Tcllib

Reference manual pages

2026-10-26

Contents

apps **15**
dtplite 15
mkdoc 20
nns 22
nnsd 24
nnslog 25
page 26
pt 32
tcldocstrip 43
devdoc **46**
tcl_community_communication 46
tcllib_devguide 48
tcllib_installer 63
tcllib_license 68
tcllib_releasemgr 69
tcllib_sources 70
aes **71**
aes 71
amazon-s3 **74**
S3 74
xsxp 95
asn **98**
asn 98
base32 **106**
base32 106
base32core 108
base32hex 109
base64 **111**
ascii85 111
base64 113
uuencode 116
yencode 119
bee **122**
bee 122
bench **128**
bench 128
bench_intro 132
bench_lang_intro 133
bench_lang_spec 135
bench_read 137
bench_wcsv 138
bench_wtext 139
bibtex **140**
bibtex 140

blowfish **142**
blowfish 142
cache **145**
async 145
clay **147**
clay 147
clock **161**
iso8601 161
rfc2822 163
cmdline **163**
cmdline 163
comm **167**
comm 167
comm_wire 183
control **186**
control 186
coroutine **188**
coro_auto 188
tcllib_coroutine 189
counter **191**
counter 191
crc **194**
cksum 194
crc16 196
crc32 201
sum 203
cron **205**
cron 205
csv **208**
csv 208
debug **212**
debug 212
debug_caller 216
debug_heartbeat 217
debug_timestamp 218
defer **219**
defer 219
des **220**
des 220
tcldes 223
tcldesjr 224
dicttool **225**
dicttool 225
dns **227**
tcllib_dns 227
tcllib_ip 231

docstrip **237**
docstrip 237
docstrip_util 245
doctools **256**
changelog 256
cvs 258
docidx 259
docidx_intro 264
docidx_lang_cmdref 265
docidx_lang_faq 267
docidx_lang_intro 268
docidx_lang_syntax 271
docidx_plugin_apiref 273
doctoc 279
doctoc_intro 284
doctoc_lang_cmdref 285
doctoc_lang_faq 287
doctoc_lang_intro 288
doctoc_lang_syntax 293
doctoc_plugin_apiref 294
doctools 300
doctools_intro 307
doctools_lang_cmdref 308
doctools_lang_faq 316
doctools_lang_intro 317
doctools_lang_syntax 326
doctools_plugin_apiref 328
mpexpand 335
doctools2base **336**
html_cssdefaults 336
nroff_manmacros 337
tcl_parse 338
tcllib_msgcat 340
doctools2idx **342**
export_docidx 342
idx_container 345
idx_export 350
idx_export_html 356
idx_export_json 360
idx_export_nroff 364
idx_export_text 366
idx_export_wiki 369
idx_import 371
idx_import_json 378
idx_introduction 381
idx_msgcat_c 384

idx_msgcat_de 385
idx_msgcat_en 386
idx_msgcat_fr 387
idx_parse 388
idx_structure 392
import_docidx 395
doctools2toc 398
export_doctoc 398
import_doctoc 401
toc_container 404
toc_export 411
toc_export_html 417
toc_export_json 422
toc_export_nroff 427
toc_export_text 430
toc_export_wiki 433
toc_import 435
toc_import_json 443
toc_introduction 447
toc_msgcat_c 450
toc_msgcat_de 451
toc_msgcat_en 452
toc_msgcat_fr 453
toc_parse 454
toc_structure 458
dtplite 462
pkg_dtplite 462
fileutil 463
fileutil 463
multi 470
multiop 471
paths 478
traverse 479
ftp 482
ftp 482
ftp_geturl 488
ftpd 489
ftpd 489
fumagic 493
cfront 493
cgen 494
filetypes 495
rtcore 496
generator 499
generator 499
gpx 506

gpx 506
grammar_aycock 509
aycock 509
grammar_fa 511
dacceptor 511
dexec 513
fa 515
faop 524
grammar_me 531
gasm 531
me_ast 536
me_cpu 538
me_cpucore 542
me_intro 548
me_tcl 549
me_util 555
me_vm 556
grammar_peg 563
peg 563
peg_interp 571
hook 573
hook 573
html 578
html 578
htmlparse 585
htmlparse 585
http 589
autoproxy 589
httpd 593
httpd 593
ident 606
ident 606
imap4 607
imap4 607
inifile 615
ini 615
interp 617
deleg_method 617
deleg_proc 618
tcllib_interp 619
irc 621
irc 621
picoirc 624
javascript 627
javascript 627
jpeg 629

jpeg 629
json 632
json 632
json_write 635
lambda 637
lambda 637
lazyset 640
lazyset 640
ldap 642
ldap 642
ldapx 651
log 663
log 663
logger 667
loggerAppender 674
loggerUtils 675
map 678
map_geocode_nominatim 678
map_slippy 680
map_slippy_cache 687
map_slippy_fetcher 689
mapproj 690
mapproj 690
markdown 696
markdown 696
math 698
bigfloat 698
bignum 707
calculus 711
changepoint 720
combinatorics 722
constants 727
decimal 729
exact 734
figurate 737
filtergen 743
fourier 745
fuzzy 747
interpolate 750
linalg 754
machineparameters 770
math 773
math_geometry 775
numtheory 788
optimize 792
pca 798

polynomials 801
probopt 804
qcomplex 807
quasirandom 811
rational_funcs 813
roman 816
romberg 817
special 822
statistics 830
symdiff 865
trig 867
md4 871
md4 871
md5 873
md5 874
md5crypt 876
md5crypt 876
mime 878
mime 878
smtp 883
mkdoc 886
mkdoc 886
multiplexer 889
multiplexer 889
namespacex 891
namespacex 891
ncgi 893
ncgi 893
nettool 898
nettool 898
nmea 901
nmea 901
nns 903
nns_auto 903
nns_client 905
nns_common 910
nns_intro 911
nns_protocol 913
nns_server 915
nntp 917
nntp 917
ntp 923
ntp_time 923
oauth 925
oauth 925
oometa 930

oometa 930
ooutil **933**
ooutil 933
otp **936**
otp 936
page **937**
page_intro 938
page_pluginmgr 938
page_util_flow 949
page_util_norm_lemon 951
page_util_norm_peg 952
page_util_peg 953
page_util_quote 955
pki **956**
pki 956
pluginmgr **962**
pluginmgr 962
png **967**
png 967
pop3 **970**
pop3 970
pop3d **974**
pop3d 974
pop3d_dbox 978
pop3d_udb 980
practcl **982**
practcl 982
processman **1011**
processman 1011
profiler **1012**
profiler 1012
pt **1014**
pt_astree 1014
pt_cparam_config_critcl 1019
pt_cparam_config_tea 1020
pt_from_api 1021
pt_introduction 1028
pt_json_language 1031
pt_param 1038
pt_parse_peg 1047
pt_parser_api 1049
pt_peg_container 1055
pt_peg_container_peg 1064
pt_peg_export 1065
pt_peg_export_container 1072
pt_peg_export_json 1079

pt_peg_export_peg 1088
pt_peg_from_container 1096
pt_peg_from_json 1097
pt_peg_from_peg 1105
pt_peg_import 1113
pt_peg_import_container 1120
pt_peg_import_json 1121
pt_peg_import_peg 1129
pt_peg_interp 1137
pt_peg_introduction 1143
pt_peg_language 1145
pt_peg_op 1153
pt_peg_to_container 1157
pt_peg_to_cparam 1164
pt_peg_to_json 1172
pt_peg_to_param 1181
pt_peg_to_peg 1204
pt_peg_to_tclparam 1213
pt_pegrammar 1220
pt_pexpr_op 1226
pt_pexpression 1229
pt_pgen 1236
pt_rdengine 1239
pt_tclparam_config_nx 1254
pt_tclparam_config_snit 1255
pt_tclparam_config_tcloo 1256
pt_to_api 1257
pt_util 1264
rc4 1266
rc4 1266
rcs 1268
rcs 1268
report 1272
report 1272
rest 1278
rest 1278
ripemd 1287
ripemd128 1287
ripemd160 1290
sasl 1292
gtoken 1292
ntlm 1294
sasl 1294
scram 1299
sha1 1300
sha1 1300

sha256 1303
simulation 1306
annealing 1306
montecarlo 1310
simulation_random 1313
smtpd 1316
smtpd 1316
snit 1321
snit 1321
snitfaq 1359
soundex 1417
soundex 1417
stooop 1418
stooop 1418
switched 1421
string 1426
token 1426
token_shell 1429
stringprep 1432
stringprep 1432
stringprep_data 1435
unicode 1436
unicode_data 1439
struct 1439
disjointset 1439
graph 1444
graph1 1460
graphops 1465
matrix 1484
pool 1492
prioqueue 1499
queue 1501
record 1502
skiplist 1509
stack 1510
struct_list 1512
struct_map 1523
struct_set 1524
struct_tree 1527
struct_tree1 1537
tar 1542
tar 1542
tepam 1545
tepam_argument_dialogbox 1545
tepam_doc_gen 1560
tepam_introduction 1568

tepam_procedure 1576
term 1597
ansi_cattr 1597
ansi_cctrl 1601
ansi_cmacros 1608
ansi_code 1609
ansi_ctrlu 1610
ansi_send 1612
imenu 1619
ipager 1622
receive 1624
term 1625
term_bind 1626
term_send 1628
textutil 1629
adjust 1629
expander 1633
patch 1640
repeat 1642
tabify 1643
textutil 1645
textutil_split 1657
textutil_string 1659
trim 1662
wcswidth 1664
tie 1666
tie 1666
tie_std 1673
tiff 1674
tiff 1674
tool 1677
meta 1677
tool 1680
tool_dict_ensemble 1685
transfer 1686
connect 1686
copyops 1689
ddest 1692
dsource 1694
receiver 1697
tqueue 1701
transmitter 1703
treeql 1708
treeql 1708
try 1717
tcllib_fhome 1717

tcllib_throw 1718
tcllib_try 1719
udpcluster 1721
udpcluster 1721
uev 1722
uevent 1722
uevent_onidle 1725
units 1726
units 1726
uri 1732
uri 1732
urn-scheme 1738
uuid 1739
uuid 1739
valtype 1740
cc_amex 1740
cc_discover 1742
cc_mastercard 1744
cc_visa 1745
ean13 1747
iban 1749
imei 1751
isbn 1752
luhn 1754
luhn5 1756
usnpi 1757
valtype_common 1759
verhoeff 1761
virtchannel_base 1762
cat 1762
facade 1763
halfpipe 1765
nullzero 1766
randseed 1767
std 1768
tcllib_fifo 1769
tcllib_fifo2 1770
tcllib_memchan 1771
tcllib_null 1772
tcllib_random 1773
tcllib_string 1774
tcllib_variable 1775
tcllib_zero 1775
textwindow 1776
virtchannel_core 1777
core 1777

events 1779
transformcore 1780
virtchannel_transform 1782
adler32 1782
hex 1783
identity 1784
limitsize 1785
observe 1786
rot 1787
spacer 1788
tcllib_zlib 1789
vt_base64 1790
vt_counter 1791
vt_crc32 1792
vt_otp 1793
websocket 1794
websocket 1794
wip 1801
wip 1801
yaml 1806
huddle 1806
yaml 1817
zip 1820
decode 1820
encode 1823
mkzip 1824
License 1826
license license terms 1826

apps

dtplite

Keywords: dtplite, HTML, TMML, conversion, docidx, doctoc, doctools, manpage, markup, nroff

Name

dtplite - Lightweight DocTools Markup Processor

Synopsis

dtplite -o *output* ?*options*? *format inputfile*
dtplite validate *inputfile*
dtplite -o *output* ?*options*? *format inputdirectory*
dtplite -merge -o *output* ?*options*? *format inputdirectory*

Description

The application described by this document, **dtplite**, is the successor to the extremely simple **mpexpand**. Influenced in its functionality by the **dtp** doctools processor it is much more powerful than **mpexpand**, yet still as easy to use; definitely easier than **dtp** with its myriad of subcommands and options.

dtplite is based upon the package **doctools**, like the other two processors.

Use cases

dtplite was written with the following three use cases in mind.

1. Validation of a single document, i.e. checking that it was written in valid doctools format. This mode can also be used to get a preliminary version of the formatted output for a single document, for display in a browser, nroff, etc., allowing proofreading of the formatting.
2. Generation of the formatted documentation for a single package, i.e. all the manpages, plus a table of contents and an index of keywords.
3. An extension of the previous mode of operation, a method for the easy generation of one documentation tree for several packages, and especially of a unified table of contents and keyword index.

Beyond the above we also want to make use of the customization features provided by the HTML formatter. It is not the only format the application should be able to generate, but we anticipiate it to be the most commonly used, and it is one of the few which do provide customization hooks.

We allow the caller to specify a header string, footer string, a stylesheet, and data for a bar of navigation links at the top of the generated document. While all can be set as long as the formatting engine provides an appropriate engine parameter (See section Options) the last two have internal processing which make them specific to HTML.

Command line

dtplite -o *output* ?options? *format inputfile*

This is the form for use case [1]. The *options* will be explained later, in section Options.

path *output* (in)

This argument specifies where to write the generated document. It can be the path to a file or directory, or **-**. The last value causes the application to write the generated documented

to **stdout**.

If the *output* does not exist then [file dirname $output] has to exist and must be a writable directory. The generated document will be written to a file in that directory, and the name of that file will be derived from the *inputfile*, the *format*, and the value given to option **-ext** (if present).

(path|handle) *format* (in)
This argument specifies the formatting engine to use when processing the input, and thus the format of the generated document. See section Formats for the possibilities recognized by the application.

path *inputfile* (in)
This argument specifies the path to the file to process. It has to exist, must be readable, and written in *doctools* format.

dtplite validate *inputfile*
This is a simpler form for use case [1]. The "validate" format generates no output at all, only syntax checks are performed. As such the specification of an output file or other options is not necessary and left out.

dtplite -o *output* ?options? *format inputdirectory*
This is the form for use case [2]. It differs from the form for use case [1] by having the input documents specified through a directory instead of a file. The other arguments are identical, except for *output*, which now has to be the path to an existing and writable directory.

The input documents are all files in *inputdirectory* or any of its subdirectories which were recognized by **fileutil::fileType** as containing text in *doctools* format.

dtplite -merge -o *output* ?options? *format inputdirectory*
This is the form for use case [3]. The only difference to the form for use case [2] is the additional option **-merge**.

Each such call will merge the generated documents coming from processing the input documents under *inputdirectory* or any of its subdirectories to the files under *output*. In this manner it is possible to incrementally build the unified documentation for any number of packages. Note that it is necessary to run through all the packages twice to get fully correct cross-references (for formats supporting them).

Options

This section describes all the options available to the user of the application, with the exception of the options **-o** and **-merge**. These two were described already, in section Command line.

-exclude string
This option specifies an exclude (glob) pattern. Any files identified as manpages to process which match the exclude pattern are ignored. The option can be provided multiple times, each usage adding an additional pattern to the list of exclusions.

-ext string
If the name of an output file has to be derived from the name of an input file it will use the name of the *format* as the extension by default. This option here will override this however, forcing it to use *string* as the file extension. This option is ignored if the name of the output

file is fully specified through option **-o**.

When used multiple times only the last definition is relevant.

-header file
This option can be used if and only if the selected *format* provides an engine parameter named "header". It takes the contents of the specified file and assign them to that parameter, for whatever use by the engine. The HTML engine will insert the text just after the tag. If navigation buttons are present (see option **-nav** below), then the HTML generated for them is appended to the header data originating here before the final assignment to the parameter.

When used multiple times only the last definition is relevant.

-footer file
Like **-header**, except that: Any navigation buttons are ignored, the corresponding required engine parameter is named "footer", and the data is inserted just before the tag.

When used multiple times only the last definition is relevant.

-style file
This option can be used if and only if the selected *format* provides an engine parameter named "meta". When specified it will generate a piece of HTML code declaring the *file* as the stylesheet for the generated document and assign that to the parameter. The HTML engine will insert this inot the document, just after the tag.

When processing an input directory the stylesheet file is copied into the output directory and the generated HTML will refer to the copy, to make the result more self-contained. When processing an input file we have no location to copy the stylesheet to and so just reference it as specified.

When used multiple times only the last definition is relevant.

-toc path
This option specifies a doctoc file to use for the table of contents instead of generating our own.

When used multiple times only the last definition is relevant.

-pre+toc label path|text
see below ...

-post+toc label path|text
This option specifies additional doctoc files (or texts) to use in the navigation bar.

Positioning and handling of multiple uses is like for options **-prenav** and **-postnav**, see below.

-nav label url
see below ...

-prenav label url
Use this option to specify a navigation button with *label* to display and the *url* to link to. This option can be used if and only if the selected *format* provides an engine parameter named

"header". The HTML generated for this is appended to whatever data we got from option **-header** before it is inserted into the generated documents.

When used multiple times all definitions are collected and a navigation bar is created, with the first definition shown at the left edge and the last definition to the right.

The url can be relative. In that case it is assumed to be relative to the main files (TOC and Keyword index), and will be transformed for all others to still link properly.

-postnav label url

Use this option to specify a navigation button with *label* to display and the *url* to link to. This option can be used if and only if the selected *format* provides an engine parameter named "header". The HTML generated for this is appended to whatever data we got from option **-header** before it is inserted into the generated documents.

When used multiple times all definitions are collected and a navigation bar is created, with the last definition shown at the right edge and the first definition to the left.

The url can be relative. In that case it is assumed to be relative to the main files (TOC and Keyword index), and will be transformed for all others to still link properly.

Formats

At first the *format* argument will be treated as a path to a tcl file containing the code for the requested formatting engine. The argument will be treated as the name of one of the predefined formats listed below if and only if the path does not exist.

Note a limitation: If treating the format as path to the tcl script implementing the engine was sucessful, then this script has to implement not only the engine API for doctools, i.e. *doctools_api*, but for *doctoc_api* and *docidx_api* as well. Otherwise the generation of a table of contents and of a keyword index will fail.

List of predefined formats, i.e. as provided by the package **doctools**:

nroff

The processor generates *roff output, the standard format for unix manpages.

html

The processor generates HTML output, for usage in and display by web browsers. This engine is currently the only one providing the various engine parameters required for the additional customaization of the output.

tmml

The processor generates TMML output, the Tcl Manpage Markup Language, a derivative of XML.

latex

The processor generates LaTeX output.

wiki

The processor generates Wiki markup as understood by **wikit**.

list

The processor extracts the information provided by **manpage_begin**. This format is used internally to extract the meta data from which both table of contents and keyword index are

derived from.

null
The processor does not generate any output. This is equivalent to **validate**.

Directory structures

In this section we describe the directory structures generated by the application under *output* when processing all documents in an *inputdirectory*. In other words, this is only relevant to the use cases [2] and [3].

[2]
The following directory structure is created when processing a single set of input documents. The file extension used is for output in HTML, but that is not relevant to the structure and was just used to have proper file names.

```
output/
    toc.html
    index.html
    files/
        path/to/FOO.html
```

The last line in the example shows the document generated for a file FOO located at

```
inputdirectory/path/to/FOO
```

[3]
When merging many packages into a unified set of documents the generated directory structure is a bit deeper:

```
output
    .toc
    .idx
    .tocdoc
    .idxdoc
    .xrf
    toc.html
    index.html
    FOO1/
        ...
    FOO2/
        toc.html
        files/
            path/to/BAR.html
```

Each of the directories FOO1, ... contains the documents generated for the package FOO1, ... and follows the structure shown for use case [2]. The only exception is that there is no per-package index.

The files "*.toc*", "*.idx*", and "*.xrf*" contain the internal status of the whole output and will be read and updated by the next invokation. Their contents will not be documented. Remove

these files when all packages wanted for the output have been processed, i.e. when the output is complete.

The files "*.tocdoc*", and "*.idxdoc*", are intermediate files in doctoc and docidx markup, respectively, containing the main table of contents and keyword index for the set of documents before their conversion to the chosen output format. They are left in place, i.e. not deleted, to serve as demonstrations of doctoc and docidx markup.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Documentation tools

Copyright


```
Copyright (c) 2004-2013 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


mkdoc

Keywords: mkdoc

Name

mkdoc - Source code documentation extractor/converter application

Synopsis

mkdoc --help
mkdoc --version
mkdoc --license
mkdoc *input output* ?*--css cssfile*?

Description

This document describes **mkdoc**, an application to extract documentation embedded in source code files, be they "*.tcl*", or other.

Command line

mkdoc --help

The application prints a short help to standard output and exits.

mkdoc --version

The application prints its version number to standard output and exits.

mkdoc --license

The application prints its license to standard output and exits.

mkdoc *input output* ?--css *cssfile*?

The application reads the *input* file, extracts the embedded documentation, and writes it to the *output* file.

If the output file is not a "*.md*" file the extracted documentation is converted to HTML before being written.

When generating and writing HTML the default CSS stylesheet can be overridden by specifying the path to a custom stylesheet via option **--css**.

If the input file is a "*.md*" file it is expected to contain Markdown as-is, instead of Markdown embedded into code.

On the other side, when the file is considered code then the documentation is expected to be contained in all lines starting with the marker **#'**. For script languages like Tcl the **#** character of this marker means that the documentation is contained in the so-flagged comments. For other languages the marker and documentation may have to be embedded into multi-line comments.

Examples

```
# Create HTML manual for a CPP file using a custom style sheet
mkdoc sample.cpp sample.html --css manual.css

# Extract the documentation from code as simple Markdown, ready to
be processed
# further, for example with pandoc, or similar
mkdoc sample.cpp sample.md

# Convert a Markdown file to HTML
mkdoc sample.md sample.html
```

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such to the author of this package. Please also report any ideas for enhancements you may have for either package and/or documentation.

Code copyright

BSD License type:

The following terms apply to all files a ssociated with the software unless explicitly disclaimed in individual files.

The authors hereby grant permission to use, copy, modify, distribute, and license this software and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions. No written agreement, license, or royalty fee is required for any of the authorized uses. Modifications to this software

may be copyrighted by their authors and need not follow the licensing terms described here, provided that the new terms are clearly indicated on the first page of each file where they apply.

In no event shall the authors or distributors be liable to any party for direct, indirect, special, incidental, or consequential damages arising out of the use of this software, its documentation, or any derivatives thereof, even if the authors have been advised of the possibility of such damage.

The authors and distributors specifically disclaim any warranties, including, but not limited to, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. This software is provided on an "as is" basis, and the authors and distributors have no obligation to provide maintenance, support, updates, enhancements, or modifications.

RESTRICTED RIGHTS: Use, duplication or disclosure by the government is subject to the restrictions as set forth in subparagraph (c) (1) (ii) of the Rights in Technical Data and Computer Software Clause as DFARS 252.227-7013 and FAR 52.227-19.

Category
Text processing

Copyright
```
Copyright (c) 2019-2022, Detlef Groth <detlef(at)dgroth(dot)de>
```

nns
Keywords: nns, application, client, name service

Name
nns - Name service facility, Commandline Client Application

Synopsis
nns bind ?*-host host*? ?*-port port*? *name data*
nns search ?*-host host*? ?*-port port*? ?**-continuous**? ?*pattern*?
nns ident ?*-host host*? ?*-port port*?
nns who

Description
Please read *Name service facility, introduction* first.

The application described by this document, **nns**, is a simple command line client for the nano name service facility provided by the Tcllib packages **nameserv**, and **nameserv::server**. Beyond that the application's sources also serve as an example of how to use the client package **nameserv**. All abilities of a client are covered, from configuration to registration of names to searching.

This name service facility has nothing to do with the Internet's *Domain Name System*, otherwise known as *DNS*. If the reader is looking for a package dealing with that please see either of the packages **dns** and **resolv**, both found in Tcllib too.

Use cases
nns was written with the following two main use cases in mind.

1. Registration of a name/data pair in the name service.
2. Searching the name service for entries matching a glob pattern.

Beyond the above we also want to be able to identify the client, and get information about the name service.

Command line

nns bind ?-host *host*? ?-port *port*? *name data*

This form registers the *name*/*data* pair in the specified name service. In this form the command will *not* exit to keep the registration alive. The user has to kill it explicitly, either by sending a signal, or through the job-control facilities of the shell in use. It will especially survive the loss of the connection to the name service and reestablish the *name*/*data* pair when the connection is restored.

The options to specify the name service will be explained later, in section Options.

nns search ?-host *host*? ?-port *port*? ?-continuous? ?*pattern*?

This form searches the specified name service for entries matching the glob-*pattern* and prints them to stdout, with each entry on its own line. If no pattern is specified it defaults to *, matching everything.

The options to specify the name service will be explained later, in section Options.

If the option **-continuous** is specified the client will not exit after performing the search, but start to continuously monitor the service for changes to the set of matching entries, appropriately updating the display as changes arrive. In that form it will especially also survive the loss of the connection to the name service and reestablish the search when the connection is restored.

nns ident ?-host *host*? ?-port *port*?

This form asks the specified name service for the version and features of the name service protocol it supports and prints the results to stdout.

The options to specify the name service will be explained later, in section Options.

nns who

This form prints name, version, and protocol version of the application to stdout.

Options

This section describes all the options available to the user of the application

-host name|ipaddress

If this option is not specified it defaults to **localhost**. It specifies the name or ip-address of the host the name service to talk to is running on.

-port number

If this option is not specified it defaults to **38573**. It specifies the TCP port the name service to talk to is listening on for requests.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib

Trackers and specify *nameserv* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Networking

Copyright

```
Copyright (c) 2007-2008 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


nnsd
Keywords: nnsd, application, name service, server

Name
nnsd - Name service facility, Commandline Server Application

Synopsis
nnsd ?*-localonly flag*? ?*-port port*?

Description
Please read *Name service facility, introduction* first.

The application described by this document, nns, is a simple command line server for the nano name service facility provided by the Tcllib packages **nameserv**, and **nameserv::server**. Beyond that the application's sources also serve as an example of how to use the server package **nameserv::server**.

This name service facility has nothing to do with the Internet's *Domain Name System*, otherwise known as *DNS*. If the reader is looking for a package dealing with that please see either of the packages **dns** and **resolv**, both found in Tcllib too.

Use cases
nnsd was written with the following main use case in mind.

1. Run a nano name service on some host.

Command line
nnsd ?-localonly *flag*? ?-port *port*?

The command configures a server per the specified options and starts it. The command will not exit on its own, as it keeps the name service database wholly in memory. The user has to kill it explicitly, either by sending a a signal, or through the job-control facilities of the shell in use.

The options to configure the name service are explained in section Options.

Options

This section describes all the options available to the user of the application

-localonly bool
: If this option is not specified it defaults to **true**, i.e. acceptance of only local connections. The server will accept remote connections, i.e. connections from other hosts, if and only if this option is configured to **false**.

-port number
: If this option is not specified it defaults to **38573**. It specifies the TCP port the server has to listen on for requests.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *nameserv* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Networking

Copyright


```
Copyright (c) 2007-2008 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


nnslog

Keywords: nnslog, application, client, name service

Name

nnslog - Name service facility, Commandline Logging Client Application

Synopsis

nnslog ?*-host host*? ?*-port port*?

Description

Please read *Name service facility, introduction* first.

The application described by this document, **nnslog**, is a simple command line client for the nano name service facility provided by the Tcllib packages **nameserv**, and **nameserv::server**.

It essentially implements "nns search -continuous *", but uses a different output formatting. Instead of continuously showing the current contents of the server in the terminal it simply logs all received add/remove events to **stdout**.

This name service facility has nothing to do with the Internet's *Domain Name System*, otherwise known as *DNS*. If the reader is looking for a package dealing with that please see either of the

packages **dns** and **resolv**, both found in Tcllib too.

Use cases

nnslog was written with the following main use case in mind.

1. Monitoring the name service for all changes and logging them in a text terminal.

Command line

nnslog ?-host *host*? ?-port *port*?

The command connects to the specified name service, sets up a search for all changes and then prints all received events to stdout, with each events on its own line. The command will not exit until it is explicitly terminated by the user. It will especially survive the loss of the connection to the name service and reestablish the search and log when the connection is restored.

The options to specify the name service will be explained later, in section Options.

Options

This section describes all the options available to the user of the application

-host name|ipaddress

If this option is not specified it defaults to **localhost**. It specifies the name or ip-address of the host the name service to talk to is running on.

-port number

If this option is not specified it defaults to **38573**. It specifies the TCP port the name service to talk to is listening on for requests.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *nameserv* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Networking

Copyright


```
Copyright (c) 2008 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


page

Keywords: page, parser generator, text processing

Name

page - Parser Generator

Synopsis

page ?*options...*? ?*input ?output*?

Description

The application described by this document, **page**, is actually not just a parser generator, as the name implies, but a generic tool for the execution of arbitrary transformations on texts.

Its genericity comes through the use of *plugins* for reading, transforming, and writing data, and the predefined set of plugins provided by Tcllib is for the generation of memoizing recursive descent parsers (aka *packrat parsers*) from grammar specifications (*Parsing Expression Grammars*).

page is written on top of the package **page::pluginmgr**, wrapping its functionality into a command line based application. All the other **page::*** packages are plugin and/or supporting packages for the generation of parsers. The parsers themselves are based on the packages **grammar::peg**, **grammar::peg::interp**, and **grammar::mengine**.

Command line

page ?*options*...? ?*input* ?*output*??

This is general form for calling **page**. The application will read the contents of the file *input*, process them under the control of the specified *options*, and then write the result to the file *output*.

If *input* is the string **-** the data to process will be read from **stdin** instead of a file. Analogously the result will be written to **stdout** instead of a file if *output* is the string **-**. A missing output or input specification causes the application to assume **-**.

The detailed specifications of the recognized *options* are provided in section Options.

path *input* (in)

This argument specifies the path to the file to be processed by the application, or **-**. The last value causes the application to read the text from **stdin**. Otherwise it has to exist, and be readable. If the argument is missing **-** is assumed.

path *output* (in)

This argument specifies where to write the generated text. It can be the path to a file, or **-**. The last value causes the application to write the generated documented to **stdout**.

If the file *output* does not exist then [file dirname $output] has to exist and must be a writable directory, as the application will create the fileto write to.

If the argument is missing **-** is assumed.

Operation

... reading ... transforming ... writing - plugins - pipeline ...

Options

This section describes all the options available to the user of the application. Options are always processed in order. I.e. of both **--help** and **--version** are specified the option encountered first

has precedence.

Unknown options specified before any of the options **-rd**, **-wr**, or **-tr** will cause processing to abort with an error. Unknown options coming in between these options, or after the last of them are assumed to always take a single argument and are associated with the last plugin option coming before them. They will be checked after all the relevant plugins, and thus the options they understand, are known. I.e. such unknown options cause error if and only if the plugin option they are associated with does not understand them, and was not superceded by a plugin option coming after.

Default options are used if and only if the command line did not contain any options at all. They will set the application up as a PEG-based parser generator. The exact list of options is

```
-c peg
```

And now the recognized options and their arguments, if they have any:

--help
: see below ...

-h
: see below ...

-?
: When one of these options is found on the command line all arguments coming before or after are ignored. The application will print a short description of the recognized options and exit.

--version
: see below ...

-V
: When one of these options is found on the command line all arguments coming before or after are ignored. The application will print its own revision and exit.

-P
: This option signals the application to activate visual feedback while reading the input.

-T
: This option signals the application to collect statistics while reading the input and to print them after reading has completed, before processing started.

-D
: This option signals the application to activate logging in the Safe base, for the debugging of problems with plugins.

-r parser
: see below ...

-rd parser
: see below ...

--reader parser
: These options specify the plugin the application has to use for reading the *input*. If the options are used multiple times the last one will be used.

-w generator
: see below ...

-wr generator

see below ...

--writer generator
These options specify the plugin the application has to use for generating and writing the final *output*. If the options are used multiple times the last one will be used.

-t process
see below ...

-tr process
see below ...

--transform process
These options specify a plugin to run on the input. In contrast to readers and writers each use will *not* supersede previous uses, but add each chosen plugin to a list of transformations, either at the front, or the end, per the last seen use of either option **-p** or **-a**. The initial default is to append the new transformations.

-a
see below ...

--append
These options signal the application that all following transformations should be added at the end of the list of transformations.

-p
see below ...

--prepend
These options signal the application that all following transformations should be added at the beginning of the list of transformations.

--reset
This option signals the application to clear the list of transformations. This is necessary to wipe out the default transformations used.

-c file
see below ...

--configuration file
This option causes the application to load a configuration file and/or plugin. This is a plugin which in essence provides a pre-defined set of commandline options. They are processed exactly as if they have been specified in place of the option and its arguments. This means that unknown options found at the beginning of the configuration file are associated with the last plugin, even if that plugin was specified before the configuration file itself. Conversely, unknown options coming after the configuration file can be associated with a plugin specified in the file.

If the argument is a file which cannot be loaded as a plugin the application will assume that its contents are a list of options and their arguments, separated by space, tabs, and newlines. Options and argumentes containing spaces can be quoted via double-quotes (") and quotes ('). The quote character can be specified within in a quoted string by doubling it. Newlines in a quoted string are accepted as is.

Plugins

page makes use of four different types of plugins, namely: readers, writers, transformations,

and configurations. Here we provide only a basic introduction on how to use them from **page**. The exact APIs provided to and expected from the plugins can be found in the documentation for **page::pluginmgr**, for those who wish to write their own plugins.

Plugins are specified as arguments to the options **-r**, **-w**, **-t**, **-c**, and their equivalent longer forms. See the section Options for reference.

Each such argument will be first treated as the name of a file and this file is loaded as the plugin. If however there is no file with that name, then it will be translated into the name of a package, and this package is then loaded. For each type of plugins the package management searches not only the regular paths, but a set application- and type-specific paths as well. Please see the section Plugin locations for a listing of all paths and their sources.

-c *name*
Configurations. The name of the package for the plugin *name* is "page::config::*name*".

We have one predefined plugin:

peg
It sets the application up as a parser generator accepting parsing expression grammars and writing a packrat parser in Tcl. The actual arguments it specifies are:

```
--reset
--append
--reader    peg
--transform reach
--transform use
--writer    me
```

-r *name*
Readers. The name of the package for the plugin *name* is "page::reader::*name*".

We have five predefined plugins:

peg
Interprets the input as a parsing expression grammar (*PEG*) and generates a tree representation for it. Both the syntax of PEGs and the structure of the tree representation are explained in their own manpages.

hb
Interprets the input as Tcl code as generated by the writer plugin *hb* and generates its tree representation.

ser
Interprets the input as the serialization of a PEG, as generated by the writer plugin *ser*, using the package **grammar::peg**.

lemon
Interprets the input as a grammar specification as understood by Richard Hipp's *LEMON* parser generator and generates a tree representation for it. Both the input syntax and the structure of the tree representation are explained in their own manpages.

treeser
Interprets the input as the serialization of a **struct::tree**. It is validated as such, but nothing else. It is *not* assumed to be the tree representation of a grammar.

-w *name*

Writers. The name of the package for the plugin *name* is "page::writer::*name*".

We have eight predefined plugins:

identity

Simply writes the incoming data as it is, without making any changes. This is good for inspecting the raw result of a reader or transformation.

null

Generates nothing, and ignores the incoming data structure.

tree

Assumes that the incoming data structure is a **struct::tree** and generates an indented textual representation of all nodes, their parental relationships, and their attribute information.

peg

Assumes that the incoming data structure is a tree representation of a *PEG* or other other grammar and writes it out as a PEG. The result is nicely formatted and partially simplified (strings as sequences of characters). A pretty printer in essence, but can also be used to obtain a canonical representation of the input grammar.

tpc

Assumes that the incoming data structure is a tree representation of a *PEG* or other other grammar and writes out Tcl code defining a package which defines a **grammar::peg** object containing the grammar when it is loaded into an interpreter.

hb

This is like the writer plugin *tpc*, but it writes only the statements which define stat expression and grammar rules. The code making the result a package is left out.

ser

Assumes that the incoming data structure is a tree representation of a *PEG* or other other grammar, transforms it internally into a **grammar::peg** object and writes out its serialization.

me

Assumes that the incoming data structure is a tree representation of a *PEG* or other other grammar and writes out Tcl code defining a package which implements a memoizing recursive descent parser based on the match engine (ME) provided by the package **grammar::mengine**.

-t *name*

Transformers. The name of the package for the plugin *name* is "page::transform::*name*".

We have two predefined plugins:

reach

Assumes that the incoming data structure is a tree representation of a *PEG* or other other grammar. It determines which nonterminal symbols and rules are reachable from start-symbol/expression. All nonterminal symbols which were not reached are removed.

use

Assumes that the incoming data structure is a tree representation of a *PEG* or other other grammar. It determines which nonterminal symbols and rules are able to generate a *finite* sequences of terminal symbols (in the sense for a Context Free Grammar). All nonterminal

symbols which were not deemed useful in this sense are removed.

Plugin locations

The application-specific paths searched by **page** either are, or come from:

1. The directory "*~/.page/plugin*"
2. The environment variable *PAGE_PLUGINS*
3. The registry entry *HKEY_LOCAL_MACHINE\SOFTWARE\PAGE\PLUGINS*
4. The registry entry *HKEY_CURRENT_USER\SOFTWARE\PAGE\PLUGINS*

The type-specific paths searched by **page** either are, or come from:

1. The directory "*~/.page/plugin/*"
2. The environment variable *PAGE__PLUGINS*
3. The registry entry *HKEY_LOCAL_MACHINE\SOFTWARE\PAGE\\PLUGINS*
4. The registry entry *HKEY_CURRENT_USER\SOFTWARE\PAGE\\PLUGINS*

Where the placeholder is always one of the values below, properly capitalized.

1. reader
2. writer
3. transform
4. config

The registry entries are specific to the Windows(tm) platform, all other platforms will ignore them.

The contents of both environment variables and registry entries are interpreted as a list of paths, with the elements separated by either colon (Unix), or semicolon (Windows).

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *page* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Page Parser Generator

Copyright


```
Copyright (c) 2005 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt

Keywords: pt, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt - Parser Tools Application

Synopsis

package require Tcl 8.5 9
pt generate *resultformat* ?*options...*? *resultfile inputformat inputfile*

Description

Are you lost ? Do you have trouble understanding this document ? In that case please read the overview provided by the *Introduction to Parser Tools*. This document is the entrypoint to the whole system the current package is a part of.

This document describes **pt**, the main application of the module, a *parser generator*. Its intended audience are people who wish to create a parser for some language of theirs. Should you wish to modify the application instead, please see the section about the application's Internals for the basic references.

It resides in the User Application Layer of Parser Tools.

IMAGE: arch_user_app

Command line

pt generate *resultformat ?options...? resultfile inputformat inputfile*

This sub-command of the application reads the parsing expression grammar stored in the *inputfile* in the format *inputformat*, converts it to the *resultformat* under the direction of the (format-specific) set of options specified by the user and stores the result in the *resultfile*.

The *inputfile* has to exist, while the *resultfile* may be created, overwriting any pre-existing content of the file. Any missing directory in the path to the *resultfile* will be created as well.

The exact form of the result for, and the set of options supported by the known result-formats, are explained in the upcoming sections of this document, with the list below providing an index mapping between format name and its associated section. In alphabetical order:

c
A *resultformat*. See section C parser.

container
A *resultformat*. See section Grammar container.

critcl
A *resultformat*. See section C parser embedded in Tcl.

json
A *input*- and *resultformat*. See section JSON grammar exchange.

oo
A *resultformat*. See section TclOO parser.

peg
A *input*- and *resultformat*. See section Peg specification language.

snit
A *resultformat*. See section Snit parser.

Of the seven possible results four are parsers outright (**c**, **critcl**, **oo**, and **snit**), one (**container**)

provides code which can be used in conjunction with a generic parser (also known as a grammar interpreter), and the last two (**json** and **peg**) are doing double-duty as input formats, allowing the transformation of grammars for exchange, reformatting, and the like.

The created parsers fall into three categories:

```
+ --- C ---> critcl, c
|
+ --- specialized -+
|                  |
---+                  + --- Tcl -> snit, oo
|
+ --- interpreted (Tcl) ------> container
```

Specialized parsers implemented in C

The fastest parsers are created when using the result formats **c** and **critcl**. The first returns the raw C code for the parser, while the latter wraps it into a Tcl package using *CriTcl*.

This makes the latter much easier to use than the former. On the other hand, the former can be adapted to the users' requirements through a multitude of options, allowing for things like usage of the parser outside of a Tcl environment, something the **critcl** format doesn't support. As such the **c** format is meant for more advanced users, or users with special needs.

A disadvantage of all the parsers in this section is the need to run them through a C compiler to make them actually executable. This is not something everyone has the necessary tools for. The parsers in the next section are for people under such restrictions.

Specialized parsers implemented in Tcl

As the parsers in this section are implemented in Tcl they are quite a bit slower than anything from the previous section. On the other hand this allows them to be used in pure-Tcl environments, or in environments which allow only a limited set of binary packages. In the latter case it will be advantageous to lobby for the inclusion of the C-based runtime support (notes below) into the environment to reduce the impact of Tcl's on the speed of these parsers.

The relevant formats are **snit** and **oo**. Both place their result into a Tcl package containing a **snit::type**, or TclOO **class** respectively.

Of the supporting runtime, which is the package **pt::rde**, the user has to know nothing but that it does exist and that the parsers are dependent on it. Knowledge of the API exported by the runtime for the parsers' consumption is *not* required by the parsers' users.

Interpreted parsing implemented in Tcl

The last category, grammar interpretation. This means that an interpreter for parsing expression grammars takes the description of the grammar to parse input for, and uses it guide the parsing process. This is the slowest of the available options, as the interpreter has to continually run through the configured grammar, whereas the specialized parsers of the previous sections have the relevant knowledge about the grammar baked into them.

The only places where using interpretation make sense is where the grammar for some input

may be changed interactively by the user, as the interpretation allows for quick turnaround after each change, whereas the previous methods require the generation of a whole new parser, which is not as fast. On the other hand, wherever the grammar to use is fixed, the previous methods are much more advantageous as the time to generate the parser is minuscule compared to the time the parser code is in use.

The relevant result format is **container**. It (quickly) generates grammar descriptions (instead of a full parser) which match the API expected by ParserTools' grammar interpreter. The latter is provided by the package **pt::peg::interp**.

All the parsers generated by **critcl**, **snit**, and **oo**, and the grammar interpreter share a common API for access to the actual parsing functionality, making them all plug-compatible. It is described in the *Parser API* specification document.

Peg specification language

peg, a language for the specification of parsing expression grammars is meant to be human readable, and writable as well, yet strict enough to allow its processing by machine. Like any computer language. It was defined to make writing the specification of a grammar easy, something the other formats found in the Parser Tools do not lend themselves too.

For either an introduction to or the formal specification of the language, please go and read the *PEG Language Tutorial*.

When used as a result-format this format supports the following options:

-file string
: The value of this option is the name of the file or other entity from which the grammar came, for which the command is run. The default value is **unknown**.

-name string
: The value of this option is the name of the grammar we are processing. The default value is **a_pe_grammar**.

-user string
: The value of this option is the name of the user for which the command is run. The default value is **unknown**.

-template string
: The value of this option is a string into which to put the generated text and the values of the other options. The various locations for user-data are expected to be specified with the placeholders listed below. The default value is "**@code@**".

  @user@
  : To be replaced with the value of the option **-user**.

  @format@
  : To be replaced with the the constant **PEG**.

  @file@
  : To be replaced with the value of the option **-file**.

  @name@
  : To be replaced with the value of the option **-name**.

  @code@
  : To be replaced with the generated text.

JSON grammar exchange

The **json** format for parsing expression grammars was written as a data exchange format not bound to Tcl. It was defined to allow the exchange of grammars with PackRat/PEG based parser generators for other languages.

For the formal specification of the JSON grammar exchange format, please go and read *The JSON Grammar Exchange Format*.

When used as a result-format this format supports the following options:

-file string
: The value of this option is the name of the file or other entity from which the grammar came, for which the command is run. The default value is **unknown**.

-name string
: The value of this option is the name of the grammar we are processing. The default value is **a_pe_grammar**.

-user string
: The value of this option is the name of the user for which the command is run. The default value is **unknown**.

-indented boolean
: If this option is set the system will break the generated JSON across lines and indent it according to its inner structure, with each key of a dictionary on a separate line.

  If the option is not set (the default), the whole JSON object will be written on a single line, with minimum spacing between all elements.

-aligned boolean
: If this option is set the system will ensure that the values for the keys in a dictionary are vertically aligned with each other, for a nice table effect. To make this work this also implies that **-indented** is set.

  If the option is not set (the default), the output is formatted as per the value of **indented**, without trying to align the values for dictionary keys.

C parser embedded in Tcl

The **critcl** format is executable code, a parser for the grammar. It is a Tcl package with the actual parser implementation written in C and embedded in Tcl via the **critcl** package.

This result-format supports the following options:

-file string
: The value of this option is the name of the file or other entity from which the grammar came, for which the command is run. The default value is **unknown**.

-name string
: The value of this option is the name of the grammar we are processing. The default value is **a_pe_grammar**.

-user string
: The value of this option is the name of the user for which the command is run. The default value is **unknown**.

-class string

The value of this option is the name of the class to generate, without leading colons. The default value is **CLASS**.

For a simple value **X** without colons, like CLASS, the parser command will be **X**::**X**. Whereas for a namespaced value **X::Y** the parser command will be **X::Y**.

-package string
The value of this option is the name of the package to generate. The default value is **PACKAGE**.

-version string
The value of this option is the version of the package to generate. The default value is **1**.

C parser

The **c** format is executable code, a parser for the grammar. The parser implementation is written in C and can be tweaked to the users' needs through a multitude of options.

The **critcl** format, for example, is implemented as a canned configuration of these options on top of the generator for **c**.

This result-format supports the following options:

-file string
The value of this option is the name of the file or other entity from which the grammar came, for which the command is run. The default value is **unknown**.

-name string
The value of this option is the name of the grammar we are processing. The default value is **a_pe_grammar**.

-user string
The value of this option is the name of the user for which the command is run. The default value is **unknown**.

-template string
The value of this option is a string into which to put the generated text and the other configuration settings. The various locations for user-data are expected to be specified with the placeholders listed below. The default value is "**@code@**".

@user@
To be replaced with the value of the option **-user**.

@format@
To be replaced with the the constant **C/PARAM**.

@file@
To be replaced with the value of the option **-file**.

@name@
To be replaced with the value of the option **-name**.

@code@
To be replaced with the generated Tcl code.

The following options are special, in that they will occur within the generated code, and are replaced there as well.

@statedecl@

To be replaced with the value of the option **state-decl**.

@stateref@
To be replaced with the value of the option **state-ref**.

@strings@
To be replaced with the value of the option **string-varname**.

@self@
To be replaced with the value of the option **self-command**.

@def@
To be replaced with the value of the option **fun-qualifier**.

@ns@
To be replaced with the value of the option **namespace**.

@main@
To be replaced with the value of the option **main**.

@prelude@
To be replaced with the value of the option **prelude**.

-state-decl string
A C string representing the argument declaration to use in the generated parsing functions to refer to the parsing state. In essence type and argument name. The default value is the string **RDE_PARAM p**.

-state-ref string
A C string representing the argument named used in the generated parsing functions to refer to the parsing state. The default value is the string **p**.

-self-command string
A C string representing the reference needed to call the generated parser function (methods ...) from another parser fonction, per the chosen framework (template). The default value is the empty string.

-fun-qualifier string
A C string containing the attributes to give to the generated functions (methods ...), per the chosen framework (template). The default value is **static**.

-namespace string
The name of the C namespace the parser functions (methods, ...) shall reside in, or a general prefix to add to the function names. The default value is the empty string.

-main string
The name of the main function (method, ...) to be called by the chosen framework (template) to start parsing input. The default value is **__main**.

-string-varname string
The name of the variable used for the table of strings used by the generated parser, i.e. error messages, symbol names, etc. The default value is **p_string**.

-prelude string
A snippet of code to be inserted at the head of each generated parsing function. The default value is the empty string.

-indent integer
The number of characters to indent each line of the generated code by. The default value is **0**.

-comments boolean
A flag controlling the generation of code comments containing the original parsing expression a parsing function is for. The default value is **on**.

Snit parser

The **snit** format is executable code, a parser for the grammar. It is a Tcl package holding a **snit::type**, i.e. a class, whose instances are parsers for the input grammar.

This result-format supports the following options:

-file string
The value of this option is the name of the file or other entity from which the grammar came, for which the command is run. The default value is **unknown**.

-name string
The value of this option is the name of the grammar we are processing. The default value is **a_pe_grammar**.

-user string
The value of this option is the name of the user for which the command is run. The default value is **unknown**.

-class string
The value of this option is the name of the class to generate, without leading colons. Note, it serves double-duty as the name of the package to generate too, if option **-package** is not specified, see below. The default value is **CLASS**, applying if neither option **-class** nor **-package** were specified.

-package string
The value of this option is the name of the package to generate, without leading colons. Note, it serves double-duty as the name of the class to generate too, if option **-class** is not specified, see above. The default value is **PACKAGE**, applying if neither option **-package** nor **-class** were specified.

-version string
The value of this option is the version of the package to generate. The default value is **1**.

TclOO parser

The **oo** format is executable code, a parser for the grammar. It is a Tcl package holding a **TclOO** class, whose instances are parsers for the input grammar.

This result-format supports the following options:

-file string
The value of this option is the name of the file or other entity from which the grammar came, for which the command is run. The default value is **unknown**.

-name string
The value of this option is the name of the grammar we are processing. The default value is **a_pe_grammar**.

-user string
The value of this option is the name of the user for which the command is run. The default value is **unknown**.

-class string

The value of this option is the name of the class to generate, without leading colons. Note, it serves double-duty as the name of the package to generate too, if option **-package** is not specified, see below. The default value is **CLASS**, applying if neither option **-class** nor **-package** were specified.

-package string
The value of this option is the name of the package to generate, without leading colons. Note, it serves double-duty as the name of the class to generate too, if option **-class** is not specified, see above. The default value is **PACKAGE**, applying if neither option **-package** nor **-class** were specified.

-version string
The value of this option is the version of the package to generate. The default value is **1**.

Grammar container

The **container** format is another form of describing parsing expression grammars. While data in this format is executable it does not constitute a parser for the grammar. It always has to be used in conjunction with the package **pt::peg::interp**, a grammar interpreter.

The format represents grammars by a **snit::type**, i.e. class, whose instances are API-compatible to the instances of the **pt::peg::container** package, and which are preloaded with the grammar in question.

This result-format supports the following options:

-file string
The value of this option is the name of the file or other entity from which the grammar came, for which the command is run. The default value is **unknown**.

-name string
The value of this option is the name of the grammar we are processing. The default value is **a_pe_grammar**.

-user string
The value of this option is the name of the user for which the command is run. The default value is **unknown**.

-mode bulk|incremental
The value of this option controls which methods of **pt::peg::container** instances are used to specify the grammar, i.e. preload it into the container. There are two legal values, as listed below. The default is **bulk**.

bulk
In this mode the methods **start**, **add**, **modes**, and **rules** are used to specify the grammar in a bulk manner, i.e. as a set of nonterminal symbols, and two dictionaries mapping from the symbols to their semantic modes and parsing expressions.

This mode is the default.

incremental
In this mode the methods **start**, **add**, **mode**, and **rule** are used to specify the grammar piecemal, with each nonterminal having its own block of defining commands.

-template string
The value of this option is a string into which to put the generated code and the other

configuration settings. The various locations for user-data are expected to be specified with the placeholders listed below. The default value is "**@code@**".

@user@
To be replaced with the value of the option **-user**.

@format@
To be replaced with the the constant **CONTAINER**.

@file@
To be replaced with the value of the option **-file**.

@name@
To be replaced with the value of the option **-name**.

@mode@
To be replaced with the value of the option **-mode**.

@code@
To be replaced with the generated code.

Example

In this section we are working a complete example, starting with a PEG grammar and ending with running the parser generated from it over some input, following the outline shown in the figure below:

IMAGE: flow

Our grammar, assumed to the stored in the file "*calculator.peg*" is

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

From this we create a snit-based parser via

```
pt generate snit calculator.tcl -class calculator -name calculator peg
    calculator.peg
```

which leaves us with the parser package and class written to the file "*calculator.tcl*". Assuming that this package is then properly installed in a place where Tcl can find it we can now use this class via a script like

```
    package require calculator

    lassign $argv input
    set channel [open $input r]
```

```
    set parser [calculator]
    set ast [$parser parse $channel]
    $parser destroy
    close $channel

    ... now process the returned abstract syntax tree ...
```

where the abstract syntax tree stored in the variable will look like

```
set ast {Expression 0 4
    {Factor 0 4
        {Term 0 2
            {Number 0 2
                {Digit 0 0}
                {Digit 1 1}
                {Digit 2 2}
            }
        }
        {AddOp 3 3}
        {Term 4 4
            {Number 4 4
                {Digit 4 4}
            }
        }
    }
}
```

assuming that the input file and channel contained the text

```
 120+5
```

A more graphical representation of the tree would be

```
+- Digit 0 0 | 1
|            |
+- Term 0 2 --- Number 0 2 -+- Digit 1 1 | 2
|                           |            |
|                           +- Digit 2 2 | 0
|                                        |
Expression 0 4 --- Factor 0 4 -+----------------------------- AddOp 3
    3 | +
|                                        |
+- Term 4 4 --- Number 4 4 --- Digit 4 4 | 5
```

Regardless, at this point it is the user's responsibility to work with the tree to reach whatever goal she desires. I.e. analyze it, transform it, etc. The package **pt::ast** should be of help here, providing commands to walk such ASTs structures in various ways.

One important thing to note is that the parsers used here return a data structure representing the structure of the input per the grammar underlying the parser. There are *no* callbacks during

the parsing process, i.e. no *parsing actions*, as most other parsers will have.

Going back to the last snippet of code, the execution of the parser for some input, note how the parser instance follows the specified *Parser API*.

Internals

This section is intended for users of the application which wish to modify or extend it. Users only interested in the generation of parsers can ignore it.

The main functionality of the application is encapsulated in the package **pt::pgen**. Please read it for more information.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pt* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Parsing and Grammars

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

tcldocstrip

Keywords: tcldocstrip, .dtx, LaTeX, conversion, docstrip, documentation, literate programming, markup, source

Name

tcldocstrip - Tcl-based Docstrip Processor

Synopsis

tcldocstrip *output* ?*options*? *input* ?*guards*?
tcldocstrip ?*options*? *output* (?*options*? *input guards*)...
tcldocstrip -guards *input*

Description

The application described by this document, **tcldocstrip**, is a relative of **docstrip**, a simple literate programming tool for LaTeX.

tcldocstrip is based upon the package **docstrip**.

Use cases

tcldocstrip was written with the following three use cases in mind.

1. Conversion of a single input file according to the listed guards into the stripped output. This handles the most simple case of a set of guards specifying a single document found in a single input file.
2. Stitching, or the assembly of an output from several sets of guards, in a specific order, and possibly from different files. This is the second common case. One document spread over several inputs, and/or spread over different guard sets.
3. Extraction and listing of all the unique guard expressions and guards used within a document to help a person which did not author the document in question in familiarizing itself with it.

Command line

tcldocstrip *output* ?options? *input* ?*guards*?

This is the form for use case [1]. It converts the *input* file according to the specified *guards* and options. The result is written to the named *output* file. Usage of the string **-** as the name of the output signals that the result should be written to **stdout**. The guards are document-specific and have to be known to the caller. The *options* will be explained later, in section Options.

path *output* (in)

This argument specifies where to write the generated document. It can be the path to a file or directory, or **-**. The last value causes the application to write the generated documented to **stdout**.

If the *output* does not exist then [file dirname $output] has to exist and must be a writable directory.

path *inputfile* (in)

This argument specifies the path to the file to process. It has to exist, must be readable, and written in *docstrip* format.

tcldocstrip ?options? *output* (?options? *input guards*)...

This is the form for use case [2]. It differs from the form for use case [1] by the possibility of having options before the output file, which apply in general, and specifying more than one inputfile, each with its own set of input specific options and guards.

It extracts data from the various *input* files, according to the specified *options* and *guards*, and writes the result to the given *output*, in the order of their specification on the command line. Options specified before the output are global settings, whereas the options specified before each input are valid only just for this input file. Unspecified values are taken from the global settings, or defaults. As for form [1] using the string **-** as output causes the application to write to stdout. Using the string**.** for an input file signals that the last input file should be used again. This enables the assembly of the output from one input file using multiple and different sets of guards, without having to specify the full name of the file every time.

tcldocstrip -guards *input*

This is the form for use case [3]. It determines the guards, and unique guard expressions used within the provided *input* document. The found strings are written to stdout, one string per line.

Options

This section describes all the options available to the user of the application, with the exception

of the option **-guards**. This option was described already, in section Command line.

-metaprefix string
This option is inherited from the command **docstrip::extract** provided by the package **docstrip**.

It specifies the string by which the '%%' prefix of a metacomment line will be replaced. Defaults to '%%'. For Tcl code this would typically be '#'.

-onerror mode
This option is inherited from the command **docstrip::extract** provided by the package **docstrip**.

It controls what will be done when a format error in the *text* being processed is detected. The settings are:

ignore
Just ignore the error; continue as if nothing happened.

puts
Write an error message to **stderr**, then continue processing.

throw
Throw an error. **::errorCode** is set to a list whose first element is **DOCSTRIP**, second element is the type of error, and third element is the line number where the error is detected. This is the default.

-trimlines bool
This option is inherited from the command **docstrip::extract** provided by the package **docstrip**.

Controls whether *spaces* at the end of a line should be trimmed away before the line is processed. Defaults to **true**.

-preamble text
see below ...

-postamble text
see below ...

-nopreamble
see below ...

-nopostamble
The -no*amble options deactivate file pre- and postambles altogether, whereas the -*amble options specify the *user* part of the file pre- and postambles. This part can be empty, in that case only the standard parts are shown. This is the default.

Preambles, when active, are written before the actual content of a generated file. In the same manner postambles are, when active, written after the actual content of a generated file.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *docstrip* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Documentation tools

Copyright
```
Copyright (c) 2005 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

devdoc

tcl_community_communication
Keywords: tcl_community_communication

Name
tcl_community_communication - Tcl Community - Kind Communication

Description
The Tcl Community encourages contributions from anyone who wishes to advance the development of:

- The Tcl Language
- Tcl derived languages
- Tcl related libraries
- Tcl extensions
- External Projects that Integrate Tcl

We welcome those contributions from anyone. We are blind to gender, race, religion, cultural background, cybernetic nature, and any other demographic characteristics, as well as personal political views.

A community lives and dies by communications. And occasionally our communications are peppered with patterns that are harsh, unfriendly, unwelcoming and/or otherwise unkind. As a volunteer community, we need all of the help we can get. Therefore, we ask all contributors to make a conscious effort, in Tcl Community discussions, to communicate in ways that are welcoming. Ways that are friendly. Ways that are, in a word: kind.

These guidelines suggest specific ways to accomplish that goal.

Please note: for the balance of this document any reference to "People", "Persons", "anybody" or "somebody" can refer to any sentient being, not merely corporeal members of the species Homo Sapien.

We are a Sanctuary not a Clubhouse
: The Tcl Community is a collective of amateurs and professionals who code, test, and use tools. Our community is open to all. There is no velvet rope. There is no bouncer at the door. There are no secret handshakes. Any sentient being who enters our midst is welcome. If

someone is ever asked to leave, it is only because they are being disruptive to the functioning of the community.

We Merit Ideas, Not People

A good idea can come from anyone, regardless of how little time they have been with us. A bad idea can come from anyone, regardless of how much time or how little time they have been with us. We judge a concept by how it stands up to scrutiny of logic, implementation, and regression testing. We don't judge ideas based on who had the idea first, who agrees with the idea, or who disagrees with it.

Treat Everyone with Respect

Everyone is deserving of respect and courtesy at all times.

Refer to people by the names they use.

If grammar requires you to state a gender for a person, honor their preferences about their gender identity. If you are unsure as to the gender of an individual, ask. If someone had to guess about your gender and got it wrong, please correct them and do not take it personally.

Do not take a harsh tone towards other participants.

Do not make personal attacks against anyone (participant or not.)

Criticize statements and actions, never people.

Don't Take Things Personally

When in doubt, assume the best in people. A criticism of your statements is not a personal attack on you.

Persons, not People

Stereotypes are an unhelpful tool on many accounts. They are generally oversimplified. They are usually flat out wrong. And even if "right" they are of absolutely no utility in determining the capabilities, motivations, or fitness of an individual.

Don't use them in Tcl Community communications.

Mistakes Happen

The human condition is a series of trials and errors. Progress is when we get one more trial than error. Being wrong or making a mistake is the default state of humanity. Accept the errors of your fellow sentient beings, and be aware that you are also fallible.

Keep it Real

Please respond to what people actually say. We are all amazing individuals, but none among us are mind readers. If you find yourself responding to what you imagine someone is thinking, odds are you are going to be wrong.

If you must criticize someone, stick to things they have actually done. Never criticize for something you speculate they have done. Or imagine they have done. Or something someone who shares some attribute with them has done in the past.

Keep discussions about any non-Tcl subjects to what can be stated factually and without emotion or judgement.

When Trouble Arises, Don't Escalate

If you feel you are being personally attacked or offended, take the high road. Punching back in a public forum will only makes things worse. Address the matter in a private correspondence.

Be polite. Express your feelings, but note that you are expressing your feelings. When writing, look for a way to calm matters down. And when in doubt, sleep on your letter before pressing send. And when not in doubt, sleep on it for another day after that.

If you are a spectator to a fight in progress, politely request the two parties take the matter to a more private forum.

Always get the Last Word: I'm Sorry

If an personal argument does arise, be the first to apologize. An apology does not concede a logical point. It merely acknowledges that at some point the discussion left either logic, community decency, or both. Return to the topic when cooler heads can prevail.

Nobody is Keeping Score

There is no prize for being right. There is no cost for being wrong. A hard sell is not going to advance your idea along any more than a logical argument. You aren't running for office. This isn't debate club. If you find yourself continuing a discussion beyond where a topic can be logically discussed, stop.

No Evangelizing

The Tcl Community is not the place to promote your chosen operating system, political outlook, religion, marketing scheme, or economic model. Period.

(And if you do bring it up, be prepared to have your chosen topic discussed logically. And odds are, not favorably.)

Respect the Community

If the Community has come to a decision on a course of action, please stop arguing.

If someone complains about how you are expressing your ideas, listen.

If your words are hurting people, stop. There is no amount of being "right" that makes up for someone leaving our midst because they felt insulted, threatened, or ignored.

By following these guidelines, we will build our community, encourage more contribution to our projects, and our discussions will be friendlier and reach conclusions more easily.

Thank You.

Signatories

- Sean "the Hypnotoad" Woods
- Andreas Kupries

Authors

Primary

Sean "the Hypnotoad" Woods

Light editing

Andreas Kupries

tcllib_devguide

Keywords: tcllib_devguide

Name

tcllib_devguide - Tcllib - The Developer's Guide

Synopsis

Module *name code-action doc-action example-action*
Application *name*
Exclude *name*

Description

Welcome to Tcllib, the Tcl Standard Library. Note that Tcllib is not a package itself. It is a collection of (semi-independent) *Tcl* packages that provide utility functions useful to a large collection of Tcl programmers.

This document is a guide for developers working on Tcllib, i.e. maintainers fixing bugs, extending the collection's functionality, etc.

Please read

1. *Tcllib - How To Get The Sources* and
2. *Tcllib - The Installer's Guide*

first, if that was not done already.

Here we assume that the sources are already available in a directory of your choice, and that you not only know how to build and install them, but also have all the necessary requisites to actually do so. The guide to the sources in particular also explains which source code management system is used, where to find it, how to set it up, etc.

Commitments

Contributor

As a contributor to Tcllib you are committing yourself to:

1. keep the guidelines written down in *Tcl Community - Kind Communication* in your mind. The main point to take away from there is *to be kind to each other*.
2. Your contributions getting distributed under a BSD/MIT license. For the details see *Tcllib - License*

Contributions are made by entering tickets into our tracker, providing patches, bundles or branches of code for inclusion, or posting to the Tcllib related mailing lists.

Maintainer

When contributing one or more packages for full inclusion into Tcllib you are committing yourself to

1. Keep the guidelines written down in *Tcl Community - Kind Communication* (as any contributor) in your mind. The main point to take away from there is *to be kind to each other*.
2. Your packages getting distributed under a BSD/MIT license. For the details see *Tcllib - License*
3. Maintenance of the new packages for a period of two years under the following rules, and responsibilities:
   1. A maintainer may step down after the mandatory period as they see fit.
   2. A maintainer may step down before the end of the mandatory period, under the condition that a replacement maintainer is immediately available and has agreed to serve the

remainder of the period, plus their own mandatory period (see below).
3. When stepping down without a replacement maintainer taking over the relevant packages have to be flagged as **unmaintained**.
4. When a replacement mantainer is brought in for a package it is (kept) marked as **maintained** (again).

   A replacement maintainer is bound by the same rules as the original maintainer, except that the mandatory period of maintenance is shortened to one year.
5. For any **unmaintained** package a contributor interested in becoming its maintainer can become so by flagging them as **maintained** with their name and contact information, committing themselves to the rules of a replacement maintainer (see previous point).
6. For any already **maintained** package a contributor interested in becoming a co-maintainer can become so with the agreement of the existing maintainer(s), committing themselves to the rules of a replacement maintainer (see two points previous).

```
The responsibilities as a maintainer include:

1. Watching Tcllib's ticket tracker for bugs, bug fixes, and feature
    requests related to the new packages.

2. Reviewing the aforementioned tickets, rejecting or applying them

3. Coordination and discussion with ticket submitter during the
    development and/or application of bug fixes.
```

1. Follow the Branching and Workflow of this guide.

Branching and Workflow

Package dependencies

Regarding packages and dependencies between them Tcllib occupies a middle position between two extremes:

1. On one side a strongly interdependent set of packages, usually by a single author, for a single project. Looking at my (Andreas Kupries) own work examples of such are Marpa, CRIMP, Kinetcl, etc.

   For every change the author of the project handles all the modifications cascading from any incompatibilities it introduced to the system.
2. On the other side, the world of semi-independent projects by many different authors where authors know what packages their own creations depend on, yet usually do not know who else depends on them.

   The best thing an author making an (incompatible) change to their project can do is to for one announce such changes in some way, and for two use versioning to distinguish the code before and after the change.

   The world is then responsible for adapting, be it by updating their own projects to the new version, or by sticking to the old.

As mentioned already, Tcllib lives in the middle of that.

While we as maintainers cannot be aware of all users of Tcllib's packages, and thus have to rely on the mechanisms touched on in point 2 above for that, the dependencies between the packages contained in Tcllib are a different matter.

As we are collectively responsible for the usability of Tcllib in toto to the outside world, it behooves us to be individually mindful even of Tcllib packages we are not directly maintaining, when they depend on packages under our maintainership. This may be as simple as coordinating with the maintainers of the affected packages. It may also require us to choose how to adapt affected packages which do not have maintainers, i.e. modify them to use our changed package properly, or modify them to properly depend on the unchanged version of our package.

Note that the above is not only a chore but an opportunity as well. Additional insight can be had by forcing ourselves to look at our package and the planned change(s) from an outside perspective, to consider the ramifications of our actions on others in general, and on dependent packages in particular.

Trunk

The management and use of branches is an important part of working with a *Distributed Version Control System* (*DVCS*) like fossil.

For Tcllib the main branch of the collection is *trunk*. In *git* this branch would be called *master*, and this is exactly the case in the github mirror of Tcllib.

To properly support debugging *each commit* on this branch *has to pass the entire testsuite* of the collection. Using bisection to determine when an issue appeared is an example of an action made easier by this constraint.

This is part of our collective responsibility for the usability of Tcllib in toto to the outside world. As *fossil* has no mechanism to enforce this condition this is handled on the honor system for developers and maintainers.

To make the task easier Tcllib comes with a tool ("*sak.tcl*") providing a number of commands in support. These commands are explained in the following sections of this guide.

While it is possible and allowed to commit directly to trunk remember the above constraint regarding the testsuite, and the coming notes about other possible issues with a commit.

Branches

Given the constraints placed on the *trunk* branch of the repository it is (strongly) recommended to perform any development going beyond trivial changes on a non-trunk branch.

Outside of the trunk developers are allowed to commit intermediate broken states of their work. Only at the end of a development cycle, when the relevant branch is considered ready for merging, will it be necessary to perform full the set of validations ensuring that the merge to come will create a good commit on trunk.

Note that while a review from a second developer is not a required condition for merging a branch it is recommended to seek out such an independent opinion as a means of cross-checking the work.

It also recommended to give any new branch a name which aids in determining additional details about it. Examples of good things to stick into a branch name would be

- Developer (nick)name
- Ticket hash/reference

- One or two keywords applicable to the work
- ...

Further, while most development branches are likely quite short-lived, no prohibitions exist against making longer-lived branches. Creators should however be mindful that the longer such a branch exists without merges the more divergent they will tend to be, with an associated increase in the effort which will have to be spent on either merging from and merging to trunk.

Working with branches

In the hope of engendering good work practices now a few example operations which will come up with branches, and their associated fossil command (sequences).

Awareness

When developing we have to keep ourselves aware of the context of our work. On what branch are we ? What files have we changed ? What new files are not yet known to the repository ? What has happened remotely since we used our checkout ? The answers to these questions become especially important when using a long-lived checkout and coming back to it after some time away.

Commands to answer questions like the above are:

fossil pull
: Get all changes done on the remote since the last pull or sync from it. This has to be done first, before any of the commands below.

  Even if the commit in our checkout refers to the branch we want right now control operations committed to the remote may have changed that from underneath us.

fossil info | grep tags
: see below ...

fossil branch list | grep '*'
: Two different ways of determining the branch our checkout is on.

fossil timeline
: What have we (and others) done recently ?

  Attention, this information is very likely outdated, the more the longer we did not use this checkout. Run **fossil pull** first to get latest information from the remote repository of the project.

fossil timeline current
: Place the commit our checkout is based on at the top of the timeline.

fossil changes
: Lists the files we have changed compared to the commit the checkout is based on.

fossil extra
: Lists the files we have in the checkout the repository does not know about. This may be leftover chaff from our work, or something we have forgotten to **fossil add** to the repository yet.

Clean checkouts

Be aware of where you are (see first definition).

For pretty much all the operation recipes below a clean checkout is at least desired, often

required. To check that a checkout is clean invoke

```
fossil changes
fossil extra
```

How to clean up when uncommitted changes of all sorts are found is context-specific and outside of the scope of this guide.

Starting a new branch

Be aware of where you are (see first definition).

Ensure that you have clean checkout (see second definition). It is *required*.

In most situations you want to be on branch *trunk*, and you want to be on the latest commit for it. To get there use

```
fossil pull
fossil update trunk
```

If some other branch is desired as the starting point for the coming work replace *trunk* in the commands above with the name of that branch.

With the base line established we now have two ways of creating the new branch, with differing (dis)advantages. The simpler way is to

```
fossil branch new NAME_OF_NEW_BRANCH
```

and start developing. The advantage here is that you cannot forget to create the branch. The disadvantages are that we have a branch commit unchanged from where we branched from, and that we have to use high-handed techniques like hiding or shunning to get rid of the commit should we decide to abandon the work before the first actual commit on the branch.

The other way of creating the branch is to start developing, and then on the first commit use the option **--branch** to tell **fossil** that we are starting a branch now. I.e. run

```
fossil commit --branch NAME_OF_NEW_BRANCH ...
```

where*...* are any other options used to supply the commit message, files to commit, etc.

The (dis)advantages are now reversed.

We have no superflous commit, only what is actually developed. The work is hidden until we commit to make our first commit.

We may forget to use **--branch NAME_OF_NEW_BRANCH** and then have to correct that oversight via the fossil web interface (I am currently unaware of ways of doing such from the command line, although some magic incantantion of **fossil tag create** may work).

It helps to keep awareness, like checking before any commit that we are on the desired branch.

Merging a branch into trunk

Be aware of where you are (see first definition).

Ensure that you have clean checkout (see second definition). In the full-blown sequence

(zig-zag) it is *required*, due to the merging from trunk. In the shorter sequence it is only desired. That said, keeping the checkout clean before any major operations is a good habit to have, in my opinion.

The full-blown sequencing with checks all the way is to

1. Validate the checkout, i.e. last commit on your branch. Run the full test suite and other validations, fix all the issues which have cropped up.
2. Merge the latest state of the *trunk* (see next definition).
3. Validate the checkout again. The incoming trunk changes may have broken something now. Do any required fixes.
4. Now merge to the trunk using

```
fossil update trunk
fossil merge --integrate YOUR_BRANCH
```

5. At this point the checkout should be in the same state as at the end of point (3) above, because we resolved any issues with the trunk already. Thus a simple

```
fossil commit ...
```

6. should be sufficient now to commit the merge back and close the branch (due to the **--integrate** we used on the merge).

   The more paranoid may validate the checkout a third time before commiting.

I call this a *zig-zag merge* because of how the arrows look in the timeline, from trunk to feature branch for the first merge, and then back for the final merge.

A less paranoid can do what I call a *simple merge*, which moves step (2) after step (4) and skips step (3) entirely. The resulting shorter sequence is

1. Validate
2. Merge to trunk
3. Validate again
4. Commit to trunk

The last step after either zig-zag or plain merge is to

```
fossil sync
```

This saves our work to the remote side, and further gives us any other work done while we were doing our merge. It especially allows us to check if we raced somebody else, resulting in a split trunk.

When that happens we should coordinate with the other developer on who fixes the split, to ensure that we do not race each other again.

Merging from trunk

Be aware of where you are (see first definition).

Ensure that you have clean checkout (see second definition). It is *required*.

In most situations you want to import the latest commit of branch *trunk* (or other origin). To get it use

```
    fossil pull
```

With that done we can now import this commit into our current branch with

```
    fossil merge trunk
```

Even if **fossil** does not report any conflicts it is a good idea to check that the operation has not broken the new and/or changed functionality we are working on.

With the establishment of a good merge we then save the state with

```
    fossil commit ...
```

before continuing development.

Version numbers

In Tcllib all changes to a package have to come with an increment of its version number. What part is incremented (patchlevel, minor, major version) depends on the kind of change made. With multiple changes in a commit the highest "wins".

When working in a development branch the version change can be deferred until it is time to merge, and then has to cover all the changes in the branch.

Below a list of the kinds of changes and their associated version increments:

D - documentation
: No increment

T - testsuite
: No increment

B - bugfix
: Patchlevel

I - implementation tweak
: Patchlevel

P - performance tweak
: Patchlevel

E - backward-compatible extension
: Minor

API - incompatible change
: Major

Note that a commit containing a version increment has to mention the new version number in its commit message, as well as the kind of change which caused it.

Note further that the version number of a package currently exists in three places. An increment has to update all of them:

1. The package implementation.
2. The package index ("*pkgIndex.tcl*")
3. The package documentation.

The "*sak.tcl*" command **validate version** helps finding discrepancies between the first two. All the other **validate** methods are also of interest to any developer. Invoke it with

```
sak.tcl help validate
```

to see their documentation.

Structural overview

Main directories

The main directories in the Tcllib toplevel directory and of interest to a developer are:

"*modules*"
: Each child directory represents one or more packages. In the case of the latter the packages are usually related in some way. Examples are "*base64*", "*math*", and "*struct*", with loose (base64) to strong (math) relations between the packages in the directory.

"*apps*"
: This directory contains all the installable applications, with their documentation. Note that this directory is currently *not* split into sub-directories.

"*examples*"
: Each child directory "*foo*" contains one or more example application for the packages in "*modules/foo*". These examples are generally not polished enough to be considered for installation.

More directories

"*config*"
: This directory contains files supporting the Unix build system, i.e. "*configure*" and "*Makefile.in*".

"*devdoc*"
: This directories contains the doctools sources for the global documentation, like this document and its sibling guides.

"*embedded*"
: This directory contains the entire documentation formatted for *HTML* and styled to properly mix into the web site generated by fossil for the repository.

  This is the documentation accessible from the Tcllib home directory, represented in the repository as "*embedded/index.md*".

"*idoc*"
: This directory contains the entire documentation formatted for *nroff* and *HTML*, the latter without any styling. This is the documentation which will be installed.

"*support*"
: This directory contains the sources of internal packages and utilities used in the implementation of the "*installer.tcl*" and "*sak.tcl*" scripts/tools.

Top files

"*aclocal.m4*"
: see below ...

"*configure*"
: see below ...

"*configure.in*"

see below ...

"***Makefile.in***"
: These four files comprise the Unix build system layered on top of the "*installer.tcl*" script.

"***installer.tcl***"
: The Tcl-based installation script/tool.

"***project.shed***"
: Configuration file for *Sean Wood*'s **PracTcl** buildsystem.

"***sak.tcl***"
: This is the main tool for developers and release managers, the *Swiss Army Knife* of management operations on the collection.

"***ChangeLog***"
: The log of changes to the global support, when the sources were held in *CVS*. Not relevant any longer with the switch to the *fossil* SCM.

"***license.terms***"
: The license in plain ASCII. See also *Tcllib - License* for the nicely formatted form. The text is identical.

"***README.md***"
: see below ...

"***.github/CONTRIBUTING.md***"
: see below ...

"***.github/ISSUE_TEMPLATE.md***"
: see below ...

"***.github/PULL_REQUEST_TEMPLATE.md***"
: These markdown-formatted documents are used and shown by the github mirror of these sources, pointing people back to the official location and issue trackers.

"***DESCRIPTION.txt***"
: see below ...

"***STATUS***"
: see below ...

"***tcllib.spec***"
: see below ...

"***tcllib.tap***"
: see below ...

"***tcllib.yml***"
: ????

File types

The most common file types, by file extension, are:

"***.tcl***"
: Tcl code for a package, application, or example.

"***.man***"
: Doctools-formatted documentation, usually for a package.

"***.test***"
: Test suite for a package, or part of. Based on **tcltest**.

"***.bench***"
: Performance benchmarks for a package, or part of. Based on "*modules/bench*".

"***.pcx***"
: Syntax rules for *TclDevKit*'s **tclchecker**. Using these rules allows the checker to validate the use of commands of a Tcllib package **foo** without having to scan the "*.tcl*" files implementing it.

Testsuite tooling

Testsuites in Tcllib are based on Tcl's standard test package **tcltest**, plus utilities found in the directory "*modules/devtools*"

Tcllib developers invoke the suites through the **test run** method of the "*sak.tcl*" tool, with other methods of **test** providing management operations, for example setting a list of standard Tcl shells to use.

Invoke the testsuites of a specific module

Invoke either

```
./sak.tcl test run foo
```

or

```
./sak.tcl test run modules/foo
```

to invoke the testsuites found in a specific module "*foo*".

Invoke the testsuites of all modules

Invoke the tool without a module name, i.e.

```
./sak.tcl test run
```

to invoke the testsuites of all modules.

Detailed test logs

In all the previous examples the test runner will write a combination of progress display and testsuite log to the standard output, showing for each module only the tests that passed or failed and how many of each in a summary at the end.

To get a detailed log, it is necessary to invoke the test runner with additional options.

For one:

```
 ./sak.tcl test run --log LOG foo
```

While this shows the same short log on the terminal as before, it also writes a detailed log to the file "*LOG.log*", and excerpts to other files ("*LOG.summary*", "*LOG.failures*", etc.).

For two:

```
./sak.tcl test run -v foo
```

This writes the detailed log to the standard output, instead of the short log.

Regardless of form, the detailed log contains a list of all test cases executed, which failed, and how they failed (expected versus actual results).

Shell selection

By default the test runner will use all the Tcl shells specified via **test add** to invoke the specified testsuites, if any. If no such are specified it will fall back to the Tcl shell used to run the tool itself.

Use option **--shell** to explicitly specify the Tcl shell to use, like

```
./sak.tcl test run --shell /path/to/tclsh ...
```

Help

Invoke the tool as

```
./sak.tcl help test
```

to see the detailed help for all methods of **test**, and the associated options.

Documentation tooling

The standard format used for documentation of packages and other things in Tcllib is *doctools*. Its supporting packages are a part of Tcllib, see the directories "*modules/doctools*" and "*modules/dtplite*". The latter is an application package, with the actual application "*apps/dtplite*" a light wrapper around it.

Tcllib developers gain access to these through the **doc** method of the "*sak.tcl*" tool, another (internal) wrapper around the "*modules/dtplite*" application package.

Generate documentation for a specific module

Invoke either

```
./sak.tcl doc html foo
```

or

```
./sak.tcl doc html modules/foo
```

to generate HTML for the documentation found in the module "*foo*". Instead of **html** any other supported format can be used here, of course.

The generated formatted documentation will be placed into a directory "*doc*" in the current working directory.

Generate documentation for all modules

Invoke the tool without a module name, i.e.

```
./sak.tcl doc html
```

to generate HTML for the documentation found in all modules. Instead of **html** any other supported format can be used here, of course.

The generated formatted documentation will be placed into a directory "*doc*" in the current working directory.

Available output formats, help

Invoke the tool as

```
./sak.tcl help doc
```

to see the entire set of supported output formats which can be generated.

Validation without output

Note the special format **validate**.

Using this value as the name of the format to generate forces the tool to simply check that the documentation is syntactically correct, without generating actual output.

Invoke it as either

```
./sak.tcl doc validate (modules/)foo
```

or

```
./sak.tcl doc validate
```

to either check the packages of a specific module or check all of them.

Notes on writing a testsuite

While previous sections talked about running the testsuites for a module and the packages therein, this has no meaning if the module in question has no testsuites at all.

This section gives a very basic overview on possible methodologies for writing tests and testsuites.

First there are "drudgery" tests. Written to check absolutely basic assumptions which should never fail.

For example for a command FOO taking two arguments, three tests calling it with zero, one, and three arguments. The basic checks that the command fails if it has not enough arguments, or too many.

After that come the tests checking things based on our knowledge of the command, about its properties and assumptions. Some examples based on the graph operations added during Google's Summer of Code 2009 are:

- The BellmanFord command in struct::graph::ops takes a *startnode* as argument, and this node should be a node of the graph. This equals one test case checking the behavior when the specified node is not a node of the graph.

  This often gives rise to code in the implementation which explicitly checks the assumption and throws an understandable error, instead of letting the algorithm fail later in some weird non-deterministic way.

  It is not always possible to do such checks. The graph argument for example is just a command in itself, and while we expect it to exhibit a certain interface, i.e. a set of sub-commands aka methods, we cannot check that it has them, except by actually trying to use them. That is done by the algorithm anyway, so an explicit check is just overhead we can get by without.
- IIRC one of the distinguishing characteristic of either BellmanFord and/or Johnson is that they are able to handle negative weights. Whereas Dijkstra requires positive weights.

  This induces (at least) three testcases ... Graph with all positive weights, all negative, and a mix of positive and negative weights. Thinking further does the algorithm handle the weight **0** as well ? Another test case, or several, if we mix zero with positive and negative weights.
- The two algorithms we are currently thinking about are about distances between nodes, and

distance can be 'Inf'inity, i.e. nodes may not be connected. This means that good test cases are

1. Strongly connected graph
2. Connected graph
3. Disconnected graph.

```
At the extremes of strongly connected and disconnected we have the
    fully connected graphs and graphs without edges, only nodes, i.e.
    completely disconnected.
```

- IIRC both of the algorithms take weighted arcs, and fill in a default if arcs are left unweighted in the input graph.

This also induces three test cases:

1. Graph will all arcs with explicit weights.
2. Graph without weights at all.
3. Graph with mixture of weighted and unweighted graphs.

What was described above via examples is called *black-box* testing. Test cases are designed and written based on the developer's knowledge of the properties of the algorithm and its inputs, without referencing a particular implementation.

Going further, a complement to *black-box* testing is *white-box*. For this we know the implementation of the algorithm, we look at it and design our tests cases so that they force the code through all possible paths in the implementation. Wherever a decision is made we have a test case forcing a specific direction of the decision, for all possible combinations and directions. It is easy to get a combinatorial explosion in the number of needed test-cases.

In practice I often hope that the black-box tests I have made are enough to cover all the paths, obviating the need for white-box tests.

The above should be enough to make it clear that writing tests for an algorithm takes at least as much time as coding the algorithm, and often more time. Much more time. See for example also http://sqlite.org/testing.html, a writeup on how the Sqlite database engine is tested. Another article of interest might be https://www.researchgate.net/publication/298896236. While geared to a particular numerical algorithm it still shows that even a simple-looking algorithm can lead to an incredible number of test cases.

An interesting connection is to documentation. In one direction, the properties checked with black-box testing are exactly the properties which should be documented in the algorithm's man page. And conversely, the documentation of the properties of an algorithm makes a good reference to base the black-box tests on.

In practice test cases and documentation often get written together, cross-influencing each other. And the actual writing of test cases is a mix of black and white box, possibly influencing the implementation while writing the tests. Like writing a test for a condition like *startnode not in input graph* serving as reminder to put a check for this condition into the code.

Installation tooling

A last thing to consider when adding a new package to the collection is installation.

How to *use* the "*installer.tcl*" script is documented in *Tcllib - The Installer's Guide*.

Here we document how to extend said installer so that it may install new package(s) and/or application(s).

In most cases only a single file has to be modified, the "*support/installation/modules.tcl*" holding one command per module and application to install.

The relevant commands are:

Module *name code-action doc-action example-action*

Install the packages of module *name*, found in "*modules/name*".

The *code-action* is responsible for installing the packages and their index. The system currently provides

_tcl
: Copy all "*.tcl*" files found in "*modules/name*" into the installation.

_tcr
: As **_tcl**, copy the "*.tcl*" files found in the subdirectories of "*modules/name*" as well.

_tci
: As **_tcl**, and copy the "*tclIndex.tcl*" file as well.

_msg
: As **_tcl**, and copy the subdirectory "*msgs*" as well.

_doc
: As **_tcl**, and copy the subdirectory "*mpformats*" as well.

_tex
: As **_tcl**, and copy "*.tex*" files as well.

The *doc-action* is responsible for installing the package documentation. The system currently provides

_null
: No documentation available, do nothing.

_man
: Process the "*.man*" files found in "*modules/name*" and install the results (nroff and/or HTML) in the proper location, as given to the installer.

  This is actually a fallback, normally the installer uses the pre-made formatted documentation found under "*idoc*".

The *example-action* is responsible for installing the examples. The system currently provides

_null
: No examples available, do nothing.

_exa
: Copy the the directory "*examples/name*" recursively to the install location for examples.

Application *name*

Install the application with *name*, found in "*apps*".

Exclude *name*

This command signals to the installer which of the listed modules to *not* install. I.e. they name the deprecated modules of Tcllib.

If, and only if the above actions are not suitable for the new module then a second file has to be modified, "*support/installation/actions.tcl*".

This file contains the implementations of the available actions, and is the place where any custom action needed to handle the special circumstances of module has to be added.

tcllib_installer

Keywords: tcllib_installer

Name

tcllib_install_guide - Tcllib - The Installer's Guide

Description

Welcome to Tcllib, the Tcl Standard Library. Note that Tcllib is not a package itself. It is a collection of (semi-independent) *Tcl* packages that provide utility functions useful to a large collection of Tcl programmers.

This document is targeted at anybody wishing to build and install the packages found in Tcllib, for either themselves, or others.

We additionally provide

1. *Tcllib - The Developer's Guide*.

for developers wishing to work on the packages themselves.

Please read *Tcllib - How To Get The Sources* first, if that was not done already. Here we assume that the sources are already available in a directory of your choice.

Requisites

Before Tcllib can be build and used a number of requisites must be installed. These are:

1. The scripting language Tcl. For details see Tcl.
2. Optionally, the **critcl** package (C embedding) for Tcl. For details see Critcl.

This list assumes that the machine where Tcllib is to be installed is essentially clean. Of course, if parts of the dependencies listed below are already installed the associated steps can be skipped. It is still recommended to read their sections though, to validate that the dependencies they talk about are indeed installed.

Tcl

As we are installing a number of Tcl packages and applications it should be pretty much obvious that a working installation of Tcl itself is needed, and I will not belabor the point.

Out of the many possibilities use whatever you are comfortable with, as long as it provides at the very least Tcl 8.2, or higher. This may be a Tcl installation provided by your operating system distribution, from a distribution-independent vendor, or built by yourself.

Note that the packages in Tcllib have begun to require 8.4, 8.5, and even 8.6. Older versions of Tcl will not be able to use such packages. Trying to use them will result in *package not found* errors, as their package index files will not register them in versions of the core unable to use them.

Myself, I used (and still use) ActiveState's ActiveTcl 8.5 distribution during development, as I am most familiar with it.

(Disclosure: I, Andreas Kupries, worked for ActiveState until 2016, maintaining ActiveTcl and TclDevKit for them).. I am currently working for SUSE Software Canada ULC, although not in Tcl-related areas.

This distribution can be found at http://www.activestate.com/activetcl. Retrieve the archive of ActiveTcl 8.5 (or higher) for your platform and install it as directed by ActiveState.

For those wishing to build and install Tcl on their own, the relevant sources can be found at

Tcl
: http://core.tcl-lang.org/tcl/

together with the necessary instructions on how to build it.

If there are problems with building, installing, or using Tcl, please file a ticket against *Tcl*, or the vendor of your distribution, and *not Tcllib*.

Critcl

The **critcl** tool is an *optional* dependency.

It is only required when trying to build the C-based *accelerators* for a number of packages, as explained in Critcl & accelerators

Beware: The C accelerators are *built by default*.

For Unix Tcllib's build system (**sak.tcl**) looks for it in the, using the name **critcl**.

On Windows on the other hand the search is more complex. The build system looks first looks for a proper application **critcl.exe**. When that is not found it looks for a combination of interpreter (**tclkitsh.exe**, **tclsh.exe**) and starkit (**critcl.kit**, **critcl**) instead. *Note* that the choice of starkit can be overriden via the environment variable.

Tcllib 1.21 and older requires Critcl version 2 or higher.

Tcllib 2.0 and newer on the other hand requires Critcl version 3.3 or higher.

The Critcl provides these releases and the associated sources.

Any branch of the repository can be used (if not using the prebuild starkit or starpack), although the use of the stable branch *master* is recommended.

At the above url is also an explanation on how to build and install Critcl, including a list of its dependencies.

Its instructions will not be repeated here. If there are problems with these directions please file a ticket against the *Critcl* project, and not Tcllib.

Build & installation instructions

As Tcllib is mainly a bundle of packages written in pure Tcl building it is the same as installing it. The exceptions to this have their own subsection, Critcl & accelerators, later on.

Before that however comes the standard case, differentiated by the platforms with material differences in the instruction, i.e. *Unix*-like, versus *Windows*.

Regarding the latter it should also be noted that it is possible set up an *Unix*-like environment using projects like *MSYS*, *Cygwin*, and others. In that case the user has the choice of which instructions to follow.

Regardless of environment or platform, a suitable *Tcl* has to be installed, and its **tclsh** should be placed on the (*Unix*) or associated with "*.tcl*" files (*Windows*).

Installing on Unix

For *Unix*-like environments Tcllib comes with the standard set of files to make

```
./configure
make install
# or just
make
```

a suitable way of installing it. This is a standard non-interactive install automatically figuring out where to place everything, i.e. packages, applications, and the manpages.

This includes the Tcllibc binary package. Therefore requires an installation of Critcl.

To install only specific parts of the project use

```
make install-binaries      # Tcllibc. Requires an installation of
  Critcl.
make install-tcl           # Tcl packages and applications.
make install-libraries     # Tcl packages alone.
make install-applications  # Applications alone.
make install-doc           # Nroff manpages.
```

For a graphical installer invoke

```
./installer.tcl
```

instead. *Note* however that this installer handles only the Tcl packages, applications, and the documentation. It does *not* handle the binary Tcllibc.

Installing on Windows

In a Windows environment we have the **installer.tcl** script to perform installation.

If the desired **tclsh** is associated "*.tcl*" files then double-clicking / opening the **installer.tcl** is enough to invoke it in graphical mode. This assumes that *Tk* is installed and available as well.

Without *Tk* the only way to invoke the installer are to open a DOS window, i.e. **cmd.exe**, and then to invoke

```
./installer.tcl
```

inside it. *Note* that this installer handles only the Tcl packages, applications, and the documentation. It does *not* handle the binary Tcllibc.

Critcl & accelerators

While the majority of Tcllib consists of packages written in pure Tcl a number of packages also have *accelerators* associated with them.

These are **critcl**-based C packages whose use will boost the performance of the packages using them. While they are built by default on Unix (see *make install*), they are not required, they are *optional*.

When building the accelerators the normally optional dependency on **critcl** becomes required. If they are built according to the instructions below then they will also be installed as well.

To build and install Tcllib with the accelerators in a Unix-like environment invoke:

```
./configure
make
# or
make install
# or
make install-binaries
```

The underlying tool is "*sak.tcl*" in the toplevel directory of Tcllib and the **make install-binaries** target is just a wrapper around

```
./sak.tcl critcl
```

Therefore in a Windows environment instead invoke

```
./sak.tcl critcl
./installer.tcl
```

from within a DOS window, i.e. **cmd.exe**.

Beware that the directory tree generated by this command sequence differs from the directory tree generated by **make install** and variants.

make install generates two package directories, one for Tcllib, the other for Tcllibc.

The above sequence on the other hand places Tcllibc into a sub-directory of the Tcllib package directory.

Tooling

The core of Tcllib's build system is the script "*installer.tcl*" found in the toplevel directory of a checkout or release.

The

```
        configure ; make install
```

setup available to developers on Unix-like systems is just a wrapper around it. To go beyond the standard embodied in the wrapper it is necessary to directly invoke this script.

On Windows system using it directly is the only way to invoke it.

For basic help invoke it as

```
        ./installer.tcl -help
```

This will print a short list of all the available options to the standard output channel.

The commands associated with the various *install* targets in the *Makefile.in* for Unix can be used as additional examples on how to use this tool as well.

The installer can operate in GUI and CLI modes. By default it chooses the mode automatically, based on if the Tcl package **Tk** can be used or not. The option **-no-gui** can be used to force CLI mode.

Note that it is possible to specify options on the command line even if the installer ultimatively selects GUI mode. In that case the hardwired defaults and the options determine the data presented to the user for editing.

The installer will select a number of defaults for the locations of packages, examples, and

documentation, and also the format of the documentation. The user can overide these defaults in the GUI, or by specifying additional options. The defaults depend on the platform detected (Unix/Windows) and on the **tclsh** executable used to run the installer.

Options

-help
Show the list of options explained here on the standard output channel and exit.

+excluded
Include deprecated packages in the installation.

-no-gui
Force command line operation of the installer

-no-wait
In CLI mode the installer will by default ask the user to confirm that the chosen configuration (destination paths, things to install) is correct before performing any action. Using this option causes the installer to skip this query and immediately jump to installation.

-app-path *path*
see below ...

-example-path *path*
see below ...

-html-path *path*
see below ...

-nroff-path *path*
see below ...

-pkg-path *path*
Declare the destination paths for the applications, examples, html documentation, nroff manpages, and packages. The defaults are derived from the location of the **tclsh** used to run the installer.

-dry-run
see below ...

-simulate
Run the installer without modifying the destination directories.

-apps
see below ...

-no-apps
see below ...

-examples
see below ...

-no-examples
see below ...

-pkgs
see below ...

-no-pkgs
see below ...

-html

see below ...

-no-html
see below ...

-nroff
see below ...

-no-nroff
(De)activate the installation of applications, examples, packages, html documentation, and nroff manpages.

Applications, examples, and packages are installed by default.

On Windows the html documentation is installed by default.

On Unix the nroff manpages are installed by default.

tcllib_license

Keywords: tcllib_license

Name

tcllib_license - Tcllib - License

Description

Welcome to Tcllib, the Tcl Standard Library. Note that Tcllib is not a package itself. It is a collection of (semi-independent) *Tcl* packages that provide utility functions useful to a large collection of Tcl programmers.

The collection is under the BSD license.

License

This software is copyrighted by Ajuba Solutions and other parties. The following terms apply to all files associated with the software unless explicitly disclaimed in individual files.

The authors hereby grant permission to use, copy, modify, distribute, and license this software and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions. No written agreement, license, or royalty fee is required for any of the authorized uses. Modifications to this software may be copyrighted by their authors and need not follow the licensing terms described here, provided that the new terms are clearly indicated on the first page of each file where they apply.

IN NO EVENT SHALL THE AUTHORS OR DISTRIBUTORS BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OF THIS SOFTWARE, ITS DOCUMENTATION, OR ANY DERIVATIVES THEREOF, EVEN IF THE AUTHORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

THE AUTHORS AND DISTRIBUTORS SPECIFICALLY DISCLAIM ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. THIS SOFTWARE IS PROVIDED ON AN "AS IS" BASIS, AND THE AUTHORS AND DISTRIBUTORS HAVE NO OBLIGATION TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

GOVERNMENT USE: If you are acquiring this software on behalf of the U.S. government, the Government shall have only "Restricted Rights" in the software and related documentation as defined in the Federal Acquisition Regulations (FARs) in Clause 52.227.19 (c) (2). If you are acquiring the software on behalf of the Department of Defense, the software shall be classified as "Commercial Computer Software" and the Government shall have only "Restricted Rights" as defined in Clause 252.227-7013 (c) (1) of DFARs. Notwithstanding the foregoing, the authors grant the U.S. Government and others acting in its behalf permission to use and distribute the software in accordance with the terms specified in this license.

tcllib_releasemgr

Keywords: tcllib_releasemgr

Name

tcllib_releasemgr - Tcllib - The Release Manager's Guide

Description

Welcome to Tcllib, the Tcl Standard Library. Note that Tcllib is not a package itself. It is a collection of (semi-independent) *Tcl* packages that provide utility functions useful to a large collection of Tcl programmers.

The audience of this document is the release manager for Tcllib, their deputies, and anybody else interested in the task of creating an official release of Tcllib for distribution.

Please read *Tcllib - How To Get The Sources* first, if that was not done already. Here we assume that the sources are already available in a directory of your choice.

Tools

The "*sak.tcl*" script in the toplevel directory of a Tcllib checkout is the one tool used by the release manager to perform its Tasks.

The main commands to be used are

```
    sak.tcl validate
    sak.tcl test run
    sak.tcl review
    sak.tcl readme
    sak.tcl localdoc
    sak.tcl release
```

More detail will be provided in the explanations of the various Tasks.

Tasks

Start a release candidate

todo: open a candidate for release

Ready the candidate

todo: test, validate and check that the candidate is worthy of release fix testsuites, possibly fix packages, documentation regenerate docs coordinate with package maintainers wrt fixes big thing: going over the packages, classify changes since last release to generate a nice readme.

Make it official

todo: finalize release, make candidate official

Distribute the release

With the release made it has to be published and the world notified of its existence.

1. Create a proper fossil event for the release, via http://core.tcl-lang.org/tcllib/eventedit.

   An existing event should be used as template.
2. Update a number of web locations:
   1. Home page
   2. Downloads
   3. Past Releases
   4. http://www.tcl-lang.org/home/release.txt
   5. http://www.tcl-lang.org/software/tcllib/*.tml
   6. http://wiki.tcl-lang.org/page/Tcllib

1. The first location maps to the file "*embedded/index.md*" in the repository itself, as such it can edited as part of the release process. This is where reference to the new fossil event is added, as the new current release.

   The next two locations are in the fossil tcllib wiki and require admin or wiki write permissions for http://core.tcl-lang.org/tcllib.

   The last two locations require ssh access to http://www.tcl-lang.org and permission to edit files in the web area.
2. ***TODO*** mailing lists and other places to send notes to.

tcllib_sources

Keywords: tcllib_sources

Name

tcllib_sources - Tcllib - How To Get The Sources

Description

Welcome to Tcllib, the Tcl Standard Library. Note that Tcllib is not a package itself. It is a collection of (semi-independent) *Tcl* packages that provide utility functions useful to a large collection of Tcl programmers.

The audience of this document is anyone wishing to either have just a look at Tcllib's source code, or build the packages, or to extend and modify them.

For builders and developers we additionally provide

1. *Tcllib - The Installer's Guide*.
2. *Tcllib - The Developer's Guide*.

respectively.

Source location

The official repository for Tcllib is found at http://core.tcl-lang.org/tcllib. This repository is managed by the Fossil SCM.

Get archives for head and releases

This is done easiest by going to the official repository and following the links in the *Releases* section at the top, immediately underneath the entry field for searching the package documentation.

Retrieval of arbitrary commits

For anything beyond head state and releases the process is a bit more involved.

If the commit id (commit hash) **((ID))** of the revision of interest is already known then links to the desired archives can be constructed using the forms below:

```
    https://core.tcl-lang.org/tcllib/tarball/((ID))/Tcl+Library+Source+
    Code.tar.gz
    https://core.tcl-lang.org/tcllib/zip/((ID))/Tcl+Library+Source+Code
    .zip
```

Note that branch names can be used for the **((ID))** also, this returns archives containing the head revision of the named branch.

The part of of the url after the **((ID))** is the name of the file to return and can be modified to suit.

Without a known commit id the process is longer again:

1. Go to the official repository.
2. Find the login link/button in the top right corner of the page.
3. Log in as "anonymous", using the semi-random password in the captcha.
4. Go to the "Timeline" following the link/button in the middle of the nav bar.
5. Choose the revision you wish to have.
6. Follow its link to its detailed information page.
7. On that page, choose either the "ZIP" or "Tarball" link to get a copy of this revision in the format of your choice.

Source Code management

The sources are managed with the Fossil SCM. Binaries for popular platforms can be found directly at its download page.

With that tool available the full history can be retrieved via:

```
    fossil clone  http://core.tcl-lang.org/tcllib  tcllib.fossil
```

followed by

```
    mkdir tcllib
    cd tcllib
    fossil open ../tcllib.fossil
```

to get a checkout of the head of the trunk.

aes

aes

Keywords: aes, block cipher, data integrity, encryption, security

Name

aes - Implementation of the AES block cipher

Synopsis

package require Tcl 8.5 9
package require aes ?**1.2.2**?
::aes::aes ?*-mode [ecb|cbc]*? ?*-dir [encrypt|decrypt]*? *-key keydata* ?*-iv vector*? ?*-hex*? ?*-out channel*? ?*-chunksize size*? [*-in channel* | ?**--**? *data*]
::aes::Init *mode keydata iv*
::aes::Encrypt *Key data*
::aes::Decrypt *Key data*
::aes::Reset *Key iv*
::aes::Final *Key*

Description

This is a Tcl implementation of the Advanced Encryption Standard (AES) as published by the U.S. National Institute of Standards and Technology [1]. AES is a 128-bit block cipher with a variable key size of 128, 192 or 256 bits. This implementation supports CBC and ECB modes.

Commands

::aes::aes ?*-mode [ecb|cbc]*? ?*-dir [encrypt|decrypt]*? *-key keydata* ?*-iv vector*? ?*-hex*? ?*-out channel*? ?*-chunksize size*? [*-in channel* | ?--? *data*]

Perform the **aes** algorithm on either the data provided by the argument or on the data read from the *-in* channel. If an *-out* channel is given then the result will be written to this channel.

The *-key* option must be given. This parameter takes a binary string of either 16, 24 or 32 bytes in length and is used to generate the key schedule.

The *-mode* and *-dir* options are optional and default to CBC mode and encrypt respectively. The initialization vector *-iv* takes a 16 byte binary argument which defaults to all zeros. See Modes of operation for more about available modes and their uses.

Since AES is a 128-bit block cipher, the data must be provided in units that are a multiple of 16 bytes.

Programming interface

Internal state is maintained in an opaque structure that is returned from the **Init** function. For CBC mode some input dependent state is maintained and may be reset by calling the **Reset** function with a new initialization vector value. In ECB mode the state is not affected by the input.

::aes::Init *mode keydata iv*

Construct a new AES key schedule using the specified key data and the given initialization vector. The initialization vector is important for CBC mode; it is not used with ECB mode. See Modes of operation for details about cipher modes.

::aes::Encrypt *Key data*

Use a prepared key acquired by calling **Init** to encrypt the provided data. The data argument

should be a binary array that is a multiple of the AES block size of 16 bytes. The result is a binary array the same size as the input of encrypted data.

::aes::Decrypt *Key data*
Decipher data using the key. Note that the same key may be used to encrypt and decrypt data provided that the initialization vector is reset appropriately for CBC mode.

::aes::Reset *Key iv*
Reset the initialization vector. This permits the programmer to re-use a key and avoid the cost of re-generating the key schedule where the same key data is being used multiple times.

::aes::Final *Key*
This should be called to clean up resources associated with *Key*. Once this function has been called the key may not be used again.

Modes of operation

Cipher Block Chaining (CBC)
CBC mode uses the output of the last block encryption to affect the current block. An initialization vector of the same size as the cipher block size is used to handle the first block. The initialization vector should be chosen randomly and transmitted as the first block of the output. Errors in encryption affect the current block and the next block after which the cipher will correct itself. CBC is the most commonly used mode in software encryption, and this module's default mode of operation.

Electronic Code Book (ECB)
ECB is the basic mode of all block ciphers. Each block is encrypted independently and so identical plain text will produce identical output when encrypted with the same key. Any encryption errors will only affect a single block. *This mode is vulnerable to known plaintext attacks.*

Examples

```
% set nil_block [string repeat \\0 16]
% aes::aes -hex -mode cbc -dir encrypt -key $nil_block $nil_block
66e94bd4ef8a2c3b884cfa59ca342b2e
```

```
set Key [aes::Init cbc $sixteen_bytes_key_data $sixteen_byte_iv]
append ciphertext [aes::Encrypt $Key $plaintext]
append ciphertext [aes::Encrypt $Key $additional_plaintext]
aes::Final $Key
```

References

1. "Advanced Encryption Standard", Federal Information Processing Standards Publication 197, 2001 (http://csrc.nist.gov/publications/fips/fips197/fips-197.pdf)

Authors

Thorsten Schloermann, Pat Thoyts

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib

Trackers and specify *aes* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Hashes, checksums, and encryption

Copyright

```
Copyright (c) 2005, Pat Thoyts <patthoyts@users.sourceforge.net>
Copyright (c) 2012-2014, Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


amazon-s3

S3
Keywords: S3, amazon, cloud

Name
S3 - Amazon S3 Web Service Interface

Synopsis
package require Tcl 8.5 9
package require S3 ?**1.0.5**?
package require sha1 1.1
package require md5 2.1
package require base64 2.4
package require xsxp 1.1
S3::Configure ?*-reset boolean*? ?*-retries integer*? ?*-accesskeyid idstring*? ?*-secretaccesskey idstring*? ?*-service-access-point FQDN*? ?*-use-tls boolean*? ?*-default-compare always|never|exists|missing|newer|date|checksum|different*? ?*-default-separator string*? ?*-default-acl private|public-read|public-read-write|authenticated-read|keep|calc*? ?*-default-bucket bucketname*?
S3::SuggestBucket ?*name*?
S3::REST *dict*
S3::ListAllMyBuckets ?*-blocking boolean*? ?*-parse-xml xmlstring*? ?*-result-type REST|xml|pxml|dict|names|owner*?
S3::PutBucket ?*-bucket bucketname*? ?*-blocking boolean*? ?*-acl {}|private|public-read|public-read-write|authenticated-read*?
S3::DeleteBucket ?*-bucket bucketname*? ?*-blocking boolean*?
S3::GetBucket ?*-bucket bucketname*? ?*-blocking boolean*? ?*-parse-xml xmlstring*? ?*-max-count integer*? ?*-prefix prefixstring*? ?*-delimiter delimiterstring*? ?*-result-type REST|xml|pxml|names|dict*?
S3::Put ?*-bucket bucketname*? **-resource** *resourcename* ?*-blocking boolean*? ?*-file filename*? ?

-content contentstring? ?*-acl private|public-read|public-read-write|authenticated-read|calc|keep*? ?*-content-type contenttypestring*? ?*-x-amz-meta-* metadatatext*? ?*-compare comparemode*?
S3::Get ?*-bucket bucketname*? **-resource** *resourcename* ?*-blocking boolean*? ?*-compare comparemode*? ?*-file filename*? ?*-content contentvarname*? ?*-timestamp aws|now*? ?*-headers headervarname*?
S3::Head ?*-bucket bucketname*? **-resource** *resourcename* ?*-blocking boolean*? ?*-dict dictvarname*? ?*-headers headersvarname*? ?*-status statusvarname*?
S3::GetAcl ?*-blocking boolean*? ?*-bucket bucketname*? **-resource** *resourcename* ?*-result-type REST|xml|pxml*?
S3::PutAcl ?*-blocking boolean*? ?*-bucket bucketname*? **-resource** *resourcename* ?*-acl new-acl*?
S3::Delete ?*-bucket bucketname*? **-resource** *resourcename* ?*-blocking boolean*? ?*-status statusvar*?
S3::Push ?*-bucket bucketname*? **-directory** *directoryname* ?*-prefix prefixstring*? ?*-compare comparemode*? ?*-x-amz-meta-* metastring*? ?*-acl aclcode*? ?*-delete boolean*? ?*-error throw|break|continue*? ?*-progress scriptprefix*?
S3::Pull ?*-bucket bucketname*? **-directory** *directoryname* ?*-prefix prefixstring*? ?*-blocking boolean*? ?*-compare comparemode*? ?*-delete boolean*? ?*-timestamp aws|now*? ?*-error throw|break|continue*? ?*-progress scriptprefix*?
S3::Toss ?*-bucket bucketname*? **-prefix** *prefixstring* ?*-blocking boolean*? ?*-error throw|break|continue*? ?*-progress scriptprefix*?

Description

This package provides access to Amazon's Simple Storage Solution web service.

As a quick summary, Amazon Simple Storage Solution provides a for-fee web service allowing the storage of arbitrary data as "resources" within "buckets" online. See http://www.amazonaws.com/ for details on that system. Access to the service is via HTTP (SOAP or REST). Much of this documentation will not make sense if you're not familiar with the terms and functionality of the Amazon S3 service.

This package provides services for reading and writing the data items via the REST interface. It also provides some higher-level operations. Other packages in the same distribution provide for even more functionality.

Copyright 2006 Darren New. All Rights Reserved. NO WARRANTIES OF ANY TYPE ARE PROVIDED. COPYING OR USE INDEMNIFIES THE AUTHOR IN ALL WAYS. This software is licensed under essentially the same terms as Tcl. See LICENSE.txt for the terms.

Error reporting

The error reporting from this package makes use of $errorCode to provide more details on what happened than simply throwing an error. Any error caught by the S3 package (and we try to catch them all) will return with an $errorCode being a list having at least three elements. In all cases, the first element will be "S3". The second element will take on one of six values, with that element defining the value of the third and subsequent elements. S3::REST does not throw an error, but rather returns a dictionary with the keys "error", "errorInfo", and "errorCode" set. This allows for reliable background use. The possible second elements are these:

usage
The usage of the package is incorrect. For example, a command has been invoked which

requires the library to be configured before the library has been configured, or an invalid combination of options has been specified. The third element of $errorCode supplies the name of the parameter that was wrong. The fourth usually provides the arguments that were actually supplied to the throwing proc, unless the usage error isn't confined to a single proc.

local
Something happened on the local system which threw an error. For example, a request to upload or download a file was made and the file permissions denied that sort of access. The third element of $errorCode is the original $errorCode.

socket
Something happened with the socket. It closed prematurely, or some other condition of failure-to-communicate-with-Amazon was detected. The third element of $errorCode is the original $errorCode, or sometimes the message from fcopy, or ...?

remote
The Amazon web service returned an error code outside the 2xx range in the HTTP header. In other words, everything went as documented, except this particular case was documented not to work. The third element is the dictionary returned from **::S3::REST**. Note that S3::REST itself never throws this error, but just returns the dictionary. Most of the higher-level commands throw for convenience, unless an argument indicates they should not. If something is documented as "not throwing an S3 remote error", it means a status return is set rather than throwing an error if Amazon returns a non-2XX HTTP result code.

notyet
The user obeyed the documentation, but the author has not yet gotten around to implementing this feature. (Right now, only TLS support and sophisticated permissions fall into this category, as well as the S3::Acl command.)

xml
The service has returned invalid XML, or XML whose schema is unexpected. For the high-level commands that accept service XML as input for parsing, this may also be thrown.

Commands

This package provides several separate levels of complexity.

- The lowest level simply takes arguments to be sent to the service, sends them, retrieves the result, and provides it to the caller. *Note:* This layer allows both synchronous and event-driven processing. It depends on the MD5 and SHA1 and base64 packages from Tcllib (available at http://core.tcl.tk/tcllib/). Note that **S3::Configure** is required for **S3::REST** to work due to the authentication portion, so we put that in the "lowest level."
- The next layer parses the results of calls, allowing for functionality such as uploading only changed files, synchronizing directories, and so on. This layer depends on the **TclXML** package as well as the included **xsxp** package. These packages are package required when these more-sophisticated routines are called, so nothing breaks if they are not correctly installed.
- Also included is a separate program that uses the library. It provides code to parse $argv0 and $argv from the command line, allowing invocation as a tclkit, etc. (Not yet implmented.)
- Another separate program provides a GUI interface allowing drag-and-drop and other such functionality. (Not yet implemented.)
- Also built on this package is the OddJob program. It is a separate program designed to allow distribution of computational work units over Amazon's Elastic Compute Cloud web service.

The goal is to have at least the bottom-most layers implemented in pure Tcl using only that which comes from widely-available sources, such as Tcllib.

Low level commands

These commands do not require any packages not listed above. They talk directly to the service, or they are utility or configuration routines. Note that the "xsxp" package was written to support this package, so it should be available wherever you got this package.

S3::Configure ?-reset *boolean*? ?-retries *integer*? ?-accesskeyid *idstring*? ? -secretaccesskey *idstring*? ?-service-access-point *FQDN*? ?-use-tls *boolean*? ? -default-compare *always*|*never*|*exists*|*missing*|*newer*|*date*|*checksum*|*different*? ? -default-separator *string*? ?-default-acl *private*|*public-read*|*public-read-write*|*authenticated-read*|*keep*|*calc*? ?-default-bucket *bucketname*?

There is one command for configuration, and that is **S3::Configure**. If called with no arguments, it returns a dictionary of key/value pairs listing all current settings. If called with one argument, it returns the value of that single argument. If called with two or more arguments, it must be called with pairs of arguments, and it applies the changes in order. There is only one set of configuration information per interpreter.

The following options are accepted:

-reset *boolean*

By default, false. If true, any previous changes and any changes on the same call before the reset option will be returned to default values.

-retries *integer*

Default value is 3. If Amazon returns a 500 error, a retry after an exponential backoff delay will be tried this many times before finally throwing the 500 error. This applies to each call to **S3::REST** from the higher-level commands, but not to **S3::REST** itself. That is, **S3::REST** will always return httpstatus 500 if that's what it receives. Functions like **S3::Put** will retry the PUT call, and will also retry the GET and HEAD calls used to do content comparison. Changing this to 0 will prevent retries and their associated delays. In addition, socket errors (i.e., errors whose errorCode starts with "S3 socket") will be similarly retried after backoffs.

-accesskeyid *idstring*

see below ...

-secretaccesskey *idstring*

Each defaults to an empty string. These must be set before any calls are made. This is your S3 ID. Once you sign up for an account, go to http://www.amazonaws.com/, sign in, go to the "Your Web Services Account" button, pick "AWS Access Identifiers", and your access key ID and secret access keys will be available. All **S3::REST** calls are authenticated. Blame Amazon for the poor choice of names.

-service-access-point *FQDN*

Defaults to "s3.amazonaws.com". This is the fully-qualified domain name of the server to contact for **S3::REST** calls. You should probably never need to touch this, unless someone else implements a compatible service, or you wish to test something by pointing the library at your own service.

-slop-seconds *integer*

When comparing dates between Amazon and the local machine, two dates within this many seconds of each other are considered the same. Useful for clock drift correction, processing overhead time, and so on.

-use-tls *boolean*
Defaults to false. This is not yet implemented. If true, **S3::REST** will negotiate a TLS connection to Amazon. If false, unencrypted connections are used.

-bucket-prefix *string*
Defaults to "TclS3". This string is used by **S3::SuggestBucketName** if that command is passed an empty string as an argument. It is used to distinguish different applications using the Amazon service. Your application should always set this to keep from interfering with the buckets of other users of Amazon S3 or with other buckets of the same user.

-default-compare *always*|*never*|*exists*|*missing*|*newer*|*date*|*checksum*|*different*
Defaults to "always." If no -compare is specified on **S3::Put**, **S3::Get**, or **S3::Delete**, this comparison is used. See those commands for a description of the meaning.

-default-separator *string*
Defaults to "/". This is currently unused. It might make sense to use this for **S3::Push** and **S3::Pull**, but allowing resources to have slashes in their names that aren't marking directories would be problematic. Hence, this currently does nothing.

-default-acl *private*|*public-read*|*public-read-write*|*authenticated-read*|*keep*|*calc*
Defaults to an empty string. If no -acl argument is provided to **S3::Put** or **S3::Push**, this string is used (given as the x-amz-acl header if not keep or calc). If this is also empty, no x-amz-acl header is generated. This is *not* used by **S3::REST**.

-default-bucket *bucketname*
If no bucket is given to **S3::GetBucket**, **S3::PutBucket**, **S3::Get**, **S3::Put**, **S3::Head**, **S3::Acl**, **S3::Delete**, **S3::Push**, **S3::Pull**, or **S3::Toss**, and if this configuration variable is not an empty string (and not simply "/"), then this value will be used for the bucket. This is useful if one program does a large amount of resource manipulation within a single bucket.

S3::SuggestBucket ?*name*?
The **S3::SuggestBucket** command accepts an optional string as a prefix and returns a valid bucket containing the *name* argument and the Access Key ID. This makes the name unique to the owner and to the application (assuming the application picks a good *name* argument). If no name is provided, the name from **S3::Configure** *-bucket-prefix* is used. If that too is empty (which is not the default), an error is thrown.

S3::REST *dict*
The **S3::REST** command takes as an argument a dictionary and returns a dictionary. The return dictionary has the same keys as the input dictionary, and includes additional keys as the result. The presence or absence of keys in the input dictionary can control the behavior of the routine. It never throws an error directly, but includes keys "error", "errorInfo", and "errorCode" if necessary. Some keys are required, some optional. The routine can run either in blocking or non-blocking mode, based on the presense of **resultvar** in the input dictionary. This requires the *-accesskeyid* and *-secretaccesskey* to be configured via **S3::Configure** before being called.

The possible input keys are these:
verb *GET*|*PUT*|*DELETE*|*HEAD*

This required item indicates the verb to be used.

resource *string*
This required item indicates the resource to be accessed. A leading / is added if not there already. It will be URL-encoded for you if necessary. Do not supply a resource name that is already URL-encoded.

?rtype *torrent*|*acl*?
This indicates a torrent or acl resource is being manipulated. Do not include this in the **resource** key, or the "?" separator will get URL-encoded.

?parameters *dict*?
This optional dictionary provides parameters added to the URL for the transaction. The keys must be in the correct case (which is confusing in the Amazon documentation) and the values must be valid. This can be an empty dictionary or omitted entirely if no parameters are desired. No other error checking on parameters is performed.

?headers *dict*?
This optional dictionary provides headers to be added to the HTTP request. The keys must be in *lower case* for the authentication to work. The values must not contain embedded newlines or carriage returns. This is primarily useful for adding x-amz-* headers. Since authentication is calculated by **S3::REST**, do not add that header here. Since content-type gets its own key, also do not add that header here.

?inbody *contentstring*?
This optional item, if provided, gives the content that will be sent. It is sent with a tranfer encoding of binary, and only the low bytes are used, so use [encoding convertto utf-8] if the string is a utf-8 string. This is written all in one blast, so if you are using non-blocking mode and the **inbody** is especially large, you may wind up blocking on the write socket.

?infile *filename*?
This optional item, if provided, and if **inbody** is not provided, names the file from which the body of the HTTP message will be constructed. The file is opened for reading and sent progressively by [fcopy], so it should not block in non-blocking mode even if the file is very large. The file is transfered in binary mode, so the bytes on your disk will match the bytes in your resource. Due to HTTP restrictions, it must be possible to use [file size] on this file to determine the size at the start of the transaction.

?S3chan *channel*?
This optional item, if provided, indicates the already-open socket over which the transaction should be conducted. If not provided, a connection is made to the service access point specified via **S3::Configure**, which is normally s3.amazonaws.com. If this is provided, the channel is not closed at the end of the transaction.

?outchan *channel*?
This optional item, if provided, indicates the already-open channel to which the body returned from S3 should be written. That is, to retrieve a large resource, open a file, set the translation mode, and pass the channel as the value of the key outchan. Output will be written to the channel in pieces so memory does not fill up unnecessarily. The channel is not closed at the end of the transaction.

?resultvar *varname*?
This optional item, if provided, indicates that **S3::REST** should run in non-blocking mode. The *varname* should be fully qualified with respect to namespaces and cannot be local to a

proc. If provided, the result of the **S3::REST** call is assigned to this variable once everything has completed; use trace or vwait to know when this has happened. If this key is not provided, the result is simply returned from the call to **S3::REST** and no calls to the eventloop are invoked from within this call.

?throwsocket *throw*|*return*?
This optional item, if provided, indicates that **S3::REST** should throw an error if throwmode is throw and a socket error is encountered. It indicates that **S3::REST** should return the error code in the returned dictionary if a socket error is encountered and this is set to return. If **throwsocket** is set to *return* or if the call is not blocking, then a socket error (i.e., an error whose error code starts with "S3 socket" will be returned in the dictionary as **error** , **errorInfo**, and **errorCode**. If a foreground call is made (i.e., **resultvar** is not provided), and this option is not provided or is set to *throw*, then **error** will be invoked instead.

Once the call to **S3::REST** completes, a new dict is returned, either in the *resultvar* or as the result of execution. This dict is a copy of the original dict with the results added as new keys. The possible new keys are these:

error *errorstring*
see below ...

errorInfo *errorstring*
see below ...

errorCode *errorstring*
If an error is caught, these three keys will be set in the result. Note that **S3::REST** does *not* consider a non-2XX HTTP return code as an error. The **errorCode** value will be formatted according to the Error reporting description. If these are present, other keys described here might not be.

httpstatus *threedigits*
The three-digit code from the HTTP transaction. 2XX for good, 5XX for server error, etc.

httpmessage *text*
The textual result after the status code. "OK" or "Forbidden" or etc.

outbody *contentstring*
If *outchan* was not specified, this key will hold a reference to the (unencoded) contents of the body returned. If Amazon returned an error (a la the httpstatus not a 2XX value), the error message will be in **outbody** or written to **outchan** as appropriate.

outheaders *dict*
This contains a dictionary of headers returned by Amazon. The keys are always lower case. It's mainly useful for finding the x-amz-meta-* headers, if any, although things like last-modified and content-type are also useful. The keys of this dictionary are always lower case. Both keys and values are trimmed of extraneous whitespace.

High level commands

The routines in this section all make use of one or more calls to **S3::REST** to do their work, then parse and manage the data in a convenient way. All these commands throw errors as described in Error reporting unless otherwise noted.

In all these commands, all arguments are presented as name/value pairs, in any order. All the argument names start with a hyphen.

There are a few options that are common to many of the commands, and those common options are documented here.

-blocking *boolean*
: If provided and specified as false, then any calls to **S3:REST** will be non-blocking, and internally these routines will call [vwait] to get the results. In other words, these routines will return the same value, but they'll have event loops running while waiting for Amazon.

-parse-xml *xmlstring*
: If provided, the routine skips actually communicating with Amazon, and instead behaves as if the XML string provided was returned as the body of the call. Since several of these routines allow the return of data in various formats, this argument can be used to parse existing XML to extract the bits of information that are needed. It's also helpful for testing.

-bucket *bucketname*
: Almost every high-level command needs to know what bucket the resources are in. This option specifies that. (Only the command to list available buckets does not require this parameter.) This does not need to be URL-encoded, even if it contains special or non-ASCII characters. May or may not contain leading or trailing spaces - commands normalize the bucket. If this is not supplied, the value is taken from **S3::Configure -default-bucket** if that string isn't empty. Note that spaces and slashes are always trimmed from both ends and the rest must leave a valid bucket.

-resource *resourcename*
: This specifies the resource of interest within the bucket. It may or may not start with a slash - both cases are handled. This does not need to be URL-encoded, even if it contains special or non-ASCII characters.

-compare *always*|*never*|*exists*|*missing*|*newer*|*date*|*checksum*|*different*
: When commands copy resources to files or files to resources, the caller may specify that the copy should be skipped if the contents are the same. This argument specifies the conditions under which the files should be copied. If it is not passed, the result of **S3::Configure -default-compare** is used, which in turn defaults to "always." The meanings of the various values are these:

  always
  : Always copy the data. This is the default.

  never
  : Never copy the data. This is essentially a no-op, except in **S3::Push** and **S3::Pull** where the -delete flag might make a difference.

  exists
  : Copy the data only if the destination already exists.

  missing
  : Copy the data only if the destination does not already exist.

  newer
  : Copy the data if the destination is missing, or if the date on the source is newer than the date on the destination by at least **S3::Configure -slop-seconds** seconds. If the source is Amazon, the date is taken from the Last-Modified header. If the source is local, it is taken as the mtime of the file. If the source data is specified in a string rather than a file, it is taken as right now, via [clock seconds].

  date

Like *newer*, except copy if the date is newer *or* older.

checksum
Calculate the MD5 checksum on the local file or string, ask Amazon for the eTag of the resource, and copy the data if they're different. Copy the data also if the destination is missing. Note that this can be slow with large local files unless the C version of the MD5 support is available.

different
Copy the data if the destination does not exist. If the destination exists and an actual file name was specified (rather than a content string), and the date on the file differs from the date on the resource, copy the data. If the data is provided as a content string, the "date" is treated as "right now", so it will likely always differ unless slop-seconds is large. If the dates are the same, the MD5 checksums are compared, and the data is copied if the checksums differ.

Note that "newer" and "date" don't care about the contents, and "checksum" doesn't care about the dates, but "different" checks both.

S3::ListAllMyBuckets ?-blocking *boolean*? ?-parse-xml *xmlstring*? ?-result-type *REST*|*xml*|*pxml*|*dict*|*names*|*owner*?

This routine performs a GET on the Amazon S3 service, which is defined to return a list of buckets owned by the account identified by the authorization header. (Blame Amazon for the dumb names.)

-blocking *boolean*
See above for standard definition.

-parse-xml *xmlstring*
See above for standard definition.

-result-type *REST*
The dictionary returned by **S3::REST** is the return value of **S3::ListAllMyBuckets**. In this case, a non-2XX httpstatus will not throw an error. You may not combine this with *-parse-xml*.

-result-type *xml*
The raw XML of the body is returned as the result (with no encoding applied).

-result-type *pxml*
The XML of the body as parsed by xsxp::parse is returned.

-result-type *dict*
A dictionary of interesting portions of the XML is returned. The dictionary contains the following keys:

Owner/ID
The Amazon AWS ID (in hex) of the owner of the bucket.

Owner/DisplayName
The Amazon AWS ID's Display Name.

Bucket/Name
A list of names, one for each bucket.

Bucket/CreationDate
A list of dates, one for each bucket, in the same order as Bucket/Name, in ISO format (as returned by Amazon).

-result-type *names*
A list of bucket names is returned with all other information stripped out. This is the default result type for this command.

-result-type *owner*
A list containing two elements is returned. The first element is the owner's ID, and the second is the owner's display name.

S3::PutBucket ?-bucket *bucketname*? ?-blocking *boolean*? ?-acl *{}*|*private*|*public-read*|*public-read-write*|*authenticated-read*?
This command creates a bucket if it does not already exist. Bucket names are globally unique, so you may get a "Forbidden" error from Amazon even if you cannot see the bucket in **S3::ListAllMyBuckets**. See **S3::SuggestBucket** for ways to minimize this risk. The x-amz-acl header comes from the **-acl** option, or from **S3::Configure -default-acl** if not specified.

S3::DeleteBucket ?-bucket *bucketname*? ?-blocking *boolean*?
This command deletes a bucket if it is empty and you have such permission. Note that Amazon's list of buckets is a global resource, requiring far-flung synchronization. If you delete a bucket, it may be quite a few minutes (or hours) before you can recreate it, yielding "Conflict" errors until then.

S3::GetBucket ?-bucket *bucketname*? ?-blocking *boolean*? ?-parse-xml *xmlstring*? ?-max-count *integer*? ?-prefix *prefixstring*? ?-delimiter *delimiterstring*? ?-result-type *REST*|*xml*|*pxml*|*names*|*dict*?
This lists the contents of a bucket. That is, it returns a directory listing of resources within a bucket, rather than transfering any user data.

-bucket *bucketname*
The standard bucket argument.

-blocking *boolean*
The standard blocking argument.

-parse-xml *xmlstring*
The standard parse-xml argument.

-max-count *integer*
If supplied, this is the most number of records to be returned. If not supplied, the code will iterate until all records have been found. Not compatible with -parse-xml. Note that if this is supplied, only one call to **S3::REST** will be made. Otherwise, enough calls will be made to exhaust the listing, buffering results in memory, so take care if you may have huge buckets.

-prefix *prefixstring*
If present, restricts listing to resources with a particular prefix. One leading / is stripped if present.

-delimiter *delimiterstring*
If present, specifies a delimiter for the listing. The presence of this will summarize multiple resources into one entry, as if S3 supported directories. See the Amazon documentation for details.

-result-type *REST*|*xml*|*pxml*|*names*|*dict*
This indicates the format of the return result of the command.

REST
: If *-max-count* is specified, the dictionary returned from **S3::REST** is returned. If *-max-count* is not specified, a list of all the dictionaries returned from the one or more calls to **S3::REST** is returned.

xml
: If *-max-count* is specified, the body returned from **S3::REST** is returned. If *-max-count* is not specified, a list of all the bodies returned from the one or more calls to **S3::REST** is returned.

pxml
: If *-max-count* is specified, the body returned from **S3::REST** is passed throught xsxp::parse and then returned. If *-max-count* is not specified, a list of all the bodies returned from the one or more calls to **S3::REST** are each passed through xsxp::parse and then returned.

names
: Returns a list of all names found in either the Contents/Key fields or the CommonPrefixes/Prefix fields. If no *-delimiter* is specified and no *-max-count* is specified, this returns a list of all resources with the specified *-prefix*.

dict
: Returns a dictionary. (Returns only one dictionary even if *-max-count* wasn't specified.) The keys of the dictionary are as follows:

  Name
  : The name of the bucket (from the final call to **S3::REST**).

  Prefix
  : From the final call to **S3::REST**.

  Marker
  : From the final call to **S3::REST**.

  MaxKeys
  : From the final call to **S3::REST**.

  IsTruncated
  : From the final call to **S3::REST**, so always false if *-max-count* is not specified.

  NextMarker
  : Always provided if IsTruncated is true, and calculated of Amazon does not provide it. May be empty if IsTruncated is false.

  Key
  : A list of names of resources in the bucket matching the *-prefix* and *-delimiter* restrictions.

  LastModified
  : A list of times of resources in the bucket, in the same order as Key, in the format returned by Amazon. (I.e., it is not parsed into a seconds-from-epoch.)

  ETag
  : A list of entity tags (a.k.a. MD5 checksums) in the same order as Key.

  Size
  : A list of sizes in bytes of the resources, in the same order as Key.

  Owner/ID

A list of owners of the resources in the bucket, in the same order as Key.

Owner/DisplayName
A list of owners of the resources in the bucket, in the same order as Key. These are the display names.

CommonPrefixes/Prefix
A list of prefixes common to multiple entities. This is present only if *-delimiter* was supplied.

S3::Put ?-bucket *bucketname*? -resource *resourcename* ?-blocking *boolean*? ?-file *filename*? ?-content *contentstring*? ?-acl *private*|*public-read*|*public-read-write*|*authenticated-read*|*calc*|*keep*? ?-content-type *contenttypestring*? ?-x-amz-meta-* *metadatatext*? ?-compare *comparemode*?

This command sends data to a resource on Amazon's servers for storage, using the HTTP PUT command. It returns 0 if the **-compare** mode prevented the transfer, 1 if the transfer worked, or throws an error if the transfer was attempted but failed. Server 5XX errors and S3 socket errors are retried according to **S3:Configure -retries** settings before throwing an error; other errors throw immediately.

-bucket
This specifies the bucket into which the resource will be written. Leading and/or trailing slashes are removed for you, as are spaces.

-resource
This is the full name of the resource within the bucket. A single leading slash is removed, but not a trailing slash. Spaces are not trimmed.

-blocking
The standard blocking flag.

-file
If this is specified, the *filename* must exist, must be readable, and must not be a special or directory file. [file size] must apply to it and must not change for the lifetime of the call. The default content-type is calculated based on the name and/or contents of the file. Specifying this is an error if **-content** is also specified, but at least one of **-file** or **-content** must be specified. (The file is allowed to not exist or not be readable if **-compare** *never* is specified.)

-content
If this is specified, the *contentstring* is sent as the body of the resource. The content-type defaults to "application/octet-string". Only the low bytes are sent, so non-ASCII should use the appropriate encoding (such as [encoding convertto utf-8]) before passing it to this routine, if necessary. Specifying this is an error if **-file** is also specified, but at least one of **-file** or **-content** must be specified.

-acl
This defaults to **S3::Configure -default-acl** if not specified. It sets the x-amz-acl header on the PUT operation. If the value provided is *calc*, the x-amz-acl header is calculated based on the I/O permissions of the file to be uploaded; it is an error to specify *calc* and **-content**. If the value provided is *keep*, the acl of the resource is read before the PUT (or the default is used if the resource does not exist), then set back to what it was after the PUT (if it existed). An error will occur if the resource is successfully written but the kept ACL

cannot be then applied. This should never happen. *Note: calc* is not currently fully implemented.

-x-amz-meta-*
If any header starts with "-x-amz-meta-", its contents are added to the PUT command to be stored as metadata with the resource. Again, no encoding is performed, and the metadata should not contain characters like newlines, carriage returns, and so on. It is best to stick with simple ASCII strings, or to fix the library in several places.

-content-type
This overrides the content-type calculated by **-file** or sets the content-type for **-content**.

-compare
This is the standard compare mode argument. **S3::Put** returns 1 if the data was copied or 0 if the data was skipped due to the comparison mode so indicating it should be skipped.

S3::Get ?-bucket *bucketname*? -resource *resourcename* ?-blocking *boolean*? ? -compare *comparemode*? ?-file *filename*? ?-content *contentvarname*? ?-timestamp *aws|now*? ?-headers *headervarname*?

This command retrieves data from a resource on Amazon's S3 servers, using the HTTP GET command. It returns 0 if the **-compare** mode prevented the transfer, 1 if the transfer worked, or throws an error if the transfer was attempted but failed. Server 5XX errors and S3 socket errors are are retried according to **S3:Configure** settings before throwing an error; other errors throw immediately. Note that this is always authenticated as the user configured in via **S3::Configure -accesskeyid**. Use the Tcllib http for unauthenticated GETs.

-bucket
This specifies the bucket from which the resource will be read. Leading and/or trailing slashes are removed for you, as are spaces.

-resource
This is the full name of the resource within the bucket. A single leading slash is removed, but not a trailing slash. Spaces are not trimmed.

-blocking
The standard blocking flag.

-file
If this is specified, the body of the resource will be read into this file, incrementally without pulling it entirely into memory first. The parent directory must already exist. If the file already exists, it must be writable. If an error is thrown part-way through the process and the file already existed, it may be clobbered. If an error is thrown part-way through the process and the file did not already exist, any partial bits will be deleted. Specifying this is an error if **-content** is also specified, but at least one of **-file** or **-content** must be specified.

-timestamp
This is only valid in conjunction with **-file**. It may be specified as *now* or *aws*. The default is *now*. If *now*, the file's modification date is left up to the system. If *aws*, the file's mtime is set to match the Last-Modified header on the resource, synchronizing the two appropriately for **-compare** *date* or **-compare** *newer*.

-content
If this is specified, the *contentvarname* is a variable in the caller's scope (not necessarily

global) that receives the value of the body of the resource. No encoding is done, so if the resource (for example) represents a UTF-8 byte sequence, use [encoding convertfrom utf-8] to get a valid UTF-8 string. If this is specified, the **-compare** is ignored unless it is *never*, in which case no assignment to *contentvarname* is performed. Specifying this is an error if **-file** is also specified, but at least one of **-file** or **-content** must be specified.

-compare
This is the standard compare mode argument. **S3::Get** returns 1 if the data was copied or 0 if the data was skipped due to the comparison mode so indicating it should be skipped.

-headers
If this is specified, the headers resulting from the fetch are stored in the provided variable, as a dictionary. This will include content-type and x-amz-meta-* headers, as well as the usual HTTP headers, the x-amz-id debugging headers, and so on. If no file is fetched (due to **-compare** or other errors), no assignment to this variable is performed.

S3::Head ?-bucket *bucketname*? -resource *resourcename* ?-blocking *boolean*? ?-dict *dictvarname*? ?-headers *headersvarname*? ?-status *statusvarname*?

This command requests HEAD from the resource. It returns whether a 2XX code was returned as a result of the request, never throwing an S3 remote error. That is, if this returns 1, the resource exists and is accessible. If this returns 0, something went wrong, and the **-status** result can be consulted for details.

-bucket
This specifies the bucket from which the resource will be read. Leading and/or trailing slashes are removed for you, as are spaces.

-resource
This is the full name of the resource within the bucket. A single leading slash is removed, but not a trailing slash. Spaces are not trimmed.

-blocking
The standard blocking flag.

-dict
If specified, the resulting dictionary from the **S3::REST** call is assigned to the indicated (not necessarily global) variable in the caller's scope.

-headers
If specified, the dictionary of headers from the result are assigned to the indicated (not necessarily global) variable in the caller's scope.

-status
If specified, the indicated (not necessarily global) variable in the caller's scope is assigned a 2-element list. The first element is the 3-digit HTTP status code, while the second element is the HTTP message (such as "OK" or "Forbidden").

S3::GetAcl ?-blocking *boolean*? ?-bucket *bucketname*? -resource *resourcename* ? -result-type *REST*|*xml*|*pxml*?

This command gets the ACL of the indicated resource or throws an error if it is unavailable.

-blocking *boolean*
See above for standard definition.

-bucket
This specifies the bucket from which the resource will be read. Leading and/or trailing

slashes are removed for you, as are spaces.

-resource
This is the full name of the resource within the bucket. A single leading slash is removed, but not a trailing slash. Spaces are not trimmed.

-parse-xml *xml*
The XML from a previous GetACL can be passed in to be parsed into dictionary form. In this case, -result-type must be pxml or dict.

-result-type *REST*
The dictionary returned by **S3::REST** is the return value of **S3::GetAcl**. In this case, a non-2XX httpstatus will not throw an error.

-result-type *xml*
The raw XML of the body is returned as the result (with no encoding applied).

-result-type *pxml*
The XML of the body as parsed by xsxp::parse is returned.

-result-type *dict*
This fetches the ACL, parses it, and returns a dictionary of two elements.

The first element has the key "owner" whose value is the canonical ID of the owner of the resource.

The second element has the key "acl" whose value is a dictionary. Each key in the dictionary is one of Amazon's permissions, namely "READ", "WRITE", "READ_ACP", "WRITE_ACP", or "FULL_CONTROL". Each value of each key is a list of canonical IDs or group URLs that have that permission. Elements are not in the list in any particular order, and not all keys are necessarily present. Display names are not returned, as they are not especially useful; use pxml to obtain them if necessary.

S3::PutAcl ?-blocking *boolean*? ?-bucket *bucketname*? -resource *resourcename* ?-acl *new-acl*?
This sets the ACL on the indicated resource. It returns the XML written to the ACL, or throws an error if anything went wrong.

-blocking *boolean*
See above for standard definition.

-bucket
This specifies the bucket from which the resource will be read. Leading and/or trailing slashes are removed for you, as are spaces.

-resource
This is the full name of the resource within the bucket. A single leading slash is removed, but not a trailing slash. Spaces are not trimmed.

-owner
If this is provided, it is assumed to match the owner of the resource. Otherwise, a GET may need to be issued against the resource to find the owner. If you already have the owner (such as from a call to **S3::GetAcl**, you can pass the value of the "owner" key as the value of this option, and it will be used in the construction of the XML.

-acl

If this option is specified, it provides the ACL the caller wishes to write to the resource. If this is not supplied or is empty, the value is taken from **S3::Configure -default-acl**. The ACL is written with a PUT to the ?acl resource.

If the value passed to this option starts with "<", it is taken to be a body to be PUT to the ACL resource.

If the value matches one of the standard Amazon x-amz-acl headers (i.e., a canned access policy), that header is translated to XML and then applied. The canned access policies are private, public-read, public-read-write, and authenticated-read (in lower case).

Otherwise, the value is assumed to be a dictionary formatted as the "acl" sub-entry within the dict returns by **S3::GetAcl -result-type dict**. The proper XML is generated and applied to the resource. Note that a value containing "//" is assumed to be a group, a value containing "@" is assumed to be an AmazonCustomerByEmail, and otherwise the value is assumed to be a canonical Amazon ID.

Note that you cannot change the owner, so calling GetAcl on a resource owned by one user and applying it via PutAcl on a resource owned by another user may not do exactly what you expect.

S3::Delete ?-bucket *bucketname*? -resource *resourcename* ?-blocking *boolean*? ?-status *statusvar*?

This command deletes the specified resource from the specified bucket. It returns 1 if the resource was deleted successfully, 0 otherwise. It returns 0 rather than throwing an S3 remote error.

-bucket
This specifies the bucket from which the resource will be deleted. Leading and/or trailing slashes are removed for you, as are spaces.

-resource
This is the full name of the resource within the bucket. A single leading slash is removed, but not a trailing slash. Spaces are not trimmed.

-blocking
The standard blocking flag.

-status
If specified, the indicated (not necessarily global) variable in the caller's scope is set to a two-element list. The first element is the 3-digit HTTP status code. The second element is the HTTP message (such as "OK" or "Forbidden"). Note that Amazon's DELETE result is 204 on success, that being the code indicating no content in the returned body.

S3::Push ?-bucket *bucketname*? -directory *directoryname* ?-prefix *prefixstring*? ?-compare *comparemode*? ?-x-amz-meta-* *metastring*? ?-acl *aclcode*? ?-delete *boolean*? ?-error *throw*|*break*|*continue*? ?-progress *scriptprefix*?

This synchronises a local directory with a remote bucket by pushing the differences using **S3::Put**. Note that if something has changed in the bucket but not locally, those changes could be lost. Thus, this is not a general two-way synchronization primitive. (See **S3::Sync** for

that.) Note too that resource names are case sensitive, so changing the case of a file on a Windows machine may lead to otherwise-unnecessary transfers. Note that only regular files are considered, so devices, pipes, symlinks, and directories are not copied.

-bucket
This names the bucket into which data will be pushed.

-directory
This names the local directory from which files will be taken. It must exist, be readable via [glob] and so on. If only some of the files therein are readable, **S3::Push** will PUT those files that are readable and return in its results the list of files that could not be opened.

-prefix
This names the prefix that will be added to all resources. That is, it is the remote equivalent of **-directory**. If it is not specified, the root of the bucket will be treated as the remote directory. An example may clarify.

```
S3::Push -bucket test -directory /tmp/xyz -prefix hello/world
```

In this example, /tmp/xyz/pdq.html will be stored as http://s3.amazonaws.com/test/hello/world/pdq.html in Amazon's servers. Also, /tmp/xyz/abc/def/Hello will be stored as http://s3.amazonaws.com/test/hello/world/abc/def/Hello in Amazon's servers. Without the **-prefix** option, /tmp/xyz/pdq.html would be stored as http://s3.amazonaws.com/test/pdq.html.

-blocking
This is the standard blocking option.

-compare
If present, this is passed to each invocation of **S3::Put**. Naturally, **S3::Configure -default-compare** is used if this is not specified.

-x-amz-meta-*
If present, this is passed to each invocation of **S3::Put**. All copied files will have the same metadata.

-acl
If present, this is passed to each invocation of **S3::Put**.

-delete
This defaults to false. If true, resources in the destination that are not in the source directory are deleted with **S3::Delete**. Since only regular files are considered, the existance of a symlink, pipe, device, or directory in the local source will *not* prevent the deletion of a remote resource with a corresponding name.

-error
This controls the behavior of **S3::Push** in the event that **S3::Put** throws an error. Note that errors encountered on the local file system or in reading the list of resources in the remote bucket always throw errors. This option allows control over "partial" errors, when some files were copied and some were not. **S3::Delete** is always finished up, with errors simply recorded in the return result.

throw
The error is rethrown with the same errorCode.

break
Processing stops without throwing an error, the error is recorded in the return value, and the command returns with a normal return. The calls to **S3::Delete** are not started.

continue
This is the default. Processing continues without throwing, recording the error in the return result, and resuming with the next file in the local directory to be copied.

-progress
If this is specified and the indicated script prefix is not empty, the indicated script prefix will be invoked several times in the caller's context with additional arguments at various points in the processing. This allows progress reporting without backgrounding. The provided prefix will be invoked with additional arguments, with the first additional argument indicating what part of the process is being reported on. The prefix is initially invoked with *args* as the first additional argument and a dictionary representing the normalized arguments to the **S3::Push** call as the second additional argument. Then the prefix is invoked with *local* as the first additional argument and a list of suffixes of the files to be considered as the second argument. Then the prefix is invoked with *remote* as the first additional argument and a list of suffixes existing in the remote bucket as the second additional argument. Then, for each file in the local list, the prefix will be invoked with *start* as the first additional argument and the common suffix as the second additional argument. When **S3::Put** returns for that file, the prefix will be invoked with *copy* as the first additional argument, the common suffix as the second additional argument, and a third argument that will be "copied" (if **S3::Put** sent the resource), "skipped" (if **S3::Put** decided not to based on **-compare**), or the errorCode that **S3::Put** threw due to unexpected errors (in which case the third argument is a list that starts with "S3"). When all files have been transfered, the prefix may be invoked zero or more times with *delete* as the first additional argument and the suffix of the resource being deleted as the second additional argument, with a third argument being either an empty string (if the delete worked) or the errorCode from **S3::Delete** if it failed. Finally, the prefix will be invoked with *finished* as the first additional argument and the return value as the second additional argument.

The return result from this command is a dictionary. They keys are the suffixes (i.e., the common portion of the path after the **-directory** and **-prefix**), while the values are either "copied", "skipped" (if **-compare** indicated not to copy the file), or the errorCode thrown by **S3::Put**, as appropriate. If **-delete** was true, there may also be entries for suffixes with the value "deleted" or "notdeleted", indicating whether the attempted **S3::Delete** worked or not, respectively. There is one additional pair in the return result, whose key is the empty string and whose value is a nested dictionary. The keys of this nested dictionary include "filescopied" (the number of files successfully copied), "bytescopied" (the number of data bytes in the files copied, excluding headers, metadata, etc), "compareskipped" (the number of files not copied due to **-compare** mode), "errorskipped" (the number of files not copied due to thrown errors), "filesdeleted" (the number of resources deleted due to not having corresponding files locally, or 0 if **-delete** is false), and "filesnotdeleted" (the number of resources whose deletion was attempted but failed).

Note that this is currently implemented somewhat inefficiently. It fetches the bucket listing

(including timestamps and eTags), then calls **S3::Put**, which uses HEAD to find the timestamps and eTags again. Correcting this with no API change is planned for a future upgrade.

S3::Pull ?-bucket *bucketname*? -directory *directoryname* ?-prefix *prefixstring*? ?-blocking *boolean*? ?-compare *comparemode*? ?-delete *boolean*? ?-timestamp *aws*|*now*? ?-error *throw*|*break*|*continue*? ?-progress *scriptprefix*?

This synchronises a remote bucket with a local directory by pulling the differences using **S3::Get** If something has been changed locally but not in the bucket, those difference may be lost. This is not a general two-way synchronization mechanism. (See **S3::Sync** for that.) This creates directories if needed; new directories are created with default permissions. Note that resource names are case sensitive, so changing the case of a file on a Windows machine may lead to otherwise-unnecessary transfers. Also, try not to store data in resources that end with a slash, or which are prefixes of resources that otherwise would start with a slash; i.e., don't use this if you store data in resources whose names have to be directories locally.

Note that this is currently implemented somewhat inefficiently. It fetches the bucket listing (including timestamps and eTags), then calls **S3::Get**, which uses HEAD to find the timestamps and eTags again. Correcting this with no API change is planned for a future upgrade.

-bucket
: This names the bucket from which data will be pulled.

-directory
: This names the local directory into which files will be written It must exist, be readable via [glob], writable for file creation, and so on. If only some of the files therein are writable, **S3::Pull** will GET those files that are writable and return in its results the list of files that could not be opened.

-prefix
: The prefix of resources that will be considered for retrieval. See **S3::Push** for more details, examples, etc. (Of course, **S3::Pull** reads rather than writes, but the prefix is treated similarly.)

-blocking
: This is the standard blocking option.

-compare
: This is passed to each invocation of **S3::Get** if provided. Naturally, **S3::Configure -default-compare** is used if this is not provided.

-timestamp
: This is passed to each invocation of **S3::Get** if provided.

-delete
: If this is specified and true, files that exist in the **-directory** that are not in the **-prefix** will be deleted after all resources have been copied. In addition, empty directories (other than the top-level **-directory**) will be deleted, as Amazon S3 has no concept of an empty directory.

-error
: See **S3::Push** for a description of this option.

-progress
: See **S3::Push** for a description of this option. It differs slightly in that local directories may

be included with a trailing slash to indicate they are directories.

The return value from this command is a dictionary. It is identical in form and meaning to the description of the return result of **S3::Push**. It differs only in that directories may be included, with a trailing slash in their name, if they are empty and get deleted.

S3::Toss ?-bucket *bucketname*? -prefix *prefixstring* ?-blocking *boolean*? ?-error *throw|break|continue*? ?-progress *scriptprefix*?

This deletes some or all resources within a bucket. It would be considered a "recursive delete" had Amazon implemented actual directories.

-bucket
The bucket from which resources will be deleted.

-blocking
The standard blocking option.

-prefix
The prefix for resources to be deleted. Any resource that starts with this string will be deleted. This is required. To delete everything in the bucket, pass an empty string for the prefix.

-error
If this is "throw", **S3::Toss** rethrows any errors it encounters. If this is "break", **S3::Toss** returns with a normal return after the first error, recording that error in the return result. If this is "continue", which is the default, **S3::Toss** continues on and lists all errors in the return result.

-progress
If this is specified and not an empty string, the script prefix will be invoked several times in the context of the caller with additional arguments appended. Initially, it will be invoked with the first additional argument being *args* and the second being the processed list of arguments to **S3::Toss**. Then it is invoked with *remote* as the first additional argument and the list of suffixes in the bucket to be deleted as the second additional argument. Then it is invoked with the first additional argument being *delete* and the second additional argument being the suffix deleted and the third additional argument being "deleted" or "notdeleted" depending on whether **S3::Delete** threw an error. Finally, the script prefix is invoked with a first additional argument of "finished" and a second additional argument of the return value.

The return value is a dictionary. The keys are the suffixes of files that **S3::Toss** attempted to delete, and whose values are either the string "deleted" or "notdeleted". There is also one additional pair, whose key is the empty string and whose value is an embedded dictionary. The keys of this embedded dictionary include "filesdeleted" and "filesnotdeleted", each of which has integer values.

Limitations

- The pure-Tcl MD5 checking is slow. If you are processing files in the megabyte range, consider ensuring binary support is available.
- The commands **S3::Pull** and **S3::Push** fetch a directory listing which includes timestamps and MD5 hashes, then invoke **S3::Get** and **S3::Put**. If a complex **-compare** mode is specified, **S3::Get** and **S3::Put** will invoke a HEAD operation for each file to fetch timestamps and MD5

hashes of each resource again. It is expected that a future release of this package will solve this without any API changes.
- The commands **S3::Pull** and **S3::Push** fetch a directory listing without using **-max-count**. The entire directory is pulled into memory at once. For very large buckets, this could be a performance problem. The author, at this time, does not plan to change this behavior. Welcome to Open Source.
- **S3::Sync** is neither designed nor implemented yet. The intention would be to keep changes synchronised, so changes could be made to both the bucket and the local directory and be merged by **S3::Sync**.
- Nor is **-compare** *calc* fully implemented. This is primarily due to Windows not providing a convenient method for distinguishing between local files that are "public-read" or "public-read-write". Assistance figuring out TWAPI for this would be appreciated. The U**X semantics are difficult to map directly as well. See the source for details. Note that there are not tests for calc, since it isn't done yet.
- The HTTP processing is implemented within the library, rather than using a "real" HTTP package. Hence, multi-line headers are not (yet) handled correctly. Do not include carriage returns or linefeeds in x-amz-meta-* headers, content-type values, and so on. The author does not at this time expect to improve this.
- Internally, **S3::Push** and **S3::Pull** and **S3::Toss** are all very similar and should be refactored.
- The idea of using **-compare** *never* **-delete** *true* to delete files that have been deleted from one place but not the other yet not copying changed files is untested.

Usage suggestions

To fetch a "directory" out of a bucket, make changes, and store it back:

```
file mkdir ./tempfiles
S3::Pull -bucket sample -prefix of/interest -directory ./tempfiles \
  -timestamp aws
do_my_process ./tempfiles other arguments
S3::Push -bucket sample -prefix of/interest -directory ./tempfiles \
  -compare newer -delete true
```

To delete files locally that were deleted off of S3 but not otherwise update files:

```
S3::Pull -bucket sample -prefix of/interest -directory ./myfiles \
  -compare never -delete true
```

Future developments

The author intends to work on several additional projects related to this package, in addition to finishing the unfinished features.

First, a command-line program allowing browsing of buckets and transfer of files from shell scripts and command prompts is useful.

Second, a GUI-based program allowing visual manipulation of bucket and resource trees not unlike Windows Explorer would be useful.

Third, a command-line (and perhaps a GUI-based) program called "OddJob" that will use S3 to synchronize computation amongst multiple servers running OddJob. An S3 bucket will be set up with a number of scripts to run, and the OddJob program can be invoked on multiple machines to

run scripts on all the machines, each moving on to the next unstarted task as it finishes each. This is still being designed, and it is intended primarily to be run on Amazon's Elastic Compute Cloud.

TLS security considerations

This package uses the **TLS** package to handle the security for **https** urls and other socket connections.

Policy decisions like the set of protocols to support and what ciphers to use are not the responsibility of **TLS**, nor of this package itself however. Such decisions are the responsibility of whichever application is using the package, and are likely influenced by the set of servers the application will talk to as well.

For example, in light of the recent POODLE attack discovered by Google many servers will disable support for the SSLv3 protocol. To handle this change the applications using **TLS** must be patched, and not this package, nor **TLS** itself. Such a patch may be as simple as generally activating **tls1** support, as shown in the example below.

```
    package require tls
    tls::init -tls1 1 ;# forcibly activate support for the TLS1
    protocol

    ... your own application code ...
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *amazon-s3* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Networking

Copyright

```
2006,2008 Darren New. All Rights Reserved. See LICENSE.TXT for terms.
```

xsxp

Keywords: xsxp, dom, parser, xml

Name

xsxp - eXtremely Simple Xml Parser

Synopsis

package require Tcl 8.5 9
package require xsxp 1.1

package require xml
xsxp::parse *xml*
xsxp::fetch *pxml path* ?*part*?
xsxp::fetchall *pxml_list path* ?*part*?
xsxp::only *pxml tagname*
xsxp::prettyprint *pxml* ?*chan*?

Description

This package provides a simple interface to parse XML into a pure-value list. It also provides accessor routines to pull out specific subtags, not unlike DOM access. This package was written for and is used by Darren New's Amazon S3 access package.

This is pretty lame, but I needed something like this for S3, and at the time, TclDOM would not work with the new 8.5 Tcl due to version number problems.

In addition, this is a pure-value implementation. There is no garbage to clean up in the event of a thrown error, for example. This simplifies the code for sufficiently small XML documents, which is what Amazon's S3 guarantees.

Copyright 2006 Darren New. All Rights Reserved. NO WARRANTIES OF ANY TYPE ARE PROVIDED. COPYING OR USE INDEMNIFIES THE AUTHOR IN ALL WAYS. This software is licensed under essentially the same terms as Tcl. See LICENSE.txt for the terms.

Commands

The package implements five rather simple procedures. One parses, one is for debugging, and the rest pull various parts of the parsed document out for processing.

xsxp::parse *xml*

This parses an XML document (using the standard xml tcllib module in a SAX sort of way) and builds a data structure which it returns if the parsing succeeded. The return value is referred to herein as a "pxml", or "parsed xml". The list consists of two or more elements:
- The first element is the name of the tag.
- The second element is an array-get formatted list of key/value pairs. The keys are attribute names and the values are attribute values. This is an empty list if there are no attributes on the tag.
- The third through end elements are the children of the node, if any. Each child is, recursively, a pxml.
- Note that if the zero'th element, i.e. the tag name, is "%PCDATA", then the attributes will be empty and the third element will be the text of the element. In addition, if an element's contents consists only of PCDATA, it will have only one child, and all the PCDATA will be concatenated. In other words, this parser works poorly for XML with elements that contain both child tags and PCDATA. Since Amazon S3 does not do this (and for that matter most uses of XML where XML is a poor choice don't do this), this is probably not a serious limitation.

xsxp::fetch *pxml path* ?*part*?

pxml is a parsed XML, as returned from xsxp::parse. *path* is a list of element tag names. Each element is the name of a child to look up, optionally followed by a hash ("#") and a string of digits. An empty list or an initial empty element selects *pxml*. If no hash sign is present, the behavior is as if "#0" had been appended to that element. (In addition to a list, slashes can

separate subparts where convenient.)

An element of *path* scans the children at the indicated level for the n'th instance of a child whose tag matches the part of the element before the hash sign. If an element is simply "#" followed by digits, that indexed child is selected, regardless of the tags in the children. Hence, an element of "#3" will always select the fourth child of the node under consideration.

part defaults to "%ALL". It can be one of the following case-sensitive terms:

%ALL
returns the entire selected element.

%TAGNAME
returns lindex 0 of the selected element.

%ATTRIBUTES
returns index 1 of the selected element.

%CHILDREN
returns lrange 2 through end of the selected element, resulting in a list of elements being returned.

%PCDATA
returns a concatenation of all the bodies of direct children of this node whose tag is %PCDATA. It throws an error if no such children are found. That is, part=%PCDATA means return the textual content found in that node but not its children nodes.

%PCDATA?
is like %PCDATA, but returns an empty string if no PCDATA is found.

For example, to fetch the first bold text from the fifth paragraph of the body of your HTML file,

```
xsxp::fetch $pxml {body p#4 b} %PCDATA
```

xsxp::fetchall *pxml_list path* ?*part*?
This iterates over each PXML in *pxml_list* (which must be a list of pxmls) selecting the indicated path from it, building a new list with the selected data, and returning that new list.

For example, *pxml_list* might be the %CHILDREN of a particular element, and the *path* and *part* might select from each child a sub-element in which we're interested.

xsxp::only *pxml tagname*
This iterates over the direct children of *pxml* and selects only those with *tagname* as their tag. Returns a list of matching elements.

xsxp::prettyprint *pxml* ?*chan*?
This outputs to *chan* (default stdout) a pretty-printed version of *pxml*.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *amazon-s3* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the

left-most button in the secondary navigation bar.

Category
Text processing

Copyright
2006 Darren New. All Rights Reserved.

asn

asn
Keywords: asn, ber, cer, der, internet, protocol, x.208, x.209

Name
asn - ASN.1 BER encoder/decoder

Synopsis
package require Tcl 8.5 9
package require asn ?**0.8.5**?
::asn::asnSequence ?*evalue* ...?
::asn::asnSequenceFromList *elist*
::asn::asnSet ?*evalue* ...?
::asn::asnSetFromList *elist*
::asn::asnApplicationConstr *appNumber* ?*evalue* ...?
::asn::asnApplication *appNumber data*
::asn::asnChoice *appNumber* ?*evalue* ...?
::asn::asnChoiceConstr *appNumber* ?*evalue* ...?
::asn::asnInteger *number*
::asn::asnEnumeration *number*
::asn::asnBoolean *bool*
::asn::asnContext *context data*
::asn::asnContextConstr *context* ?*evalue* ...?
::asn::asnObjectIdentifier *idlist*
::asn::asnUTCTime *utcstring*
::asn::asnNull
::asn::asnBitString *string*
::asn::asnOctetString *string*
::asn::asnNumericString *string*
::asn::asnPrintableString *string*
::asn::asnIA5String *string*
::asn::asnBMPString *string*
::asn::asnUTF8String *string*
::asn::asnString *string*
::asn::defaultStringType ?*type*?
::asn::asnPeekByte *dataVar byteVar*
::asn::asnGetLength *dataVar lengthVar*

::asn::asnGetResponse *chan dataVar*
::asn::asnGetInteger *dataVar intVar*
::asn::asnGetEnumeration *dataVar enumVar*
::asn::asnGetOctetString *dataVar stringVar*
::asn::asnGetString *dataVar stringVar* ?*typeVar*?
::asn::asnGetNumericString *dataVar stringVar*
::asn::asnGetPrintableString *dataVar stringVar*
::asn::asnGetIA5String *dataVar stringVar*
::asn::asnGetBMPString *dataVar stringVar*
::asn::asnGetUTF8String *dataVar stringVar*
::asn::asnGetUTCTime *dataVar utcVar*
::asn::asnGetBitString *dataVar bitsVar*
::asn::asnGetObjectIdentifier *dataVar oidVar*
::asn::asnGetBoolean *dataVar boolVar*
::asn::asnGetNull *dataVar*
::asn::asnGetSequence *dataVar sequenceVar*
::asn::asnGetSet *dataVar setVar*
::asn::asnGetApplication *dataVar appNumberVar* ?*contentVar*? ?*encodingTypeVar*?
::asn::asnGetContext *dataVar contextNumberVar* ?*contentVar*? ?*encodingTypeVar*?
::asn::asnPeekTag *dataVar tagVar tagtypeVar constrVar*
::asn::asnTag *tagnumber* ?*class*? ?*tagstyle*?
::asn::asnRetag *dataVar newTag*

Description

The **asn** package provides *partial* de- and encoder commands for BER encoded ASN.1 data. It can also be used for decoding DER, which is a restricted subset of BER.

ASN.1 is a standard *Abstract Syntax Notation*, and BER are its *Basic Encoding Rules*.

See http://asn1.elibel.tm.fr/en/standards/index.htm for more information about the standard.

Also see http://luca.ntop.org/Teaching/Appunti/asn1.html for *A Layman's Guide to a Subset of ASN.1, BER, and DER*, an RSA Laboratories Technical Note by Burton S. Kaliski Jr. (Revised November 1, 1993). A text version of this note is part of the module sources and should be read by any implementor.

API

Encoder

::asn::asnSequence ?*evalue ...*?
: Takes zero or more encoded values, packs them into an ASN sequence and returns its encoded binary form.

::asn::asnSequenceFromList *elist*
: Takes a list of encoded values, packs them into an ASN sequence and returns its encoded binary form.

::asn::asnSet ?*evalue ...*?
: Takes zero or more encoded values, packs them into an ASN set and returns its encoded binary form.

::asn::asnSetFromList *elist*

Takes a list of encoded values, packs them into an ASN set and returns its encoded binary form.

::asn::asnApplicationConstr *appNumber ?evalue ...?*
Takes zero or more encoded values, packs them into an ASN application construct and returns its encoded binary form.

::asn::asnApplication *appNumber data*
Takes a single encoded value *data*, packs it into an ASN application construct and returns its encoded binary form.

::asn::asnChoice *appNumber ?evalue ...?*
Takes zero or more encoded values, packs them into an ASN choice construct and returns its encoded binary form.

::asn::asnChoiceConstr *appNumber ?evalue ...?*
Takes zero or more encoded values, packs them into an ASN choice construct and returns its encoded binary form.

::asn::asnInteger *number*
Returns the encoded form of the specified integer *number*.

::asn::asnEnumeration *number*
Returns the encoded form of the specified enumeration id *number*.

::asn::asnBoolean *bool*
Returns the encoded form of the specified boolean value *bool*.

::asn::asnContext *context data*
Takes an encoded value and packs it into a constructed value with application tag, the *context* number.

::asn::asnContextConstr *context ?evalue ...?*
Takes zero or more encoded values and packs them into a constructed value with application tag, the *context* number.

::asn::asnObjectIdentifier *idlist*
Takes a list of at least two integers describing an object identifier (OID) value, and returns the encoded value.

::asn::asnUTCTime *utcstring*
Returns the encoded form of the specified UTC time string.

::asn::asnNull
Returns the NULL encoding.

::asn::asnBitString *string*
Returns the encoded form of the specified *string*.

::asn::asnOctetString *string*
Returns the encoded form of the specified *string*.

::asn::asnNumericString *string*
Returns the *string* encoded as an ASN.1 NumericString. Raises an error if the *string* contains characters other than decimal digits and spaces.

::asn::asnPrintableString *string*
Returns the *string* encoding as ASN.1 PrintableString. Raises an error if the *string* contains characters which are not allowed by the Printable String datatype. The allowed characters are A-Z, a-z, 0-9, space, apostrophe, colon, parentheses, plus, minus, comma, period, forward

slash, question mark, and the equals sign.

::asn::asnIA5String *string*
Returns the *string* encoded as ASN.1 IA5String. Raises an error if the *string* contains any characters outside of the US-ASCII range.

::asn::asnBMPString *string*
Returns the *string* encoded as ASN.1 Basic Multilingual Plane string (Which is essentialy big-endian UCS2).

::asn::asnUTF8String *string*
Returns the *string* encoded as UTF8 String. Note that some legacy applications such as Windows CryptoAPI may not work correctly with UTF8 strings. It may be safest to use BMPStrings.

::asn::asnString *string*
Returns an encoded form of *string*, choosing the most restricted ASN.1 string type possible. If the string contains non-ASCII characters, then there is more than one string type which can be used. See **::asn::defaultStringType**.

::asn::defaultStringType ?*type*?
Selects the string type to use for the encoding of non-ASCII strings. Returns current default when called without argument. If the argument *type* is supplied, it should be either **UTF8** or **BMP** to choose UTF8String or BMPString respectively.

Decoder

General notes:

1. Nearly all getter (decoder) commands take two arguments. These arguments are variable names, *except* for **::asn::asnGetResponse**. The *dataVar* variable contains the encoded ASN values to decode. When a value getter (decoder) is called, the value is stored in the second variable, and the remaining input is stored back in the first *dataVar* variable.
2. When getting (decoding), the *dataVar* variable is modified first, before the decoded value is stored in the second variable. This means that if both arguments refer to the same variable, it will always contain the decoded value after the call, and not the remainder of the input.

::asn::asnPeekByte *dataVar byteVar*
Return a copy of the first byte of the data, without modifing *dataVar*. This can be used to check for implicit tags.

::asn::asnGetLength *dataVar lengthVar*
Decode the length information for a block of BER data. The tag has already to be removed from the data.

::asn::asnGetResponse *chan dataVar*
Reads an encoded ASN *sequence* from the channel *chan* and stores it into the variable called *dataVar*.

::asn::asnGetInteger *dataVar intVar*
Decodes the integer value encoded at the start of the data stored in the variable called *dataVar*. The integer is stored in the variable called *intVar*, and the decoded bytes are removed from the start of the data in *dataVar*, ready for the next value to be decoded. If the value at the start is not an encoded integer an error is thrown.

::asn::asnGetEnumeration *dataVar enumVar*
Decodes the enumeration ID value encoded at the start of the data stored in the variable

called *dataVar*. The enumeration ID is stored in the variable called *enumVar*, and the decoded bytes are removed from the start of the data in *dataVar*, ready for the next value to be decoded. If the value at the start is not an encoded enumeration ID an error is thrown.

::asn::asnGetOctetString *dataVar stringVar*

Decodes the string value encoded at the start of the data stored in the variable called *dataVar* . The string is stored in the variable called *stringVar*, and the decoded bytes are removed from the start of the data in *dataVar*, ready for the next value to be decoded. If the value at the start is not an encoded string an error is thrown.

::asn::asnGetString *dataVar stringVar* ?*typeVar*?

Decodes the user-readable string value encoded at the start of the data stored in the variable called *dataVar*. The string is stored in the variable called *stringVar*, and the decoded bytes are removed from the start of the data in *dataVar*, ready for the next value to be decoded. If the value at the start is not an encoded string an error is thrown.

This is a convenience function which is able to automatically distinguish all supported ASN.1 string types and convert the input value appropriately.

See **::asn::asnGetPrintableString**, **::asnGetIA5String**, etc., below for the type-specific conversion commands.

If the optional third argument *typeVar* is supplied, then the incoming string's type is stored in the variable it names.

The function throws the error "Invalid command name asnGetSome**UnsupportedString**" if the unsupported string type **Unsupported** is encountered. You can create the appropriate function "asn::asnGetSome**UnsupportedString**" in your application if neccessary.

::asn::asnGetNumericString *dataVar stringVar*

Decodes the numeric string value encoded at the start of the data stored in the variable called *dataVar*. The numeric string is stored in the variable called *stringVar*, and the decoded bytes are removed from the start of the data in *dataVar*, ready for the next value to be decoded. If the value at the start is not an encoded numeric string an error is thrown.

::asn::asnGetPrintableString *dataVar stringVar*

Decodes the printable string value encoded at the start of the data stored in the variable called *dataVar*. The printable string is stored in the variable called *stringVar*, and the decoded bytes are removed from the start of the data in *dataVar*, ready for the next value to be decoded. If the value at the start is not an encoded printable string an error is thrown.

::asn::asnGetIA5String *dataVar stringVar*

Decodes the IA5 (US-ASCII) string value encoded at the start of the data stored in the variable called *dataVar*. The IA5 string is stored in the variable called *stringVar*, and the decoded bytes are removed from the start of the data in *dataVar*, ready for the next value to be decoded. If the value at the start is not an encoded IA5 string an error is thrown.

::asn::asnGetBMPString *dataVar stringVar*

Decodes the BMP (two-byte unicode—UCS2) string value encoded at the start of the data stored in the variable called *dataVar*. The BMP string is converted into a proper Tcl string which is then stored in the variable called *stringVar*, and the decoded bytes are removed from

the start of the data in *dataVar*, ready for the next value to be decoded. If the value at the start is not an encoded BMP string an error is thrown.

::asn::asnGetUTF8String *dataVar stringVar*

Decodes the UTF8 string value encoded at the start of the data stored in the variable called *dataVar*. The UTF8 string is converted into a proper Tcl string which is then stored in the variable called *stringVar*, and the decoded bytes are removed from the start of the data in *dataVar*, ready for the next value to be decoded. If the value at the start is not an encoded UTF8 string an error is thrown.

::asn::asnGetUTCTime *dataVar utcVar*

Decodes the UTC time value encoded at the start of the data stored in the variable called *dataVar*. The UTC time value is stored in the variable called *utcVar*, and the decoded bytes are removed from the start of the data in *dataVar*, ready for the next value to be decoded. The value stored in *utcVar* is a string; to get an actual time value, use the **clock scan** command. If the value at the start is not an encoded UTC time value an error is thrown.

::asn::asnGetBitString *dataVar bitsVar*

Decodes the bit string value encoded at the start of the data stored in the variable called *dataVar*. The bit string is stored in the variable called *bitsVar*, and the decoded bytes are removed from the start of the data in *dataVar*, ready for the next value to be decoded. The value stored in *bitsVar* is a string that contains only **"0"** and **"1"** characters. If the value at the start is not an encoded bit string an error is thrown.

::asn::asnGetObjectIdentifier *dataVar oidVar*

Decodes the object identifier (OID) value encoded at the start of the data stored in the variable called *dataVar*. The OID is stored in the variable called *oidVar*, and the decoded bytes are removed from the start of the data in *dataVar*, ready for the next value to be decoded. The value stored in *oidVar* is a list of integers. If the value at the start is not an encoded OID an error is thrown.

::asn::asnGetBoolean *dataVar boolVar*

Decodes the Boolean value encoded at the start of the data stored in the variable called *dataVar*. The Boolean is stored in the variable called *boolVar*, and the decoded bytes are removed from the start of the data in *dataVar*, ready for the next value to be decoded. The value stored in *boolVar* is either **0** or **1**. If the value at the start is not an encoded Boolean an error is thrown.

::asn::asnGetNull *dataVar*

Decodes the NULL value encoded at the start of the data stored in the variable called *dataVar* and removes the decoded bytes from the start of the data in *dataVar*, ready for the next value to be decoded. If the value at the start is not an encoded NULL value an error is thrown.

::asn::asnGetSequence *dataVar sequenceVar*

Decodes the ASN sequence value encoded at the start of the data stored in the variable called *dataVar*. The ASN sequence is stored in the variable called *sequenceVar*, and the decoded bytes are removed from the start of the data in *dataVar*, ready for the next value to be decoded. If the value at the start is not an encoded ASN sequence an error is thrown.

The value in *sequenceVar* is encoded binary and must be further decoded according to the definition of the sequence, using the other decoder "asnGet..." commands.

::asn::asnGetSet *dataVar setVar*

Decodes the ASN set value encoded at the start of the data stored in the variable called *dataVar*. The ASN set is stored in the variable called *setVar*, and the decoded bytes are removed from the start of the data in *dataVar*, ready for the next value to be decoded. If the value at the start is not an encoded ASN set an error is thrown.

The value in *setVar* is encoded binary and must be further decoded according to the definition of the sequence, using the other decoder "asnGet..." commands.

::asn::asnGetApplication *dataVar appNumberVar* ?*contentVar*? ?*encodingTypeVar*?

Decodes the ASN application construct value encoded at the start of the data stored in the variable called *dataVar*. The ASN application construct's ID is stored in the variable called *appNumberVar*, and the decoded bytes are removed from the start of the data in *dataVar*, ready for the next value to be decoded. If the value at the start is not an encoded ASN application construct an error is thrown.

If a *contentVar* is specified, then the command places all data associated with the ASN application construct into the named variable, in the binary form which can be processed using the this package's decoder commands.

If a *encodingTypeVar* is specified, then that variable is set to **1** if the encoding is constructed and **0** if it is primitive.

Without the optional arguments, it is the responsibility of the caller to decode the remainder of the ASN application construct based on the ID retrieved by this command, using the package's decoder commands.

::asn::asnGetContext *dataVar contextNumberVar* ?*contentVar*? ?*encodingTypeVar*?

Decodes the ASN context tag construct encoded at the start of the data stored in the variable called *dataVar*. The ASN context tag construct's ID is stored in the variable called *contextNumberVar*, and the decoded bytes are removed from the start of the data in *dataVar*, ready for the next value to be decoded. If the value at the start is not an encoded ASN context tag construct an error is thrown.

If a *contentVar* is specified, then the command places all data associated with the ASN context tag construct into the named variable, in the binary form which can be processed using the this package's decoder commands.

If a *encodingTypeVar* is specified, then that variable is set to **1** if the encoding is constructed and **0** if it is primitive.

Without the optional arguments, it is the responsibility of the caller to decode the remainder of the ASN context tag construct based on the ID retrieved by this command, using the package's decoder commands.

Handling tags

When working with ASN.1 you often need to decode tagged values which use a tag that's different from a type's universal tag. In these cases, to decode the value, you must replace the tag with the type's universal tag. To decode a tagged value use **::asn::asnRetag** to change the

tag to an appropriate type so that you can use one of the decoders for primitive values. To assist with this the package provides three helper functions:

::asn::asnPeekTag *dataVar tagVar tagtypeVar constrVar*
: The **::asn::asnPeekTag** command can be used to take a peek at the data and decode the tag value, without removing it from the data. The *tagVar* is set to the tag number. The *tagtypeVar* is set to the tag's class, which is one of **UNIVERSAL**, **CONTEXT**, **APPLICATION**, or **PRIVATE**. The *constrVar* is set to 1 if the tag is for a constructed value, or to 0 for a primitive. The command returns the tag's length.

::asn::asnTag *tagnumber* ?*class*? ?*tagstyle*?
: The **::asn::asnTag** command is used to create a tag value. The *tagnumber* specifies the tag's number. If given, the tag's *class* must be one of **UNIVERSAL**, **CONTEXT**, **APPLICATION**, or **PRIVATE**; or just the first letter as an abbreviation (**U**, **C**, **A**, or **P**.) The default class is **UNIVERSAL**. If given, the *tagstyle* must be either **C** or **1** for a constructed encoding, or **P** or **0** for a primitive encoding. The default is **P**. (The use of **1** and **0** makes it easy to use this command with the values returned by **::asn::asnPeekTag**.)

::asn::asnRetag *dataVar newTag*
: Replaces the tag at the start of the data in *dataVar* with *newTag*. The new tag can be created using the **::asn::asnTag** command.

Examples

Examples that show this package in use can be found in the implementation of the **ldap** package.

When encoding text it is best to *avoid* **::asn::asnUTF8String** since in rare cases it may not work correctly. For pure 7-bit ASCII it is easiest to encode using **::asn::asnIA5String**. Outside the 7-bit ASCII range convert Tcl strings into UCS2 (i.e., utf-16le) and encode with **::asn::asnBMPString**. For decoding any string use **::asn::asnGetString**; however, for each UCS2 string, remember to convert it back to a Tcl string.

```
# Encode
set eflag [::asn::asnBoolean 1]
set etotal [::asn::asnInteger 734]
set line [encoding convertto utf-16le "€2 ÷ 3 → €⅔ ✓"]
set eline [::asn::asnBMPString $line]
set enote [::asn::asnIA5String "7-bit ASCII ~^"]
set encoded [::asn::asnSequence $eflag $etotal $eline $enote]
# Decode
::asn::asnGetSequence encoded sequence
::asn::asnGetBoolean sequence flag
::asn::asnGetInteger sequence total
::asn::asnGetString sequence bmp_line
set line [encoding convertfrom utf-16le $bmp_line]
::asn::asnGetString sequence note
puts "flag=$flag total=$total line='$line' note='$note'"
=>
flag=1 total=734 line='€2 ÷ 3 → €⅔ ✓' note='7-bit ASCII ~^'
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *asn* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Networking

Copyright


```
Copyright (c) 2004 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
Copyright (c) 2004 Jochen Loewer <loewerj@web.de>
Copyright (c) 2004-2011 Michael Schlenker <mic42@users.sourceforge.net>
```


base32

base32

Keywords: base32, rfc3548, rfc4648

Name

base32 - base32 standard encoding

Synopsis

package require Tcl 8.5 9
package require base32::core ?**0.2**?
package require base32 ?**0.2**?
::base32::encode *bstring*
::base32::decode *estring*

Description

This package provides commands for encoding and decoding binary strings to and from the standard base32 encoding as specified in RFC 4648 *(and also the older* RFC 3548*)*.

API

::base32::encode ***bstring***

Returns a base32-encoded version of the binary string *bstring* as its result.

The result may be padded at the end with **=** characters to ensure that the result's length is a multiple of eight.

The encoder will throw an error if *bstring* is neither a binary string, nor a string containing

only 7-bit ASCII.

::base32::decode *estring*

Returns a binary string that has been base32-decoded from the *estring*.

Note that while the encoder produces only uppercase characters, this decoder is case-insensitive, i.e., more permissive.

The decoder may throw an error on invalid input. For example, if:
- the input contains characters which are not valid for base32-encoded data;
- the length of the input is not a multiple of eight;
- padding appears in the middle rather than at the end of the input;
- the padding's length is not exactly one, three, four, or six characters.

It is also possible to have invalid input that does *not* throw an error—for example, plain text that doesn't do any of the things listed above.

Code map

The code map used to convert 5-bit sequences is shown below, with the numeric ID of the bit sequences to the left and the character used to encode it to the right. Note that the characters "0" and "1" are *not* used by the encoding, to avoid confusion with "O", "o" and "l" (L).

```
 0 A    9 J   18 S   27 3
 1 B   10 K   19 T   28 4
 2 C   11 L   20 U   29 5
 3 D   12 M   21 V   30 6
 4 E   13 N   22 W   31 7
 5 F   14 O   23 X
 6 G   15 P   24 Y
 7 H   16 Q   25 Z
 8 I   17 R   26 2
```

Examples

This example shows how to encode and decode a Tcl string to and from base32, taking account of the fact that the base32 commands work in terms of binary strings.

```
const UTF8_LINE "Δ÷ "Utf-8" ♞ℤ"
set bytes [encoding convertto utf-8 $UTF8_LINE]
set enc32 [::base32::encode $bytes]
set dec32 [::base32::decode $enc32]
set line [encoding convertfrom utf-8 $dec32]
puts "base32='$enc32' [expr {$UTF8_LINE eq $line}]"
=>
base32='Z2KMHNZA4KAJYVLUMYWTRYUATUQOFGM64KCKI===' 1
```

If the original string is 7-bit ASCII the conversions to and from raw bytes using the built-in encoding command are not needed. For example:

```
const ASCII_LINE "! 7-bit ASCII {~^}"
set enc32 [::base32::encode $ASCII_LINE]
set dec32 [::base32::decode $enc32]
set line [encoding convertfrom utf-8 $dec32]
puts "base32='$enc32' [expr {$ASCII_LINE eq $line}]"
=>
base32='EEQDOLLCNF2CAQKTINEUSID3PZPH2===' 1
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *base32* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Text processing

Copyright

```
Public domain
```

base32core

Keywords: base32core, base32

Name

base32::core - Expanding basic base32 maps

Synopsis

package require Tcl 8.5 9
package require base32::core ?**0.2**?
::base32::core::define *map forwvar backwvar ivar*
::base32::core::valid *string pattern mvar*

Description

This package provides generic commands for the construction of full base32 mappings from a basic mapping listing just the codes and associated characters. The full mappings, regular and inverse, created here map to and from bit sequences, and also handle the partial mappings at the end of a string.

This is essentially an internal package to be used by implementors of base32 encoders and decoders. Regular users do not need this package.

API

::base32::core::define *map forwvar backwvar ivar*

This command computes full forward and backward (inverse) mappings from the basic *map* and stores them in the variables named by *forwvar* and *backwvar* resp. It also constructs a regexp pattern for the detection of invalid characters in supposedly base32 encoded input and stores it in the variable named by *ivar*.

::base32::core::valid *string pattern mvar*

This command checks if the input *string* is a valid base32 encoded string, based on the *pattern* of invalid characters as generated by **::base32::core::define**, and some other general rules.

The result of the command is a boolean flag. Its value is **True** for a valid *string*, and **False** otherwise. In the latter case an error message describing the problem with the input is stored into the variable named by *mvar*. The variable is not touched if the input was found to be valid.

The rules checked by the command, beyond rejection of bad characters, are:
1. The length of the input is not a multiple of eight,
2. The padding appears not at the end of input, but in the middle,
3. The padding has not of length six, four, three, or one characters,

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *base32* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Text processing

Copyright

```
Public domain
```

base32hex

Keywords: base32hex, base32, hex, rfc4648

Name

base32::hex - base32 extended hex encoding

Synopsis

package require Tcl 8.5 9
package require base32::core ?**0.2**?
package require base32::hex ?**0.2**?
::base32::hex::encode *bstring*
::base32::hex::decode *estring*

Description

This package provides commands for encoding and decoding binary strings to and from the extended hex base32 encoding as specified in RFC 4648.

(Note that Tcl's built-in binary command provides simple encoding and decoding of bytes to and from pairs of hex digits.)

API

::base32::hex::encode *bstring*

Returns an extended hex base32-encoded version of the binary string *bstring* as its result.

The result may be padded at the end with **=** characters to ensure that the result's length is a multiple of eight.

The encoder will throw an error if *bstring* is neither a binary string, nor a string containing only 7-bit ASCII.

::base32::hex::decode *estring*

Returns a binary string that has been extended hex base32-decoded from the *estring*.

Note that while the encoder produces only uppercase characters, this decoder is case-insensitive, i.e., more permissive.

The decoder may throw an error on invalid input. For example, if:
- the input contains characters which are not valid for extended hex base32-encoded data;
- the length of the input is not a multiple of eight;
- padding appears in the middle rather than at the end of the input;
- the padding's length is not exactly one, three, four, or six characters.

It is also possible to have invalid input that does *not* throw an error—for example, plain text that doesn't do any of the things listed above.

Code map

The code map used to convert 5-bit sequences is shown below, with the numeric ID of the bit sequences to the left and the character used to encode it to the right. The important feature of the extended hex mapping is that the first 16 codes map to the digits and hex characters.

```
 0 0    9 9        18 I   27 R
 1 1   10 A        19 J   28 S
 2 2   11 B        20 K   29 T
 3 3   12 C        21 L   30 U
 4 4   13 D        22 M   31 V
 5 5   14 E        23 N
 6 6   15 F        24 O
 7 7        16 G   25 P
 8 8        17 H   26 Q
```

Examples

This example shows how to encode and decode a Tcl string to and from extended hex base32, taking account of the fact that the extended hex base32 commands work in terms of binary

strings.

```
const UTF8_LINE "Δ÷ "Utf-8" ♞ℤ"
set bytes [encoding convertto utf-8 $UTF8_LINE]
set enc32 [::base32::hex::encode $bytes]
set dec32 [::base32::hex::decode $enc32]
set line [encoding convertfrom utf-8 $dec32]
puts "base32::hex='$enc32' [expr {$UTF8_LINE eq $line}]"
=>
base32::hex='PQAC7DP0SA09OLBKCOMJHOK0JKGE56CUSA2A8===' 1
```

If the original string is 7-bit ASCII the conversions to and from raw bytes using the **encoding** command are not needed. For example:

```
const ASCII_LINE "! 7-bit ASCII {~^}"
set enc32 [::base32::hex::encode $ASCII_LINE]
set dec32 [::base32::hex::decode $enc32]
set line [encoding convertfrom utf-8 $dec32]
puts "base32::hex='$enc32' [expr {$ASCII_LINE eq $line}]"
=>
base32::hex='44G3EBB2D5Q20GAJ8D4KI83RFPF7Q===' 1
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *base32* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Text processing

Copyright

```
Public domain
```

base64

ascii85

Keywords: ascii85, encoding

Name

ascii85 - ascii85-encode/decode binary data

Synopsis

package require Tcl 8.5 9

package require ascii85 ?**1.1**?
::ascii85::encode ?*-maxlen maxlen*? ?*-wrapchar wrapchar*? *bstring*
::ascii85::decode *estring*

Description

This package provides commands for encoding and decoding binary strings to and from the ascii85 encoding. (This encoding is also known as base85.)

API

::ascii85::encode ?-maxlen *maxlen*? ?-wrapchar *wrapchar*? *bstring*

Returns an ascii85-encoded version of the binary string *bstring* as its result.

The result string contains the *wrapchar* character after every *maxlen* characters of output. The *wrapchar* defaults to newline and *maxlen* defaults to 76. To prevent wrapping, set *maxlen* to **0**.

The command will throw an error if *maxlen* is not an integer or is negative, or if *bstring* is neither a binary string, nor a string containing only 7-bit ASCII.

::ascii85::decode *estring*

Returns a binary string that has been ascii85-decoded from the *estring*. Any whitespace (spaces, tabs, newlines) in *estring* is ignored.

Examples

This example shows how to encode and decode a Tcl string to and from ascii85, taking account of the fact that the ascii85 commands work in terms of binary strings.

```
const UTF8_LINE "Δ÷ "Utf-8" ♞ℤ"
set bytes [encoding convertto utf-8 $UTF8_LINE]
set enc85 [::ascii85::encode $bytes]
set dec85 [::ascii85::decode $enc85]
set line [encoding convertfrom utf-8 $dec85]
puts "ascii85='$enc85' [expr {$UTF8_LINE eq $line}]"
=>
ascii85='cBQ5U+Q@pA<HMh)39#IZ+QAf\ie4*' 1
```

If the original string is 7-bit ASCII the conversions to and from raw bytes using the built-in **encoding** command are not needed. For example:

```
const ASCII_LINE "! 7-bit ASCII {~^}"
set enc85 [::ascii85::encode $ASCII_LINE]
set dec85 [::ascii85::decode $enc85]
set line [encoding convertfrom utf-8 $dec85]
puts "ascii85='$enc85' [expr {$ASCII_LINE eq $line}]"
=>
ascii85='+Wr]q@VKp,5uU-B8K`A/?@;' 1
```

Wrapping can be prevented so that no whitespace is introduced:

```
ascii85::encode [string repeat xyz 24]
=>
G^4U[H$X^\H?a^]G^4U[H$X^\H?a^]G^4U[H$X^\H?a^]G^4U[H$X^\H?a^]G^4U[H$X^\H
    ?a^]G
^4U[H$X^\H?a^]
ascii85::encode -maxlen 0 [string repeat xyz 24]
=>
G^4U[H$X^\H?a^]G^4U[H$X^\H?a^]G^4U[H$X^\H?a^]G^4U[H$X^\H?a^]G^4U[H$X^\H
    ?a^]G^4U[H$X^\H?a^]
```

References

1. http://en.wikipedia.org/wiki/Ascii85
2. Postscript Language Reference Manual, 3rd Edition, page 131.
   http://www.adobe.com/devnet/postscript/pdfs/PLRM.pdf

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *base64* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Text processing

Copyright

```
Copyright (c) 2010, Emiliano Gavilán
```

base64

Keywords: base64, encoding

Name

base64 - base64-encode/decode binary data

Synopsis

package require Tcl 8.5 9
package require base64 ?**2.6.1**?
::base64::encode ?*-maxlen maxlen*? ?*-wrapchar wrapchar*? *bstring*
::base64::decode *estring*

Description

This package provides commands for encoding and decoding binary strings to and from the standard base64 encoding as specified in RFC 4648.

It is recommended that for new code the built-in binary subcommands for encoding and decoding base64 (introduced in Tcl 8.6), are used in preference to this package's commands.

Beware: variations in decoding behaviour

Tcl 8.6 introduced built-in support for encoding and decoding base64 using subcommands of the binary command:

```
    binary encode base64
    binary decode base64
```

Beware that although these subcommands have the same signatures as the commands provided by this package, the decoders are *not behaviourally compatible*.

The built-in binary decode base64 subcommand accepts the option **-strict**, enabling the user to choose between strict and nonstrict modes. In strict mode invalid characters, and partial data at the end of the input are reported as errors. In nonstrict mode they are ignored. (See RFC 2045.)

The implementations provided by this package do not offer control over strictness. Instead they mix strict and nonstrict. Partial data at the end of the input is reported as an error, but invalid characters are ignored.

By default this package's encoder wraps using newlines after every 76 characters of output. The built-in binary encode base64 subcommand defaults to not wrapping, but does support wrapping using a **-maxlen** option.

API

::base64::encode ?-maxlen *maxlen*? ?-wrapchar *wrapchar*? *bstring*

Returns a base64-encoded version of the binary string *bstring* as its result.

The result string contains the *wrapchar* character after every *maxlen* characters of output. The *wrapchar* defaults to newline and *maxlen* defaults to 76. To prevent wrapping, set *maxlen* to **0**.

The command will throw an error if *maxlen* is not an integer or is negative, or if *bstring* is neither a binary string, nor a string containing only 7-bit ASCII.

::base64::decode *estring*

Returns a binary string that has been base64-decoded from the *estring*. Any invalid characters or whitespace (spaces, tabs, newlines) in *estring* are ignored.

Implementation notes

This package contains three different implementations for base64 encoding and decoding, and chooses among them based on the environment it finds itself in.

All three implementations have the same behaviour. See also Beware: variations in decoding behaviour at the beginning of this document.

1. If Tcl 8.6 or higher is found the commands are implemented in terms of the then-available built-in commands.
2. If the Trf extension can be loaded the commands are implemented in terms of its commands.
3. If neither of the above are possible a Tcl-only implementation is used. This is much slower.

Examples

This example shows how to encode and decode a Tcl string to and from base64, taking account of the fact that the base64 commands work in terms of binary strings.

```
const UTF8_LINE "Δ÷ "Utf-8" ♞ℤ"
# Legacy (this package)
set bytes [encoding convertto utf-8 $UTF8_LINE]
set enc64 [::base64::encode $bytes]
set dec64 [::base64::decode $enc64]
set line [encoding convertfrom utf-8 $dec64]
puts "base64='$enc64' [expr {$UTF8_LINE eq $line}]"
=>
base64='zpTDtyDigJxVdGYtOOKAnSDimZ7ihKQ=' 1
# Modern (using built-in binary subcommands)
set bytes [encoding convertto utf-8 $UTF8_LINE]
set enc64 [binary encode base64 $bytes]
set dec64 [binary decode base64 $enc64]
set line [encoding convertfrom utf-8 $dec64]
puts "base64='$enc64' [expr {$UTF8_LINE eq $line}]"
=>
base64='zpTDtyDigJxVdGYtOOKAnSDimZ7ihKQ=' 1
```

If the original string is 7-bit ASCII the conversions to and from raw bytes using the built-in encoding command are not needed. For example:

```
const ASCII_LINE "! 7-bit ASCII {~^}"
# Legacy (this package)
set enc64 [::base64::encode $ASCII_LINE]
set dec64 [::base64::decode $enc64]
set line [encoding convertfrom utf-8 $dec64]
puts "base64='$enc64' [expr {$ASCII_LINE eq $line}]"
=>
base64='ISA3LWJpdCBBU0NJSSB7fl59' 1
# Modern (using built-in binary subcommands)
set enc64 [binary encode base64 $ASCII_LINE]
set dec64 [binary decode base64 $enc64]
set line [encoding convertfrom utf-8 $dec64]
puts "base64='$enc64' [expr {$ASCII_LINE eq $line}]"
=>
base64='ISA3LWJpdCBBU0NJSSB7fl59' 1
```

Wrapping can be prevented so that no whitespace is introduced:

```
base64::encode [string repeat xyz 20]
=>
eHl6eHl6eHl6eHl6eHl6eHl6eHl6eHl6eHl6eHl6eHl6eHl6eHl6eHl6eHl6eHl6eHl6
eHl6eHl6eHl6
base64::encode -maxlen 0 [string repeat xyz 20]
```

```
=>
eHl6eHl6eHl6eHl6eHl6eHl6eHl6eHl6eHl6eHl6eHl6eHl6eHl6eHl6eHl6eHl6eHl6eHl
    6
```

Note that the built-in binary encode base64 subcommand defaults to not wrapping, but will wrap if the **-maxlen** option is used.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *base64* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Text processing

Copyright

```
Copyright (c) 2000, Eric Melski
Copyright (c) 2001, Miguel Sofer
```

uuencode

Keywords: uuencode, encoding

Name

uuencode - UU-encode/decode binary data

Synopsis

package require Tcl 8.5 9
package require uuencode ?**1.1.6**?
::uuencode::encode *bstring*
::uuencode::decode *estring*
::uuencode::uuencode ?*-name string*? ?*-mode octal*? (**-file** *filename* | ?**--**? *bstring*)
::uuencode::uudecode (**-file** *filename* | ?**--**? *estring*)

Description

This package provides a Tcl-only implementation of the **uuencode(1p)** https://www.man7.org/linux/man-pages/man1/uuencode.1p.html and **uudecode(1p)** https://www.man7.org/linux/man-pages/man1/uudecode.1p.html commands. This encoding packs binary data into printable 7-bit ASCII characters.

API

::uuencode::encode *bstring*

Returns a uuencoded version of the binary string *bstring* as its result.

All the given bytes are encoded even if they exceed the uuencode maximum line length. If the number of input bytes is not a multiple of 3 then the string is padded with additional **\x00** NUL bytes.

The command will throw an error if *bstring* is neither a binary string, nor a string containing only 7-bit ASCII.

::uuencode::decode *estring*
Returns a binary string that has been uuencode-decoded from the *estring*, including any padding **\x00** NUL bytes at the end.

::uuencode::uuencode ?-name *string*? ?-mode *octal*? (-file *filename* | ?--? *bstring*)
Returns a wrapped uuencoded version of the data in the file specified by **-file** *filename* or by the binary string *bstring* as its result.

Each wrapped uuencoded result consists of a header, body, and footer. The header is a line starting with **begin** followed by a suggested file mode and suggested filename. The body consists of lines of uuencoded data, each prefixed with a line length character. The footer is **end** on its own line. See **uuencode(1)** https://www.man7.org/linux/man-pages/man1/uuencode.1p.html.

::uuencode::uudecode (-file *filename* | ?--? *estring*)
Returns a list of 3-element lists of filename, file mode, uuencode-decoded bytes, from the file specified by **-file** *filename* or from the *estring*.

Options

-file *filename*
Cause the **uuencode** or **uudecode** commands to read their data from the named file rather that taking a binary string parameter.

-name *string*
The uuencoded data header line contains the suggested file name to be used when unpacking the data. Use this option to change this from the default of "data.dat".

-mode *octal*
The uuencoded data header line contains a suggested permissions bit pattern expressed as an octal string. To change the default of 0644 you can set this option. For instance, 0755 would be suitable for an executable. See **chmod(1)** https://www.man7.org/linux/man-pages/man1/chmod.1.html.

Examples

The examples use the subcommand **string trimright** to trim any **\x00** NUL bytes that may have been added during encoding. *Note however, that if the original binary strings might end with* **\x00** NUL *bytes, a more sophisticated approach will be needed.*

This example shows how to uuencode and uudecode a Tcl string, taking account of the fact that the uuencode commands work in terms of binary strings.

```
const UTF8_LINE "Δ÷ "Utf-8" ♞ℤ"
set bytes [encoding convertto utf-8 $UTF8_LINE]
set encuu [::uuencode::encode $bytes]
set decuu [string trimright [::uuencode::decode $encuu] \x00]
set line [encoding convertfrom utf-8 $decuu]
puts "uuencode='$encuu' [expr {$UTF8_LINE eq $line}]"
=>
uuencode='SI3#MR#B@)Q5=&8M..*`G2#BF9[BA*0`' 1
```

If the original string is 7-bit ASCII the conversions to and from raw bytes using the built-in **encoding** command are not needed. For example:

```
const ASCII_LINE "! 7-bit ASCII {~^}"
set encuu [::uuencode::encode $ASCII_LINE]
set decuu [string trimright [::uuencode::decode $encuu] \x00]
set line [encoding convertfrom utf-8 $decuu]
puts "uuencode='$encuu' [expr {$ASCII_LINE eq $line}]"
=>
uuencode='(2`W+6)I="!!4T-)22![?EY]' 1
```

This example shows how to create a uuencoded byte string and write it to a file.

```
const UTF8_LINE "Δ÷ "Utf-8" ♞ℤ"
set bytes [encoding convertto utf-8 $UTF8_LINE]
set encuu [::uuencode::uuencode -name test.uu $bytes]
writeFile test.uu binary $encuu
puts $encuu
=>
begin 644 test.uu
7SI3#MR#B@)Q5=&8M..*`G2#BF9[BA*0`
`
end
```

This example shows how to read a file containing one or more uuencoded files.

```
foreach lst [::uuencode::uudecode -file test.uu] {
    lassign $lst name mode decuu
    set decuu [string trimright $decuu \x00]
    set line [encoding convertfrom utf-8 $decuu]
    puts "name=$name mode=$mode line='$line'"
}
=>
name=test.uu mode=644 line='Δ÷ "Utf-8" ♞ℤ'
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *base64* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Text processing

Copyright
```
Copyright (c) 2002, Pat Thoyts
```

yencode
Keywords: yencode, encoding, yEnc, ydecode

Name
yencode - Y-encode/decode binary data

Synopsis
package require Tcl 8.5 9
package require yencode ?**1.1.4**?
::yencode::encode *bstring*
::yencode::decode *estring*
::yencode::yencode ?*-name string*? ?*-line integer*? ?*-crc32 boolean*? (**-file** *filename* | ?**--**? *bstring*)
::yencode::ydecode (**-file** *filename* | ?**--**? *estring*)

Description
This package provides a Tcl-only implementation of the yEncode file encoding used for Usenet messages. This encoding packs binary data into a format that requires an 8-bit clean transmission layer but that escapes characters special to the *NNTP* posting protocols. See http://www.yenc.org for the encoding's details.

API
::yencode::encode *bstring*
Returns a yEncoded version of the binary string *bstring* as its result.

The command will throw an error if *bstring* is neither a binary string, nor a string containing only 7-bit ASCII.

::yencode::decode *estring*
Returns a binary string that has been yEncode-decoded from the *estring*.

::yencode::yencode ?-name *string*? ?-line *integer*? ?-crc32 *boolean*? (-file *filename* | ?-- ? *bstring*)
Returns a wrapped yEncoded version of the data in the file specified by **-file** *filename* or by the binary string *bstring* as its result.

Each wrapped yEncoded result consists of a header, body, and trailer. The header is a line

starting with **=ybegin** followed by a line length of form **line=***length*, the size of the original data (in bytes) of form **size=***size*, and the name of the original file of form **name=***filename*, all space-separated. The body is the yEncoded data. The trailer is a line starting with **=yend** followed by the size of the original data of form **size=***size*, and optionally (but recommended), a CRC32 checksum value of form **crc32=***checksum*, all space-separated.

::yencode::ydecode (-file *filename* | ?--? *estring*)
Returns a list of 3-element lists of filename, file size, yEncode-decoded bytes, from the file specified by **-file** *filename* or from the *estring*.

Options

-file *filename*
Cause the **yencode** or **ydecode** commands to read their data from the named file rather that taking a binary string parameter.

-name *string*
The yEncoded data header line contains the original file name to be used when unpacking the data. Use this option to change this from the default of "default.bin".

-line *integer*
The yEncoded header line specifies the line length used for the encoding, and which defaults to 128. Use this option to set a different line length. Note that NNTP imposes a 1000 character line length limit and some systems may have trouble with more than 255 characters per line.

-crc32 *boolean*
When encoding this package puts a cyclic redundancy check (CRC) value in the trailer as recommended in the yEncode specification. This can be prevented by using this option and passing a false value for *boolean*, e.g., **0**.

Examples

The yEncoded data is not shown in the examples because it is just raw bytes.

This example shows how to yEncode and yEncode-decode a Tcl string, taking account of the fact that the yEncode commands work in terms of binary strings.

```
const UTF8_LINE "Δ÷ "Utf-8" ♞ℤ"
set bytes [encoding convertto utf-8 $UTF8_LINE]
set ency [::yencode::encode $bytes]
set decy [::yencode::decode $ency]
set line [encoding convertfrom utf-8 $decy]
puts "[expr {$UTF8_LINE eq $line}]"
=>
1
```

If the original string is 7-bit ASCII the conversions to and from raw bytes using the built-in **encoding** command are not needed. For example:

```
const ASCII_LINE "! 7-bit ASCII {~^}"
set ency [::yencode::encode $ASCII_LINE]
set decy [::yencode::decode $ency]
set line [encoding convertfrom utf-8 $decy]
puts "[expr {$ASCII_LINE eq $line}]"
=>
1
```

This example shows how to create a yEncoded byte string and write it to a file.

```
set bytes [encoding convertto utf-8 $UTF8_LINE]
set ency [::yencode::yencode -name test.y $bytes]
writeFile test.y binary $ency
puts $ency
=>
=ybegin line=128 size=23 name=test.y
    … raw binary data elided …
=yend size=23 crc32=fc01071
```

This example shows how to read a file containing one or more yEncoded files.

```
foreach lst [::yencode::ydecode $ency] {
    lassign $lst name size decy
    set line [encoding convertfrom utf-8 $decy]
    puts "name=$name size=$size line='$line'"
}
=>
name=test.y size=23 line='Δ÷ "Utf-8" ♞ℤ'
```

References

1. http://www.yenc.org/yenc-draft.1.3.txt

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *base64* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Text processing

Copyright

```
Copyright (c) 2002, Pat Thoyts
```

bee

bee

Keywords: bee, BitTorrent, serialization, torrent

Name

bee - BitTorrent Serialization Format Encoder/Decoder

Synopsis

package require Tcl 8.5 9
package require bee ?**0.3**?
::bee::encodeString *string*
::bee::encodeNumber *integer*
::bee::encodeListArgs ?*beevalue* ...?
::bee::encodeList *listofbees*
::bee::encodeDictArgs ?*key beevalue* ...?
::bee::encodeDict *dictofbees*
::bee::decode *beestring* ?*endVar*? ?*start*?
::bee::decodeIndices *beestring* ?*endVar*? ?*start*?
::bee::decodeChannel *chan* **-command** *on_decode* ?**-exact**? ?*-prefix data*?
on_decode value *handle value*
on_decode error *handle message*
on_decode eof *handle*
::bee::decodeCancel *handle*
::bee::decodePush *handle string*

Description

This package provides encoding and decoding commands for data in Bencode (spoken as "bee-encode"), the BitTorrent protocol's serialization format used for data and messages.

API

Encoder

This package provides an encoder command for each value type (string and integer), and two commands per collection type (list and dict), one taking a complete Tcl collection to encode, the other taking the same data given as arguments.

::bee::encodeString *string*
Returns the bee-encoding of the *string*.

::bee::encodeNumber *integer*
Returns the bee-encoding of the *integer* number.

This command will throw an error if *integer* is not an integer.

::bee::encodeListArgs ?*beevalue ...*?
Returns a bee-encoded list of the given zero or more bee-encoded *beevalue*s.

::bee::encodeList *listofbees*
Returns a bee-encoded list of the given Tcl list, *listofbees*, of bee-encoded values.

::bee::encodeDictArgs ?*key beevalue ...*?
Returns a bee-encoded dict of the given zero or more pairs of Tcl string keys and bee-encoded *beevalue*s.

This command will bee-encode each string key itself.

::bee::encodeDict *dictofbees*
Returns a bee-encoded dict of the given Tcl dict, *dictofbees*, whose keys must be Tcl strings and whose values must be bee-encoded values.

This command will bee-encode each string key itself.

Decoder

This package provides decoding commands for bee-encoded strings and for bee-encoded data received over a channel.

The commands for decoding a bee-encoded string expect the string to contain one or more complete bee-encoded values (which may themselves contain nested bee-encoded lists and dicts).

The command for the incremental decoding of bee-values arriving on a channel is asynchronous and provides the completed decoded values via a callback.

::bee::decode *beestring* ?*endVar*? ?*start*?
Returns a bee-decoded value from the *beestring*, starting from index position *start* if given—or from its default of **0**, and sets *endVar* if given.

If the bee-encoded value is a string or integer, the equivalent Tcl string or integer is returned.

If the bee-encoded value is a collection (a bee-list or a bee-dict), then a corresponding Tcl list or Tcl dict is returned. And in either of these cases, all contained values (including nested bee-lists and bee-dicts) are all bee-decoded, so that the result is a Tcl list or dict with all the nested values fully bee-decoded into Tcl lists, dicts, strings, or integers, as appropriate.

If the option *endVar* is given, it is the name of a variable used to store the index position of the first character in the *beestring* that *follows* the value that's just been decoded, i.e., the start position of the next value to decode. This is useful if the *beestring* contains two or more bee-encoded values concatenated together (as opposed to a single bee-encoded list of bee-encoded values). See the last example in the Examples.

The optional *start* is the index position from which the next bee-encoded value will be read and decoded.

::bee::decodeIndices *beestring* ?*endVar*? ?*start*?
Returns a list based on the bee-decoded value from the *beestring*, starting from index position *start* if given—or from its default of **0**, and sets *endVar* if given. The meaning and use of *endVar* and *start* are exactly the same as for **::bee::decode**.

The returned list's elements depend on the type of the bee-decoded value. In all cases though, the first element is a type name (one of **string integer list dict**), and the second and third elements are the start and end index positions in the *beestring* for the bee-encoded

value. For bee-lists and bee-dicts there is a fourth element which provides details of their contained bee-values.

Formally the result lists for the various types of bee-values are:

string

A list containing three elements:
- A constant string indicating the value's type, for strings it is **string**.
- The index position (>= *0*) of the first character of the bee-value in the given *beestring*.
- The index position (>= *0*) of the last character of the bee-value in the given *beestring*.

Every result list, for every bee-value, begins with these three elements.

integer

A list containing the constant string **integer** and—just the same as for strings—the start and end index positions of the bee-value in the *beestring*.

list

A list containing the constant string **list** and—just the same as for strings—the start and end index positions of the bee-value in the *beestring*.

In addition the list has a fourth element, which itself contains the index data as described here for all the elements of the bee-list.

dictionary

A list containing the constant string **dict** and—just the same as for strings—the start and end index positions of the bee-value in the *beestring*.

In addition the dict has a fourth element. This element is a dictionary mapping from each of the bee-dict's string keys to a 2-element list of 2-element lists. The list's first element's list contains the index positions for the key, and the second element's list contains the index positions for the value the key maps to. This structure is the only one which contains not only index positions, but actual values from the *beestring*. While each key's index positions are unique, i.e., usable as a key, they are not easy to navigate when trying to find a particular element. Using actual keys makes navigation much easier, which is why they're provided.

::bee::decodeChannel *chan* -command *on_decode* ?-exact? ?-prefix *data*?

This command creates a decoder for a series of bee-values arriving on the channel *chan* and returns a decoder handle. Each decoding event is reported to the required *on_decode* callback command.

The returned handle can be used to push data into the decoder or to cancel the decoder.

Attempting to create another bee decoder on the *chan* channel while an existing bee decoder is still active will throw an error.

-command

The *on_decode* command specified by the *required* option **-command** is used to report decoded values, errors, and channel EOF.

The callback is executed at the interpreter's global level, with two or three arguments. The possible callback signatures are:

on_decode value *handle value*

The decoder received and successfully decoded a bee-value. The format of the equivalent Tcl *value* is the same as returned by **::bee::decode**. The channel is still open and the decoder handle is valid. This means that the callback is able to push data into the decoder or to cancel the decoder.

on_decode error *handle message*
The decoder encountered an error, which is not EOF. For example, an invalid bee-value. The *message* provides details about the error. The decoder handle is in the same state as for EOF, i.e., invalid, and cannot be used. However, the channel remains open.

on_decode eof *handle*
The decoder has reached EOF on the *chan* channel. No further calls to the callback will be made after this. The channel has already been closed at the time of the call, and the *handle* is invalid and cannot be used.

-exact
By default the decoder assumes that all the channel's data consists of bee-values, and reads as much as possible per event, without regard for boundaries between bee-values. This means that if the the channel contains non-bee data after a series of bee-values, the beginning of any non-bee data may be lost, because it has already been read by the decoder, but not processed.

In such cases use the **-exact** option. When this is specified, the decoder will not read any characters beyond the currently processed bee-value, so that any non-bee data is left in the channel for further processing after the decoder is closed.

-prefix
If this option is specified its value is assumed to be the beginning of a bee-value and is used to initialize the decoder's internal buffer. This feature is required if the creator of the decoder read data from the channel to determine if it should create the decoder or not. Without the option this initial data would be lost to the decoding.

::bee::decodeCancel *handle*
This command cancels the decoder set up by **::bee::decodeChannel** and represented by the handle *handle*.

::bee::decodePush *handle string*
This command appends the *string* to the decoder's internal buffer. It is the runtime equivalent of the **::bee::decodeChannel** command's **-prefix** option. Use this command to push data back into the decoder when the **on_decode value** callback used data from the channel to determine if it should decode another bee-value or not.

Format definition

Data in the Bencode serialization format is constructed from two basic forms, and two collection forms. The basic forms are strings and integerss, and the collections are lists and dictionaries.

String *S*
A string *S* of length *L* is encoded by the string "*L*:*S*", where the length is in textual form. For example, the string **"Bencode format"** would be encoded as **"14:Bencode format"**.

Integer *N*
An integer *N* is encoded by the string "**i***N***e**". For example, the integer **-482** would be encoded

as "**i-482e**".

List *v1* ... *vn*
: A list of the values *v1* to *vn* is encoded by the string "**l***BV1*...*BVn***e**" where "BV**i**" is the bee-encoding of the value "v**i**".

Dict *k1* -> *v1* ...
: A dictionary mapping the string key *k***i** to the value *v***i**, for **i** in **1** ... **n** is encoded by the string "**d***BK***i***BV***i**...**e**" for i in **1** ... **n**, where "BK**i**" is the bee-encoding of the key string "k**i**". and "BV**i**" is the bee-encoding of the value "v**i**".

  Note: The bee-encoding does not preserve the order of the keys in the input, but stores in key-sorted order.

Note that the type of each encoded item can be determined immediately from the first character of its encoding:

i
: Integer.

l
: List.

d
: Dictionary.

[0-9]
: String.

By prefixing integers with **i** the format ensures that they are different from strings, which all begin with a digit.

Examples

This example shows how to bee-encode and bee-decode individual strings and integers.

```
const LABEL "Δ÷ "Utf-8" ♞ℤ"
set labelbeeEnc [::bee::encodeString $LABEL]
set labelbeeDec [::bee::decode $labelbeeEnc]
puts "labelbeeEnc='$labelbeeEnc' eq LABEL → [expr {$labelbeeDec eq
    $LABEL}]"
set xbeeEnc [::bee::encodeNumber 834]
set ybeeEnc [::bee::encodeNumber -172]
set ybeeDec [::bee::decode $ybeeEnc]
puts "ybeeEnc='$ybeeEnc' == -172 → [expr {$ybeeDec == -172}]"
=>
labelbeeEnc='13:Δ÷ "Utf-8" ♞ℤ' eq LABEL → 1
ybeeEnc='i-172e' == -172 → 1
```

This example shows how to bee-encode collections of bee-encoded values.

```
set pointbeeEnc [::bee::encodeList [list $xbeeEnc $ybeeEnc]]
set placebeeEnc [::bee::encodeDictArgs label $labelbeeEnc point
    $pointbeeEnc]
puts "pointbeeEnc='$pointbeeEnc'"
```

```
puts "placebeeEnc='$placebeeEnc'"
=>
pointbeeEnc='li834ei-172ee'
placebeeEnc='d5:label13:Δ÷ "Utf-8" ♞ℤ5:pointli834ei-172eee'
```

This example shows how to bee-decode collections of bee-encoded values.

```
set placebeeDec [::bee::decode $placebeeEnc]
set label [dict get $placebeeDec label]
puts "label='$label' eq LABEL → [expr {$label eq $LABEL}]"
lassign [dict get $placebeeDec point] x y
puts "x=$x == 834 → [expr {$x == 834}] • y=$y == -172 → [expr {$y ==
    -172}]"
=>
label='Δ÷ "Utf-8" ♞ℤ' eq LABEL → 1
x=834 == 834 → 1 • y=-172 == -172 → 1
```

This example shows how to bee-decode using the **::bee::decodeIndices** command.

```
set labelbeeDecInd [::bee::decodeIndices $labelbeeEnc]
lassign $labelbeeDecInd type i j
set label [::bee::decode [string range $labelbeeEnc $i $j]]
puts "labelbeeDecInd type=$type i=$i j=$j label='$label'"
=>
labelbeeDecInd type=string i=0 j=15 label='Δ÷ "Utf-8" ♞ℤ'
```

This example shows how to bee-encode and then bee-decode a sequence of *separate* bee-encoded values.

```
set stream [::bee::encodeString "€⅔•"][::bee::encodeNumber
    -91][::bee::encodeString "α-ω"][::bee::encodeNumber 68]
for {set i 0} {$i < [string length $stream]} {} {
    lappend items [::bee::decode $stream i $i]
}
puts "stream='$stream' items={$items} size=[llength $items]"
=>
stream='3:€⅔•i-91e3:α-ωi68e' items={€⅔• -91 α-ω 68} size=4
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *bee* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Networking

Copyright

```
Copyright (c) 2004 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

bench

bench

Keywords: bench, benchmark, merging, normalization, performance, testing

Name

bench - bench - Processing benchmark suites

Synopsis

package require Tcl 8.5 9
package require bench ?**0.6**?
::bench::locate *pattern paths*
::bench::run ?*option value...*? *interp_list file...*
::bench::versions *interp_list*
::bench::del *bench_result column*
::bench::edit *bench_result column newvalue*
::bench::merge *bench_result...*
::bench::norm *bench_result column*
::bench::out::raw *bench_result*

Description

This package provides commands for the execution of benchmarks written in the bench language, and for the processing of results generated by such execution.

A reader interested in the bench language itself should start with the *bench language introduction* and proceed from there to the formal *bench language specification*.

Public API

Benchmark execution

::bench::locate *pattern paths*

This command locates Tcl interpreters and returns a list containing their paths. It searches them in the list of *paths* specified by the caller, using the glob *pattern*.

The command resolves soft links to find the actual executables matching the pattern. Note that only interpreters which are marked as executable and are actually executable on the current platform are put into the result.

::bench::run ?*option value*...? *interp_list file*...

This command executes the benchmarks declared in the set of files, once per Tcl interpreter specified via the *interp_list*, and per the configuration specified by the options, and then returns the accumulated timing results. The format of this result is described in section Result

format.

It is assumed that the contents of the files are written in the bench language.

The available options are

-errors *flag*
: The argument is a boolean value. If set errors in benchmarks are propagated to the command, aborting benchmark execution. Otherwise they are recorded in the timing result via a special result code. The default is to propagate and abort.

-threads *n*
: The argument is a non-negative integer value declaring the number of threads to use while executing the benchmarks. The default value is **0**, to not use threads.

-match *pattern*
: The argument is a glob pattern. Only benchmarks whose description matches the pattern are executed. The default is the empty string, to execute all patterns.

-rmatch *pattern*
: The argument is a regular expression pattern. Only benchmarks whose description matches the pattern are executed. The default is the empty string, to execute all patterns.

-iters *n*
: The argument is positive integer number, the maximal number of iterations for any benchmark. The default is **1000**. Individual benchmarks can override this.

-pkgdir *path*
: The argument is a path to an existing, readable directory. Multiple paths can be specified, simply use the option multiple times, each time with one of the paths to use.

  If no paths were specified the system will behave as before. If one or more paths are specified, say **N**, each of the specified interpreters will be invoked **N** times, with one of the specified paths. The chosen path is put into the interpreters' **auto_path**, thus allowing it to find specific versions of a package.

  In this way the use of **-pkgdir** allows the user to benchmark several different versions of a package, against one or more interpreters.

  Note: The empty string is allowed as a path and causes the system to run the specified interpreters with an unmodified **auto_path**. In case the package in question is available there as well.

::bench::versions *interp_list*
: This command takes a list of Tcl interpreters, identified by their path, and returns a dictionary mapping from the interpreters to their versions. Interpreters which are not actually executable, or fail when interrogated, are not put into the result. I.e the result may contain less interpreters than there in the input list.

  The command uses builtin command **info patchlevel** to determine the version of each interpreter.

Result manipulation

::bench::del *bench_result column*
This command removes a column, i.e. all benchmark results for a specific Tcl interpreter, from the specified benchmark result and returns the modified result.

The benchmark results are in the format described in section Result format.

The column is identified by an integer number.

::bench::edit *bench_result column newvalue*
This command renames a column in the specified benchmark result and returns the modified result. This means that the path of the Tcl interpreter in the identified column is changed to an arbitrary string.

The benchmark results are in the format described in section Result format.

The column is identified by an integer number.

::bench::merge *bench_result...*
This commands takes one or more benchmark results, merges them into one big result, and returns that as its result.

All benchmark results are in the format described in section Result format.

::bench::norm *bench_result column*
This command normalizes the timing results in the specified benchmark result and returns the modified result. This means that the cell values are not times anymore, but factors showing how much faster or slower the execution was relative to the baseline.

The baseline against which the command normalizes are the timing results in the chosen column. This means that after the normalization the values in this column are all **1**, as these benchmarks are neither faster nor slower than the baseline.

A factor less than **1** indicates a benchmark which was faster than the baseline, whereas a factor greater than **1** indicates a slower execution.

The benchmark results are in the format described in section Result format.

The column is identified by an integer number.

::bench::out::raw *bench_result*
This command formats the specified benchmark result for output to a file, socket, etc. This specific command does no formatting at all, it passes the input through unchanged.

For other formatting styles see the packages **bench::out::text** and **bench::out::csv** which provide commands to format benchmark results for human consumption, or as CSV data importable by spread sheets, respectively.

Complementary, to read benchmark results from files, sockets etc. look for the package **bench::in** and the commands provided by it.

Result format

After the execution of a set of benchmarks the raw result returned by this package is a Tcl dictionary containing all the relevant information. The dictionary is a compact representation, i.e. serialization, of a 2-dimensional table which has Tcl interpreters as columns and benchmarks as rows. The cells of the table contain the timing results. The Tcl interpreters / columns are identified by their paths. The benchmarks / rows are identified by their description.

The possible keys are all valid Tcl lists of two or three elements and have one of the following forms:

{interp *}
: The set of keys matching this glob pattern capture the information about all the Tcl interpreters used to run the benchmarks. The second element of the key is the path to the interpreter.

  The associated value is the version of the Tcl interpreter.

{desc *}
: The set of keys matching this glob pattern capture the information about all the benchmarks found in the executed benchmark suite. The second element of the key is the description of the benchmark, which has to be unique.

  The associated value is irrelevant, and set to the empty string.

{usec * *}
: The set of keys matching this glob pattern capture the performance information, i.e. timing results. The second element of the key is the description of the benchmark, the third element the path of the Tcl interpreter which was used to run it.

  The associated value is either one of several special result codes, or the time it took to execute the benchmark, in microseconds. The possible special result codes are

  ERR
  : Benchmark could not be executed, failed with a Tcl error.

  BAD_RES
  : The benchmark could be executed, however the result from its body did not match the declared expectations.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *bench* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Benchmark tools

Copyright

```
Copyright (c) 2007-2008 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

bench_intro

Keywords: bench_intro, bench language, benchmark, performance, testing

Name

bench_intro - bench introduction

Description

The *bench* (short for *benchmark tools*), is a set of related, yet different, entities which are working together for the easy creation and execution of performance test suites, also known as benchmarks. These are

1. A tcl based language for the declaration of test cases. A test case is represented by a tcl command declaring the various parts needed to execute it, like setup, cleanup, the commands to test, etc.
2. A package providing the ability to execute test cases written in that language.

Which of the more detailed documents are relevant to the reader of this introduction depends on their role in the benchmarking process.

1. A *writer* of benchmarks has to understand the bench language itself. A beginner to bench should read the more informally written *bench language introduction* first. Having digested this the formal *bench language specification* should become understandable. A writer experienced with bench may only need this last document from time to time, to refresh her memory.
2. A *user* of benchmark suites written in the *bench* language has to know which tools are available for use. At the bottom level sits the package **bench**, providing the basic facilities to read and execute files containing benchmarks written in the bench language, and to manipulate benchmark results.

Historical notes

This module and package have been derived from Jeff Hobbs' **tclbench** application for the benchmarking of the Tcl core and its ancestor "*runbench.tcl*".

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *bench* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Benchmark tools

Copyright


```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


bench_lang_intro

Keywords: bench_lang_intro, bench language, benchmark, examples, performance, testing

Name

bench_lang_intro - bench language introduction

Description

This document is an informal introduction to version 1 of the bench language based on a multitude of examples. After reading this a benchmark writer should be ready to understand the formal *bench language specification*.

Fundamentals

In the broadest terms possible the *bench language* is essentially Tcl, plus a number of commands to support the declaration of benchmarks. A document written in this language is a Tcl script and has the same syntax.

Basics

One of the most simplest benchmarks which can be written in bench is

```
bench -desc LABEL -body {
    set a b
}
```

This code declares a benchmark named **LABEL** which measures the time it takes to assign a value to a variable. The Tcl code doing this assignment is the **-body** of the benchmark.

Pre- and postprocessing

Our next example demonstrates how to declare *initialization* and *cleanup* code, i.e. code computing information for the use of the **-body**, and for releasing such resources after the measurement is done. They are the **-pre**- and the **-post**-body, respectively.

In our example, directly drawn from the benchmark suite of Tcllib's **aes** package, the concrete initialization code constructs the key schedule used by the encryption command whose speed we measure, and the cleanup code releases any resources bound to that schedule.

```
bench -desc "AES-${len} ECB encryption core" -pre {
    set key [aes::Init ecb $k $i]
} -body {
    aes::Encrypt $key $p
} -post {
    aes::Final $key
}
```

Advanced pre- and postprocessing

Our last example again deals with initialization and cleanup code. To see the difference to the

regular initialization and cleanup discussed in the last section it is necessary to know a bit more about how bench actually measures the speed of the the **-body**.

Instead of running the **-body** just once the system actually executes the **-body** several hundred times and then returns the average of the found execution times. This is done to remove environmental effects like machine load from the result as much as possible, with outliers canceling each other out in the average.

The drawback of doing things this way is that when we measure operations which are not idempotent we will most likely not measure the time for the operation we want, but of the state(s) the system is in after the first iteration, a mixture of things we have no interest in.

Should we wish, for example, to measure the time it takes to include an element into a set, with the element not yet in the set, and the set having specific properties like being a shared Tcl_Obj, then the first iteration will measure the time for this. *However* all subsequent iterations will measure the time to include an element which is already in the set, and the Tcl_Obj holding the set will not be shared anymore either. In the end the timings taken for the several hundred iterations of this state will overwhelm the time taken from the first iteration, the only one which actually measured what we wanted.

The advanced initialization and cleanup codes, **-ipre**- and the **-ipost**-body respectively, are present to solve this very problem. While the regular initialization and cleanup codes are executed before and after the whole series of iterations the advanced codes are executed before and after each iteration of the body, without being measured themselves. This allows them to bring the system into the exact state the body wishes to measure.

Our example, directly drawn from the benchmark suite of Tcllib's **struct::set** package, is for exactly the example we used above to demonstrate the necessity for the advanced initialization and cleanup. Its concrete initialization code constructs a variable refering to a set with specific properties (The set has a string representation, which is shared) affecting the speed of the inclusion command, and the cleanup code releases the temporary variables created by this initialization.

```
bench -desc "set include, missing <SC> x$times $n" -ipre {
    set A $sx($times,$n)
    set B $A
} -body {
    struct::set include A x
} -ipost {
    unset A B
}
```

Further reading

Now that this document has been digested the reader, assumed to be a *writer* of benchmarks, he should be fortified enough to be able to understand the formal *bench language specfication*. It will also serve as the detailed specification and cheat sheet for all available commands and their syntax.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you

want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *bench* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Benchmark tools

Copyright


```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


bench_lang_spec

Keywords: bench_lang_spec, bench language, benchmark, performance, specification, testing

Name

bench_lang_spec - bench language specification

Synopsis

bench_rm *path...*
bench_tmpfile
bench *options...*

Description

This document specifies both names and syntax of all the commands which together are the bench language, version 1. As this document is intended to be a reference the commands are listed in alphabetical order, and the descriptions are relatively short. A beginner should read the more informally written *bench language introduction* first.

Commands

bench_rm *path*...

This command silently removes the files specified as its arguments and then returns the empty string as its result. The command is *trusted*, there is no checking if the specified files are outside of whatever restricted area the benchmarks are run in.

bench_tmpfile

This command returns the path to a bench specific unique temporary file. The uniqueness means that multiple calls will return different paths. While the path may exist from previous runs, the command itself does *not* create aynthing.

The base location of the temporary files is platform dependent:

Unix, and indeterminate platform

"*/tmp*"

Windows

$TEMP

Anything else
The current working directory.

bench *options*...
This command declares a single benchmark. Its result is the empty string. All parts of the benchmark are declared via options, and their values. The options can occur in any order. The accepted options are:

-body script
The argument of this option declares the body of the benchmark, the Tcl script whose performance we wish to measure. This option, and **-desc**, are the two required parts of each benchmark.

-desc msg
The argument of this option declares the name of the benchmark. It has to be unique, or timing data from different benchmarks will be mixed together.

Beware! This requirement is not checked when benchmarks are executed, and the system will silently produce bogus data. This option, and **-body**, are the two required parts of each benchmark.

-ipost script
The argument of this option declares a script which is run immediately *after* each iteration of the body. Its responsibility is to release resources created by the body, or **-ipre**-bodym which we do not wish to live into the next iteration.

-ipre script
The argument of this option declares a script which is run immediately *before* each iteration of the body. Its responsibility is to create the state of the system expected by the body so that we measure the right thing.

-iterations num
The argument of this option declares the maximum number of times to run the **-body** of the benchmark. During execution this and the global maximum number of iterations are compared and the smaller of the two values is used.

This option should be used only for benchmarks which are expected or known to take a long time per run. I.e. reduce the number of times they are run to keep the overall time for the execution of the whole benchmark within manageable limits.

-post script
The argument of this option declares a script which is run *after* all iterations of the body have been run. Its responsibility is to release resources created by the body, or **-pre**-body.

-pre script
The argument of this option declares a script which is run *before* any of the iterations of the body are run. Its responsibility is to create whatever resources are needed by the body to run without failing.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib

Trackers and specify *bench* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Benchmark tools

Copyright

```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


bench_read
Keywords: bench_read, benchmark, csv, formatting, human readable, parsing, performance, reading, testing, text

Name
bench::in - bench::in - Reading benchmark results

Synopsis
package require Tcl 8.5 9
package require csv
package require bench::in ?**0.2**?
::bench::in::read *file*

Description
This package provides a command for reading benchmark results from files, sockets, etc.

A reader interested in the creation, processing or writing of such results should go and read *bench - Processing benchmark suites* instead.

If the bench language itself is the actual interest please start with the *bench language introduction* and then proceed from there to the formal *bench language specification*.

Public API
::bench::in::read *file*
This command reads a benchmark result from the specified *file* and returns it as its result. The command understands the three formats created by the commands
bench::out::raw
Provided by package **bench**.
bench::out::csv
Provided by package **bench::out::csv**.
bench::out::text
Provided by package **bench::out::text**.

and automatically detects which format is used by the input file.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *bench* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Benchmark tools

Copyright

```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

bench_wcsv

Keywords: bench_wcsv, benchmark, csv, formatting, performance, testing

Name

bench::out::csv - bench::out::csv - Formatting benchmark results as CSV

Synopsis

package require Tcl 8.5 9
package require bench::out::csv ?**0.1.3**?
::bench::out::csv *bench_result*

Description

This package provides commands for fomatting of benchmark results into a CSV table importable by spread sheets.

A reader interested in the generation or processing of such results should go and read *bench - Processing benchmark suites* instead.

If the bench language itself is the actual interest please start with the *bench language introduction* and then proceed from there to the formal *bench language specification*.

Public API

::bench::out::csv *bench_result*

This command formats the specified benchmark result for output to a file, socket, etc. This specific command generates CSV data importable by spread sheets.

For other formatting styles see the packages **bench** and **bench::out::text** which provide commands to format benchmark results in raw form, or for human consumption, respectively.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you

want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *bench* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Benchmark tools

Copyright


```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


bench_wtext

Keywords: bench_wtext, benchmark, formatting, human readable, performance, testing, text

Name

bench::out::text - bench::out::text - Formatting benchmark results as human readable text

Synopsis

package require Tcl 8.5 9
package require bench::out::text ?**0.1.3**?
::bench::out::text *bench_result*

Description

This package provides commands for fomatting of benchmark results into human readable text.

A reader interested in the generation or processing of such results should go and read *bench - Processing benchmark suites* instead.

If the bench language itself is the actual interest please start with the *bench language introduction* and then proceed from there to the formal *bench language specification*.

Public API

::bench::out::text *bench_result*

This command formats the specified benchmark result for output to a file, socket, etc. This specific command generates human readable text.

For other formatting styles see the packages **bench** and **bench::out::csv** which provide commands to format benchmark results in raw form, or as importable CSV data, respectively.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *bench* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Benchmark tools

Copyright

```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


bibtex

bibtex
Keywords: bibtex, bibliography, parsing, text processing

Name
bibtex - Parse bibtex files

Synopsis
package require Tcl 8.5 9
package require bibtex ?**0.8**?
::bibtex::parse ?*options*? ?*text*?
::bibtex::parse *text*
::bibtex::parse ?*-command cmd*? **-channel** *chan*
::bibtex::parse ?*-recordcommand recordcmd*? ?*-preamblecommand preamblecmd*? ?*-stringcommand stringcmd*? ?*-commentcommand commentcmd*? ?*-progresscommand progresscmd*? ?*-casesensitivestrings bool*? (*text* | **-channel** *chan*)
::bibtex::wait *token*
::bibtex::destroy *token*
::bibtex::addStrings *token stringdict*

Description
This package provides commands for the parsing of bibliographies in BibTeX format.

::bibtex::parse ?*options*? ?*text*?
This is the general form of the command for parsing a bibliography. Depending on the options used to invoke it it will either return a token for the parser, or the parsed entries of the input bibliography. Instead of performing an immediate parse returning a predefined format the command can also enter an event-based parsing style where all relevant entries in the input are reported through callback commands, in the style of SAX.

::bibtex::parse *text*
In this form the command will assume that the specified *text* is a bibliography in BibTeX format, parse it, and then return a list containing one element per record found in the bibliography. Note that comments, string definitions, preambles, etc. will not show up in the result. Each element will be a list containing record type, bibliography key and record data, in this order. The record data will be a dictionary, its keys the keys of the record, with the

associated values.

::bibtex::parse ?-command *cmd*? -channel *chan*

In this form the command will reads the bibliography from the specified Tcl channel *chan* and then returns the same data structure as described above.

If however the option **-command** is specified the result will be a handle for the parser instead and all processing will be incremental and happen in the background. When the input has been exhausted the callback *cmd* will be invoked with the result of the parse. The exact definition for the callback is

cmd *token parseresult*

The parse result will have the structure explained above, for the simpler forms of the parser.

Note that the parser will *not* close the channel after it has exhausted it. This is still the responsibility of the user of the parser.

::bibtex::parse ?-recordcommand *recordcmd*? ?-preamblecommand *preamblecmd*? ?-stringcommand *stringcmd*? ?-commentcommand *commentcmd*? ?-progresscommand *progresscmd*? ?-casesensitivestrings *bool*? (*text* | -channel *chan*)

This is the most low-level form for the parser. The returned result will be a handle for the parser. During processing it will invoke the invoke the specified callback commands for each type of data found in the bibliography.

The processing will be incremental and happen in the background if, and only if a Tcl channel *chan* is specified. For a *text* the processing will happen immediately and all callbacks will be invoked before the command itself returns.

The callbacks, i.e. **cmd*, are all command prefixes and will be invoked with additional arguments appended to them. The meaning of the arguments depends on the callback and is explained below. The first argument will however always be the handle of the parser invoking the callback.

-casesensitivestrings

This option takes a boolean value. When set string macro processing becomes case-sensitive. The default is case-insensitive string macro processing.

recordcmd *token type key recorddict*

This callback is invoked whenever the parser detects a bibliography record in the input. Its arguments are the record type, the bibliography key for the record, and a dictionary containing the keys and values describing the record. Any string macros known to the parser have already been expanded.

preamblecmd *token preambletext*

This callback is invoked whenever the parser detects an @preamble block in the input. The only additional argument is the text found in the preamble block. By default such entries are ignored.

stringcmd *token stringdict*

This callback is invoked whenever the parser detects an @string-based macro definition in the input. The argument is a dictionary with the macro names as keys and their replacement strings as values. By default such definitions are added to the parser state for use in future bibliography records.

commentcmd *token commenttext*
This callback is invoked whenever the parser detects a comment in the input. The only additional argument is the comment text. By default such entries are ignored.

progresscmd *token percent*
This callback is invoked during processing to tell the user about the progress which has been made. Its argument is the percentage of data processed, as integer number between **0** and **100**. In the case of incremental processing the perecentage will always be **-1** as the total number of entries is not known beforehand.

::bibtex::wait *token*
This command waits for the parser represented by the *token* to complete and then returns. The returned result is the empty string.

::bibtex::destroy *token*
This command cleans up all internal state associated with the parser represented by the handle *token*, effectively destroying it. This command can be called from within the parser callbacks to terminate processing.

::bibtex::addStrings *token stringdict*
This command adds the macro definitions stored in the dictionary *stringdict* to the parser represented by the handle *token*.

The dictionary keys are the macro names and the values their replacement strings. This command has the correct signature for use as a **-stringcommand** callback in an invokation of the command **::bibtex::parse**.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *bibtex* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Text processing

Copyright

```
Copyright (c) 2005 for documentation, Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

blowfish

blowfish

Keywords: blowfish, block cipher, cryptography, encryption, security

Name

blowfish - Implementation of the Blowfish block cipher

Synopsis

package require Tcl 8.5 9
package require blowfish ?**1.0.6**?
::blowfish::blowfish ?*-mode [ecb|cbc]*? ?*-dir [encrypt|decrypt]*? *-key keydata* ?*-iv vector*? ?*-out channel*? ?*-chunksize size*? ?*-pad padchar*? [*-in channel* | ?--? *data*]
::blowfish::Init *mode keydata iv*
::blowfish::Encrypt *Key data*
::blowfish::Decrypt *Key data*
::blowfish::Reset *Key iv*
::blowfish::Final *Key*

Description

This package is an implementation in Tcl of the Blowfish algorithm developed by Bruce Schneier [1]. Blowfish is a 64-bit block cipher designed to operate quickly on 32 bit architectures and accepting a variable key length. This implementation supports ECB and CBC mode blowfish encryption.

Commands

::blowfish::blowfish ?*-mode [ecb|cbc]*? ?*-dir [encrypt|decrypt]*? *-key keydata* ?*-iv vector*? ?*-out channel*? ?*-chunksize size*? ?*-pad padchar*? [*-in channel* | ?--? *data*]

Perform the **blowfish** algorithm on either the data provided by the argument or on the data read from the *-in* channel. If an *-out* channel is given then the result will be written to this channel.

The *-key* option must be given. This parameter takes a binary string of variable length and is used to generate the **blowfish** key schedule. You should be aware that creating a key schedule is quite an expensive operation in blowfish so it is worth reusing the key where possible. See **Reset**.

The *-mode* and *-dir* options are optional and default to cbc mode and encrypt respectively. The initialization vector *-iv* takes an 8 byte binary argument which defaults to 8 zeros. See Modes of operation for more about available modes and their uses.

Blowfish is a 64-bit block cipher. This means that the data must be provided in units that are a multiple of 8 bytes. The **blowfish** command will by default add nul characters to pad the input data to a multiple of 8 bytes if necessary. The programming api commands will never add padding and instead will raise an error if the input is not a multiple of the block size. The *-pad* option can be used to change the padding character or to disable padding if the empty string is provided as the argument.

Programming interface

::blowfish::Init *mode keydata iv*

Construct a new blowfish key schedule using the specified key data and the given

initialization vector. The initialization vector is not used with ECB mode but is important for CBC mode. See Modes of operation for details about cipher modes.

::blowfish::Encrypt *Key data*
Use a prepared key acquired by calling **Init** to encrypt the provided data. The data argument should be a binary array that is a multiple of the block size of 8 bytes. The result is a binary array the same size as the input of encrypted data.

::blowfish::Decrypt *Key data*
Decipher data using the key. Note that the same key may be used to encrypt and decrypt data provided that the initialization vector is reset appropriately for CBC mode.

::blowfish::Reset *Key iv*
Reset the initialization vector. This permits the programmer to re-use a key and avoid the cost of re-generating the key schedule where the same key data is being used multiple times.

::blowfish::Final *Key*
This should be called to clean up resources associated with *Key*. Once this function has been called the key may not be used again.

Modes of operation

Electronic Code Book (ECB)
ECB is the basic mode of all block ciphers. Each block is encrypted independently and so identical plain text will produce identical output when encrypted with the same key. Any encryption errors will only affect a single block however this is vulnerable to known plaintext attacks.

Cipher Block Chaining (CBC)
CBC mode uses the output of the last block encryption to affect the current block. An initialization vector of the same size as the cipher block size is used to handle the first block. The initialization vector should be chosen randomly and transmitted as the first block of the output. Errors in encryption affect the current block and the next block after which the cipher will correct itself. CBC is the most commonly used mode in software encryption.

Examples

```
% blowfish::blowfish -hex -mode ecb -dir encrypt -key secret01 "hello,
    world!"
d0d8f27e7a374b9e2dbd9938dd04195a
```

```
set Key [blowfish::Init cbc $eight_bytes_key_data $eight_byte_iv]
append ciphertext [blowfish::Encrypt $Key $plaintext]
append ciphertext [blowfish::Encrypt $Key $additional_plaintext]
blowfish::Final $Key
```

References

1. Schneier, B. "Applied Cryptography, 2nd edition", 1996, ISBN 0-471-11709-9, pub. John Wiley & Sons.

Authors

Frank Pilhofer, Pat Thoyts

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *blowfish* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Hashes, checksums, and encryption

Copyright

```
Copyright (c) 2003, Pat Thoyts <patthoyts@users.sourceforge.net>
```

cache

async

Keywords: async, asynchronous, cache, callback, synchronous

Name

cache::async - Asynchronous in-memory cache

Synopsis

package require Tcl 8.5 9
package require cache::async ?**0.3.2**?
::cache::async *objectName commandprefix* ?*options...*?
objectName **get** *key donecmdprefix*
objectName **set** *key value*
objectName **unset** *key*
objectName **exists** *key*
objectName **clear** ?*key*?

Description

This package provides objects which cache data in memory, and operate asynchronously with regard to request and responses. The objects are agnostic with regard to cache keys and values, and unknown methods are delegated to the provider of cached data. These two properties make it easy to use caches as a facade for any data provider.

API

The package exports a class, **cache::async**, as specified below.

::cache::async *objectName commandprefix ?options...?*

The command creates a new *cache* object with an associated global Tcl command whose name is *objectName*. This command may be used to invoke various operations on the object.

The *commandprefix* is the action to perform when an user asks for data in the cache and the

cache doesn't yet know about the key. When run the commandprefix is given three additional arguments, the string **get**, the key requested, and the cache object itself, in the form of its object command, in this order. The execution of the action is done in an idle-handler, decoupling it from the original request.

The only supported option is

-full-async-results

This option defines the behaviour of the cache for when requested keys are known to the cache at the time of **get** request. By default such requeste are responded to asynchronously as well. Setting this option to **false** forces the cache to respond to them synchronuously, although still through the specified callback.

The object commands created by the class commands above have the form:

objectName* get *key donecmdprefix

This method requests the data for the *key* from the cache. If the data is not yet known the command prefix specified during construction of the cache object is used to ask for this information.

Whenever the information is/becomes available the *donecmdprefix* will be run to transfer the result to the caller. This command prefix is invoked with either 2 or 3 arguments, i.e.
1. The string **set**, the *key*, and the value.
2. The string **unset**, and the *key*.

These two possibilities are used to either signal the value for the *key*, or that the *key* has no value defined for it. The latter is distinct from the cache not knowing about the *key*.

For a cache object configured to be fully asynchronous (default) the *donecmdprefix* is always run in an idle-handler, decoupling it from the request. Otherwise the callback will be invoked synchronously when the *key* is known to the cache at the time of the invokation.

Another important part of the cache's behaviour, as it is asynchronous it is possible that multiple **get** requests are issued for the same *key* before it can respond. In that case the cache will issue only one data request to the provider, for the first of these, and suspend the others, and then notify all of them when the data becomes available.

objectName* set *key value

see below ...

objectName* unset *key

These two methods are provided to allow users of the cache to make keys known to the cache, as either having a *value*, or as undefined.

It is expected that the data provider (see *commandprefix* of the constructor) uses them in response to data requests for unknown keys.

Note how this matches the cache's own API towards its caller, calling the *donecmd* of **get** -requests issued to itself with either "set key value" or "unset key", versus issuing **get** -requests to its own provider with itself in the place of the *donecmd*, expecting to be called with either "set key value" or "unset key".

This also means that these methods invoke the *donecmd* of all **get**-requests waiting for

information about the modified *key*.

objectName* exists *key
This method queries the cache for knowledge about the *key* and returns a boolean value. The result is **true** if the key is known, and **false** otherwise.

***objectName* clear ?*key*?**
This method resets the state of either the specified *key* or of all keys known to the cache, making it unkown. This forces future **get**-requests to reload the information from the provider.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *cache* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Copyright


```
Copyright (c) 2008 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


clay

clay

Keywords: clay, TclOO, oo

Name

clay - A minimalist framework for large scale OO Projects

Synopsis

package require Tcl 8.6 9
package require uuid
package require oo::dialect

proc **clay::PROC** *name arglist body* ?*ninja* ?

proc **clay::_ancestors** *resultvar class*

proc **clay::ancestors** ?*args*?

proc **clay::args_to_dict** ?*args*?

proc **clay::args_to_options** ?*args*?

proc **clay::dynamic_arguments** *ensemble method arglist* ?*args*?

proc **clay::dynamic_wrongargs_message** *arglist*

proc **clay::is_dict** *d*

proc **clay::is_null** *value*

proc **clay::leaf** ?*args*?
proc **clay::K** *a b*
proc **clay::noop** ?*args*?
proc **clay::cleanup**
proc **clay::object_create** *objname* ?*class* ?
proc **clay::object_rename** *object newname*
proc **clay::object_destroy** ?*args*?
proc **clay::path** ?*args*?
proc **clay::putb** ?*map*? *text*
proc **clay::script_path**
proc **clay::NSNormalize** *qualname*
proc **clay::uuid_generate** ?*args*?
proc **clay::uuid::generate_tcl_machinfo**
proc **clay::uuid::tostring** *uuid*
proc **clay::uuid::fromstring** *uuid*
proc **clay::uuid::equal** *left right*
proc **clay::uuid** *cmd* ?*args*?
proc **clay::tree::sanitize** *dict*
proc **clay::tree::_sanitizeb** *path varname dict*
proc **clay::tree::storage** *rawpath*
proc **clay::tree::dictset** *varname* ?*args*?
proc **clay::tree::dictmerge** *varname* ?*args*?
proc **clay::tree::merge** ?*args*?
proc **dictargs::proc** *name argspec body*
proc **dictargs::method** *name argspec body*
proc **clay::dialect::Push** *class*
proc **clay::dialect::Peek**
proc **clay::dialect::Pop**
proc **clay::dialect::create** *name* ?*parent* ?
proc **clay::dialect::NSNormalize** *namespace qualname*
proc **clay::dialect::DefineThunk** *target* ?*args*?
proc **clay::dialect::Canonical** *namespace NSpace class*
proc **clay::dialect::Define** *namespace class* ?*args*?
proc **clay::dialect::Aliases** *namespace* ?*args*?
proc **clay::dialect::SuperClass** *namespace* ?*args*?
proc **clay::dynamic_methods** *class*
proc **clay::dynamic_methods_class** *thisclass*

proc **clay::define::Array** *name* ?*values* ?
proc **clay::define::Delegate** *name info*
proc **clay::define::constructor** *arglist rawbody*
proc **clay::define::Class_Method** *name arglist body*
proc **clay::define::class_method** *name arglist body*
proc **clay::define::clay** ?*args*?
proc **clay::define::destructor** *rawbody*
proc **clay::define::Dict** *name* ?*values* ?
proc **clay::define::Option** *name* ?*args*?
proc **clay::define::Method** *name argstyle argspec body*
proc **clay::define::Option_Class** *name* ?*args*?
proc **clay::define::Variable** *name* ?*default* ?
proc **clay::ensemble_methodbody** *ensemble einfo*
proc **clay::define::Ensemble** *rawmethod* ?*args*?
proc **clay::event::cancel** *self* ?*task* *****?
proc **clay::event::generate** *self event* ?*args*?
proc **clay::event::nextid**
proc **clay::event::Notification_list** *self event* ?*stackvar* ?
proc **clay::event::notify** *rcpt sender event eventinfo*
proc **clay::event::process** *self handle script*
proc **clay::event::schedule** *self handle interval script*
proc **clay::event::subscribe** *self who event*
proc **clay::event::unsubscribe** *self* ?*args*?
proc **clay::singleton** *name script*
method **clay ancestors**
method **clay dump**
method **clay find** *path* ?**path...**?
method **clay get** *path* ?**path...**?
method **clay GET** *path* ?**path...**?
method **clay merge** *dict* ?**dict...**?
method **clay replace** *dictionary*
method **clay search** *path* ?**path...**?
method **clay set** *path* ?**path...**? *value*
method **clay ancestors**
method **clay cache** *path value*
method **clay cget** *field*
method **clay delegate** ?*stub*? ?*object*?

method **clay dump**
method **clay ensemble_map**
method **clay eval** *script*
method **clay evolve**
method **clay exists** *path* ?**path...**?
method **clay flush**
method **clay forward** *method object*
method **clay get** *path* ?**path...**?
method **clay leaf** *path* ?**path...**?
method **clay merge** *dict* ?**dict...**?
method **clay mixin** *class* ?**class...**?
method **clay mixinmap** ?*stub*? ?*classes*?
method **clay provenance** *path* ?**path...**?
method **clay replace** *dictionary*
method **clay search** *path valuevar isleafvar*
method **clay source** *filename*
method **clay set** *path* ?**path...**? *value*
method **InitializePublic**

Description

Clay introduces a method ensemble to both **oo::class** and **oo::object** called clay. This ensemble handles all of the high level interactions within the framework. Clay stores structured data. Clan manages method delegation. Clay has facilities to manage the complex interactions that come about with mixins.

The central concept is that inside of every object and class (which are actually objects too) is a dict called clay. What is stored in that dict is left to the imagination. But because this dict is exposed via a public method, we can share structured data between object, classes, and mixins.

Structured data

Clay uses a standardized set of method interactions and introspection that TclOO already provides to perform on-the-fly searches. On-the-fly searches mean that the data is never stale, and we avoid many of the sorts of collisions that would arise when objects start mixing in other classes during operation.

The **clay** methods for both classes and objects have a get and a set method. For objects, get will search through the local clay dict. If the requested leaf is not found, or the query is for a branch, the system will then begin to poll the clay methods of all of the class that implements the object, all of that classes' ancestors, as well as all of the classes that have been mixed into this object, and all of their ancestors.

Intended branches on a tree end with a directory slash (/). Intended leaves are left unadorned. This is a guide for the tool that builds the search results to know what parts of a dict are intended to be branches and which are intended to be leaves. For simple cases, branch marking

can be ignored:

```
::oo::class create ::foo { }
::foo clay set property/ color blue
::foo clay set property/ shape round

set A [::foo new]
$A clay get property/
{color blue shape round}

$A clay set property/ shape square
$A clay get property/
{color blue shape square}
```

But when you start storing blocks of text, guessing what field is a dict and what isn't gets messy:

```
::foo clay set description {A generic thing of designated color and
    shape}

$A clay get description
{A generic thing of designated color and shape}

Without a convention for discerning branches for leaves what should
    have been a value can be accidentally parsed as a dictionary, and
    merged with all of the other values that were never intended to be
    merge. Here is an example of it all going wrong:
::oo::class create ::foo { }
# Add description as a leaf
::foo clay set description  {A generic thing of designated color and
    shape}
# Add description as a branch
::foo clay set description/  {A generic thing of designated color and
    shape}

::oo::class create ::bar {
  superclass foo
}
# Add description as a leaf
::bar clay set description  {A drinking establishment of designated
    color and shape and size}
# Add description as a branch
::bar clay set description/  {A drinking establishment of designated
    color and shape and size}

set B [::bar new]
# As a leaf we get the value verbatim from he nearest ancestor
$B clay get description
```

```
  {A drinking establishment of designated color and shape and size}
# As a branch we get a recursive merge
$B clay get description/
{A drinking establishment of designated color and size thing of}
```

Clay dialect

Clay is built using the oo::dialect module from Tcllib. oo::dialect allows you to either add keywords directly to clay, or to create your own metaclass and keyword set using Clay as a foundation. For details on the keywords and what they do, consult the functions in the ::clay::define namespace.

Method delegation

Method Delegation It is sometimes useful to have an external object that can be invoked as if it were a method of the object. Clay provides a delegate ensemble method to perform that delegation, as well as introspect which methods are delegated in that manner. All delegated methods are marked with html-like tag markings (< >) around them.

```
::clay::define counter {
  Variable counter 0
  method incr {{howmuch 1}} {
    my variable counter
    incr counter $howmuch
  }
  method value {} {
    my variable counter
    return $counter
  }
  method reset {} {
    my variable counter
    set counter 0
  }
}
::clay::define example {
  variable buffer
  constructor {} {
    # Build a counter object
    set obj [namespace current]::counter
    ::counter create $obj
    # Delegate the counter
    my delegate <counter> $obj
  }
  method line {text} {
    my <counter> incr
    append buffer $text
  }
}
```

```
set A [example new]
$A line {Who's line is it anyway?}
$A <counter> value
1
```

Commands

proc clay::PROC *name arglist body* ?*ninja* ?
Because many features in this package may be added as commands to future tcl cores, or be provided in binary form by packages, I need a declaritive way of saying *Create this command if there isn't one already*. The *ninja* argument is a script to execute if the command is created by this mechanism.

proc clay::_ancestors *resultvar class*
see below ...

proc clay::ancestors ?*args*?
see below ...

proc clay::args_to_dict ?*args*?
see below ...

proc clay::args_to_options ?*args*?
see below ...

proc clay::dynamic_arguments *ensemble method arglist* ?*args*?
see below ...

proc clay::dynamic_wrongargs_message *arglist*
see below ...

proc clay::is_dict *d*
see below ...

proc clay::is_null *value*
see below ...

proc clay::leaf ?*args*?
see below ...

proc clay::K *a b*
see below ...

proc clay::noop ?*args*?
Perform a noop. Useful in prototyping for commenting out blocks of code without actually having to comment them out. It also makes a handy default for method delegation if a delegate has not been assigned yet.

proc clay::cleanup
Process the queue of objects to be destroyed

proc clay::object_create *objname* ?*class* ?
see below ...

proc clay::object_rename *object newname*
see below ...

proc clay::object_destroy ?*args*?
Mark an objects for destruction on the next cleanup

proc clay::path ?*args*?
see below ...

proc clay::putb ?*map*? *text*
Append a line of text to a variable. Optionally apply a string mapping.

proc clay::script_path
see below ...

proc clay::NSNormalize *qualname*
see below ...

proc clay::uuid_generate ?*args*?
see below ...

proc clay::uuid::generate_tcl_machinfo
see below ...

proc clay::uuid::tostring *uuid*
see below ...

proc clay::uuid::fromstring *uuid*
Convert a string representation of a uuid into its binary format.

proc clay::uuid::equal *left right*
Compare two uuids for equality.

proc clay::uuid *cmd* ?*args*?
uuid generate -> string rep of a new uuid uuid equal uuid1 uuid2

proc clay::tree::sanitize *dict*
Output a dictionary removing any. entries added by **clay::tree::merge**

proc clay::tree::_sanitizeb *path varname dict*
Helper function for ::clay::tree::sanitize Formats the string representation for a dictionary element within a human readable stream of lines, and determines if it needs to call itself with further indentation to express a sub-branch

proc clay::tree::storage *rawpath*
Return the path as a storage path for clay::tree with all branch terminators removed. This command will also break arguments up if they contain /.

Example:

```
> clay::tree::storage {foo bar baz bang}
foo bar baz bang
> clay::tree::storage {foo bar baz bang/}
foo bar baz bang
> clay::tree::storage {foo bar baz bang:}
foo bar baz bang:
> clay::tree::storage {foo/bar/baz bang:}
foo bar baz bang:
> clay::tree::storage {foo/bar/baz/bang}
foo bar baz bang
```

proc clay::tree::dictset *varname* ?*args*?
Set an element with a recursive dictionary, marking all branches on the way down to the final element. If the value does not exists in the nested dictionary it is added as a leaf. If the value

already exists as a branch the value given is merged if the value is a valid dict. If the incoming value is not a valid dict, the value overrides the value stored, and the value is treated as a leaf from then on.

Example:

```
> set r {}
> ::clay::tree::dictset r option color default Green
. {} option {. {} color {. {} default Green}}
> ::clay::tree::dictset r option {Something not dictlike}
. {} option {Something not dictlike}
# Note that if the value is not a dict, and you try to force it to
   be
# an error with be thrown on the merge
> ::clay::tree::dictset r option color default Blue
missing value to go with key
```

proc clay::tree::dictmerge *varname* ?*args*?

A recursive form of dict merge, intended for modifying variables in place.

Example:

```
> set mydict {sub/ {sub/ {description {a block of text}}}}
> ::clay::tree::dictmerge mydict {sub/ {sub/ {field {another block
   of text}}}}]
> clay::tree::print $mydict
sub/ {
  sub/ {
    description {a block of text}
    field {another block of text}
  }
}
```

proc clay::tree::merge ?*args*?

A recursive form of dict merge

A routine to recursively dig through dicts and merge adapted from http://stevehavelka.com/tcl-dict-operation-nested-merge/

Example:

```
> set mydict {sub/ {sub/ {description {a block of text}}}}
> set odict [clay::tree::merge $mydict {sub/ {sub/ {field {another
   block of text}}}}]
> clay::tree::print $odict
sub/ {
  sub/ {
    description {a block of text}
    field {another block of text}
  }
}
```

proc dictargs::proc *name argspec body*
Named Procedures as new command

proc dictargs::method *name argspec body*
see below ...

proc clay::dialect::Push *class*
see below ...

proc clay::dialect::Peek
see below ...

proc clay::dialect::Pop
see below ...

proc clay::dialect::create *name* ?*parent* ?
This proc will generate a namespace, a "mother of all classes", and a rudimentary set of policies for this dialect.

proc clay::dialect::NSNormalize *namespace qualname*
Support commands; not intended to be called directly.

proc clay::dialect::DefineThunk *target* ?*args*?
see below ...

proc clay::dialect::Canonical *namespace NSpace class*
see below ...

proc clay::dialect::Define *namespace class* ?*args*?
Implementation of the languages' define command

proc clay::dialect::Aliases *namespace* ?*args*?
see below ...

proc clay::dialect::SuperClass *namespace* ?*args*?
see below ...

proc clay::dynamic_methods *class*
see below ...

proc clay::dynamic_methods_class *thisclass*
see below ...

proc clay::define::Array *name* ?*values* ?
New OO Keywords for clay

proc clay::define::Delegate *name info*
An annotation that objects of this class interact with delegated methods. The annotation is

intended to be a dictionary, and the only reserved key is *description*, a human readable description.

proc clay::define::constructor *arglist rawbody*
see below ...

proc clay::define::Class_Method *name arglist body*
Specify the a method for the class object itself, instead of for objects of the class

proc clay::define::class_method *name arglist body*
And alias to the new Class_Method keyword

proc clay::define::clay ?*args*?
see below ...

proc clay::define::destructor *rawbody*
see below ...

proc clay::define::Dict *name* ?*values* ?
see below ...

proc clay::define::Option *name* ?*args*?
Define an option for the class

proc clay::define::Method *name argstyle argspec body*
see below ...

proc clay::define::Option_Class *name* ?*args*?
Define a class of options All field / value pairs will be be inherited by an option that specify *name* as it class field.

proc clay::define::Variable *name* ?*default* ?
This keyword can also be expressed:

```
property variable NAME {default DEFAULT}
```

Variables registered in the variable property are also initialized (if missing) when the object changes class via the *morph* method.

proc clay::ensemble_methodbody *ensemble einfo*
Produce the body of an ensemble's public dispatch method ensemble is the name of the the ensemble. einfo is a dictionary of methods for the ensemble, and each value is a script to execute on dispatch

Example:

```
::clay::ensemble_methodbody foo {
  bar {tailcall my Foo_bar {*}$args}
  baz {tailcall my Foo_baz {*}$args}
  clock {return [clock seconds]}
  default {puts "You gave me $method"}
}
```

proc clay::define::Ensemble *rawmethod* ?*args*?
see below ...

proc clay::event::cancel *self* ?*task* *?
Cancel a scheduled event

proc clay::event::generate *self event* ?*args*?
Generate an event Adds a subscription mechanism for objects to see who has recieved this event and prevent spamming or infinite recursion

proc clay::event::nextid
see below ...

proc clay::event::Notification_list *self event* ?*stackvar* ?
Called recursively to produce a list of who recieves notifications

proc clay::event::notify *rcpt sender event eventinfo*
Final delivery to intended recipient object

proc clay::event::process *self handle script*
Evaluate an event script in the global namespace

proc clay::event::schedule *self handle interval script*
Schedule an event to occur later

proc clay::event::subscribe *self who event*
Subscribe an object to an event pattern

proc clay::event::unsubscribe *self* ?*args*?
Unsubscribe an object from an event pattern

proc clay::singleton *name script*
An object which is intended to be it's own class.

Classes

Class clay::class

Methods

method clay ancestors
Return this class and all ancestors in search order.

method clay dump
Return a complete dump of this object's clay data, but only this object's clay data.

method clay find *path* ?path...?
Pull a chunk of data from the clay system. If the last element of *path* is a branch, returns a recursive merge of all data from this object and it's constituent classes of the data in that branch. If the last element is a leaf, search this object for a matching leaf, or search all constituent classes for a matching leaf and return the first value found. If no value is found, returns an empty string. If a branch is returned the topmost. entry is omitted.

method clay get *path* ?path...?
Pull a chunk of data from the class's clay system. If no value is found, returns an empty string. If a branch is returned the topmost. entry is omitted.

method clay GET *path* ?path...?
Pull a chunk of data from the class's clay system. If no value is found, returns an empty string.

method clay merge *dict* ?dict...?
Recursively merge the dictionaries given into the object's local clay storage.

method clay replace *dictionary*
Replace the contents of the internal clay storage with the dictionary given.

method clay search *path* ?path...?

Return the first matching value for the path in either this class's clay data or one of its ancestors

method clay set *path* ?path...? *value*

Merge the conents of **value** with the object's clay storage at **path**.

Class clay::object

clay::object This class is inherited by all classes that have options.

Methods

method clay ancestors

Return the class this object belongs to, all classes mixed into this object, and all ancestors of those classes in search order.

method clay cache *path value*

Store VALUE in such a way that request in SEARCH for PATH will always return it until the cache is flushed

method clay cget *field*

Pull a value from either the object's clay structure or one of its constituent classes that matches the field name. The order of search us:

1. The as a value in local dict variable config
2. The as a value in local dict variable clay
3. As a leaf in any ancestor as a root of the clay tree
4. As a leaf in any ancestor as **const** *field*
5. As a leaf in any ancestor as **option** *field* **default**

method clay delegate ?*stub*? ?*object*?

Introspect or control method delegation. With no arguments, the method will return a key/value list of stubs and objects. With just the *stub* argument, the method will return the object (if any) attached to the stub. With a *stub* and an *object* this command will forward all calls to the method *stub* to the *object*.

method clay dump

Return a complete dump of this object's clay data, as well as the data from all constituent classes recursively blended in.

method clay ensemble_map

Return a dictionary describing the method ensembles to be assembled for this object

method clay eval *script*

Evaluated a script in the namespace of this object

method clay evolve

Trigger the **InitializePublic** private method

method clay exists *path* ?path...?

Returns 1 if *path* exists in either the object's clay data. Values greater than one indicate the element exists in one of the object's constituent classes. A value of zero indicates the path could not be found.

method clay flush

Wipe any caches built by the clay implementation

method clay forward *method object*

A convenience wrapper for

```
oo::objdefine [self] forward {*}$args
```

method clay get *path* ?path...?
Pull a chunk of data from the clay system. If the last element of *path* is a branch (ends in a slash /), returns a recursive merge of all data from this object and it's constituent classes of the data in that branch. If the last element is a leaf, search this object for a matching leaf, or search all constituent classes for a matching leaf and return the first value found. If no value is found, returns an empty string.

method clay leaf *path* ?path...?
A modified get which is tailored to pull only leaf elements

method clay merge *dict* ?dict...?
Recursively merge the dictionaries given into the object's local clay storage.

method clay mixin *class* ?class...?
Perform [oo::objdefine [self] mixin] on this object, with a few additional rules: Prior to the call, for any class was previously mixed in, but not in the new result, execute the script registered to mixin/ unmap-script (if given.) For all new classes, that were not present prior to this call, after the native TclOO mixin is invoked, execute the script registered to mixin/ map-script (if given.) Fall all classes that are now present and "mixed in", execute the script registered to mixin/ react-script (if given.)

method clay mixinmap ?*stub*? ?*classes*?
With no arguments returns the map of stubs and classes mixed into the current object. When only stub is given, returns the classes mixed in on that stub. When stub and classlist given, replace the classes currently on that stub with the given classes and invoke clay mixin on the new matrix of mixed in classes.

method clay provenance *path* ?path...?
Return either **self** if that path exists in the current object, or return the first class (if any) along the clay search path which contains that element.

method clay replace *dictionary*
Replace the contents of the internal clay storage with the dictionary given.

method clay search *path valuevar isleafvar*
Return true, and set valuevar to the value and isleafar to true for false if PATH was found in the cache.

method clay source *filename*
Source the given filename within the object's namespace

method clay set *path* ?path...? *value*
Merge the conents of **value** with the object's clay storage at **path**.

method InitializePublic
Instantiate variables. Called on object creation and during clay mixin.

Authors

Sean Woods *mailto:*yoda@etoyoc.com

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib

Trackers and specify *oo* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Programming tools

Copyright
```
Copyright (c) 2018 Sean Woods <yoda@etoyoc.com>
```

clock

iso8601
Keywords: iso8601

Name
clock_iso8601 - Parsing ISO 8601 dates/times

Synopsis
package require Tcl 8.5 9
package require clock::iso8601 ?**0.2**?
::clock::iso8601 parse_date *date options...*
::clock::iso8601 parse_time *time options...*

Description
This package provides functionality to parse dates and times in ISO 8601 format.

::clock::iso8601 parse_date *date options...*
This command parses an ISO8601 date string in an unknown variant and returns the given date/time in seconds since epoch.

The acceptable options are **-base**, **-gmt**, **-locale**, and **-timezone** of the builtin command **clock scan**.

::clock::iso8601 parse_time *time options...*
This command parses a full ISO8601 timestamp string (date and time) in an unknown variant and returns the given time in seconds since epoch.

The acceptable options are **-base**, **-gmt**, **-locale**, and **-timezone** of the builtin command **clock scan**.

Date formats
The commands accept the following 23 date formats:

```
(year)-(month)-(day)
(year)(month)(day)
(year)-(day in year)
(year)(day in year)
(year in century)-(month)-(day)
(year)-(month)          Day defaults to the 1st of the month
(year in century)(month)(day)
(year in century)-(day in year)
(year in century)(day in year)
--(month)-(day)         Year defaults to the current year
--(month)(day)          Year defaults to the current year
--(day in year)         Year defaults to the current year
---(day)                Year defaults to the current year, month to
    current month
(fiscal year)-W(week)-(wday)
(fiscal year)W(week)-(wday)
(fiscal year in century)-W(week)-(wday)
(fiscal year in century)W(week)(wday)
(fiscal year)-W(week)       Weekday defaults to monday
(fiscal year)W(week)        Weekday defaults to monday
-W(week)-(wday)         Year defaults to current fiscal year
-W(week)(wday)          Year defaults to current fiscal year
(wday)              Year defaults to current fiscal year, week to current week
(year)              Month defaults to january, day to 1st of the month
```

The possible parts/fields in the above, and their meanings, are:

year
: Year with century, 4 digits

month
: Month in year, 2 digits

day
: Day in month, 2 digits.

year in century
: Year without century, 2 digits

day in year
: Day in year, 3 digits

fiscal year
: ISO 8601 fiscal year with century, 4 digits

fiscal year in century
: ISO 8601 fiscal year without century, 2 digits

week
: ISO 8601 week number

wday
: Week day, 1 digit, Monday (1) to Sunday (7,0)

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *clock::iso8601* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Text processing

rfc2822

Keywords: rfc2822

Name

clock_rfc2822 - Parsing RFC 2822 dates/times

Synopsis

package require Tcl 8.5 9
package require clock::rfc2822 ?**0.2**?
::clock::rfc2822 parse_date *date*

Description

This package provides functionality to parse dates in RFC 2822 format.

::clock::rfc2822 parse_date *date*
: This command parses an RFC2822 date string and returns the given date in seconds since epoch. An error is thrown if the command is unable to parse the date.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *clock::rfc2822* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Text processing

cmdline

cmdline

Keywords: cmdline, argument processing, argv, argv0, cmdline processing, command line

processing

Name
cmdline - Procedures to process command lines and options.

Synopsis
package require Tcl 8.5 9
package require cmdline ?**1.5.3**?
::cmdline::getopt *argvVar optstring optVar valVar*
::cmdline::getKnownOpt *argvVar optstring optVar valVar*
::cmdline::getoptions *argvVar optlist* ?*usage*?
::cmdline::getKnownOptions *argvVar optlist* ?*usage*?
::cmdline::usage *optlist* ?*usage*?
::cmdline::getfiles *patterns quiet*
::cmdline::getArgv0

Description
This package provides commands to parse command lines and options.

::argv handling
One of the most common variables this package will be used with is **::argv**, which holds the command line of the current application. This variable has a companion **::argc** which is initialized to the number of elements in **::argv** at the beginning of the application.

The commands in this package will *not* modify the **::argc** companion when called with **::argv**. Keeping the value consistent, if such is desired or required, is the responsibility of the caller.

API
::cmdline::getopt *argvVar optstring optVar valVar*
This command works in a fashion like the standard C based **getopt** function. Given an option string and a pointer to an array of args this command will process the *first argument* and return info on how to proceed. The command returns 1 if an option was found, 0 if no more options were found, and -1 if an error occurred.

argvVar contains the name of the list of arguments to process. If options are found the list is modified and the processed arguments are removed from the start of the list.

optstring contains a list of command options that the application will accept. If the option ends in ".arg" the command will use the next argument as an argument to the option, or extract it from the current argument, if it is of the form "option=value". Otherwise the option is a boolean that is set to 1 if present.

optVar refers to the variable the command will store the found option into (without the leading '-' and without the .arg extension).

valVar refers to the variable to store either the value for the specified option into upon success or an error message in the case of failure. The stored value comes from the command line for .arg options, otherwise the value is 1.

::cmdline::getKnownOpt *argvVar optstring optVar valVar*
Like **::cmdline::getopt**, except it ignores any unknown options in the input.

::cmdline::getoptions *argvVar optlist* ?*usage*?
Processes the entire set of command line options found in the list variable named by *argvVar* and fills in defaults for those not specified. This also generates an error message that lists the allowed flags if an incorrect flag is specified. The optional *usage*-argument contains a string to include in front of the generated message. If not present it defaults to "options:".

argvVar contains the name of the list of arguments to process. If options are found the list is modified and the processed arguments are removed from the start of the list.

optlist contains a list of lists where each element specifies an option in the form: *flag default comment*.

If *flag* ends in ".arg" then the value is taken from the command line. Otherwise it is a boolean and appears in the result if present on the command line. If *flag* ends in ".secret", it will not be displayed in the usage.

The options **-?**, **-help**, and **--** are implicitly understood. The first two abort option processing by throwing an error and force the generation of the usage message, whereas the the last aborts option processing without an error, leaving all arguments coming after for regular processing, even if starting with a dash.

The result of the command is a dictionary mapping all options to their values, be they user-specified or defaults.

::cmdline::getKnownOptions *argvVar optlist* ?*usage*?
Like **::cmdline::getoptions**, but ignores any unknown options in the input.

::cmdline::usage *optlist* ?*usage*?
Generates and returns an error message that lists the allowed flags. *optlist* is defined as for **::cmdline::getoptions**. The optional *usage*-argument contains a string to include in front of the generated message. If not present it defaults to "options:".

::cmdline::getfiles *patterns quiet*
Given a list of file *patterns* this command computes the set of valid files. On windows, file globbing is performed on each argument. On Unix, only file existence is tested. If a file argument produces no valid files, a warning is optionally generated (set *quiet* to true).

This code also uses the full path for each file. If not given it prepends the current working directory to the filename. This ensures that these files will never conflict with files in a wrapped zip file. The last sentence refers to the pro-tools.

::cmdline::getArgv0
This command returns the "sanitized" version of *argv0*. It will strip off the leading path and removes the extension ".bin". The latter is used by the TclPro applications because they must be wrapped by a shell script.

Error codes

Starting with version 1.5 all errors thrown by the package have a proper **::errorCode** for use

with Tcl's **try** command. This code always has the word **CMDLINE** as its first element.

Examples

Cmdline::getoptions

This example, taken from the package **fileutil** and slightly modified, demonstrates how to use **cmdline::getoptions**. First, a list of options is created, then the 'args' list is passed to cmdline for processing. Subsequently, different options are checked to see if they have been passed to the script, and what their value is.

```
        package require Tcl 8.5
        package require try         ;# Tcllib.
        package require cmdline 1.5 ;# First version with proper
    error-codes.

        # Notes:
        # - Tcl 8.6+ has 'try' as a builtin command and therefore does
    not
        #   need the 'try' package.
        # - Before Tcl 8.5 we cannot support 'try' and have to use
    'catch'.
        #   This then requires a dedicated test (if) on the contents of
        #   ::errorCode to separate the CMDLINE USAGE signal from
    actual errors.

        set options {
            {a          "set the atime only"}
            {m          "set the mtime only"}
            {c          "do not create non-existent files"}
            {r.arg  ""  "use time from ref_file"}
            {t.arg  -1  "use specified time"}
        }
        set usage ": MyCommandName \[options] filename ...\noptions:"

        try {
            array set params [::cmdline::getoptions argv $options
    $usage]

        # Note: argv is modified now. The recognized options are
        # removed from it, leaving the non-option arguments behind.
        } trap {CMDLINE USAGE} {msg o} {
            # Trap the usage signal, print the message, and exit the
    application.
            # Note: Other errors are not caught and passed through to
    higher levels!
        puts $msg
        exit 1
```

```
        }

        if {  $params(a) } { set set_atime "true" }
        set has_t [expr {$params(t) != -1}]
        set has_r [expr {[string length $params(r)] > 0}]
        if {$has_t && $has_r} {
            return -code error "Cannot specify both -r and -t"
        } elseif {$has_t} {
        ...
        }
```

Cmdline::getopt

This example shows the core loop of **cmdline::getoptions** from the previous example. It demonstrates how it uses **cmdline::get** to process the options one at a time.

```
    while {[set err [getopt argv $opts opt arg]]} {
    if {$err < 0} {
            set result(?) ""
            break
    }
    set result($opt) $arg
    }
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *cmdline* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Programming tools

comm

comm

Keywords: comm, communication, ipc, message, remote communication, remote execution, rpc, secure, send, socket, ssl, tls

Name

comm - A remote communication facility for Tcl (8.5 and later)

Synopsis

package require Tcl 8.5 9

package require comm ?**4.7.3**?
::comm::comm send ?*-async*? ?*-command callback*? *id cmd* ?*arg arg* ...?
::comm::comm self
::comm::comm interps
::comm::comm connect ?*id*?
::comm::comm new *chan* ?*name value* ...?
::comm::comm channels
::comm::comm config
::comm::comm config *name*
::comm::comm config ?*name value* ...?
::comm::comm shutdown *id*
::comm::comm abort
::comm::comm destroy
::comm::comm hook *event* ?**+**? ?*script*?
::comm::comm remoteid
::comm::comm_send
::comm::comm return_async
$future return ?*-code code*? ?*value*?
$future configure ?*-command cmdprefix*?
$future cget -command

Description

The **comm** command provides an inter-interpreter remote execution facility much like Tk's **send(n)**, except that it uses sockets rather than the X server for the communication path. As a result, **comm** works with multiple interpreters, works on Windows and Macintosh systems, and provides control over the remote execution path.

These commands work just like **send** and **winfo interps**:

```
    ::comm::comm send ?-async? id cmd ?arg arg ...?
    ::comm::comm interps
```

This is all that is really needed to know in order to use **comm**

Commands

The package initializes **::comm::comm** as the default *chan*.

comm names communication endpoints with an *id* unique to each machine. Before sending commands, the *id* of another interpreter is needed. Unlike Tk's send, **comm** doesn't implicitly know the *id*'s of all the interpreters on the system. The following four methods make up the basic **comm** interface.

::comm::comm send ?-async? ?-command *callback*? *id cmd* ?*arg arg ...*?
: This invokes the given command in the interpreter named by *id*. The command waits for the result and remote errors are returned unless the **-async** or **-command** option is given. If **-async** is given, send returns immediately and there is no further notification of result. If **-command** is used, *callback* specifies a command to invoke when the result is received. These options are mutually exclusive. The callback will receive arguments in the form *-option value*, suitable for **array set**. The options are: *-id*, the comm id of the interpreter that received the command; *-serial*, a unique serial for each command sent to a particular comm

interpreter; *-chan*, the comm channel name; *-code*, the result code of the command; *-errorcode*, the errorcode, if any, of the command; *-errorinfo*, the errorinfo, if any, of the command; and *-result*, the return value of the command. If connection is lost before a reply is received, the callback will be invoked with a connection lost message with -code equal to -1. When **-command** is used, the command returns the unique serial for the command.

::comm::comm self
Returns the *id* for this channel.

::comm::comm interps
Returns a list of all the remote *id*'s to which this channel is connected. **comm** learns a new remote *id* when a command is first issued it, or when a remote *id* first issues a command to this comm channel. **::comm::comm ids** is an alias for this method.

::comm::comm connect ?*id*?
Whereas **::comm::comm send** will automatically connect to the given *id*, this forces a connection to a remote *id* without sending a command. After this, the remote *id* will appear in **::comm::comm interps**.

Eval semantics

The evaluation semantics of **::comm::comm send** are intended to match Tk's **send** *exactly*. This means that **comm** evaluates arguments on the remote side.

If you find that **::comm::comm send** doesn't work for a particular command, try the same thing with Tk's send and see if the result is different. If there is a problem, please report it. For instance, there was had one report that this command produced an error. Note that the equivalent **send** command also produces the same error.

```
% ::comm::comm send id llength {a b c}
wrong # args: should be "llength list"
% send name llength {a b c}
wrong # args: should be "llength list"
```

The **eval** hook (described below) can be used to change from **send**'s double eval semantics to single eval semantics.

Multiple channels

More than one **comm** channel (or *listener*) can be created in each Tcl interpreter. This allows flexibility to create full and restricted channels. For instance, *hook* scripts are specific to the channel they are defined against.

::comm::comm new *chan* ?*name value ...*?
This creates a new channel and Tcl command with the given channel name. This new command controls the new channel and takes all the same arguments as **::comm::comm**. Any remaining arguments are passed to the **config** method. The fully qualified channel name is returned.

::comm::comm channels
This lists all the channels allocated in this Tcl interpreter.

The default configuration parameters for a new channel are:

```
"-port 0 -local 1 -listen 0 -silent 0"
```

The default channel **::comm::comm** is created with:

```
    "::comm::comm new ::comm::comm -port 0 -local 1 -listen 1 -silent
    0"
```

Channel configuration

The **config** method acts similar to **fconfigure** in that it sets or queries configuration variables associated with a channel.

::comm::comm config
see below ...

::comm::comm config *name*
see below ...

::comm::comm config ?*name value...*?
When given no arguments, **config** returns a list of all variables and their value With one argument, **config** returns the value of just that argument. With an even number of arguments, the given variables are set to the given values.

These configuration variables can be changed (descriptions of them are elsewhere in this manual page):

- **-listen** ?*0|1*?
- **-local** ?*0|1*?
- **-port** ?*port*?
- **-silent** ?*0|1*?
- **-socketcmd** ?*commandname*?
- **-interp** ?*interpreter*?
- **-events** ?*eventlist*?

These configuration variables are read only:

- **-chan** *chan*
- **-serial** *n*
- **-socket** sock*In*

When **config** changes the parameters of an existing channel (with the exception of **-interp** and **-events**), it closes and reopens the listening socket. An automatically assigned channel *id* will change when this happens. Recycling the socket is done by invoking **::comm::comm abort**, which causes all active sends to terminate.

Id/port assignments

comm uses a TCP port for endpoint *id*. The **interps** (or **ids**) method merely lists all the TCP ports to which the channel is connected. By default, each channel's *id* is randomly assigned by the operating system (but usually starts at a low value around 1024 and increases each time a new socket is opened). This behavior is accomplished by giving the **-port** config option a value of 0. Alternately, a specific TCP port number may be provided for a given channel. As a special case, comm contains code to allocate a a high-numbered TCP port (>10000) by using **-port {}**. Note that a channel won't be created and initialized unless the specific port can be allocated.

As a special case, if the channel is configured with **-listen 0**, then it will not create a listening socket and will use an id of **0** for itself. Such a channel is only good for outgoing connections (although once a connection is established, it can carry send traffic in both directions). As

another special case, if the channel is configured with **-silent 0**, then the listening side will ignore connection attempts where the protocol negotiation phase failed, instead of throwing an error.

Execution environment

A communication channel in its default configuration will use the current interpreter for the execution of all received scripts, and of the event scripts associated with the various hooks.

This insecure setup can be changed by the user via the two options **-interp**, and **-events**.

When **-interp** is set all received scripts are executed in the slave interpreter specified as the value of the option. This interpreter is expected to exist before configuration. I.e. it is the responsibility of the user to create it. However afterward the communication channel takes ownership of this interpreter, and will destroy it when the communication channel is destroyed. Note that reconfiguration of the communication channel to either a different interpreter or the empty string will release the ownership *without* destroying the previously configured interpreter. The empty string has a special meaning, it restores the default behaviour of executing received scripts in the current interpreter.

Also of note is that replies and callbacks (a special form of reply) are *not* considered as received scripts. They are trusted, part of the internal machinery of comm, and therefore always executed in the current interpreter.

Even if an interpreter has been configured as the execution environment for received scripts the event scripts associated with the various hooks will by default still be executed in the current interpreter. To change this use the option **-events** to declare a list of the events whose scripts should be executed in the declared interpreter as well. The contents of this option are ignored if the communication channel is configured to execute received scripts in the current interpreter.

Remote interpreters

By default, each channel is restricted to accepting connections from the local system. This can be overridden by using the **-local 0** configuration option For such channels, the *id* parameter takes the form *{ id host }*.

WARNING: The *host* must always be specified in the same form (e.g., as either a fully qualified domain name, plain hostname or an IP address).

Closing connections

These methods give control over closing connections:

::comm::comm shutdown *id*
: This closes the connection to *id*, aborting all outstanding commands in progress. Note that nothing prevents the connection from being immediately reopened by another incoming or outgoing command.

::comm::comm abort
: This invokes shutdown on all open connections in this comm channel.

::comm::comm destroy
: This aborts all connections and then destroys the this comm channel itself, including closing the listening socket. Special code allows the default **::comm::comm** channel to be closed such that the **::comm::comm** command it is not destroyed. Doing so closes the listening socket, preventing both incoming and outgoing commands on the channel. This sequence

reinitializes the default channel:

```
"::comm::comm destroy; ::comm::comm new ::comm::comm"
```

When a remote connection is lost (because the remote exited or called **shutdown**), **comm** can invoke an application callback. This can be used to cleanup or restart an ancillary process, for instance. See the *lost* callback below.

Callbacks

This is a mechanism for setting hooks for particular events:

::comm::comm hook *event* ?+? ?*script*?

This uses a syntax similar to Tk's **bind** command. Prefixing *script* with a **+** causes the new script to be appended. Without this, a new *script* replaces any existing script. When invoked without a script, no change is made. In all cases, the new hook script is returned by the command.

When an *event* occurs, the *script* associated with it is evaluated with the listed variables in scope and available. The return code (*not* the return value) of the script is commonly used decide how to further process after the hook.

Common variables include:

chan
: the name of the comm channel (and command)

id
: the id of the remote in question

fid
: the file id for the socket of the connection

These are the defined *events*:

connecting

Variables: **chan**, **id**

This hook is invoked before making a connection to the remote named in *id*. An error return (via **error**) will abort the connection attempt with the error. Example:

```
% ::comm::comm hook connecting {
    if {[string match {*[02468]} $id]} {
        error "Can't connect to even ids"
    }
}
% ::comm::comm send 10000 puts ok
Connect to remote failed: Can't connect to even ids
%
```

connected

Variables: **chan**, **fid**, **id**, **host**, and **port**.

This hook is invoked immediately after making a remote connection to *id*, allowing arbitrary authentication over the socket named by *fid*. An error return (via **error**) will close the

connection with the error. *host* and *port* are merely extracted from the *id*; changing any of these will have no effect on the connection, however. It is also possible to substitute and replace *fid*.

incoming

Variables: **chan**, **fid**, **addr**, and **remport**.

Hook invoked when receiving an incoming connection, allowing arbitrary authentication over socket named by *fid*. An error return (via **error**) will close the connection with the error. Note that the peer is named by *remport* and *addr* but that the remote *id* is still unknown. Example:

```
    ::comm::comm hook incoming {
        if {[string match 127.0.0.1 $addr]} {
            error "I don't talk to myself"
        }
    }
```

eval

Variables: **chan**, **id**, **cmd**, and **buffer**.

This hook is invoked after collecting a complete script from a remote but *before* evaluating it. This allows complete control over the processing of incoming commands. *cmd* contains either **send** or **async**. *buffer* holds the script to evaluate. At the time the hook is called, *$chan remoteid* is identical in value to *id*.

By changing *buffer*, the hook can change the script to be evaluated. The hook can short circuit evaluation and cause a value to be immediately returned by using **return** *result* (or, from within a procedure, **return -code return** *result*). An error return (via **error**) will return an error result, as is if the script caused the error. Any other return will evaluate the script in *buffer* as normal. For compatibility with 3.2, **break** and **return -code break** *result* is supported, acting similarly to **return {}** and **return -code return** *result*.

Examples:

1. augmenting a command

```
    % ::comm::comm send [::comm::comm self] pid
    5013
    % ::comm::comm hook eval {puts "going to execute $buffer"}
    % ::comm::comm send [::comm::comm self] pid
    going to execute pid
    5013
```

2. short circuiting a command

```
    % ::comm::comm hook eval {puts "would have executed $buffer";
    return 0}
    % ::comm::comm send [::comm::comm self] pid
    would have executed pid
    0
```

3. Replacing double eval semantics

```
% ::comm::comm send [::comm::comm self] llength {a b c}
wrong # args: should be "llength list"
% ::comm::comm hook eval {return [uplevel #0 $buffer]}
return [uplevel #0 $buffer]
% ::comm::comm send [::comm::comm self] llength {a b c}
3
```

4. Using a slave interpreter

```
% interp create foo
% ::comm::comm hook eval {return [foo eval $buffer]}
% ::comm::comm send [::comm::comm self] set myvar 123
123
% set myvar
can't read "myvar": no such variable
% foo eval set myvar
123
```

5. Using a slave interpreter (double eval)

```
% ::comm::comm hook eval {return [eval foo eval $buffer]}
```

6. Subverting the script to execute

```
% ::comm::comm hook eval {
    switch -- $buffer {
        a {return A-OK}
        b {return B-OK}
        default {error "$buffer is a no-no"}
    }
}
% ::comm::comm send [::comm::comm self] pid
pid is a no-no
% ::comm::comm send [::comm::comm self] a
A-OK
```

reply

Variables: **chan**, **id**, **buffer**, **ret**, and **return()**.

This hook is invoked after collecting a complete reply script from a remote but *before* evaluating it. This allows complete control over the processing of replies to sent commands. The reply *buffer* is in one of the following forms
- return result
- return -code code result
- return -code code -errorinfo info -errorcode ecode msg

For safety reasons, this is decomposed. The return result is in *ret*, and the return switches are in the return array:
- *return(-code)*
- *return(-errorinfo)*

- *return(-errorcode)*

Any of these may be the empty string. Modifying these four variables can change the return value, whereas modifying *buffer* has no effect.

callback
Variables: **chan**, **id**, **buffer**, **ret**, and **return()**.

Similar to *reply*, but used for callbacks.

lost
Variables: **chan**, **id**, and **reason**.

This hook is invoked when the connection to **id** is lost. Return value (or thrown error) is ignored. *reason* is an explanatory string indicating why the connection was lost. Example:

```
::comm::comm hook lost {
    global myvar
    if {$myvar(id) == $id} {
        myfunc
        return
    }
}
```

Unsupported

These interfaces may change or go away in subsequence releases.

::comm::comm remoteid
Returns the *id* of the sender of the last remote command executed on this channel. If used by a proc being invoked remotely, it must be called before any events are processed. Otherwise, another command may get invoked and change the value.

::comm::comm_send
Invoking this procedure will substitute the Tk **send** and **winfo interps** commands with these equivalents that use **::comm::comm**.

```
proc send {args} {
    eval ::comm::comm send $args
}
rename winfo tk_winfo
proc winfo {cmd args} {
    if {![string match in* $cmd]} {
        return [eval [list tk_winfo $cmd] $args]
    }
    return [::comm::comm interps]
}
```

Security

Starting with version 4.6 of the package an option **-socketcmd** is supported, allowing the user of a comm channel to specify which command to use when opening a socket. Anything which is

API-compatible with the builtin **::socket** (the default) can be used.

The envisioned main use is the specification of the **tls::socket** command, see package **tls**, to secure the communication.

```
    # Load and initialize tls
    package require tls
    tls::init  -cafile /path/to/ca/cert -keyfile ...

    # Create secured comm channel
    ::comm::comm new SECURE -socketcmd tls::socket -listen 1
    ...
```

The sections Execution environment and Callbacks are also relevant to the security of the system, providing means to restrict the execution to a specific environment, perform additional authentication, and the like.

Blocking semantics

There is one outstanding difference between **comm** and **send**. When blocking in a synchronous remote command, **send** uses an internal C hook (Tk_RestrictEvents) to the event loop to look ahead for send-related events and only process those without processing any other events. In contrast, **comm** uses the **vwait** command as a semaphore to indicate the return message has arrived. The difference is that a synchronous **send** will block the application and prevent all events (including window related ones) from being processed, while a synchronous **::comm::comm send** will block the application but still allow other events to get processed. In particular, **after idle** handlers will fire immediately when comm blocks.

What can be done about this? First, note that this behavior will come from any code using **vwait** to block and wait for an event to occur. At the cost of multiple channel support, **comm** could be changed to do blocking I/O on the socket, giving send-like blocking semantics. However, multiple channel support is a very useful feature of comm that it is deemed too important to lose. The remaining approaches involve a new loadable module written in C (which is somewhat against the philosophy of **comm**) One way would be to create a modified version of the **vwait** command that allow the event flags passed to Tcl_DoOneEvent to be specified. For **comm**, just the TCL_FILE_EVENTS would be processed. Another way would be to implement a mechanism like Tk_RestrictEvents, but apply it to the Tcl event loop (since **comm** doesn't require Tk). One of these approaches will be available in a future **comm** release as an optional component.

Asynchronous result generation

By default the result returned by a remotely invoked command is the result sent back to the invoker. This means that the result is generated synchronously, and the server handling the call is blocked for the duration of the command.

While this is tolerable as long as only short-running commands are invoked on the server long-running commands, like database queries make this a problem. One command can prevent the processing requests of all other clients for an arbitrary period of time.

Before version 4.5 of comm the only solution was to rewrite the server command to use the Tcl builtin command **vwait**, or one of its relatives like **tkwait**, to open a new event loop which processes requests while the long-running operation is executed. This however has its own perils, as this makes it possible to both overflow the Tcl stack with a large number of event loop,

and to have a newer requests block the return of older ones, as the eventloop have to be unwound in the order of their creation.

The proper solution is to have the invoked command indicate to **comm** that it cannot or will not deliver an immediate, synchronous result, but will do so later. At that point the framework can put sending the actual result on hold and continue processing requests using the main event loop. No blocking, no nesting of event loops. At some future date the long running operation delivers the result to comm, via the future object, which is then forwarded to the invoker as usual.

The necessary support for this solution has been added to comm since version 4.5, in the form of the new method **return_async**.

::comm::comm return_async

This command is used by a remotely invoked script to notify the comm channel which invoked it that the result to send back to the invoker is not generated synchronously. If this command is not called the default/standard behaviour of comm is to send the synchronously generated result of the script itself to the invoker.

The result of **return_async** is an object. This object, called a *future* is where the result of the script has to be delivered to when it becomes ready. When that happens it will take all the necessary actions to deliver the result to the invoker of the script, and then destroy itself. Should comm have lost the connection to the invoker while the result is being computed the future will not try to deliver the result it got, but just destroy itself. The future can be configured with a command to call when the invoker is lost. This enables the user to implement an early abort of the long-running operation, should this be supported by it.

An example:

```
# Procedure invoked by remote clients to run database operations.
proc select {sql} {
    # Signal the async generation of the result

    set future [::comm::comm return_async]

    # Generate an async db operation and tell it where to deliver
    the result.

    set query [db query -command [list $future return] $sql]

    # Tell the database system which query to cancel if the
    connection
    # goes away while it is running.

    $future configure -command [list db cancel $query]

    # Note: The above will work without problem only if the async
    # query will nover run its completion callback immediately, but
    # only from the eventloop. Because otherwise the future we wish
    to
```

```
    # configure may already be gone. If that is possible use 'catch'
    # to prevent the error from propagating.
    return
}
```

The API of a future object is:

$future return ?-code *code*? ?*value*?

Use this method to tell the future that long-running operation has completed. Arguments are an optional return value (defaults to the empty string), and the Tcl return code (defaults to OK).

The future will deliver this information to invoker, if the connection was not lost in the meantime, and then destroy itself. If the connection was lost it will do nothing but destroy itself.

$future configure ?-command ?*cmdprefix*??

see below ...

$future cget -command

These methods allow the user to retrieve and set a command to be called if the connection the future belongs to has been lost.

Compatibility

comm exports itself as a package. The package version number is in the form *major. minor*, where the major version will only change when a non-compatible change happens to the API or protocol. Minor bug fixes and changes will only affect the minor version. To load **comm** this command is usually used:

```
package require comm 3
```

Note that requiring no version (or a specific version) can also be done.

The revision history of **comm** includes these releases:

4.6.3

Fixed ticket [ced0d60fc9]. Added proper detection of eof on a socket, properly closing it.

4.6.2

Fixed bugs 2972571 and 3066872, the first a misdetection of quoted brace after double backslash, the other a blocking gets making for an obvious (hinsight) DoS attack on comm channels.

4.6.1

Changed the implementation of **comm::commCollect** to emulate lindex's pre-Tcl 8 behaviour, i.e. it was given the ability to parse out the first word of a list, even if the whole buffer is not a well-formed list. Without this change the first word could only be extracted if the whole buffer was a well-formed list (ever since Tcl 8), and in a ver-high-load situation, i.e. a server sending lots and/or large commands very fast, this may never happen, eventually crashing the receiver when it runs out of memory. With the change the receiver is always able to process the first word when it becomes well-formed, regardless of the structure of the remainder of the buffer.

4.6
Added the option **-socketcmd** enabling users to override how a socket is opened. The envisioned main use is the specification of the **tls::socket** command, see package **tls**, to secure the communication.

4.5.7
Changed handling of ports already in use to provide a proper error message.

4.5.6
Bugfix in the replacement for **vwait**, made robust against of variable names containing spaces.

4.5.5
Bugfix in the handling of hooks, typo in variable name.

4.5.4
Bugfix in the handling of the result received by the **send** method. Replaced an *after idle unset result* with an immediate **unset**, with the information saved to a local variable.

The **after idle** can spill into a forked child process if there is no event loop between its setup and the fork. This may bork the child if the next event loop is the **vwait** of **comm**'s **send** a few lines above the **after idle**, and the child used the same serial number for its next request. In that case the parent's **after idle unset** will delete the very array element the child is waiting for, unlocking the **vwait**, causing it to access a now missing array element, instead of the expected result.

4.5.3
Bugfixes in the wrappers for the builtin **update** and **vwait** commands.

4.5.2
Bugfix in the wrapper for the builtin **update** command.

4.5.1
Bugfixes in the handling of -interp for regular scripts. The handling of the buffer was wrong for scripts which are a single statement as list. Fixed missing argument to new command **commSendReply**, introduced by version 4.5. Affected debugging.

4.5
New server-side feature. The command invoked on the server can now switch comm from the standard synchronous return of its result to an asynchronous (defered) return. Due to the use of snit to implement the *future* objects used by this feature from this version on comm requires at least Tcl 8.3 to run. Please read the section Asynchronous result generation for more details.

4.4.1
Bugfix in the execution of hooks.

4.4
Bugfixes in the handling of -interp for regular and hook scripts. Bugfixes in channel cleanup.

4.3.1
Introduced -interp and -events to enable easy use of a slave interp for execution of received scripts, and of event scripts.

4.3
Bugfixes, and introduces -silent to allow the user to force the server/listening side to silently ignore connection attempts where the protocol negotiation failed.

4.2
Bugfixes, and most important, switched to utf-8 as default encoding for full i18n without any problems.

4.1
Rewrite of internal code to remove old pseudo-object model. Addition of send -command asynchronous callback option.

4.0
Per request by John LoVerso. Improved handling of error for async invoked commands.

3.7
Moved into tcllib and placed in a proper namespace.

3.6
A bug in the looking up of the remoteid for a executed command could be triggered when the connection was closed while several asynchronous sends were queued to be executed.

3.5
Internal change to how reply messages from a **send** are handled. Reply messages are now decoded into the *value* to pass to **return**; a new return statement is then cons'd up to with this value. Previously, the return code was passed in from the remote as a command to evaluate. Since the wire protocol has not changed, this is still the case. Instead, the reply handling code decodes the **reply** message.

3.4
Added more source commentary, as well as documenting config variables in this man page. Fixed bug were loss of connection would give error about a variable named **pending** rather than the message about the lost connection. **comm ids** is now an alias for **comm interps** (previously, it an alias for **comm chans**). Since the method invocation change of 3.0, break and other exceptional conditions were not being returned correctly from **comm send**. This has been fixed by removing the extra level of indirection into the internal procedure **commSend**. Also added propagation of the *errorCode* variable. This means that these commands return exactly as they would with **send**:

```
    comm send id break
    catch {comm send id break}
    comm send id expr 1 / 0
```

Added a new hook for reply messages. Reworked method invocation to avoid the use of comm:* procedures; this also cut the invocation time down by 40%. Documented **comm config** (as this manual page still listed the defunct **comm init**!)

3.3
Some minor bugs were corrected and the documentation was cleaned up. Added some examples for hooks. The return semantics of the **eval** hook were changed.

3.2
A new wire protocol, version 3, was added. This is backwards compatible with version 2 but adds an exchange of supported protocol versions to allow protocol negotiation in the future. Several bugs with the hook implementation were fixed. A new section of the man page on blocking semantics was added.

3.1

All the documented hooks were implemented. **commLostHook** was removed. A bug in **comm new** was fixed.

3.0
This is a new version of **comm** with several major changes. There is a new way of creating the methods available under the **comm** command. The **comm init** method has been retired and is replaced by **comm configure** which allows access to many of the well-defined internal variables. This also generalizes the options available to **comm new**. Finally, there is now a protocol version exchanged when a connection is established. This will allow for future on-wire protocol changes. Currently, the protocol version is set to 2.

2.3
comm ids was renamed to **comm channels**. General support for **comm hook** was fully implemented, but only the *lost* hook exists, and it was changed to follow the general hook API. **commLostHook** was unsupported (replaced by **comm hook lost**) and **commLost** was removed.

2.2
The *died* hook was renamed *lost*, to be accessed by **commLostHook** and an early implementation of **comm lost hook**. As such, **commDied** is now **commLost**.

2.1
Unsupported method **comm remoteid** was added.

2.0
comm has been rewritten from scratch (but is fully compatible with Comm 1.0, without the requirement to use obTcl).

Tls security considerations

This package uses the **TLS** package to handle the security for **https** urls and other socket connections.

Policy decisions like the set of protocols to support and what ciphers to use are not the responsibility of **TLS**, nor of this package itself however. Such decisions are the responsibility of whichever application is using the package, and are likely influenced by the set of servers the application will talk to as well.

For example, in light of the recent POODLE attack discovered by Google many servers will disable support for the SSLv3 protocol. To handle this change the applications using **TLS** must be patched, and not this package, nor **TLS** itself. Such a patch may be as simple as generally activating **tls1** support, as shown in the example below.

```
package require tls
tls::init -tls1 1 ;# forcibly activate support for the TLS1
protocol

... your own application code ...
```

Author

John LoVerso, John@LoVerso.Southborough.MA.US

http://www.opengroup.org/~loverso/tcl-tk/#comm

License

Please see the file *comm.LICENSE* that accompanied this source, or http://www.opengroup.org/www/dist_client/caubweb/COPYRIGHT.free.html.

This license for **comm**, new as of version 3.2, allows it to be used for free, without any licensing fee or royalty.

Bugs

- If there is a failure initializing a channel created with **::comm::comm new**, then the channel should be destroyed. Currently, it is left in an inconsistent state.
- There should be a way to force a channel to quiesce when changing the configuration.

The following items can be implemented with the existing hooks and are listed here as a reminder to provide a sample hook in a future version.

- Allow easier use of a slave interp for actual command execution (especially when operating in "not local" mode).
- Add host list (xhost-like) or "magic cookie" (xauth-like) authentication to initial handshake.

The following are outstanding todo items.

- Add an interp discovery and name->port mapping. This is likely to be in a separate, optional nameserver. (See also the related work, below.)
- Fix the *{id host}* form so as not to be dependent upon canonical hostnames. This requires fixes to Tcl to resolve hostnames!

This man page is bigger than the source file.

On using old versions of Tcl

Tcl7.5 under Windows contains a bug that causes the interpreter to hang when EOF is reached on non-blocking sockets. This can be triggered with a command such as this:

```
    "comm send $other exit"
```

Always make sure the channel is quiescent before closing/exiting or use at least Tcl7.6 under Windows.

Tcl7.6 on the Mac contains several bugs. It is recommended you use at least Tcl7.6p2.

Tcl8.0 on UNIX contains a socket bug that can crash Tcl. It is recommended you use Tcl8.0p1 (or Tcl7.6p2).

Related work

Tcl-DP provides an RPC-based remote execution interface, but is a compiled Tcl extension. See http://www.cs.cornell.edu/Info/Projects/zeno/Projects/Tcl-DP.html.

Michael Doyle miked@eolas.com has code that implements the Tcl-DP RPC interface using standard Tcl sockets, much like **comm**. The DpTcl package is available at http://chiselapp.com/user/gwlester/repository/DpTcl.

Andreas Kupries uses **comm** and has built a simple nameserver as part of his Pool library. See http://www.purl.org/net/akupries/soft/pool/index.htm.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you

want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *comm* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Programming tools

Copyright

```
Copyright (c) 1995-1998 The Open Group. All Rights Reserved.
Copyright (c) 2003-2004 ActiveState Corporation.
Copyright (c) 2006-2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


comm_wire
Keywords: comm_wire, comm, communication, ipc, message, remote communication, remote execution, rpc, socket

Name
comm_wire - The comm wire protocol

Synopsis
package require comm

Description
The **comm** command provides an inter-interpreter remote execution facility much like Tk's **send(n)**, except that it uses sockets rather than the X server for the communication path. As a result, **comm** works with multiple interpreters, works on Windows and Macintosh systems, and provides control over the remote execution path.

This document contains a specification of the various versions of the wire protocol used by comm internally for the communication between its endpoints. It has no relevance to users of **comm**, only to developers who wish to modify the package, write a compatible facility in a different language, or some other facility based on the same protocol.

Wire protocol version 3

Basic Layer
The basic encoding for *all* data is UTF-8. Because of this binary data, including the NULL character, can be sent over the wire as is, without the need for armoring it.

Basic message Layer
On top of the Basic Layer we have a *message oriented* exchange of data. The totality of all characters written to the channel is a Tcl list, with each element a separate *message*, each itself a list. The messages in the overall list are separated by EOL. Note that EOL characters can occur

within the list as well. They can be distinguished from the message-separating EOL by the fact that the data from the beginning up to their location is not a valid Tcl list.

EOL is signaled through the linefeed character, i.e **LF**, or, hex **0x0a**. This is following the unix convention for line-endings.

As a list each message is composed of *words*. Their meaning depends on when the message was sent in the overall exchange. This is described in the upcoming sections.

Negotiation messages - initial handshake

The command protocol is defined like this:

- The first message send by a client to a server, when opening the connection, contains two words. The first word is a list as well, and contains the versions of the wire protocol the client is willing to accept, with the most preferred version first. The second word is the TCP port the client is listening on for connections to itself. The value **0** is used here to signal that the client will not listen for connections, i.e. that it is purely for sending commands, and not receiving them.
- The first message sent by the server to the client, in response to the message above contains only one word. This word is a list, containing the string **vers** as its first element, and the version of the wire protocol the server has selected from the offered versions as the second.

Script/command messages

All messages coming after the **initial handshake** consist of three words. These are an instruction, a transaction id, and the payload. The valid instructions are shown below. The transaction ids are used by the client to match any incoming replies to the command messages it sent. This means that a server has to copy the transaction id from a command message to the reply it sends for that message.

send
: see below ...

async
: see below ...

command
: The payload is the Tcl script to execute on the server. It is actually a list containing the script fragments. These fragment are **concat**enated together by the server to form the full script to execute on the server side. This emulates the Tcl "eval" semantics. In most cases it is best to have only one word in the list, a list containing the exact command.

  Examples:

```
    (a)     {send 1 {{array get tcl_platform}}}
    (b)     {send 1 {array get tcl_platform}}
    (c)     {send 1 {array {get tcl_platform}}}

    are all valid representations of the same command. They are
    generated via

    (a')    send {array get tcl_platform}
    (b')    send array get tcl_platform
```

```
    (c')    send array {get tcl_platform}

    respectively
```

Note that (a), generated by (a'), is the usual form, if only single commands are sent by the client. For example constructed using **list**, if the command contains variable arguments. Like

```
    send [list array get $the_variable]
```

These three instructions all invoke the script on the server side. Their difference is in the treatment of result values, and thus determines if a reply is expected.

send
A reply is expected. The sender is waiting for the result.

async
No reply is expected, the sender has no interest in the result and is not waiting for any.

command
A reply is expected, but the sender is not waiting for it. It has arranged to get a process-internal notification when the result arrives.

reply
Like the previous three command, however the tcl script in the payload is highly restricted. It has to be a syntactically valid Tcl **return** command. This contains result code, value, error code, and error info.

Examples:

```
    {reply 1 {return -code 0 {}}}
    {reply 1 {return -code 0 {osVersion 2.4.21-99-default byteOrder
    littleEndian machine i686 platform unix os Linux user andreask
    wordSize 4}}}
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *comm* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Programming tools

Copyright


```
Copyright (c) 2005 Docs. Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

control

control

Keywords: control, assert, do, flow, no-op, structure

Name

control - Procedures for control flow structures.

Synopsis

package require Tcl 8.5 9
package require control ?**0.1.4**?
control::control *command option* ?*arg arg* ...?
control::assert *expr* ?*arg arg* ...?
control::do *body* ?*option test*?
control::no-op ?*arg arg* ...?

Description

The **control** package provides a variety of commands that provide additional flow of control structures beyond the built-in ones provided by Tcl. These are commands that in many programming languages might be considered *keywords*, or a part of the language itself. In Tcl, control flow structures are just commands like everything else.

Commands

control::control *command option* ?*arg arg ...*?

The **control** command is used as a configuration command for customizing the other public commands of the control package. The *command* argument names the command to be customized. The set of valid *option* and subsequent arguments are determined by the command being customized, and are documented with the command.

control::assert *expr* ?*arg arg ...*?

When disabled, the **assert** command behaves exactly like the **no-op** command.

When enabled, the **assert** command evaluates *expr* as an expression (in the same way that **expr** evaluates its argument). If evaluation reveals that *expr* is not a valid boolean expression (according to [**string is boolean -strict**]), an error is raised. If *expr* evaluates to a true boolean value (as recognized by **if**), then **assert** returns an empty string. Otherwise, the remaining arguments to **assert** are used to construct a message string. If there are no arguments, the message string is "assertion failed: $expr". If there are arguments, they are joined by **join** to form the message string. The message string is then appended as an argument to a callback command, and the completed callback command is evaluated in the global namespace.

The **assert** command can be customized by the **control** command in two ways:

[**control::control assert enabled** ?*boolean*?] queries or sets whether **control::assert** is enabled. When called without a *boolean* argument, a boolean value is returned indicating whether the **control::assert** command is enabled. When called with a valid boolean value as the *boolean* argument, the **control::assert** command is enabled or disabled to match the argument, and an empty string is returned.

[**control::control assert callback** ?*command*?] queries or sets the callback command that will be called by an enabled **assert** on assertion failure. When called without a *command* argument, the current callback command is returned. When called with a *command* argument, that argument becomes the new assertion failure callback command. Note that an assertion failure callback command is always defined, even when **assert** is disabled. The default callback command is [**return -code error**].

Note that **control::assert** has been written so that in combination with [**namespace import**], it is possible to use enabled **assert** commands in some namespaces and disabled **assert** commands in other namespaces at the same time. This capability is useful so that debugging efforts can be independently controlled module by module.

```
% package require control
% control::control assert enabled 1
% namespace eval one namespace import ::control::assert
% control::control assert enabled 0
% namespace eval two namespace import ::control::assert
% one::assert {1 == 0}
assertion failed: 1 == 0
% two::assert {1 == 0}
```

control::do *body* ?*option test*?

The **do** command evaluates the script *body* repeatedly *until* the expression *test* becomes true or as long as (*while*) *test* is true, depending on the value of *option* being **until** or **while**. If *option* and *test* are omitted the body is evaluated exactly once. After normal completion, **do** returns an empty string. Exceptional return codes (**break**, **continue**, **error**, etc.) during the evaluation of *body* are handled in the same way the **while** command handles them, except as noted in Limitations, below.

control::no-op ?*arg arg ...*?

The **no-op** command takes any number of arguments and does nothing. It returns an empty string.

Limitations

Several of the commands provided by the **control** package accept arguments that are scripts to be evaluated. Due to fundamental limitations of Tcl's **catch** and **return** commands, it is not possible for these commands to properly evaluate the command [**return -code $code**] within one of those script arguments for any value of *$code* other than *ok*. In this way, the commands of the **control** package are limited as compared to Tcl's built-in control flow commands (such as **if**, **while**, etc.) and those control flow commands that can be provided by packages coded in C. An example of this difference:

```
% package require control
% proc a {} {while 1 {return -code error a}}
% proc b {} {control::do {return -code error b} while 1}
% catch a
1
% catch b
0
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *control* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Programming tools

coroutine

coro_auto

Keywords: coro_auto, after, channel, coroutine, events, exit, gets, global, green threads, read, threads, update, vwait

Name

coroutine::auto - Automatic event and IO coroutine awareness

Synopsis

package require Tcl 8.6 9
package require coroutine::auto 1.3
package require coroutine 1.4

Description

The **coroutine::auto** package provides no commands or other directly visible functionality. Built on top of the package **coroutine**, it intercepts various builtin commands of the Tcl core to make any code using them coroutine-oblivious, i.e. able to run inside and outside of a coroutine without changes.

The commands so affected by this package are

- **after**
- **exit**
- **gets**
- **global**
- **puts**
- **read**
- **socket**
- **update**
- **vwait**

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib

Trackers and specify *coroutine* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Coroutine

Copyright

```
Copyright (c) 2010-2014 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


tcllib_coroutine
Keywords: tcllib_coroutine, after, channel, coroutine, events, exit, gets, global, green threads, read, threads, update, vwait

Name
coroutine - Coroutine based event and IO handling

Synopsis
package require Tcl 8.6 9
package require coroutine 1.4
coroutine::util after *delay*
coroutine::util await *varname...*
coroutine::util create *arg...*
coroutine::util exit ?*status*?
coroutine::util gets *chan* ?*varname*?
coroutine::util gets_safety *chan limit varname*
coroutine::util global *varname...*
coroutine::util puts ?**-nonewline**? *channel string*
coroutine::util read -nonewline *chan* ?*n*?
coroutine::util socket ?*options...*? *host port*
coroutine::util update ?**idletasks**?
coroutine::util vwait *varname*

Description
The **coroutine** package provides coroutine-aware implementations of various event- and channel related commands. It can be in multiple modes:

1. Call the commands through their ensemble, in code which is explicitly written for use within coroutines.
2. Import the commands into a namespace, either directly, or through **namespace path**. This allows the use from within code which is not coroutine-aware per se and restricted to specific namespaces.

A more agressive form of making code coroutine-oblivious than point 2 above is available

through the package coroutine::auto, which intercepts the relevant builtin commands and changes their implementation dependending on the context they are run in, i.e. inside or outside of a coroutine.

API

All the commands listed below are synchronous with respect to the coroutine invoking them, i.e. this coroutine blocks until the result is available. The overall eventloop is not blocked however.

coroutine::util after *delay*
: This command delays the coroutine invoking it by *delay* milliseconds.

coroutine::util await *varname...*
: This command is an extension form of the **coroutine::util vwait** command (see below) which waits on a write to one of many named namespace variables.

coroutine::util create *arg...*
: This command creates a new coroutine with an automatically assigned name and causes it to run the code specified by the arguments.

coroutine::util exit ?*status*?
: This command exits the current coroutine, causing it to return *status*. If no status was specified the default *0* is returned.

coroutine::util gets *chan* ?*varname*?
: This command reads a line from the channel *chan* and returns it either as its result, or, if a *varname* was specified, writes it to the named variable and returns the number of characters read.

coroutine::util gets_safety *chan limit varname*
: This command reads a line from the channel *chan* up to size *limit* and stores the result in *varname*. Of *limit* is reached before the set first newline, an error is thrown. The command returns the number of characters read.

coroutine::util global *varname...*
: This command imports the named global variables of the coroutine into the current scope. From the technical point of view these variables reside in level **#1** of the Tcl stack. I.e. these are not the regular global variable in to the global namespace, and each coroutine can have their own set, independent of all others.

coroutine::util puts ?-nonewline? *channel string*
: This commands writes the string to the specified *channel*. Contrary to the builtin **puts** this command waits until the *channel* is writable before actually writing to it.

coroutine::util read -nonewline *chan* ?*n*?
: This command reads *n* characters from the channel *chan* and returns them as its result. If *n* is not specified the command will read the channel until EOF is reached.

coroutine::util socket ?*options...*? *host port*
: This command connects to the specified host and port and returns when that is done. Contrary to the builtin command it performs a non-blocking connect in the background. As such, while its blocks the calling coroutine, the overall application is not blocked.

coroutine::util update ?idletasks?
: This command causes the coroutine invoking it to run pending events or idle handlers before proceeding.

coroutine::util vwait *varname*

This command causes the coroutine calling it to wait for a write to the named namespace variable *varname*.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *coroutine* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Coroutine

Copyright

```
Copyright (c) 2010-2015 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

counter

counter

Keywords: counter, counting, histogram, statistics, tallying

Name

counter - Procedures for counters and histograms

Synopsis

package require Tcl 8.5 9
package require counter ?**2.0.6**?
::counter::init *tag args*
::counter::count *tag* ?*delta*? ?*instance*?
::counter::start *tag instance*
::counter::stop *tag instance*
::counter::get *tag args*
::counter::exists *tag*
::counter::names
::counter::histHtmlDisplay *tag args*
::counter::reset *tag args*

Description

The **counter** package provides a counter facility and can compute statistics and histograms over the collected data.

::counter::init *tag args*

This defines a counter with the name *tag*. The *args* determines the characteristics of the

counter. The *args* are

-group *name*
Keep a grouped counter where the name of the histogram bucket is passed into **::counter::count**.

-hist *bucketsize*
Accumulate the counter into histogram buckets of size *bucketsize*. For example, if the samples are millisecond time values and *bucketsize* is 10, then each histogram bucket represents time values of 0 to 10 msec, 10 to 20 msec, 20 to 30 msec, and so on.

-hist2x *bucketsize*
Accumulate the statistic into histogram buckets. The size of the first bucket is *bucketsize*, each other bucket holds values 2 times the size of the previous bucket. For example, if *bucketsize* is 10, then each histogram bucket represents time values of 0 to 10 msec, 10 to 20 msec, 20 to 40 msec, 40 to 80 msec, and so on.

-hist10x *bucketsize*
Accumulate the statistic into histogram buckets. The size of the first bucket is *bucketsize*, each other bucket holds values 10 times the size of the previous bucket. For example, if *bucketsize* is 10, then each histogram bucket represents time values of 0 to 10 msec, 10 to 100 msec, 100 to 1000 msec, and so on.

-lastn *N*
Save the last *N* values of the counter to maintain a "running average" over the last *N* values.

-timehist *secsPerMinute*
Keep a time-based histogram. The counter is summed into a histogram bucket based on the current time. There are 60 per-minute buckets that have a size determined by *secsPerMinute*, which is normally 60, but for testing purposes can be less. Every "hour" (i.e., 60 "minutes") the contents of the per-minute buckets are summed into the next hourly bucket. Every 24 "hours" the contents of the per-hour buckets are summed into the next daily bucket. The counter package keeps all time-based histograms in sync, so the first *secsPerMinute* value seen by the package is used for all subsequent time-based histograms.

::counter::count *tag* ?*delta*? ?*instance*?
Increment the counter identified by *tag*. The default increment is 1, although you can increment by any value, integer or real, by specifying *delta*. You must declare each counter with **::counter::init** to define the characteristics of counter before you start to use it. If the counter type is **-group**, then the counter identified by *instance* is incremented.

::counter::start *tag instance*
Record the starting time of an interval. The *tag* is the name of the counter defined as a **-hist** value-based histogram. The *instance* is used to distinguish this interval from any other intervals that might be overlapping this one.

::counter::stop *tag instance*
Record the ending time of an interval. The delta time since the corresponding **::counter::start** call for *instance* is recorded in the histogram identified by *tag*.

::counter::get *tag args*
Return statistics about a counter identified by *tag*. The *args* determine what value to return:

-total
Return the total value of the counter. This is the default if *args* is not specified.

-totalVar
Return the name of the total variable. Useful for specifying with -textvariable in a Tk widget.

-N
Return the number of samples accumulated into the counter.

-avg
Return the average of samples accumulated into the counter.

-avgn
Return the average over the last *N* samples taken. The *N* value is set in the **::counter::init** call.

-hist *bucket*
If *bucket* is specified, then the value in that bucket of the histogram is returned. Otherwise the complete histogram is returned in array get format sorted by bucket.

-histVar
Return the name of the histogram array variable.

-histHour
Return the complete hourly histogram in array get format sorted by bucket.

-histHourVar
Return the name of the hourly histogram array variable.

-histDay
Return the complete daily histogram in array get format sorted by bucket.

-histDayVar
Return the name of the daily histogram array variable.

-resetDate
Return the clock seconds value recorded when the counter was last reset.

-all
Return an array get of the array used to store the counter. This includes the total, the number of samples (N), and any type-specific information. This does not include the histogram array.

::counter::exists *tag*
Returns 1 if the counter is defined.

::counter::names
Returns a list of all counters defined.

::counter::histHtmlDisplay *tag args*
Generate HTML to display a histogram for a counter. The *args* control the format of the display. They are:

-title *string*
Label to display above bar chart

-unit *unit*
Specify **minutes**, **hours**, or **days** for the time-base histograms. For value-based histograms, the *unit* is used in the title.

-images *url*
URL of /images directory.

-gif *filename*
Image for normal histogram bars. The *filename* is relative to the **-images** directory.

-ongif *filename*
Image for the active histogram bar. The *filename* is relative to the **-images** directory.

-max *N*
Maximum number of value-based buckets to display.

-height *N*
Pixel height of the highest bar.

-width *N*
Pixel width of each bar.

-skip *N*
Buckets to skip when labeling value-based histograms.

-format *string*
Format used to display labels of buckets.

-text *boolean*
If 1, a text version of the histogram is dumped, otherwise a graphical one is generated.

::counter::reset *tag args*
Resets the counter with the name *tag* to an initial state. The *args* determine the new characteristics of the counter. They have the same meaning as described for **::counter::init**.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *counter* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Data structures

crc

cksum

Keywords: cksum, checksum, crc, crc32, cyclic redundancy check, data integrity, security

Name

cksum - Calculate a cksum(1) compatible checksum

Synopsis

package require Tcl 8.5 9

package require cksum ?**1.1.5**?
::crc::cksum ?*-format format*? ?*-chunksize size*? [*-channel chan* | *-filename file* | *string*]
::crc::CksumInit
::crc::CksumUpdate *token data*
::crc::CksumFinal *token*

Description

This package provides a Tcl implementation of the cksum(1) algorithm based upon information provided at in the GNU implementation of this program as part of the GNU Textutils 2.0 package.

Commands

::crc::cksum ?*-format format*? ?*-chunksize size*? [*-channel chan* | *-filename file* | *string*]

The command takes string data or a channel or file name and returns a checksum value calculated using the **cksum(1)** algorithm. The result is formatted using the *format*(n) specifier provided or as an unsigned integer (%u) by default.

Options

-channel *name*

Return a checksum for the data read from a channel. The command will read data from the channel until the **eof** is true. If you need to be able to process events during this calculation see the Programming interface section

-filename *name*

This is a convenience option that opens the specified file, sets the encoding to binary and then acts as if the *-channel* option had been used. The file is closed on completion.

-format *string*

Return the checksum using an alternative format template.

Programming interface

The cksum package implements the checksum using a context variable to which additional data can be added at any time. This is expecially useful in an event based environment such as a Tk application or a web server package. Data to be checksummed may be handled incrementally during a **fileevent** handler in discrete chunks. This can improve the interactive nature of a GUI application and can help to avoid excessive memory consumption.

::crc::CksumInit

Begins a new cksum context. Returns a token ID that must be used for the remaining functions. An optional seed may be specified if required.

::crc::CksumUpdate *token data*

Add data to the checksum identified by token. Calling *CksumUpdate $token* "*abcd*" is equivalent to calling *CksumUpdate $token* "*ab*" followed by *CksumUpdate $token* "*cb*". See Examples.

::crc::CksumFinal *token*

Returns the checksum value and releases any resources held by this token. Once this command completes the token will be invalid. The result is a 32 bit integer value.

Examples

```
% crc::cksum "Hello, World!"
2609532967
```

```
% crc::cksum -format 0x%X "Hello, World!"
0x9B8A5027
```

```
% crc::cksum -file cksum.tcl
1828321145
```

```
% set tok [crc::CksumInit]
% crc::CksumUpdate $tok "Hello, "
% crc::CksumUpdate $tok "World!"
% crc::CksumFinal $tok
2609532967
```

Authors

Pat Thoyts

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *crc* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Hashes, checksums, and encryption

Copyright

```
Copyright (c) 2002, Pat Thoyts
```

crc16

Keywords: crc16, checksum, cksum, crc, crc32, cyclic redundancy check, data integrity, security

Name

crc16 - Perform a 16bit Cyclic Redundancy Check

Synopsis

package require Tcl 8.5 9
package require crc16 ?**1.1.5**?
::crc::crc16 ?*-format format*? ?*-seed value*? ?*-implementation procname*? **--** *message*
::crc::crc16 ?*-format format*? ?*-seed value*? ?*-implementation procname*? -filename *file*
::crc::crc-ccitt ?*-format format*? ?*-seed value*? ?*-implementation procname*? **--** *message*

::crc::crc-ccitt ?*-format format*? ?*-seed value*? ?*-implementation procname*? -filename *file*
::crc::xmodem ?*-format format*? ?*-seed value*? ?*-implementation procname*? **--** *message*
::crc::xmodem ?*-format format*? ?*-seed value*? ?*-implementation procname*? -filename *file*
::crc::kermit ?*-format format*? ?*-seed value*? ?*-implementation procname*? **--** *message*
::crc::kermit ?*-format format*? ?*-seed value*? ?*-implementation procname*? -filename *file*
::crc::modbus ?*-format format*? ?*-seed value*? ?*-implementation procname*? **--** *message*
::crc::modbus ?*-format format*? ?*-seed value*? ?*-implementation procname*? -filename *file*
::crc::mcrf4xx ?*-format format*? ?*-seed value*? ?*-implementation procname*? **--** *message*
::crc::mcrf4xx ?*-format format*? ?*-seed value*? ?*-implementation procname*? -filename *file*
::crc::genibus ?*-format format*? ?*-seed value*? ?*-implementation procname*? **--** *message*
::crc::genibus ?*-format format*? ?*-seed value*? ?*-implementation procname*? -filename *file*
::crc::crc-x25 ?*-format format*? ?*-seed value*? ?*-implementation procname*? **--** *message*
::crc::crc-x25 ?*-format format*? ?*-seed value*? ?*-implementation procname*? -filename *file*
::crc::crc-sdlc ?*-format format*? ?*-seed value*? ?*-implementation procname*? **--** *message*
::crc::crc-sdlc ?*-format format*? ?*-seed value*? ?*-implementation procname*? -filename *file*
::crc::crc-usb ?*-format format*? ?*-seed value*? ?*-implementation procname*? **--** *message*
::crc::crc-usb ?*-format format*? ?*-seed value*? ?*-implementation procname*? -filename *file*
::crc::buypass ?*-format format*? ?*-seed value*? ?*-implementation procname*? **--** *message*
::crc::buypass ?*-format format*? ?*-seed value*? ?*-implementation procname*? -filename *file*
::crc::umts ?*-format format*? ?*-seed value*? ?*-implementation procname*? **--** *message*
::crc::umts ?*-format format*? ?*-seed value*? ?*-implementation procname*? -filename *file*
::crc::gsm ?*-format format*? ?*-seed value*? ?*-implementation procname*? **--** *message*
::crc::gsm ?*-format format*? ?*-seed value*? ?*-implementation procname*? -filename *file*
::crc::unknown2 ?*-format format*? ?*-seed value*? ?*-implementation procname*? **--** *message*
::crc::unknown2 ?*-format format*? ?*-seed value*? ?*-implementation procname*? -filename *file*
::crc::maxim ?*-format format*? ?*-seed value*? ?*-implementation procname*? **--** *message*
::crc::maxim ?*-format format*? ?*-seed value*? ?*-implementation procname*? -filename *file*
::crc::unknown3 ?*-format format*? ?*-seed value*? ?*-implementation procname*? **--** *message*
::crc::unknown3 ?*-format format*? ?*-seed value*? ?*-implementation procname*? -filename *file*
::crc::unknown4 ?*-format format*? ?*-seed value*? ?*-implementation procname*? **--** *message*
::crc::unknown4 ?*-format format*? ?*-seed value*? ?*-implementation procname*? -filename *file*
::crc::cms ?*-format format*? ?*-seed value*? ?*-implementation procname*? **--** *message*
::crc::cms ?*-format format*? ?*-seed value*? ?*-implementation procname*? -filename *file*

Description

This package provides a Tcl-only implementation of the CRC algorithms based upon information provided at http://www.microconsultants.com/tips/crc/crc.txt There are a number of permutations available for calculating CRC checksums and this package can handle all of them. Defaults are set up for the most common cases.

Commands

::crc::crc16 ?-format *format*? ?-seed *value*? ?-implementation *procname*? -- *message*

see below ...

::crc::crc16 ?-format *format*? ?-seed *value*? ?-implementation *procname*? -filename *file*

see below ...

::crc::crc-ccitt ?-format *format*? ?-seed *value*? ?-implementation *procname*? -- *message*

see below ...

::crc::crc-ccitt ?-format *format*? ?-seed *value*? ?-implementation *procname*? -filename *file*

see below ...

::crc::xmodem ?-format *format*? ?-seed *value*? ?-implementation *procname*? -- *message*

see below ...

::crc::xmodem ?-format *format*? ?-seed *value*? ?-implementation *procname*? -filename *file*

see below ...

::crc::kermit ?-format *format*? ?-seed *value*? ?-implementation *procname*? -- *message*

see below ...

::crc::kermit ?-format *format*? ?-seed *value*? ?-implementation *procname*? -filename *file*

see below ...

::crc::modbus ?-format *format*? ?-seed *value*? ?-implementation *procname*? -- *message*

see below ...

::crc::modbus ?-format *format*? ?-seed *value*? ?-implementation *procname*? -filename *file*

see below ...

::crc::mcrf4xx ?-format *format*? ?-seed *value*? ?-implementation *procname*? -- *message*

see below ...

::crc::mcrf4xx ?-format *format*? ?-seed *value*? ?-implementation *procname*? -filename *file*

see below ...

::crc::genibus ?-format *format*? ?-seed *value*? ?-implementation *procname*? -- *message*

see below ...

::crc::genibus ?-format *format*? ?-seed *value*? ?-implementation *procname*? -filename *file*

see below ...

::crc::crc-x25 ?-format *format*? ?-seed *value*? ?-implementation *procname*? -- *message*

see below ...

::crc::crc-x25 ?-format *format*? ?-seed *value*? ?-implementation *procname*? -filename *file*

see below ...

::crc::crc-sdlc ?-format *format*? ?-seed *value*? ?-implementation *procname*? -- *message*

see below ...

::crc::crc-sdlc ?-format *format*? ?-seed *value*? ?-implementation *procname*? -filename

file
see below ...

::crc::crc-usb ?-format *format*? ?-seed *value*? ?-implementation *procname*? -- *message*
see below ...

::crc::crc-usb ?-format *format*? ?-seed *value*? ?-implementation *procname*? -filename *file*
see below ...

::crc::buypass ?-format *format*? ?-seed *value*? ?-implementation *procname*? -- *message*
see below ...

::crc::buypass ?-format *format*? ?-seed *value*? ?-implementation *procname*? -filename *file*
see below ...

::crc::umts ?-format *format*? ?-seed *value*? ?-implementation *procname*? -- *message*
see below ...

::crc::umts ?-format *format*? ?-seed *value*? ?-implementation *procname*? -filename *file*
see below ...

::crc::gsm ?-format *format*? ?-seed *value*? ?-implementation *procname*? -- *message*
see below ...

::crc::gsm ?-format *format*? ?-seed *value*? ?-implementation *procname*? -filename *file*
see below ...

::crc::unknown2 ?-format *format*? ?-seed *value*? ?-implementation *procname*? -- *message*
see below ...

::crc::unknown2 ?-format *format*? ?-seed *value*? ?-implementation *procname*? -filename *file*
see below ...

::crc::maxim ?-format *format*? ?-seed *value*? ?-implementation *procname*? -- *message*
see below ...

::crc::maxim ?-format *format*? ?-seed *value*? ?-implementation *procname*? -filename *file*
see below ...

::crc::unknown3 ?-format *format*? ?-seed *value*? ?-implementation *procname*? -- *message*
see below ...

::crc::unknown3 ?-format *format*? ?-seed *value*? ?-implementation *procname*? -filename *file*
see below ...

::crc::unknown4 ?-format *format*? ?-seed *value*? ?-implementation *procname*? -- *message*
see below ...

::crc::unknown4 ?-format *format*? ?-seed *value*? ?-implementation *procname*? -filename *file*
see below ...

::crc::cms ?-format *format*? ?-seed *value*? ?-implementation *procname*? -- *message*
see below ...

::crc::cms ?-format *format*? ?-seed *value*? ?-implementation *procname*? -filename *file*
The command takes either string data or a file name and returns a checksum value calculated using the CRC algorithm. The command used sets up the CRC polynomial, initial value and bit ordering for the desired standard checksum calculation. The result is formatted using the *format*(n) specifier provided or as an unsigned integer (%u) by default.

A number of common polynomials are in use with the CRC algorithm and the most commonly used of these are included in this package. For convenience each of these has a command alias in the crc namespace.

It is possible to implement the CRC-32 checksum using this crc16 package as the implementation is sufficiently generic to extend to 32 bit checksums. As an example this has been done already - however this is not the fastest method to implement this algorithm in Tcl and a separate **crc32** package is available.

Options

-filename *name*
Return a checksum for the file contents instead of for parameter data.

-format *string*
Return the checksum using an alternative format template.

-seed *value*
Select an alternative seed value for the CRC calculation. The default is 0 for the CRC16 calculation and 0xFFFF for the CCITT version. This can be useful for calculating the CRC for data structures without first converting the whole structure into a string. The CRC of the previous member can be used as the seed for calculating the CRC of the next member. It is also used for accumulating a checksum from fragments of a large message (or file)

-implementation *procname*
This hook is provided to allow users to provide their own implementation (perhaps a C compiled extension). The procedure specfied is called with two parameters. The first is the data to be checksummed and the second is the seed value. An integer is expected as the result.

The package provides some implementations of standard CRC polynomials for the XMODEM, CCITT and the usual CRC-16 checksum. For convenience, additional commands have been provided that make use of these implementations.

--
Terminate option processing. Please note that using the option termination flag is important when processing data from parameters. If the binary data looks like one of the options given above then the data will be read as an option if this marker is not included. Always use the -- option termination flag before giving the data argument.

Examples

```
% crc::crc16 -- "Hello, World!"
64077
```

```
% crc::crc-ccitt -- "Hello, World!"
26586
```

```
% crc::crc16 -format 0x%X -- "Hello, World!"
0xFA4D
```

```
% crc::crc16 -file crc16.tcl
51675
```

Authors
Pat Thoyts

Bugs, ideas, feedback
If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *crc* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Hashes, checksums, and encryption

Copyright
```
Copyright (c) 2002, 2017, Pat Thoyts
```

crc32
Keywords: crc32, checksum, cksum, crc, cyclic redundancy check, data integrity, security

Name
crc32 - Perform a 32bit Cyclic Redundancy Check

Synopsis
package require Tcl 8.5 9
package require crc32 ?**1.3.4**?
::crc::crc32 ?*-format format*? ?*-seed value*? [*-channel chan* | *-filename file* | *message*]
::crc::Crc32Init ?*seed*?
::crc::Crc32Update *token data*
::crc::Crc32Final *token*

Description
This package provides a Tcl implementation of the CRC-32 algorithm based upon information

provided at http://www.naaccr.org/standard/crc32/document.html If either the **critcl** package or the **Trf** package are available then a compiled version may be used internally to accelerate the checksum calculation.

Commands

::crc::crc32 ?-format *format*? ?-seed *value*? [*-channel chan* | *-filename file* | *message*]

The command takes either string data or a channel or file name and returns a checksum value calculated using the CRC-32 algorithm. The result is formatted using the *format*(n) specifier provided. The default is to return the value as an unsigned integer (format %u).

Options

-channel *name*

Return a checksum for the data read from a channel. The command will read data from the channel until the **eof** is true. If you need to be able to process events during this calculation see the Programming interface section

-filename *name*

This is a convenience option that opens the specified file, sets the encoding to binary and then acts as if the *-channel* option had been used. The file is closed on completion.

-format *string*

Return the checksum using an alternative format template.

-seed *value*

Select an alternative seed value for the CRC calculation. The default is 0xffffffff. This can be useful for calculating the CRC for data structures without first converting the whole structure into a string. The CRC of the previous member can be used as the seed for calculating the CRC of the next member. Note that the crc32 algorithm includes a final XOR step. If incremental processing is desired then this must be undone before using the output of the algorithm as the seed for further processing. A simpler alternative is to use the Programming interface which is intended for this mode of operation.

Programming interface

The CRC-32 package implements the checksum using a context variable to which additional data can be added at any time. This is expecially useful in an event based environment such as a Tk application or a web server package. Data to be checksummed may be handled incrementally during a **fileevent** handler in discrete chunks. This can improve the interactive nature of a GUI application and can help to avoid excessive memory consumption.

::crc::Crc32Init ?*seed*?

Begins a new CRC32 context. Returns a token ID that must be used for the remaining functions. An optional seed may be specified if required.

::crc::Crc32Update *token data*

Add data to the checksum identified by token. Calling *Crc32Update $token* "*abcd*" is equivalent to calling *Crc32Update $token* "*ab*" followed by *Crc32Update $token* "*cb*". See Examples.

::crc::Crc32Final *token*

Returns the checksum value and releases any resources held by this token. Once this command completes the token will be invalid. The result is a 32 bit integer value.

Examples

```
% crc::crc32 "Hello, World!"
3964322768
```

```
% crc::crc32 -format 0x%X "Hello, World!"
0xEC4AC3D0
```

```
% crc::crc32 -file crc32.tcl
483919716
```

```
% set tok [crc::Crc32Init]
% crc::Crc32Update $tok "Hello, "
% crc::Crc32Update $tok "World!"
% crc::Crc32Final $tok
3964322768
```

Authors

Pat Thoyts

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *crc* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Hashes, checksums, and encryption

Copyright

```
Copyright (c) 2002, Pat Thoyts
```

sum

Keywords: sum, checksum, cksum, crc, crc32, cyclic redundancy check, data integrity, security

Name

sum - Calculate a sum(1) compatible checksum

Synopsis

package require Tcl 8.5 9
package require sum ?**1.1.3**?
::crc::sum ?*-bsd* | *-sysv*? ?*-format fmt*? ?*-chunksize size*? [*-filename file* | *-channel chan* | *string*]

Description

This package provides a Tcl-only implementation of the sum(1) command which calculates a 16 bit checksum value from the input data. The BSD sum algorithm is used by default but the SysV algorithm is also available.

Commands

::crc::sum ?*-bsd* | *-sysv*? ?*-format fmt*? ?*-chunksize size*? [*-filename file* | *-channel chan* | *string*]

The command takes string data or a file name or a channel and returns a checksum value calculated using the **sum(1)** algorithm. The result is formatted using the *format*(n) specifier provided or as an unsigned integer (%u) by default.

Options

-sysv

The SysV algorithm is fairly naive. The byte values are summed and any overflow is discarded. The lowest 16 bits are returned as the checksum. Input with the same content but different ordering will give the same result.

-bsd

This algorithm is similar to the SysV version but includes a bit rotation step which provides a dependency on the order of the data values.

-filename *name*

Return a checksum for the file contents instead of for parameter data.

-channel *chan*

Return a checksum for the contents of the specified channel. The channel must be open for reading and should be configured for binary translation. The channel will no be closed on completion.

-chunksize *size*

Set the block size used when reading data from either files or channels. This value defaults to 4096.

-format *string*

Return the checksum using an alternative format template.

Examples

```
% crc::sum "Hello, World!"
37287
```

```
% crc::sum -format 0x%X "Hello, World!"
0x91A7
```

```
% crc::sum -file sum.tcl
13392
```

Authors

Pat Thoyts

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *crc* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Hashes, checksums, and encryption

Copyright


```
Copyright (c) 2002, Pat Thoyts <patthoyts@users.sourceforge.net>
```


cron

cron

Keywords: cron, odie

Name

cron - Tool for automating the period callback of commands

Synopsis

package require Tcl 8.6 9
package require cron ?**2.2**?
::cron::at ?*processname*? *timecode command*
::cron::cancel *processname*
::cron::every *processname frequency command*
::cron::in ?*processname*? *timecode command*
::cron::object_coroutine *object coroutine* ?*info*?
::cron::sleep *milliseconds*
::cron::task delete *process*
::cron::task exists *process*
::cron::task info *process*
::cron::task set *process field value* ?*field* ...? ?*value* ...?
::cron::wake ?*who*?
::cron::clock_step *milliseconds*
::cron::clock_delay *milliseconds*
::cron::clock_sleep *seconds* ?*offset*?
::cron::clock_set *newtime*

Description

The **cron** package provides a Pure-tcl set of tools to allow programs to schedule tasks to occur at regular intervals. Rather than force each task to issue it's own call to the event loop, the cron

system mimics the cron utility in Unix: on task periodically checks to see if something is to be done, and issues all commands for a given time step at once.

Changes in version 2.0

While cron was originally designed to handle time scales > 1 second, the latest version's internal understand time granularity down to the millisecond, making it easier to integrate with other timed events. Version 2.0 also understands how to properly integrate coroutines and objects. It also adds a facility for an external (or script driven) clock. Note that vwait style events won't work very well with an external clock.

Commands

::cron::at*?processname? timecode command*

This command registers a *command* to be called at the time specified by *timecode*. If *timecode* is expressed as an integer, the timecode is assumed to be in unixtime. All other inputs will be interpreted by **clock scan** and converted to unix time. This task can be modified by subsequent calls to this package's commands by referencing *processname*. If *processname* exists, it will be replaced. If *processname* is not given, one is generated and returned by the command.

```
::cron::at start_coffee {Tomorrow at 9:00am}
    {remote::exec::coffeepot power on}
::cron::at shutdown_coffee {Tomorrow at 12:00pm}
    {remote::exec::coffeepot power off}
```

::cron::cancel *processname*

This command unregisters the process *processname* and cancels any pending commands. Note: processname can be a process created by either **::cron::at** or **::cron::every**.

```
::cron::cancel check_mail
```

::cron::every *processname frequency command*

This command registers a *command* to be called at the interval of *frequency*. *frequency* is given in seconds. This task can be modified by subsequent calls to this package's commands by referencing *processname*. If *processname* exists, it will be replaced.

```
::cron::every check_mail 900  ::imap_client::check_mail
::cron::every backup_db  3600 {::backup_procedure ::mydb}
```

::cron::in*?processname? timecode command*

This command registers a *command* to be called after a delay of time specified by *timecode*. *timecode* is expressed as an seconds. This task can be modified by subsequent calls to this package's commands by referencing *processname*. If *processname* exists, it will be replaced. If *processname* is not given, one is generated and returned by the command.

::cron::object_coroutine *object coroutine?info?*

This command registers a *coroutine*, associated with *object* to be called given the parameters of *info*. If now parameters are given, the coroutine is assumed to be an idle task which will self-terminate. *info* can be given in any form compadible with **::cron::task set**

::cron::sleep *milliseconds*

When run within a coroutine, this command will register the coroutine for a callback at the

appointed time, and immediately yield.

If the ::cron::time variable is > 0 this command will advance the internal time, 100ms at a time.

In all other cases this command will generate a fictious variable, generate an after call, and vwait the variable:

```
set eventid [incr ::cron::eventcount]
set var ::cron::event_#$eventid
set $var 0
::after $ms "set $var 1"
::vwait $var
::unset $var
```

Usage:

```
::cron::sleep 250
```

::cron::task delete *process*
Delete the process specified the *process*

::cron::task exists *process*
Returns true if *process* is registered with cron.

::cron::task info *process*
Returns a dict describing *process*. See **::cron::task set** for a description of the options.

::cron::task set *process field value?field...??value...?*
If *process* does not exist, it is created. Options Include:

command
If **coroutine** is black, a global command which implements this process. If **coroutine** is not black, the command to invoke to create or recreate the coroutine.

coroutine
The name of the coroutine (if any) which implements this process.

frequency
If -1, this process is terminated after the next event. If 0 this process should be called during every idle event. If positive, this process should generate events periodically. The frequency is an integer number of milliseconds between events.

object
The object associated with this process or coroutine.

scheduled
If non-zero, the absolute time from the epoch (in milliseconds) that this process will trigger an event. If zero, and the **frequency** is also zero, this process is called every idle loop.

running
A boolean flag. If true it indicates the process never returned or yielded during the event loop, and will not be called again until it does so.

::cron::wake*?who?*
Wake up cron, and arrange for its event loop to be run during the next Idle cycle.

```
::cron::wake {I just did something important}
```

Several utility commands are provided that are used internally within cron and for testing cron, but may or may not be useful in the general cases.

::cron::clock_step *milliseconds*
: Return a clock time absolute to the epoch which falls on the next border between one second and the next for the value of *milliseconds*

::cron::clock_delay *milliseconds*
: Return a clock time absolute to the epoch which falls on the next border between one second and the next *milliseconds* in the future.

::cron::clock_sleep *seconds?offset?*
: Return a clock time absolute to the epoch which falls exactly *seconds* in the future. If offset is given it may be positive or negative, and will shift the final time to before or after the second would flip.

::cron::clock_set *newtime*
: Sets the internal clock for cron. This command will advance the time in 100ms increment, triggering events, until the internal time catches up with *newtime*.

  newtime is expressed in absolute milliseconds since the beginning of the epoch.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *odie* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

System

Copyright

```
Copyright (c) 2016-2018 Sean Woods <yoda@etoyoc.com>
```

CSV

csv

Keywords: csv, matrix, package, queue, tcllib

Name

csv - Procedures to handle CSV data.

Synopsis

package require Tcl 8.5 9

package require csv ?**0.8.1**?
::csv::iscomplete *data*
::csv::join *values* ?*sepChar*? ?*delChar*? ?*delMode*?
::csv::joinlist *values* ?*sepChar*? ?*delChar*? ?*delMode*?
::csv::joinmatrix *matrix* ?*sepChar*? ?*delChar*? ?*delMode*?
::csv::read2matrix ?**-alternate**? *chan m {sepChar ,} {expand none}*
::csv::read2queue ?**-alternate**? *chan q {sepChar ,}*
::csv::report *cmd matrix* ?*chan*?
::csv::split ?**-alternate**? *line* ?*sepChar*? ?*delChar*?
::csv::split2matrix ?**-alternate**? *m line {sepChar ,} {expand none}*
::csv::split2queue ?**-alternate**? *q line {sepChar ,}*
::csv::writematrix *m chan* ?*sepChar*? ?*delChar*?
::csv::writequeue *q chan* ?*sepChar*? ?*delChar*?

Description

The **csv** package provides commands to manipulate information in CSV Format (CSV = Comma Separated Values).

Commands

The following commands are available:

::csv::iscomplete *data*
: A predicate checking if the argument *data* is a complete csv record. The result is a boolean flag indicating the completeness of the data. The result is true if the data is complete.

::csv::join *values* ?*sepChar*? ?*delChar*? ?*delMode*?
: Takes a list of values and returns a string in CSV format containing these values. The separator character can be defined by the caller, but this is optional. The default is ",". The quoting aka delimiting character can be defined by the caller, but this is optional. The default is '"'. By default the quoting mode *delMode* is "auto", surrounding values with *delChar* only when needed. When set to "always" however, values are always surrounded by the *delChar* instead.

::csv::joinlist *values* ?*sepChar*? ?*delChar*? ?*delMode*?
: Takes a list of lists of values and returns a string in CSV format containing these values. The separator character can be defined by the caller, but this is optional. The default is ",". The quoting character can be defined by the caller, but this is optional. The default is '"'. By default the quoting mode *delMode* is "auto", surrounding values with *delChar* only when needed. When set to "always" however, values are always surrounded by the *delChar* instead. Each element of the outer list is considered a record, these are separated by newlines in the result. The elements of each record are formatted as usual (via **::csv::join**).

::csv::joinmatrix *matrix* ?*sepChar*? ?*delChar*? ?*delMode*?
: Takes a *matrix* object following the API specified for the struct::matrix package and returns a string in CSV format containing these values. The separator character can be defined by the caller, but this is optional. The default is ",". The quoting character can be defined by the caller, but this is optional. The default is '"'. By default the quoting mode *delMode* is "auto", surrounding values with *delChar* only when needed. When set to "always" however, values are always surrounded by the *delChar* instead. Each row of the matrix is considered a record, these are separated by newlines in the result. The elements of each record are formatted as

usual (via **::csv::join**).

::csv::read2matrix ?-alternate? *chan m* {*sepChar*,} {*expand* none}
A wrapper around **::csv::split2matrix** (see below) reading CSV-formatted lines from the specified channel (until EOF) and adding them to the given matrix. For an explanation of the *expand* argument see **::csv::split2matrix**.

::csv::read2queue ?-alternate? *chan q* {*sepChar*,}
A wrapper around **::csv::split2queue** (see below) reading CSV-formatted lines from the specified channel (until EOF) and adding them to the given queue.

::csv::report *cmd matrix* ?*chan*?
A report command which can be used by the matrix methods **format 2string** and **format 2chan**. For the latter this command delegates the work to **::csv::writematrix**. *cmd* is expected to be either **printmatrix** or **printmatrix2channel**. The channel argument, *chan*, has to be present for the latter and must not be present for the first.

::csv::split ?-alternate? *line* ?*sepChar*? ?*delChar*?
converts a *line* in CSV format into a list of the values contained in the line. The character used to separate the values from each other can be defined by the caller, via *sepChar*, but this is optional. The default is ",". The quoting character can be defined by the caller, but this is optional. The default is '"'.

If the option **-alternate** is specified a slightly different syntax is used to parse the input. This syntax is explained below, in the section Format.

::csv::split2matrix ?-alternate? *m line* {*sepChar*,} {*expand* none}
The same as **::csv::split**, but appends the resulting list as a new row to the matrix *m*, using the method **add row**. The expansion mode specified via *expand* determines how the command handles a matrix with less columns than contained in *line*. The allowed modes are:

none
This is the default mode. In this mode it is the responsibility of the caller to ensure that the matrix has enough columns to contain the full line. If there are not enough columns the list of values is silently truncated at the end to fit.

empty
In this mode the command expands an empty matrix to hold all columns of the specified line, but goes no further. The overall effect is that the first of a series of lines determines the number of columns in the matrix and all following lines are truncated to that size, as if mode **none** was set.

auto
In this mode the command expands the matrix as needed to hold all columns contained in *line*. The overall effect is that after adding a series of lines the matrix will have enough columns to hold all columns of the longest line encountered so far.

::csv::split2queue ?-alternate? *q line* {*sepChar*,}
The same as **::csv::split**, but appending the resulting list as a single item to the queue *q*, using the method **put**.

::csv::writematrix *m chan* ?*sepChar*? ?*delChar*?
A wrapper around **::csv::join** taking all rows in the matrix *m* and writing them CSV formatted into the channel *chan*.

::csv::writequeue *q chan* ?*sepChar*? ?*delChar*?
A wrapper around **::csv::join** taking all items in the queue *q* (assumes that they are lists) and writing them CSV formatted into the channel *chan*.

Format

The format of regular CSV files is specified as

1. Each record of a csv file (comma-separated values, as exported e.g. by Excel) is a set of ASCII values separated by ",". For other languages it may be ";" however, although this is not important for this case as the functions provided here allow any separator character.
2. If and only if a value contains itself the separator ",", then it (the value) has to be put between "". If the value does not contain the separator character then quoting is optional.
3. If a value contains the character ", that character is represented by "".
4. The output string "" represents the value ". In other words, it is assumed that it was created through rule 3, and only this rule, i.e. that the value was not quoted.

An alternate format definition mainly used by MS products specifies that the output string "" is a representation of the empty string. In other words, it is assumed that the output was generated out of the empty string by quoting it (i.e. rule 2), and not through rule 3. This is the only difference between the regular and the alternate format.

The alternate format is activated through specification of the option **-alternate** to the various split commands.

Example

Using the regular format the record

```
123,"123,521.2","Mary says ""Hello, I am Mary""",""
```

is parsed into the items

```
a) 123
b) 123,521.2
c) Mary says "Hello, I am Mary"
d) "
```

Using the alternate format the result is

```
a) 123
b) 123,521.2
c) Mary says "Hello, I am Mary"
d) (the empty string)
```

instead. As can be seen only item (d) is different, now the empty string instead of a ".

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *csv* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be

made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Text processing

Copyright
```
Copyright (c) 2002-2015 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

debug

debug
Keywords: debug, log, narrative, trace

Name
debug - debug narrative - core

Synopsis
package require Tcl 8.5 9
package require debug ?**1.0.7**?
debug.tag *message* ?*level*?
debug 2array
debug define *tag*
debug header *text*
debug level *tag* ?*level*? ?*fd*?
debug names
debug off *tag*
debug on *tag*
debug parray *arrayvarname*
debug pdict *dict*
debug hexl *data* ?*prefix*?
debug nl
debug tab
debug prefix *tag* ?*text*?
debug setting (*tag level*) ... ?*fd*?
debug suffix *tag* ?*text*?
debug trailer *text*

Description
Debugging areas of interest are represented by 'tags' which have independently settable levels of interest (an integer, higher is more detailed).

API
debug.tag *message* ?*level*?
For each known tag the package creates a command with this signature the user can then

use to provide the debug narrative of the tag. The narrative *message* is provided as a Tcl script whose value is **subst**ed in the caller's scope if and only if the current level of interest for the *tag* matches or exceeds the call's *level* of detail. This is useful, as one can place arbitrarily complex narrative in code without unnecessarily evaluating it.

See methods **level** and **setting** for querying and manipulating the current level of detail for tags.

The actually printed text consists of not only the *message*, but also global and tag-specific prefix and suffix, should they exist, with each line in the message having the specified headers and trailers.

All these parts are **subst**ableTcl scripts, which are substituted once per message before assembly.

debug 2array

This method returns a dictionary mapping the names of all debug tags currently known to the package to their state and log level. The latter are encoded in a single numeric value, where a negative number indicates an inactive tag at the level given by the absolute value, and a positive number is an active tag at that level.

See also method **settings** below.

debug define *tag*

This method registers the named *tag* with the package. If the tag was not known before it is placed in an inactive state. The state of an already known tag is left untouched.

The result of the method is the empty string.

debug header *text*

This method defines a global **subst**able Tcl script which provides a text printed before each line of output.

Note how this is tag-independent.

Further note that the header substitution happens only once per actual printed message, i.e. all lines of the same message will have the same actual heading text.

The result of the method is the specified text.

debug level *tag* ?*level*? ?*fd*?

This method sets the detail-*level* for the *tag*, and the channel *fd* to write the tags narration into. The level is an integer value >= 0 defaulting to **1**. The channel defaults to **stderr**.

The result of the method is the new detail-level for the tag.

debug names

This method returns a list containing the names of all debug tags currently known to the package.

debug off *tag*

This method registers the named *tag* with the package and sets it inactive.

The result of the method is the empty string.

debug on *tag*

This method registers the named *tag* with the package, as active.

The result of the method is the empty string.

debug parray *arrayvarname*

This is a convenience method formatting the named array like the builtin command **parray**, except it returns the resulting string instead of writing it directly to **stdout**.

This makes it suitable for use in debug messages.

debug pdict *dict*

This is a convenience method formatting the dictionary similarly to how the builtin command **parray** does for array, and returns the resulting string.

This makes it suitable for use in debug messages.

debug hexl *data* ?*prefix*?

This is a convenience method formatting arbitrary data into a hex-dump and returns the resulting string.

This makes it suitable for use in debug messages.

Each line of the dump is prefixed with *prefix*. This prefix defaults to the empty string.

debug nl

This is a convenience method to insert a linefeed character (ASCII 0x0a) into a debug message.

debug tab

This is a convenience method to insert a TAB character (ASCII 0x09) into a debug message.

debug prefix *tag* ?*text*?

This method is similar to the method **header** above, in that it defines **subst**able Tcl script which provides more text for debug messages.

In contrast to **header** the generated text is added to the user's message before it is split into lines, making it a per-message extension.

Furthermore the script is tag-dependent.

In exception to that, a script for tag **::** is applied to all messages.

If both global and tag-dependent prefix exist, both are applied, with the global prefix coming before the tag-dependent prefix.

Note that the prefix substitution happens only once per actual printed message.

The result of the method is the empty string.

If the *tag* was not known at the time of the call it is registered, and set inactive.

debug setting (*tag level*) ... ?*fd*?

This method is a multi-tag variant of method **level** above, with the functionality of methods **on**, and **off** also folded in.

Each named *tag* is set to the detail-*level* following it, with a negative level deactivating the tag, and a positive level activating it.

If the last argument is not followed by a level it is not treated as tag name, but as the channel all the named tags should print their messages to.

The result of the method is the empty string.

debug suffix *tag* ?*text*?

This method is similar to the method **trailer** below, in that it defines **subst**able Tcl script which provides more text for debug messages.

In contrast to **trailer** the generated text is added to the user's message before it is split into lines, making it a per-message extension.

Furthermore the script is tag-dependent.

In exception to that, a script for tag **::** is applied to all messages.

If both global and tag-dependent suffix exist, both are applied, with the global suffix coming after the tag-dependent suffix.

Note that the suffix substitution happens only once per actual printed message.

The result of the method is the empty string.

If the *tag* was not known at the time of the call it is registered, and set inactive.

debug trailer *text*

This method defines a global **subst**able Tcl script which provides a text printed after each line of output (before the EOL however).

Note how this is tag-independent.

Further note that the trailer substitution happens only once per actual printed message, i.e. all lines of the same message will have the same actual trailing text.

The result of the method is the specified text.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you

want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *debug* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

debugging, tracing, and logging

Copyright


```
Copyright (c) 200?, Colin McCormack, Wub Server Utilities
Copyright (c) 2012-2014, Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


debug_caller

Keywords: debug_caller, debug, log, narrative, trace

Name

debug::caller - debug narrative - caller

Synopsis

package require Tcl 8.5 9
package require debug::caller ?**1.2**?
debug caller ?*args...*?

Description

API

debug **caller ?*args*...?**

This method is useful in a tag-specific prefix to automatically provide caller information for all uses of the tag. Or in a message, when only specific places need such detail.

Beyond that it recognizing the various internal forms of method calls generated by the **snit** OO system and rewrites these to their original form, for better readability. Similarly for **TclOO**.

If *args* are specified then they are treated as the integer indices of command arguments to *not* show in the output. The referenced arguments are replaced by * instead. The main anticipated use case for this is the exclusion of arguments expected to contain large Tcl values, i.e. long lists, large dictionaries, etc. to prevent them from overwhelming the narrative.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *debug* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
debugging, tracing, and logging

Copyright

```
Copyright (c) 2012-2015, Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


debug_heartbeat
Keywords: debug_heartbeat, debug, heartbeat, log, narrative, trace

Name
debug::heartbeat - debug narrative - heartbeat

Synopsis
package require Tcl 8.5 9
package require debug::heartbeat ?**1.0.2**?
package require debug ?**1**?
debug heartbeat ?*delta*?

Description

API
debug **heartbeat ?*delta*?**

This method activates or disables a heartbeat with which to monitor the event loop of an event-based Tcl application.

It reserves the debug tag **heartbeat** for its operation and writes a message every *delta* milliseconds.

A *delta*-value <= 0 disables the heartbeat.

The message produced by the heartbeat contains a sequence counter and the time in milliseconds since the last beat, thus providing insight into timing variationsn and deviations from the nominal *delta*.

Bugs, ideas, feedback
If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *debug* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be

made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
debugging, tracing, and logging

Copyright

```
Copyright (c) 200?, Colin McCormack, Wub Server Utilities
Copyright (c) 2012, Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


debug_timestamp
Keywords: debug_timestamp, debug, log, narrative, timestamps, trace

Name
debug::timestamp - debug narrative - timestamping

Synopsis
package require Tcl 8.5 9
package require debug::timestamp ?**1.1**?
package require debug ?**1.0.7**?
debug timestamp

Description

API
debug **timestamp**

This method returns millisecond timing information since a baseline or last call, making it useful in a tag-specific prefix to automatically provide caller information for all uses of the tag. Or in a message, when only specific places need such detail.

Bugs, ideas, feedback
If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *debug* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
debugging, tracing, and logging

Copyright

```
Copyright (c) 200?, Colin McCormack, Wub Server Utilities
Copyright (c) 2012, Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

defer

defer

Keywords: defer, cleanup, golang

Name

defer - Defered execution

Synopsis

package require Tcl 8.6 9
package require defer ?**1.1**?
::defer::defer ?*command*? ?*arg1*? ?*arg2*? ?*argN...*?
::defer::with *variableList script*
::defer::autowith *script*
::defer::cancel ?*id...*?

Description

The **defer** commands allow a developer to schedule actions to happen as part of the current variable scope terminating. This is most useful for dealing with cleanup activities. Since the defered actions always execute, and always execute in the reverse order from which the defer statements themselves execute, the programmer can schedule the cleanup of a resource (for example, a channel) as soon as that resource is acquired. Then, later if the procedure or lambda ends, either due to an error, or an explicit return, the cleanup of that resource will always occur.

Commands

::defer::defer ?*command*? ?*arg1*? ?*arg2*? ?*argN...*?
: Defers execution of some code until the current variable scope ends. Each argument is concatencated together to form the script to execute at deferal time. Multiple defer statements may be used, they are executed in the order of last-in, first-out. The return value is an identifier which can be used later with **defer::cancel**

::defer::with *variableList script*
: Defers execution of a script while copying the current value of some variables, whose names specified in *variableList*, into the script. The script acts like a lambda but executes at the same level as the **defer::with** call. The return value is the same as **::defer::defer**

::defer::autowith *script*
: The same as **::defer::with** but uses all local variables in the variable list.

::defer::cancel ?*id...*?
: Cancels the execution of a defered action. The *id* argument is the identifier returned by **::defer::defer**, **::defer::with**, or **::defer::autowith**. Any number of arguments may be supplied, and all of the IDs supplied will be cancelled.

Examples

```
    package require defer 1
    apply {{} {
        set fd [open /dev/null]
        defer::defer close $fd
    }}
```

References

1. see below ...

Authors

Roy Keene

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *defer* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Utility

Copyright

```
Copyright (c) 2017, Roy Keene
```

des

des

Keywords: des, 3DES, block cipher, data integrity, encryption, security

Name

des - Implementation of the DES and triple-DES ciphers

Synopsis

package require Tcl 8.5 9
package require des 1.2
::DES::des ?*-mode [ecb|cbc|cfb|ofb]*? ?*-dir [encrypt|decrypt]*? *-key keydata* ?*-iv vector*? ?*-hex*? ?*-weak*? ?*-out channel*? ?*-chunksize size*? [*-in channel* | *data*]
::DES::Init *mode keydata iv* ?*weak*?
::DES::Encrypt *Key data*
::DES::Decrypt *Key data*
::DES::Reset *Key iv*

::DES::Final *Key*

Description

This is an implementation in Tcl of the Data Encryption Standard (DES) as published by the U.S. National Institute of Standards and Technology (NIST) [1]. This implementation also supports triple DES (3DES) extension to DES. DES is a 64-bit block cipher that uses a 56-bit key. 3DES uses a 168-bit key. DES has now officially been superceeded by AES but is in common use in many protocols.

The tcllib implementation of DES and 3DES uses an implementation by Mac Cody and is available as a separate download from [2]. For anyone concerned about the details of exporting this code please see the TclDES web pages. The tcllib specific code is a wrapper to the TclDES API that presents same API for the DES cipher as for other ciphers in the library.

Commands

::DES::des ?*-mode [ecb|cbc|cfb|ofb]*? ?*-dir [encrypt|decrypt]*? *-key keydata* ?*-iv vector*? ?*-hex*? ?*-weak*? ?*-out channel*? ?*-chunksize size*? [*-in channel* | *data*]

Perform the **DES** algorithm on either the data provided by the argument or on the data read from the *-in* channel. If an *-out* channel is given then the result will be written to this channel.

The *-key* option must be given. This parameter takes a binary string of 8 bytes in length and is used to generate the key schedule. In DES only 56 bits of key data are used. The highest bit from each byte is discarded.

The *-mode* and *-dir* options are optional and default to cbc mode and encrypt respectively. The initialization vector *-iv* takes an 8 byte binary argument. This defaults to all zeros. See Modes of operation for more about *-mode* and the use of the initialization vector.

DES is a 64-bit block cipher. This means that the data must be provided in units that are a multiple of 8 bytes.

Programming interface

Internal state is maintained in an opaque structure that is returned from the **Init** function. In ECB mode the state is not affected by the input but for other modes some input dependent state is maintained and may be reset by calling the **Reset** function with a new initialization vector value.

::DES::Init *mode keydata iv* ?*weak*?

Construct a new DES key schedule using the specified key data and the given initialization vector. The initialization vector is not used with ECB mode but is important for other usage modes. See Modes of operation.

There are a small number of keys that are known to be weak when used with DES. By default if such a key is passed in then an error will be raised. If there is a need to accept such keys then the *weak* parameter can be set true to avoid the error being thrown.

::DES::Encrypt *Key data*

Use a prepared key acquired by calling **Init** to encrypt the provided data. The data argument should be a binary array that is a multiple of the DES block size of 8 bytes. The result is a

binary array the same size as the input of encrypted data.

::DES::Decrypt *Key data*

Decipher data using the key. Note that the same key may be used to encrypt and decrypt data provided that the initialization vector is reset appropriately for CBC mode.

::DES::Reset *Key iv*

Reset the initialization vector. This permits the programmer to re-use a key and avoid the cost of re-generating the key schedule where the same key data is being used multiple times.

::DES::Final *Key*

This should be called to clean up resources associated with *Key*. Once this function has been called the key may not be used again.

Modes of operation

Electronic Code Book (ECB)

ECB is the basic mode of all block ciphers. Each block is encrypted independently and so identical plain text will produce identical output when encrypted with the same key. Any encryption errors will only affect a single block however this is vulnerable to known plaintext attacks.

Cipher Block Chaining (CBC)

CBC mode uses the output of the last block encryption to affect the current block. An initialization vector of the same size as the cipher block size is used to handle the first block. The initialization vector should be chosen randomly and transmitted as the first block of the output. Errors in encryption affect the current block and the next block after which the cipher will correct itself. CBC is the most commonly used mode in software encryption.

Cipher Feedback (CFB)

CFB mode can be used to convert block ciphers into stream ciphers. In CFB mode the initialization vector is encrypted and the output is then xor'd with the plaintext stream. The result is then used as the initialization vector for the next round. Errors will affect the current block and the next block.

Output Feedback (OFB)

OFB is similar to CFB except that the output of the cipher is fed back into the next round and not the xor'd plain text. This means that errors only affect a single block but the cipher is more vulnerable to attack.

Examples

```
% set ciphertext [DES::des -mode cbc -dir encrypt -key $secret
    $plaintext]
% set plaintext [DES::des -mode cbc -dir decrypt -key $secret
    $ciphertext]
```

```
set iv [string repeat \\0 8]
set Key [DES::Init cbc \\0\\1\\2\\3\\4\\5\\6\\7 $iv]
set ciphertext [DES::Encrypt $Key "somedata"]
append ciphertext [DES::Encrypt $Key "moredata"]
DES::Reset $Key $iv
set plaintext [DES::Decrypt $Key $ciphertext]
DES::Final $Key
```

References

1. "Data Encryption Standard", Federal Information Processing Standards Publication 46-3, 1999, (http://csrc.nist.gov/publications/fips/fips46-3/fips46-3.pdf)
2. "TclDES: munitions-grade Tcl scripting" http://tcldes.sourceforge.net/

Authors

Jochen C Loewer, Mac Cody, Pat Thoyts

Bugs, ideas, Feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *des* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Hashes, checksums, and encryption

Copyright

```
Copyright (c) 2005, Pat Thoyts <patthoyts@users.sourceforge.net>
```

tcldes

Keywords: tcldes, 3DES, DES, block cipher, data integrity, encryption, security

Name

tclDES - Implementation of the DES and triple-DES ciphers

Synopsis

package require Tcl 8.5 9
package require tclDES 1.1

Description

The **tclDES** package is a helper package for **des**.

Please see the documentation of **des** for details.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *des* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Hashes, checksums, and encryption

Copyright


```
Copyright (c) 2005, Pat Thoyts <patthoyts@users.sourceforge.net>
```


tcldesjr

Keywords: tcldesjr, 3DES, DES, block cipher, data integrity, encryption, security

Name

tclDESjr - Implementation of the DES and triple-DES ciphers

Synopsis

package require Tcl 8.5 9
package require tclDESjr 1.1

Description

The **tclDESjr** package is a helper package for **des**.

Please see the documentation of **des** for details.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *des* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Hashes, checksums, and encryption

Copyright


```
Copyright (c) 2005, Pat Thoyts <patthoyts@users.sourceforge.net>
```

dicttool

dicttool

Keywords: dicttool, dict

Name

dicttool - Dictionary Tools

Synopsis

package require Tcl 8.5 9
package require dicttool ?**1.2**?
ladd *dictVar* ?*key value* ...?
ldelete *dictVar* ?*key value* ...?
dict getnull *key*
dict print *aDict*
dict is_dict *value*
dict rmerge *dict1 dict2* ?*dict3* ...?

Description

This package enhances the built-in dict command with several new commands. In addition, the package also defines several convenient list commands. (Each command only adds itself if a command of the same name doesn't already exist, in case any of these are added to the core.)

API

ladd *dictVar* ?*key value ...*?

This command adds every pair of *key value* elements to *dictVar*, but only if that element is not already present (whether as a key or a value). This command *must* be given pairs of *key value* elements to preserve *dictVar* as a **dict**.

ldelete *dictVar* ?*key value ...*?

This command deletes every pair of *key value* elements from *dictVar*. Harmlessly does nothing if the pair is not already in *dictVar*. This command *must* be given pairs of *key value* elements to preserve *dictVar* as a **dict**.

Use the built-in dict command's **dict remove** subcommand to remove elements by key alone.

dict getnull *key*

Operates like **dict get**, however if the key *key* does not exist, it returns an empty list instead of throwing an error.

An alternative to this command is the built-in dict command's **dict getwithdefault** subcommand *or its alias* **dict getdef**. Using **dict getdef** with a default of an empty list, *{}*, is equivalent to using **dict getnull**.

dict print *aDict*

Returns a "pretty printed" string representation of **dict** *aDict*, with each nested branch on a newline, and indented with two spaces for every level.

dict is_dict *value*

Returns **1** (true) if *value* can be interpreted as a **dict**; otherwise **0** (false).

This command does *not* force an existing **dict** representation to change into another internal representation.

dict rmerge *dict1 dict2 ?dict3 ...?*

Returns a **dict** which is the result of a recursive merge of all of the **dict** arguments. Unlike the built-in dict command's **dict merge** subcommand, this command descends into all of the **dict**'s levels. Note that **dict** keys which end in a: indicate a leaf, which will be interpreted as a literal value, and not descended into further.

```
set items [dict rmerge {
  option {color {default: green}}
} {
  option {fruit {default: mango}}
} {
  option {color {default: blue} fruit {widget: select values: {mango
    apple cherry grape}}}
}]
puts [dict print $items]
=>
option {
  color {
    default: blue
  }
  fruit {
    default: mango
    widget: select
    values: {mango apple cherry grape}
  }
}
```

If the built-in dict command's **dict merge** subcommand had been used above, the *fruit* item's *default* key-value entry would have been lost.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *dict* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Utilities

Copyright

```
Copyright (c) 2017 Sean Woods <yoda@etoyoc.com>
```

dns

tcllib_dns

Keywords: tcllib_dns, DNS, domain name service, resolver, rfc 1034, rfc 1035, rfc 1886, rfc 7858

Name

dns - Tcl Domain Name Service Client

Synopsis

package require Tcl 8.5 9
package require dns ?**1.6.1**?
::dns::resolve *query* ?*options*?
::dns::configure ?*options*?
::dns::name *token*
::dns::address *token*
::dns::cname *token*
::dns::result *token*
::dns::status *token*
::dns::error *token*
::dns::reset *token*
::dns::wait *token*
::dns::cleanup *token*
::dns::nameservers

Description

The dns package provides a Tcl only Domain Name Service client. You should refer to (1) and (2) for information about the DNS protocol or read resolver(3) to find out how the C library resolves domain names. The intention of this package is to insulate Tcl scripts from problems with using the system library resolver for slow name servers. It may or may not be of practical use. Internet name resolution is a complex business and DNS is only one part of the resolver. You may find you are supposed to be using hosts files, NIS or WINS to name a few other systems. This package is not a substitute for the C library resolver - it does however implement name resolution over DNS. The package also extends the package **uri** to support DNS URIs (4) of the form *dns:what.host.com* or *dns://my.nameserver/what.host.com*. The **dns::resolve** command can handle DNS URIs or simple domain names as a query.

Note: The package defaults to using DNS over TCP connections. If you wish to use UDP you will need to have the **tcludp** package installed and have a version that correctly handles binary data (> 1.0.4). This is available at http://tcludp.sourceforge.net/. If the **udp** package is present then UDP will be used by default.

Note: The package supports DNS over TLS (RFC 7858) for enhanced privacy of DNS queries. Using this feature requires the TLS package.

Commands

::dns::resolve *query* ?*options*?

Resolve a domain name using the *DNS* protocol. *query* is the domain name to be lookup up. This should be either a fully qualified domain name or a DNS URI.

-nameserver *hostname* or -server *hostname*

Specify an alternative name server for this request.

-protocol *tcp*|*udp*

Specify the network protocol to use for this request. Can be one of *tcp* or *udp*.

-port *portnum*

Specify an alternative port.

-search *domainlist*

see below ...

-timeout *milliseconds*

Override the default timeout.

-type *TYPE*

Specify the type of DNS record you are interested in. Valid values are A, NS, MD, MF, CNAME, SOA, MB, MG, MR, NULL, WKS, PTR, HINFO, MINFO, MX, TXT, SPF, SRV, AAAA, AXFR, MAILB, MAILA and *. See RFC1035 for details about the return values. See http://spf.pobox.com/ about SPF. See (3) about AAAA records and RFC2782 for details of SRV records.

-class *CLASS*

Specify the class of domain name. This is usually IN but may be one of IN for internet domain names, CS, CH, HS or * for any class.

-recurse *boolean*

Set to *false* if you do not want the name server to recursively act upon your request. Normally set to *true*.

-command *procname*

Set a procedure to be called upon request completion. The procedure will be passed the token as its only argument.

-usetls *boolean*

Set the *true* to use DNS over TLS. This will force the use of TCP and change the default port to 853. Certificate validation is required so a source of trusted certificate authority certificates must be provided using *-cafile* or *-cadir*.

-cafile *filepath*

Specify a file containing a collection of trusted certificate authority certficates. See the **update-ca-certificates** command manual page for details or the **-CAfile** option help from **openssl**.

-cadir *dirpath*

Specify a directory containing trusted certificate authority certificates. This must be provided if **-cafile** is not specified for certificate validation to work when **-usetls** is enabled. See the **openssl** documentation for the required structure of this directory.

::dns::configure ?*options*?

The **::dns::configure** command is used to setup the dns package. The server to query, the protocol and domain search path are all set via this command. If no arguments are provided

then a list of all the current settings is returned. If only one argument then it must the the name of an option and the value for that option is returned.

-nameserver *hostname*
Set the default name server to be used by all queries. The default is *localhost*.

-protocol *tcp*|*udp*
Set the default network protocol to be used. Default is *tcp*.

-port *portnum*
Set the default port to use on the name server. The default is 53.

-search *domainlist*
Set the domain search list. This is currently not used.

-timeout *milliseconds*
Set the default timeout value for DNS lookups. Default is 30 seconds.

-loglevel *level*
Set the log level used for emitting diagnostic messages from this package. The default is *warn*. See the **log** package for details of the available levels.

-cafile *filepath*
Set the default file path to be used for the **-cafile** option to **dns::resolve**.

-cadir *dirpath*
Set the default directory path to be used for the **-cadir** option to **dns::resolve**.

::dns::name *token*
Returns a list of all domain names returned as an answer to your query.

::dns::address *token*
Returns a list of the address records that match your query.

::dns::cname *token*
Returns a list of canonical names (usually just one) matching your query.

::dns::result *token*
Returns a list of all the decoded answer records provided for your query. This permits you to extract the result for more unusual query types.

::dns::status *token*
Returns the status flag. For a successfully completed query this will be *ok*. May be *error* or *timeout* or *eof*. See also **::dns::error**

::dns::error *token*
Returns the error message provided for requests whose status is *error*. If there is no error message then an empty string is returned.

::dns::reset *token*
Reset or cancel a DNS query.

::dns::wait *token*
Wait for a DNS query to complete and return the status upon completion.

::dns::cleanup *token*
Remove all state variables associated with the request.

::dns::nameservers
Attempts to return a list of the nameservers currently configured for the users system. On a unix machine this parses the /etc/resolv.conf file for nameservers (if it exists) and on Windows

systems we examine certain parts of the registry. If no nameserver can be found then the loopback address (127.0.0.1) is used as a default.

Examples

```
% set tok [dns::resolve www.tcl.tk]
::dns::1
% dns::status $tok
ok
% dns::address $tok
199.175.6.239
% dns::name $tok
www.tcl.tk
% dns::cleanup $tok
```

Using DNS URIs as queries:

```
% set tok [dns::resolve "dns:tcl.tk;type=MX"]
% set tok [dns::resolve "dns://l.root-servers.net/www.tcl.tk"]
```

Reverse address lookup:

```
% set tok [dns::resolve 127.0.0.1]
::dns::1
% dns::name $tok
localhost
% dns::cleanup $tok
```

Using DNS over TLS (RFC 7858):

```
% set tok [dns::resolve www.tcl.tk -nameserver
    dns-tls.bitwiseshift.net  -usetls 1 -cafile
    /etc/ssl/certs/ca-certificates.crt]
::dns::12
% dns::wait $tok
ok
% dns::address $tok
104.25.119.118 104.25.120.118
```

References

1. Mockapetris, P., "Domain Names - Concepts and Facilities", RFC 1034, November 1987. (http://www.ietf.org/rfc/rfc1034.txt)
2. Mockapetris, P., "Domain Names - Implementation and Specification", RFC 1035, November 1087. (http://www.ietf.org/rfc/rfc1035.txt)
3. Thompson, S. and Huitema, C., "DNS Extensions to support IP version 6", RFC 1886, December 1995. (http://www.ietf.org/rfc/rfc1886.txt)
4. Josefsson, S., "Domain Name System Uniform Resource Identifiers", Internet-Draft, October 2003, (http://www.ietf.org/internet-drafts/draft-josefsson-dns-url-09.txt)
5. Gulbrandsen, A., Vixie, P. and Esibov, L., "A DNS RR for specifying the location of services (DNS SRV)", RFC 2782, February 2000, (http://www.ietf.org/rfc/rfc2782.txt)

6. Ohta, M. "Incremental Zone Transfer in DNS", RFC 1995, August 1996, (http://www.ietf.org/rfc/rfc1995.txt)
7. Hu, Z., etc al. "Specification for DNS over Transport Layer Security (TLS)", RFC 7858, May 2016, (http://www.ietf.org/rfc/rfc7858.txt)

Authors
Pat Thoyts

Bugs, ideas, feedback
If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *dns* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Networking

Copyright
```
Copyright (c) 2002, Pat Thoyts
```

tcllib_ip
Keywords: tcllib_ip, internet address, ip, ipv4, ipv6, rfc 3513

Name
tcllib_ip - IPv4 and IPv6 address manipulation

Synopsis
package require Tcl 8.5 9
package require ip ?**1.5.1**?
::ip::version *address*
::ip::is *class address*
::ip::equal *address address*
::ip::normalize *address*
::ip::contract *address*
::ip::distance *ipaddr1 ipaddr2*
::ip::nextIp *ipaddr* ?*offset*?
::ip::prefix *address*
::ip::type *address*
::ip::mask *address*
::ip::prefixToNative *prefix*
::ip::nativeToPrefix *nativeList*|*native* ?**-ipv4**?
::ip::intToString *number* ?**-ipv4**?
::ip::toInteger *ipaddr*

::ip::toHex *ipaddr*
::ip::maskToInt *ipmask*
::ip::broadcastAddress *prefix* ?**-ipv4**?
::ip::maskToLength *dottedMask|integerMask|hexMask* ?**-ipv4**?
::ip::lengthToMask *maskLength* ?**-ipv4**?
::ip::nextNet *ipaddr ipmask* ?*count*? ?**-ipv4**?
::ip::isOverlap *prefix prefix...*
::ip::isOverlapNative ?**-all**? ?**-inline**? ?**-ipv4**? *hexipaddr hexipmask hexiplist*
::ip::ipToLayer2Multicast *ipaddr*
::ip::ipHostFromPrefix *prefix* ?*-exclude prefixExcludeList*?
::ip::reduceToAggregates *prefixlist*
::ip::longestPrefixMatch *ipaddr prefixlist* ?**-ipv4**?
::ip::collapse *prefixlist*
::ip::subtract *prefixlist*

Description

This package provides a set of commands to help in parsing, displaying and comparing internet addresses. The package can handle both IPv4 (1) and IPv6 (2) address types.

Commands

::ip::version *address*
Returns the protocol version of the address (**4** or **6**), or **-1** if the address is neither IPv4 or IPv6.

::ip::is *class address*
Returns true if the address is a member of the given protocol class. The class parameter may be either **ipv4** or **ipv6** This is effectively a boolean equivalent of the **version** command. The *class* argument may be shortened to **4** or **6**.

::ip::equal *address address*
Compare two address specifications for equivalence. The arguments are normalized and the address prefix determined (if a mask is supplied). The normalized addresses are then compared bit-by-bit and the procedure returns true if they match.

::ip::normalize *address*
Convert an IPv4 or IPv6 address into a fully expanded version. There are various shorthand ways to write internet addresses, missing out redundant parts or digits. This procedure is the opposite of **contract**.

::ip::contract *address*
Convert a **normalize**d internet address into a more compact form suitable for displaying to users.

::ip::distance *ipaddr1 ipaddr2*
This command computes the (integer) distance from IPv4 address *ipaddr1* to IPv4 address *ipaddr2*, i.e. "ipaddr2 - ipaddr1"

```
% ::ip::distance 1.1.1.1  1.1.1.5
4
```

::ip::nextIp *ipaddr* ?*offset*?

This command adds the integer *offset* to the IPv4 address *ipaddr* and returns the new IPv4 address.

```
% ::ip::distance 1.1.1.1  4
1.1.1.5
```

::ip::prefix *address*
Returns the address prefix generated by masking the address part with the mask if provided. If there is no mask then it is equivalent to calling **normalize**

::ip::type *address*
see below ...

::ip::mask *address*
If the address supplied includes a mask then this is returned otherwise returns an empty string.

::ip::prefixToNative *prefix*
This command converts the string *prefix* from dotted form (/ format) to native (hex) form. Returns a list containing two elements, ipaddress and mask, in this order, in hexadecimal notation.

```
% ip::prefixToNative 1.1.1.0/24
0x01010100 0xffffff00
```

::ip::nativeToPrefix *nativeList*|*native* ?-ipv4?
This command converts from native (hex) form to dotted form. It is the complement of **::ip::prefixToNative**.

list *nativeList* (in)
List of several ip addresses in native form. The native form is a list as returned by **::ip::prefixToNative**.

list *native* (in)
A list as returned by **::ip::prefixToNative**.

The command returns a list of addresses in dotted form if it was called with a list of addresses. Otherwise a single address in dotted form is returned.

```
% ip::nativeToPrefix {0x01010100 0xffffff00} -ipv4
1.1.1.0/24
```

::ip::intToString *number* ?-ipv4?
This command converts from an ip address specified as integer number to dotted form.

```
      ip::intToString 4294967295
      255.255.255.255
```

::ip::toInteger *ipaddr*
This command converts a dotted form ip into an integer number.

```
% ::ip::toInteger 1.1.1.0
16843008
```

::ip::toHex *ipaddr*
This command converts dotted form ip into a hexadecimal number.

```
% ::ip::toHex 1.1.1.0
0x01010100
```

::ip::maskToInt *ipmask*
This command convert an ipmask in either dotted (255.255.255.0) form or mask length form (24) into an integer number.

```
::ip::maskToInt 24
4294967040
```

::ip::broadcastAddress *prefix* ?-ipv4?
This commands returns a broadcast address in dotted form for the given route *prefix*, either in the form "addr/mask", or in native form. The result is in dotted form.

```
::ip::broadcastAddress 1.1.1.0/24
1.1.1.255

::ip::broadcastAddress {0x01010100 0xffffff00}
0x010101ff
```

::ip::maskToLength *dottedMask*|*integerMask*|*hexMask* ?-ipv4?
This command converts the dotted or integer form of an ipmask to the mask length form.

```
::ip::maskToLength 0xffffff00 -ipv4
24

% ::ip::maskToLength 255.255.255.0
24
```

::ip::lengthToMask *maskLength* ?-ipv4?
This command converts an ipmask in mask length form to its dotted form.

```
::ip::lengthToMask 24
255.255.255.0
```

::ip::nextNet *ipaddr ipmask* ?*count*? ?-ipv4?
This command returns an ipaddress in the same position in the *count* next network. The default value for *count* is **1**.

The address can be specified as either integer number or in dotted form. The mask can be specified as either integer number, dotted form, or mask length form.

The result is in hex form.

::ip::isOverlap *prefix prefix*...
This command checks if the given ip prefixes overlap. All arguments are in dotted "addr/mask" form. All arguments after the first prefix are compared against the first prefix.

The result is a boolean value. It is true if an overlap was found for any of the prefixes.

```
% ::ip::isOverlap 1.1.1.0/24 2.1.0.1/32
0

::ip::isOverlap 1.1.1.0/24 2.1.0.1/32 1.1.1.1/32
1
```

::ip::isOverlapNative ?-all? ?-inline? ?-ipv4? *hexipaddr hexipmask hexiplist*

This command is similar to **::ip::isOverlap**, however the arguments are in the native form, and the form of the result is under greater control of the caller. If the option **-all** is specified it checks all addresses for overlap, not only until the first one is found. If the option **-inline** is specified the command returns the overlapping prefix instead of index values.

The result of the command is, depending on the specified options,

no options
: The index of the first overlap found, or 0 if there is none.

-all
: A list containing the indices of all overlaps found, or an empty list if there are none.

-inline
: The first overlapping prefix, or an empoty string if there is none.

-all -inline
: A list containing the prefixes of all overlaps found, or an empty list if there are none.

```
% ::ip::isOverlapNative 0x01010100 0xffffff00 {{0x02010001
  0xffffffff}}
0

% ::ip::isOverlapNative 0x01010100 0xffffff00 {{0x02010001
  0xffffffff} {0x01010101 0xffffffff}}
2
```

::ip::ipToLayer2Multicast *ipaddr*

This command an converts ipv4 address in dotted form into a layer 2 multicast address, also in dotted form.

```
% ::ip::ipToLayer2Multicast 224.0.0.2
01.00.5e.00.00.02
```

::ip::ipHostFromPrefix *prefix* ?-exclude *prefixExcludeList*?

This command returns a host address from a prefix in the form "ipaddr/masklen", also making sure that the result is not an address found in the *prefixExcludeList*. The result is an ip address in dotted form.

```
%::ip::ipHostFromPrefix  1.1.1.5/24
1.1.1.1

%::ip::ipHostFromPrefix  1.1.1.1/32
1.1.1.1
```

::ip::reduceToAggregates *prefixlist*

This command finds nets that overlap and filters out the more specific nets. The prefixes are in either addr/mask form or in native format. The result is a list containing the non-overlapping ip prefixes from the input.

```
% ::ip::reduceToAggregates {1.1.1.0/24 1.1.0.0/8  2.1.1.0/24
  1.1.1.1/32 }
1.0.0.0/8 2.1.1.0/24
```

::ip::longestPrefixMatch *ipaddr prefixlist* ?-ipv4?

This command finds longest prefix match from set of prefixes, given a specific host address. The prefixes in the list are in either native or dotted form, whereas the host address is in either ipprefix format, dotted form, or integer form. The result is the prefix which is the most specific match to the host address.

```
% ::ip::longestPrefixMatch 1.1.1.1 {1.1.1.0/24 1.0.0.0/8
  2.1.1.0/24 1.1.1.0/28 }
1.1.1.0/28
```

::ip::collapse *prefixlist*

This commands takes a list of prefixes and returns a list prefixes with the largest possible subnet masks covering the input, in this manner collapsing adjacent prefixes into larger ranges.

This is different from **::ip::reduceToAggregates** in that the latter only removes specific nets from a list when they are covered by other elements of the input whereas this command actively merges nets into larger ranges when they are adjacent to each other.

```
% ::ip::collapse {1.2.2.0/24 1.2.3.0/24}
1.2.2.0/23
```

::ip::subtract *prefixlist*

This command takes a list of prefixes, some of which are prefixed by a dash. These latter *negative* prefixes are used to punch holes into the ranges described by the other, *positive*, prefixes. I.e. the negative prefixes are subtracted frrom the positive ones, resulting in a larger list of describes describing the covered ranges only as positives.

Examples

```
% ip::version ::1
6
% ip::version 127.0.0.1
4
```

```
% ip::normalize 127/8
127.0.0.0/8
% ip::contract 192.168.0.0
192.168
%
% ip::normalize fec0::1
fec0:0000:0000:0000:0000:0000:0000:0001
% ip::contract fec0:0000:0000:0000:0000:0000:0000:0001
fec0::1
```

```
% ip::equal 192.168.0.4/16 192.168.0.0/16
1
% ip::equal fec0::1/10 fec0::fe01/10
1
```

References

1. Postel, J. "Internet Protocol." RFC 791, September 1981, (http://www.ietf.org/rfc/rfc791.txt)
2. Hinden, R. and Deering, S., "Internet Protocol Version 6 (IPv6) Addressing Architecture", RFC 3513, April 2003 (http://www.ietf.org/rfc/rfc3513.txt)

Authors

Pat Thoyts

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *dns* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Networking

Copyright

```
Copyright (c) 2004, Pat Thoyts
Copyright (c) 2005 Aamer Akhter <aakhter@cisco.com>
```

docstrip

docstrip

Keywords: docstrip, .dtx, LaTeX, documentation, literate programming, source

Name

docstrip - Docstrip style source code extraction

Synopsis

package require Tcl 8.5 9
package require docstrip ?**1.3**?
docstrip::extract *text terminals* ?*option value* ...?
docstrip::sourcefrom *filename terminals* ?*option value* ...?

Description

Docstrip is a tool created to support a brand of Literate Programming. It is most common in the (La)TeX community, where it is being used for pretty much everything from the LaTeX core and up, but there is nothing about **docstrip** which prevents using it for other types of software.

In short, the basic principle of literate programming is that program source should primarily be written and structured to suit the developers (and advanced users who want to peek "under the hood"), not to suit the whims of a compiler or corresponding source code consumer. This means literate sources often need some kind of "translation" to an illiterate form that dumb software can understand. The **docstrip** Tcl package handles this translation.

Even for those who do not whole-hartedly subscribe to the philosophy behind literate programming, **docstrip** can bring greater clarity to in particular:

- programs employing non-obvious mathematics
- projects where separate pieces of code, perhaps in different languages, need to be closely coordinated.

The first is by providing access to much more powerful typographical features for source code comments than are possible in plain text. The second is because all the separate pieces of code can be kept next to each other in the same source file.

The way it works is that the programmer edits directly only one or several "master" source code files, from which **docstrip** generates the more traditional "source" files compilers or the like would expect. The master sources typically contain a large amount of documentation of the code, sometimes even in places where the code consumers would not allow any comments. The etymology of "docstrip" is that this *doc*umentation was *strip*ped away (although "code extraction" might be a better description, as it has always been a matter of copying selected pieces of the master source rather than deleting text from it). The **docstrip** Tcl package contains a reimplementation of the basic extraction functionality from the **docstrip** program, and thus makes it possible for a Tcl interpreter to read and interpret the master source files directly.

Readers who are not previously familiar with **docstrip** but want to know more about it may consult the following sources.

1. *The tclldoc package and class*, http://ctan.org/tex-archive/macros/latex/contrib/tclldoc/.
2. *The DocStrip utility*, http://ctan.org/tex-archive/macros/latex/base/docstrip.dtx.
3. *The doc and shortvrb Packages*, http://ctan.org/tex-archive/macros/latex/base/doc.dtx.
4. Chapter 14 of *The LaTeX Companion* (second edition), Addison-Wesley, 2004; ISBN 0-201-36299-6.

File format

The basic unit **docstrip** operates on are the *lines* of a master source file. Extraction consists of selecting some of these lines to be copied from input text to output text. The basic distinction is

that between *code lines* (which are copied and do not begin with a percent character) and *comment lines* (which begin with a percent character and are not copied).

```
docstrip::extract [join {
  {% comment}
  {% more comment !"#$%&/(}
  {some command}
  { % blah $blah "Not a comment."}
  {% abc; this is comment}
  {# def; this is code}
  {ghi}
  {% jkl}
} \n] {}
```

returns the same sequence of lines as

```
join {
  {some command}
  { % blah $blah "Not a comment."}
  {# def; this is code}
  {ghi} ""
} \n
```

It does not matter to **docstrip** what format is used for the documentation in the comment lines, but in order to do better than plain text comments, one typically uses some markup language. Most commonly LaTeX is used, as that is a very established standard and also provides the best support for mathematical formulae, but the **docstrip::util** package also gives some support for *doctools*-like markup.

Besides the basic code and comment lines, there are also *guard lines*, which begin with the two characters '%<', and *meta-comment lines*, which begin with the two characters '%%'. Within guard lines there is furthermore the distinction between *verbatim guard lines*, which begin with '%<<', and ordinary guard lines, where the '%<' is not followed by another '<'. The last category is by far the most common.

Ordinary guard lines conditions extraction of the code line(s) they guard by the value of a boolean expression; the guarded block of code lines will only be included if the expression evaluates to true. The syntax of an ordinary guard line is one of

```
 '%' '<' STARSLASH EXPRESSION '>'
 '%' '<' PLUSMINUS EXPRESSION '>' CODE
```

where

```
 STARSLASH  ::=  '*' | '/'
 PLUSMINUS  ::=  | '+' | '-'
 EXPRESSION ::= SECONDARY | SECONDARY ',' EXPRESSION
              | SECONDARY '|' EXPRESSION
 SECONDARY  ::= PRIMARY | PRIMARY '&' SECONDARY
 PRIMARY    ::= TERMINAL | '!' PRIMARY | '(' EXPRESSION ')'
 CODE       ::= { any character except end-of-line }
```

Comma and vertical bar both denote 'or'. Ampersand denotes 'and'. Exclamation mark denotes 'not'. A TERMINAL can be any nonempty string of characters not containing '>', '&', '|', comma, '(', or ')', although the **docstrip** manual is a bit restrictive and only guarantees proper operation for strings of letters (although even the LaTeX core sources make heavy use also of digits in TERMINALs). The second argument of **docstrip::extract** is the list of those TERMINALs that should count as having the value 'true'; all other TERMINALs count as being 'false' when guard expressions are evaluated.

In the case of a '%<*_EXPRESSION_>' guard, the lines guarded are all lines up to the next '%</_EXPRESSION_>' guard with the same *EXPRESSION* (compared as strings). The blocks of code delimited by such '*' and '/' guard lines must be properly nested.

```
set text [join {
   {begin}
   {%<*foo>}
   {1}
   {%<*bar>}
   {2}
   {%</bar>}
   {%<*!bar>}
   {3}
   {%</!bar>}
   {4}
   {%</foo>}
   {5}
   {%<*bar>}
   {6}
   {%</bar>}
   {end}
} \n]
set res [docstrip::extract $text foo]
append res [docstrip::extract $text {foo bar}]
append res [docstrip::extract $text bar]
```

sets $res to the result of

```
join {
   {begin}
   {1}
   {3}
   {4}
   {5}
   {end}
   {begin}
   {1}
   {2}
   {4}
   {5}
```

```
      {6}
      {end}
      {begin}
      {5}
      {6}
      {end} ""
   } \n
```

In guard lines without a '*', '/', '+', or '-' modifier after the '%<', the guard applies only to the CODE following the '>' on that single line. A '+' modifier is equivalent to no modifier. A '-' modifier is like the case with no modifier, but the expression is implicitly negated, i.e., the CODE of a '%<-' guard line is only included if the expression evaluates to false.

Metacomment lines are "comment lines which should not be stripped away", but be extracted like code lines; these are sometimes used for copyright notices and similar material. The '%%' prefix is however not kept, but substituted by the current **-metaprefix**, which is customarily set to some "comment until end of line" character (or character sequence) of the language of the code being extracted.

```
   set text [join {
      {begin}
      {%<foo> foo}
      {%<+foo>plusfoo}
      {%<-foo>minusfoo}
      {middle}
      {%% some metacomment}
      {%<*foo>}
      {%%another metacomment}
      {%</foo>}
      {end}
   } \n]
   set res [docstrip::extract $text foo -metaprefix {# }]
   append res [docstrip::extract $text bar -metaprefix {#}]
```

sets $res to the result of

```
   join {
      {begin}
      { foo}
      {plusfoo}
      {middle}
      {#  some metacomment}
      {# another metacomment}
      {end}
      {begin}
      {minusfoo}
      {middle}
      {# some metacomment}
      {end} ""
   } \n
```

Verbatim guards can be used to force code line interpretation of a block of lines even if some of them happen to look like any other type of lines to docstrip. A verbatim guard has the form '%<<*END-TAG*' and the verbatim block is terminated by the first line that is exactly '%*END-TAG*'.

```
   set text [join {
      {begin}
      {%<*myblock>}
      {some stupid()}
      {   #computer<program>}
      {%<<QQQ-98765}
      {% These three lines are copied verbatim (including percents}
      {%% even if -metaprefix is something different than %%).}
      {%</myblock>}
      {%QQQ-98765}
      {   using*strange@programming<language>}
      {%</myblock>}
      {end}
   } \n]
   set res [docstrip::extract $text myblock -metaprefix {# }]
   append res [docstrip::extract $text {}]
```

sets $res to the result of

```
  join {
     {begin}
     {some stupid()}
     {   #computer<program>}
     {% These three lines are copied verbatim (including percents}
     {%% even if -metaprefix is something different than %%).}
     {%</myblock>}
     {   using*strange@programming<language>}
     {end}
     {begin}
     {end} ""
  } \n
```

The processing of verbatim guards takes place also inside blocks of lines which due to some outer block guard will not be copied.

The final piece of **docstrip** syntax is that extraction stops at a line that is exactly "\endinput"; this is often used to avoid copying random whitespace at the end of a file. In the unlikely case that one wants such a code line, one can protect it with a verbatim guard.

Commands

The package defines two commands.

docstrip::extract *text terminals* ?*option value* ...?

The **extract** command docstrips the *text* and returns the extracted lines of code, as a string with each line terminated with a newline. The *terminals* is the list of those guard expression terminals which should evaluate to true. The available options are:

-annotate *lines*

Requests the specified number of lines of annotation to follow each extracted line in the result. Defaults to 0. Annotation lines are mostly useful when the extracted lines are to undergo some further transformation. A first annotation line is a list of three elements: line type, prefix removed in extraction, and prefix inserted in extraction. The line type is one of: 'V' (verbatim), 'M' (metacomment), '+' (+ or no modifier guard line), '-' (- modifier guard line), '.' (normal line). A second annotation line is the source line number. A third annotation line is the current stack of block guards. Requesting more than three lines of annotation is currently not supported.

-metaprefix *string*

The string by which the '%%' prefix of a metacomment line will be replaced. Defaults to '%%'. For Tcl code this would typically be '#'.

-onerror *keyword*

Controls what will be done when a format error in the *text* being processed is detected. The settings are:

ignore

Just ignore the error; continue as if nothing happened.

puts

Write an error message to **stderr**, then continue processing.

throw

Throw an error. The **-errorcode** is set to a list whose first element is **DOCSTRIP**, second element is the type of error, and third element is the line number where the error is detected. This is the default.

-trimlines ***boolean***
Controls whether *spaces* at the end of a line should be trimmed away before the line is processed. Defaults to true.

It should be remarked that the *terminals* are often called "options" in the context of the **docstrip** program, since these specify which optional code fragments should be included.

docstrip::sourcefrom ***filename terminals*** **?*option value* ...?**
The **sourcefrom** command is a docstripping emulation of **source**. It opens the file *filename*, reads it, closes it, docstrips the contents as specified by the *terminals*, and evaluates the result in the local context of the caller, during which time the **info script** value will be the *filename*. The options are passed on to **fconfigure** to configure the file before its contents are read. The **-metaprefix** is set to '#', all other **extract** options have their default values.

Document structure

The file format (as described above) determines whether a master source code file can be processed correctly by **docstrip**, but the usefulness of the format is to no little part also dependent on that the code and comment lines together constitute a well-formed document.

For a document format that does not require any non-Tcl software, see the **ddt2man** command in the **docstrip::util** package. It is suggested that files employing that document format are given the suffix "*.ddt*", to distinguish them from the more traditional LaTeX-based "*.dtx*" files.

Master source files with "*.dtx*" extension are usually set up so that they can be typeset directly by **latex** without any support from other files. This is achieved by beginning the file with the lines

```
% \iffalse
%<*driver>
\documentclass{tclldoc}
\begin{document}
\DocInput{filename.dtx}
\end{document}
%</driver>
% \fi
```

or some variation thereof. The trick is that the file gets read twice. With normal LaTeX reading rules, the first two lines are comments and therefore ignored. The third line is the document preamble, the fourth line begins the document body, and the sixth line ends the document, so LaTeX stops there — non-comments below that point in the file are never subjected to the normal LaTeX reading rules. Before that, however, the \DocInput command on the fifth line is processed, and that does two things: it changes the interpretation of '%' from "comment" to "ignored", and it inputs the file specified in the argument (which is normally the name of the file the command is in). It is this second time that the file is being read that the comments and code in it are typeset.

The function of the \iffalse ... \fi is to skip lines two to seven on this second time through; this is

similar to the "if 0 { ... }" idiom for block comments in Tcl code, and it is needed here because (amongst other things) the \documentclass command may only be executed once. The function of the guards is to prevent this short piece of LaTeX code from being extracted by **docstrip**. The total effect is that the file can function both as a LaTeX document and as a **docstrip** master source code file.

It is not necessary to use the tclldoc document class, but that does provide a number of features that are convenient for "*.dtx*" files containing Tcl code. More information on this matter can be found in the references above.

Category
Documentation tools

Copyright

```
Copyright (c) 2003–2010 Lars Hellström <Lars dot Hellstrom at
    residenset dot net>
```


docstrip_util
Keywords: docstrip_util, .ddt, .dtx, LaTeX, Tcl module, catalogue, diff, docstrip, doctools, documentation, literate programming, module, package indexing, patch, source

Name
docstrip_util - Docstrip-related utilities

Synopsis
package require Tcl 8.5 9
package require docstrip ?**1.3**?
package require docstrip::util ?**1.3.3**?
pkgProvide *name version terminals*
pkgIndex ?*terminal* ...?
fileoptions ?*option value* ...?
docstrip::util::index_from_catalogue *dir pattern* ?*option value* ...?
docstrip::util::modules_from_catalogue *target source* ?*option value* ...?
docstrip::util::classical_preamble *metaprefix message target* ?*source terminals* ...?
docstrip::util::classical_postamble *metaprefix message target* ?*source terminals* ...?
docstrip::util::packages_provided *text* ?*setup-script*?
docstrip::util::ddt2man *text*
docstrip::util::guards *subcmd text*
docstrip::util::patch *source-var terminals fromtext diff* ?*option value* ...?
docstrip::util::thefile *filename* ?*option value* ...?
docstrip::util::import_unidiff *diff-text* ?*warning-var*?

Description
The **docstrip::util** package is meant for collecting various utility procedures that are mainly useful at installation or development time. It is separate from the base package to avoid overhead when the latter is used to **source** code.

Package indexing commands

Like raw "*.tcl*" files, code lines in docstrip source files can be searched for package declarations and corresponding indices constructed. A complication is however that one cannot tell from the code blocks themselves which will fit together to make a working package; normally that information would be found in an accompanying "*.ins*" file, but parsing one of those is not an easy task. Therefore **docstrip::util** introduces an alternative encoding of such information, in the form of a declarative Tcl script: the *catalogue* (of the contents in a source file).

The special commands which are available inside a catalogue are:

pkgProvide *name version terminals*
: Declares that the code for a package with name *name* and version *version* is made up from those modules in the source file which are selected by the *terminals* list of guard expression terminals. This code should preferably not contain a **package provide** command for the package, as one will be provided by the package loading mechanisms.

pkgIndex ?*terminal* ...?
: Declares that the code for a package is made up from those modules in the source file which are selected by the listed guard expression *terminal*s. The name and version of this package is determined from **package provide** command(s) found in that code (hence there must be such a command in there).

fileoptions ?*option value* ...?
: Declares the **fconfigure** options that should be in force when reading the source; this can usually be ignored for pure ASCII files, but if the file needs to be interpreted according to some other **-encoding** then this is how to specify it. The command should normally appear first in the catalogue, as it takes effect only for commands following it.

Other Tcl commands are supported too — a catalogue is parsed by being evaluated in a safe interpreter — but they are rarely needed. To allow for future extensions, unknown commands in the catalogue are silently ignored.

To simplify distribution of catalogues together with their source files, the catalogue is stored *in the source file itself* as a module selected by the terminal '**docstrip.tcl::catalogue**'. This supports both the style of collecting all catalogue lines in one place and the style of putting each catalogue line in close proximity of the code that it declares.

Putting catalogue entries next to the code they declare may look as follows

```
%    First there's the catalogue entry
%    \begin{tcl}
%<docstrip.tcl::catalogue>pkgProvide foo::bar 1.0 {foobar load}
%    \end{tcl}
%    second a metacomment used to include a copyright message
%    \begin{macrocode}
%<*foobar>
%% This file is placed in the public domain.
%    \end{macrocode}
%    third the package implementation
%    \begin{tcl}
namespace eval foo::bar {
```

```
   # ... some clever piece of Tcl code elided ...
%    \end{tcl}
%    which at some point may have variant code to make use of a
%    |load|able extension
%    \begin{tcl}
%<*load>
   load [file rootname [info script]][info sharedlibextension]
%</load>
%<*!load>
   # ... even more clever scripted counterpart of the extension
   # also elided ...
%</!load>
}
%</foobar>
%    \end{tcl}
%    and that's it!
```

The corresponding set-up with **pkgIndex** would be

```
%    First there's the catalogue entry
%    \begin{tcl}
%<docstrip.tcl::catalogue>pkgIndex foobar load
%    \end{tcl}
%    second a metacomment used to include a copyright message
%    \begin{tcl}
%<*foobar>
%% This file is placed in the public domain.
%    \end{tcl}
%    third the package implementation
%    \begin{tcl}
package provide foo::bar 1.0
namespace eval foo::bar {
   # ... some clever piece of Tcl code elided ...
%    \end{tcl}
%    which at some point may have variant code to make use of a
%    |load|able extension
%    \begin{tcl}
%<*load>
   load [file rootname [info script]][info sharedlibextension]
%</load>
%<*!load>
   # ... even more clever scripted counterpart of the extension
   # also elided ...
%</!load>
}
%</foobar>
%    \end{tcl}
%    and that's it!
```

docstrip::util::index_from_catalogue *dir pattern ?option value ...?*

This command is a sibling of the standard **pkg_mkIndex** command, in that it adds package entries to "*pkgIndex.tcl*" files. The difference is that it indexes **docstrip**-style source files rather than raw "*.tcl*" or loadable library files. Only packages listed in the catalogue of a file are considered.

The *dir* argument is the directory in which to look for files (and whose "*pkgIndex.tcl*" file should be amended). The *pattern* argument is a **glob** pattern of files to look into; a typical value would be ***.dtx** or ***.{dtx,ddt}**. Remaining arguments are option-value pairs, where the supported options are:

-recursein *dirpattern*

If this option is given, then the **index_from_catalogue** operation will be repeated in each subdirectory whose name matches the *dirpattern*. **-recursein *** will cause the entire subtree rooted at *dir* to be indexed.

-sourceconf *dictionary*

Specify **fileoptions** to use when reading the catalogues of files (and also for reading the packages if the catalogue does not contain a **fileoptions** command). Defaults to being

empty. Primarily useful if your system encoding is very different from that of the source file (e.g., one is a two-byte encoding and the other is a one-byte encoding). **ascii** and **utf-8** are not very different in that sense.

-options *terminals*
The *terminals* is a list of terminals in addition to **docstrip.tcl::catalogue** that should be held as true when extracting the catalogue. Defaults to being empty. This makes it possible to make use of "variant sections" in the catalogue itself, e.g. gaurd some entries with an extra "experimental" and thus prevent them from appearing in the index unless that is generated with "experimental" among the **-options**.

-report *boolean*
If the *boolean* is true then the return value will be a textual, probably multiline, report on what was done. Defaults to false, in which case there is no particular return value.

-reportcmd *commandPrefix*
Every item in the report is handed as an extra argument to the command prefix. Since **index_from_catalogue** would typically be used at a rather high level in installation scripts and the like, the *commandPrefix* defaults to "**puts stdout**". Use **list** to effectively disable this feature. The return values from the prefix are ignored.

The **package ifneeded** scripts that are generated contain one **package require docstrip** command and one docstrip::sourcefrom command. If the catalogue entry was of the **pkgProvide** kind then the **package ifneeded** script also contains the **package provide** command.

Note that **index_from_catalogue** never removes anything from an existing "*pkgIndex.tcl*" file. Hence you may need to delete it (or have **pkg_mkIndex** recreate it from scratch) before running **index_from_catalogue** to update some piece of information, such as a package version number.

docstrip::util::modules_from_catalogue *target source ?option value ...?*
This command is an alternative to **index_from_catalogue** which creates Tcl Module ("*.tm*") files rather than "*pkgIndex.tcl*" entries. Since this action is more similar to what **docstrip** classically does, it has features for putting pre- and postambles on the generated files.

The *source* argument is the name of the source file to generate "*.tm*" files from. The *target* argument is the directory which should count as a module path, i.e., this is what the relative paths derived from package names are joined to. The supported options are:

-preamble *message*
A message to put in the preamble (initial block of comments) of generated files. Defaults to a space. May be several lines, which are then separated by newlines. Traditionally used for copyright notices or the like, but metacomment lines provide an alternative to that.

-postamble *message*
Like **-preamble**, but the message is put at the end of the file instead of the beginning. Defaults to being empty.

-sourceconf *dictionary*
Specify **fileoptions** to use when reading the catalogue of the *source* (and also for reading the packages if the catalogue does not contain a **fileoptions** command). Defaults to being empty. Primarily useful if your system encoding is very different from that of the source file (e.g., one is a two-byte encoding and the other is a one-byte encoding). **ascii** and **utf-8**

are not very different in that sense.

-options *terminals*
The *terminals* is a list of terminals in addition to **docstrip.tcl::catalogue** that should be held as true when extracting the catalogue. Defaults to being empty. This makes it possible to make use of "variant sections" in the catalogue itself, e.g. gaurd some entries with an extra "experimental" guard and thus prevent them from contributing packages unless those are generated with "experimental" among the **-options**.

-formatpreamble *commandPrefix*
Command prefix used to actually format the preamble. Takes four additional arguments *message*, *targetFilename*, *sourceFilename*, and *terminalList* and returns a fully formatted preamble. Defaults to using **classical_preamble** with a *metaprefix* of '##'.

-formatpostamble *commandPrefix*
Command prefix used to actually format the postamble. Takes four additional arguments *message*, *targetFilename*, *sourceFilename*, and *terminalList* and returns a fully formatted postamble. Defaults to using **classical_postamble** with a *metaprefix* of '##'.

-report *boolean*
If the *boolean* is true (which is the default) then the return value will be a textual, probably multiline, report on what was done. If it is false then there is no particular return value.

-reportcmd *commandPrefix*
Every item in the report is handed as an extra argument to this command prefix. Defaults to **list**, which effectively disables this feature. The return values from the prefix are ignored. Use for example "**puts stdout**" to get report items written immediately to the terminal.

An existing file of the same name as one to be created will be overwritten.

docstrip::util::classical_preamble *metaprefix message target* ?*source terminals* ...?
This command returns a preamble in the classical **docstrip** style

```
##
## This is `TARGET',
## generated by the docstrip::util package.
##
## The original source files were:
##
## SOURCE (with options: `foo,bar')
##
## Some message line 1
## line2
## line3
```

if called as

```
docstrip::util::classical_preamble {##}\
  "\nSome message line 1\nline2\nline3" TARGET SOURCE {foo bar}
```

The command supports preambles for files generated from multiple sources, even though

modules_from_catalogue at present does not need that.

docstrip::util::classical_postamble ***metaprefix message target*** **?*source terminals* ...?**

This command returns a postamble in the classical **docstrip** style

```
## Some message line 1
## line2
## line3
##
## End of file `TARGET'.
```

if called as

```
docstrip::util::classical_postamble {##}\
  "Some message line 1\nline2\nline3" TARGET SOURCE {foo bar}
```

In other words, the *source* and *terminals* arguments are ignored, but supported for symmetry with **classical_preamble**.

docstrip::util::packages_provided ***text*** **?*setup-script*?**

This command returns a list where every even index element is the name of a package **provide**d by *text* when that is evaluated as a Tcl script, and the following odd index element is the corresponding version. It is used to do package indexing of extracted pieces of code, in the manner of **pkg_mkIndex**.

One difference to **pkg_mkIndex** is that the *text* gets evaluated in a safe interpreter. **package require** commands are silently ignored, as are unknown commands (which includes **source** and **load**). Other errors cause processing of the *text* to stop, in which case only those package declarations that had been encountered before the error will be included in the return value.

The *setup-script* argument can be used to customise the evaluation environment, if the code in *text* has some very special needs. The *setup-script* is evaluated in the local context of the **packages_provided** procedure just before the *text* is processed. At that time, the name of the slave command for the safe interpreter that will do this processing is kept in the local variable **c**. To for example copy the contents of the **::env** array to the safe interpreter, one might use a *setup-script* of

```
  $c eval [list array set env [array get ::env]]
```

Source processing commands

Unlike the previous group of commands, which would use docstrip::extract to extract some code lines and then process those further, the following commands operate on text consisting of all types of lines.

docstrip::util::ddt2man ***text***

The **ddt2man** command reformats *text* from the general **docstrip** format to **doctools** "*.man* " format (Tcl Markup Language for Manpages). The different line types are treated as follows:

comment and metacomment lines

The '%' and '%%' prefixes are removed, the rest of the text is kept as it is.

empty lines
: These are kept as they are. (Effectively this means that they will count as comment lines after a comment line and as code lines after a code line.)

code lines
: **example_begin** and **example_end** commands are placed at the beginning and end of every block of consecutive code lines. Brackets in a code line are converted to **lb** and **rb** commands.

verbatim guards
: These are processed as usual, so they do not show up in the result but every line in a verbatim block is treated as a code line.

other guards
: These are treated as code lines, except that the actual guard is **emph**asised.

At the time of writing, no project has employed **doctools** markup in master source files, so experience of what works well is not available. A source file could however look as follows

```
% [manpage_begin gcd n 1.0]
% [keywords divisor]
% [keywords math]
% [moddesc {Greatest Common Divisor}]
% [require gcd [opt 1.0]]
% [description]
%
% [list_begin definitions]
% [call [cmd gcd] [arg a] [arg b]]
%   The [cmd gcd] procedure takes two arguments [arg a] and [arg b]
    which
%   must be integers and returns their greatest common divisor.
proc gcd {a b} {
%   The first step is to take the absolute values of the arguments.
%   This relieves us of having to worry about how signs will be
    treated
%   by the remainder operation.
   set a [expr {abs($a)}]
   set b [expr {abs($b)}]
%   The next line does all of Euclid's algorithm! We can make do
%   without a temporary variable, since $a is substituted before the
%   [lb]set a $b[rb] and thus continues to hold a reference to the
%   "old" value of [var a].
   while {$b>0} { set b [expr { $a % [set a $b] }] }
%   In Tcl 8.3 we might want to use [cmd set] instead of [cmd return]
%   to get the slight advantage of byte-compilation.
%<tcl83>  set a
%<!tcl83>   return $a
}
% [list_end]
%
% [manpage_end]
```

If the above text is fed through **docstrip::util::ddt2man** then the result will be a syntactically correct **doctools** manpage, even though its purpose is a bit different.

It is suggested that master source code files with **doctools** markup are given the suffix "*.ddt* ", hence the "ddt" in **ddt2man**.

docstrip::util::guards *subcmd text*

The **guards** command returns information (mostly of a statistical nature) about the ordinary docstrip guards that occur in the *text*. The *subcmd* selects what is returned.

counts

List the guard expression terminals with counts. The format of the return value is a dictionary which maps the terminal name to the number of occurencies of it in the file.

exprcount

List the guard expressions with counts. The format of the return value is a dictionary which

maps the expression to the number of occurencies of it in the file.

exprerr
List the syntactically incorrect guard expressions (e.g. parentheses do not match, or a terminal is missing). The return value is a list, with the elements in no particular order.

expressions
List the guard expressions. The return value is a list, with the elements in no particular order.

exprmods
List the guard expressions with modifiers. The format of the return value is a dictionary where each index is a guard expression and each entry is a string with one character for every guard line that has this expression. The characters in the entry specify what modifier was used in that line: +, -, *, /, or (for guard without modifier:) space. This is the most primitive form of the information gathered by **guards**.

names
List the guard expression terminals. The return value is a list, with the elements in no particular order.

rotten
List the malformed guard lines (this does not include lines where only the expression is malformed, though). The format of the return value is a dictionary which maps line numbers to their contents.

docstrip::util::patch ***source-var terminals fromtext diff ?option value ...?***

This command tries to apply a **diff** file (for example a contributed patch) that was computed for a generated file to the **docstrip** source. This can be useful if someone has edited a generated file, thus mistaking it for being the source. This command makes no presumptions which are specific for the case that the generated file is a Tcl script.

patch requires that the source file to patch is kept as a list of lines in a variable, and the name of that variable in the calling context is what goes into the *source-var* argument. The *terminals* is the list of terminals used to extract the file that has been patched. The *diff* is the actual diff to apply (in a format as explained below) and the *fromtext* is the contents of the file which served as "from" when the diff was computed. Options can be used to further control the process.

The process works by "lifting" the hunks in the *diff* from generated to source file, and then applying them to the elements of the *source-var*. In order to do this lifting, it is necessary to determine how lines in the *fromtext* correspond to elements of the *source-var*, and that is where the *terminals* come in; the source is first **extract**ed under the given *terminals*, and the result of that is then matched against the *fromtext*. This produces a map which translates line numbers stated in the *diff* to element numbers in *source-var*, which is what is needed to lift the hunks.

The reason that both the *terminals* and the *fromtext* must be given is twofold. First, it is very difficult to keep track of how many lines of preamble are supplied some other way than by copying lines from source files. Second, a generated file might contain material from several source files. Both make it impossible to predict what line number an extracted file would have in the generated file, so instead the algorithm for computing the line number map looks for a block of lines in the *fromtext* which matches what can be extracted from the source. This

matching is affected by the following options:

-matching *mode*
How equal must two lines be in order to match? The supported *mode*s are:

exact
Lines must be equal as strings. This is the default.

anyspace
All sequences of whitespace characters are converted to single spaces before comparing.

nonspace
Only non-whitespace characters are considered when comparing.

none
Any two lines are considered to be equal.

-metaprefix *string*
The **-metaprefix** value to use when extracting. Defaults to "%%", but for Tcl code it is more likely that "#" or "##" had been used for the generated file.

-trimlines *boolean*
The **-trimlines** value to use when extracting. Defaults to true.

The return value is in the form of a unified diff, containing only those hunks which were not applied or were only partially applied; a comment in the header of each hunk specifies which case is at hand. It is normally necessary to manually review both the return value from **patch** and the patched text itself, as this command cannot adjust comment lines to match new content.

An example use would look like

```
set sourceL [split [docstrip::util::thefile from.dtx] \n]
set terminals {foo bar baz}
set fromtext [docstrip::util::thefile from.tcl]
set difftext [exec diff --unified from.tcl to.tcl]
set leftover [docstrip::util::patch sourceL $terminals $fromtext\
  [docstrip::util::import_unidiff $difftext] -metaprefix {#}]
set F [open to.dtx w]; puts $F [join $sourceL \n]; close $F
return $leftover
```

Here, "*from.dtx*" was used as source for "*from.tcl*", which someone modified into "*to.tcl*". We're trying to construct a "*to.dtx*" which can be used as source for "*to.tcl*".

docstrip::util::thefile *filename* ?*option value* ...?
The **thefile** command opens the file *filename*, reads it to end, closes it, and returns the contents (dropping a final newline if there is one). The option-value pairs are passed on to **fconfigure** to configure the open file channel before anything is read from it.

docstrip::util::import_unidiff *diff-text* ?*warning-var*?
This command parses a unified (**diff** flags **-U** and **--unified**) format diff into the list-of-hunks format expected by **docstrip::util::patch**. The *diff-text* argument is the text to parse and the *warning-var* is, if specified, the name in the calling context of a variable to which any warnings about parsing problems will be **append**ed.

The return value is a list of *hunks*. Each hunk is a list of five elements "*start1 end1 start2 end2 lines*". *start1* and *end1* are line numbers in the "from" file of the first and last respectively lines of the hunk. *start2* and *end2* are the corresponding line numbers in the "to" file. Line numbers start at 1. The *lines* is a list with two elements for each line in the hunk; the first specifies the type of a line and the second is the actual line contents. The type is **-** for lines only in the "from" file, **+** for lines that are only in the "to" file, and **0** for lines that are in both.

Category
Documentation tools

Copyright
```
Copyright (c) 2003–2010 Lars Hellström <Lars dot Hellstrom at
    residenset dot net>
```

doctools

changelog
Keywords: changelog, doctools, emacs

Name
doctools::changelog - Processing text in Emacs ChangeLog format

Synopsis
package require Tcl 8.5 9
package require textutil
package require doctools::changelog ?**1.2**?
::doctools::changelog::scan *text*
::doctools::changelog::flatten *entries*
::doctools::changelog::toDoctools *title module version entries*
::doctools::changelog::merge *entries...*

Description
This package provides Tcl commands for the processing and reformatting of text in the "*ChangeLog*" format generated by **emacs**.

API
::doctools::changelog::scan *text*

The command takes the *text* and parses it under the assumption that it contains a ChangeLog as generated by **emacs**. It returns a data structure describing the contents of this ChangeLog.

This data structure is a list where each element describes one entry in the ChangeLog. Each element/entry is then a list of three elements describing the date of the entry, its author, and the comments made, in this order. The last item in each element/entry, the comments, is a list of sections. Each section is described by a list containing two elements, a list of file

names, and a string containing the true comment associated with the files of the section.

```
    {
    {
        date
        author
        {
        {
            {file ...}
            commenttext
        }
        ...
        }
    }
    {...}
    }
```

::doctools::changelog::flatten *entries*
This command converts a list of entries as generated by **change::scan** above into a simpler list of plain text blocks each containing all the information of a single entry.

::doctools::changelog::toDoctools *title module version entries*
This command converts the pre-parsed ChangeLog *entries* as generated by the command **::doctools::changelog::scan** into a document in *doctools* format and returns it as the result of the command.

The other three arguments supply the information for the header of that document which is not available from the changelog itself.

::doctools::changelog::merge *entries...*
Each argument of the command is assumed to be a pre-parsed Changelog as generated by the command **::doctools::changelog::scan**. This command merges all of them into a single structure, and collapses multiple entries for the same date and author into a single entry. The new structure is returned as the result of the command.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Documentation tools

Copyright

```
Copyright (c) 2003-2013 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

cvs

Keywords: cvs, changelog, cvs log, emacs, log

Name

doctools::cvs - Processing text in 'cvs log' format

Synopsis

package require Tcl 8.5 9
package require textutil
package require doctools::cvs ?**1.1**?
::doctools::cvs::scanLog *text evar cvar fvar*
::doctools::cvs::toChangeLog *evar cvar fvar*

Description

This package provides Tcl commands for the processing and reformatting text in the format generated by the **cvs log** command.

The commands **::doctools::cvs::scanLog** and **::doctools::cvs::toChangeLog** are derived from code found on the Tcl'ers Wiki. See the references at the end of the page.

API

::doctools::cvs::scanLog *text evar cvar fvar*

The command takes the *text* and parses it under the assumption that it contains a CVS log as generated by **cvs log**. The resulting information is stored in the variables whose names were specified via *evar*, *cvar*, and *fvar*.

Already existing information in the referenced variables is preserved, allowing the caller to merge data from multiple logs into one database.

varname *evar* (in)

Has to refer to a scalar variable. After the call this variable will contain a list of all the entries found in the log file. An entry is identified through the combination of date and author, and can be split over multiple physical entries, one per touched file.

It should be noted that the entries are listed in the same order as they were found in the *text*. This is not necessarily sorted by date or author.

Each item in the list is a list containing two elements, the date of the entry, and its author, in this order. The date is formatted as **year**/**month**/**day**.

varname *cvar* (in)

Has to refer to an array variable. Keys are strings containing the date and author of log entries, in this order, separated by a comma.

The values are lists of comments made for the entry.

varname *fvar* (in)

Has to refer to an array variable. Keys are strings containing date, author of a log entry, and a comment for that entry, in this order, separated by commas.

The values are lists of the files the entry is touching.

::doctools::cvs::toChangeLog ***evar cvar fvar***
The three arguments for this command are the same as the last three arguments of the command **::doctools::cvs::scanLog**. This command however expects them to be filled with information about one or more logs. It takes this information and converts it into a text in the format of a ChangeLog as accepted and generated by **emacs**. The constructed text is returned as the result of the command.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Documentation tools

Copyright


```
Copyright (c) 2003-2008 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


docidx

Keywords: docidx, HTML, TMML, conversion, documentation, index, keyword index, latex, manpage, markdown, markup, nroff, wiki

Name

doctools::idx - docidx - Processing indices

Synopsis

package require Tcl 8.5 9
package require doctools::idx ?**1.2.1**?
::doctools::idx::new *objectName* ?*-option value* ...?
::doctools::idx::help
::doctools::idx::search *path*
objectName method ?*arg arg* ...?
objectName **configure**
objectName **configure** *option*
objectName **configure -option** *value...*
objectName **cget -option**

objectName **destroy**
objectName **format** *text*
objectName **map** *symbolic actual*
objectName **parameters**
objectName **search** *path*
objectName **setparam** *name value*
objectName **warnings**

Description

This package provides a class for the creation of objects able to process and convert text written in the *docidx* markup language into any output format X for which a *formatting engine* is available.

A reader interested in the markup language itself should start with the *docidx language introduction* and proceed from there to the formal specifications, i.e. the *docidx language syntax* and the *docidx language command reference*.

If on the other hand the reader wishes to write her own formatting engine for some format, i.e. is a *plugin writer* then reading and understanding the *docidx plugin API reference* is an absolute necessity, as that document specifies the interaction between this package and its plugins, i.e. the formatting engines, in detail.

Public API

Package commands

::doctools::idx::new *objectName* ?-option *value* ...?

This command creates a new docidx object with an associated Tcl command whose name is *objectName*. This *object* command is explained in full detail in the sections Object command and Object methods. The object command will be created under the current namespace if the *objectName* is not fully qualified, and in the specified namespace otherwise.

The options and their values coming after the name of the object are used to set the initial configuration of the object.

::doctools::idx::help

This is a convenience command for applications wishing to provide their user with a short description of the available formatting commands and their meanings. It returns a string containing a standard help text.

::doctools::idx::search *path*

Whenever an object created by this the package has to map the name of a format to the file containing the code for its formatting engine it will search for the file in a number of directories stored in a list. See section Format mapping for more explanations.

This list not only contains three default directories which are declared by the package itself, but is also extensible user of the package. This command is the means to do so. When given a *path* to an existing and readable directory it will prepend that directory to the list of directories to search. This means that the *path* added last is later searched through first.

An error will be thrown if the *path* either does not exist, is not a directory, or is not readable.

Object command

All commands created by **::doctools::idx::new** have the following general form and may be used to invoke various operations on their docidx converter object.

objectName method ?*arg arg ...*?

The method **method** and its *arg*'uments determine the exact behavior of the command. See section Object methods for the detailed specifications.

Object methods

***objectName* configure**

The method returns a list of all known options and their current values when called without any arguments.

objectName* configure *option

The method behaves like the method **cget** when called with a single argument and returns the value of the option specified by said argument.

***objectName* configure -option *value*...**

The method reconfigures the specified **option**s of the object, setting them to the associated *value*s, when called with an even number of arguments, at least two.

The legal options are described in the section Object configuration.

***objectName* cget -option**

This method expects a legal configuration option as argument and will return the current value of that option for the object the method was invoked for.

The legal configuration options are described in section Object configuration.

***objectName* destroy**

This method destroys the object it is invoked for.

objectName* format *text

This method runs the *text* through the configured formatting engine and returns the generated string as its result. An error will be thrown if no **-format** was configured for the object.

The method assumes that the *text* is in *docidx* format as specified in the companion document *docidx_fmt*. Errors will be thrown otherwise.

objectName* map *symbolic actual

This methods add one entry to the per-object mapping from *symbolic* filenames to the *actual* uris. The object just stores this mapping and makes it available to the configured formatting engine through the command dt_fmap. This command is described in more detail in the *docidx plugin API reference* which specifies the interaction between the objects created by this package and index formatting engines.

***objectName* parameters**

This method returns a list containing the names of all engine parameters provided by the configured formatting engine. It will return an empty list if the object is not yet configured for a specific format.

objectName* search *path

This method extends the per-object list of paths searched for index formatting engines. See

also the command **::doctools::idx::search** on how to extend the per-package list of paths. Note that the path entered last will be searched first. For more details see section Format mapping.

objectName* setparam *name value
This method sets the *name*d engine parameter to the specified *value*. It will throw an error if the object is either not yet configured for a specific format, or if the formatting engine for the configured format does not provide a parameter with the given *name*. The list of parameters provided by the configured formatting engine can be retrieved through the method **parameters**.

***objectName* warnings**
This method returns a list containing all the warnings which were generated by the configured formatting engine during the last invocation of the method **format**.

Object configuration

All docidx objects understand the following configuration options:

-file *file*
The argument of this option is stored in the object and made available to the configured formatting engine through the command dt_file. This command is described in more detail in the companion document *docidx_api* which specifies the API between the object and formatting engines.

The default value of this option is the empty string.

The configured formatting engine should interpret the value as the name of the file containing the document which is currently processed.

-format *text*
The argument of this option specifies the format to generate and by implication the formatting engine to use when converting text via the method **format**. Its default value is the empty string. The method **format** cannot be used if this option is not set to a valid value at least once.

The package will immediately try to map the given name to a file containing the code for a formatting engine generating that format. An error will be thrown if this mapping fails. In that case a previously configured format is left untouched.

The section Format mapping explains in detail how the package and object will look for engine implementations.

Format mapping

The package and object will perform the following algorithm when trying to map a format name *foo* to a file containing an implementation of a formatting engine for *foo*:

1. If *foo* is the name of an existing file then this file is directly taken as the implementation.
2. If not, the list of per-object search paths is searched. For each directory in the list the package checks if that directory contains a file "*idx.foo*". If yes, then that file is taken as the implementation.

Note that this list of paths is initially empty and can be extended through the object method **search**.

3. If not, the list of package paths is searched. For each directory in the list the package checks if that directory contains a file "*idx.foo*". If yes, then that file is taken as the implementation.

   This list of paths can be extended through the command **::doctools::idx::search**. It contains initially one path, the subdirectory "*mpformats*" of the directory the package itself is located in. In other words, if the package implementation "*docidx.tcl*" is installed in the directory "*/usr/local/lib/tcllib/doctools*" then it will by default search the directory "*/usr/local/lib/tcllib/doctools/mpformats*" for format implementations.
4. The mapping fails.

Predefined engines

The package provides predefined formatting engines for the following formats. Some of the formatting engines support engine parameters. These will be explicitly highlighted.

html

This engine generates HTML markup, for processing by web browsers and the like. This engine supports three parameters:

footer

The value for this parameter has to be valid selfcontained HTML markup for the body section of a HTML document. The default value is the empty string. The value is inserted into the generated output just before the tag, closing the body of the generated HTML.

This can be used to insert boilerplate footer markup into the generated document.

header

The value for this parameter has to be valid selfcontained HTML markup for the body section of a HTML document. The default value is the empty string. The value is inserted into the generated output just after the tag, starting the body of the generated HTML.

This can be used to insert boilerplate header markup into the generated document.

meta

The value for this parameter has to be valid selfcontained HTML markup for the header section of a HTML document. The default value is the empty string. The value is inserted into the generated output just after the tag, starting the header section of the generated HTML.

This can be used to insert boilerplate meta data markup into the generated document, like references to a stylesheet, standard meta keywords, etc.

latex

This engine generates output suitable for the **latex** text processor coming out of the TeX world.

list

This engine retrieves version, section and title of the manpage from the document. As such it can be used to generate a directory listing for a set of manpages.

markdown

This engine generates *Markdown* markup.

nroff
: This engine generates nroff output, for processing by **nroff**, or **groff**. The result will be standard man pages as they are known in the unix world.

null
: This engine generates no outout at all. This can be used if one just wants to validate some input.

tmml
: This engine generates TMML markup as specified by Joe English. The Tcl Manpage Markup Language is a derivate of XML.

wiki
: This engine generates Wiki markup as understood by Jean Claude Wippler's **wikit** application.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Documentation tools

Copyright


```
Copyright (c) 2003-2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


docidx_intro

Keywords: docidx_intro, index, keyword index, markup, semantic markup

Name

docidx_intro - docidx introduction

Description

docidx (short for *documentation tables of contents*) stands for a set of related, yet different, entities which are working together for the easy creation and transformation of keyword-based indices for documentation. These are

1. A tcl based language for the semantic markup of a keyword index. Markup is represented by Tcl commands.
2. A package providing the ability to read and transform texts written in that markup language. It is important to note that the actual transformation of the input text is delegated to plugins.
3. An API describing the interface between the package above and a plugin.

Which of the more detailed documents are relevant to the reader of this introduction depends on

their role in the documentation process.

1. A *writer* of documentation has to understand the markup language itself. A beginner to docidx should read the more informally written *docidx language introduction* first. Having digested this the formal *docidx language syntax* specification should become understandable. A writer experienced with docidx may only need the *docidx language command reference* from time to time to refresh her memory.

   While a document is written the **dtp** application can be used to validate it, and after completion it also performs the conversion into the chosen system of visual markup, be it *roff, HTML, plain text, wiki, etc. The simpler **dtplite** application makes internal use of docidx when handling directories of documentation, automatically generating a proper keyword index for them.
2. A *processor* of documentation written in the *docidx* markup language has to know which tools are available for use.

   The main tool is the aforementioned **dtp** application provided by Tcllib. The simpler **dtplite** does not expose docidx to the user. At the bottom level, common to both applications, however sits the package **doctoools::idx**, providing the basic facilities to read and process files containing text in the docidx format.
3. At last, but not least, *plugin writers* have to understand the interaction between the **doctools::idx** package and its plugins, as described in the *docidx plugin API reference*.

Related formats

docidx does not stand alone, it has two companion formats. These are called *doctoc* and *doctools*, and they are for the markup of *tables of contents*, and general documentation, respectively. They are described in their own sets of documents, starting at the *doctoc introduction* and the *doctools introduction*, respectively.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Documentation tools

Copyright


```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


docidx_lang_cmdref

Keywords: docidx_lang_cmdref, docidx commands, docidx language, docidx markup, markup, semantic markup

Name

docidx_lang_cmdref - docidx language command reference

Synopsis

comment *plaintext*
include *filename*
index_begin *text title*
index_end
key *text*
lb
manpage *file text*
rb
url *url label*
vset *varname value*
vset *varname*

Description

This document specifies both names and syntax of all the commands which together are the docidx markup language, version 1. As this document is intended to be a reference the commands are listed in alphabetical order, and the descriptions are relatively short. A beginner should read the much more informally written *docidx language introduction* first.

Commands

comment *plaintext*
: Index markup. The argument text is marked up as a comment standing outside of the actual text of the document. Main use is in free-form text.

include *filename*
: Templating. The contents of the named file are interpreted as text written in the docidx markup and processed in the place of the include command. The markup in the file has to be self-contained. It is not possible for a markup command to cross the file boundaries.

index_begin *text title*
: Document structure. The command to start an index. The arguments are a label for the whole group of documents the index refers to (*text*) and the overall title text for the index (*title*), without markup.

  The label often is the name of the package (or extension) the documents belong to.

index_end
: Document structure. Command to end an index. Anything in the document coming after this command is in error.

key *text*
: Index structure. This command adds the keyword *text* to the index.

lb
: Text. The command is replaced with a left bracket. Use in free-form text. Required to avoid interpretation of a left bracket as the start of a markup command. Its usage is restricted to the arguments of other markup commands.

manpage *file text*

Index structure. This command adds an element to the index which refers to a document. The document is specified through the symbolic name *file*. The *text* argument is used to label the reference.

Symbolic names are used to preserve the convertibility of this format to any output format. The actual name of the file will be inserted by the chosen formatting engine when converting the input. This will be based on a mapping from symbolic to actual names given to the engine.

rb
Text. The command is replaced with a right bracket. Use in free-form text. Required to avoid interpretation of a right bracket as the end of a markup command. Its usage is restricted to the arguments of other commands.

url *url label*
Index structure. This is the second command to add an element to the index. To refer to a document it is not using a symbolic name however, but a (possibly format-specific) url describing the exact location of the document indexed here.

vset *varname value*
Templating. In this form the command sets the named document variable to the specified *value*. It does not generate output. I.e. the command is replaced by the empty string.

vset *varname*
Templating. In this form the command is replaced by the value of the named document variable

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Documentation tools

Copyright


```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


docidx_lang_faq

Keywords: docidx_lang_faq, docidx commands, docidx language, docidx markup, docidx syntax, examples, faq, markup, semantic markup

Name

docidx_lang_faq - docidx language faq

Description

Overview

What is this document?

This document is currently mainly a placeholder, to be filled with commonly asked questions about the docidx markup language and companions, and their answers.

Please report any questions (and, if possible, answers) we should consider for this document as explained in the section Bugs, Ideas, Feedback below.

Examples

Where do I find docidx examples?

We have no direct examples of documents written using docidx markup. However the doctools processor **dtplite** does generate a table of contents when processing a set of documents written in doctools markup. The intermediate file for it uses docidx markup and is not deleted when generation completes. Such files can therefore serve as examples.

dtplite is distributed as part of Tcllib, so to get it you need one of

1. A snapshot of Tcllib. How to retrieve such a snapshot and the tools required for this are described at *Development Snapshots* [/wiki?name=Development+Snapshots]
2. A Tcllib release archive. They are available at the *home* [/home] page.

Bugs, Ideas, Feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Documentation tools

Copyright


```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


docidx_lang_intro

Keywords: docidx_lang_intro, docidx commands, docidx language, docidx markup, docidx syntax , markup, semantic markup

Name

docidx_lang_intro - docidx language introduction

Description

This document is an informal introduction to version 1 of the docidx markup language based on a multitude of examples. After reading this a writer should be ready to understand the two parts of the formal specification, i.e. the *docidx language syntax* specification and the *docidx language command reference*.

Fundamentals

While the *docidx markup language* is quite similar to the *doctools markup language*, in the broadest terms possible, there is one key difference. An index consists essentially only of markup commands, with no plain text interspersed between them, except for whitespace.

Each markup command is a Tcl command surrounded by a matching pair of **[** and **]**. Inside of these delimiters the usual rules for a Tcl command apply with regard to word quotation, nested commands, continuation lines, etc. I.e.

```
    ... [key {markup language}] ...
```

```
  ... [manpage thefile \
          {file description}] ...
```

Basic structure

The most simple document which can be written in docidx is

```
    [index_begin GROUPTITLE TITLE]
    [index_end]
```

Not very useful, but valid. This also shows us that all docidx documents consist of only one part where we will list all keys and their references.

A more useful index will contain at least keywords, or short 'keys', i.e. the phrases which were indexed. So:

```
[index_begin GROUPTITLE TITLE]
[key markup]
[key {semantic markup}]]
[key {docidx markup}]
[key {docidx language}]
[key {docidx commands}]
[index_end]
```

In the above example the command key is used to declare the keyword phrases we wish to be part of the index.

However a truly useful index does not only list the keyword phrases, but will also contain references to documents associated with the keywords. Here is a made-up index for all the manpages in the module *base64*:

```
[index_begin tcllib/base64 {De- & Encoding}]
[key base64]
[manpage base64]
[key encoding]
[manpage base64]
```

```
[manpage uuencode]
[manpage yencode]
[key uuencode]
[manpage uuencode]
[key yEnc]
[manpage yencode]
[key ydecode]
[manpage yencode]
[key yencode]
[manpage yencode]
[index_end]
```

In the above example the command manpage is used to insert references to documents, using symbolic file names, with each command belonging to the last key command coming before it.

The other command to insert references is **url**. In contrast to manpage it uses explicit (possibly format-specific) urls to describe the location of the referenced document. As such this command is intended for the creation of references to external documents which could not be handled in any other way.

Advanced structure

In all previous examples we fudged a bit regarding the markup actually allowed to be used before the index_begin command opening the document.

Instead of only whitespace the two templating commands include and vset are also allowed, to enable the writer to either set and/or import configuration settings relevant to the table of contents. I.e. it is possible to write

```
[include FILE]
[vset VAR VALUE]
[index_begin GROUPTITLE TITLE]
...
[index_end]
```

Even more important, these two commands are allowed anywhere where a markup command is allowed, without regard for any other structure.

```
[index_begin GROUPTITLE TITLE]
[include FILE]
[vset VAR VALUE]
...
[index_end]
```

The only restriction include has to obey is that the contents of the included file must be valid at the place of the inclusion. I.e. a file included before index_begin may contain only the templating commands vset and include, a file included after a key may contain only manape or url references, and other keys, etc.

Escapes

Beyond the 6 commands shown so far we have two more available. However their function is not

the marking up of index structure, but the insertion of characters, namely **[** and **]**. These commands, lb and rb respectively, are required because our use of [and] to bracket markup commands makes it impossible to directly use [and] within the text.

Our example of their use are the sources of the last sentence in the previous paragraph, with some highlighting added.

```
...
These commands, [cmd lb] and [cmd lb] respectively, are required
because our use of [lb] and [rb] to bracket markup commands makes it
impossible to directly use [lb] and [rb] within the text.
...
```

Further reading

Now that this document has been digested the reader, assumed to be a *writer* of documentation should be fortified enough to be able to understand the formal *docidx language syntax* specification as well. From here on out the *docidx language command reference* will also serve as the detailed specification and cheat sheet for all available commands and their syntax.

To be able to validate a document while writing it, it is also recommended to familiarize oneself with Tclapps' ultra-configurable **dtp**.

On the other hand, docidx is perfectly suited for the automatic generation from doctools documents, and this is the route Tcllib's easy and simple **dtplite** goes, creating an index for a set of documents behind the scenes, without the writer having to do so on their own.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Documentation tools

Copyright


```
Copyright (c) 2007-2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


docidx_lang_syntax

Keywords: docidx_lang_syntax, docidx commands, docidx language, docidx markup, docidx syntax, markup, semantic markup

Name

docidx_lang_syntax - docidx language syntax

Description

This document contains the formal specification of the syntax of the docidx markup language, version 1 in Backus-Naur-Form. This document is intended to be a reference, complementing the *docidx language command reference*. A beginner should read the much more informally written *docidx language introduction* first before trying to understand either this document or the command reference.

Fundamentals

In the broadest terms possible the *docidx markup language* is like SGML and similar languages. A document written in this language consists primarily of markup commands, with text embedded into it at some places.

Each markup command is a just Tcl command surrounded by a matching pair of **[** and **]**. Which commands are available, and their arguments, i.e. syntax is specified in the *docidx language command reference*.

In this document we specify first the lexeme, and then the syntax, i.e. how we can mix text and markup commands with each other.

Lexical definitions

In the syntax rules listed in the next section

1. stands for all text except markup commands.
2. Any XXX stands for the markup command [xxx] including its arguments. Each markup command is a Tcl command surrounded by a matching pair of **[** and **]**. Inside of these delimiters the usual rules for a Tcl command apply with regard to word quotation, nested commands, continuation lines, etc.
3. stands for all text consisting only of spaces, newlines, tabulators and the comment markup command.

Syntax

The rules listed here specify only the syntax of docidx documents. The lexical level of the language was covered in the previous section.

Regarding the syntax of the (E)BNF itself

1. The construct { X } stands for zero or more occurrences of X.
2. The construct [X] stands for zero or one occurrence of X.

The syntax:

```
index     = defs
            INDEX_BEGIN
            [ contents ]
            INDEX_END
            { <WHITE> }

defs      = { INCLUDE | VSET | <WHITE> }
contents  = keyword { keyword }

keyword   = defs KEY ref { ref }
ref       = MANPAGE | URL | defs
```

At last a rule we were unable to capture in the EBNF syntax, as it is about the arguments of the markup commands, something which is not modeled here.

1. The arguments of all markup commands have to be plain text, and/or text markup commands, i.e. one of
   1. lb,
   2. rb, or
   3. vset (1-argument form).

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Documentation tools

Copyright


```
Copyright (c) 2007-2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


docidx_plugin_apiref

Keywords: docidx_plugin_apiref, formatting engine, index, index formatter, keywords, markup, plugin, semantic markup

Name

docidx_plugin_apiref - docidx plugin API reference

Synopsis

dt_fmap *symfname*

dt_format
dt_read *file*
dt_source *file*
ex_cappend *text*
ex_cget *varname*
ex_cis *cname*
ex_cname
ex_cpop *cname*
ex_cpush *cname*
ex_cset *varname value*
ex_lb ?*newbracket*?
ex_rb ?*newbracket*?
idx_initialize
idx_listvariables
idx_numpasses
idx_postprocess *text*
idx_setup *n*
idx_shutdown
idx_varset *varname text*
fmt_plain_text *text*

Description

This document is intended for *plugin writers*, i.e. developers wishing to write an index *formatting engine* for some output format X.

It specifies the interaction between the **doctools::idx** package and its plugins, i.e. the interface any index formatting engine has to comply with.

This document deals with version 1 of the interface.

A reader who is on the other hand more interested in the markup language itself should start with the *docidx language introduction* and proceed from there to the formal specifications, i.e. the *docidx language syntax* and the *docidx language command reference*.

Overview

The API for an index formatting engine consists of two major sections.

On the one side we have a set of commands through which the plugin is able to query the frontend. These commands are provided by the frontend and linked into the plugin interpreter. Please see section Frontend commands for their detailed specification.

And on the other side the plugin has to provide its own set of commands which will then be called by the frontend in a specific sequence while processing input. They, again, fall into two categories, management and formatting. Please see section Plugin commands and its subsections for their detailed specification.

Frontend commands

This section specifies the set of commands through which a plugin, also known as an index formatting engine, is able to query the frontend. These commands are provided by the frontend and linked into the plugin interpreter.

I.e. an index formatting engine can assume that all of the following commands are present when any of its own commands (as specified in section Plugin commands) are executed.

Beyond that it can also assume that it has full access to its own safe interpreter and thus is not able to damage the other parts of the processor, nor can it damage the filesystem. It is however able to either kill or hang the whole process, by exiting, or running an infinite loop.

Coming back to the imported commands, all the commands with prefix *dt_* provide limited access to specific parts of the frontend, whereas the commands with prefix *ex_* provide access to the state of the **textutil::expander** object which does the main parsing of the input within the frontend. These commands should not be except under very special circumstances.

dt_fmap *symfname*
: Query command. It returns the actual pathname to use in the output in place of the symbolic filename *symfname*. It will return the unchanged input if no mapping was established for *symfname*.

  The required mappings are established with the method **map** of a frontend, as explained in section **OBJECT METHODS** of the documentation for the package **doctools::idx**.

dt_format
: Query command. It returns the name of the format associated with the index formatting engine.

dt_read *file*
: Controlled filesystem access. Returns contents of *file* for whatever use desired by the plugin. Only files which are either in the same directory as the file containing the engine, or below it, can be loaded. Trying to load a file outside of this directory causes an error.

dt_source *file*
: Controlled filesystem access. This command allows the index formatting engine to load additional Tcl code it may need. Only files which are either in the same directory as the file containing the engine, or below it, can be loaded. Trying to load a file outside of this directory causes an error.

ex_cappend *text*
: Appends a string to the output in the current context. This command should rarely be used by macros or application code.

ex_cget *varname*
: Retrieves the value of variable *varname*, defined in the current context.

ex_cis *cname*
: Determines whether or not the name of the current context is *cname*.

ex_cname
: Returns the name of the current context.

ex_cpop *cname*
: Pops a context from the context stack, returning all accumulated output in that context. The context must be named *cname*, or an error results.

ex_cpush *cname*
: Pushes a context named *cname* onto the context stack. The context must be popped by **cpop** before expansion ends or an error results.

ex_cset *varname value*

Sets variable *varname* to *value* in the current context.

ex_lb ?*newbracket*?
Returns the current value of the left macro expansion bracket; this is for use as or within a macro, when the bracket needs to be included in the output text. If *newbracket* is specified, it becomes the new bracket, and is returned.

ex_rb ?*newbracket*?
Returns the current value of the right macro expansion bracket; this is for use as or within a macro, when the bracket needs to be included in the output text. If *newbracket* is specified, it becomes the new bracket, and is returned.

Plugin commands

The plugin has to provide its own set of commands which will then be called by the frontend in a specific sequence while processing input. They fall into two categories, management and formatting. Their expected names, signatures, and responsibilities are specified in the following two subsections.

Management commands

The management commands a plugin has to provide are used by the frontend to

1. initialize and shutdown the plugin
2. determine the number of passes it has to make over the input
3. initialize and shutdown each pass
4. query and initialize engine parameters

After the plugin has been loaded and the frontend commands are established the commands will be called in the following sequence:

```
    idx_numpasses -> n
    idx_listvariables -> vars

    idx_varset var1 value1
    idx_varset var2 value2
    ...
    idx_varset varK valueK
    idx_initialize
    idx_setup 1
    ...
    idx_setup 2
    ...
    ...
    idx_setup n
    ...
    idx_postprocess
    idx_shutdown
    ...
```

I.e. first the number of passes and the set of available engine parameters is established, followed by calls setting the parameters. That second part is optional.

After that the plugin is initialized, the specified number of passes executed, the final result run through a global post processing step and at last the plugin is shutdown again. This can be followed by more conversions, restarting the sequence at **idx_varset**.

In each of the passes, i.e. after the calls of **idx_setup** the frontend will process the input and call the formatting commands as markup is encountered. This means that the sequence of formatting commands is determined by the grammar of the docidx markup language, as specified in the *docidx language syntax* specification.

A different way of looking at the sequence is:

- First some basic parameters are determined.
- Then everything starting at the first **idx_varset** to **idx_shutdown** forms a *run*, the formatting of a single input. Each run can be followed by more.
- Embedded within each run we have one or more *passes*, each starting with **idx_setup** and going until either the next **idx_setup** or **idx_postprocess** is reached.

  If more than one pass is required to perform the formatting only the output of the last pass is relevant. The output of all the previous, preparatory passes is ignored.

The commands, their names, signatures, and responsibilities are, in detail:

idx_initialize
: *Initialization/Shutdown*. This command is called at the beginning of every conversion run, as the first command of that run. Note that a run is not a pass, but may consist of multiple passes. It has to initialize the general state of the plugin, beyond the initialization done during the load. No return value is expected, and any returned value is ignored.

idx_listvariables
: *Initialization/Shutdown* and *Engine parameters*. Second command is called after the plugin code has been loaded, i.e. immediately after **idx_numpasses**. It has to return a list containing the names of the parameters the frontend can set to configure the engine. This list can be empty.

idx_numpasses
: *Initialization/Shutdown* and *Pass management*. First command called after the plugin code has been loaded. No other command of the engine will be called before it. It has to return the number of passes this engine requires to fully process the input document. This value has to be an integer number greater or equal to one.

idx_postprocess *text*
: *Initialization/Shutdown*. This command is called immediately after the last pass in a run. Its argument is the result of the conversion generated by that pass. It is provided to allow the engine to perform any global modifications of the generated document. If no post-processing is required for a specific format the command has to just return the argument.

  Expected to return a value, the final result of formatting the input.

idx_setup *n*
: *Initialization/Shutdown* and *Pass management*. This command is called at the beginning of each pass over the input in a run. Its argument is the number of the pass which has begun. Passes are counted from **1** upward. The command has to set up the internal state of the plugin for this particular pass. No return value is expected, and any returned value is ignored.

idx_shutdown

Initialization/Shutdown. This command is called at the end of every conversion run. It is the last command called in a run. It has to clean up of all the run-specific state in the plugin. After the call the engine has to be in a state which allows the initiation of another run without fear that information from the last run is leaked into this new run. No return value is expected, and any returned value is ignored.

idx_varset *varname text*

Engine parameters. This command is called by the frontend to set an engine parameter to a particular value. The parameter to change is specified by *varname*, the value to set in *text*.

The command has to throw an error if an unknown *varname* is used. Only the names returned by **idx_listvariables** have to be considered as known.

The values of all engine parameters have to persist between passes and runs.

Formatting commands

The formatting commands have to implement the formatting for the output format, for all the markup commands of the docidx markup language, except lb, rb, vset, include, and comment. These exceptions are processed by the frontend and are never seen by the plugin. In return a command for the formatting of plain text has to be provided, something which has no markup in the input at all.

This means, that each of the five markup commands specified in the *docidx language command reference* and outside of the set of exceptions listed above has an equivalent formatting command which takes the same arguments as the markup command and whose name is the name of markup command with the prefix *fmt_* added to it.

All commands are expected to format their input in some way per the semantics specified in the command reference and to return whatever part of this that they deem necessary as their result, which will be added to the output.

To avoid essentially duplicating the command reference we do not list any of the command here and simply refer the reader to the *docidx language command reference* for their signature and description. The sole exception is the plain text formatter, which has no equivalent markup command.

The calling sequence of formatting commands is not as rigid as for the management commands, but determined by the grammar of the docidx markup language, as specified in the *docidx language syntax* specification.

fmt_plain_text *text*

No associated markup command.

Called by the frontend for any plain text encountered in the input. It has to perform any and all special processing required for plain text.

The formatted text is expected as the result of the command, and added to the output. If no special processing is required it has to simply return its argument without change.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you

want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Documentation tools

Copyright

```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

doctoc

Keywords: doctoc, HTML, TMML, conversion, documentation, latex, manpage, markdown, markup, nroff, table of contents, toc, wiki

Name

doctools::toc - doctoc - Processing tables of contents

Synopsis

package require Tcl 8.5 9
package require doctools::toc ?**1.3.1**?
::doctools::toc::new *objectName* ?*-option value* ...?
::doctools::toc::help
::doctools::toc::search *path*
objectName method ?*arg arg* ...?
objectName **configure**
objectName **configure** *option*
objectName **configure -option** *value...*
objectName **cget -option**
objectName **destroy**
objectName **format** *text*
objectName **map** *symbolic actual*
objectName **parameters**
objectName **search** *path*
objectName **setparam** *name value*
objectName **warnings**

Description

This package provides a class for the creation of objects able to process and convert text written in the *doctoc* markup language into any output format X for which a *formatting engine* is available.

A reader interested in the markup language itself should start with the *doctoc language*

introduction and proceed from there to the formal specifications, i.e. the *doctoc language syntax* and the *doctoc language command reference*.

If on the other hand the reader wishes to write her own formatting engine for some format, i.e. is a *plugin writer* then reading and understanding the *doctoc plugin API reference* is an absolute necessity, as that document specifies the interaction between this package and its plugins, i.e. the formatting engines, in detail.

Public API

Package commands

::doctools::toc::new *objectName* ?-option *value* ...?

This command creates a new doctoc object with an associated Tcl command whose name is *objectName*. This *object* command is explained in full detail in the sections Object command and Object methods. The object command will be created under the current namespace if the *objectName* is not fully qualified, and in the specified namespace otherwise.

The options and their values coming after the name of the object are used to set the initial configuration of the object.

::doctools::toc::help

This is a convenience command for applications wishing to provide their user with a short description of the available formatting commands and their meanings. It returns a string containing a standard help text.

::doctools::toc::search *path*

Whenever an object created by this the package has to map the name of a format to the file containing the code for its formatting engine it will search for the file in a number of directories stored in a list. See section Format mapping for more explanations.

This list not only contains three default directories which are declared by the package itself, but is also extensible user of the package. This command is the means to do so. When given a *path* to an existing and readable directory it will prepend that directory to the list of directories to search. This means that the *path* added last is later searched through first.

An error will be thrown if the *path* either does not exist, is not a directory, or is not readable.

Object command

All commands created by **::doctools::toc::new** have the following general form and may be used to invoke various operations on their doctoc converter object.

objectName method ?*arg arg ...*?

The method **method** and its *arg*'uments determine the exact behavior of the command. See section Object methods for the detailed specifications.

Object methods

***objectName* configure**

The method returns a list of all known options and their current values when called without any arguments.

objectName* configure *option

The method behaves like the method **cget** when called with a single argument and returns the value of the option specified by said argument.

***objectName* configure -option *value*...**
The method reconfigures the specified **option**s of the object, setting them to the associated *value*s, when called with an even number of arguments, at least two.

The legal options are described in the section Object configuration.

***objectName* cget -option**
This method expects a legal configuration option as argument and will return the current value of that option for the object the method was invoked for.

The legal configuration options are described in section Object configuration.

***objectName* destroy**
This method destroys the object it is invoked for.

objectName* format *text
This method runs the *text* through the configured formatting engine and returns the generated string as its result. An error will be thrown if no **-format** was configured for the object.

The method assumes that the *text* is in *doctoc* format as specified in the companion document *doctoc_fmt*. Errors will be thrown otherwise.

objectName* map *symbolic actual
This methods add one entry to the per-object mapping from *symbolic* filenames to the *actual* uris. The object just stores this mapping and makes it available to the configured formatting engine through the command dt_fmap. This command is described in more detail in the *doctoc plugin API reference* which specifies the interaction between the objects created by this package and toc formatting engines.

***objectName* parameters**
This method returns a list containing the names of all engine parameters provided by the configured formatting engine. It will return an empty list if the object is not yet configured for a specific format.

objectName* search *path
This method extends the per-object list of paths searched for toc formatting engines. See also the command **::doctools::toc::search** on how to extend the per-package list of paths. Note that the path entered last will be searched first. For more details see section Format mapping.

objectName* setparam *name value
This method sets the *name*d engine parameter to the specified *value*. It will throw an error if the object is either not yet configured for a specific format, or if the formatting engine for the configured format does not provide a parameter with the given *name*. The list of parameters provided by the configured formatting engine can be retrieved through the method **parameters**.

***objectName* warnings**
This method returns a list containing all the warnings which were generated by the configured formatting engine during the last invocation of the method **format**.

Object configuration

All doctoc objects understand the following configuration options:

-file *file*
: The argument of this option is stored in the object and made available to the configured formatting engine through the command dt_file. This command is described in more detail in the companion document *doctoc_api* which specifies the API between the object and formatting engines.

  The default value of this option is the empty string.

  The configured formatting engine should interpret the value as the name of the file containing the document which is currently processed.

-format *text*
: The argument of this option specifies the format to generate and by implication the formatting engine to use when converting text via the method **format**. Its default value is the empty string. The method **format** cannot be used if this option is not set to a valid value at least once.

  The package will immediately try to map the given name to a file containing the code for a formatting engine generating that format. An error will be thrown if this mapping fails. In that case a previously configured format is left untouched.

  The section Format mapping explains in detail how the package and object will look for engine implementations.

Format mapping

The package and object will perform the following algorithm when trying to map a format name *foo* to a file containing an implementation of a formatting engine for *foo*:

1. If *foo* is the name of an existing file then this file is directly taken as the implementation.
2. If not, the list of per-object search paths is searched. For each directory in the list the package checks if that directory contains a file "*toc.foo*". If yes, then that file is taken as the implementation.

   Note that this list of paths is initially empty and can be extended through the object method **search**.
3. If not, the list of package paths is searched. For each directory in the list the package checks if that directory contains a file "*toc.foo*". If yes, then that file is taken as the implementation.

   This list of paths can be extended through the command **::doctools::toc::search**. It contains initially one path, the subdirectory "*mpformats*" of the directory the package itself is located in. In other words, if the package implementation "*doctoc.tcl*" is installed in the directory "*/usr/local/lib/tcllib/doctools*" then it will by default search the directory "*/usr/local/lib/tcllib/doctools/mpformats*" for format implementations.
4. The mapping fails.

Predefined engines

The package provides predefined formatting engines for the following formats. Some of the

formatting engines support engine parameters. These will be explicitly highlighted.

html
: This engine generates HTML markup, for processing by web browsers and the like. This engine supports three parameters:

  footer
  : The value for this parameter has to be valid selfcontained HTML markup for the body section of a HTML document. The default value is the empty string. The value is inserted into the generated output just before the tag, closing the body of the generated HTML.

    This can be used to insert boilerplate footer markup into the generated document.

  header
  : The value for this parameter has to be valid selfcontained HTML markup for the body section of a HTML document. The default value is the empty string. The value is inserted into the generated output just after the tag, starting the body of the generated HTML.

    This can be used to insert boilerplate header markup into the generated document.

  meta
  : The value for this parameter has to be valid selfcontained HTML markup for the header section of a HTML document. The default value is the empty string. The value is inserted into the generated output just after the tag, starting the header section of the generated HTML.

    This can be used to insert boilerplate meta data markup into the generated document, like references to a stylesheet, standard meta keywords, etc.

latex
: This engine generates output suitable for the **latex** text processor coming out of the TeX world.

list
: This engine retrieves version, section and title of the manpage from the document. As such it can be used to generate a directory listing for a set of manpages.

markdown
: This engine generates *Markdown* markup.

nroff
: This engine generates nroff output, for processing by **nroff**, or **groff**. The result will be standard man pages as they are known in the unix world.

null
: This engine generates no outout at all. This can be used if one just wants to validate some input.

tmml
: This engine generates TMML markup as specified by Joe English. The Tcl Manpage Markup Language is a derivate of XML.

wiki
: This engine generates Wiki markup as understood by Jean Claude Wippler's **wikit** application.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Documentation tools

Copyright


```
Copyright (c) 2003-2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


doctoc_intro

Keywords: doctoc_intro, markup, semantic markup, table of contents, toc

Name

doctoc_intro - doctoc introduction

Description

doctoc (short for *documentation tables of contents*) stands for a set of related, yet different, entities which are working together for the easy creation and transformation of tables of contents for documentation. These are

1. A tcl based language for the semantic markup of a table of contents. Markup is represented by Tcl commands.
2. A package providing the ability to read and transform texts written in that markup language. It is important to note that the actual transformation of the input text is delegated to plugins.
3. An API describing the interface between the package above and a plugin.

Which of the more detailed documents are relevant to the reader of this introduction depends on their role in the documentation process.

1. A *writer* of documentation has to understand the markup language itself. A beginner to doctoc should read the more informally written *doctoc language introduction* first. Having digested this the formal *doctoc language syntax* specification should become understandable. A writer experienced with doctoc may only need the *doctoc language command reference* from time to time to refresh her memory.

   While a document is written the **dtp** application can be used to validate it, and after completion it also performs the conversion into the chosen system of visual markup, be it *roff, HTML, plain text, wiki, etc. The simpler **dtplite** application makes internal use of doctoc when handling directories of documentation, automatically generating a proper table of contents for them.
2. A *processor* of documentation written in the *doctoc* markup language has to know which tools

are available for use.

The main tool is the aforementioned **dtp** application provided by Tcllib. The simpler **dtplite** does not expose doctoc to the user. At the bottom level, common to both applications, however sits the package **doctoools::toc**, providing the basic facilities to read and process files containing text in the doctoc format.

3. At last, but not least, *plugin writers* have to understand the interaction between the **doctools::toc** package and its plugins, as described in the *doctoc plugin API reference*.

Related formats

doctoc does not stand alone, it has two companion formats. These are called *docidx* and *doctools*, and they are for the markup of *keyword indices*, and general documentation, respectively. They are described in their own sets of documents, starting at the *docidx introduction* and the *doctools introduction*, respectively.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Documentation tools

Copyright


```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


doctoc_lang_cmdref

Keywords: doctoc_lang_cmdref, doctoc commands, doctoc language, doctoc markup, markup, semantic markup

Name

doctoc_lang_cmdref - doctoc language command reference

Synopsis

comment *plaintext*
division_end
division_start *text* ?*symfile*?
include *filename*
item *file text desc*
lb
rb

toc_begin *text title*
toc_end
vset *varname value*
vset *varname*

Description

This document specifies both names and syntax of all the commands which together are the doctoc markup language, version 1. As this document is intended to be a reference the commands are listed in alphabetical order, and the descriptions are relatively short. A beginner should read the much more informally written *doctoc language introduction* first.

Commands

comment *plaintext*
: Toc markup. The argument text is marked up as a comment standing outside of the actual text of the document. Main use is in free-form text.

division_end
: Toc structure. This command closes the division opened by the last **division_begin** command coming before it, and not yet closed.

division_start *text* ?*symfile*?
: Toc structure. This command opens a division in the table of contents. Its counterpart is **division_end**. Together they allow a user to give a table of contents additional structure.

  The title of the new division is provided by the argument *text*.

  If the symbolic filename *symfile* is present then the section title should link to the referenced document, if links are supported by the output format.

include *filename*
: Templating. The contents of the named file are interpreted as text written in the doctoc markup and processed in the place of the include command. The markup in the file has to be self-contained. It is not possible for a markup command to cross the file boundaries.

item *file text desc*
: Toc structure. This command adds an individual element to the table of contents. Each such element refers to a document. The document is specified through the symbolic name *file*. The *text* argument is used to label the reference, whereas the *desc* provides a short descriptive text of that document.

  The symbolic names are used to preserve the convertibility of this format to any output format. The actual name of the file will be inserted by the chosen formatting engine when converting the input. This will be based on a mapping from symbolic to actual names given to the engine.

lb
: Text. The command is replaced with a left bracket. Use in free-form text. Required to avoid interpretation of a left bracket as the start of a markup command. Its usage is restricted to the arguments of other markup commands.

rb
: Text. The command is replaced with a right bracket. Use in free-form text. Required to avoid

interpretation of a right bracket as the end of a markup command. Its usage is restricted to the arguments of other commands.

toc_begin *text title*
Document structure. The command to start a table of contents. The arguments are a label for the whole group of documents the index refers to (*text*) and the overall title text for the index (*title*), without markup.

The label often is the name of the package (or extension) the documents belong to.

toc_end
Document structure. Command to end a table of contents. Anything in the document coming after this command is in error.

vset *varname value*
Templating. In this form the command sets the named document variable to the specified *value*. It does not generate output. I.e. the command is replaced by the empty string.

vset *varname*
Templating. In this form the command is replaced by the value of the named document variable

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Documentation tools

Copyright


```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


doctoc_lang_faq

Keywords: doctoc_lang_faq, doctoc commands, doctoc language, doctoc markup, doctoc syntax, examples, faq, markup, semantic markup

Name

doctoc_lang_faq - doctoc language faq

Description

Overview

What is this document?

This document is currently mainly a placeholder, to be filled with commonly asked questions about the doctoc markup language and companions, and their answers.

Please report any questions (and, if possible, answers) we should consider for this document as explained in the section Bugs, Ideas, Feedback below.

Examples

Where do I find doctoc examples?

We have no direct examples of documents written using doctoc markup. However the doctools processor **dtplite** does generate a table of contents when processing a set of documents written in doctools markup. The intermediate file for it uses doctoc markup and is not deleted when generation completes. Such files can therefore serve as examples.

dtplite is distributed as part of Tcllib, so to get it you need one of

1. A snapshot of Tcllib. How to retrieve such a snapshot and the tools required for this are described at *Development Snapshots* [/wiki?name=Development+Snapshots]
2. A Tcllib release archive. They are available at the *home* [/home] page.

Bugs, Ideas, Feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Documentation tools

Copyright


```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


doctoc_lang_intro

Keywords: doctoc_lang_intro, doctoc commands, doctoc language, doctoc markup, doctoc syntax, markup, semantic markup

Name

doctoc_lang_intro - doctoc language introduction

Description

This document is an informal introduction to version 1.1 of the doctoc markup language based on a multitude of examples. After reading this a writer should be ready to understand the two parts of the formal specification, i.e. the *doctoc language syntax* specification and the *doctoc language command reference*.

Fundamentals

While the *doctoc markup language* is quite similar to the *doctools markup language*, in the broadest terms possible, there is one key difference. A table of contents consists essentially only of markup commands, with no plain text interspersed between them, except for whitespace.

Each markup command is a Tcl command surrounded by a matching pair of **[** and **]**. Inside of these delimiters the usual rules for a Tcl command apply with regard to word quotation, nested commands, continuation lines, etc. I.e.

```
    ... [division_start {Appendix 1}] ...
```

```
  ... [item thefile \
          label {file description}] ...
```

Basic structure

The most simple document which can be written in doctoc is

```
    [toc_begin GROUPTITLE TITLE]
    [toc_end]
```

This also shows us that all doctoc documents consist of only one part where we will list *items* and *divisions*.

The user is free to mix these as she sees fit. This is a change from version 1 of the language, which did not allow this mixing, but only the use of either a series of items or a series of divisions.

We will discuss the commands for each of these two possibilities in the next sections.

Items

Use the command item to put an *item* into a table of contents. This is essentially a reference to a section, subsection, etc. in the document, or set of documents, the table of contents is for. The command takes three arguments, a symbolic name for the file the item is for and two text to label the item and describe the referenced section.

Symbolic names are used to preserve the convertibility of this format to any output format. The actual name of any file will be inserted by the chosen formatting engine when converting the input, based on a mapping from symbolic to actual names given to the engine.

Here a made up example for a table of contents of this document:

```
[toc_begin Doctoc {Language Introduction}]
[item 1 DESCRIPTION]
[item 1.1 {Basic structure}]
[item 1.2 Items]
[item 1.3 Divisions]
[item 2 {FURTHER READING}]
[toc_end]
```

Divisions

One thing of notice in the last example in the previous section is that the referenced sections actually had a nested structure, something which was expressed in the item labels, by using a

common prefix for all the sections nested under section 1.

This kind of structure can be made more explicit in the doctoc language by using divisions. Instead of using a series of plain items we use a series of divisions for the major references, and then place the nested items inside of these.

Of course, instead of the nested items we can again use divisions and thus nest arbitrarily deep.

A division is marked by two commands instead of one, one to start it, the other to close the last opened division. They are:

division_start
: This command opens a new division. It takes one or two arguments, the title of the division, and the symbolic name of the file it refers to. The latter is optional. If the symbolic filename is present then the section title should link to the referenced document, if links are supported by the output format.

division_end
: This command closes the last opened and not yet closed division.

Using this we can recast the last example like this

```
[toc_begin Doctoc {Language Introduction}]
[division_start DESCRIPTION]
[item 1 {Basic structure}]
[item 2 Items]
[item 3 Divisions]
[division_end]
[division_start {FURTHER READING}]
[division_end]
[toc_end]
```

Or, to demonstrate deeper nesting

```
[toc_begin Doctoc {Language Introduction}]
[division_start DESCRIPTION]
[division_start {Basic structure}]
[item 1 Do]
[item 2 Re]
[division_end]
[division_start Items]
[item a Fi]
[item b Fo]
[item c Fa]
[division_end]
[division_start Divisions]
[item 1 Sub]
[item 1 Zero]
[division_end]
[division_end]
[division_start {FURTHER READING}]
[division_end]
[toc_end]
```

And do not forget, it is possible to freely mix items and divisions, and to have empty divisions.

```
[toc_begin Doctoc {Language Introduction}]
[item 1 Do]
[division_start DESCRIPTION]
[division_start {Basic structure}]
[item 2 Re]
[division_end]
[item a Fi]
[division_start Items]
[item b Fo]
[item c Fa]
[division_end]
[division_start Divisions]
[division_end]
[division_end]
[division_start {FURTHER READING}]
[division_end]
[toc_end]
```

Advanced structure

In all previous examples we fudged a bit regarding the markup actually allowed to be used before the toc_begin command opening the document.

Instead of only whitespace the two templating commands include and vset are also allowed, to enable the writer to either set and/or import configuration settings relevant to the table of contents. I.e. it is possible to write

```
[include FILE]
[vset VAR VALUE]
[toc_begin GROUPTITLE TITLE]
...
[toc_end]
```

Even more important, these two commands are allowed anywhere where a markup command is allowed, without regard for any other structure.

```
[toc_begin GROUPTITLE TITLE]
[include FILE]
[vset VAR VALUE]
...
[toc_end]
```

The only restriction include has to obey is that the contents of the included file must be valid at the place of the inclusion. I.e. a file included before toc_begin may contain only the templating commands vset and include, a file included in a division may contain only items or divisions commands, etc.

Escapes

Beyond the 6 commands shown so far we have two more available. However their function is not the marking up of toc structure, but the insertion of characters, namely **[** and **]**. These commands, lb and rb respectively, are required because our use of [and] to bracket markup commands makes it impossible to directly use [and] within the text.

Our example of their use are the sources of the last sentence in the previous paragraph, with some highlighting added.

```
  ...
  These commands, [cmd lb] and [cmd lb] respectively, are required
  because our use of [lb] and [rb] to bracket markup commands makes it
  impossible to directly use [lb] and [rb] within the text.
  ...
```

Further reading

Now that this document has been digested the reader, assumed to be a *writer* of documentation should be fortified enough to be able to understand the formal *doctoc language syntax* specification as well. From here on out the *doctoc language command reference* will also serve as the detailed specification and cheat sheet for all available commands and their syntax.

To be able to validate a document while writing it, it is also recommended to familiarize oneself with Tclapps' ultra-configurable **dtp**.

On the other hand, doctoc is perfectly suited for the automatic generation from doctools documents, and this is the route Tcllib's easy and simple **dtplite** goes, creating a table of contents for a set of documents behind the scenes, without the writer having to do so on their own.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Documentation tools

Copyright


```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


doctoc_lang_syntax

Keywords: doctoc_lang_syntax, doctoc commands, doctoc language, doctoc markup, doctoc syntax, markup, semantic markup

Name

doctoc_lang_syntax - doctoc language syntax

Description

This document contains the formal specification of the syntax of the doctoc markup language, version 1.1 in Backus-Naur-Form. This document is intended to be a reference, complementing the *doctoc language command reference*. A beginner should read the much more informally written *doctoc language introduction* first before trying to understand either this document or the command reference.

Fundamentals

In the broadest terms possible the *doctoc markup language* is like SGML and similar languages. A document written in this language consists primarily of markup commands, with text embedded into it at some places.

Each markup command is a just Tcl command surrounded by a matching pair of **[** and **]**. Which commands are available, and their arguments, i.e. syntax is specified in the *doctoc language command reference*.

In this document we specify first the lexeme, and then the syntax, i.e. how we can mix text and markup commands with each other.

Lexical definitions

In the syntax rules listed in the next section

1. stands for all text except markup commands.
2. Any XXX stands for the markup command [xxx] including its arguments. Each markup command is a Tcl command surrounded by a matching pair of **[** and **]**. Inside of these delimiters the usual rules for a Tcl command apply with regard to word quotation, nested

commands, continuation lines, etc.
3. stands for all text consisting only of spaces, newlines, tabulators and the comment markup command.

Syntax

The rules listed here specify only the syntax of doctoc documents. The lexical level of the language was covered in the previous section.

Regarding the syntax of the (E)BNF itself

1. The construct { X } stands for zero or more occurrences of X.
2. The construct [X] stands for zero or one occurrence of X.

The syntax:

```
toc       = defs
            TOC_BEGIN
            contents
            TOC_END
            { <WHITE> }

defs      = { INCLUDE | VSET | <WHITE> }
contents  = { defs entry } [ defs ]

entry     = ITEM | division

division  = DIVISION_START
            contents
            DIVISION_END
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Documentation tools

Copyright

```
Copyright (c) 2007-2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

doctoc_plugin_apiref

Keywords: doctoc_plugin_apiref, formatting engine, markup, plugin, semantic markup, table of

contents, toc, toc formatter

Name

doctoc_plugin_apiref - doctoc plugin API reference

Synopsis

dt_fmap *symfname*
dt_format
dt_read *file*
dt_source *file*
ex_cappend *text*
ex_cget *varname*
ex_cis *cname*
ex_cname
ex_cpop *cname*
ex_cpush *cname*
ex_cset *varname value*
ex_lb ?*newbracket*?
ex_rb ?*newbracket*?
toc_initialize
toc_listvariables
toc_numpasses
toc_postprocess *text*
toc_setup *n*
toc_shutdown
toc_varset *varname text*
fmt_plain_text *text*

Description

This document is intended for *plugin writers*, i.e. developers wishing to write a toc *formatting engine* for some output format X.

It specifies the interaction between the **doctools::toc** package and its plugins, i.e. the interface any toc formatting engine has to comply with.

This document deals with version 1 of the interface.

A reader who is on the other hand more interested in the markup language itself should start with the *doctoc language introduction* and proceed from there to the formal specifications, i.e. the *doctoc language syntax* and the *doctoc language command reference*.

Overview

The API for a toc formatting engine consists of two major sections.

On the one side we have a set of commands through which the plugin is able to query the frontend. These commands are provided by the frontend and linked into the plugin interpreter. Please see section Frontend commands for their detailed specification.

And on the other side the plugin has to provide its own set of commands which will then be called by the frontend in a specific sequence while processing input. They, again, fall into two

categories, management and formatting. Please see section Plugin commands and its subsections for their detailed specification.

Frontend commands

This section specifies the set of commands through which a plugin, also known as a toc formatting engine, is able to query the frontend. These commands are provided by the frontend and linked into the plugin interpreter.

I.e. a toc formatting engine can assume that all of the following commands are present when any of its own commands (as specified in section Plugin commands) are executed.

Beyond that it can also assume that it has full access to its own safe interpreter and thus is not able to damage the other parts of the processor, nor can it damage the filesystem. It is however able to either kill or hang the whole process, by exiting, or running an infinite loop.

Coming back to the imported commands, all the commands with prefix *dt_* provide limited access to specific parts of the frontend, whereas the commands with prefix *ex_* provide access to the state of the **textutil::expander** object which does the main parsing of the input within the frontend. These commands should not be except under very special circumstances.

dt_fmap *symfname*
: Query command. It returns the actual pathname to use in the output in place of the symbolic filename *symfname*. It will return the unchanged input if no mapping was established for *symfname*.

  The required mappings are established with the method **map** of a frontend, as explained in section **OBJECT METHODS** of the documentation for the package **doctools::toc**.

dt_format
: Query command. It returns the name of the format associated with the toc formatting engine.

dt_read *file*
: Controlled filesystem access. Returns contents of *file* for whatever use desired by the plugin. Only files which are either in the same directory as the file containing the engine, or below it, can be loaded. Trying to load a file outside of this directory causes an error.

dt_source *file*
: Controlled filesystem access. This command allows the toc formatting engine to load additional Tcl code it may need. Only files which are either in the same directory as the file containing the engine, or below it, can be loaded. Trying to load a file outside of this directory causes an error.

ex_cappend *text*
: Appends a string to the output in the current context. This command should rarely be used by macros or application code.

ex_cget *varname*
: Retrieves the value of variable *varname*, defined in the current context.

ex_cis *cname*
: Determines whether or not the name of the current context is *cname*.

ex_cname
: Returns the name of the current context.

ex_cpop *cname*

Pops a context from the context stack, returning all accumulated output in that context. The context must be named *cname*, or an error results.

ex_cpush *cname*
Pushes a context named *cname* onto the context stack. The context must be popped by **cpop** before expansion ends or an error results.

ex_cset *varname value*
Sets variable *varname* to *value* in the current context.

ex_lb ?*newbracket*?
Returns the current value of the left macro expansion bracket; this is for use as or within a macro, when the bracket needs to be included in the output text. If *newbracket* is specified, it becomes the new bracket, and is returned.

ex_rb ?*newbracket*?
Returns the current value of the right macro expansion bracket; this is for use as or within a macro, when the bracket needs to be included in the output text. If *newbracket* is specified, it becomes the new bracket, and is returned.

Plugin commands

The plugin has to provide its own set of commands which will then be called by the frontend in a specific sequence while processing input. They fall into two categories, management and formatting. Their expected names, signatures, and responsibilities are specified in the following two subsections.

Management commands

The management commands a plugin has to provide are used by the frontend to

1. initialize and shutdown the plugin
2. determine the number of passes it has to make over the input
3. initialize and shutdown each pass
4. query and initialize engine parameters

After the plugin has been loaded and the frontend commands are established the commands will be called in the following sequence:

```
    toc_numpasses -> n
    toc_listvariables -> vars

    toc_varset var1 value1
    toc_varset var2 value2
    ...
    toc_varset varK valueK
    toc_initialize
    toc_setup 1
    ...
    toc_setup 2
    ...
    ...
    toc_setup n
    ...
```

```
    toc_postprocess
    toc_shutdown
    ...
```

I.e. first the number of passes and the set of available engine parameters is established, followed by calls setting the parameters. That second part is optional.

After that the plugin is initialized, the specified number of passes executed, the final result run through a global post processing step and at last the plugin is shutdown again. This can be followed by more conversions, restarting the sequence at **toc_varset**.

In each of the passes, i.e. after the calls of **toc_setup** the frontend will process the input and call the formatting commands as markup is encountered. This means that the sequence of formatting commands is determined by the grammar of the doctoc markup language, as specified in the *doctoc language syntax* specification.

A different way of looking at the sequence is:

- First some basic parameters are determined.
- Then everything starting at the first **toc_varset** to **toc_shutdown** forms a *run*, the formatting of a single input. Each run can be followed by more.
- Embedded within each run we have one or more *passes*, each starting with **toc_setup** and going until either the next **toc_setup** or **toc_postprocess** is reached.

  If more than one pass is required to perform the formatting only the output of the last pass is relevant. The output of all the previous, preparatory passes is ignored.

The commands, their names, signatures, and responsibilities are, in detail:

toc_initialize
: *Initialization/Shutdown*. This command is called at the beginning of every conversion run, as the first command of that run. Note that a run is not a pass, but may consist of multiple passes. It has to initialize the general state of the plugin, beyond the initialization done during the load. No return value is expected, and any returned value is ignored.

toc_listvariables
: *Initialization/Shutdown* and *Engine parameters*. Second command is called after the plugin code has been loaded, i.e. immediately after **toc_numpasses**. It has to return a list containing the names of the parameters the frontend can set to configure the engine. This list can be empty.

toc_numpasses
: *Initialization/Shutdown* and *Pass management*. First command called after the plugin code has been loaded. No other command of the engine will be called before it. It has to return the number of passes this engine requires to fully process the input document. This value has to be an integer number greater or equal to one.

toc_postprocess *text*
: *Initialization/Shutdown*. This command is called immediately after the last pass in a run. Its argument is the result of the conversion generated by that pass. It is provided to allow the engine to perform any global modifications of the generated document. If no post-processing is required for a specific format the command has to just return the argument.

  Expected to return a value, the final result of formatting the input.

toc_setup *n*
: *Initialization/Shutdown* and *Pass management*. This command is called at the beginning of each pass over the input in a run. Its argument is the number of the pass which has begun. Passes are counted from **1** upward. The command has to set up the internal state of the plugin for this particular pass. No return value is expected, and any returned value is ignored.

toc_shutdown
: *Initialization/Shutdown*. This command is called at the end of every conversion run. It is the last command called in a run. It has to clean up of all the run-specific state in the plugin. After the call the engine has to be in a state which allows the initiation of another run without fear that information from the last run is leaked into this new run. No return value is expected, and any returned value is ignored.

toc_varset *varname text*
: *Engine parameters*. This command is called by the frontend to set an engine parameter to a particular value. The parameter to change is specified by *varname*, the value to set in *text*.

  The command has to throw an error if an unknown *varname* is used. Only the names returned by **toc_listvariables** have to be considered as known.

  The values of all engine parameters have to persist between passes and runs.

Formatting commands

The formatting commands have to implement the formatting for the output format, for all the markup commands of the doctoc markup language, except lb, rb, vset, include, and comment. These exceptions are processed by the frontend and are never seen by the plugin. In return a command for the formatting of plain text has to be provided, something which has no markup in the input at all.

This means, that each of the five markup commands specified in the *doctoc language command reference* and outside of the set of exceptions listed above has an equivalent formatting command which takes the same arguments as the markup command and whose name is the name of markup command with the prefix *fmt_* added to it.

All commands are expected to format their input in some way per the semantics specified in the command reference and to return whatever part of this that they deem necessary as their result, which will be added to the output.

To avoid essentially duplicating the command reference we do not list any of the command here and simply refer the reader to the *doctoc language command reference* for their signature and description. The sole exception is the plain text formatter, which has no equivalent markup command.

The calling sequence of formatting commands is not as rigid as for the management commands, but determined by the grammar of the doctoc markup language, as specified in the *doctoc language syntax* specification.

fmt_plain_text *text*
: *No associated markup command*.

  Called by the frontend for any plain text encountered in the input. It has to perform any and all special processing required for plain text.

The formatted text is expected as the result of the command, and added to the output. If no special processing is required it has to simply return its argument without change.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Documentation tools

Copyright


```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


doctools

Keywords: doctools, HTML, TMML, conversion, documentation, manpage, markdown, markup, nroff

Name

doctools - doctools - Processing documents

Synopsis

package require Tcl 8.5 9
package require doctools ?**1.6.1**?
::doctools::new *objectName* ?*option value...*?
::doctools::help
::doctools::search *path*
objectName method ?*arg arg ...*?
objectName **configure**
objectName **configure** *option*
objectName **configure -option** *value...*
objectName **cget -option**
objectName **destroy**
objectName **format** *text*
objectName **map** *symbolic actual*
objectName **parameters**
objectName **search** *path*
objectName **setparam** *name value*
objectName **warnings**

Description

This package provides a class for the creation of objects able to process and convert text written in the *doctools* markup language into any output format X for which a *formatting engine* is available.

A reader interested in the markup language itself should start with the *doctools language introduction* and proceed from there to the formal specifications, i.e. the *doctools language syntax* and the *doctools language command reference*.

If on the other hand the reader wishes to write her own formatting engine for some format, i.e. is a *plugin writer* then reading and understanding the *doctools plugin API reference* is an absolute necessity, as that document specifies the interaction between this package and its plugins, i.e. the formatting engines, in detail.

Public API

Package commands

::doctools::new *objectName* ?*option value*...?

This command creates a new doctools object with an associated Tcl command whose name is *objectName*. This *object* command is explained in full detail in the sections Object command and Object methods. The object command will be created under the current namespace if the *objectName* is not fully qualified, and in the specified namespace otherwise.

The options and their values coming after the name of the object are used to set the initial configuration of the object.

::doctools::help

This is a convenience command for applications wishing to provide their user with a short description of the available formatting commands and their meanings. It returns a string containing a standard help text.

::doctools::search *path*

Whenever an object created by this the package has to map the name of a format to the file containing the code for its formatting engine it will search for the file in a number of directories stored in a list. See section Format mapping for more explanations.

This list not only contains three default directories which are declared by the package itself, but is also extensible user of the package. This command is the means to do so. When given a *path* to an existing and readable directory it will prepend that directory to the list of directories to search. This means that the *path* added last is later searched through first.

An error will be thrown if the *path* either does not exist, is not a directory, or is not readable.

Object command

All commands created by **::doctools::new** have the following general form and may be used to invoke various operations on their doctools converter object.

objectName method ?*arg arg ...*?

The method **method** and its *arg*'uments determine the exact behavior of the command. See section Object methods for the detailed specifications.

Object methods

***objectName* configure**

The method returns a list of all known options and their current values when called without any arguments.

objectName* configure *option

The method behaves like the method **cget** when called with a single argument and returns the value of the option specified by said argument.

***objectName* configure -option *value*...**

The method reconfigures the specified **option**s of the object, setting them to the associated *value*s, when called with an even number of arguments, at least two.

The legal options are described in the section Object configuration.

***objectName* cget -option**

This method expects a legal configuration option as argument and will return the current value of that option for the object the method was invoked for.

The legal configuration options are described in section Object configuration.

***objectName* destroy**

This method destroys the object it is invoked for.

objectName* format *text

This method runs the *text* through the configured formatting engine and returns the generated string as its result. An error will be thrown if no **-format** was configured for the object.

The method assumes that the *text* is in *doctools* format as specified in the companion document *doctools_fmt*. Errors will be thrown otherwise.

objectName* map *symbolic actual

This methods add one entry to the per-object mapping from *symbolic* filenames to the *actual* uris. The object just stores this mapping and makes it available to the configured formatting engine through the command dt_fmap. This command is described in more detail in the *doctools plugin API reference* which specifies the interaction between the objects created by this package and doctools formatting engines.

***objectName* parameters**

This method returns a list containing the names of all engine parameters provided by the configured formatting engine. It will return an empty list if the object is not yet configured for a specific format.

objectName* search *path

This method extends the per-object list of paths searched for doctools formatting engines. See also the command **::doctools::search** on how to extend the per-package list of paths. Note that the path entered last will be searched first. For more details see section Format mapping.

objectName* setparam *name value

This method sets the *name*d engine parameter to the specified *value*. It will throw an error if the object is either not yet configured for a specific format, or if the formatting engine for the configured format does not provide a parameter with the given *name*. The list of parameters

provided by the configured formatting engine can be retrieved through the method **parameters**.

***objectName* warnings**
This method returns a list containing all the warnings which were generated by the configured formatting engine during the last invocation of the method **format**.

Object configuration

All doctools objects understand the following configuration options:

-file *file*
The argument of this option is stored in the object and made available to the configured formatting engine through the commands dt_file and dt_mainfile. These commands are described in more detail in the companion document *doctools_api* which specifies the API between the object and formatting engines.

The default value of this option is the empty string.

The configured formatting engine should interpret the value as the name of the file containing the document which is currently processed.

-ibase *file*
The argument of this option is stored in the object and used as the base path for resolution of relative include paths. If this option is not set (empty string) the value of **-file** is used instead.

Note that **-file** and **-ibase**, while similar looking, are actually very different. The value of **-file** is used by some engines for the generation of proper relative references between output documents (HTML). As such this is a *destination* path. The **-ibase** on the other hand is used to resolve relative include paths, and as such deals with *source* paths.

The default value of this option is the empty string.

-module *text*
The argument of this option is stored in the object and made available to the configured formatting engine through the command dt_module. This command is described in more detail in the companion document *doctools_api* which specifies the API between the object and formatting engines.

The default value of this option is the empty string.

The configured formatting engine should interpret the value as the name of the module the file containing the document which is currently processed belongs to.

-format *text*
The argument of this option specifies the format to generate and by implication the formatting engine to use when converting text via the method **format**. Its default value is the empty string. The method **format** cannot be used if this option is not set to a valid value at least once.

The package will immediately try to map the given name to a file containing the code for a formatting engine generating that format. An error will be thrown if this mapping fails. In that

case a previously configured format is left untouched.

The section Format mapping explains in detail how the package and object will look for engine implementations.

-deprecated *boolean*

This option is a boolean flag. The object will generate warnings if this flag is set and the text given to method **format** contains the deprecated markup command strong. Its default value is **FALSE**. In other words, no warnings will be generated.

-copyright *text*

The argument of this option is stored in the object and made available to the configured formatting engine through the command dt_copyright. This command is described in more detail in the companion document *doctools_api* which specifies the API between the object and formatting engines.

The default value of this option is the empty string.

The configured formatting engine should interpret the value as a copyright assignment for the document which is currently processed, or the package described by it.

This information must be used if and only if the engine is unable to find any copyright assignments within the document itself. Such are specified by the formatting command copyright. This command is described in more detail in the companion document *doctools_fmt* which specifies the *doctools* format itself.

Format mapping

The package and object will perform the following algorithm when trying to map a format name *foo* to a file containing an implementation of a formatting engine for *foo*:

1. If *foo* is the name of an existing file then this file is directly taken as the implementation.
2. If not, the list of per-object search paths is searched. For each directory in the list the package checks if that directory contains a file "*fmt.foo*". If yes, then that file is taken as the implementation.

   Note that this list of paths is initially empty and can be extended through the object method **search**.
3. If not, the list of package paths is searched. For each directory in the list the package checks if that directory contains a file "*fmt.foo*". If yes, then that file is taken as the implementation.

   This list of paths can be extended through the command **::doctools::search**. It contains initially one path, the subdirectory "*mpformats*" of the directory the package itself is located in. In other words, if the package implementation "*doctools.tcl*" is installed in the directory "*/usr/local/lib/tcllib/doctools*" then it will by default search the directory "*/usr/local/lib/tcllib/doctools/mpformats*" for format implementations.
4. The mapping fails.

Predefined engines

The package provides predefined engines for the following formats. Some of the engines support parameters. These will be explained below as well.

html

This engine generates HTML markup, for processing by web browsers and the like. This engine supports four parameters:

footer

The value for this parameter has to be valid selfcontained HTML markup for the body section of a HTML document. The default value is the empty string. The value is inserted into the generated output just before the tag, closing the body of the generated HTML.

This can be used to insert boilerplate footer markup into the generated document.

header

The value for this parameter has to be valid selfcontained HTML markup for the body section of a HTML document. The default value is the empty string. The value is inserted into the generated output just after the tag, starting the body of the generated HTML.

This can be used to insert boilerplate header markup into the generated document.

meta

The value for this parameter has to be valid selfcontained HTML markup for the header section of a HTML document. The default value is the empty string. The value is inserted into the generated output just after the tag, starting the header section of the generated HTML.

This can be used to insert boilerplate meta data markup into the generated document, like references to a stylesheet, standard meta keywords, etc.

xref

The value for this parameter has to be a list of triples specifying cross-reference information. This information is used by the engine to create more hyperlinks. Each triple is a list containing a pattern, symbolic filename and fragment reference, in this order. If a pattern is specified multiple times the last occurrence of the pattern will be used.

The engine will consult the xref database when encountering specific commands and will create a link if the relevant text matches one of the patterns. No link will be created if no match was found. The link will go to the uri **file#fragment** listed in the relevant triple, after conversion of the symbolic file name to the actual uri via dt_fmap (see the *doctools plugin API reference*). This file-to-uri mapping was build by calls to the method **map** of the doctools object (See section Object methods).

The following formatting commands will consult the xref database:

cmd ***word***

The command will look for the patterns **sa,***word*, and *word*, in this order. If this fails if it will convert *word* to all lowercase and try again.

syscmd ***word***

The command will look for the patterns **sa,***word*, and *word*, in this order. If this fails if it will convert *word* to all lowercase and try again.

term ***word***

The command will look for the patterns **kw,***word*, **sa,***word*, and *word*, in this order. If this fails if it will convert *word* to all lowercase and try again.

package *word*

The command will look for the patterns **sa,***word*, **kw,***word*, and *word*, in this order. If this fails if it will convert *word* to all lowercase and try again.

see_also ***word...***
The command will look for the patterns **sa,***word*, and *word*, in this order, for each *word* given to the command. If this fails if it will convert *word* to all lowercase and try again.

keywords ***word...***
The command will look for the patterns **kw,***word*, and *word*, in this order, for each *word* given to the command. If this fails if it will convert *word* to all lowercase and try again.

latex
This engine generates output suitable for the **latex** text processor coming out of the TeX world.

list
This engine retrieves version, section and title of the manpage from the document. As such it can be used to generate a directory listing for a set of manpages.

nroff
This engine generates nroff output, for processing by **nroff**, or **groff**. The result will be standard man pages as they are known in the unix world.

markdown
This engine generates *Markdown* markup. This engine supports two parameters:

header
The value for this parameter has to be valid selfcontained markdown markup for the body section of a markdown document. The default value is the empty string. The value is inserted into the generated output just before the table of contents.

This can be used to insert boilerplate header markup into the generated document.

xref
The value for this parameter has to be a list of triples specifying cross-reference information.

The full details of expected syntax and engine-internal use are explained above for the *html* engine.

null
This engine generates no outout at all. This can be used if one just wants to validate some input.

tmml
This engine generates TMML markup as specified by Joe English. The Tcl Manpage Markup Language is a derivate of XML.

wiki
This engine generates Wiki markup as understood by Jean Claude Wippler's **wikit** application.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib

Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Documentation tools

Copyright


```
Copyright (c) 2003-2024 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


doctools_intro

Keywords: doctools_intro, markup, semantic markup

Name

doctools_intro - doctools introduction

Description

doctools (short for *documentation tools*) stands for a set of related, yet different, entities which are working together for the easy creation and transformation of documentation. These are

1. A tcl based language for the semantic markup of text. Markup is represented by Tcl commands interspersed with the actual text.
2. A package providing the ability to read and transform texts written in that markup language. It is important to note that the actual transformation of the input text is delegated to plugins.
3. An API describing the interface between the package above and a plugin.

Which of the more detailed documents are relevant to the reader of this introduction depends on their role in the documentation process.

1. A *writer* of documentation has to understand the markup language itself. A beginner to doctools should read the more informally written *doctools language introduction* first. Having digested this the formal *doctools language syntax* specification should become understandable. A writer experienced with doctools may only need the *doctools language command reference* from time to time to refresh her memory.

   While a document is written the **dtplite** application can be used to validate it, and after completion it also performs the conversion into the chosen system of visual markup, be it *roff, HTML, plain text, wiki, etc.
2. A *processor* of documentation written in the *doctools* markup language has to know which tools are available for use.

   The main tool is the aforementioned **dtplite** application provided by Tcllib. A more powerful one (in terms of options and ability to configure it) is the **dtp** application, provided by Tclapps. At the bottom level, common to both applications, however sits the package **doctools**, providing the basic facilities to read and process files containing text in the doctools format.
3. At last, but not least, *plugin writers* have to understand the interaction between the **doctools**

package and its plugins, as described in the *doctools plugin API reference*.

Related formats

doctools does not stand alone, it has two companion formats. These are called *docidx* and *doctoc*, and they are for the markup of *keyword indices*, and *tables of contents*, respectively. They are described in their own sets of documents, starting at the *docidx introduction* and the *doctoc introduction*, respectively.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Documentation tools

Copyright


```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


doctools_lang_cmdref

Keywords: doctools_lang_cmdref, doctools commands, doctools language, doctools markup, markup, semantic markup

Name

doctools_lang_cmdref - doctools language command reference

Synopsis

arg *text*
arg_def *type name* ?*mode*?
bullet *(deprecated — use* **item)**
call *args*
category *text*
class *text*
cmd *text*
cmd_def *command*
comment *plaintext*
const *text*
copyright *text*
def *text*
description
enum

emph *text*
example *text*
example_begin
example_end
file *text*
fun *text*
image *name* ?*label*?
include *filename*
item
keywords *args*
lb
list_begin *what*
list_end
lst_item *text (deprecated — use* **def)**
manpage_begin *command section version*
manpage_end
method *text*
moddesc *text*
namespace *text*
nl *(deprecated — use* **para)**
opt *text*
opt_def *name* ?*arg*?
option *text*
package *text*
para
rb
require *package* ?*version...*?
section *name*
sectref *id* ?*text*?
sectref-external *text*
see_also *args*
strong *text (deprecated — use* **emph)**
subsection *name*
syscmd *text*
term *text*
titledesc *desc*
tkoption_def *name dbname dbclass*
type *text*
uri *text* ?*text*?
usage *args*
var *text*
vset *varname value*
vset *varname*
widget *text*

Description

This document specifies both names and syntax of all the commands which together are the doctools markup language, version 1. As this document is intended to be a reference the commands are listed in alphabetical order, and the descriptions are relatively short. A beginner should read the much more informally written *doctools language introduction* first.

Commands

arg *text*
: Text markup. The argument text is marked up as the *argument* of a command. Main uses are the highlighting of command arguments in free-form text, and for the argument parameters of the markup commands **call** and **usage**.

arg_def *type name* ?*mode*?
: Text structure. List element. Argument list. Automatically closes the previous list element. Specifies the data-*type* of the described argument of a command, its *name* and its i/o-*mode*. The latter is optional.

bullet *(deprecated — use* item*)*
: *Deprecated*. Text structure. List element. Itemized list. See **item** for the canonical command to open a list item in an itemized list.

call *args*
: Text structure. List element. Definition list. Automatically closes the previous list element. Defines the term as a command and its arguments. The first argument is the name of the command described by the following free-form text, and all arguments coming after that are descriptions of the command's arguments. It is expected that the arguments are marked up with **arg**, **method**, **option** etc., as is appropriate, and that the command itself is marked up with **cmd**. It is expected that the formatted term is not only printed in place, but also in the table of contents of the document, or synopsis, depending on the output format.

category *text*
: Document information. Anywhere. This command registers its plain text arguments as the category this document belongs to. If this command is used multiple times the last value specified is used.

class *text*
: Text markup. The argument is marked up as the name of a *class*. The text may have other markup already applied to it. Main use is the highlighting of class names in free-form text.

cmd *text*
: Text markup. The argument text is marked up as the name of a *Tcl command*. The text may have other markup already applied to it. Main uses are the highlighting of commands in free-form text, and for the command parameters of the markup commands **call** and **usage**.

cmd_def *command*
: Text structure. List element. Command list. Automatically closes the previous list element. The argument specifies the name of the *Tcl command* to be described by the list element. Expected to be marked up in the output as if it had been formatted with **cmd**.

comment *plaintext*
: Text markup. The argument text is marked up as a comment standing outside of the actual text of the document. Main use is in free-form text.

const *text*
: Text markup. The argument is marked up as a *constant* value. The text may have other

markup already applied to it. Main use is the highlighting of constants in free-form text.

copyright *text*
Document information. Anywhere. The command registers the plain text argument as a copyright assignment for the manpage. When invoked more than once the assignments are accumulated.

def *text*
Text structure. List element. Definition list. Automatically closes the previous list element. The argument text is the term defined by the new list element. Text markup can be applied to it.

description
Document structure. This command separates the header from the document body. Implicitly starts a section named "DESCRIPTION" (See command **section**).

enum
Text structure. List element. Enumerated list. Automatically closes the previous list element.

emph *text*
Text markup. The argument text is marked up as emphasized. Main use is for general highlighting of pieces of free-form text without attaching special meaning to the pieces.

example *text*
Text structure, Text markup. This command marks its argument up as an *example*. Main use is the simple embedding of examples in free-form text. It should be used if the example does *not* need special markup of its own. Otherwise use a sequence of **example_begin** ... **example_end**.

example_begin
Text structure. This commands starts an example. All text until the next **example_end** belongs to the example. Line breaks, spaces, and tabs have to be preserved literally. Examples cannot be nested.

example_end
Text structure. This command closes the example started by the last **example_begin**.

file *text*
Text markup. The argument is marked up as a *file* or *directory*, i.e. in general a *path*. The text may have other markup already applied to it. Main use is the highlighting of paths in free-form text.

fun *text*
Text markup. The argument is marked up as the name of a *function*. The text may have other markup already applied to it. Main use is the highlighting of function names in free-form text.

image *name* ?*label*?
Text markup. The argument is the symbolic name of an *image* and replaced with the image itself, if a suitable variant is found by the backend. The second argument, should it be present, will be interpreted the human-readable description of the image, and put into the output in a suitable position, if such is supported by the format. The HTML format, for example, can place it into the *alt* attribute of image references.

include *filename*
Templating. The contents of the named file are interpreted as text written in the doctools markup and processed in the place of the include command. The markup in the file has to be self-contained. It is not possible for a markup command to cross the file boundaries.

item
Text structure. List element. Itemized list. Automatically closes the previous list element.

keywords *args*
Document information. Anywhere. This command registers all its plain text arguments as keywords applying to this document. Each argument is a single keyword. If this command is used multiple times all the arguments accumulate.

lb
Text. The command is replaced with a left bracket. Use in free-form text. Required to avoid interpretation of a left bracket as the start of a markup command.

list_begin *what*
Text structure. This command starts a list. The exact nature of the list is determined by the argument *what* of the command. This further determines which commands are have to be used to start the list elements. Lists can be nested, i.e. it is allowed to start a new list within a list element.

The allowed types (and their associated item commands) are:

arguments
arg_def.

commands
cmd_def.

definitions
def and **call**.

enumerated
enum

itemized
item

options
opt_def

tkoptions
tkoption_def

Additionally the following names are recognized as shortcuts for some of the regular types:

args
Short for **arguments**.

cmds
Short for **commands**.

enum
Short for **enumerated**.

item
Short for **itemized**.

opts
Short for **options**.

At last the following names are still recognized for backward compatibility, but are otherwise considered to be *deprecated*.

arg
Deprecated. See **arguments**.

bullet
Deprecated. See **itemized**.

cmd
Deprecated. See **commands**.

opt
Deprecated. See **options**.

tkoption
Deprecated. See **tkoptions**.

list_end
Text structure. This command closes the list opened by the last **list_begin** command coming before it.

lst_item *text (deprecated — use* def*)*
Deprecated. Text structure. List element. Definition list. See **def** for the canonical command to open a general list item in a definition list.

manpage_begin *command section version*
Document structure. The command to start a manpage. The arguments are the name of the *command* described by the manpage, the *section* of the manpages this manpage resides in, and the *version* of the module containing the command. All arguments have to be plain text, without markup.

manpage_end
Document structure. Command to end a manpage/document. Anything in the document coming after this command is in error.

method *text*
Text markup. The argument text is marked up as the name of an *object method*, i.e. subcommand of a Tcl command. The text may have other markup already applied to it. Main uses are the highlighting of method names in free-form text, and for the command parameters of the markup commands **call** and **usage**.

moddesc *text*
Document information. Header. Registers the plain text argument as a short description of the module the manpage resides in.

namespace *text*
Text markup. The argument text is marked up as a namespace name. The text may have other markup already applied to it. Main use is the highlighting of namespace names in free-form text.

nl *(deprecated — use* para*)*
Deprecated. Text structure. See **para** for the canonical command to insert paragraph breaks into the text.

opt *text*
Text markup. The argument text is marked up as *optional*. The text may have other markup already applied to it. Main use is the highlighting of optional arguments, see the command arg **arg**.

opt_def *name* ?*arg*?

Text structure. List element. Option list. Automatically closes the previous list element. Specifies *name* and arguments of the *option* described by the list element. It is expected that the name is marked up using **option**.

option *text*
Text markup. The argument is marked up as *option*. The text may have other markup already applied to it. Main use is the highlighting of options, also known as command-switches, in either free-form text, or the arguments of the **call** and **usage** commands.

package *text*
Text markup. The argument is marked up as the name of a *package*. The text may have other markup already applied to it. Main use is the highlighting of package names in free-form text.

para
Text structure. This command breaks free-form text into paragraphs. Each command closes the paragraph coming before it and starts a new paragraph for the text coming after it. Higher-level forms of structure are sections and subsections.

rb
Text. The command is replaced with a right bracket. Use in free-form text. Required to avoid interpretation of a right bracket as the end of a markup command.

require *package* ?*version*...?
Document information. Header. This command registers its argument *package* as the name of a package or application required by the described package or application. A set of required versions can be provided as well. This argument can be marked up. The usual markup is **opt**.

section *name*
Text structure. This command starts a new named document section. The argument has to be plain text. Implicitly closes the last paragraph coming before it and also implicitly opens the first paragraph of the new section.

sectref *id* ?*text*?
Text markup. Formats a reference to the section identified by *id*. If no *text* is specified the title of the referenced section is used in the output, otherwise *text* is used.

sectref-external *text*
Text markup. Like **sectref**, except that the section is assumed to be in a different document and therefore doesn't need to be identified, nor are any checks for existence made. Only the text to format is needed.

see_also *args*
Document information. Anywhere. The command defines direct cross-references to other documents. Each argument is a plain text label identifying the referenced document. If this command is used multiple times all the arguments accumulate.

strong *text (deprecated — use* emph*)*
Deprecated. Text markup. See **emph** for the canonical command to emphasize text.

subsection *name*
Text structure. This command starts a new named subsection of a section. The argument has to be plain text. Implicitly closes the last paragraph coming before it and also implicitly opens the first paragraph of the new subsection.

syscmd *text*
Text markup. The argument text is marked up as the name of an external command. The text may have other markup already applied to it. Main use is the highlighting of external

commands in free-form text.

term *text*
: Text markup. The argument is marked up as unspecific terminology. The text may have other markup already applied to it. Main use is the highlighting of important terms and concepts in free-form text.

titledesc *desc*
: Document information. Header. Optional. Registers the plain text argument as the title of the manpage. Defaults to the value registered by **moddesc**.

tkoption_def *name dbname dbclass*
: Text structure. List element. Widget option list. Automatically closes the previous list element. Specifies the *name* of the option as used in scripts, the name used by the option database (*dbname*), and its class (*dbclass*), i.e. its type. It is expected that the name is marked up using **option**.

type *text*
: Text markup. The argument is marked up as the name of a *data type*. The text may have other markup already applied to it. Main use is the highlighting of data types in free-form text.

uri *text* ?*text*?
: Text markup. The argument is marked up as an *uri* (i.e. a *uniform resource identifier*. The text may have other markup already applied to it. Main use is the highlighting of uris in free-form text. The second argument, should it be present, will be interpreted the human-readable description of the uri. In other words, as its label. Without an explicit label the uri will be its own label.

usage *args*
: Text markup. See **call** for the full description, this command is syntactically identical, as it is in its expectations for the markup of its arguments. In contrast to **call** it is however not allowed to generate output where this command occurs in the text. The command is *silent*. The formatted text may only appear in a different section of the output, for example a table of contents, or synopsis, depending on the output format.

var *text*
: Text markup. The argument is marked up as the name of a *variable*. The text may have other markup already applied to it. Main use is the highlighting of variables in free-form text.

vset *varname value*
: Templating. In this form the command sets the named document variable to the specified *value*. It does not generate output. I.e. the command is replaced by the empty string.

vset *varname*
: Templating. In this form the command is replaced by the value of the named document variable

widget *text*
: Text markup. The argument is marked up as the name of a *widget*. The text may have other markup already applied to it. Main use is the highlighting of widget names in free-form text.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Documentation tools

Copyright

```
Copyright (c) 2007-2010 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


doctools_lang_faq
Keywords: doctools_lang_faq, doctools commands, doctools language, doctools markup, doctools syntax, examples, faq, markup, semantic markup

Name
doctools_lang_faq - doctools language faq

Description

Overview

What is this document?
This document is currently mainly a placeholder, to be filled with commonly asked questions about the doctools markup language and companions, and their answers.

Please report any questions (and, if possible, answers) we should consider for this document as explained in the section Bugs, Ideas, Feedback below.

Examples

Where do I find doctools examples?
We have no direct examples of documents written using doctools markup. However the doctools processor **dtplite** does generate a table of contents when processing a set of documents written in doctools markup. The intermediate file for it uses doctools markup and is not deleted when generation completes. Such files can therefore serve as examples.

dtplite is distributed as part of Tcllib, so to get it you need one of

1. A snapshot of Tcllib. How to retrieve such a snapshot and the tools required for this are described at *Development Snapshots* [/wiki?name=Development+Snapshots]
2. A Tcllib release archive. They are available at the *home* [/home] page.

Bugs, Ideas, Feedback
If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Documentation tools

Copyright

```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


doctools_lang_intro
Keywords: doctools_lang_intro, doctools commands, doctools language, doctools markup, doctools syntax, markup, semantic markup

Name
doctools_lang_intro - doctools language introduction

Description
This document is an informal introduction to version 1 of the doctools markup language based on a multitude of examples. After reading this a writer should be ready to understand the two parts of the formal specification, i.e. the *doctools language syntax* specification and the *doctools language command reference*.

Fundamentals
In the broadest terms possible the *doctools markup language* is LaTeX-like, instead of like SGML and similar languages. A document written in this language consists primarily of text, with markup commands embedded into it.

Each markup command is a Tcl command surrounded by a matching pair of **[** and **]**. Inside of these delimiters the usual rules for a Tcl command apply with regard to word quotation, nested commands, continuation lines, etc. I.e.

```
... [list_begin enumerated] ...
```

```
... [call [cmd foo] \
        [arg bar]] ...
```

```
... [term {complex concept}] ...
```

```
... [opt "[arg key] [arg value]"] ...
```

Basic structure
The most simple document which can be written in doctools is

```
    [manpage_begin NAME SECTION VERSION]
[see_also doctools_intro]
[see_also doctools_lang_cmdref]
[see_also doctools_lang_faq]
[see_also doctools_lang_syntax]
[keywords {doctools commands}]
[keywords {doctools language}]
[keywords {doctools markup}]
[keywords {doctools syntax}]
[keywords markup]
[keywords {semantic markup}]
    [description]
    [vset CATEGORY doctools]
[include ../common-text/feedback.inc]
[manpage_end]
```

This also shows us that all doctools documents are split into two parts, the *header* and the *body*. Everything coming before [description] belongs to the header, and everything coming after belongs to the body, with the whole document bracketed by the two **manpage_*** commands. Before and after these opening and closing commands we have only *whitespace*.

In the remainder of this section we will discuss only the contents of the header, the structure of the body will be discussed in the section Text structure.

The header section can be empty, and otherwise may contain only an arbitrary sequence of the four so-called *header* commands, plus *whitespace*. These commands are

- titledesc
- moddesc
- require
- copyright

They provide, through their arguments, additional information about the document, like its title, the title of the larger group the document belongs to (if applicable), the requirements of the documented packages (if applicable), and copyright assignments. All of them can occur multiple times, including none, and they can be used in any order. However for titledesc and moddesc only the last occurrence is taken. For the other two the specified information is accumulated, in the given order. Regular text is not allowed within the header.

Given the above a less minimal example of a document is

```
[manpage_begin NAME SECTION VERSION]
[copyright {YEAR AUTHOR}]
[titledesc TITLE]
[moddesc   MODULE_TITLE]
[require   PACKAGE VERSION]
[require   PACKAGE]
[description]
[manpage_end]
```

Remember that the whitespace is optional. The document

```
    [manpage_begin NAME SECTION VERSION]
    [copyright {YEAR AUTHOR}][titledesc TITLE][moddesc MODULE_TITLE]
    [require PACKAGE VERSION][require PACKAGE][description]
    [vset CATEGORY doctools]
[include ../common-text/feedback.inc]
[manpage_end]
```

has the same meaning as the example before.

On the other hand, if *whitespace* is present it consists not only of any sequence of characters containing the space character, horizontal and vertical tabs, carriage return, and newline, but it may contain comment markup as well, in the form of the comment command.

```
[comment { ... }]
[manpage_begin NAME SECTION VERSION]
[copyright {YEAR AUTHOR}]
[titledesc TITLE]
[moddesc   MODULE_TITLE][comment { ... }]
[require   PACKAGE VERSION]
[require   PACKAGE]
[description]
[manpage_end]
[comment { ... }]
```

Advanced structure

In the simple examples of the last section we fudged a bit regarding the markup actually allowed to be used before the manpage_begin command opening the document.

Instead of only whitespace the two templating commands include and vset are also allowed, to enable the writer to either set and/or import configuration settings relevant to the document. I.e. it is possible to write

```
[include FILE]
[vset VAR VALUE]
[manpage_begin NAME SECTION VERSION]
[description]
[manpage_end]
```

Even more important, these two commands are allowed anywhere where a markup command is allowed, without regard for any other structure. I.e. for example in the header as well.

```
[manpage_begin NAME SECTION VERSION]
[include FILE]
[vset VAR VALUE]
[description]
[manpage_end]
```

The only restriction include has to obey is that the contents of the included file must be valid at the place of the inclusion. I.e. a file included before manpage_begin may contain only the templating commands vset and include, a file included in the header may contain only header

commands, etc.

Text structure

The body of the document consists mainly of text, possibly split into sections, subsections, and paragraphs, with parts marked up to highlight various semantic categories of text, and additional structure through the use of examples and (nested) lists.

This section explains the high-level structural commands, with everything else deferred to the following sections.

The simplest way of structuring the body is through the introduction of paragraphs. The command for doing so is para. Each occurrence of this command closes the previous paragraph and automatically opens the next. The first paragraph is automatically opened at the beginning of the body, by description. In the same manner the last paragraph automatically ends at manpage_end.

```
[manpage_begin NAME SECTION VERSION]
[description]
 ...
[para]
 ...
[para]
 ...
[manpage_end]
```

Empty paragraphs are ignored.

A structure coarser than paragraphs are sections, which allow the writer to split a document into larger, and labeled, pieces. The command for doing so is **section**. Each occurrence of this command closes the previous section and automatically opens the next, including its first paragraph. The first section is automatically opened at the beginning of the body, by description (This section is labeled "DESCRIPTION"). In the same manner the last section automatically ends at manpage_end.

Empty sections are *not* ignored. We are free to (not) use paragraphs within sections.

```
[manpage_begin NAME SECTION VERSION]
[description]
 ...
[section {Section A}]
 ...
[para]
 ...
[section {Section B}]
 ...
[manpage_end]
```

Between sections and paragraphs we have subsections, to split sections. The command for doing so is subsection. Each occurrence of this command closes the previous subsection and automatically opens the next, including its first paragraph. A subsection is automatically opened at the beginning of the body, by description, and at the beginning of each section. In the same

manner the last subsection automatically ends at manpage_end.

Empty subsections are *not* ignored. We are free to (not) use paragraphs within subsections.

```
[manpage_begin NAME SECTION VERSION]
[description]
 ...
[section {Section A}]
 ...
[subsection {Sub 1}]
 ...
[para]
 ...
[subsection {Sub 2}]
 ...
[section {Section B}]
 ...
[manpage_end]
```

Text markup

Having handled the overall structure a writer can impose on the document we now take a closer at the text in a paragraph.

While most often this is just the unadorned content of the document we do have situations where we wish to highlight parts of it as some type of thing or other, like command arguments, command names, concepts, uris, etc.

For this we have a series of markup commands which take the text to highlight as their single argument. It should be noted that while their predominant use is the highlighting of parts of a paragraph they can also be used to mark up the arguments of list item commands, and of other markup commands.

The commands available to us are

arg
: Its argument is a the name of a command argument.

class
: Its argument is a class name.

cmd
: Its argument is a command name (Tcl command).

const
: Its argument is a constant.

emph
: General, non-semantic emphasis.

file
: Its argument is a filename / path.

fun
: Its argument is a function name.

method

Its argument is a method name

namespace
: Its argument is namespace name.

opt
: Its argument is some optional syntax element.

option
: Its argument is a command line switch / widget option.

package
: Its argument is a package name.

sectref
: Its argument is the title of a section or subsection, i.e. a section reference.

syscmd
: Its argument is a command name (external, system command).

term
: Its argument is a concept, or general terminology.

type
: Its argument is a type name.

uri
: Its argument is a uniform resource identifier, i.e an external reference. A second argument can be used to specify an explicit label for the reference in question.

usage
: The arguments describe the syntax of a Tcl command.

var
: Its argument is a variable.

widget
: Its argument is a widget name.

The example demonstrating the use of text markup is an excerpt from the *doctools language command reference*, with some highlighting added. It shows their use within a block of text, as the arguments of a list item command (call), and our ability to nest them.

```
...
[call [cmd arg_def] [arg type] [arg name] [opt [arg mode]]]

Text structure. List element. Argument list. Automatically closes the
previous list element. Specifies the data-[arg type] of the described
argument of a command, its [arg name] and its i/o-[arg mode]. The
latter is optional.
...
```

Escapes

Beyond the 20 commands for simple markup shown in the previous section we have two more available which are technically simple markup. However their function is not the marking up of phrases as specific types of things, but the insertion of characters, namely **[** and **]**. These commands, lb and rb respectively, are required because our use of [and] to bracket markup commands makes it impossible to directly use [and] within the text.

Our example of their use are the sources of the last sentence in the previous paragraph, with some highlighting added.

```
...
These commands, [cmd lb] and [cmd lb] respectively, are required
because our use of [lb] and [rb] to bracket markup commands makes it
impossible to directly use [lb] and [rb] within the text.
...
```

Cross-references

The last two commands we have to discuss are for the declaration of cross-references between documents, explicit and implicit. They are keywords and see_also. Both take an arbitrary number of arguments, all of which have to be plain unmarked text. I.e. it is not allowed to use markup on them. Both commands can be used multiple times in a document. If that is done all arguments of all occurrences of one of them are put together into a single set.

keywords
: The arguments of this command are interpreted as keywords describing the document. A processor can use this information to create an index indirectly linking the containing document to all documents with the same keywords.

see_also
: The arguments of this command are interpreted as references to other documents. A processor can format them as direct links to these documents.

All the cross-reference commands can occur anywhere in the document between manpage_begin and manpage_end. As such the writer can choose whether she wants to have them at the beginning of the body, or at its end, maybe near the place a keyword is actually defined by the main content, or considers them as meta data which should be in the header, etc.

Our example shows the sources for the cross-references of this document, with some highlighting added. Incidentally they are found at the end of the body.

```
...
[see_also doctools_intro]
[see_also doctools_lang_syntax]
[see_also doctools_lang_cmdref]
[keywords markup {semantic markup}]
[keywords {doctools markup} {doctools language}]
[keywords {doctools syntax} {doctools commands}]
[manpage_end]
```

Examples

Where ever we can write plain text we can write examples too. For simple examples we have the command **example** which takes a single argument, the text of the argument. The example text must not contain markup. If we wish to have markup within an example we have to use the 2-command combination example_begin / example_end instead.

The first opens an example block, the other closes it, and in between we can write plain text and use all the regular text markup commands. Note that text structure commands are not allowed.

This also means that it is not possible to embed examples and lists within an example. On the other hand, we *can* use templating commands within example blocks to read their contents from a file (Remember section Advanced structure).

The source for the very first example in this document (see section Fundamentals), with some highlighting added, is

```
[example {
  ... [list_begin enumerated] ...
}]
```

Using example_begin / example_end this would look like

```
[example_begin]
  ... [list_begin enumerated] ...
[example_end]
```

Lists

Where ever we can write plain text we can write lists too. The main commands are list_begin to start a list, and list_end to close one. The opening command takes an argument specifying the type of list started it, and this in turn determines which of the eight existing list item commands are allowed within the list to start list items.

After the opening command only whitespace is allowed, until the first list item command opens the first item of the list. Each item is a regular series of paragraphs and is closed by either the next list item command, or the end of the list. If closed by a list item command this command automatically opens the next list item. A consequence of a list item being a series of paragraphs is that all regular text markup can be used within a list item, including examples and other lists.

The list types recognized by list_begin and their associated list item commands are:

arguments
: (arg_def) This opens an *argument (declaration) list*. It is a specialized form of a term definition list where the term is an argument name, with its type and i/o-mode.

commands
: (cmd_def) This opens a *command (declaration) list*. It is a specialized form of a term definition list where the term is a command name.

definitions
: (def and call) This opens a general *term definition list*. The terms defined by the list items are specified through the argument(s) of the list item commands, either general terms, possibly with markup (def), or Tcl commands with their syntax (call).

enumerated
: (enum) This opens a general *enumerated list*.

itemized
: (item) This opens a general *itemized list*.

options
: (opt_def) This opens an *option (declaration) list*. It is a specialized form of a term definition list where the term is an option name, possibly with the option's arguments.

tkoptions
: (tkoption_def) This opens a *widget option (declaration) list*. It is a specialized form of a term

definition list where the term is the name of a configuration option for a widget, with its name and class in the option database.

Our example is the source of the definition list in the previous paragraph, with most of the content in the middle removed.

```
...
[list_begin definitions]
[def [const arg]]

([cmd arg_def]) This opens an argument (declaration) list. It is a
specialized form of a definition list where the term is an argument
name, with its type and i/o-mode.

[def [const itemized]]

([cmd item])
This opens a general itemized list.

...
[def [const tkoption]]

([cmd tkoption_def]) This opens a widget option (declaration) list.
  It
is a specialized form of a definition list where the term is the name
of a configuration option for a widget, with its name and class in
  the
option database.

[list_end]
...
```

Note that a list cannot begin in one (sub)section and end in another. Differently said, (sub)section breaks are not allowed within lists and list items. An example of this *illegal* construct is

```
...
[list_begin itemized]
[item]
...
[section {ILLEGAL WITHIN THE LIST}]
...
[list_end]
...
```

Further reading

Now that this document has been digested the reader, assumed to be a *writer* of documentation should be fortified enough to be able to understand the formal *doctools language syntax*

specification as well. From here on out the *doctools language command reference* will also serve as the detailed specification and cheat sheet for all available commands and their syntax.

To be able to validate a document while writing it, it is also recommended to familiarize oneself with one of the applications for the processing and conversion of doctools documents, i.e. either Tcllib's easy and simple **dtplite**, or Tclapps' ultra-configurable **dtp**.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Documentation tools

Copyright


```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


doctools_lang_syntax

Keywords: doctools_lang_syntax, doctools commands, doctools language, doctools markup, doctools syntax, markup, semantic markup

Name

doctools_lang_syntax - doctools language syntax

Description

This document contains the formal specification of the syntax of the doctools markup language, version 1 in Backus-Naur-Form. This document is intended to be a reference, complementing the *doctools language command reference*. A beginner should read the much more informally written *doctools language introduction* first before trying to understand either this document or the command reference.

Fundamentals

In the broadest terms possible the *doctools markup language* is LaTeX-like, instead of like SGML and similar languages. A document written in this language consists primarily of text, with markup commands embedded into it.

Each markup command is a just Tcl command surrounded by a matching pair of **[** and **]**. Which commands are available, and their arguments, i.e. syntax is specified in the *doctools language command reference*.

In this document we specify first the lexeme, and then the syntax, i.e. how we can mix text and markup commands with each other.

Lexical definitions

In the syntax rules listed in the next section

1. stands for all text except markup commands.
2. Any XXX stands for the markup command [xxx] including its arguments. Each markup command is a Tcl command surrounded by a matching pair of **[** and **]**. Inside of these delimiters the usual rules for a Tcl command apply with regard to word quotation, nested commands, continuation lines, etc.
3. stands for all text consisting only of spaces, newlines, tabulators and the comment markup command.

Syntax

The rules listed here specify only the syntax of doctools documents. The lexical level of the language was covered in the previous section.

Regarding the syntax of the (E)BNF itself

1. The construct { X } stands for zero or more occurrences of X.
2. The construct [X] stands for zero or one occurrence of X.
3. The construct LIST_BEGIN stands for the markup command list_begin with **X** as its type argument.

The syntax:

```
manpage = defs
          MANPAGE_BEGIN
          header
          DESCRIPTION
          body
          MANPAGE_END
          { <WHITE> }

defs    = { INCLUDE | VSET | <WHITE> }

header  = { TITLEDESC | MODDESC | COPYRIGHT | REQUIRE | defs | xref }

xref    = KEYWORDS | SEE_ALSO | CATEGORY

body    = paras { SECTION    sbody  }
sbody   = paras { SUBSECTION ssbody }
ssbody  = paras

paras   = tblock { (PARA | NL) tblock }

tblock  = { <TEXT> | defs | markup | xref | an_example | a_list }

markup  = ARG     | CLASS | CMD     | CONST     | EMPH    | FILE
        | FUN     | LB    | METHOD  | NAMESPACE | OPT     | OPTION
        | PACKAGE | RB    | SECTREF | STRONG    | SYSCMD | TERM
```

```
        | TYPE    | URI    | USAGE    | VAR        | WIDGET

example = EXAMPLE
        | EXAMPLE_BEGIN extext EXAMPLE_END

extext  = { <TEXT> | defs | markup }

a_list  = LIST_BEGIN<arguments>   argd_list   LIST_END
        | LIST_BEGIN<commands>    cmdd_list   LIST_END
        | LIST_BEGIN<definitions> def_list    LIST_END
        | LIST_BEGIN<enumerated>  enum_list   LIST_END
        | LIST_BEGIN<itemized>    item_list   LIST_END
        | LIST_BEGIN<options>     optd_list   LIST_END
        | LIST_BEGIN<tkoptions>   tkoptd_list LIST_END

argd_list   = [ <WHITE> ] { ARG_DEF      paras }
cmdd_list   = [ <WHITE> ] { CMD_DEF      paras }
def_list    = [ <WHITE> ] { (DEF|CALL)   paras }
enum_list   = [ <WHITE> ] { ENUM         paras }
item_list   = [ <WHITE> ] { ITEM         paras }
optd_list   = [ <WHITE> ] { OPT_DEF      paras }
tkoptd_list = [ <WHITE> ] { TKOPTION_DEF paras }
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Documentation tools

Copyright


```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


doctools_plugin_apiref

Keywords: doctools_plugin_apiref, document, formatter, formatting engine, manpage, markup, semantic markup

Name

doctools_plugin_apiref - doctools plugin API reference

Synopsis

dt_copyright
dt_file
dt_mainfile
dt_fileid
dt_fmap *symfname*
dt_format
dt_imgdata *key extensions*
dt_imgdst *key extensions*
dt_imgsrc *key extensions*
dt_lnesting
dt_module
dt_read *file*
dt_source *file*
dt_user
ex_cappend *text*
ex_cget *varname*
ex_cis *cname*
ex_cname
ex_cpop *cname*
ex_cpush *cname*
ex_cset *varname value*
ex_lb ?*newbracket*?
ex_rb ?*newbracket*?
fmt_initialize
fmt_listvariables
fmt_numpasses
fmt_postprocess *text*
fmt_setup *n*
fmt_shutdown
fmt_varset *varname text*
fmt_plain_text *text*

Description

This document is intended for *plugin writers*, i.e. developers wishing to write a doctools *formatting engine* for some output format X.

It specifies the interaction between the **doctools** package and its plugins, i.e. the interface any doctools formatting engine has to comply with.

This document deals with version 1 of the interface.

A reader who is on the other hand more interested in the markup language itself should start with the *doctools language introduction* and proceed from there to the formal specifications, i.e. the *doctools language syntax* and the *doctools language command reference*.

Overview

The API for a doctools formatting engine consists of two major sections.

On the one side we have a set of commands through which the plugin is able to query the frontend. These commands are provided by the frontend and linked into the plugin interpreter. Please see section Frontend commands for their detailed specification.

And on the other side the plugin has to provide its own set of commands which will then be called by the frontend in a specific sequence while processing input. They, again, fall into two categories, management and formatting. Please see section Plugin commands and its subsections for their detailed specification.

Frontend commands

This section specifies the set of commands through which a plugin, also known as a doctools formatting engine, is able to query the frontend. These commands are provided by the frontend and linked into the plugin interpreter.

I.e. a doctools formatting engine can assume that all of the following commands are present when any of its own commands (as specified in section Plugin commands) are executed.

Beyond that it can also assume that it has full access to its own safe interpreter and thus is not able to damage the other parts of the processor, nor can it damage the filesystem. It is however able to either kill or hang the whole process, by exiting, or running an infinite loop.

Coming back to the imported commands, all the commands with prefix *dt_* provide limited access to specific parts of the frontend, whereas the commands with prefix *ex_* provide access to the state of the **textutil::expander** object which does the main parsing of the input within the frontend. These commands should not be except under very special circumstances.

dt_copyright
: Query command. It returns a string containing the copyright information the doctools processor was configured with. The relevant option is **-copyright**).

dt_file
: Query command. It returns the full path of the file containing the input currently processed by the engine. This may be an included file.

dt_mainfile
: Query command. It returns the full path of the toplevel file containing the input currently processed by the engine.

dt_fileid
: Query command. It returns the name of the file containing the input currently processed by the engine, without path, nor extension.

dt_fmap *symfname*
: Query command. It returns the actual pathname to use in the output in place of the symbolic filename *symfname*. It will return the unchanged input if no mapping was established for *symfname*.

  The required mappings are established with the method **map** of a frontend, as explained in section **OBJECT METHODS** of the documentation for the package **doctools**.

dt_format
: Query command. It returns the name of the format associated with the doctools formatting engine.

dt_imgdata *key extensions*

Query command. Access to the image map. Looks for an image recorded under the *key* and having on the specified *extension*. If a matching image is found its data is returned as the result of the command. Otherwise an empty string is returned.

dt_imgdst *key extensions*
Query command. Access to the image map. Looks for an image recorded under the *key* and having on the specified *extension*. If a matching image is found its destination path in the output is returned as the result of the command. Otherwise an empty string is returned.

dt_imgsrc *key extensions*
Query command. Access to the image map. Looks for an image recorded under the *key* and having on the specified *extension*. If a matching image is found its origin path is returned as the result of the command. Otherwise an empty string is returned.

dt_lnesting
Query command. It returns the number of lists currently open.

dt_module
Query command. It returns the name of the module the input currently processed belongs to.

dt_read *file*
Controlled filesystem access. Returns contents of *file* for whatever use desired by the plugin. Only files which are either in the same directory as the file containing the engine, or below it, can be loaded. Trying to load a file outside of this directory causes an error.

dt_source *file*
Controlled filesystem access. This command allows the doctools formatting engine to load additional Tcl code it may need. Only files which are either in the same directory as the file containing the engine, or below it, can be loaded. Trying to load a file outside of this directory causes an error.

dt_user
Query command. It returns the name of the current user as known to the tcl interpreter the frontend controlling the formatting engine resides in.

ex_cappend *text*
Appends a string to the output in the current context. This command should rarely be used by macros or application code.

ex_cget *varname*
Retrieves the value of variable *varname*, defined in the current context.

ex_cis *cname*
Determines whether or not the name of the current context is *cname*.

ex_cname
Returns the name of the current context.

ex_cpop *cname*
Pops a context from the context stack, returning all accumulated output in that context. The context must be named *cname*, or an error results.

ex_cpush *cname*
Pushes a context named *cname* onto the context stack. The context must be popped by **cpop** before expansion ends or an error results.

ex_cset *varname value*
Sets variable *varname* to *value* in the current context.

ex_lb ?*newbracket*?
: Returns the current value of the left macro expansion bracket; this is for use as or within a macro, when the bracket needs to be included in the output text. If *newbracket* is specified, it becomes the new bracket, and is returned.

ex_rb ?*newbracket*?
: Returns the current value of the right macro expansion bracket; this is for use as or within a macro, when the bracket needs to be included in the output text. If *newbracket* is specified, it becomes the new bracket, and is returned.

Plugin commands

The plugin has to provide its own set of commands which will then be called by the frontend in a specific sequence while processing input. They fall into two categories, management and formatting. Their expected names, signatures, and responsibilities are specified in the following two subsections.

Management commands

The management commands a plugin has to provide are used by the frontend to

1. initialize and shutdown the plugin
2. determine the number of passes it has to make over the input
3. initialize and shutdown each pass
4. query and initialize engine parameters

After the plugin has been loaded and the frontend commands are established the commands will be called in the following sequence:

```
    fmt_numpasses -> n
    fmt_listvariables -> vars

    fmt_varset var1 value1
    fmt_varset var2 value2
    ...
    fmt_varset varK valueK
    fmt_initialize
    fmt_setup 1
    ...
    fmt_setup 2
    ...
    ...
    fmt_setup n
    ...
    fmt_postprocess
    fmt_shutdown
    ...
```

I.e. first the number of passes and the set of available engine parameters is established, followed by calls setting the parameters. That second part is optional.

After that the plugin is initialized, the specified number of passes executed, the final result run

through a global post processing step and at last the plugin is shutdown again. This can be followed by more conversions, restarting the sequence at **fmt_varset**.

In each of the passes, i.e. after the calls of **fmt_setup** the frontend will process the input and call the formatting commands as markup is encountered. This means that the sequence of formatting commands is determined by the grammar of the doctools markup language, as specified in the *doctools language syntax* specification.

A different way of looking at the sequence is:

- First some basic parameters are determined.
- Then everything starting at the first **fmt_varset** to **fmt_shutdown** forms a *run*, the formatting of a single input. Each run can be followed by more.
- Embedded within each run we have one or more *passes*, each starting with **fmt_setup** and going until either the next **fmt_setup** or **fmt_postprocess** is reached.

  If more than one pass is required to perform the formatting only the output of the last pass is relevant. The output of all the previous, preparatory passes is ignored.

The commands, their names, signatures, and responsibilities are, in detail:

fmt_initialize
: *Initialization/Shutdown*. This command is called at the beginning of every conversion run, as the first command of that run. Note that a run is not a pass, but may consist of multiple passes. It has to initialize the general state of the plugin, beyond the initialization done during the load. No return value is expected, and any returned value is ignored.

fmt_listvariables
: *Initialization/Shutdown* and *Engine parameters*. Second command is called after the plugin code has been loaded, i.e. immediately after **fmt_numpasses**. It has to return a list containing the names of the parameters the frontend can set to configure the engine. This list can be empty.

fmt_numpasses
: *Initialization/Shutdown* and *Pass management*. First command called after the plugin code has been loaded. No other command of the engine will be called before it. It has to return the number of passes this engine requires to fully process the input document. This value has to be an integer number greater or equal to one.

fmt_postprocess *text*
: *Initialization/Shutdown*. This command is called immediately after the last pass in a run. Its argument is the result of the conversion generated by that pass. It is provided to allow the engine to perform any global modifications of the generated document. If no post-processing is required for a specific format the command has to just return the argument.

  Expected to return a value, the final result of formatting the input.

fmt_setup *n*
: *Initialization/Shutdown* and *Pass management*. This command is called at the beginning of each pass over the input in a run. Its argument is the number of the pass which has begun. Passes are counted from **1** upward. The command has to set up the internal state of the plugin for this particular pass. No return value is expected, and any returned value is ignored.

fmt_shutdown
: *Initialization/Shutdown*. This command is called at the end of every conversion run. It is the

last command called in a run. It has to clean up of all the run-specific state in the plugin. After the call the engine has to be in a state which allows the initiation of another run without fear that information from the last run is leaked into this new run. No return value is expected, and any returned value is ignored.

fmt_varset *varname text*

Engine parameters. This command is called by the frontend to set an engine parameter to a particular value. The parameter to change is specified by *varname*, the value to set in *text*.

The command has to throw an error if an unknown *varname* is used. Only the names returned by **fmt_listvariables** have to be considered as known.

The values of all engine parameters have to persist between passes and runs.

Formatting commands

The formatting commands have to implement the formatting for the output format, for all the markup commands of the doctools markup language, except lb, rb, vset, include, and comment. These exceptions are processed by the frontend and are never seen by the plugin. In return a command for the formatting of plain text has to be provided, something which has no markup in the input at all.

This means, that each of the 49 markup commands specified in the *doctools language command reference* and outside of the set of exceptions listed above has an equivalent formatting command which takes the same arguments as the markup command and whose name is the name of markup command with the prefix *fmt_* added to it.

All commands are expected to format their input in some way per the semantics specified in the command reference and to return whatever part of this that they deem necessary as their result, which will be added to the output.

To avoid essentially duplicating the command reference we do not list any of the command here and simply refer the reader to the *doctools language command reference* for their signature and description. The sole exception is the plain text formatter, which has no equivalent markup command.

The calling sequence of formatting commands is not as rigid as for the management commands, but determined by the grammar of the doctools markup language, as specified in the *doctools language syntax* specification.

fmt_plain_text *text*

No associated markup command.

Called by the frontend for any plain text encountered in the input. It has to perform any and all special processing required for plain text.

The formatted text is expected as the result of the command, and added to the output. If no special processing is required it has to simply return its argument without change.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib

Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Documentation tools

Copyright


```
Copyright (c) 2007-2010 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


mpexpand

Keywords: mpexpand, HTML, TMML, conversion, manpage, markup, nroff

Name

mpexpand - Markup processor

Synopsis

mpexpand ?*-module module*? *format infile*|- *outfile*|-
mpexpand.all ?*-verbose*? ?*module*?

Description

This manpage describes a processor / converter for manpages in the doctools format as specified in **doctools_fmt**. The processor is based upon the package **doctools**.

mpexpand ?-module *module*? *format infile*|- *outfile*|-

The processor takes three arguments, namely the code describing which formatting to generate as the output, the file to read the markup from, and the file to write the generated output into. If the *infile* is "**-**" the processor will read from **stdin**. If *outfile* is "**-**" the processor will write to **stdout**.

If the option *-module* is present its value overrides the internal definition of the module name.

The currently known output formats are

nroff
: The processor generates *roff output, the standard format for unix manpages.

html
: The processor generates HTML output, for usage in and display by web browsers.

tmml
: The processor generates TMML output, the Tcl Manpage Markup Language, a derivative of XML.

latex
: The processor generates LaTeX output.

wiki
: The processor generates Wiki markup as understood by **wikit**.

list
The processor extracts the information provided by manpage_begin.

null
The processor does not generate any output.

mpexpand.all ?*-verbose*? ?*module*?
This command uses **mpexpand** to generate all possible output formats for all manpages in the current directory. The manpages are recognized through the extension "*.man*". If *-verbose* is specified the command will list its actions before executing them.

The *module* information is passed to **mpexpand**.

Notes
Possible future formats are plain text, pdf and postscript.

Bugs, ideas, feedback
If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Documentation tools

Copyright

```
Copyright (c) 2002 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
Copyright (c) 2003 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


doctools2base

html_cssdefaults
Keywords: html_cssdefaults, CSS, HTML, doctools, export, plugin, style

Name
doctools::html::cssdefaults - Default CSS style for HTML export plugins

Synopsis
package require Tcl 8.5 9
package require doctools::html::cssdefaults ?**0.2**?
::doctools::html::cssdefaults::contents

Description

This package provides a single command providing access to the text of the default CSS style to use for HTML markup generated by the various HTML export plugins.

This is an internal package of doctools, for use by *export* plugins, i.e. the packages converting doctools related documented into other formats, most notably *HTML*.

API

::doctools::html::cssdefaults::contents

This command returns the text of the default CSS style to use for HTML markup generated by the various HTML export plugins.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Documentation tools

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


nroff_manmacros

Keywords: nroff_manmacros, doctools, export, macros, man_macros, nroff, plugin

Name

doctools::nroff::man_macros - Default CSS style for NROFF export plugins

Synopsis

package require Tcl 8.5 9
package require doctools::nroff::man_macros ?**0.2**?
::doctools::nroff::man_macros::contents

Description

This package provides a single command providing access to the definition of the nroff *man* macro set to use for NROFF markup generated by the various NROFF export plugins.

This is an internal package of doctools, for use by *export* plugins, i.e. the packages converting doctools related documented into other formats, most notably *nroff*.

API

::doctools::nroff::man_macros::contents
This command returns the text of the default CSS style to use for NROFF generated by the various NROFF export plugins.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Documentation tools

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


tcl_parse

Keywords: tcl_parse, Tcl syntax, command, doctools, parser, subst, word

Name

doctools::tcl::parse - Processing text in 'subst -novariables' format

Synopsis

package require Tcl 8.5 9
package require snit
package require fileutil
package require logger
package require struct::list
package require struct::stack
package require struct::set
package require treeql
package require doctools::tcl::parse ?**0.2**?
::doctools::tcl::parse text *tree text* ?*root*?
::doctools::tcl::parse file *tree path* ?*root*?

Description

This package provides commands for parsing text with embedded Tcl commands as accepted by the Tcl builtin command **subst -novariables**. The result of the parsing is an abstract syntax tree.

This is an internal package of doctools, for use by the higher level parsers processing the *docidx*, *doctoc*, and *doctools* markup languages.

API

::doctools::tcl::parse text *tree text* ?*root*?

The command takes the *text* and parses it under the assumption that it contains a string acceptable to the Tcl builtin command **subst -novariables**. Errors are thrown otherwise during the parsing. The format used for these errors in described in section Error format.

The command returns the empty string as it result. The actual result of the parsing is entered into the tree structure *tree*, under the node *root*. If *root* is not specified the root of *tree* is used. The *tree* has to exist and be the command of a tree object which supports the same methods as trees created by the package **struct::tree**.

In case of errors *tree* will be left in an undefined state.

::doctools::tcl::parse file *tree path* ?*root*?

The same as **text**, except that the text to parse is read from the file specified by *path*.

Error format

When the parser encounters a problem in the input it will throw an error using the format described here.

1. The message will contain the reason for the problem (unexpected character or end of input in input), the character in question, if any, and the line and column the problem was found at, in a human readable form. This part is not documented further as its format may change as we see fit. It is intended for human consumption, not machine.
2. The error code however will contain a machine-readable representation of the problem, in the form of a 5-element list containing, in the order listed below
   1. the constant string **doctools::tcl::parse**
   2. the cause of the problem, one of

      char
      : Unexpected character in input

      eof
      : Unexpected end of the input

1. The location of the problem as offset from the beginning of the input, counted in characters. Note: Line markers count as one character.
2. The line the problem was found on (counted from 1 (one)),
3. The column the problem was found at (counted from 0 (zero))

Tree structure

After successfully parsing a string the generated tree will have the following structure:

1. In the following items the word 'root' refers to the node which was specified as the root of the tree when invoking either **text** or **file**. This may be the actual root of the tree.
2. All the following items further ignore the possibility of pre-existing attributes in the pre-existing nodes. If attributes exists with the same names as the attributes used by the parser the pre-existing values are written over. Attributes with names not clashing with the parser's attributes are not touched.
3. The root node has no attributes.

4. All other nodes have the attributes
   type
   : The value is a string from the set { Command, Text, Word }

   range
   : The value is either empty or a 2-element list containing integer numbers. The numbers are the offsets of the first and last character in the input text, of the token described by the node,.

   line
   : The value is an integer, it describes the line in the input the token described by the node ends on. Lines are counted from 1 (**one**).

   col
   : The value is an integer, it describes the column in the line in the input the token described by the node ends on. Columns are counted from 0 (**zero**).

1. The children of the root, if any, are of type Command and Text, in semi-alternation. This means: After a Text node a Command node has to follow, and anything can follow a Command node, a Text or other Command node.
2. The children of a Command node, if any, are of type Command, and Text, and Word, they describe the arguments of the command.
3. The children of a Word node, if any, are of type Command, Text, in semi-alternation. This means: After a Text node a Command node has to follow, and anything can follow a Command node, a Text or other Command node.
4. A Word node without children represents the empty string.
5. All Text nodes are leaves of the tree.
6. All leaves of the tree are either Text or Command nodes. Word nodes cannot be leaves.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Documentation tools

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

tcllib_msgcat

Keywords: tcllib_msgcat, catalog package, docidx, doctoc, doctools, i18n, internationalization, l10n, localization, message catalog, message package

Name

doctools::msgcat - Message catalog management for the various document parsers

Synopsis

package require Tcl 8.5 9
package require msgcat
package require doctools::msgcat ?**0.2**?
::doctools::msgcat::init *prefix*

Description

The package **doctools::msgcat** is a support module handling the selection of message catalogs for the various document processing packages in the doctools system version 2. As such it is an internal package a regular user (developer) should not be in direct contact with.

If you are such please go the documentation of either

1. **doctools::doc**,
2. **doctools::toc**, or
3. **doctools::idx**

Within the system architecture this package resides under the various parser packages, and is shared by them. Underneath it, but not explicit dependencies, are the packages providing the message catalogs for the various languages.

API

::doctools::msgcat::init ***prefix***

The command locates and loads the message catalogs for all the languages returned by **msgcat::mcpreferences**, provided that they could be found. It returns an integer number describing how many packages were found and loaded.

The names of the packages the command will look for have the form "doctools::msgcat::*prefix*::**langcode**", with *prefix* the argument to the command, and the **langcode** supplied by the result of **msgcat::mcpreferences**.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Documentation tools

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

doctools2idx

export_docidx

Keywords: export_docidx, docidx, doctools, export, index, serialization

Name

doctools::idx::export::docidx - docidx export plugin

Synopsis

package require Tcl 8.5 9
package require doctools::idx::export::docidx ?**0.2.2**?
export *serial configuration*

Description

This package implements the doctools keyword index export plugin for the generation of docidx markup.

This is an internal package of doctools, for use by the higher level management packages handling keyword indices, especially **doctools::idx::export**, the export manager.

Using it from a regular interpreter is possible, however only with contortions, and is not recommended. The proper way to use this functionality is through the package **doctools::idx::export** and the export manager objects it provides.

API

The API provided by this package satisfies the specification of the docidx export plugin API version 2.

export *serial configuration*

This command takes the canonical serialization of a keyword index, as specified in section Keyword index serialization format, and contained in *serial*, the *configuration*, a dictionary, and generates docidx markup encoding the index. The created string is then returned as the result of the command.

[docidx] notation of keyword indices

The docidx format for keyword indices, also called the *docidx markup language*, is too large to be covered in single section. The interested reader should start with the document

1. *docidx language introduction*

and then proceed from there to the formal specifications, i.e. the documents

1. *docidx language syntax* and
2. *docidx language command reference*.

to get a thorough understanding of the language.

Configuration

The docidx export plugin recognizes the following configuration variables and changes its behaviour as they specify.

string *user*
: This standard configuration variable contains the name of the user running the process which invoked the export plugin. The plugin puts this information into the provenance comment at the beginning of the generated document.

string *file*
: This standard configuration variable contains the name of the file the index came from. This variable may not be set or contain the empty string. The plugin puts this information, if defined, i.e. set and not the empty string, into the provenance comment at the beginning of the generated document.

boolean *newlines*
: If this flag is set the plugin will break the generated docidx code across lines, with each markup command on a separate line.

  If this flag is not set (the default), the whole document will be written on a single line, with minimum spacing between all elements.

boolean *indented*
: If this flag is set the plugin will indent the markup commands according to the structure of indices. To make this work this also implies that **newlines** is set. This effect is independent of the value for **aligned** however.

  If this flag is not set (the default), the output is formatted as per the values of **newlines** and **aligned**, and no indenting is done.

boolean *aligned*
: If this flag is set the generator ensures that the arguments for the **manpage** and **url** commands in a keyword section are aligned vertically for a nice table effect. To make this work this also implies that **newlines** is set. This effect is independent of the value for **indented** however.

  If this flag is not set (the default), the output is formatted as per the values of **newlines** and **indented**, and no alignment is done.

Note that this plugin ignores the standard configuration variables **format**, and **map**, and their values.

Keyword index serialization format

Here we specify the format used by the doctools v2 packages to serialize keyword indices as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a keyword index may have more than one regular serialization only exactly one of them will be *canonical*.

regular serialization

1. An index serialization is a nested Tcl dictionary.
1. This dictionary holds a single key, **doctools::idx**, and its value. This value holds the contents of the index.
2. The contents of the index are a Tcl dictionary holding the title of the index, a label, and the

keywords and references. The relevant keys and their values are

title
: The value is a string containing the title of the index.

label
: The value is a string containing a label for the index.

keywords
: The value is a Tcl dictionary, using the keywords known to the index as keys. The associated values are lists containing the identifiers of the references associated with that particular keyword.

  Any reference identifier used in these lists has to exist as a key in the **references** dictionary, see the next item for its definition.

references
: The value is a Tcl dictionary, using the identifiers for the references known to the index as keys. The associated values are 2-element lists containing the type and label of the reference, in this order.

  Any key here has to be associated with at least one keyword, i.e. occur in at least one of the reference lists which are the values in the **keywords** dictionary, see previous item for its definition.

1. The *type* of a reference can be one of two values,

   manpage
   : The identifier of the reference is interpreted as symbolic file name, referring to one of the documents the index was made for.

   url
   : The identifier of the reference is interpreted as an url, referring to some external location, like a website, etc.

canonical serialization
: The canonical serialization of a keyword index has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of the keyword index.

  1. The keys found in all the nested Tcl dictionaries are sorted in ascending dictionary order, as generated by Tcl's builtin command **lsort -increasing -dict**.
  2. The references listed for each keyword of the index, if any, are listed in ascending dictionary order of their *labels*, as generated by Tcl's builtin command **lsort -increasing -dict**.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be

made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Text formatter plugin

Copyright

```
Copyright (c) 2009-2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


idx_container
Keywords: idx_container, HTML, TMML, conversion, docidx markup, documentation, formatting, generation, index, json, keyword index, latex, manpage, markup, nroff, parsing, plugin, reference , tcler's wiki, text, url, wiki

Name
doctools::idx - Holding keyword indices

Synopsis
package require doctools::idx ?**2.1**?
package require Tcl 8.5 9
package require doctools::idx::structure
package require snit
::doctools::idx *objectName*
objectName method ?*arg arg ...*?
objectName **destroy**
objectName **key add** *name*
objectName **key remove** *name*
objectName **key references** *name*
objectName **keys**
objectName **reference add** *type key name label*
objectName **reference remove** *name*
objectName **reference label** *name*
objectName **reference keys** *name*
objectName **reference type** *name*
objectName **references**
objectName **title**
objectName **title** *text*
objectName **label**
objectName **label** *text*
objectName **importer**
objectName **importer** *object*
objectName **exporter**
objectName **exporter** *object*
objectName **deserialize =** *data* ?*format*?
objectName **deserialize +=** *data* ?*format*?

objectName **serialize** ?*format*?

Description

This package provides a class to contain and programmatically manipulate keyword indices

This is one of the three public pillars the management of keyword indices resides on. The other two pillars are

1. *Exporting keyword indices*, and
2. *Importing keyword indices*

For information about the Concepts of keyword indices, and their parts, see the same-named section. For information about the data structure which is used to encode keyword indices as values see the section Keyword index serialization format. This is the only format directly known to this class. Conversions from and to any other format are handled by export and import manager objects. These may be attached to a container, but do not have to be, it is merely a convenience.

Concepts

1. A *keyword index* consists of a (possibly empty) set of *keywords*.
2. Each keyword in the set is identified by its name.
3. Each keyword has a (possibly empty) set of *references*.
4. A reference can be associated with more than one keyword.
5. A reference not associated with at least one keyword is not possible however.
6. Each reference is identified by its target, specified as either an url or symbolic filename, depending on the type of reference (**url**, or **manpage**).
7. The type of a reference (url, or manpage) depends only on the reference itself, and not the keywords it is associated with.
8. In addition to a type each reference has a descriptive label as well. This label depends only on the reference itself, and not the keywords it is associated with.

A few notes

1. Manpage references are intended to be used for references to the documents the index is made for. Their target is a symbolic file name identifying the document, and export plugins may replace symbolic with actual file names, if specified.
2. Url references are intended on the othre hand are inteded to be used for links to anything else, like websites. Their target is an url.
3. While url and manpage references share a namespace for their identifiers, this should be no problem, given that manpage identifiers are symbolic filenames and as such they should never look like urls, the identifiers for url references.

API

Package commands

::doctools::idx *objectName*

This command creates a new container object with an associated Tcl command whose name is *objectName*. This *object* command is explained in full detail in the sections Object command and Object methods. The object command will be created under the current namespace if the *objectName* is not fully qualified, and in the specified namespace otherwise.

Object command

All objects created by the **::doctools::idx** command have the following general form:

objectName method ?*arg arg ...*?

The method **method** and its *arg*'uments determine the exact behavior of the command. See section Object methods for the detailed specifications.

Object methods

***objectName* destroy**

This method destroys the object it is invoked for.

objectName* key add *name

This method adds the keyword *name* to the index. If the keyword is already known nothing is done. The result of the method is the empty string.

objectName* key remove *name

This method removes the keyword *name* from the index. If the keyword is already gone nothing is done. Any references for whom this keyword was the last association are removed as well. The result of the method is the empty string.

objectName* key references *name

This method returns a list containing the names of all references associated with the keyword *name*. An error is thrown in the keyword is not known to the index. The order of the references in the list is undefined.

***objectName* keys**

This method returns a list containing the names of all keywords known to the index. The order of the keywords in the list is undefined.

objectName* reference add *type key name label

This method adds the reference *name* to the index and associates it with the keyword *key*. The other two arguments hold the *type* and *label* of the reference, respectively. The type has to match the stored information, should the reference exist already, i.e. this information is immutable after the reference is known. The only way to change it is delete and recreate the reference. The label on the other hand is automatically updated to the value of the argument, overwriting any previously stored information. Should the reference exists already it is simply associated with the *key*. If that is true already as well nothing is done, but the *label* updated to the new value. The result of the method is the empty string.

The *type* argument has be to one of **manpage** or **url**.

objectName* reference remove *name

The reference *name* is removed from the index. All associations with keywords are released and the relevant reference labels removed. The result of the method is the empty string.

objectName* reference label *name

This method returns the label associated with the reference *name*. An error is thrown if the reference is not known.

objectName* reference keys *name

This method returns a list containing the names of all keywords associated with the reference *name*. An error is thrown in the reference is not known to the index. The order of the keywords in the list is undefined.

objectName* reference type *name

This method returns the type of the reference *name*. An error is thrown in the reference is not known to the index.

***objectName* references**
This method returns a list containing the names of all references known to the index. The order of the references in the list is undefined.

***objectName* title**
Returns the currently defined title of the keyword index.

objectName* title *text
Sets the title of the keyword index to *text*, and returns it as the result of the command.

***objectName* label**
Returns the currently defined label of the keyword index.

objectName* label *text
Sets the label of the keyword index to *text*, and returns it as the result of the command.

***objectName* importer**
Returns the import manager object currently attached to the container, if any.

objectName* importer *object
Attaches the *object* as import manager to the container, and returns it as the result of the command. Note that the *object* is *not* put into ownership of the container. I.e., destruction of the container will *not* destroy the *object*.

It is expected that *object* provides a method named **import text** which takes a text and a format name, and returns the canonical serialization of the keyword index contained in the text, assuming the given format.

***objectName* exporter**
Returns the export manager object currently attached to the container, if any.

objectName* exporter *object
Attaches the *object* as export manager to the container, and returns it as the result of the command. Note that the *object* is *not* put into ownership of the container. I.e., destruction of the container will *not* destroy the *object*.

It is expected that *object* provides a method named **export object** which takes the container and a format name, and returns a text encoding keyword index stored in the container, in the given format. It is further expected that the *object* will use the container's method **serialize** to obtain the serialization of the keyword index from which to generate the text.

***objectName* deserialize = *data* ?*format*?**
This method replaces the contents of the index object with the index contained in the *data*. If no *format* was specified it is assumed to be the regular serialization of a keyword index.

Otherwise the object will use the attached import manager to convert the data from the specified format to a serialization it can handle. In that case an error will be thrown if the container has no import manager attached to it.

The result of the method is the empty string.

***objectName* deserialize += *data* ?*format*?**
This method behaves like **deserialize =** in its essentials, except that it merges the keyword

index in the *data* to its contents instead of replacing it. The method will throw an error if merging is not possible, i.e. would produce an invalid index. The existing content is left unchanged in that case.

The result of the method is the empty string.

***objectName* serialize ?*format*?**

This method returns the keyword index contained in the object. If no *format* is not specified the returned result is the canonical serialization of its contents.

Otherwise the object will use the attached export manager to convert the data to the specified format. In that case an error will be thrown if the container has no export manager attached to it.

Keyword index serialization format

Here we specify the format used by the doctools v2 packages to serialize keyword indices as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a keyword index may have more than one regular serialization only exactly one of them will be *canonical*.

regular serialization

1. An index serialization is a nested Tcl dictionary.
1. This dictionary holds a single key, **doctools::idx**, and its value. This value holds the contents of the index.
2. The contents of the index are a Tcl dictionary holding the title of the index, a label, and the keywords and references. The relevant keys and their values are

   title
   The value is a string containing the title of the index.

   label
   The value is a string containing a label for the index.

   keywords
   The value is a Tcl dictionary, using the keywords known to the index as keys. The associated values are lists containing the identifiers of the references associated with that particular keyword.

   Any reference identifier used in these lists has to exist as a key in the **references** dictionary, see the next item for its definition.

   references
   The value is a Tcl dictionary, using the identifiers for the references known to the index as keys. The associated values are 2-element lists containing the type and label of the reference, in this order.

   Any key here has to be associated with at least one keyword, i.e. occur in at least one of the reference lists which are the values in the **keywords** dictionary, see previous item for its definition.

1. The *type* of a reference can be one of two values,

manpage
The identifier of the reference is interpreted as symbolic file name, referring to one of the documents the index was made for.

url
The identifier of the reference is interpreted as an url, referring to some external location, like a website, etc.

canonical serialization
The canonical serialization of a keyword index has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of the keyword index.

1. The keys found in all the nested Tcl dictionaries are sorted in ascending dictionary order, as generated by Tcl's builtin command **lsort -increasing -dict**.
2. The references listed for each keyword of the index, if any, are listed in ascending dictionary order of their *labels*, as generated by Tcl's builtin command **lsort -increasing -dict**.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Documentation tools

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


idx_export

Keywords: idx_export, HTML, conversion, docidx, documentation, export, formatting, generation, index, json, keyword index, manpage, markup, nroff, plugin, reference, tcler's wiki, text, url, wiki

Name

doctools::idx::export - Exporting keyword indices

Synopsis

package require doctools::idx::export ?**0.2.2**?
package require Tcl 8.5 9
package require struct::map
package require doctools::idx::structure

package require snit
package require pluginmgr
::doctools::idx::export *objectName*
objectName method ?*arg arg ...*?
objectName **destroy**
objectName **export serial** *serial* ?*format*?
objectName **export object** *object* ?*format*?
objectName **config names**
objectName **config get**
objectName **config set** *name* ?*value*?
objectName **config unset** *pattern...*
export *serial configuration*

Description

This package provides a class to manage the plugins for the export of keyword indices to other formats, i.e. their conversion to, for example *docidx*, *HTML*, etc.

This is one of the three public pillars the management of keyword indices resides on. The other two pillars are

1. *Importing keyword indices*, and
2. *Holding keyword indices*

For information about the Concepts of keyword indices, and their parts, see the same-named section. For information about the data structure which is the major input to the manager objects provided by this package see the section Keyword index serialization format.

The plugin system of our class is based on the package **pluginmgr**, and configured to look for plugins using

1. the environment variable **DOCTOOLS_IDX_EXPORT_PLUGINS**,
2. the environment variable **DOCTOOLS_IDX_PLUGINS**,
3. the environment variable **DOCTOOLS_PLUGINS**,
4. the path "*~/.doctools/idx/export/plugin*"
5. the path "*~/.doctools/idx/plugin*"
6. the path "*~/.doctools/plugin*"
7. the path "*~/.doctools/idx/export/plugins*"
8. the path "*~/.doctools/idx/plugins*"
9. the path "*~/.doctools/plugins*"
10. the registry entry "HKEY_CURRENT_USER\SOFTWARE\DOCTOOLS\IDX\EXPORT\PLUGINS"
11. the registry entry "HKEY_CURRENT_USER\SOFTWARE\DOCTOOLS\IDX\PLUGINS"
12. the registry entry "HKEY_CURRENT_USER\SOFTWARE\DOCTOOLS\PLUGINS"

The last three are used only when the package is run on a machine using Windows(tm) operating system.

The whole system is delivered with six predefined export plugins, namely

docidx
: See *docidx export plugin* for details.

html
: See *html export plugin* for details.

json
: See *json export plugin* for details.

nroff
: See *nroff export plugin* for details.

text
: See *text export plugin* for details.

wiki
: See *wiki export plugin* for details.

Readers wishing to write their own export plugin for some format, i.e. *plugin writer*s reading and understanding the section containing the Export plugin API v2 reference is an absolute necessity, as it specifies the interaction between this package and its plugins in detail.

Concepts

1. A *keyword index* consists of a (possibly empty) set of *keywords*.
2. Each keyword in the set is identified by its name.
3. Each keyword has a (possibly empty) set of *references*.
4. A reference can be associated with more than one keyword.
5. A reference not associated with at least one keyword is not possible however.
6. Each reference is identified by its target, specified as either an url or symbolic filename, depending on the type of reference (**url**, or **manpage**).
7. The type of a reference (url, or manpage) depends only on the reference itself, and not the keywords it is associated with.
8. In addition to a type each reference has a descriptive label as well. This label depends only on the reference itself, and not the keywords it is associated with.

A few notes

1. Manpage references are intended to be used for references to the documents the index is made for. Their target is a symbolic file name identifying the document, and export plugins may replace symbolic with actual file names, if specified.
2. Url references are intended on the othre hand are inteded to be used for links to anything else, like websites. Their target is an url.
3. While url and manpage references share a namespace for their identifiers, this should be no problem, given that manpage identifiers are symbolic filenames and as such they should never look like urls, the identifiers for url references.

API

Package commands

::doctools::idx::export *objectName*
: This command creates a new export manager object with an associated Tcl command whose name is *objectName*. This *object* command is explained in full detail in the sections Object command and Object methods. The object command will be created under the current namespace if the *objectName* is not fully qualified, and in the specified namespace otherwise.

Object command

All objects created by the **::doctools::idx::export** command have the following general form:

objectName method ?*arg arg ...*?

The method **method** and its *arg*'uments determine the exact behavior of the command. See section Object methods for the detailed specifications.

Object methods

***objectName* destroy**

This method destroys the object it is invoked for.

***objectName* export serial *serial* ?*format*?**

This method takes the canonical serialization of a keyword index stored in *serial* and converts it to the specified *format*, using the export plugin for the format. An error is thrown if no plugin could be found for the format. The string generated by the conversion process is returned as the result of this method.

If no format is specified the method defaults to **docidx**.

The specification of what a *canonical* serialization is can be found in the section Keyword index serialization format.

The plugin has to conform to the interface specified in section Export plugin API v2 reference.

***objectName* export object *object* ?*format*?**

This method is a convenient wrapper around the **export serial** method described by the previous item. It expects that *object* is an object command supporting a **serialize** method returning the canonical serialization of a keyword index. It invokes that method, feeds the result into **export serial** and returns the resulting string as its own result.

***objectName* config names**

This method returns a list containing the names of all configuration variables currently known to the object.

***objectName* config get**

This method returns a dictionary containing the names and values of all configuration variables currently known to the object.

***objectName* config set *name* ?*value*?**

This method sets the configuration variable *name* to the specified *value* and returns the new value of the variable.

If no value is specified it simply returns the current value, without changing it.

Note that while the user can set the predefined configuration variables **user** and **format** doing so will have no effect, these values will be internally overridden when invoking an import plugin.

***objectName* config unset *pattern*...**

This method unsets all configuration variables matching the specified glob *pattern*s. If no pattern is specified it will unset all currently defined configuration variables.

Export plugin API v2 reference

Plugins are what this package uses to manage the support for any output format beyond the Keyword index serialization format. Here we specify the API the objects created by this package use to interact with their plugins.

A plugin for this package has to follow the rules listed below:

1. A plugin is a package.
2. The name of a plugin package has the form doctools::idx::export::**FOO**, where **FOO** is the name of the format the plugin will generate output for. This name is also the argument to provide to the various **export** methods of export manager objects to get a string encoding a keyword index in that format.
3. The plugin can expect that the package **doctools::idx::export::plugin** is present, as indicator that it was invoked from a genuine plugin manager.
4. A plugin has to provide one command, with the signature shown below.

   export *serial configuration*

   Whenever an export manager of **doctools::idx** has to generate output for an index it will invoke this command.

   string *serial*

   This argument will contain the *canonical* serialization of the index for which to generate the output. The specification of what a *canonical* serialization is can be found in the section Keyword index serialization format.

   dictionary *configuration*

   This argument will contain the current configuration to apply to the generation, as a dictionary mapping from variable names to values.

   The following configuration variables have a predefined meaning all plugins have to obey, although they can ignore this information at their discretion. Any other other configuration variables recognized by a plugin will be described in the manpage for that plugin.

   user

   This variable is expected to contain the name of the user owning the process invoking the plugin.

   format

   This variable is expected to contain the name of the format whose plugin is invoked.

   file

   This variable, if defined by the user of the index object is expected to contain the name of the input file for which the plugin is generating its output for.

   map

   This variable, if defined by the user of the index object is expected to contain a dictionary mapping from symbolic file names used in the references of type **manpage** to actual paths (or urls). A plugin has to be able to handle the possibility that a symbolic name is without entry in this mapping.

1. A single usage cycle of a plugin consists of the invokations of the command **export**. This call has to leave the plugin in a state where another usage cycle can be run without problems.

Keyword index serialization format

Here we specify the format used by the doctools v2 packages to serialize keyword indices as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a keyword index may have

more than one regular serialization only exactly one of them will be *canonical*.

regular serialization

1. An index serialization is a nested Tcl dictionary.
1. This dictionary holds a single key, **doctools::idx**, and its value. This value holds the contents of the index.
2. The contents of the index are a Tcl dictionary holding the title of the index, a label, and the keywords and references. The relevant keys and their values are

   title
   : The value is a string containing the title of the index.

   label
   : The value is a string containing a label for the index.

   keywords
   : The value is a Tcl dictionary, using the keywords known to the index as keys. The associated values are lists containing the identifiers of the references associated with that particular keyword.

     Any reference identifier used in these lists has to exist as a key in the **references** dictionary, see the next item for its definition.

   references
   : The value is a Tcl dictionary, using the identifiers for the references known to the index as keys. The associated values are 2-element lists containing the type and label of the reference, in this order.

     Any key here has to be associated with at least one keyword, i.e. occur in at least one of the reference lists which are the values in the **keywords** dictionary, see previous item for its definition.

1. The *type* of a reference can be one of two values,

   manpage
   : The identifier of the reference is interpreted as symbolic file name, referring to one of the documents the index was made for.

   url
   : The identifier of the reference is interpreted as an url, referring to some external location, like a website, etc.

canonical serialization

The canonical serialization of a keyword index has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of the keyword index.

1. The keys found in all the nested Tcl dictionaries are sorted in ascending dictionary order, as generated by Tcl's builtin command **lsort -increasing -dict**.
2. The references listed for each keyword of the index, if any, are listed in ascending dictionary order of their *labels*, as generated by Tcl's builtin command **lsort -increasing -dict**.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Documentation tools

Copyright


```
Copyright (c) 2009-2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


idx_export_html

Keywords: idx_export_html, HTML, doctools, export, index, serialization

Name

doctools::idx::export::html - HTML export plugin

Synopsis

package require Tcl 8.5 9
package require doctools::idx::export::html ?**0.3**?
package require doctools::text
package require doctools::html
package require doctools::html::cssdefaults
export *serial configuration*

Description

This package implements the doctools keyword index export plugin for the generation of HTML markup.

This is an internal package of doctools, for use by the higher level management packages handling keyword indices, especially **doctools::idx::export**, the export manager.

Using it from a regular interpreter is possible, however only with contortions, and is not recommended. The proper way to use this functionality is through the package **doctools::idx::export** and the export manager objects it provides.

API

The API provided by this package satisfies the specification of the docidx export plugin API version 2.

export *serial configuration*

This command takes the canonical serialization of a keyword index, as specified in section Keyword index serialization format, and contained in *serial*, the *configuration*, a dictionary,

and generates HTML markup encoding the index. The created string is then returned as the result of the command.

Configuration

The html export plugin recognizes the following configuration variables and changes its behaviour as they specify.

string *user*
: This standard configuration variable contains the name of the user running the process which invoked the export plugin. The plugin puts this information into the provenance comment at the beginning of the generated document.

string *file*
: This standard configuration variable contains the name of the file the index came from. This variable may not be set or contain the empty string. The plugin puts this information, if defined, i.e. set and not the empty string, into the provenance comment at the beginning of the generated document.

dictionary *map*
: This standard configuration variable contains a dictionary mapping from the symbolic files names in manpage references to the actual filenames and/or urls to be used in the output.

  Url references and symbolic file names without a mapping are used unchanged.

boolean *newlines*
: If this flag is set the plugin will break the generated html code across lines, with each markup command on a separate line.

  If this flag is not set (the default), the whole document will be written on a single line, with minimum spacing between all elements.

boolean *indented*
: If this flag is set the plugin will indent the markup commands according to the structure of indices. To make this work this also implies that **newlines** is set.

  If this flag is not set (the default), the output is formatted as per the value of **newlines**, and no indenting is done.

string *meta*
: This variable is meant to hold a fragment of HTML (default: empty). The fragment it contains will be inserted into the generated output in the section of the document, just after the tag.

string *header*
: This variable is meant to hold a fragment of HTML (default: empty). The fragment it contains will be inserted into the generated output just after the title tag in the body of the document, in the class.header 'ision.

string *footer*
: This variable is meant to hold a fragment of HTML (default: empty). The fragment it contains will be inserted into the generated output just before the tag, in the class.footer 'ision.

dictionary *kwid*
: The value of this variable (default: empty) maps keywords to the identifiers to use as their anchor names. Each keyword **FOO** not found in the dictionary uses **KW-FOO** as anchor, i.e.

itself prefixed with the string **KW-**.

string *sepline*
The value of this variable is the string to use for the separator comments inserted into the output when the outpout is broken across lines and/or indented. The default string consists of 60 dashes.

integer *kwidth*
This variable holds the size of the keyword column in the main table generated by the plugin, in percent of the total width of the table. This is an integer number in the range of 1 to 99. Choosing a value outside of that range causes the generator to switch back to the defauly setting, 35 percent.

string *dot*
This variable contains a HTML fragment inserted between the entries of the navigation bar, and the references associated with each keyword. The default is the HTML entity · i.e. the bullet character, also known as the "Greek middle dot", i.e. the unicode character 00B7.

string *class.main*
This variable contains the class name for the main 'ivision of the generated document. The default is **doctools**.

string *class.header*
This variable contains the class name for the header 'ision of the generated document. The default is **idx-header**. This division contains the document title, the user specified **header**, if any, a visible separator line, and the navigation bar for quick access to each keyword section.

string *class.title*
This variable contains the class name for the tag enclosing the document title. The default is **idx-title**.

string *class.navsep*
This variable contains the class name for the separators in the header and footer sections of the generated document. The default is **idx-navsep**.

string *class.navbar*
This variable contains the class name for the navigation 'ision enclosing the navigation bar of the generated document. The default is **idx-kwnav**.

string *class.contents*
This variable contains the class name for the holding the keywords and their references in the generated document. The default is **idx-contents**.

string *class.leader*
This variable contains the class name for the anchor names the plugin inserts into the keyword table when switching from one section to the next (Each section holds all keywords with a particular first character). The default is **idx-leader**.

string *class.row0*
This variable contains the class name used to label the even rows () of the keyword table. The default is **idx-even**.

string *class.row1*
This variable contains the class name used to label the odd rows () of the keyword table. The default is **idx-odd**.

string *class.keyword*

This variable contains the class name used to label the keyword cells/column () in the keyword table of the document. The default is **idx-keyword**.

string *class.refs*
This variable contains the class name used to label the reference cells/column () in the keyword table of the document. The default is **idx-refs**.

string *class.footer*
This variable contains the class name for the footer 'ision of the generated document. The default is **idx-footer**. This division contains a browser-visible separator line and the user specified **footer**, if any.

Note that this plugin ignores the standard configuration variable **format**, and its value.

Keyword index serialization format

Here we specify the format used by the doctools v2 packages to serialize keyword indices as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a keyword index may have more than one regular serialization only exactly one of them will be *canonical*.

regular serialization

1. An index serialization is a nested Tcl dictionary.
1. This dictionary holds a single key, **doctools::idx**, and its value. This value holds the contents of the index.
2. The contents of the index are a Tcl dictionary holding the title of the index, a label, and the keywords and references. The relevant keys and their values are

   title
   The value is a string containing the title of the index.

   label
   The value is a string containing a label for the index.

   keywords
   The value is a Tcl dictionary, using the keywords known to the index as keys. The associated values are lists containing the identifiers of the references associated with that particular keyword.

   Any reference identifier used in these lists has to exist as a key in the **references** dictionary, see the next item for its definition.

   references
   The value is a Tcl dictionary, using the identifiers for the references known to the index as keys. The associated values are 2-element lists containing the type and label of the reference, in this order.

   Any key here has to be associated with at least one keyword, i.e. occur in at least one of the reference lists which are the values in the **keywords** dictionary, see previous item for its definition.

1. The *type* of a reference can be one of two values,

   manpage
   The identifier of the reference is interpreted as symbolic file name, referring to one of

the documents the index was made for.

url
The identifier of the reference is interpreted as an url, referring to some external location, like a website, etc.

canonical serialization
The canonical serialization of a keyword index has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of the keyword index.

1. The keys found in all the nested Tcl dictionaries are sorted in ascending dictionary order, as generated by Tcl's builtin command **lsort -increasing -dict**.
2. The references listed for each keyword of the index, if any, are listed in ascending dictionary order of their *labels*, as generated by Tcl's builtin command **lsort -increasing -dict**.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Text formatter plugin

Copyright


```
Copyright (c) 2009-2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


idx_export_json

Keywords: idx_export_json, JSON, doctools, export, index, serialization

Name

doctools::idx::export::json - JSON export plugin

Synopsis

package require Tcl 8.5 9
package require doctools::idx::export::json ?**0.2**?
package require textutil::adjust
export *serial configuration*

Description

This package implements the doctools keyword index export plugin for the generation of JSON

markup.

This is an internal package of doctools, for use by the higher level management packages handling keyword indices, especially **doctools::idx::export**, the export manager.

Using it from a regular interpreter is possible, however only with contortions, and is not recommended. The proper way to use this functionality is through the package **doctools::idx::export** and the export manager objects it provides.

API

The API provided by this package satisfies the specification of the docidx export plugin API version 2.

export *serial configuration*

This command takes the canonical serialization of a keyword index, as specified in section Keyword index serialization format, and contained in *serial*, the *configuration*, a dictionary, and generates JSON markup encoding the index. The created string is then returned as the result of the command.

JSON notation of keyword indices

The JSON format used for keyword indices is a direct translation of the Keyword index serialization format, mapping Tcl dictionaries as JSON objects and Tcl lists as JSON arrays. For example, the Tcl serialization

```
doctools::idx {
    label {Keyword Index}
    keywords {
        changelog  {changelog.man cvs.man}
        conversion {doctools.man docidx.man doctoc.man apps/dtplite.man
    mpexpand.man}
        cvs        cvs.man
    }
    references {
        apps/dtplite.man {manpage dtplite}
        changelog.man    {manpage doctools::changelog}
        cvs.man          {manpage doctools::cvs}
        docidx.man       {manpage doctools::idx}
        doctoc.man       {manpage doctools::toc}
        doctools.man     {manpage doctools}
        mpexpand.man     {manpage mpexpand}
    }
    title {}
}
```

is equivalent to the JSON string

```
{
    "doctools::idx" : {
        "label"      : "Keyword Index",
        "keywords"   : {
```

```
            "changelog"  : ["changelog.man","cvs.man"],
            "conversion" :
    ["doctools.man","docidx.man","doctoc.man","apps\/dtplite.man","mpex
    pand.man"],
            "cvs"        : ["cvs.man"],
        },
        "references" : {
            "apps\/dtplite.man" : ["manpage","dtplite"],
            "changelog.man"     : ["manpage","doctools::changelog"],
            "cvs.man"           : ["manpage","doctools::cvs"],
            "docidx.man"        : ["manpage","doctools::idx"],
            "doctoc.man"        : ["manpage","doctools::toc"],
            "doctools.man"      : ["manpage","doctools"],
            "mpexpand.man"      : ["manpage","mpexpand"]
        },
        "title"      : ""
    }
}
```

Configuration

The JSON export plugin recognizes the following configuration variables and changes its behaviour as they specify.

boolean *indented*

If this flag is set the plugin will break the generated JSON code across lines and indent it according to its inner structure, with each key of a dictionary on a separate line.

If this flag is not set (the default), the whole JSON object will be written on a single line, with minimum spacing between all elements.

boolean *aligned*

If this flag is set the generator ensures that the values for the keys in a dictionary are vertically aligned with each other, for a nice table effect. To make this work this also implies that **indented** is set.

If this flag is not set (the default), the output is formatted as per the value of **indented**, without trying to align the values for dictionary keys.

Note that this plugin ignores the standard configuration variables **user**, **format**, **file**, and **map** and their values.

Keyword index serialization format

Here we specify the format used by the doctools v2 packages to serialize keyword indices as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a keyword index may have more than one regular serialization only exactly one of them will be *canonical*.

regular serialization

1. An index serialization is a nested Tcl dictionary.

1. This dictionary holds a single key, **doctools::idx**, and its value. This value holds the contents of the index.
2. The contents of the index are a Tcl dictionary holding the title of the index, a label, and the keywords and references. The relevant keys and their values are

   title
   The value is a string containing the title of the index.

   label
   The value is a string containing a label for the index.

   keywords
   The value is a Tcl dictionary, using the keywords known to the index as keys. The associated values are lists containing the identifiers of the references associated with that particular keyword.

   Any reference identifier used in these lists has to exist as a key in the **references** dictionary, see the next item for its definition.

   references
   The value is a Tcl dictionary, using the identifiers for the references known to the index as keys. The associated values are 2-element lists containing the type and label of the reference, in this order.

   Any key here has to be associated with at least one keyword, i.e. occur in at least one of the reference lists which are the values in the **keywords** dictionary, see previous item for its definition.

1. The *type* of a reference can be one of two values,

   manpage
   The identifier of the reference is interpreted as symbolic file name, referring to one of the documents the index was made for.

   url
   The identifier of the reference is interpreted as an url, referring to some external location, like a website, etc.

canonical serialization
The canonical serialization of a keyword index has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of the keyword index.

1. The keys found in all the nested Tcl dictionaries are sorted in ascending dictionary order, as generated by Tcl's builtin command **lsort -increasing -dict**.
2. The references listed for each keyword of the index, if any, are listed in ascending dictionary order of their *labels*, as generated by Tcl's builtin command **lsort -increasing -dict**.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Text formatter plugin

Copyright

```
Copyright (c) 2009-2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


idx_export_nroff
Keywords: idx_export_nroff, doctools, export, index, nroff, serialization

Name
doctools::idx::export::nroff - nroff export plugin

Synopsis
package require Tcl 8.5 9
package require doctools::idx::export::nroff ?**0.4**?
package require doctools::text
package require doctools::nroff::man_macros
export *serial configuration*

Description
This package implements the doctools keyword index export plugin for the generation of nroff markup.

This is an internal package of doctools, for use by the higher level management packages handling keyword indices, especially **doctools::idx::export**, the export manager.

Using it from a regular interpreter is possible, however only with contortions, and is not recommended. The proper way to use this functionality is through the package **doctools::idx::export** and the export manager objects it provides.

API
The API provided by this package satisfies the specification of the docidx export plugin API version 2.

export *serial configuration*
This command takes the canonical serialization of a keyword index, as specified in section Keyword index serialization format, and contained in *serial*, the *configuration*, a dictionary, and generates nroff markup encoding the index. The created string is then returned as the result of the command.

Configuration
The nroff export plugin recognizes the following configuration variables and changes its

behaviour as they specify.

string *user*
: This standard configuration variable contains the name of the user running the process which invoked the export plugin. The plugin puts this information into the provenance comment at the beginning of the generated document.

string *file*
: This standard configuration variable contains the name of the file the index came from. This variable may not be set or contain the empty string. The plugin puts this information, if defined, i.e. set and not the empty string, into the provenance comment at the beginning of the generated document.

boolean *inline*
: If this flag is set (default) the plugin will place the definitions of the man macro set directly into the output.

  If this flag is not set, the plugin will place a reference to the definitions of the man macro set into the output, but not the macro definitions themselves.

Note that this plugin ignores the standard configuration variables **format**, and **map**, and their values.

Keyword index serialization format

Here we specify the format used by the doctools v2 packages to serialize keyword indices as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a keyword index may have more than one regular serialization only exactly one of them will be *canonical*.

regular serialization

1. An index serialization is a nested Tcl dictionary.
1. This dictionary holds a single key, **doctools::idx**, and its value. This value holds the contents of the index.
2. The contents of the index are a Tcl dictionary holding the title of the index, a label, and the keywords and references. The relevant keys and their values are

   title
   : The value is a string containing the title of the index.

   label
   : The value is a string containing a label for the index.

   keywords
   : The value is a Tcl dictionary, using the keywords known to the index as keys. The associated values are lists containing the identifiers of the references associated with that particular keyword.

     Any reference identifier used in these lists has to exist as a key in the **references** dictionary, see the next item for its definition.

   references
   : The value is a Tcl dictionary, using the identifiers for the references known to the index as keys. The associated values are 2-element lists containing the type and label of the

reference, in this order.

Any key here has to be associated with at least one keyword, i.e. occur in at least one of the reference lists which are the values in the **keywords** dictionary, see previous item for its definition.

1. The *type* of a reference can be one of two values,
   manpage
   The identifier of the reference is interpreted as symbolic file name, referring to one of the documents the index was made for.
   url
   The identifier of the reference is interpreted as an url, referring to some external location, like a website, etc.

canonical serialization
The canonical serialization of a keyword index has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of the keyword index.

1. The keys found in all the nested Tcl dictionaries are sorted in ascending dictionary order, as generated by Tcl's builtin command **lsort -increasing -dict**.
2. The references listed for each keyword of the index, if any, are listed in ascending dictionary order of their *labels*, as generated by Tcl's builtin command **lsort -increasing -dict**.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Text formatter plugin

Copyright


```
Copyright (c) 2009-2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


idx_export_text

Keywords: idx_export_text, doctools, export, index, plain text, serialization

Name

doctools::idx::export::text - plain text export plugin

Synopsis

package require Tcl 8.5 9
package require doctools::idx::export::text ?**0.3**?
package require doctools::text
export *serial configuration*

Description

This package implements the doctools keyword index export plugin for the generation of plain text markup.

This is an internal package of doctools, for use by the higher level management packages handling keyword indices, especially **doctools::idx::export**, the export manager.

Using it from a regular interpreter is possible, however only with contortions, and is not recommended. The proper way to use this functionality is through the package **doctools::idx::export** and the export manager objects it provides.

API

The API provided by this package satisfies the specification of the docidx export plugin API version 2.

export *serial configuration*
: This command takes the canonical serialization of a keyword index, as specified in section Keyword index serialization format, and contained in *serial*, the *configuration*, a dictionary, and generates plain text markup encoding the index. The created string is then returned as the result of the command.

Configuration

The text export plugin recognizes the following configuration variables and changes its behaviour as they specify.

dictionary *map*
: This standard configuration variable contains a dictionary mapping from the symbolic files names in manpage references to the actual filenames and/or urls to be used in the output.

  Url references and symbolic file names without a mapping are used unchanged.

Note that this plugin ignores the standard configuration variables **user**, **file**, and **format**, and their values.

Keyword index serialization format

Here we specify the format used by the doctools v2 packages to serialize keyword indices as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a keyword index may have more than one regular serialization only exactly one of them will be *canonical*.

regular serialization

1. An index serialization is a nested Tcl dictionary.
1. This dictionary holds a single key, **doctools::idx**, and its value. This value holds the contents of the index.

2. The contents of the index are a Tcl dictionary holding the title of the index, a label, and the keywords and references. The relevant keys and their values are

   title
   The value is a string containing the title of the index.

   label
   The value is a string containing a label for the index.

   keywords
   The value is a Tcl dictionary, using the keywords known to the index as keys. The associated values are lists containing the identifiers of the references associated with that particular keyword.

   Any reference identifier used in these lists has to exist as a key in the **references** dictionary, see the next item for its definition.

   references
   The value is a Tcl dictionary, using the identifiers for the references known to the index as keys. The associated values are 2-element lists containing the type and label of the reference, in this order.

   Any key here has to be associated with at least one keyword, i.e. occur in at least one of the reference lists which are the values in the **keywords** dictionary, see previous item for its definition.

1. The *type* of a reference can be one of two values,

   manpage
   The identifier of the reference is interpreted as symbolic file name, referring to one of the documents the index was made for.

   url
   The identifier of the reference is interpreted as an url, referring to some external location, like a website, etc.

canonical serialization
The canonical serialization of a keyword index has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of the keyword index.

1. The keys found in all the nested Tcl dictionaries are sorted in ascending dictionary order, as generated by Tcl's builtin command **lsort -increasing -dict**.
2. The references listed for each keyword of the index, if any, are listed in ascending dictionary order of their *labels*, as generated by Tcl's builtin command **lsort -increasing -dict**.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Text formatter plugin

Copyright

```
Copyright (c) 2009-2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


idx_export_wiki
Keywords: idx_export_wiki, doctools, export, index, serialization, wiki

Name
doctools::idx::export::wiki - wiki export plugin

Synopsis
package require Tcl 8.5 9
package require doctools::idx::export::wiki ?**0.3**?
package require doctools::text
export *serial configuration*

Description
This package implements the doctools keyword index export plugin for the generation of wiki markup.

This is an internal package of doctools, for use by the higher level management packages handling keyword indices, especially **doctools::idx::export**, the export manager.

Using it from a regular interpreter is possible, however only with contortions, and is not recommended. The proper way to use this functionality is through the package **doctools::idx::export** and the export manager objects it provides.

API
The API provided by this package satisfies the specification of the docidx export plugin API version 2.

export *serial configuration*
This command takes the canonical serialization of a keyword index, as specified in section Keyword index serialization format, and contained in *serial*, the *configuration*, a dictionary, and generates wiki markup encoding the index. The created string is then returned as the result of the command.

Wiki markup
The basic syntax of the wiki markup generated by this plugin are described at http://wiki.tcl.tk/14.

The plugin goes beyond the classic markup to generate proper headers and either a table or

indented list of the keywords and their references.

Configuration

The wiki export plugin recognizes the following configuration variables and changes its behaviour as they specify.

dictionary *map*
: This standard configuration variable contains a dictionary mapping from the symbolic files names in manpage references to the actual filenames and/or urls to be used in the output.

  Url references and symbolic file names without a mapping are used unchanged.

enum *style*
: This variable recognizes two values as legal, **list** (default), and **table**. Depending on the value the plugin generates either a list- or table-based wiki page for the index.

Note that this plugin ignores the standard configuration variables **user**, **file** and **format**, and their values.

Keyword index serialization format

Here we specify the format used by the doctools v2 packages to serialize keyword indices as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a keyword index may have more than one regular serialization only exactly one of them will be *canonical*.

regular serialization

1. An index serialization is a nested Tcl dictionary.
1. This dictionary holds a single key, **doctools::idx**, and its value. This value holds the contents of the index.
2. The contents of the index are a Tcl dictionary holding the title of the index, a label, and the keywords and references. The relevant keys and their values are

   title
   : The value is a string containing the title of the index.

   label
   : The value is a string containing a label for the index.

   keywords
   : The value is a Tcl dictionary, using the keywords known to the index as keys. The associated values are lists containing the identifiers of the references associated with that particular keyword.

     Any reference identifier used in these lists has to exist as a key in the **references** dictionary, see the next item for its definition.

   references
   : The value is a Tcl dictionary, using the identifiers for the references known to the index as keys. The associated values are 2-element lists containing the type and label of the reference, in this order.

     Any key here has to be associated with at least one keyword, i.e. occur in at least one of the reference lists which are the values in the **keywords** dictionary, see previous item

for its definition.

1. The *type* of a reference can be one of two values,
   manpage
   The identifier of the reference is interpreted as symbolic file name, referring to one of the documents the index was made for.
   url
   The identifier of the reference is interpreted as an url, referring to some external location, like a website, etc.

canonical serialization
The canonical serialization of a keyword index has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of the keyword index.
1. The keys found in all the nested Tcl dictionaries are sorted in ascending dictionary order, as generated by Tcl's builtin command **lsort -increasing -dict**.
2. The references listed for each keyword of the index, if any, are listed in ascending dictionary order of their *labels*, as generated by Tcl's builtin command **lsort -increasing -dict**.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Text formatter plugin

Copyright


```
Copyright (c) 2009-2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


idx_import

Keywords: idx_import, conversion, docidx, documentation, import, index, json, keyword index, manpage, markup, parsing, plugin, reference, url

Name

doctools::idx::import - Importing keyword indices

Synopsis

package require doctools::idx::import ?**0.2.2**?

package require Tcl 8.5 9
package require struct::map
package require doctools::idx::structure
package require snit
package require pluginmgr
::doctools::idx::import *objectName*
objectName method ?*arg arg ...*?
objectName **destroy**
objectName **import text** *text* ?*format*?
objectName **import file** *path* ?*format*?
objectName **import object text** *object text* ?*format*?
objectName **import object file** *object path* ?*format*?
objectName **config names**
objectName **config get**
objectName **config set** *name* ?*value*?
objectName **config unset** *pattern...*
objectName **includes**
objectName **include add** *path*
objectName **include remove** *path*
objectName **include clear**
IncludeFile *currentfile path*
import *text configuration*

Description

This package provides a class to manage the plugins for the import of keyword indices from other formats, i.e. their conversion from, for example *docidx*, *json*, etc.

This is one of the three public pillars the management of keyword indices resides on. The other two pillars are

1. *Exporting keyword indices*, and
2. *Holding keyword indices*

For information about the Concepts of keyword indices, and their parts, see the same-named section. For information about the data structure which is the major output of the manager objects provided by this package see the section Keyword index serialization format.

The plugin system of our class is based on the package **pluginmgr**, and configured to look for plugins using

1. the environment variable **DOCTOOLS_IDX_IMPORT_PLUGINS**,
2. the environment variable **DOCTOOLS_IDX_PLUGINS**,
3. the environment variable **DOCTOOLS_PLUGINS**,
4. the path "*~/.doctools/idx/import/plugin*"
5. the path "*~/.doctools/idx/plugin*"
6. the path "*~/.doctools/plugin*"
7. the path "*~/.doctools/idx/import/plugins*"
8. the path "*~/.doctools/idx/plugins*"
9. the path "*~/.doctools/plugins*"
10. the registry entry "HKEY_CURRENT_USER\SOFTWARE\DOCTOOLS\IDX\IMPORT\PLUGINS"

11. the registry entry "HKEY_CURRENT_USER\SOFTWARE\DOCTOOLS\IDX\PLUGINS"
12. the registry entry "HKEY_CURRENT_USER\SOFTWARE\DOCTOOLS\PLUGINS"

The last three are used only when the package is run on a machine using Windows(tm) operating system.

The whole system is delivered with two predefined import plugins, namely

docidx
: See *docidx import plugin* for details.

json
: See *json import plugin* for details.

Readers wishing to write their own import plugin for some format, i.e. *plugin writer*s reading and understanding the section containing the Import plugin API v2 reference is an absolute necessity, as it specifies the interaction between this package and its plugins in detail.

Concepts

1. A *keyword index* consists of a (possibly empty) set of *keywords*.
2. Each keyword in the set is identified by its name.
3. Each keyword has a (possibly empty) set of *references*.
4. A reference can be associated with more than one keyword.
5. A reference not associated with at least one keyword is not possible however.
6. Each reference is identified by its target, specified as either an url or symbolic filename, depending on the type of reference (**url**, or **manpage**).
7. The type of a reference (url, or manpage) depends only on the reference itself, and not the keywords it is associated with.
8. In addition to a type each reference has a descriptive label as well. This label depends only on the reference itself, and not the keywords it is associated with.

A few notes

1. Manpage references are intended to be used for references to the documents the index is made for. Their target is a symbolic file name identifying the document, and export plugins may replace symbolic with actual file names, if specified.
2. Url references are intended on the othre hand are inteded to be used for links to anything else, like websites. Their target is an url.
3. While url and manpage references share a namespace for their identifiers, this should be no problem, given that manpage identifiers are symbolic filenames and as such they should never look like urls, the identifiers for url references.

API

Package commands

::doctools::idx::import *objectName*
: This command creates a new import manager object with an associated Tcl command whose name is *objectName*. This *object* command is explained in full detail in the sections Object command and Object methods. The object command will be created under the current namespace if the *objectName* is not fully qualified, and in the specified namespace otherwise.

Object command

All objects created by the **::doctools::idx::import** command have the following general form:

objectName method ?*arg arg ...*?
The method **method** and its *arg*'uments determine the exact behavior of the command. See section Object methods for the detailed specifications.

Object methods

***objectName* destroy**
This method destroys the object it is invoked for.

***objectName* import text *text* ?*format*?**
This method takes the *text* and converts it from the specified *format* to the canonical serialization of a keyword index using the import plugin for the format. An error is thrown if no plugin could be found for the format. The serialization generated by the conversion process is returned as the result of this method.

If no format is specified the method defaults to **docidx**.

The specification of what a *canonical* serialization is can be found in the section Keyword index serialization format.

The plugin has to conform to the interface specified in section Import plugin API v2 reference.

***objectName* import file *path* ?*format*?**
This method is a convenient wrapper around the **import text** method described by the previous item. It reads the contents of the specified file into memory, feeds the result into **import text** and returns the resulting serialization as its own result.

***objectName* import object text *object text* ?*format*?**
This method is a convenient wrapper around the **import text** method described by the previous item. It expects that *object* is an object command supporting a **deserialize** method expecting the canonical serialization of a keyword index. It imports the text using **import text** and then feeds the resulting serialization into the *object* via **deserialize**. This method returns the empty string as it result.

***objectName* import object file *object path* ?*format*?**
This method behaves like **import object text**, except that it reads the text to convert from the specified file instead of being given it as argument.

***objectName* config names**
This method returns a list containing the names of all configuration variables currently known to the object.

***objectName* config get**
This method returns a dictionary containing the names and values of all configuration variables currently known to the object.

***objectName* config set *name* ?*value*?**
This method sets the configuration variable *name* to the specified *value* and returns the new value of the variable.

If no value is specified it simply returns the current value, without changing it.

Note that while the user can set the predefined configuration variables **user** and **format** doing so will have no effect, these values will be internally overridden when invoking an import plugin.

objectName* config unset *pattern...
This method unsets all configuration variables matching the specified glob *pattern*s. If no pattern is specified it will unset all currently defined configuration variables.

***objectName* includes**
This method returns a list containing the currently specified paths to use to search for include files when processing input. The order of paths in the list corresponds to the order in which they are used, from first to last, and also corresponds to the order in which they were added to the object.

objectName* include add *path
This methods adds the specified *path* to the list of paths to use to search for include files when processing input. The path is added to the end of the list, causing it to be searched after all previously added paths. The result of the command is the empty string.

The method does nothing if the path is already known.

objectName* include remove *path
This methods removes the specified *path* from the list of paths to use to search for include files when processing input. The result of the command is the empty string.

The method does nothing if the path is not known.

***objectName* include clear**
This method clears the list of paths to use to search for include files when processing input. The result of the command is the empty string.

Import plugin API v2 reference

Plugins are what this package uses to manage the support for any input format beyond the Keyword index serialization format. Here we specify the API the objects created by this package use to interact with their plugins.

A plugin for this package has to follow the rules listed below:

1. A plugin is a package.
2. The name of a plugin package has the form doctools::idx::import::**FOO**, where **FOO** is the name of the format the plugin will generate output for. This name is also the argument to provide to the various **import** methods of import manager objects to get a string encoding a keyword index in that format.
3. The plugin can expect that the package **doctools::idx::export::plugin** is present, as indicator that it was invoked from a genuine plugin manager.
4. The plugin can expect that a command named **IncludeFile** is present, with the signature
   IncludeFile *currentfile path*
   This command has to be invoked by the plugin when it has to process an included file, if the format has the concept of such. An example of such a format would be *docidx*.

   The plugin has to supply the following arguments
   string *currentfile*
   The path of the file it is currently processing. This may be the empty string if no such is

known.

string *path*

The path of the include file as specified in the include directive being processed.

```
The result of the command will be a 5-element list containing

1. A boolean flag indicating the success (**True**) or failure
(**False**) of the operation.

2. In case of success the contents of the included file, and the
empty string otherwise.

3. The resolved, i.e. absolute path of the included file, if
possible, or the unchanged *path* argument. This is for display in
an error message, or as the *currentfile* argument of another call
to **IncludeFile** should this file contain more files.

4. In case of success an empty string, and for failure a code
indicating the reason for it, one of

    notfound
    : The specified file could not be found.

    notread
    : The specified file was found, but not be read into memory.


5. An empty string in case of success of a **notfound** failure,
and an additional error message describing the reason for a
**notread** error in more detail.
```

1. A plugin has to provide one command, with the signature shown below.

import *text configuration*

Whenever an import manager of **doctools::idx** has to parse input for an index it will invoke this command.

string *text*

This argument will contain the text encoding the index per the format the plugin is for.

dictionary *configuration*

This argument will contain the current configuration to apply to the parsing, as a dictionary mapping from variable names to values.

The following configuration variables have a predefined meaning all plugins have to obey, although they can ignore this information at their discretion. Any other other configuration variables recognized by a plugin will be described in the manpage for that plugin.

user

This variable is expected to contain the name of the user owning the process invoking

the plugin.

format
This variable is expected to contain the name of the format whose plugin is invoked.

1. A single usage cycle of a plugin consists of the invokations of the command **import**. This call has to leave the plugin in a state where another usage cycle can be run without problems.

Keyword index serialization format

Here we specify the format used by the doctools v2 packages to serialize keyword indices as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a keyword index may have more than one regular serialization only exactly one of them will be *canonical*.

regular serialization

1. An index serialization is a nested Tcl dictionary.
1. This dictionary holds a single key, **doctools::idx**, and its value. This value holds the contents of the index.
2. The contents of the index are a Tcl dictionary holding the title of the index, a label, and the keywords and references. The relevant keys and their values are

title
The value is a string containing the title of the index.

label
The value is a string containing a label for the index.

keywords
The value is a Tcl dictionary, using the keywords known to the index as keys. The associated values are lists containing the identifiers of the references associated with that particular keyword.

Any reference identifier used in these lists has to exist as a key in the **references** dictionary, see the next item for its definition.

references
The value is a Tcl dictionary, using the identifiers for the references known to the index as keys. The associated values are 2-element lists containing the type and label of the reference, in this order.

Any key here has to be associated with at least one keyword, i.e. occur in at least one of the reference lists which are the values in the **keywords** dictionary, see previous item for its definition.

1. The *type* of a reference can be one of two values,

manpage
The identifier of the reference is interpreted as symbolic file name, referring to one of the documents the index was made for.

url
The identifier of the reference is interpreted as an url, referring to some external location, like a website, etc.

canonical serialization

The canonical serialization of a keyword index has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of the keyword index.

1. The keys found in all the nested Tcl dictionaries are sorted in ascending dictionary order, as generated by Tcl's builtin command **lsort -increasing -dict**.
2. The references listed for each keyword of the index, if any, are listed in ascending dictionary order of their *labels*, as generated by Tcl's builtin command **lsort -increasing -dict**.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Documentation tools

Copyright


```
Copyright (c) 2009-2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


idx_import_json

Keywords: idx_import_json, JSON, deserialization, doctools, import, index

Name

doctools::idx::import::json - JSON import plugin

Synopsis

package require Tcl 8.5 9
package require doctools::idx::import::json ?**0.2.1**?
package require doctools::idx::structure
package require json
import *string configuration*

Description

This package implements the doctools keyword index import plugin for the parsing of JSON markup.

This is an internal package of doctools, for use by the higher level management packages handling keyword indices, especially **doctools::idx::import**, the import manager.

Using it from a regular interpreter is possible, however only with contortions, and is not

recommended. The proper way to use this functionality is through the package **doctools::idx::import** and the import manager objects it provides.

API

The API provided by this package satisfies the specification of the docidx import plugin API version 2.

import *string configuration*

This command takes the *string* and parses it as JSON markup encoding a keyword index, in the context of the specified *configuration* (a dictionary). The result of the command is the canonical serialization of that keyword index, in the form specified in section Keyword index serialization format.

JSON notation of keyword indices

The JSON format used for keyword indices is a direct translation of the Keyword index serialization format, mapping Tcl dictionaries as JSON objects and Tcl lists as JSON arrays. For example, the Tcl serialization

```
doctools::idx {
    label {Keyword Index}
    keywords {
        changelog  {changelog.man cvs.man}
        conversion {doctools.man docidx.man doctoc.man apps/dtplite.man
    mpexpand.man}
        cvs        cvs.man
    }
    references {
        apps/dtplite.man {manpage dtplite}
        changelog.man    {manpage doctools::changelog}
        cvs.man          {manpage doctools::cvs}
        docidx.man       {manpage doctools::idx}
        doctoc.man       {manpage doctools::toc}
        doctools.man     {manpage doctools}
        mpexpand.man     {manpage mpexpand}
    }
    title {}
}
```

is equivalent to the JSON string

```
{
    "doctools::idx" : {
        "label"      : "Keyword Index",
        "keywords"   : {
            "changelog"  : ["changelog.man","cvs.man"],
            "conversion" :
    ["doctools.man","docidx.man","doctoc.man","apps\/dtplite.man","mpex
    pand.man"],
```

```
            "cvs"        : ["cvs.man"],
        },
        "references" : {
            "apps\/dtplite.man" : ["manpage","dtplite"],
            "changelog.man"     : ["manpage","doctools::changelog"],
            "cvs.man"           : ["manpage","doctools::cvs"],
            "docidx.man"        : ["manpage","doctools::idx"],
            "doctoc.man"        : ["manpage","doctools::toc"],
            "doctools.man"      : ["manpage","doctools"],
            "mpexpand.man"      : ["manpage","mpexpand"]
        },
        "title"      : ""
    }
}
```

Keyword index serialization format

Here we specify the format used by the doctools v2 packages to serialize keyword indices as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a keyword index may have more than one regular serialization only exactly one of them will be *canonical*.

regular serialization

1. An index serialization is a nested Tcl dictionary.
1. This dictionary holds a single key, **doctools::idx**, and its value. This value holds the contents of the index.
2. The contents of the index are a Tcl dictionary holding the title of the index, a label, and the keywords and references. The relevant keys and their values are

   title
   : The value is a string containing the title of the index.

   label
   : The value is a string containing a label for the index.

   keywords
   : The value is a Tcl dictionary, using the keywords known to the index as keys. The associated values are lists containing the identifiers of the references associated with that particular keyword.

     Any reference identifier used in these lists has to exist as a key in the **references** dictionary, see the next item for its definition.

   references
   : The value is a Tcl dictionary, using the identifiers for the references known to the index as keys. The associated values are 2-element lists containing the type and label of the reference, in this order.

     Any key here has to be associated with at least one keyword, i.e. occur in at least one of the reference lists which are the values in the **keywords** dictionary, see previous item for its definition.

1. The *type* of a reference can be one of two values,
   manpage
   : The identifier of the reference is interpreted as symbolic file name, referring to one of the documents the index was made for.

   url
   : The identifier of the reference is interpreted as an url, referring to some external location, like a website, etc.

canonical serialization
: The canonical serialization of a keyword index has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of the keyword index.

1. The keys found in all the nested Tcl dictionaries are sorted in ascending dictionary order, as generated by Tcl's builtin command **lsort -increasing -dict**.
2. The references listed for each keyword of the index, if any, are listed in ascending dictionary order of their *labels*, as generated by Tcl's builtin command **lsort -increasing -dict**.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Text formatter plugin

Copyright


```
Copyright (c) 2009-2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


idx_introduction

Keywords: idx_introduction, conversion, formatting, index, keyword index, markup, parsing, plugin, semantic markup

Name

doctools2idx_introduction - DocTools - Keyword indices

Description

docidx (short for *documentation indices*) stands for a set of related, yet different, entities which are working together for the easy creation and transformation of keyword indices for documentation.

These are

1. A tcl based language for the semantic markup of a keyword index. Markup is represented by Tcl commands. Beginners should start with the *docidx language introduction*. The formal specification is split over two documents, one dealing with the *docidx language syntax*, the other a *docidx language command reference*.
2. A set of packages for the programmatic manipulation of keyword indices in memory, and their conversion between various formats, reading and writing. The aforementioned markup language is one of the formats which can be both read from and written to.
3. The system for the conversion of indices is based on a plugin mechanism, for this we have two APIs describing the interface between the packages above and the import/export plugins.

Which of the more detailed documents are relevant to the reader of this introduction depends on their role in the documentation process.

1. A *writer* of documentation has to understand the markup language itself. A beginner to docidx should read the more informally written *docidx language introduction* first. Having digested this the formal *docidx language syntax* specification should become understandable. A writer experienced with docidx may only need the *docidx language command reference* from time to time to refresh her memory.

   While a document is written the **dtp** application can be used to validate it, and after completion it also performs the conversion into the chosen system of visual markup, be it *roff, HTML, plain text, wiki, etc. The simpler **dtplite** application makes internal use of docidx when handling directories of documentation, automatically generating a proper keyword index for them.
2. A *processor* of documentation written in the *docidx* markup language has to know which tools are available for use.

   The main tool is the aforementioned **dtp** application provided by Tcllib. The simpler **dtplite** does not expose docidx to the user. At the bottom level, common to both applications, however we find the three packages providing the basic facilities to handle keyword indices, i.e. import from textual formats, programmatic manipulation in memory, and export to textual formats. These are

   doctools::idx
   : Programmatic manipulation of keyword indices in memory.

   doctools::idx::import
   : Import of keyword indices from various textual formats. The set of supported formats is extensible through plugin packages.

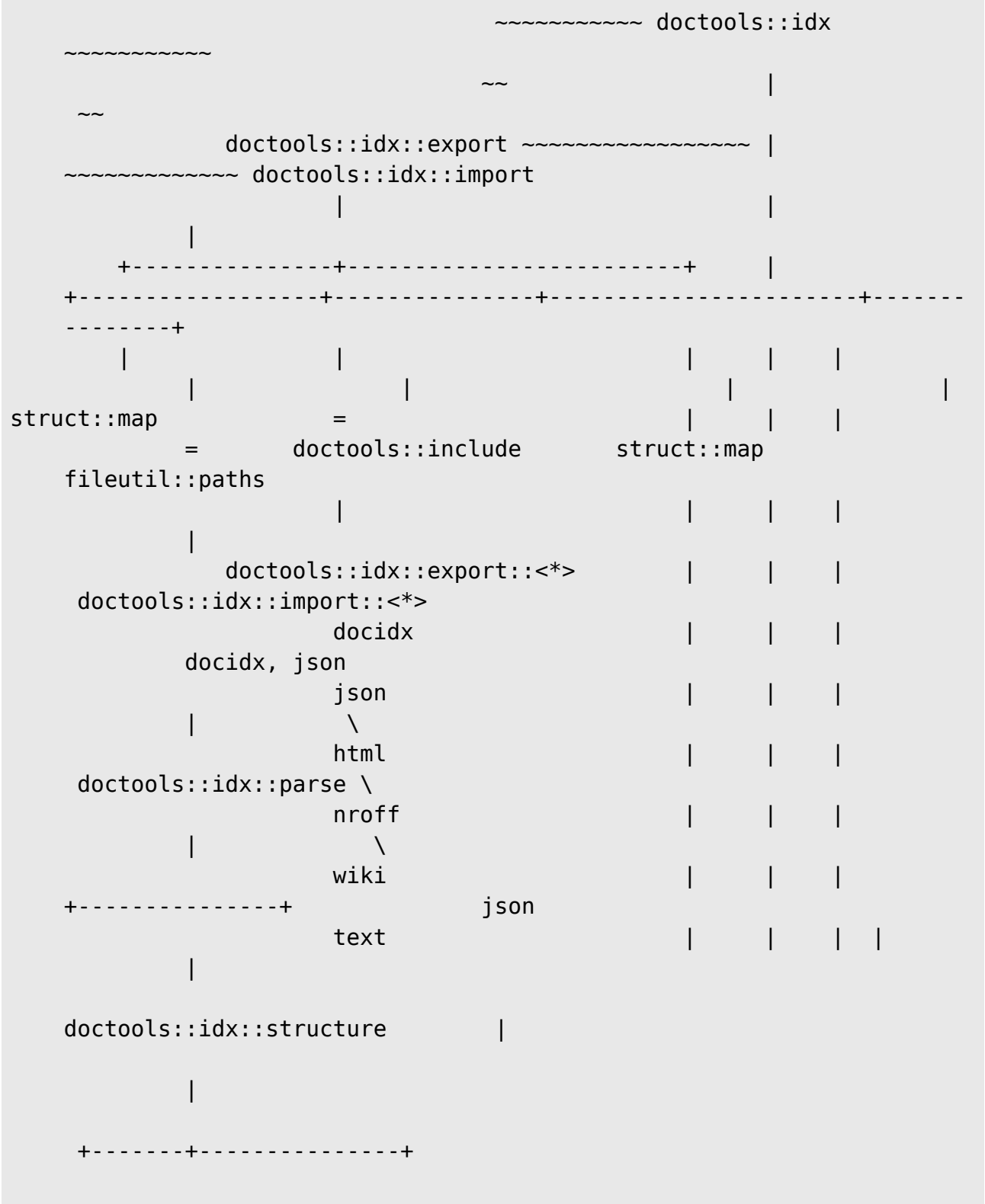
   doctools::idx::export
   : Export of keyword indices to various textual formats. The set of supported formats is extensible through plugin packages.

1. See also section Package overview for an overview of the dependencies between these and other, supporting packages.
2. At last, but not least, *plugin writers* have to understand the interaction between the import and export packages and their plugins. These APIs are described in the documentation for the two relevant packages, i.e.
   - **doctools::idx::import**
   - **doctools::idx::export**

Related formats

The docidx format does not stand alone, it has two companion formats. These are called *doctoc* and *doctools*, and they are intended for the markup of *tables of contents*, and of general documentation, respectively. They are described in their own sets of documents, starting at the *DocTools - Tables Of Contents* and the *DocTools - General*, respectively.

Package overview



```
     |                        |
          doctools::html  doctools::html::cssdefaults
    doctools::tcl::parse    doctools::msgcat
                |
                               |
          doctools::text  doctools::nroff::man_macros
                               =

                               |

                     doctools::msgcat::idx::<*>

                               c, en, de, fr

                               (fr == en for now)
        ~~      Interoperable objects, without actual package
    dependencies
        --      Package dependency, higher requires lower package
        =       Dynamic dependency through plugin system
        <*>     Multiple packages following the given form of naming.
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Documentation tools

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


idx_msgcat_c

Keywords: idx_msgcat_c, C, catalog package, docidx, doctools, i18n, internationalization, l10n, localization, message catalog, message package

Name

doctools::msgcat::idx::c - Message catalog for the docidx parser (C)

Synopsis

package require Tcl 8.5 9
package require msgcat
package require doctools::msgcat::idx::c ?**0.1**?

Description

The package **doctools::msgcat::idx::c** is a support module providing the C language message catalog for the docidx parser in the doctools system version 2. As such it is an internal package a regular user (developer) should not be in direct contact with.

If you are such please go the documentation of either

1. **doctools::doc**,
2. **doctools::toc**, or
3. **doctools::idx**

Within the system architecture this package resides under the package **doctools::msgcat** providing the general message catalog management within the system. *Note* that there is *no* explicit dependency between the manager and catalog packages. The catalog is a plugin which is selected and loaded dynamically.

API

This package has no exported API.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Documentation tools

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


idx_msgcat_de

Keywords: idx_msgcat_de, DE, catalog package, docidx, doctools, i18n, internationalization, l10n , localization, message catalog, message package

Name

doctools::msgcat::idx::de - Message catalog for the docidx parser (DE)

Synopsis

package require Tcl 8.5 9

package require msgcat
package require doctools::msgcat::idx::de ?**0.1**?

Description

The package **doctools::msgcat::idx::de** is a support module providing the DE (german) language message catalog for the docidx parser in the doctools system version 2. As such it is an internal package a regular user (developer) should not be in direct contact with.

If you are such please go the documentation of either

1. **doctools::doc**,
2. **doctools::toc**, or
3. **doctools::idx**

Within the system architecture this package resides under the package **doctools::msgcat** providing the general message catalog management within the system. *Note* that there is *no* explicit dependency between the manager and catalog packages. The catalog is a plugin which is selected and loaded dynamically.

API

This package has no exported API.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Documentation tools

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


idx_msgcat_en

Keywords: idx_msgcat_en, EN, catalog package, docidx, doctools, i18n, internationalization, l10n , localization, message catalog, message package

Name

doctools::msgcat::idx::en - Message catalog for the docidx parser (EN)

Synopsis

package require Tcl 8.5 9
package require msgcat

package require doctools::msgcat::idx::en ?**0.1**?

Description

The package **doctools::msgcat::idx::en** is a support module providing the EN (english) language message catalog for the docidx parser in the doctools system version 2. As such it is an internal package a regular user (developer) should not be in direct contact with.

If you are such please go the documentation of either

1. **doctools::doc**,
2. **doctools::toc**, or
3. **doctools::idx**

Within the system architecture this package resides under the package **doctools::msgcat** providing the general message catalog management within the system. *Note* that there is *no* explicit dependency between the manager and catalog packages. The catalog is a plugin which is selected and loaded dynamically.

API

This package has no exported API.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Documentation tools

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


idx_msgcat_fr

Keywords: idx_msgcat_fr, FR, catalog package, docidx, doctools, i18n, internationalization, l10n, localization, message catalog, message package

Name

doctools::msgcat::idx::fr - Message catalog for the docidx parser (FR)

Synopsis

package require Tcl 8.5 9
package require msgcat
package require doctools::msgcat::idx::fr ?**0.1**?

Description

The package **doctools::msgcat::idx::fr** is a support module providing the FR (french) language message catalog for the docidx parser in the doctools system version 2. As such it is an internal package a regular user (developer) should not be in direct contact with.

If you are such please go the documentation of either

1. **doctools::doc**,
2. **doctools::toc**, or
3. **doctools::idx**

Within the system architecture this package resides under the package **doctools::msgcat** providing the general message catalog management within the system. *Note* that there is *no* explicit dependency between the manager and catalog packages. The catalog is a plugin which is selected and loaded dynamically.

API

This package has no exported API.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Documentation tools

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


idx_parse

Keywords: idx_parse, docidx, doctools, lexer, parser

Name

doctools::idx::parse - Parsing text in docidx format

Synopsis

package require doctools::idx::parse ?**0.2**?
package require Tcl 8.5 9
package require doctools::idx::structure
package require doctools::msgcat
package require doctools::tcl::parse
package require fileutil

package require logger
package require snit
package require struct::list
package require struct::stack
::doctools::idx::parse text *text*
::doctools::idx::parse file *path*
::doctools::idx::parse includes
::doctools::idx::parse include add *path*
::doctools::idx::parse include remove *path*
::doctools::idx::parse include clear
::doctools::idx::parse vars
::doctools::idx::parse var set *name value*
::doctools::idx::parse var unset *name*
::doctools::idx::parse var clear ?*pattern*?

Description

This package provides commands to parse text written in the *docidx* markup language and convert it into the canonical serialization of the keyword index encoded in the text. See the section Keyword index serialization format for specification of their format.

This is an internal package of doctools, for use by the higher level packages handling *docidx* documents.

API

::doctools::idx::parse text *text*

The command takes the string contained in *text* and parses it under the assumption that it contains a document written using the *docidx* markup language. An error is thrown if this assumption is found to be false. The format of these errors is described in section Parse errors .

When successful the command returns the canonical serialization of the keyword index which was encoded in the text. See the section Keyword index serialization format for specification of that format.

::doctools::idx::parse file *path*

The same as **text**, except that the text to parse is read from the file specified by *path*.

::doctools::idx::parse includes

This method returns the current list of search paths used when looking for include files.

::doctools::idx::parse include add *path*

This method adds the *path* to the list of paths searched when looking for an include file. The call is ignored if the path is already in the list of paths. The method returns the empty string as its result.

::doctools::idx::parse include remove *path*

This method removes the *path* from the list of paths searched when looking for an include file. The call is ignored if the path is not contained in the list of paths. The method returns the empty string as its result.

::doctools::idx::parse include clear

This method clears the list of search paths for include files.

::doctools::idx::parse vars
This method returns a dictionary containing the current set of predefined variables known to the **vset** markup command during processing.

::doctools::idx::parse var set *name value*
This method adds the variable *name* to the set of predefined variables known to the **vset** markup command during processing, and gives it the specified *value*. The method returns the empty string as its result.

::doctools::idx::parse var unset *name*
This method removes the variable *name* from the set of predefined variables known to the **vset** markup command during processing. The method returns the empty string as its result.

::doctools::idx::parse var clear ?*pattern*?
This method removes all variables matching the *pattern* from the set of predefined variables known to the **vset** markup command during processing. The method returns the empty string as its result.

The pattern matching is done with **string match**, and the default pattern used when none is specified, is *.

Parse errors

The format of the parse error messages thrown when encountering violations of the *docidx* markup syntax is human readable and not intended for processing by machines. As such it is not documented.

However, the errorCode attached to the message is machine-readable and has the following format:

1. The error code will be a list, each element describing a single error found in the input. The list has at least one element, possibly more.
2. Each error element will be a list containing six strings describing an error in detail. The strings will be
   1. The path of the file the error occurred in. This may be empty.
   2. The range of the token the error was found at. This range is a two-element list containing the offset of the first and last character in the range, counted from the beginning of the input (file). Offsets are counted from zero.
   3. The line the first character after the error is on. Lines are counted from one.
   4. The column the first character after the error is at. Columns are counted from zero.
   5. The message code of the error. This value can be used as argument to **msgcat::mc** to obtain a localized error message, assuming that the application had a suitable call of **doctools::msgcat::init** to initialize the necessary message catalogs (See package **doctools::msgcat**).
   6. A list of details for the error, like the markup command involved. In the case of message code **docidx/include/syntax** this value is the set of errors found in the included file, using the format described here.

[docidx] notation of keyword indices

The docidx format for keyword indices, also called the *docidx markup language*, is too large to be covered in single section. The interested reader should start with the document

1. *docidx language introduction*

and then proceed from there to the formal specifications, i.e. the documents

1. *docidx language syntax* and
2. *docidx language command reference*.

to get a thorough understanding of the language.

Keyword index serialization format

Here we specify the format used by the doctools v2 packages to serialize keyword indices as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a keyword index may have more than one regular serialization only exactly one of them will be *canonical*.

regular serialization

1. An index serialization is a nested Tcl dictionary.
1. This dictionary holds a single key, **doctools::idx**, and its value. This value holds the contents of the index.
2. The contents of the index are a Tcl dictionary holding the title of the index, a label, and the keywords and references. The relevant keys and their values are

   title
   : The value is a string containing the title of the index.

   label
   : The value is a string containing a label for the index.

   keywords
   : The value is a Tcl dictionary, using the keywords known to the index as keys. The associated values are lists containing the identifiers of the references associated with that particular keyword.

     Any reference identifier used in these lists has to exist as a key in the **references** dictionary, see the next item for its definition.

   references
   : The value is a Tcl dictionary, using the identifiers for the references known to the index as keys. The associated values are 2-element lists containing the type and label of the reference, in this order.

     Any key here has to be associated with at least one keyword, i.e. occur in at least one of the reference lists which are the values in the **keywords** dictionary, see previous item for its definition.

1. The *type* of a reference can be one of two values,

   manpage
   : The identifier of the reference is interpreted as symbolic file name, referring to one of the documents the index was made for.

   url
   : The identifier of the reference is interpreted as an url, referring to some external location, like a website, etc.

canonical serialization

The canonical serialization of a keyword index has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of the keyword index.

1. The keys found in all the nested Tcl dictionaries are sorted in ascending dictionary order, as generated by Tcl's builtin command **lsort -increasing -dict**.
2. The references listed for each keyword of the index, if any, are listed in ascending dictionary order of their *labels*, as generated by Tcl's builtin command **lsort -increasing -dict**.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Documentation tools

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


idx_structure

Keywords: idx_structure, deserialization, docidx, doctools, serialization

Name

doctools::idx::structure - Docidx serialization utilities

Synopsis

package require doctools::idx::structure ?**0.2**?
package require Tcl 8.5 9
package require logger
package require snit
::doctools::idx::structure verify *serial* ?*canonvar*?
::doctools::idx::structure verify-as-canonical *serial*
::doctools::idx::structure canonicalize *serial*
::doctools::idx::structure print *serial*
::doctools::idx::structure merge *seriala serialb*

Description

This package provides commands to work with the serializations of keyword indices as managed

by the doctools system v2, and specified in section Keyword index serialization format.

This is an internal package of doctools, for use by the higher level packages handling keyword indices and their conversion into and out of various other formats, like documents written using *docidx* markup.

API

::doctools::idx::structure verify *serial* ?*canonvar*?

This command verifies that the content of *serial* is a valid *regular* serialization of a keyword index and will throw an error if that is not the case. The result of the command is the empty string.

If the argument *canonvar* is specified it is interpreted as the name of a variable in the calling context. This variable will be written to if and only if *serial* is a valid regular serialization. Its value will be a boolean, with **True** indicating that the serialization is not only valid, but also *canonical*. **False** will be written for a valid, but non-canonical serialization.

For the specification of regular and canonical keyword index serializations see the section Keyword index serialization format.

::doctools::idx::structure verify-as-canonical *serial*

This command verifies that the content of *serial* is a valid *canonical* serialization of a keyword index and will throw an error if that is not the case. The result of the command is the empty string.

For the specification of canonical keyword index serializations see the section Keyword index serialization format.

::doctools::idx::structure canonicalize *serial*

This command assumes that the content of *serial* is a valid *regular* serialization of a keyword index and will throw an error if that is not the case.

It will then convert the input into the *canonical* serialization of the contained keyword index and return it as its result. If the input is already canonical it will be returned unchanged.

For the specification of regular and canonical keyword index serializations see the section Keyword index serialization format.

::doctools::idx::structure print *serial*

This command assumes that the argument *serial* contains a valid regular serialization of a keyword index and returns a string containing that index in a human readable form.

The exact format of this form is not specified and cannot be relied on for parsing or other machine-based activities.

For the specification of regular keyword index serializations see the section Keyword index serialization format.

::doctools::idx::structure merge *seriala serialb*

This command accepts the regular serializations of two keyword indices and uses them to create their union. The result of the command is the canonical serialization of this unified

keyword index.

Title and label of the resulting index are taken from the index contained in *serialb*. The set of keys, references and their connections is the union of the set of keys and references of the two inputs.

For the specification of regular and canonical keyword index serializations see the section Keyword index serialization format.

Keyword index serialization format

Here we specify the format used by the doctools v2 packages to serialize keyword indices as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a keyword index may have more than one regular serialization only exactly one of them will be *canonical*.

regular serialization

1. An index serialization is a nested Tcl dictionary.
1. This dictionary holds a single key, **doctools::idx**, and its value. This value holds the contents of the index.
2. The contents of the index are a Tcl dictionary holding the title of the index, a label, and the keywords and references. The relevant keys and their values are

   title
   The value is a string containing the title of the index.

   label
   The value is a string containing a label for the index.

   keywords
   The value is a Tcl dictionary, using the keywords known to the index as keys. The associated values are lists containing the identifiers of the references associated with that particular keyword.

   Any reference identifier used in these lists has to exist as a key in the **references** dictionary, see the next item for its definition.

   references
   The value is a Tcl dictionary, using the identifiers for the references known to the index as keys. The associated values are 2-element lists containing the type and label of the reference, in this order.

   Any key here has to be associated with at least one keyword, i.e. occur in at least one of the reference lists which are the values in the **keywords** dictionary, see previous item for its definition.

1. The *type* of a reference can be one of two values,

   manpage
   The identifier of the reference is interpreted as symbolic file name, referring to one of the documents the index was made for.

   url

The identifier of the reference is interpreted as an url, referring to some external location, like a website, etc.

canonical serialization

The canonical serialization of a keyword index has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of the keyword index.

1. The keys found in all the nested Tcl dictionaries are sorted in ascending dictionary order, as generated by Tcl's builtin command **lsort -increasing -dict**.
2. The references listed for each keyword of the index, if any, are listed in ascending dictionary order of their *labels*, as generated by Tcl's builtin command **lsort -increasing -dict**.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Documentation tools

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


import_docidx

Keywords: import_docidx, deserialization, docidx, doctools, import, index

Name

doctools::idx::import::docidx - docidx import plugin

Synopsis

package require Tcl 8.5 9
package require doctools::idx::import::docidx ?**0.2.1**?
package require doctools::idx::parse
package require doctools::idx::structure
package require doctools::msgcat
package require doctools::tcl::parse
package require fileutil
package require logger
package require snit

package require struct::list
package require struct::set
package require struct::stack
package require struct::tree
package require treeql
import *string configuration*

Description

This package implements the doctools keyword index import plugin for the parsing of docidx markup.

This is an internal package of doctools, for use by the higher level management packages handling keyword indices, especially **doctools::idx::import**, the import manager.

Using it from a regular interpreter is possible, however only with contortions, and is not recommended. The proper way to use this functionality is through the package **doctools::idx::import** and the import manager objects it provides.

API

The API provided by this package satisfies the specification of the docidx import plugin API version 2.

import *string configuration*

This command takes the *string* and parses it as docidx markup encoding a keyword index, in the context of the specified *configuration* (a dictionary). The result of the command is the canonical serialization of that keyword index, in the form specified in section Keyword index serialization format.

[docidx] notation of keyword indices

The docidx format for keyword indices, also called the *docidx markup language*, is too large to be covered in single section. The interested reader should start with the document

1. *docidx language introduction*

and then proceed from there to the formal specifications, i.e. the documents

1. *docidx language syntax* and
2. *docidx language command reference*.

to get a thorough understanding of the language.

Keyword index serialization format

Here we specify the format used by the doctools v2 packages to serialize keyword indices as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a keyword index may have more than one regular serialization only exactly one of them will be *canonical*.

regular serialization

1. An index serialization is a nested Tcl dictionary.
1. This dictionary holds a single key, **doctools::idx**, and its value. This value holds the contents of the index.
2. The contents of the index are a Tcl dictionary holding the title of the index, a label, and the

keywords and references. The relevant keys and their values are

title
: The value is a string containing the title of the index.

label
: The value is a string containing a label for the index.

keywords
: The value is a Tcl dictionary, using the keywords known to the index as keys. The associated values are lists containing the identifiers of the references associated with that particular keyword.

  Any reference identifier used in these lists has to exist as a key in the **references** dictionary, see the next item for its definition.

references
: The value is a Tcl dictionary, using the identifiers for the references known to the index as keys. The associated values are 2-element lists containing the type and label of the reference, in this order.

  Any key here has to be associated with at least one keyword, i.e. occur in at least one of the reference lists which are the values in the **keywords** dictionary, see previous item for its definition.

1. The *type* of a reference can be one of two values,

   manpage
   : The identifier of the reference is interpreted as symbolic file name, referring to one of the documents the index was made for.

   url
   : The identifier of the reference is interpreted as an url, referring to some external location, like a website, etc.

canonical serialization
: The canonical serialization of a keyword index has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of the keyword index.

  1. The keys found in all the nested Tcl dictionaries are sorted in ascending dictionary order, as generated by Tcl's builtin command **lsort -increasing -dict**.
  2. The references listed for each keyword of the index, if any, are listed in ascending dictionary order of their *labels*, as generated by Tcl's builtin command **lsort -increasing -dict**.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be

made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Text formatter plugin

Copyright

```
Copyright (c) 2009-2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


doctools2toc

export_doctoc
Keywords: export_doctoc, doctoc, doctools, export, serialization, table of contents, toc

Name
doctools::toc::export::doctoc - doctoc export plugin

Synopsis
package require Tcl 8.5 9
package require doctools::toc::export::doctoc ?**0.2.2**?
export *serial configuration*

Description
This package implements the doctools table of contents export plugin for the generation of doctoc markup.

This is an internal package of doctools, for use by the higher level management packages handling tables of contents, especially **doctools::toc::export**, the export manager.

Using it from a regular interpreter is possible, however only with contortions, and is not recommended. The proper way to use this functionality is through the package **doctools::toc::export** and the export manager objects it provides.

API
The API provided by this package satisfies the specification of the doctoc export plugin API version 2.

export *serial configuration*
This command takes the canonical serialization of a table of contents, as specified in section Toc serialization format, and contained in *serial*, the *configuration*, a dictionary, and generates doctoc markup encoding the table. The created string is then returned as the result of the command.

[doctoc] notation of tables of contents
The doctoc format for tables of contents, also called the *doctoc markup language*, is too large to be covered in single section. The interested reader should start with the document

1. *doctoc language introduction*

and then proceed from there to the formal specifications, i.e. the documents

1. *doctoc language syntax* and
2. *doctoc language command reference*.

to get a thorough understanding of the language.

Configuration

The doctoc export plugin recognizes the following configuration variables and changes its behaviour as they specify.

string *user*
: This standard configuration variable contains the name of the user running the process which invoked the export plugin. The plugin puts this information into the provenance comment at the beginning of the generated document.

string *file*
: This standard configuration variable contains the name of the file the table of contents came from. This variable may not be set or contain the empty string. The plugin puts this information, if defined, i.e. set and not the empty string, into the provenance comment at the beginning of the generated document.

boolean *newlines*
: If this flag is set the plugin will break the generated doctoc code across lines, with each markup command on a separate line.

  If this flag is not set (the default), the whole document will be written on a single line, with minimum spacing between all elements.

boolean *indented*
: If this flag is set the plugin will indent the markup commands according to the structure of tables of contents. To make this work this also implies that **newlines** is set. This effect is independent of the value for **aligned** however.

  If this flag is not set (the default), the output is formatted as per the value of **newlines**, and no indenting is done.

boolean *aligned*
: If this flag is set the generator ensures that the arguments for the **item** commands in a division are aligned vertically for a nice table effect. To make this work this also implies that **newlines** is set. This effect is independent of the value for **indented** however.

  If this flag is not set (the default), the output is formatted as per the values of **newlines** and **indented**, and no alignment is done.

Note that this plugin ignores the standard configuration variables **format**, and **map**, and their values.

Toc serialization format

Here we specify the format used by the doctools v2 packages to serialize tables of contents as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a table of contents may have

more than one regular serialization only exactly one of them will be *canonical*.

regular serialization

1. The serialization of any table of contents is a nested Tcl dictionary.
1. This dictionary holds a single key, **doctools::toc**, and its value. This value holds the contents of the table of contents.
2. The contents of the table of contents are a Tcl dictionary holding the title of the table of contents, a label, and its elements. The relevant keys and their values are

   title
   The value is a string containing the title of the table of contents.

   label
   The value is a string containing a label for the table of contents.

   items
   The value is a Tcl list holding the elements of the table, in the order they are to be shown.

   Each element is a Tcl list holding the type of the item, and its description, in this order. An alternative description would be that it is a Tcl dictionary holding a single key, the item type, mapped to the item description.

   The two legal item types and their descriptions are

   reference
   This item describes a single entry in the table of contents, referencing a single document. To this end its value is a Tcl dictionary containing an id for the referenced document, a label, and a longer textual description which can be associated with the entry. The relevant keys and their values are

   id
   The value is a string containing the id of the document associated with the entry.

   label
   The value is a string containing a label for this entry. This string also identifies the entry, and no two entries (references and divisions) in the containing list are allowed to have the same label.

   desc
   The value is a string containing a longer description for this entry.

```
**division**
: This item describes a group of entries in the table of
contents, inducing a hierarchy of entries. To this end its value
is a Tcl dictionary containing a label for the group, an
optional id to a document for the whole group, and the list of
entries in the group. The relevant keys and their values are

    **id**
    : The value is a string containing the id of the document
associated with the whole group. This key is optional.
```

```
        **label**
        : The value is a string containing a label for the group.
    This string also identifies the entry, and no two entries
    (references and divisions) in the containing list are allowed to
    have the same label.

        **items**
        : The value is a Tcl list holding the elements of the group,
    in the order they are to be shown. This list has the same
    structure as the value for the keyword **items** used to
    describe the whole table of contents, see above. This closes the
    recusrive definition of the structure, with divisions holding
    the same type of elements as the whole table of contents,
    including other divisions.
```

canonical serialization

The canonical serialization of a table of contents has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this table of contents.

1. The keys found in all the nested Tcl dictionaries are sorted in ascending dictionary order, as generated by Tcl's builtin command **lsort -increasing -dict**.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Text formatter plugin

Copyright


```
Copyright (c) 2009-2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


import_doctoc

Keywords: import_doctoc, deserialization, doctoc, doctools, import, table of contents, toc

Name

doctools::toc::import::doctoc - doctoc import plugin

Synopsis

package require Tcl 8.5 9
package require doctools::toc::import::doctoc ?**0.2.1**?
package require doctools::toc::parse
package require doctools::toc::structure
package require doctools::msgcat
package require doctools::tcl::parse
package require fileutil
package require logger
package require snit
package require struct::list
package require struct::set
package require struct::stack
package require struct::tree
package require treeql
import *string configuration*

Description

This package implements the doctools table of contents import plugin for the parsing of doctoc markup.

This is an internal package of doctools, for use by the higher level management packages handling tables of contents, especially **doctools::toc::import**, the import manager.

Using it from a regular interpreter is possible, however only with contortions, and is not recommended. The proper way to use this functionality is through the package **doctools::toc::import** and the import manager objects it provides.

API

The API provided by this package satisfies the specification of the doctoc import plugin API version 2.

import *string configuration*

This command takes the *string* and parses it as doctoc markup encoding a table of contents, in the context of the specified *configuration* (a dictionary). The result of the command is the canonical serialization of that table of contents, in the form specified in section Toc serialization format.

[doctoc] notation of tables of contents

The doctoc format for tables of contents, also called the *doctoc markup language*, is too large to be covered in single section. The interested reader should start with the document

1. *doctoc language introduction*

and then proceed from there to the formal specifications, i.e. the documents

1. *doctoc language syntax* and
2. *doctoc language command reference*.

to get a thorough understanding of the language.

Toc serialization format

Here we specify the format used by the doctools v2 packages to serialize tables of contents as

immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a table of contents may have more than one regular serialization only exactly one of them will be *canonical*.

regular serialization

1. The serialization of any table of contents is a nested Tcl dictionary.
1. This dictionary holds a single key, **doctools::toc**, and its value. This value holds the contents of the table of contents.
2. The contents of the table of contents are a Tcl dictionary holding the title of the table of contents, a label, and its elements. The relevant keys and their values are

   title
   The value is a string containing the title of the table of contents.

   label
   The value is a string containing a label for the table of contents.

   items
   The value is a Tcl list holding the elements of the table, in the order they are to be shown.

   Each element is a Tcl list holding the type of the item, and its description, in this order. An alternative description would be that it is a Tcl dictionary holding a single key, the item type, mapped to the item description.

   The two legal item types and their descriptions are

   reference
   This item describes a single entry in the table of contents, referencing a single document. To this end its value is a Tcl dictionary containing an id for the referenced document, a label, and a longer textual description which can be associated with the entry. The relevant keys and their values are

   id
   The value is a string containing the id of the document associated with the entry.

   label
   The value is a string containing a label for this entry. This string also identifies the entry, and no two entries (references and divisions) in the containing list are allowed to have the same label.

   desc
   The value is a string containing a longer description for this entry.

```
**division**
: This item describes a group of entries in the table of
contents, inducing a hierarchy of entries. To this end its value
is a Tcl dictionary containing a label for the group, an
optional id to a document for the whole group, and the list of
entries in the group. The relevant keys and their values are

    **id**
    : The value is a string containing the id of the document
```

```
    associated with the whole group. This key is optional.

        **label**
        : The value is a string containing a label for the group.
    This string also identifies the entry, and no two entries
    (references and divisions) in the containing list are allowed to
    have the same label.

        **items**
        : The value is a Tcl list holding the elements of the group,
    in the order they are to be shown. This list has the same
    structure as the value for the keyword **items** used to
    describe the whole table of contents, see above. This closes the
    recusrive definition of the structure, with divisions holding
    the same type of elements as the whole table of contents,
    including other divisions.
```

canonical serialization

The canonical serialization of a table of contents has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this table of contents.

1. The keys found in all the nested Tcl dictionaries are sorted in ascending dictionary order, as generated by Tcl's builtin command **lsort -increasing -dict**.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Text formatter plugin

Copyright


```
Copyright (c) 2009-2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


toc_container

Keywords: toc_container, HTML, TMML, conversion, doctoc markup, documentation, formatting, generation, json, latex, markup, nroff, parsing, plugin, reference, table, table of contents, tcler's wiki, text, wiki

Name

doctools::toc - Holding tables of contents

Synopsis

package require doctools::toc ?**2**?
package require Tcl 8.5 9
package require doctools::toc::structure
package require struct::tree
package require snit
::doctools::toc *objectName*
objectName method ?*arg arg ...*?
objectName **destroy**
objectName **+ reference** *id label docid desc*
objectName **+ division** *id label* ?*docid*?
objectName **remove** *id*
objectName **up** *id*
objectName **next** *id*
objectName **prev** *id*
objectName **child** *id label* ?*...*?
objectName **element** ?*...*?
objectName **children** *id*
objectName **type** *id*
objectName **full-label** *id*
objectName **elabel** *id* ?*newlabel*?
objectName **description** *id* ?*newdesc*?
objectName **document** *id* ?*newdocid*?
objectName **title**
objectName **title** *text*
objectName **label**
objectName **label** *text*
objectName **importer**
objectName **importer** *object*
objectName **exporter**
objectName **exporter** *object*
objectName **deserialize =** *data* ?*format*?
objectName **deserialize +=** *data* ?*format*?
objectName **serialize** ?*format*?

Description

This package provides a class to contain and programmatically manipulate tables of contents.

This is one of the three public pillars the management of tables of contents resides on. The other two pillars are

1. *Exporting tables of contents*, and
2. *Importing tables of contents*

For information about the Concepts of tables of contents, and their parts, see the same-named

section. For information about the data structure which is used to encode tables of contents as values see the section Toc serialization format. This is the only format directly known to this class. Conversions from and to any other format are handled by export and import manager objects. These may be attached to a container, but do not have to be, it is merely a convenience.

Concepts

1. A *table of contents* consists of a (possibly empty) list of *elements*.
2. Each element in the list is identified by its label.
3. Each element is either a *reference*, or a *division*.
4. Each reference has an associated document, identified by a symbolic id, and a textual description.
5. Each division may have an associated document, identified by a symbolic id.
6. Each division consists consists of a (possibly empty) list of *elements*, with each element following the rules as specified in item 2 and above.

A few notes

1. The above rules span up a tree of elements, with references as the leaf nodes, and divisions as the inner nodes, and each element representing an entry in the whole table of contents.
2. The identifying labels of any element E are unique within their division (or toc), and the full label of any element E is the list of labels for all nodes on the unique path from the root of the tree to E, including E.

API

Package commands

::doctools::toc *objectName*

This command creates a new container object with an associated Tcl command whose name is *objectName*. This *object* command is explained in full detail in the sections Object command and Object methods. The object command will be created under the current namespace if the *objectName* is not fully qualified, and in the specified namespace otherwise.

Object command

All objects created by the **::doctools::toc** command have the following general form:

objectName method ?*arg arg ...*?

The method **method** and its *arg*'uments determine the exact behavior of the command. See section Object methods for the detailed specifications.

Object methods

***objectName* destroy**

This method destroys the object it is invoked for.

objectName* + reference *id label docid desc

This method adds a new reference element to the table of contents, under the element specified via its handle *id*. This parent element has to be a division element, or the root. An error is thrown otherwise. The new element will be externally identified by its *label*, which has to be be unique within the parent element. An error is thrown otherwise.

As a reference element it will refer to a document identified by the symbolic *docid*. This reference must not be the empty string, an error is thrown otherwise. Beyond the label the element also has a longer descriptive string, supplied via *desc*.

The result of the method is the handle (id) of the new element.

***objectName* + division *id label* ?*docid*?**

This method adds a new division element to the table of contents, under the element specified via its handle *id*. This parent element has to be a division element, or the root. An error is thrown otherwise. The new element will be externally identified by its *label*, which has to be be unique within the parent element. An error is thrown otherwise.

As a division element it is can refer to a document, identified by the symbolic *docid*, but may choose not to.

The result of the method is the handle (id) of the new element.

objectName* remove *id

This method removes the element identified by the handle *id* from the table of contents. If the element is a division all of its children, if any, are removed as well. The root element/division of the table of contents cannot be removed however, only its children.

The result of the method is the empty string.

objectName* up *id

This method returns the handle of the parent for the element identified by its handle *id*, or the empty string if *id* referred to the root element.

objectName* next *id

This method returns the handle of the right sibling for the element identified by its handle *id*, or the handle of the parent if the element has no right sibling, or the empty string if *id* referred to the root element.

objectName* prev *id

This method returns the handle of the left sibling for the element identified by its handle *id*, or the handle of the parent if the element has no left sibling, or the empty string if *id* referred to the root element.

***objectName* child *id label* ?*...*?**

This method returns the handle of a child of the element identified by its handle *id*. The child itself is identified by a series of labels.

***objectName* element ?*...*?**

This method returns the handle of the element identified by a series of labels, starting from the root of the table of contents. The series of labels is allowed to be empty, in which case the handle of the root element is returned.

objectName* children *id

This method returns a list containing the handles of all children of the element identified by the handle *id*, from first to last, in that order.

objectName* type *id

This method returns the type of the element, either **reference**, or **division**.

objectName* full-label *id

This method is the complement of the method **element**, converting the handle *id* of an element into a list of labels full identifying the element within the whole table of contents.

***objectName* elabel *id* ?*newlabel*?**

This method queries and/or changes the label of the element identified by the handle *id*. If the argument *newlabel* is present then the label is changed to that value. Regardless of this, the result of the method is the current value of the label.

If the label is changed the new label has to be unique within the containing division, or an error is thrown.

Further, of the *id* refers to the root element of the table of contents, then using this method is equivalent to using the method *label*, i.e. it is accessing the global label for the whole table.

***objectName* description *id* ?*newdesc*?**

This method queries and/or changes the description of the element identified by the handle *id* . If the argument *newdesc* is present then the description is changed to that value. Regardless of this, the result of the method is the current value of the description.

The element this method operates on has to be a reference element, or an error will be thrown.

***objectName* document *id* ?*newdocid*?**

This method queries and/or changes the document reference of the element identified by the handle *id*. If the argument *newdocid* is present then the description is changed to that value. Regardless of this, the result of the method is the current value of the document reference.

Setting the reference to the empty string means unsetting it, and is allowed only for division elements. Conversely, if the result is the empty string then the element has no document reference, and this can happen only for division elements.

***objectName* title**

Returns the currently defined title of the table of contents.

objectName* title *text

Sets the title of the table of contents to *text*, and returns it as the result of the command.

***objectName* label**

Returns the currently defined label of the table of contents.

objectName* label *text

Sets the label of the table of contents to *text*, and returns it as the result of the command.

***objectName* importer**

Returns the import manager object currently attached to the container, if any.

objectName* importer *object

Attaches the *object* as import manager to the container, and returns it as the result of the command. Note that the *object* is *not* put into ownership of the container. I.e., destruction of the container will *not* destroy the *object*.

It is expected that *object* provides a method named **import text** which takes a text and a format name, and returns the canonical serialization of the table of contents contained in the text, assuming the given format.

***objectName* exporter**
Returns the export manager object currently attached to the container, if any.

objectName* exporter *object
Attaches the *object* as export manager to the container, and returns it as the result of the command. Note that the *object* is *not* put into ownership of the container. I.e., destruction of the container will *not* destroy the *object*.

It is expected that *object* provides a method named **export object** which takes the container and a format name, and returns a text encoding table of contents stored in the container, in the given format. It is further expected that the *object* will use the container's method **serialize** to obtain the serialization of the table of contents from which to generate the text.

***objectName* deserialize = *data* ?*format*?**
This method replaces the contents of the table object with the table contained in the *data*. If no *format* was specified it is assumed to be the regular serialization of a table of contents.

Otherwise the object will use the attached import manager to convert the data from the specified format to a serialization it can handle. In that case an error will be thrown if the container has no import manager attached to it.

The result of the method is the empty string.

***objectName* deserialize += *data* ?*format*?**
This method behaves like **deserialize =** in its essentials, except that it merges the table of contents in the *data* to its contents instead of replacing it. The method will throw an error if merging is not possible, i.e. would produce an invalid table. The existing content is left unchanged in that case.

The result of the method is the empty string.

***objectName* serialize ?*format*?**
This method returns the table of contents contained in the object. If no *format* is not specified the returned result is the canonical serialization of its contents.

Otherwise the object will use the attached export manager to convert the data to the specified format. In that case an error will be thrown if the container has no export manager attached to it.

Toc serialization format

Here we specify the format used by the doctools v2 packages to serialize tables of contents as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a table of contents may have more than one regular serialization only exactly one of them will be *canonical*.

regular serialization

1. The serialization of any table of contents is a nested Tcl dictionary.
1. This dictionary holds a single key, **doctools::toc**, and its value. This value holds the contents of the table of contents.
2. The contents of the table of contents are a Tcl dictionary holding the title of the table of

contents, a label, and its elements. The relevant keys and their values are

title
: The value is a string containing the title of the table of contents.

label
: The value is a string containing a label for the table of contents.

items
: The value is a Tcl list holding the elements of the table, in the order they are to be shown.

  Each element is a Tcl list holding the type of the item, and its description, in this order. An alternative description would be that it is a Tcl dictionary holding a single key, the item type, mapped to the item description.

  The two legal item types and their descriptions are

  reference
  : This item describes a single entry in the table of contents, referencing a single document. To this end its value is a Tcl dictionary containing an id for the referenced document, a label, and a longer textual description which can be associated with the entry. The relevant keys and their values are

    id
    : The value is a string containing the id of the document associated with the entry.

    label
    : The value is a string containing a label for this entry. This string also identifies the entry, and no two entries (references and divisions) in the containing list are allowed to have the same label.

    desc
    : The value is a string containing a longer description for this entry.

```
**division**
: This item describes a group of entries in the table of
contents, inducing a hierarchy of entries. To this end its value
is a Tcl dictionary containing a label for the group, an
optional id to a document for the whole group, and the list of
entries in the group. The relevant keys and their values are

    **id**
    : The value is a string containing the id of the document
associated with the whole group. This key is optional.

    **label**
    : The value is a string containing a label for the group.
This string also identifies the entry, and no two entries
(references and divisions) in the containing list are allowed to
have the same label.
```

```
        **items**
        : The value is a Tcl list holding the elements of the group,
    in the order they are to be shown. This list has the same
    structure as the value for the keyword **items** used to
    describe the whole table of contents, see above. This closes the
    recusrive definition of the structure, with divisions holding
    the same type of elements as the whole table of contents,
    including other divisions.
```

canonical serialization

The canonical serialization of a table of contents has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this table of contents.

1. The keys found in all the nested Tcl dictionaries are sorted in ascending dictionary order, as generated by Tcl's builtin command **lsort -increasing -dict**.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Documentation tools

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

toc_export

Keywords: toc_export, HTML, conversion, doctoc, documentation, export, formatting, generation, json, manpage, markup, nroff, plugin, reference, table, table of contents, tcler's wiki, text, url, wiki

Name

doctools::toc::export - Exporting tables of contents

Synopsis

package require doctools::toc::export ?**0.2.2**?
package require Tcl 8.5 9
package require struct::map
package require doctools::toc::structure

package require snit
package require pluginmgr
::doctools::toc::export *objectName*
objectName method ?*arg arg ...*?
objectName **destroy**
objectName **export serial** *serial* ?*format*?
objectName **export object** *object* ?*format*?
objectName **config names**
objectName **config get**
objectName **config set** *name* ?*value*?
objectName **config unset** *pattern...*
export *serial configuration*

Description

This package provides a class to manage the plugins for the export of tables of contents to other formats, i.e. their conversion to, for example *doctoc*, *HTML*, etc.

This is one of the three public pillars the management of tables of contents resides on. The other two pillars are

1. *Importing tables of contents*, and
2. *Holding tables of contents*

For information about the Concepts of tables of contents, and their parts, see the same-named section. For information about the data structure which is the major input to the manager objects provided by this package see the section Toc serialization format.

The plugin system of our class is based on the package **pluginmgr**, and configured to look for plugins using

1. the environment variable **DOCTOOLS_TOC_EXPORT_PLUGINS**,
2. the environment variable **DOCTOOLS_TOC_PLUGINS**,
3. the environment variable **DOCTOOLS_PLUGINS**,
4. the path "*~/.doctools/toc/export/plugin*"
5. the path "*~/.doctools/toc/plugin*"
6. the path "*~/.doctools/plugin*"
7. the path "*~/.doctools/toc/export/plugins*"
8. the path "*~/.doctools/toc/plugins*"
9. the path "*~/.doctools/plugins*"
10. the registry entry "HKEY_CURRENT_USER\SOFTWARE\DOCTOOLS\TOC\EXPORT\PLUGINS"
11. the registry entry "HKEY_CURRENT_USER\SOFTWARE\DOCTOOLS\TOC\PLUGINS"
12. the registry entry "HKEY_CURRENT_USER\SOFTWARE\DOCTOOLS\PLUGINS"

The last three are used only when the package is run on a machine using Windows(tm) operating system.

The whole system is delivered with six predefined export plugins, namely

doctoc
: See *doctoc export plugin* for details.

html
: See *html export plugin* for details.

json
: See *json export plugin* for details.

nroff
: See *nroff export plugin* for details.

text
: See *text export plugin* for details.

wiki
: See *wiki export plugin* for details.

Readers wishing to write their own export plugin for some format, i.e. *plugin writer*s reading and understanding the section containing the Export plugin API v2 reference is an absolute necessity, as it specifies the interaction between this package and its plugins in detail.

Concepts

1. A *table of contents* consists of a (possibly empty) list of *elements*.
2. Each element in the list is identified by its label.
3. Each element is either a *reference*, or a *division*.
4. Each reference has an associated document, identified by a symbolic id, and a textual description.
5. Each division may have an associated document, identified by a symbolic id.
6. Each division consists consists of a (possibly empty) list of *elements*, with each element following the rules as specified in item 2 and above.

A few notes

1. The above rules span up a tree of elements, with references as the leaf nodes, and divisions as the inner nodes, and each element representing an entry in the whole table of contents.
2. The identifying labels of any element E are unique within their division (or toc), and the full label of any element E is the list of labels for all nodes on the unique path from the root of the tree to E, including E.

API

Package commands

::doctools::toc::export *objectName*
: This command creates a new export manager object with an associated Tcl command whose name is *objectName*. This *object* command is explained in full detail in the sections Object command and Object methods. The object command will be created under the current namespace if the *objectName* is not fully qualified, and in the specified namespace otherwise.

Object command

All objects created by the **::doctools::toc::export** command have the following general form:

objectName method ?*arg arg ...*?
: The method **method** and its *arg*'uments determine the exact behavior of the command. See section Object methods for the detailed specifications.

Object methods

***objectName* destroy**
: This method destroys the object it is invoked for.

***objectName* export serial *serial* ?*format*?**
This method takes the canonical serialization of a table of contents stored in *serial* and converts it to the specified *format*, using the export plugin for the format. An error is thrown if no plugin could be found for the format. The string generated by the conversion process is returned as the result of this method.

If no format is specified the method defaults to **doctoc**.

The specification of what a *canonical* serialization is can be found in the section Toc serialization format.

The plugin has to conform to the interface specified in section Export plugin API v2 reference.

***objectName* export object *object* ?*format*?**
This method is a convenient wrapper around the **export serial** method described by the previous item. It expects that *object* is an object command supporting a **serialize** method returning the canonical serialization of a table of contents. It invokes that method, feeds the result into **export serial** and returns the resulting string as its own result.

***objectName* config names**
This method returns a list containing the names of all configuration variables currently known to the object.

***objectName* config get**
This method returns a dictionary containing the names and values of all configuration variables currently known to the object.

***objectName* config set *name* ?*value*?**
This method sets the configuration variable *name* to the specified *value* and returns the new value of the variable.

If no value is specified it simply returns the current value, without changing it.

Note that while the user can set the predefined configuration variables **user** and **format** doing so will have no effect, these values will be internally overridden when invoking an import plugin.

***objectName* config unset *pattern*...**
This method unsets all configuration variables matching the specified glob *pattern*s. If no pattern is specified it will unset all currently defined configuration variables.

Export plugin API v2 reference

Plugins are what this package uses to manage the support for any output format beyond the Toc serialization format. Here we specify the API the objects created by this package use to interact with their plugins.

A plugin for this package has to follow the rules listed below:

1. A plugin is a package.
2. The name of a plugin package has the form doctools::toc::export::**FOO**, where **FOO** is the name of the format the plugin will generate output for. This name is also the argument to provide to the various **export** methods of export manager objects to get a string encoding a

table of contents in that format.

3. The plugin can expect that the package **doctools::toc::export::plugin** is present, as indicator that it was invoked from a genuine plugin manager.
4. A plugin has to provide one command, with the signature shown below.

   export *serial configuration*

   Whenever an export manager of **doctools::toc** has to generate output for a table of contents it will invoke this command.

   string *serial*

   This argument will contain the *canonical* serialization of the table of contents for which to generate the output. The specification of what a *canonical* serialization is can be found in the section Toc serialization format.

   dictionary *configuration*

   This argument will contain the current configuration to apply to the generation, as a dictionary mapping from variable names to values.

   The following configuration variables have a predefined meaning all plugins have to obey, although they can ignore this information at their discretion. Any other other configuration variables recognized by a plugin will be described in the manpage for that plugin.

   user

   This variable is expected to contain the name of the user owning the process invoking the plugin.

   format

   This variable is expected to contain the name of the format whose plugin is invoked.

   file

   This variable, if defined by the user of the table object is expected to contain the name of the input file for which the plugin is generating its output for.

   map

   This variable, if defined by the user of the table object is expected to contain a dictionary mapping from symbolic document ids used in the table entries to actual paths (or urls). A plugin has to be able to handle the possibility that a document id is without entry in this mapping.

1. A single usage cycle of a plugin consists of the invokations of the command **export**. This call has to leave the plugin in a state where another usage cycle can be run without problems.

Toc serialization format

Here we specify the format used by the doctools v2 packages to serialize tables of contents as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a table of contents may have more than one regular serialization only exactly one of them will be *canonical*.

regular serialization

1. The serialization of any table of contents is a nested Tcl dictionary.
1. This dictionary holds a single key, **doctools::toc**, and its value. This value holds the contents of the table of contents.

2. The contents of the table of contents are a Tcl dictionary holding the title of the table of contents, a label, and its elements. The relevant keys and their values are

   title
   The value is a string containing the title of the table of contents.

   label
   The value is a string containing a label for the table of contents.

   items
   The value is a Tcl list holding the elements of the table, in the order they are to be shown.

   Each element is a Tcl list holding the type of the item, and its description, in this order. An alternative description would be that it is a Tcl dictionary holding a single key, the item type, mapped to the item description.

   The two legal item types and their descriptions are

   reference
   This item describes a single entry in the table of contents, referencing a single document. To this end its value is a Tcl dictionary containing an id for the referenced document, a label, and a longer textual description which can be associated with the entry. The relevant keys and their values are

   id
   The value is a string containing the id of the document associated with the entry.

   label
   The value is a string containing a label for this entry. This string also identifies the entry, and no two entries (references and divisions) in the containing list are allowed to have the same label.

   desc
   The value is a string containing a longer description for this entry.

```
**division**
: This item describes a group of entries in the table of
contents, inducing a hierarchy of entries. To this end its value
is a Tcl dictionary containing a label for the group, an
optional id to a document for the whole group, and the list of
entries in the group. The relevant keys and their values are

    **id**
    : The value is a string containing the id of the document
associated with the whole group. This key is optional.

    **label**
    : The value is a string containing a label for the group.
This string also identifies the entry, and no two entries
(references and divisions) in the containing list are allowed to
have the same label.
```

```
    **items**
    : The value is a Tcl list holding the elements of the group,
in the order they are to be shown. This list has the same
structure as the value for the keyword **items** used to
describe the whole table of contents, see above. This closes the
recusrive definition of the structure, with divisions holding
the same type of elements as the whole table of contents,
including other divisions.
```

canonical serialization

The canonical serialization of a table of contents has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this table of contents.

1. The keys found in all the nested Tcl dictionaries are sorted in ascending dictionary order, as generated by Tcl's builtin command **lsort -increasing -dict**.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Documentation tools

Copyright


```
Copyright (c) 2009-2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


toc_export_html

Keywords: toc_export_html, HTML, doctools, export, serialization, table of contents, toc

Name

doctools::toc::export::html - HTML export plugin

Synopsis

package require Tcl 8.5 9
package require doctools::toc::export::html ?**0.2**?
package require doctools::text
package require doctools::html
package require doctools::html::cssdefaults

export *serial configuration*

Description

This package implements the doctools table of contents export plugin for the generation of HTML markup.

This is an internal package of doctools, for use by the higher level management packages handling tables of contents, especially **doctools::toc::export**, the export manager.

Using it from a regular interpreter is possible, however only with contortions, and is not recommended. The proper way to use this functionality is through the package **doctools::toc::export** and the export manager objects it provides.

API

The API provided by this package satisfies the specification of the doctoc export plugin API version 2.

export *serial configuration*

This command takes the canonical serialization of a table of contents, as specified in section Toc serialization format, and contained in *serial*, the *configuration*, a dictionary, and generates HTML markup encoding the table. The created string is then returned as the result of the command.

Configuration

The html export plugin recognizes the following configuration variables and changes its behaviour as they specify.

string *user*

This standard configuration variable contains the name of the user running the process which invoked the export plugin. The plugin puts this information into the provenance comment at the beginning of the generated document.

string *file*

This standard configuration variable contains the name of the file the table of contents came from. This variable may not be set or contain the empty string. The plugin puts this information, if defined, i.e. set and not the empty string, into the provenance comment at the beginning of the generated document.

dictionary *map*

This standard configuration variable contains a dictionary mapping from the (symbolic) document ids in reference entries to the actual filenames and/or urls to be used in the output.

Document ids without a mapping are used unchanged.

boolean *newlines*

If this flag is set the plugin will break the generated html code across lines, with each markup command on a separate line.

If this flag is not set (the default), the whole document will be written on a single line, with minimum spacing between all elements.

boolean *indented*

If this flag is set the plugin will indent the markup commands according to the structure of

indices. To make this work this also implies that **newlines** is set.

If this flag is not set (the default), the output is formatted as per the value of **newlines**, and no indenting is done.

string *meta*
This variable is meant to hold a fragment of HTML (default: empty). The fragment it contains will be inserted into the generated output in the section of the document, just after the tag.

string *header*
This variable is meant to hold a fragment of HTML (default: empty). The fragment it contains will be inserted into the generated output just after the title tag in the body of the document, in the class.header 'ision.

string *footer*
This variable is meant to hold a fragment of HTML (default: empty). The fragment it contains will be inserted into the generated output just before the tag, in the class.footer 'ision.

dictionary *rid*
The value of this variable (default: empty) maps references to the identifiers to use as their anchor names. Each reference **FOO** not found in the dictionary uses **REF-FOO** as anchor, i.e. itself prefixed with the string **REF-**.

string *sepline*
The value of this variable is the string to use for the separator comments inserted into the output when the outpout is broken across lines and/or indented. The default string consists of 60 dashes.

string *class.main*
This variable contains the class name for the main 'ivision of the generated document. The default is **doctools**.

string *class.header*
This variable contains the class name for the header 'ision of the generated document. The default is **toc-header**. This division contains the document title, the user specified **header**, if any, and a visible separator line.

string *class.title*
This variable contains the class name for the tag enclosing the document title. The default is **toc-title**.

string *class.navsep*
This variable contains the class name for the separators in the header and footer sections of the generated document. The default is **toc-navsep**.

string *class.contents*
This variable contains the class name for the XXXXX holding the keywords and their references in the generated document. The default is **toc-contents**.

string *class.ref*
This variable contains the class name for the table elements which are references to other documents. The default is **toc-ref**.

string *class.div*
This variable contains the class name for the table elements which are divisions. The default is **toc-div**.

string *class.footer*
: This variable contains the class name for the footer 'ision of the generated document. The default is **toc-footer**. This division contains a browser-visible separator line and the user specified **footer**, if any.

Note that this plugin ignores the standard configuration variable **format**, and its value.

Toc serialization format

Here we specify the format used by the doctools v2 packages to serialize tables of contents as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a table of contents may have more than one regular serialization only exactly one of them will be *canonical*.

regular serialization

1. The serialization of any table of contents is a nested Tcl dictionary.
1. This dictionary holds a single key, **doctools::toc**, and its value. This value holds the contents of the table of contents.
2. The contents of the table of contents are a Tcl dictionary holding the title of the table of contents, a label, and its elements. The relevant keys and their values are

   title
   : The value is a string containing the title of the table of contents.

   label
   : The value is a string containing a label for the table of contents.

   items
   : The value is a Tcl list holding the elements of the table, in the order they are to be shown.

     Each element is a Tcl list holding the type of the item, and its description, in this order. An alternative description would be that it is a Tcl dictionary holding a single key, the item type, mapped to the item description.

     The two legal item types and their descriptions are

     reference
     : This item describes a single entry in the table of contents, referencing a single document. To this end its value is a Tcl dictionary containing an id for the referenced document, a label, and a longer textual description which can be associated with the entry. The relevant keys and their values are

       id
       : The value is a string containing the id of the document associated with the entry.

       label
       : The value is a string containing a label for this entry. This string also identifies the entry, and no two entries (references and divisions) in the containing list are allowed to have the same label.

       desc
       : The value is a string containing a longer description for this entry.

```
    **division**
    : This item describes a group of entries in the table of
    contents, inducing a hierarchy of entries. To this end its value
    is a Tcl dictionary containing a label for the group, an
    optional id to a document for the whole group, and the list of
    entries in the group. The relevant keys and their values are

        **id**
        : The value is a string containing the id of the document
    associated with the whole group. This key is optional.

        **label**
        : The value is a string containing a label for the group.
    This string also identifies the entry, and no two entries
    (references and divisions) in the containing list are allowed to
    have the same label.

        **items**
        : The value is a Tcl list holding the elements of the group,
    in the order they are to be shown. This list has the same
    structure as the value for the keyword **items** used to
    describe the whole table of contents, see above. This closes the
    recusrive definition of the structure, with divisions holding
    the same type of elements as the whole table of contents,
    including other divisions.
```

canonical serialization

The canonical serialization of a table of contents has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this table of contents.

1. The keys found in all the nested Tcl dictionaries are sorted in ascending dictionary order, as generated by Tcl's builtin command **lsort -increasing -dict**.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Text formatter plugin

Copyright


```
Copyright (c) 2009-2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


toc_export_json

Keywords: toc_export_json, JSON, doctools, export, serialization, table of contents, toc

Name

doctools::toc::export::json - JSON export plugin

Synopsis

package require Tcl 8.5 9
package require doctools::toc::export::json ?**0.2**?
package require textutil::adjust
export *serial configuration*

Description

This package implements the doctools table of contents export plugin for the generation of JSON markup.

This is an internal package of doctools, for use by the higher level management packages handling tables of contents, especially **doctools::toc::export**, the export manager.

Using it from a regular interpreter is possible, however only with contortions, and is not recommended. The proper way to use this functionality is through the package **doctools::toc::export** and the export manager objects it provides.

API

The API provided by this package satisfies the specification of the doctoc export plugin API version 2.

export *serial configuration*
: This command takes the canonical serialization of a table of contents, as specified in section Toc serialization format, and contained in *serial*, the *configuration*, a dictionary, and generates JSON markup encoding the table. The created string is then returned as the result of the command.

JSON notation of tables of contents

The JSON format used for tables of contents is a direct translation of the Toc serialization format, mapping Tcl dictionaries as JSON objects and Tcl lists as JSON arrays. For example, the Tcl serialization

```
doctools::toc {
    items {
        {reference {
        desc {DocTools - Tables of Contents}
         id introduction.man
         label doctools::toc::introduction
```

```
    }}
    {division {
         id processing.man
         items {
             {reference {
             desc {doctoc serialization utilities}
             id structure.man
             label doctools::toc::structure
         }}
         {reference {
             desc {Parsing text in doctoc format}
             id parse.man
             label doctools::toc::parse
         }}
         }
             label Processing
        }}
    }
    label {Table of Contents}
    title TOC
}
```

is equivalent to the JSON string

```
{
    "doctools::toc" : {
        "items" : [{
            "reference" : {
                "desc"  : "DocTools - Tables of Contents",
                "id"    : "introduction.man",
                "label" : "doctools::toc::introduction"
            }
        },{
            "division" : {
                "id"    : "processing.man",
                "items" : [{
                    "reference" : {
                        "desc"  : "doctoc serialization utilities",
                        "id"    : "structure.man",
                        "label" : "doctools::toc::structure"
                    }
                },{
                    "reference" : {
                        "desc"  : "Parsing text in doctoc format",
                        "id"    : "parse.man",
                        "label" : "doctools::toc::parse"
                    }
```

```
                }],
                "label" : "Processing"
            }
        }],
        "label" : "Table of Contents",
        "title" : "TOC"
    }
}
```

Configuration

The JSON export plugin recognizes the following configuration variables and changes its behaviour as they specify.

boolean *indented*
: If this flag is set the plugin will break the generated JSON code across lines and indent it according to its inner structure, with each key of a dictionary on a separate line.

  If this flag is not set (the default), the whole JSON object will be written on a single line, with minimum spacing between all elements.

boolean *aligned*
: If this flag is set the generator ensures that the values for the keys in a dictionary are vertically aligned with each other, for a nice table effect. To make this work this also implies that **indented** is set.

  If this flag is not set (the default), the output is formatted as per the value of **indented**, without trying to align the values for dictionary keys.

Note that this plugin ignores the standard configuration variables **user**, **format**, **file**, and **map** and their values.

Toc serialization format

Here we specify the format used by the doctools v2 packages to serialize tables of contents as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a table of contents may have more than one regular serialization only exactly one of them will be *canonical*.

regular serialization

1. The serialization of any table of contents is a nested Tcl dictionary.
1. This dictionary holds a single key, **doctools::toc**, and its value. This value holds the contents of the table of contents.
2. The contents of the table of contents are a Tcl dictionary holding the title of the table of contents, a label, and its elements. The relevant keys and their values are

   title
   : The value is a string containing the title of the table of contents.

   label
   : The value is a string containing a label for the table of contents.

   items

The value is a Tcl list holding the elements of the table, in the order they are to be shown.

Each element is a Tcl list holding the type of the item, and its description, in this order. An alternative description would be that it is a Tcl dictionary holding a single key, the item type, mapped to the item description.

The two legal item types and their descriptions are

reference

This item describes a single entry in the table of contents, referencing a single document. To this end its value is a Tcl dictionary containing an id for the referenced document, a label, and a longer textual description which can be associated with the entry. The relevant keys and their values are

id

The value is a string containing the id of the document associated with the entry.

label

The value is a string containing a label for this entry. This string also identifies the entry, and no two entries (references and divisions) in the containing list are allowed to have the same label.

desc

The value is a string containing a longer description for this entry.

```
    **division**
    : This item describes a group of entries in the table of
    contents, inducing a hierarchy of entries. To this end its value
    is a Tcl dictionary containing a label for the group, an
    optional id to a document for the whole group, and the list of
    entries in the group. The relevant keys and their values are

        **id**
        : The value is a string containing the id of the document
    associated with the whole group. This key is optional.

        **label**
        : The value is a string containing a label for the group.
    This string also identifies the entry, and no two entries
    (references and divisions) in the containing list are allowed to
    have the same label.

        **items**
        : The value is a Tcl list holding the elements of the group,
    in the order they are to be shown. This list has the same
    structure as the value for the keyword **items** used to
    describe the whole table of contents, see above. This closes the
    recusrive definition of the structure, with divisions holding
    the same type of elements as the whole table of contents,
    including other divisions.
```

canonical serialization

The canonical serialization of a table of contents has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this table of contents.

1. The keys found in all the nested Tcl dictionaries are sorted in ascending dictionary order, as generated by Tcl's builtin command **lsort -increasing -dict**.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Text formatter plugin

Copyright

```
Copyright (c) 2009-2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

toc_export_nroff

Keywords: toc_export_nroff, doctools, export, nroff, serialization, table of contents, toc

Name

doctools::toc::export::nroff - nroff export plugin

Synopsis

package require Tcl 8.5 9
package require doctools::toc::export::nroff ?**0.3**?
package require doctools::text
package require doctools::nroff::man_macros
export *serial configuration*

Description

This package implements the doctools table of contents export plugin for the generation of nroff markup.

This is an internal package of doctools, for use by the higher level management packages handling tables of contents, especially **doctools::toc::export**, the export manager.

Using it from a regular interpreter is possible, however only with contortions, and is not recommended. The proper way to use this functionality is through the package **doctools::toc::export** and the export manager objects it provides.

API

The API provided by this package satisfies the specification of the doctoc export plugin API version 2.

export *serial configuration*
: This command takes the canonical serialization of a table of contents, as specified in section Toc serialization format, and contained in *serial*, the *configuration*, a dictionary, and generates nroff markup encoding the table. The created string is then returned as the result of the command.

Configuration

The nroff export plugin recognizes the following configuration variables and changes its behaviour as they specify.

string *user*
: This standard configuration variable contains the name of the user running the process which invoked the export plugin. The plugin puts this information into the provenance comment at the beginning of the generated document.

string *file*
: This standard configuration variable contains the name of the file the table of contents came from. This variable may not be set or contain the empty string. The plugin puts this

information, if defined, i.e. set and not the empty string, into the provenance comment at the beginning of the generated document.

boolean *inline*

If this flag is set (default) the plugin will place the definitions of the man macro set directly into the output.

If this flag is not set, the plugin will place a reference to the definitions of the man macro set into the output, but not the macro definitions themselves.

Note that this plugin ignores the standard configuration variables **format**, and **map**, and their values.

Toc serialization format

Here we specify the format used by the doctools v2 packages to serialize tables of contents as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a table of contents may have more than one regular serialization only exactly one of them will be *canonical*.

regular serialization

1. The serialization of any table of contents is a nested Tcl dictionary.
1. This dictionary holds a single key, **doctools::toc**, and its value. This value holds the contents of the table of contents.
2. The contents of the table of contents are a Tcl dictionary holding the title of the table of contents, a label, and its elements. The relevant keys and their values are

   title

   The value is a string containing the title of the table of contents.

   label

   The value is a string containing a label for the table of contents.

   items

   The value is a Tcl list holding the elements of the table, in the order they are to be shown.

   Each element is a Tcl list holding the type of the item, and its description, in this order. An alternative description would be that it is a Tcl dictionary holding a single key, the item type, mapped to the item description.

   The two legal item types and their descriptions are

   reference

   This item describes a single entry in the table of contents, referencing a single document. To this end its value is a Tcl dictionary containing an id for the referenced document, a label, and a longer textual description which can be associated with the entry. The relevant keys and their values are

   id

   The value is a string containing the id of the document associated with the entry.

   label

   The value is a string containing a label for this entry. This string also identifies the entry, and no two entries (references and divisions) in the containing list are allowed to have the same label.

desc
The value is a string containing a longer description for this entry.

```
    **division**
    : This item describes a group of entries in the table of
    contents, inducing a hierarchy of entries. To this end its value
    is a Tcl dictionary containing a label for the group, an
    optional id to a document for the whole group, and the list of
    entries in the group. The relevant keys and their values are

        **id**
        : The value is a string containing the id of the document
    associated with the whole group. This key is optional.

        **label**
        : The value is a string containing a label for the group.
    This string also identifies the entry, and no two entries
    (references and divisions) in the containing list are allowed to
    have the same label.

        **items**
        : The value is a Tcl list holding the elements of the group,
    in the order they are to be shown. This list has the same
    structure as the value for the keyword **items** used to
    describe the whole table of contents, see above. This closes the
    recusrive definition of the structure, with divisions holding
    the same type of elements as the whole table of contents,
    including other divisions.
```

canonical serialization
The canonical serialization of a table of contents has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this table of contents.

1. The keys found in all the nested Tcl dictionaries are sorted in ascending dictionary order, as generated by Tcl's builtin command **lsort -increasing -dict**.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Text formatter plugin

Copyright

```
Copyright (c) 2009-2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


toc_export_text
Keywords: toc_export_text, doctools, export, plain text, serialization, table of contents, toc

Name
doctools::toc::export::text - plain text export plugin

Synopsis
package require Tcl 8.5 9
package require doctools::toc::export::text ?**0.2**?
package require doctools::text
export *serial configuration*

Description
This package implements the doctools table of contents export plugin for the generation of plain text markup.

This is an internal package of doctools, for use by the higher level management packages handling tables of contents, especially **doctools::toc::export**, the export manager.

Using it from a regular interpreter is possible, however only with contortions, and is not recommended. The proper way to use this functionality is through the package **doctools::toc::export** and the export manager objects it provides.

API
The API provided by this package satisfies the specification of the doctoc export plugin API version 2.

export *serial configuration*
This command takes the canonical serialization of a table of contents, as specified in section Toc serialization format, and contained in *serial*, the *configuration*, a dictionary, and generates plain text markup encoding the table. The created string is then returned as the result of the command.

Configuration
The text export plugin recognizes the following configuration variables and changes its behaviour as they specify.

dictionary *map*
This standard configuration variable contains a dictionary mapping from the (symbolic) document ids in reference entries to the actual filenames and/or urls to be used in the output.

Document ids without a mapping are used unchanged.

Note that this plugin ignores the standard configuration variables **user**, **file**, and **format**, and their values.

Toc serialization format

Here we specify the format used by the doctools v2 packages to serialize tables of contents as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a table of contents may have more than one regular serialization only exactly one of them will be *canonical*.

regular serialization

1. The serialization of any table of contents is a nested Tcl dictionary.
1. This dictionary holds a single key, **doctools::toc**, and its value. This value holds the contents of the table of contents.
2. The contents of the table of contents are a Tcl dictionary holding the title of the table of contents, a label, and its elements. The relevant keys and their values are

   title
   : The value is a string containing the title of the table of contents.

   label
   : The value is a string containing a label for the table of contents.

   items
   : The value is a Tcl list holding the elements of the table, in the order they are to be shown.

     Each element is a Tcl list holding the type of the item, and its description, in this order. An alternative description would be that it is a Tcl dictionary holding a single key, the item type, mapped to the item description.

     The two legal item types and their descriptions are

     reference
     : This item describes a single entry in the table of contents, referencing a single document. To this end its value is a Tcl dictionary containing an id for the referenced document, a label, and a longer textual description which can be associated with the entry. The relevant keys and their values are

       id
       : The value is a string containing the id of the document associated with the entry.

       label
       : The value is a string containing a label for this entry. This string also identifies the entry, and no two entries (references and divisions) in the containing list are allowed to have the same label.

       desc
       : The value is a string containing a longer description for this entry.

```
    **division**
    : This item describes a group of entries in the table of
    contents, inducing a hierarchy of entries. To this end its value
    is a Tcl dictionary containing a label for the group, an
```

```
    optional id to a document for the whole group, and the list of
    entries in the group. The relevant keys and their values are

        **id**
        : The value is a string containing the id of the document
    associated with the whole group. This key is optional.

        **label**
        : The value is a string containing a label for the group.
    This string also identifies the entry, and no two entries
    (references and divisions) in the containing list are allowed to
    have the same label.

        **items**
        : The value is a Tcl list holding the elements of the group,
    in the order they are to be shown. This list has the same
    structure as the value for the keyword **items** used to
    describe the whole table of contents, see above. This closes the
    recusrive definition of the structure, with divisions holding
    the same type of elements as the whole table of contents,
    including other divisions.
```

canonical serialization

The canonical serialization of a table of contents has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this table of contents.

1. The keys found in all the nested Tcl dictionaries are sorted in ascending dictionary order, as generated by Tcl's builtin command **lsort -increasing -dict**.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Text formatter plugin

Copyright


```
Copyright (c) 2009-2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

toc_export_wiki

Keywords: toc_export_wiki, doctools, export, serialization, table of contents, toc, wiki

Name

doctools::toc::export::wiki - wiki export plugin

Synopsis

package require Tcl 8.5 9
package require doctools::toc::export::wiki ?**0.2**?
package require doctools::text
export *serial configuration*

Description

This package implements the doctools table of contents export plugin for the generation of wiki markup.

This is an internal package of doctools, for use by the higher level management packages handling tables of contents, especially **doctools::toc::export**, the export manager.

Using it from a regular interpreter is possible, however only with contortions, and is not recommended. The proper way to use this functionality is through the package **doctools::toc::export** and the export manager objects it provides.

API

The API provided by this package satisfies the specification of the doctoc export plugin API version 2.

export *serial configuration*

This command takes the canonical serialization of a table of contents, as specified in section Toc serialization format, and contained in *serial*, the *configuration*, a dictionary, and generates wiki markup encoding the table. The created string is then returned as the result of the command.

Wiki markup

The basic syntax of the wiki markup generated by this plugin are described at http://wiki.tcl.tk/14.

The plugin goes beyond the classic markup to generate proper headers and indenting.

Configuration

The wiki export plugin recognizes the following configuration variables and changes its behaviour as they specify.

dictionary *map*

This standard configuration variable contains a dictionary mapping from the (symbolic) document ids in reference entries to the actual filenames and/or urls to be used in the output.

Document ids without a mapping are used unchanged.

Note that this plugin ignores the standard configuration variables **user**, **file** and **format**, and their values.

Toc serialization format

Here we specify the format used by the doctools v2 packages to serialize tables of contents as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a table of contents may have more than one regular serialization only exactly one of them will be *canonical*.

regular serialization

1. The serialization of any table of contents is a nested Tcl dictionary.
1. This dictionary holds a single key, **doctools::toc**, and its value. This value holds the contents of the table of contents.
2. The contents of the table of contents are a Tcl dictionary holding the title of the table of contents, a label, and its elements. The relevant keys and their values are

   title
   : The value is a string containing the title of the table of contents.

   label
   : The value is a string containing a label for the table of contents.

   items
   : The value is a Tcl list holding the elements of the table, in the order they are to be shown.

     Each element is a Tcl list holding the type of the item, and its description, in this order. An alternative description would be that it is a Tcl dictionary holding a single key, the item type, mapped to the item description.

     The two legal item types and their descriptions are

     reference
     : This item describes a single entry in the table of contents, referencing a single document. To this end its value is a Tcl dictionary containing an id for the referenced document, a label, and a longer textual description which can be associated with the entry. The relevant keys and their values are

       id
       : The value is a string containing the id of the document associated with the entry.

       label
       : The value is a string containing a label for this entry. This string also identifies the entry, and no two entries (references and divisions) in the containing list are allowed to have the same label.

       desc
       : The value is a string containing a longer description for this entry.

```
**division**
: This item describes a group of entries in the table of
contents, inducing a hierarchy of entries. To this end its value
is a Tcl dictionary containing a label for the group, an
optional id to a document for the whole group, and the list of
entries in the group. The relevant keys and their values are
```

```
        **id**
        : The value is a string containing the id of the document
    associated with the whole group. This key is optional.

        **label**
        : The value is a string containing a label for the group.
    This string also identifies the entry, and no two entries
    (references and divisions) in the containing list are allowed to
    have the same label.

        **items**
        : The value is a Tcl list holding the elements of the group,
    in the order they are to be shown. This list has the same
    structure as the value for the keyword **items** used to
    describe the whole table of contents, see above. This closes the
    recusrive definition of the structure, with divisions holding
    the same type of elements as the whole table of contents,
    including other divisions.
```

canonical serialization

The canonical serialization of a table of contents has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this table of contents.

1. The keys found in all the nested Tcl dictionaries are sorted in ascending dictionary order, as generated by Tcl's builtin command **lsort -increasing -dict**.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Text formatter plugin

Copyright


```
Copyright (c) 2009-2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


toc_import

Keywords: toc_import, conversion, doctoc, documentation, import, json, manpage, markup, parsing, plugin, reference, table, table of contents, url

Name

doctools::toc::import - Importing keyword indices

Synopsis

package require doctools::toc::import ?**0.2.2**?
package require Tcl 8.5 9
package require struct::map
package require doctools::toc::structure
package require snit
package require pluginmgr
::doctools::toc::import *objectName*
objectName method ?*arg arg ...*?
objectName **destroy**
objectName **import text** *text* ?*format*?
objectName **import file** *path* ?*format*?
objectName **import object text** *object text* ?*format*?
objectName **import object file** *object path* ?*format*?
objectName **config names**
objectName **config get**
objectName **config set** *name* ?*value*?
objectName **config unset** *pattern...*
objectName **includes**
objectName **include add** *path*
objectName **include remove** *path*
objectName **include clear**
IncludeFile *currentfile path*
import *text configuration*

Description

This package provides a class to manage the plugins for the import of tables of contents from other formats, i.e. their conversion from, for example *doctoc*, *json*, etc.

This is one of the three public pillars the management of tables of contents resides on. The other two pillars are

1. *Exporting tables of contents*, and
2. *Holding tables of contents*

For information about the Concepts of tables of contents, and their parts, see the same-named section. For information about the data structure which is the major output of the manager objects provided by this package see the section Toc serialization format.

The plugin system of our class is based on the package **pluginmgr**, and configured to look for plugins using

1. the environment variable **DOCTOOLS_TOC_IMPORT_PLUGINS**,
2. the environment variable **DOCTOOLS_TOC_PLUGINS**,
3. the environment variable **DOCTOOLS_PLUGINS**,
4. the path "*~/.doctools/toc/import/plugin*"
5. the path "*~/.doctools/toc/plugin*"

6. the path "*~/.doctools/plugin*"
7. the path "*~/.doctools/toc/import/plugins*"
8. the path "*~/.doctools/toc/plugins*"
9. the path "*~/.doctools/plugins*"
10. the registry entry "HKEY_CURRENT_USER\SOFTWARE\DOCTOOLS\TOC\IMPORT\PLUGINS"
11. the registry entry "HKEY_CURRENT_USER\SOFTWARE\DOCTOOLS\TOC\PLUGINS"
12. the registry entry "HKEY_CURRENT_USER\SOFTWARE\DOCTOOLS\PLUGINS"

The last three are used only when the package is run on a machine using Windows(tm) operating system.

The whole system is delivered with two predefined import plugins, namely

doctoc
: See *doctoc import plugin* for details.

json
: See *json import plugin* for details.

Readers wishing to write their own import plugin for some format, i.e. *plugin writer*s reading and understanding the section containing the Import plugin API v2 reference is an absolute necessity, as it specifies the interaction between this package and its plugins in detail.

Concepts

1. A *table of contents* consists of a (possibly empty) list of *elements*.
2. Each element in the list is identified by its label.
3. Each element is either a *reference*, or a *division*.
4. Each reference has an associated document, identified by a symbolic id, and a textual description.
5. Each division may have an associated document, identified by a symbolic id.
6. Each division consists consists of a (possibly empty) list of *elements*, with each element following the rules as specified in item 2 and above.

A few notes

1. The above rules span up a tree of elements, with references as the leaf nodes, and divisions as the inner nodes, and each element representing an entry in the whole table of contents.
2. The identifying labels of any element E are unique within their division (or toc), and the full label of any element E is the list of labels for all nodes on the unique path from the root of the tree to E, including E.

API

Package commands

::doctools::toc::import *objectName*
: This command creates a new import manager object with an associated Tcl command whose name is *objectName*. This *object* command is explained in full detail in the sections Object command and Object methods. The object command will be created under the current namespace if the *objectName* is not fully qualified, and in the specified namespace otherwise.

Object command

All objects created by the **::doctools::toc::import** command have the following general form:

objectName method ?*arg arg ...*?
The method **method** and its *arg*'uments determine the exact behavior of the command. See section Object methods for the detailed specifications.

Object methods

***objectName* destroy**
This method destroys the object it is invoked for.

***objectName* import text *text* ?*format*?**
This method takes the *text* and converts it from the specified *format* to the canonical serialization of a table of contents using the import plugin for the format. An error is thrown if no plugin could be found for the format. The serialization generated by the conversion process is returned as the result of this method.

If no format is specified the method defaults to **doctoc**.

The specification of what a *canonical* serialization is can be found in the section Toc serialization format.

The plugin has to conform to the interface specified in section Import plugin API v2 reference.

***objectName* import file *path* ?*format*?**
This method is a convenient wrapper around the **import text** method described by the previous item. It reads the contents of the specified file into memory, feeds the result into **import text** and returns the resulting serialization as its own result.

***objectName* import object text *object text* ?*format*?**
This method is a convenient wrapper around the **import text** method described by the previous item. It expects that *object* is an object command supporting a **deserialize** method expecting the canonical serialization of a table of contents. It imports the text using **import text** and then feeds the resulting serialization into the *object* via **deserialize**. This method returns the empty string as it result.

***objectName* import object file *object path* ?*format*?**
This method behaves like **import object text**, except that it reads the text to convert from the specified file instead of being given it as argument.

***objectName* config names**
This method returns a list containing the names of all configuration variables currently known to the object.

***objectName* config get**
This method returns a dictionary containing the names and values of all configuration variables currently known to the object.

***objectName* config set *name* ?*value*?**
This method sets the configuration variable *name* to the specified *value* and returns the new value of the variable.

If no value is specified it simply returns the current value, without changing it.

Note that while the user can set the predefined configuration variables **user** and **format**

doing so will have no effect, these values will be internally overridden when invoking an import plugin.

objectName* config unset *pattern...
This method unsets all configuration variables matching the specified glob *pattern*s. If no pattern is specified it will unset all currently defined configuration variables.

***objectName* includes**
This method returns a list containing the currently specified paths to use to search for include files when processing input. The order of paths in the list corresponds to the order in which they are used, from first to last, and also corresponds to the order in which they were added to the object.

objectName* include add *path
This methods adds the specified *path* to the list of paths to use to search for include files when processing input. The path is added to the end of the list, causing it to be searched after all previously added paths. The result of the command is the empty string.

The method does nothing if the path is already known.

objectName* include remove *path
This methods removes the specified *path* from the list of paths to use to search for include files when processing input. The result of the command is the empty string.

The method does nothing if the path is not known.

***objectName* include clear**
This method clears the list of paths to use to search for include files when processing input. The result of the command is the empty string.

Import plugin API v2 reference

Plugins are what this package uses to manage the support for any input format beyond the Toc serialization format. Here we specify the API the objects created by this package use to interact with their plugins.

A plugin for this package has to follow the rules listed below:

1. A plugin is a package.
2. The name of a plugin package has the form doctools::toc::import::**FOO**, where **FOO** is the name of the format the plugin will generate output for. This name is also the argument to provide to the various **import** methods of import manager objects to get a string encoding a table of contents in that format.
3. The plugin can expect that the package **doctools::toc::export::plugin** is present, as indicator that it was invoked from a genuine plugin manager.
4. The plugin can expect that a command named **IncludeFile** is present, with the signature
   IncludeFile *currentfile path*
   This command has to be invoked by the plugin when it has to process an included file, if the format has the concept of such. An example of such a format would be *doctoc*.

   The plugin has to supply the following arguments
   string *currentfile*
   The path of the file it is currently processing. This may be the empty string if no such is known.

string *path*
The path of the include file as specified in the include directive being processed.

```
The result of the command will be a 5-element list containing

1. A boolean flag indicating the success (**True**) or failure
(**False**) of the operation.

2. In case of success the contents of the included file, and the
empty string otherwise.

3. The resolved, i.e. absolute path of the included file, if
possible, or the unchanged *path* argument. This is for display in
an error message, or as the *currentfile* argument of another call
to **IncludeFile** should this file contain more files.

4. In case of success an empty string, and for failure a code
indicating the reason for it, one of

    notfound
    : The specified file could not be found.

    notread
    : The specified file was found, but not be read into memory.


5. An empty string in case of success of a **notfound** failure,
and an additional error message describing the reason for a
**notread** error in more detail.
```

1. A plugin has to provide one command, with the signature shown below.
import *text configuration*
Whenever an import manager of **doctools::toc** has to parse input for a table of contents it will invoke this command.
string *text*
This argument will contain the text encoding the table of contents per the format the plugin is for.
dictionary *configuration*
This argument will contain the current configuration to apply to the parsing, as a dictionary mapping from variable names to values.
The following configuration variables have a predefined meaning all plugins have to obey, although they can ignore this information at their discretion. Any other other configuration variables recognized by a plugin will be described in the manpage for that plugin.
user
This variable is expected to contain the name of the user owning the process invoking

the plugin.

format
This variable is expected to contain the name of the format whose plugin is invoked.

1. A single usage cycle of a plugin consists of the invokations of the command **import**. This call has to leave the plugin in a state where another usage cycle can be run without problems.

Toc serialization format

Here we specify the format used by the doctools v2 packages to serialize tables of contents as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a table of contents may have more than one regular serialization only exactly one of them will be *canonical*.

regular serialization

1. The serialization of any table of contents is a nested Tcl dictionary.
1. This dictionary holds a single key, **doctools::toc**, and its value. This value holds the contents of the table of contents.
2. The contents of the table of contents are a Tcl dictionary holding the title of the table of contents, a label, and its elements. The relevant keys and their values are

   title
   The value is a string containing the title of the table of contents.

   label
   The value is a string containing a label for the table of contents.

   items
   The value is a Tcl list holding the elements of the table, in the order they are to be shown.

   Each element is a Tcl list holding the type of the item, and its description, in this order. An alternative description would be that it is a Tcl dictionary holding a single key, the item type, mapped to the item description.

   The two legal item types and their descriptions are

   reference
   This item describes a single entry in the table of contents, referencing a single document. To this end its value is a Tcl dictionary containing an id for the referenced document, a label, and a longer textual description which can be associated with the entry. The relevant keys and their values are

   id
   The value is a string containing the id of the document associated with the entry.

   label
   The value is a string containing a label for this entry. This string also identifies the entry, and no two entries (references and divisions) in the containing list are allowed to have the same label.

   desc
   The value is a string containing a longer description for this entry.

```
    **division**
    : This item describes a group of entries in the table of
    contents, inducing a hierarchy of entries. To this end its value
    is a Tcl dictionary containing a label for the group, an
    optional id to a document for the whole group, and the list of
    entries in the group. The relevant keys and their values are

        **id**
        : The value is a string containing the id of the document
    associated with the whole group. This key is optional.

        **label**
        : The value is a string containing a label for the group.
    This string also identifies the entry, and no two entries
    (references and divisions) in the containing list are allowed to
    have the same label.

        **items**
        : The value is a Tcl list holding the elements of the group,
    in the order they are to be shown. This list has the same
    structure as the value for the keyword **items** used to
    describe the whole table of contents, see above. This closes the
    recusrive definition of the structure, with divisions holding
    the same type of elements as the whole table of contents,
    including other divisions.
```

canonical serialization

The canonical serialization of a table of contents has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this table of contents.

1. The keys found in all the nested Tcl dictionaries are sorted in ascending dictionary order, as generated by Tcl's builtin command **lsort -increasing -dict**.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Documentation tools

Copyright


```
Copyright (c) 2009-2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


toc_import_json

Keywords: toc_import_json, JSON, deserialization, doctools, import, table of contents, toc

Name

doctools::toc::import::json - JSON import plugin

Synopsis

package require Tcl 8.5 9
package require doctools::toc::import::json ?**0.2.1**?
package require doctools::toc::structure
package require json
import *string configuration*

Description

This package implements the doctools table of contents import plugin for the parsing of JSON markup.

This is an internal package of doctools, for use by the higher level management packages handling tables of contents, especially **doctools::toc::import**, the import manager.

Using it from a regular interpreter is possible, however only with contortions, and is not recommended. The proper way to use this functionality is through the package **doctools::toc::import** and the import manager objects it provides.

API

The API provided by this package satisfies the specification of the doctoc import plugin API version 2.

import *string configuration*

This command takes the *string* and parses it as JSON markup encoding a table of contents, in the context of the specified *configuration* (a dictionary). The result of the command is the canonical serialization of that table of contents, in the form specified in section Toc serialization format.

JSON notation of tables of contents

The JSON format used for tables of contents is a direct translation of the Toc serialization format, mapping Tcl dictionaries as JSON objects and Tcl lists as JSON arrays. For example, the Tcl serialization

```
doctools::toc {
    items {
        {reference {
        desc {DocTools - Tables of Contents}
         id introduction.man
```

```
         label doctools::toc::introduction
    }}
    {division {
         id processing.man
         items {
             {reference {
             desc {doctoc serialization utilities}
             id structure.man
             label doctools::toc::structure
         }}
         {reference {
             desc {Parsing text in doctoc format}
             id parse.man
             label doctools::toc::parse
         }}
         }
             label Processing
        }}
    }
    label {Table of Contents}
    title TOC
}
```

is equivalent to the JSON string

```
{
    "doctools::toc" : {
        "items" : [{
            "reference" : {
                "desc"  : "DocTools - Tables of Contents",
                "id"    : "introduction.man",
                "label" : "doctools::toc::introduction"
            }
        },{
            "division" : {
                "id"    : "processing.man",
                "items" : [{
                    "reference" : {
                        "desc"  : "doctoc serialization utilities",
                        "id"    : "structure.man",
                        "label" : "doctools::toc::structure"
                    }
                },{
                    "reference" : {
                        "desc"  : "Parsing text in doctoc format",
                        "id"    : "parse.man",
                        "label" : "doctools::toc::parse"
```

```
                    }
                }],
                "label" : "Processing"
            }
        }],
        "label" : "Table of Contents",
        "title" : "TOC"
    }
}
```

Toc serialization format

Here we specify the format used by the doctools v2 packages to serialize tables of contents as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a table of contents may have more than one regular serialization only exactly one of them will be *canonical*.

regular serialization

1. The serialization of any table of contents is a nested Tcl dictionary.
1. This dictionary holds a single key, **doctools::toc**, and its value. This value holds the contents of the table of contents.
2. The contents of the table of contents are a Tcl dictionary holding the title of the table of contents, a label, and its elements. The relevant keys and their values are

   title
   The value is a string containing the title of the table of contents.

   label
   The value is a string containing a label for the table of contents.

   items
   The value is a Tcl list holding the elements of the table, in the order they are to be shown.

   Each element is a Tcl list holding the type of the item, and its description, in this order. An alternative description would be that it is a Tcl dictionary holding a single key, the item type, mapped to the item description.

   The two legal item types and their descriptions are

   reference
   This item describes a single entry in the table of contents, referencing a single document. To this end its value is a Tcl dictionary containing an id for the referenced document, a label, and a longer textual description which can be associated with the entry. The relevant keys and their values are

   id
   The value is a string containing the id of the document associated with the entry.

   label
   The value is a string containing a label for this entry. This string also identifies the entry, and no two entries (references and divisions) in the containing list are allowed to have the same label.

   desc

The value is a string containing a longer description for this entry.

```
    **division**
    : This item describes a group of entries in the table of
    contents, inducing a hierarchy of entries. To this end its value
    is a Tcl dictionary containing a label for the group, an
    optional id to a document for the whole group, and the list of
    entries in the group. The relevant keys and their values are

        **id**
        : The value is a string containing the id of the document
    associated with the whole group. This key is optional.

        **label**
        : The value is a string containing a label for the group.
    This string also identifies the entry, and no two entries
    (references and divisions) in the containing list are allowed to
    have the same label.

        **items**
        : The value is a Tcl list holding the elements of the group,
    in the order they are to be shown. This list has the same
    structure as the value for the keyword **items** used to
    describe the whole table of contents, see above. This closes the
    recusrive definition of the structure, with divisions holding
    the same type of elements as the whole table of contents,
    including other divisions.
```

canonical serialization

The canonical serialization of a table of contents has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this table of contents.

1. The keys found in all the nested Tcl dictionaries are sorted in ascending dictionary order, as generated by Tcl's builtin command **lsort -increasing -dict**.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Text formatter plugin

Copyright

```
Copyright (c) 2009-2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


toc_introduction
Keywords: toc_introduction, contents, conversion, formatting, markup, parsing, plugin, semantic markup, table of contents

Name
doctools2toc_introduction - DocTools - Tables of Contents

Description
doctoc (short for *documentation tables of contents*) stands for a set of related, yet different, entities which are working together for the easy creation and transformation of tables and contents for documentation.

These are

1. A tcl based language for the semantic markup of a table of contents. Markup is represented by Tcl commands. Beginners should start with the *doctoc language introduction*. The formal specification is split over two documents, one dealing with the *doctoc language syntax*, the other a *doctoc language command reference*.
2. A set of packages for the programmatic manipulation of tables of contents in memory, and their conversion between various formats, reading and writing. The aforementioned markup language is one of the formats which can be both read from and written to.
3. The system for the conversion of tables of contents is based on a plugin mechanism, for this we have two APIs describing the interface between the packages above and the import/export plugins.

Which of the more detailed documents are relevant to the reader of this introduction depends on their role in the documentation process.

1. A *writer* of documentation has to understand the markup language itself. A beginner to doctoc should read the more informally written *doctoc language introduction* first. Having digested this the formal *doctoc language syntax* specification should become understandable. A writer experienced with doctoc may only need the *doctoc language command reference* from time to time to refresh her memory.

   While a document is written the **dtp** application can be used to validate it, and after completion it also performs the conversion into the chosen system of visual markup, be it *roff, HTML, plain text, wiki, etc. The simpler **dtplite** application makes internal use of doctoc when handling directories of documentation, automatically generating a proper table of contents for them.
2. A *processor* of documentation written in the *doctoc* markup language has to know which tools are available for use.

   The main tool is the aforementioned **dtp** application provided by Tcllib. The simpler **dtplite**

does not expose doctoc to the user. At the bottom level, common to both applications, however we find the three packages providing the basic facilities to handle tables of contents, i.e. import from textual formats, programmatic manipulation in memory, and export to textual formats. These are

doctoools::toc
: Programmatic manipulation of tables of contents in memory.

doctoools::toc::import
: Import of tables of contents from various textual formats. The set of supported formats is extensible through plugin packages.

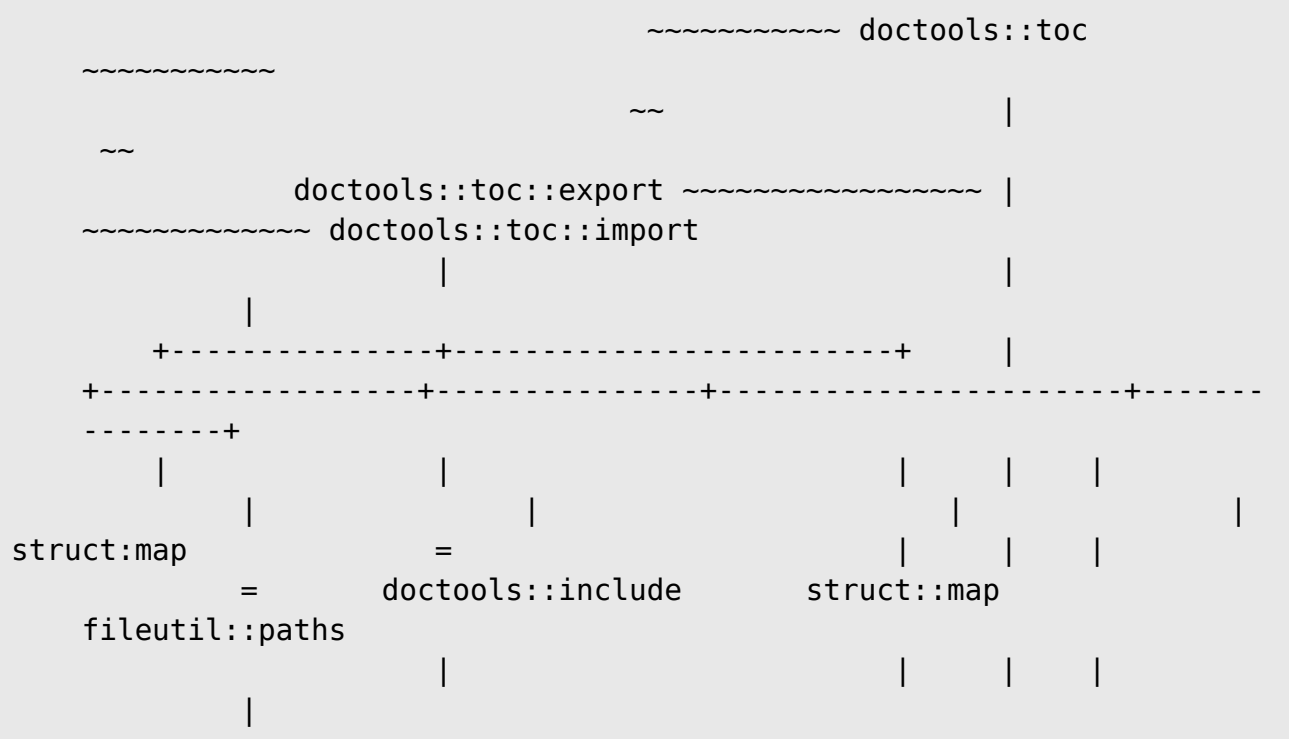
doctoools::toc::export
: Export of tables of contents to various textual formats. The set of supported formats is extensible through plugin packages.

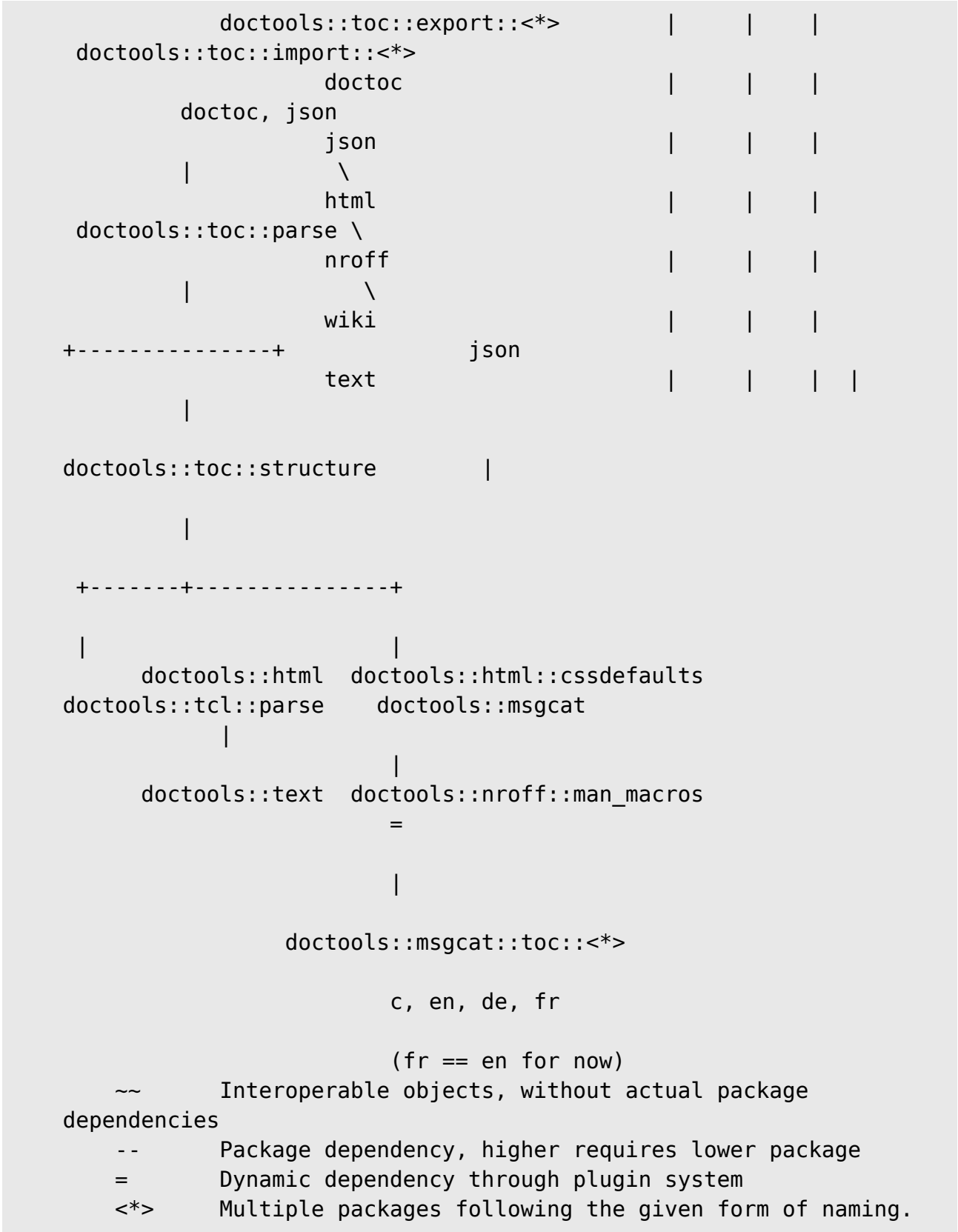
1. See also section Package overview for an overview of the dependencies between these and other, supporting packages.
2. At last, but not least, *plugin writers* have to understand the interaction between the import and export packages and their plugins. These APIs are described in the documentation for the two relevant packages, i.e.
   - **doctoools::toc::import**
   - **doctoools::toc::export**

Related formats

The doctoc format does not stand alone, it has two companion formats. These are called *docidx* and *doctools*, and they are intended for the markup of *keyword indices*, and of general documentation, respectively. They are described in their own sets of documents, starting at the *DocTools - Keyword Indices* and the *DocTools - General*, respectively.

Package overview





Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Documentation tools

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


toc_msgcat_c
Keywords: toc_msgcat_c, C, catalog package, doctoc, doctools, i18n, internationalization, l10n, localization, message catalog, message package

Name
doctools::msgcat::toc::c - Message catalog for the doctoc parser (C)

Synopsis
package require Tcl 8.5 9
package require msgcat
package require doctools::msgcat::toc::c ?**0.1**?

Description
The package **doctools::msgcat::toc::c** is a support module providing the C language message catalog for the doctoc parser in the doctools system version 2. As such it is an internal package a regular user (developer) should not be in direct contact with.

If you are such please go the documentation of either

1. **doctools::doc**,
2. **doctools::toc**, or
3. **doctools::idx**

Within the system architecture this package resides under the package **doctools::msgcat** providing the general message catalog management within the system. *Note* that there is *no* explicit dependency between the manager and catalog packages. The catalog is a plugin which is selected and loaded dynamically.

API
This package has no exported API.

Bugs, ideas, feedback
If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Documentation tools

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


toc_msgcat_de
Keywords: toc_msgcat_de, DE, catalog package, doctoc, doctools, i18n, internationalization, l10n , localization, message catalog, message package

Name
doctools::msgcat::toc::de - Message catalog for the doctoc parser (DE)

Synopsis
package require Tcl 8.5 9
package require msgcat
package require doctools::msgcat::toc::de ?**0.1**?

Description
The package **doctools::msgcat::toc::de** is a support module providing the DE (german) language message catalog for the doctoc parser in the doctools system version 2. As such it is an internal package a regular user (developer) should not be in direct contact with.

If you are such please go the documentation of either

1. **doctools::doc**,
2. **doctools::toc**, or
3. **doctools::idx**

Within the system architecture this package resides under the package **doctools::msgcat** providing the general message catalog management within the system. *Note* that there is *no* explicit dependency between the manager and catalog packages. The catalog is a plugin which is selected and loaded dynamically.

API
This package has no exported API.

Bugs, ideas, feedback
If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be

made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Documentation tools

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


toc_msgcat_en
Keywords: toc_msgcat_en, EN, catalog package, doctoc, doctools, i18n, internationalization, l10n , localization, message catalog, message package

Name
doctools::msgcat::toc::en - Message catalog for the doctoc parser (EN)

Synopsis
package require Tcl 8.5 9
package require msgcat
package require doctools::msgcat::toc::en ?**0.1**?

Description
The package **doctools::msgcat::toc::en** is a support module providing the EN (english) language message catalog for the doctoc parser in the doctools system version 2. As such it is an internal package a regular user (developer) should not be in direct contact with.

If you are such please go the documentation of either

1. **doctools::doc**,
2. **doctools::toc**, or
3. **doctools::idx**

Within the system architecture this package resides under the package **doctools::msgcat** providing the general message catalog management within the system. *Note* that there is *no* explicit dependency between the manager and catalog packages. The catalog is a plugin which is selected and loaded dynamically.

API
This package has no exported API.

Bugs, ideas, feedback
If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the

left-most button in the secondary navigation bar.

Category
Documentation tools

Copyright
```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

toc_msgcat_fr
Keywords: toc_msgcat_fr, FR, catalog package, doctoc, doctools, i18n, internationalization, l10n, localization, message catalog, message package

Name
doctools::msgcat::toc::fr - Message catalog for the doctoc parser (FR)

Synopsis
package require Tcl 8.5 9
package require msgcat
package require doctools::msgcat::toc::fr ?**0.1**?

Description
The package **doctools::msgcat::toc::fr** is a support module providing the FR (french) language message catalog for the doctoc parser in the doctools system version 2. As such it is an internal package a regular user (developer) should not be in direct contact with.

If you are such please go the documentation of either

1. **doctools::doc**,
2. **doctools::toc**, or
3. **doctools::idx**

Within the system architecture this package resides under the package **doctools::msgcat** providing the general message catalog management within the system. *Note* that there is *no* explicit dependency between the manager and catalog packages. The catalog is a plugin which is selected and loaded dynamically.

API
This package has no exported API.

Bugs, ideas, feedback
If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Documentation tools

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


toc_parse
Keywords: toc_parse, doctoc, doctools, lexer, parser

Name
doctools::toc::parse - Parsing text in doctoc format

Synopsis
package require doctools::toc::parse ?**0.2**?
package require Tcl 8.5 9
package require doctools::toc::structure
package require doctools::msgcat
package require doctools::tcl::parse
package require fileutil
package require logger
package require snit
package require struct::list
package require struct::stack
::doctools::toc::parse text *text*
::doctools::toc::parse file *path*
::doctools::toc::parse includes
::doctools::toc::parse include add *path*
::doctools::toc::parse include remove *path*
::doctools::toc::parse include clear
::doctools::toc::parse vars
::doctools::toc::parse var set *name value*
::doctools::toc::parse var unset *name*
::doctools::toc::parse var clear ?*pattern*?

Description
This package provides commands to parse text written in the *doctoc* markup language and convert it into the canonical serialization of the table of contents encoded in the text. See the section Toc serialization format for specification of their format.

This is an internal package of doctools, for use by the higher level packages handling *doctoc* documents.

API
::doctools::toc::parse text *text*
The command takes the string contained in *text* and parses it under the assumption that it contains a document written using the *doctoc* markup language. An error is thrown if this

assumption is found to be false. The format of these errors is described in section Parse errors .

When successful the command returns the canonical serialization of the table of contents which was encoded in the text. See the section Toc serialization format for specification of that format.

::doctools::toc::parse file *path*
The same as **text**, except that the text to parse is read from the file specified by *path*.

::doctools::toc::parse includes
This method returns the current list of search paths used when looking for include files.

::doctools::toc::parse include add *path*
This method adds the *path* to the list of paths searched when looking for an include file. The call is ignored if the path is already in the list of paths. The method returns the empty string as its result.

::doctools::toc::parse include remove *path*
This method removes the *path* from the list of paths searched when looking for an include file. The call is ignored if the path is not contained in the list of paths. The method returns the empty string as its result.

::doctools::toc::parse include clear
This method clears the list of search paths for include files.

::doctools::toc::parse vars
This method returns a dictionary containing the current set of predefined variables known to the **vset** markup command during processing.

::doctools::toc::parse var set *name value*
This method adds the variable *name* to the set of predefined variables known to the **vset** markup command during processing, and gives it the specified *value*. The method returns the empty string as its result.

::doctools::toc::parse var unset *name*
This method removes the variable *name* from the set of predefined variables known to the **vset** markup command during processing. The method returns the empty string as its result.

::doctools::toc::parse var clear ?*pattern*?
This method removes all variables matching the *pattern* from the set of predefined variables known to the **vset** markup command during processing. The method returns the empty string as its result.

The pattern matching is done with **string match**, and the default pattern used when none is specified, is *.

Parse errors

The format of the parse error messages thrown when encountering violations of the *doctoc* markup syntax is human readable and not intended for processing by machines. As such it is not documented.

However, the errorCode attached to the message is machine-readable and has the following format:

1. The error code will be a list, each element describing a single error found in the input. The list has at least one element, possibly more.
2. Each error element will be a list containing six strings describing an error in detail. The strings will be
   1. The path of the file the error occurred in. This may be empty.
   2. The range of the token the error was found at. This range is a two-element list containing the offset of the first and last character in the range, counted from the beginning of the input (file). Offsets are counted from zero.
   3. The line the first character after the error is on. Lines are counted from one.
   4. The column the first character after the error is at. Columns are counted from zero.
   5. The message code of the error. This value can be used as argument to **msgcat::mc** to obtain a localized error message, assuming that the application had a suitable call of **doctools::msgcat::init** to initialize the necessary message catalogs (See package **doctools::msgcat**).
   6. A list of details for the error, like the markup command involved. In the case of message code **doctoc/include/syntax** this value is the set of errors found in the included file, using the format described here.

[doctoc] notation of tables of contents

The doctoc format for tables of contents, also called the *doctoc markup language*, is too large to be covered in single section. The interested reader should start with the document

1. *doctoc language introduction*

and then proceed from there to the formal specifications, i.e. the documents

1. *doctoc language syntax* and
2. *doctoc language command reference*.

to get a thorough understanding of the language.

Toc serialization format

Here we specify the format used by the doctools v2 packages to serialize tables of contents as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a table of contents may have more than one regular serialization only exactly one of them will be *canonical*.

regular serialization

1. The serialization of any table of contents is a nested Tcl dictionary.
1. This dictionary holds a single key, **doctools::toc**, and its value. This value holds the contents of the table of contents.
2. The contents of the table of contents are a Tcl dictionary holding the title of the table of contents, a label, and its elements. The relevant keys and their values are

   title
   : The value is a string containing the title of the table of contents.

   label
   : The value is a string containing a label for the table of contents.

   items
   : The value is a Tcl list holding the elements of the table, in the order they are to be

shown.

Each element is a Tcl list holding the type of the item, and its description, in this order. An alternative description would be that it is a Tcl dictionary holding a single key, the item type, mapped to the item description.

The two legal item types and their descriptions are

reference
: This item describes a single entry in the table of contents, referencing a single document. To this end its value is a Tcl dictionary containing an id for the referenced document, a label, and a longer textual description which can be associated with the entry. The relevant keys and their values are

  id
  : The value is a string containing the id of the document associated with the entry.

  label
  : The value is a string containing a label for this entry. This string also identifies the entry, and no two entries (references and divisions) in the containing list are allowed to have the same label.

  desc
  : The value is a string containing a longer description for this entry.

```
**division**
: This item describes a group of entries in the table of
contents, inducing a hierarchy of entries. To this end its value
is a Tcl dictionary containing a label for the group, an
optional id to a document for the whole group, and the list of
entries in the group. The relevant keys and their values are

    **id**
    : The value is a string containing the id of the document
associated with the whole group. This key is optional.

    **label**
    : The value is a string containing a label for the group.
This string also identifies the entry, and no two entries
(references and divisions) in the containing list are allowed to
have the same label.

    **items**
    : The value is a Tcl list holding the elements of the group,
in the order they are to be shown. This list has the same
structure as the value for the keyword **items** used to
describe the whole table of contents, see above. This closes the
recusrive definition of the structure, with divisions holding
the same type of elements as the whole table of contents,
including other divisions.
```

canonical serialization

The canonical serialization of a table of contents has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this table of contents.

1. The keys found in all the nested Tcl dictionaries are sorted in ascending dictionary order, as generated by Tcl's builtin command **lsort -increasing -dict**.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Documentation tools

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


toc_structure

Keywords: toc_structure, deserialization, doctoc, doctools, serialization

Name

doctools::toc::structure - Doctoc serialization utilities

Synopsis

package require doctools::toc::structure ?**0.2**?
package require Tcl 8.5 9
package require logger
package require snit
::doctools::toc::structure verify *serial* ?*canonvar*?
::doctools::toc::structure verify-as-canonical *serial*
::doctools::toc::structure canonicalize *serial*
::doctools::toc::structure print *serial*
::doctools::toc::structure merge *seriala serialb*

Description

This package provides commands to work with the serializations of tables of contents as managed by the doctools system v2, and specified in section Toc serialization format.

This is an internal package of doctools, for use by the higher level packages handling tables of contents and their conversion into and out of various other formats, like documents written

using *doctoc* markup.

API

::doctools::toc::structure verify *serial* ?*canonvar*?

This command verifies that the content of *serial* is a valid *regular* serialization of a table of contents and will throw an error if that is not the case. The result of the command is the empty string.

If the argument *canonvar* is specified it is interpreted as the name of a variable in the calling context. This variable will be written to if and only if *serial* is a valid regular serialization. Its value will be a boolean, with **True** indicating that the serialization is not only valid, but also *canonical*. **False** will be written for a valid, but non-canonical serialization.

For the specification of regular and canonical serializations see the section Toc serialization format.

::doctools::toc::structure verify-as-canonical *serial*

This command verifies that the content of *serial* is a valid *canonical* serialization of a table of contents and will throw an error if that is not the case. The result of the command is the empty string.

For the specification of canonical serializations see the section Toc serialization format.

::doctools::toc::structure canonicalize *serial*

This command assumes that the content of *serial* is a valid *regular* serialization of a table of contents and will throw an error if that is not the case.

It will then convert the input into the *canonical* serialization of the contained table of contents and return it as its result. If the input is already canonical it will be returned unchanged.

For the specification of regular and canonical serializations see the section Toc serialization format.

::doctools::toc::structure print *serial*

This command assumes that the argument *serial* contains a valid regular serialization of a table of contents and returns a string containing that table in a human readable form.

The exact format of this form is not specified and cannot be relied on for parsing or other machine-based activities.

For the specification of regular serializations see the section Toc serialization format.

::doctools::toc::structure merge *seriala serialb*

This command accepts the regular serializations of two tables of contents and uses them to create their union. The result of the command is the canonical serialization of this unified table of contents.

Title and label of the resulting table are taken from the table contained in *serialb*.

The whole table and its divisions are merged recursively in the same manner:
1. All reference elements which occur in both divisions (identified by their label) are unified

with document id's and descriptions taken from the second table.
2. All division elements which occur in both divisions (identified by their label) are unified with the optional document id taken from the second table, if any, or from the first if none is in the second. The elements in the division are merged recursively using the same algorithm as described in this list.
3. Type conflicts between elements, i.e. finding two elements with the same label but different types result in a merge error.
4. All elements found in the second division but not in the first are added to the end of the list of elements in the merge result.

For the specification of regular and canonical serializations see the section Toc serialization format.

Toc serialization format

Here we specify the format used by the doctools v2 packages to serialize tables of contents as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a table of contents may have more than one regular serialization only exactly one of them will be *canonical*.

regular serialization

1. The serialization of any table of contents is a nested Tcl dictionary.
1. This dictionary holds a single key, **doctools::toc**, and its value. This value holds the contents of the table of contents.
2. The contents of the table of contents are a Tcl dictionary holding the title of the table of contents, a label, and its elements. The relevant keys and their values are

   title
   The value is a string containing the title of the table of contents.

   label
   The value is a string containing a label for the table of contents.

   items
   The value is a Tcl list holding the elements of the table, in the order they are to be shown.

   Each element is a Tcl list holding the type of the item, and its description, in this order. An alternative description would be that it is a Tcl dictionary holding a single key, the item type, mapped to the item description.

   The two legal item types and their descriptions are

   reference
   This item describes a single entry in the table of contents, referencing a single document. To this end its value is a Tcl dictionary containing an id for the referenced document, a label, and a longer textual description which can be associated with the entry. The relevant keys and their values are

   id
   The value is a string containing the id of the document associated with the entry.

   label
   The value is a string containing a label for this entry. This string also identifies the entry, and no two entries (references and divisions) in the containing list are

allowed to have the same label.

desc
The value is a string containing a longer description for this entry.

```
    **division**
    : This item describes a group of entries in the table of
    contents, inducing a hierarchy of entries. To this end its value
    is a Tcl dictionary containing a label for the group, an
    optional id to a document for the whole group, and the list of
    entries in the group. The relevant keys and their values are

        **id**
        : The value is a string containing the id of the document
    associated with the whole group. This key is optional.

        **label**
        : The value is a string containing a label for the group.
    This string also identifies the entry, and no two entries
    (references and divisions) in the containing list are allowed to
    have the same label.

        **items**
        : The value is a Tcl list holding the elements of the group,
    in the order they are to be shown. This list has the same
    structure as the value for the keyword **items** used to
    describe the whole table of contents, see above. This closes the
    recusrive definition of the structure, with divisions holding
    the same type of elements as the whole table of contents,
    including other divisions.
```

canonical serialization
The canonical serialization of a table of contents has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this table of contents.

1. The keys found in all the nested Tcl dictionaries are sorted in ascending dictionary order, as generated by Tcl's builtin command **lsort -increasing -dict**.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the

left-most button in the secondary navigation bar.

Category
Documentation tools

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


dtplite

pkg_dtplite
Keywords: pkg_dtplite, HTML, TMML, conversion, docidx, doctoc, doctools, manpage, markup, nroff

Name
pkg_dtplite - Lightweight DocTools Markup Processor

Synopsis
package require dtplite ?**1.3.2**?
dtplite::print-via *cmd*
dtplite::do *arguments*

Description
The package provided by this document, **dtplite**, is the foundation for the **dtplite** application. It is a light wrapper around the various **doctools** packages.

Commands
dtplite::print-via *cmd*
Redirect print operations of the package to the specified *cmd*.

The result of the command is the empty string.

dtplite::do *arguments*
The main command it takes a *single list* of *arguments*, processes them, and performs the specified action.

The result of the command is the empty string.

The details of the syntax inside of the *arguments* list are explained in section *COMMAND LINE* of the documentation for the **dtplite** application.

Bugs, ideas, feedback
If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *doctools* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Documentation tools

Copyright
```
Copyright (c) 2004-2013 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

fileutil

fileutil
Keywords: fileutil, cat, file utilities, grep, temp file, test, touch, type

Name
fileutil - Procedures implementing some file utilities

Synopsis
package require Tcl 8.5 9
package require fileutil ?**1.16.4**?
::fileutil::lexnormalize *path*
::fileutil::fullnormalize *path*
::fileutil::test *path codes* ?*msgvar*? ?*label*?
::fileutil::cat (?*options*? *file*)...
::fileutil::writeFile ?*options*? *file data*
::fileutil::appendToFile ?*options*? *file data*
::fileutil::insertIntoFile ?*options*? *file at data*
::fileutil::removeFromFile ?*options*? *file at n*
::fileutil::replaceInFile ?*options*? *file at n data*
::fileutil::updateInPlace ?*options*? *file cmd*
::fileutil::fileType *filename*
::fileutil::find ?*basedir ?filtercmd*?
::fileutil::findByPattern *basedir* ?**-regexp|-glob**? ?**--**? *patterns*
::fileutil::foreachLine *var filename cmd*
::fileutil::grep *pattern* ?*files*?
::fileutil::install ?*-m mode*? *source destination*
::fileutil::stripN *path n*
::fileutil::stripPwd *path*
::fileutil::stripPath *prefix path*
::fileutil::jail *jail path*
::fileutil::touch ?**-a**? ?**-c**? ?**-m**? ?*-r ref_file*? ?*-t time*? *filename* ?*...*?
::fileutil::tempdir
::fileutil::tempdir *path*
::fileutil::tempdirReset

::fileutil::tempfile ?*prefix*?
::fileutil::maketempdir ?*-prefix str*? ?*-suffix str*? ?*-dir str*?
::fileutil::relative *base dst*
::fileutil::relativeUrl *base dst*

Description

This package provides implementations of standard unix utilities.

::fileutil::lexnormalize *path*

This command performs purely lexical normalization on the *path* and returns the changed path as its result. Symbolic links in the path are *not* resolved.

Examples:

```
fileutil::lexnormalize /foo/./bar
=> /foo/bar

fileutil::lexnormalize /foo/../bar
=> /bar
```

::fileutil::fullnormalize *path*

This command resolves all symbolic links in the *path* and returns the changed path as its result. In contrast to the builtin **file normalize** this command resolves a symbolic link in the last element of the path as well.

::fileutil::test *path codes* ?*msgvar*? ?*label*?

A command for the testing of several properties of a *path*. The properties to test for are specified in *codes*, either as a list of keywords describing the properties, or as a string where each letter is a shorthand for a property to test. The recognized keywords, shorthands, and associated properties are shown in the list below. The tests are executed in the order given to the command.

The result of the command is a boolean value. It will be true if and only if the *path* passes all the specified tests. In the case of the *path* not passing one or more test the first failing test will leave a message in the variable referenced by *msgvar*, if such is specified. The message will be prefixed with *label*, if it is specified. *Note* that the variabled referenced by *msgvar* is not touched at all if all the tests pass.

***r*ead**
: **file readable**

***w*rite**
: **file writable**

***e*xists**
: **file exists**

e*x*ec
: **file executable**

***f*ile**
: **file isfile**

***d*ir**
: **file isdirectory**

::fileutil::cat (?*options*? *file*)...
A tcl implementation of the UNIX **cat** command. Returns the contents of the specified file(s). The arguments are files to read, with interspersed options configuring the process. If there are problems reading any of the files, an error will occur, and no data will be returned.

The options accepted are **-encoding**, **-translation**, **-eofchar**, and **--**. With the exception of the last all options take a single value as argument, as specified by the tcl builtin command **fconfigure**. The **--** has to be used to terminate option processing before a file if that file's name begins with a dash.

Each file can have its own set of options coming before it, and for anything not specified directly the defaults are inherited from the options of the previous file. The first file inherits the system default for unspecified options.

::fileutil::writeFile ?*options*? *file data*
The command replaces the current contents of the specified *file* with *data*, with the process configured by the options. The command accepts the same options as **::fileutil::cat**. The specification of a non-existent file is legal and causes the command to create the file (and all required but missing directories).

::fileutil::appendToFile ?*options*? *file data*
This command is like **::fileutil::writeFile**, except that the previous contents of *file* are not replaced, but appended to. The command accepts the same options as **::fileutil::cat**

::fileutil::insertIntoFile ?*options*? *file at data*
This comment is similar to **::fileutil::appendToFile**, except that the new data is not appended at the end, but inserted at a specified location within the file. In further contrast this command has to be given the path to an existing file. It will not create a missing file, but throw an error instead.

The specified location *at* has to be an integer number in the range **0** ... [file size *file*]. **0** will cause insertion of the new data before the first character of the existing content, whereas [file size *file*] causes insertion after the last character of the existing content, i.e. appending.

The command accepts the same options as **::fileutil::cat**.

::fileutil::removeFromFile ?*options*? *file at n*
This command is the complement to **::fileutil::insertIntoFile**, removing *n* characters from the *file*, starting at location *at*. The specified location *at* has to be an integer number in the range **0** ... [file size *file*] - *n*. **0** will cause the removal of the new data to start with the first character of the existing content, whereas [file size *file*] - *n* causes the removal of the tail of the existing content, i.e. the truncation of the file.

The command accepts the same options as **::fileutil::cat**.

::fileutil::replaceInFile ?*options*? *file at n data*
This command is a combination of **::fileutil::removeFromFile** and **::fileutil::insertIntoFile**. It first removes the part of the contents specified by the arguments *at* and *n*, and then inserts *data* at the given location, effectively replacing the removed by content with *data*. All constraints imposed on *at* and *n* by **::fileutil::removeFromFile** and **::fileutil::insertIntoFile** are obeyed.

The command accepts the same options as **::fileutil::cat**.

::fileutil::updateInPlace ?*options*? *file cmd*

This command can be seen as the generic core functionality of **::fileutil::replaceInFile**. It first reads the contents of the specified *file*, then runs the command prefix *cmd* with that data appended to it, and at last writes the result of that invokation back as the new contents of the file.

If the executed command throws an error the *file* is not changed.

The command accepts the same options as **::fileutil::cat**.

::fileutil::fileType *filename*

An implementation of the UNIX **file** command, which uses various heuristics to guess the type of a file. Returns a list specifying as much type information as can be determined about the file, from most general (eg, "binary" or "text") to most specific (eg, "gif"). For example, the return value for a GIF file would be "binary graphic gif". The command will detect the following types of files: directory, empty, binary, text, script (with interpreter), executable elf, executable dos, executable ne, executable pe, graphic gif, graphic jpeg, graphic png, graphic tiff, graphic bitmap, html, xml (with doctype if available), message pgp, binary pdf, text ps, text eps, binary gravity_wave_data_frame, compressed bzip, compressed gzip, compressed zip, compressed tar, audio wave, audio mpeg, and link. It further detects doctools, doctoc, and docidx documentation files, and tklib diagrams.

::fileutil::find ?*basedir* ?*filtercmd*??

An implementation of the unix command **find**. Adapted from the Tcler's Wiki. Takes at most two arguments, the path to the directory to start searching from and a command to use to evaluate interest in each file. The path defaults to ".", i.e. the current directory. The command defaults to the empty string, which means that all files are of interest. The command takes care *not* to lose itself in infinite loops upon encountering circular link structures. The result of the command is a list containing the paths to the interesting files.

The *filtercmd*, if specified, is interpreted as a command prefix and one argument is added to it, the name of the file or directory find is currently looking at. Note that this name is *not* fully qualified. It has to be joined it with the result of **pwd** to get an absolute filename.

The result of *filtercmd* is a boolean value that indicates if the current file should be included in the list of interesting files.

Example:

```
    # find .tcl files
    package require fileutil
    proc is_tcl {name} {return [string match *.tcl $name]}
    set tcl_files [fileutil::find . is_tcl]
```

::fileutil::findByPattern *basedir* ?-regexp|-glob? ?--? *patterns*

This command is based upon the **TclX** command **recursive_glob**, except that it doesn't allow recursion over more than one directory at a time. It uses **::fileutil::find** internally and is thus able to and does follow symbolic links, something the **TclX** command does not do. First

argument is the directory to start the search in, second argument is a list of *patterns*. The command returns a list of all files reachable through *basedir* whose names match at least one of the patterns. The options before the pattern-list determine the style of matching, either regexp or glob. glob-style matching is the default if no options are given. Usage of the option **--** stops option processing. This allows the use of a leading '-' in the patterns.

::fileutil::foreachLine *var filename cmd*

The command reads the file *filename* and executes the script *cmd* for every line in the file. During the execution of the script the variable *var* is set to the contents of the current line. The return value of this command is the result of the last invocation of the script *cmd* or the empty string if the file was empty.

::fileutil::grep *pattern* ?*files*?

Implementation of **grep**. Adapted from the Tcler's Wiki. The first argument defines the *pattern* to search for. This is followed by a list of *files* to search through. The list is optional and **stdin** will be used if it is missing. The result of the procedures is a list containing the matches. Each match is a single element of the list and contains filename, number and contents of the matching line, separated by a colons.

::fileutil::install ?-m *mode*? *source destination*

The **install** command is similar in functionality to the **install** command found on many unix systems, or the shell script distributed with many source distributions (unix/install-sh in the Tcl sources, for example). It copies *source*, which can be either a file or directory to *destination*, which should be a directory, unless *source* is also a single file. The ?-m? option lets the user specify a unix-style mode (either octal or symbolic - see **file attributes**.

::fileutil::stripN *path n*

Removes the first *n* elements from the specified *path* and returns the modified path. If *n* is greater than the number of components in *path* an empty string is returned. The number of components in a given path may be determined by performing **llength** on the list returned by **file split**.

::fileutil::stripPwd *path*

If, and only if the *path* is inside of the directory returned by [**pwd**] (or the current working directory itself) it is made relative to that directory. In other words, the current working directory is stripped from the *path*. The possibly modified path is returned as the result of the command. If the current working directory itself was specified for *path* the result is the string "**.**".

::fileutil::stripPath *prefix path*

If, and only of the *path* is inside of the directory "*prefix*" (or the prefix directory itself) it is made relative to that directory. In other words, the prefix directory is stripped from the *path*. The possibly modified path is returned as the result of the command. If the prefix directory itself was specified for *path* the result is the string "**.**".

::fileutil::jail *jail path*

This command ensures that the *path* is not escaping the directory *jail*. It always returns an absolute path derived from *path* which is within *jail*.

If *path* is an absolute path and already within *jail* it is returned unmodified.

An absolute path outside of *jail* is stripped of its root element and then put into the *jail* by

prefixing it with it. The same happens if *path* is relative, except that nothing is stripped of it. Before adding the *jail* prefix the *path* is lexically normalized to prevent the caller from using**..** segments in *path* to escape the jail.

::fileutil::touch ?-a? ?-c? ?-m? ?-r *ref_file*? ?-t *time*? *filename* ?...?

Implementation of **touch**. Alter the atime and mtime of the specified files. If **-c**, do not create files if they do not already exist. If **-r**, use the atime and mtime from *ref_file*. If **-t**, use the integer clock value *time*. It is illegal to specify both **-r** and **-t**. If **-a**, only change the atime. If **-m**, only change the mtime.

This command is not available for Tcl versions less than 8.3.

::fileutil::tempdir

The command returns the path of a directory where the caller can place temporary files, such as "*/tmp*" on Unix systems. The algorithm we use to find the correct directory is as follows:

1. The directory set by an invokation of **::fileutil::tempdir** with an argument. If this is present it is tried exclusively and none of the following item are tried.
2. The directory named in the TMPDIR environment variable.
3. The directory named in the TEMP environment variable.
4. The directory named in the TMP environment variable.
5. A platform specific location:

   Windows

   "*C:\TEMP*", "*C:\TMP*", "*\TEMP*", and "*\TMP*" are tried in that order.

   (classic) Macintosh

   The TRASH_FOLDER environment variable is used. This is most likely not correct.

   Unix

   The directories "*/tmp*", "*/var/tmp*", and "*/usr/tmp*" are tried in that order.

The algorithm utilized is mainly that used in the Python standard library. The exception is the first item, the ability to have the search overridden by a user-specified directory.

::fileutil::tempdir *path*

In this mode the command sets the *path* as the first and only directory to try as a temp. directory. See the previous item for the use of the set directory. The command returns the empty string.

::fileutil::tempdirReset

Invoking this command clears the information set by the last call of [**::fileutil::tempdir** *path*]. See the last item too.

::fileutil::tempfile ?*prefix*?

The command generates a temporary file name suitable for writing to, and the associated file. The file name will be unique, and the file will be writable and contained in the appropriate system specific temp directory. The name of the file will be returned as the result of the command.

The code was taken from http://wiki.tcl.tk/772, attributed to Igor Volobouev and anon.

::fileutil::maketempdir ?-prefix *str*? ?-suffix *str*? ?-dir *str*?

The command generates a temporary directory suitable for writing to. The directory name will be unique, and the directory will be writable and contained in the appropriate system specific

temp directory. The name of the directory will be returned as the result of the command.

The three options can used to tweak the behaviour of the command:

-prefix str
: The initial, fixed part of the directory name. Defaults to **tmp** if not specified.

-suffix str
: The fixed tail of the directory. Defaults to the empty string if not specified.

-dir str
: The directory to place the new directory into. Defaults to the result of **fileutil::tempdir** if not specified.

The initial code for this was supplied by Miguel Martinez Lopez.

::fileutil::relative *base dst*
: This command takes two directory paths, both either absolute or relative and computes the path of *dst* relative to *base*. This relative path is returned as the result of the command. As implied in the previous sentence, the command is not able to compute this relationship between the arguments if one of the paths is absolute and the other relative.

  Note: The processing done by this command is purely lexical. Symbolic links are *not* taken into account.

::fileutil::relativeUrl *base dst*
: This command takes two file paths, both either absolute or relative and computes the path of *dst* relative to *base*, as seen from inside of the *base*. This is the algorithm how a browser resolves a relative link found in the currently shown file.

  The computed relative path is returned as the result of the command. As implied in the previous sentence, the command is not able to compute this relationship between the arguments if one of the paths is absolute and the other relative.

  Note: The processing done by this command is purely lexical. Symbolic links are *not* taken into account.

Warnings and incompatibilities

1.14.9

In this version **fileutil::find**'s broken system for handling symlinks was replaced with one working correctly and properly enumerating all the legal non-cyclic paths under a base directory.

While correct this means that certain pathological directory hierarchies with cross-linked sym-links will now take about O(n**2) time to enumerate whereas the original broken code managed O(n) due to its brokenness.

A concrete example and extreme case is the "*/sys*" hierarchy under Linux where some hundred devices exist under both "*/sys/devices*" and "*/sys/class*" with the two sub-hierarchies linking to the other, generating millions of legal paths to enumerate. The structure, reduced to three devices, roughly looks like

```
    /sys/class/tty/tty0 --> ../../dev/tty0
    /sys/class/tty/tty1 --> ../../dev/tty1
    /sys/class/tty/tty2 --> ../../dev/tty1

    /sys/dev/tty0/bus
    /sys/dev/tty0/subsystem --> ../../class/tty
    /sys/dev/tty1/bus
    /sys/dev/tty1/subsystem --> ../../class/tty
    /sys/dev/tty2/bus
    /sys/dev/tty2/subsystem --> ../../class/tty
```

The command **fileutil::find** currently has no way to escape this. When having to handle such a pathological hierarchy It is recommended to switch to package **fileutil::traverse** and the same-named command it provides, and then use the **-prefilter** option to prevent the traverser from following symbolic links, like so:

```
    package require fileutil::traverse

    proc NoLinks {fileName} {
        if {[string equal [file type $fileName] link]} {
            return 0
        }
        return 1
    }

    fileutil::traverse T /sys/devices -prefilter NoLinks
    T foreach p {
        puts $p
    }
    T destroy
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *fileutil* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Programming tools

multi

Keywords: multi, copy, file utilities, move, multi-file, remove

Name
fileutil::multi - Multi-file operation, scatter/gather, standard object

Synopsis
package require Tcl 8.5 9
package require fileutil::multi ?**0.2**?
package require fileutil::multi::op ?**0.5.4**?
package require wip ?**1.1**?
::fileutil::multi ?*word...*?

Description
This package provides a single command to perform actions on multiple files selected by glob patterns. It is a thin layer over the package **fileutil::multi::op** which provides objects for the same. This package simply creates a single such object and directs all file commands to it.

At the core is a domain specific language allowing the easy specification of multi-file copy and/or move and/or deletion operations. Alternate names would be scatter/gather processor, or maybe even assembler. For the detailed specification of this language, and examples, please see the documention for the package **fileutil::multi::op**.

Public API
The main command of the package is:

::fileutil::multi ?*word*...?
: This command interprets the specified words as file commands to execute. See the section **FILE API** of the documentation for the package **fileutil::multi::op** for the set of acceptable commands, their syntax, and semantics.

  The result of the command is the result generated by the last file command it executed.

Bugs, ideas, feedback
If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *fileutil* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Programming tools

multiop
Keywords: multiop, copy, file utilities, move, multi-file, remove

Name
fileutil::multi::op - Multi-file operation, scatter/gather

Synopsis

package require Tcl 8.5 9
package require fileutil::multi::op ?**0.5.4**?
package require wip ?**1.0**?
::fileutil::multi::op ?*opName*? ?*word...*?
opName *option* ?*arg arg* ...?
$opName do ?*word...*?
into *directory*
in *directory*
to *directory*
from *directory*
not *pattern*
for *pattern*
exclude *pattern*
but
except
as *name*
recursive
recursively
copy
move
remove
expand
invoke *cmdprefix*
reset
(
)
cd *directory*
up
for-windows
for-win
for-unix
the *pattern*
the-set *varname*
-> *varname*
strict
!strict
files
links
directories
dirs
all
state?
as?
excluded?
from?

into?
operation?
recursive?
strict?
type?

Description

This package provides objects which are able to perform actions on multiple files selected by glob patterns.

At the core is a domain specific language allowing the easy specification of multi-file copy and/or move and/or deletion operations. Alternate names would be scatter/gather processor, or maybe even assembler.

Class API

The main command of the package is:

::fileutil::multi::op ?*opName*? ?*word*...?

The command creates a new multi-file operation object with an associated global Tcl command whose name is *opName*. This command can be used to invoke the various possible file operations. It has the following general form:

opName *option* ?*arg arg ...*?

Option and the *arg*s determine the exact behavior of the command.

If the string **%AUTO%** is used as the *opName* then the package will generate a unique name on its own.

If one or more *word*s are specified they are interpreted as an initial set of file commands to execute. I.e. the method **do** of the newly constructed object is implicitly invoked using the words as its arguments.

Object API

The following methods are possible for multi-file operation objects:

$opName do ?*word*...?

This method interprets the specified words as file commands to execute. See the section File API for the set of acceptable commands, their syntax, and semantics.

The result of the method is the result generated by the last file command it executed.

File API

Both object constructor and method **do** take a list of words and interpret them as file commands to execute. The names were chosen to allow the construction of operations as sentences in near-natural language. Most of the commands influence just the state of the object, i.e. are simply providing the configuration used by the command triggering the actual action.

into *directory*

Specifies the destination directory for operations.

in *directory*

Alias for **into**.

to *directory*
Alias for **into**.

from *directory*
Specifies the source directory for operations.

not *pattern*
Specifies a glob pattern for paths to be excluded from the operation.

for *pattern*
Alias for **not**.

exclude *pattern*
Alias for **not**.

but
Has no arguments of its own, but looks ahead in the list of words and executes all **not** commands immediately following it. This allows the construction of "but not" and "but exclude" clauses for a more natural sounding specification of excluded paths.

except
A semi-alias for **but**. Has no arguments of its own, but looks ahead in the list of words and executes all **for** commands immediately following it. This allows the construction of "except for" clauses for a more natural sounding specification of excluded paths.

as *name*
Specifies a new name for the first file handled by the current operation. I.e. for the renaming of a single file during the operation.

recursive
Signals that file expansion should happen in the whole directory hierarchy and not just the directory itself.

recursively
An alias for **recursive**.

copy
Signals that the operation is the copying of files from source to destination directory per the specified inclusion and exclusion patterns.

move
Signals that the operation is the moving of files from source to destination directory per the specified inclusion and exclusion patterns.

remove
Signals that the operation is the removal of files in the destination directory per the specified inclusion and exclusion patterns.

expand
Signals that there is no operation but the calculation of the set of files from the include and exclude patterns. This operation is not available if **the-set** is used.

invoke *cmdprefix*
Signals that the user-specified command prefix *cmdprefix* is the operation to perform. The command prefix is executed at the global level and given the source directory, destination directory, and set of files (as dictionary mapping from source to destination files), in this order.

reset

Forces the object into the ground state where all parts of the configuration have default values.

(

Saves a copy of the current object state on a stack.

)

Takes the state at the top of the state stack and restores it, i.e. makes it the new current object state.

cd *directory*

Changes the destination directory to the sub-directory *directory* of the current destination.

up

Changes the destination directory to the parent directory of the current destination.

for-windows

Checks that Windows is the current platform. Aborts processing if not.

for-win

An alias for **for-windows**.

for-unix

Checks that Unix is the current platform. Aborts processing if not.

the *pattern*

This command specifies the files to operate on per a glob pattern, and is also the active element, i.e. the command which actually performs the specified operation. All the other commands only modified the object state to set the operation up, but di nothing else.

To allow for a more natural sounding syntax this command also looks ahead in the list of words looks and executes several commands immediately following it before performing its own actions. These commands are **as**, **but**, **exclude**, **except**, **from**, and **into** (and aliases). That way these commands act like qualifiers, and still take effect as if they had been written before this command.

After the operation has been performed the object state the exclude patterns and the alias name, if specified, are reset to their default values (i.e. empty), but nothing else.

the-set *varname*

Like **the**, however the set of files to use is not specified implicitly per a glob pattern, but contained and loaded from the specified variable. The operation **expand** is not available if this command is used.

-> *varname*

Saves the set of files from the last expansion into the specified variable.

strict

Make file expansion and definition of destination directory (**in** and aliases) strict, i.e. report errors for missing directories, and empty expansion.

!strict

Complement of **strict**. A missing destination directory or empty expansion are not reported as errors.

files

Limit the search to files. Default is to accept every type of path.

links
Limit the search to symbolic links. Default is to accept every type of path.

directories
Limit the search to directories. Default is to accept every type of path.

dirs
An alias for **directories**.

all
Accept all types of paths (default).

state?
Returns the current state of the object as dictionary. The dictionary keys and their meanings are:

as
Last setting made by **as**.

excluded
List of currently known exclusion patterns.

from
Current source directory, set by **from**.

into
Current destination directory, set by **into** (and aliases).

operation
Current operation to perform, set by **copy**, **move**, **remove**, **expand**, or **invoke**.

recursive
Current recursion status. Set/unset by **recursive** and**!recursive**.

strict
Current strictness. Set/unset by **strict** and**!strict**.

type
Current path type limiter. Set by either **files**, **directories**, **links**, or **all**.

as?
Returns the current alias name.

excluded?
Returns the current set of exclusion patterns.

from?
Returns the current source directory.

into?
Returns the current destination directory.

operation?
Returns the current operation to perform.

recursive?
Returns the current recursion status.

strict?
Returns the current strictness.

type?
Returns the current path type limiter.

Examples

The following examples assume that the variable **F** contains a reference to a multi-file operation object.

```
    $F do copy                       \
    the  *.dll                    \
    from c:/TDK/PrivateOpenSSL/bin \
    to   [installdir_of tls]
```

```
    $F do move      \
    the  *       \
    from /sources \
    into /scratch  \
    but not *.html

    # Alternatively use 'except for *.html'.
```

```
    $F do           \
    move         \
    the  index    \
    from /sources  \
    into /scratch   \
    as   pkgIndex.tcl
```

```
    $F do         \
    remove     \
    the *.txt  \
    in /scratch
```

Note that the fact that most commands just modify the object state allows us to use more off forms as specifications instead of just nearly-natural language sentences. For example the second example in this section can re-arranged into:

```
    $F do            \
    from /sources \
    into /scratch  \
    but not *.html \
    move           \
    the  *
```

and the result is not only still a valid specification, but even stays relatively readable.

Further note that the information collected by the commands **but**, **except**, and **as** is automatically reset after the associated **the** was executed. However no other state is reset in that manner, allowing the user to avoid repetitions of unchanging information. For example the second and third examples of this section can be merged and rewritten into the equivalent:

```
$F do                    \
    move                  \
    the  *                 \
    from /sources           \
    into /scratch            \
    but not *.html not index \
    the  index                \
    as   pkgIndex.tcl
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *fileutil* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Programming tools

paths

Keywords: paths

Name

fileutil::paths - Manage search path pools

Synopsis

package require Tcl 8.5 9
package require fileutil::paths ?**1.1**?
::fileutil::paths *poolName*
poolName method ?*arg arg* ...?
poolName **add** *path*
poolName **clear**
poolName **paths**
poolName **remove** *path*

Description

Provides a snit class whose instances manage a pool of (search) paths.

API

The main command provides construction of search path pools:

::fileutil::paths *poolName*

Creates a new, empty pool of search paths with an associated global Tcl command whose name is *poolName*. It may be used to invoke various operations on the pool. It has the following general form:

poolName method ?*arg arg ...*?
method and *arg*uments determine the exact behavior of the command.

If *poolName* is specified as **%AUTO%** a unique name will be generated by the package itself. The result of the command is the fully-qualified name of the instance command.

The following commands are possible for pool objects:

poolName* add *path
Adds the *path* to the pool. Nothing is done if the *path* is already known to the pool. The result of the command is the empty string.

***poolName* clear**
Clears the entire pool. In other words, removes all paths from it. The result of the command is the empty string.

***poolName* paths**
Returns the list of all paths known to the pool, in the order they were added.

poolName* remove *path
Removes the *path* from the pool, if it is known to the pool. Unknown paths are ignored without error. The result of the command is the empty string.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *fileutil* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

traverse

Keywords: traverse, directory traversal, traversal

Name

fileutil_traverse - Iterative directory traversal

Synopsis

package require Tcl 8.5 9
package require fileutil::traverse ?**0.7**?
package require fileutil
package require control
::fileutil::traverse ?*objectName*? *path* ?*option value...*?
$traverser command ?*arg arg* ...?
$traverser files
$traverser foreach *filevar script*
$traverser next *filevar*

Description

This package provides objects for the programmable traversal of directory hierarchies. The main command exported by the package is:

::fileutil::traverse ?*objectName*? *path* ?*option value*...?

The command creates a new traversal object with an associated global Tcl command whose name is *objectName*. This command may be used to invoke various operations on the traverser. If the string **%AUTO%** is used as the *objectName* then a unique name will be generated by the package itself.

Regarding the recognized options see section Options. Note that all these options can be set only during the creation of the traversal object. Changing them later is not possible and causes errors to be thrown if attempted.

The object command has the following general form:

$traverser command ?*arg arg ...*?

Command and its *arg*uments determine the exact behavior of the object.

The following commands are possible for traversal objects:

$traverser files

This method is the most highlevel one provided by traversal objects. When invoked it returns a list containing the names of all files and directories matching the current configuration of the traverser.

$traverser foreach *filevar script*

The highlevel **files** method (see above) is based on this mid-level method. When invoked it finds all files and directories matching per the current configuration and executes the *script* for each path. The current path under consideration is stored in the variable named by *filevar*. Both variable and script live / are executed in the context of the caller of the method. In the method files the script simply saves the found paths into the list to return.

$traverser next *filevar*

This is the lowest possible interface to the traverser, the core all higher methods are built on. When invoked it returns a boolean value indicating whether it found a path matching the current configuration (**True**), or not (**False**). If a path was found it is stored into the variable named by *filevar*, in the context of the caller.

The **foreach** method simply calls this method in a loop until it returned **False**. This method is exposed so that we are also able to incrementally traverse a directory hierarchy in an event-based manner.

Note that the traverser does follow symbolic links, except when doing so would cause it to enter a link-cycle. In other words, the command takes care to *not* lose itself in infinite loops upon encountering circular link structures. Note that even links which are not followed will still appear in the result.

Options

-prefilter command_prefix

This callback is executed for directories. Its result determines if the traverser recurses into the directory or not. The default is to always recurse into all directories. The callback is invoked with a single argument, the *absolute* path of the directory, and has to return a boolean value,

True when the directory passes the filter, and **False** if not.

-filter command_prefix
This callback is executed for all paths. Its result determines if the current path is a valid result, and returned by **next**. The default is to accept all paths as valid. The callback is invoked with a single argument, the *absolute* path to check, and has to return a boolean value, **True** when the path passes the filter, and **False** if not.

-errorcmd command_prefix
This callback is executed for all paths the traverser has trouble with. Like being unable to change into them, get their status, etc. The default is to ignore any such problems. The callback is invoked with a two arguments, the *absolute* path for which the error occured, and the error message. Errors thrown by the filter callbacks are handled through this callback too. Errors thrown by the error callback itself are not caught and ignored, but allowed to pass to the caller, i.e. however invoked the **next**. Any other results from the callback are ignored.

Warnings and incompatibilities

0.4.4

In this version the traverser's broken system for handling symlinks was replaced with one working correctly and properly enumerating all the legal non-cyclic paths under a base directory.

While correct this means that certain pathological directory hierarchies with cross-linked sym-links will now take about O(n**2) time to enumerate whereas the original broken code managed O(n) due to its brokenness.

A concrete example and extreme case is the "*/sys*" hierarchy under Linux where some hundred devices exist under both "*/sys/devices*" and "*/sys/class*" with the two sub-hierarchies linking to the other, generating millions of legal paths to enumerate. The structure, reduced to three devices, roughly looks like

```
    /sys/class/tty/tty0 --> ../../dev/tty0
    /sys/class/tty/tty1 --> ../../dev/tty1
    /sys/class/tty/tty2 --> ../../dev/tty1

    /sys/dev/tty0/bus
    /sys/dev/tty0/subsystem --> ../../class/tty
    /sys/dev/tty1/bus
    /sys/dev/tty1/subsystem --> ../../class/tty
    /sys/dev/tty2/bus
    /sys/dev/tty2/subsystem --> ../../class/tty
```

When having to handle such a pathological hierarchy it is recommended to use the **-prefilter** option to prevent the traverser from following symbolic links, like so:

```
    package require fileutil::traverse

    proc NoLinks {fileName} {
        if {[string equal [file type $fileName] link]} {
            return 0
```

```
        }
        return 1
    }

    fileutil::traverse T /sys/devices -prefilter NoLinks
    T foreach p {
        puts $p
    }
    T destroy
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *fileutil* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Programming tools

ftp

ftp

Keywords: ftp, internet, net, rfc 959

Name

ftp - Client-side tcl implementation of the ftp protocol

Synopsis

package require Tcl 8.5 9
package require ftp ?**2.4.14**?
::ftp::Open *server user passwd* ?*options*?
::ftp::Close *handle*
::ftp::Cd *handle directory*
::ftp::Pwd *handle*
::ftp::Type *handle* ?**ascii|binary|tenex**?
::ftp::List *handle* ?*pattern*?
::ftp::NList *handle* ?*directory*?
::ftp::FileSize *handle file*
::ftp::ModTime *handle file*
::ftp::Delete *handle file*
::ftp::Rename *handle from to*
::ftp::Put *handle* (*local* | -data *data* | -channel *chan*) ?*remote*?

::ftp::Append *handle* (*local* | -data *data* | -channel *chan*) ?*remote*?
::ftp::Get *handle remote* ?(local | -variable *varname* | -channel *chan)?*
::ftp::Reget *handle remote* ?*local*? ?*from*? ?*to*?
::ftp::Newer *handle remote* ?*local*?
::ftp::MkDir *handle directory*
::ftp::RmDir *handle directory*
::ftp::Quote *handle arg1 arg2 ...*
::ftp::DisplayMsg *handle msg* ?*state*?

Description

The ftp package provides the client side of the ftp protocol as specified in RFC 959 (http://www.rfc-editor.org/rfc/rfc959.txt). The package implements both active (default) and passive ftp sessions.

A new ftp session is started with the **::ftp::Open** command. To shutdown an existing ftp session use **::ftp::Close**. All other commands are restricted to usage in an an open ftp session. They will generate errors if they are used out of context. The ftp package includes file and directory manipulating commands for remote sites. To perform the same operations on the local site use commands built into the core, like **cd** or **file**.

The output of the package is controlled by two state variables, **::ftp::VERBOSE** and **::ftp::DEBUG**. Setting **::ftp::VERBOSE** to "1" forces the package to show all responses from a remote server. The default value is "0". Setting **::ftp::DEBUG** to "1" enables debugging and forces the package to show all return codes, states, state changes and "real" ftp commands. The default value is "0".

The command **::ftp::DisplayMsg** is used to show the different messages from the ftp session. The setting of **::ftp::VERBOSE** determines if this command is called or not. The current implementation of the command uses the **log** package of tcllib to write the messages to their final destination. This means that the behaviour of **::ftp::DisplayMsg** can be customized without changing its implementation. For more radical changes overwriting its implementation by the application is of course still possible. Note that the default implementation honors the option **-output** to **::ftp::Open** for a session specific log command.

Caution: The default implementation logs error messages like all other messages. If this behaviour is changed to throwing an error instead all commands in the API will change their behaviour too. In such a case they will not return a failure code as described below but pass the thrown error to their caller.

API

::ftp::Open *server user passwd* ?*options*?

This command is used to start a FTP session by establishing a control connection to the FTP server. The defaults are used for any option not specified by the caller.

The command takes a host name *server*, a user name *user* and a password *password* as its parameters and returns a session handle that is an integer number greater than or equal to "0", if the connection is successfully established. Otherwise it returns "-1". The *server* parameter must be the name or internet address (in dotted decimal notation) of the ftp server to connect to. The *user* and *passwd* parameters must contain a valid user name and password to complete the login process.

The options overwrite some default values or set special abilities:

-blocksize *size*
: The blocksize is used during data transfer. At most *size* bytes are transfered at once. The default value for this option is 4096. The package will evaluate the **-progress callback** for the session after the transfer of each block.

-timeout *seconds*
: If *seconds* is non-zero, then **::ftp::Open** sets up a timeout which will occur after the specified number of seconds. The default value is 600.

-port *number*
: The port *number* specifies an alternative remote port on the ftp server on which the ftp service resides. Most ftp services listen for connection requests on the default port 21. Sometimes, usually for security reasons, port numbers other than 21 are used for ftp connections.

-mode *mode*
: The transfer *mode* option determines if a file transfer occurs in **active** or **passive** mode. In passive mode the client will ask the ftp server to listen on a data port and wait for the connection rather than to initiate the process by itself when a data transfer request comes in. Passive mode is normally a requirement when accessing sites via a firewall. The default mode is **active**.

-progress *callback*
: This *callback* is evaluated whenever a block of data was transfered. See the option **-blocksize** for how to specify the size of the transfered blocks.

  When evaluating the *callback* one argument is appended to the callback script, the current accumulated number of bytes transferred so far.

-command *callback*
: Specifying this option places the connection into asynchronous mode. The *callback* is evaluated after the completion of any operation. When an operation is running no further operations must be started until a callback has been received for the currently executing operation.

  When evaluating the *callback* several arguments are appended to the callback script, namely the keyword of the operation that has completed and any additional arguments specific to the operation. If an error occurred during the execution of the operation the callback is given the keyword **error**.

-output *callback*
: This option has no default. If it is set the default implementation of **::ftp::DisplayMsg** will use its value as command prefix to log all internal messages. The callback will have three arguments appended to it before evaluation, the id of the session, the message itself, and the connection state, in this order.

::ftp::Close *handle*
: This command terminates the specified ftp session. If no file transfer is in progress, the server will close the control connection immediately. If a file transfer is in progress however, the control connection will remain open until the transfers completes. When that happens the server will write the result response for the transfer to it and close the connection afterward.

::ftp::Cd *handle directory*
This command changes the current working directory on the ftp server to a specified target *directory*. The command returns 1 if the current working directory was successfully changed to the specified directory or 0 if it fails. The target directory can be
- a subdirectory of the current directory,
- Two dots,**..** (as an indicator for the parent directory of the current directory)
- or a fully qualified path to a new working directory.

::ftp::Pwd *handle*
This command returns the complete path of the current working directory on the ftp server, or an empty string in case of an error.

::ftp::Type *handle* ?ascii|binary|tenex?
This command sets the ftp file transfer type to either **ascii**, **binary**, or **tenex**. The command always returns the currently set type. If called without type no change is made.

Currently only **ascii** and **binary** types are supported. There is some early (alpha) support for Tenex mode. The type **ascii** is normally used to convert text files into a format suitable for text editors on the platform of the destination machine. This mainly affects end-of-line markers. The type **binary** on the other hand allows the undisturbed transfer of non-text files, such as compressed files, images and executables.

::ftp::List *handle* ?*pattern*?
This command returns a human-readable list of files. Wildcard expressions such as "**.tcl*" are allowed. If *pattern* refers to a specific directory, then the contents of that directory are returned. If the *pattern* is not a fully-qualified path name, the command lists entries relative to the current remote directory. If no *pattern* is specified, the contents of the current remote directory is returned.

The listing includes any system-dependent information that the server chooses to include. For example most UNIX systems produce output from the command **ls -l**. The command returns the retrieved information as a tcl list with one item per entry. Empty lines and UNIX's "total" lines are ignored and not included in the result as reported by this command.

If the command fails an empty list is returned.

::ftp::NList *handle* ?*directory*?
This command has the same behavior as the **::ftp::List** command, except that it only retrieves an abbreviated listing. This means only file names are returned in a sorted list.

::ftp::FileSize *handle file*
This command returns the size of the specified *file* on the ftp server. If the command fails an empty string is returned.

ATTENTION! It will not work properly when in ascii mode and is not supported by all ftp server implementations.

::ftp::ModTime *handle file*
This command retrieves the time of the last modification of the *file* on the ftp server as a system dependent integer value in seconds or an empty string if an error occurred. Use the built-in command **clock** to convert the retrieves value into other formats.

::ftp::Delete *handle file*
This command deletes the specified *file* on the ftp server. The command returns 1 if the specified file was successfully deleted or 0 if it failed.

::ftp::Rename *handle from to*
This command renames the file *from* in the current directory of the ftp server to the specified new file name *to*. This new file name must not be the same as any existing subdirectory or file name. The command returns 1 if the specified file was successfully renamed or 0 if it failed.

::ftp::Put *handle* (*local* | -data *data* | -channel *chan*) ?*remote*?
This command transfers a local file *local* to a remote file *remote* on the ftp server. If the file parameters passed to the command do not fully qualified path names the command will use the current directory on local and remote host. If the remote file name is unspecified, the server will use the name of the local file as the name of the remote file. The command returns 1 to indicate a successful transfer and 0 in the case of a failure.

If **-data** *data* is specified instead of a local file, the system will not transfer a file, but the *data* passed into it. In this case the name of the remote file has to be specified.

If **-channel** *chan* is specified instead of a local file, the system will not transfer a file, but read the contents of the channel *chan* and write this to the remote file. In this case the name of the remote file has to be specified. After the transfer *chan* will be closed.

::ftp::Append *handle* (*local* | -data *data* | -channel *chan*) ?*remote*?
This command behaves like **::ftp::Puts**, but appends the transfered information to the remote file. If the file did not exist on the server it will be created.

::ftp::Get *handle remote* ?(*local* | -variable *varname* | -channel *chan*)?
This command retrieves a remote file *remote* on the ftp server and stores its contents into the local file *local*. If the file parameters passed to the command are not fully qualified path names the command will use the current directory on local and remote host. If the local file name is unspecified, the server will use the name of the remote file as the name of the local file. The command returns 1 to indicate a successful transfer and 0 in the case of a failure. The command will throw an error if the directory the file *local* is to be placed in does not exist.

If **-variable** *varname* is specified, the system will store the retrieved data into the variable *varname* instead of a file.

If **-channel** *chan* is specified, the system will write the retrieved data into the channel *chan* instead of a file. The system will *not* close *chan* after the transfer, this is the responsibility of the caller to **::ftp::Get**.

::ftp::Reget *handle remote* ?*local*? ?*from*? ?*to*?
This command behaves like **::ftp::Get**, except that if local file *local* exists and is smaller than remote file *remote*, the local file is presumed to be a partially transferred copy of the remote file and the transfer is continued from the apparent point of failure. The command will throw an error if the directory the file *local* is to be placed in does not exist. This command is useful when transferring very large files over networks that tend to drop connections.

Specifying the additional byte offsets *from* and *to* will cause the command to change its behaviour and to download exactly the specified slice of the remote file. This mode is possible

only if a local destination is explicitly provided. Omission of *to* leads to downloading till the end of the file.

::ftp::Newer *handle remote* ?*local*?
This command behaves like **::ftp::Get**, except that it retrieves the remote file only if the modification time of the remote file is more recent than the file on the local system. If the file does not exist on the local system, the remote file is considered newer. The command will throw an error if the directory the file *local* is to be placed in does not exist.

::ftp::MkDir *handle directory*
This command creates the specified *directory* on the ftp server. If the specified path is relative the new directory will be created as a subdirectory of the current working directory. Else the created directory will have the specified path name. The command returns 1 to indicate a successful creation of the directory and 0 in the case of a failure.

::ftp::RmDir *handle directory*
This command removes the specified directory on the ftp server. The remote directory has to be empty or the command will fail. The command returns 1 to indicate a successful removal of the directory and 0 in the case of a failure.

::ftp::Quote *handle arg1 arg2...*
This command is used to send an arbitrary ftp command to the server. It cannot be used to obtain a directory listing or for transferring files. It is included to allow an application to execute commands on the ftp server which are not provided by this package. The arguments are sent verbatim, i.e. as is, with no changes.

In contrast to the other commands in this package this command will not parse the response it got from the ftp server but return it verbatim to the caller.

::ftp::DisplayMsg *handle msg* ?*state*?
This command is used by the package itself to show the different messages from the ftp sessions. The package itself declares this command very simple, writing the messages to **stdout** (if **::ftp::VERBOSE** was set, see below) and throwing tcl errors for error messages. It is the responsibility of the application to overwrite it as needed. A state variable for different states assigned to different colors is recommended by the author. The package **log** is useful for this.

::ftp::VERBOSE
A state variable controlling the output of the package. Setting **::ftp::VERBOSE** to "1" forces the package to show all responses from a remote server. The default value is "0".

::ftp::DEBUG
A state variable controlling the output of ftp. Setting **::ftp::DEBUG** to "1" enables debugging and forces the package to show all return codes, states, state changes and "real" ftp commands. The default value is "0".

Bugs

The correct execution of many commands depends upon the proper behavior by the remote server, network and router configuration.

An update command placed in the procedure **::ftp::DisplayMsg** may run into persistent errors or infinite loops. The solution to this problem is to use **update idletasks** instead of **update**.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *ftp* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Networking

ftp_geturl

Keywords: ftp_geturl, ftp, internet, net, rfc 959

Name

ftp::geturl - Uri handler for ftp urls

Synopsis

package require Tcl 8.5 9
package require ftp::geturl ?**0.2.3**?
::ftp::geturl *url*

Description

This package provides a command which wraps around the client side of the *ftp* protocol provided by package **ftp** to allow the retrieval of urls using the *ftp* schema.

API

::ftp::geturl ***url***

This command can be used by the generic command **::uri::geturl** (See package **uri**) to retrieve the contents of ftp urls. Internally it uses the commands of the package **ftp** to fulfill the request.

The contents of a *ftp* url are defined as follows:

file
: The contents of the specified file itself.

directory
: A listing of the contents of the directory in key value notation where the file name is the key and its attributes the associated value.

link
: The attributes of the link, including the path it refers to.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *ftp* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Networking

ftpd

ftpd
Keywords: ftpd, ftp, ftpserver, rfc 959, services

Name
ftpd - Tcl FTP server implementation

Synopsis
package require Tcl 8.5 9
package require ftpd ?**1.4.1**?
::ftpd::server ?*myaddr*?
::ftpd::config ?*option value*? ?*option value* ...?
fsCmd **append** *path*
fsCmd **delete** *path channel*
fsCmd **dlist** *path style channel*
fsCmd **exists** *path*
fsCmd **mkdir** *path channel*
fsCmd **mtime** *path channel*
fsCmd **permissions** *path*
fsCmd **rename** *path newpath channel*
fsCmd **retr** *path*
fsCmd **rmdir** *path channel*
fsCmd **size** *path channel*
fsCmd **store** *path*

Description
The **ftpd** package provides a simple Tcl-only server library for the FTP protocol as specified in RFC 959 (http://www.rfc-editor.org/rfc/rfc959.txt). It works by listening on the standard FTP socket. Most server errors are returned as error messages with the appropriate code attached to them. Since the server code for the ftp daemon is executed in the event loop, it is possible that a **bgerror** will be thrown on the server if there are problems with the code in the module.

Commands
::ftpd::server ?*myaddr*?
Open a listening socket to listen to and accept ftp connections. myaddr is an optional argument. *myaddr* is the domain-style name or numerical IP address of the client-side network interface to use for the connection.

::ftpd::config ?*option value*? ?*option value ...*?

The value is always the name of the command to call as the callback. The option specifies which callback should be configured. See section Callbacks for descriptions of the arguments and return values for each of the callbacks.

-authIpCmd *proc*

Callback to authenticate new connections based on the ip-address of the peer.

-authUsrCmd *proc*

Callback to authenticate new connections based on the user logging in (and the users password).

-authFileCmd *proc*

Callback to accept or deny a users access to read and write to a specific path or file.

-logCmd *proc*

Callback for log information generated by the FTP engine.

-fsCmd *proc*

Callback to connect the engine to the filesystem it operates on.

-closeCmd *proc*

Callback to be called when a connection is closed. This allows the embedding application to perform its own cleanup operations.

-xferDoneCmd *proc*

Callback for transfer completion notification. In other words, it is called whenever a transfer of data to or from the client has completed.

Callbacks

authIpCmd callback

The authIpCmd receives the ip-address of the peer attempting to connect to the ftp server as its argument. It returns a 1 to allow users from the specified IP to attempt to login and a 0 to reject the login attempt from the specified IP.

authUsrCmd callback

The authUsrCmd receives the username and password as its two arguments. It returns a 1 to accept the attempted login to the ftpd and a 0 to reject the attempted login.

authFileCmd callback

The authFileCmd receives the user (that is currently logged in), the path or filename that is about to be read or written, and **read** or **write** as its three arguments. It returns a 1 to allow the path or filename to be read or written, and a 0 to reject the attempted read or write with a permissions error code.

logCmd callback

The logCmd receives a severity and a message as its two arguments. The severities used within the ftpd package are **note**, **debug**, and **error**. The logCmd doesn't return anything.

fsCmd callback

The fsCmd receives a subcommand, a filename or path, and optional additional arguments (depending on the subcommand).

The subcommands supported by the fsCmd are:

fsCmd* append *path

The append subcommand receives the filename to append to as its argument. It returns a

writable tcl channel as its return value.

fsCmd* delete *path channel

The delete subcommand receives the filename to delete, and a channel to write to as its two arguments. The file specified is deleted and the appropriate ftp message is written to the channel that is passed as the second argument. The delete subcommand returns nothing.

fsCmd* dlist *path style channel

The dlist subcommand receives the path that it should list the files that are in, the style in which the files should be listed which is either **nlst** or **list**, and a channel to write to as its three arguments. The files in the specified path are printed to the specified channel one per line. If the style is **nlst** only the name of the file is printed to the channel. If the style is **list** then the file permissions, number of links to the file, the name of the user that owns the file, the name of the group that owns the file, the size (in bytes) of the file, the modify time of the file, and the filename are printed out to the channel in a formatted space separated format. The **dlist** subcommand returns nothing.

fsCmd* exists *path

The exists subcommand receives the name of a file to check the existence of as its only argument. The exists subcommand returns a 1 if the path specified exists and the path is not a directory.

fsCmd* mkdir *path channel

The mkdir subcommand receives the path of a directory to create and a channel to write to as its two arguments. The mkdir subcommand creates the specified directory if necessary and possible. The mkdir subcommand then prints the appropriate success or failure message to the channel. The mkdir subcommand returns nothing.

fsCmd* mtime *path channel

The mtime subcommand receives the path of a file to check the modify time on and a channel as its two arguments. If the file exists the mtime is printed to the channel in the proper FTP format, otherwise an appropriate error message and code are printed to the channel. The mtime subcommand returns nothing.

fsCmd* permissions *path

The permissions subcommand receives the path of a file to retrieve the permissions of. The permissions subcommand returns the octal file permissions of the specified file. The file is expected to exist.

fsCmd* rename *path newpath channel

The rename subcommand receives the path of the current file, the new file path, and a channel to write to as its three arguments. The rename subcommand renames the current file to the new file path if the path to the new file exists, and then prints out the appropriate message to the channel. If the new file path doesn't exist the appropriate error message is printed to the channel. The rename subcommand returns nothing.

fsCmd* retr *path

The retr subcommand receives the path of a file to read as its only argument. The retr subcommand returns a readable channel that the specified file can be read from.

fsCmd* rmdir *path channel

The rmdir subcommand receives the path of a directory to remove and a channel to write to as its two arguments. The rmdir subcommand removes the specified directory (if

possible) and prints the appropriate message to the channel (which may be an error if the specified directory does not exist or is not empty). The rmdir subcommand returns nothing.

fsCmd* size *path channel
The size subcommand receives the path of a file to get the size (in bytes) of and a channel to write to as its two arguments. The size subcommand prints the appropriate code and the size of the file if the specified path is a file, otherwise an appropriate error code and message are printed to the channel. The size subcommand returns nothing.

fsCmd* store *path
The store subcommand receives the path of a file to write as its only argument. The store subcommand returns a writable channel.

closeCmd
The **closeCmd** receives no arguments when it is invoked, and any return value it may generate is discarded.

xferDoneCmd sock sock2 file bytes filename err
The **xferDoneCmd** receives six arguments when invoked. These are, in this order, the channel handle of the control socket for the connection, the channel handle of the data socket used for the transfer (already closed), the handle of the channel containing the transfered file, the number of bytes transfered, the path of the file which was transfered, and a (possibly empty) error message. Any return value it may generate is discarded.

Variables

::ftpd::cwd
The current working directory for a session when someone first connects to the FTPD or when the **REIN** ftp command is received.

::ftpd::contact
The e-mail address of the person that is the contact for the ftp server. This address is printed out as part of the response to the **FTP HELP** command.

::ftpd::port
The port that the ftp server should listen on. If port is specified as zero, the operating system will allocate an unused port for use as a server socket; afterwards, the variable will contain the port number that was allocated.

::ftpd::welcome
The message that is printed out when the user first connects to the ftp server.

::ftpd::CurrentSocket
Accessible to all callbacks and all filesystem commands (which are a special form of callback) and contains the handle of the socket channel which was active when the callback was invoked.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *ftpd* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be

made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Networking

fumagic

cfront
Keywords: cfront, file recognition, file type, file utilities, mime, type

Name
fileutil::magic::cfront - Generator core for compiler of magic(5) files

Synopsis
package require Tcl 8.5 9
package require fileutil::magic::cfront ?**1.3.3**?
package require fileutil::magic::cgen ?**1.2.0**?
package require fileutil::magic::rt ?**1.2.0**?
package require struct::list
package require fileutil
::fileutil::magic::cfront::compile *path...*
::fileutil::magic::cfront::generate ?*compressed*?
boolean? -- *path...*
::fileutil::magic::cfront::install *path...*

Description
This package provides the frontend of a compiler of magic(5) files into recognizers based on the **fileutil::magic::rt** recognizer runtime package. For the generator backed used by this compiler see the package **fileutil::magic::cgen**.

Commands
::fileutil::magic::cfront::compile *path*...
This command takes the paths of one or more files and directories and compiles all the files, and the files in all the directories into a single analyzer for all the file types specified in these files. It returns a list whose first item is a list per-file dictionaries of analyzer scripts and whose second item is a list of analyzer commands.

All the files have to be in the format specified by magic(5).

The result of the command is a Tcl script containing the generated recognizer.

::fileutil::magic::cfront::generate ?*compressed*
boolean? -- *path*... This command behaves like **::fileutil::magic::cfront::compile** with regard to the specified path arguments, then wraps the resulting recognizer script into a variable named **named** and a procedure named **analyzer**, and returns the resulting script.

Argument *compressed* defaults to **boolean true**, meaning that whitespace is minimised in the script that the command returns.

The argument -- is always required.

::fileutil::magic::cfront::install *path*...
This command behaves like **::fileutil::magic::cfront::compile** with regard to the specified path arguments. It uses **::fileutil::magic::cfront::generate** to compile a recognizer script, and evaluates this script in the current interpreter so that it will be used by command ::fileutil::magic::filetype.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *fileutil :: magic* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Programming tools

cgen

Keywords: cgen, file recognition, file type, file utilities, mime, type

Name

fileutil::magic::cgen - Generator core for compiler of magic(5) files

Synopsis

package require Tcl 8.5 9
package require fileutil::magic::cgen ?**1.2.0**?
package require fileutil::magic::rt ?**1.2.0**?
package require struct::tree
package require struct::list
::fileutil::magic::cgen::2tree *script*
::fileutil::magic::cgen::treedump *tree*
::fileutil::magic::cgen::treegen *tree node*

Description

This package provides the generator backend for a compiler of magic(5) files into recognizers based on the **fileutil::magic::rt** recognizer runtime package. For the compiler frontend using this generator see the package **fileutil::magic::cfront**.

Commands

::fileutil::magic::cgen::2tree *script*
This command converts the recognizer specified by the *script* into a tree and returns the

object command of that tree as its result. It uses the package **struct::tree** for the tree.

The *script* is in the format specified by magic(5).

::fileutil::magic::cgen::treedump *tree*
This command takes a *tree* as generated by **::fileutil::magic::cgen::2tree** and returns a string encoding the tree for human consumption, to aid in debugging.

::fileutil::magic::cgen::treegen *tree node*
This command takes a *tree* as generated by **::fileutil::magic::cgen::2tree** and returns a Tcl script, the recognizer for the file types represented by the sub-tree rooted at the *node*. The generated script makes extensive use of the commands provided by the recognizer runtime package **fileutil::magic::rt** to perform its duties.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *fileutil :: magic* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Programming tools

filetypes

Keywords: filetypes, file recognition, file type, file utilities, type

Name

fileutil::magic::filetype - Procedures implementing file-type recognition

Synopsis

package require Tcl 8.6 9
package require fileutil::magic::filetype ?**2.0**?
::fileutil::magic::filetype *filename*

Description

This package provides a command for the recognition of file types in pure Tcl.

The core part of the recognizer was generated from a "magic(5)" file containing the checks to perform to recognize files, and associated file-types.

Beware! This recognizer is large, about 752 Kilobyte of generated Tcl code.

::fileutil::magic::filetype *filename*
This command is similar to the command **fileutil::fileType**.

Returns a list containing a list of descriptions, a list of mimetype components, and a list file extensions. Returns an empty string if the file content is not recognized.

References

1. File(1) sources This site contains the current sources for the file command, including the magic definitions used by it. The latter were used by us to generate this recognizer.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *fileutil :: magic* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Programming tools

rtcore

Keywords: rtcore, file recognition, file type, file utilities, mime, type

Name

fileutil::magic::rt - Runtime core for file type recognition engines written in pure Tcl

Synopsis

package require Tcl 8.5 9
package require fileutil::magic::rt ?**2.0**?
::fileutil::magic::rt::>
::fileutil::magic::rt::<
::fileutil::magic::rt::new *chan named analyze*
::fileutil::magic::rt::file_start *name*
::fileutil::magic::rt::emit *msg*
::fileutil::magic::rt::O *where*
::fileutil::magic::rt::R *where*
::fileutil::magic::rt::Nv *type offset compinvert comp expected*
::fileutil::magic::rt::N *type offset testinvert compinvert mod mand comp expected*
::fileutil::magic::rt::S *type offset testinvert mod mand comp val*
::fileutil::magic::rt::L *newlevel*
::fileutil::magic::rt::I *offset it ioi ioo iir io*
::fileutil::magic::rt::R *offset*
::fileutil::magic::rt::U *fileindex name*

Description

This package provides the runtime core for file type recognition engines written in pure Tcl and is thus used by all other packages in this module such as **fileutil::magic::filetype** and the two compiler packages **fileutil::magic::cgen** and **fileutil::magic::cfront**.

Commands

::fileutil::magic::rt::>
Increment the level and perform related housekeeping

::fileutil::magic::rt::<
Decrement the level and perform related housekeeping

::fileutil::magic::rt::new *chan named analyze*
Create a new command which returns one description of the file each time it is called, and a code of *break* when there are no more descriptions. *chan* is the channel containing the data to describe. The channel configuration is then managed as needed. *named* is a dictionary of named tests, as generated by **fileutil::magic::cfront::compile**. *test* is a command prefix for a routine composed of the list of commands as returned by **fileutil::magic::cfront::compile** .

::fileutil::magic::rt::file_start *name*
This command marks the start of a magic file when debugging. It returns the empty string as its result.

::fileutil::magic::rt::emit *msg*
This command adds the text *msg* to the result buffer. The message may contain the following special character sequences. They will be replaced with buffered values before the message is added to the result. The command returns the empty string as its result.

\b
This sequence is removed

%s
Replaced with the last buffered string value.

%ld
Replaced with the last buffered numeric value.

%d
See above.

${x:...?...}
Substitute one string if the file is executable, and another string otherwise.

::fileutil::magic::rt::O *where*
Produce an offset from *where*, relative to the cursor one level up. Produce an offset from *where*, relative to the offset one level up.

::fileutil::magic::rt::Nv *type offset compinvert comp expected*
A limited form of **::fileutile::magic::rt::N** that only checks for equality and can't be told to invert the test.

::fileutil::magic::rt::N *type offset testinvert compinvert mod mand comp expected*
Fetch the numeric value with *type* from the absolute location *offset*, compare it with *expected* using *comp* as the comparision operator, and returns the result.

The argument *comp* must be one of Tcl's comparison operators.

```
    <comp> <fetched-and-masked-value> <comp> <expected>
```

The special comparison operator **x** signals that no comparison should be done, or, in other words, that the fetched value will always match *val*.

::fileutil::magic::rt::S *type offset testinvert mod mand comp val*

Like **::fileutil::magic::rt::N** except that it fetches and compares string types, not numeric data.

::fileutil::magic::rt::L *newlevel*
Sets the current level in the calling context to *newlevel*. The command returns the empty string as its result.

::fileutil::magic::rt::I *offset it ioi ioo iir io*
Calculates an offset based on an initial offset and the provided modifiers.

::fileutil::magic::rt::R *offset*
Given an initial offset, calculates an offset relative to the cursor at the next level up. The cursor is the position in the data one character after the data extracted from the file one level up.

::fileutil::magic::rt::U *fileindex name*
Add a level and use a named test script.

Numeric types

byte
8-bit integer

short
16-bit integer, stored in native endianess

beshort
see above, stored in big endian

leshort
see above, stored in small/little endian

long
32-bit integer, stored in native endianess

belong
see above, stored in big endian

lelong
see above, stored in small/little endian

All of the types above exit in an unsigned form as well. The type names are the same, with the character "u" added as prefix.

date
32-bit integer timestamp, stored in native endianess

bedate
see above, stored in big endian

ledate
see above, stored in small/little endian

ldate
32-bit integer timestamp, stored in native endianess

beldate
see above, stored in big endian

leldate
see above, stored in small/little endian

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *fileutil :: magic* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Programming tools

generator

generator

Keywords: generator, control structure, coroutine, filter, foldl, foldr, foreach, iterator, map, reduce, scanl

Name

generator - Procedures for creating and using generators.

Synopsis

package require Tcl 8.6 9
package require generator ?**0.3**?
generator define *name params body*
generator yield *arg* ?*args..*?
generator foreach *varList generator varList generator* ?*...*? *body*
generator next *generator* ?*varName..*?
generator exists *generator*
generator names
generator destroy ?*generator..*?
generator finally *cmd* ?*arg..*?
generator from *format value*
generator to *format generator*
generator map *function generator*
generator filter *predicate generator*
generator reduce *function zero generator*
generator foldl *function zero generator*
generator foldr *function zero generator*
generator all *predicate generator*
generator and *generator*
generator any *generator*
generator concat *generator* ?*generator..*?
generator concatMap *function generator*
generator drop *n generator*
generator dropWhile *predicate generator*

generator contains *element generator*
generator foldl1 *function generator*
generator foldli *function zero generator*
generator foldri *function zero generator*
generator head *generator*
generator tail *generator*
generator init *generator*
generator takeList *n generator*
generator take *n generator*
generator iterate *function init*
generator last *generator*
generator length *generator*
generator or *predicate generator*
generator product *generator*
generator repeat *n value..*
generator sum *generator*
generator takeWhile *predicate generator*
generator splitWhen *predicate generator*
generator scanl *function zero generator*

Description

The **generator** package provides commands to define and iterate over generator expressions. A *generator* is a command that returns a sequence of values. However, unlike an ordinary command that returns a list, a generator *yields* each value and then suspends, allowing subsequent values to be fetched on-demand. As such, generators can be used to efficiently iterate over a set of values, without having to generate all answers in-memory. Generators can be used to iterate over elements of a data structure, or rows in the result set of a database query, or to decouple producer/consumer software designs such as parsers and tokenizers, or to implement sophisticated custom control strategies such as backtracking search. Generators reduce the need to implement custom control structures, as many such structures can be recast as generators, leading to both a simpler implementation and a more standardised interface. The generator mechanism is built on top of the Tcl 8.6 coroutine mechanism.

The package exports a single ensemble command, **generator**. All functionality is provided as subcommands of this command. The core subcommands of the package are **define**, **yield**, and **foreach**. The **define** command works like Tcl's **proc** command, but creates a generator procedure; that is, a procedure that returns a generator when called. The generator itself is a command that can be called multiple times: each time it returns the next value in the generated series. When the series has been exhausted, the generator command returns an empty list and then destroys itself. Rather than manually call a generator, however, the package also provides a flexible **foreach** command that loops through the values of one or more generators. This loop construct mimicks the functionality of the built-in Tcl **foreach** command, including handling multiple return values and looping over multiple generators at once. Writing a generator is also a simple task, much like writing a normal procedure: simply use the **define** command to define the generator, and then call **yield** instead of **return**. For example, we can define a generator for looping through the integers in a particular range:

```
    generator define range {n m} {
        for {set i $n} {$i <= $m} {incr i} { generator yield $i }
    }
    generator foreach x [range 1 10] {
        puts "x = $x"
    }
```

The above example will print the numbers from 1 to 10 in sequence, as you would expect. The difference from a normal loop over a list is that the numbers are only generated as they are needed. If we insert a break into the loop then any remaining numbers in the sequence would never be generated. To illustrate, we can define a generator that produces the sequence of natural numbers: an infinite series. A normal procedure would never return trying to produce this series as a list. By using a generator we only have to generate those values which are actually used:

```
    generator define nats {} {
        while 1 { generator yield [incr nat] }
    }
    generator foreach n [nats] {
        if {$n > 100} { break }
    }
```

Commands

generator define *name params body*

Creates a new generator procedure. The arguments to the command are identical to those for **proc**: a *name*, a list of parameters, and a body. The parameter list format is identical to a procedure. In particular, default values and the ?args? syntax can be used as usual. Each time the resulting generator procedure is called it creates a new generator command (coroutine) that will yield a list of values on each call. Each result from a generator is guaranteed to be a non-empty list of values. When a generator is exhausted it returns an empty list and then destroys itself to free up resources. It is an error to attempt to call an exhausted generator as the command no longer exists.

generator yield *arg ?args..?*

Used in the definition of a generator, this command returns the next set of values to the consumer. Once the **yield** command has been called the generator will suspend to allow the consumer to process that value. When the next value is requested, the generator will resume as if the yield command had just returned, and can continue processing to yield the next result. The **yield** command must be called with at least one argument, but can be called with multiple arguments, in which case this is equivalent to calling **yield** once for each argument.

generator foreach *varList generator varList generator ?...? body*

Loops through one or more generators, assigning the next values to variables and then executing the loop body. Works much like the built-in **foreach** command, but working with generators rather than lists. Multiple generators can be iterated over in parallel, and multiple results can be retrieved from a single generator at once. Like the built-in **foreach**, the loop will continue until all of the generators have been exhausted: variables for generators that are exhausted early will be set to the empty string.

The **foreach** command will automatically clean-up all of the generators at the end of the loop, regardless of whether the loop terminated early or not. This behaviour is provided as a convenience to avoid having to explicitly clean up a generator in the usual cases. Generators can however be destroyed before the end of the loop, in which case the loop will continue as normal until all the other generators have been destroyed or exhausted.

The **foreach** command does not take a snapshot of the generator. Any changes in the state of the generator made inside the loop or by other code will affect the state of the loop. In particular, if the code in the loop invokes the generator to manually retrieve the next element, this element will then be excluded from the loop, and the next iteration will continue from the element after that one. Care should be taken to avoid concurrent updates to generators unless this behaviour is required (e.g., in argument processing).

generator next *generator* ?*varName..*?

Manually retrieves the next values from a generator. One value is retrieved for each variable supplied and assigned to the corresponding variable. If the generator becomes exhausted at any time then any remaining variables are set to the empty string.

generator exists *generator*

Returns 1 if the generator (still) exists, or 0 otherwise.

generator names

Returns a list of all currently existing generator commands.

generator destroy ?*generator..*?

Destroys one or more generators, freeing any associated resources.

generator finally *cmd* ?*arg..*?

Used in the definition of a generator procedure, this command arranges for a resource to be cleaned up whenever the generator is destroyed, either explicitly or implicitly when the generator is exhausted. This command can be used like a **finally** block in the **try** command, except that it is tied to the life-cycle of the generator rather than to a particular scope. For example, if we create a generator to iterate over the lines in a text file, we can use **finally** to ensure that the file is closed whenever the generator is destroyed:

```
    generator define lines file {
        set in [open $file]
        # Ensure file is always closed
        generator finally close $in
        while {[gets $in line] >= 0} {
            generator yield $line
        }
    }
    generator foreach line [lines /etc/passwd] {
        puts "[incr count]: $line"
        if {$count > 10} { break }
    }
    # File will be closed even on early exit
```

If you create a generator that consumes another generator (such as the standard **map** and

filter generators defined later), then you should use a **finally** command to ensure that this generator is destroyed when its parent is. For example, the **map** generator is defined as follows:

```
generator define map {f xs} {
    generator finally generator destroy $xs
    generator foreach x $xs { generator yield [{*}$f $x] }
}
```

generator from *format value*

Creates a generator from a data structure. Currently, supported formats are **list**, **dict**, or **string**. The list format yields each element in turn. For dictionaries, each key and value are yielded separately. Finally, strings are yielded a character at a time.

generator to *format generator*

Converts a generator into a data structure. This is the reverse operation of the **from** command, and supports the same data structures. The two operations obey the following identity laws (where **=** is interpreted appropriately):

```
[generator to $fmt [generator from $fmt $value]] = $value
[generator from $fmt [generator to $fmt $gen]]   = $gen
```

Prelude

The following commands are provided as a standard library of generator combinators and functions that perform convenience operations on generators. The functions in this section are loosely modelled on the equivalent functions from the Haskell Prelude. *Warning:* most of the functions in this prelude destroy any generator arguments they are passed as a side-effect. If you want to have persistent generators, see the streams library.

generator map *function generator*

Apply a function to every element of a generator, returning a new generator of the results. This is the classic map function from functional programming, applied to generators. For example, we can generate all the square numbers using the following code (where **nats** is defined as earlier):

```
proc square x { expr {$x * $x} }
generator foreach n [generator map square [nats]] {
    puts "n = $n"
    if {$n > 1000} { break }
}
```

generator filter *predicate generator*

Another classic functional programming gem. This command returns a generator that yields only those items from the argument generator that satisfy the predicate (boolean function). For example, if we had a generator **employees** that returned a stream of dictionaries representing people, we could filter all those whose salaries are above 100,000 dollars (or whichever currency you prefer) using a simple filter:

```
    proc salary> {amount person} { expr {[dict get $person salary] >
    $amount} }
    set fat-cats [generator filter {salary> 100000} $employees]
```

generator reduce *function zero generator*

This is the classic left-fold operation. This command takes a function, an initial value, and a generator of values. For each element in the generator it applies the function to the current accumulator value (the *zero* argument initially) and that element, and then uses the result as the new accumulator value. This process is repeated through the entire generator (eagerly) and the final accumulator value is then returned. If we consider the function to be a binary operator, and the zero argument to be the left identity element of that operation, then we can consider the **reduce** command as *folding* the operator between each successive pair of values in the generator in a left-associative fashion. For example, the sum of a sequence of numbers can be calculated by folding a **+** operator between them, with 0 as the identity:

```
    # sum xs          = reduce + 0 xs
    # sum [range 1 5] = reduce + 0 [range 1 5]
    #                 = reduce + [+ 0 1] [range 2 5]
    #                 = reduce + [+ 1 2] [range 3 5]
    #                 = ...
    #                 = reduce + [+ 10 5] <empty>
    #                 = ((((0+1)+2)+3)+4)+5
    #                 = 15
    proc + {a b} { expr {$a + $b} }
    proc sum gen { generator reduce + 0 $gen }
    puts [sum [range 1 10]]
```

The **reduce** operation is an extremely useful one, and a great variety of different operations can be defined using it. For example, we can define a factorial function as the product of a range using generators. This definition is both very clear and also quite efficient (in both memory and running time):

```
    proc * {x y} { expr {$x * $y} }
    proc prod gen { generator reduce * 0 $gen }
    proc fac n { prod [range 1 $n] }
```

However, while the **reduce** operation is efficient for finite generators, care should be taken not to apply it to an infinite generator, as this will result in an infinite loop:

```
    sum [nats]; # Never returns
```

generator foldl *function zero generator*

This is an alias for the **reduce** command.

generator foldr *function zero generator*

This is the right-associative version of **reduce**. This operation is generally inefficient, as the entire generator needs to be evaluated into memory (as a list) before the reduction can commence. In an eagerly evaluated language like Tcl, this operation has limited use, and should be avoided if possible.

generator all *predicate generator*
Returns true if all elements of the generator satisfy the given predicate.

generator and *generator*
Returns true if all elements of the generator are true (i.e., takes the logical conjunction of the elements).

generator any *generator*
Returns true if any of the elements of the generator are true (i.e., logical disjunction).

generator concat *generator* ?*generator..*?
Returns a generator which is the concatenation of each of the argument generators.

generator concatMap *function generator*
Given a function which maps a value to a series of values, and a generator of values of that type, returns a generator of all of the results in one flat series. Equivalent to **concat** applied to the result of **map**.

generator drop *n generator*
Removes the given number of elements from the front of the generator and returns the resulting generator with those elements removed.

generator dropWhile *predicate generator*
Removes all elements from the front of the generator that satisfy the predicate.

generator contains *element generator*
Returns true if the generator contains the given element. Note that this will destroy the generator!

generator foldl1 *function generator*
A version of **foldl** that takes the *zero* argument from the first element of the generator. Therefore this function is only valid on non-empty generators.

generator foldli *function zero generator*
A version of **foldl** that supplies the integer index of each element as the first argument to the function. The first element in the generator at this point is given index 0.

generator foldri *function zero generator*
Right-associative version of **foldli**.

generator head *generator*
Returns the first element of the generator.

generator tail *generator*
Removes the first element of the generator, returning the rest.

generator init *generator*
Returns a new generator consisting of all elements except the last of the argument generator.

generator takeList *n generator*
Returns the next *n* elements of the generator as a list. If not enough elements are left in the generator, then just the remaining elements are returned.

generator take *n generator*
Returns the next *n* elements of the generator as a new generator. The old generator is destroyed.

generator iterate *function init*
Returns an infinite generator formed by repeatedly applying the function to the initial argument. For example, the Fibonacci numbers can be defined as follows:

```
    proc fst pair { lindex $pair 0 }
    proc snd pair { lindex $pair 1 }
    proc nextFib ab { list [snd $ab] [expr {[fst $ab] + [snd $ab]}] }
    proc fibs {} { generator map fst [generator iterate nextFib {0
    1}] }
```

generator last *generator*
Returns the last element of the generator (if it exists).

generator length *generator*
Returns the length of the generator, destroying it in the process.

generator or *predicate generator*
Returns 1 if any of the elements of the generator satisfy the predicate.

generator product *generator*
Returns the product of the numbers in a generator.

generator repeat *n value..*
Returns a generator that consists of *n* copies of the given elements. The special value *Inf* can be used to generate an infinite sequence.

generator sum *generator*
Returns the sum of the values in the generator.

generator takeWhile *predicate generator*
Returns a generator of the first elements in the argument generator that satisfy the predicate.

generator splitWhen *predicate generator*
Splits the generator into lists of elements using the predicate to identify delimiters. The resulting lists are returned as a generator. Elements matching the delimiter predicate are discarded. For example, to split up a generator using the string "|" as a delimiter:

```
    set xs [generator from list {a | b | c}]
    generator split {string equal "|"} $xs ;# returns a then b then c
```

generator scanl *function zero generator*
Similar to **foldl**, but returns a generator of all of the intermediate values for the accumulator argument. The final element of this generator is equivalent to **foldl** called on the same arguments.

Bugs, ideas, feedback

Please report any errors in this document, or in the package it describes, to Neil Madden.

gpx

gpx

Keywords: gpx, gps

Name

gpx - Extracts waypoints, tracks and routes from GPX files

Synopsis

package require Tcl 8.5 9
package require gpx ?**1.1**?
::gpx::Create *gpxFilename* ?*rawXML*?
::gpx::Cleanup *token*
::gpx::GetGPXMetadata *token*
::gpx::GetWaypointCount *token*
::gpx::GetAllWaypoints *token*
::gpx::GetTrackCount *token*
::gpx::GetTrackMetadata *token whichTrack*
::gpx::GetTrackPoints *token whichTrack*
::gpx::GetRouteCount *token*
::gpx::GetRouteMetadata *token whichRoute*
::gpx::GetRoutePoints *token whichRoute*

Description

This module parses and extracts waypoints, tracks, routes and metadata from a GPX (GPS eXchange) file. Both GPX version 1.0 and 1.1 are supported.

Commands

::gpx::Create *gpxFilename* ?*rawXML*?
: The **::gpx::Create** is the first command called to process GPX data. It takes the GPX data from either the *rawXML* parameter if present or from the contents of *gpxFilename*, and parses it using *tdom*. It returns a token value that is used by all the other commands.

::gpx::Cleanup *token*
: This procedure cleans up resources associated with *token*. It is *strongly* recommended that you call this function after you are done with a given GPX file. Not doing so will result in memory not being freed, and if your app calls **::gpx::Create** enough times, the memory leak could cause a performance hit...or worse.

::gpx::GetGPXMetadata *token*
: This procedure returns a dictionary of the metadata associated with the GPX data identified by *token*. The format of the metadata dictionary is described below, but keys *version* and *creator* will always be present.

::gpx::GetWaypointCount *token*
: This procedure returns the number of waypoints defined in the GPX data identified by *token*.

::gpx::GetAllWaypoints *token*
: This procedure returns the a list of waypoints defined in the GPX data identified by *token*. The format of each waypoint item is described below.

::gpx::GetTrackCount *token*
: This procedure returns the number of tracks defined in the GPX data identified by *token*.

::gpx::GetTrackMetadata *token whichTrack*
: This procedure returns a dictionary of the metadata associated track number *whichTrack* (1 based) in the GPX data identified by *token*. The format of the metadata dictionary is described below.

::gpx::GetTrackPoints *token whichTrack*
: The procedure returns a list of track points comprising track number *whichTrack* (1 based) in the GPX data identified by *token*. The format of the metadata dictionary is described below.

::gpx::GetRouteCount *token*
This procedure returns the number of routes defined in the GPX data identified by *token*.

::gpx::GetRouteMetadata *token whichRoute*
This procedure returns a dictionary of the metadata associated route number *whichRoute* (1 based) in the GPX data identified by *token*. The format of the metadata dictionary is described below.

::gpx::GetRoutePoints *token whichRoute*
The procedure returns a list of route points comprising route number *whichRoute* (1 based) in the GPX data identified by *token*. The format of the metadata dictionary is described below.

Data structures

metadata dictionary
The metadata associated with either the GPX document, a track, a route, a waypoint, a track point or route point is returned in a dictionary. The keys of that dictionary will be whatever optional GPX elements are present. The value for each key depends on the GPX schema for that element. For example, the value for a version key will be a string, while for a link key will be a sub-dictionary with keys *href* and optionally *text* and *type*.

point item
Each item in a track or route list of points consists of a list of three elements: *latitude*, *longitude* and *metadata dictionary*. *Latitude* and *longitude* are decimal numbers. The *metadata dictionary* format is described above. For points in a track, typically there will always be ele (elevation) and time metadata keys.

Example

```
% set token [::gpx::Create myGpxFile.gpx]
% set version [dict get [::gpx::GetGPXMetadata $token] version]
% set trackCnt [::gpx::GetTrackCount $token]
% set firstPoint [lindex [::gpx::GetTrackPoints $token 1] 0]
% lassign $firstPoint lat lon ptMetadata
% puts "first point in the first track is at $lat, $lon"
% if {[dict exists $ptMetadata ele]} {
     puts "at elevation [dict get $ptMetadata ele] meters"
  }
% ::gpx::Cleanup $token
```

References

1. GPX: the GPS Exchange Format (http://www.topografix.com/gpx.asp)
2. GPX 1.1 Schema Documentation (http://www.topografix.com/GPX/1/1/)
3. GPX 1.0 Developer's Manual (http://www.topografix.com/gpx_manual.asp)

Author

Keith Vetter

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib

Trackers and specify *gpx* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
File formats

Copyright

```
Copyright (c) 2010, Keith Vetter <kvetter@gmail.com>
```


grammar_aycock

aycock
Keywords: aycock, Earley, Horspool, parser, compiler, ambiguous, grammar, parsing, transducer

Name
grammar::aycock - Aycock-Horspool-Earley parser generator for Tcl

Synopsis
package require Tcl 8.5 9
package require grammar::aycock ?**1.1**?
::aycock::parser *grammar* ?**-verbose**?
parserName **parse** *symList valList* ?*clientData*?
parserName **destroy**
parserName **terminals**
parserName **nonterminals**
parserName **save**

Description
The **grammar::aycock** package implements a parser generator for the class of parsers described in John Aycock and R. Nigel Horspool. Practical Earley Parsing. *The Computer Journal, 45*(6):620-630, 2002. http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.12.4254

Procedures
The **grammar::aycock** package exports the single procedure:

::aycock::parser *grammar* ?-verbose?
Generates a parser for the given *grammar*, and returns its name. If the optional **-verbose** flag is given, dumps verbose information relating to the generated parser to the standard output. The returned parser is an object that accepts commands as shown in Object command below.

Object command
***parserName* parse *symList valList* ?*clientData*?**

Invokes a parser returned from **::aycock::parser**. *symList* is a list of grammar symbols representing the terminals in an input string, and *valList* is a list of their semantic values. The result is the semantic value of the entire string when parsed.

***parserName* destroy**
Destroys a parser constructed by **::aycock::parser**.

***parserName* terminals**
Returns a list of terminal symbols that may be presented in the *symList* argument to the **parse** object command.

***parserName* nonterminals**
Returns a list of nonterminal symbols that were defined in the parser's grammar.

***parserName* save**
Returns a Tcl script that will reconstruct the parser without needing all the mechanism of the parser generator at run time. The reconstructed parser depends on a set of commands in the package **grammar::aycock::runtime**, which is also automatically loaded when the **grammar::aycock** package is loaded.

Description

The **grammar::aycock::parser** command accepts a grammar expressed as a Tcl list. The list must be structured as the concatenation of a set of *rule*s. Each *rule* comprises a variable number of elements in the list:

- The name of the nonterminal symbol that the rule reduces.
- The literal string, **::=**
- Zero or more names of terminal or nonterminal symbols that comprise the right-hand-side of the rule.
- Finally, a Tcl script to execute when the rule is reduced. Within the given script, a variable called _ contains a list of the semantic values of the symbols on the right-hand side. The value returned by the script is expected to be the semantic value of the left-hand side. If the *clientData* parameter was passed to the **parse** method, it is available in a variable called **clientData**. It is permissible for the script to be the empty string. In this case, the semantic value of the rule will be the same as the semantic value of the first symbol on the right-hand side. If the right-hand side is also empty, the semantic value will be the empty string.

Parsing is done with an Earley parser, which is not terribly efficient in speed or memory consumption, but which deals effectively with ambiguous grammars. For this reason, the **grammar::aycock** package is perhaps best adapted to natural-language processing or the parsing of extraordinarily complex languages in which ambiguity can be tolerated.

Example

The following code demonstrates a trivial desk calculator, admitting only **+**, * and parentheses as its operators. It also shows the format in which the lexical analyzer is expected to present terminal symbols to the parser.

```
set p [aycock::parser {
    start ::= E {}
    E ::= E + T {expr {[lindex $_ 0] + [lindex $_ 2]}}
    E ::= T {}
```

```
    T ::= T * F {expr {[lindex $_ 0] * [lindex $_ 2]}}
    T ::= F {}
    F ::= NUMBER {}
    F ::= ( E ) {lindex $_ 1}
}]
puts [$p parse {(  NUMBER +  NUMBER )  *  ( NUMBER +  NUMBER ) }  {{}
    2      {} 3      {} {} {} 7     {} 1      {}}]
$p destroy
```

The example, when run, prints **40**.

Category

Grammars and finite automata

Copyright

```
Copyright (c) 2006 by Kevin B. Kenny <kennykb@acm.org>
Redistribution permitted under the terms of the Open Publication
    License <http://www.opencontent.org/openpub/>
```

grammar_fa

dacceptor

Keywords: dacceptor, acceptance, acceptor, automaton, finite automaton, grammar, parsing, regular expression, regular grammar, regular languages, state, transducer

Name

grammar::fa::dacceptor - Create and use deterministic acceptors

Synopsis

package require Tcl 8.5 9
package require snit
package require struct::set
package require grammar::fa::dacceptor ?**0.1.2**?
::grammar::fa::dacceptor *daName fa* ?*-any any*?
daName *option* ?*arg arg* ...?
daName **destroy**
daName **accept?** *symbols*

Description

This package provides a class for acceptors constructed from deterministic *finite automatons* (DFA). Acceptors are objects which can be given a string of symbols and tell if the DFA they are constructed from would *accept* that string. For the actual creation of the DFAs the acceptors are based on we have the packages **grammar::fa** and **grammar::fa::op**.

API

The package exports the API described here.

::grammar::fa::dacceptor *daName fa* ?-any *any*?

Creates a new deterministic acceptor with an associated global Tcl command whose name is *daName*. This command may be used to invoke various operations on the acceptor. It has the following general form:

daName *option* ?*arg arg ...*?

Option and the *arg*s determine the exact behavior of the command. See section Acceptor methods for more explanations.

The acceptor will be based on the deterministic finite automaton stored in the object *fa*. It will keep a copy of the relevant data of the FA in its own storage, in a form easy to use for its purposes. This also means that changes made to the *fa* after the construction of the acceptor *will not* influence the acceptor.

If *any* has been specified, then the acceptor will convert all symbols in the input which are unknown to the base FA to that symbol before proceeding with the processing.

Acceptor methods

All acceptors provide the following methods for their manipulation:

***daName* destroy**

Destroys the automaton, including its storage space and associated command.

daName* accept? *symbols

Takes the list of *symbols* and checks if the FA the acceptor is based on would accept it. The result is a boolean value. **True** is returned if the symbols are accepted, and **False** otherwise. Note that bogus symbols in the input are either translated to the *any* symbol (if specified), or cause the acceptance test to simply fail. No errors will be thrown. The method will process only just that prefix of the input which is enough to fully determine (non-)acceptance.

Examples

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *grammar_fa* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Grammars and finite automata

Copyright


```
Copyright (c) 2004 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

dexec

Keywords: dexec, automaton, execution, finite automaton, grammar, parsing, regular expression , regular grammar, regular languages, running, state, transducer

Name

grammar::fa::dexec - Execute deterministic finite automatons

Synopsis

package require Tcl 8.5 9
package require snit
package require grammar::fa::dexec ?**0.3**?
::grammar::fa::dexec *daName fa* ?*-any any*? ?*-command cmdprefix*?
daName *option* ?*arg arg* ...?
daName **destroy**
daName **put** *symbol*
daName **reset**
daName **state**
cmdprefix **error** *code message*
cmdprefix **final** *stateid*
cmdprefix **reset**
cmdprefix **state** *stateid*

Description

This package provides a class for executors constructed from deterministic *finite automatons* (DFA). Executors are objects which are given a string of symbols in a piecemal fashion, perform state transitions and report back when they enter a final state, or find an error in the input. For the actual creation of the DFAs the executors are based on we have the packages **grammar::fa** and **grammar::fa::op**.

The objects follow a push model. Symbols are pushed into the executor, and when something important happens, i.e. error occurs, a state transition, or a final state is entered this will be reported via the callback specified via the option **-command**. Note that conversion of this into a pull model where the environment retrieves messages from the object and the object uses a callback to ask for more symbols is a trivial thing.

Side note: The acceptor objects provided by **grammar::fa::dacceptor** could have been implemented on top of the executors provided here, but were not, to get a bit more performance (we avoid a number of method calls and the time required for their dispatch).

API

The package exports the API described here.

::grammar::fa::dexec *daName fa* ?-any *any*? ?-command *cmdprefix*?

Creates a new deterministic executor with an associated global Tcl command whose name is *daName*. This command may be used to invoke various operations on the executor. It has the following general form:

daName *option* ?*arg arg ...*?

Option and the *arg*s determine the exact behavior of the command. See section Executor methods for more explanations.

The executor will be based on the deterministic finite automaton stored in the object *fa*. It will keep a copy of the relevant data of the FA in its own storage, in a form easy to use for its purposes. This also means that changes made to the *fa* after the construction of the executor *will not* influence the executor.

If *any* has been specified, then the executor will convert all symbols in the input which are unknown to the base FA to that symbol before proceeding with the processing.

Executor methods

All executors provide the following methods for their manipulation:

***daName* destroy**
Destroys the automaton, including its storage space and associated command.

daName* put *symbol
Takes the current state of the executor and the *symbol* and performs the appropriate state transition. Reports any errors encountered via the command callback, as well as entering a final state of the underlying FA.

When an error is reported all further invokations of **put** will do nothing, until the error condition has been cleared via an invokation of method **reset**.

***daName* reset**
Unconditionally sets the executor into the start state of the underlying FA. This also clears any error condition **put** may have encountered.

***daName* state**
Returns the current state of the underlying FA. This allow for introspection without the need to pass data from the callback command.

Executor callback

The callback command *cmdprefix* given to an executor via the option **-command** will be executed by the object at the global level, using the syntax described below. Note that *cmdprefix* is not simply the name of a command, but a full command prefix. In other words it may contain additional fixed argument words beyond the command word.

cmdprefix* error *code message
The executor has encountered an error, and *message* contains a human-readable text explaining the nature of the problem. The *code* on the other hand is a fixed machine-readable text. The following error codes can be generated by executor objects.

BADSYM
An unknown symbol was found in the input. This can happen if and only if no **-any** symbol was specified.

BADTRANS
The underlying FA has no transition for the current combination of input symbol and state. In other words, the executor was not able to compute a new state for this combination.

cmdprefix* final *stateid
The executor has entered the final state *stateid*.

***cmdprefix* reset**
The executor was reset.

cmdprefix* state *stateid
The FA changed state due to a transition. *stateid* is the new state.

Examples

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *grammar_fa* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Grammars and finite automata

Copyright


```
Copyright (c) 2004 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
Copyright (c) 2007 Bogdan <rftghost@users.sourceforge.net>
```


fa

Keywords: fa, automaton, finite automaton, grammar, parsing, regular expression, regular grammar, regular languages, state, transducer

Name

grammar::fa - Create and manipulate finite automatons

Synopsis

package require Tcl 8.5 9
package require snit 1.3
package require struct::list
package require struct::set
package require grammar::fa::op ?**0.3**?
package require grammar::fa ?**0.6**?
::grammar::fa *faName* ?=|:=|<--|*as*|*deserialize src*|*fromRegex re over*?
faName *option* ?*arg arg* ...?
faName **destroy**
faName **clear**
faName **=** *srcFA*
faName **-->** *dstFA*
faName **serialize**
faName **deserialize** *serialization*

faName **states**
faName **state add** *s1* ?*s2* ...?
faName **state delete** *s1* ?*s2* ...?
faName **state exists** *s*
faName **state rename** *s snew*
faName **startstates**
faName **start add** *s1* ?*s2* ...?
faName **start remove** *s1* ?*s2* ...?
faName **start?** *s*
faName **start?set** *stateset*
faName **finalstates**
faName **final add** *s1* ?*s2* ...?
faName **final remove** *s1* ?*s2* ...?
faName **final?** *s*
faName **final?set** *stateset*
faName **symbols**
faName **symbols@** *s* ?*d*?
faName **symbols@set** *stateset*
faName **symbol add** *sym1* ?*sym2* ...?
faName **symbol delete** *sym1* ?*sym2* ...?
faName **symbol rename** *sym newsym*
faName **symbol exists** *sym*
faName **next** *s sym* ?--> *next*?
faName **!next** *s sym* ?--> *next*?
faName **nextset** *stateset sym*
faName **is deterministic**
faName **is complete**
faName **is useful**
faName **is epsilon-free**
faName **reachable_states**
faName **unreachable_states**
faName **reachable** *s*
faName **useful_states**
faName **unuseful_states**
faName **useful** *s*
faName **epsilon_closure** *s*
faName **reverse**
faName **complete**
faName **remove_eps**
faName **trim** ?*what*?
faName **determinize** ?*mapvar*?
faName **minimize** ?*mapvar*?
faName **complement**
faName **kleene**
faName **optional**
faName **union** *fa* ?*mapvar*?

faName **intersect** *fa* ?*mapvar*?
faName **difference** *fa* ?*mapvar*?
faName **concatenate** *fa* ?*mapvar*?
faName **fromRegex** *regex* ?*over*?

Description

This package provides a container class for *finite automatons* (Short: FA). It allows the incremental definition of the automaton, its manipulation and querying of the definition. While the package provides complex operations on the automaton (via package **grammar::fa::op**), it does not have the ability to execute a definition for a stream of symbols. Use the packages **grammar::fa::dacceptor** and **grammar::fa::dexec** for that. Another package related to this is **grammar::fa::compiler**. It turns a FA into an executor class which has the definition of the FA hardwired into it. The output of this package is configurable to suit a large number of different implementation languages and paradigms.

For more information about what a finite automaton is see section Finite automatons.

API

The package exports the API described here.

::grammar::fa *faName* ?=|:=|<--|as|deserialize *src*|fromRegex *re* ?*over*??

Creates a new finite automaton with an associated global Tcl command whose name is *faName*. This command may be used to invoke various operations on the automaton. It has the following general form:

faName *option* ?*arg arg ...*?

Option and the *arg*s determine the exact behavior of the command. See section FA methods for more explanations. The new automaton will be empty if no *src* is specified. Otherwise it will contain a copy of the definition contained in the *src*. The *src* has to be a FA object reference for all operators except **deserialize** and **fromRegex**. The **deserialize** operator requires *src* to be the serialization of a FA instead, and **fromRegex** takes a regular expression in the form a of a syntax tree. See ::grammar::fa::op::fromRegex for more detail on that.

FA methods

All automatons provide the following methods for their manipulation:

***faName* destroy**

Destroys the automaton, including its storage space and associated command.

***faName* clear**

Clears out the definition of the automaton contained in *faName*, but does *not* destroy the object.

faName = srcFA

Assigns the contents of the automaton contained in *srcFA* to *faName*, overwriting any existing definition. This is the assignment operator for automatons. It copies the automaton contained in the FA object *srcFA* over the automaton definition in *faName*. The old contents of *faName* are deleted by this operation.

This operation is in effect equivalent to

```
    faName deserialize [srcFA serialize]
```

faName --> dstFA

This is the reverse assignment operator for automatons. It copies the automation contained in the object *faName* over the automaton definition in the object *dstFA*. The old contents of *dstFA* are deleted by this operation.

This operation is in effect equivalent to

```
    dstFA deserialize [faName serialize]
```

***faName* serialize**

This method serializes the automaton stored in *faName*. In other words it returns a tcl *value* completely describing that automaton. This allows, for example, the transfer of automatons over arbitrary channels, persistence, etc. This method is also the basis for both the copy constructor and the assignment operator.

The result of this method has to be semantically identical over all implementations of the **grammar::fa** interface. This is what will enable us to copy automatons between different implementations of the same interface.

The result is a list of three elements with the following structure:

1. The constant string **grammar::fa**.
2. A list containing the names of all known input symbols. The order of elements in this list is not relevant.
3. The last item in the list is a dictionary, however the order of the keys is important as well. The keys are the states of the serialized FA, and their order is the order in which to create the states when deserializing. This is relevant to preserve the order relationship between states.

   The value of each dictionary entry is a list of three elements describing the state in more detail.
   1. A boolean flag. If its value is **true** then the state is a start state, otherwise it is not.
   2. A boolean flag. If its value is **true** then the state is a final state, otherwise it is not.
   3. The last element is a dictionary describing the transitions for the state. The keys are symbols (or the empty string), and the values are sets of successor states.

Assuming the following FA (which describes the life of a truck driver in a very simple way:)

```
    Drive -- yellow --> Brake -- red --> (Stop) -- red/yellow -->
    Attention -- green --> Drive
    (...) is the start state.
```

a possible serialization is

```
grammar::fa \
{yellow red green red/yellow} \
{Drive     {0 0 {yellow     Brake}} \
 Brake     {0 0 {red        Stop}} \
 Stop      {1 0 {red/yellow Attention}} \
 Attention {0 0 {green      Drive}}}
```

A possible one, because I did not care about creation order here

faName* deserialize *serialization
This is the complement to **serialize**. It replaces the automaton definition in *faName* with the automaton described by the *serialization* value. The old contents of *faName* are deleted by this operation.

***faName* states**
Returns the set of all states known to *faName*.

***faName* state add *s1* ?*s2* ...?**
Adds the states *s1*, *s2*, et cetera to the FA definition in *faName*. The operation will fail any of the new states is already declared.

***faName* state delete *s1* ?*s2* ...?**
Deletes the state *s1*, *s2*, et cetera, and all associated information from the FA definition in *faName*. The latter means that the information about in- or outbound transitions is deleted as well. If the deleted state was a start or final state then this information is invalidated as well. The operation will fail if the state *s* is not known to the FA.

faName* state exists *s
A predicate. It tests whether the state *s* is known to the FA in *faName*. The result is a boolean value. It will be set to **true** if the state *s* is known, and **false** otherwise.

faName* state rename *s snew
Renames the state *s* to *snew*. Fails if *s* is not a known state. Also fails if *snew* is already known as a state.

***faName* startstates**
Returns the set of states which are marked as *start* states, also known as *initial* states. See Finite automatons for explanations what this means.

***faName* start add *s1* ?*s2* ...?**
Mark the states *s1*, *s2*, et cetera in the FA *faName* as *start* (aka *initial*).

***faName* start remove *s1* ?*s2* ...?**
Mark the states *s1*, *s2*, et cetera in the FA *faName* as *not start* (aka *not accepting*).

faName* start? *s
A predicate. It tests if the state *s* in the FA *faName* is *start* or not. The result is a boolean value. It will be set to **true** if the state *s* is *start*, and **false** otherwise.

faName* start?set *stateset
A predicate. It tests if the set of states *stateset* contains at least one start state. They operation will fail if the set contains an element which is not a known state. The result is a boolean value. It will be set to **true** if a start state is present in *stateset*, and **false** otherwise.

***faName* finalstates**
Returns the set of states which are marked as *final* states, also known as *accepting* states.

See Finite automatons for explanations what this means.

***faName* final add *s1* ?*s2* ...?**
Mark the states *s1*, *s2*, et cetera in the FA *faName* as *final* (aka *accepting*).

***faName* final remove *s1* ?*s2* ...?**
Mark the states *s1*, *s2*, et cetera in the FA *faName* as *not final* (aka *not accepting*).

faName* final? *s
A predicate. It tests if the state *s* in the FA *faName* is *final* or not. The result is a boolean value. It will be set to **true** if the state *s* is *final*, and **false** otherwise.

faName* final?set *stateset
A predicate. It tests if the set of states *stateset* contains at least one final state. They operation will fail if the set contains an element which is not a known state. The result is a boolean value. It will be set to **true** if a final state is present in *stateset*, and **false** otherwise.

***faName* symbols**
Returns the set of all symbols known to the FA *faName*.

***faName* symbols@ *s* ?*d*?**
Returns the set of all symbols for which the state *s* has transitions. If the empty symbol is present then *s* has epsilon transitions. If two states are specified the result is the set of symbols which have transitions from *s* to *t*. This set may be empty if there are no transitions between the two specified states.

faName* symbols@set *stateset
Returns the set of all symbols for which at least one state in the set of states *stateset* has transitions. In other words, the union of [*faName* **symbols@ s**] for all states **s** in *stateset*. If the empty symbol is present then at least one state contained in *stateset* has epsilon transitions.

***faName* symbol add *sym1* ?*sym2* ...?**
Adds the symbols *sym1*, *sym2*, et cetera to the FA definition in *faName*. The operation will fail any of the symbols is already declared. The empty string is not allowed as a value for the symbols.

***faName* symbol delete *sym1* ?*sym2* ...?**
Deletes the symbols *sym1*, *sym2* et cetera, and all associated information from the FA definition in *faName*. The latter means that all transitions using the symbols are deleted as well. The operation will fail if any of the symbols is not known to the FA.

faName* symbol rename *sym newsym
Renames the symbol *sym* to *newsym*. Fails if *sym* is not a known symbol. Also fails if *newsym* is already known as a symbol.

faName* symbol exists *sym
A predicate. It tests whether the symbol *sym* is known to the FA in *faName*. The result is a boolean value. It will be set to **true** if the symbol *sym* is known, and **false** otherwise.

***faName* next *s sym* ?--> *next*?**
Define or query transition information.

If *next* is specified, then the method will add a transition from the state *s* to the *successor* state *next* labeled with the symbol *sym* to the FA contained in *faName*. The operation will fail if *s*, or *next* are not known states, or if *sym* is not a known symbol. An exception to the latter is that *sym* is allowed to be the empty string. In that case the new transition is an *epsilon*

transition which will not consume input when traversed. The operation will also fail if the combination of (*s*, *sym*, and *next*) is already present in the FA.

If *next* was not specified, then the method will return the set of states which can be reached from *s* through a single transition labeled with symbol *sym*.

***faName*!next *s sym* ?--> *next*?**
Remove one or more transitions from the Fa in *faName*.

If *next* was specified then the single transition from the state *s* to the state *next* labeled with the symbol *sym* is removed from the FA. Otherwise *all* transitions originating in state *s* and labeled with the symbol *sym* will be removed.

The operation will fail if *s* and/or *next* are not known as states. It will also fail if a non-empty *sym* is not known as symbol. The empty string is acceptable, and allows the removal of epsilon transitions.

faName* nextset *stateset sym
Returns the set of states which can be reached by a single transition originating in a state in the set *stateset* and labeled with the symbol *sym*.

In other words, this is the union of [*faName* next **s** *symbol*] for all states **s** in *stateset*.

***faName* is deterministic**
A predicate. It tests whether the FA in *faName* is a deterministic FA or not. The result is a boolean value. It will be set to **true** if the FA is deterministic, and **false** otherwise.

***faName* is complete**
A predicate. It tests whether the FA in *faName* is a complete FA or not. A FA is complete if it has at least one transition per state and symbol. This also means that a FA without symbols, or states is also complete. The result is a boolean value. It will be set to **true** if the FA is deterministic, and **false** otherwise.

Note: When a FA has epsilon-transitions transitions over a symbol for a state S can be indirect, i.e. not attached directly to S, but to a state in the epsilon-closure of S. The symbols for such indirect transitions count when computing completeness.

***faName* is useful**
A predicate. It tests whether the FA in *faName* is an useful FA or not. A FA is useful if all states are *reachable* and *useful*. The result is a boolean value. It will be set to **true** if the FA is deterministic, and **false** otherwise.

***faName* is epsilon-free**
A predicate. It tests whether the FA in *faName* is an epsilon-free FA or not. A FA is epsilon-free if it has no epsilon transitions. This definition means that all deterministic FAs are epsilon-free as well, and epsilon-freeness is a necessary pre-condition for deterministic'ness. The result is a boolean value. It will be set to **true** if the FA is deterministic, and **false** otherwise.

***faName* reachable_states**
Returns the set of states which are reachable from a start state by one or more transitions.

***faName* unreachable_states**
Returns the set of states which are not reachable from any start state by any number of

transitions. This is

```
[faName states] - [faName reachable_states]
```

faName* reachable *s
A predicate. It tests whether the state *s* in the FA *faName* can be reached from a start state by one or more transitions. The result is a boolean value. It will be set to **true** if the state can be reached, and **false** otherwise.

***faName* useful_states**
Returns the set of states which are able to reach a final state by one or more transitions.

***faName* unuseful_states**
Returns the set of states which are not able to reach a final state by any number of transitions. This is

```
[faName states] - [faName useful_states]
```

faName* useful *s
A predicate. It tests whether the state *s* in the FA *faName* is able to reach a final state by one or more transitions. The result is a boolean value. It will be set to **true** if the state is useful, and **false** otherwise.

faName* epsilon_closure *s
Returns the set of states which are reachable from the state *s* in the FA *faName* by one or more epsilon transitions, i.e transitions over the empty symbol, transitions which do not consume input. This is called the *epsilon closure* of *s*.

***faName* reverse**
see below ...

***faName* complete**
see below ...

***faName* remove_eps**
see below ...

***faName* trim ?*what*?**
see below ...

***faName* determinize ?*mapvar*?**
see below ...

***faName* minimize ?*mapvar*?**
see below ...

***faName* complement**
see below ...

***faName* kleene**
see below ...

***faName* optional**
see below ...

***faName* union *fa* ?*mapvar*?**
see below ...

***faName* intersect *fa* ?*mapvar*?**
see below ...

***faName* difference *fa* ?*mapvar*?**
see below ...

***faName* concatenate *fa* ?*mapvar*?**
see below ...

***faName* fromRegex *regex* ?*over*?**
These methods provide more complex operations on the FA. Please see the same-named commands in the package **grammar::fa::op** for descriptions of what they do.

Examples

Finite automatons

For the mathematically inclined, a FA is a 5-tuple (S,Sy,St,Fi,T) where

- S is a set of *states*,
- Sy a set of *input symbols*,
- St is a subset of S, the set of *start* states, also known as *initial* states.
- Fi is a subset of S, the set of *final* states, also known as *accepting*.
- T is a function from S x (Sy + epsilon) to {S}, the *transition function*. Here **epsilon** denotes the empty input symbol and is distinct from all symbols in Sy; and {S} is the set of subsets of S. In other words, T maps a combination of State and Input (which can be empty) to a set of *successor states*.

In computer theory a FA is most often shown as a graph where the nodes represent the states, and the edges between the nodes encode the transition function: For all n in S' = T (s, sy) we have one edge between the nodes representing s and n resp., labeled with sy. The start and accepting states are encoded through distinct visual markers, i.e. they are attributes of the nodes.

FA's are used to process streams of symbols over Sy.

A specific FA is said to *accept* a finite stream sy_1 sy_2 ... sy_n if there is a path in the graph of the FA beginning at a state in St and ending at a state in Fi whose edges have the labels sy_1, sy_2, etc. to sy_n. The set of all strings accepted by the FA is the *language* of the FA. One important equivalence is that the set of languages which can be accepted by an FA is the set of *regular languages*.

Another important concept is that of deterministic FAs. A FA is said to be *deterministic* if for each string of input symbols there is exactly one path in the graph of the FA beginning at the start state and whose edges are labeled with the symbols in the string. While it might seem that non-deterministic FAs to have more power of recognition, this is not so. For each non-deterministic FA we can construct a deterministic FA which accepts the same language (--> Thompson's subset construction).

While one of the premier applications of FAs is in *parsing*, especially in the *lexer* stage (where symbols == characters), this is not the only possibility by far.

Quite a lot of processes can be modeled as a FA, albeit with a possibly large set of states. For these the notion of accepting states is often less or not relevant at all. What is needed instead is the ability to act to state changes in the FA, i.e. to generate some output in response to the input. This transforms a FA into a *finite transducer*, which has an additional set OSy of *output symbols* and also an additional *output function* O which maps from "S x (Sy + epsilon)" to "(Osy

+ epsilon)", i.e a combination of state and input, possibly empty to an output symbol, or nothing.

For the graph representation this means that edges are additional labeled with the output symbol to write when this edge is traversed while matching input. Note that for an application "writing an output symbol" can also be "executing some code".

Transducers are not handled by this package. They will get their own package in the future.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *grammar_fa* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Grammars and finite automata

Copyright


```
Copyright (c) 2004-2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


faop

Keywords: faop, automaton, finite automaton, grammar, parsing, regular expression, regular grammar, regular languages, state, transducer

Name

grammar::fa::op - Operations on finite automatons

Synopsis

package require Tcl 8.5 9
package require snit
package require struct::list
package require struct::set
package require grammar::fa::op ?**0.4.2**?
::grammar::fa::op::constructor *cmd*
::grammar::fa::op::reverse *fa*
::grammar::fa::op::complete *fa* ?*sink*?
::grammar::fa::op::remove_eps *fa*
::grammar::fa::op::trim *fa* ?*what*?
::grammar::fa::op::determinize *fa* ?*mapvar*?
::grammar::fa::op::minimize *fa* ?*mapvar*?
::grammar::fa::op::complement *fa*
::grammar::fa::op::kleene *fa*

::grammar::fa::op::optional *fa*
::grammar::fa::op::union *fa fb* ?*mapvar*?
::grammar::fa::op::intersect *fa fb* ?*mapvar*?
::grammar::fa::op::difference *fa fb* ?*mapvar*?
::grammar::fa::op::concatenate *fa fb* ?*mapvar*?
::grammar::fa::op::fromRegex *fa regex* ?*over*?
::grammar::fa::op::toRegexp *fa*
::grammar::fa::op::toRegexp2 *fa*
::grammar::fa::op::toTclRegexp *regexp symdict*
::grammar::fa::op::simplifyRegexp *regexp*

Description

This package provides a number of complex operations on finite automatons (Short: FA), as provided by the package **grammar::fa**. The package does not provide the ability to create and/or manipulate such FAs, nor the ability to execute a FA for a stream of symbols. Use the packages **grammar::fa** and **grammar::fa::interpreter** for that. Another package related to this is **grammar::fa::compiler** which turns a FA into an executor class which has the definition of the FA hardwired into it.

For more information about what a finite automaton is see section *FINITE AUTOMATONS* in package **grammar::fa**.

API

The package exports the API described here. All commands modify their first argument. I.e. whatever FA they compute is stored back into it. Some of the operations will construct an automaton whose states are all new, but related to the states in the source automaton(s). These operations take variable names as optional arguments where they will store mappings which describe the relationship(s). The operations can be loosely partitioned into structural and language operations. The latter are defined in terms of the language the automaton(s) accept, whereas the former are defined in terms of the structural properties of the involved automaton(s). Some operations are both. *Structure operations*

::grammar::fa::op::constructor *cmd*
: This command has to be called by the user of the package before any other operations is performed, to establish a command which can be used to construct a FA container object. If this is not done several operations will fail as they are unable to construct internal and transient containers to hold state and/or partial results.

  Any container class using this package for complex operations should set its own class command as the constructor. See package **grammar::fa** for an example.

::grammar::fa::op::reverse *fa*
: Reverses the *fa*. This is done by reversing the direction of all transitions and swapping the sets of *start* and *final* states. The language of *fa* changes unpredictably.

::grammar::fa::op::complete *fa* ?*sink*?
: Completes the *fa complete*, but nothing is done if the *fa* is already *complete*. This implies that only the first in a series of multiple consecutive complete operations on *fa* will perform anything. The remainder will be null operations.

The language of *fa* is unchanged by this operation.

This is done by adding a single new state, the *sink*, and transitions from all other states to that sink for all symbols they have no transitions for. The sink itself is made complete by adding loop transitions for all symbols.

Note: When a FA has epsilon-transitions transitions over a symbol for a state S can be indirect, i.e. not attached directly to S, but to a state in the epsilon-closure of S. The symbols for such indirect transitions count when computing completeness of a state. In other words, these indirectly reached symbols are *not* missing.

The argument *sink* provides the name for the new state and most not be present in the *fa* if specified. If the name is not specified the command will name the state "sink**n**", where **n** is set so that there are no collisions with existing states.

Note that the sink state is *not useful* by definition. In other words, while the FA becomes complete, it is also *not useful* in the strict sense as it has a state from which no final state can be reached.

::grammar::fa::op::remove_eps *fa*

Removes all epsilon-transitions from the *fa* in such a manner the the language of *fa* is unchanged. However nothing is done if the *fa* is already *epsilon-free*. This implies that only the first in a series of multiple consecutive complete operations on *fa* will perform anything. The remainder will be null operations.

Note: This operation may cause states to become unreachable or not useful. These states are not removed by this operation. Use **::grammar::fa::op::trim** for that instead.

::grammar::fa::op::trim *fa* ?*what*?

Removes unwanted baggage from *fa*. The legal values for *what* are listed below. The command defaults to**!reachable|!useful** if no specific argument was given.

!reachable
: Removes all states which are not reachable from a start state.

!useful
: Removes all states which are unable to reach a final state.

!reachable&!useful
: see below ...

!(reachable|useful)
: Removes all states which are not reachable from a start state and are unable to reach a final state.

!reachable|!useful
: see below ...

!(reachable&useful)
: Removes all states which are not reachable from a start state or are unable to reach a final state.

::grammar::fa::op::determinize *fa* ?*mapvar*?

Makes the *fa* deterministic without changing the language accepted by the *fa*. However nothing is done if the *fa* is already *deterministic*. This implies that only the first in a series of multiple consecutive complete operations on *fa* will perform anything. The remainder will be null operations.

The command will store a dictionary describing the relationship between the new states of the resulting dfa and the states of the input nfa in *mapvar*, if it has been specified. Keys of the dictionary are the handles for the states of the resulting dfa, values are sets of states from the input nfa.

Note: An empty dictionary signals that the command was able to make the *fa* deterministic without performing a full subset construction, just by removing states and shuffling transitions around (As part of making the FA epsilon-free).

Note: The algorithm fails to make the FA deterministic in the technical sense if the FA has no start state(s), because determinism requires the FA to have exactly one start states. In that situation we make a best effort; and the missing start state will be the only condition preventing the generated result from being *deterministic*. It should also be noted that in this case the possibilities for trimming states from the FA are also severely reduced as we cannot declare states unreachable.

::grammar::fa::op::minimize *fa* ?*mapvar*?

Creates a FA which accepts the same language as *fa*, but has a minimal number of states. Uses Brzozowski's method to accomplish this.

The command will store a dictionary describing the relationship between the new states of the resulting minimal fa and the states of the input fa in *mapvar*, if it has been specified. Keys of the dictionary are the handles for the states of the resulting minimal fa, values are sets of states from the input fa.

Note: An empty dictionary signals that the command was able to minimize the *fa* without having to compute new states. This should happen if and only if the input FA was already minimal.

Note: If the algorithm has no start or final states to work with then the result might be technically minimal, but have a very unexpected structure. It should also be noted that in this case the possibilities for trimming states from the FA are also severely reduced as we cannot declare states unreachable.

Language operations All operations in this section require that all input FAs have at least one start and at least one final state. Otherwise the language of the FAs will not be defined, making the operation senseless (as it operates on the languages of the FAs in a defined manner).

::grammar::fa::op::complement *fa*

Complements *fa*. This is possible if and only if *fa* is *complete* and *deterministic*. The resulting FA accepts the complementary language of *fa*. In other words, all inputs not accepted by the input are accepted by the result, and vice versa.

The result will have all states and transitions of the input, and different final states.

::grammar::fa::op::kleene *fa*

Applies Kleene's closure to *fa*. The resulting FA accepts all strings **S** for which we can find a natural number **n** (0 inclusive) and strings **A1** ... **An** in the language of *fa* such that **S** is the concatenation of **A1** ... **An**. In other words, the language of the result is the infinite union over finite length concatenations over the language of *fa*.

The result will have all states and transitions of the input, and new start and final states.

::grammar::fa::op::optional *fa*

Makes the *fa* optional. In other words it computes the FA which accepts the language of *fa* and the empty the word (epsilon) as well.

The result will have all states and transitions of the input, and new start and final states.

::grammar::fa::op::union *fa fb* ?*mapvar*?

Combines the FAs *fa* and *fb* such that the resulting FA accepts the union of the languages of the two FAs.

The result will have all states and transitions of the two input FAs, and new start and final states. All states of *fb* which exist in *fa* as well will be renamed, and the *mapvar* will contain a mapping from the old states of *fb* to the new ones, if present.

It should be noted that the result will be non-deterministic, even if the inputs are deterministic.

::grammar::fa::op::intersect *fa fb* ?*mapvar*?

Combines the FAs *fa* and *fb* such that the resulting FA accepts the intersection of the languages of the two FAs. In other words, the result will accept a word if and only if the word is accepted by both *fa* and *fb*. The result will be useful, but not necessarily deterministic or minimal.

The command will store a dictionary describing the relationship between the new states of the resulting fa and the pairs of states of the input FAs in *mapvar*, if it has been specified. Keys of the dictionary are the handles for the states of the resulting fa, values are pairs of states from the input FAs. Pairs are represented by lists. The first element in each pair will be a state in *fa*, the second element will be drawn from *fb*.

::grammar::fa::op::difference *fa fb* ?*mapvar*?

Combines the FAs *fa* and *fb* such that the resulting FA accepts the difference of the languages of the two FAs. In other words, the result will accept a word if and only if the word is accepted by *fa*, but not by *fb*. This can also be expressed as the intersection of *fa* with the complement of *fb*. The result will be useful, but not necessarily deterministic or minimal.

The command will store a dictionary describing the relationship between the new states of the resulting fa and the pairs of states of the input FAs in *mapvar*, if it has been specified. Keys of the dictionary are the handles for the states of the resulting fa, values are pairs of states from the input FAs. Pairs are represented by lists. The first element in each pair will be a state in *fa*, the second element will be drawn from *fb*.

::grammar::fa::op::concatenate *fa fb* ?*mapvar*?

Combines the FAs *fa* and *fb* such that the resulting FA accepts the cross-product of the languages of the two FAs. I.e. a word W will be accepted by the result if there are two words A and B accepted by *fa*, and *fb* resp. and W is the concatenation of A and B.

The result FA will be non-deterministic.

::grammar::fa::op::fromRegex *fa regex* ?*over*?

Generates a non-deterministic FA which accepts the same language as the regular expression *regex*. If the *over* is specified it is treated as the set of symbols the regular expression and the automaton are defined over. The command will compute the set from the "S" constructors in *regex* when *over* was not specified. This set is important if and only if the complement operator "!" is used in *regex* as the complementary language of an FA is quite different for different sets of symbols.

The regular expression is represented by a nested list, which forms a syntax tree. The following structures are legal:

{S x}

Atomic regular expression. Everything else is constructed from these. Accepts the **S**ymbol "x".

{. A1 A2 ...}

Concatenation operator. Accepts the concatenation of the regular expressions **A1**, **A2**, etc.

Note that this operator accepts zero or more arguments. With zero arguments the represented language is *epsilon*, the empty word.

{| A1 A2 ...}

Choice operator, also called "Alternative". Accepts all input accepted by at least one of the regular expressions **A1**, **A2**, etc. In other words, the union of **A1**, **A2**.

Note that this operator accepts zero or more arguments. With zero arguments the represented language is the *empty* language, the language without words.

{& A1 A2 ...}

Intersection operator, logical and. Accepts all input accepted which is accepted by all of the regular expressions **A1**, **A2**, etc. In other words, the intersection of **A1**, **A2**.

{? A}

Optionality operator. Accepts the empty word and anything from the regular expression **A**.

{* A}

Kleene closure. Accepts the empty word and any finite concatenation of words accepted by the regular expression **A**.

{+ A}

Positive Kleene closure. Accepts any finite concatenation of words accepted by the regular expression **A**, but not the empty word.

{! A}

Complement operator. Accepts any word not accepted by the regular expression **A**. Note that the complement depends on the set of symbol the result should run over. See the discussion of the argument *over* before.

::grammar::fa::op::toRegexp *fa*
This command generates and returns a regular expression which accepts the same language as the finite automaton *fa*. The regular expression is in the format as described above, for **::grammar::fa::op::fromRegex**.

::grammar::fa::op::toRegexp2 *fa*
This command has the same functionality as **::grammar::fa::op::toRegexp**, but uses a different algorithm to simplify the generated regular expressions.

::grammar::fa::op::toTclRegexp *regexp symdict*
This command generates and returns a regular expression in Tcl syntax for the regular expression *regexp*, if that is possible. *regexp* is in the same format as expected by **::grammar::fa::op::fromRegex**.

The command will fail and throw an error if *regexp* contains complementation and intersection operations.

The argument *symdict* is a dictionary mapping symbol names to pairs of *syntactic type* and Tcl-regexp. If a symbol occurring in the *regexp* is not listed in this dictionary then single-character symbols are considered to designate themselves whereas multiple-character symbols are considered to be a character class name.

::grammar::fa::op::simplifyRegexp *regexp*
This command simplifies a regular expression by applying the following algorithm first to the main expression and then recursively to all sub-expressions:
1. Convert the expression into a finite automaton.
2. Minimize the automaton.
3. Convert the automaton back to a regular expression.
4. Choose the shorter of original expression and expression from the previous step.

Examples

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *grammar_fa* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Grammars and finite automata

Copyright

```
Copyright (c) 2004-2008 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

grammar_me

gasm

Keywords: gasm, assembler, grammar, graph, parsing, tree, virtual machine

Name

grammar::me::cpu::gasm - ME assembler

Synopsis

package require grammar::me::cpu::gasm ?**0.2**?
::grammar::me::cpu::gasm::begin *g n* ?*mode*? ?*note*?
**::grammar::me::cpu::gasm::done --> ***t*
::grammar::me::cpu::gasm::state
::grammar::me::cpu::gasm::state! *s*
::grammar::me::cpu::gasm::lift *t dst* **=** *src*
::grammar::me::cpu::gasm::Inline *t node label*
::grammar::me::cpu::gasm::Cmd *cmd* ?*arg...*?
::grammar::me::cpu::gasm::Bra
::grammar::me::cpu::gasm::Nop *text*
::grammar::me::cpu::gasm::Note *text*
::grammar::me::cpu::gasm::Jmp *label*
::grammar::me::cpu::gasm::Exit
::grammar::me::cpu::gasm::Who *label*
::grammar::me::cpu::gasm::/Label *name*
::grammar::me::cpu::gasm::/Clear
::grammar::me::cpu::gasm::/Ok
::grammar::me::cpu::gasm::/Fail
::grammar::me::cpu::gasm::/At *name*
::grammar::me::cpu::gasm::/CloseLoop

Description

This package provides a simple in-memory assembler. Its origin is that of a support package for use by packages converting PEG and other grammars into a corresponding matcher based on the ME virtual machine, like **page::compiler::peg::mecpu**. Despite that it is actually mostly agnostic regarding the instructions, users can choose any instruction set they like.

The program under construction is held in a graph structure (See package **struct::graph**) during assembly and subsequent manipulation, with instructions represented by nodes, and the flow of execution between instructions explicitly encoded in the arcs between them.

In this model jumps are not encoded explicitly, they are implicit in the arcs. The generation of explicit jumps is left to any code converting the graph structure into a more conventional representation. The same goes for branches. They are implicitly encoded by all instructions which have two outgoing arcs, whereas all other instructions have only one outgoing arc. Their conditonality is handled by tagging their outgoing arcs with information about the conditions under which they are taken.

While the graph the assembler operates on is supplied from the outside, i.e. external, it does

manage some internal state, namely:

1. The handle of the graph node most assembler operations will work on, the *anchor*.
2. A mapping from arbitrary strings to instructions. I.e. it is possible to *label* an instruction during assembly, and later recall that instruction by its label.
3. The condition code to use when creating arcs between instructions, which is one of **always**, **ok**, and **fail**.
4. The current operation mode, one of **halt**, **okfail**, and**!okfail**.
5. The name of a node in a tree. This, and the operation mode above are the parts most heavily influenced by the needs of a grammar compiler, as they assume some basic program structures (selected through the operation mode), and intertwine the graph with a tree, like the AST for the grammar to be compiled.

Definitions

As the graph the assembler is operating on, and the tree it is intertwined with, are supplied to the assembler from the outside it is necessary to specify the API expected from them, and to describe the structures expected and/or generated by the assembler in either.

1. Any graph object command used by the assembler has to provide the API as specified in the documentation for the package **struct::graph**.
2. Any tree object command used by the assembler has to provide the API as specified in the documentation for the package **struct::tree**.
3. Any instruction (node) generated by the assembler in a graph will have at least two, and at most three attributes:

   instruction
   : The value of this attribute is the name of the instruction. The only names currently defined by the assembler are the three pseudo-instructions

     NOP
     : This instruction does nothing. Useful for fixed framework nodes, unchanging jump destinations, and the like. No arguments.

     C
     : A .NOP to allow the insertion of arbitrary comments into the instruction stream, i.e. a comment node. One argument, the text of the comment.

     BRA
     : A .NOP serving as explicitly coded conditional branch. No arguments.

```
    However we reserve the space of all instructions whose names begin
    with a "." (dot) for future use by the assembler.

**arguments**
: The value of this attribute is a list of strings, the arguments of
    the instruction. The contents are dependent on the actual
    instruction and the assembler doesn't know or care about them.
    This means for example that it has no builtin knowledge about what
    instruction need which arguments and thus doesn't perform any type
    of checking.
```

```
**expr**
: This attribute is optional. When it is present its value is the name
    of a node in the tree intertwined with the graph.
```

1. Any arc between two instructions will have one attribute:
   condition
   The value of this attribute determines under which condition execution will take this arc. It is one of **always**, **ok**, and **fail**. The first condition is used for all arcs which are the single outgoing arc of an instruction. The other two are used for the two outgoing arcs of an instruction which implicitly encode a branch.

1. A tree node given to the assembler for cross-referencing will be written to and given the following attributes, some fixed, some dependent on the operation mode. All values will be references to nodes in the instruction graph. Some of the instruction will expect some or specific sets of these attributes.
   gas::entry
   Always written.
   gas::exit
   Written for all modes but **okfail**.
   gas::exit::ok
   Written for mode **okfail**.
   gas::exit::fail
   Written for mode **okfail**.

API

::grammar::me::cpu::gasm::begin *g n* ?*mode*? ?*note*?

This command starts the assembly of an instruction sequence, and (re)initializes the state of the assembler. After completion of the instruction sequence use **::grammar::me::cpu::gasm::done** to finalize the assembler.

It will operate on the graph *g* in the specified *mode* (Default is **okfail**). As part of the initialization it will always create a standard .NOP instruction and label it "entry". The creation of the remaining standard instructions is *mode*-dependent:

halt
An "icf_halt" instruction labeled "exit/return".

!okfail
An "icf_ntreturn" instruction labeled "exit/return".

okfail
Two .NOP instructions labeled "exit/ok" and "exit/fail" respectively.

The *note*, if specified (default is not), is given to the "entry" .NOP instruction.

The node reference *n* is simply stored for use by **::grammar::me::cpu::gasm::done**. It has to refer to a node in the tree *t* argument of that command.

After the initialization is done the "entry" instruction will be the *anchor*, and the condition code will be set to **always**.

The command returns the empy string as its result.

::grammar::me::cpu::gasm::done --> *t*

This command finalizes the creation of an instruction sequence and then clears the state of the assembler. *NOTE* that this *does not* delete any of the created instructions. They can be made available to future begin/done cycles. Further assembly will be possible only after reinitialization of the system via **::grammar::me::cpu::gasm::begin**.

Before the state is cleared selected references to selected instructions will be written to attributes of the node *n* in the tree *t*. Which instructions are saved is *mode*-dependent. Both *mode* and the destination node *n* were specified during invokation of **::grammar::me::cpu::gasm::begin**.

Independent of the mode a reference to the instruction labeled "entry" will be saved to the attribute **gas::entry** of *n*. The reference to the node *n* will further be saved into the attribute "expr" of the "entry" instruction. Beyond that

halt
: A reference to the instruction labeled "exit/return" will be saved to the attribute **gas::exit** of *n*.

okfail
: See **halt**.

!okfail
: Reference to the two instructions labeled "exit/ok" and "exit/fail" will be saved to the attributes **gas::exit::ok** and **gas::exit::fail** of *n* respectively.

The command returns the empy string as its result.

::grammar::me::cpu::gasm::state

This command returns the current state of the assembler. Its format is not documented and considered to be internal to the package.

::grammar::me::cpu::gasm::state! *s*

This command takes a serialized assembler state *s* as returned by **::grammar::me::cpu::gasm::state** and makes it the current state of the assembler.

Note that this may overwrite label definitions, however all non-conflicting label definitions in the state before are not touched and merged with *s*.

The command returns the empty string as its result.

::grammar::me::cpu::gasm::lift *t dst = src*

This command operates on the tree *t*. It copies the contents of the attributes **gas::entry**, **gas::exit::ok** and **gas::exit::fail** from the node *src* to the node *dst*. It returns the empty string as its result.

::grammar::me::cpu::gasm::Inline *t node label*

This command links an instruction sequence created by an earlier begin/done pair into the current instruction sequence.

To this end it

1. reads the instruction references from the attributes **gas::entry**, **gas::exit::ok**, and **gas::exit::fail** from the node *n* of the tree *t* and makes them available to assembler und

the labels *label*/entry, *label*/exit::ok, and *label*/exit::fail respectively.
2. Creates an arc from the *anchor* to the node labeled *label*/entry, and tags it with the current condition code.
3. Makes the node labeled *label*/exit/ok the new *anchor*.

The command returns the empty string as its result.

::grammar::me::cpu::gasm::Cmd *cmd* ?*arg*...?
This is the basic command to add instructions to the graph. It creates a new instruction of type *cmd* with the given arguments *arg*... If the *anchor* was defined it will also create an arc from the *anchor* to the new instruction using the current condition code. After the call the new instruction will be the *anchor* and the current condition code will be set to **always**.

The command returns the empty string as its result.

::grammar::me::cpu::gasm::Bra
This is a convenience command to create a .BRA pseudo-instruction. It uses **::grammar::me::cpu::gasm::Cmd** to actually create the instruction and inherits its behaviour.

::grammar::me::cpu::gasm::Nop *text*
This is a convenience command to create a .NOP pseudo-instruction. It uses **::grammar::me::cpu::gasm::Cmd** to actually create the instruction and inherits its behaviour. The *text* will be saved as the first and only argument of the new instruction.

::grammar::me::cpu::gasm::Note *text*
This is a convenience command to create a .C pseudo-instruction, i.e. a comment. It uses **::grammar::me::cpu::gasm::Cmd** to actually create the instruction and inherits its behaviour. The *text* will be saved as the first and only argument of the new instruction.

::grammar::me::cpu::gasm::Jmp *label*
This command creates an arc from the *anchor* to the instruction labeled with *label*, and tags with the the current condition code.

The command returns the empty string as its result.

::grammar::me::cpu::gasm::Exit
This command creates an arc from the *anchor* to one of the exit instructions, based on the operation mode (see **::grammar::me::cpu::gasm::begin**), and tags it with current condition code.

For mode **okfail** it links to the instruction labeled either "exit/ok" or "exit/fail", depending on the current condition code, and tagging it with the current condition code For the other two modes it links to the instruction labeled "exit/return", tagging it condition code **always**, independent the current condition code.

The command returns the empty string as its result.

::grammar::me::cpu::gasm::Who *label*
This command returns a reference to the instruction labeled with *label*.

::grammar::me::cpu::gasm::/Label *name*
This command labels the *anchor* with *name*. *Note* that an instruction can have more than one

label.

The command returns the empty string as its result.

::grammar::me::cpu::gasm::/Clear
This command clears the *anchor*, leaving it undefined, and further resets the current condition code to **always**.

The command returns the empty string as its result.

::grammar::me::cpu::gasm::/Ok
This command sets the current condition code to **ok**.

The command returns the empty string as its result.

::grammar::me::cpu::gasm::/Fail
This command sets the current condition code to **fail**.

The command returns the empty string as its result.

::grammar::me::cpu::gasm::/At *name*
This command sets the *anchor* to the instruction labeled with *name*, and further resets the current condition code to **always**.

The command returns the empty string as its result.

::grammar::me::cpu::gasm::/CloseLoop
This command marks the *anchor* as the last instruction in a loop body, by creating the attribute **LOOP**.

The command returns the empty string as its result.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *grammar_me* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Grammars and finite automata

Copyright

```
Copyright (c) 2005 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

me_ast

Keywords: me_ast, AST, abstract syntax tree

Name

grammar::me_ast - Various representations of ASTs

Description

This document specifies various representations for the *abstract syntax tree*s (short *AST*) generated by instances of ME virtual machines, independent of variant. Please go and read the document grammar::me_intro first if you do not know what a ME virtual machine is.

ASTs and all the representations we specify distinguish between two types of nodes, namely:

Terminal
: Terminal nodes refer to the terminal symbols found in the token stream. They are always leaf nodes. I.e. terminal nodes never have children.

Nonterminal
: Nonterminal nodes represent a nonterminal symbol of the grammar used during parsing. They can occur as leaf and inner nodes of the tree.

Both types of nodes carry basic range information telling a user which parts of the input are covered by the node by providing the location of the first and last tokens found within the range. Locations are provided as non-negative integer offsets from the beginning of the token stream, with the first token found in the stream located at offset 0 (zero).

The root of an AS tree can be either a terminal or nonterminal node.

AST values

This representation of ASTs is a Tcl list. The main list represents the root node of the tree, with the representations of the children nested within.

Each node is represented by a single Tcl list containing three or more elements. The first element is either the empty string or the name of a nonterminal symbol (which is never the empty string). The second and third elements are then the locations of the first and last tokens. Any additional elements after the third are then the representations of the children, with the leftmost child first, i.e. as the fourth element of the list representing the node.

AST objects

In this representation an AST is represented by a Tcl object command whose API is compatible to the tree objects provided by the package **struct::tree**. I.e it has to support at least all of the methods described by that package, and may support more.

Because of this the remainder of the specifications is written using the terms of **struct::tree**.

Each node of the AST directly maps to a node in the tree object. All data beyond the child nodes, i.e. node type and input locations, are stored in attributes of the node in the tree object. They are:

type
: The type of the AST node. The recognized values are **terminal** and **nonterminal**.

range
: The locations of the first and last token of the terminal data in the input covered by the node. This is a list containing two locations.

detail
: This attribute is present only for nonterminal nodes. It contains the name of the nonterminal

symbol stored in the node.

Extended AST objects

Extended AST objects are like AST objects, with additional information.

detail
: This attribute is now present at all nodes. Its contents are unchanged for nonterminal nodes. For terminal nodes it contains a list describing all tokens from the input which are covered by the node.

  Each element of the list contains the token name, the associated lexeme attribute, line number, and column index, in this order.

range_lc
: This new attribute is defined for all nodes, and contains the locations from attribute *range* translated into line number and column index. Lines are counted from 1, columns are counted from 0.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *grammar_me* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Grammars and finite automata

Copyright


```
Copyright (c) 2005 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


me_cpu

Keywords: me_cpu, grammar, parsing, virtual machine

Name

grammar::me::cpu - Virtual machine implementation II for parsing token streams

Synopsis

package require Tcl 8.5 9
package require grammar::me::cpu ?**0.3**?
::grammar::me::cpu *meName matchcode*
meName option ?*arg arg* ...?
meName **lc** *location*
meName **tok** ?*from* ?*to*?

meName **pc** *state*
meName **iseof** *state*
meName **at** *state*
meName **cc** *state*
meName **sv**
meName **ok**
meName **error**
meName **lstk** *state*
meName **astk** *state*
meName **mstk** *state*
meName **estk** *state*
meName **rstk** *state*
meName **nc** *state*
meName **ast**
meName **halted**
meName **code**
meName **eof**
meName **put** *tok lex line col*
meName **putstring** *string lvar cvar*
meName **run** ?*n*?
meName **pull** *nextcmd*
meName **reset**
meName **destroy**

Description

This package provides an implementation of the ME virtual machine. Please go and read the document grammar::me_intro first if you do not know what a ME virtual machine is.

This implementation provides an object-based API and the machines are not truly tied to Tcl. A C implementation of the same API is quite possible.

Internally the package actually uses the value-based machine manipulation commands as provided by the package **grammar::me::cpu::core** to perform its duties.

API

Class API

The package directly provides only a single command for the construction of ME virtual machines.

::grammar::me::cpu *meName matchcode*

The command creates a new ME machine object with an associated global Tcl command whose name is *meName*. This command may be used to invoke various operations on the machine. It has the following general form:

meName option ?*arg arg ...*?

Option and the *arg*s determine the exact behavior of the command.

The argument *matchcode* contains the match instructions the machine has to execute while parsing the input stream. Please read section **MATCH CODE REPRESENTATION** of the

documentation for the package **grammar::me::cpu::core** for the specification of the structure of this value.

The *tokmap* argument taken by the implementation provided by the package **grammar::me::tcl** is here hidden inside of the match instructions and therefore not needed.

Object API

All ME virtual machine objects created by the class command specified in section Class API support the methods listed below.

The machines provided by this package provide methods for operation in both push- and pull-styles. Push-style means that tokens are pushed into the machine state when they arrive, triggering further execution until they are consumed. In other words, this allows the machine to be suspended and resumed at will and an arbitrary number of times, the quasi-parallel operation of several machines, and the operation as part of the event loop.

meName* lc *location

This method converts the location of a token given as offset in the input stream into the associated line number and column index. The result of the command is a 2-element list containing the two values, in the order mentioned in the previous sentence. This allows higher levels to convert the location information found in the error status and the generated AST into more human readable data.

Note that the command is not able to convert locations which have not been reached by the machine yet. In other words, if the machine has read 7 tokens the command is able to convert the offsets **0** to **6**, but nothing beyond that. This also shows that it is not possible to convert offsets which refer to locations before the beginning of the stream.

***meName* tok ?*from* ?*to*??**

This method returns a Tcl list containing the part of the input stream between the locations *from* and *to* (both inclusive). If *to* is not specified it will default to the value of *from*. If *from* is not specified either the whole input stream is returned.

Each element of the returned list is a list of four elements, the token, its associated lexeme, line number, and column index, in this order. This command places the same restrictions on its location arguments as the method **lc**.

meName* pc *state

This method takes the state value of a ME virtual machine and returns the current value of the stored program counter.

meName* iseof *state

This method takes the state value of a ME virtual machine and returns the current value of the stored eof flag.

meName* at *state

This method takes the state value of a ME virtual machine and returns the current location in the input stream.

meName* cc *state

This method takes the state value of a ME virtual machine and returns the current token.

***meName* sv**

This command returns the current semantic value *SV* stored in the machine. This is an

abstract syntax tree as specified in the document grammar::me_ast, section **AST VALUES**.

***meName* ok**
This method returns the current match status *OK*.

***meName* error**
This method returns the current error status *ER*.

meName* lstk *state
This method takes the state value of a ME virtual machine and returns the location stack.

meName* astk *state
This method takes the state value of a ME virtual machine and returns the AST stack.

meName* mstk *state
This method takes the state value of a ME virtual machine and returns the AST marker stack.

meName* estk *state
This method takes the state value of a ME virtual machine and returns the error stack.

meName* rstk *state
This method takes the state value of a ME virtual machine and returns the subroutine return stack.

meName* nc *state
This method takes the state value of a ME virtual machine and returns the nonterminal match cache as a dictionary.

***meName* ast**
This method returns the current top entry of the AST stack *AS*. This is an abstract syntax tree as specified in the document grammar::me_ast, section **AST VALUES**.

***meName* halted**
This method returns a boolean value telling the caller whether the engine has halted execution or not. Halt means that no further matching is possible, and the information retrieved via the other method is final. Attempts to **run** the engine will be ignored, until a **reset** is made.

***meName* code**
This method returns the *code* information used to construct the object. In other words, the match program executed by the machine.

***meName* eof**
This method adds an end of file marker to the end of the input stream. This signals the machine that the current contents of the input queue are the final parts of the input and nothing will come after. Attempts to put more characters into the queue will fail.

meName* put *tok lex line col
This method adds the token *tok* to the end of the input stream, with associated lexeme data *lex* and *line*/*col*umn information.

meName* putstring *string lvar cvar
This method adds each individual character in the *string* as a token to the end of the input stream, from first to last. The lexemes will be empty and the line/col information is computed based on the characters encountered and the data in the variables *lvar* and *cvar*.

***meName* run ?*n*?**
This methods causes the engine to execute match instructions until either
- *n* instructions have been executed, or

- a halt instruction was executed, or
- the input queue is empty and the code is asking for more tokens to process.

If no limit *n* was set only the last two conditions are checked for.

meName* pull *nextcmd

This method implements pull-style operation of the machine. It causes it to execute match instructions until either a halt instruction is reached, or the command prefix *nextcmd* ceases to deliver more tokens.

The command prefix *nextcmd* represents the input stream of characters and is invoked by the machine whenever the a new character from the stream is required. The instruction for handling this is *ict_advance*. The callback has to return either the empty list, or a list of 4 elements containing the token, its lexeme attribute, and its location as line number and column index, in this order. The empty list is the signal that the end of the input stream has been reached. The lexeme attribute is stored in the terminal cache, but otherwise not used by the machine.

The end of the input stream for this method does not imply that method **eof** is called for the machine as a whole. By avoiding this and still asking for an explicit call of the method it is possible to mix push- and pull-style operation during the lifetime of the machine.

***meName* reset**

This method resets the machine to its initial state, discarding any state it may have.

***meName* destroy**

This method deletes the object and releases all resurces it claimed.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *grammar_me* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Grammars and finite automata

Copyright


```
Copyright (c) 2005-2006 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


me_cpucore

Keywords: me_cpucore, grammar, parsing, virtual machine

Name

grammar::me::cpu::core - ME virtual machine state manipulation

Synopsis

package require Tcl 8.5 9
package require grammar::me::cpu::core ?**0.4**?
::grammar::me::cpu::core disasm *asm*
::grammar::me::cpu::core asm *asm*
::grammar::me::cpu::core new *asm*
::grammar::me::cpu::core lc *state location*
::grammar::me::cpu::core tok *state* ?*from* ?*to*?
::grammar::me::cpu::core pc *state*
::grammar::me::cpu::core iseof *state*
::grammar::me::cpu::core at *state*
::grammar::me::cpu::core cc *state*
::grammar::me::cpu::core sv *state*
::grammar::me::cpu::core ok *state*
::grammar::me::cpu::core error *state*
::grammar::me::cpu::core lstk *state*
::grammar::me::cpu::core astk *state*
::grammar::me::cpu::core mstk *state*
::grammar::me::cpu::core estk *state*
::grammar::me::cpu::core rstk *state*
::grammar::me::cpu::core nc *state*
::grammar::me::cpu::core ast *state*
::grammar::me::cpu::core halted *state*
::grammar::me::cpu::core code *state*
::grammar::me::cpu::core eof *statevar*
::grammar::me::cpu::core put *statevar tok lex line col*
::grammar::me::cpu::core run *statevar* ?*n*?

Description

This package provides an implementation of the ME virtual machine. Please go and read the document grammar::me_intro first if you do not know what a ME virtual machine is.

This implementation represents each ME virtual machine as a Tcl value and provides commands to manipulate and query such values to show the effects of executing instructions, adding tokens, retrieving state, etc.

The values fully follow the paradigm of Tcl that every value is a string and while also allowing C implementations for a proper Tcl_ObjType to keep all the important data in native data structures. Because of the latter it is recommended to access the state values *only* through the commands of this package to ensure that internal representation is not shimmered away.

The actual structure used by all state values is described in section Cpu state.

API

The package directly provides only a single command, and all the functionality is made available through its methods.

::grammar::me::cpu::core disasm *asm*

This method returns a list containing a disassembly of the match instructions in *asm*. The format of *asm* is specified in the section Match program representation.

Each element of the result contains instruction label, instruction name, and the instruction arguments, in this order. The label can be the empty string. Jump destinations are shown as labels, strings and tokens unencoded. Token names are prefixed with their numeric id, if, and only if a tokmap is defined. The two components are separated by a colon.

::grammar::me::cpu::core asm *asm*

This method returns code in the format as specified in section Match program representation generated from ME assembly code *asm*, which is in the format as returned by the method **disasm**.

::grammar::me::cpu::core new *asm*

This method creates state value for a ME virtual machine in its initial state and returns it as its result.

The argument *matchcode* contains a Tcl representation of the match instructions the machine has to execute while parsing the input stream. Its format is specified in the section Match program representation.

The *tokmap* argument taken by the implementation provided by the package **grammar::me::tcl** is here hidden inside of the match instructions and therefore not needed.

::grammar::me::cpu::core lc *state location*

This method takes the state value of a ME virtual machine and uses it to convert a location in the input stream (as offset) into a line number and column index. The result of the method is a 2-element list containing the two pieces in the order mentioned in the previous sentence.

Note that the method cannot convert locations which the machine has not yet read from the input stream. In other words, if the machine has read 7 characters so far it is possible to convert the offsets **0** to **6**, but nothing beyond that. This also shows that it is not possible to convert offsets which refer to locations before the beginning of the stream.

This utility allows higher levels to convert the location offsets found in the error status and the AST into more human readable data.

::grammar::me::cpu::core tok *state* ?*from* ?*to*??

This method takes the state value of a ME virtual machine and returns a Tcl list containing the part of the input stream between the locations *from* and *to* (both inclusive). If *to* is not specified it will default to the value of *from*. If *from* is not specified either the whole input stream is returned.

This method places the same restrictions on its location arguments as the method **lc**.

::grammar::me::cpu::core pc *state*

This method takes the state value of a ME virtual machine and returns the current value of the stored program counter.

::grammar::me::cpu::core iseof *state*

This method takes the state value of a ME virtual machine and returns the current value of

the stored eof flag.

::grammar::me::cpu::core at *state*
This method takes the state value of a ME virtual machine and returns the current location in the input stream.

::grammar::me::cpu::core cc *state*
This method takes the state value of a ME virtual machine and returns the current token.

::grammar::me::cpu::core sv *state*
This method takes the state value of a ME virtual machine and returns the current semantic value stored in it. This is an abstract syntax tree as specified in the document grammar::me_ast, section **AST VALUES**.

::grammar::me::cpu::core ok *state*
This method takes the state value of a ME virtual machine and returns the match status stored in it.

::grammar::me::cpu::core error *state*
This method takes the state value of a ME virtual machine and returns the current error status stored in it.

::grammar::me::cpu::core lstk *state*
This method takes the state value of a ME virtual machine and returns the location stack.

::grammar::me::cpu::core astk *state*
This method takes the state value of a ME virtual machine and returns the AST stack.

::grammar::me::cpu::core mstk *state*
This method takes the state value of a ME virtual machine and returns the AST marker stack.

::grammar::me::cpu::core estk *state*
This method takes the state value of a ME virtual machine and returns the error stack.

::grammar::me::cpu::core rstk *state*
This method takes the state value of a ME virtual machine and returns the subroutine return stack.

::grammar::me::cpu::core nc *state*
This method takes the state value of a ME virtual machine and returns the nonterminal match cache as a dictionary.

::grammar::me::cpu::core ast *state*
This method takes the state value of a ME virtual machine and returns the abstract syntax tree currently at the top of the AST stack stored in it. This is an abstract syntax tree as specified in the document grammar::me_ast, section **AST VALUES**.

::grammar::me::cpu::core halted *state*
This method takes the state value of a ME virtual machine and returns the current halt status stored in it, i.e. if the machine has stopped or not.

::grammar::me::cpu::core code *state*
This method takes the state value of a ME virtual machine and returns the code stored in it, i.e. the instructions executed by the machine.

::grammar::me::cpu::core eof *statevar*
This method takes the state value of a ME virtual machine as stored in the variable named by *statevar* and modifies it so that the eof flag inside is set. This signals to the machine that whatever token are in the input queue are the last to be processed. There will be no more.

::grammar::me::cpu::core put *statevar tok lex line col*

This method takes the state value of a ME virtual machine as stored in the variable named by *statevar* and modifies it so that the token *tok* is added to the end of the input queue, with associated lexeme data *lex* and *line*/*col*umn information.

The operation will fail with an error if the eof flag of the machine has been set through the method **eof**.

::grammar::me::cpu::core run *statevar* ?*n*?

This method takes the state value of a ME virtual machine as stored in the variable named by *statevar*, executes a number of instructions and stores the state resulting from their modifications back into the variable.

The execution loop will run until either
- *n* instructions have been executed, or
- a halt instruction was executed, or
- the input queue is empty and the code is asking for more tokens to process.

If no limit *n* was set only the last two conditions are checked for.

Match program representation

A match program is represented by nested Tcl list. The first element, *asm*, is a list of integer numbers, the instructions to execute, and their arguments. The second element, *pool*, is a list of strings, referenced by the instructions, for error messages, token names, etc. The third element, *tokmap*, provides ordering information for the tokens, mapping their names to their numerical rank. This element can be empty, forcing lexicographic comparison when matching ranges.

All ME instructions are encoded as integer numbers, with the mapping given below. A number of the instructions, those which handle error messages, have been given an additional argument to supply that message explicitly instead of having it constructed from token names, etc. This allows the machine state to store only the message ids instead of the full strings.

Jump destination arguments are absolute indices into the *asm* element, refering to the instruction to jump to. Any string arguments are absolute indices into the *pool* element. Tokens, characters, messages, and token (actually character) classes to match are coded as references into the *pool* as well.

1. "**ict_advance** *message*"
2. "**ict_match_token** *tok message*"
3. "**ict_match_tokrange** *tokbegin tokend message*"
4. "**ict_match_tokclass** *code message*"
5. "**inc_restore** *branchlabel nt*"
6. "**inc_save** *nt*"
7. "**icf_ntcall** *branchlabel*"
8. "**icf_ntreturn**"
9. "**iok_ok**"
10. "**iok_fail**"
11. "**iok_negate**"
12. "**icf_jalways** *branchlabel*"
13. "**icf_jok** *branchlabel*"

14. "**icf_jfail** *branchlabel*"
15. "**icf_halt**"
16. "**icl_push**"
17. "**icl_rewind**"
18. "**icl_pop**"
19. "**ier_push**"
20. "**ier_clear**"
21. "**ier_nonterminal** *message*"
22. "**ier_merge**"
23. "**isv_clear**"
24. "**isv_terminal**"
25. "**isv_nonterminal_leaf** *nt*"
26. "**isv_nonterminal_range** *nt*"
27. "**isv_nonterminal_reduce** *nt*"
28. "**ias_push**"
29. "**ias_mark**"
30. "**ias_mrewind**"
31. "**ias_mpop**"

Cpu state

A state value is a list containing the following elements, in the order listed below:

1. *code*: Match instructions, see Match program representation.
2. *pc*: Program counter, *int*.
3. *halt*: Halt flag, *boolean*.
4. *eof*: Eof flag, *boolean*
5. *tc*: Terminal cache, and input queue. Structure see below.
6. *cl*: Current location, *int*.
7. *ct*: Current token, *string*.
8. *ok*: Match status, *boolean*.
9. *sv*: Semantic value, *list*.
10. *er*: Error status, *list*.
11. *ls*: Location stack, *list*.
12. *as*: AST stack, *list*.
13. *ms*: AST marker stack, *list*.
14. *es*: Error stack, *list*.
15. *rs*: Return stack, *list*.
16. *nc*: Nonterminal cache, *dictionary*.

tc, the input queue of tokens waiting for processing and the terminal cache containing the tokens already processing are one unified data structure simply holding all tokens and their information, with the current location separating that which has been processed from that which is waiting. Each element of the queue/cache is a list containing the token, its lexeme information, line number, and column index, in this order.

All stacks have their top element aat the end, i.e. pushing an item is equivalent to appending to the list representing the stack, and popping it removes the last element.

er, the error status is either empty or a list of two elements, a location in the input, and a list of

messages, encoded as references into the *pool* element of the *code*.

nc, the nonterminal cache is keyed by nonterminal name and location, each value a four-element list containing current location, match status, semantic value, and error status, in this order.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *grammar_me* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Grammars and finite automata

Copyright


```
Copyright (c) 2005-2006 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


me_intro

Keywords: me_intro, CFG, CFL, LL(k), PEG, TPDL, context-free grammar, context-free languages, expression, grammar, matching, parsing, parsing expression grammar, push down automaton, recursive descent, top-down parsing languages, transducer, virtual machine

Name

grammar::me_intro - Introduction to virtual machines for parsing token streams

Description

This document is an introduction to and overview of the basic facilities for the parsing and/or matching of *token* streams. One possibility often used for the token domain are characters.

The packages themselves all provide variants of one *virtual machine*, called a *match engine* (short *ME*), which has all the facilities needed for the matching and parsing of a stream, and which are either controlled directly, or are customized with a match program. The virtual machine is basically a pushdown automaton, with additional elements for backtracking and/or handling of semantic data and construction of abstract syntax trees (*AST*).

Because of the high degree of similarity in the actual implementations of the aforementioned virtual machine and the data structures they receive and generate these common parts are specified in a separate document which will be referenced by the documentation for packages actually implementing it.

The relevant documents are:

grammar::me_vm
: Virtual machine specification.

grammar::me_ast
: Specification of various representations used for abstract syntax trees.

grammar::me::util
: Utility commands.

grammar::me::tcl
: Singleton ME virtual machine implementation tied to Tcl for control flow and stacks. Hardwired for pull operation. Uninteruptible during processing.

grammar::me::cpu
: Object-based ME virtual machine implementation with explicit control flow, and stacks, using bytecodes. Suspend/Resumable. Push/pull operation.

grammar::me::cpu::core
: Core functionality for state manipulation and stepping used in the bytecode based implementation of ME virtual machines.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *grammar_me* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Grammars and finite automata

Copyright


```
Copyright (c) 2005 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


me_tcl

Keywords: me_tcl, grammar, parsing, virtual machine

Name

grammar::me::tcl - Virtual machine implementation I for parsing token streams

Synopsis

package require Tcl 8.5 9
package require grammar::me::tcl ?**0.2**?
::grammar::me::tcl cmd *...*
::grammar::me::tcl init *nextcmd* ?*tokmap*?
::grammar::me::tcl lc *location*
::grammar::me::tcl tok *from* ?*to*?
::grammar::me::tcl tokens
::grammar::me::tcl sv

::grammar::me::tcl ast
::grammar::me::tcl astall
::grammar::me::tcl ctok
::grammar::me::tcl nc
::grammar::me::tcl next
::grammar::me::tcl ord
::grammar::me::tcl::ict_advance *message*
::grammar::me::tcl::ict_match_token *tok message*
::grammar::me::tcl::ict_match_tokrange *tokbegin tokend message*
::grammar::me::tcl::ict_match_tokclass *code message*
::grammar::me::tcl::inc_restore *nt*
::grammar::me::tcl::inc_save *nt startlocation*
::grammar::me::tcl::iok_ok
::grammar::me::tcl::iok_fail
::grammar::me::tcl::iok_negate
::grammar::me::tcl::icl_get
::grammar::me::tcl::icl_rewind *oldlocation*
::grammar::me::tcl::ier_get
::grammar::me::tcl::ier_clear
::grammar::me::tcl::ier_nonterminal *message location*
::grammar::me::tcl::ier_merge *olderror*
::grammar::me::tcl::isv_clear
::grammar::me::tcl::isv_terminal
::grammar::me::tcl::isv_nonterminal_leaf *nt startlocation*
::grammar::me::tcl::isv_nonterminal_range *nt startlocation*
::grammar::me::tcl::isv_nonterminal_reduce *nt startlocation* ?*marker*?
::grammar::me::tcl::ias_push
::grammar::me::tcl::ias_mark
::grammar::me::tcl::ias_pop2mark *marker*

Description

This package provides an implementation of the ME virtual machine. Please go and read the document grammar::me_intro first if you do not know what a ME virtual machine is.

This implementation is tied very strongly to Tcl. All the stacks in the machine state are handled through the Tcl stack, all control flow is handled by Tcl commands, and the remaining machine instructions are directly mapped to Tcl commands. Especially the matching of nonterminal symbols is handled by Tcl procedures as well, essentially extending the machine implementation with custom instructions.

Further on the implementation handles only a single machine which is uninteruptible during execution and hardwired for pull operation. I.e. it explicitly requests each new token through a callback, pulling them into its state.

A related package is **grammar::peg::interp** which provides a generic interpreter / parser for parsing expression grammars (PEGs), implemented on top of this implementation of the ME virtual machine.

API

The commands documented in this section do not implement any of the instructions of the ME virtual machine. They provide the facilities for the initialization of the machine and the retrieval of important information.

::grammar::me::tcl cmd*...*

This is an ensemble command providing access to the commands listed in this section. See the methods themselves for detailed specifications.

::grammar::me::tcl init *nextcmd* ?*tokmap*?

This command (re)initializes the machine. It returns the empty string. This command has to be invoked before any other command of this package.

The command prefix *nextcmd* represents the input stream of characters and is invoked by the machine whenever the a new character from the stream is required. The instruction for handling this is *ict_advance*. The callback has to return either the empty list, or a list of 4 elements containing the token, its lexeme attribute, and its location as line number and column index, in this order. The empty list is the signal that the end of the input stream has been reached. The lexeme attribute is stored in the terminal cache, but otherwise not used by the machine.

The optional dictionary *tokmap* maps from tokens to integer numbers. If present the numbers impose an order on the tokens, which is subsequently used by *ict_match_tokrange* to determine if a token is in the specified range or not. If no token map is specified the lexicographic order of th token names will be used instead. This choice is especially asensible when using characters as tokens.

::grammar::me::tcl lc *location*

This command converts the location of a token given as offset in the input stream into the associated line number and column index. The result of the command is a 2-element list containing the two values, in the order mentioned in the previous sentence. This allows higher levels to convert the location information found in the error status and the generated AST into more human readable data.

Note that the command is not able to convert locations which have not been reached by the machine yet. In other words, if the machine has read 7 tokens the command is able to convert the offsets **0** to **6**, but nothing beyond that. This also shows that it is not possible to convert offsets which refer to locations before the beginning of the stream.

After a call of **init** the state used for the conversion is cleared, making further conversions impossible until the machine has read tokens again.

::grammar::me::tcl tok *from* ?*to*?

This command returns a Tcl list containing the part of the input stream between the locations *from* and *to* (both inclusive). If *to* is not specified it will default to the value of *from*.

Each element of the returned list is a list of four elements, the token, its associated lexeme, line number, and column index, in this order. In other words, each element has the same structure as the result of the *nextcmd* callback given to **::grammar::me::tcl::init**

This command places the same restrictions on its location arguments as **::grammar::me::tcl::lc**.

::grammar::me::tcl tokens
This command returns the number of tokens currently known to the ME virtual machine.

::grammar::me::tcl sv
This command returns the current semantic value *SV* stored in the machine. This is an abstract syntax tree as specified in the document grammar::me_ast, section **AST VALUES**.

::grammar::me::tcl ast
This method returns the abstract syntax tree currently at the top of the AST stack of the ME virtual machine. This is an abstract syntax tree as specified in the document grammar::me_ast, section **AST VALUES**.

::grammar::me::tcl astall
This method returns the whole stack of abstract syntax trees currently known to the ME virtual machine. Each element of the returned list is an abstract syntax tree as specified in the document grammar::me_ast, section **AST VALUES**. The top of the stack resides at the end of the list.

::grammar::me::tcl ctok
This method returns the current token considered by the ME virtual machine.

::grammar::me::tcl nc
This method returns the contents of the nonterminal cache as a dictionary mapping from "**symbol**,**location**" to match information.

::grammar::me::tcl next
This method returns the next token callback as specified during initialization of the ME virtual machine.

::grammar::me::tcl ord
This method returns a dictionary containing the *tokmap* specified during initialization of the ME virtual machine. **::grammar::me::tcl::ok** This variable contains the current match status *OK*. It is provided as variable instead of a command because that makes access to this information faster, and the speed of access is considered very important here as this information is used constantly to determine the control flow.

Machine state

Please go and read the document grammar::me_vm first for a specification of the basic ME virtual machine and its state.

This implementation manages the state described in that document, except for the stacks minus the AST stack. In other words, location stack, error stack, return stack, and ast marker stack are implicitly managed through standard Tcl scoping, i.e. Tcl variables in procedures, outside of this implementation.

Machine instructions

Please go and read the document grammar::me_vm first for a specification of the basic ME virtual machine and its instruction set.

This implementation maps all instructions to Tcl commands in the namespace "::grammar::me::tcl", except for the stack related commands, nonterminal symbols and control

flow. Here we simply list the commands and explain the differences to the specified instructions, if there are any. For their semantics see the aforementioned specification. The machine commands are *not* reachable through the ensemble command **::grammar::me::tcl**.

::grammar::me::tcl::ict_advance *message*
No changes.

::grammar::me::tcl::ict_match_token *tok message*
No changes.

::grammar::me::tcl::ict_match_tokrange *tokbegin tokend message*
If, and only if a token map was specified during initialization then the arguments are the numeric representations of the smallest and largest tokens in the range. Otherwise they are the relevant tokens themselves and lexicographic comparison is used.

::grammar::me::tcl::ict_match_tokclass *code message*
No changes.

::grammar::me::tcl::inc_restore *nt*
Instead of taking a branchlabel the command returns a boolean value. The result will be **true** if and only if cached information was found. The caller has to perform the appropriate branching.

::grammar::me::tcl::inc_save *nt startlocation*
The command takes the start location as additional argument, as it is managed on the Tcl stack, and not in the machine state.

icf_ntcall *branchlabel*
see below ...

icf_ntreturn
These two instructions are not mapped to commands. They are control flow instructions and handled in Tcl.

::grammar::me::tcl::iok_ok
No changes.

::grammar::me::tcl::iok_fail
No changes.

::grammar::me::tcl::iok_negate
No changes.

icf_jalways *branchlabel*
see below ...

icf_jok *branchlabel*
see below ...

icf_jfail *branchlabel*
see below ...

icf_halt
These four instructions are not mapped to commands. They are control flow instructions and handled in Tcl.

::grammar::me::tcl::icl_get
This command returns the current location *CL* in the input. It replaces *icl_push*.

::grammar::me::tcl::icl_rewind *oldlocation*
The command takes the location as argument as it comes from the Tcl stack, not the machine

state.

icl_pop
Not mapped, the stacks are not managed by the package.

::grammar::me::tcl::ier_get
This command returns the current error state *ER*. It replaces *ier_push*.

::grammar::me::tcl::ier_clear
No changes.

::grammar::me::tcl::ier_nonterminal *message location*
The command takes the location as argument as it comes from the Tcl stack, not the machine state.

::grammar::me::tcl::ier_merge *olderror*
The command takes the second error state to merge as argument as it comes from the Tcl stack, not the machine state.

::grammar::me::tcl::isv_clear
No changes.

::grammar::me::tcl::isv_terminal
No changes.

::grammar::me::tcl::isv_nonterminal_leaf *nt startlocation*
The command takes the start location as argument as it comes from the Tcl stack, not the machine state.

::grammar::me::tcl::isv_nonterminal_range *nt startlocation*
The command takes the start location as argument as it comes from the Tcl stack, not the machine state.

::grammar::me::tcl::isv_nonterminal_reduce *nt startlocation* ?*marker*?
The command takes start location and marker as argument as it comes from the Tcl stack, not the machine state.

::grammar::me::tcl::ias_push
No changes.

::grammar::me::tcl::ias_mark
This command returns a marker for the current state of the AST stack *AS*. The marker stack is not managed by the machine.

::grammar::me::tcl::ias_pop2mark *marker*
The command takes the marker as argument as it comes from the Tcl stack, not the machine state. It replaces *ias_mpop*.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *grammar_me* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Grammars and finite automata

Copyright

```
Copyright (c) 2005 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


me_util
Keywords: me_util, abstract syntax tree, syntax tree, tree

Name
grammar::me::util - AST utilities

Synopsis
package require Tcl 8.5 9
package require grammar::me::util ?**0.2**?
::grammar::me::util::ast2tree *ast tree* ?*root*?
::grammar::me::util::ast2etree *ast mcmd tree* ?*root*?
mcmd lc *location*
mcmd tok *from* ?*to*?
::grammar::me::util::tree2ast *tree* ?*root*?

Description
This package provides a number of utility command for the conversion between the various representations of abstract syntax trees as specified in the document grammar::me_ast.

::grammar::me::util::ast2tree *ast tree* ?*root*?
This command converts an *ast* from value to object representation. All nodes in the *ast* will be converted into nodes of this *tree*, with the root of the AST a child of the node *root*. If this node is not explicitly specified the root of the tree is used. Existing content of tree is not touched, i.e. neither removed nor changed, with the exception of the specified root node, which will gain a new child.

::grammar::me::util::ast2etree *ast mcmd tree* ?*root*?
This command is like **::grammar::me::util::ast2tree**, except that the result is in the extended object representation of the input AST. The source of the extended information is the command prefix *mcmd*. It has to understand two methods, **lc**, and **tok**, with the semantics specified below.

mcmd lc *location*
Takes the location of a token given as offset in the input stream and return a 2-element list containing the associated line number and column index, in this order.

mcmd tok *from* ?*to*?
Takes one or two locations *from* and *to* as offset in the input stream and returns a Tcl list containing the specified part of the input stream. Both location are inclusive. If *to* is not specified it will default to the value of *from*.

Each element of the returned list is a list containing the token, its associated lexeme, the

line number, and column index, in this order.

Both the ensemble command ::grammar::me::tcl provided by the package **grammar::me::tcl** and the objects command created by the package ::grammar::me::cpu fit the above specification.

::grammar::me::util::tree2ast *tree* ?*root*?

This command converts an *ast* in (extended) object representation into a value and returns it. If a *root* node is specified the AST is generated from that node downward. Otherwise the root of the tree object is used as the starting point.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *grammar_me* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Grammars and finite automata

Copyright


```
Copyright (c) 2005 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


me_vm

Keywords: me_vm, grammar, parsing, virtual machine

Name

grammar::me_vm - Virtual machine for parsing token streams

Description

Please go and read the document grammar::me_intro first for an overview of the various documents and their relations.

This document specifies a virtual machine for the controlled matching and parsing of token streams, creating an *abstract syntax tree* (short *AST*) reflecting the structure of the input. Special machine features are the caching and reuse of partial results, caching of the encountered input, and the ability to backtrack in both input and AST creation.

These features make the specified virtual machine especially useful to packrat parsers based on parsing expression grammars. It is however not restricted to this type of parser. Normal LL and LR parsers can be implemented with it as well.

The following sections will discuss first the abstract state kept by ME virtual machines, and then their instruction set.

Machine state

A ME virtual machine manages the following state:

***Current token* CT**
: The token from the input under consideration by the machine.

  This information is used and modified by the instructions defined in the section Terminal matching.

***Current location* CL**
: The location of the *current token* in the input stream, as offset relative to the beginning of the stream. The first token is considered to be at offset **0**.

  This information is implicitly used and modified by the instructions defined in the sections Terminal matching and Nonterminal matching, and can be directly queried and modified by the instructions defined in section Input location handling.

***Location stack* LS**
: In addition to the above a stack of locations, for backtracking. Locations can put on the stack, removed from it, and removed with setting the current location.

  This information is implicitly used and modified by the instructions defined in the sections Terminal matching and Nonterminal matching, and can be directly queried and modified by the instructions defined in section Input location handling.

***Match status* OK**
: A boolean value, the result of the last attempt at matching input. It is set to **true** if that attempt was successful, and **false** otherwise.

  This information is influenced by the instructions defined in the sections Terminal matching, Nonterminal matching, and Unconditional matching. It is queried by the instructions defined in the section Control flow.

***Semantic value* SV**
: The semantic value associated with (generated by) the last attempt at matching input. Contains either the empty string or a node for the abstract syntax tree constructed from the input.

  This information is influenced by the instructions defined in the sections Semantic values, and AST stack handling.

***AST stack* AS**
: A stack of partial abstract syntax trees constructed by the machine during matching.

  This information is influenced by the instructions defined in the sections Semantic values, and AST stack handling.

***AST Marker stack* MS**
: In addition to the above a stack of stacks, for backtracking. This is actually a stack of markers into the AST stack, thus implicitly snapshooting the state of the AST stack at some point in time. Markers can be put on the stack, dropped from it, and used to roll back the AST stack to an earlier state.

This information is influenced by the instructions defined in the sections Semantic values, and AST stack handling.

Error status **ER**

Error information associated with the last attempt at matching input. Contains either the empty string or a list of 2 elements, a location in the input and a list of error messages associated with it, in this order.

Note that error information can be set even if the last attempt at matching input was successful. For example the *-operator (matching a sub-expression zero or more times) in a parsing expression grammar is always successful, even if it encounters a problem further in the input and has to backtrack. Such problems must not be forgotten when continuing to match.

This information is queried and influenced by the instructions defined in the sections Terminal matching, Nonterminal matching, and Error handling.

Error stack **ES**

In addition to the above a stack of error information, to allow the merging of current and older error information when performing backtracking in choices after an unsucessful match.

This information is queried and influenced by the instructions defined in the sections Terminal matching, Nonterminal matching, and Error handling.

Return stack **RS**

A stack of program counter values, i.e. locations in the code controlling the virtual machine, for the management of subroutine calls, i.e. the matching of nonterminal symbols.

This information is queried and influenced by the instructions defined in the section Nonterminal matching.

Nonterminal cache **NC**

A cache of machine states (A 4-tuple containing a location in the input, match status *OK*, semantic value *SV*, and error status *ER*) keyed by name of nonterminal symbol and location in the input stream.

The key location is where machine started the attempt to match the named nonterminal symbol, and the location in the value is where machine ended up after the attempt completed, independent of the success of the attempt.

This status is queried and influenced by the instructions defined in the section Nonterminal matching.

Machine instructions

With the machine state specified it is now possible to explain the instruction set of ME virtual machines. They are grouped roughly by the machine state they influence and/or query.

Terminal matching

First the instructions to match tokens from the input stream, and by extension all terminal

symbols.

These instructions are the only ones which may retrieve a new token from the input stream. This is a *may* and not a *will* because the instructions will a retrieve new token if, and only if the current location *CL* is at the head of the stream. If the machine has backtracked (see **icl_rewind**) the instructions will retrieve the token to compare against from the internal cache.

ict_advance *message*

This instruction tries to advance to the next token in the input stream, i.e. the one after the current location *CL*. The instruction will fail if, and only if the end of the input stream is reached, i.e. if there is no next token.

The sucess/failure of the instruction is remembered in the match status *OK*. In the case of failure the error status *ER* is set to the current location and the message *message*. In the case of success the error status *ER* is cleared, the new token is made the current token *CT*, and the new location is made the current location *CL*.

The argument *message* is a reference to the string to put into the error status *ER*, if such is needed.

ict_match_token *tok message*

This instruction tests the current token *CT* for equality with the argument *tok* and records the result in the match status *OK*. The instruction fails if the current token is not equal to *tok*.

In case of failure the error status *ER* is set to the current location *CL* and the message *message*, and the current location *CL* is moved one token backwards. Otherwise, i.e. upon success, the error status *ER* is cleared and the current location *CL* is not touched.

ict_match_tokrange *tokbegin tokend message*

This instruction tests the current token *CT* for being in the range of tokens from *tokbegin* to *tokend* (inclusive) and records the result in the match status *OK*. The instruction fails if the current token is not inside the range.

In case of failure the error status *ER* is set to the current location *CL* and the message *message*, and the current location *CL* is moved one token backwards. Otherwise, i.e. upon success, the error status *ER* is cleared and the current location *CL* is not touched.

ict_match_tokclass *code message*

This instruction tests the current token *CT* for being a member of the token class *code* and records the result in the match status *OK*. The instruction fails if the current token is not a member of the specified class.

In case of failure the error status *ER* is set to the current location *CL* and the message *message*, and the current location *CL* is moved one token backwards. Otherwise, i.e. upon success, the error status *ER* is cleared and the current location *CL* is not touched.

Currently the following classes are legal:

alnum

A token is accepted if it is a unicode alphabetical character, or a digit.

alpha

A token is accepted if it is a unicode alphabetical character.

digit
A token is accepted if it is a unicode digit character.

xdigit
A token is accepted if it is a hexadecimal digit character.

punct
A token is accepted if it is a unicode punctuation character.

space
A token is accepted if it is a unicode space character.

Nonterminal matching

The instructions in this section handle the matching of nonterminal symbols. They query the nonterminal cache *NC* for saved information, and put such information into the cache.

The usage of the cache is a performance aid for backtracking parsers, allowing them to avoid an expensive rematch of complex nonterminal symbols if they have been encountered before.

inc_restore *branchlabel nt*
This instruction checks if the nonterminal cache *NC* contains information about the nonterminal symbol *nt*, at the current location *CL*. If that is the case the instruction will update the machine state (current location *CL*, match status *OK*, semantic value *SV*, and error status *ER*) with the found information and continue execution at the instruction refered to by the *branchlabel*. The new current location *CL* will be the last token matched by the nonterminal symbol, i.e. belonging to it.

If no information was found the instruction will continue execution at the next instruction.

Together with **icf_ntcall** it is possible to generate code for memoized and non-memoized matching of nonterminal symbols, either as subroutine calls, or inlined in the caller.

inc_save *nt*
This instruction saves the current state of the machine (current location *CL*, match status *OK*, semantic value *SV*, and error status *ER*), to the nonterminal cache *NC*. It will also pop an entry from the location stack *LS* and save it as the start location of the match.

It is expected to be called at the end of matching a nonterminal symbol, with *nt* the name of the nonterminal symbol the code was working on. This allows the instruction **inc_restore** to check for and retrieve the data, should we have to match this nonterminal symbol at the same location again, during backtracking.

icf_ntcall *branchlabel*
This instruction invokes the code for matching the nonterminal symbol *nt* as a subroutine. To this end it stores the current program counter *PC* on the return stack *RS*, the current location *CL* on the location stack *LS*, and then continues execution at the address *branchlabel*.

The next matching **icf_ntreturn** will cause the execution to continue at the instruction coming after the call.

icf_ntreturn
This instruction will pop an entry from the return stack *RS*, assign it to the program counter *PC*, and then continue execution at the new address.

Unconditional matching

The instructions in this section are the remaining match operators. They change the match status *OK* directly and unconditionally.

iok_ok
: This instruction sets the match status *OK* to **true**, indicating a successful match.

iok_fail
: This instruction sets the match status *OK* to **false**, indicating a failed match.

iok_negate
: This instruction negates the match status *OK*, turning a failure into a success and vice versa.

Control flow

The instructions in this section implement both conditional and unconditional control flow. The conditional jumps query the match status *OK*.

icf_jalways *branchlabel*
: This instruction sets the program counter *PC* to the address specified by *branchlabel* and then continues execution from there. This is an unconditional jump.

icf_jok *branchlabel*
: This instruction sets the program counter *PC* to the address specified by *branchlabel*. This happens if, and only if the match status *OK* indicates a success. Otherwise it simply continues execution at the next instruction. This is a conditional jump.

icf_jfail *branchlabel*
: This instruction sets the program counter *PC* to the address specified by *branchlabel*. This happens if, and only if the match status *OK* indicates a failure. Otherwise it simply continues execution at the next instruction. This is a conditional jump.

icf_halt
: This instruction halts the machine and blocks any further execution.

Input location handling

The instructions in this section are for backtracking, they manipulate the current location *CL* of the machine state. They allow a user of the machine to query and save locations in the input, and to rewind the current location *CL* to saved locations, making them one of the components enabling the implementation of backtracking parsers.

icl_push
: This instruction pushes a copy of the current location *CL* on the location stack *LS*.

icl_rewind
: This instruction pops an entry from the location stack *LS* and then moves the current location *CL* back to this point in the input.

icl_pop
: This instruction pops an entry from the location stack *LS* and discards it.

Error handling

The instructions in this section provide read and write access to the error status *ER* of the machine.

ier_push
: This instruction pushes a copy of the current error status *ER* on the error stack *ES*.

ier_clear
This instruction clears the error status *ER*.

ier_nonterminal *message*
This instruction checks if the error status *ER* contains an error whose location is just past the location found in the top entry of the location stack *LS*. Nothing happens if no such error is found. Otherwise the found error is replaced by an error at the location found on the stack, having the message *message*.

ier_merge
This instruction pops an entry from the error stack *ES*, merges it with the current error status *ER* and stores the result of the merge as the new error status *ER*.

The merge is performed as described below:

If one of the two error states is empty the other is chosen. If neither error state is empty, and refering to different locations, then the error state with the location further in the input is chosen. If both error states refer to the same location their messages are merged (with removing duplicates).

Semantic values

The instructions in this section manipulate the semantic value *SV*.

isv_clear
This instruction clears the semantic value *SV*.

isv_terminal
This instruction creates a terminal AST node for the current token *CT*, makes it the semantic value *SV*, and also pushes the node on the AST stack *AS*.

isv_nonterminal_leaf *nt*
This instruction creates a nonterminal AST node without any children for the nonterminal *nt*, and makes it the semantic value *SV*.

This instruction should be executed if, and only if the match status *OK* indicates a success. In the case of a failure **isv_clear** should be called.

isv_nonterminal_range *nt*
This instruction creates a nonterminal AST node for the nonterminal *nt*, with a single terminal node as its child, and makes this AST the semantic value *SV*. The terminal node refers to the input string from the location found on top of the location stack *LS* to the current location *CL* (both inclusive).

This instruction should be executed if, and only if the match status *OK* indicates a success. In the case of a failure **isv_clear** should be called.

isv_nonterminal_reduce *nt*
This instruction creates a nonterminal AST node for the nonterminal *nt* and makes it the semantic value *SV*.

All entries on the AST stack *AS* above the marker found in the top entry of the AST Marker stack *MS* become children of the new node, with the entry at the stack top becoming the

rightmost child. If the AST Marker stack *MS* is empty the whole stack is used. The AST marker stack *MS* is left unchanged.

This instruction should be executed if, and only if the match status *OK* indicates a success. In the case of a failure **isv_clear** should be called.

AST stack handling

The instructions in this section manipulate the AST stack *AS*, and the AST Marker stack *MS*.

ias_push
This instruction pushes the semantic value *SV* on the AST stack *AS*.

ias_mark
This instruction pushes a marker for the current state of the AST stack *AS* on the AST Marker stack *MS*.

ias_mrewind
This instruction pops an entry from the AST Marker stack *MS* and then proceeds to pop entries from the AST stack *AS* until the state represented by the popped marker has been reached again. Nothing is done if the AST stack *AS* is already smaller than indicated by the popped marker.

ias_mpop
This instruction pops an entry from the AST Marker stack *MS* and discards it.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *grammar_me* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Grammars and finite automata

Copyright

```
Copyright (c) 2005 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

grammar_peg

peg

Keywords: peg, LL(k), TDPL, context-free languages, expression, grammar, parsing, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

grammar::peg - Create and manipulate parsing expression grammars

Synopsis

package require Tcl 8.5 9
package require snit
package require grammar::peg ?**0.3**?
::grammar::peg *pegName* ?=|*:=*|<--|*as*|*deserialize src*?
pegName **destroy**
pegName **clear**
pegName **=** *srcPEG*
pegName **-->** *dstPEG*
pegName **serialize**
pegName **deserialize** *serialization*
pegName **is valid**
pegName **start** ?*pe*?
pegName **nonterminals**
pegName **nonterminal add** *nt pe*
pegName **nonterminal delete** *nt1* ?*nt2* ...?
pegName **nonterminal exists** *nt*
pegName **nonterminal rename** *nt ntnew*
pegName **nonterminal mode** *nt* ?*mode*?
pegName **nonterminal rule** *nt*
pegName **unknown nonterminals**

Description

This package provides a container class for *parsing expression grammars* (Short: PEG). It allows the incremental definition of the grammar, its manipulation and querying of the definition. The package neither provides complex operations on the grammar, nor has it the ability to execute a grammar definition for a stream of symbols. Two packages related to this one are **grammar::mengine** and **grammar::peg::interpreter**. The first of them defines a general virtual machine for the matching of a character stream, and the second implements an interpreter for parsing expression grammars on top of that virtual machine.

Terms & concepts

PEGs are similar to context-free grammars, but not equivalent; in some cases PEGs are strictly more powerful than context-free grammars (there exist PEGs for some non-context-free languages). The formal mathematical definition of parsing expressions and parsing expression grammars can be found in section Parsing expression grammars.

In short, we have *terminal symbols*, which are the most basic building blocks for *sentences*, and *nonterminal symbols* with associated *parsing expressions*, defining the grammatical structure of the sentences. The two sets of symbols are distinctive, and do not overlap. When speaking about symbols the word "symbol" is often left out. The union of the sets of terminal and nonterminal symbols is called the set of *symbols*.

Here the set of *terminal symbols* is not explicitly managed, but implicitly defined as the set of all characters. Note that this means that we inherit from Tcl the ability to handle all of Unicode.

A pair of *nonterminal* and *parsing expression* is also called a *grammatical rule*, or *rule* for short. In the context of a rule the nonterminal is often called the left-hand-side (LHS), and the parsing expression the right-hand-side (RHS).

The *start expression* of a grammar is a parsing expression from which all the sentences contained in the language specified by the grammar are *derived*. To make the understanding of this term easier let us assume for a moment that the RHS of each rule, and the start expression, is either a sequence of symbols, or a series of alternate parsing expressions. In the latter case the rule can be seen as a set of rules, each providing one alternative for the nonterminal. A parsing expression A' is now a derivation of a parsing expression A if we pick one of the nonterminals N in the expression, and one of the alternative rules R for N, and then replace the nonterminal in A with the RHS of the chosen rule. Here we can see why the terminal symbols are called such. They cannot be expanded any further, thus terminate the process of deriving new expressions. An example

```
    Rules
      (1)  A <- a B c
      (2a) B <- d B
      (2b) B <- e

    Some derivations, using starting expression A.

      A -/1/-> a B c -/2a/-> a d B c -/2b/-> a d e c
```

A derived expression containing only terminal symbols is a *sentence*. The set of all sentences which can be derived from the start expression is the *language* of the grammar.

Some definitions for nonterminals and expressions:

1. A nonterminal A is called *reachable* if it is possible to derive a parsing expression from the start expression which contains A.
2. A nonterminal A is called *useful* if it is possible to derive a sentence from it.
3. A nonterminal A is called *recursive* if it is possible to derive a parsing expression from it which contains A, again.
4. The *FIRST set* of a nonterminal A contains all the symbols which can occur of as the leftmost symbol in a parsing expression derived from A. If the FIRST set contains A itself then that nonterminal is called *left-recursive*.
5. The *LAST set* of a nonterminal A contains all the symbols which can occur of as the rightmost symbol in a parsing expression derived from A. If the LAST set contains A itself then that nonterminal is called *right-recursive*.
6. The *FOLLOW set* of a nonterminal A contains all the symbols which can occur after A in a parsing expression derived from the start expression.
7. A nonterminal (or parsing expression) is called *nullable* if the empty sentence can be derived from it.

And based on the above definitions for grammars:

1. A grammar G is *recursive* if and only if it contains a nonterminal A which is recursive. The terms *left-* and *right-recursive*, and *useful* are analogously defined.
2. A grammar is *minimal* if it contains only *reachable* and *useful* nonterminals.
3. A grammar is *wellformed* if it is not left-recursive. Such grammars are also *complete*, which

means that they always succeed or fail on all input sentences. For an incomplete grammar on the other hand input sentences exist for which an attempt to match them against the grammar will not terminate.

4. As we wish to allow ourselves to build a grammar incrementally in a container object we will encounter stages where the RHS of one or more rules reference symbols which are not yet known to the container. Such a grammar we call *invalid*. We cannot use the term *incomplete* as this term is already taken, see the last item.

Container class API

The package exports the API described here.

::grammar::peg *pegName* ?=|:=|<--|as|deserialize *src*?

The command creates a new container object for a parsing expression grammar and returns the fully qualified name of the object command as its result. The API the returned command is following is described in the section Container object API. It may be used to invoke various operations on the container and the grammar within.

The new container, i.e. grammar will be empty if no *src* is specified. Otherwise it will contain a copy of the grammar contained in the *src*. The *src* has to be a container object reference for all operators except **deserialize**. The **deserialize** operator requires *src* to be the serialization of a parsing expression grammar instead.

An empty grammar has no nonterminal symbols, and the start expression is the empty expression, i.e. epsilon. It is *valid*, but not *useful*.

Container object API

All grammar container objects provide the following methods for the manipulation of their contents:

***pegName* destroy**

Destroys the grammar, including its storage space and associated command.

***pegName* clear**

Clears out the definition of the grammar contained in *pegName*, but does *not* destroy the object.

pegName = srcPEG

Assigns the contents of the grammar contained in *srcPEG* to *pegName*, overwriting any existing definition. This is the assignment operator for grammars. It copies the grammar contained in the grammar object *srcPEG* over the grammar definition in *pegName*. The old contents of *pegName* are deleted by this operation.

This operation is in effect equivalent to

```
    pegName deserialize [srcPEG serialize]
```

pegName --> dstPEG

This is the reverse assignment operator for grammars. It copies the automation contained in the object *pegName* over the grammar definition in the object *dstPEG*. The old contents of *dstPEG* are deleted by this operation.

This operation is in effect equivalent to

```
    dstPEG deserialize [pegName serialize]
```

***pegName* serialize**

This method serializes the grammar stored in *pegName*. In other words it returns a tcl *value* completely describing that grammar. This allows, for example, the transfer of grammars over arbitrary channels, persistence, etc. This method is also the basis for both the copy constructor and the assignment operator.

The result of this method has to be semantically identical over all implementations of the **grammar::peg** interface. This is what will enable us to copy grammars between different implementations of the same interface.

The result is a list of four elements with the following structure:

1. The constant string **grammar::peg**.
2. A dictionary. Its keys are the names of all known nonterminal symbols, and their associated values are the parsing expressions describing their sentennial structure.
3. A dictionary. Its keys are the names of all known nonterminal symbols, and their associated values hints to a matcher regarding the semantic values produced by the symbol.
4. The last item is a parsing expression, the *start expression* of the grammar.

Assuming the following PEG for simple mathematical expressions

```
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'
    Sign       <- '+' / '-'
    Number     <- Sign? Digit+
    Expression <- '(' Expression ')' / (Factor (MulOp Factor)*)
    MulOp      <- '*' / '/'
    Factor     <- Term (AddOp Term)*
    AddOp      <- '+'/'-'
    Term       <- Number
```

a possible serialization is

```
grammar::peg \
{Expression {/ {x ( Expression )} {x Factor {* {x MulOp
Factor}}}} \
 Factor     {x Term {* {x AddOp Term}}} \
 Term       Number \
 MulOp      {/ * /} \
 AddOp      {/ + -} \
 Number     {x {? Sign} {+ Digit}} \
 Sign       {/ + -} \
 Digit      {/ 0 1 2 3 4 5 6 7 8 9} \
} \
{Expression value     Factor     value \
 Term       value     MulOp      value \
 AddOp      value     Number     value \
 Sign       value     Digit      value \
}
Expression
```

A possible one, because the order of the nonterminals in the dictionary is not relevant.

pegName* deserialize *serialization

This is the complement to **serialize**. It replaces the grammar definition in *pegName* with the grammar described by the *serialization* value. The old contents of *pegName* are deleted by this operation.

***pegName* is valid**

A predicate. It tests whether the PEG in *pegName* is *valid*. See section Terms & concepts for the definition of this grammar property. The result is a boolean value. It will be set to **true** if the PEG has the tested property, and **false** otherwise.

***pegName* start ?*pe*?**

This method defines the *start expression* of the grammar. It replaces the previously defined start expression with the parsing expression *pe*. The method fails and throws an error if *pe* does not contain a valid parsing expression as specified in the section Parsing expressions. In that case the existing start expression is not changed. The method returns the empty string as its result.

If the method is called without an argument it will return the currently defined start expression.

***pegName* nonterminals**

Returns the set of all nonterminal symbols known to the grammar.

pegName* nonterminal add *nt pe

This method adds the nonterminal *nt* and its associated parsing expression *pe* to the set of nonterminal symbols and rules of the PEG contained in the object *pegName*. The method fails and throws an error if either the string *nt* is already known as a symbol of the grammar, or if *pe* does not contain a valid parsing expression as specified in the section Parsing expressions. In that case the current set of nonterminal symbols and rules is not changed. The method returns the empty string as its result.

***pegName* nonterminal delete *nt1* ?*nt2* ...?**
This method removes the named symbols *nt1*, *nt2* from the set of nonterminal symbols of the PEG contained in the object *pegName*. The method fails and throws an error if any of the strings is not known as a nonterminal symbol. In that case the current set of nonterminal symbols is not changed. The method returns the empty string as its result.

The stored grammar becomes invalid if the deleted nonterminals are referenced by the RHS of still-known rules.

pegName* nonterminal exists *nt
A predicate. It tests whether the nonterminal symbol *nt* is known to the PEG in *pegName*. The result is a boolean value. It will be set to **true** if the symbol *nt* is known, and **false** otherwise.

pegName* nonterminal rename *nt ntnew
This method renames the nonterminal symbol *nt* to *ntnew*. The method fails and throws an error if either *nt* is not known as a nonterminal, or if *ntnew* is a known symbol. The method returns the empty string as its result.

***pegName* nonterminal mode *nt* ?*mode*?**
This mode returns or sets the semantic mode associated with the nonterminal symbol *nt*. If no *mode* is specified the current mode of the nonterminal is returned. Otherwise the current mode is set to *mode*. The method fails and throws an error if *nt* is not known as a nonterminal. The grammar interpreter implemented by the package **grammar::peg::interpreter** recognizes the following modes:

value
The semantic value of the nonterminal is the abstract syntax tree created from the AST's of the RHS and a node for the nonterminal itself.

match
The semantic value of the nonterminal is an the abstract syntax tree consisting of single a node for the string matched by the RHS. The ASTs generated by the RHS are discarded.

leaf
The semantic value of the nonterminal is an the abstract syntax tree consisting of single a node for the nonterminal itself. The ASTs generated by the RHS are discarded.

discard
The nonterminal has no semantic value. The ASTs generated by the RHS are discarded (as well).

pegName* nonterminal rule *nt
This method returns the parsing expression associated with the nonterminal *nt*. The method fails and throws an error if *nt* is not known as a nonterminal.

***pegName* unknown nonterminals**
This method returns a list containing the names of all nonterminal symbols which are referenced on the RHS of a grammatical rule, but have no rule defininig their structure. In other words, a list of the nonterminal symbols which make the grammar invalid. The grammar is valid if this list is empty.

Parsing expressions

Various methods of PEG container objects expect a parsing expression as their argument, or will return such. This section specifies the format such parsing expressions are in.

1. The string **epsilon** is an atomic parsing expression. It matches the empty string.
2. The string **alnum** is an atomic parsing expression. It matches any alphanumeric character.
3. The string **alpha** is an atomic parsing expression. It matches any alphabetical character.
4. The string **dot** is an atomic parsing expression. It matches any character.
5. The expression [list t **x**] is an atomic parsing expression. It matches the terminal string **x**.
6. The expression [list n **A**] is an atomic parsing expression. It matches the nonterminal **A**.
7. For parsing expressions **e1**, **e2**, ... the result of [list / **e1 e2** ...] is a parsing expression as well. This is the *ordered choice*, aka *prioritized choice*.
8. For parsing expressions **e1**, **e2**, ... the result of [list x **e1 e2** ...] is a parsing expression as well. This is the *sequence*.
9. For a parsing expression **e** the result of [list * **e**] is a parsing expression as well. This is the *kleene closure*, describing zero or more repetitions.
10. For a parsing expression **e** the result of [list + **e**] is a parsing expression as well. This is the *positive kleene closure*, describing one or more repetitions.
11. For a parsing expression **e** the result of [list & **e**] is a parsing expression as well. This is the *and lookahead predicate*.
12. For a parsing expression **e** the result of [list! **e**] is a parsing expression as well. This is the *not lookahead predicate*.
13. For a parsing expression **e** the result of [list ? **e**] is a parsing expression as well. This is the *optional input*.

Examples of parsing expressions where already shown, in the description of the method **serialize**.

Parsing expression grammars

For the mathematically inclined, a PEG is a 4-tuple (VN,VT,R,eS) where

- VN is a set of *nonterminal symbols*,
- VT is a set of *terminal symbols*,
- R is a finite set of rules, where each rule is a pair (A,e), A in VN, and *e* a *parsing expression*.
- eS is a parsing expression, the *start expression*.

Further constraints are

- The intersection of VN and VT is empty.
- For all A in VT exists exactly one pair (A,e) in R. In other words, R is a function from nonterminal symbols to parsing expressions.

Parsing expression are inductively defined via

- The empty string (epsilon) is a parsing expression.
- A terminal symbol *a* is a parsing expression.
- A nonterminal symbol *A* is a parsing expression.
- *e1e2* is a parsing expression for parsing expressions *e1* and *2*. This is called *sequence*.
- *e1*/*e2* is a parsing expression for parsing expressions *e1* and *2*. This is called *ordered choice*.
- *e** is a parsing expression for parsing expression *e*. This is called *zero-or-more repetitions*, also known as *kleene closure*.
- *e*+ is a parsing expression for parsing expression *e*. This is called *one-or-more repetitions*, also known as *positive kleene closure*.
- !*e* is a parsing expression for parsing expression *e1*. This is called a *not lookahead predicate*.
- &*e* is a parsing expression for parsing expression *e1*. This is called an *and lookahead predicate*.

PEGs are used to define a grammatical structure for streams of symbols over VT. They are a modern phrasing of older formalisms invented by Alexander Birham. These formalisms were called TS (TMG recognition scheme), and gTS (generalized TS). Later they were renamed to TPDL (Top-Down Parsing Languages) and gTPDL (generalized TPDL).

They can be easily implemented by recursive descent parsers with backtracking. This makes them relatives of LL(k) Context-Free Grammars.

References

1. The Packrat Parsing and Parsing Expression Grammars Page, by Bryan Ford, Massachusetts Institute of Technology. This is the main entry page to PEGs, and their realization through Packrat Parsers.
2. *Parsing Techniques - A Practical Guide* http://www.cs.vu.nl/~dick/PTAPG.html, an online book offering a clear, accessible, and thorough discussion of many different parsing techniques with their interrelations and applicabilities, including error recovery techniques.
3. Compilers and Compiler Generators, an online book using CoCo/R, a generator for recursive descent parsers.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *grammar_peg* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Grammars and finite automata

Copyright


```
Copyright (c) 2005 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


peg_interp

Keywords: peg_interp, LL(k), TDPL, context-free languages, expression, grammar, matching, parsing, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer, virtual machine

Name

grammar::peg::interp - Interpreter for parsing expression grammars

Synopsis

package require Tcl 8.5 9
package require grammar::mengine ?**0.1**?
package require grammar::peg::interp ?**0.1.2**?
::grammar::peg::interp::setup *peg*

::grammar::peg::interp::parse *nextcmd errorvar astvar*

Description

This package provides commands for the controlled matching of a character stream via a parsing expression grammar and the creation of an abstract syntax tree for the stream and partials.

It is built on top of the virtual machine provided by the package **grammar::me::tcl** and directly interprets the parsing expression grammar given to it. In other words, the grammar is *not* pre-compiled but used as is.

The grammar to be interpreted is taken from a container object following the interface specified by the package **grammar::peg::container**. Only the relevant parts are copied into the state of this package.

It should be noted that the package provides exactly one instance of the interpreter, and interpreting a second grammar requires the user to either abort or complete a running interpretation, or to put them into different Tcl interpreters.

Also of note is that the implementation assumes a pull-type handling of the input. In other words, the interpreter pulls characters from the input stream as it needs them. For usage in a push environment, i.e. where the environment pushes new characters as they come we have to put the engine into its own thread.

The interpreter API

The package exports the following API

::grammar::peg::interp::setup *peg*

This command (re)initializes the interpreter. It returns the empty string. This command has to be invoked first, before any matching run.

Its argument *peg* is the handle of an object containing the parsing expression grammar to interpret. This grammar has to be valid, or an error will be thrown.

::grammar::peg::interp::parse *nextcmd errorvar astvar*

This command interprets the loaded grammar and tries to match it against the stream of characters represented by the command prefix *nextcmd*.

The command prefix *nextcmd* represents the input stream of characters and is invoked by the interpreter whenever the a new character from the stream is required. The callback has to return either the empty list, or a list of 4 elements containing the token, its lexeme attribute, and its location as line number and column index, in this order. The empty list is the signal that the end of the input stream has been reached. The lexeme attribute is stored in the terminal cache, but otherwise not used by the machine.

The result of the command is a boolean value indicating whether the matching process was successful (**true**), or not (**false**). In the case of a match failure error information will be stored into the variable referenced by *errorvar*. The variable referenced by *astvar* will always contain the generated abstract syntax tree, however in the case of an error it will be only partial and possibly malformed.

The abstract syntax tree is represented by a nested list, as described in section **AST VALUES** of document *grammar::me_ast*.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *grammar_peg* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Grammars and finite automata

Copyright

```
Copyright (c) 2005-2011 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

hook

hook

Keywords: hook, callback, event, observer, producer, publisher, subject, subscriber, uevent

Name

hook - Hooks

Synopsis

package require Tcl 8.5 9
package require hook ?**0.3**?
hook bind ?*subject*? ?*hook*? ?*observer*? ?*cmdPrefix*?
hook call *subject hook* ?*args...*?
hook forget *object*
hook cget *option*
hook configure option *value* ...

Description

This package provides the **hook** ensemble command, which implements the Subject/Observer pattern. It allows *subjects*, which may be *modules*, *objects*, *widgets*, and so forth, to synchronously call *hooks* which may be bound to an arbitrary number of subscribers, called *observers*. A subject may call any number of distinct hooks, and any number of observers can bind callbacks to a particular hook called by a particular subject. Hook bindings can be queried and deleted.

This man page is intended to be a reference only.

Concepts

Introduction

Tcl modules usually send notifications to other modules in two ways: via Tk events, and via callback options like the text widget's **-yscrollcommand** option. Tk events are available only in Tk, and callback options require tight coupling between the modules sending and receiving the notification.

Loose coupling between sender and receiver is often desirable, however. In Model/View/Controller terms, a View can send a command (stemming from user input) to the Controller, which updates the Model. The Model can then call a hook *to which all relevant Views subscribe.* The Model is decoupled from the Views, and indeed need not know whether any Views actually exist. At present, Tcl/Tk has no standard mechanism for implementing loose coupling of this kind. This package defines a new command, **hook**, which implements just such a mechanism.

Bindings

The **hook** command manages a collection of hook bindings. A hook binding has four elements:

1. A *subject*: the name of the entity that will be calling the hook.
2. The *hook* itself. A hook usually reflects some occurrence in the life of the *subject* that other entities might care to know about. A *hook* has a name, and may also have arguments. Hook names are arbitrary strings. Each *subject* must document the names and arguments of the hooks it can call.
3. The name of the *observer* that wishes to receive the *hook* from the *subject*.
4. A command prefix to which the *hook* arguments will be appended when the binding is executed.

Subjects and observers

For convenience, this document collectively refers to subjects and observers as *objects*, while placing no requirements on how these *objects* are actually implemented. An object can be a **TclOO** or **Snit** or **XOTcl** object, a Tcl command, a namespace, a module, a pseudo-object managed by some other object (as tags are managed by the Tk text widget) or simply a well-known name.

Subject and observer names are arbitrary strings; however, as **hook** might be used at the package level, it's necessary to have conventions that avoid name collisions between packages written by different people.

Therefore, any subject or observer name used in core or package level code should look like a Tcl command name, and should be defined in a namespace owned by the package. Consider, for example, an ensemble command **::foo** that creates a set of pseudo-objects and uses **hook** to send notifications. The pseudo-objects have names that are not commands and exist in their own namespace, rather like file handles do. To avoid name collisions with subjects defined by other packages, users of **hook**, these **::foo** handles should have names like **::foo::1**, **::foo::2**, and so on.

Because object names are arbitrary strings, application code can use whatever additional conventions are dictated by the needs of the application.

Reference

Hook provides the following commands:

hook bind ?*subject*? ?*hook*? ?*observer*? ?*cmdPrefix*?

This subcommand is used to create, update, delete, and query hook bindings.

Called with no arguments it returns a list of the subjects with hooks to which observers are currently bound.

Called with one argument, a *subject*, it returns a list of the subject's hooks to which observers are currently bound.

Called with two arguments, a *subject* and a *hook*, it returns a list of the observers which are currently bound to this *subject* and *hook*.

Called with three arguments, a *subject*, a *hook*, and an *observer*, it returns the binding proper, the command prefix to be called when the hook is called, or the empty string if there is no such binding.

Called with four arguments, it creates, updates, or deletes a binding. If *cmdPrefix* is the empty string, it deletes any existing binding for the *subject*, *hook*, and *observer*; nothing is returned. Otherwise, *cmdPrefix* must be a command prefix taking as many additional arguments as are documented for the *subject* and *hook*. The binding is added or updated, and the observer is returned.

If the *observer* is the empty string, "", it will create a new binding using an automatically generated observer name of the form **::hook::ob<number>**. The automatically generated name will be returned, and can be used to query, update, and delete the binding as usual. If automated observer names are always used, the observer name effectively becomes a unique binding ID.

It is possible to call **hook bind** to create or delete a binding to a *subject* and *hook* while in an observer binding for that same *subject* and *hook*. The following rules determine what happens when

```
    hook bind $s $h $o $binding
```

is called during the execution of

```
    hook call $s $h
```

1. No binding is ever called after it is deleted.
2. When a binding is called, the most recently given command prefix is always used.
3. The set of observers whose bindings are to be called is determined when this method begins to execute, and does not change thereafter, except that deleted bindings are not called.

In particular:

1. If **$o**s binding to **$s** and **$h** is deleted, and **$o**s binding has not yet been called during this execution of
   ```
       hook call $s $h
   ```
2. it will not be called. (Note that it might already have been called; and in all likelihood, it is probably deleting itself.)

3. If **$o** changes the command prefix that's bound to **$s** and **$h**, and if **$o**s binding has not yet been called during this execution of

```
    hook call $s $h
```

4. the new binding will be called when the time comes. (But again, it is probably **$o**s binding that is is making the change.)
5. If a new observer is bound to **$s** and **$h**, its binding will not be called until the next invocation of

```
    hook call $s $h
```

hook call *subject hook* ?*args*...?

This command is called when the named *subject* wishes to call the named *hook*. All relevant bindings are called with the specified arguments in the global namespace. Note that the bindings are called synchronously, before the command returns; this allows the *args* to include references to entities that will be cleaned up as soon as the hook has been called.

The order in which the bindings are called is not guaranteed. If sequence among observers must be preserved, define one observer and have its bindings call the other callbacks directly in the proper sequence.

Because the **hook** mechanism is intended to support loose coupling, it is presumed that the *subject* has no knowledge of the observers, nor any expectation regarding return values. This has a number of implications:

1. **hook call** returns the empty string.
2. Normal return values from observer bindings are ignored.
3. Errors and other exceptional returns propagate normally by default. This will rarely be what is wanted, because the subjects usually have no knowledge of the observers and will therefore have no particular competence at handling their errors. That makes it an application issue, and so applications will usually want to define an **-errorcommand**.

If the **-errorcommand** configuration option has a non-empty value, its value will be invoked for all errors and other exceptional returns in observer bindings. See **hook configure**, below, for more information on configuration options.

hook forget *object*

This command deletes any existing bindings in which the named *object* appears as either the *subject* or the *observer*. Bindings deleted by this method will never be called again. In particular,

1. If an observer is forgotten during a call to **hook call**, any uncalled binding it might have had to the relevant subject and hook will *not* be called subsequently.
2. If a subject **$s** is forgotten during a call to

```
hook call $s $h
```

3. then **hook call** will return as soon as the current binding returns. No further bindings will be called.

hook cget *option*

This command returns the value of one of the **hook** command's configuration options.

hook configure option *value* ...

This command sets the value of one or more of the **hook** command's configuration options:

-errorcommand *cmdPrefix*

If the value of this option is the empty string, "", then errors and other exception returns in binding scripts are propagated normally. Otherwise, it must be a command prefix taking three additional arguments:

1. a 4-element list {subject hook arglist observer},
2. the result string, and
3. the return options dictionary.

Given this information, the **-errorcommand** can choose to log the error, call **interp bgerror**, delete the errant binding (thus preventing the error from arising a second time) and so forth.

-tracecommand *cmdPrefix*

The option's value should be a command prefix taking four arguments:

1. a *subject*,
2. a *hook*,
3. a list of the hook's argument values, and
4. a list of *objects* the hook was called for.

The command will be called for each hook that is called. This allows the application to trace hook execution for debugging purposes.

Example

The **::model** module calls the hook in response to commands that change the model's data:

```
hook call ::model <Update>
```

The **.view** megawidget displays the model state, and needs to know about model updates. Consequently, it subscribes to the ::model's hook.

```
hook bind ::model <Update> .view [list .view ModelUpdate]
```

When the **::model** calls the hook, the **.view**s ModelUpdate subcommand will be called.

Later the **.view** megawidget is destroyed. In its destructor, it tells the *hook* that it no longer exists:

```
hook forget .view
```

All bindings involving **.view** are deleted.

Credits

Hook has been designed and implemented by William H. Duquette.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *hook* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be

made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Programming tools

Copyright

```
Copyright (c) 2010, by William H. Duquette
```


html

html
Keywords: html, checkbox, checkbutton, form, radiobutton, table

Name
html - Procedures to generate HTML structures

Synopsis
package require Tcl 8.5 9
package require html ?**1.6**?
::html::author *author*
::html::bodyTag *args*
::html::cell *param value* ?*tag*?
::html::checkbox *name value*
::html::checkSet *key sep list*
::html::checkValue *name* ?*value*?
::html::closeTag
::html::default *key* ?*param*?
::html::description *description*
::html::end
::html::eval *arg* ?*args*?
::html::extractParam *param key* ?*varName*?
::html::font *args*
::html::for *start test next body*
::html::foreach *varlist1 list1* ?*varlist2 list2 ...*? *body*
::html::formValue *name* ?*defvalue*?
::html::getFormInfo *args*
::html::getTitle
::html::h *level string* ?*param*?
::html::h1 *string* ?*param*?
::html::h2 *string* ?*param*?
::html::h3 *string* ?*param*?
::html::h4 *string* ?*param*?
::html::h5 *string* ?*param*?
::html::h6 *string* ?*param*?
::html::hdrRow *args*

::html::head *title*
::html::headTag *string*
::html::html_entities *string*
::html::if *expr1 body1 ?elseif expr2 body2 ...? ?else bodyN?*
::html::init *?list?*
::html::keywords *args*
::html::mailto *email ?subject?*
::html::meta *args*
::html::meta_name *args*
::html::meta_equiv *args*
::html::meta_charset *charset*
::html::css *href*
::html::css-clear
::html::js *href*
::html::js-clear
::html::minorList *list ?ordered?*
::html::minorMenu *list ?sep?*
::html::nl2br *string*
::html::openTag *tag ?param?*
::html::paramRow *list ?rparam? ?cparam?*
::html::passwordInput *?name?*
::html::passwordInputRow *label ?name?*
::html::quoteFormValue *value*
::html::radioSet *key sep list*
::html::radioValue *name value*
::html::refresh *seconds url*
::html::row *args*
::html::select *name param choices ?current?*
::html::selectPlain *name param choices ?current?*
::html::set *var val*
::html::submit *label ?name? ?title?*
::html::tableFromArray *arrname ?param? ?pat?*
::html::tableFromList *querylist ?param?*
::html::textarea *name ?param? ?current?*
::html::textInput *name value args*
::html::textInputRow *label name value args*
::html::varEmpty *name*
::html::while *test body*
::html::doctype *id*
::html::wrapTag *tag ?text? ?args?*

Description

The package **html** provides commands that generate HTML. These commands typically return an HTML string as their result. In particular, they do not output their result to **stdout**.

The command **::html::init** should be called early to initialize the module. You can also use this procedure to define default values for HTML tag parameters.

::html::author *author*
Side effect only. Call this before **::html::head** to define an author for the page. The author is noted in a comment in the HEAD section.

::html::bodyTag *args*
Generate a *body* tag. The tag parameters are taken from *args* or from the body.* attributes define with **::html::init**.

::html::cell *param value* ?*tag*?
Generate a *td* (or *th*) tag, a value, and a closing *td* (or *th*) tag. The tag parameters come from *param* or TD.* attributes defined with **::html::init**. This uses **::html::font** to insert a standard *font* tag into the table cell. The *tag* argument defaults to "td".

::html::checkbox *name value*
Generate a *checkbox* form element with the specified name and value. This uses **::html::checkValue**.

::html::checkSet *key sep list*
Generate a set of *checkbox* form elements and associated labels. The *list* should contain an alternating list of labels and values. This uses **::html::checkbox**. All the *checkbox* buttons share the same *key* for their name. The *sep* is text used to separate the elements.

::html::checkValue *name* ?*value*?
Generate the "name=*name* value=*value*" for a *checkbox* form element. If the CGI variable *name* has the value *value*, then SELECTED is added to the return value. *value* defaults to "1".

::html::closeTag
Pop a tag off the stack created by **::html::openTag** and generate the corresponding close tag (e.g.,).

::html::default *key* ?*param*?
This procedure is used by **::html::tagParam** to generate the name, value list of parameters for a tag. The **::html::default** procedure is used to generate default values for those items not already in *param*. If the value identified by *key* matches a value in *param* then this procedure returns the empty string. Otherwise, it returns a "parameter=value" string for a form element identified by *key*. The *key* has the form "tag.parameter" (e.g., body.bgcolor). Use **::html::init** to register default values. *param* defaults to the empty string.

::html::description *description*
Side effect only. Call this before **::html::head** to define a description *meta* tag for the page. This tag is generated later in the call to **::html::head**.

::html::end
Pop all open tags from the stack and generate the corresponding close HTML tags, (e.g.,).

::html::eval *arg* ?*args*?
This procedure is similar to the built-in Tcl **eval** command. The only difference is that it returns "" so it can be called from an HTML template file without appending unwanted results.

::html::extractParam *param key* ?*varName*?
This is a parsing procedure that extracts the value of *key* from *param*, which is a HTML-style "name=quotedvalue" list. *varName* is used as the name of a Tcl variable that is changed to have the value found in the parameters. The function returns 1 if the parameter was found in *param*, otherwise it returns 0. If the *varName* is not specified, then *key* is used as the variable name.

::html::font *args*

Generate a standard *font* tag. The parameters to the tag are taken from *args* and the HTML defaults defined with **::html::init**.

::html::for *start test next body*
This procedure is similar to the built-in Tcl **for** control structure. Rather than evaluating the body, it returns the subst'ed *body*. Each iteration of the loop causes another string to be concatenated to the result value.

::html::foreach *varlist1 list1 ?varlist2 list2 ...? body*
This procedure is similar to the built-in Tcl **foreach** control structure. Rather than evaluating the body, it returns the subst'ed *body*. Each iteration of the loop causes another string to be concatenated to the result value.

::html::formValue *name ?defvalue?*
Return a name and value pair, where the value is initialized from existing CGI data, if any. The result has this form:

```
name="fred" value="freds value"
```

::html::getFormInfo *args*
Generate hidden fields to capture form values. If *args* is empty, then hidden fields are generated for all CGI values. Otherwise args is a list of string match patterns for form element names.

::html::getTitle
Return the title string, with out the surrounding *title* tag, set with a previous call to **::html::title**.

::html::h *level string ?param?*
Generate a heading (e.g., *h***level**) tag. The *string* is nested in the heading, and *param* is used for the tag parameters.

::html::h1 *string ?param?*
Generate an *h1* tag. See **::html::h**.

::html::h2 *string ?param?*
Generate an *h2* tag. See **::html::h**.

::html::h3 *string ?param?*
Generate an *h3* tag. See **::html::h**.

::html::h4 *string ?param?*
Generate an *h4* tag. See **::html::h**.

::html::h5 *string ?param?*
Generate an *h5* tag. See **::html::h**.

::html::h6 *string ?param?*
Generate an *h6* tag. See **::html::h**.

::html::hdrRow *args*
Generate a table row, including *tr* and *th* tags. Each value in *args* is place into its own table cell. This uses **::html::cell**.

::html::head *title*
Generate the *head* section that includes the page *title*. If previous calls have been made to **::html::author**, **::html::keywords**, **::html::description**, or **::html::meta** then additional tags are inserted into the *head* section. This leaves an open *html* tag pushed on the stack

with **::html::openTag**.

::html::headTag *string*
Save a tag for inclusion in the *head* section generated by **::html::head**. The *string* is everything in the tag except the enclosing angle brackets, < >.

::html::html_entities *string*
This command replaces all special characters in the *string* with their HTML entities and returns the modified text.

::html::if *expr1 body1* ?elseif *expr2 body2 ...*? ?else *bodyN*?
This procedure is similar to the built-in Tcl **if** control structure. Rather than evaluating the body of the branch that is taken, it returns the subst'ed *body*. Note that the syntax is slightly more restrictive than that of the built-in Tcl **if** control structure.

::html::init ?*list*?
::html::init accepts a Tcl-style name-value list that defines values for items with a name of the form "tag.parameter". For example, a default with key "body.bgcolor" defines the background color for the *body* tag.

::html::keywords *args*
Side effect only. Call this before **::html::head** to define a keyword *meta* tag for the page. The *meta* tag is included in the result of **::html::head**.

::html::mailto *email* ?*subject*?
Generate a hypertext link to a mailto: URL.

::html::meta *args*
Compatibility name for **html::meta_name**.

::html::meta_name *args*
Side effect only. Call this before **::html::head** to define a *meta* tag for the page. The arguments (*args*) are a Tcl-style name, value list that is used for the **name=** and **content=** attributes of the *meta* tag. The *meta* tag is included in the result of **::html::head**.

::html::meta_equiv *args*
Side effect only. Call this before **::html::head** to define a *meta* tag for the page. The arguments (*args*) are a Tcl-style name, value list that is used for the **http-equiv=** and **content=** attributes of the *meta* tag. The *meta* tag is included in the result of **::html::head**.

::html::meta_charset *charset*
Side effect only. Call this before **::html::head** to define a *meta* tag for the page. The *charset* is used with the **charset=** attribute of the *meta* tag. The *meta* tag is included in the result of **::html::head**.

::html::css *href*
Side effect only. Call this before **::html::head** to define a *link* tag for a linked CSS document. The *href* value is a HTTP URL to a CSS document. The *link* tag is included in the result of **::html::head**.

Multiple calls of this command are allowed, enabling the use of multiple CSS document references. In other words, the arguments of multiple calls are accumulated, and do not overwrite each other.

::html::css-clear
Side effect only. Call this before **::html::head** to clear all links to CSS documents.

Multiple calls of this command are allowed, doing nothing after the first of a sequence with no intervening **::html::css**.

::html::js *href*

Side effect only. Call this before **::html::head** to define a *script* tag for a linked JavaScript document. The *href* is a HTTP URL to a JavaScript document. The *script* tag is included in the result of **::html::head**.

Multiple calls of this command are allowed, enabling the use of multiple JavaScript document references. In other words, the arguments of multiple calls are accumulated, and do not overwrite each other.

::html::js-clear

Side effect only. Call this before **::html::head** to clear all links to JavaScript documents.

Multiple calls of this command are allowed, doing nothing after the first of a sequence with no intervening **::html::js**.

::html::minorList *list* ?*ordered*?

Generate an ordered or unordered list of links. The *list* is a Tcl-style name, value list of labels and urls for the links. *ordered* is a boolean used to choose between an ordered or unordered list. It defaults to **false**.

::html::minorMenu *list* ?*sep*?

Generate a series of hypertext links. The *list* is a Tcl-style name, value list of labels and urls for the links. The *sep* is the text to put between each link. It defaults to " | ".

::html::nl2br *string*

This command replaces all line-endings in the *string* with a *br* tag and returns the modified text.

::html::openTag *tag* ?*param*?

Push *tag* onto a stack and generate the opening tag for *tag*. Use **::html::closeTag** to pop the tag from the stack. The second argument provides any tag arguments, as a list whose elements are formatted to be in the form "**key**=**value**".

::html::paramRow *list* ?*rparam*? ?*cparam*?

Generate a table row, including *tr* and *td* tags. Each value in *list* is placed into its own table cell. This uses **::html::cell**. The value of *rparam* is used as parameter for the *tr* tag. The value of *cparam* is passed to **::html::cell** as parameter for the *td* tags.

::html::passwordInput ?*name*?

Generate an *input* tag of type *password*. The *name* defaults to "password".

::html::passwordInputRow *label* ?*name*?

Format a table row containing a label and an *input* tag of type *password*. The *name* defaults to "password".

::html::quoteFormValue *value*

Quote special characters in *value* by replacing them with HTML entities for quotes, ampersand, and angle brackets.

::html::radioSet *key sep list*

Generate a set of *input* tags of type *radio* and an associated text label. All the radio buttons share the same *key* for their name. The *sep* is text used to separate the elements. The *list* is a Tcl-style label, value list.

::html::radioValue *name value*
Generate the "name=*name* value=*value*" for a *radio* form element. If the CGI variable *name* has the value *value*, then SELECTED is added to the return value.

::html::refresh *seconds url*
Set up a refresh *meta* tag. Call this before **::html::head** and the HEAD section will contain a *meta* tag that causes the document to refresh in *seconds* seconds. The *url* is optional. If specified, it specifies a new page to load after the refresh interval.

::html::row *args*
Generate a table row, including *tr* and *td* tags. Each value in *args* is place into its own table cell. This uses **::html::cell**. Ignores any default information set up via **::html::init**.

::html::select *name param choices* ?*current*?
Generate a *select* form element and nested *option* tags. The *name* and *param* are used to generate the *select* tag. The *choices* list is a Tcl-style name, value list.

::html::selectPlain *name param choices* ?*current*?
Like **::html::select** except that *choices* is a Tcl list of values used for the *option* tags. The label and the value for each *option* are the same.

::html::set *var val*
This procedure is similar to the built-in Tcl **set** command. The main difference is that it returns "" so it can be called from an HTML template file without appending unwanted results. The other difference is that it must take two arguments.

::html::submit *label* ?*name*? ?*title*?
Generate an *input* tag of type *submit*. The *name* defaults to "submit". When a non-empty *title* string is specified the button gains a **title=** attribute with that value.

::html::tableFromArray *arrname* ?*param*? ?*pat*?
Generate a two-column *table* and nested rows to display a Tcl array. The table gets a heading that matches the array name, and each generated row contains a name, value pair. The array names are sorted (**lsort** without special options). The argument *param* is for the *table* tag and has to contain a pre-formatted string. The *pat* is a **string match** pattern used to select the array elements to show in the table. It defaults to *, i.e. the whole array is shown.

::html::tableFromList *querylist* ?*param*?
Generate a two-column *table* and nested rows to display *querylist*, which is a Tcl dictionary. Each generated row contains a name, value pair. The information is shown in the same order as specified in the dictionary. The argument *param* is for the *table* tag and has to contain a pre-formatted string.

::html::textarea *name* ?*param*? ?*current*?
Generate a *textarea* tag wrapped around its current values.

::html::textInput *name value args*
Generate an *input* form tag with type *text*. This uses **::html::formValue**. The args is any additional tag attributes you want to put into the *input* tag.

::html::textInputRow *label name value args*
Generate an *input* form tag with type *text* formatted into a table row with an associated label. The args is any additional tag attributes you want to put into the *input* tag.

::html::varEmpty *name*
This returns 1 if the named variable either does not exist or has the empty string for its value.

::html::while *test body*
This procedure is similar to the built-in Tcl **while** control structure. Rather than evaluating the body, it returns the subst'ed *body*. Each iteration of the loop causes another string to be concatenated to the result value.

::html::doctype *id*
This procedure can be used to build the standard DOCTYPE declaration string. It will return the standard declaration string for the id, or throw an error if the id is not known. The following id's are defined:
1. HTML32
2. HTML40
3. HTML40T
4. HTML40F
5. HTML401
6. HTML401T
7. HTML401F
8. XHTML10S
9. XHTML10T
10. XHTML10F
11. XHTML11
12. XHTMLB

::html::wrapTag *tag* ?*text*? ?*args*?
A helper to wrap a *text* in a pair of open/close *tag*s. The arguments (*args*) are a Tcl-style name, value list that is used to provide attributes and associated values to the opening tag. The result is a string with the open *tag* along with the optional attributes, the optional text, and the closed tag.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *html* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

CGI programming

htmlparse

htmlparse

Keywords: htmlparse, html, parsing, queue, tree

Name

htmlparse - Procedures to parse HTML strings

Synopsis

package require Tcl 8.5 9
package require struct::stack 1.3
package require cmdline 1.1
package require htmlparse ?**1.2.3**?
::htmlparse::parse ?*-cmd cmd*? ?*-vroot tag*? ?*-split n*? ?*-incvar var*? ?*-queue q*? *html*
::htmlparse::debugCallback ?*clientdata*? *tag slash param textBehindTheTag*
::htmlparse::mapEscapes *html*
::htmlparse::2tree *html tree*
::htmlparse::removeVisualFluff *tree*
::htmlparse::removeFormDefs *tree*

Description

The **htmlparse** package provides commands that allow libraries and applications to parse HTML in a string into a representation of their choice.

The following commands are available:

::htmlparse::parse ?-cmd *cmd*? ?-vroot *tag*? ?-split *n*? ?-incvar *var*? ?-queue *q*? *html*

This command is the basic parser for HTML. It takes an HTML string, parses it and invokes a command prefix for every tag encountered. It is not necessary for the HTML to be valid for this parser to function. It is the responsibility of the command invoked for every tag to check this. Another responsibility of the invoked command is the handling of tag attributes and character entities (escaped characters). The parser provides the un-interpreted tag attributes to the invoked command to aid in the former, and the package at large provides a helper command, **::htmlparse::mapEscapes**, to aid in the handling of the latter. The parser *does* ignore leading DOCTYPE declarations and all valid HTML comments it encounters.

All information beyond the HTML string itself is specified via options, these are explained below.

To help understand the options, some more background information about the parser.

It is capable of detecting incomplete tags in the HTML string given to it. Under normal circumstances this will cause the parser to throw an error, but if the option *-incvar* is used to specify a global (or namespace) variable, the parser will store the incomplete part of the input into this variable instead. This will aid greatly in the handling of incrementally arriving HTML, as the parser will handle whatever it can and defer the handling of the incomplete part until more data has arrived.

Another feature of the parser are its two possible modes of operation. The normal mode is activated if the option *-queue* is not present on the command line invoking the parser. If it is present, the parser will go into the incremental mode instead.

The main difference is that a parser in normal mode will immediately invoke the command prefix for each tag it encounters. In incremental mode however the parser will generate a number of scripts which invoke the command prefix for groups of tags in the HTML string and then store these scripts in the specified queue. It is then the responsibility of the caller of the parser to ensure the execution of the scripts in the queue.

Note: The queue object given to the parser has to provide the same interface as the queue

defined in tcllib -> struct. This means, for example, that all queues created via that tcllib module can be immediately used here. Still, the queue doesn't have to come from tcllib -> struct as long as the same interface is provided.

In both modes the parser will return an empty string to the caller.

The *-split* option may be given to a parser in incremental mode to specify the size of the groups it creates. In other words, -split 5 means that each of the generated scripts will invoke the command prefix for 5 consecutive tags in the HTML string. A parser in normal mode will ignore this option and its value.

The option *-vroot* specifies a virtual root tag. A parser in normal mode will invoke the command prefix for it immediately before and after it processes the tags in the HTML, thus simulating that the HTML string is enclosed in a combination. In incremental mode however the parser is unable to provide the closing virtual root as it never knows when the input is complete. In this case the first script generated by each invocation of the parser will contain an invocation of the command prefix for the virtual root as its first command. The following options are available:

-cmd *cmd*
: The command prefix to invoke for every tag in the HTML string. Defaults to *::htmlparse::debugCallback*.

-vroot *tag*
: The virtual root tag to add around the HTML in normal mode. In incremental mode it is the first tag in each chunk processed by the parser, but there will be no closing tags. Defaults to *hmstart*.

-split *n*
: The size of the groups produced by an incremental mode parser. Ignored when in normal mode. Defaults to 10. Values <= 0 are not allowed.

-incvar *var*
: The name of the variable where to store any incomplete HTML into. This makes most sense for the incremental mode. The parser will throw an error if it sees incomplete HTML and has no place to store it to. This makes sense for the normal mode. Only incomplete tags are detected, not missing tags. Optional, defaults to 'no variable'.

Interface to the command prefix
: In normal mode the parser will invoke the command prefix with four arguments appended. See **::htmlparse::debugCallback** for a description.

  In incremental mode, however, the generated scripts will invoke the command prefix with five arguments appended. The last four of these are the same which were mentioned above. The first is a placeholder string (**@win@**) for a clientdata value to be supplied later during the actual execution of the generated scripts. This could be a tk window path, for example. This allows the user of this package to preprocess HTML strings without committing them to a specific window, object, whatever during parsing. This connection can be made later. This also means that it is possible to cache preprocessed HTML. Of course, nothing prevents the user of the parser from replacing the placeholder with an empty string.

::htmlparse::debugCallback ?*clientdata*? *tag slash param textBehindTheTag*

This command is the standard callback used by the parser in **::htmlparse::parse** if none was specified by the user. It simply dumps its arguments to stdout. This callback can be used for both normal and incremental mode of the calling parser. In other words, it accepts four or five arguments. The last four arguments are described below. The optional fifth argument contains the clientdata value passed to the callback by a parser in incremental mode. All callbacks have to follow the signature of this command in the last four arguments, and callbacks used in incremental parsing have to follow this signature in the last five arguments.

The first argument, *clientdata*, is optional and present only if this command is invoked by a parser in incremental mode. It contains whatever the user of this package wishes.

The second argument, *tag*, contains the name of the tag which is currently processed by the parser.

The third argument, *slash*, is either empty or contains a slash character. It allows the callback to distinguish between opening (slash is empty) and closing tags (slash contains a slash character).

The fourth argument, *param*, contains the un-interpreted list of parameters to the tag.

The fifth and last argument, *textBehindTheTag*, contains the text found by the parser behind the tag named in *tag*.

::htmlparse::mapEscapes *html*

This command takes a HTML string, substitutes all escape sequences with their actual characters and then returns the resulting string. HTML strings which do not contain escape sequences are returned unchanged.

::htmlparse::2tree *html tree*

This command is a wrapper around **::htmlparse::parse** which takes an HTML string (in *html*) and converts it into a tree containing the logical structure of the parsed document. The name of the tree is given to the command as its second argument (*tree*). The command does **not** generate the tree by itself but expects that the caller provided it with an existing and empty tree. It also expects that the specified tree object follows the same interface as the tree object in tcllib -> struct. It doesn't have to be from tcllib -> struct, but it must provide the same interface.

The internal callback does some basic checking of HTML validity and tries to recover from the most basic errors. The command returns the contents of its second argument. Side effects are the creation and manipulation of a tree object.

Each node in the generated tree represent one tag in the input. The name of the tag is stored in the attribute *type* of the node. Any html attributes coming with the tag are stored unmodified in the attribute *data* of the tag. In other words, the command does *not* parse html attributes into their names and values.

If a tag contains text its node will have children of type *PCDATA* containing this text. The text

will be stored in the attribute *data* of these children.

::htmlparse::removeVisualFluff ***tree***
This command walks a tree as generated by **::htmlparse::2tree** and removes all the nodes which represent visual tags and not structural ones. The purpose of the command is to make the tree easier to navigate without getting bogged down in visual information not relevant to the search. Its only argument is the name of the tree to cut down.

::htmlparse::removeFormDefs ***tree***
Like **::htmlparse::removeVisualFluff** this command is here to cut down on the size of the tree as generated by **::htmlparse::2tree**. It removes all nodes representing forms and form elements. Its only argument is the name of the tree to cut down.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *htmlparse* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Text processing

http

autoproxy

Keywords: autoproxy, authentication, http, proxy

Name

autoproxy - Automatic HTTP proxy usage and authentication

Synopsis

package require Tcl 8.5 9
package require http ?**2.0**?
package require autoproxy ?**1.8.1**?
::autoproxy::init
::autoproxy::cget *-option*
::autoproxy::configure ?*-option value*?
::autoproxy::tls_connect *args*
::autoproxy::tunnel_connect *args*
::autoproxy::tls_socket *args*

Description

This package attempts to automate the use of HTTP proxy servers in Tcl HTTP client code. It tries to initialize the web access settings from system standard locations and can be configured to negotiate authentication with the proxy if required.

On Unix the standard for identifying the local HTTP proxy server seems to be to use the environment variable http_proxy or ftp_proxy and no_proxy to list those domains to be excluded from proxying. On Windows we can retrieve the Internet Settings values from the registry to obtain pretty much the same information. With this information we can setup a suitable filter procedure for the Tcl http package and arrange for automatic use of the proxy.

There seem to be a number of ways that the http_proxy environment variable may be set up. Either a plain host:port or more commonly a URL and sometimes the URL may contain authentication parameters or these may be requested from the user or provided via http_proxy_user and http_proxy_pass. This package attempts to deal with all these schemes. It will do it's best to get the required parameters from the environment or registry and if it fails can be reconfigured.

TLS security considerations

Note This section only applies if TLS support is provided by the **TLS** package. It does not apply when **autoproxy** was configured to use some other package which can provide the same (i.e **twapi**), via the **-tls_package** configuration option.

This package uses the **TLS** package to handle the security for **https** urls and other socket connections.

Policy decisions like the set of protocols to support and what ciphers to use are not the responsibility of **TLS**, nor of this package itself however. Such decisions are the responsibility of whichever application is using the package, and are likely influenced by the set of servers the application will talk to as well.

For example, in light of the recent POODLE attack discovered by Google many servers will disable support for the SSLv3 protocol. To handle this change the applications using **TLS** must be patched, and not this package, nor **TLS** itself. Such a patch may be as simple as generally activating **tls1** support, as shown in the example below.

```
    package require tls
    tls::init -tls1 1 ;# forcibly activate support for the TLS1
    protocol

    ... your own application code ...
```

Commands

::autoproxy::init
: Initialize the autoproxy package from system resources. Under unix this means we look for environment variables. Under windows we look for the same environment variables but also look at the registry settings used by Internet Explorer.

::autoproxy::cget *-option*
: Retrieve individual package configuration options. See Options.

::autoproxy::configure ?-option *value*?
: Configure the autoproxy package. Calling **configure** with no options will return a list of all option names and values. See Options.

::autoproxy::tls_connect *args*
: Connect to a secure socket through a proxy. HTTP proxy servers permit the use of the

CONNECT HTTP command to open a link through the proxy to the target machine. This function hides the details. For use with the http package see **tls_socket**.

The *args* list may contain any of the options supported by the specific TLS package that is in use but must end with the host and port as the last two items.

::autoproxy::tunnel_connect *args*

Connect to a target host throught a proxy. This uses the same CONNECT HTTP command as the **tls_connect** but does not promote the link security once the connection is established.

The *args* list may contain any of the options supported by the specific TLS package that is in use but must end with the host and port as the last two items.

Note that many proxy servers will permit CONNECT calls to a limited set of ports - typically only port 443 (the secure HTTP port).

::autoproxy::tls_socket *args*

This function is to be used to register a proxy-aware secure socket handler for the https protocol. It may only be used with the Tcl http package and should be registered using the http::register command (see the examples below). The job of actually creating the tunnelled connection is done by the tls_connect command and this may be used when not registering with the http package.

Options

-host hostname

see below ...

-proxy_host hostname

Set the proxy hostname. This is normally set up by **init** but may be configured here as well.

-port number

see below ...

-proxy_port number

Set the proxy port number. This is normally set up by **init**. e.g. **configure -port** *3128*

-no_proxy list

You may manipulate the **no_proxy** list that was setup by **init**. The value of this option is a tcl list of strings that are matched against the http request host using the tcl **string match** command. Therefore glob patterns are permitted. For instance, **configure -no_proxy** **.localdomain*

-authProc procedure

This option may be used to set an application defined procedure to be called when **configure -basic** is called with either no or insufficient authentication details. This can be used to present a dialog to the user to request the additional information.

-basic

Following options are for configuring the Basic authentication scheme parameters. See Basic authentication. To unset the proxy authentication information retained from a previous call of this function either "--" or no additional parameters can be supplied. This will remove the existing authentication information.

-tls_package packagename

This option may be used to configure the Tcl package to use for TLS support. Valid package names are **tls** (default) and **twapi**.

Basic authentication

Basic is the simplest and most commonly use HTTP proxy authentication scheme. It is described in (1 section 11) and also in (2). It offers no privacy whatsoever and its use should be discouraged in favour of more secure alternatives like Digest. To perform Basic authentication the client base64 encodes the username and plaintext password separated by a colon. This encoded text is prefixed with the word "Basic" and a space.

The following options exists for this scheme:

-username name
The username required to authenticate with the configured proxy.

-password password
The password required for the username specified.

-realm realm
This option is not used by this package but may be used in requesting authentication details from the user.

--
The end-of-options indicator may be used alone to unset any authentication details currently enabled.

Examples

```
package require autoproxy
autoproxy::init
autoproxy::configure -basic -username ME -password SEKRET
set tok [http::geturl http://wiki.tcl.tk/]
http::data $tok
```

```
package require http
package require tls
package require autoproxy
autoproxy::init
http::register https 443 autoproxy::tls_socket
set tok [http::geturl https://www.example.com/]
```

References

1. Berners-Lee, T., Fielding R. and Frystyk, H. "Hypertext Transfer Protocol -- HTTP/1.0", RFC 1945, May 1996, (http://www.rfc-editor.org/rfc/rfc1945.txt)
2. Franks, J. et al. "HTTP Authentication: Basic and Digest Access Authentication", RFC 2617, June 1999 (http://www.rfc-editor.org/rfc/rfc2617.txt)

Bugs

At this time only Basic authentication (1) (2) is supported. It is planned to add support for Digest (2) and NTLM in the future.

Authors
Pat Thoyts

Bugs, ideas, feedback
If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *http :: autoproxy* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Networking

httpd

httpd
Keywords: httpd, TclOO, WWW, http, httpserver, services

Name
httpd - A TclOO and coroutine based web server

Synopsis
package require Tcl 8.6 9
package require uuid
package require clay
package require coroutine
package require fileutil
package require fileutil::magic::filetype
package require websocket
package require mime
package require cron
package require uri
package require Markdown

method **ChannelCopy** *in out* ?*args*?

method **html_header** ?*title* ? ?*args*?

method **html_footer** ?*args*?

method **http_code_string** *code*

method **HttpHeaders** *sock* ?*debug* ?

method **HttpHeaders_Default**

method **HttpServerHeaders**

method **MimeParse** *mimetext*

method **Url_Decode** *data*
method **Url_PathCheck** *urlsuffix*
method **wait** *mode sock*
variable **ChannelRegister**
variable **reply**
variable **request**
delegate
method **constructor** *ServerObj* ?*args*?
method **destructor** ?*dictargs*?
method **ChannelRegister** ?*args*?
method **close**
method **Log_Dispatched**
method **dispatch** *newsock datastate*
method **Dispatch**
method **html_header** *title* ?*args*?
method **html_footer** ?*args*?
method **error** *code* ?*msg* ? ?*errorInfo* ?
method **content**
method **EncodeStatus** *status*
method **log** *type* ?*info* ?
method **CoroName**
method **DoOutput**
method **FormData**
method **PostData** *length*
method **Session_Load**
method **puts** *line*
method **RequestFind** *field*
method **request** *subcommand* ?*args*?
method **reply** *subcommand* ?*args*?
method **reset**
method **timeOutCheck**
method **timestamp**
variable **template**
variable **url_patterns**
method **constructor** *args* ?*port* **auto**? ?*myaddr* **127.0.0.1**? ?*string* **auto**? ?*name* **auto**? ?*doc_root* ? ?*reverse_dns* **0**? ?*configuration_file* ? ?*protocol* **HTTP/1.1**?
method **destructor** ?*dictargs*?

method **connect** *sock ip port*
method **ServerHeaders** *ip http_request mimetxt*
method **Connect** *uuid sock ip*
method **counter** *which*
method **CheckTimeout**
method **debug** ?*args*?
method **dispatch** *data*
method **Dispatch_Default** *reply*
method **Dispatch_Local** *data*
method **Headers_Local** *varname*
method **Headers_Process** *varname*
method **HostName** *ipaddr*
method **log** ?*args*?
method **plugin** *slot* ?*class* ?
method **port_listening**
method **PrefixNormalize** *prefix*
method **source** *filename*
method **start**
method **stop**
method **SubObject {} db**
method **SubObject {} default**
method **template** *page*
method **TemplateSearch** *page*
method **Thread_start**
method **Uuid_Generate**
method **Validate_Connection** *sock ip*
method **reset**
method **content**
method **Dispatch**
method **content**
method **FileName**
method **DirectoryListing** *local_file*
method **content**
method **Dispatch**
variable **exename**
method **CgiExec** *execname script arglist*
method **Cgi_Executable** *script*

method **proxy_channel**
method **proxy_path**
method **ProxyRequest** *chana chanb*
method **ProxyReply** *chana chanb* ?*args*?
method **Dispatch**
method **FileName**
method **proxy_channel**
method **ProxyRequest** *chana chanb*
method **ProxyReply** *chana chanb* ?*args*?
method **DirectoryListing** *local_file*
method **EncodeStatus** *status*
method **scgi_info**
method **proxy_channel**
method **ProxyRequest** *chana chanb*
method **ProxyReply** *chana chanb* ?*args*?
method **debug** ?*args*?
method **Connect** *uuid sock ip*
method **Dispatch_Dict** *data*
method **uri {} add** *vhosts patterns info*
method **uri {} direct** *vhosts patterns info body*
method **output**
method **DoOutput**
method **close**
method **local_memchan** *command* ?*args*?
method **Connect_Local** *uuid sock* ?*args*?

Description

This module implements a web server suitable for embedding in an application. The server is object oriented, and contains all fundamentals needed for a full service website.

Minimal example

Starting a web service requires starting a class of type **httpd::server**, and providing that server with one or more URIs to service, and **httpd::reply** derived classes to generate them.

```
package require httpd

oo::class create ::reply.hello {
  method content {} {
    my puts "<HTML><HEAD><TITLE>IRM Dispatch
    Server</TITLE></HEAD><BODY>"
```

```
    my puts "<h1>Hello World!</h1>"
    my puts </BODY></HTML>
  }
}
::httpd::server create HTTPD port 8015 myaddr 127.0.0.1 doc_root [file
    home]/htdocs
HTTPD plugin dispatch ::httpd::plugin.dict_dispatch
HTTPD uri add * /hello [list mixin reply.hello]
vwait forever
```

The bare module has facilities to host files from a file system. Files that end in a **.tml** will be substituted in the style of Tclhttpd:

```
<!-- hello.tml -->
[my html_header {Hello World!}]
Your Server is running.
<p>
The time is now [clock format [clock seconds]]
[my html_footer]
```

A complete httpd server example in the Tcllib's /examples directory. This also shows how to dispatch URIs to other processes via SCGI and HTTP proxies.

```
cd ~/tcl/sandbox/tcllib
tclsh examples/httpd.tcl
```

Classes

Class httpd::mime

A metaclass for MIME handling behavior across a live socket

Methods

method ChannelCopy *in out* ?*args*?
see below ...

method html_header ?*title* ? ?*args*?
Returns a block of HTML

method html_footer ?*args*?
see below ...

method http_code_string *code*
see below ...

method HttpHeaders *sock* ?*debug* ?
see below ...

method HttpHeaders_Default
see below ...

method HttpServerHeaders
see below ...

method MimeParse *mimetext*
Converts a block of mime encoded text to a key/value list. If an exception is encountered, the

method will generate its own call to the **error** method, and immediately invoke the **output** method to produce an error code and close the connection.

method Url_Decode *data*
De-httpizes a string.

- method **Url_PathCheck** *urlsuffix*
- method **wait** *mode sock*

Class httpd::reply

ancestors: **httpd::mime**

A class which shephards a request through the process of generating a reply. The socket associated with the reply is available at all times as the *chan* variable. The process of generating a reply begins with an **httpd::server** generating a **http::class** object, mixing in a set of behaviors and then invoking the reply object's **dispatch** method. In normal operations the **dispatch** method:

1. Invokes the **reset** method for the object to populate default headers.
2. Invokes the **HttpHeaders** method to stream the MIME headers out of the socket
3. Invokes the **request parse** method to convert the stream of MIME headers into a dict that can be read via the **request** method.
4. Stores the raw stream of MIME headers in the *rawrequest* variable of the object.
5. Invokes the **content** method for the object, generating an call to the **error** method if an exception is raised.
6. Invokes the **output** method for the object

Developers have the option of streaming output to a buffer via the **puts** method of the reply, or simply populating the *reply_body* variable of the object. The information returned by the **content** method is not interpreted in any way. If an exception is thrown (via the **error** command in Tcl, for example) the caller will auto-generate a 500 {Internal Error} message. A typical implementation of **content** look like:

```
clay::define ::test::content.file {
   superclass ::httpd::content.file
   # Return a file
   # Note: this is using the content.file mixin which looks for the
   reply_file variable
   # and will auto-compute the Content-Type
   method content {} {
     my reset
    set doc_root [my request get DOCUMENT_ROOT]
    my variable reply_file
    set reply_file [file join $doc_root index.html]
   }
}
clay::define ::test::content.time {
  # return the current system time
   method content {} {
       my variable reply_body
```

```
    my reply set Content-Type text/plain
       set reply_body [clock seconds]
   }
}
clay::define ::test::content.echo {
   method content {} {
       my variable reply_body
    my reply set Content-Type [my request get CONTENT_TYPE]
       set reply_body [my PostData [my request get CONTENT_LENGTH]]
   }
}
clay::define ::test::content.form_handler {
   method content {} {
     set form [my FormData]
     my reply set Content-Type {text/html; charset=UTF-8}
    my puts [my html_header {My Dynamic Page}]
    my puts "<BODY>"
    my puts "You Sent<p>"
    my puts "<TABLE>"
    foreach {f v} $form {
      my puts "<TR><TH>$f</TH><TD><verbatim>$v</verbatim></TD>"
    }
    my puts "</TABLE><p>"
    my puts "Send some info:<p>"
    my puts "<FORM action=/[my request get REQUEST_PATH] method POST>"
    my puts "<TABLE>"
    foreach field {name rank serial_number} {
      set line "<TR><TH>$field</TH><TD><input name=\"$field\" "
      if {[dict exists $form $field]} {
        append line " value=\"[dict get $form $field]\""
      }
      append line " /></TD></TR>"
      my puts $line
    }
    my puts "</TABLE>"
    my puts [my html footer]
   }
}
```

Variable

variable ChannelRegister
see below ...

variable reply
A dictionary which will converted into the MIME headers of the reply

variable request
A dictionary containing the SCGI transformed HTTP headers for the request

Delegate

delegate
The server object which spawned this reply

Methods

method constructor *ServerObj* ?*args*?
see below ...

method destructor ?*dictargs*?
clean up on exit

method ChannelRegister ?*args*?
Registers a channel to be closed by the close method

method close
Close channels opened by this object

method Log_Dispatched
Record a dispatch event

method dispatch *newsock datastate*
Accept the handoff from the server object of the socket *newsock* and feed it the state *datastate*. Fields the *datastate* are looking for in particular are:

* **mixin** - A key/value list of slots and classes to be mixed into the object prior to invoking **Dispatch**.

* **http** - A key/value list of values to populate the object's *request* ensemble

All other fields are passed along to the **clay** structure of the object.

method Dispatch
see below ...

method html_header *title* ?*args*?
see below ...

method html_footer ?*args*?
see below ...

method error *code* ?*msg* ? ?*errorInfo* ?
see below ...

method content
REPLACE ME: This method is the "meat" of your application. It writes to the result buffer via the "puts" method and can tweak the headers via "clay put header_reply"

method EncodeStatus *status*
Formulate a standard HTTP status header from he string provided.

method log *type* ?*info* ?
see below ...

method CoroName
see below ...

method DoOutput
Generates the the HTTP reply, streams that reply back across *chan*, and destroys the object.

method FormData
For GET requests, converts the QUERY_DATA header into a key/value list. For POST requests, reads the Post data and converts that information to a key/value list for application/x-www-form-urlencoded posts. For multipart posts, it composites all of the MIME headers of the post to a singular key/value list, and provides MIME_* information as computed by the **mime** package, including the MIME_TOKEN, which can be fed back into the mime package to read out the contents.

method PostData *length*
Stream *length* bytes from the *chan* socket, but only of the request is a POST or PUSH. Returns an empty string otherwise.

method Session_Load
Manage session data

method puts *line*
Appends the value of *string* to the end of *reply_body*, as well as a trailing newline character.

method RequestFind *field*
see below ...

method request *subcommand* ?*args*?
see below ...

method reply *subcommand* ?*args*?
see below ...

method reset
Clear the contents of the *reply_body* variable, and reset all headers in the **reply** structure back to the defaults for this object.

method timeOutCheck
Called from the **http::server** object which spawned this reply. Checks to see if too much time has elapsed while waiting for data or generating a reply, and issues a timeout error to the request if it has, as well as destroy the object and close the *chan* socket.

method timestamp
Return the current system time in the format:

```
%a, %d %b %Y %T %Z
```

Class httpd::server

ancestors: **httpd::mime**

Variable

- variable **template**
- variable **url_patterns**

Methods

method constructor *args* ?*port* auto? ?*myaddr* 127.0.0.1? ?*string* auto? ?*name* auto? ?*doc_root* ? ?*reverse_dns* 0? ?*configuration_file* ? ?*protocol* HTTP/1.1?
see below ...

method destructor ?*dictargs*?
see below ...

method connect *sock ip port*

Reply to an open socket. This method builds a coroutine to manage the remainder of the connection. The coroutine's operations are driven by the **Connect** method.

method ServerHeaders *ip http_request mimetxt*

see below ...

method Connect *uuid sock ip*

This method reads HTTP headers, and then consults the **dispatch** method to determine if the request is valid, and/or what kind of reply to generate. Under normal cases, an object of class **::http::reply** is created, and that class's **dispatch** method. This action passes control of the socket to the reply object. The reply object manages the rest of the transaction, including closing the socket.

method counter *which*

Increment an internal counter.

method CheckTimeout

Check open connections for a time out event.

method debug ?*args*?

see below ...

method dispatch *data*

Given a key/value list of information, return a data structure describing how the server should reply.

method Dispatch_Default *reply*

Method dispatch method of last resort before returning a 404 NOT FOUND error. The default behavior is to look for a file in *DOCUMENT_ROOT* which matches the query.

method Dispatch_Local *data*

Method dispatch method invoked prior to invoking methods implemented by plugins. If this method returns a non-empty dictionary, that structure will be passed to the reply. The default is an empty implementation.

method Headers_Local *varname*

Introspect and possibly modify a data structure destined for a reply. This method is invoked before invoking Header methods implemented by plugins. The default implementation is empty.

method Headers_Process *varname*

Introspect and possibly modify a data structure destined for a reply. This method is built dynamically by the **plugin** method.

method HostName *ipaddr*

Convert an ip address to a host name. If the server/ reverse_dns flag is false, this method simply returns the IP address back. Internally, this method uses the *dns* module from tcllib.

method log ?*args*?

Log an event. The input for args is free form. This method is intended to be replaced by the user, and is a noop for a stock http::server object.

method plugin *slot* ?*class* ?

Incorporate behaviors from a plugin. This method dynamically rebuilds the **Dispatch** and **Headers** method. For every plugin, the server looks for the following entries in *clay plugin/*:

load - A script to invoke in the server's namespace during the **plugin** method invokation.

dispatch - A script to stitch into the server's **Dispatch** method.

headers - A script to stitch into the server's **Headers** method.

thread - A script to stitch into the server's **Thread_start** method.

method port_listening
Return the actual port that httpd is listening on.

method PrefixNormalize *prefix*
For the stock version, trim trailing /'s and *'s from a prefix. This method can be replaced by the end user to perform any other transformations needed for the application.

method source *filename*
see below ...

method start
Open the socket listener.

method stop
Shut off the socket listener, and destroy any pending replies.

method SubObject {} db
see below ...

method SubObject {} default
see below ...

method template *page*
Return a template for the string *page*

method TemplateSearch *page*
Perform a search for the template that best matches *page*. This can include local file searches, in-memory structures, or even database lookups. The stock implementation simply looks for files with a .tml or .html extension in the ?doc_root? directory.

method Thread_start
Built by the **plugin** method. Called by the **start** method. Intended to allow plugins to spawn worker threads.

method Uuid_Generate
Generate a GUUID. Used to ensure every request has a unique ID. The default implementation is:

```
   return [::clay::uuid generate]
```

method Validate_Connection *sock ip*
Given a socket and an ip address, return true if this connection should be terminated, or false if it should be allowed to continue. The stock implementation always returns 0. This is intended for applications to be able to implement black lists and/or provide security based on IP address.

Class httpd::server::dispatch

ancestors: **httpd::server**

Provide a backward compadible alias

Class httpd::content.redirect
Methods

- method **reset**
- method **content**

Class httpd::content.cache
Methods

- method **Dispatch**

Class httpd::content.template
Methods

- method **content**

Class httpd::content.file
Class to deliver Static content When utilized, this class is fed a local filename by the dispatcher

Methods

- method **FileName**
- method **DirectoryListing** *local_file*
- method **content**
- method **Dispatch**

Class httpd::content.exec
Variable

- variable **exename**

Methods

- method **CgiExec** *execname script arglist*
- method **Cgi_Executable** *script*

Class httpd::content.proxy
ancestors: **httpd::content.exec**

Return data from an proxy process

Methods

- method **proxy_channel**
- method **proxy_path**
- method **ProxyRequest** *chana chanb*
- method **ProxyReply** *chana chanb* ?*args*?
- method **Dispatch**

Class httpd::content.CGI
ancestors: **httpd::content.proxy**

Methods

method FileName
see below ...

method proxy_channel

see below ...

method ProxyRequest *chana chanb*
see below ...

method ProxyReply *chana chanb* ?*args*?
see below ...

method DirectoryListing *local_file*
For most CGI applications a directory list is vorboten

Class httpd::protocol.SCGI
Return data from an SCGI process

Methods

- method **EncodeStatus** *status*

Class httpd::content.SCGI
ancestors: **httpd::content.proxy**

Methods

- method **scgi_info**
- method **proxy_channel**
- method **ProxyRequest** *chana chanb*
- method **ProxyReply** *chana chanb* ?*args*?

Class httpd::server.SCGI
ancestors: **httpd::server**

Act as an SCGI Server

Methods

- method **debug** ?*args*?
- method **Connect** *uuid sock ip*

Class httpd::content.websocket
Upgrade a connection to a websocket

Class httpd::plugin
httpd plugin template

Class httpd::plugin.dict_dispatch
A rudimentary plugin that dispatches URLs from a dict data structure

Methods

method Dispatch_Dict *data*
Implementation of the dispatcher

- method **uri {} add** *vhosts patterns info*
- method **uri {} direct** *vhosts patterns info body*

Class httpd::reply.memchan
ancestors: **httpd::reply**

Methods

- method **output**
- method **DoOutput**
- method **close**

Class httpd::plugin.local_memchan

Methods

method local_memchan *command* ?*args*?
see below ...

method Connect_Local *uuid sock* ?*args*?
A modified connection method that passes simple GET request to an object and pulls data directly from the reply_body data variable in the object Needed because memchan is bidirectional, and we can't seem to communicate that the server is one side of the link and the reply is another

Authors

Sean Woods

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *network* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Networking

Copyright


```
Copyright (c) 2018 Sean Woods <yoda@etoyoc.com>
```


ident

ident

Keywords: ident, identification, rfc 1413

Name

ident - Ident protocol client

Synopsis

package require Tcl 8.5 9
package require ident ?**0.44**?
::ident::query *socket* ?*callback*?

Description

The **ident** package provides a client implementation of the ident protocol as defined in RFC 1413 (http://www.rfc-editor.org/rfc/rfc1413.txt).

::ident::query *socket* ?*callback*?

This command queries the ident daemon on the remote side of the given socket, and returns the result of the query as a dictionary. Interpreting the dictionary as list the first key will always be **resp-type**, and can have one of the values **USERID**, **ERROR**, and **FATAL**. These *response types* have the following meanings:

USERID

This indicates a successful response. Two more keys and associated values are returned, **opsys**, and **user-id**.

ERROR

This means the ident server has returned an error. A second key named **error** is present whose value contains the **error-type** field from the server response.

FATAL

Fatal errors happen when no ident server is listening on the remote side, or when the ident server gives a response that does not conform to the RFC. A detailed error message is returned under the **error** key.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *ident* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Networking

Copyright

```
Copyright (c) 2004 Reinhard Max <max@tclers.tk>
```

imap4

imap4

Keywords: imap4, email, imap, internet, mail, net, rfc3501, ssl, tls

Name

imap4 - imap client-side tcl implementation of imap protocol

Synopsis

package require Tcl 8.5 9

package require imap4 ?**0.5.5**?
::imap4::open *hostname* ?*port*?
::imap4::starttls *chan*
::imap4::login *chan user pass*
::imap4::folders *chan* ?*-inline*? ?*mboxref*? ?*mboxname*?
::imap4::select *chan* ?*mailbox*?
::imap4::examine *chan* ?*mailbox*?
::imap4::fetch *chan range* ?*-inline*? ?*attr ...*?
::imap4::noop *chan*
::imap4::check *chan*
::imap4::folderinfo *chan* ?*info*?
::imap4::msginfo *chan msgid* ?*info*? ?*defval*?
::imap4::mboxinfo *chan* ?*info*?
::imap4::isableto *chan* ?*capability*?
::imap4::create *chan mailbox*
::imap4::delete *chan mailbox*
::imap4::rename *chan oldname newname*
::imap4::subscribe *chan mailbox*
::imap4::unsubscribe *chan mailbox*
::imap4::search *chan expr* ?*...*?
::imap4::close *chan*
::imap4::cleanup *chan*
::imap4::debugmode *chan* ?*errormsg*?
::imap4::store *chan range data flaglist*
::imap4::expunge *chan*
::imap4::copy *chan msgid mailbox*
::imap4::logout *chan*

Description

The **imap4** library package provides the client side of the *Internet Message Access Protocol* (IMAP) using standard sockets or secure connection via TLS/SSL. The package is fully implemented in Tcl.

This document describes the procedures and explains their usage.

Procedures

This package defines the following public procedures:

::imap4::open *hostname* ?*port*?

Open a new IMAP connection and initalize the handler, the imap communication channel (handler) is returned.

hostname - mail server

port - connection port, defaults to 143

The namespace variable **::imap4::use_ssl** can be used to establish to a secure connection via TSL/SSL if set to true. In this case default connection port defaults to 993.

Note: For connecting via SSL the Tcl module *tls* must be already loaded otherwise an error is

raised.

```
    package require tls              ; # must be loaded for TLS/SSL
    set ::imap4::use_ssl 1           ; # request a secure connection
    set chan [::imap4::open $server] ; # default port is now 993
```

::imap4::starttls *chan*

Use this when tasked with connecting to an unsecure port which must be changed to a secure port prior to user login. This feature is known as *STARTTLS*.

::imap4::login *chan user pass*

Login using the IMAP LOGIN command, 0 is returned on successful login.

chan - imap channel

user - username

pass - password

::imap4::folders *chan* ?*-inline*? ?*mboxref*? ?*mboxname*?

Get list of matching folders, 0 is returned on success.

Wildcards '*' as '%' are allowed for *mboxref* and *mboxname*, command **::imap4::folderinfo** can be used to retrieve folder information.

chan - imap channel

mboxref - mailbox reference, defaults to ""

mboxname - mailbox name, defaults to "*"

If **-inline** is specified a compact folderlist is returned instead of the result code. All flags are converted to lowercase and leading special characters are removed.

```
{{Arc08 noselect} {Arc08/Private {noinferiors unmarked}} {INBOX
    noinferiors}}
```

::imap4::select *chan* ?*mailbox*?

Select a mailbox, 0 is returned on success.

chan - imap channel

mailbox - Path of the mailbox, defaults to *INBOX*

Prior to examine/select an open mailbox must be closed - see: **::imap4::close**.

::imap4::examine *chan* ?*mailbox*?

"Examines" a mailbox, read-only equivalent of **::imap4::select**.

chan - imap channel

mailbox - mailbox name or path to mailbox, defaults to *INBOX*

Prior to examine/select an open mailbox must be closed - see: **::imap4::close**.

::imap4::fetch *chan range* ?*-inline*? ?*attr ...*?

Fetch attributes from messages.

The attributes are fetched and stored in the internal state which can be retrieved with command **::imap4::msginfo**, 0 is returned on success. If **-inline** is specified, alle records are returned as list in order as defined in the *attr* argument.

chan - imap channel

range - message index in format *FROM*:*TO*

attr - imap attributes to fetch

Note: If *FROM* is omitted, the 1st message is assumed, if *TO* is ommitted the last message is assumed. All message index ranges are 1-based.

::imap4::noop *chan*

Send NOOP command to server. May get information as untagged data.

chan - imap channel

::imap4::check *chan*

Send CHECK command to server. Flush to disk.

chan - imap channel

::imap4::folderinfo *chan* ?*info*?

Get information on the recently selected folderlist. If the *info* argument is omitted or a null string, the full list of information available for the mailbox is returned.

If the required information name is suffixed with a ? character, the command returns true if the information is available, or false if it is not.

chan - imap channel

info - folderlist options to retrieve

Currently supported options: *delim* - hierarchy delimiter only, *match* - ref and mbox search patterns (see **::imap4::folders**), *names* - list of folder names only, *flags* - list of folder names with flags in format *{ {name {flags}} ... }* (see also compact format in function **::imap4::folders**).

```
{{Arc08 {{\NoSelect}}} {Arc08/Private {{\NoInferiors} {\UnMarked}}}
    {INBOX {\NoInferiors}}}
```

::imap4::msginfo *chan msgid* ?*info*? ?*defval*?

Get information (from previously collected using fetch) from a given *msgid*. If the 'info' argument is omitted or a null string, the list of available information options for the given message is returned.

If the required information name is suffixed with a ? character, the command returns true if

the information is available, or false if it is not.

chan - imap channel

msgid - message number

info - imap keyword to retrieve

defval - default value, returned if info is empty

Note: All message index ranges are 1-based.

::imap4::mboxinfo *chan* ?*info*?

Get information on the currently selected mailbox. If the *info* argument is omitted or a null string, the list of available information options for the mailbox is returned.

If the required information name is suffixed with a ? character, the command returns true if the information is available, or false if it is not.

chan - imap channel

opt - mailbox option to retrieve

Currently supported options: *EXISTS* (noof msgs), *RECENT* (noof 'recent' flagged msgs), *FLAGS*

In conjunction with OK: *PERMFLAGS*, *UIDNEXT*, *UIDVAL*, *UNSEEN*

Div. states: *CURRENT*, *FOUND*, *PERM*.

```
    ::imap4::select $chan INBOX
    puts "[::imap4::mboxinfo $chan exists] mails in INBOX"
```

::imap4::isableto *chan* ?*capability*?

Test for capability. It returns 1 if requested capability is supported, 0 otherwise. If *capability* is omitted all capability imap codes are retured as list.

chan - imap channel

info - imap keyword to retrieve

Note: Use the capability command to ask the server if not already done by the user.

::imap4::create *chan mailbox*

Create a new mailbox.

chan - imap channel

mailbox - mailbox name

::imap4::delete *chan mailbox*

Delete a new mailbox.

chan - imap channel

mailbox - mailbox name

::imap4::rename *chan oldname newname*

Rename a new mailbox.

chan - imap channel

mailbox - old mailbox name

mailbox - new mailbox name

::imap4::subscribe *chan mailbox*

Subscribe a new mailbox.

chan - imap channel

mailbox - mailbox name

::imap4::unsubscribe *chan mailbox*

Unsubscribe a new mailbox.

chan - imap channel

mailbox - mailbox name

::imap4::search *chan expr* ?*...*?

Search for mails matching search criterions, 0 is returned on success.

chan - imap channel

expr - imap search expression

Notes: Currently the following search expressions are handled:

Mail header flags: all mail header entries (ending with a colon ":"), like "From:", "Bcc:", ...

Imap message search flags: ANSWERED, DELETED, DRAFT, FLAGGED, RECENT, SEEN, NEW, OLD, UNANSWERED, UNDELETED, UNDRAFT, UNFLAGGED, UNSEEN, ALL

Imap header search flags: BODY, CC, FROM, SUBJECT, TEXT, KEYWORD, BCC

Imap conditional search flags: SMALLER, LARGER, ON, SENTBEFORE, SENTON, SENTSINCE, SINCE, BEFORE (not implemented), UID (not implemented)

Logical search conditions: OR, NOT

```
::imap4::search $chan larger 4000 seen
puts "Found messages: [::imap4::mboxinfo $chan found]"
Found messages: 1 3 6 7 8 9 13 14 15 19 20
```

::imap4::close *chan*

Close the mailbox. Permanently removes \Deleted messages and return to the AUTH state.

chan - imap channel

::imap4::cleanup *chan*
Destroy an IMAP connection and free the used space. Close the mailbox. Permanently removes \Deleted messages and return to the AUTH state.

chan - imap channel

::imap4::debugmode *chan* ?*errormsg*?
Switch client into command line debug mode.

This is a developers mode only that pass the control to the programmer. Every line entered is sent verbatim to the server (after the addition of the request identifier). The ::imap4::debug variable is automatically set to '1' on enter.

It's possible to execute Tcl commands starting the line with a slash.

chan - imap channel

errormsg - optional error message to display

::imap4::store *chan range data flaglist*
Alters data associated with a message in the selected mailbox.

chan - imap channel

range - message index in format *FROM*:*TO*

flaglist - Flags the *data* operates on.

data - The currently defined *data* items that can be stored are shown below. *Note* that all of these data types may also be suffixed with ".SILENT" to suppress the untagged FETCH response.

FLAGS
Replace the flags for the message (other than \Recent) with the *flaglist*.

+FLAGS
Add the flags in *flaglist* to the existing flags for the message.

-FLAGS
Remove the flags in *flaglist* to the existing flags for the message.

For example:

```
    ::imap4::store $chan $start_msgid:$end_msgid +FLAGS "Deleted"
```

::imap4::expunge *chan*
Permanently removes all messages that have the \Deleted flag set from the currently selected mailbox, without the need to close the connection.

chan - imap channel

::imap4::copy *chan msgid mailbox*
Copies the specified message (identified by its message number) to the named mailbox, i.e. imap folder.

chan - imap channel

msgid - message number

mailbox - mailbox name

::imap4::logout *chan*

Informs the server that the client is done with the connection and closes the network connection. Permanently removes \Deleted messages.

A new connection will need to be established to login once more.

chan - imap channel

Examples

```
set user myusername
set pass xtremescrt
set server imap.test.tld
set FOLDER INBOX
# Connect to server
set imap [::imap4::open $server]
::imap4::login $imap $user $pass
::imap4::select $imap $FOLDER
# Output all the information about that mailbox
foreach info [::imap4::mboxinfo $imap] {
    puts "$info -> [::imap4::mboxinfo $imap $info]"
}
# fetch 3 records inline
set fields {from: to: subject: size}
foreach rec [::imap4::fetch $imap :3 -inline {*}$fields] {
    puts -nonewline "#[incr idx])"
    for {set j 0} {$j<[llength $fields]} {incr j} {
        puts "\t[lindex $fields $j] [lindex $rec $j]"
    }
}

# Show all the information available about the message ID 1
puts "Available info about message 1: [::imap4::msginfo $imap 1]"

# Use the capability stuff
puts "Capabilities: [::imap4::isableto $imap]"
puts "Is able to imap4rev1? [::imap4::isableto $imap imap4rev1]"

# Cleanup
::imap4::cleanup $imap
```

Tls security considerations

This package uses the **TLS** package to handle the security for **https** urls and other socket connections.

Policy decisions like the set of protocols to support and what ciphers to use are not the responsibility of **TLS**, nor of this package itself however. Such decisions are the responsibility of whichever application is using the package, and are likely influenced by the set of servers the application will talk to as well.

For example, in light of the recent POODLE attack discovered by Google many servers will disable support for the SSLv3 protocol. To handle this change the applications using **TLS** must be patched, and not this package, nor **TLS** itself. Such a patch may be as simple as generally activating **tls1** support, as shown in the example below.

```
    package require tls
    tls::init -tls1 1 ;# forcibly activate support for the TLS1
    protocol

    ... your own application code ...
```

References

Mark R. Crispin, "INTERNET MESSAGE ACCESS PROTOCOL - VERSION 4rev1", RFC 3501, March 2003, http://www.rfc-editor.org/rfc/rfc3501.txt

OpenSSL, http://www.openssl.org/

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *imap4* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar. Only a small part of rfc3501 implemented.

Category

Networking

inifile

ini

Keywords: ini

Name

inifile - Parsing of Windows INI files

Synopsis

package require Tcl 8.5 9

package require inifile ?**0.3.3**?
::ini::open *file* ?*-encoding encoding*? ?*access*?
::ini::close *ini*
::ini::commit *ini*
::ini::revert *ini*
::ini::filename *ini*
::ini::sections *ini*
::ini::keys *ini section*
::ini::get *ini section*
::ini::exists *ini section* ?*key*?
::ini::value *ini section key* ?*default*?
::ini::set *ini section key value*
::ini::delete *ini section* ?*key*?
::ini::comment *ini section* ?*key*? ?*text*?
::ini::commentchar ?*char*?

Description

This package provides an interface for easy manipulation of Windows INI files.

::ini::open *file* ?-encoding *encoding*? ?*access*?
Opens an INI file and returns a handle that is used by other commands. *access* is the same as the first form (non POSIX) of the **open** command, with the exception that mode **a** is not supported. The default mode is **r+**.

The default *encoding* is the system encoding.

::ini::close *ini*
Close the specified handle. If any changes were made and not written by **commit** they are lost.

::ini::commit *ini*
Writes the file and all changes to disk. The sections are written in arbitrary order. The keys in a section are written in alphabetical order. If the ini was opened in read only mode an error will be thrown.

::ini::revert *ini*
Rolls all changes made to the inifile object back to the last committed state.

::ini::filename *ini*
Returns the name of the file the *ini* object is associated with.

::ini::sections *ini*
Returns a list of all the names of the existing sections in the file handle specified.

::ini::keys *ini section*
Returns a list of all they key names in the section and file specified.

::ini::get *ini section*
Returns a list of key value pairs that exist in the section and file specified.

::ini::exists *ini section* ?*key*?
Returns a boolean value indicating the existance of the specified section as a whole or the specified key within that section.

::ini::value *ini section key* ?*default*?

Returns the value of the named key and section. If specified, the default value will be returned if the key does not exist. If the key does not exist and no default is specified an error will be thrown.

::ini::set *ini section key value*
Sets the value of the key in the specified section. If the section does not exist then a new one is created.

::ini::delete *ini section* ?*key*?
Removes the key or the entire section and all its keys. A section is not automatically deleted when it has no remaining keys.

::ini::comment *ini section* ?*key*? ?*text*?
Reads and modifies comments for sections and keys. To write a section comment use an empty string for the *key*. To remove all comments use an empty string for *text*. *text* may consist of a list of lines or one single line. Any embedded newlines in *text* are properly handled. Comments may be written to nonexistant sections or keys and will not return an error. Reading a comment from a nonexistant section or key will return an empty string.

::ini::commentchar ?char?
Reads and sets the comment character. Lines that begin with this character are treated as comments. When comments are written out each line is preceded by this character. The default is**;**.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *inifile* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Text processing

interp

deleg_method

Keywords: deleg_method, comm, delegation, interpreter, method, snit

Name

deleg_method - Creation of comm delegates (snit methods)

Synopsis

package require Tcl 8.5 9
package require snit ?**1.1**?
package require interp::delegate::method ?**0.3**?
::interp::delegate::method ?**-async**? *name arguments comm id*

Description

This package provides a single command for use within **snit** type definition (i.e. actually a **snit::macro**) for the convenient creation of methods which delegate the actual work to a remote location via a "channel" created by the package **comm**.

API

::interp::delegate::method ?-async? *name arguments comm id*

This commands creates a method which is named by *name*. All invokations of this method will delegate the actual work to the remote location identified by the comm channel *comm* and the endpoint *id*.

The name of the remote method invoked by the delegator is identical to the name of the method itself.

Normally the generated method marshalls the *arguments*, and returns the result from the remote method as its own result. If however the option **-async** was specified then the generated method will not wait for a result and return immediately.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *interp* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Programming tools

Copyright


```
Copyright (c) 2006 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


deleg_proc

Keywords: deleg_proc, comm, delegation, interpreter, procedure

Name

deleg_proc - Creation of comm delegates (procedures)

Synopsis

package require Tcl 8.5 9
package require interp::delegate::proc ?**0.3**?
::interp::delegate::proc ?**-async**? *name arguments comm id*

Description

This package provides a single command for the convenient creation of procedures which delegate the actual work to a remote location via a "channel" created by the package **comm**.

API

::interp::delegate::proc ?-async? *name arguments comm id*

This commands creates a procedure which is named by *name* and returns its fully-qualified name. All invokations of this procedure will delegate the actual work to the remote location identified by the comm channel *comm* and the endpoint *id*.

The name of the remote procedure invoked by the delegator is [namespace tail *name*]. I.e., namespace information is stripped from the call.

Normally the generated procedure marshalls the *arguments*, and returns the result from the remote procedure as its own result. If however the option **-async** was specified then the generated procedure will not wait for a result and return immediately.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *interp* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Programming tools

Copyright


```
Copyright (c) 2006 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


tcllib_interp

Keywords: tcllib_interp, alias, empty interpreter, interpreter, method, snit

Name

interp - Interp creation and aliasing

Synopsis

package require Tcl 8.5 9
package require interp ?**0.1.3**?
::interp::createEmpty ?*path*?
::interp::snitLink *path methodlist*
::interp::snitDictLink *path methoddict*

Description

This package provides a number of commands for the convenient creation of Tcl interpreters for highly restricted execution.

API

::interp::createEmpty ?*path*?

This commands creates an empty Tcl interpreter and returns it name. Empty means that the new interpreter has neither namespaces, nor any commands. It is useful only for the creation of aliases.

If a *path* is specified then it is taken as the name of the new interpreter.

::interp::snitLink *path methodlist*

This command assumes that it was called from within a method of a snit object, and that the command **mymethod** is available.

It extends the interpreter specified by *path* with aliases for all methods found in the *methodlist*, with the alias directing execution to the same-named method of the snit object invoking this command. Each element of *methodlist* is actually interpreted as a command prefix, with the first word of each prefix the name of the method to link to.

The result of the command is the empty string.

::interp::snitDictLink *path methoddict*

This command behaves like **::interp::snitLink**, except that it takes a dictionary mapping from commands to methods as its input, and not a list of methods. Like for **::interp::snitLink** the method references are actually command prefixes. This command allows the creation of more complex command-method mappings than **::interp::snitLink**.

The result of the command is the empty string.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *interp* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Programming tools

Copyright


```
Copyright (c) 2006 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

irc

irc

Keywords: irc, chat

Name

irc - Create IRC connection and interface.

Synopsis

package require Tcl 8.6 9
package require irc ?**0.8.0**?
::irc::config ?*key*? ?*value*?
::irc::connection
::irc::connections
net **registerevent** *event script*
net **getevent** *event script*
net **eventexists** *event script*
net **connect** *hostname* ?*port*?
net **config** ?*key*? ?*value*?
net **log** *level message*
net **logname**
net **connected**
net **sockname**
net **peername**
net **socket**
net **user** *username localhostname localdomainname userinfo*
net **nick** *nick*
net **ping** *target*
net **serverping**
net **join** *channel* ?*key*?
net **part** *channel* ?*message*?
net **quit** ?*message*?
net **privmsg** *target message*
net **notice** *target message*
net **ctcp** *target message*
net **kick** *channel target* ?*message*?
net **mode** *target args*
net **topic** *channel message*
net **invite** *channel target*
net **send** *text*
net **destroy**
who ?**address**?
action
target
additional
header

msg

Description

This package provides low-level commands to deal with the IRC protocol (Internet Relay Chat) for immediate and interactive multi-cast communication.

::irc::config ?key? ?value?
: Sets configuration ?key? to ?value?. The configuration keys currently defined are the boolean flags **logger** and **debug**. **logger** makes **irc** use the logger package for printing error. **debug** requires **logger** and prints extra debug output. If no ?key? or ?value? is given the current values are returned.

::irc::connection
: The command creates a new object to deal with an IRC connection. Creating this IRC object does not automatically create the network connection. It returns a new irc namespace command which can be used to interact with the new IRC connection. NOTE: the old form of the connection command, which took a hostname and port as arguments, is deprecated. Use **connect** instead to specify this information.

::irc::connections
: Returns a list of all the current connections that were created with **connection**

Per-connection commands

In the following list of available connection methods *net* represents a connection command as returned by **::irc::connection**.

net* registerevent *event script
: Registers a callback handler for the specific event. Events available are those described in RFC 1459 http://www.rfc-editor.org/rfc/rfc1459.txt. In addition, there are several other events defined. **defaultcmd** adds a command that is called if no other callback is present. **EOF** is called if the connection signals an End of File condition. The events **defaultcmd**, **defaultnumeric**, **defaultevent**, and **EOF** are required. *script* is executed in the connection namespace, which can take advantage of several commands (see Callback commands below) to aid in the parsing of data.

net* getevent *event script
: Returns the current handler for the event if one exists. Otherwise an empty string is returned.

net* eventexists *event script
: Returns a boolean value indicating the existence of the event handler.

***net* connect *hostname* ?port?**
: This causes the socket to be established. **::irc::connection** created the namespace and the commands to be used, but did not actually open the socket. This is done here. NOTE: the older form of 'connect' did not require the user to specify a hostname and port, which were specified with 'connection'. That form is deprecated.

***net* config ?key? ?value?**
: The same as **::irc::config** but sets and gets options for the *net* connection only.

net* log *level message
: If logger is turned on by **config** this will write a log *message* at *level*.

***net* logname**
: Returns the name of the logger instance if logger is turned on.

***net* connected**
Returns a boolean value indicating if this connection is connected to a server.

***net* sockname**
Returns a 3 element list consisting of the ip address, the hostname, and the port of the local end of the connection, if currently connected.

***net* peername**
Returns a 3 element list consisting of the ip address, the hostname, and the port of the remote end of the connection, if currently connected.

***net* socket**
Return the Tcl channel for the socket used by the connection.

net* user *username localhostname localdomainname userinfo
Sends USER command to server. *username* is the username you want to appear. *localhostname* is the host portion of your hostname, *localdomainname* is your domain name, and *userinfo* is a short description of who you are. The 2nd and 3rd arguments are normally ignored by the IRC server.

net* nick *nick
NICK command. *nick* is the nickname you wish to use for the particular connection.

net* ping *target
Send a CTCP PING to *target*.

***net* serverping**
PING the server.

***net* join *channel* ?*key*?**
channel is the IRC channel to join. IRC channels typically begin with a hashmark ("#") or ampersand ("&").

***net* part *channel* ?*message*?**
Makes the client leave *channel*. Some networks may support the optional argument *message*

***net* quit ?*message*?**
Instructs the IRC server to close the current connection. The package will use a generic default if no *message* was specified.

net* privmsg *target message
Sends *message* to *target*, which can be either a channel, or another user, in which case their nick is used.

net* notice *target message
Sends a **notice** with message *message* to *target*, which can be either a channel, or another user, in which case their nick is used.

net* ctcp *target message
Sends a CTCP of type *message* to *target*

***net* kick *channel target* ?*message*?**
Kicks the user *target* from the channel *channel* with a *message*. The latter can be left out.

net* mode *target args
Sets the mode *args* on the target *target*. *target* may be a channel, a channel user, or yourself.

net* topic *channel message
Sets the topic on *channel* to *message* specifying an empty string will remove the topic.

net* invite *channel target

Invites *target* to join the channel *channel*

net* send *text
Sends *text* to the IRC server.

***net* destroy**
Deletes the connection and its associated namespace and information.

Callback commands

These commands can be used within callbacks

who ?address?
Returns the nick of the user who performed a command. The optional keyword **address** causes the command to return the user in the format "username@address".

action
Returns the action performed, such as KICK, PRIVMSG, MODE, etc... Normally not useful, as callbacks are bound to a particular event.

target
Returns the target of a particular command, such as the channel or user to whom a PRIVMSG is sent.

additional
Returns a list of any additional arguments after the target.

header
Returns the entire event header (everything up to the:) as a proper list.

msg
Returns the message portion of the command (the part after the:).

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *irc* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Networking

picoirc

Keywords: picoirc, chat, irc

Name

picoirc - Small and simple embeddable IRC client.

Synopsis

package require Tcl 8.6 9
package require picoirc ?**0.14.1**?

::picoirc::connect *callback nick ?password? url*
::picoirc::post *context channel message*

Description

This package provides a general purpose minimal IRC client suitable for embedding in other applications. All communication with the parent application is done via an application provided callback procedure. Each connection has its own state so you can hook up multiple servers in a single application instance.

To initiate an IRC connection you must call **picoirc::connect** with a callback procedure, a nick-name to use on IRC and the IRC URL that describes the connection. This will return a variable name that is the irc connection context. See Callback for details.

This package is a fairly simple IRC client. If you need something with more capability investigate the **irc** package.

Commands

::picoirc::connect *callback nick ?password? url*

Creates a new irc connection to the server specified by *url* and login using the *nick* as the username and optionally *password*. If the *url* starts with *ircs://* then a TLS connection is created. The *callback* must be as specified in Callback. Returns a package-specific variable that is used when calling other commands in this package.

Note: For connecting via TLS the Tcl module *tls* must be already loaded, otherwise an error is raised.

```
# must be loaded for TLS
package require tls
# default arguments
tls::init -autoservername true -command workaround \
    -require 1 -cadir /etc/ssl/certs -tls1 0 -tls1.1 0
# avoid annoying bgerror, errors are already catched internally
proc workaround {state args} {
    if {$state == "verify"} {
        return [lindex $args 3]
    }
}
```

::picoirc::post *context channel message*

This should be called to process user input and send it to the server. If *message* is multiline then each line will be processed and sent individually. A number of commands are recognised when prefixed with a forward-slash (/). Such commands are converted to IRC command sequences and then sent. If *channel* is empty then all raw output to the server is handled. The default action is to write the *message* to the irc socket. However, before this happens the callback is called with "debug write". This permits the application author to inspect the raw IRC data and if desired to return a break error code to halt further processing. In this way the application can override the default send via the callback procedure.

Callback

The callback must look like:

```
proc Callback {context state args} {
}
```

where context is the irc context variable name (in case you need to pass it back to a picoirc procedure). state is one of a number of states as described below.

init
: called just before the socket is created

connect
: called once we have connected, before we join any channels

close
: called when the socket gets closed, before the context is deleted. If an error occurs before we get connected the only argument will be the socket error message.

userlist *channel nicklist*
: called to notify the application of an updated userlist. This is generated when the output of the NAMES irc command is seen. The package collects the entire output which can span a number of output lines from the server and calls this callback when they have all been received.

userinfo *nick info*
: called as a response of WHOIS command. *nick* is the user the command was targeted for. *info* is the dictionary containing detailed information about that user: name, host, channels and userinfo. userinfo typically contains name and version of user's IRC client.

chat *target nick message type*
: called when a message arrives. *target* is the identity that the message was targetted for. This can be the logged in nick or a channel name. *nick* is the name of the sender of the message. *message* is the message text. *type* is set to "ACTION" if the message was sent as a CTCP ACTION. *type* is set to "NOTICE" if the message was sent as a NOTICE command, in that case *target* is empty if it matches current user nick or it's "*", in later case empty *target* means that notice comes from server.

mode *nick target flags*
: called when mode of user or channel changes. *nick* is the name of the user who requested a change, can be empty if it's the server. *target* is the identity that has its mode changed. *flags* are the changes in mode.

system *channel message*
: called when a system message is received

topic *channel topic*
: called when the channel topic string is seen. *topic* is the text of the channel topic.

traffic *action channel nick ?newnick?*
: called when users join, leave or change names. *action* is either entered, left or nickchange and *nick* is the user doing the action. *newnick* is the new name if *action* is nickchange.

  NOTE: *channel* is often empty for these messages as nick activities are global for the irc server. You will have to manage the nick for all connected channels yourself.

version
: This is called to request a version string to use to override the internal version. If

implemented, you should return as colon delimited string as

Appname:Appversion:LibraryVersion

For example, the default is

PicoIRC:[package provide picoirc]:Tcl [info patchlevel]

debug *type raw*
called when data is either being read or written to the network socket. *type* is set to **read** when reading data and **write** if the data is to be written. *raw* is the unprocessed IRC protocol data.

In both cases the application can return a break error code to interrupt further processing of the raw data. If this is a **read** operation then the package will not handle this line. If the operation is **write** then the package will not send the data. This callback is intended for debugging protocol issues but could be used to redirect all input and output if desired.

Category
Networking

javascript

javascript
Keywords: javascript, checkbox, html, selectionbox, submitbutton

Name
javascript - Procedures to generate HTML and Java Script structures.

Synopsis
package require Tcl 8.5 9
package require javascript ?**1.0.3**?
::javascript::makeSelectorWidget *id leftLabel leftValueList rightLabel rightValueList rightNameList* ?*length*? ?*minWidth*?
::javascript::makeSubmitButton *name value*
::javascript::makeProtectedSubmitButton *name value msg*
::javascript::makeMasterButton *master value slavePattern boolean*
::javascript::makeParentCheckbox *parentName childName*
::javascript::makeChildCheckbox *parentName childName*

Description
The **::javascript** package provides commands that generate HTML and Java Script code. These commands typically return an HTML string as their result. In particular, they do not output their result to **stdout**.

::javascript::makeSelectorWidget *id leftLabel leftValueList rightLabel rightValueList rightNameList* ?*length*? ?*minWidth*?
Construct HTML code to create a dual-multi-selection megawidget. This megawidget consists

of two side-by-side multi-selection boxes separated by a left arrow and a right arrow button. The right arrow button moves all items selected in the left box to the right box. The left arrow button moves all items selected in the right box to the left box. The *id* argument is the suffix of all HTML objects in this megawidget. The *leftLabel* argument is the text that appears above the left selection box. The *leftValueList* argument is the values of items in the left selection box. The *leftNameList* argument is the names to appear in the left selection box. The *rightLabel* argument is the text that appears above the right selection box. The *rightValueList* argument is the values of items in the right selection box. The *rightNameList* argument is the names to appear in the right selection box. The *length* argument (optional) determines the number of elts to show before adding a vertical scrollbar; it defaults to 8. The *minWidth* argument (optional) is the number of spaces to determine the minimum box width; it defaults to 32.

::javascript::makeSubmitButton *name value*
Create an HTML submit button that resets a hidden field for each registered multi-selection box. The *name* argument is the name of the HTML button object to create. The *value* argument is the label of the HTML button object to create.

::javascript::makeProtectedSubmitButton *name value msg*
Create an HTML submit button that prompts the user with a continue/cancel shutdown warning before the form is submitted. The *name* argument is the name of the HTML button object to create. The *value* argument is the label of the HTML button object to create. The *msg* argument is the message to display when the button is pressed.

::javascript::makeMasterButton *master value slavePattern boolean*
Create an HTML button that sets its slave checkboxs to the boolean value. The *master* argument is the name of the child's parent html checkbox object. The *value* argument is the value of the master. The *slaves* argument is the name of child html checkbox object to create. The *boolean* argument is the java script boolean value that will be given to all the slaves; it must be "true" or "false".

::javascript::makeParentCheckbox *parentName childName*
Create an HTML checkbox and tie its value to that of its child checkbox. If the parent is unchecked, the child is automatically unchecked. The *parentName* argument is the name of parent html checkbox object to create. The *childName* argument is the name of the parent's child html checkbox object.

::javascript::makeChildCheckbox *parentName childName*
Create an HTML checkbox and tie its value to that of its parent checkbox. If the child is checked, the parent is automatically checked. The *parentName* argument is the name of the child's parent html checkbox object. The *childName* argument is the name of child html checkbox object to create.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *javascript* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the

left-most button in the secondary navigation bar.

Category
CGI programming

jpeg

jpeg
Keywords: jpeg, comment, exif, image, jfif, thumbnail

Name
jpeg - JPEG querying and manipulation of meta data

Synopsis
package require Tcl 8.5 9
package require jpeg ?**0.7**?
::jpeg::isJPEG *file*
::jpeg::imageInfo *file*
::jpeg::dimensions *file*
::jpeg::getThumbnail *file*
::jpeg::getExif *file* ?*section*?
::jpeg::getExifFromChannel *channel* ?*section*?
::jpeg::formatExif *keys*
::jpeg::exifKeys
::jpeg::removeExif *file*
::jpeg::stripJPEG *file*
::jpeg::getComments *file*
::jpeg::addComment *file text...*
::jpeg::removeComments *file*
::jpeg::replaceComment *file text*
::jpeg::debug *file*
::jpeg::markers *channel*

Description
This package provides commands to query and modify JPEG images. JPEG stands for *Joint Photography Experts Group* and is a standard for the lossy compression of photographical images. It is specified at *LINK_HERE*.

Commands
::jpeg::isJPEG *file*
Returns a boolean value indicating if *file* is a JPEG image.

::jpeg::imageInfo *file*
Returns a dictionary with keys **version**, **units**, **xdensity**, **ydensity**, **xthumb**, and **ythumb**. The values are the associated properties of the JPEG image in *file*. Throws an error if *file* is not a JPEG image.

::jpeg::dimensions *file*

Returns the dimensions of the JPEG *file* as a list of the horizontal and vertical pixel count. Throws an error if *file* is not a JPEG image.

::jpeg::getThumbnail *file*

This procedure will return the binary thumbnail image data, if a JPEG thumbnail is included in *file*, and the empty string otherwise. Note that it is possible to include thumbnails in formats other than JPEG although that is not common. The command finds thumbnails that are encoded in either the JFXX or EXIF segments of the JPEG information. If both are present the EXIF thumbnail will take precedence. Throws an error if *file* is not a JPEG image.

```
    set fh [open thumbnail.jpg w+]
    fconfigure $fh -translation binary -encoding binary
    puts -nonewline $fh [::jpeg::getThumbnail photo.jpg]
    close $fh
```

::jpeg::getExif *file* ?*section*?

section must be one of **main** or **thumbnail**. The default is **main**. Returns a dictionary containing the EXIF information for the specified section. For example:

```
    set exif {
    Make     Canon
    Model    {Canon DIGITAL IXUS}
    DateTime {2001:06:09 15:17:32}
    }
```

Throws an error if *file* is not a JPEG image.

::jpeg::getExifFromChannel *channel* ?*section*?

This command is as per **::jpeg::getExif** except that it uses a previously opened channel. *channel* should be a seekable channel and *section* is as described in the documentation of **::jpeg::getExif**.

Note: The jpeg parser expects that the start of the channel is the start of the image data. If working with an image embedded in a container file format it may be necessary to read the jpeg data into a temporary container: either a temporary file or a memory channel.

Attention: It is the resonsibility of the caller to close the channel after its use.

::jpeg::formatExif *keys*

Takes a list of key-value pairs as returned by **getExif** and formats many of the values into a more human readable form. As few as one key-value may be passed in, the entire exif is not required.

```
    foreach {key val} [::jpeg::formatExif [::jpeg::getExif
    photo.jpg]] {
        puts "$key: $val"
    }
```

```
    array set exif [::jpeg::getExif photo.jpg]
    puts "max f-stop: [::jpeg::formatExif [list MaxAperture
    $exif(MaxAperture)]]
```

::jpeg::exifKeys
Returns a list of the EXIF keys which are currently understood. There may be keys present in **getExif** data that are not understood. Those keys will appear in a 4 digit hexadecimal format.

::jpeg::removeExif *file*
Removes the Exif data segment from the specified file and replaces it with a standard JFIF segment. Throws an error if *file* is not a JPEG image.

::jpeg::stripJPEG *file*
Removes all metadata from the JPEG file leaving only the image. This includes comments, EXIF segments, JFXX segments, and application specific segments. Throws an error if *file* is not a JPEG image.

::jpeg::getComments *file*
Returns a list containing all the JPEG comments found in the *file*. Throws an error if *file* is not a valid JPEG image.

::jpeg::addComment *file text...*
Adds one or more plain *text* comments to the JPEG image in *file*. Throws an error if *file* is not a valid JPEG image.

::jpeg::removeComments *file*
Removes all comments from the file specified. Throws an error if *file* is not a valid JPEG image.

::jpeg::replaceComment *file text*
Replaces the first comment in the file with the new *text*. This is merely a shortcut for **::jpeg::removeComments** and **::jpeg::addComment** Throws an error if *file* is not a valid JPEG image.

::jpeg::debug *file*
Prints everything we know about the given file in a nice format.

::jpeg::markers *channel*
This is an internal helper command, we document it for use by advanced users of the package. The argument *channel* is an open file handle positioned at the start of the first marker (usually 2 bytes). The command returns a list with one element for each JFIF marker found in the file. Each element consists of a list of the marker name, its offset in the file, and its length. The offset points to the beginning of the sections data, not the marker itself. The length is the length of the data from the offset listed to the start of the next marker.

Limitations

can only work with files cant write exif data gps exif data not parsed makernote data not yet implemented

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *jpeg* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
File formats

Copyright

```
Copyright (c) 2004-2005, Code: Aaron Faupell
    <afaupell@users.sourceforge.net>
Copyright (c) 2007, Code:  Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
Copyright (c) 2004-2009, Doc:  Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
Copyright (c) 2011, Code: Pat Thoyts <patthoyts@users.sourceforge.net>
```


json

json
Keywords: json, data exchange, exchange format, javascript

Name
json - JSON parser

Synopsis
package require Tcl 8.5 9
package require json ?**1.3.6**?
::json::json2dict *json_txt*
::json::many-json2dict *json_txt* ?*max*?

Description
The **json** package provides a simple Tcl-only library for parsing the JSON data exchange format as specified in RFC 4627. There is some ambiguity in parsing JSON because JSON has type information that is not maintained by this module's conversion. Furthermore, JSON **null** values are converted to the string literal "null" (see ticket). The **json** package returns data either as a Tcl **dict** or a **list** which itself may contain nested **dict**s or **list**s. (Either the **dict** package or Tcl 8.5 or later is required for use.)

For an alternative Tcl-only type-preserving implementation, see the typed-json module.

Commands
::json::json2dict *json_txt*
Returns a **dict** or **list** (which itself may contain nested **dict**s and/or **list**s) which is the result of parsing the JSON formatted text passed in *json_txt*.

If *json_txt* contains more than one JSON entity only the first one is returned.

::json::many-json2dict *json_txt* ?*max*?

Returns a **list** (which itself will contain nested **dict**s and/or **list**s) which is the result of parsing all the JSON entities (or if *max* is specified, the first *max* entities), in the JSON formatted text passed in *json_txt*.

If *max* is specified it must be a positive integer **> 0** or the command will throw and error.

If the JSON formatted text contains nested lists and dicts then so will the returned dict. In such cases accessing a data value may require a *mixture* of using dict get and lindex. (See the second example.)

Examples

An example of a JSON array converted to Tcl. A JSON array is returned as a single item with multiple elements.

```
[
    {
       "precision": "zip",
       "Latitude":  37.7668,
       "Longitude": -122.3959,
       "Address":   "",
       "City":      "SAN FRANCISCO",
       "State":     "CA",
       "Zip":       "94107",
       "Country":   "US"
    },
    {
       "precision": "zip",
       "Latitude":  37.371991,
       "Longitude": -122.026020,
       "Address":   "",
       "City":      "SUNNYVALE",
       "State":     "CA",
       "Zip":       "94085",
       "Country":   "US"
    }
]
=>
{Country US Latitude 37.7668 precision zip State CA City {SAN
    FRANCISCO} Address {} Zip 94107 Longitude -122.3959} {Country US
    Latitude 37.371991 precision zip State CA City SUNNYVALE Address
    {} Zip 94085 Longitude -122.026020}
```

An example of a JSON dict which contains an entry whose value is a list of dicts.

```
{
  "items": [
     {
      "snippet": {
```

```
        "title": "In The Shade",
        "channelTitle": "Flox - Topic"
      },
      "statistics": {
        "viewCount": "274"
      }
    }
  ]
}
=>
items {{snippet {title {In The Shade} channelTitle {Flox - Topic}}
    statistics {viewCount 274}}}
```

Assuming that the JSON formatted text is in variable **jdata**, it can be converted to a **dict** and elements extracted.

```
set d [json::json2dict $jdata]
set title [dict get [lindex [dict get $d items] 0] snippet title]
=>
In The Shade
```

An example of a JSON object converted to Tcl. A JSON object is returned as a multi-element list (a dict).

```
{
    "Image": {
        "Width":  800,
        "Height": 600,
        "Title":  "View from 15th Floor",
        "Thumbnail": {
            "Url":    "http://www.example.com/image/481989943",
            "Height": 125,
            "Width":  "100"
        },
        "IDs": [116, 943, 234, 38793]
    }
}
=>
Image {IDs {116 943 234 38793} Thumbnail {Width 100 Height 125 Url
    http://www.example.com/image/481989943} Width 800 Height 600 Title
    {View from 15th Floor}}
```

Related

To write json, instead of parsing it, see package **json::write**.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib

Trackers and specify *json* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

CGI programming

Copyright

```
Copyright (c) 2006 ActiveState Software Inc.
Copyright (c) 2009 Thomas Maeder, Glue Software Engineering AG
```

json_write

Keywords: json_write, data exchange, exchange format, javascript, json

Name

json::write - JSON generation

Synopsis

package require Tcl 8.5 9
package require json::write ?**1.0.5**?
::json::write indented
::json::write indented *flag*
::json::write aligned
::json::write aligned *flag*
::json::write string *s*
::json::write array *arg...*
::json::write array-strings *arg...*
::json::write object *key value...*
::json::write object-strings *key value...*

Description

The **json::write** package provides a simple Tcl-only library for generation of text in the JSON http://www.json.org/ data exchange format as specified in RFC 4627 http://www.ietf.org/rfc/rfc4627.txt.

Commands

::json::write indented

This method returns the current state of the indentation setting.

::json::write indented *flag*

This and the method **aligned** configure the layout of the JSON generated by the package.

If this *flag* is set (default) the package will break the generated JSON code across lines and indent it according to its inner structure, with each key of an object on a separate line.

If this flag is not set, the whole JSON object will be written on a single line, with minimum spacing between all elements.

::json::write aligned
This method returns the current state of the alignment setting.

::json::write aligned *flag*
This and the method **indented** configure the layout of the JSON generated by the package.

If this *flag* is set (default) the package ensures that the values for the keys in an object are vertically aligned with each other, for a nice table effect. To make this work this also implies that **indented** is set as well.

If this flag is not set, the output is formatted as per the value of **indented**, without trying to align the values for object keys.

::json::write string *s*
This method takes the string *s* and returns it properly quoted for JSON as its result.

::json::write array *arg*...
This method takes a series of JSON formatted arguments and returns them as a properly formatted JSON array as its result.

::json::write array-strings *arg*...
This is a convenience variant of **array**. It assumes that all arguments are plain strings and formats them as JSON strings before passing them into the **array**.

::json::write object *key value*...
This method takes a series of key/value arguments, the values already formatted for JSON, and returns them as a properly formatted JSON object as its result, with the keys formatted as JSON strings.

::json::write object-strings *key value*...
This is a convenience variant of **object**. It assumes that all *value* arguments are plain strings and formats them as JSON strings before passing them into the **object**.

Related

To parse json, instead of writing it, see package **json**.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *json* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

CGI programming

Copyright

Copyright (c) 2009-2013,2022 Andreas Kupries
    <andreas_kupries@sourceforge.net>

lambda

lambda

Keywords: lambda, anonymous procedure, callback, command prefix, currying, partial application, proc

Name

lambda - Utility commands for anonymous procedures

Synopsis

package require Tcl 8.5 9
package require lambda ?**1.1**?
::lambda *arguments body* ?*arg* ...?
::lambda@ *namespace arguments body* ?*arg* ...?

Description

This package provides two convenience commands to make the creation of anonymous commands, i.e., lambdas, more **proc**-like. Instead of writing, for example

```
     set f {::apply {{x} {
        ....
     }}}
```

with its deep nesting of braces, or

```
     set f [list ::apply {{x y} {
        ....
     }} $value_for_x]
```

with a **list** command to insert some of the arguments of a partial application, simply write

```
     set f [lambda {x} {
        ....
     }]
```

or

```
     set f [lambda {x y} {
        ....
     } $value_for_x]
```

Lambdas are particularly useful for commands that accept a **-command** option.

API

::lambda *arguments body* ?*arg ...*?

Returns an anonymous command based on the *arguments* list, the *body* script, and any (optional) predefined argument values, *arg* ....

When invoked, the *body* is run in a new procedure scope just below the global scope, with the arguments set to the values supplied at both construction (any *arg ...* arguments), and invocation time.

::lambda@ *namespace arguments body ?arg ...?*

Returns an anonymous command in namespace *namespace*, and based on the *arguments* list, the *body* script, and any (optional) predefined argument values, *arg ...*.

When invoked, the *body* is run in a new procedure scope in the *namespace* namespace, with the arguments set to the values supplied at both construction (any *arg ...* arguments), and invocation time.

Examples

Partial function application

Lambdas can be used for partial function application (PFA), although Tcl's **expr** contexts mean that this isn't often needed. Nonetheless, here is a classic PFA example:

```
proc make_adder addend { lambda {x y} { expr { $x + $y } } $addend }
set add5 [make_adder 5]
puts "[{*}$add5 92] [{*}$add5 17]"
=>
97 22
```

The **make_adder** command has been used to create an anonymous command, i.e., a lambda, which has then been assigned to the **add5** variable. To call a variable as a command, it is necessary to prefix it with the **{*}** expansion ("splat") operator. Notice that these calls need only *one* argument; the first argument was set when the lambda was created.

Using regusb with lambda

This example shows how to use a lambda with the built-in regsub command's **-command** option.

```
# Simple URL-decoding: + → space; %XX → char
set encoded_url "www.tcl-lang.org%2FSet%20or%20%24%20usage%3F"
set url [regsub -all -command {(\+)|%([0-9A-Fa-f]{2})}  $encoded_url
    [lambda {_ plus hex} {
                if {$plus eq "+"} { return " " }
                scan $hex %x charCode
                format %c $charCode
            }]]
puts $url
=>
www.tcl-lang.org/Set or $ usage?
```

Notice that when using the built-in regsub command's **-command** option, the command itself *follows* the string to match (in the *subSpec* argument's position).

Using lsort with lambda

This example shows how to use lambdas to sort objects using the built-in lsort command's **-command** option.

First, a minimal definition of an **Id** class whose objects are to be sorted.

```
oo::class create Id {
    variable Id
    variable Name
    constructor {id name} { set Id $id ; set Name $name }
    method id {} { return $Id }
    method name {} { return $Name }
    method to_string {} { return "$Id'$Name'" }
}
```

Next, a list of **Id**s is created and then sorted in two different ways using lambdas to get the required sort orders.

```
set ids [list [Id new 3 Jane] [Id new 1 Bill] [Id new 4 Sam] [Id new 5
    Nell]]
#
puts -nonewline "Original order:     "
foreach id $ids { puts -nonewline "[$id to_string] " }
#
puts -nonewline "\nName,Id order:      "
foreach id [lsort -command [lambda {a b} {
        if {[set i [string compare [$a name] [$b name]]]} { return $i }
        if {[$a id] < [$b id]} { return -1 }
        if {[$a id] > [$b id]} { return 1 }
        return 0}
            ] $ids] {
    puts -nonewline "[$id to_string] "
}
#
puts -nonewline "\nId desc,Name order: "
foreach id [lsort -command [lambda {a b} {
        if {[$a id] < [$b id]} { return 1 }
        if {[$a id] > [$b id]} { return -1 }
        return [string compare [$a name] [$b name]]}
            ] $ids] {
    puts -nonewline "[$id to_string] "
}
puts ""
=>
Original order:     3'Jane' 1'Bill' 4'Sam' 5'Nell'
Name,Id order:      1'Bill' 3'Jane' 5'Nell' 4'Sam'
Id desc,Name order: 5'Nell' 4'Sam' 3'Jane' 1'Bill'
```

Notice that when using the built-in lsort command's **-command** option, the command itself *precedes* the list to sort.

Authors
Andreas Kupries

Bugs, ideas, feedback
If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *lambda* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Utility

Copyright

```
Copyright (c) 2011 Andreas Kupries, BSD licensed
```


lazyset

lazyset
Keywords: lazyset

Name
lazyset - Lazy evaluation

Synopsis
package require Tcl 8.5 9
package require lazyset ?**1.1**?
::lazyset::variable ?*-array boolean*? ?*-appendArgs boolean*? *someVar commandPrefix*

Description
The **lazyset** package provides a mechanism for deferring code execution until a specific variable or any index of an array is referenced.

(Another way to defer execution is to create a singleton class. The first time its constructor is called—if ever—the object will be created. Any subsequent constructor call will get back the first created object. See oo::singleton.)

API
::lazyset::variable ?*-array boolean*? ?*-appendArgs boolean*? *someVar commandPrefix*

Arrange for the code specified as *commandPrefix* to be executed when the variable whose name is specified by *someVar* is read for the first time.

If the optional argument *-array boolean* is specified as true, then the variable specified as *someVar* is treated as an array and attempting to read any index of the array causes that

index to be set by the *commandPrefix* as they are read.

If the optional argument *-appendArgs boolean* is specified as false, then the variable name and subnames are not appended to the *commandPrefix* before it is evaluated.

If the argument *-appendArgs boolean* is not specified or is specified as true, then 1 or 2 additional arguments are appended to the *commandPrefix*. If *-array boolean* is specified as true, then 2 arguments are appended corresponding to the name of the variable and the index; otherwise 1 argument is appended containing the name of the variable.

The *commandPrefix* code is run in the same scope as the variable that is read.

Examples

```
::lazyset::variable page {apply {{name} {
    package require http
    set token [http::geturl http://www.tcl-lang.org/]
    set data [http::data $token]
    return $data
}}}

puts $page
```

```
::lazyset::variable -array true page {apply {{name index} {
    package require http
    set token [http::geturl $index]
    set data [http::data $token]
    return $data
}}}

puts $page(http://www.tcl-lang.org/)
```

```
::lazyset::variable -appendArgs false simple {
    return -level 0 42
}

puts $simple
```

Authors

Roy Keene

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *utility* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be

made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Utility

Copyright

```
Copyright (c) 2018 Roy Keene
```


ldap

ldap
Keywords: ldap, directory access, internet, ldap client, protocol, rfc 2251, rfc 4511, x.500

Name
ldap - LDAP client

Synopsis
package require Tcl 8.5 9
package require ldap ?**1.10.2**?
::ldap::connect *host* ?*port*?
::ldap::tlsoptions reset
::ldap::tlsoptions ?*opt1 val1*? ?*opt2 val2*? ...
::ldap::secure_connect *host* ?*port*?
::ldap::secure_connect *host* ?*port*? ?*verify_cert*? ?*sni_servername*?
::ldap::disconnect *handle*
::ldap::starttls *handle*
::ldap::starttls *handle* ?*cafile*? ?*certfile*? ?*keyfile*? ?*verify_cert*? ?*sni_servername*?
::ldap::bind *handle* ?*name*? ?*password*?
::ldap::bindSASL *handle* ?*name*? ?*password*?
::ldap::unbind *handle*
::ldap::search *handle baseObject filterString attributes options*
::ldap::searchInit *handle baseObject filterString attributes options*
::ldap::searchNext *handle*
::ldap::searchEnd *handle*
::ldap::modify *handle dn attrValToReplace* ?*attrToDelete*? ?*attrValToAdd*?
::ldap::modifyMulti *handle dn attrValToReplace* ?*attrValToDelete*? ?*attrValToAdd*?
::ldap::add *handle dn attrValueTuples*
::ldap::addMulti *handle dn attrValueTuples*
::ldap::delete *handle dn*
::ldap::modifyDN *handle dn newrdn* ?*deleteOld*? ?*newSuperior*?
::ldap::info ip *handle*
::ldap::info bound *handle*
::ldap::info bounduser *handle*
::ldap::info connections
::ldap::info tls *handle*

::ldap::info tlsstatus *handle*
::ldap::info saslmechanisms *handle*
::ldap::info control *handle*
::ldap::info extensions *extensions*
::ldap::info whoami *handle*

Description

The **ldap** package provides a Tcl-only client library for the LDAPv3 protocol as specified in RFC 4511 (http://www.rfc-editor.org/rfc/rfc4511.txt). It works by opening the standard (or secure) LDAP socket on the server, and then providing a Tcl API to access the LDAP protocol commands. All server errors are returned as Tcl errors (thrown) which must be caught with the Tcl **catch** command.

TLS security considerations

This package uses the **TLS** package to handle the security for **LDAPS** connections.

Policy decisions like the set of protocols to support and what ciphers to use are not the responsibility of **TLS**, nor of this package itself however. Such decisions are the responsibility of whichever application is using the package, and are likely influenced by the set of servers the application will talk to as well.

For example, in light of the recent POODLE attack discovered by Google many servers will disable support for the SSLv3 protocol. To handle this change the applications using **TLS** must be patched, and not this package, nor **TLS** itself. Such a patch may be as simple as generally activating **tls1** support, as shown in the example below.

```
    ldap::tlsoptions -tls1 1 -ssl2 0 -ssl3 0 ;# forcibly activate
    support for the TLS1 protocol

    ... your own application code ...
```

Commands

::ldap::connect ***host*** **?*port*?**

Opens a LDAPv3 connection to the specified *host*, at the given *port*, and returns a token for the connection. This token is the *handle* argument for all other commands. If no *port* is specified it will default to **389**.

The command blocks until the connection has been established, or establishment definitely failed.

::ldap::tlsoptions reset

This command resets TLS options to default values. It returns the set of options. Using this command is incompatible with the obsolete form of **::ldap::secure_connect** and **::ldap_starttls**.

::ldap::tlsoptions ?*opt1 val1*? ?*opt2 val2*? ...

This commands adds one or more options to some value, and may be used more than one time in order to add options in several steps. A complete description of options may be found in the **tls** package documentation. Valid options and values are:
-cadir directory

Provide the directory containing the CA certificates. No default.

-cafile file
Provide the CA file. No default.

-cipher string
Provide the cipher suites to use. No default.

-dhparams file
Provide a Diffie-Hellman parameters file. No default.

-request boolean
Request a certificate from peer during SSL handshake. Default: true.

-require boolean
Require a valid certificate from peer during SSL handshake. If this is set to true then -request must also be set to true. Default: false

-servername host
Only available if the OpenSSL library the TLS package is linked against supports the TLS hostname extension for 'Server Name Indication' (SNI). Use to name the logical host we are talking to and expecting a certificate for. No default.

-ssl2 bool
Enable use of SSL v2. Default: false

-ssl3 bool
Enable use of SSL v3. Default: false

-tls1 bool
Enable use of TLS v1 Default: true

-tls1.1 bool
Enable use of TLS v1.1 Default: true

-tls1.2 bool
Enable use of TLS v1.2 Default: true

This command returns the current set of TLS options and values. In particular, one may use this command without any arguments to get the current set of options.

Using this command is incompatible with the obsolete form of **::ldap::secure_connect** and **::ldap_starttls** (see below).

::ldap::secure_connect ***host*** **?*port*?**
Like **::ldap::connect**, except that the created connection is secured by SSL. The port defaults to **636**. This command depends on the availability of the package **TLS**, which is a SSL binding for Tcl. If **TLS** is not available, then this command will fail.

TLS options are specified with **::ldap::tlsoptions**.

The command blocks until the connection has been established, or establishment definitely failed.

::ldap::secure_connect ***host*** **?*port*? ?*verify_cert*? ?*sni_servername*?**
Note: this form of the command is deprecated, since TLS options had to be specified with a combination of parameters to this command (*verify_cert* and *sni_servername*) and arguments to **::tls::init** (from package **tls**) for example to setup defaults for trusted certificates. Prefer

the above form (without the *verify_cert* and *sni_servername* parameters) and set TLS options with **::ldap::tlsoptions**.

If *verify_cert* is set to 1, the default, this checks the server certificate against the known hosts. If *sni_servername* is set, the given hostname is used as the hostname for Server Name Indication in the TLS handshake.

Use **::tls::init** to setup defaults for trusted certificates.

TLS supports different protocol levels. In common use are the versions 1.0, 1.1 and 1.2. By default all those versions are offered. If you need to modify the acceptable protocols, you can change the ::ldap::tlsProtocols list (deprecated).

::ldap::disconnect *handle*
Closes the ldap connection refered to by the token *handle*. Returns the empty string as its result.

::ldap::starttls *handle*
Start TLS negotiation on the connection denoted by *handle*, with TLS parameters set with **::ldap::tlsoptions**.

::ldap::starttls *handle* ?*cafile*? ?*certfile*? ?*keyfile*? ?*verify_cert*? ?*sni_servername*?
Note: this form of the command is deprecated, since TLS options had to be specified with a combination of parameters to this command (*cafile*, *certfile*, *keyfile*, *verify_cert* and *sni_servername*) and arguments to **::tls::init** (from package **tls**). Prefer the above form (without specific TLS arguments) and set TLS options with **::ldap::tlsoptions**.

Start TLS negotiation on the connection denoted by *handle*. You need to set at least the *cafile* argument to a file with trusted certificates, if *verify_cert* is 1, which is the default. The *sni_servername* can be used to signal a different hostname during the TLS handshake. The announced protocols are determined in the same way as **::ldap::secure_connect**. You can specify a TLS client certificate with the *certfile* and *keyfile* options.

::ldap::bind *handle* ?*name*? ?*password*?
This command authenticates the ldap connection refered to by the token in *handle*, with a user name and associated password. It blocks until a response from the ldap server arrives. Its result is the empty string. Both *name* and *passwd* default to the empty string if they are not specified. By leaving out *name* and *passwd* you can make an anonymous bind to the ldap server. You can issue **::ldap::bind** again to bind with different credentials.

::ldap::bindSASL *handle* ?*name*? ?*password*?
This command uses SASL authentication mechanisms to do a multistage bind. Its otherwise identical to the standard **::ldap::bind**. This feature is currently experimental and subject to change. See the documentation for the **SASL** and the "*SASL.txt*" in the tcllib CVS repository for details how to setup and use SASL with openldap.

::ldap::unbind *handle*
This command asks the ldap server to release the last bind done for the connection refered to by the token in *handle*. The *handle* is invalid after the unbind, as the server closes the connection. So this is effectivly just a more polite disconnect operation.

::ldap::search *handle baseObject filterString attributes options*

This command performs a LDAP search below the *baseObject* tree using a complex LDAP search expression *filterString* and returns the specified *attributes* of all matching objects (DNs). If the list of *attributes* was empty all attributes are returned. The command blocks until it has received all results. The valid *options* are identical to the options listed for **::ldap::searchInit**.

An example of a search expression is

```
    set filterString "|(cn=Linus*)(sn=Torvalds*)"
```

The return value of the command is a list of nested dictionaries. The first level keys are object identifiers (DNs), second levels keys are attribute names. In other words, it is in the form

```
    {dn1 {attr1 {val11 val12 ...} attr2 {val21...} ...}} {dn2 {a1
    {v11 ...} ...}} ...
```

::ldap::searchInit *handle baseObject filterString attributes options*

This command initiates a LDAP search below the *baseObject* tree using a complex LDAP search expression *filterString*. The search gets the specified *attributes* of all matching objects (DNs). The command itself just starts the search, to retrieve the actual results, use **::ldap::searchNext**. A search can be terminated at any time by **::ldap::searchEnd**. This informs the server that no further results should be sent by sending and ABANDON message and cleans up the internal state of the search. Only one **::ldap::search** can be active at a given time, this includes the introspection commands **::ldap::info saslmechanisms**, **ldap::info control** and **ldap::info extensions**, which invoke a search internally. Error responses from the server due to wrong arguments or similar things are returned with the first **::ldap::searchNext** call and should be checked and dealed with there. If the list of requested *attributes* is empty all attributes will be returned. The parameter *options* specifies the options to be used in the search, and has the following format:

```
    {-option1 value1 -option2 value2 ... }
```

Following options are available:

-scope base one sub
: Control the scope of the search to be one of **base**, **one**, or **sub**, to specify a base object, one-level or subtree search. The default is **sub**.

-derefaliases never search find always
: Control how aliases dereferencing is done. Should be one of **never**, **always**, **search**, or **find** to specify that aliases are never dereferenced, always dereferenced, dereferenced when searching, or dereferenced only when locating the base object for the search. The default is to never dereference aliases.

-sizelimit num
: Determines the maximum number of entries to return in a search. If specified as 0 no limit is enforced. The server may enforce a configuration dependent sizelimit, which may be lower than the one given by this option. The default is 0, no limit.

-timelimit seconds
: Asks the server to use a timelimit of *seconds* for the search. Zero means no limit. The default is 0, no limit.

-attrsonly boolean
If set to 1 only the attribute names but not the values will be present in the search result. The default is to retrieve attribute names and values.

-referencevar varname
If set the search result reference LDAPURIs, if any, are returned in the given variable. The caller can than decide to follow those references and query other LDAP servers for further results.

::ldap::searchNext *handle*
This command returns the next entry from a LDAP search initiated by **::ldap::searchInit**. It returns only after a new result is received or when no further results are available, but takes care to keep the event loop alive. The returned entry is a list with two elements: the first is the DN of the entry, the second is the list of attributes and values, under the format:

```
    dn {attr1 {val11 val12 ...} attr2 {val21...} ...}
```

The **::ldap::searchNext** command returns an empty list at the end of the search.

::ldap::searchEnd *handle*
This command terminates a LDAP search initiated by **::ldap::searchInit**. It also cleans up the internal state so a new search can be initiated. If the client has not yet received all results, the client sends an ABANDON message to inform the server that no further results for the previous search should to be sent.

::ldap::modify *handle dn attrValToReplace ?attrToDelete? ?attrValToAdd?*
This command modifies the object *dn* on the ldap server we are connected to via *handle*. It replaces attributes with new values, deletes attributes, and adds new attributes with new values. All arguments are dictionaries mapping attribute names to values. The optional arguments default to the empty dictionary, which means that no attributes will be deleted nor added.

dictionary *attrValToReplace* (in)
No attributes will be changed if this argument is empty. The dictionary contains the new attributes and their values. They *replace all* attributes known to the object.

dictionary *attrToDelete* (in)
No attributes will be deleted if this argument is empty. The dictionary values are restrictions on the deletion. An attribute listed here will be deleted if and only if its current value at the server matches the value specified in the dictionary, or if the value in the dictionary is the empty string.

dictionary *attrValToAdd* (in)
No attributes will be added if this argument is empty. The dictionary values are the values for the new attributes.

The command blocks until all modifications have completed. Its result is the empty string.

::ldap::modifyMulti *handle dn attrValToReplace ?attrValToDelete? ?attrValToAdd?*
This command modifies the object *dn* on the ldap server we are connected to via *handle*. It replaces attributes with new values, deletes attributes, and adds new attributes with new values. All arguments are lists with the format:

```
    attr1 {val11 val12 ...} attr2 {val21...} ...
```

where each value list may be empty for deleting all attributes. The optional arguments default to empty lists of attributes to delete and to add.

list *attrValToReplace* (in)
No attributes will be changed if this argument is empty. The dictionary contains the new attributes and their values. They *replace all* attributes known to the object.

list *attrValToDelete* (in)
No attributes will be deleted if this argument is empty. If no value is specified, the whole set of values for an attribute will be deleted.

list *attrValToAdd* (in)
No attributes will be added if this argument is empty.

The command blocks until all modifications have completed. Its result is the empty string.

::ldap::add *handle dn attrValueTuples*
This command creates a new object using the specified *dn*. The attributes of the new object are set to the values in the list *attrValueTuples*. Multiple valuated attributes may be specified using multiple tuples. The command blocks until the operation has completed. Its result is the empty string.

::ldap::addMulti *handle dn attrValueTuples*
This command is the preferred one to create a new object using the specified *dn*. The attributes of the new object are set to the values in the dictionary *attrValueTuples* (which is keyed by the attribute names). Each tuple is a list containing multiple values. The command blocks until the operation has completed. Its result is the empty string.

::ldap::delete *handle dn*
This command removes the object specified by *dn*, and all its attributes from the server. The command blocks until the operation has completed. Its result is the empty string.

::ldap::modifyDN *handle dn newrdn* ?*deleteOld*? ?*newSuperior*?
This command moves or copies the object specified by *dn* to a new location in the tree of object. This location is specified by *newrdn*, a *relative* designation, or by *newrdn* and *newSuperior*, a *absolute* designation. The optional argument *deleteOld* defaults to **true**, i.e. a move operation. If *deleteOld* is not set, then the operation will create a copy of *dn* in the new location. The optional argument *newSuperior* defaults an empty string, meaning that the object must not be relocated in another branch of the tree. If this argument is given, the argument *deleteOld* must be specified also. The command blocks until the operation has completed. Its result is the empty string.

::ldap::info ip *handle*
This command returns the IP address of the remote LDAP server the handle is connected to.

::ldap::info bound *handle*
This command returns 1 if a handle has successfully completed a **::ldap::bind**. If no bind was done or it failed, a 0 is returned.

::ldap::info bounduser *handle*
This command returns the username used in the bind operation if a handle has successfully completed a **::ldap::bind**. If no bound was done or it failed, an empty string is returned.

::ldap::info connections

This command returns all currently existing ldap connection handles.

::ldap::info tls *handle*
This command returns 1 if the ldap connection *handle* used TLS/SSL for connection via **ldap::secure_connect** or completed **ldap::starttls**, 0 otherwise.

::ldap::info tlsstatus *handle*
This command returns the current security status of an TLS secured channel. The result is a list of key-value pairs describing the connected peer (see the **TLS** package documentation for the returned values). If the connection is not secured with TLS, an empty list is returned.

::ldap::info saslmechanisms *handle*
Return the supported SASL mechanisms advertised by the server. Only valid in a bound state (anonymous or other).

::ldap::info control *handle*
Return the supported controls advertised by the server as a list of OIDs. Only valid in a bound state. This is currently experimental and subject to change.

::ldap::info extensions *extensions*
Returns the supported LDAP extensions as list of OIDs. Only valid in a bound state. This is currently experimental and subject to change.

::ldap::info whoami *handle*
Returns authzId for the current connection. This implements the RFC 4532 protocol extension.

Examples

A small example, extracted from the test application coming with this code.

```
    package require ldap

    # Connect, bind, add a new object, modify it in various ways

    set handle [ldap::connect localhost 9009]

    set dn "cn=Manager, o=University of Michigan, c=US"
    set pw secret

    ldap::bind $handle $dn $pw

    set dn "cn=Test User,ou=People,o=University of Michigan,c=US"

    ldap::add $handle $dn {
    objectClass     OpenLDAPperson
    cn              {Test User}
    mail            test.user@google.com
    uid             testuid
    sn              User
    telephoneNumber +31415926535
    telephoneNumber +27182818285
    }
```

```
    set dn "cn=Another User,ou=People,o=University of Michigan,c=US"

    ldap::addMulti $handle $dn {
    objectClass     {OpenLDAPperson}
    cn              {{Anotther User}}
    mail            {test.user@google.com}
    uid             {testuid}
    sn              {User}
    telephoneNumber {+31415926535 +27182818285}
    }

    # Replace all attributes
    ldap::modify $handle $dn [list drink icetea uid JOLO]

    # Add some more
    ldap::modify $handle $dn {} {} [list drink water  drink
    orangeJuice pager "+1 313 555 7671"]

    # Delete
    ldap::modify $handle $dn {} [list drink water  pager ""]

    # Move
    ldap::modifyDN $handle $dn "cn=Tester"

    # Kill the test object, and shut the connection down.
    set dn "cn=Tester,ou=People,o=University of Michigan,c=US"
    ldap::delete $handle $dn

    ldap::unbind     $handle
    ldap::disconnect $handle
```

And another example, a simple query, and processing the results.

```
    package require ldap
    set handle [ldap::connect ldap.acme.com 389]
    ldap::bind $handle
    set results [ldap::search $handle "o=acme,dc=com" "(uid=jdoe)" {}]
    foreach result $results {
    foreach {object attributes} $result break

    # The processing here is similar to what 'parray' does.
    # I.e. finding the longest attribute name and then
    # generating properly aligned output listing all attributes
    # and their values.

    set width 0
    set sortedAttribs {}
```

```
    foreach {type values} $attributes {
        if {[string length $type] > $width} {
        set width [string length $type]
        }
        lappend sortedAttribs [list $type $values]
    }

    puts "object='$object'"

    foreach sortedAttrib  $sortedAttribs {
        foreach {type values} $sortedAttrib break
        foreach value $values {
        regsub -all "\[\x01-\x1f\]" $value ? value
        puts [format "  %-${width}s %s" $type $value]
        }
    }
    puts ""
    }
    ldap::unbind $handle
    ldap::disconnect $handle
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *ldap* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Networking

Copyright

```
Copyright (c) 2004 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
Copyright (c) 2004 Jochen Loewer <loewerj@web.de>
Copyright (c) 2006 Michael Schlenker <mic42@users.sourceforge.net>
```

ldapx

Keywords: ldapx, directory access, internet, ldap, ldap client, ldif, protocol, rfc 2251, rfc 2849

Name

ldapx - LDAP extended object interface

Synopsis

package require Tcl 8.5 9
package require ldapx ?**1.3**?
e **reset**
e **dn** ?*newdn*?
e **rdn**
e **superior**
e **print**
se **isempty**
se **get** *attr*
se **get1** *attr*
se **set** *attr values*
se **set1** *attr value*
se **add** *attr values*
se **add1** *attr value*
se **del** *attr* ?*values*?
se **del1** *attr value*
se **getattr**
se **getall**
se **setall** *avpairs*
se **backup** ?*other*?
se **swap**
se **restore** ?*other*?
se **apply** *centry*
ce **change** ?*new*?
ce **diff** *new* ?*old*?
la **error** ?*newmsg*?
la **connect** *url* ?*binddn*? ?*bindpw*? ?*starttls*?
la **disconnect**
la **traverse** *base filter attrs entry body*
la **search** *base filter attrs*
la **read** *base filter entry ... entry*
la **commit** *entry ... entry*
li **channel** *chan*
li **error** ?*newmsg*?
li **read** *entry*
li **write** *entry*

Description

The **ldapx** package provides an extended Tcl interface to LDAP directores and LDIF files. The **ldapx** package is built upon the **ldap** package in order to get low level LDAP access.

LDAP access is compatible with RFC 2251 (http://www.rfc-editor.org/rfc/rfc2251.txt). LDIF access is compatible with RFC 2849 (http://www.rfc-editor.org/rfc/rfc2849.txt).

Overview

The **ldapx** package provides objects to interact with LDAP directories and LDIF files with an easy

to use programming interface. It implements three **snit**::type classes.

The first class, **entry**, is used to store individual entries. Two different formats are available: the first one is the *standard* format, which represents an entry as read from the directory. The second format is the *change* format, which stores differences between two standard entries.

With these entries, an application which wants to modify an entry in a directory needs to read a (standard) entry from the directory, create a fresh copy into a new (standard) entry, modify the new copy, and then compute the differences between the two entries into a new (change) entry, which may be commited to the directory.

Such kinds of modifications are so heavily used that standard entries may contain their own copy of the original data. With such a copy, the application described above reads a (standard) entry from the directory, backs-up the original data, modifies the entry, and computes the differences between the entry and its backup. These differences are then commited to the directory.

Methods are provided to compute differences between two entries, to apply differences to an entry in order to get a new entry, and to get or set attributes in standard entries.

The second class is the **ldap** class. It provides a method to **connect** and bind to the directory with a uniform access to LDAP and LDAPS through an URL (ldap:// or ldaps://). The **traverse** control structure executes a body for each entry found in the directory. The **commit** method applies some changes (represented as **entry** objects) to the directory. Since some attributes are represented as UTF-8 strings, the option **-utf8** controls which attributes must be converted and which attributes must not be converted.

The last class is the **ldif** class. It provides a method to associate a standard Tcl *channel* to an LDIF object. Then, methods **read** and **write** read or write entries from or to this channel. This class can make use of standard or change entries, according to the type of the LDIF file which may contain either standard entries or change entries (but not both at the same time). The option **-utf8** works exactly as with the **ldap** class.

Entry class

Entry instance data

An instance of the **entry** class keeps the following data:

dn
: This is the DN of the entry, which includes (in LDAP terminology) the RDN (relative DN) and the Superior parts.

format
: The format may be *uninitialized* (entry not yet used), *standard* or *change*. Most methods check the format of the entry, which can be reset with the **reset** method.

attrvals
: In a *standard* entry, this is where the attributes and associated values are stored. Many methods provide access to these informations. Attribute names are always converted into lower case.

backup
: In a *standard* entry, the backup may contain a copy of the dn and all attributes and values. Methods **backup** and **restore** manipulate these data, and method **diff** may use this backup.

change
In a *change* entry, these data represent the modifications. Such modifications are handled by specialized methods such as **apply** or **commit**. Detailed format should not be used directly by programs.

Internally, modifications are represented as a list of elements, each element has one of the following formats (which match the corresponding LDAP operations):

1. {**add** {attr1 {val1...valn} attr2 {...} ...}}

   Addition of a new entry.
2. {**mod** {modop {attr1 ?val1...valn?} attr2 ...} {modop ...} ...}

   Modification of one or more attributes and/or values, where can be **modadd**, **moddel** or **modrepl** (see the LDAP modify operation).
3. {**del**}

   Deletion of an old entry.
4. {**modrdn** newrdn deleteoldrdn ?newsuperior?}

   Renaming of an entry.

Entry options

No option is defined by this class.

Methods for all kinds of entries

***e* reset**
This method resets the entry to an uninitialized state.

***e* dn ?*newdn*?**
This method returns the current DN of the entry. If the optional *newdn* is specified, it replaces the current DN of the entry.

***e* rdn**
This method returns the RDN part of the DN of the entry.

***e* superior**
This method returns the superior part of the DN of the entry.

***e* print**
This method returns the entry as a string ready to be printed.

Methods for standard entries only

In all methods, attribute names are converted in lower case.

***se* isempty**
This method returns 1 if the entry is empty (i.e. without any attribute).

se* get *attr
This method returns all values of the attribute *attr*, or the empty list if the attribute is not fond.

se* get1 *attr
This method returns the first value of the attribute.

se* set *attr values
This method sets the values (list *values*) of the attribute *attr*. If the list is empty, this method

deletes all

se* set1 *attr value
This method sets the values of the attribute *attr* to be an unique value *value*. Previous values, if any, are replaced by the new value.

se* add *attr values
This method adds all elements the list *values* to the values of the attribute *attr*.

se* add1 *attr value
This method adds a single value given by the parameter *value* to the attribute *attr*.

***se* del *attr* ?*values*?**
If the optional list *values* is specified, this method deletes all specified values from the attribute *attr*. If the argument *values* is not specified, this method deletes all values.

se* del1 *attr value
This method deletes a unique *value* from the attribute *attr*.

***se* getattr**
This method returns all attributes names.

***se* getall**
This method returns all attributes and values from the entry, packed in a list of pairs.

se* setall *avpairs
This method sets at once all attributes and values. The format of the *avpairs* argument is the same as the one returned by method **getall**.

***se* backup ?*other*?**
This method stores in an *other* standard entry object a copy of the current DN and attributes/values. If the optional *other* argument is not specified, copy is done in the current entry (in a specific place, see section Overview).

***se* swap**
This method swaps the current and backup contexts of the entry.

***se* restore ?*other*?**
If the optional argument *other* is given, which must then be a *standard* entry, this method restores the current entry into the *other* entry. If the argument *other* argument is not specified, this methods restores the current entry from its internal backup (see section Overview).

se* apply *centry
This method applies changes defined in the *centry* argument, which must be a *change* entry.

Methods for change entries only

***ce* change ?*new*?**
If the optional argument *new* is specified, this method modifies the change list (see subsection Entry instance data for the exact format). In both cases, current change list is returned. Warning: values returned by this method should only be used by specialized methods such as **apply** or **commit**.

***ce* diff *new* ?*old*?**
This method computes the differences between the *new* and *old* entries under the form of a change list, and stores this list into the current *change* entry. If the optional argument *old* is not specified, difference is computed from the entry and its internal backup (see section Overview). Return value is the computed change list.

Entry example

```
package require ldapx

#
# Create an entry and fill it as a standard entry with
# attributes and values
#
::ldapx::entry create e
e dn "uid=joe,ou=people,o=mycomp"
e set1 "uid"             "joe"
e set  "objectClass"     {person anotherObjectClass}
e set1 "givenName"       "Joe"
e set1 "sn"              "User"
e set  "telephoneNumber" {+31415926535 +2182818}
e set1 "anotherAttr"     "This is a beautiful day, isn't it?"

puts stdout "e\n[e print]"

#
# Create a second entry as a backup of the first, and
# make some changes on it.
# Entry is named automatically by snit.
#

set b [::ldapx::entry create %AUTO%]
e backup $b

puts stdout "$b\n[$b print]"

$b del  "anotherAttr"
$b del1 "objectClass" "anotherObjectClass"

#
# Create a change entry, a compute differences between first
# and second entry.
#

::ldapx::entry create c
c diff e $b

puts stdout "$c\n[$c print]"

#
# Apply changes to first entry. It should be the same as the
# second entry, now.
#
```

```
e apply c

::ldapx::entry create nc
nc diff e $b

puts stdout "nc\n[nc print]"

#
# Clean-up
#

e destroy
$b destroy
c destroy
nc destroy
```

LDAP class

LDAP instance data

An instance of the **ldap** class keeps the following data:

channel
: This is the channel used by the **ldap** package for communication with the LDAP server.

lastError
: This variable contains the error message which appeared in the last method of the **ldap** class (this string is modified in nearly all methods). The **error** method may be used to fetch this message.

LDAP options

Options are configured on **ldap** instances using the **configure** method.

The first option is used for TLS parameters:

-tlsoptions *list*
: Specify the set of TLS options to use when connecting to the LDAP server (see the **connect** method). For the list of valid options, see the **LDAP** package documentation.

  The default is **-request 1 -require 1 -ssl2 no -ssl3 no -tls1 yes -tls1.1 yes -tls1.2 yes**.

  Example:

  ```
  $l configure -tlsoptions {-request yes -require yes}
  ```

A set of options of the **ldap** class is used during search operations (methods **traverse**, **search** and **read**, see below).

-scope base|one|sub
: Specify the scope of the LDAP search to be one of **base**, **one** or **sub** to specify a base object, one-level or subtree search.

The default is **sub**.

-derefaliases never|seach|find|always
Specify how aliases dereferencing is handled: **never** is used to specify that aliases are never derefenced, **always** that aliases are always derefenced, **search** that aliases are dereferenced when searching, or **find** that aliases are dereferenced only when locating the base object for the search.

The default is **never**.

-sizelimit integer
Specify the maximum number of entries to be retreived during a search. A value of **0** means no limit.

Default is **0**.

-timelimit integer
Specify the time limit for a search to complete. A value of **0** means no limit.

Default is **0**.

-attrsonly 0|1
Specify if only attribute names are to be retrieved (value **1**). Normally (value **0**), attribute values are also retrieved.

Default is **0**.

The last option is used when getting entries or committing changes in the directory:

-utf8 pattern-yes pattern-no
Specify which attribute values are encoded in UTF-8. This information is specific to the LDAP schema in use by the application, since some attributes such as jpegPhoto, for example, are not encoded in UTF-8. This option takes the form of a list with two regular expressions suitable for the **regexp** command (anchored by ^ and $). The first specifies which attribute names are to be UTF-8 encoded, and the second selects, among those, the attribute names which will not be UTF-8 encoded. It is thus possible to say: convert all attributes, except jpegPhoto.

Default is {{.*} {}}, meaning: all attributes are converted, without exception.

LDAP methods

***la* error ?*newmsg*?**
This method returns the error message that occurred in the last call to a **ldap** class method. If the optional argument *newmsg* is supplied, it becomes the last error message.

***la* connect *url* ?*binddn*? ?*bindpw*? ?*starttls*?**
This method connects to the LDAP server using given URL (which can be of the form *ldap://host:port* or *ldaps://host:port*). If an optional *binddn* argument is given together with the *bindpw* argument, the **connect** binds to the LDAP server using the specified DN and password.

If the *starttls* argument is given a true value (**1**, **yes**, etc.) and the URL uses the *ldap://*

scheme, a TLS negotiation is initiated with the newly created connection, before LDAP binding. Default value: **no**.

This method returns 1 if connection was successful, or 0 if an error occurred (use the **error** method to get the message).

***la* disconnect**
This method disconnects (and unbinds, if necessary) from the LDAP server.

la* traverse *base filter attrs entry body
This method is a new control structure. It searches the LDAP directory from the specified base DN (given by the *base* argument) and selects entries based on the argument *filter*. For each entry found, this method fetches attributes specified by the *attrs* argument (or all attributes if it is an empty list), stores them in the *entry* instance of class **entry** and executes the script defined by the argument *body*. Options are used to refine the search.

Caution: when this method is used, the script *body* cannot perform another LDAP search (methods **traverse**, **search** or **read**).

la* search *base filter attrs
This method searches the directory using the same way as method **traverse**. All found entries are stored in newly created instances of class **entry**, which are returned in a list. The newly created instances should be destroyed when they are no longer used.

la* read *base filter entry ... entry
This method reads one or more entries, using the same search criteria as methods **traverse** and **search**. All attributes are stored in the entries. This method provides a quick way to read some entries. It returns the number of entries found in the directory (which may be more than the number of read entries). If called without any *entry* argument, this method just returns the number of entries found, without returning any data.

la* commit *entry ... entry
This method commits the changes stored in the *entry* arguments. Each *entry* may be either a *change* entry, or a *standard* entry with a backup.

Note: in the future, this method should use the LDAP transaction extension provided by OpenLDAP 2.3 and later.

LDAP example

```
package require ldapx

#
# Connects to the LDAP directory using StartTLS
#

::ldapx::ldap create l
l configure -tlsoptions {-cadir /etc/ssl/certs -request yes
-require yes}
set url "ldap://server.mycomp.com"
if {! [l connect $url "cn=admin,o=mycomp" "mypasswd" yes]} then {
```

```
    puts stderr "error: [l error]"
    exit 1
    }

    #
    # Search all entries matching some criterion
    #

    l configure -scope one
    ::ldapx::entry create e
    set n 0
    l traverse "ou=people,o=mycomp" "(sn=Joe*)" {sn givenName} e {
    puts "dn: [e dn]"
    puts "  sn:        [e get1 sn]"
    puts "  givenName: [e get1 givenName]"
    incr n
    }
    puts "$n entries found"
    e destroy

    #
    # Add a telephone number to some entries
    # Note this modification cannot be done in the "traverse"
    operation.
    #

    set lent [l search "ou=people,o=mycomp" "(sn=Joe*)" {}]
    ::ldapx::entry create c
    foreach e $lent {
    $e backup
    $e add1 "telephoneNumber" "+31415926535"
    c diff $e
    if {! [l commit c]} then {
        puts stderr "error: [l error]"
        exit 1
    }
    $e destroy
    }
    c destroy

    l disconnect
    l destroy
```

LDIF class

LDIF instance data

An instance of the **ldif** class keeps the following data:

channel
: This is the Tcl channel used to retrieve or store LDIF file contents. The association between an instance and a channel is made by the method **channel**. There is no need to disrupt this association when the LDIF file operation has ended.

format
: LDIF files may contain *standard* entries or *change* entries, but not both. This variable contains the detected format of the file (when reading) or the format of entries written to the file (when writing).

lastError
: This variable contains the error message which appeared in the last method of the **ldif** class (this string is modified in nearly all methods). The **error** method may be used to fetch this message.

version
: This is the version of the LDIF file. Only version 1 is supported: the method **read** can only read from version 1 files, and method **write** only creates version 1 files.

LDIF options

This class defines two options:

-ignore list-of-attributes
: This option is used to ignore certain attribute names on reading. For example, to read OpenLDAP replica files (replog), one must ignore **replica** and **time** attributes since they do not conform to the RFC 2849 standard for LDIF files.

  Default is empty list: no attribute is ignored.

-utf8 pattern-yes pattern-no
: Specify which attribute values are encoded in UTF-8. This information is specific to the LDAP schema in use by the application, since some attributes such as jpegPhoto, for example, are not encoded in UTF-8. This option takes the form of a list with two regular expressions suitable for the **regexp** command (anchored by ^ and $). The first specifies which attribute names are to be UTF-8 encoded, and the second selects, among those, the attribute names which will not be UTF-8 encoded. It is thus possible to say: convert all attributes, except jpegPhoto.

  Default is {{.*} {}}, meaning: all attributes are converted, without exception.

LDIF methods

li* channel *chan
: This method associates the Tcl channel named *chan* with the LDIF instance. It resets the type of LDIF object to *uninitialized*.

***li* error ?*newmsg*?**
: This method returns the error message that occurred in the last call to a **ldif** class method. If the optional argument *newmsg* is supplied, it becomes the last error message.

li* read *entry
: This method reads the next entry from the LDIF file and stores it in the *entry* object of class

entry. The entry may be a *standard* or *change* entry.

li* write *entry

This method writes the entry given in the argument *entry* to the LDIF file.

LDIF example

```
package require ldapx

# This examples reads a LDIF file containing entries,
# compare them to a LDAP directory, and writes on standard
# output an LDIF file containing changes to apply to the
# LDAP directory to match exactly the LDIF file.

::ldapx::ldif create liin
liin channel stdin

::ldapx::ldif create liout
liout channel stdout

::ldapx::ldap create la
if {! [la connect "ldap://server.mycomp.com"]} then {
puts stderr "error: [la error]"
exit 1
}
la configure -scope one

# Reads LDIF file

::ldapx::entry create e1
::ldapx::entry create e2
::ldapx::entry create c

while {[liin read e1] != 0} {
set base [e1 superior]
set id [e1 rdn]
if {[la read $base "($id)" e2] == 0} then {
    e2 reset
}

c diff e1 e2
if {[llength [c change]] != 0} then {
    liout write c
}
}

la disconnect
la destroy
```

```
    e1 destroy
    e2 destroy
    c destroy
    liout destroy
    liin destroy
```

References

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *ldap* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Networking

Copyright


```
Copyright (c) 2006-2018 Pierre David <pdav@users.sourceforge.net>
```


log

log

Keywords: log, log level, message, message level

Name

log - Procedures to log messages of libraries and applications.

Synopsis

package require Tcl 8.5 9
package require log ?**1.5**?
::log::levels
::log::lv2longform *level*
::log::lv2color *level*
::log::lv2priority *level*
::log::lv2cmd *level*
::log::lv2channel *level*
::log::lvCompare *level1 level2*
::log::lvSuppress *level {suppress 1}*
::log::lvSuppressLE *level {suppress 1}*
::log::lvIsSuppressed *level*
::log::lvCmd *level cmd*

::log::lvCmdForall *cmd*
::log::lvChannel *level chan*
::log::lvChannelForall *chan*
::log::lvColor *level color*
::log::lvColorForall *color*
::log::log *level text*
::log::logarray *level arrayvar* ?*pattern*?
::log::loghex *level text data*
::log::logsubst *level msg*
::log::logMsg *text*
::log::logError *text*
::log::Puts *level text*

Description

The **log** package provides commands that allow libraries and applications to selectively log information about their internal operation and state.

To use the package just execute

```
    package require log
    log::log notice "Some message"
```

As can be seen above, each message given to the log facility is associated with a *level* determining the importance of the message. The user can then select which levels to log, what commands to use for the logging of each level and the channel to write the message to. In the following example the logging of all message with level **debug** is deactivated.

```
    package require log
    log::lvSuppress debug
    log::log debug "Unseen message" ; # No output
```

By default all messages associated with an error-level (**emergency**, **alert**, **critical**, and **error**) are written to **stderr**. Messages with any other level are written to **stdout**. In the following example the log module is reconfigured to write **debug** messages to **stderr** too.

```
    package require log
    log::lvChannel debug stderr
    log::log debug "Written to stderr"
```

Each message level is also associated with a command to use when logging a message with that level. The behaviour above for example relies on the fact that all message levels use by default the standard command **::log::Puts** to log any message. In the following example all messages of level **notice** are given to the non-standard command **toText** for logging. This disables the channel setting for such messages, assuming that **toText** does not use it by itself.

```
    package require log
    log::lvCmd notice toText
    log::log notice "Handled by \"toText\""
```

Another database maintained by this facility is a map from message levels to colors. The information in this database has *no* influence on the behaviour of the module. It is merely

provided as a convenience and in anticipation of the usage of this facility in **tk**-based application which may want to colorize message logs.

API

The following commands are available:

::log::levels
Returns the names of all known levels, in alphabetical order.

::log::lv2longform *level*
Converts any unique abbreviation of a level name to the full level name.

::log::lv2color *level*
Converts any level name including unique abbreviations to the corresponding color.

::log::lv2priority *level*
Converts any level name including unique abbreviations to the corresponding priority.

::log::lv2cmd *level*
Converts any level name including unique abbreviations to the command prefix used to write messages with that level.

::log::lv2channel *level*
Converts any level name including unique abbreviations to the channel used by **::log::Puts** to write messages with that level.

::log::lvCompare *level1 level2*
Compares two levels (including unique abbreviations) with respect to their priority. This command can be used by the -command option of lsort. The result is one of -1, 0 or 1 or an error. A result of -1 signals that level1 is of less priority than level2. 0 signals that both levels have the same priority. 1 signals that level1 has higher priority than level2.

::log::lvSuppress *level* {*suppress* 1}
(Un)suppresses the output of messages having the specified level. Unique abbreviations for the level are allowed here too.

::log::lvSuppressLE *level* {*suppress* 1}
(Un)suppresses the output of messages having the specified level or one of lesser priority. Unique abbreviations for the level are allowed here too.

::log::lvIsSuppressed *level*
Asks the package whether the specified level is currently suppressed. Unique abbreviations of level names are allowed.

::log::lvCmd *level cmd*
Defines for the specified level with which command to write the messages having this level. Unique abbreviations of level names are allowed. The command is actually a command prefix and this facility will append 2 arguments before calling it, the level of the message and the message itself, in this order.

::log::lvCmdForall *cmd*
Defines for all known levels with which command to write the messages having this level. The command is actually a command prefix and this facility will append 2 arguments before calling it, the level of the message and the message itself, in this order.

::log::lvChannel *level chan*
Defines for the specified level into which channel **::log::Puts** (the standard command) shall

write the messages having this level. Unique abbreviations of level names are allowed. The command is actually a command prefix and this facility will append 2 arguments before calling it, the level of the message and the message itself, in this order.

::log::lvChannelForall *chan*
Defines for all known levels with which which channel **::log::Puts** (the standard command) shall write the messages having this level. The command is actually a command prefix and this facility will append 2 arguments before calling it, the level of the message and the message itself, in this order.

::log::lvColor *level color*
Defines for the specified level the color to return for it in a call to **::log::lv2color**. Unique abbreviations of level names are allowed.

::log::lvColorForall *color*
Defines for all known levels the color to return for it in a call to **::log::lv2color**. Unique abbreviations of level names are allowed.

::log::log *level text*
Log a message according to the specifications for commands, channels and suppression. In other words: The command will do nothing if the specified level is suppressed. If it is not suppressed the actual logging is delegated to the specified command. If there is no command specified for the level the message won't be logged. The standard command **::log::Puts** will write the message to the channel specified for the given level. If no channel is specified for the level the message won't be logged. Unique abbreviations of level names are allowed. Errors in the actual logging command are *not* caught, but propagated to the caller, as they may indicate misconfigurations of the log facility or errors in the callers code itself.

::log::logarray *level arrayvar* ?*pattern*?
Like **::log::log**, but logs the contents of the specified array variable *arrayvar*, possibly restricted to entries matching the *pattern*. The pattern defaults to * (i.e. all entries) if none was specified.

::log::loghex *level text data*
Like **::log::log**, but assumes that *data* contains binary data. It converts this into a mixed hex/ascii representation before writing them to the log.

::log::logsubst *level msg*
Like **::log::log**, but *msg* may contain substitutions and variable references, which are evaluated in the caller scope first. The purpose of this command is to avoid overhead in the non-logging case, if the log message building is expensive. Any substitution errors raise an error in the command execution. The following example shows an xml text representation, which is only generated in debug mode:

```
    log::logsubst debug {XML of node $node is '[$node toXml]'}
```

::log::logMsg *text*
Convenience wrapper around **::log::log**. Equivalent to **::log::log info text**.

::log::logError *text*
Convenience wrapper around **::log::log**. Equivalent to **::log::log error text**.

::log::Puts *level text*
The standard log command, it writes messages and their levels to user-specified channels. Assumes that the suppression checks were done by the caller. Expects full level names,

abbreviations are *not allowed*.

Levels

The package currently defines the following log levels, the level of highest importance listed first.

- emergency
- alert
- critical
- error
- warning
- notice
- info
- debug

Note that by default all messages with levels **warning** down to **debug** are suppressed. This is done intentionally, because (we believe that) in most situations debugging output is not wanted. Most people wish to have such output only when actually debugging an application.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *log* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Programming tools

Copyright


```
Copyright (c) 2001-2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


logger

Keywords: logger, log, log level, service

Name

logger - System to control logging of events.

Synopsis

package require Tcl 8.5 9
package require logger ?**0.9.5**?
logger::init *service*
logger::import ?**-all**? ?**-force**? ?*-prefix prefix*? ?*-namespace namespace*? *service*
logger::initNamespace *ns* ?*level*?

logger::services
logger::enable *level*
logger::disable *level*
logger::setlevel *level*
logger::levels
logger::servicecmd *service*
${log}::debug *message*
${log}::info *message*
${log}::notice *message*
${log}::warn *message*
${log}::error *message*
${log}::critical *message*
${log}::alert *message*
${log}::emergency *message*
${log}::setlevel *level*
${log}::enable *level*
${log}::disable *level*
${log}::lvlchangeproc *command*
${log}::lvlchangeproc
${log}::logproc *level*
${log}::logproc *level command*
${log}::logproc *level argname body*
${log}::services
${log}::servicename
${log}::currentloglevel
${log}::delproc *command*
${log}::delproc
${log}::delete
${log}::trace *command*
${log}::trace on
${log}::trace off
${log}::trace status ?*procName*? ?*...*?
${log}::trace add *procName* ?*...*?
${log}::trace add ?*-ns*? *nsName* ?*...*?
${log}::trace remove *procName* ?*...*?
${log}::trace remove ?*-ns*? *nsName* ?*...*?

Description

The **logger** package provides a flexible system for logging messages from different services, at priority levels, with different commands.

To begin using the logger package, we do the following:

```
    package require logger
    set log [logger::init myservice]
    ${log}::notice "Initialized myservice logging"
```

```
    ... code ...

    ${log}::notice "Ending myservice logging"
    ${log}::delete
```

In the above code, after the package is loaded, the following things happen:

logger::init *service*
: Initializes the service *service* for logging. The service names are actually Tcl namespace names, so they are separated with '::'. The service name may not be the empty string or only ':'s. When a logger service is initialized, it "inherits" properties from its parents. For instance, if there were a service *foo*, and we did a **logger::init** *foo::bar* (to create a *bar* service underneath *foo*), *bar* would copy the current configuration of the *foo* service, although it would of course, also be possible to then separately configure *bar*. If a logger service is initialized and the parent does not yet exist, the parent is also created. The new logger service is initialized with the default loglevel set with **logger::setlevel**.

logger::import ?-all? ?-force? ?-prefix *prefix*? ?-namespace *namespace*? *service*
: Import the logger service commands into the current namespace. Without the **-all** option only the commands corresponding to the log levels are imported. If **-all** is given, all the **${log}::cmd** style commands are imported. If the import would overwrite a command an error is returned and no command is imported. Use the **-force** option to force the import and overwrite existing commands without complaining. If the **-prefix** option is given, the commands are imported with the given *prefix* prepended to their names. If the **-namespace** option is given, the commands are imported into the given namespace. If the namespace does not exist, it is created. If a namespace without a leading :: is given, it is interpreted as a child namespace to the current namespace.

logger::initNamespace *ns* ?*level*?
: Convenience command for setting up a namespace for logging. Creates a logger service named after the namespace *ns* (a :: prefix is stripped), imports all the log commands into the namespace, and sets the default logging level, either as specified by *level*, or inherited from a service in the parent namespace, or a hardwired default, **warn**.

logger::services
: Returns a list of all the available services.

logger::enable *level*
: Globally enables logging at and "above" the given level. Levels are **debug**, **info**, **notice**, **warn**, **error**, **critical**, **alert**, **emergency**.

logger::disable *level*
: Globally disables logging at and "below" the given level. Levels are those listed above.

logger::setlevel *level*
: Globally enable logging at and "above" the given level. Levels are those listed above. This command changes the default loglevel for new loggers created with **logger::init**.

logger::levels
: Returns a list of the available log levels (also listed above under **enable**).

logger::servicecmd *service*
: Returns the **${log}** token created by **logger::init** for this service.

${log}::debug *message*

see below ...

${log}::info *message*
see below ...

${log}::notice *message*
see below ...

${log}::warn *message*
see below ...

${log}::error *message*
see below ...

${log}::critical *message*
see below ...

${log}::alert *message*
see below ...

${log}::emergency *message*
These are the commands called to actually log a message about an event. **${log}** is the variable obtained from **logger::init**.

${log}::setlevel *level*
Enable logging, in the service referenced by **${log}**, and its children, at and above the level specified, and disable logging below it.

${log}::enable *level*
Enable logging, in the service referenced by **${log}**, and its children, at and above the level specified. Note that this does *not* disable logging below this level, so you should probably use **setlevel** instead.

${log}::disable *level*
Disable logging, in the service referenced by **${log}**, and its children, at and below the level specified. Note that this does *not* enable logging above this level, so you should probably use **setlevel** instead. Disabling the loglevel **emergency** switches logging off for the service and its children.

${log}::lvlchangeproc *command*
see below ...

${log}::lvlchangeproc
Set the script to call when the log instance in question changes its log level. If called without a command it returns the currently registered command. The command gets two arguments appended, the old and the new loglevel. The callback is invoked after all changes have been done. If child loggers are affected, their callbacks are called before their parents callback.

```
proc lvlcallback {old new} {
    puts "Loglevel changed from $old to $new"
}
${log}::lvlchangeproc lvlcallback
```

${log}::logproc *level*
see below ...

${log}::logproc *level command*
see below ...

${log}::logproc *level argname body*

This command comes in three forms - the third, older one is deprecated and may be removed from future versions of the logger package. The current set version takes one argument, a command to be executed when the level is called. The callback command takes on argument, the text to be logged. If called only with a valid level **logproc** returns the name of the command currently registered as callback command. **logproc** specifies which command will perform the actual logging for a given level. The logger package ships with default commands for all log levels, but with **logproc** it is possible to replace them with custom code. This would let you send your logs over the network, to a database, or anything else. For example:

```
    proc logtoserver {txt} {
        variable socket
        puts $socket "Notice: $txt"
    }

    ${log}::logproc notice logtoserver
```

Trace logs are slightly different: instead of a plain text argument, the argument provided to the logproc is a dictionary consisting of the **enter** or **leave** keyword along with another dictionary of details about the trace. These include:
- **proc** - Name of the procedure being traced.
- **level** - The stack level for the procedure invocation (from **info level**).
- **script** - The name of the file in which the procedure is defined, or an empty string if defined in interactive mode.
- **caller** - The name of the procedure calling the procedure being traced, or an empty string if the procedure was called from the global scope (stack level 0).
- **procargs** - A dictionary consisting of the names of arguments to the procedure paired with values given for those arguments (**enter** traces only).
- **status** - The Tcl return code (e.g. **ok**, **continue**, etc.) (**leave** traces only).
- **result** - The value returned by the procedure (**leave** traces only).

${log}::services

Returns a list of the registered logging services which are children of this service.

${log}::servicename

Returns the name of this service.

${log}::currentloglevel

Returns the currently enabled log level for this service. If no logging is enabled returns **none**.

${log}::delproc *command*

see below ...

${log}::delproc

Set the script to call when the log instance in question is deleted. If called without a command it returns the currently registered command. For example:

```
    ${log}::delproc [list closesock $logsock]
```

${log}::delete

This command deletes a particular logging service, and its children. You must call this to clean

up the resources used by a service.

${log}::trace *command*

This command controls logging of enter/leave traces for specified procedures. It is used to enable and disable tracing, query tracing status, and specify procedures are to be traced. Trace handlers are unregistered when tracing is disabled. As a result, there is not performance impact to a library when tracing is disabled, just as with other log level commands.

```
  proc tracecmd { dict } {
      puts $dict
  }

  set log [::logger::init example]
  ${log}::logproc trace tracecmd

  proc foo { args } {
      puts "In foo"
      bar 1
      return "foo_result"
  }

  proc bar { x } {
      puts "In bar"
      return "bar_result"
  }

  ${log}::trace add foo bar
  ${log}::trace on

  foo

# Output:
enter {proc ::foo level 1 script {} caller {} procargs {args {}}}
In foo
enter {proc ::bar level 2 script {} caller ::foo procargs {x 1}}
In bar
leave {proc ::bar level 2 script {} caller ::foo status ok result
    bar_result}
leave {proc ::foo level 1 script {} caller {} status ok result
    foo_result}
```

${log}::trace on

Turns on trace logging for procedures registered through the **trace add** command. This is similar to the **enable** command for other logging levels, but allows trace logging to take place at any level. The trace logging mechanism takes advantage of the execution trace feature of Tcl 8.4 and later. The **trace on** command will return an error if called from earlier versions of Tcl.

${log}::trace off

Turns off trace logging for procedures registered for trace logging through the **trace add** command. This is similar to the **disable** command for other logging levels, but allows trace logging to take place at any level. Procedures are not unregistered, so logging for them can be turned back on with the **trace on** command. There is no overhead imposed by trace registration when trace logging is disabled.

${log}::trace status ?procName? ?...?
This command returns a list of the procedures currently registered for trace logging, or a flag indicating whether or not a trace is registered for one or more specified procedures.

${log}::trace add *procName* ?...?
see below ...

${log}::trace add ?-ns? *nsName* ?...?
This command registers one or more procedures for logging of entry/exit traces. Procedures can be specified via a list of procedure names or namespace names (in which case all procedure within the namespace are targeted by the operation). By default, each name is first interpreted as a procedure name or glob-style search pattern, and if not found its interpreted as a namespace name. The *-ns* option can be used to force interpretation of all provided arguments as namespace names. Procedures must be defined prior to registering them for tracing through the **trace add** command. Any procedure or namespace names/patterns that don't match any existing procedures will be silently ignored.

${log}::trace remove *procName* ?...?
see below ...

${log}::trace remove ?-ns? *nsName* ?...?
This command unregisters one or more procedures so that they will no longer have trace logging performed, with the same matching rules as that of the **trace add** command.

Implementation

The logger package is implemented in such a way as to optimize (for Tcl 8.4 and newer) log procedures which are disabled. They are aliased to a proc which has no body, which is compiled to a no op in bytecode. This should make the peformance hit minimal. If you really want to pull out all the stops, you can replace the ${log} token in your code with the actual namespace and command (${log}::warn becomes ::logger::tree::myservice::warn), so that no variable lookup is done. This puts the performance of disabled logger commands very close to no logging at all.

The "object orientation" is done through a hierarchy of namespaces. Using an actual object oriented system would probably be a better way of doing things, or at least provide for a cleaner implementation.

The service "object orientation" is done with namespaces.

Logprocs and callstack

The logger package takes extra care to keep the logproc out of the call stack. This enables logprocs to execute code in the callers scope by using uplevel or linking to local variables by using upvar. This may fire traces with all usual side effects.

```
    # Print caller and current vars in the calling proc
    proc log_local_var {txt} {
         set caller [info level -1]
         set vars [uplevel 1 info vars]
         foreach var [lsort $vars] {
            if {[uplevel 1 [list array exists $var]] == 1} {
               lappend val $var <Array>
            } else {
               lappend val $var [uplevel 1 [list set $var]]
            }
         }
         puts "$txt"
         puts "Caller: $caller"
         puts "Variables in callers scope:"
         foreach {var value} $val {
           puts "$var = $value"
         }
    }

    # install as logproc
    ${log}::logproc debug log_local_var
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *logger* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Programming tools

loggerAppender

Keywords: loggerAppender, appender, logger

Name

logger::appender - Collection of predefined appenders for logger

Synopsis

package require Tcl 8.5 9
package require logger::appender ?**1.4**?
::logger::appender::console -level *level* **-service** *service* ?*options...*?
::logger::appender::colorConsole -level *level* **-service** *service* ?*options...*?

Description

This package provides a predefined set of logger templates.

::logger::appender::console -level *level* -service *service* ?*options*...?

see below ...

-level level

Name of the level to fill in as "priority" in the log procedure.

-service service

Name of the service to fill in as "category" in the log procedure.

-appenderArgs appenderArgs

Any additional arguments for the log procedure in list form

-conversionPattern conversionPattern

The log pattern to use (see **logger::utils::createLogProc** for the allowed substitutions).

-procName procName

Explicitly set the name of the created procedure.

-procNameVar procNameVar

Name of the variable to set in the calling context. This variable will contain the name of the procedure.

::logger::appender::colorConsole -level *level* -service *service* ?*options*...?

See **::logger::appender::colorConsole** for a description of the applicable options.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *logger* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Programming tools

Copyright


```
Copyright (c) 2005 Aamer Akhter <aakhter@cisco.com>
```


loggerUtils

Keywords: loggerUtils, appender, logger

Name

logger::utils - Utilities for logger

Synopsis

package require Tcl 8.5 9
package require logger::utils ?**1.3.2**?

::logger::utils::createFormatCmd *formatString*
::logger::utils::createLogProc -procName *procName* ?*options...*?
::logger::utils::applyAppender -appender *appenderType* ?*options...*?
::logger::utils::autoApplyAppender *command command-string log op args...*

Description

This package adds template based *appenders*.

::logger::utils::createFormatCmd *formatString*

This command translates *formatString* into an expandable command string. The following strings are the known substitutions (from log4perl) allowed to occur in the *formatString*:

%c
Category of the logging event

%C
Fully qualified name of logging event

%d
Current date in yyyy/MM/dd hh:mm:ss

%H
Hostname

%m
Message to be logged

%M
Method where logging event was issued

%p
Priority of logging event

%P
Pid of current process

::logger::utils::createLogProc -procName *procName* ?*options*...?

This command ...

-procName procName
The name of the procedure to create.

-conversionPattern pattern
See **::logger::utils::createFormatCmd** for the substitutions allowed in the *pattern*.

-category category
The category (service).

-priority priority
The priority (level).

-outputChannel channel
channel to output on (default stdout)

::logger::utils::applyAppender -appender *appenderType* ?*options*...?

This command will create an appender for the specified logger services. If no service is specified then the appender will be added as the default appender for the specified levels. If no levels are specified, then all levels are assumed.

-service loggerservices

see below ...

-serviceCmd loggerserviceCmds
Name of the logger instance to modify. **-serviceCmd** takes as input the return of logger::init.

-appender appenderType
Type of the appender to use. One of **console**, **colorConsole**.

-appenderArgs appenderArgs
Additional arguments to apply to the appender. The argument of the option is a list of options and their arguments.

For example

```
logger::utils::applyAppender -serviceCmd $log -appender console
    -appenderArgs {-conversionPattern {\[%M\] \[%p\] - %m}}
```

The usual Tcl quoting rules apply.

-levels levelList
The list of levels to apply this appender to. If not specified all levels are assumed.

Example of usage:

```
 % set log [logger::init testLog]
 ::logger::tree::testLog
 % logger::utils::applyAppender -appender console -serviceCmd $log
 % ${log}::error "this is an error"
 [2005/08/22 10:14:13] [testLog] [global] [error] this is an error
```

::logger::utils::autoApplyAppender *command command-string log op args...*
This command is designed to be added via **trace leave** to calls of logger::init. It will look at preconfigured state (via **::logger::utils::applyAppender**) to autocreate appenders for newly created logger instances. It will return its argument *log*.

Example of usage:

```
logger::utils::applyAppender -appender console
set log [logger::init applyAppender-3]
${log}::error "this is an error"
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *logger* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Programming tools

Copyright

```
Copyright (c) 2005 Aamer Akhter <aakhter@cisco.com>
```


map

map_geocode_nominatim
Keywords: map_geocode_nominatim, geocoding, http, location, map, nominatim, server, url

Name
map::geocode::nominatim - Resolving geographical names with a Nominatim service

Synopsis
package require Tcl 8.6 9
package require http
package require json
package require uri
package require snit
package require map::geocode::nominatim ?**0.3**?
::map::geocode::nominatim *requestor* ?*-baseurl url*? ?*-callback callback*? ?*-error error callback*?
$cmdprefix *result*
$cmdprefix *errorstring*
requestor **search** *query*

Description
This package provides a class for accessing geocoding services which implement the *Nominatim* interface (see References)

API
::map::geocode::nominatim *requestor* ?-baseurl *url*? ?-callback *callback*? ?-error *error callback*?

Creates a geocoding request object *requestor*, which will send its requests to the *Nominatim* server.

The result of the command is *name*.

Options
-baseurl *url*

The base URL of the *Nominatim* service. Default value is *OpenStreetMap's* service at http://nominatim.openstreetmap.org/search A possible free alternative is at http://open.mapquestapi.com//nominatim/v1/search

-callback *cmdprefix*

A command prefix to be invoked when search result become available. The default setting,

active when nothing was specified on object creation, is to print the *result* (see below) to *stdout*. The result of the command prefix is ignored. Errors thrown by the command prefix are caught and cause the invokation of the error callback (see option **-error** below), with the error message as argument.

The signature of the command prefix is:
$cmdprefix *result*

The *result* is a list of dictionaries, containing one item per hit. Each dictionary will have the following entries:

place_id
The place ID (FIXME: what's this?)

licence
The data licence string

osm_type
The OSM type of the location

osm_id
FIXME

boundingbox
The coordinates of the bounding box (min and max latitude, min and max longitude)

lat
The location's latitude

lon
The location's longitude

display_name
the location's human readable name

class
FIXME

type
FIXME

icon
FIXME

-error *cmdprefix*

A command prefix to be invoked when encountering errors. Typically these are HTTP errors. The default setting, active when nothing was specified on object creation, is to print the *errorstring* (see below) to *stderr*. The result of the command prefix is ignored. Errors thrown by the command prefix are passed to higher levels.

The signature of the command prefix is:
- **$cmdprefix** *errorstring*

Methods

requestor* search *query

This method returns a list of dictionaries, one item per hit for the specified *query*.

References

1. http://wiki.openstreetmap.org/wiki/Nominatim
2. http://open.mapquestapi.com/nominatim/

map_slippy

Keywords: map_slippy, geodesy, geography, latitute, location, longitude, map, slippy, zoom

Name

map::slippy - Common code for slippy based map packages

Synopsis

package require Tcl 8.6 9
package require map::slippy ?**0.10**?
::map slippy geo box 2point *zoom geobox*
::map slippy geo box center *geobox*
::map slippy geo box corners *geobox*
::map slippy geo box diameter *geobox*
::map slippy geo box dimensions *geobox*
::map slippy geo box fit *geobox canvdim zmax* ?*zmin*?
::map slippy geo box inside *geobox geo*
::map slippy geo box limit *geobox*
::map slippy geo box opposites *geobox*
::map slippy geo box perimeter *geobox*
::map slippy geo box valid *geobox*
::map slippy geo box valid-list *geoboxes*
::map slippy geo distance *geo1 geo2*
::map slippy geo distance* *closed geo...*
::map slippy geo distance-list *closed geo-list*
::map slippy geo limit *geo*
::map slippy geo bbox *geo...*
::map slippy geo bbox-list *geo-list*
::map slippy geo center *geo...*
::map slippy geo center-list *geo-list*
::map slippy geo diameter *geo...*
::map slippy geo diameter-list *geo-list*
::map slippy geo 2point *zoom geo*
::map slippy geo 2point* *zoom geo...*
::map slippy geo 2point-list *zoom geo-list*
::map slippy geo valid *geo*
::map slippy geo valid-list *geos*
::map slippy length *level*
::map slippy limit2 *x*
::map slippy limit3 *x*
::map slippy limit6 *x*
::map slippy point box 2geo *zoom pointbox*
::map slippy point box center *pointbox*

::map slippy point box corners *pointbox*
::map slippy point box diameter *pointbox*
::map slippy point box dimensions *pointbox*
::map slippy point box inside *pointbox point*
::map slippy point box opposites *pointbox*
::map slippy point box perimeter *pointbox*
::map slippy point distance *point1 point2*
::map slippy point distance* *closed point...*
::map slippy point distance-list *closed point-list*
::map slippy point bbox *point...*
::map slippy point bbox-list *point-list*
::map slippy point center *point...*
::map slippy point center-list *point-list*
::map slippy point diameter *point...*
::map slippy point diameter-list *point-list*
::map slippy point 2geo *zoom point*
::map slippy point 2geo* *zoom point...*
::map slippy point 2geo-list *zoom point-list*
::map slippy point simplify radial *threshold point-list*
::map slippy point simplify rdp *point-list*
::map slippy pretty-distance *x*
::map slippy tiles *level*
::map slippy tile size
::map slippy tile valid *zoom row column levels* ?*msgvar*?
::map slippy valid latitude *x*
::map slippy valid longitude *x*

Description

This package provides a number of methods doing things needed by all types of slippy-based map packages.

BEWARE, *Attention* Version *0.9* is *NOT backward compatible* with version 0.7 and earlier. The entire API was *heavily revised and changed*.

Note: For the representation of locations in the various coordinate systems used by the commands of this package please read section Coordinate systems. The command descriptions will not repeat them, and assume that they are understood already.

Coordinate systems

The commands of this package operate in two distinct coordinate systems, geographical locations, and points. The former represents coordinates for locations on Earth, while the latter is for use on Tk *canvas* widgets.

Geographic

Geographical locations (short: *geo*) are represented by a pair of *Latitude* and *Longitude* values, each of which is measured in degrees, as they are essentially angles.

The **Zero** longitude is the *Greenwich meridian*, with positive values going *east*, and negative values going *west*, for a total range of +/- 180 degrees. Note that +180 and -180 longitude are

the same *meridian*, opposite to Greenwich.

The **zero** latitude is the *Equator*, with positive values going *north* and negative values going *south*. While the true range is +/- 90 degrees the projection used by the package requires us to cap the range at roughly +/- **85.05112877983284** degrees. This means that the North and South poles are not representable and not part of any map.

A geographical location is represented by a list containing two values, the latitude, and longitude of the location, in this order.

A geographical bounding box is represented by a list containing four values, the minimal latitude and longitude of the box, and the maximal latitude and longitude of the box, in this order.

Geographical locations and boxes can be converted to points and their boxes by means of an additional parameter, the *zoom* level. This parameter indicates the size of the map in the canvas the coordinates are to be projected into.

Points

Points (short: *point*) are represented by a pair of *x* and *y* values, each of which is measured in pixels. They reference a location in a Tk *canvas* widget. As a map can be shown at different degrees of magnification, the exact pixel coordinates for a geographical location depend on this *zoom* level.

For the following explanation to make sense it should be noted that a map shown in a Tk *canvas* widget is split into equal-sized quadratic *tiles*.

Point coordinates are tile coordinates scaled by the size of these tiles. This package uses tiles of size **256**, which is the standard size used by many online servers providing map tiles of some kind or other.

A point is represented by a list containing the x- and y-coordinates of the lcoation, in this in this order.

A point bounding box is represented by a list containing four values, the minimal x and y of the box, and the maximal x and y of the box, in this order.

Point locations and boxes can be converted to geographical locations and their boxes by means of an additional parameter, the *zoom* level. This parameter indicates the size of the map in the canvas the coordinates are projected from.

Tile coordinates appear only in one place of the API, in the signature of command **map slippy tile valid**. Everything else uses Point coordinates.

Using tile coordinates in the following however makes the structure of the map at the various *zoom* levels (maginification factors) easier to explain.

Generally the integer part of the tile coordinates represent the row and column number of a tile of the map, wheras the fractional parts signal how far inside that tile the location in question is, with pure integer coordinates (no fractional part) representing the upper left corner of a tile.

The zero point of the map is at the upper left corner, regardless of zoom level, with larger coordinates going right (east) and down (south), and smaller coordinates going left (west) and up (north). Again regardless of zoom level.

Negative coordinates are not allowed.

At zoom level **0** the entire map is represented by a single tile, putting the geographic zero at

1/2, 1/2 in terms of tile coordinates, and the range of tile coordinates as [0...1].

When going from zoom level **N** to the next deeper (magnified) level (**N**+1) each tile of level **N** is split into its four quadrants, which then are the tiles of level **N**+1.

This means that at zoom level **N** the map is sliced (horizontally and vertically) into **2^N** rows and columns, for a total of **4^N** tiles, with the tile coordinates ranging from **0** to **2^N+1**.

API

::map slippy geo box 2point *zoom geobox*
: The command converts the geographical box *geobox* to a point box in the canvas, for the specified *zoom* level, and returns that box.

::map slippy geo box center *geobox*
: The command returns the center of the geographical box *geobox*.

::map slippy geo box corners *geobox*
: This command returns a list containing the four corner locations implied by the geographical box *geobox*. The four points are top-left, bottom-left, top-right, and bottom-right, in that order.

::map slippy geo box diameter *geobox*
: The command returns the diameter of the geographical box *geobox*, in meters.

::map slippy geo box dimensions *geobox*
: The command returns the dimensions of the geographical box *geobox*, width and height, in this order.

::map slippy geo box fit *geobox canvdim zmax* ?*zmin*?
: This command calculates the zoom level such that the *geobox* will fit into a viewport given by *canvdim* (a 2-element list containing the width and height of said viewport) and returns it.

  The zoom level will be made to fit within the range *zmin*...*zmax*. When *zmin* is not specified it will default to **0**.

::map slippy geo box inside *geobox geo*
: The command tests if the geographical location *geo* is contained in the geographical box *geobox* or not. It returns **true** if so, and **false** else.

::map slippy geo box limit *geobox*
: This command limits the geographical box to at most 6 decimals and returns the result.

  For geographical coordinates 6 decimals is roughly equivalent to a grid of 11.1 cm.

::map slippy geo box opposites *geobox*
: This command returns a list containing the two principal corner locations implied by the geographical box *geobox*. The two points are top-left, and bottom-right, in that order.

::map slippy geo box perimeter *geobox*
: The command returns the perimeter of the geographical box *geobox*, in meters.

::map slippy geo box valid *geobox*
: This commands tests if the specified geographical box contains only valid latitudes and longitudes. It returns **true** if the box is valid, and **false** else.

::map slippy geo box valid-list *geoboxes*
: This commands tests if the list of geographical boxes contains only valid latitudes and longitudes. It returns **true** if all the boxes are valid, and **false** else.

::map slippy geo distance *geo1 geo2*
This command computes the great-circle distance between the two geographical locations in meters and returns that value.

The code is based on https://wiki.tcl-lang.org/page/geodesy take on the haversine formula.

::map slippy geo distance* *closed geo...*
An extension of **map slippy geo distance** this command computes the cumulative distance along the path specified by the ordered set of geographical locations in meters, and returns it.

If the path is marked as *closed* (i.e. a polygon/loop) the result contains the distance between last and first element of the path as well, making the result the length of the perimeter of the area described by the locations.

::map slippy geo distance-list *closed geo-list*
As a variant of **map slippy geo distance*** this command takes the path to compute the length of as a single list of geographical locations, instead of a varying number of arguments.

::map slippy geo limit *geo*
This command limits the geographical location to at most 6 decimals and returns the result.

For geographical coordinates 6 decimals is roughly equivalent to a grid of 11.1 cm.

::map slippy geo bbox *geo...*
see below ...

::map slippy geo bbox-list *geo-list*
These two commands compute the bounding box for the specified set of geographical locations and return a geographical box.

When no geographical locations are specified the box is "**0 0 0 0**".

The locations are specified as either a varying number of arguments, or as a single list.

::map slippy geo center *geo...*
see below ...

::map slippy geo center-list *geo-list*
These two commands compute the center of the bounding box for the specified set of geographical locations.

When no geographical locations are specified the center is "**0 0**".

The locations are specified as either a varying number of arguments, or as a single list.

::map slippy geo diameter *geo...*
see below ...

::map slippy geo diameter-list *geo-list*
These two commands compute the diameter for the specified set of geographical locations, in meters. The diameter is the maximum of the pair-wise distances between all locations.

When less than two geographical locations are specified the diameter is "**0**".

The locations are specified as either a varying number of arguments, or as a single list.

::map slippy geo 2point *zoom geo*
This command converts the geographical location *geo* to a point in the canvas, for the specified *zoom* level, and returns that point.

::map slippy geo 2point* *zoom geo...*
see below ...

::map slippy geo 2point-list *zoom geo-list*
These two commands are extensions of **map slippy geo 2point** which take a series of geographical locations as either a varying number of arguments or a single list, convert them all to points as per the specified *zoom* level and return a list of the results.

::map slippy geo valid *geo*
This commands tests if the specified geographical location contains only valid latitudes and longitudes. It returns **true** if the location is valid, and **false** else.

::map slippy geo valid-list *geos*
This commands tests if the list of geographical locations contains only valid latitudes and longitudes. It returns **true** if all the locations are valid, and **false** else.

::map slippy length *level*
This command returns the width/height of a slippy-based map at the specified zoom *level*, in pixels. This is, in essence, the result of

```
expr { [tiles $level] * [tile size] }
```

::map slippy limit2 *x*
see below ...

::map slippy limit3 *x*
see below ...

::map slippy limit6 *x*
This command limits the value to at most 2, 3, or 6 decimals and returns the result.

For geographical coordinates 6 decimals is roughly equivalent to a grid of 11.1 cm.

::map slippy point box 2geo *zoom pointbox*
The command converts the point box *pointbox* to a geographical box in the canvas, as per the specified *zoom* level, and returns that box.

::map slippy point box center *pointbox*
The command returns the center of the *pointbox*.

::map slippy point box corners *pointbox*
This command returns a list containing the four corner locations implied by the point box *pointbox*. The four points are top-left, bottom-left, top-right, and bottom-right, in that order.

::map slippy point box diameter *pointbox*
The command returns the diameter of the *pointbox*, in pixels.

::map slippy point box dimensions *pointbox*
The command returns the dimensions of the *pointbox*, width and height, in this order.

::map slippy point box inside *pointbox point*
The command tests if the *point* is contained in the *pointbox* or not. It returns **true** if so, and **false** else.

::map slippy point box opposites *pointbox*
This command returns a list containing the two principal corner locations implied by the point box *pointbox*. The two points are top-left, and bottom-right, in that order.

::map slippy point box perimeter *pointbox*
The command returns the perimeter of the *pointbox*, in pixels.

::map slippy point distance *point1 point2*
This command computes the euclidena distance between the two points in pixels and returns that value.

::map slippy point distance* *closed point...*
An extension of **map slippy point distance** this command computes the cumulative distance along the path specified by the ordered set of points, and returns it.

If the path is marked as *closed* (i.e. a polygon/loop) the result contains the distance between last and first element of the path as well, making the result the length of the perimeter of the area described by the locations.

::map slippy point distance-list *closed point-list*
As a variant of **map slippy point distance*** this command takes the path to compute the length of as a single list of points, instead of a varying number of arguments.

::map slippy point bbox *point...*
see below ...

::map slippy point bbox-list *point-list*
These two commands compute the bounding box for the specified set of points and return a point box.

When no points are specified the box is "**0 0 0 0**".

The locations are specified as either a varying number of arguments, or as a single list.

::map slippy point center *point...*
see below ...

::map slippy point center-list *point-list*
These two commands compute the center of the bounding box for the specified set of points.

When no points are specified the center is "**0 0**".

The locations are specified as either a varying number of arguments, or as a single list.

::map slippy point diameter *point...*
see below ...

::map slippy point diameter-list *point-list*
These two commands compute the diameter for the specified set of points, in pixels. The diameter is the maximum of the pair-wise distances between all locations.

When less than two points are specified the diameter is "**0**".

The locations are specified as either a varying number of arguments, or as a single list.

::map slippy point 2geo *zoom point*

This command converts the *point* in the canvas, for the specified *zoom* level, to a geograhical location, and returns that location.

::map slippy point 2geo* *zoom point...*
see below ...

::map slippy point 2geo-list *zoom point-list*
These two commands are extensions of **map slippy point 2geo** which take a series of points as either a varying number of arguments or a single list, convert them all to geographical locations as per the specified *zoom* level and return a list of the results.

::map slippy point simplify radial *threshold point-list*
This command takes a path of points (as a single list), simplifies the path using the *Radial Distance* algorithm and returns the simplified path as list of points.

In essence the algorithm keeps only the first of adjacent points nearer to that first point than the threshold, and drops the others.

::map slippy point simplify rdp *point-list*
This command takes a patch of points (as a single list), simplifies it using the *non-parametric Ramer-Douglas-Peucker* algorithm and returns the simplified path as list of points.

::map slippy pretty-distance *x*
This methods formats the distance *x* (in meters) for display and returns the resulting string (including the chosen unit).

Sub-kilometer distances are limited to 2 decimals, i.e. centimeters, whereas Kilometers are limited to 3 decimals, i.e. meters.

::map slippy tiles *level*
This command returns the width/height of a slippy-based map at the specified zoom *level*, in *tiles*.

::map slippy tile size
This command returns the width/height of a tile in a slippy-based map, in pixels.

::map slippy tile valid *zoom row column levels* ?*msgvar*?
This command checks if the tile described by *zoom*, *row*, and *column* is valid for a slippy-based map having that many zoom *levels*, or not. The result is a boolean value, **true** if the tile is valid, and **false** otherwise. In the latter case a message is left in the variable named by *msgvar*, should it be specified.

::map slippy valid latitude *x*
This commands tests if the argument *x* is a valid latitude value, and returns the boolean result of that test. I.e. **true** if the value is valid, and **false** else.

::map slippy valid longitude *x*
This commands tests if the argument *x* is a valid longitude value, and returns the boolean result of that test. I.e. **true** if the value is valid, and **false** else.

References

1. http://wiki.openstreetmap.org/wiki/Main_Page

map_slippy_cache

Keywords: map_slippy_cache, cache, filesystem, location, map, slippy, tile, zoom

Name

map::slippy::cache - Management of a tile cache in the local filesystem

Synopsis

package require Tcl 8.6 9
package require Tk 8.6
package require map::slippy
package require map::slippy::cache ?**0.5**?
::map::slippy::cache *cacheName cachedir provider*
cacheName **valid** *tile* ?*msgvar*?
cacheName **exists** *tile*
cacheName **get** *tile donecmd*

Description

This package provides a class for managing a cache of tiles for slippy-based maps in the local filesystem.

BEWARE, *Attention* Version *0.5* is *NOT backward compatible* with version 0.3 and earlier. This is due to switching this package's internals to the use of version 0.8 of **map::slippy**.

API

::map::slippy::cache *cacheName cachedir provider*

Creates the cache *cacheName* and configures it with both the path to the directory contaiing the locally cached tiles (*cachedir*), and the command prefix from which it will pull tiles asked for and not yet known to the cache itself (*provider*).

The result of the command is *cacheName*.

Methods

***cacheName* valid *tile* ?*msgvar*?**

This method checks the validity of a the given *tile* identifier. This is a convenience wrapper to **::map::slippy tile valid** and has the same interface.

cacheName* exists *tile

This methods tests whether the cache contains the specified *tile* or not. The result is a boolean value, **true** if the tile is known, and **false** otherwise. The tile is identified by a list containing three elements, zoom level, row, and column number, in this order.

cacheName* get *tile donecmd

This is the main method of the cache, retrieving the image for the specified *tile* from the cache. The tile identifier is a list containing three elements, the zoom level, row, and column number of the tile, in this order.

The command refix *donecmd* will be invoked when the cache either knows the image for the tile or that no image will forthcoming. It will be invoked with either 2 or 3 arguments, i.e.
1. The string **set**, the *tile*, and the image.
2. The string **unset**, and the *tile*.

These two possibilities are used to either signal the image for the *tile*, or that the *tile* has no image defined for it.

When the cache has no information about the tile it will invoke the *provider* command prefix specified during its construction, adding three arguments: The string **get**, the *tile*, and a callback into the cache. The latter will be invoked by the provider to either transfer the image to the cache, or signal that the tile has no image.

When multiple requests for the same tile are made only one request will be issued to the provider.

References

1. http://wiki.openstreetmap.org/wiki/Main_Page

map_slippy_fetcher

Keywords: map_slippy_fetcher, http, location, map, server, slippy, tile, url, zoom

Name

map::slippy::fetcher - Accessing a server providing tiles for slippy-based maps

Synopsis

package require Tcl 8.6 9
package require Tk 8.6
package require map::slippy
package require map::slippy::fetcher ?**0.7**?
::map::slippy::fetcher *fetcherName levels url*
fetcherName **levels**
fetcherName **tileheight**
fetcherName **tilewidth**
fetcherName **get** *tile donecmd*

Description

This package provides a class for accessing http servers providing tiles for slippy-based maps.

BEWARE, *Attention* Version *0.7* is *NOT backward compatible* with version 0.5 and earlier. This is due to switching this package's internals to the use of version 0.8 of **map::slippy**.

API

::map::slippy::fetcher *fetcherName levels url*

Creates the fetcher *fetcherName* and configures it with the number of zoom *levels* supported by the tile server, and the *url* it is listening on for tile requests.

The result of the command is *fetcherName*.

Methods

***fetcherName* levels**

This method returns the number of zoom levels supported by the fetcher object, and the tile server it is accessing.

***fetcherName* tileheight**

This method returns the height of tiles served, in pixels.

***fetcherName* tilewidth**

This method returns the width of tiles served, in pixels.

fetcherName* get *tile donecmd

This is the main method of the fetcher, retrieving the image for the specified *tile*. The tile identifier is a list containing three elements, the zoom level, row, and column number of the tile, in this order.

The command refix *donecmd* will be invoked when the fetcher either knows the image for the tile or that no image will forthcoming. It will be invoked with either 2 or 3 arguments, i.e.

1. The string **set**, the *tile*, and the image.
2. The string **unset**, and the *tile*.

These two possibilities are used to either signal the image for the *tile*, or that the *tile* has no image defined for it.

References

1. http://wiki.openstreetmap.org/wiki/Main_Page

mapproj

mapproj

Keywords: mapproj, geodesy, map, projection

Name

mapproj - Map projection routines

Synopsis

package require Tcl ?**8.5 9**?
package require math::interpolate ?**1.1**?
package require math::special ?**0.2.2**?
package require mapproj ?**1.1**?
::mapproj::toPlateCarree *lambda_0 phi_0 lambda phi*
::mapproj::fromPlateCarree *lambda_0 phi_0 x y*
::mapproj::toCylindricalEqualArea *lambda_0 phi_0 lambda phi*
::mapproj::fromCylindricalEqualArea *lambda_0 phi_0 x y*
::mapproj::toMercator *lambda_0 phi_0 lambda phi*
::mapproj::fromMercator *lambda_0 phi_0 x y*
::mapproj::toMillerCylindrical *lambda_0 lambda phi*
::mapproj::fromMillerCylindrical *lambda_0 x y*
::mapproj::toSinusoidal *lambda_0 phi_0 lambda phi*
::mapproj::fromSinusoidal *lambda_0 phi_0 x y*
::mapproj::toMollweide *lambda_0 lambda phi*
::mapproj::fromMollweide *lambda_0 x y*
::mapproj::toEckertIV *lambda_0 lambda phi*
::mapproj::fromEckertIV *lambda_0 x y*
::mapproj::toEckertVI *lambda_0 lambda phi*
::mapproj::fromEckertVI *lambda_0 x y*
::mapproj::toRobinson *lambda_0 lambda phi*

::mapproj::fromRobinson *lambda_0 x y*
::mapproj::toCassini *lambda_0 phi_0 lambda phi*
::mapproj::fromCassini *lambda_0 phi_0 x y*
::mapproj::toPeirceQuincuncial *lambda_0 lambda phi*
::mapproj::fromPeirceQuincuncial *lambda_0 x y*
::mapproj::toOrthographic *lambda_0 phi_0 lambda phi*
::mapproj::fromOrthographic *lambda_0 phi_0 x y*
::mapproj::toStereographic *lambda_0 phi_0 lambda phi*
::mapproj::fromStereographic *lambda_0 phi_0 x y*
::mapproj::toGnomonic *lambda_0 phi_0 lambda phi*
::mapproj::fromGnomonic *lambda_0 phi_0 x y*
::mapproj::toAzimuthalEquidistant *lambda_0 phi_0 lambda phi*
::mapproj::fromAzimuthalEquidistant *lambda_0 phi_0 x y*
::mapproj::toLambertAzimuthalEqualArea *lambda_0 phi_0 lambda phi*
::mapproj::fromLambertAzimuthalEqualArea *lambda_0 phi_0 x y*
::mapproj::toHammer *lambda_0 lambda phi*
::mapproj::fromHammer *lambda_0 x y*
::mapproj::toConicEquidistant *lambda_0 phi_0 phi_1 phi_2 lambda phi*
::mapproj::fromConicEquidistant *lambda_0 phi_0 phi_1 phi_2 x y*
::mapproj::toAlbersEqualAreaConic *lambda_0 phi_0 phi_1 phi_2 lambda phi*
::mapproj::fromAlbersEqualAreaConic *lambda_0 phi_0 phi_1 phi_2 x y*
::mapproj::toLambertConformalConic *lambda_0 phi_0 phi_1 phi_2 lambda phi*
::mapproj::fromLambertConformalConic *lambda_0 phi_0 phi_1 phi_2 x y*
::mapproj::toLambertCylindricalEqualArea *lambda_0 phi_0 lambda phi*
::mapproj::fromLambertCylindricalEqualArea *lambda_0 phi_0 x y*
::mapproj::toBehrmann *lambda_0 phi_0 lambda phi*
::mapproj::fromBehrmann *lambda_0 phi_0 x y*
::mapproj::toTrystanEdwards *lambda_0 phi_0 lambda phi*
::mapproj::fromTrystanEdwards *lambda_0 phi_0 x y*
::mapproj::toHoboDyer *lambda_0 phi_0 lambda phi*
::mapproj::fromHoboDyer *lambda_0 phi_0 x y*
::mapproj::toGallPeters *lambda_0 phi_0 lambda phi*
::mapproj::fromGallPeters *lambda_0 phi_0 x y*
::mapproj::toBalthasart *lambda_0 phi_0 lambda phi*
::mapproj::fromBalthasart *lambda_0 phi_0 x y*

Description

The **mapproj** package provides a set of procedures for converting between world co-ordinates (latitude and longitude) and map co-ordinates on a number of different map projections.

Commands

The following commands convert between world co-ordinates and map co-ordinates:

::mapproj::toPlateCarree *lambda_0 phi_0 lambda phi*
Converts to the *plate carrée* (cylindrical equidistant) projection.

::mapproj::fromPlateCarree *lambda_0 phi_0 x y*
Converts from the *plate carrée* (cylindrical equidistant) projection.

::mapproj::toCylindricalEqualArea *lambda_0 phi_0 lambda phi*
Converts to the cylindrical equal-area projection.

::mapproj::fromCylindricalEqualArea *lambda_0 phi_0 x y*
Converts from the cylindrical equal-area projection.

::mapproj::toMercator *lambda_0 phi_0 lambda phi*
Converts to the Mercator (cylindrical conformal) projection.

::mapproj::fromMercator *lambda_0 phi_0 x y*
Converts from the Mercator (cylindrical conformal) projection.

::mapproj::toMillerCylindrical *lambda_0 lambda phi*
Converts to the Miller Cylindrical projection.

::mapproj::fromMillerCylindrical *lambda_0 x y*
Converts from the Miller Cylindrical projection.

::mapproj::toSinusoidal *lambda_0 phi_0 lambda phi*
Converts to the sinusoidal (Sanson-Flamsteed) projection. projection.

::mapproj::fromSinusoidal *lambda_0 phi_0 x y*
Converts from the sinusoidal (Sanson-Flamsteed) projection. projection.

::mapproj::toMollweide *lambda_0 lambda phi*
Converts to the Mollweide projection.

::mapproj::fromMollweide *lambda_0 x y*
Converts from the Mollweide projection.

::mapproj::toEckertIV *lambda_0 lambda phi*
Converts to the Eckert IV projection.

::mapproj::fromEckertIV *lambda_0 x y*
Converts from the Eckert IV projection.

::mapproj::toEckertVI *lambda_0 lambda phi*
Converts to the Eckert VI projection.

::mapproj::fromEckertVI *lambda_0 x y*
Converts from the Eckert VI projection.

::mapproj::toRobinson *lambda_0 lambda phi*
Converts to the Robinson projection.

::mapproj::fromRobinson *lambda_0 x y*
Converts from the Robinson projection.

::mapproj::toCassini *lambda_0 phi_0 lambda phi*
Converts to the Cassini (transverse cylindrical equidistant) projection.

::mapproj::fromCassini *lambda_0 phi_0 x y*
Converts from the Cassini (transverse cylindrical equidistant) projection.

::mapproj::toPeirceQuincuncial *lambda_0 lambda phi*
Converts to the Peirce Quincuncial Projection.

::mapproj::fromPeirceQuincuncial *lambda_0 x y*
Converts from the Peirce Quincuncial Projection.

::mapproj::toOrthographic *lambda_0 phi_0 lambda phi*
Converts to the orthographic projection.

::mapproj::fromOrthographic *lambda_0 phi_0 x y*

Converts from the orthographic projection.

::mapproj::toStereographic *lambda_0 phi_0 lambda phi*
Converts to the stereographic (azimuthal conformal) projection.

::mapproj::fromStereographic *lambda_0 phi_0 x y*
Converts from the stereographic (azimuthal conformal) projection.

::mapproj::toGnomonic *lambda_0 phi_0 lambda phi*
Converts to the gnomonic projection.

::mapproj::fromGnomonic *lambda_0 phi_0 x y*
Converts from the gnomonic projection.

::mapproj::toAzimuthalEquidistant *lambda_0 phi_0 lambda phi*
Converts to the azimuthal equidistant projection.

::mapproj::fromAzimuthalEquidistant *lambda_0 phi_0 x y*
Converts from the azimuthal equidistant projection.

::mapproj::toLambertAzimuthalEqualArea *lambda_0 phi_0 lambda phi*
Converts to the Lambert azimuthal equal-area projection.

::mapproj::fromLambertAzimuthalEqualArea *lambda_0 phi_0 x y*
Converts from the Lambert azimuthal equal-area projection.

::mapproj::toHammer *lambda_0 lambda phi*
Converts to the Hammer projection.

::mapproj::fromHammer *lambda_0 x y*
Converts from the Hammer projection.

::mapproj::toConicEquidistant *lambda_0 phi_0 phi_1 phi_2 lambda phi*
Converts to the conic equidistant projection.

::mapproj::fromConicEquidistant *lambda_0 phi_0 phi_1 phi_2 x y*
Converts from the conic equidistant projection.

::mapproj::toAlbersEqualAreaConic *lambda_0 phi_0 phi_1 phi_2 lambda phi*
Converts to the Albers equal-area conic projection.

::mapproj::fromAlbersEqualAreaConic *lambda_0 phi_0 phi_1 phi_2 x y*
Converts from the Albers equal-area conic projection.

::mapproj::toLambertConformalConic *lambda_0 phi_0 phi_1 phi_2 lambda phi*
Converts to the Lambert conformal conic projection.

::mapproj::fromLambertConformalConic *lambda_0 phi_0 phi_1 phi_2 x y*
Converts from the Lambert conformal conic projection.

Among the cylindrical equal-area projections, there are a number of choices of standard parallels that have names:

::mapproj::toLambertCylindricalEqualArea *lambda_0 phi_0 lambda phi*
Converts to the Lambert cylindrical equal area projection. (standard parallel is the Equator.)

::mapproj::fromLambertCylindricalEqualArea *lambda_0 phi_0 x y*
Converts from the Lambert cylindrical equal area projection. (standard parallel is the Equator.)

::mapproj::toBehrmann *lambda_0 phi_0 lambda phi*
Converts to the Behrmann cylindrical equal area projection. (standard parallels are 30 degrees North and South)

::mapproj::fromBehrmann *lambda_0 phi_0 x y*

Converts from the Behrmann cylindrical equal area projection. (standard parallels are 30 degrees North and South.)

::mapproj::toTrystanEdwards *lambda_0 phi_0 lambda phi*
Converts to the Trystan Edwards cylindrical equal area projection. (standard parallels are 37.4 degrees North and South)

::mapproj::fromTrystanEdwards *lambda_0 phi_0 x y*
Converts from the Trystan Edwards cylindrical equal area projection. (standard parallels are 37.4 degrees North and South.)

::mapproj::toHoboDyer *lambda_0 phi_0 lambda phi*
Converts to the Hobo-Dyer cylindrical equal area projection. (standard parallels are 37.5 degrees North and South)

::mapproj::fromHoboDyer *lambda_0 phi_0 x y*
Converts from the Hobo-Dyer cylindrical equal area projection. (standard parallels are 37.5 degrees North and South.)

::mapproj::toGallPeters *lambda_0 phi_0 lambda phi*
Converts to the Gall-Peters cylindrical equal area projection. (standard parallels are 45 degrees North and South)

::mapproj::fromGallPeters *lambda_0 phi_0 x y*
Converts from the Gall-Peters cylindrical equal area projection. (standard parallels are 45 degrees North and South.)

::mapproj::toBalthasart *lambda_0 phi_0 lambda phi*
Converts to the Balthasart cylindrical equal area projection. (standard parallels are 50 degrees North and South)

::mapproj::fromBalthasart *lambda_0 phi_0 x y*
Converts from the Balthasart cylindrical equal area projection. (standard parallels are 50 degrees North and South.)

Arguments

The following arguments are accepted by the projection commands:

lambda
Longitude of the point to be projected, in degrees.

phi
Latitude of the point to be projected, in degrees.

lambda_0
Longitude of the center of the sheet, in degrees. For many projections, this figure is also the reference meridian of the projection.

phi_0
Latitude of the center of the sheet, in degrees. For the azimuthal projections, this figure is also the latitude of the center of the projection.

phi_1
Latitude of the first reference parallel, for projections that use reference parallels.

phi_2
Latitude of the second reference parallel, for projections that use reference parallels.

x

X co-ordinate of a point on the map, in units of Earth radii.

y

Y co-ordinate of a point on the map, in units of Earth radii.

Results

For all of the procedures whose names begin with 'to', the return value is a list comprising an *x* co-ordinate and a *y* co-ordinate. The co-ordinates are relative to the center of the map sheet to be drawn, measured in Earth radii at the reference location on the map. For all of the functions whose names begin with 'from', the return value is a list comprising the longitude and latitude, in degrees.

Choosing a projection

This package offers a great many projections, because no single projection is appropriate to all maps. This section attempts to provide guidance on how to choose a projection.

First, consider the type of data that you intend to display on the map. If the data are *directional* (*e.g.,* winds, ocean currents, or magnetic fields) then you need to use a projection that preserves angles; these are known as *conformal* projections. Conformal projections include the Mercator, the Albers azimuthal equal-area, the stereographic, and the Peirce Quincuncial projection. If the data are *thematic*, describing properties of land or water, such as temperature, population density, land use, or demographics; then you need a projection that will show these data with the areas on the map proportional to the areas in real life. These so-called *equal area* projections include the various cylindrical equal area projections, the sinusoidal projection, the Lambert azimuthal equal-area projection, the Albers equal-area conic projection, and several of the world-map projections (Miller Cylindrical, Mollweide, Eckert IV, Eckert VI, Robinson, and Hammer). If the significant factor in your data is distance from a central point or line (such as air routes), then you will do best with an *equidistant* projection such as *plate carrée*, Cassini, azimuthal equidistant, or conic equidistant. If direction from a central point is a critical factor in your data (for instance, air routes, radio antenna pointing), then you will almost surely want to use one of the azimuthal projections. Appropriate choices are azimuthal equidistant, azimuthal equal-area, stereographic, and perhaps orthographic.

Next, consider how much of the Earth your map will cover, and the general shape of the area of interest. For maps of the entire Earth, the cylindrical equal area, Eckert IV and VI, Mollweide, Robinson, and Hammer projections are good overall choices. The Mercator projection is traditional, but the extreme distortions of area at high latitudes make it a poor choice unless a conformal projection is required. The Peirce projection is a better choice of conformal projection, having less distortion of landforms. The Miller Cylindrical is a compromise designed to give shapes similar to the traditional Mercator, but with less polar stretching. The Peirce Quincuncial projection shows all the continents with acceptable distortion if a reference meridian close to +20 degrees is chosen. The Robinson projection yields attractive maps for things like political divisions, but should be avoided in presenting scientific data, since other projections have moe useful geometric properties.

If the map will cover a hemisphere, then choose stereographic, azimuthal-equidistant, Hammer, or Mollweide projections; these all project the hemisphere into a circle.

If the map will cover a large area (at least a few hundred km on a side), but less than a hemisphere, then you have several choices. Azimuthal projections are usually good (choose

stereographic, azimuthal equidistant, or Lambert azimuthal equal-area according to whether shapes, distances from a central point, or areas are important). Azimuthal projections (and possibly the Cassini projection) are the only really good choices for mapping the polar regions.

If the large area is in one of the temperate zones and is round or has a primarily east-west extent, then the conic projections are good choices. Choose the Lambert conformal conic, the conic equidistant, or the Albers equal-area conic according to whether shape, distance, or area are the most important parameters. For any of these, the reference parallels should be chosen at approximately 1/6 and 5/6 of the range of latitudes to be displayed. For instance, maps of the 48 coterminous United States are attractive with reference parallels of 28.5 and 45.5 degrees.

If the large area is equatorial and is round or has a primarily east-west extent, then the Mercator projection is a good choice for a conformal projection; Lambert cylindrical equal-area and sinusoidal projections are good equal-area projections; and the *plate carrée* is a good equidistant projection.

Large areas having a primarily North-South aspect, particularly those spanning the Equator, need some other choices. The Cassini projection is a good choice for an equidistant projection (for instance, a Cassini projection with a central meridian of 80 degrees West produces an attractive map of the Americas). The cylindrical equal-area, Albers equal-area conic, sinusoidal, Mollweide and Hammer projections are possible choices for equal-area projections. A good conformal projection in this situation is the Transverse Mercator, which alas, is not yet implemented.

Small areas begin to get into a realm where the ellipticity of the Earth affects the map scale. This package does not attempt to handle accurate mapping for large-scale topographic maps. If slight scale errors are acceptable in your application, then any of the projections appropriate to large areas should work for small ones as well.

There are a few projections that are included for their special properties. The orthographic projection produces views of the Earth as seen from space. The gnomonic projection produces a map on which all great circles (the shortest distance between two points on the Earth's surface) are rendered as straight lines. While this projection is useful for navigational planning, it has extreme distortions of shape and area, and can display only a limited area of the Earth (substantially less than a hemisphere).

Copyright


```
Copyright (c) 2007 Kevin B. Kenny <kennykb@acm.org>
```


markdown

markdown

Keywords: markdown

Name

markdown - Converts Markdown text to HTML

Synopsis

package require Tcl 8.5 9

package require Markdown 1.2.4
package require textutil ?**0.8**?
::Markdown::convert *markdown*
::Markdown::register *langspec converter*
::Markdown::get_lang_counter
::Markdown::reset_lang_counter

Description

The package **Markdown** provides a command to convert Markdown annotated text into HMTL.

::Markdown::convert *markdown*
This command takes in a block of Markdown text, and returns a block of HTML.

The converter supports two types of syntax highlighting for fenced code blocks: highlighting via a registered converter (see **::Markdown::register**), or pure JavaScript highlighting, e.g. via "highlight.js", where the language specifier used in the markup is set as CSS class of the "code" element in the returned markup.

::Markdown::register *langspec converter*
Register a language specific converter for prettifying a code block (e.g. syntax highlighting). Markdown supports fenced code blocks with an optional language specifier (e.g. "tcl"). When the markdown parser processes such a code block and a converter for the specified langspec is registered, the converter is called with the raw code block as argument. The converter is supposed to return the markup of the code block as result. The specified converter can be an arbitrary Tcl command, the raw text block is added as last argument upon invocation.

::Markdown::get_lang_counter
Return a dict of language specifier and number of occurrences in fenced code blocks. This function can be used e.g. to detect, whether some CSS or JavaScript headers should be included for rendering without the need of postprocessing the rendered result.

::Markdown::reset_lang_counter
Reset the language counters.

Supported markdown syntax

This markdown converter supports the original markdown by Gruber and Swartz(see their syntax page for details):

- paragraphs
- atx- and setext-style headers
- blockquotes
- emphasis and strong emphasis
- unordered and ordered lists
- inline-style, reference-style and automatic links
- inline- and reference-style images
- inline code
- code blocks (with four indent spaces or one tab)
- inline HTML
- backslash escapes
- horizontal rules

In addition, the following extended markdown sytax is supported, taken from PHP Markdown Extra and GFM (Github Flavoured Markdown):

- pipe tables
- fenced code blocks (with an optional language specifier)

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *textutil* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Text processing

math

bigfloat

Keywords: bigfloat, computations, floating-point, interval, math, multiprecision, tcl

Name

math::bigfloat - Arbitrary precision floating-point numbers

Synopsis

package require Tcl 8.5 9
package require math::bigfloat ?**2.0.6**?
fromstr *number* ?*trailingZeros*?
tostr ?**-nosci**? *number*
fromdouble *double* ?*decimals*?
todouble *number*
isInt *number*
isFloat *number*
int2float *integer* ?*decimals*?
add *x y*
sub *x y*
mul *x y*
div *x y*
mod *x y*
abs *x*
opp *x*
pow *x n*
iszero *x*
equal *x y*
compare *x y*

sqrt *x*
log *x*
exp *x*
cos *x*
sin *x*
tan *x*
cotan *x*
acos *x*
asin *x*
atan *x*
cosh *x*
sinh *x*
tanh *x*
pi *n*
rad2deg *radians*
deg2rad *degrees*
round *x*
ceil *x*
floor *x*

Description

The bigfloat package provides arbitrary precision floating-point math capabilities to the Tcl language. It is designed to work with Tcl 8.5, but for Tcl 8.4 is provided an earlier version of this package. See What about Tcl 8.4 ? for more explanations. By convention, we will talk about the numbers treated in this library as:

- BigFloat for floating-point numbers of arbitrary length.
- integers for arbitrary length signed integers, just as basic integers since Tcl 8.5.

Each BigFloat is an interval, namely [*m-d, m+d*], where *m* is the mantissa and *d* the uncertainty, representing the limitation of that number's precision. This is why we call such mathematics *interval computations*. Just take an example in physics: when you measure a temperature, not all digits you read are *significant*. Sometimes you just cannot trust all digits - not to mention if doubles (f.p. numbers) can handle all these digits. BigFloat can handle this problem - trusting the digits you get - plus the ability to store numbers with an arbitrary precision. BigFloats are internally represented at Tcl lists: this package provides a set of procedures operating against the internal representation in order to:

- perform math operations on BigFloats and (optionnaly) with integers.
- convert BigFloats from their internal representations to strings, and vice versa.

Introduction

fromstr *number* ?*trailingZeros*?

Converts *number* into a BigFloat. Its precision is at least the number of digits provided by *number*. If the *number* contains only digits and eventually a minus sign, it is considered as an integer. Subsequently, no conversion is done at all.

trailingZeros - the number of zeros to append at the end of the floating-point number to get more precision. It cannot be applied to an integer.

```
# x and y are BigFloats : the first string contained a dot, and the
    second an e sign
set x [fromstr -1.000000]
set y [fromstr 2000e30]
# let's see how we get integers
set t 20000000000000
# the old way (package 1.2) is still supported for backwards
    compatibility :
set m [fromstr 10000000000]
# but we do not need fromstr for integers anymore
set n -39
# t, m and n are integers
```

The *number*'s last digit is considered by the procedure to be true at +/-1, For example, 1.00 is the interval [0.99, 1.01], and 0.43 the interval [0.42, 0.44]. The Pi constant may be approximated by the number "3.1415". This string could be considered as the interval [3.1414 , 3.1416] by **fromstr**. So, when you mean 1.0 as a double, you may have to write 1.000000 to get enough precision. To learn more about this subject, see Precision.

For example:

```
set x [fromstr 1.0000000000]
# the next line does the same, but smarter
set y [fromstr 1. 10]
```

tostr ?-nosci? *number*

Returns a string form of a BigFloat, in which all digits are exacts. *All exact digits* means a rounding may occur, for example to zero, if the uncertainty interval does not clearly show the true digits. *number* may be an integer, causing the command to return exactly the input argument. With the **-nosci** option, the number returned is never shown in scientific notation, i.e. not like '3.4523e+5' but like '345230.'.

```
puts [tostr [fromstr 0.99999]] ;# 1.0000
puts [tostr [fromstr 1.00001]] ;# 1.0000
puts [tostr [fromstr 0.002]] ;# 0.e-2
```

See Precision for that matter. See also **iszero** for how to detect zeros, which is useful when performing a division.

fromdouble *double* ?*decimals*?

Converts a double (a simple floating-point value) to a BigFloat, with exactly *decimals* digits. Without the *decimals* argument, it behaves like **fromstr**. Here, the only important feature you might care of is the ability to create BigFloats with a fixed number of *decimals*.

```
tostr [fromstr 1.111 4]
# returns : 1.111000 (3 zeros)
tostr [fromdouble 1.111 4]
# returns : 1.111
```

todouble *number*
Returns a double, that may be used in *expr*, from a BigFloat.

isInt *number*
Returns 1 if *number* is an integer, 0 otherwise.

isFloat *number*
Returns 1 if *number* is a BigFloat, 0 otherwise.

int2float *integer* ?*decimals*?
Converts an integer to a BigFloat with *decimals* trailing zeros. The default, and minimal, number of *decimals* is 1. When converting back to string, one decimal is lost:

```
set n 10
set x [int2float $n]; # like fromstr 10.0
puts [tostr $x]; # prints "10."
set x [int2float $n 3]; # like fromstr 10.000
puts [tostr $x]; # prints "10.00"
```

Arithmetics

add *x y*
see below ...

sub *x y*
see below ...

mul *x y*
Return the sum, difference and product of *x* by *y*. *x* - may be either a BigFloat or an integer *y* - may be either a BigFloat or an integer When both are integers, these commands behave like **expr**.

div *x y*
see below ...

mod *x y*
Return the quotient and the rest of *x* divided by *y*. Each argument (*x* and *y*) can be either a BigFloat or an integer, but you cannot divide an integer by a BigFloat Divide by zero throws an error.

abs *x*
Returns the absolute value of *x*

opp *x*
Returns the opposite of *x*

pow *x n*
Returns *x* taken to the *n*th power. It only works if *n* is an integer. *x* might be a BigFloat or an integer.

Comparisons

iszero *x*
Returns 1 if *x* is:
- a BigFloat close enough to zero to raise "divide by zero".
- the integer 0.

See here how numbers that are close to zero are converted to strings:

```
tostr [fromstr 0.001] ; # -> 0.e-2
tostr [fromstr 0.000000] ; # -> 0.e-5
tostr [fromstr -0.000001] ; # -> 0.e-5
tostr [fromstr 0.0] ; # -> 0.
tostr [fromstr 0.002] ; # -> 0.e-2

set a [fromstr 0.002] ; # uncertainty interval : 0.001, 0.003
tostr  $a ; # 0.e-2
iszero $a ; # false

set a [fromstr 0.001] ; # uncertainty interval : 0.000, 0.002
tostr  $a ; # 0.e-2
iszero $a ; # true
```

equal *x y*
Returns 1 if *x* and *y* are equal, 0 elsewhere.

compare *x y*
Returns 0 if both BigFloat arguments are equal, 1 if *x* is greater than *y*, and -1 if *x* is lower than *y*. You would not be able to compare an integer to a BigFloat: the operands should be both BigFloats, or both integers.

Analysis

sqrt *x*
see below ...

log *x*
see below ...

exp *x*
see below ...

cos *x*
see below ...

sin *x*
see below ...

tan *x*
see below ...

cotan *x*
see below ...

acos *x*
see below ...

asin *x*
see below ...

atan *x*
see below ...

cosh *x*

see below ...

sinh *x*

see below ...

tanh *x*

The above functions return, respectively, the following: square root, logarithm, exponential, cosine, sine, tangent, cotangent, arc cosine, arc sine, arc tangent, hyperbolic cosine, hyperbolic sine, hyperbolic tangent, of a BigFloat named *x*.

pi *n*

Returns a BigFloat representing the Pi constant with *n* digits after the dot. *n* is a positive integer.

rad2deg *radians*

see below ...

deg2rad *degrees*

radians - angle expressed in radians (BigFloat)

degrees - angle expressed in degrees (BigFloat)

Convert an angle from radians to degrees, and *vice versa*.

Rounding

round *x*

see below ...

ceil *x*

see below ...

floor *x*

The above functions return the *x* BigFloat, rounded like with the same mathematical function in *expr*, and returns it as an integer.

Precision

How do conversions work with precision ?

- When a BigFloat is converted from string, the internal representation holds its uncertainty as 1 at the level of the last digit.
- During computations, the uncertainty of each result is internally computed the closest to the reality, thus saving the memory used.
- When converting back to string, the digits that are printed are not subject to uncertainty. However, some rounding is done, as not doing so causes severe problems.

Uncertainties are kept in the internal representation of the number; it is recommended to use **tostr** only for outputting data (on the screen or in a file), and NEVER call **fromstr** with the result of **tostr**. It is better to always keep operands in their internal representation. Due to the internals of this library, the uncertainty interval may be slightly wider than expected, but this should not cause false digits.

Now you may ask this question: What precision am I going to get after calling add, sub, mul or div? First you set a number from the string representation and, by the way, its uncertainty is set:

```
set a [fromstr 1.230]
# $a belongs to [1.229, 1.231]
set a [fromstr 1.000]
# $a belongs to [0.999, 1.001]
# $a has a relative uncertainty of 0.1% : 0.001(the
    uncertainty)/1.000(the medium value)
```

The uncertainty of the sum, or the difference, of two numbers, is the sum of their respective uncertainties.

```
set a [fromstr 1.230]
set b [fromstr 2.340]
set sum [add $a $b]]
# the result is : [3.568, 3.572] (the last digit is known with an
    uncertainty of 2)
tostr $sum ; # 3.57
```

But when, for example, we add or substract an integer to a BigFloat, the relative uncertainty of the result is unchanged. So it is desirable not to convert integers to BigFloats:

```
set a [fromstr 0.999999999]
# now something dangerous
set b [fromstr 2.000]
# the result has only 3 digits
tostr [add $a $b]

# how to keep precision at its maximum
puts [tostr [add $a 2]]
```

For multiplication and division, the relative uncertainties of the product or the quotient, is the sum of the relative uncertainties of the operands. Take care of division by zero: check each divider with **iszero**.

```
set num [fromstr 4.00]
set denom [fromstr 0.01]

puts [iszero $denom];# true
set quotient [div $num $denom];# error : divide by zero

# opposites of our operands
puts [compare $num [opp $num]]; # 1
puts [compare $denom [opp $denom]]; # 0 !!!
# No suprise ! 0 and its opposite are the same...
```

Effects of the precision of a number considered equal to zero to the cos function:

```
puts [tostr [cos [fromstr 0. 10]]]; # -> 1.000000000
puts [tostr [cos [fromstr 0. 5]]]; # -> 1.0000
puts [tostr [cos [fromstr 0e-10]]]; # -> 1.000000000
puts [tostr [cos [fromstr 1e-10]]]; # -> 1.000000000
```

BigFloats with different internal representations may be converted to the same string.

For most analysis functions (cosine, square root, logarithm, etc.), determining the precision of the result is difficult. It seems however that in many cases, the loss of precision in the result is of one or two digits. There are some exceptions: for example,

```
tostr [exp [fromstr 100.0 10]]
# returns : 2.688117142e+43 which has only 10 digits of precision,
    although the entry
# has 14 digits of precision.
```

What about Tcl 8.4 ?

If your setup do not provide Tcl 8.5 but supports 8.4, the package can still be loaded, switching back to *math::bigfloat* 1.2. Indeed, an important function introduced in Tcl 8.5 is required - the ability to handle bignums, that we can do with **expr**. Before 8.5, this ability was provided by several packages, including the pure-Tcl *math::bignum* package provided by *tcllib*. In this case, all you need to know, is that arguments to the commands explained here, are expected to be in their internal representation. So even with integers, you will need to call **fromstr** and **tostr** in order to convert them between string and internal representations.

```
#
# with Tcl 8.5
# ============
set a [pi 20]
# round returns an integer and 'everything is a string' applies to
    integers
# whatever big they are
puts [round [mul $a 10000000000]]
#
# the same with Tcl 8.4
# ====================
set a [pi 20]
# bignums (arbitrary length integers) need a conversion hook
set b [fromstr 10000000000]
# round returns a bignum:
# before printing it, we need to convert it with 'tostr'
puts [tostr [round [mul $a $b]]]
```

Namespaces and other packages

We have not yet discussed about namespaces because we assumed that you had imported public commands into the global namespace, like this:

```
namespace import ::math::bigfloat::*
```

If you matter much about avoiding names conflicts, I considere it should be resolved by the following:

```
package require math::bigfloat
# beware: namespace ensembles are not available in Tcl 8.4
namespace eval ::math::bigfloat {namespace ensemble create -command
    ::bigfloat}
# from now, the bigfloat command takes as subcommands all original
    math::bigfloat::* commands
set a [bigfloat sub [bigfloat fromstr 2.000] [bigfloat fromstr 0.530]]
puts [bigfloat tostr $a]
```

Examples

Guess what happens when you are doing some astronomy. Here is an example:

```
# convert acurrate angles with a millisecond-rated accuracy
proc degree-angle {degrees minutes seconds milliseconds} {
    set result 0
    set div 1
    foreach factor {1 1000 60 60} var [list $milliseconds $seconds
    $minutes $degrees] {
        # we convert each entry var into milliseconds
        set div [expr {$div*$factor}]
        incr result [expr {$var*$div}]
    }
    return [div [int2float $result] $div]
}
# load the package
package require math::bigfloat
namespace import ::math::bigfloat::*
# work with angles : a standard formula for navigation (taking
    bearings)
set angle1 [deg2rad [degree-angle 20 30 40   0]]
set angle2 [deg2rad [degree-angle 21  0 50 500]]
set opposite3 [deg2rad [degree-angle 51  0 50 500]]
set sinProduct [mul [sin $angle1] [sin $angle2]]
set cosProduct [mul [cos $angle1] [cos $angle2]]
set angle3 [asin [add [mul $sinProduct [cos $opposite3]] $cosProduct]]
puts "angle3 : [tostr [rad2deg $angle3]]"
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *math :: bignum :: float* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be

made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Mathematics

Copyright

```
Copyright (c) 2004-2008, by Stephane Arnold <stephanearnold at yahoo
    dot fr>
```


bignum
Keywords: bignum, bignums, math, multiprecision, tcl

Name
math::bignum - Arbitrary precision integer numbers

Synopsis
package require Tcl ?**8.5 9**?
package require math::bignum ?**3.1.2**?
::math::bignum::fromstr *string* ?*radix*?
::math::bignum::tostr *bignum* ?*radix*?
::math::bignum::sign *bignum*
::math::bignum::abs *bignum*
::math::bignum::cmp *a b*
::math::bignum::iszero *bignum*
::math::bignum::lt *a b*
::math::bignum::le *a b*
::math::bignum::gt *a b*
::math::bignum::ge *a b*
::math::bignum::eq *a b*
::math::bignum::ne *a b*
::math::bignum::isodd *bignum*
::math::bignum::iseven *bignum*
::math::bignum::add *a b*
::math::bignum::sub *a b*
::math::bignum::mul *a b*
::math::bignum::divqr *a b*
::math::bignum::div *a b*
::math::bignum::rem *a b*
::math::bignum::mod *n m*
::math::bignum::pow *base exp*
::math::bignum::powm *base exp m*
::math::bignum::sqrt *bignum*
::math::bignum::rand *bits*
::math::bignum::lshift *bignum bits*
::math::bignum::rshift *bignum bits*

::math::bignum::bitand *a b*
::math::bignum::bitor *a b*
::math::bignum::bitxor *a b*
::math::bignum::setbit *bignumVar bit*
::math::bignum::clearbit *bignumVar bit*
::math::bignum::testbit *bignum bit*
::math::bignum::bits *bignum*

Description

The bignum package provides arbitrary precision integer math (also known as "big numbers") capabilities to the Tcl language. Big numbers are internally represented at Tcl lists: this package provides a set of procedures operating against the internal representation in order to:

- perform math operations
- convert bignums from the internal representation to a string in the desired radix and vice versa.

But the two constants "0" and "1" are automatically converted to the internal representation, in order to easily compare a number to zero, or increment a big number.

The bignum interface is opaque, so operations on bignums that are not returned by procedures in this package (but created by hand) may lead to unspecified behaviours. It's safe to treat bignums as pure values, so there is no need to free a bignum, or to duplicate it via a special operation.

Examples

This section shows some simple example. This library being just a way to perform math operations, examples may be the simplest way to learn how to work with it. Consult the API section of this man page for information about individual procedures.

```
    package require math::bignum

    # Multiplication of two bignums
    set a [::math::bignum::fromstr 88888881111111]
    set b [::math::bignum::fromstr 22222220000000]
    set c [::math::bignum::mul $a $b]
    puts [::math::bignum::tostr $c] ; # => will output
    1975308271604953086420000000
    set c [::math::bignum::sqrt $c]
    puts [::math::bignum::tostr $c] ; # => will output 44444440277777

    # From/To string conversion in different radix
    set a [::math::bignum::fromstr 11000101010101110010011110101111 2]
    puts [::math::bignum::tostr $a 16] ; # => will output 62ab93d7

    # Factorial example
    proc fact n {
        # fromstr is not needed for 0 and 1
        set z 1
```

```
        for {set i 2} {$i <= $n} {incr i} {
            set z [::math::bignum::mul $z [::math::bignum::fromstr $i]]
        }
        return $z
    }

    puts [::math::bignum::tostr [fact 100]]
```

API

::math::bignum::fromstr *string* ?*radix*?
Convert *string* into a bignum. If *radix* is omitted or zero, the string is interpreted in hex if prefixed with *0x*, in octal if prefixed with *ox*, in binary if it's pefixed with *bx*, as a number in radix 10 otherwise. If instead the *radix* argument is specified in the range 2-36, the *string* is interpreted in the given radix. Please note that this conversion is not needed for two constants: *0* and *1*. (see the example)

::math::bignum::tostr *bignum* ?*radix*?
Convert *bignum* into a string representing the number in the specified radix. If *radix* is omitted, the default is 10.

::math::bignum::sign *bignum*
Return the sign of the bignum. The procedure returns 0 if the number is positive, 1 if it's negative.

::math::bignum::abs *bignum*
Return the absolute value of the bignum.

::math::bignum::cmp *a b*
Compare the two bignums a and b, returning *0* if *a* == *b*, *1* if *a* > *b*, and *-1* if *a* < *b*.

::math::bignum::iszero *bignum*
Return true if *bignum* value is zero, otherwise false is returned.

::math::bignum::lt *a b*
Return true if *a* < *b*, otherwise false is returned.

::math::bignum::le *a b*
Return true if *a* <= *b*, otherwise false is returned.

::math::bignum::gt *a b*
Return true if *a* > *b*, otherwise false is returned.

::math::bignum::ge *a b*
Return true if *a* >= *b*, otherwise false is returned.

::math::bignum::eq *a b*
Return true if *a* == *b*, otherwise false is returned.

::math::bignum::ne *a b*
Return true if *a!*= *b*, otherwise false is returned.

::math::bignum::isodd *bignum*
Return true if *bignum* is odd.

::math::bignum::iseven *bignum*
Return true if *bignum* is even.

::math::bignum::add *a b*

Return the sum of the two bignums *a* and *b*.

::math::bignum::sub *a b*
Return the difference of the two bignums *a* and *b*.

::math::bignum::mul *a b*
Return the product of the two bignums *a* and *b*. The implementation uses Karatsuba multiplication if both the numbers are bigger than a given threshold, otherwise the direct algorith is used.

::math::bignum::divqr *a b*
Return a two-elements list containing as first element the quotient of the division between the two bignums *a* and *b*, and the remainder of the division as second element.

::math::bignum::div *a b*
Return the quotient of the division between the two bignums *a* and *b*.

::math::bignum::rem *a b*
Return the remainder of the division between the two bignums *a* and *b*.

::math::bignum::mod *n m*
Return *n* modulo *m*. This operation is called modular reduction.

::math::bignum::pow *base exp*
Return *base* raised to the exponent *exp*.

::math::bignum::powm *base exp m*
Return *base* raised to the exponent *exp*, modulo *m*. This function is often used in the field of cryptography.

::math::bignum::sqrt *bignum*
Return the integer part of the square root of *bignum*

::math::bignum::rand *bits*
Return a random number of at most *bits* bits. The returned number is internally generated using Tcl's *expr rand()* function and is not suitable where an unguessable and cryptographically secure random number is needed.

::math::bignum::lshift *bignum bits*
Return the result of left shifting *bignum*'s binary representation of *bits* positions on the left. This is equivalent to multiplying by 2^*bits* but much faster.

::math::bignum::rshift *bignum bits*
Return the result of right shifting *bignum*'s binary representation of *bits* positions on the right. This is equivalent to dividing by *2^bits* but much faster.

::math::bignum::bitand *a b*
Return the result of doing a bitwise AND operation on a and b. The operation is restricted to positive numbers, including zero. When negative numbers are provided as arguments the result is undefined.

::math::bignum::bitor *a b*
Return the result of doing a bitwise OR operation on a and b. The operation is restricted to positive numbers, including zero. When negative numbers are provided as arguments the result is undefined.

::math::bignum::bitxor *a b*
Return the result of doing a bitwise XOR operation on a and b. The operation is restricted to positive numbers, including zero. When negative numbers are provided as arguments the

result is undefined.

::math::bignum::setbit *bignumVar bit*
Set the bit at *bit* position to 1 in the bignum stored in the variable *bignumVar*. Bit 0 is the least significant.

::math::bignum::clearbit *bignumVar bit*
Set the bit at *bit* position to 0 in the bignum stored in the variable *bignumVar*. Bit 0 is the least significant.

::math::bignum::testbit *bignum bit*
Return true if the bit at the *bit* position of *bignum* is on, otherwise false is returned. If *bit* is out of range, it is considered as set to zero.

::math::bignum::bits *bignum*
Return the number of bits needed to represent bignum in radix 2.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *math :: bignum* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Mathematics

Copyright


```
Copyright (c) 2004 Salvatore Sanfilippo <antirez at invece dot org>
Copyright (c) 2004 Arjen Markus <arjenmarkus at users dot sourceforge
    dot net>
```


calculus

Keywords: calculus, differential equations, integration, math, roots

Name

math::calculus - Integration and ordinary differential equations

Synopsis

package require Tcl 8.5 9
package require math::calculus 1.1
::math::calculus::integral *begin end nosteps func*
::math::calculus::integralExpr *begin end nosteps expression*
::math::calculus::integral2D *xinterval yinterval func*
::math::calculus::integral2D_accurate *xinterval yinterval func*
::math::calculus::integral3D *xinterval yinterval zinterval func*
::math::calculus::integral3D_accurate *xinterval yinterval zinterval func*

::math::calculus::qk15 *xstart xend func nosteps*
::math::calculus::qk15_detailed *xstart xend func nosteps*
::math::calculus::eulerStep *t tstep xvec func*
::math::calculus::heunStep *t tstep xvec func*
::math::calculus::rungeKuttaStep *t tstep xvec func*
::math::calculus::boundaryValueSecondOrder *coeff_func force_func leftbnd rightbnd nostep*
::math::calculus::solveTriDiagonal *acoeff bcoeff ccoeff dvalue*
::math::calculus::newtonRaphson *func deriv initval*
::math::calculus::newtonRaphsonParameters *maxiter tolerance*
::math::calculus::regula_falsi *f xb xe eps*
::math::calculus::root_bisection *f xb xe eps*
::math::calculus::root_secant *f xb xe eps*
::math::calculus::root_brent *f xb xe eps*
::math::calculus::root_chandrupatla *f xb xe eps*

Description

This package implements several simple mathematical algorithms:

- The integration of a function over an interval
- The numerical integration of a system of ordinary differential equations.
- Estimating the root(s) of an equation of one variable.

The package is fully implemented in Tcl. No particular attention has been paid to the accuracy of the calculations. Instead, well-known algorithms have been used in a straightforward manner.

This document describes the procedures and explains their usage.

Procedures

This package defines the following public procedures:

::math::calculus::integral *begin end nosteps func*

Determine the integral of the given function using the Simpson rule. The interval for the integration is [*begin*, *end*]. The remaining arguments are:

nosteps
: Number of steps in which the interval is divided.

func
: Function to be integrated. It should take one single argument.

::math::calculus::integralExpr *begin end nosteps expression*

Similar to the previous proc, this one determines the integral of the given *expression* using the Simpson rule. The interval for the integration is [*begin*, *end*]. The remaining arguments are:

nosteps
: Number of steps in which the interval is divided.

expression
: Expression to be integrated. It should use the variable "x" as the only variable (the "integrate")

::math::calculus::integral2D *xinterval yinterval func*

see below ...

::math::calculus::integral2D_accurate *xinterval yinterval func*

The commands **integral2D** and **integral2D_accurate** calculate the integral of a function of two variables over the rectangle given by the first two arguments, each a list of three items, the start and stop interval for the variable and the number of steps.

The command **integral2D** evaluates the function at the centre of each rectangle, whereas the command **integral2D_accurate** uses a four-point quadrature formula. This results in an exact integration of polynomials of third degree or less.

The function must take two arguments and return the function value.

::math::calculus::integral3D *xinterval yinterval zinterval func*

see below ...

::math::calculus::integral3D_accurate *xinterval yinterval zinterval func*

The commands **integral3D** and **integral3D_accurate** are the three-dimensional equivalent of **integral2D** and **integral3D_accurate**. The function *func* takes three arguments and is integrated over the block in 3D space given by three intervals.

::math::calculus::qk15 *xstart xend func nosteps*

Determine the integral of the given function using the Gauss-Kronrod 15 points quadrature rule. The returned value is the estimate of the integral over the interval [*xstart*, *xend*]. The remaining arguments are:

func

Function to be integrated. It should take one single argument.

?nosteps?

Number of steps in which the interval is divided. Defaults to 1.

::math::calculus::qk15_detailed *xstart xend func nosteps*

Determine the integral of the given function using the Gauss-Kronrod 15 points quadrature rule. The interval for the integration is [*xstart*, *xend*]. The procedure returns a list of four values:

- The estimate of the integral over the specified interval (I).
- An estimate of the absolute error in I.
- The estimate of the integral of the absolute value of the function over the interval.
- The estimate of the integral of the absolute value of the function minus its mean over the interval.

The remaining arguments are:

func

Function to be integrated. It should take one single argument.

?nosteps?

Number of steps in which the interval is divided. Defaults to 1.

::math::calculus::eulerStep *t tstep xvec func*

Set a single step in the numerical integration of a system of differential equations. The method used is Euler's.

t

Value of the independent variable (typically time) at the beginning of the step.

tstep
Step size for the independent variable.

xvec
List (vector) of dependent values

func
Function of t and the dependent values, returning a list of the derivatives of the dependent values. (The lengths of xvec and the return value of "func" must match).

::math::calculus::heunStep *t tstep xvec func*

Set a single step in the numerical integration of a system of differential equations. The method used is Heun's.

t
Value of the independent variable (typically time) at the beginning of the step.

tstep
Step size for the independent variable.

xvec
List (vector) of dependent values

func
Function of t and the dependent values, returning a list of the derivatives of the dependent values. (The lengths of xvec and the return value of "func" must match).

::math::calculus::rungeKuttaStep *t tstep xvec func*

Set a single step in the numerical integration of a system of differential equations. The method used is Runge-Kutta 4th order.

t
Value of the independent variable (typically time) at the beginning of the step.

tstep
Step size for the independent variable.

xvec
List (vector) of dependent values

func
Function of t and the dependent values, returning a list of the derivatives of the dependent values. (The lengths of xvec and the return value of "func" must match).

::math::calculus::boundaryValueSecondOrder *coeff_func force_func leftbnd rightbnd nostep*

Solve a second order linear differential equation with boundary values at two sides. The equation has to be of the form (the "conservative" form):

```
d      dy     d
-- A(x)--  +  -- B(x)y + C(x)y  =  D(x)
dx     dx     dx
```

Ordinarily, such an equation would be written as:

```
        d2y        dy
   a(x)---  + b(x)-- + c(x) y  =  D(x)
        dx2        dx
```

The first form is easier to discretise (by integrating over a finite volume) than the second form. The relation between the two forms is fairly straightforward:

```
   A(x)  =  a(x)
   B(x)  =  b(x) - a'(x)
   C(x)  =  c(x) - B'(x)  =  c(x) - b'(x) + a''(x)
```

Because of the differentiation, however, it is much easier to ask the user to provide the functions A, B and C directly.

coeff_func
: Procedure returning the three coefficients (A, B, C) of the equation, taking as its one argument the x-coordinate.

force_func
: Procedure returning the right-hand side (D) as a function of the x-coordinate.

leftbnd
: A list of two values: the x-coordinate of the left boundary and the value at that boundary.

rightbnd
: A list of two values: the x-coordinate of the right boundary and the value at that boundary.

nostep
: Number of steps by which to discretise the interval. The procedure returns a list of x-coordinates and the approximated values of the solution.

::math::calculus::solveTriDiagonal *acoeff bcoeff ccoeff dvalue*

Solve a system of linear equations Ax = b with A a tridiagonal matrix. Returns the solution as a list.

acoeff
: List of values on the lower diagonal

bcoeff
: List of values on the main diagonal

ccoeff
: List of values on the upper diagonal

dvalue
: List of values on the righthand-side

::math::calculus::newtonRaphson *func deriv initval*

Determine the root of an equation given by

```
    func(x) = 0
```

using the method of Newton-Raphson. The procedure takes the following arguments:

func
: Procedure that returns the value the function at x

deriv
Procedure that returns the derivative of the function at x

initval
Initial value for x

::math::calculus::newtonRaphsonParameters *maxiter tolerance*
Set the numerical parameters for the Newton-Raphson method:

maxiter
Maximum number of iteration steps (defaults to 20)

tolerance
Relative precision (defaults to 0.001)

::math::calculus::regula_falsi *f xb xe eps*
Return an estimate of the zero or one of the zeros of the function contained in the interval [xb,xe]. The error in this estimate is of the order of eps*abs(xe-xb), the actual error may be slightly larger.

The method used is the so-called *regula falsi* or *false position* method. It is a straightforward implementation. The method is robust, but requires that the interval brackets a zero or at least an uneven number of zeros, so that the value of the function at the start has a different sign than the value at the end.

In contrast to Newton-Raphson there is no need for the computation of the function's derivative.

command *f*
Name of the command that evaluates the function for which the zero is to be returned

float *xb*
Start of the interval in which the zero is supposed to lie

float *xe*
End of the interval

float *eps*
Relative allowed error (defaults to 1.0e-4)

::math::calculus::root_bisection *f xb xe eps*
Return an estimate of the zero or one of the zeros of the function contained in the interval [xb,xe]. The error in this estimate is of the order of eps*abs(xe-xb), the actual error may be slightly larger.

The method used is the so-called *bisection*. For properties: see the *regula falsi* procedure.

command *f*
Name of the command that evaluates the function for which the zero is to be returned

float *xb*
Start of the interval in which the zero is supposed to lie

float *xe*
End of the interval

float *eps*
Relative allowed error (defaults to 1.0e-7)

::math::calculus::root_secant *f xb xe eps*

Return an estimate of the zero or one of the zeros of the function contained in the interval [xb,xe]. The error in this estimate is of the order of eps*abs(xe-xb), the actual error may be slightly larger.

The method used is the so-called *secant* method. For properties: see the *regula falsi* procedure. Note that this method is not guaranteed to produce a zero, but it is fast.

command *f*
Name of the command that evaluates the function for which the zero is to be returned

float *xb*
Start of the interval in which the zero is supposed to lie

float *xe*
End of the interval

float *eps*
Relative allowed error (defaults to 1.0e-7)

::math::calculus::root_brent *f xb xe eps*

Return an estimate of the zero or one of the zeros of the function contained in the interval [xb,xe]. The error in this estimate is of the order of eps*abs(xe-xb), the actual error may be slightly larger.

The method used is the so-called *Brent* method. For properties: see the *regula falsi* procedure.

command *f*
Name of the command that evaluates the function for which the zero is to be returned

float *xb*
Start of the interval in which the zero is supposed to lie

float *xe*
End of the interval

float *eps*
Relative allowed error (defaults to 1.0e-7)

::math::calculus::root_chandrupatla *f xb xe eps*

Return an estimate of the zero or one of the zeros of the function contained in the interval [xb,xe]. The error in this estimate is of the order of eps*abs(xe-xb), the actual error may be slightly larger.

The method used is the so-called *Chandrupatla* method. It is a variant of the Brent method and is reputed to be faster. See https://www.embeddedrelated.com/showarticle/855.php for a discussion.

command *f*
Name of the command that evaluates the function for which the zero is to be returned

float *xb*
Start of the interval in which the zero is supposed to lie

float *xe*
End of the interval

float *eps*
Relative allowed error (defaults to 1.0e-7)

Notes:

Several of the above procedures take the *names* of procedures as arguments. To avoid problems with the *visibility* of these procedures, the fully-qualified name of these procedures is determined inside the calculus routines. For the user this has only one consequence: the named procedure must be visible in the calling procedure. For instance:

```
    namespace eval ::mySpace {
       namespace export calcfunc
       proc calcfunc { x } { return $x }
    }
    #
    # Use a fully-qualified name
    #
    namespace eval ::myCalc {
       proc detIntegral { begin end } {
          return [integral $begin $end 100 ::mySpace::calcfunc]
       }
    }
    #
    # Import the name
    #
    namespace eval ::myCalc {
       namespace import ::mySpace::calcfunc
       proc detIntegral { begin end } {
          return [integral $begin $end 100 calcfunc]
       }
    }
```

Enhancements for the second-order boundary value problem:

- Other types of boundary conditions (zero gradient, zero flux)
- Other schematisation of the first-order term (now central differences are used, but upstream differences might be useful too).
- The various root finding methods differ in robustness, ease of use and convergence rates. While the Newton-Raphson method is quite fast (quadratic convergence), it is not guaranteed to produce an answer. The methods that bracket the root, require two starting points, but except for the secant method are guaranteed to deliver a good estimate.

Examples

Let us take a few simple examples:

Integrate x over the interval [0,100] (20 steps):

```
proc linear_func { x } { return $x }
puts "Integral: [::math::calculus::integral 0 100 20 linear_func]"
```

For simple functions, the alternative could be:

```
puts "Integral: [::math::calculus::integralExpr 0 100 20 {$x}]"
```

Do not forget the braces!

The differential equation for a dampened oscillator:

```
x'' + rx' + wx = 0
```

can be split into a system of first-order equations:

```
x' = y
y' = -ry - wx
```

Then this system can be solved with code like this:

```
proc dampened_oscillator { t xvec } {
   set x  [lindex $xvec 0]
   set x1 [lindex $xvec 1]
   return [list $x1 [expr {-$x1-$x}]]
}

set xvec   { 1.0 0.0 }
set t      0.0
set tstep  0.1
for { set i 0 } { $i < 20 } { incr i } {
   set result [::math::calculus::eulerStep $t $tstep $xvec
    dampened_oscillator]
   puts "Result ($t): $result"
   set t      [expr {$t+$tstep}]
   set xvec   $result
}
```

Suppose we have the boundary value problem:

```
    Dy'' + ky = 0
    x = 0: y = 1
    x = L: y = 0
```

This boundary value problem could originate from the diffusion of a decaying substance.

It can be solved with the following fragment:

```
   proc coeffs { x } { return [list $::Diff 0.0 $::decay] }
   proc force  { x } { return 0.0 }

   set Diff   1.0e-2
   set decay  0.0001
   set length 100.0

   set y [::math::calculus::boundaryValueSecondOrder \
      coeffs force {0.0 1.0} [list $length 0.0] 100]
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you

want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *math :: calculus* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Mathematics

Copyright

```
Copyright (c) 2002,2003,2004 Arjen Markus
```


changepoint
Keywords: changepoint, control, statistics

Name
math::changepoint - Change point detection methods

Synopsis
package require Tcl 8.6 9
package require TclOO
package require math::statistics
package require math::changepoint ?**0.2**?
::math::changepoint::cusum-detect *data* ?*args*?
::math::changepoint::cusum-online ?*args*?
$cusumObj examine *value*
$cusumObj reset
::math::changepoint::binary-segmentation *data* ?*args*?

Description
The **math::changepoint** package implements a number of well-known methods to determine if a series of data contains a shift in the mean or not. Note that these methods only indicate if a shift in the mean is probably. Due to the stochastic nature of the data that will be analysed, false positives are possible. The CUSUM method is implemented in both an "offline" and an "online" version, so that it can be used either for a complete data series or for detecting changes in data that come in one by one. The implementation has been based on these websites mostly:

- https://www.itl.nist.gov/div898/handbook/pmc/section3/pmc323.htm
- https://en.wikipedia.org/wiki/CUSUM

Basically, the deviation of the data from a given target value is accumulated and when the total deviation becomes too large, a change point is reported. A second method, binary segmentation, is implemented only as an "offline" method, as it needs to examine the data series as a whole. In the variant contained here the following ideas have been used:

- The segments in which the data series may be separated shold not be too short, otherwise the ultimate result could be segments of only one data point long. So a minimum length is used.

- To make the segmentation worthwhile there should be a minimum gain in reducing the cost function (the sum of the squared deviations from the mean for each segment).

This may not be in agreement with the descriptions of the method found in various publications, but it is simple to understand and intuitive. One publication that provides more information on the method in general is "Selective review of offline change point detection methods" by Truong et al. https://arxiv.org/abs/1801.00718.

Procedures

The package defines the following public procedures:

::math::changepoint::cusum-detect *data* ?args?

Examine a given data series and return the location of the first change (if any)

double *data*

Series of data to be examined

list *args*

Optional list of key-value pairs:

-target *value*

The target (or mean) for the time series

-tolerance *value*

The tolerated standard deviation

-kfactor *value*

The factor by which to multiply the standard deviation (defaults to 0.5, typically between 0.5 and 1.0)

-hfactor *value*

The factor determining the limits betweem which the "cusum" statistic is accepted (typicaly 3.0-5.0, default 4.0)

::math::changepoint::cusum-online ?args?

Class to examine data passed in against expected properties. At least the keywords *-target* and *-tolerance* must be given.

list *args*

List of key-value pairs:

-target *value*

The target (or mean) for the time series

-tolerance *value*

The tolerated standard deviation

-kfactor *value*

The factor by which to multiply the standard deviation (defaults to 0.5, typically between 0.5 and 1.0)

-hfactor *value*

The factor determining the limits betweem which the "cusum" statistic is accepted (typicaly 3.0-5.0, default 4.0)

$cusumObj examine *value*

Pass a value to the *cusum-online* object and examine it. If, with this new value, the

cumulative sum remains within the bounds, zero (0) is returned, otherwise one (1) is returned.

double *value*

The new value

$cusumObj reset

Reset the cumulative sum, so that the examination can start afresh.

::math::changepoint::binary-segmentation *data* ?args?

Apply the binary segmentation method recursively to find change points. Returns a list of indices of potential change points

list *data*

Data to be examined

list *args*

Optional key-value pairs:

-minlength *number*

Minimum number of points in each segment (default: 5)

-threshold *value*

Factor applied to the standard deviation functioning as a threshold for accepting the change in cost function as an improvement (default: 1.0)

Category

Mathematics

Copyright

```
Copyright (c) 2020 by Arjen Markus
```

combinatorics

Keywords: combinatorics

Name

math::combinatorics - Combinatorial functions in the Tcl Math Library

Synopsis

package require Tcl 8.5 9
package require math ?**1.2.6**?
package require Tcl 8.6 9
package require TclOO
package require math::combinatorics ?**2.1**?
::math::ln_Gamma *z*
::math::factorial *x*
::math::choose *n k*
::math::Beta *z w*
::math::combinatorics::permutations *n*
::math::combinatorics::variations *n k*
::math::combinatorics::combinations *n k*
::math::combinatorics::derangements *n*

::math::combinatorics::catalan *n*
::math::combinatorics::firstStirling *n m*
::math::combinatorics::secondStirling *n m*
::math::combinatorics::partitionP *n*
::math::combinatorics::list-permutations *n*
::math::combinatorics::list-variations *n k*
::math::combinatorics::list-combinations *n k*
::math::combinatorics::list-derangements *n*
::math::combinatorics::list-powerset *n*
::math::combinatorics::permutationObj new/create NAME *n*
$perm next
$perm reset
$perm setElements *elements*
$perm setElements
::math::combinatorics::combinationObj new/create NAME *n k*
$combin next
$combin reset
$combin setElements *elements*
$combin setElements

Description

The **math** package contains implementations of several functions useful in combinatorial problems. The **math::combinatorics** extends the collections based on features in Tcl 8.6. Note: the meaning of the partitionP function, Catalan and Stirling numbers is explained on the MathWorld website

Commands

::math::ln_Gamma *z*

Returns the natural logarithm of the Gamma function for the argument *z*.

The Gamma function is defined as the improper integral from zero to positive infinity of

```
t**(x-1)*exp(-t) dt
```

The approximation used in the Tcl Math Library is from Lanczos, *ISIAM J. Numerical Analysis, series B*, volume 1, p. 86. For "**x** > 1", the absolute error of the result is claimed to be smaller than 5.5*10**-10 -- that is, the resulting value of Gamma when

```
exp( ln_Gamma( x) )
```

is computed is expected to be precise to better than nine significant figures.

::math::factorial *x*

Returns the factorial of the argument *x*.

For integer *x*, 0 <= *x* <= 12, an exact integer result is returned.

For integer *x*, 13 <= *x* <= 21, an exact floating-point result is returned on machines with IEEE floating point.

For integer *x*, 22 <= *x* <= 170, the result is exact to 1 ULP.

For real *x*, *x* >= 0, the result is approximated by computing *Gamma(x+1)* using the **::math::ln_Gamma** function, and the result is expected to be precise to better than nine significant figures.

It is an error to present *x* <= -1 or *x* > 170, or a value of *x* that is not numeric.

::math::choose *n k*

Returns the binomial coefficient *C(n, k)*

```
C(n,k) = n! / k! (n-k)!
```

If both parameters are integers and the result fits in 32 bits, the result is rounded to an integer.

Integer results are exact up to at least *n* = 34. Floating point results are precise to better than nine significant figures.

::math::Beta *z w*

Returns the Beta function of the parameters *z* and *w*.

```
Beta(z,w) = Beta(w,z) = Gamma(z) * Gamma(w) / Gamma(z+w)
```

Results are returned as a floating point number precise to better than nine significant digits provided that *w* and *z* are both at least 1.

::math::combinatorics::permutations *n*

Return the number of permutations of n items. The returned number is always an integer, it is not limited by the range of 32-or 64-bits integers using the arbitrary precision integers available in Tcl 8.5 and later.

int *n*

The number of items to be permuted.

::math::combinatorics::variations *n k*

Return the number of variations k items selected from the total of n items. The order of the items is taken into account.

int *n*

The number of items to be selected from.

int *k*

The number of items to be selected in each variation.

::math::combinatorics::combinations *n k*

Return the number of combinations of k items selected from the total of n items. The order of the items is not important.

int *n*

The number of items to be selected from.

int *k*

The number of items to be selected in each combination.

::math::combinatorics::derangements *n*

Return the number of derangements of n items. A derangement is a permutation where each item is displaced from the original position.

int *n*

The number of items to be rearranged.

::math::combinatorics::catalan *n*

Return the n'th Catalan number. The number n is expected to be 1 or larger. These numbers occur in various combinatorial problems.

int *n*

The index of the Catalan number

::math::combinatorics::firstStirling *n m*

Calculate a Stirling number of the first kind (signed version, m cycles in a permutation of n items)

int *n*

Number of items

int *m*

Number of cycles

::math::combinatorics::secondStirling *n m*

Calculate a Stirling number of the second kind (m non-empty subsets from n items)

int *n*

Number of items

int *m*

Number of subsets

::math::combinatorics::partitionP *n*

Calculate the number of ways an integer n can be written as the sum of positive integers.

int *n*

Number in question

::math::combinatorics::list-permutations *n*

Return the list of permutations of the numbers 0, ..., n-1.

int *n*

The number of items to be permuted.

::math::combinatorics::list-variations *n k*

Return the list of variations of k numbers selected from the numbers 0, ..., n-1. The order of the items is taken into account.

int *n*

The number of items to be selected from.

int *k*

The number of items to be selected in each variation.

::math::combinatorics::list-combinations *n k*

Return the list of combinations of k numbers selected from the numbers 0, ..., n-1. The order

of the items is ignored.

int *n*

The number of items to be selected from.

int *k*

The number of items to be selected in each combination.

::math::combinatorics::list-derangements *n*

Return the list of derangements of the numbers 0, ..., n-1.

int *n*

The number of items to be rearranged.

::math::combinatorics::list-powerset *n*

Return the list of all subsets of the numbers 0, ..., n-1.

int *n*

The number of items to be rearranged.

::math::combinatorics::permutationObj new/create NAME *n*

Create a TclOO object for returning permutations one by one. If the last permutation has been reached an empty list is returned.

int *n*

The number of items to be rearranged.

$perm next

Return the next permutation of n objects.

$perm reset

Reset the object, so that the command *next* returns the complete list again.

$perm setElements *elements*

Register a list of items to be permuted, using the *nextElements* command.

list *elements*

The list of n items that will be permuted.

$perm setElements

Return the next permulation of the registered items.

::math::combinatorics::combinationObj new/create NAME *n k*

Create a TclOO object for returning combinations one by one. If the last combination has been reached an empty list is returned.

int *n*

The number of items to be rearranged.

$combin next

Return the next combination of n objects.

$combin reset

Reset the object, so that the command *next* returns the complete list again.

$combin setElements *elements*

Register a list of items to be permuted, using the *nextElements* command.

list *elements*

The list of n items that will be permuted.

$combin setElements
Return the next combination of the registered items.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *math* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Mathematics

constants

Keywords: constants, degrees, e, math, pi, radians

Name

math::constants - Mathematical and numerical constants

Synopsis

package require Tcl ?**8.3**?
package require math::constants ?**1.0.4**?
::math::constants::constants *args*
::math::constants::print-constants *args*

Description

This package defines some common mathematical and numerical constants. By using the package you get consistent values for numbers like pi and ln(10).

It defines two commands:

- One for importing the constants
- One for reporting which constants are defined and what values they actually have.

The motivation for this package is that quite often, with (mathematical) computations, you need a good approximation to, say, the ratio of degrees to radians. You can, of course, define this like:

```
    variable radtodeg [expr {180.0/(4.0*atan(1.0))}]
```

and use the variable radtodeg whenever you need the conversion.

This has two drawbacks:

- You need to remember the proper formula or value and that is error-prone.
- Especially with the use of mathematical functions like *atan* you assume that they have been accurately implemented. This is seldom or never the case and for each platform you can get subtle differences.

Here is the way you can do it with the *math::constants* package:

```
    package require math::constants
    ::math::constants::constants radtodeg degtorad
```

which creates two variables, radtodeg and (its reciprocal) degtorad in the calling namespace.

Constants that have been defined (their values are mostly taken from mathematical tables with more precision than usually can be handled) include:

- basic constants like pi, e, gamma (Euler's constant)
- derived values like ln(10) and sqrt(2)
- purely numerical values such as 1/3 that are included for convenience and for the fact that certain seemingly trivial computations like:

```
    set value [expr {3.0*$onethird}]
```

- give *exactly* the value you expect (if IEEE arithmetic is available).

The full set of named constants is listed in section Constants.

Procedures

The package defines the following public procedures:

::math::constants::constants *args*
Import the constants whose names are given as arguments

::math::constants::print-constants *args*
Print the constants whose names are given as arguments on the screen (name, value and description) or, if no arguments are given, print all defined constants. This is mainly a convenience procedure.

Constants

pi
Ratio of circle circumference to diameter

e
Base for natural logarithm

ln10
Natural logarithm of 10

phi
Golden ratio

gamma
Euler's constant

sqrt2
Square root of 2

thirdrt2
One-third power of 2

sqrt3
Square root of 3

radtodeg
Conversion from radians to degrees

degtorad

Conversion from degrees to radians

onethird
One third (0.3333....)

twothirds
Two thirds (0.6666....)

onesixth
One sixth (0.1666....)

huge
(Approximately) largest number

tiny
(Approximately) smallest number not equal zero

eps
Smallest number such that 1+eps!= 1

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *math :: constants* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Mathematics

Copyright

```
Copyright (c) 2004 Arjen Markus <arjenmarkus@users.sourceforge.net>
```

decimal

Keywords: decimal, math, tcl

Name

math::decimal - General decimal arithmetic

Synopsis

package require Tcl ?**8.5 9**?
package require math::decimal 1.0.3
::math::decimal::fromstr *string*
::math::decimal::tostr *decimal*
::math::decimal::setVariable *variable setting*
::math::decimal::add *a b*
::math::decimal::+ *a b*
::math::decimal::subtract *a b*
::math::decimal::- *a b*

::math::decimal::multiply *a b*
::math::decimal::* *a b*
::math::decimal::divide *a b*
::math::decimal::/ *a b*
::math::decimal::divideint *a b*
::math::decimal::remainder *a b*
::math::decimal::abs *decimal*
::math::decimal::compare *a b*
::math::decimal::max *a b*
::math::decimal::maxmag *a b*
::math::decimal::min *a b*
::math::decimal::minmag *a b*
::math::decimal::plus *a*
::math::decimal::minus *a*
::math::decimal::copynegate *a*
::math::decimal::copysign *a b*
::math::decimal::is-signed *decimal*
::math::decimal::is-zero *decimal*
::math::decimal::is-NaN *decimal*
::math::decimal::is-infinite *decimal*
::math::decimal::is-finite *decimal*
::math::decimal::fma *a b c*
::math::decimal::round_half_even *decimal digits*
::math::decimal::round_half_up *decimal digits*
::math::decimal::round_half_down *decimal digits*
::math::decimal::round_down *decimal digits*
::math::decimal::round_up *decimal digits*
::math::decimal::round_floor *decimal digits*
::math::decimal::round_ceiling *decimal digits*
::math::decimal::round_05up *decimal digits*

Description

The decimal package provides decimal arithmetic support for both limited precision floating point and arbitrary precision floating point. Additionally, integer arithmetic is supported.

More information and the specifications on which this package depends can be found on the general decimal arithmetic page at http://speleotrove.com/decimal This package provides for:

- A new data type decimal which is represented as a list containing sign, mantissa and exponent.
- Arithmetic operations on those decimal numbers such as addition, subtraction, multiplication, etc...

Numbers are converted to decimal format using the operation ::math::decimal::fromstr.

Numbers are converted back to string format using the operation ::math::decimal::tostr.

Examples

This section shows some simple examples. Since the purpose of this library is to perform decimal math operations, examples may be the simplest way to learn how to work with it and to

see the difference between using this package and sticking with expr. Consult the API section of this man page for information about individual procedures.

```
    package require math::decimal

    # Various operations on two numbers.
    # We first convert them to decimal format.
    set a [::math::decimal::fromstr 8.2]
    set b [::math::decimal::fromstr .2]

    # Then we perform our operations. Here we add
    set c [::math::decimal::+ $a $b]

    # Finally we convert back to string format for presentation to the
    user.
    puts [::math::decimal::tostr $c] ; # => will output 8.4

    # Other examples
    #
    # Subtraction
    set c [::math::decimal::- $a $b]
    puts [::math::decimal::tostr $c] ; # => will output 8.0

    # Why bother using this instead of simply expr?
    puts [expr {8.2 + .2}] ; # => will output 8.399999999999999
    puts [expr {8.2 - .2}] ; # => will output 7.999999999999999
    # See http://speleotrove.com/decimal to learn more about why this
    happens.
```

API

::math::decimal::fromstr *string*
Convert *string* into a decimal.

::math::decimal::tostr *decimal*
Convert *decimal* into a string representing the number in base 10.

::math::decimal::setVariable *variable setting*
Sets the *variable* to *setting*. Valid variables are:
- *rounding* - Method of rounding to use during rescale. Valid methods are round_half_even, round_half_up, round_half_down, round_down, round_up, round_floor, round_ceiling.
- *precision* - Maximum number of digits allowed in mantissa.
- *extended* - Set to 1 for extended mode. 0 for simplified mode.
- *maxExponent* - Maximum value for the exponent. Defaults to 999.
- *minExponent* - Minimum value for the exponent. Default to -998.

::math::decimal::add *a b*
see below ...

::math::decimal::+ *a b*

Return the sum of the two decimals *a* and *b*.

::math::decimal::subtract *a b*
see below ...

::math::decimal::- *a b*
Return the differnece of the two decimals *a* and *b*.

::math::decimal::multiply *a b*
see below ...

::math::decimal::* *a b*
Return the product of the two decimals *a* and *b*.

::math::decimal::divide *a b*
see below ...

::math::decimal::/ *a b*
Return the quotient of the division between the two decimals *a* and *b*.

::math::decimal::divideint *a b*
Return a the integer portion of the quotient of the division between decimals *a* and *b*

::math::decimal::remainder *a b*
Return the remainder of the division between the two decimals *a* and *b*.

::math::decimal::abs *decimal*
Return the absolute value of the decimal.

::math::decimal::compare *a b*
Compare the two decimals a and b, returning *0* if *a* == *b*, *1* if *a* > *b*, and *-1* if *a* < *b*.

::math::decimal::max *a b*
Compare the two decimals a and b, and return *a* if *a* >= *b*, and *b* if *a* < *b*.

::math::decimal::maxmag *a b*
Compare the two decimals a and b while ignoring their signs, and return *a* if *abs(a)* >= *abs(b)* , and *b* if *abs(a)* < *abs(b)*.

::math::decimal::min *a b*
Compare the two decimals a and b, and return *a* if *a* <= *b*, and *b* if *a* > *b*.

::math::decimal::minmag *a b*
Compare the two decimals a and b while ignoring their signs, and return *a* if *abs(a)* <= *abs(b)* , and *b* if *abs(a)* > *abs(b)*.

::math::decimal::plus *a*
Return the result from *::math::decimal::+ 0 $a*.

::math::decimal::minus *a*
Return the result from *::math::decimal::- 0 $a*.

::math::decimal::copynegate *a*
Returns *a* with the sign flipped.

::math::decimal::copysign *a b*
Returns *a* with the sign set to the sign of the *b*.

::math::decimal::is-signed *decimal*
Return the sign of the decimal. The procedure returns 0 if the number is positive, 1 if it's negative.

::math::decimal::is-zero *decimal*
Return true if *decimal* value is zero, otherwise false is returned.

::math::decimal::is-NaN *decimal*
Return true if *decimal* value is NaN (not a number), otherwise false is returned.

::math::decimal::is-infinite *decimal*
Return true if *decimal* value is Infinite, otherwise false is returned.

::math::decimal::is-finite *decimal*
Return true if *decimal* value is finite, otherwise false is returned.

::math::decimal::fma *a b c*
Return the result from first multiplying *a* by *b* and then adding *c*. Rescaling only occurs after completion of all operations. In this way the result may vary from that returned by performing the operations individually.

::math::decimal::round_half_even *decimal digits*
Rounds *decimal* to *digits* number of decimal points with the following rules: Round to the nearest. If equidistant, round so the final digit is even.

::math::decimal::round_half_up *decimal digits*
Rounds *decimal* to *digits* number of decimal points with the following rules: Round to the nearest. If equidistant, round up.

::math::decimal::round_half_down *decimal digits*
Rounds *decimal* to *digits* number of decimal points with the following rules: Round to the nearest. If equidistant, round down.

::math::decimal::round_down *decimal digits*
Rounds *decimal* to *digits* number of decimal points with the following rules: Round toward 0. (Truncate)

::math::decimal::round_up *decimal digits*
Rounds *decimal* to *digits* number of decimal points with the following rules: Round away from 0

::math::decimal::round_floor *decimal digits*
Rounds *decimal* to *digits* number of decimal points with the following rules: Round toward -Infinity.

::math::decimal::round_ceiling *decimal digits*
Rounds *decimal* to *digits* number of decimal points with the following rules: Round toward Infinity

::math::decimal::round_05up *decimal digits*
Rounds *decimal* to *digits* number of decimal points with the following rules: Round zero or five away from 0. The same as round-up, except that rounding up only occurs if the digit to be rounded up is 0 or 5, and after overflow the result is the same as for round-down.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *decimal* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Mathematics

Copyright

```
Copyright (c) 2011 Mark Alston <mark at beernut dot com>
```


exact
Keywords: exact

Name
math::exact - Exact Real Arithmetic

Synopsis
package require Tcl 8.6 9
package require grammar::aycock 1.0
package require math::exact 1.0.2
::math::exact::exactexpr *expr*
number **ref**
number **unref**
number **asPrint** *precision*
number **asFloat** *precision*

Description
The **exactexpr** command in the **math::exact** package allows for exact computations over the computable real numbers. These are not arbitrary-precision calculations; rather they are exact, with numbers represented by algorithms that produce successive approximations. At the end of a calculation, the caller can request a given precision for the end result, and intermediate results are computed to whatever precision is necessary to satisfy the request.

Procedures
The following procedure is the primary entry into the **math::exact** package.

::math::exact::exactexpr *expr*
Accepts a mathematical expression in Tcl syntax, and returns an object that represents the program to calculate successive approximations to the expression's value. The result will be referred to as an exact real number.

***number* ref**
Increases the reference count of a given exact real number.

***number* unref**
Decreases the reference count of a given exact real number, and destroys the number if the reference count is zero.

number* asPrint *precision
Formats the given *number* for printing, with the specified *precision*. (See below for how *precision* is interpreted). Numbers that are known to be rational are formatted as fractions.

number* asFloat *precision
Formats the given *number* for printing, with the specified *precision*. (See below for how

precision is interpreted). All numbers are formatted in floating-point E format.

Parameters

expr
Expression to evaluate. The syntax for expressions is the same as it is in Tcl, but the set of operations is smaller. See Expressions below for details.

number
The object returned by an earlier invocation of **math::exact::exactexpr**

precision
The requested 'precision' of the result. The precision is (approximately) the absolute value of the binary exponent plus the number of bits of the binary significand. For instance, to return results to IEEE-754 double precision, 56 bits plus the exponent are required. Numbers between 1/2 and 2 will require a precision of 57; numbers between 1/4 and 1/2 or between 2 and 4 will require 58; numbers between 1/8 and 1/4 or between 4 and 8 will require 59; and so on.

Expressions

The **math::exact::exactexpr** command accepts expressions in a subset of Tcl's syntax. The following components may be used in an expression.

- Decimal integers.
- Variable references with the dollar sign (**$**). The value of the variable must be the result of another call to **math::exact::exactexpr**. The reference count of the value will be increased by one for each position at which it appears in the expression.
- The exponentiation operator (**).
- Unary plus (**+**) and minus (**-**) operators.
- Multiplication (*) and division (/) operators.
- Parentheses used for grouping.
- Functions. See Functions below for the functions that are available.

Functions

The following functions are available for use within exact real expressions.

acos(*x*)
The inverse cosine of *x*. The result is expressed in radians. The absolute value of *x* must be less than 1.

acosh(*x*)
The inverse hyperbolic cosine of *x*. *x* must be greater than 1.

asin(*x*)
The inverse sine of *x*. The result is expressed in radians. The absolute value of *x* must be less than 1.

asinh(*x*)
The inverse hyperbolic sine of *x*.

atan(*x*)
The inverse tangent of *x*. The result is expressed in radians.

atanh(*x*)
The inverse hyperbolic tangent of *x*. The absolute value of *x* must be less than 1.

cos(*x*)
The cosine of *x*. *x* is expressed in radians.

cosh(*x*)
The hyperbolic cosine of *x*.

e()
The base of the natural logarithms = **2.71828...**

exp(*x*)
The exponential function of *x*.

log(*x*)
The natural logarithm of *x*. *x* must be positive.

pi()
The value of pi = **3.15159...**

sin(*x*)
The sine of *x*. *x* is expressed in radians.

sinh(*x*)
The hyperbolic sine of *x*.

sqrt(*x*)
The square root of *x*. *x* must be positive.

tan(*x*)
The tangent of *x*. *x* is expressed in radians.

tanh(*x*)
The hyperbolic tangent of *x*.

Summary

The **math::exact::exactexpr** command provides a system that performs exact arithmetic over computable real numbers, representing the numbers as algorithms for successive approximation. An example, which implements the high-school quadratic formula, is shown below.

```
namespace import math::exact::exactexpr
proc exactquad {a b c} {
    set d [[exactexpr {sqrt($b*$b - 4*$a*$c)}] ref]
    set r0 [[exactexpr {(-$b - $d) / (2 * $a)}] ref]
    set r1 [[exactexpr {(-$b + $d) / (2 * $a)}] ref]
    $d unref
    return [list $r0 $r1]
}

set a [[exactexpr 1] ref]
set b [[exactexpr 200] ref]
set c [[exactexpr {(-3/2) * 10**-12}] ref]
lassign [exactquad $a $b $c] r0 r1
$a unref; $b unref; $c unref
puts [list [$r0 asFloat 70] [$r1 asFloat 110]]
$r0 unref; $r1 unref
```

The program prints the result:

```
-2.0000000000000075e2 7.499999999999999719e-15
```

Note that if IEEE-754 floating point had been used, a catastrophic roundoff error would yield a smaller root that is a factor of two too high:

```
-200.0 1.4210854715202004e-14
```

The invocations of **exactexpr** should be fairly self-explanatory. The other commands of note are **ref** and **unref**. It is necessary for the caller to keep track of references to exact expressions - to call **ref** every time an exact expression is stored in a variable and **unref** every time the variable goes out of scope or is overwritten. The **asFloat** method emits decimal digits as long as the requested precision supports them. It terminates when the requested precision yields an uncertainty of more than one unit in the least significant digit.

Category
Mathematics

Copyright

```
Copyright (c) 2015 Kevin B. Kenny <kennykb@acm.org>
Redistribution permitted under the terms of the Open Publication
    License <http://www.opencontent.org/openpub/>
```


figurate
Keywords: figurate, figurate numbers, mathematics

Name
math::figurate - Evaluate figurate numbers

Synopsis
package require Tcl 8.6 9
package require math::figurate 1.0
::math::figurate::sum_sequence *n*
::math::figurate::sum_squares *n*
::math::figurate::sum_cubes *n*
::math::figurate::sum_4th_power *n*
::math::figurate::sum_5th_power *n*
::math::figurate::sum_6th_power *n*
::math::figurate::sum_7th_power *n*
::math::figurate::sum_8th_power *n*
::math::figurate::sum_9th_power *n*
::math::figurate::sum_10th_power *n*
::math::figurate::sum_sequence_odd *n*
::math::figurate::sum_squares_odd *n*
::math::figurate::sum_cubes_odd *n*
::math::figurate::sum_4th_power_odd *n*
::math::figurate::sum_5th_power_odd *n*

::math::figurate::sum_6th_power_odd *n*
::math::figurate::sum_7th_power_odd *n*
::math::figurate::sum_8th_power_odd *n*
::math::figurate::sum_9th_power_odd *n*
::math::figurate::sum_10th_power_odd *n*
::math::figurate::oblong *n*
::math::figurate::pronic *n*
::math::figurate::triangular *n*
::math::figurate::square *n*
::math::figurate::cubic *n*
::math::figurate::biquadratic *n*
::math::figurate::centeredTriangular *n*
::math::figurate::centeredSquare *n*
::math::figurate::centeredPentagonal *n*
::math::figurate::centeredHexagonal *n*
::math::figurate::centeredCube *n*
::math::figurate::decagonal *n*
::math::figurate::heptagonal *n*
::math::figurate::hexagonal *n*
::math::figurate::octagonal *n*
::math::figurate::octahedral *n*
::math::figurate::pentagonal *n*
::math::figurate::squarePyramidral *n*
::math::figurate::tetrahedral *n*
::math::figurate::pentatope *n*

Description

Sums of numbers that follow a particular pattern are called figurate numbers. A simple example is the sum of integers 1, 2, ... up to n. You can arrange 1, 1+2=3, 1+2+3=6, ... objects in a triangle, hence the name triangular numbers:

```
        *
        *   *
        *   *   *
        *   *   *   *
        ...
```

The **math::figurate** package consists of a collection of procedures to evaluate a wide variety of figurate numbers. While all formulae are straightforward, the details are sometimes puzzling. *Note:* The procedures consider arguments lower than zero as to mean "no objects to be counted" and therefore return 0.

Procedures

The procedures can be arranged in a few categories: sums of integers raised to a particular power, sums of odd integers and general figurate numbers, for instance the pentagonal numbers.

::math::figurate::sum_sequence *n*

Return the sum of integers 1, 2, ..., n.
int *n*
Highest integer in the sum

::math::figurate::sum_squares *n*
Return the sum of squares 1**2, 2**2, ..., n**2.
int *n*
Highest base integer in the sum

::math::figurate::sum_cubes *n*
Return the sum of cubes 1**3, 2**3, ..., n**3.
int *n*
Highest base integer in the sum

::math::figurate::sum_4th_power *n*
Return the sum of 4th powers 1**4, 2**4, ..., n**4.
int *n*
Highest base integer in the sum

::math::figurate::sum_5th_power *n*
Return the sum of 5th powers 1**5, 2**5, ..., n**5.
int *n*
Highest base integer in the sum

::math::figurate::sum_6th_power *n*
Return the sum of 6th powers 1**6, 2**6, ..., n**6.
int *n*
Highest base integer in the sum

::math::figurate::sum_7th_power *n*
Return the sum of 7th powers 1**7, 2**7, ..., n**7.
int *n*
Highest base integer in the sum

::math::figurate::sum_8th_power *n*
Return the sum of 8th powers 1**8, 2**8, ..., n**8.
int *n*
Highest base integer in the sum

::math::figurate::sum_9th_power *n*
Return the sum of 9th powers 1**9, 2**9, ..., n**9.
int *n*
Highest base integer in the sum

::math::figurate::sum_10th_power *n*
Return the sum of 10th powers 1**10, 2**10, ..., n**10.
int *n*
Highest base integer in the sum

::math::figurate::sum_sequence_odd *n*
Return the sum of odd integers 1, 3, ..., 2n-1
int *n*
Highest integer in the sum

::math::figurate::sum_squares_odd *n*
Return the sum of odd squares 1**2, 3**2, ..., (2n-1)**2.
int *n*
Highest base integer in the sum

::math::figurate::sum_cubes_odd *n*
Return the sum of odd cubes 1**3, 3**3, ..., (2n-1)**3.
int *n*
Highest base integer in the sum

::math::figurate::sum_4th_power_odd *n*
Return the sum of odd 4th powers 1**4, 2**4, ..., (2n-1)**4.
int *n*
Highest base integer in the sum

::math::figurate::sum_5th_power_odd *n*
Return the sum of odd 5th powers 1**5, 2**5, ..., (2n-1)**5.
int *n*
Highest base integer in the sum

::math::figurate::sum_6th_power_odd *n*
Return the sum of odd 6th powers 1**6, 2**6, ..., (2n-1)**6.
int *n*
Highest base integer in the sum

::math::figurate::sum_7th_power_odd *n*
Return the sum of odd 7th powers 1**7, 2**7, ..., (2n-1)**7.
int *n*
Highest base integer in the sum

::math::figurate::sum_8th_power_odd *n*
Return the sum of odd 8th powers 1**8, 2**8, ..., (2n-1)**8.
int *n*
Highest base integer in the sum

::math::figurate::sum_9th_power_odd *n*
Return the sum of odd 9th powers 1**9, 2**9, ..., (2n-1)**9.
int *n*
Highest base integer in the sum

::math::figurate::sum_10th_power_odd *n*
Return the sum of odd 10th powers 1**10, 2**10, ..., (2n-1)**10.
int *n*

Highest base integer in the sum

::math::figurate::oblong *n*
Return the nth oblong number (twice the nth triangular number)
int *n*
Required index

::math::figurate::pronic *n*
Return the nth pronic number (synonym for oblong)
int *n*
Required index

::math::figurate::triangular *n*
Return the nth triangular number
int *n*
Required index

::math::figurate::square *n*
Return the nth square number
int *n*
Required index

::math::figurate::cubic *n*
Return the nth cubic number
int *n*
Required index

::math::figurate::biquadratic *n*
Return the nth biquaratic number (i.e. n**4)
int *n*
Required index

::math::figurate::centeredTriangular *n*
Return the nth centered triangular number (items arranged in concentric squares)
int *n*
Required index

::math::figurate::centeredSquare *n*
Return the nth centered square number (items arranged in concentric squares)
int *n*
Required index

::math::figurate::centeredPentagonal *n*
Return the nth centered pentagonal number (items arranged in concentric pentagons)
int *n*
Required index

::math::figurate::centeredHexagonal *n*

Return the nth centered hexagonal number (items arranged in concentric hexagons)
int *n*
Required index

::math::figurate::centeredCube *n*
Return the nth centered cube number (items arranged in concentric cubes)
int *n*
Required index

::math::figurate::decagonal *n*
Return the nth decagonal number (items arranged in decagons with one common vertex)
int *n*
Required index

::math::figurate::heptagonal *n*
Return the nth heptagonal number (items arranged in heptagons with one common vertex)
int *n*
Required index

::math::figurate::hexagonal *n*
Return the nth hexagonal number (items arranged in hexagons with one common vertex)
int *n*
Required index

::math::figurate::octagonal *n*
Return the nth octagonal number (items arranged in octagons with one common vertex)
int *n*
Required index

::math::figurate::octahedral *n*
Return the nth octahedral number (items arranged in octahedrons with a common centre)
int *n*
Required index

::math::figurate::pentagonal *n*
Return the nth pentagonal number (items arranged in pentagons with one common vertex)
int *n*
Required index

::math::figurate::squarePyramidral *n*
Return the nth square pyramidral number (items arranged in a square pyramid)
int *n*
Required index

::math::figurate::tetrahedral *n*
Return the nth tetrahedral number (items arranged in a triangular pyramid)
int *n*
Required index

::math::figurate::pentatope ***n***
Return the nth pentatope number (items arranged in the four-dimensional analogue of a triangular pyramid)
int ***n***
Required index

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *math :: figurate* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Mathematics

filtergen

Keywords: filtergen, digital, filtering

Name

math::filters - Digital filters

Synopsis

package require Tcl 8.6 9
package require TclOO
package require math::filters ?**0.4**?
::math::filters::filterButterworth *lowpass order samplefreq cutofffreq*
::math::filters::filter *coeffs data*
::math::filters::filterChebyshev *lowpass order samplefreq cutofffreq* ?*epsilon*?
::math::filters::filter *coeffs data*
::math::filters::filterObject new *coeffs yinit*
$filterObj filter *x*
$filterObj reset

Description

The **math::filters** package implements digital filters, notably Butterworth and Chebyshev low-pass and high-pass filters. The procedures allow to filter an entire data series as well as filter data one by one. Note: https://en.wikipedia.org/wiki/Digital_filter and other pages on Wikipedia describe the principles and design considerations of these digital filters. Note: This is a reimplementation compared to version 0.3, which was based on GPL-licensed code. One possible incompatibility with that version is that the order now needs to be even.

Procedures

The package defines the following public procedures:

::math::filters::filterButterworth *lowpass order samplefreq cutofffreq*

Determine the coefficients for a Butterworth filter of given (even) order. The coefficients are returned as a nested list, suitable for the *filter* command or method,

bool *lowpass*

Generate a low-pass filter (1) or a high-pass filter (0)

integer *order*

The order of the filter to be generated - it must be even

double *samplefreq*

Sampling frequency of the data series

double *cutofffreq*

Cut-off frequency for the filter (at most half the sampling frequency)

::math::filters::filter *coeffs data*

Filter the entire data series based on the filter coefficients.

list *coeffs*

List of coefficients as generated by *filterButterworth* (or in fact any similar list of coefficients)

list *data*

Data to be filtered

::math::filters::filterChebyshev *lowpass order samplefreq cutofffreq ?epsilon?*

Determine the coefficients for a Chebyshev filter of given (even) order. The coefficients are returned as a nested list, suitable for the *filter* command or method, For this type of filters an extra parameter can be given, the so-called ripple factor. This controls the smoothness of the gain factor.

bool *lowpass*

Generate a low-pass filter (1) or a high-pass filter (0)

integer *order*

The order of the filter to be generated - it must be even

double *samplefreq*

Sampling frequency of the data series

double *cutofffreq*

Cut-off frequency for the filter (must be less than half the sampling frequency)

double *epsilon*

Ripple factor, defaults to 0.1

::math::filters::filter *coeffs data*

Filter the entire data series based on the filter coefficients.

list *coeffs*

List of coefficients as generated by *filterButterworth* (or in fact any similar list of coefficients)

list *data*

Data to be filtered

::math::filters::filterObject new *coeffs yinit*

Create a filter object. The initial x data are taken as zero. The initial y data can be prescribed.

If they are not given, they are taken as zero as well.

list *coeffs*
List of coefficients as generated by *filterButterworth* (or in fact any similar list of coefficients)

list *yinit*
(Optional) initial data for the filter result.

$filterObj filter *x*
Filter a single value and return the result.

double *x*
The value to be filtered

$filterObj reset
Reset the filter object (start anew)

Category
Mathematics

Copyright

```
Copyright (c) 2020 by Arjen Markus
```


fourier
Keywords: fourier, FFT, Fourier transform, complex numbers, mathematics

Name
math::fourier - Discrete and fast fourier transforms

Synopsis
package require Tcl 8.5 9
package require math::fourier 1.0.3
::math::fourier::dft *in_data*
::math::fourier::inverse_dft *in_data*
::math::fourier::lowpass *cutoff in_data*
::math::fourier::highpass *cutoff in_data*

Description
The **math::fourier** package uses the fast Fourier transform, if applicable, or the ordinary transform to implement the discrete Fourier transform. It also provides a few simple filter procedures as an illustration of how such filters can be implemented.

The purpose of this document is to describe the implemented procedures and provide some examples of their usage. As there is ample literature on the algorithms involved, we refer to relevant text books for more explanations. We also refer to the original Wiki page on the subject which describes some of the considerations behind the current implementation.

General information
The two top-level procedures defined are

- dft data-list
- inverse_dft data-list

Both take a list of *complex numbers* and apply a Discrete Fourier Transform (DFT) or its inverse respectively to these lists of numbers. A "complex number" in this case is either (i) a pair (two element list) of numbers, interpreted as the real and imaginary parts of the complex number, or (ii) a single number, interpreted as the real part of a complex number whose imaginary part is zero. The return value is always in the first format. (The DFT generally produces complex results even if the input is purely real.) Applying first one and then the other of these procedures to a list of complex numbers will (modulo rounding errors due to floating point arithmetic) return the original list of numbers.

If the input length N is a power of two then these procedures will utilize the O(N log N) Fast Fourier Transform algorithm. If input length is not a power of two then the DFT will instead be computed using the naive quadratic algorithm.

Some examples:

```
% dft {1 2 3 4}
{10 0.0} {-2.0 2.0} {-2 0.0} {-2.0 -2.0}
% inverse_dft {{10 0.0} {-2.0 2.0} {-2 0.0} {-2.0 -2.0}}
{1.0 0.0} {2.0 0.0} {3.0 0.0} {4.0 0.0}
% dft {1 2 3 4 5}
{15.0 0.0} {-2.5 3.44095480118} {-2.5 0.812299240582} {-2.5
-0.812299240582} {-2.5 -3.44095480118}
% inverse_dft {{15.0 0.0} {-2.5 3.44095480118} {-2.5
0.812299240582} {-2.5 -0.812299240582} {-2.5 -3.44095480118}}
{1.0 0.0} {2.0 8.881784197e-17} {3.0 4.4408920985e-17} {4.0
4.4408920985e-17} {5.0 -8.881784197e-17}
```

In the last case, the imaginary parts <1e-16 would have been zero in exact arithmetic, but aren't here due to rounding errors.

Internally, the procedures use a flat list format where every even index element of a list is a real part and every odd index element is an imaginary part. This is reflected in the variable names by Re_ and Im_ prefixes.

The package includes two simple filters. They have an analogue equivalent in a simple electronic circuit, a resistor and a capacitance in series. Using these filters requires the **math::complexnumbers** package.

Procedures

The public Fourier transform procedures are:

::math::fourier::dft *in_data*

Determine the *Fourier transform* of the given list of complex numbers. The result is a list of complex numbers representing the (complex) amplitudes of the Fourier components.

list *in_data*

List of data

::math::fourier::inverse_dft *in_data*

Determine the *inverse Fourier transform* of the given list of complex numbers (interpreted as

amplitudes). The result is a list of complex numbers representing the original (complex) data

list *in_data*
List of data (amplitudes)

::math::fourier::lowpass *cutoff in_data*
Filter the (complex) amplitudes so that high-frequency components are suppressed. The implemented filter is a first-order low-pass filter, the discrete equivalent of a simple electronic circuit with a resistor and a capacitance.

float *cutoff*
Cut-off frequency

list *in_data*
List of data (amplitudes)

::math::fourier::highpass *cutoff in_data*
Filter the (complex) amplitudes so that low-frequency components are suppressed. The implemented filter is a first-order low-pass filter, the discrete equivalent of a simple electronic circuit with a resistor and a capacitance.

float *cutoff*
Cut-off frequency

list *in_data*
List of data (amplitudes)

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *math :: fourier* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Mathematics

fuzzy

Keywords: fuzzy, floating-point, math, rounding

Name

math::fuzzy - Fuzzy comparison of floating-point numbers

Synopsis

package require Tcl ?**8.3**?
package require math::fuzzy ?**0.2**?
::math::fuzzy::teq *value1 value2*
::math::fuzzy::tne *value1 value2*
::math::fuzzy::tge *value1 value2*

::math::fuzzy::tle *value1 value2*
::math::fuzzy::tlt *value1 value2*
::math::fuzzy::tgt *value1 value2*
::math::fuzzy::tfloor *value*
::math::fuzzy::tceil *value*
::math::fuzzy::tround *value*
::math::fuzzy::troundn *value ndigits*

Description

The package Fuzzy is meant to solve common problems with floating-point numbers in a systematic way:

- Comparing two numbers that are "supposed" to be identical, like 1.0 and 2.1/(1.2+0.9) is not guaranteed to give the intuitive result.
- Rounding a number that is halfway two integer numbers can cause strange errors, like int(100.0*2.8)!= 28 but 27

The Fuzzy package is meant to help sorting out this type of problems by defining "fuzzy" comparison procedures for floating-point numbers. It does so by allowing for a small margin that is determined automatically - the margin is three times the "epsilon" value, that is three times the smallest number *eps* such that 1.0 and 1.0+$eps canbe distinguished. In Tcl, which uses double precision floating-point numbers, this is typically 1.1e-16.

Procedures

Effectively the package provides the following procedures:

::math::fuzzy::teq *value1 value2*
Compares two floating-point numbers and returns 1 if their values fall within a small range. Otherwise it returns 0.

::math::fuzzy::tne *value1 value2*
Returns the negation, that is, if the difference is larger than the margin, it returns 1.

::math::fuzzy::tge *value1 value2*
Compares two floating-point numbers and returns 1 if their values either fall within a small range or if the first number is larger than the second. Otherwise it returns 0.

::math::fuzzy::tle *value1 value2*
Returns 1 if the two numbers are equal according to [teq] or if the first is smaller than the second.

::math::fuzzy::tlt *value1 value2*
Returns the opposite of [tge].

::math::fuzzy::tgt *value1 value2*
Returns the opposite of [tle].

::math::fuzzy::tfloor *value*
Returns the integer number that is lower or equal to the given floating-point number, within a well-defined tolerance.

::math::fuzzy::tceil *value*
Returns the integer number that is greater or equal to the given floating-point number, within a well-defined tolerance.

::math::fuzzy::tround *value*

Rounds the floating-point number off.

::math::fuzzy::troundn *value ndigits*

Rounds the floating-point number off to the specified number of decimals (Pro memorie).

Usage:

```
if { [teq $x $y] } { puts "x == y" }
if { [tne $x $y] } { puts "x != y" }
if { [tge $x $y] } { puts "x >= y" }
if { [tgt $x $y] } { puts "x > y" }
if { [tlt $x $y] } { puts "x < y" }
if { [tle $x $y] } { puts "x <= y" }

set fx      [tfloor $x]
set fc      [tceil  $x]
set rounded [tround $x]
set roundn  [troundn $x $nodigits]
```

Test cases

The problems that can occur with floating-point numbers are illustrated by the test cases in the file "fuzzy.test":

- Several test case use the ordinary comparisons, and they fail invariably to produce understandable results
- One test case uses [expr] without braces ({ and }). It too fails.

The conclusion from this is that any expression should be surrounded by braces, because otherwise very awkward things can happen if you need accuracy. Furthermore, accuracy and understandable results are enhanced by using these "tolerant" or fuzzy comparisons.

Note that besides the Tcl-only package, there is also a C-based version.

References

Original implementation in Fortran by dr. H.D. Knoble (Penn State University).

P. E. Hagerty, "More on Fuzzy Floor and Ceiling," APL QUOTE QUAD 8(4):20-24, June 1978. Note that TFLOOR=FL5 took five years of refereed evolution (publication).

L. M. Breed, "Definitions for Fuzzy Floor and Ceiling", APL QUOTE QUAD 8(3):16-23, March 1978.

D. Knuth, Art of Computer Programming, Vol. 1, Problem 1.2.4-5.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *math :: fuzzy* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Mathematics

interpolate
Keywords: interpolate, interpolation, math, spatial interpolation

Name
math::interpolate - Interpolation routines

Synopsis
package require Tcl ?**8.5 9**?
package require struct
package require math::interpolate ?**1.1.4**?
::math::interpolate::defineTable *name colnames values*
::math::interpolate::interp-1d-table *name xval*
::math::interpolate::interp-table *name xval yval*
::math::interpolate::interp-linear *xyvalues xval*
::math::interpolate::interp-lagrange *xyvalues xval*
::math::interpolate::prepare-cubic-splines *xcoord ycoord*
::math::interpolate::interp-cubic-splines *coeffs x*
::math::interpolate::interp-spatial *xyvalues coord*
::math::interpolate::interp-spatial-params *max_search power*
::math::interpolate::neville *xlist ylist x*

Description
This package implements several interpolation algorithms:

- Interpolation into a table (one or two independent variables), this is useful for example, if the data are static, like with tables of statistical functions.
- Linear interpolation into a given set of data (organised as (x,y) pairs).
- Lagrange interpolation. This is mainly of theoretical interest, because there is no guarantee about error bounds. One possible use: if you need a line or a parabola through given points (it will calculate the values, but not return the coefficients).

  A variation is Neville's method which has better behaviour and error bounds.
- Spatial interpolation using a straightforward distance-weight method. This procedure allows any number of spatial dimensions and any number of dependent variables.
- Interpolation in one dimension using cubic splines.

This document describes the procedures and explains their usage.

Incompatibility with version 1.0.3
The interpretation of the tables in the **::math::interpolate::interpolate-1d-table** command has been changed to be compatible with the interpretation for 2D interpolation in the **::math::interpolate::interpolate-table** command. As a consequence this version is incompatible with the previous versions of the command (1.0.x).

Procedures
The interpolation package defines the following public procedures:

::math::interpolate::defineTable *name colnames values*

Define a table with one or two independent variables (the distinction is implicit in the data). The procedure returns the name of the table - this name is used whenever you want to interpolate the values. *Note:* this procedure is a convenient wrapper for the struct::matrix procedure. Therefore you can access the data at any location in your program.

string *name* (in)

Name of the table to be created

list *colnames* (in)

List of column names

list *values* (in)

List of values (the number of elements should be a multiple of the number of columns. See Examples for more information on the interpretation of the data.

The values must be sorted with respect to the independent variable(s).

::math::interpolate::interp-1d-table *name xval*

Interpolate into the one-dimensional table "name" and return a list of values, one for each dependent column.

string *name* (in)

Name of an existing table

float *xval* (in)

Value of the independent *row* variable

::math::interpolate::interp-table *name xval yval*

Interpolate into the two-dimensional table "name" and return the interpolated value.

string *name* (in)

Name of an existing table

float *xval* (in)

Value of the independent *row* variable

float *yval* (in)

Value of the independent *column* variable

::math::interpolate::interp-linear *xyvalues xval*

Interpolate linearly into the list of x,y pairs and return the interpolated value.

list *xyvalues* (in)

List of pairs of (x,y) values, sorted to increasing x. They are used as the breakpoints of a piecewise linear function.

float *xval* (in)

Value of the independent variable for which the value of y must be computed.

::math::interpolate::interp-lagrange *xyvalues xval*

Use the list of x,y pairs to construct the unique polynomial of lowest degree that passes through all points and return the interpolated value.

list *xyvalues* (in)

List of pairs of (x,y) values

float *xval* (in)

Value of the independent variable for which the value of y must be computed.

::math::interpolate::prepare-cubic-splines *xcoord ycoord*

Returns a list of coefficients for the second routine *interp-cubic-splines* to actually interpolate.

list *xcoord*

List of x-coordinates for the value of the function to be interpolated is known. The coordinates must be strictly ascending. At least three points are required.

list *ycoord*

List of y-coordinates (the values of the function at the given x-coordinates).

::math::interpolate::interp-cubic-splines *coeffs x*

Returns the interpolated value at coordinate x. The coefficients are computed by the procedure *prepare-cubic-splines*.

list *coeffs*

List of coefficients as returned by prepare-cubic-splines

float *x*

x-coordinate at which to estimate the function. Must be between the first and last x-coordinate for which values were given.

::math::interpolate::interp-spatial *xyvalues coord*

Use a straightforward interpolation method with weights as function of the inverse distance to interpolate in 2D and N-dimensional space

The list xyvalues is a list of lists:

```
{   {x1 y1 z1 {v11 v12 v13 v14}}
{x2 y2 z2 {v21 v22 v23 v24}}
...
}
```

The last element of each inner list is either a single number or a list in itself. In the latter case the return value is a list with the same number of elements.

The method is influenced by the search radius and the power of the inverse distance

list *xyvalues* (in)

List of lists, each sublist being a list of coordinates and of dependent values.

list *coord* (in)

List of coordinates for which the values must be calculated

::math::interpolate::interp-spatial-params *max_search power*

Set the parameters for spatial interpolation

float *max_search* (in)

Search radius (data points further than this are ignored)

integer *power* (in)

Power for the distance (either 1 or 2; defaults to 2)

::math::interpolate::neville *xlist ylist x*

Interpolates between the tabulated values of a function whose abscissae are *xlist* and whose

ordinates are *ylist* to produce an estimate for the value of the function at *x*. The result is a two-element list; the first element is the function's estimated value, and the second is an estimate of the absolute error of the result. Neville's algorithm for polynomial interpolation is used. Note that a large table of values will use an interpolating polynomial of high degree, which is likely to result in numerical instabilities; one is better off using only a few tabulated values near the desired abscissa.

Examples

Example of using one-dimensional tables:

Suppose you have several tabulated functions of one variable:

```
  x     y1     y2
0.0    0.0    0.0
1.0    1.0    1.0
2.0    4.0    8.0
3.0    9.0   27.0
4.0   16.0   64.0
```

Then to estimate the values at 0.5, 1.5, 2.5 and 3.5, you can use:

```
set table [::math::interpolate::defineTable table1  {x y1 y2} {   -
      1      2
                 0.0    0.0    0.0
                 1.0    1.0    1.0
                 2.0    4.0    8.0
                 3.0    9.0   27.0
                 4.0   16.0   64.0}]
foreach x {0.5 1.5 2.5 3.5} {
    puts "$x: [::math::interpolate::interp-1d-table $table $x]"
}
```

For one-dimensional tables the first row is not used. For two-dimensional tables, the first row represents the values for the second independent variable.

Example of using the cubic splines:

Suppose the following values are given:

```
  x       y
0.1     1.0
0.3     2.1
0.4     2.2
0.8     4.11
1.0     4.12
```

Then to estimate the values at 0.1, 0.2, 0.3, ... 1.0, you can use:

```
   set coeffs [::math::interpolate::prepare-cubic-splines  {0.1 0.3
    0.4 0.8  1.0}  {1.0 2.1 2.2 4.11 4.12}]
   foreach x {0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1.0} {
      puts "$x: [::math::interpolate::interp-cubic-splines $coeffs $x]"
   }
```

to get the following output:

```
0.1: 1.0
0.2: 1.68044117647
0.3: 2.1
0.4: 2.2
0.5: 3.11221507353
0.6: 4.25242647059
0.7: 5.41804227941
0.8: 4.11
0.9: 3.95675857843
1.0: 4.12
```

As you can see, the values at the abscissae are reproduced perfectly.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *math :: interpolate* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Mathematics

Copyright

```
Copyright (c) 2004 Arjen Markus <arjenmarkus@users.sourceforge.net>
Copyright (c) 2004 Kevn B. Kenny <kennykb@users.sourceforge.net>
```

linalg

Keywords: linalg, least squares, linear algebra, linear equations, math, matrices, matrix, vectors

Name

math::linearalgebra - Linear Algebra

Synopsis

package require Tcl ?**8.5 9**?
package require math::linearalgebra ?**1.1.7**?
::math::linearalgebra::mkVector *ndim value*

::math::linearalgebra::mkUnitVector *ndim ndir*
::math::linearalgebra::mkMatrix *nrows ncols value*
::math::linearalgebra::getrow *matrix row ?imin? ?imax?*
::math::linearalgebra::setrow *matrix row newvalues ?imin? ?imax?*
::math::linearalgebra::getcol *matrix col ?imin? ?imax?*
::math::linearalgebra::setcol *matrix col newvalues ?imin? ?imax?*
::math::linearalgebra::getelem *matrix row col*
::math::linearalgebra::setelem *matrix row ?col? newvalue*
::math::linearalgebra::swaprows *matrix irow1 irow2 ?imin? ?imax?*
::math::linearalgebra::swapcols *matrix icol1 icol2 ?imin? ?imax?*
::math::linearalgebra::show *obj ?format? ?rowsep? ?colsep?*
::math::linearalgebra::dim *obj*
::math::linearalgebra::shape *obj*
::math::linearalgebra::conforming *type obj1 obj2*
::math::linearalgebra::symmetric *matrix ?eps?*
::math::linearalgebra::norm *vector type*
::math::linearalgebra::norm_one *vector*
::math::linearalgebra::norm_two *vector*
::math::linearalgebra::norm_max *vector ?index?*
::math::linearalgebra::normMatrix *matrix type*
::math::linearalgebra::dotproduct *vect1 vect2*
::math::linearalgebra::unitLengthVector *vector*
::math::linearalgebra::normalizeStat *mv*
::math::linearalgebra::axpy *scale mv1 mv2*
::math::linearalgebra::add *mv1 mv2*
::math::linearalgebra::sub *mv1 mv2*
::math::linearalgebra::scale *scale mv*
::math::linearalgebra::rotate *c s vect1 vect2*
::math::linearalgebra::transpose *matrix*
::math::linearalgebra::matmul *mv1 mv2*
::math::linearalgebra::angle *vect1 vect2*
::math::linearalgebra::crossproduct *vect1 vect2*
::math::linearalgebra::matmul *mv1 mv2*
::math::linearalgebra::mkIdentity *size*
::math::linearalgebra::mkDiagonal *diag*
::math::linearalgebra::mkRandom *size*
::math::linearalgebra::mkTriangular *size ?uplo? ?value?*
::math::linearalgebra::mkHilbert *size*
::math::linearalgebra::mkDingdong *size*
::math::linearalgebra::mkOnes *size*
::math::linearalgebra::mkMoler *size*
::math::linearalgebra::mkFrank *size*
::math::linearalgebra::mkBorder *size*
::math::linearalgebra::mkWilkinsonW+ *size*
::math::linearalgebra::mkWilkinsonW- *size*
::math::linearalgebra::solveGauss *matrix bvect*

::math::linearalgebra::solvePGauss *matrix bvect*
::math::linearalgebra::solveTriangular *matrix bvect* ?*uplo*?
::math::linearalgebra::solveGaussBand *matrix bvect*
::math::linearalgebra::solveTriangularBand *matrix bvect*
::math::linearalgebra::determineSVD *A eps*
::math::linearalgebra::eigenvectorsSVD *A eps*
::math::linearalgebra::leastSquaresSVD *A y qmin eps*
::math::linearalgebra::choleski *matrix*
::math::linearalgebra::orthonormalizeColumns *matrix*
::math::linearalgebra::orthonormalizeRows *matrix*
::math::linearalgebra::dger *matrix alpha x y* ?*scope*?
::math::linearalgebra::dgetrf *matrix*
::math::linearalgebra::det *matrix*
::math::linearalgebra::largesteigen *matrix tolerance maxiter*
::math::linearalgebra::to_LA *mv*
::math::linearalgebra::from_LA *mv*

Description

This package offers both low-level procedures and high-level algorithms to deal with linear algebra problems:

- robust solution of linear equations or least squares problems
- determining eigenvectors and eigenvalues of symmetric matrices
- various decompositions of general matrices or matrices of a specific form
- (limited) support for matrices in band storage, a common type of sparse matrices

It arose as a re-implementation of Hume's LA package and the desire to offer low-level procedures as found in the well-known BLAS library. Matrices are implemented as lists of lists rather linear lists with reserved elements, as in the original LA package, as it was found that such an implementation is actually faster.

It is advisable, however, to use the procedures that are offered, such as *setrow* and *getrow*, rather than rely on this representation explicitly: that way it is to switch to a possibly even faster compiled implementation that supports the same API.

Note: When using this package in combination with Tk, there may be a naming conflict, as both this package and Tk define a command *scale*. See the Naming conflict section below.

Procedures

The package defines the following public procedures (several exist as specialised procedures, see below):

Constructing matrices and vectors

::math::linearalgebra::mkVector *ndim value*

Create a vector with ndim elements, each with the value *value*.

integer *ndim*

Dimension of the vector (number of components)

double *value*

Uniform value to be used (default: 0.0)

::math::linearalgebra::mkUnitVector *ndim ndir*

Create a unit vector in *ndim*-dimensional space, along the *ndir*-th direction.

integer *ndim*

Dimension of the vector (number of components)

integer *ndir*

Direction (0, ..., ndim-1)

::math::linearalgebra::mkMatrix *nrows ncols value*

Create a matrix with *nrows* rows and *ncols* columns. All elements have the value *value*.

integer *nrows*

Number of rows

integer *ncols*

Number of columns

double *value*

Uniform value to be used (default: 0.0)

::math::linearalgebra::getrow *matrix row* ?imin? ?imax?

Returns a single row of a matrix as a list

list *matrix*

Matrix in question

integer *row*

Index of the row to return

integer *imin*

Minimum index of the column (default: 0)

integer *imax*

Maximum index of the column (default: ncols-1)

::math::linearalgebra::setrow *matrix row newvalues* ?imin? ?imax?

Set a single row of a matrix to new values (this list must have the same number of elements as the number of *columns* in the matrix)

list *matrix*

name of the matrix in question

integer *row*

Index of the row to update

list *newvalues*

List of new values for the row

integer *imin*

Minimum index of the column (default: 0)

integer *imax*

Maximum index of the column (default: ncols-1)

::math::linearalgebra::getcol *matrix col* ?imin? ?imax?

Returns a single column of a matrix as a list

list *matrix*

Matrix in question

integer *col*

Index of the column to return

integer *imin*
Minimum index of the row (default: 0)

integer *imax*
Maximum index of the row (default: nrows-1)

::math::linearalgebra::setcol *matrix col newvalues* ?imin? ?imax?
Set a single column of a matrix to new values (this list must have the same number of elements as the number of *rows* in the matrix)

list *matrix*
name of the matrix in question

integer *col*
Index of the column to update

list *newvalues*
List of new values for the column

integer *imin*
Minimum index of the row (default: 0)

integer *imax*
Maximum index of the row (default: nrows-1)

::math::linearalgebra::getelem *matrix row col*
Returns a single element of a matrix/vector

list *matrix*
Matrix or vector in question

integer *row*
Row of the element

integer *col*
Column of the element (not present for vectors)

::math::linearalgebra::setelem *matrix row* ?col? *newvalue*
Set a single element of a matrix (or vector) to a new value

list *matrix*
name of the matrix in question

integer *row*
Row of the element

integer *col*
Column of the element (not present for vectors)

::math::linearalgebra::swaprows *matrix irow1 irow2* ?imin? ?imax?
Swap two rows in a matrix completely or only a selected part

list *matrix*
name of the matrix in question

integer *irow1*
Index of first row

integer *irow2*

Index of second row

integer *imin*
Minimum column index (default: 0)

integer *imin*
Maximum column index (default: ncols-1)

::math::linearalgebra::swapcols *matrix icol1 icol2* ?imin? ?imax?
Swap two columns in a matrix completely or only a selected part

list *matrix*
name of the matrix in question

integer *irow1*
Index of first column

integer *irow2*
Index of second column

integer *imin*
Minimum row index (default: 0)

integer *imin*
Maximum row index (default: nrows-1)

Querying matrices and vectors

::math::linearalgebra::show *obj* ?format? ?rowsep? ?colsep?
Return a string representing the vector or matrix, for easy printing. (There is currently no way to print fixed sets of columns)

list *obj*
Matrix or vector in question

string *format*
Format for printing the numbers (default: %6.4f)

string *rowsep*
String to use for separating rows (default: newline)

string *colsep*
String to use for separating columns (default: space)

::math::linearalgebra::dim *obj*
Returns the number of dimensions for the object (either 0 for a scalar, 1 for a vector and 2 for a matrix)

any *obj*
Scalar, vector, or matrix

::math::linearalgebra::shape *obj*
Returns the number of elements in each dimension for the object (either an empty list for a scalar, a single number for a vector and a list of the number of rows and columns for a matrix)

any *obj*
Scalar, vector, or matrix

::math::linearalgebra::conforming *type obj1 obj2*

Checks if two objects (vector or matrix) have conforming shapes, that is if they can be applied in an operation like addition or matrix multiplication.

string *type*

Type of check:
- "shape" - the two objects have the same shape (for all element-wise operations)
- "rows" - the two objects have the same number of rows (for use as A and b in a system of linear equations $Ax = b$
- "matmul" - the first object has the same number of columns as the number of rows of the second object. Useful for matrix-matrix or matrix-vector multiplication.

list *obj1*

First vector or matrix (left operand)

list *obj2*

Second vector or matrix (right operand)

::math::linearalgebra::symmetric *matrix* ?eps?

Checks if the given (square) matrix is symmetric. The argument eps is the tolerance.

list *matrix*

Matrix to be inspected

float *eps*

Tolerance for determining approximate equality (defaults to 1.0e-8)

Basic operations

::math::linearalgebra::norm *vector type*

Returns the norm of the given vector. The type argument can be: 1, 2, inf or max, respectively the sum of absolute values, the ordinary Euclidean norm or the max norm.

list *vector*

Vector, list of coefficients

string *type*

Type of norm (default: 2, the Euclidean norm)

::math::linearalgebra::norm_one *vector*

Returns the L1 norm of the given vector, the sum of absolute values

list *vector*

Vector, list of coefficients

::math::linearalgebra::norm_two *vector*

Returns the L2 norm of the given vector, the ordinary Euclidean norm

list *vector*

Vector, list of coefficients

::math::linearalgebra::norm_max *vector* ?index?

Returns the Linf norm of the given vector, the maximum absolute coefficient

list *vector*

Vector, list of coefficients

integer *index*

(optional) if non zero, returns a list made of the maximum value and the index where that

maximum was found. if zero, returns the maximum value.

::math::linearalgebra::normMatrix *matrix type*

Returns the norm of the given matrix. The type argument can be: 1, 2, inf or max, respectively the sum of absolute values, the ordinary Euclidean norm or the max norm.

list *matrix*

Matrix, list of row vectors

string *type*

Type of norm (default: 2, the Euclidean norm)

::math::linearalgebra::dotproduct *vect1 vect2*

Determine the inproduct or dot product of two vectors. These must have the same shape (number of dimensions)

list *vect1*

First vector, list of coefficients

list *vect2*

Second vector, list of coefficients

::math::linearalgebra::unitLengthVector *vector*

Return a vector in the same direction with length 1.

list *vector*

Vector to be normalized

::math::linearalgebra::normalizeStat *mv*

Normalize the matrix or vector in a statistical sense: the mean of the elements of the columns of the result is zero and the standard deviation is 1.

list *mv*

Vector or matrix to be normalized in the above sense

::math::linearalgebra::axpy *scale mv1 mv2*

Return a vector or matrix that results from a "daxpy" operation, that is: compute a*x+y (a a scalar and x and y both vectors or matrices of the same shape) and return the result.

Specialised variants are: axpy_vect and axpy_mat (slightly faster, but no check on the arguments)

double *scale*

The scale factor for the first vector/matrix (a)

list *mv1*

First vector or matrix (x)

list *mv2*

Second vector or matrix (y)

::math::linearalgebra::add *mv1 mv2*

Return a vector or matrix that is the sum of the two arguments (x+y)

Specialised variants are: add_vect and add_mat (slightly faster, but no check on the arguments)

list *mv1*

First vector or matrix (x)

list *mv2*
Second vector or matrix (y)

::math::linearalgebra::sub *mv1 mv2*
Return a vector or matrix that is the difference of the two arguments (x-y)

Specialised variants are: sub_vect and sub_mat (slightly faster, but no check on the arguments)

list *mv1*
First vector or matrix (x)

list *mv2*
Second vector or matrix (y)

::math::linearalgebra::scale *scale mv*
Scale a vector or matrix and return the result, that is: compute a*x.

Specialised variants are: scale_vect and scale_mat (slightly faster, but no check on the arguments)

double *scale*
The scale factor for the vector/matrix (a)

list *mv*
Vector or matrix (x)

::math::linearalgebra::rotate *c s vect1 vect2*
Apply a planar rotation to two vectors and return the result as a list of two vectors: c*x-s*y and s*x+c*y. In algorithms you can often easily determine the cosine and sine of the angle, so it is more efficient to pass that information directly.

double *c*
The cosine of the angle

double *s*
The sine of the angle

list *vect1*
First vector (x)

list *vect2*
Seocnd vector (x)

::math::linearalgebra::transpose *matrix*
Transpose a matrix

list *matrix*
Matrix to be transposed

::math::linearalgebra::matmul *mv1 mv2*
Multiply a vector/matrix with another vector/matrix. The result is a matrix, if both x and y are matrices or both are vectors, in which case the "outer product" is computed. If one is a vector and the other is a matrix, then the result is a vector.

list *mv1*
First vector/matrix (x)

list *mv2*
Second vector/matrix (y)

::math::linearalgebra::angle *vect1 vect2*
Compute the angle between two vectors (in radians)
list *vect1*
First vector
list *vect2*
Second vector

::math::linearalgebra::crossproduct *vect1 vect2*
Compute the cross product of two (three-dimensional) vectors
list *vect1*
First vector
list *vect2*
Second vector

::math::linearalgebra::matmul *mv1 mv2*
Multiply a vector/matrix with another vector/matrix. The result is a matrix, if both x and y are matrices or both are vectors, in which case the "outer product" is computed. If one is a vector and the other is a matrix, then the result is a vector.
list *mv1*
First vector/matrix (x)
list *mv2*
Second vector/matrix (y)

Common matrices and test matrices

::math::linearalgebra::mkIdentity *size*
Create an identity matrix of dimension *size*.
integer *size*
Dimension of the matrix

::math::linearalgebra::mkDiagonal *diag*
Create a diagonal matrix whose diagonal elements are the elements of the vector *diag*.
list *diag*
Vector whose elements are used for the diagonal

::math::linearalgebra::mkRandom *size*
Create a square matrix whose elements are uniformly distributed random numbers between 0 and 1 of dimension *size*.
integer *size*
Dimension of the matrix

::math::linearalgebra::mkTriangular *size* ?uplo? ?value?
Create a triangular matrix with non-zero elements in the upper or lower part, depending on argument *uplo*.
integer *size*

Dimension of the matrix

string *uplo*

Fill the upper (U) or lower part (L)

double *value*

Value to fill the matrix with

::math::linearalgebra::mkHilbert *size*

Create a Hilbert matrix of dimension *size*. Hilbert matrices are very ill-conditioned with respect to eigenvalue/eigenvector problems. Therefore they are good candidates for testing the accuracy of algorithms and implementations.

integer *size*

Dimension of the matrix

::math::linearalgebra::mkDingdong *size*

Create a "dingdong" matrix of dimension *size*. Dingdong matrices are imprecisely represented, but have the property of being very stable in such algorithms as Gauss elimination.

integer *size*

Dimension of the matrix

::math::linearalgebra::mkOnes *size*

Create a square matrix of dimension *size* whose entries are all 1.

integer *size*

Dimension of the matrix

::math::linearalgebra::mkMoler *size*

Create a Moler matrix of size *size*. (Moler matrices have a very simple Choleski decomposition. It has one small eigenvalue and it can easily upset elimination methods for systems of linear equations.)

integer *size*

Dimension of the matrix

::math::linearalgebra::mkFrank *size*

Create a Frank matrix of size *size*. (Frank matrices are fairly well-behaved matrices)

integer *size*

Dimension of the matrix

::math::linearalgebra::mkBorder *size*

Create a bordered matrix of size *size*. (Bordered matrices have a very low rank and can upset certain specialised algorithms.)

integer *size*

Dimension of the matrix

::math::linearalgebra::mkWilkinsonW+ *size*

Create a Wilkinson W+ of size *size*. This kind of matrix has pairs of eigenvalues that are very close together. Usually the order (size) is odd.

integer *size*

Dimension of the matrix

::math::linearalgebra::mkWilkinsonW- *size*

Create a Wilkinson W- of size *size*. This kind of matrix has pairs of eigenvalues with opposite signs, when the order (size) is odd.

integer *size*

Dimension of the matrix

Common algorithms

::math::linearalgebra::solveGauss *matrix bvect*

Solve a system of linear equations (Ax=b) using Gauss elimination. Returns the solution (x) as a vector or matrix of the same shape as bvect.

list *matrix*

Square matrix (matrix A)

list *bvect*

Vector or matrix whose columns are the individual b-vectors

::math::linearalgebra::solvePGauss *matrix bvect*

Solve a system of linear equations (Ax=b) using Gauss elimination with partial pivoting. Returns the solution (x) as a vector or matrix of the same shape as bvect.

list *matrix*

Square matrix (matrix A)

list *bvect*

Vector or matrix whose columns are the individual b-vectors

::math::linearalgebra::solveTriangular *matrix bvect* ?uplo?

Solve a system of linear equations (Ax=b) by backward substitution. The matrix is supposed to be upper-triangular.

list *matrix*

Lower or upper-triangular matrix (matrix A)

list *bvect*

Vector or matrix whose columns are the individual b-vectors

string *uplo*

Indicates whether the matrix is lower-triangular (L) or upper-triangular (U). Defaults to "U".

::math::linearalgebra::solveGaussBand *matrix bvect*

Solve a system of linear equations (Ax=b) using Gauss elimination, where the matrix is stored as a band matrix (*cf.* Storage). Returns the solution (x) as a vector or matrix of the same shape as bvect.

list *matrix*

Square matrix (matrix A; in band form)

list *bvect*

Vector or matrix whose columns are the individual b-vectors

::math::linearalgebra::solveTriangularBand *matrix bvect*

Solve a system of linear equations (Ax=b) by backward substitution. The matrix is supposed

to be upper-triangular and stored in band form.

list *matrix*
Upper-triangular matrix (matrix A)

list *bvect*
Vector or matrix whose columns are the individual b-vectors

::math::linearalgebra::determineSVD *A eps*
Determines the Singular Value Decomposition of a matrix: A = U S Vtrans. Returns a list with the matrix U, the vector of singular values S and the matrix V.

list *A*
Matrix to be decomposed

float *eps*
Tolerance (defaults to 2.3e-16)

::math::linearalgebra::eigenvectorsSVD *A eps*
Determines the eigenvectors and eigenvalues of a real *symmetric* matrix, using SVD. Returns a list with the matrix of normalized eigenvectors and their eigenvalues.

list *A*
Matrix whose eigenvalues must be determined

float *eps*
Tolerance (defaults to 2.3e-16)

::math::linearalgebra::leastSquaresSVD *A y qmin eps*
Determines the solution to a least-sqaures problem Ax ~ y via singular value decomposition. The result is the vector x.

Note that if you add a column of 1s to the matrix, then this column will represent a constant like in: y = a*x1 + b*x2 + c. To force the intercept to be zero, simply leave it out.

list *A*
Matrix of independent variables

list *y*
List of observed values

float *qmin*
Minimum singular value to be considered (defaults to 0.0)

float *eps*
Tolerance (defaults to 2.3e-16)

::math::linearalgebra::choleski *matrix*
Determine the Choleski decomposition of a symmetric positive semidefinite matrix (this condition is not checked!). The result is the lower-triangular matrix L such that L Lt = matrix.

list *matrix*
Matrix to be decomposed

::math::linearalgebra::orthonormalizeColumns *matrix*
Use the modified Gram-Schmidt method to orthogonalize and normalize the *columns* of the given matrix and return the result.

list *matrix*

Matrix whose columns must be orthonormalized

::math::linearalgebra::orthonormalizeRows *matrix*

Use the modified Gram-Schmidt method to orthogonalize and normalize the *rows* of the given matrix and return the result.

list *matrix*

Matrix whose rows must be orthonormalized

::math::linearalgebra::dger *matrix alpha x y* ?scope?

Perform the rank 1 operation A + alpha*x*y' inline (that is: the matrix A is adjusted). For convenience the new matrix is also returned as the result.

list *matrix*

Matrix whose rows must be adjusted

double *alpha*

Scale factor

list *x*

A column vector

list *y*

A column vector

list *scope*

If not provided, the operation is performed on all rows/columns of A if provided, it is expected to be the list {imin imax jmin jmax} where:
- *imin* Minimum row index
- *imax* Maximum row index
- *jmin* Minimum column index
- *jmax* Maximum column index

::math::linearalgebra::dgetrf *matrix*

Computes an LU factorization of a general matrix, using partial, pivoting with row interchanges. Returns the permutation vector.

The factorization has the form

```
P * A = L * U
```

where P is a permutation matrix, L is lower triangular with unit diagonal elements, and U is upper triangular. Returns the permutation vector, as a list of length n-1. The last entry of the permutation is not stored, since it is implicitely known, with value n (the last row is not swapped with any other row). At index #i of the permutation is stored the index of the row #j which is swapped with row #i at step #i. That means that each index of the permutation gives the permutation at each step, not the cumulated permutation matrix, which is the product of permutations.

list *matrix*

On entry, the matrix to be factored. On exit, the factors L and U from the factorization P*A = L*U; the unit diagonal elements of L are not stored.

::math::linearalgebra::det *matrix*

Returns the determinant of the given matrix, based on PA=LU decomposition, i.e. Gauss partial pivotal.

list *matrix*
Square matrix (matrix A)

list *ipiv*
The pivots (optionnal). If the pivots are not provided, a PA=LU decomposition is performed. If the pivots are provided, we assume that it contains the pivots and that the matrix A contains the L and U factors, as provided by dgterf. b-vectors

::math::linearalgebra::largesteigen *matrix tolerance maxiter*
Returns a list made of the largest eigenvalue (in magnitude) and associated eigenvector. Uses iterative Power Method as provided as algorithm #7.3.3 of Golub & Van Loan. This algorithm is used here for a dense matrix (but is usually used for sparse matrices).

list *matrix*
Square matrix (matrix A)

double *tolerance*
The relative tolerance of the eigenvalue (default:1.e-8).

integer *maxiter*
The maximum number of iterations (default:10).

Compability with the LA package Two procedures are provided for compatibility with Hume's LA package:

::math::linearalgebra::to_LA *mv*
Transforms a vector or matrix into the format used by the original LA package.

list *mv*
Matrix or vector

::math::linearalgebra::from_LA *mv*
Transforms a vector or matrix from the format used by the original LA package into the format used by the present implementation.

list *mv*
Matrix or vector as used by the LA package

Storage

While most procedures assume that the matrices are given in full form, the procedures *solveGaussBand* and *solveTriangularBand* assume that the matrices are stored as *band matrices* . This common type of "sparse" matrices is related to ordinary matrices as follows:

- "A" is a full-size matrix with N rows and M columns.
- "B" is a band matrix, with m upper and lower diagonals and n rows.
- "B" can be stored in an ordinary matrix of (2m+1) columns (one for each off-diagonal and the main diagonal) and n rows.
- Element i,j (i = -m,...,m; j =1,...,n) of "B" corresponds to element k,j of "A" where k = M+i-1 and M is at least (!) n, the number of rows in "B".
- To set element (i,j) of matrix "B" use:

```
    setelem B $j [expr {$N+$i-1}] $value
```

(There is no convenience procedure for this yet)

Remarks on the implementation

There is a difference between the original LA package by Hume and the current implementation. Whereas the LA package uses a linear list, the current package uses lists of lists to represent matrices. It turns out that with this representation, the algorithms are faster and easier to implement.

The LA package was used as a model and in fact the implementation of, for instance, the SVD algorithm was taken from that package. The set of procedures was expanded using ideas from the well-known BLAS library and some algorithms were updated from the second edition of J.C. Nash's book, Compact Numerical Methods for Computers, (Adam Hilger, 1990) that inspired the LA package.

Two procedures are provided to make the transition between the two implementations easier: *to_LA* and *from_LA*. They are described above.

Todo

Odds and ends: the following algorithms have not been implemented yet:

- determineQR
- certainlyPositive, diagonallyDominant

Naming conflict

If you load this package in a Tk-enabled shell like wish, then the command

```
namespace import ::math::linearalgebra
```

results in an error message about "scale". This is due to the fact that Tk defines all its commands in the global namespace. The solution is to import the linear algebra commands in a namespace that is not the global one:

```
package require math::linearalgebra
namespace eval compute {
    namespace import ::math::linearalgebra::*
    ... use the linear algebra version of scale ...
}
```

To use Tk's scale command in that same namespace you can rename it:

```
namespace eval compute {
    rename ::scale scaleTk
    scaleTk .scale ...
}
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *math :: linearalgebra* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Mathematics

Copyright

```
Copyright (c) 2004-2008 Arjen Markus
    <arjenmarkus@users.sourceforge.net>
Copyright (c) 2004 Ed Hume <http://www.hume.com/contact.us.htm>
Copyright (c) 2008 Michael Buadin <relaxkmike@users.sourceforge.net>
```


machineparameters
Keywords: machineparameters

Name
math::machineparameters - Compute double precision machine parameters.

Synopsis
package require Tcl 8.5 9
package require snit
package require math::machineparameters 0.2
machineparameters create *objectname* ?*options...*?
objectname **configure** ?*options...*?
objectname **cget** *opt*
objectname **destroy**
objectname **compute**
objectname **get** *key*
objectname **tostring**
objectname **print**

Description
The *math::machineparameters* package is the Tcl equivalent of the DLAMCH LAPACK function. In floating point systems, a floating point number is represented by

```
x = +/- d1 d2 ... dt basis^e
```

where digits satisfy

```
0 <= di <= basis - 1, i = 1, t
```

with the convention:

- t is the size of the mantissa
- basis is the basis (the "radix")

The **compute** method computes all machine parameters. Then, the **get** method can be used to get each parameter. The **print** method prints a report on standard output.

Example

In the following example, one compute the parameters of a desktop under Linux with the following Tcl 8.4.19 properties:

```
% parray tcl_platform
tcl_platform(byteOrder) = littleEndian
tcl_platform(machine)   = i686
tcl_platform(os)        = Linux
tcl_platform(osVersion) = 2.6.24-19-generic
tcl_platform(platform)  = unix
tcl_platform(tip,268)   = 1
tcl_platform(tip,280)   = 1
tcl_platform(user)      = <username>
tcl_platform(wordSize)  = 4
```

The following example creates a machineparameters object, computes the properties and displays it.

```
    set pp [machineparameters create %AUTO%]
    $pp compute
    $pp print
    $pp destroy
```

This prints out:

```
    Machine parameters
    Epsilon : 1.11022302463e-16
    Beta : 2
    Rounding : proper
    Mantissa : 53
    Maximum exponent : 1024
    Minimum exponent : -1021
    Overflow threshold : 8.98846567431e+307
    Underflow threshold : 2.22507385851e-308
```

That compares well with the results produced by Lapack 3.1.1:

```
    Epsilon                      =   1.11022302462515654E-016
    Safe minimum                 =   2.22507385850720138E-308
    Base                         =    2.0000000000000000
    Precision                    =   2.22044604925031308E-016
    Number of digits in mantissa =    53.000000000000000
    Rounding mode                =   1.00000000000000000
    Minimum exponent             =   -1021.0000000000000
    Underflow threshold          =   2.22507385850720138E-308
    Largest exponent             =    1024.0000000000000
    Overflow threshold           =   1.79769313486231571E+308
    Reciprocal of safe minimum   =   4.49423283715578977E+307
```

The following example creates a machineparameters object, computes the properties and gets

the epsilon for the machine.

```
    set pp [machineparameters create %AUTO%]
    $pp compute
    set eps [$pp get -epsilon]
    $pp destroy
```

References

- "Algorithms to Reveal Properties of Floating-Point Arithmetic", Michael A. Malcolm, Stanford University, Communications of the ACM, Volume 15, Issue 11 (November 1972), Pages: 949 - 951
- "More on Algorithms that Reveal Properties of Floating, Point Arithmetic Units", W. Morven Gentleman, University of Waterloo, Scott B. Marovich, Purdue University, Communications of the ACM, Volume 17, Issue 5 (May 1974), Pages: 276 - 277

Class API

machineparameters create *objectname* ?*options*...?

The command creates a new machineparameters object and returns the fully qualified name of the object command as its result.

-verbose *verbose*

Set this option to 1 to enable verbose logging. This option is mainly for debug purposes. The default value of *verbose* is 0.

Object API

***objectname* configure ?*options*...?**

The command configure the options of the object *objectname*. The options are the same as the static method **create**.

objectname* cget *opt

Returns the value of the option which name is *opt*. The options are the same as the method **create** and **configure**.

***objectname* destroy**

Destroys the object *objectname*.

***objectname* compute**

Computes the machine parameters.

objectname* get *key

Returns the value corresponding with given key. The following is the list of available keys.

- -epsilon: smallest value so that 1+epsilon>1 is false
- -rounding: The rounding mode used on the machine. The rounding occurs when more than t digits would be required to represent the number. Two modes can be determined with the current system: "chop" means than only t digits are kept, no matter the value of the number "proper" means that another rounding mode is used, be it "round to nearest", "round up", "round down".
- -basis: the basis of the floating-point representation. The basis is usually 2, i.e. binary representation (for example IEEE 754 machines), but some machines (like HP calculators for example) uses 10, or 16, etc...
- -mantissa: the number of bits in the mantissa

- -exponentmax: the largest positive exponent before overflow occurs
- -exponentmin: the largest negative exponent before (gradual) underflow occurs
- -vmax: largest positive value before overflow occurs
- -vmin: largest negative value before (gradual) underflow occurs

***objectname* tostring**
Return a report for machine parameters.

***objectname* print**
Print machine parameters on standard output.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *math* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Copyright

```
Copyright (c) 2008 Michael Baudin <michael.baudin@sourceforge.net>
```

math

Keywords: math, statistics

Name

math - Tcl Math Library

Synopsis

package require Tcl 8.5 9
package require math ?**1.2.6**?
::math::cov *value value* ?*value* ...?
::math::integrate *list of xy value pairs*
::math::fibonacci *n*
::math::max *value* ?*value* ...?
::math::mean *value* ?*value* ...?
::math::min *value* ?*value* ...?
::math::product *value* ?*value* ...?
::math::random ?*value1*? ?*value2*?
::math::sigma *value value* ?*value* ...?
::math::stats *value value* ?*value* ...?
::math::sum *value* ?*value* ...?

Description

The **math** package provides utility math functions.

Besides a set of basic commands, available via the package *math*, there are more specialised

packages:

- **math::bigfloat** - Arbitrary-precision floating-point arithmetic
- **math::bignum** - Arbitrary-precision integer arithmetic
- **math::calculus::romberg** - Robust integration methods for functions of one variable, using Romberg integration
- **math::calculus** - Integration of functions, solving ordinary differential equations
- **math::combinatorics** - Procedures for various combinatorial functions (for instance the Gamma function and "k out of n")
- **math::complexnumbers** - Complex number arithmetic
- **math::constants** - A set of well-known mathematical constants, such as Pi, E, and the golden ratio
- **math::fourier** - Discrete Fourier transforms
- **math::fuzzy** - Fuzzy comparisons of floating-point numbers
- **math::geometry** - 2D geometrical computations
- **math::interpolate** - Various interpolation methods
- **math::linearalgebra** - Linear algebra package
- **math::optimize** - Optimization methods
- **math::polynomials** - Polynomial arithmetic (includes families of classical polynomials)
- **math::rationalfunctions** - Arithmetic of rational functions
- **math::roman** - Manipulation (including arithmetic) of Roman numerals
- **math::special** - Approximations of special functions from mathematical physics
- **math::statistics** - Statistical operations and tests

Basic commands

::math::cov *value value* ?*value ...*?
Return the coefficient of variation expressed as percent of two or more numeric values.

::math::integrate *list of xy value pairs*
Return the area under a "curve" defined by a set of x,y pairs and the error bound as a list.

::math::fibonacci *n*
Return the *n*'th Fibonacci number.

::math::max *value* ?*value ...*?
Return the maximum of one or more numeric values.

::math::mean *value* ?*value ...*?
Return the mean, or "average" of one or more numeric values.

::math::min *value* ?*value ...*?
Return the minimum of one or more numeric values.

::math::product *value* ?*value ...*?
Return the product of one or more numeric values.

::math::random ?*value1*? ?*value2*?
Return a random number. If no arguments are given, the number is a floating point value between 0 and 1. If one argument is given, the number is an integer value between 0 and *value1*. If two arguments are given, the number is an integer value between *value1* and *value2*.

::math::sigma *value value* ?*value ...*?
Return the sample standard deviation of two or more numeric values.

::math::stats ***value value ?value ...?***
Return the mean, standard deviation, and coefficient of variation (as percent) as a list.

::math::sum ***value ?value ...?***
Return the sum of one or more numeric values.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *math* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Mathematics

math_geometry

Keywords: math_geometry, angle, distance, line, math, plane geometry, point

Name

math::geometry - Geometrical computations

Synopsis

package require Tcl ?**8.5 9**?
package require math::geometry ?**1.4.2**?
::math::geometry::+ *point1 point2*
::math::geometry::- *point1 point2*
::math::geometry::p *x y*
::math::geometry::distance *point1 point2*
::math::geometry::length *point*
::math::geometry::s* *factor point*
::math::geometry::direction *angle*
::math::geometry::h *length*
::math::geometry::v *length*
::math::geometry::between *point1 point2 s*
::math::geometry::octant *point*
::math::geometry::rect *nw se*
::math::geometry::nwse *rect*
::math::geometry::angle *line*
::math::geometry::angleBetween *vector1 vector2*
::math::geometry::inproduct *vector1 vector2*
::math::geometry::areaParallellogram *vector1 vector2*
::math::geometry::calculateDistanceToLine *P line*
::math::geometry::calculateDistanceToLineSegment *P linesegment*
::math::geometry::calculateDistanceToPolyline *P polyline*

::math::geometry::calculateDistanceToPolygon *P polygon*
::math::geometry::findClosestPointOnLine *P line*
::math::geometry::findClosestPointOnLineSegment *P linesegment*
::math::geometry::findClosestPointOnPolyline *P polyline*
::math::geometry::lengthOfPolyline *polyline*
::math::geometry::movePointInDirection *P direction dist*
::math::geometry::lineSegmentsIntersect *linesegment1 linesegment2*
::math::geometry::findLineSegmentIntersection *linesegment1 linesegment2*
::math::geometry::findLineIntersection *line1 line2*
::math::geometry::polylinesIntersect *polyline1 polyline2*
::math::geometry::polylinesBoundingIntersect *polyline1 polyline2 granularity*
::math::geometry::intervalsOverlap *y1 y2 y3 y4 strict*
::math::geometry::rectanglesOverlap *P1 P2 Q1 Q2 strict*
::math::geometry::bbox *polyline*
::math::geometry::overlapBBox *polyline1 polyline2 ?strict?*
::math::geometry::pointInsideBBox *bbox point*
::math::geometry::cathetusPoint *pa pb cathetusLength ?location?*
::math::geometry::parallel *line offset ?orient?*
::math::geometry::unitVector *line*
::math::geometry::pointInsidePolygon *P polyline*
::math::geometry::pointInsidePolygonAlt *P polyline*
::math::geometry::rectangleInsidePolygon *P1 P2 polyline*
::math::geometry::areaPolygon *polygon*
::math::geometry::translate *vector polyline*
::math::geometry::rotate *angle polyline*
::math::geometry::rotateAbout *p angle polyline*
::math::geometry::reflect *angle polyline*
::math::geometry::degToRad *angle*
::math::geometry::radToDeg *angle*
::math::geometry::circle *centre radius*
::math::geometry::circleTwoPoints *point1 point2*
::math::geometry::pointInsideCircle *point circle*
::math::geometry::lineIntersectsCircle *line circle*
::math::geometry::lineSegmentIntersectsCircle *segment circle*
::math::geometry::intersectionLineWithCircle *line circle*
::math::geometry::intersectionCircleWithCircle *circle1 circle2*
::math::geometry::tangentLinesToCircle *point circle*
::math::geometry::intersectionPolylines *polyline1 polyline2 ?mode? ?granularity?*
::math::geometry::intersectionPolylineCircle *polyline circle ?mode? ?granularity?*
::math::geometry::polylineCutOrigin *polyline1 polyline2 ?granularity?*
::math::geometry::polylineCutEnd *polyline1 polyline2 ?granularity?*
::math::geometry::splitPolyline *polyline numberVertex*
::math::geometry::enrichPolyline *polyline accuracy*
::math::geometry::cleanupPolyline *polyline*

Description

The **math::geometry** package is a collection of functions for computations and manipulations on two-dimensional geometrical objects, such as points, lines and polygons.

The geometrical objects are implemented as plain lists of coordinates. For instance a line is defined by a list of four numbers, the x- and y-coordinate of a first point and the x- and y-coordinates of a second point on the line.

Note: In version 1.4.0 an inconsistency was repaired - see https://core.tcl-lang.org/tcllib/tktview?name=fb4812f82b. More in Coordinate system

The various types of object are recognised by the number of coordinate pairs and the context in which they are used: a list of four elements can be regarded as an infinite line, a finite line segment but also as a polyline of one segment and a point set of two points.

Currently the following types of objects are distinguished:

- *point* - a list of two coordinates representing the x- and y-coordinates respectively.
- *line* - a list of four coordinates, interpreted as the x- and y-coordinates of two distinct points on the line.
- *line segment* - a list of four coordinates, interpreted as the x- and y-coordinates of the first and the last points on the line segment.
- *polyline* - a list of an even number of coordinates, interpreted as the x- and y-coordinates of an ordered set of points.
- *polygon* - like a polyline, but the implicit assumption is that the polyline is closed (if the first and last points do not coincide, the missing segment is automatically added).
- *point set* - again a list of an even number of coordinates, but the points are regarded without any ordering.
- *circle* - a list of three numbers, the first two are the coordinates of the centre and the third is the radius.

Procedures

The package defines the following public procedures:

::math::geometry::+ *point1 point2*
Compute the sum of the two vectors given as points and return it. The result is a vector as well.

::math::geometry::- *point1 point2*
Compute the difference (point1 - point2) of the two vectors given as points and return it. The result is a vector as well.

::math::geometry::p *x y*
Construct a point from its coordinates and return it as the result of the command.

::math::geometry::distance *point1 point2*
Compute the distance between the two points and return it as the result of the command. This is in essence the same as

```
    math::geometry::length [math::geomtry::- point1 point2]
```

::math::geometry::length *point*
Compute the length of the vector and return it as the result of the command.

::math::geometry::s* *factor point*
Scale the vector by the factor and return it as the result of the command. This is a vector as

well.

::math::geometry::direction *angle*
Given the angle in degrees this command computes and returns the unit vector pointing into this direction. The vector for angle == 0 points to the right (east), and for angle == 90 up (north).

::math::geometry::h *length*
Returns a horizontal vector on the X-axis of the specified length. Positive lengths point to the right (east).

::math::geometry::v *length*
Returns a vertical vector on the Y-axis of the specified length. Positive lengths point down (south).

::math::geometry::between *point1 point2 s*
Compute the point which is at relative distance *s* between the two points and return it as the result of the command. A relative distance of **0** returns *point1*, the distance **1** returns *point2*. Distances < 0 or > 1 extrapolate along the line between the two point.

::math::geometry::octant *point*
Compute the octant of the circle the point is in and return it as the result of the command. The possible results are
1. east
2. northeast
3. north
4. northwest
5. west
6. southwest
7. south
8. southeast

Each octant is the arc of the circle +/- 22.5 degrees from the cardinal direction the octant is named for.

::math::geometry::rect *nw se*
Construct a rectangle from its northwest and southeast corners and return it as the result of the command.

::math::geometry::nwse *rect*
Extract the northwest and southeast corners of the rectangle and return them as the result of the command (a 2-element list containing the points, in the named order).

::math::geometry::angle *line*
Calculate the angle from the positive x-axis to a given line (in two dimensions only).
list *line*
Coordinates of the line

::math::geometry::angleBetween *vector1 vector2*
Calculate the angle between two vectors (in degrees)
list *vector1*
First vector
list *vector2*
Second vector

::math::geometry::inproduct *vector1 vector2*

Calculate the inner product of two vectors

list *vector1*

First vector

list *vector2*

Second vector

::math::geometry::areaParallellogram *vector1 vector2*

Calculate the area of the parallellogram with the two vectors as its sides

list *vector1*

First vector

list *vector2*

Second vector

::math::geometry::calculateDistanceToLine *P line*

Calculate the distance of point P to the (infinite) line and return the result

list *P*

List of two numbers, the coordinates of the point

list *line*

List of four numbers, the coordinates of two points on the line

::math::geometry::calculateDistanceToLineSegment *P linesegment*

Calculate the distance of point P to the (finite) line segment and return the result.

list *P*

List of two numbers, the coordinates of the point

list *linesegment*

List of four numbers, the coordinates of the first and last points of the line segment

::math::geometry::calculateDistanceToPolyline *P polyline*

Calculate the distance of point P to the polyline and return the result. Note that a polyline needs not to be closed.

list *P*

List of two numbers, the coordinates of the point

list *polyline*

List of numbers, the coordinates of the vertices of the polyline

::math::geometry::calculateDistanceToPolygon *P polygon*

Calculate the distance of point P to the polygon and return the result. If the list of coordinates is not closed (first and last points differ), it is automatically closed.

list *P*

List of two numbers, the coordinates of the point

list *polygon*

List of numbers, the coordinates of the vertices of the polygon

::math::geometry::findClosestPointOnLine *P line*

Return the point on a line which is closest to a given point.

list *P*

List of two numbers, the coordinates of the point

list *line*

List of four numbers, the coordinates of two points on the line

::math::geometry::findClosestPointOnLineSegment *P linesegment*

Return the point on a *line segment* which is closest to a given point.

list *P*

List of two numbers, the coordinates of the point

list *linesegment*

List of four numbers, the first and last points on the line segment

::math::geometry::findClosestPointOnPolyline *P polyline*

Return the point on a *polyline* which is closest to a given point.

list *P*

List of two numbers, the coordinates of the point

list *polyline*

List of numbers, the vertices of the polyline

::math::geometry::lengthOfPolyline *polyline*

Return the length of the *polyline* (note: it not regarded as a polygon)

list *polyline*

List of numbers, the vertices of the polyline

::math::geometry::movePointInDirection *P direction dist*

Move a point over a given distance in a given direction and return the new coordinates (in two dimensions only).

list *P*

Coordinates of the point to be moved

double *direction*

Direction (in degrees; 0 is to the right, 90 upwards)

list *dist*

Distance over which to move the point

::math::geometry::lineSegmentsIntersect *linesegment1 linesegment2*

Check if two line segments intersect or coincide. Returns 1 if that is the case, 0 otherwise (in two dimensions only). If an endpoint of one segment lies on the other segment (or is very close to the segment), they are considered to intersect

list *linesegment1*

First line segment

list *linesegment2*

Second line segment

::math::geometry::findLineSegmentIntersection *linesegment1 linesegment2*

Find the intersection point of two line segments. Return the coordinates or the keywords "coincident" or "none" if the line segments coincide or have no points in common (in two dimensions only).

list *linesegment1*

First line segment

list *linesegment2*
Second line segment

::math::geometry::findLineIntersection *line1 line2*
Find the intersection point of two (infinite) lines. Return the coordinates or the keywords "coincident" or "none" if the lines coincide or have no points in common (in two dimensions only).

list *line1*
First line

list *line2*
Second line

See section References for details on the algorithm and math behind it.

::math::geometry::polylinesIntersect *polyline1 polyline2*
Check if two polylines intersect or not (in two dimensions only).

list *polyline1*
First polyline

list *polyline2*
Second polyline

::math::geometry::polylinesBoundingIntersect *polyline1 polyline2 granularity*
Check whether two polylines intersect, but reduce the correctness of the result to the given granularity. Use this for faster, but weaker, intersection checking.

How it works:

Each polyline is split into a number of smaller polylines, consisting of granularity points each. If a pair of those smaller lines' bounding boxes intersect, then this procedure returns 1, otherwise it returns 0.

list *polyline1*
First polyline

list *polyline2*
Second polyline

int *granularity*
Number of points in each part (<=1 means check every edge)

::math::geometry::intervalsOverlap *y1 y2 y3 y4 strict*
Check if two intervals overlap.

double *y1,y2*
Begin and end of first interval

double *y3,y4*
Begin and end of second interval

logical *strict*
Check for strict or non-strict overlap

::math::geometry::rectanglesOverlap *P1 P2 Q1 Q2 strict*

Check if two rectangles overlap.

list *P1*
upper-left corner of the first rectangle

list *P2*
lower-right corner of the first rectangle

list *Q1*
upper-left corner of the second rectangle

list *Q2*
lower-right corner of the second rectangle

list *strict*
choosing strict or non-strict interpretation

::math::geometry::bbox *polyline*
Calculate the bounding box of a polyline. Returns a list of four coordinates: the upper-left and the lower-right corner of the box.

list *polyline*
The polyline to be examined

::math::geometry::overlapBBox *polyline1 polyline2* ?strict?
Check if the bounding boxes of two polylines overlap or not.

Arguments:

list *polyline1*
The first polyline

list *polyline1*
The second polyline

int *strict*
Whether strict overlap is to checked (1) or if the bounding boxes may touch (0, default)

::math::geometry::pointInsideBBox *bbox point*
Check if the point is inside or on the bounding box or not. Arguments:

list *bbox*
The bounding box given as a list of x/y coordinates

list *point*
The point to be checked

::math::geometry::cathetusPoint *pa pb cathetusLength* ?location?
Return the third point of the rectangular triangle defined by the two given end points of the hypothenusa. The triangle's side from point A (or B, if the location is given as "b") to the third point is the cathetus length. If the cathetus' length is lower than the length of the hypothenusa, an empty list is returned.

Arguments:

list *pa*
The starting point on hypotenuse

list *pb*
The ending point on hypotenuse

float *cathetusLength*
The length of the cathetus of the triangle

string *location*
The location of the given cathetus, "a" means given cathetus shares point pa (default) "b" means given cathetus shares point pb

::math::geometry::parallel *line offset* ?orient?
Return a line parallel to the given line, with a distance "offset". The orientation is determined by the two points defining the line.

Arguments:
list *line*
The given line

float *offset*
The distance to the given line

string *orient*
Orientation of the new line with respect to the given line (defaults to "right")

::math::geometry::unitVector *line*
Return a unit vector from the given line or direction, if the *line* argument is a single point (then a line through the origin is assumed) Arguments:
list *line*
The line in question (or a single point, implying a line through the origin)

::math::geometry::pointInsidePolygon *P polyline*
Determine if a point is completely inside a polygon. If the point touches the polygon, then the point is not completely inside the polygon.
list *P*
Coordinates of the point

list *polyline*
The polyline to be examined

::math::geometry::pointInsidePolygonAlt *P polyline*
Determine if a point is completely inside a polygon. If the point touches the polygon, then the point is not completely inside the polygon. *Note:* this alternative procedure uses the so-called winding number to determine this. It handles self-intersecting polygons in a "natural" way.
list *P*
Coordinates of the point

list *polyline*
The polyline to be examined

::math::geometry::rectangleInsidePolygon *P1 P2 polyline*
Determine if a rectangle is completely inside a polygon. If polygon touches the rectangle, then the rectangle is not complete inside the polygon.
list *P1*
Upper-left corner of the rectangle

list *P2*

Lower-right corner of the rectangle

list *polygon*

The polygon in question

::math::geometry::areaPolygon *polygon*

Calculate the area of a polygon.

list *polygon*

The polygon in question

::math::geometry::translate *vector polyline*

Translate a polyline over a given vector

list *vector*

Translation vector

list *polyline*

The polyline to be translated

::math::geometry::rotate *angle polyline*

Rotate a polyline over a given angle (degrees) around the origin

list *angle*

Angle over which to rotate the polyline (degrees)

list *polyline*

The polyline to be rotated

::math::geometry::rotateAbout *p angle polyline*

Rotate a polyline around a given point p and return the new polyline.

Arguments:

list *p*

The point of rotation

float *angle*

The angle over which to rotate the polyline (degrees)

list *polyline*

The polyline to be rotated

::math::geometry::reflect *angle polyline*

Reflect a polyline in a line through the origin at a given angle (degrees) to the x-axis

list *angle*

Angle of the line of reflection (degrees)

list *polyline*

The polyline to be reflected

::math::geometry::degToRad *angle*

Convert from degrees to radians

list *angle*

Angle in degrees

::math::geometry::radToDeg *angle*

Convert from radians to degrees

list *angle*
Angle in radians

::math::geometry::circle *centre radius*

Convenience procedure to create a circle from a point and a radius.

list *centre*
Coordinates of the circle centre

list *radius*
Radius of the circle

::math::geometry::circleTwoPoints *point1 point2*

Convenience procedure to create a circle from two points on its circumference The centre is the point between the two given points, the radius is half the distance between them.

list *point1*
First point

list *point2*
Second point

::math::geometry::pointInsideCircle *point circle*

Determine if the given point is inside the circle or on the circumference (1) or outside (0).

list *point*
Point to be checked

list *circle*
Circle that may or may not contain the point

::math::geometry::lineIntersectsCircle *line circle*

Determine if the given line intersects the circle or touches it (1) or does not (0).

list *line*
Line to be checked

list *circle*
Circle that may or may not be intersected

::math::geometry::lineSegmentIntersectsCircle *segment circle*

Determine if the given line segment intersects the circle or touches it (1) or does not (0).

list *segment*
Line segment to be checked

list *circle*
Circle that may or may not be intersected

::math::geometry::intersectionLineWithCircle *line circle*

Determine the points at which the given line intersects the circle. There can be zero, one or two points. (If the line touches the circle or is close to it, then one point is returned. An arbitrary margin of 1.0e-10 times the radius is used to determine this situation.)

list *line*
Line to be checked

list *circle*

Circle that may or may not be intersected

::math::geometry::intersectionCircleWithCircle *circle1 circle2*

Determine the points at which the given two circles intersect. There can be zero, one or two points. (If the two circles touch the circle or are very close, then one point is returned. An arbitrary margin of 1.0e-10 times the mean of the radii of the two circles is used to determine this situation.)

list *circle1*

First circle

list *circle2*

Second circle

::math::geometry::tangentLinesToCircle *point circle*

Determine the tangent lines from the given point to the circle. There can be zero, one or two lines. (If the point is on the cirucmference or very close to the circle, then one line is returned. An arbitrary margin of 1.0e-10 times the radius of the circle is used to determine this situation.)

list *point*

Point in question

list *circle*

Circle to which the tangent lines are to be determined

::math::geometry::intersectionPolylines *polyline1 polyline2* ?mode? ?granularity?

Return the first point or all points where the two polylines intersect. If the number of points in the polylines is large, you can use the granularity to get an approximate answer faster.

Arguments:

list *polyline1*

The first polyline

list *polyline2*

The second polyline

string *mode*

Whether to return only the first (default) or to return all intersection points ("all")

int *granularity*

The number of points that will be skipped plus 1 in the search for intersection points (1 or smaller means an exact answer is returned)

::math::geometry::intersectionPolylineCircle *polyline circle* ?mode? ?granularity?

Return the first point or all points where the polyline intersects the circle. If the number of points in the polyline is large, you can use the granularity to get an approximate answer faster.

Arguments:

list *polyline*

The polyline that may intersect the circle

list *circle*

The circle in question

string *mode*
Whether to return only the first (default) or to return all intersection points ("all")

int *granularity*
The number of points that will be skipped plus 1 in the search for intersection points (1 or smaller means an exact answer is returned)

::math::geometry::polylineCutOrigin *polyline1 polyline2* ?granularity?
Return the part of the first polyline from the origin up to the first intersection with the second. If the number of points in the polyline is large, you can use the granularity to get an approximate answer faster.

Arguments:

list *polyline1*
The first polyline (from which a part is to be returned)

list *polyline2*
The second polyline

int *granularity*
The number of points that will be skipped plus 1 in the search for intersection points (1 or smaller means an exact answer is returned)

::math::geometry::polylineCutEnd *polyline1 polyline2* ?granularity?
Return the part of the first polyline from the last intersection point with the second to the end. If the number of points in the polyline is large, you can use the granularity to get an approximate answer faster.

Arguments:

list *polyline1*
The first polyline (from which a part is to be returned)

list *polyline2*
The second polyline

int *granularity*
The number of points that will be skipped plus 1 in the search for intersection points (1 or smaller means an exact answer is returned)

::math::geometry::splitPolyline *polyline numberVertex*
Split the poyline into a set of polylines where each separate polyline holds "numberVertex" vertices between the two end points.

Arguments:

list *polyline*
The polyline to be split up

int *numberVertex*
The number of "internal" vertices

::math::geometry::enrichPolyline *polyline accuracy*
Split up each segment of a polyline into a number of smaller segments and return the result.

Arguments:

list *polyline*

The polyline to be refined

int *accuracy*
The number of subsegments to be created

::math::geometry::cleanupPolyline *polyline*
Remove duplicate neighbouring vertices and return the result.

Arguments:
list *polyline*
The polyline to be cleaned up

Coordinate system

The coordinate system used by the package is the ordinary cartesian system, where the positive x-axis is directed to the right and the positive y-axis is directed upwards. Angles and directions are defined with respect to the positive x-axis in a counter-clockwise direction, so that an angle of 90 degrees is the direction of the positive y-axis. Note that the Tk canvas coordinates differ from this, as there the origin is located in the upper left corner of the window. Up to and including version 1.3, the direction and octant procedures of this package used this convention inconsistently.

References

1. Polygon Intersection
2. http://en.wikipedia.org/wiki/Line-line_intersection
3. http://local.wasp.uwa.edu.au/~pbourke/geometry/lineline2d/

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *math :: geometry* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Mathematics

Copyright

```
Copyright (c) 2001 by Ideogramic ApS and other parties
Copyright (c) 2010 by Andreas Kupries
Copyright (c) 2010 by Kevin Kenny
Copyright (c) 2018 by Arjen Markus
Copyright (c) 2020 by Manfred Rosenberger
```

numtheory

Keywords: numtheory, number theory, prime

Name

math::numtheory - Number Theory

Synopsis

package require Tcl ?**8.5 9**?
package require math::numtheory ?**1.1.4**?
math::numtheory::isprime *N* ?*option value ...*?
math::numtheory::firstNprimes *N*
math::numtheory::primesLowerThan *N*
math::numtheory::primeFactors *N*
math::numtheory::primesLowerThan *N*
math::numtheory::primeFactors *N*
math::numtheory::uniquePrimeFactors *N*
math::numtheory::factors *N*
math::numtheory::totient *N*
math::numtheory::moebius *N*
math::numtheory::legendre *a p*
math::numtheory::jacobi *a b*
math::numtheory::gcd *m n*
math::numtheory::lcm *m n*
math::numtheory::numberPrimesGauss *N*
math::numtheory::numberPrimesLegendre *N*
math::numtheory::numberPrimesLegendreModified *N*
math::numtheory::differenceNumberPrimesLegendreModified *lower upper*
math::numtheory::listPrimePairs *lower upper step*
math::numtheory::listPrimeProgressions *lower upper step*

Description

This package is for collecting various number-theoretic operations, with a slight bias to prime numbers.

math::numtheory::isprime *N* ?*option value* ...?

The **isprime** command tests whether the integer *N* is a prime, returning a boolean true value for prime *N* and a boolean false value for non-prime *N*. The formal definition of 'prime' used is the conventional, that the number being tested is greater than 1 and only has trivial divisors.

To be precise, the return value is one of **0** (if *N* is definitely not a prime), **1** (if *N* is definitely a prime), and **on** (if *N* is probably prime); the latter two are both boolean true values. The case that an integer may be classified as "probably prime" arises because the Miller-Rabin algorithm used in the test implementation is basically probabilistic, and may if we are unlucky fail to detect that a number is in fact composite. Options may be used to select the risk of such "false positives" in the test. **1** is returned for "small" *N* (which currently means $N < 118670087467$), where it is known that no false positives are possible.

The only option currently defined is:

-randommr *repetitions*

which controls how many times the Miller-Rabin test should be repeated with randomly chosen bases. Each repetition reduces the probability of a false positive by a factor at least

4. The default for *repetitions* is 4.

Unknown options are silently ignored.

math::numtheory::firstNprimes *N*

Return the first N primes

integer *N* (in)

Number of primes to return

math::numtheory::primesLowerThan *N*

Return the prime numbers lower/equal to N

integer *N* (in)

Maximum number to consider

math::numtheory::primeFactors *N*

Return a list of the prime numbers in the number N

integer *N* (in)

Number to be factorised

math::numtheory::primesLowerThan *N*

Return the prime numbers lower/equal to N

integer *N* (in)

Maximum number to consider

math::numtheory::primeFactors *N*

Return a list of the prime numbers in the number N

integer *N* (in)

Number to be factorised

math::numtheory::uniquePrimeFactors *N*

Return a list of the *unique* prime numbers in the number N

integer *N* (in)

Number to be factorised

math::numtheory::factors *N*

Return a list of all *unique* factors in the number N, including 1 and N itself

integer *N* (in)

Number to be factorised

math::numtheory::totient *N*

Evaluate the Euler totient function for the number N (number of numbers relatively prime to N)

integer *N* (in)

Number in question

math::numtheory::moebius *N*

Evaluate the Moebius function for the number N

integer *N* (in)

Number in question

math::numtheory::legendre *a p*
Evaluate the Legendre symbol (a/p)
integer *a* (in)
Upper number in the symbol
integer *p* (in)
Lower number in the symbol (must be non-zero)

math::numtheory::jacobi *a b*
Evaluate the Jacobi symbol (a/b)
integer *a* (in)
Upper number in the symbol
integer *b* (in)
Lower number in the symbol (must be odd)

math::numtheory::gcd *m n*
Return the greatest common divisor of *m* and *n*
integer *m* (in)
First number
integer *n* (in)
Second number

math::numtheory::lcm *m n*
Return the lowest common multiple of *m* and *n*
integer *m* (in)
First number
integer *n* (in)
Second number

math::numtheory::numberPrimesGauss *N*
Estimate the number of primes according the formula by Gauss.
integer *N* (in)
Number in question, should be larger than 0

math::numtheory::numberPrimesLegendre *N*
Estimate the number of primes according the formula by Legendre.
integer *N* (in)
Number in question, should be larger than 0

math::numtheory::numberPrimesLegendreModified *N*
Estimate the number of primes according the modified formula by Legendre.
integer *N* (in)
Number in question, should be larger than 0

math::numtheory::differenceNumberPrimesLegendreModified *lower upper*
Estimate the number of primes between tow limits according the modified formula by Legendre.
integer *lower* (in)

Lower limit for the primes, should be larger than 0

integer *upper* (in)

Upper limit for the primes, should be larger than 0

math::numtheory::listPrimePairs *lower upper step*

Return a list of pairs of primes each differing by the given step.

integer *lower* (in)

Lower limit for the primes, should be larger than 0

integer *upper* (in)

Upper limit for the primes, should be larger than the lower limit

integer *step* (in)

Step by which the primes should differ, defaults to 2

math::numtheory::listPrimeProgressions *lower upper step*

Return a list of lists of primes each differing by the given step from the previous one.

integer *lower* (in)

Lower limit for the primes, should be larger than 0

integer *upper* (in)

Upper limit for the primes, should be larger than the lower limit

integer *step* (in)

Step by which the primes should differ, defaults to 2

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *math :: numtheory* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Mathematics

Copyright


```
Copyright (c) 2010 Lars Hellström <Lars dot Hellstrom at residenset
    dot net>
```


optimize

Keywords: optimize, linear program, math, maximum, minimum, optimization

Name

math::optimize - Optimisation routines

Synopsis

package require Tcl 8.5 9
package require math::optimize ?**1.0**?
::math::optimize::minimum *begin end func maxerr*
::math::optimize::maximum *begin end func maxerr*
::math::optimize::min_bound_1d *func begin end* ?*-relerror reltol*? ?*-abserror abstol*? ? *-maxiter maxiter*? ?*-trace traceflag*?
::math::optimize::min_unbound_1d *func begin end* ?*-relerror reltol*? ?*-abserror abstol*? ? *-maxiter maxiter*? ?*-trace traceflag*?
::math::optimize::solveLinearProgram *objective constraints*
::math::optimize::linearProgramMaximum *objective result*
::math::optimize::nelderMead *objective xVector* ?*-scale xScaleVector*? ?*-ftol epsilon*? ? *-maxiter count*? ??*-trace*? *flag?*

Description

This package implements several optimisation algorithms:

- Minimize or maximize a function over a given interval
- Solve a linear program (maximize a linear function subject to linear constraints)
- Minimize a function of several variables given an initial guess for the location of the minimum.

The package is fully implemented in Tcl. No particular attention has been paid to the accuracy of the calculations. Instead, the algorithms have been used in a straightforward manner.

This document describes the procedures and explains their usage.

Procedures

This package defines the following public procedures:

::math::optimize::minimum *begin end func maxerr*

Minimize the given (continuous) function by examining the values in the given interval. The procedure determines the values at both ends and in the centre of the interval and then constructs a new interval of 1/2 length that includes the minimum. No guarantee is made that the *global* minimum is found.

The procedure returns the "x" value for which the function is minimal.

This procedure has been deprecated - use min_bound_1d instead

begin - Start of the interval

end - End of the interval

func - Name of the function to be minimized (a procedure taking one argument).

maxerr - Maximum relative error (defaults to 1.0e-4)

::math::optimize::maximum *begin end func maxerr*

Maximize the given (continuous) function by examining the values in the given interval. The procedure determines the values at both ends and in the centre of the interval and then constructs a new interval of 1/2 length that includes the maximum. No guarantee is made

that the *global* maximum is found.

The procedure returns the "x" value for which the function is maximal.

This procedure has been deprecated - use max_bound_1d instead

begin - Start of the interval

end - End of the interval

func - Name of the function to be maximized (a procedure taking one argument).

maxerr - Maximum relative error (defaults to 1.0e-4)

::math::optimize::min_bound_1d *func begin end* ?-relerror *reltol*? ?-abserror *abstol*? ?-maxiter *maxiter*? ?-trace *traceflag*?

Miminizes a function of one variable in the given interval. The procedure uses Brent's method of parabolic interpolation, protected by golden-section subdivisions if the interpolation is not converging. No guarantee is made that a *global* minimum is found. The function to evaluate, *func*, must be a single Tcl command; it will be evaluated with an abscissa appended as the last argument.

x1 and *x2* are the two bounds of the interval in which the minimum is to be found. They need not be in increasing order.

reltol, if specified, is the desired upper bound on the relative error of the result; default is 1.0e-7. The given value should never be smaller than the square root of the machine's floating point precision, or else convergence is not guaranteed. *abstol*, if specified, is the desired upper bound on the absolute error of the result; default is 1.0e-10. Caution must be used with small values of *abstol* to avoid overflow/underflow conditions; if the minimum is expected to lie about a small but non-zero abscissa, you consider either shifting the function or changing its length scale.

maxiter may be used to constrain the number of function evaluations to be performed; default is 100. If the command evaluates the function more than *maxiter* times, it returns an error to the caller.

traceFlag is a Boolean value. If true, it causes the command to print a message on the standard output giving the abscissa and ordinate at each function evaluation, together with an indication of what type of interpolation was chosen. Default is 0 (no trace).

::math::optimize::min_unbound_1d *func begin end* ?-relerror *reltol*? ?-abserror *abstol*? ?-maxiter *maxiter*? ?-trace *traceflag*?

Miminizes a function of one variable over the entire real number line. The procedure uses parabolic extrapolation combined with golden-section dilatation to search for a region where a minimum exists, followed by Brent's method of parabolic interpolation, protected by golden-section subdivisions if the interpolation is not converging. No guarantee is made that a *global* minimum is found. The function to evaluate, *func*, must be a single Tcl command; it will

be evaluated with an abscissa appended as the last argument.

x1 and *x2* are two initial guesses at where the minimum may lie. *x1* is the starting point for the minimization, and the difference between *x2* and *x1* is used as a hint at the characteristic length scale of the problem.

reltol, if specified, is the desired upper bound on the relative error of the result; default is 1.0e-7. The given value should never be smaller than the square root of the machine's floating point precision, or else convergence is not guaranteed. *abstol*, if specified, is the desired upper bound on the absolute error of the result; default is 1.0e-10. Caution must be used with small values of *abstol* to avoid overflow/underflow conditions; if the minimum is expected to lie about a small but non-zero abscissa, you consider either shifting the function or changing its length scale.

maxiter may be used to constrain the number of function evaluations to be performed; default is 100. If the command evaluates the function more than *maxiter* times, it returns an error to the caller.

traceFlag is a Boolean value. If true, it causes the command to print a message on the standard output giving the abscissa and ordinate at each function evaluation, together with an indication of what type of interpolation was chosen. Default is 0 (no trace).

::math::optimize::solveLinearProgram *objective constraints*

Solve a *linear program* in standard form using a straightforward implementation of the Simplex algorithm. (In the explanation below: The linear program has N constraints and M variables).

The procedure returns a list of M values, the values for which the objective function is maximal or a single keyword if the linear program is not feasible or unbounded (either "unfeasible" or "unbounded")

objective - The M coefficients of the objective function

constraints - Matrix of coefficients plus maximum values that implement the linear constraints. It is expected to be a list of N lists of M+1 numbers each, M coefficients and the maximum value.

::math::optimize::linearProgramMaximum *objective result*

Convenience function to return the maximum for the solution found by the solveLinearProgram procedure.

objective - The M coefficients of the objective function

result - The result as returned by solveLinearProgram

::math::optimize::nelderMead *objective xVector* ?-scale *xScaleVector*? ?-ftol *epsilon*? ?-maxiter *count*? ??-trace? *flag*?

Minimizes, in unconstrained fashion, a function of several variable over all of space. The function to evaluate, *objective*, must be a single Tcl command. To it will be appended as many

elements as appear in the initial guess at the location of the minimum, passed in as a Tcl list, *xVector*.

xScaleVector is an initial guess at the problem scale; the first function evaluations will be made by varying the co-ordinates in *xVector* by the amounts in *xScaleVector*. If *xScaleVector* is not supplied, the co-ordinates will be varied by a factor of 1.0001 (if the co-ordinate is non-zero) or by a constant 0.0001 (if the co-ordinate is zero).

epsilon is the desired relative error in the value of the function evaluated at the minimum. The default is 1.0e-7, which usually gives three significant digits of accuracy in the values of the x's.

pp *count* is a limit on the number of trips through the main loop of the optimizer. The number of function evaluations may be several times this number. If the optimizer fails to find a minimum to within *ftol* in *maxiter* iterations, it returns its current best guess and an error status. Default is to allow 500 iterations.

flag is a flag that, if true, causes a line to be written to the standard output for each evaluation of the objective function, giving the arguments presented to the function and the value returned. Default is false.

The **nelderMead** procedure returns a list of alternating keywords and values suitable for use with **array set**. The meaning of the keywords is:

x is the approximate location of the minimum.

y is the value of the function at *x*.

yvec is a vector of the best N+1 function values achieved, where N is the dimension of *x*

vertices is a list of vectors giving the function arguments corresponding to the values in *yvec*.

nIter is the number of iterations required to achieve convergence or fail.

status is 'ok' if the operation succeeded, or 'too-many-iterations' if the maximum iteration count was exceeded.

nelderMead minimizes the given function using the downhill simplex method of Nelder and Mead. This method is quite slow - much faster methods for minimization are known - but has the advantage of being extremely robust in the face of problems where the minimum lies in a valley of complex topology.

nelderMead can occasionally find itself "stuck" at a point where it can make no further progress; it is recommended that the caller run it at least a second time, passing as the initial guess the result found by the previous call. The second run is usually very fast.

nelderMead can be used in some cases for constrained optimization. To do this, add a large value to the objective function if the parameters are outside the feasible region. To work effectively in this mode, **nelderMead** requires that the initial guess be feasible and usually requires that the feasible region be convex.

Notes

Several of the above procedures take the *names* of procedures as arguments. To avoid problems with the *visibility* of these procedures, the fully-qualified name of these procedures is determined inside the optimize routines. For the user this has only one consequence: the named procedure must be visible in the calling procedure. For instance:

```
namespace eval ::mySpace {
   namespace export calcfunc
   proc calcfunc { x } { return $x }
}
#
# Use a fully-qualified name
#
namespace eval ::myCalc {
   puts [min_bound_1d ::myCalc::calcfunc $begin $end]
}
#
# Import the name
#
namespace eval ::myCalc {
   namespace import ::mySpace::calcfunc
   puts [min_bound_1d calcfunc $begin $end]
}
```

The simple procedures *minimum* and *maximum* have been deprecated: the alternatives are much more flexible, robust and require less function evaluations.

Examples

Let us take a few simple examples:

Determine the maximum of f(x) = x^3 exp(-3x), on the interval (0,10):

```
proc efunc { x } { expr {$x*$x*$x * exp(-3.0*$x)} }
puts "Maximum at: [::math::optimize::max_bound_1d efunc 0.0 10.0]"
```

The maximum allowed error determines the number of steps taken (with each step in the iteration the interval is reduced with a factor 1/2). Hence, a maximum error of 0.0001 is achieved in approximately 14 steps.

An example of a *linear program* is:

Optimise the expression 3x+2y, where:

```
   x >= 0 and y >= 0 (implicit constraints, part of the
                      definition of linear programs)

   x + y   <= 1      (constraints specific to the problem)
   2x + 5y <= 10
```

This problem can be solved as follows:

```
   set solution [::math::optimize::solveLinearProgram  { 3.0   2.0 }
    { { 1.0   1.0   1.0 }
        { 2.0   5.0  10.0 } } ]
```

Note, that a constraint like:

```
   x + y >= 1
```

can be turned into standard form using:

```
   -x  -y <= -1
```

The theory of linear programming is the subject of many a text book and the Simplex algorithm that is implemented here is the best-known method to solve this type of problems, but it is not the only one.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *math :: optimize* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Mathematics

Copyright

```
Copyright (c) 2004 Arjen Markus <arjenmarkus@users.sourceforge.net>
Copyright (c) 2004,2005 Kevn B. Kenny <kennykb@users.sourceforge.net>
```

pca

Keywords: pca, math, statistics, tcl

Name

math::PCA - Package for Principal Component Analysis

Synopsis

package require Tcl ?**8.6 9**?
package require math::linearalgebra 1.1

::math::PCA::createPCA *data* ?*args*?
$pca using ?number?|?-minproportion value?
$pca eigenvectors ?*option*?
$pca eigenvalues ?*option*?
$pca proportions ?*option*?
$pca approximate *observation*
$pca approximatOriginal
$pca scores *observation*
$pca distance *observation*
$pca qstatistic *observation* ?*option*?

Description

The PCA package provides a means to perform principal components analysis in Tcl, using an object-oriented technique as facilitated by TclOO. It actually defines a single public method, *::math::PCA::createPCA*, which constructs an object based on the data that are passed to perform the actual analysis.

The methods of the PCA objects that are created with this command allow one to examine the principal components, to approximate (new) observations using all or a selected number of components only and to examine the properties of the components and the statistics of the approximations.

The package has been modelled after the PCA example provided by the original linear algebra package by Ed Hume.

Commands

The *math::PCA* package provides one public command:

::math::PCA::createPCA *data* ?args?

Create a new object, based on the data that are passed via the *data* argument. The principal components may be based on either correlations or covariances. All observations will be normalised according to the mean and standard deviation of the original data.

list *data*

- A list of observations (see the example below).

list *args*

- A list of key-value pairs defining the options. Currently there is only one key: *-covariances*. This indicates if covariances are to be used (if the value is 1) or instead correlations (value is 0). The default is to use correlations.

The PCA object that is created has the following methods:

$pca using ?number?|?-minproportion value?

Set the number of components to be used in the analysis (the number of retained components). Returns the number of components, also if no argument is given.

int *number*

- The number of components to be retained

double *value*

- Select the number of components based on the minimum proportion of variation that is retained by them. Should be a value between 0 and 1.

$pca eigenvectors ?option?
Return the eigenvectors as a list of lists.
string *option*
- By default only the *retained* components are returned. If all eigenvectors are required, use the option *-all*.

$pca eigenvalues ?option?
Return the eigenvalues as a list of lists.
string *option*
- By default only the eigenvalues of the *retained* components are returned. If all eigenvalues are required, use the option *-all*.

$pca proportions ?option?
Return the proportions for all components, that is, the amount of variations that each components can explain.

$pca approximate *observation*
Return an approximation of the observation based on the retained components
list *observation*
- The values for the observation.

$pca approximatOriginal
Return an approximation of the original data, using the retained components. It is a convenience method that works on the complete set of original data.

$pca scores *observation*
Return the scores per retained component for the given observation.
list *observation*
- The values for the observation.

$pca distance *observation*
Return the distance between the given observation and its approximation. (Note: this distance is based on the normalised vectors.)
list *observation*
- The values for the observation.

$pca qstatistic *observation* ?option?
Return the Q statistic, basically the square of the distance, for the given observation.
list *observation*
- The values for the observation.

string *option*
- If the observation is part of the original data, you may want to use the corrected Q statistic. This is achieved with the option "-original".

Example
TODO: NIST example

Bugs, ideas, feedback
If you find errors in this document or bugs or problems with the package it describes, or if you

want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *PCA* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Mathematics

polynomials
Keywords: polynomials, math, polynomial functions

Name
math::polynomials - Polynomial functions

Synopsis
package require Tcl ?**8.5**?
package require math::polynomials ?**1.0.2**?
::math::polynomials::polynomial *coeffs*
::math::polynomials::polynCmd *coeffs*
::math::polynomials::evalPolyn *polynomial x*
::math::polynomials::addPolyn *polyn1 polyn2*
::math::polynomials::subPolyn *polyn1 polyn2*
::math::polynomials::multPolyn *polyn1 polyn2*
::math::polynomials::divPolyn *polyn1 polyn2*
::math::polynomials::remainderPolyn *polyn1 polyn2*
::math::polynomials::derivPolyn *polyn*
::math::polynomials::primitivePolyn *polyn*
::math::polynomials::degreePolyn *polyn*
::math::polynomials::coeffPolyn *polyn index*
::math::polynomials::allCoeffsPolyn *polyn*

Description
This package deals with polynomial functions of one variable:

- the basic arithmetic operations are extended to polynomials
- computing the derivatives and primitives of these functions
- evaluation through a general procedure or via specific procedures)

Procedures
The package defines the following public procedures:

::math::polynomials::polynomial *coeffs*
Return an (encoded) list that defines the polynomial. A polynomial

```
f(x) = a + b.x + c.x**2 + d.x**3
```

can be defined via:

```
set f [::math::polynomials::polynomial [list $a $b $c $d]
```

list *coeffs*
Coefficients of the polynomial (in ascending order)

::math::polynomials::polynCmd *coeffs*
Create a new procedure that evaluates the polynomial. The name of the polynomial is automatically generated. Useful if you need to evualuate the polynomial many times, as the procedure consists of a single [expr] command.
list *coeffs*
Coefficients of the polynomial (in ascending order) or the polynomial definition returned by the *polynomial* command.

::math::polynomials::evalPolyn *polynomial x*
Evaluate the polynomial at x.
list *polynomial*
The polynomial's definition (as returned by the polynomial command). order)
float *x*
The coordinate at which to evaluate the polynomial

::math::polynomials::addPolyn *polyn1 polyn2*
Return a new polynomial which is the sum of the two others.
list *polyn1*
The first polynomial operand
list *polyn2*
The second polynomial operand

::math::polynomials::subPolyn *polyn1 polyn2*
Return a new polynomial which is the difference of the two others.
list *polyn1*
The first polynomial operand
list *polyn2*
The second polynomial operand

::math::polynomials::multPolyn *polyn1 polyn2*
Return a new polynomial which is the product of the two others. If one of the arguments is a scalar value, the other polynomial is simply scaled.
list *polyn1*
The first polynomial operand or a scalar
list *polyn2*
The second polynomial operand or a scalar

::math::polynomials::divPolyn *polyn1 polyn2*
Divide the first polynomial by the second polynomial and return the result. The remainder is dropped
list *polyn1*

The first polynomial operand

list *polyn2*

The second polynomial operand

::math::polynomials::remainderPolyn *polyn1 polyn2*

Divide the first polynomial by the second polynomial and return the remainder.

list *polyn1*

The first polynomial operand

list *polyn2*

The second polynomial operand

::math::polynomials::derivPolyn *polyn*

Differentiate the polynomial and return the result.

list *polyn*

The polynomial to be differentiated

::math::polynomials::primitivePolyn *polyn*

Integrate the polynomial and return the result. The integration constant is set to zero.

list *polyn*

The polynomial to be integrated

::math::polynomials::degreePolyn *polyn*

Return the degree of the polynomial.

list *polyn*

The polynomial to be examined

::math::polynomials::coeffPolyn *polyn index*

Return the coefficient of the term of the index'th degree of the polynomial.

list *polyn*

The polynomial to be examined

int *index*

The degree of the term

::math::polynomials::allCoeffsPolyn *polyn*

Return the coefficients of the polynomial (in ascending order).

list *polyn*

The polynomial in question

Remarks on the implementation

The implementation for evaluating the polynomials at some point uses Horn's rule, which guarantees numerical stability and a minimum of arithmetic operations. To recognise that a polynomial definition is indeed a correct definition, it consists of a list of two elements: the keyword "POLYNOMIAL" and the list of coefficients in descending order. The latter makes it easier to implement Horner's rule.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you

want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *math :: polynomials* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Mathematics

Copyright

```
Copyright (c) 2004 Arjen Markus <arjenmarkus@users.sourceforge.net>
```


probopt
Keywords: probopt, mathematics, optimisation, probabilistic calculations

Name
math::probopt - Probabilistic optimisation methods

Synopsis
package require Tcl 8.6 9
package require TclOO
package require math::probopt 1.1
::math::probopt::pso *function bounds args*
::math::probopt::sce *function bounds args*
::math::probopt::diffev *function bounds args*
::math::probopt::lipoMax *function bounds args*
::math::probopt::adaLipoMax *function bounds args*

Description
The purpose of the **math::probopt** package is to provide various optimisation algorithms that are based on probabilistic techniques. The results of these algorithms may therefore vary from one run to the next. The algorithms are all well-known and well described and proponents generally claim they are efficient and reliable.

As most of these algorithms have one or more tunable parameters or even variations, the interface to each accepts options to set these parameters or the select the variation. These take the form of key-value pairs, for instance, *-iterations 100*.

This manual does not offer any recommendations with regards to these algorithms, nor does it provide much in the way of guidelines for the parameters. For this we refer to online articles on the algorithms in question.

A few notes, however:

- With the exception of LIPO, the algorithms are capable of dealing with irregular (non-smooth) and even discontinuous functions.
- The results depend on the random number seeding and are likely not to be very accurate, especially if the function varies slowly in the vicinty of the optimum. They do give a good

starting point for a deterministic algorithm.

The collection consists of the following algorithms:

- PSO - particle swarm optimisation
- SCE - shuffled complexes evolution
- DE - differential evolution
- LIPO - Lipschitz optimisation

The various procedures have a uniform interface:

```
set result [::math::probopt::algorithm function bounds args]
```

The arguments have the following meaning:

- The argument *function* is the name of the procedure that evaluates the function. Its interface is:

  ```
  set value [function coords]
  ```

- where *coords* is a list of coordinates at which to evaluate the function. It is supposed to return the function value.
- The argument *bounds* is a list of pairs of minimum and maximum for each coordinate. This list implicitly determines the dimension of the coordinate space in which the optimum is to be sought, for instance for a function like *x**2 + (y-1)**4*, you may specify the bounds as *{{-1 1} {-1 1}}*, that is, two pairs for the two coordinates.
- The rest (*args*) consists of zero or more key-value pairs to specify the options. Which options are supported by which algorithm, is documented below.

The result of the various optimisation procedures is a dictionary containing at least the following elements:

- *optimum-coordinates* is a list containing the coordinates of the optimum that was found.
- *optimum-value* is the function value at those coordinates.
- *evaluations* is the number of function evaluations.
- *best-values* is a list of successive best values, obtained as part of the iterations.

Details on the algorithms

The algorithms in the package are the following:

::math::probopt::pso *function bounds args*

The "particle swarm optimisation" algorithm uses the idea that the candidate optimum points should swarm around the best point found so far, with variations to allow for improvements.

It recognises the following options:
- *-swarmsize number*: Number of particles to consider (default: 50)
- *-vweight value*: Weight for the current "velocity" (0-1, default: 0.5)
- *-pweight value*: Weight for the individual particle's best position (0-1, default: 0.3)
- *-gweight value*: Weight for the "best" overall position as per particle (0-1, default: 0.3)
- *-type local/global*: Type of optimisation
- *-neighbours number*: Size of the neighbourhood (default: 5, used if "local")
- *-iterations number*: Maximum number of iterations
- *-tolerance value*: Absolute minimal improvement for minimum value

::math::probopt::sce *function bounds args*

The "shuffled complex evolution" algorithm is an extension of the Nelder-Mead algorithm that uses multiple complexes and reorganises these complexes to find the "global" optimum.

It recognises the following options:
- *-complexes number*: Number of particles to consider (default: 2)
- *-mincomplexes number*: Minimum number of complexes (default: 2; not currently used)
- *-newpoints number*: Number of new points to be generated (default: 1)
- *-shuffle number*: Number of iterations after which to reshuffle the complexes (if set to 0, the default, a number will be calculated from the number of dimensions)
- *-pointspercomplex number*: Number of points per complex (if set to 0, the default, a number will be calculated from the number of dimensions)
- *-pointspersubcomplex number*: Number of points per subcomplex (used to select the best points in each complex; if set to 0, the default, a number will be calculated from the number of dimensions)
- *-iterations number*: Maximum number of iterations (default: 100)
- *-maxevaluations number*: Maximum number of function evaluations (when this number is reached the iteration is broken off. Default: 1000 million)
- *-abstolerance value*: Absolute minimal improvement for minimum value (default: 0.0)
- *-reltolerance value*: Relative minimal improvement for minimum value (default: 0.001)

::math::probopt::diffev *function bounds args*

The "differential evolution" algorithm uses a number of initial points that are then updated using randomly selected points. It is more or less akin to genetic algorithms. It is controlled by two parameters, factor and lambda, where the first determines the update via random points and the second the update with the best point found sofar.

It recognises the following options:
- *-iterations number*: Maximum number of iterations (default: 100)
- *-number number*: Number of point to work with (if set to 0, the default, it is calculated from the number of dimensions)
- *-factor value*: Weight of randomly selected points in the updating (0-1, default: 0.6)
- *-lambda value*: Weight of the best point found so far in the updating (0-1, default: 0.0)
- *-crossover value*: Fraction of new points to be considered for replacing the old ones (0-1, default: 0.5)
- *-maxevaluations number*: Maximum number of function evaluations (when this number is reached the iteration is broken off. Default: 1000 million)
- *-abstolerance value*: Absolute minimal improvement for minimum value (default: 0.0)
- *-reltolerance value*: Relative minimal improvement for minimum value (default: 0.001)

::math::probopt::lipoMax *function bounds args*

The "Lipschitz optimisation" algorithm uses the "Lipschitz" property of the given function to find a *maximum* in the given bounding box. There are two variants, *lipoMax* assumes a fixed estimate for the Lipschitz parameter.

It recognises the following options:
- *-iterations number*: Number of iterations (equals the actual number of function evaluations, default: 100)
- *-lipschitz value*: Estimate of the Lipschitz parameter (default: 10.0)

::math::probopt::adaLipoMax ***function bounds args***

The "adaptive Lipschitz optimisation" algorithm uses the "Lipschitz" property of the given function to find a *maximum* in the given bounding box. The adaptive variant actually uses two phases to find a suitable estimate for the Lipschitz parameter. This is controlled by the "Bernoulli" parameter.

When you specify a large number of iterations, the algorithm may take a very long time to complete as it is trying to improve on the Lipschitz parameter and the chances of hitting a better estimate diminish fast.

It recognises the following options:
- *-iterations number*: Number of iterations (equals the actual number of function evaluations, default: 100)
- *-bernoulli value*: Parameter for random decisions (exploration versus exploitation, default: 0.1)

References

The various algorithms have been described in on-line publications. Here are a few:

- *PSO*: Maurice Clerc, Standard Particle Swarm Optimisation (2012) https://hal.archives-ouvertes.fr/file/index/docid/764996/filename/SPSO_descriptions.pdf

  Alternatively: https://en.wikipedia.org/wiki/Particle_swarm_optimization
- *SCE*: Qingyuan Duan, Soroosh Sorooshian, Vijai K. Gupta, Optimal use offo the SCE-UA global optimization method for calibrating watershed models (1994), Journal of Hydrology 158, pp 265-284

  https://www.researchgate.net/publication/223408756_Optimal_Use_of_the_SCE-UA_Global_Optimization_Me
- *DE*: Rainer Storn and Kenneth Price, Differential Evolution - A simple and efficient adaptivescheme for globaloptimization over continuous spaces (1996)

  http://www1.icsi.berkeley.edu/~storn/TR-95-012.pdf
- *LIPO*: Cedric Malherbe and Nicolas Vayatis, Global optimization of Lipschitz functions, (june 2017)

  https://arxiv.org/pdf/1703.02628.pdf

Category

Mathematics

qcomplex

Keywords: qcomplex, complex numbers, math

Name

math::complexnumbers - Straightforward complex number package

Synopsis

package require Tcl 8.5 9
package require math::complexnumbers ?**1.0.3**?
::math::complexnumbers::+ *z1 z2*
::math::complexnumbers::- *z1 z2*

::math::complexnumbers::* *z1 z2*
::math::complexnumbers::/ *z1 z2*
::math::complexnumbers::conj *z1*
::math::complexnumbers::real *z1*
::math::complexnumbers::imag *z1*
::math::complexnumbers::mod *z1*
::math::complexnumbers::arg *z1*
::math::complexnumbers::complex *real imag*
::math::complexnumbers::tostring *z1*
::math::complexnumbers::exp *z1*
::math::complexnumbers::sin *z1*
::math::complexnumbers::cos *z1*
::math::complexnumbers::tan *z1*
::math::complexnumbers::log *z1*
::math::complexnumbers::sqrt *z1*
::math::complexnumbers::pow *z1 z2*

Description

The mathematical module *complexnumbers* provides a straightforward implementation of complex numbers in pure Tcl. The philosophy is that the user knows he or she is dealing with complex numbers in an abstract way and wants as high a performance as can be had within the limitations of an interpreted language.

Therefore the procedures defined in this package assume that the arguments are valid (representations of) "complex numbers", that is, lists of two numbers defining the real and imaginary part of a complex number (though this is a mere detail: rely on the *complex* command to construct a valid number.)

Most procedures implement the basic arithmetic operations or elementary functions whereas several others convert to and from different representations:

```
    set z [complex 0 1]
    puts "z = [tostring $z]"
    puts "z**2 = [* $z $z]
```

would result in:

```
    z = i
    z**2 = -1
```

Available procedures

The package implements all or most basic operations and elementary functions.

The arithmetic operations are:

::math::complexnumbers::+ *z1 z2*
Add the two arguments and return the resulting complex number
complex *z1* (in)
First argument in the summation
complex *z2* (in)

Second argument in the summation

::math::complexnumbers::- *z1 z2*
Subtract the second argument from the first and return the resulting complex number. If there is only one argument, the opposite of z1 is returned (i.e. -z1)
complex *z1* (in)
First argument in the subtraction
complex *z2* (in)
Second argument in the subtraction (optional)

::math::complexnumbers::* *z1 z2*
Multiply the two arguments and return the resulting complex number
complex *z1* (in)
First argument in the multiplication
complex *z2* (in)
Second argument in the multiplication

::math::complexnumbers::/ *z1 z2*
Divide the first argument by the second and return the resulting complex number
complex *z1* (in)
First argument (numerator) in the division
complex *z2* (in)
Second argument (denominator) in the division

::math::complexnumbers::conj *z1*
Return the conjugate of the given complex number
complex *z1* (in)
Complex number in question

Conversion/inquiry procedures:

::math::complexnumbers::real *z1*
Return the real part of the given complex number
complex *z1* (in)
Complex number in question

::math::complexnumbers::imag *z1*
Return the imaginary part of the given complex number
complex *z1* (in)
Complex number in question

::math::complexnumbers::mod *z1*
Return the modulus of the given complex number
complex *z1* (in)
Complex number in question

::math::complexnumbers::arg *z1*
Return the argument ("angle" in radians) of the given complex number

complex *z1* (in)
Complex number in question

::math::complexnumbers::complex *real imag*
Construct the complex number "real + imag*i" and return it
float *real* (in)
The real part of the new complex number
float *imag* (in)
The imaginary part of the new complex number

::math::complexnumbers::tostring *z1*
Convert the complex number to the form "real + imag*i" and return the string
float *complex* (in)
The complex number to be converted

Elementary functions:

::math::complexnumbers::exp *z1*
Calculate the exponential for the given complex argument and return the result
complex *z1* (in)
The complex argument for the function

::math::complexnumbers::sin *z1*
Calculate the sine function for the given complex argument and return the result
complex *z1* (in)
The complex argument for the function

::math::complexnumbers::cos *z1*
Calculate the cosine function for the given complex argument and return the result
complex *z1* (in)
The complex argument for the function

::math::complexnumbers::tan *z1*
Calculate the tangent function for the given complex argument and return the result
complex *z1* (in)
The complex argument for the function

::math::complexnumbers::log *z1*
Calculate the (principle value of the) logarithm for the given complex argument and return the result
complex *z1* (in)
The complex argument for the function

::math::complexnumbers::sqrt *z1*
Calculate the (principle value of the) square root for the given complex argument and return the result
complex *z1* (in)
The complex argument for the function

::math::complexnumbers::pow *z1 z2*
Calculate "z1 to the power of z2" and return the result
complex *z1* (in)
The complex number to be raised to a power
complex *z2* (in)
The complex power to be used

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *math :: complexnumbers* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Mathematics

Copyright

```
Copyright (c) 2004 Arjen Markus <arjenmarkus@users.sourceforge.net>
```

quasirandom

Keywords: quasirandom, mathematics, quasi-random

Name

math::quasirandom - Quasi-random points for integration and Monte Carlo type methods

Synopsis

package require Tcl 8.6 9
package require TclOO
package require math::quasirandom 1.1
::math::quasirandom::qrpoint create *NAME DIM* ?*ARGS*?
gen next
gen set-start *index*
gen set-evaluations *number*
gen integral *func minmax args*

Description

In many applications pseudo-random numbers and pseudo-random points in a (limited) sample space play an important role. For instance in any type of Monte Carlo simulation. Pseudo-random numbers, however, may be too random and as a consequence a large number of data points is required to reduce the error or fluctuation in the results to the desired value.

Quasi-random numbers can be used as an alternative: instead of "completely" arbitrary points, points are generated that are diverse enough to cover the entire sample space in a more or less

uniform way. As a consequence convergence to the limit can be much faster, when such quasi-random numbers are well-chosen.

The package defines a *class* "qrpoint" that creates a command to generate quasi-random points in 1, 2 or more dimensions. The command can either generate separate points, so that they can be used in a user-defined algorithm or use these points to calculate integrals of functions defined over 1, 2 or more dimensions. It also holds several other common algorithms. (NOTE: these are not implemented yet)

One particular characteristic of the generators is that there are no tuning parameters involved, which makes the use particularly simple.

Commands

A quasi-random point generator is created using the *qrpoint* class:

::math::quasirandom::qrpoint create *NAME DIM* ?ARGS?

This command takes the following arguments:

string *NAME*

The name of the command to be created (alternatively: the *new* subcommand will generate a unique name)

integer/string *DIM*

The number of dimensions or one of: "circle", "disk", "sphere" or "ball"

strings *ARGS*

Zero or more key-value pairs. The supported options are:
- *-start index*: The index for the next point to be generated (default: 1)
- *-evaluations number*: The number of evaluations to be used by default (default: 100)

The points that are returned lie in the hyperblock [0,1[^n (n the number of dimensions) or on the unit circle, within the unit disk, on the unit sphere or within the unit ball.

Each generator supports the following subcommands:

gen next

Return the coordinates of the next quasi-random point

gen set-start *index*

Reset the index for the next quasi-random point. This is useful to control which list of points is returned. Returns the new or the current value, if no value is given.

gen set-evaluations *number*

Reset the default number of evaluations in compound algorithms. Note that the actual number is the smallest 4-fold larger or equal to the given number. (The 4-fold plays a role in the detailed integration routine.)

gen integral *func minmax args*

Calculate the integral of the given function over the block (or the circle, sphere etc.)

string *func*

The name of the function to be integrated

list *minmax*

List of pairs of minimum and maximum coordinates. This can be used to map the quasi-random coordinates to the desired hyper-block.

If the space is a circle, disk etc. then this argument should be a single value, the radius. The circle, disk, etc. is centred at the origin. If this is not what is required, then a coordinate transformation should be made within the function.

strings *args*

Zero or more key-value pairs. The following options are supported:

- *-evaluations number*: The number of evaluations to be used. If not specified use the default of the generator object.

Todo

Implement other algorithms and variants

Implement more unit tests.

Comparison to pseudo-random numbers for integration.

References

Various algorithms exist for generating quasi-random numbers. The generators created in this package are based on:
http://extremelearning.com.au/unreasonable-effectiveness-of-quasirandom-sequences/

Category

Mathematics

rational_funcs

Keywords: rational_funcs, math, rational functions

Name

math::rationalfunctions - Polynomial functions

Synopsis

package require Tcl ?**8.5 9**?
package require math::rationalfunctions ?**1.0.2**?
::math::rationalfunctions::rationalFunction *num den*
::math::rationalfunctions::ratioCmd *num den*
::math::rationalfunctions::evalRatio *rational x*
::math::rationalfunctions::addRatio *ratio1 ratio2*
::math::rationalfunctions::subRatio *ratio1 ratio2*
::math::rationalfunctions::multRatio *ratio1 ratio2*
::math::rationalfunctions::divRatio *ratio1 ratio2*
::math::rationalfunctions::derivPolyn *ratio*
::math::rationalfunctions::coeffsNumerator *ratio*
::math::rationalfunctions::coeffsDenominator *ratio*

Description

This package deals with rational functions of one variable:

- the basic arithmetic operations are extended to rational functions
- computing the derivatives of these functions

- evaluation through a general procedure or via specific procedures)

Procedures

The package defines the following public procedures:

::math::rationalfunctions::rationalFunction *num den*

Return an (encoded) list that defines the rational function. A rational function

```
           1 + x^3
f(x) = ------------
       1 + 2x + x^2
```

can be defined via:

```
set f [::math::rationalfunctions::rationalFunction [list 1 0 0 1]
  [list 1 2 1]]
```

list *num*

Coefficients of the numerator of the rational function (in ascending order)

list *den*

Coefficients of the denominator of the rational function (in ascending order)

::math::rationalfunctions::ratioCmd *num den*

Create a new procedure that evaluates the rational function. The name of the function is automatically generated. Useful if you need to evaluate the function many times, as the procedure consists of a single [expr] command.

list *num*

Coefficients of the numerator of the rational function (in ascending order)

list *den*

Coefficients of the denominator of the rational function (in ascending order)

::math::rationalfunctions::evalRatio *rational x*

Evaluate the rational function at x.

list *rational*

The rational function's definition (as returned by the rationalFunction command). order)

float *x*

The coordinate at which to evaluate the function

::math::rationalfunctions::addRatio *ratio1 ratio2*

Return a new rational function which is the sum of the two others.

list *ratio1*

The first rational function operand

list *ratio2*

The second rational function operand

::math::rationalfunctions::subRatio *ratio1 ratio2*

Return a new rational function which is the difference of the two others.

list *ratio1*

The first rational function operand

list *ratio2*
The second rational function operand

::math::rationalfunctions::multRatio *ratio1 ratio2*
Return a new rational function which is the product of the two others. If one of the arguments is a scalar value, the other rational function is simply scaled.
list *ratio1*
The first rational function operand or a scalar
list *ratio2*
The second rational function operand or a scalar

::math::rationalfunctions::divRatio *ratio1 ratio2*
Divide the first rational function by the second rational function and return the result. The remainder is dropped
list *ratio1*
The first rational function operand
list *ratio2*
The second rational function operand

::math::rationalfunctions::derivPolyn *ratio*
Differentiate the rational function and return the result.
list *ratio*
The rational function to be differentiated

::math::rationalfunctions::coeffsNumerator *ratio*
Return the coefficients of the numerator of the rational function.
list *ratio*
The rational function to be examined

::math::rationalfunctions::coeffsDenominator *ratio*
Return the coefficients of the denominator of the rational function.
list *ratio*
The rational function to be examined

Remarks on the implementation

The implementation of the rational functions relies on the math::polynomials package. For further remarks see the documentation on that package.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *math :: rationalfunctions* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Mathematics

Copyright

```
Copyright (c) 2005 Arjen Markus <arjenmarkus@users.sourceforge.net>
```


roman
Keywords: roman, conversion, integer, roman numeral

Name
math::roman - Tools for creating and manipulating roman numerals

Synopsis
package require Tcl 8.5 9
package require math::roman ?**1.1**?
::math::roman::toroman *i*
::math::roman::tointeger *r*
::math::roman::sort *list*
::math::roman::expr *args*

Description
::math::roman is a pure-Tcl library for converting between integers and roman numerals. It also provides utility functions for sorting and performing arithmetic on roman numerals.

This code was originally harvested from the Tcler's wiki at http://wiki.tcl.tk/1823 and as such is free for any use for any purpose. Many thanks to the ingeneous folk who devised these clever routines and generously contributed them to the Tcl community.

While written and tested under Tcl 8.3, I expect this library will work under all 8.x versions of Tcl.

Commands
::math::roman::toroman *i*
: Convert an integer to roman numerals. The result is always in upper case. The value zero is converted to an empty string.

::math::roman::tointeger *r*
: Convert a roman numeral into an integer.

::math::roman::sort *list*
: Sort a list of roman numerals from smallest to largest.

::math::roman::expr *args*
: Evaluate an expression where the operands are all roman numerals.

Of these commands both *toroman* and *tointeger* are exported for easier use. The other two are not, as they could interfer or be confused with existing Tcl commands.

Bugs, ideas, feedback
If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *math :: roman* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Mathematics

Copyright

```
Copyright (c) 2005 Kenneth Green <kenneth.green@gmail.com>
```


romberg
Keywords: romberg

Name
math::calculus::romberg - Romberg integration

Synopsis
package require Tcl 8.5 9
package require math::calculus 0.6
::math::calculus::romberg *f a b* ?*-option value...*?
::math::calculus::romberg_infinity *f a b* ?*-option value...*?
::math::calculus::romberg_sqrtSingLower *f a b* ?*-option value...*?
::math::calculus::romberg_sqrtSingUpper *f a b* ?*-option value...*?
::math::calculus::romberg_powerLawLower *gamma f a b* ?*-option value...*?
::math::calculus::romberg_powerLawUpper *gamma f a b* ?*-option value...*?
::math::calculus::romberg_expLower *f a b* ?*-option value...*?
::math::calculus::romberg_expUpper *f a b* ?*-option value...*?

Description
The **romberg** procedures in the **math::calculus** package perform numerical integration of a function of one variable. They are intended to be of "production quality" in that they are robust, precise, and reasonably efficient in terms of the number of function evaluations.

Procedures
The following procedures are available for Romberg integration:

::math::calculus::romberg *f a b* ?*-option value*...?
Integrates an analytic function over a given interval.

::math::calculus::romberg_infinity *f a b* ?*-option value*...?
Integrates an analytic function over a half-infinite interval.

::math::calculus::romberg_sqrtSingLower *f a b* ?*-option value*...?
Integrates a function that is expected to be analytic over an interval except for the presence of an inverse square root singularity at the lower limit.

::math::calculus::romberg_sqrtSingUpper *f a b* ?*-option value*...?
Integrates a function that is expected to be analytic over an interval except for the presence

of an inverse square root singularity at the upper limit.

::math::calculus::romberg_powerLawLower *gamma f a b* ?*-option value*...?
Integrates a function that is expected to be analytic over an interval except for the presence of a power law singularity at the lower limit.

::math::calculus::romberg_powerLawUpper *gamma f a b* ?*-option value*...?
Integrates a function that is expected to be analytic over an interval except for the presence of a power law singularity at the upper limit.

::math::calculus::romberg_expLower *f a b* ?*-option value*...?
Integrates an exponentially growing function; the lower limit of the region of integration may be arbitrarily large and negative.

::math::calculus::romberg_expUpper *f a b* ?*-option value*...?
Integrates an exponentially decaying function; the upper limit of the region of integration may be arbitrarily large.

Parameters

f
Function to integrate. Must be expressed as a single Tcl command, to which will be appended a single argument, specifically, the abscissa at which the function is to be evaluated. The first word of the command will be processed with **namespace which** in the caller's scope prior to any evaluation. Given this processing, the command may local to the calling namespace rather than needing to be global.

a
Lower limit of the region of integration.

b
Upper limit of the region of integration. For the **romberg_sqrtSingLower**, **romberg_sqrtSingUpper**, **romberg_powerLawLower**, **romberg_powerLawUpper**, **romberg_expLower**, and **romberg_expUpper** procedures, the lower limit must be strictly less than the upper. For the other procedures, the limits may appear in either order.

gamma
Power to use for a power law singularity; see section Improper integrals for details.

Options

-abserror *epsilon*
Requests that the integration machinery proceed at most until the estimated absolute error of the integral is less than *epsilon*. The error may be seriously over- or underestimated if the function (or any of its derivatives) contains singularities; see section Improper integrals for details. Default is 1.0e-08.

-relerror *epsilon*
Requests that the integration machinery proceed at most until the estimated relative error of the integral is less than *epsilon*. The error may be seriously over- or underestimated if the function (or any of its derivatives) contains singularities; see section Improper integrals for details. Default is 1.0e-06.

-maxiter *m*
Requests that integration terminate after at most *n* triplings of the number of evaluations performed. In other words, given *n* for **-maxiter**, the integration machinery will make at most

3**_n_ evaluations of the function. Default is 14, corresponding to a limit approximately 4.8 million evaluations. (Well-behaved functions will seldom require more than a few hundred evaluations.)

-degree _d_
Requests that an extrapolating polynomial of degree _d_ be used in Romberg integration; see section Description for details. Default is 4. Can be at most _m_-1.

Description

The **romberg** procedure performs Romberg integration using the modified midpoint rule. Romberg integration is an iterative process. At the first step, the function is evaluated at the midpoint of the region of integration, and the value is multiplied by the width of the interval for the coarsest possible estimate. At the second step, the interval is divided into three parts, and the function is evaluated at the midpoint of each part; the sum of the values is multiplied by three. At the third step, nine parts are used, at the fourth twenty-seven, and so on, tripling the number of subdivisions at each step.

Once the interval has been divided at least _d_ times, a polynomial is fitted to the integrals estimated in the last _d_+1 divisions. The integrals are considered to be a function of the square of the width of the subintervals (any good numerical analysis text will discuss this process under "Romberg integration"). The polynomial is extrapolated to a step size of zero, computing a value for the integral and an estimate of the error.

This process will be well-behaved only if the function is analytic over the region of integration; there may be removable singularities at either end of the region provided that the limit of the function (and of all its derivatives) exists as the ends are approached. Thus, **romberg** may be used to integrate a function like f(x)=sin(x)/x over an interval beginning or ending at zero.

Note that **romberg** will either fail to converge or else return incorrect error estimates if the function, or any of its derivatives, has a singularity anywhere in the region of integration (except for the case mentioned above). Care must be used, therefore, in integrating a function like 1/(1-x**2) to avoid the places where the derivative is singular.

Improper integrals

Romberg integration is also useful for integrating functions over half-infinite intervals or functions that have singularities. The trick is to make a change of variable to eliminate the singularity, and to put the singularity at one end or the other of the region of integration. The **math::calculus** package supplies a number of **romberg** procedures to deal with the commoner cases.

romberg_infinity
Integrates a function over a half-infinite interval; either _a_ or _b_ may be infinite. _a_ and _b_ must be of the same sign; if you need to integrate across the axis, say, from a negative value to positive infinity, use **romberg** to integrate from the negative value to a small positive value, and then **romberg_infinity** to integrate from the positive value to positive infinity. The **romberg_infinity** procedure works by making the change of variable u=1/x, so that the integral from a to b of f(x) is evaluated as the integral from 1/a to 1/b of f(1/u)/u**2.

romberg_powerLawLower and romberg_powerLawUpper
Integrate a function that has an integrable power law singularity at either the lower or upper bound of the region of integration (or has a derivative with a power law singularity there).

These procedures take a first parameter, *gamma*, which gives the power law. The function or its first derivative are presumed to diverge as (x-*a*)**(-*gamma*) or (*b*-x)**(-*gamma*). *gamma* must be greater than zero and less than 1.

These procedures are useful not only in integrating functions that go to infinity at one end of the region of integration, but also functions whose derivatives do not exist at the end of the region. For instance, integrating f(x)=pow(x,0.25) with the origin as one end of the region will result in the **romberg** procedure greatly underestimating the error in the integral. The problem can be fixed by observing that the first derivative of f(x), f'(x)=x**(-3/4)/4, goes to infinity at the origin. Integrating using **romberg_powerLawLower** with *gamma* set to 0.75 gives much more orderly convergence.

These procedures operate by making the change of variable u=(x-a)**(1-gamma) (**romberg_powerLawLower**) or u=(b-x)**(1-gamma) (**romberg_powerLawUpper**).

To summarize the meaning of gamma:
- If f(x) ~ x**(-a) (0 < a < 1), use gamma = a
- If f'(x) ~ x**(-b) (0 < b < 1), use gamma = b

romberg_sqrtSingLower and romberg_sqrtSingUpper
: These procedures behave identically to **romberg_powerLawLower** and **romberg_powerLawUpper** for the common case of *gamma*=0.5; that is, they integrate a function with an inverse square root singularity at one end of the interval. They have a simpler implementation involving square roots rather than arbitrary powers.

romberg_expLower and romberg_expUpper
: These procedures are for integrating a function that grows or decreases exponentially over a half-infinite interval. **romberg_expLower** handles exponentially growing functions, and allows the lower limit of integration to be an arbitrarily large negative number. **romberg_expUpper** handles exponentially decaying functions and allows the upper limit of integration to be an arbitrary large positive number. The functions make the change of variable u=exp(-x) and u=exp(x) respectively.

Other changes of variable

If you need an improper integral other than the ones listed here, a change of variable can be written in very few lines of Tcl. Because the Tcl coding that does it is somewhat arcane, we offer a worked example here.

Let's say that the function that we want to integrate is f(x)=exp(x)/sqrt(1-x*x) (not a very natural function, but a good example), and we want to integrate it over the interval (-1,1). The denominator falls to zero at both ends of the interval. We wish to make a change of variable from x to u so that dx/sqrt(1-x**2) maps to du. Choosing x=sin(u), we can find that dx=cos(u)*du, and sqrt(1-x**2)=cos(u). The integral from a to b of f(x) is the integral from asin(a) to asin(b) of f(sin(u))*cos(u).

We can make a function **g** that accepts an arbitrary function **f** and the parameter u, and computes this new integrand.

```
proc g { f u } {
    set x [expr { sin($u) }]
    set cmd $f; lappend cmd $x; set y [eval $cmd]
    return [expr { $y / cos($u) }]
}
```

Now integrating **f** from *a* to *b* is the same as integrating **g** from *asin(a)* to *asin(b)*. It's a little tricky to get **f** consistently evaluated in the caller's scope; the following procedure does it.

```
proc romberg_sine { f a b args } {
    set f [lreplace $f 0 0 [uplevel 1 [list namespace which [lindex $f
    0]]]]
    set f [list g $f]
    return [eval [linsert $args 0 romberg $f [expr { asin($a) }] [expr
    { asin($b) }]]]
}
```

This **romberg_sine** procedure will do any function with sqrt(1-x*x) in the denominator. Our sample function is f(x)=exp(x)/sqrt(1-x*x):

```
proc f { x } {
    expr { exp($x) / sqrt( 1. - $x*$x ) }
}
```

Integrating it is a matter of applying **romberg_sine** as we would any of the other **romberg** procedures:

```
foreach { value error } [romberg_sine f -1.0 1.0] break
puts [format "integral is %.6g +/- %.6g" $value $error]

integral is 3.97746 +/- 2.3557e-010
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *math :: calculus* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Mathematics

Copyright


```
Copyright (c) 2004 Kevin B. Kenny <kennykb@acm.org>. All rights
    reserved. Redistribution permitted under the terms of the Open
    Publication License <http://www.opencontent.org/openpub/>
```

special

Keywords: special, Bessel functions, error function, math, special functions

Name

math::special - Special mathematical functions

Synopsis

package require Tcl ?**8.5 9**?
package require math::special ?**0.5.3**?
::math::special::eulerNumber *index*
::math::special::bernoulliNumber *index*
::math::special::Beta *x y*
::math::special::incBeta *a b x*
::math::special::regIncBeta *a b x*
::math::special::Gamma *x*
::math::special::digamma *x*
::math::special::erf *x*
::math::special::erfc *x*
::math::special::invnorm *p*
::math::special::J0 *x*
::math::special::J1 *x*
::math::special::Jn *n x*
::math::special::J1/2 *x*
::math::special::J-1/2 *x*
::math::special::I_n *x*
::math::special::cn *u k*
::math::special::dn *u k*
::math::special::sn *u k*
::math::special::elliptic_K *k*
::math::special::elliptic_E *k*
::math::special::exponential_Ei *x*
::math::special::exponential_En *n x*
::math::special::exponential_li *x*
::math::special::exponential_Ci *x*
::math::special::exponential_Si *x*
::math::special::exponential_Chi *x*
::math::special::exponential_Shi *x*
::math::special::fresnel_C *x*
::math::special::fresnel_S *x*
::math::special::sinc *x*
::math::special::legendre *n*
::math::special::chebyshev *n*
::math::special::laguerre *alpha n*
::math::special::hermite *n*

Description

This package implements several so-called special functions, like the Gamma function, the Bessel functions and such.

Each function is implemented by a procedure that bears its name (well, in close approximation):

- J0 for the zeroth-order Bessel function of the first kind
- J1 for the first-order Bessel function of the first kind
- Jn for the nth-order Bessel function of the first kind
- J1/2 for the half-order Bessel function of the first kind
- J-1/2 for the minus-half-order Bessel function of the first kind
- I_n for the modified Bessel function of the first kind of order n
- Gamma for the Gamma function, erf and erfc for the error function and the complementary error function
- fresnel_C and fresnel_S for the Fresnel integrals
- elliptic_K and elliptic_E (complete elliptic integrals)
- exponent_Ei and other functions related to the so-called exponential integrals
- legendre, hermite: some of the classical orthogonal polynomials.

Overview

In the following table several characteristics of the functions in this package are summarized: the domain for the argument, the values for the parameters and error bounds.

Family	Function	Domain x	Parameter	Error bound
Bessel	J0, J1, Jn	all of R	n = integer	< 1.0e-8 (|x|<20, n<20)
Bessel	J1/2, J-1/2,	x > 0	n = integer	exact
Bessel	I_n	all of R	n = integer	< 1.0e-6
Elliptic functions	cn	0 <= x <= 1	--	< 1.0e-10
	dn	0 <= x <= 1	--	< 1.0e-10
	sn	0 <= x <= 1	--	< 1.0e-10
Elliptic integrals	K	0 <= x < 1	--	< 1.0e-6
	E	0 <= x < 1	--	< 1.0e-6
Error functions	erf		--	
	erfc			
Inverse normal distribution	invnorm	0 < x < 1	--	< 1.2e-9
Exponential (relative) integrals	Ei	x != 0	--	< 1.0e-10
	En	x > 0	--	as Ei
	li	x > 0	--	as Ei
	Chi	x > 0	--	< 1.0e-8

```
             | Shi         |  x > 0      |     --      |   < 1.0e-8
             | Ci          |  x > 0      |     --      |   < 2.0e-4
             | Si          |  x > 0      |     --      |   < 2.0e-4
             |             |             |             |
Fresnel      | C           |  all of R   |     --      |   < 2.0e-3
integrals    | S           |  all of R   |     --      |   < 2.0e-3
             |             |             |             |
general      | Beta        | (see Gamma) |     --      |   < 1.0e-9
             | Gamma       |  x != 0,-1, |     --      |   < 1.0e-9
             |             |  -2, ...    |             |
             | incBeta     |             |  a, b > 0   |   < 1.0e-9
             | regIncBeta  |             |  a, b > 0   |   < 1.0e-9
             | digamma     |  x != 0,-1  |             |   < 1.0e-9
             |             |  -2, ...    |             |
             |             |             |             |
             | sinc        |  all of R   |     --      |   exact
             |             |             |             |
orthogonal   | Legendre    |  all of R   | n = 0,1,... |   exact
polynomials  | Chebyshev   |  all of R   | n = 0,1,... |   exact
             | Laguerre    |  all of R   | n = 0,1,... |   exact
             |             |             | alpha el. R |
             | Hermite     |  all of R   | n = 0,1,... |   exact
```

Note: Some of the error bounds are estimated, as no "formal" bounds were available with the implemented approximation method, others hold for the auxiliary functions used for estimating the primary functions.

The following well-known functions are currently missing from the package:

- Bessel functions of the second kind (Y_n, K_n)
- Bessel functions of arbitrary order (and hence the Airy functions)
- Chebyshev polynomials of the second kind (U_n)
- The incomplete gamma function

Procedures

The package defines the following public procedures:

::math::special::eulerNumber *index*

Return the index'th Euler number (note: these are integer values). As the size of these numbers grows very fast, only a limited number are available.

int *index*

Index of the number to be returned (should be between 0 and 54)

::math::special::bernoulliNumber *index*

Return the index'th Bernoulli number. As the size of the numbers grows very fast, only a limited number are available.

int *index*

Index of the number to be returned (should be between 0 and 52)

::math::special::Beta *x y*
Compute the Beta function for arguments "x" and "y"
float *x*
First argument for the Beta function
float *y*
Second argument for the Beta function

::math::special::incBeta *a b x*
Compute the incomplete Beta function for argument "x" with parameters "a" and "b"
float *a*
First parameter for the incomplete Beta function, a > 0
float *b*
Second parameter for the incomplete Beta function, b > 0
float *x*
Argument for the incomplete Beta function

::math::special::regIncBeta *a b x*
Compute the regularized incomplete Beta function for argument "x" with parameters "a" and "b"
float *a*
First parameter for the incomplete Beta function, a > 0
float *b*
Second parameter for the incomplete Beta function, b > 0
float *x*
Argument for the regularized incomplete Beta function

::math::special::Gamma *x*
Compute the Gamma function for argument "x"
float *x*
Argument for the Gamma function

::math::special::digamma *x*
Compute the digamma function (psi) for argument "x"
float *x*
Argument for the digamma function

::math::special::erf *x*
Compute the error function for argument "x"
float *x*
Argument for the error function

::math::special::erfc *x*
Compute the complementary error function for argument "x"
float *x*
Argument for the complementary error function

::math::special::invnorm *p*

Compute the inverse of the normal distribution function for argument "p"

float *p*
Argument for the inverse normal distribution function (p must be greater than 0 and lower than 1)

::math::special::J0 *x*
Compute the zeroth-order Bessel function of the first kind for the argument "x"

float *x*
Argument for the Bessel function

::math::special::J1 *x*
Compute the first-order Bessel function of the first kind for the argument "x"

float *x*
Argument for the Bessel function

::math::special::Jn *n x*
Compute the nth-order Bessel function of the first kind for the argument "x"

integer *n*
Order of the Bessel function

float *x*
Argument for the Bessel function

::math::special::J1/2 *x*
Compute the half-order Bessel function of the first kind for the argument "x"

float *x*
Argument for the Bessel function

::math::special::J-1/2 *x*
Compute the minus-half-order Bessel function of the first kind for the argument "x"

float *x*
Argument for the Bessel function

::math::special::I_n *x*
Compute the modified Bessel function of the first kind of order n for the argument "x"

int *x*
Positive integer order of the function

float *x*
Argument for the function

::math::special::cn *u k*
Compute the elliptic function *cn* for the argument "u" and parameter "k".

float *u*
Argument for the function

float *k*
Parameter

::math::special::dn *u k*

Compute the elliptic function *dn* for the argument "u" and parameter "k".

float *u*
Argument for the function

float *k*
Parameter

::math::special::sn *u k*
Compute the elliptic function *sn* for the argument "u" and parameter "k".

float *u*
Argument for the function

float *k*
Parameter

::math::special::elliptic_K *k*
Compute the complete elliptic integral of the first kind for the argument "k"

float *k*
Argument for the function

::math::special::elliptic_E *k*
Compute the complete elliptic integral of the second kind for the argument "k"

float *k*
Argument for the function

::math::special::exponential_Ei *x*
Compute the exponential integral of the second kind for the argument "x"

float *x*
Argument for the function (x!= 0)

::math::special::exponential_En *n x*
Compute the exponential integral of the first kind for the argument "x" and order n

int *n*
Order of the integral (n >= 0)

float *x*
Argument for the function (x >= 0)

::math::special::exponential_li *x*
Compute the logarithmic integral for the argument "x"

float *x*
Argument for the function (x > 0)

::math::special::exponential_Ci *x*
Compute the cosine integral for the argument "x"

float *x*
Argument for the function (x > 0)

::math::special::exponential_Si *x*
Compute the sine integral for the argument "x"

float *x*
Argument for the function (x > 0)

::math::special::exponential_Chi *x*
Compute the hyperbolic cosine integral for the argument "x"
float *x*
Argument for the function (x > 0)

::math::special::exponential_Shi *x*
Compute the hyperbolic sine integral for the argument "x"
float *x*
Argument for the function (x > 0)

::math::special::fresnel_C *x*
Compute the Fresnel cosine integral for real argument x
float *x*
Argument for the function

::math::special::fresnel_S *x*
Compute the Fresnel sine integral for real argument x
float *x*
Argument for the function

::math::special::sinc *x*
Compute the sinc function for real argument x
float *x*
Argument for the function

::math::special::legendre *n*
Return the Legendre polynomial of degree n (see The orthogonal polynomials)
int *n*
Degree of the polynomial

::math::special::chebyshev *n*
Return the Chebyshev polynomial of degree n (of the first kind)
int *n*
Degree of the polynomial

::math::special::laguerre *alpha n*
Return the Laguerre polynomial of degree n with parameter alpha
float *alpha*
Parameter of the Laguerre polynomial
int *n*
Degree of the polynomial

::math::special::hermite *n*
Return the Hermite polynomial of degree n
int *n*

Degree of the polynomial

The orthogonal polynomials

For dealing with the classical families of orthogonal polynomials, the package relies on the *math::polynomials* package. To evaluate the polynomial at some coordinate, use the *evalPolyn* command:

```
set leg2 [::math::special::legendre 2]
puts "Value at x=$x: [::math::polynomials::evalPolyn $leg2 $x]"
```

The return value from the *legendre* and other commands is actually the definition of the corresponding polynomial as used in that package.

Remarks on the implementation

It should be noted, that the actual implementation of J0 and J1 depends on straightforward Gaussian quadrature formulas. The (absolute) accuracy of the results is of the order 1.0e-4 or better. The main reason to implement them like that was that it was fast to do (the formulas are simple) and the computations are fast too.

The implementation of J1/2 does not suffer from this: this function can be expressed exactly in terms of elementary functions.

The functions J0 and J1 are the ones you will encounter most frequently in practice.

The computation of I_n is based on Miller's algorithm for computing the minimal function from recurrence relations.

The computation of the Gamma and Beta functions relies on the combinatorics package, whereas that of the error functions relies on the statistics package.

The computation of the complete elliptic integrals uses the AGM algorithm.

Much information about these functions can be found in:

Abramowitz and Stegun: *Handbook of Mathematical Functions* (Dover, ISBN 486-61272-4)

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *math :: special* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Mathematics

Copyright

```
Copyright (c) 2004 Arjen Markus <arjenmarkus@users.sourceforge.net>
```

statistics

Keywords: statistics, data analysis, mathematics

Name

math::statistics - Basic statistical functions and procedures

Synopsis

package require Tcl 8.5 9
package require math::statistics 1.6.1
::math::statistics::mean *data*
::math::statistics::min *data*
::math::statistics::max *data*
::math::statistics::number *data*
::math::statistics::stdev *data*
::math::statistics::var *data*
::math::statistics::pstdev *data*
::math::statistics::pvar *data*
::math::statistics::median *data*
::math::statistics::basic-stats *data*
::math::statistics::histogram *limits values* ?*weights*?
::math::statistics::histogram-alt *limits values* ?*weights*?
::math::statistics::corr *data1 data2*
::math::statistics::interval-mean-stdev *data confidence*
::math::statistics::t-test-mean *data est_mean est_stdev alpha*
::math::statistics::test-normal *data significance*
::math::statistics::lillieforsFit *data*
::math::statistics::test-Duckworth *list1 list2 significance*
::math::statistics::test-anova-F *alpha args*
::math::statistics::test-Tukey-range *alpha args*
::math::statistics::test-Dunnett *alpha control args*
::math::statistics::quantiles *data confidence*
::math::statistics::quantiles *limits counts confidence*
::math::statistics::autocorr *data*
::math::statistics::crosscorr *data1 data2*
::math::statistics::mean-histogram-limits *mean stdev number*
::math::statistics::minmax-histogram-limits *min max number*
::math::statistics::linear-model *xdata ydata intercept*
::math::statistics::linear-residuals *xdata ydata intercept*
::math::statistics::test-2x2 *n11 n21 n12 n22*
::math::statistics::print-2x2 *n11 n21 n12 n22*
::math::statistics::control-xbar *data* ?*nsamples*?
::math::statistics::control-Rchart *data* ?*nsamples*?
::math::statistics::test-xbar *control data*
::math::statistics::test-Rchart *control data*
::math::statistics::test-Kruskal-Wallis *confidence args*
::math::statistics::analyse-Kruskal-Wallis *args*

::math::statistics::test-Levene *groups*
::math::statistics::test-Brown-Forsythe *groups*
::math::statistics::group-rank *args*
::math::statistics::test-Wilcoxon *sample_a sample_b*
::math::statistics::spearman-rank *sample_a sample_b*
::math::statistics::spearman-rank-extended *sample_a sample_b*
::math::statistics::kernel-density *data* opt *-option value ...*
::math::statistics::bootstrap *data sampleSize* ?*numberSamples*?
::math::statistics::wasserstein-distance *prob1 prob2*
::math::statistics::kl-divergence *prob1 prob2*
::math::statistics::logistic-model *xdata ydata*
::math::statistics::logistic-probability *coeffs x*
::math::statistics::tstat *dof* ?*alpha*?
::math::statistics::mv-wls *weights_and_values*
::math::statistics::mv-ols *values*
::math::statistics::pdf-normal *mean stdev value*
::math::statistics::pdf-lognormal *mean stdev value*
::math::statistics::pdf-exponential *mean value*
::math::statistics::pdf-uniform *xmin xmax value*
::math::statistics::pdf-triangular *xmin xmax value*
::math::statistics::pdf-symmetric-triangular *xmin xmax value*
::math::statistics::pdf-gamma *alpha beta value*
::math::statistics::pdf-poisson *mu k*
::math::statistics::pdf-chisquare *df value*
::math::statistics::pdf-student-t *df value*
::math::statistics::pdf-gamma *a b value*
::math::statistics::pdf-beta *a b value*
::math::statistics::pdf-weibull *scale shape value*
::math::statistics::pdf-gumbel *location scale value*
::math::statistics::pdf-pareto *scale shape value*
::math::statistics::pdf-cauchy *location scale value*
::math::statistics::pdf-laplace *location scale value*
::math::statistics::pdf-kumaraswamy *a b value*
::math::statistics::pdf-negative-binomial *r p value*
::math::statistics::cdf-normal *mean stdev value*
::math::statistics::cdf-lognormal *mean stdev value*
::math::statistics::cdf-exponential *mean value*
::math::statistics::cdf-uniform *xmin xmax value*
::math::statistics::cdf-triangular *xmin xmax value*
::math::statistics::cdf-symmetric-triangular *xmin xmax value*
::math::statistics::cdf-students-t *degrees value*
::math::statistics::cdf-gamma *alpha beta value*
::math::statistics::cdf-poisson *mu k*
::math::statistics::cdf-beta *a b value*
::math::statistics::cdf-weibull *scale shape value*
::math::statistics::cdf-gumbel *location scale value*

::math::statistics::cdf-pareto *scale shape value*
::math::statistics::cdf-cauchy *location scale value*
::math::statistics::cdf-F *nf1 nf2 value*
::math::statistics::cdf-laplace *location scale value*
::math::statistics::cdf-kumaraswamy *a b value*
::math::statistics::cdf-negative-binomial *r p value*
::math::statistics::empirical-distribution *values*
::math::statistics::random-normal *mean stdev number*
::math::statistics::random-lognormal *mean stdev number*
::math::statistics::random-exponential *mean number*
::math::statistics::random-uniform *xmin xmax number*
::math::statistics::random-triangular *xmin xmax number*
::math::statistics::random-symmetric-triangular *xmin xmax number*
::math::statistics::random-gamma *alpha beta number*
::math::statistics::random-poisson *mu number*
::math::statistics::random-chisquare *df number*
::math::statistics::random-student-t *df number*
::math::statistics::random-beta *a b number*
::math::statistics::random-weibull *scale shape number*
::math::statistics::random-gumbel *location scale number*
::math::statistics::random-pareto *scale shape number*
::math::statistics::random-cauchy *location scale number*
::math::statistics::random-laplace *location scale number*
::math::statistics::random-kumaraswamy *a b number*
::math::statistics::random-negative-binomial *r p number*
::math::statistics::histogram-uniform *xmin xmax limits number*
::math::statistics::incompleteGamma *x p* ?*tol*?
::math::statistics::incompleteBeta *a b x* ?*tol*?
::math::statistics::estimate-pareto *values*
::math::statistics::estimate-exponential *values*
::math::statistics::estimate-laplace *values*
::math::statistics::estimante-negative-binomial *r values*
::math::statistics::filter *varname data expression*
::math::statistics::map *varname data expression*
::math::statistics::samplescount *varname list expression*
::math::statistics::subdivide
::math::statistics::plot-scale *canvas xmin xmax ymin ymax*
::math::statistics::plot-xydata *canvas xdata ydata tag*
::math::statistics::plot-xyline *canvas xdata ydata tag*
::math::statistics::plot-tdata *canvas tdata tag*
::math::statistics::plot-tline *canvas tdata tag*
::math::statistics::plot-histogram *canvas counts limits tag*

Description

The **math::statistics** package contains functions and procedures for basic statistical data analysis, such as:

- Descriptive statistical parameters (mean, minimum, maximum, standard deviation)
- Estimates of the distribution in the form of histograms and quantiles
- Basic testing of hypotheses
- Probability and cumulative density functions

It is meant to help in developing data analysis applications or doing ad hoc data analysis, it is not in itself a full application, nor is it intended to rival with full (non-)commercial statistical packages.

The purpose of this document is to describe the implemented procedures and provide some examples of their usage. As there is ample literature on the algorithms involved, we refer to relevant text books for more explanations. The package contains a fairly large number of public procedures. They can be distinguished in three sets: general procedures, procedures that deal with specific statistical distributions, list procedures to select or transform data and simple plotting procedures (these require Tk). *Note:* The data that need to be analyzed are always contained in a simple list. Missing values are represented as empty list elements. *Note:* With version 1.0.1 a mistake in the procs *pdf-lognormal*, *cdf-lognormal* and *random-lognormal* has been corrected. In previous versions the argument for the standard deviation was actually used as if it was the variance.

General procedures

The general statistical procedures are:

::math::statistics::mean *data*
Determine the *mean* value of the given list of data.
list *data*
- List of data

::math::statistics::min *data*
Determine the *minimum* value of the given list of data.
list *data*
- List of data

::math::statistics::max *data*
Determine the *maximum* value of the given list of data.
list *data*
- List of data

::math::statistics::number *data*
Determine the *number* of non-missing data in the given list
list *data*
- List of data

::math::statistics::stdev *data*
Determine the *sample standard deviation* of the data in the given list
list *data*
- List of data

::math::statistics::var *data*
Determine the *sample variance* of the data in the given list

list *data*
- List of data

::math::statistics::pstdev *data*
Determine the *population standard deviation* of the data in the given list
list *data*
- List of data

::math::statistics::pvar *data*
Determine the *population variance* of the data in the given list
list *data*
- List of data

::math::statistics::median *data*
Determine the *median* of the data in the given list (Note that this requires sorting the data, which may be a costly operation)
list *data*
- List of data

::math::statistics::basic-stats *data*
Determine a list of all the descriptive parameters: mean, minimum, maximum, number of data, sample standard deviation, sample variance, population standard deviation and population variance.

(This routine is called whenever either or all of the basic statistical parameters are required. Hence all calculations are done and the relevant values are returned.)
list *data*
- List of data

::math::statistics::histogram *limits values* ?weights?
Determine histogram information for the given list of data. Returns a list consisting of the number of values that fall into each interval. (The first interval consists of all values lower than the first limit, the last interval consists of all values greater than the last limit. There is one more interval than there are limits.)

Optionally, you can use weights to influence the histogram.
list *limits*
- List of upper limits (in ascending order) for the intervals of the histogram.

list *values*
- List of data

list *weights*
- List of weights, one weight per value

::math::statistics::histogram-alt *limits values* ?weights?
Alternative implementation of the histogram procedure: the open end of the intervals is at the lower bound instead of the upper bound.
list *limits*
- List of upper limits (in ascending order) for the intervals of the histogram.

list *values*
- List of data

list *weights*
- List of weights, one weight per value

::math::statistics::corr *data1 data2*

Determine the correlation coefficient between two sets of data.

list *data1*
- First list of data

list *data2*
- Second list of data

::math::statistics::interval-mean-stdev *data confidence*

Return the interval containing the mean value and one containing the standard deviation with a certain level of confidence (assuming a normal distribution)

list *data*
- List of raw data values (small sample)

float *confidence*
- Confidence level (0.95 or 0.99 for instance)

::math::statistics::t-test-mean *data est_mean est_stdev alpha*

Test whether the mean value of a sample is in accordance with the estimated normal distribution with a certain probability. Returns 1 if the test succeeds or 0 if the mean is unlikely to fit the given distribution.

list *data*
- List of raw data values (small sample)

float *est_mean*
- Estimated mean of the distribution

float *est_stdev*
- Estimated stdev of the distribution

float *alpha*
- Probability level (0.95 or 0.99 for instance)

::math::statistics::test-normal *data significance*

Test whether the given data follow a normal distribution with a certain level of significance. Returns 1 if the data are normally distributed within the level of significance, returns 0 if not. The underlying test is the Lilliefors test. Smaller values of the significance mean a stricter testing.

list *data*
- List of raw data values

float *significance*
- Significance level (one of 0.01, 0.05, 0.10, 0.15 or 0.20). For compatibility reasons the values "1-significance", 0.80, 0.85, 0.90, 0.95 or 0.99 are also accepted.

Compatibility issue: the original implementation and documentation used the term "confidence" and used a value 1-significance (see ticket 2812473fff). This has been corrected

as of version 0.9.3.

::math::statistics::lillieforsFit *data*

Returns the goodness of fit to a normal distribution according to Lilliefors. The higher the number, the more likely the data are indeed normally distributed. The test requires at least *five* data points.

list *data*

- List of raw data values

::math::statistics::test-Duckworth *list1 list2 significance*

Determine if two data sets have the same median according to the Tukey-Duckworth test. The procedure returns 0 if the medians are unequal, 1 if they are equal, -1 if the test can not be conducted (the smallest value must be in a different set than the greatest value). # # Arguments: # list1 Values in the first data set # list2 Values in the second data set # significance Significance level (either 0.05, 0.01 or 0.001) # # Returns: Test whether the given data follow a normal distribution with a certain level of significance. Returns 1 if the data are normally distributed within the level of significance, returns 0 if not. The underlying test is the Lilliefors test. Smaller values of the significance mean a stricter testing.

list *list1*

- First list of data

list *list2*

- Second list of data

float *significance*

- Significance level (either 0.05, 0.01 or 0.001)

::math::statistics::test-anova-F *alpha args*

Determine if two or more groups with normally distributed data have the same means. The procedure returns 0 if the means are likely unequal, 1 if they are. This is a one-way ANOVA test. The groups may also be stored in a nested list: The procedure returns a list of the comparison results for each pair of groups. Each element of this list contains: the index of the first group and that of the second group, whether the means are likely to be different (1) or not (0) and the confidence interval the conclusion is based on. The groups may also be stored in a nested list:

```
    test-anova-F 0.05 $A $B $C
    #
    # Or equivalently:
    #
    test-anova-F 0.05 [list $A $B $C]
```

float *alpha*

- Significance level

list *args*

- Two or more groups of data to be checked

::math::statistics::test-Tukey-range *alpha args*

Determine if two or more groups with normally distributed data have the same means, using Tukey's range test. It is complementary to the ANOVA test. The procedure returns a list of the

comparison results for each pair of groups. Each element of this list contains: the index of the first group and that of the second group, whether the means are likely to be different (1) or not (0) and the confidence interval the conclusion is based on. The groups may also be stored in a nested list, just as with the ANOVA test.

float *alpha*

- Significance level - either 0.05 or 0.01

list *args*

- Two or more groups of data to be checked

::math::statistics::test-Dunnett *alpha control args*

Determine if one or more groups with normally distributed data have the same means as the group of control data, using Dunnett's test. It is complementary to the ANOVA test. The procedure returns a list of the comparison results for each group with the control group. Each element of this list contains: whether the means are likely to be different (1) or not (0) and the confidence interval the conclusion is based on. The groups may also be stored in a nested list, just as with the ANOVA test.

Note: some care is required if there is only one group to compare the control with:

```
    test-Dunnett-F 0.05 $control [list $A]
```

Otherwise the group A is split up into groups of one element - this is due to an ambiguity.

float *alpha*

- Significance level - either 0.05 or 0.01

list *args*

- One or more groups of data to be checked

::math::statistics::quantiles *data confidence*

Return the quantiles for a given set of data

list *data*

- List of raw data values

float *confidence*

- Confidence level (0.95 or 0.99 for instance) or a list of confidence levels.

::math::statistics::quantiles *limits counts confidence*

Return the quantiles based on histogram information (alternative to the call with two arguments)

list *limits*

- List of upper limits from histogram

list *counts*

- List of counts for for each interval in histogram

float *confidence*

- Confidence level (0.95 or 0.99 for instance) or a list of confidence levels.

::math::statistics::autocorr *data*

Return the autocorrelation function as a list of values (assuming equidistance between samples, about 1/2 of the number of raw data)

The correlation is determined in such a way that the first value is always 1 and all others are equal to or smaller than 1. The number of values involved will diminish as the "time" (the index in the list of returned values) increases

list *data*

- Raw data for which the autocorrelation must be determined

::math::statistics::crosscorr *data1 data2*

Return the cross-correlation function as a list of values (assuming equidistance between samples, about 1/2 of the number of raw data)

The correlation is determined in such a way that the values can never exceed 1 in magnitude. The number of values involved will diminish as the "time" (the index in the list of returned values) increases.

list *data1*

- First list of data

list *data2*

- Second list of data

::math::statistics::mean-histogram-limits *mean stdev number*

Determine reasonable limits based on mean and standard deviation for a histogram Convenience function - the result is suitable for the histogram function.

float *mean*

- Mean of the data

float *stdev*

- Standard deviation

int *number*

- Number of limits to generate (defaults to 8)

::math::statistics::minmax-histogram-limits *min max number*

Determine reasonable limits based on a minimum and maximum for a histogram

Convenience function - the result is suitable for the histogram function.

float *min*

- Expected minimum

float *max*

- Expected maximum

int *number*

- Number of limits to generate (defaults to 8)

::math::statistics::linear-model *xdata ydata intercept*

Determine the coefficients for a linear regression between two series of data (the model: Y = A + B*X). Returns a list of parameters describing the fit

list *xdata*

- List of independent data

list *ydata*

- List of dependent data to be fitted

boolean *intercept*

- (Optional) compute the intercept (1, default) or fit to a line through the origin (0)

The result consists of the following list:
- (Estimate of) Intercept A
- (Estimate of) Slope B
- Standard deviation of Y relative to fit
- Correlation coefficient R2
- Number of degrees of freedom df
- Standard error of the intercept A
- Significance level of A
- Standard error of the slope B
- Significance level of B

::math::statistics::linear-residuals *xdata ydata intercept*

Determine the difference between actual data and predicted from the linear model.

Returns a list of the differences between the actual data and the predicted values.

list *xdata*

- List of independent data

list *ydata*

- List of dependent data to be fitted

boolean *intercept*

- (Optional) compute the intercept (1, default) or fit to a line through the origin (0)

::math::statistics::test-2x2 *n11 n21 n12 n22*

Determine if two set of samples, each from a binomial distribution, differ significantly or not (implying a different parameter).

Returns the "chi-square" value, which can be used to the determine the significance.

int *n11*

- Number of outcomes with the first value from the first sample.

int *n21*

- Number of outcomes with the first value from the second sample.

int *n12*

- Number of outcomes with the second value from the first sample.

int *n22*

- Number of outcomes with the second value from the second sample.

::math::statistics::print-2x2 *n11 n21 n12 n22*

Determine if two set of samples, each from a binomial distribution, differ significantly or not (implying a different parameter).

Returns a short report, useful in an interactive session.

int *n11*

- Number of outcomes with the first value from the first sample.

int *n21*

- Number of outcomes with the first value from the second sample.

int *n12*

- Number of outcomes with the second value from the first sample.

int *n22*

- Number of outcomes with the second value from the second sample.

::math::statistics::control-xbar *data* ?nsamples?

Determine the control limits for an xbar chart. The number of data in each subsample defaults to 4. At least 20 subsamples are required.

Returns the mean, the lower limit, the upper limit and the number of data per subsample.

list *data*

- List of observed data

int *nsamples*

- Number of data per subsample

::math::statistics::control-Rchart *data* ?nsamples?

Determine the control limits for an R chart. The number of data in each subsample (nsamples) defaults to 4. At least 20 subsamples are required.

Returns the mean range, the lower limit, the upper limit and the number of data per subsample.

list *data*

- List of observed data

int *nsamples*

- Number of data per subsample

::math::statistics::test-xbar *control data*

Determine if the data exceed the control limits for the xbar chart.

Returns a list of subsamples (their indices) that indeed violate the limits.

list *control*

- Control limits as returned by the "control-xbar" procedure

list *data*

- List of observed data

::math::statistics::test-Rchart *control data*

Determine if the data exceed the control limits for the R chart.

Returns a list of subsamples (their indices) that indeed violate the limits.

list *control*

- Control limits as returned by the "control-Rchart" procedure

list *data*

- List of observed data

::math::statistics::test-Kruskal-Wallis *confidence args*

Check if the population medians of two or more groups are equal with a given confidence level, using the Kruskal-Wallis test.

float *confidence*

- Confidence level to be used (0-1)

list *args*

- Two or more lists of data

::math::statistics::analyse-Kruskal-Wallis *args*

Compute the statistical parameters for the Kruskal-Wallis test. Returns the Kruskal-Wallis statistic and the probability that that value would occur assuming the medians of the populations are equal.

list *args*

- Two or more lists of data

::math::statistics::test-Levene *groups*

Compute the Levene statistic to determine if groups of data have the same variance (are homoscadastic) or not. The data are organised in groups. This version uses the mean of the data as the measure to determine the deviations. The statistic is equivalent to an F statistic with degrees of freedom k-1 and N-k, k being the number of groups and N the total number of data.

list *groups*

- List of groups of data

::math::statistics::test-Brown-Forsythe *groups*

Compute the Brown-Forsythe statistic to determine if groups of data have the same variance (are homoscadastic) or not. Like the Levene test, but this version uses the median of the data.

list *groups*

- List of groups of data

::math::statistics::group-rank *args*

Rank the groups of data with respect to the complete set. Returns a list consisting of the group ID, the value and the rank (possibly a rational number, in case of ties) for each data item.

list *args*

- Two or more lists of data

::math::statistics::test-Wilcoxon *sample_a sample_b*

Compute the Wilcoxon test statistic to determine if two samples have the same median or not. (The statistic can be regarded as standard normal, if the sample sizes are both larger than 10.) Returns the value of this statistic.

list *sample_a*

- List of data comprising the first sample

list *sample_b*

- List of data comprising the second sample

::math::statistics::spearman-rank *sample_a sample_b*

Return the Spearman rank correlation as an alternative to the ordinary (Pearson's) correlation coefficient. The two samples should have the same number of data.

list *sample_a*

- First list of data

list *sample_b*

- Second list of data

::math::statistics::spearman-rank-extended *sample_a sample_b*

Return the Spearman rank correlation as an alternative to the ordinary (Pearson's) correlation coefficient as well as additional data. The two samples should have the same number of data. The procedure returns the correlation coefficient, the number of data pairs used and the z-score, an approximately standard normal statistic, indicating the significance of the correlation.

list *sample_a*

- First list of data

list *sample_b*

- Second list of data

::math::statistics::kernel-density *data* opt *-option value ...*

Return the density function based on kernel density estimation. The procedure is controlled by a small set of options, each of which is given a reasonable default.

The return value consists of three lists: the centres of the bins, the associated probability density and a list of computational parameters (begin and end of the interval, mean and standard deviation and the used bandwidth). The computational parameters can be used for further analysis.

list *data*

- The data to be examined

list *args*

- Option-value pairs:

-weights *weights*

Per data point the weight (default: 1 for all data)

-bandwidth *value*

Bandwidth to be used for the estimation (default: determined from standard deviation)

-number *value*

Number of bins to be returned (default: 100)

-interval *{begin end}*

Begin and end of the interval for which the density is returned (default: mean +/- 3*standard deviation)

-kernel *function*

Kernel to be used (One of: gaussian, cosine, epanechnikov, uniform, triangular, biweight, logistic; default: gaussian)

::math::statistics::bootstrap *data sampleSize* ?numberSamples?

Create a subsample or subsamples from a given list of data. The data in the samples are chosen from this list - multiples may occur. If there is only one subsample, the sample itself is returned (as a list of "sampleSize" values), otherwise a list of samples is returned.

list *data*

List of values to chose from

int *sampleSize*

Number of values per sample

int *numberSamples*
Number of samples (default: 1)

::math::statistics::wasserstein-distance *prob1 prob2*
Compute the Wasserstein distance or earth mover's distance for two equidstantly spaced histograms or probability densities. The histograms need not to be normalised to sum to one, but they must have the same number of entries.

Note: the histograms are assumed to be based on the same equidistant intervals. As the bounds are not passed, the value is expressed in the length of the intervals.

list *prob1*
List of values for the first histogram/probability density

list *prob2*
List of values for the second histogram/probability density

::math::statistics::kl-divergence *prob1 prob2*
Compute the Kullback-Leibler (KL) divergence for two equidstantly spaced histograms or probability densities. The histograms need not to be normalised to sum to one, but they must have the same number of entries.

Note: the histograms are assumed to be based on the same equidistant intervals. As the bounds are not passed, the value is expressed in the length of the intervals.

Note also that the KL divergence is not symmetric and that the second histogram should not contain zeroes in places where the first histogram has non-zero values.

list *prob1*
List of values for the first histogram/probability density

list *prob2*
List of values for the second histogram/probability density

::math::statistics::logistic-model *xdata ydata*
Estimate the coefficients of the logistic model that fits the data best. The data consist of independent x-values and the outcome 0 or 1 for each of the x-values. The result can be used to estimate the probability that a certain x-value gives 1.

list *xdata*
List of values for which the success (1) or failure (0) is known

list *ydata*
List of successes or failures corresponding to each value in *xdata*.

::math::statistics::logistic-probability *coeffs x*
Calculate the probability of success for the value *x* given the coefficients of the logistic model.

list *coeffs*
List of coefficients as determine by the **logistic-model** command

float *x*
X-value for which the probability needs to be determined

Multivariate linear regression

Besides the linear regression with a single independent variable, the statistics package provides

two procedures for doing ordinary least squares (OLS) and weighted least squares (WLS) linear regression with several variables. They were written by Eric Kemp-Benedict.

In addition to these two, it provides a procedure (tstat) for calculating the value of the t-statistic for the specified number of degrees of freedom that is required to demonstrate a given level of significance.

Note: These procedures depend on the math::linearalgebra package.

Description of the procedures

::math::statistics::tstat *dof* ?alpha?

Returns the value of the t-distribution t* satisfying

```
P(t*)  =  1 - alpha/2
P(-t*) =  alpha/2
```

for the number of degrees of freedom dof.

Given a sample of normally-distributed data x, with an estimate xbar for the mean and sbar for the standard deviation, the alpha confidence interval for the estimate of the mean can be calculated as

```
( xbar - t* sbar , xbar + t* sbar)
```

The return values from this procedure can be compared to an estimated t-statistic to determine whether the estimated value of a parameter is significantly different from zero at the given confidence level.

int *dof*

Number of degrees of freedom

float *alpha*

Confidence level of the t-distribution. Defaults to 0.05.

::math::statistics::mv-wls *weights_and_values*

Carries out a weighted least squares linear regression for the data points provided, with weights assigned to each point.

The linear model is of the form

```
y = b0 + b1 * x1 + b2 * x2 ... + bN * xN + error
```

and each point satisfies

```
yi = b0 + b1 * xi1 + b2 * xi2 + ... + bN * xiN + Residual_i
```

The procedure returns a list with the following elements:

- The r-squared statistic
- The adjusted r-squared statistic
- A list containing the estimated coefficients b1, ... bN, b0 (The constant b0 comes last in the list.)
- A list containing the standard errors of the coefficients
- A list containing the 95% confidence bounds of the coefficients, with each set of bounds

returned as a list with two values

Arguments:

list *weights_and_values*

A list consisting of: the weight for the first observation, the data for the first observation (as a sublist), the weight for the second observation (as a sublist) and so on. The sublists of data are organised as lists of the value of the dependent variable y and the independent variables x1, x2 to xN.

Example of the use: The weight factors are quite simple: 0.2 for negative values to indicate we put less trust in these observations and 1.0 for all positive values.

```
# Store the value of the unicode value for the "+/-" character
set pm "\u00B1"

# Provide some data
set data { 0.2 {  -.67  14.18  60.03 -7.5  }
           1.0 { 36.97  15.52  34.24 14.61 }
           0.2 {-29.57  21.85  83.36 -7.   }
           0.2 {-16.9   11.79  51.67 -6.56 }
           1.0 { 14.09  16.24  36.97 -12.84}
           1.0 { 31.52  20.93  45.99 -25.4 }
           1.0 { 24.05  20.69  50.27  17.27}
           1.0 { 22.23  16.91  45.07  -4.3 }
           1.0 { 40.79  20.49  38.92  -.73 }
           0.2 {-10.35  17.24  58.77  18.78}}

# Call the ols routine
set results [::math::statistics::mv-ols $data]

# Pretty-print the results
puts "R-squared: [lindex $results 0]"
puts "Adj R-squared: [lindex $results 1]"
puts "Coefficients $pm s.e. -- \[95% confidence interval\]:"
foreach val [lindex $results 2] se [lindex $results 3] bounds
    [lindex $results 4] {
    set lb [lindex $bounds 0]
    set ub [lindex $bounds 1]
    puts "   $val $pm $se -- \[$lb to $ub\]"
}
```

::math::statistics::mv-ols *values*

Carries out an ordinary least squares linear regression for the data points provided.

This procedure simply calls ::mvlinreg::wls with the weights set to 1.0, and returns the same information.

Example of the use:

```
# Store the value of the unicode value for the "+/-" character
set pm "\u00B1"

# Provide some data
set data {{  -.67  14.18  60.03 -7.5  }
          { 36.97  15.52  34.24 14.61 }
          {-29.57  21.85  83.36 -7.   }
          {-16.9   11.79  51.67 -6.56 }
          { 14.09  16.24  36.97 -12.84}
          { 31.52  20.93  45.99 -25.4 }
          { 24.05  20.69  50.27  17.27}
          { 22.23  16.91  45.07  -4.3 }
          { 40.79  20.49  38.92  -.73 }
          {-10.35  17.24  58.77  18.78}}

# Call the ols routine
set results [::math::statistics::mv-ols $data]

# Pretty-print the results
puts "R-squared: [lindex $results 0]"
puts "Adj R-squared: [lindex $results 1]"
puts "Coefficients $pm s.e. -- \[95% confidence interval\]:"
foreach val [lindex $results 2] se [lindex $results 3] bounds [lindex
    $results 4] {
    set lb [lindex $bounds 0]
    set ub [lindex $bounds 1]
    puts "   $val $pm $se -- \[$lb to $ub\]"
}
```

Statistical distributions

In the literature a large number of probability distributions can be found. The statistics package supports:

- The normal or Gaussian distribution as well as the log-normal distribution
- The uniform distribution - equal probability for all data within a given interval
- The exponential distribution - useful as a model for certain extreme-value distributions.
- The gamma distribution - based on the incomplete Gamma integral
- The beta distribution
- The chi-square distribution
- The student's T distribution
- The Poisson distribution
- The Pareto distribution
- The Gumbel distribution
- The Weibull distribution
- The Cauchy distribution
- The F distribution (only the cumulative density function)
- PM - binomial.

In principle for each distribution one has procedures for:

- The probability density (pdf-*)
- The cumulative density (cdf-*)
- Quantiles for the given distribution (quantiles-*)
- Histograms for the given distribution (histogram-*)
- List of random values with the given distribution (random-*)

The following procedures have been implemented:

::math::statistics::pdf-normal *mean stdev value*

Return the probability of a given value for a normal distribution with given mean and standard deviation.

float *mean*

- Mean value of the distribution

float *stdev*

- Standard deviation of the distribution

float *value*

- Value for which the probability is required

::math::statistics::pdf-lognormal *mean stdev value*

Return the probability of a given value for a log-normal distribution with given mean and standard deviation.

float *mean*

- Mean value of the distribution

float *stdev*

- Standard deviation of the distribution

float *value*

- Value for which the probability is required

::math::statistics::pdf-exponential *mean value*

Return the probability of a given value for an exponential distribution with given mean.

float *mean*

- Mean value of the distribution

float *value*

- Value for which the probability is required

::math::statistics::pdf-uniform *xmin xmax value*

Return the probability of a given value for a uniform distribution with given extremes.

float *xmin*

- Minimum value of the distribution

float *xmin*

- Maximum value of the distribution

float *value*

- Value for which the probability is required

::math::statistics::pdf-triangular *xmin xmax value*

Return the probability of a given value for a triangular distribution with given extremes. If the

argument min is lower than the argument max, then smaller values have higher probability and vice versa. In the first case the probability density function is of the form *f(x) = 2(1-x)* and the other case it is of the form *f(x) = 2x*.

float *xmin*
- Minimum value of the distribution

float *xmin*
- Maximum value of the distribution

float *value*
- Value for which the probability is required

::math::statistics::pdf-symmetric-triangular *xmin xmax value*

Return the probability of a given value for a symmetric triangular distribution with given extremes.

float *xmin*
- Minimum value of the distribution

float *xmin*
- Maximum value of the distribution

float *value*
- Value for which the probability is required

::math::statistics::pdf-gamma *alpha beta value*

Return the probability of a given value for a Gamma distribution with given shape and rate parameters

float *alpha*
- Shape parameter

float *beta*
- Rate parameter

float *value*
- Value for which the probability is required

::math::statistics::pdf-poisson *mu k*

Return the probability of a given number of occurrences in the same interval (k) for a Poisson distribution with given mean (mu)

float *mu*
- Mean number of occurrences

int *k*
- Number of occurences

::math::statistics::pdf-chisquare *df value*

Return the probability of a given value for a chi square distribution with given degrees of freedom

float *df*
- Degrees of freedom

float *value*
- Value for which the probability is required

::math::statistics::pdf-student-t *df value*

Return the probability of a given value for a Student's t distribution with given degrees of freedom

float *df*

- Degrees of freedom

float *value*

- Value for which the probability is required

::math::statistics::pdf-gamma *a b value*

Return the probability of a given value for a Gamma distribution with given shape and rate parameters

float *a*

- Shape parameter

float *b*

- Rate parameter

float *value*

- Value for which the probability is required

::math::statistics::pdf-beta *a b value*

Return the probability of a given value for a Beta distribution with given shape parameters

float *a*

- First shape parameter

float *b*

- Second shape parameter

float *value*

- Value for which the probability is required

::math::statistics::pdf-weibull *scale shape value*

Return the probability of a given value for a Weibull distribution with given scale and shape parameters

float *location*

- Scale parameter

float *scale*

- Shape parameter

float *value*

- Value for which the probability is required

::math::statistics::pdf-gumbel *location scale value*

Return the probability of a given value for a Gumbel distribution with given location and shape parameters

float *location*

- Location parameter

float *scale*

- Shape parameter

float *value*

- Value for which the probability is required

::math::statistics::pdf-pareto *scale shape value*

Return the probability of a given value for a Pareto distribution with given scale and shape parameters

float *scale*

- Scale parameter

float *shape*

- Shape parameter

float *value*

- Value for which the probability is required

::math::statistics::pdf-cauchy *location scale value*

Return the probability of a given value for a Cauchy distribution with given location and shape parameters. Note that the Cauchy distribution has no finite higher-order moments.

float *location*

- Location parameter

float *scale*

- Shape parameter

float *value*

- Value for which the probability is required

::math::statistics::pdf-laplace *location scale value*

Return the probability of a given value for a Laplace distribution with given location and shape parameters. The Laplace distribution consists of two exponential functions, is peaked and has heavier tails than the normal distribution.

float *location*

- Location parameter (mean)

float *scale*

- Shape parameter

float *value*

- Value for which the probability is required

::math::statistics::pdf-kumaraswamy *a b value*

Return the probability of a given value for a Kumaraswamy distribution with given parameters a and b. The Kumaraswamy distribution is related to the Beta distribution, but has a tractable cumulative distribution function.

float *a*

- Parameter a

float *b*

- Parameter b

float *value*

- Value for which the probability is required

::math::statistics::pdf-negative-binomial *r p value*

Return the probability of a given value for a negative binomial distribution with an allowed number of failures and the probability of success.

int *r*

- Allowed number of failures (at least 1)

float *p*

- Probability of success

int *value*

- Number of successes for which the probability is to be returned

::math::statistics::cdf-normal *mean stdev value*

Return the cumulative probability of a given value for a normal distribution with given mean and standard deviation, that is the probability for values up to the given one.

float *mean*

- Mean value of the distribution

float *stdev*

- Standard deviation of the distribution

float *value*

- Value for which the probability is required

::math::statistics::cdf-lognormal *mean stdev value*

Return the cumulative probability of a given value for a log-normal distribution with given mean and standard deviation, that is the probability for values up to the given one.

float *mean*

- Mean value of the distribution

float *stdev*

- Standard deviation of the distribution

float *value*

- Value for which the probability is required

::math::statistics::cdf-exponential *mean value*

Return the cumulative probability of a given value for an exponential distribution with given mean.

float *mean*

- Mean value of the distribution

float *value*

- Value for which the probability is required

::math::statistics::cdf-uniform *xmin xmax value*

Return the cumulative probability of a given value for a uniform distribution with given extremes.

float *xmin*

- Minimum value of the distribution

float *xmin*

- Maximum value of the distribution

float *value*

- Value for which the probability is required

::math::statistics::cdf-triangular *xmin xmax value*

Return the cumulative probability of a given value for a triangular distribution with given

extremes. If xmin < xmax, then lower values have a higher probability and vice versa, see also *pdf-triangular*

float *xmin*

- Minimum value of the distribution

float *xmin*

- Maximum value of the distribution

float *value*

- Value for which the probability is required

::math::statistics::cdf-symmetric-triangular *xmin xmax value*

Return the cumulative probability of a given value for a symmetric triangular distribution with given extremes.

float *xmin*

- Minimum value of the distribution

float *xmin*

- Maximum value of the distribution

float *value*

- Value for which the probability is required

::math::statistics::cdf-students-t *degrees value*

Return the cumulative probability of a given value for a Student's t distribution with given number of degrees.

int *degrees*

- Number of degrees of freedom

float *value*

- Value for which the probability is required

::math::statistics::cdf-gamma *alpha beta value*

Return the cumulative probability of a given value for a Gamma distribution with given shape and rate parameters.

float *alpha*

- Shape parameter

float *beta*

- Rate parameter

float *value*

- Value for which the cumulative probability is required

::math::statistics::cdf-poisson *mu k*

Return the cumulative probability of a given number of occurrences in the same interval (k) for a Poisson distribution with given mean (mu).

float *mu*

- Mean number of occurrences

int *k*

- Number of occurences

::math::statistics::cdf-beta *a b value*

Return the cumulative probability of a given value for a Beta distribution with given shape parameters

float *a*
- First shape parameter

float *b*
- Second shape parameter

float *value*
- Value for which the probability is required

::math::statistics::cdf-weibull *scale shape value*

Return the cumulative probability of a given value for a Weibull distribution with given scale and shape parameters.

float *scale*
- Scale parameter

float *shape*
- Shape parameter

float *value*
- Value for which the probability is required

::math::statistics::cdf-gumbel *location scale value*

Return the cumulative probability of a given value for a Gumbel distribution with given location and scale parameters.

float *location*
- Location parameter

float *scale*
- Scale parameter

float *value*
- Value for which the probability is required

::math::statistics::cdf-pareto *scale shape value*

Return the cumulative probability of a given value for a Pareto distribution with given scale and shape parameters

float *scale*
- Scale parameter

float *shape*
- Shape parameter

float *value*
- Value for which the probability is required

::math::statistics::cdf-cauchy *location scale value*

Return the cumulative probability of a given value for a Cauchy distribution with given location and scale parameters.

float *location*
- Location parameter

float *scale*
- Scale parameter

float *value*
- Value for which the probability is required

::math::statistics::cdf-F *nf1 nf2 value*

Return the cumulative probability of a given value for an F distribution with nf1 and nf2 degrees of freedom.

float *nf1*
- Degrees of freedom for the numerator

float *nf2*
- Degrees of freedom for the denominator

float *value*
- Value for which the probability is required

::math::statistics::cdf-laplace *location scale value*

Return the cumulative probability of a given value for a Laplace distribution with given location and shape parameters. The Laplace distribution consists of two exponential functions, is peaked and has heavier tails than the normal distribution.

float *location*
- Location parameter (mean)

float *scale*
- Shape parameter

float *value*
- Value for which the probability is required

::math::statistics::cdf-kumaraswamy *a b value*

Return the cumulative probability of a given value for a Kumaraswamy distribution with given parameters a and b. The Kumaraswamy distribution is related to the Beta distribution, but has a tractable cumulative distribution function.

float *a*
- Parameter a

float *b*
- Parameter b

float *value*
- Value for which the probability is required

::math::statistics::cdf-negative-binomial *r p value*

Return the cumulative probability of a given value for a negative binomial distribution with an allowed number of failures and the probability of success.

int *r*
- Allowed number of failures (at least 1)

float *p*
- Probability of success

int *value*
- Greatest number of successes

::math::statistics::empirical-distribution *values*

Return a list of values and their empirical probability. The values are sorted in increasing order. (The implementation follows the description at the corresponding Wikipedia page)

list *values*

- List of data to be examined

::math::statistics::random-normal *mean stdev number*

Return a list of "number" random values satisfying a normal distribution with given mean and standard deviation.

float *mean*

- Mean value of the distribution

float *stdev*

- Standard deviation of the distribution

int *number*

- Number of values to be returned

::math::statistics::random-lognormal *mean stdev number*

Return a list of "number" random values satisfying a log-normal distribution with given mean and standard deviation.

float *mean*

- Mean value of the distribution

float *stdev*

- Standard deviation of the distribution

int *number*

- Number of values to be returned

::math::statistics::random-exponential *mean number*

Return a list of "number" random values satisfying an exponential distribution with given mean.

float *mean*

- Mean value of the distribution

int *number*

- Number of values to be returned

::math::statistics::random-uniform *xmin xmax number*

Return a list of "number" random values satisfying a uniform distribution with given extremes.

float *xmin*

- Minimum value of the distribution

float *xmax*

- Maximum value of the distribution

int *number*

- Number of values to be returned

::math::statistics::random-triangular *xmin xmax number*

Return a list of "number" random values satisfying a triangular distribution with given extremes. If xmin < xmax, then lower values have a higher probability and vice versa (see also *pdf-triangular*.

float *xmin*
- Minimum value of the distribution

float *xmax*
- Maximum value of the distribution

int *number*
- Number of values to be returned

::math::statistics::random-symmetric-triangular *xmin xmax number*

Return a list of "number" random values satisfying a symmetric triangular distribution with given extremes.

float *xmin*
- Minimum value of the distribution

float *xmax*
- Maximum value of the distribution

int *number*
- Number of values to be returned

::math::statistics::random-gamma *alpha beta number*

Return a list of "number" random values satisfying a Gamma distribution with given shape and rate parameters.

float *alpha*
- Shape parameter

float *beta*
- Rate parameter

int *number*
- Number of values to be returned

::math::statistics::random-poisson *mu number*

Return a list of "number" random values satisfying a Poisson distribution with given mean.

float *mu*
- Mean of the distribution

int *number*
- Number of values to be returned

::math::statistics::random-chisquare *df number*

Return a list of "number" random values satisfying a chi square distribution with given degrees of freedom.

float *df*
- Degrees of freedom

int *number*
- Number of values to be returned

::math::statistics::random-student-t *df number*

Return a list of "number" random values satisfying a Student's t distribution with given degrees of freedom.

float *df*

- Degrees of freedom

int *number*

- Number of values to be returned

::math::statistics::random-beta *a b number*

Return a list of "number" random values satisfying a Beta distribution with given shape parameters.

float *a*

- First shape parameter

float *b*

- Second shape parameter

int *number*

- Number of values to be returned

::math::statistics::random-weibull *scale shape number*

Return a list of "number" random values satisfying a Weibull distribution with given scale and shape parameters.

float *scale*

- Scale parameter

float *shape*

- Shape parameter

int *number*

- Number of values to be returned

::math::statistics::random-gumbel *location scale number*

Return a list of "number" random values satisfying a Gumbel distribution with given location and scale parameters.

float *location*

- Location parameter

float *scale*

- Scale parameter

int *number*

- Number of values to be returned

::math::statistics::random-pareto *scale shape number*

Return a list of "number" random values satisfying a Pareto distribution with given scale and shape parameters.

float *scale*

- Scale parameter

float *shape*

- Shape parameter

int *number*

- Number of values to be returned

::math::statistics::random-cauchy *location scale number*

Return a list of "number" random values satisfying a Cauchy distribution with given location

and scale parameters.

float *location*

- Location parameter

float *scale*

- Scale parameter

int *number*

- Number of values to be returned

::math::statistics::random-laplace *location scale number*

Return a list of "number" random values satisfying a Laplace distribution with given location and shape parameters. The Laplace distribution consists of two exponential functions, is peaked and has heavier tails than the normal distribution.

float *location*

- Location parameter (mean)

float *scale*

- Shape parameter

int *number*

- Number of values to be returned

::math::statistics::random-kumaraswamy *a b number*

Return a list of "number" random values satisying a Kumaraswamy distribution with given parameters a and b. The Kumaraswamy distribution is related to the Beta distribution, but has a tractable cumulative distribution function.

float *a*

- Parameter a

float *b*

- Parameter b

int *number*

- Number of values to be returned

::math::statistics::random-negative-binomial *r p number*

Return a list of "number" random values satisying a negative binomial distribution.

int *r*

- Allowed number of failures (at least 1)

float *p*

- Probability of success

int *number*

- Number of values to be returned

::math::statistics::histogram-uniform *xmin xmax limits number*

Return the expected histogram for a uniform distribution.

float *xmin*

- Minimum value of the distribution

float *xmax*

- Maximum value of the distribution

list *limits*

- Upper limits for the buckets in the histogram

int *number*

- Total number of "observations" in the histogram

::math::statistics::incompleteGamma *x p* ?tol?

Evaluate the incomplete Gamma integral

```
                  1       / x               p-1
     P(p,x) =  --------    |   dt exp(-t) * t
               Gamma(p)  / 0
```

float *x*

- Value of x (limit of the integral)

float *p*

- Value of p in the integrand

float *tol*

- Required tolerance (default: 1.0e-9)

::math::statistics::incompleteBeta *a b x* ?tol?

Evaluate the incomplete Beta integral

float *a*

- First shape parameter

float *b*

- Second shape parameter

float *x*

- Value of x (limit of the integral)

float *tol*

- Required tolerance (default: 1.0e-9)

::math::statistics::estimate-pareto *values*

Estimate the parameters for the Pareto distribution that comes closest to the given values. Returns the estimated scale and shape parameters, as well as the standard error for the shape parameter.

list *values*

- List of values, assumed to be distributed according to a Pareto distribution

::math::statistics::estimate-exponential *values*

Estimate the parameter for the exponential distribution that comes closest to the given values. Returns an estimate of the one parameter and of the standard error.

list *values*

- List of values, assumed to be distributed according to an exponential distribution

::math::statistics::estimate-laplace *values*

Estimate the parameters for the Laplace distribution that comes closest to the given values. Returns an estimate of respectively the location and scale parameters, based on maximum likelihood.

list *values*

- List of values, assumed to be distributed according to an exponential distribution

::math::statistics::estimante-negative-binomial *r values*
Estimate the probability of success for the negative binomial distribution that comes closest to the given values. The allowed number of failures must be given.
int *r*
- Allowed number of failures (at least 1)

int *number*
- List of values, assumed to be distributed according to a negative binomial distribution.

TO DO: more function descriptions to be added

Data manipulation

The data manipulation procedures act on lists or lists of lists:

::math::statistics::filter *varname data expression*
Return a list consisting of the data for which the logical expression is true (this command works analogously to the command **foreach**).
string *varname*
- Name of the variable used in the expression

list *data*
- List of data

string *expression*
- Logical expression using the variable name

::math::statistics::map *varname data expression*
Return a list consisting of the data that are transformed via the expression.
string *varname*
- Name of the variable used in the expression

list *data*
- List of data

string *expression*
- Expression to be used to transform (map) the data

::math::statistics::samplescount *varname list expression*
Return a list consisting of the *counts* of all data in the sublists of the "list" argument for which the expression is true.
string *varname*
- Name of the variable used in the expression

list *data*
- List of sublists, each containing the data

string *expression*
- Logical expression to test the data (defaults to "true").

::math::statistics::subdivide
Routine *PM* - not implemented yet

Plot procedures

The following simple plotting procedures are available:

::math::statistics::plot-scale *canvas xmin xmax ymin ymax*

Set the scale for a plot in the given canvas. All plot routines expect this function to be called first. There is no automatic scaling provided.

widget *canvas*

- Canvas widget to use

float *xmin*

- Minimum x value

float *xmax*

- Maximum x value

float *ymin*

- Minimum y value

float *ymax*

- Maximum y value

::math::statistics::plot-xydata *canvas xdata ydata tag*

Create a simple XY plot in the given canvas - the data are shown as a collection of dots. The tag can be used to manipulate the appearance.

widget *canvas*

- Canvas widget to use

float *xdata*

- Series of independent data

float *ydata*

- Series of dependent data

string *tag*

- Tag to give to the plotted data (defaults to xyplot)

::math::statistics::plot-xyline *canvas xdata ydata tag*

Create a simple XY plot in the given canvas - the data are shown as a line through the data points. The tag can be used to manipulate the appearance.

widget *canvas*

- Canvas widget to use

list *xdata*

- Series of independent data

list *ydata*

- Series of dependent data

string *tag*

- Tag to give to the plotted data (defaults to xyplot)

::math::statistics::plot-tdata *canvas tdata tag*

Create a simple XY plot in the given canvas - the data are shown as a collection of dots. The horizontal coordinate is equal to the index. The tag can be used to manipulate the appearance. This type of presentation is suitable for autocorrelation functions for instance or for inspecting the time-dependent behaviour.

widget *canvas*
- Canvas widget to use

list *tdata*
- Series of dependent data

string *tag*
- Tag to give to the plotted data (defaults to xyplot)

::math::statistics::plot-tline *canvas tdata tag*

Create a simple XY plot in the given canvas - the data are shown as a line. See plot-tdata for an explanation.

widget *canvas*
- Canvas widget to use

list *tdata*
- Series of dependent data

string *tag*
- Tag to give to the plotted data (defaults to xyplot)

::math::statistics::plot-histogram *canvas counts limits tag*

Create a simple histogram in the given canvas

widget *canvas*
- Canvas widget to use

list *counts*
- Series of bucket counts

list *limits*
- Series of upper limits for the buckets

string *tag*
- Tag to give to the plotted data (defaults to xyplot)

Things to do

The following procedures are yet to be implemented:

- F-test-stdev
- interval-mean-stdev
- histogram-normal
- histogram-exponential
- test-histogram
- test-corr
- quantiles-*
- fourier-coeffs
- fourier-residuals
- onepar-function-fit
- onepar-function-residuals
- plot-linear-model
- subdivide

Examples

The code below is a small example of how you can examine a set of data:

```
# Simple example:
# - Generate data (as a cheap way of getting some)
# - Perform statistical analysis to describe the data
#
package require math::statistics

#
# Two auxiliary procs
#
proc pause {time} {
   set wait 0
   after [expr {$time*1000}] {set ::wait 1}
   vwait wait
}

proc print-histogram {counts limits} {
   foreach count $counts limit $limits {
      if { $limit != {} } {
         puts [format "<%12.4g\t%d" $limit $count]
         set prev_limit $limit
      } else {
         puts [format ">%12.4g\t%d" $prev_limit $count]
      }
   }
}

#
# Our source of arbitrary data
#
proc generateData { data1 data2 } {
   upvar 1 $data1 _data1
   upvar 1 $data2 _data2

   set d1 0.0
   set d2 0.0
   for { set i 0 } { $i < 100 } { incr i } {
      set d1 [expr {10.0-2.0*cos(2.0*3.1415926*$i/24.0)+3.5*rand()}]
      set d2 [expr {0.7*$d2+0.3*$d1+0.7*rand()}]
      lappend _data1 $d1
      lappend _data2 $d2
   }
   return {}
}
```

```
#
# The analysis session
#
package require Tk
console show
canvas .plot1
canvas .plot2
pack   .plot1 .plot2 -fill both -side top

generateData data1 data2

puts "Basic statistics:"
set b1 [::math::statistics::basic-stats $data1]
set b2 [::math::statistics::basic-stats $data2]
foreach label {mean min max number stdev var} v1 $b1 v2 $b2 {
   puts "$label\t$v1\t$v2"
}
puts "Plot the data as function of \"time\" and against each other"
::math::statistics::plot-scale .plot1  0 100  0 20
::math::statistics::plot-scale .plot2  0 20   0 20
::math::statistics::plot-tline .plot1 $data1
::math::statistics::plot-tline .plot1 $data2
::math::statistics::plot-xydata .plot2 $data1 $data2

puts "Correlation coefficient:"
puts [::math::statistics::corr $data1 $data2]

pause 2
puts "Plot histograms"
.plot2 delete all
::math::statistics::plot-scale .plot2  0 20 0 100
set limits         [::math::statistics::minmax-histogram-limits 7 16]
set histogram_data [::math::statistics::histogram $limits $data1]
::math::statistics::plot-histogram .plot2 $histogram_data $limits

puts "First series:"
print-histogram $histogram_data $limits

pause 2
set limits         [::math::statistics::minmax-histogram-limits 0 15
    10]
set histogram_data [::math::statistics::histogram $limits $data2]
::math::statistics::plot-histogram .plot2 $histogram_data $limits d2
.plot2 itemconfigure d2 -fill red

puts "Second series:"
```

```
print-histogram $histogram_data $limits

puts "Autocorrelation function:"
set  autoc [::math::statistics::autocorr $data1]
puts [::math::statistics::map $autoc {[format "%.2f" $x]}]
puts "Cross-correlation function:"
set  crossc [::math::statistics::crosscorr $data1 $data2]
puts [::math::statistics::map $crossc {[format "%.2f" $x]}]

::math::statistics::plot-scale .plot1  0 100 -1  4
::math::statistics::plot-tline .plot1  $autoc "autoc"
::math::statistics::plot-tline .plot1  $crossc "crossc"
.plot1 itemconfigure autoc  -fill green
.plot1 itemconfigure crossc -fill yellow

puts "Quantiles: 0.1, 0.2, 0.5, 0.8, 0.9"
puts "First:  [::math::statistics::quantiles $data1 {0.1 0.2 0.5 0.8
    0.9}]"
puts "Second: [::math::statistics::quantiles $data2 {0.1 0.2 0.5 0.8
    0.9}]"
```

If you run this example, then the following should be clear:

- There is a strong correlation between two time series, as displayed by the raw data and especially by the correlation functions.
- Both time series show a significant periodic component
- The histograms are not very useful in identifying the nature of the time series - they do not show the periodic nature.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *math :: statistics* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Mathematics

symdiff

Keywords: symdiff

Name

math::calculus::symdiff - Symbolic differentiation for Tcl

Synopsis

package require Tcl 8.5 9
package require grammar::aycock 1.0
package require math::calculus::symdiff 1.0.2
math::calculus::symdiff::symdiff *expression variable*
math::calculus::jacobian *variableDict*

Description

The **math::calculus::symdiff** package provides a symbolic differentiation facility for Tcl math expressions. It is useful for providing derivatives to packages that either require the Jacobian of a set of functions or else are more efficient or stable when the Jacobian is provided.

Procedures

The **math::calculus::symdiff** package exports the two procedures:

math::calculus::symdiff::symdiff *expression variable*
: Differentiates the given *expression* with respect to the specified *variable*. (See Expressions below for a discussion of the subset of Tcl math expressions that are acceptable to **math::calculus::symdiff**.) The result is a Tcl expression that evaluates the derivative. Returns an error if *expression* is not a well-formed expression or is not differentiable.

math::calculus::jacobian *variableDict*
: Computes the Jacobian of a system of equations. The system is given by the dictionary *variableDict*, whose keys are the names of variables in the system, and whose values are Tcl expressions giving the values of those variables. (See Expressions below for a discussion of the subset of Tcl math expressions that are acceptable to **math::calculus::symdiff**. The result is a list of lists: the i'th element of the j'th sublist is the partial derivative of the i'th variable with respect to the j'th variable. Returns an error if any of the expressions cannot be differentiated, or if *variableDict* is not a well-formed dictionary.

Expressions

The **math::calculus::symdiff** package accepts only a small subset of the expressions that are acceptable to Tcl commands such as **expr** or **if**. Specifically, the only constructs accepted are:

- Floating-point constants such as **5** or **3.14159e+00**.
- References to Tcl variable using $-substitution. The variable names must consist of alphanumerics and underscores: the **${...}** notation is not accepted.
- Parentheses.
- The **+**, **-**, *, /. and ** operators.
- Calls to the functions acos, asin, atan, **atan2**, cos, cosh, exp, **hypot**, **log**, **log10**, pow, sin, sinh . sqrt, tan, and tanh.

Command substitution, backslash substitution, and argument expansion are not accepted.

Examples

```
math::calculus::symdiff::symdiff {($a*$x+$b)*($c*$x+$d)} x
==> (($c * (($a * $x) + $b)) + ($a * (($c * $x) + $d)))
math::calculus::symdiff::jacobian {x {$a * $x + $b * $y}
                           y {$c * $x + $d * $y}}
==> {{$a} {$b}} {{$c} {$d}}
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *math :: calculus* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Copyright


```
Copyright (c) 2010 by Kevin B. Kenny <kennykb@acm.org>
Redistribution permitted under the terms of the Open Publication
    License <http://www.opencontent.org/openpub/>
```


trig

Keywords: trig, math, trigonometry

Name

math::trig - Trigonometric anf hyperbolic functions

Synopsis

package require Tcl 8.5 9
package require math::trig 1.1
::math::trig::radian_reduced *angle*
::math::trig::degree_reduced *angle*
::math::trig::cosec *angle*
::math::trig::sec *angle*
::math::trig::cotan *angle*
::math::trig::acosec *value*
::math::trig::asec *value*
::math::trig::acotan *value*
::math::trig::cosech *value*
::math::trig::sech *value*
::math::trig::cotanh *value*
::math::trig::asinh *value*
::math::trig::acosh *value*
::math::trig::atanh *value*
::math::trig::acosech *value*
::math::trig::asech *value*

::math::trig::acotanh *value*
::math::trig::sind *angle*
::math::trig::cosd *angle*
::math::trig::tand *angle*
::math::trig::cosecd *angle*
::math::trig::secd *angle*
::math::trig::cotand *angle*

Description

The *math::trig* package defines a set of trigonomic and hyperbolic functions and their inverses. In addition it defines versions of the trigonomic functions that take arguments in degrees instead of radians.

For easy use these functions may be imported into the *tcl::mathfunc* namespace, so that they can be used directly in the *expr* command.

Functions

The functions *radian_reduced* and *degree_reduced* return a reduced angle, in respectively radians and degrees, in the intervals [0, 2pi) and [0, 360):

::math::trig::radian_reduced *angle*
Return the equivalent angle in the interval [0, 2pi).
float *angle*
Angle (in radians)

::math::trig::degree_reduced *angle*
Return the equivalent angle in the interval [0, 360).
float *angle*
Angle (in degrees)

The following trigonomic functions are defined in addition to the ones defined in the *expr* command:

::math::trig::cosec *angle*
Calculate the cosecant of the angle (1/cos(angle))
float *angle*
Angle (in radians)

::math::trig::sec *angle*
Calculate the secant of the angle (1/sin(angle))
float *angle*
Angle (in radians)

::math::trig::cotan *angle*
Calculate the cotangent of the angle (1/tan(angle))
float *angle*
Angle (in radians)

For these functions also the inverses are defined:

::math::trig::acosec *value*
Calculate the arc cosecant of the value
float *value*
Value of the argument

::math::trig::asec *value*
Calculate the arc secant of the value
float *value*
Value of the argument

::math::trig::acotan *value*
Calculate the arc cotangent of the value
float *value*
Value of the argument

The following hyperbolic and inverse hyperbolic functions are defined:

::math::trig::cosech *value*
Calculate the hyperbolic cosecant of the value (1/sinh(value))
float *value*
Value of the argument

::math::trig::sech *value*
Calculate the hyperbolic secant of the value (1/cosh(value))
float *value*
Value of the argument

::math::trig::cotanh *value*
Calculate the hyperbolic cotangent of the value (1/tanh(value))
float *value*
Value of the argument

::math::trig::asinh *value*
Calculate the arc hyperbolic sine of the value
float *value*
Value of the argument

::math::trig::acosh *value*
Calculate the arc hyperbolic cosine of the value
float *value*
Value of the argument

::math::trig::atanh *value*
Calculate the arc hyperbolic tangent of the value
float *value*
Value of the argument

::math::trig::acosech *value*
Calculate the arc hyperbolic cosecant of the value

float *value*
Value of the argument

::math::trig::asech *value*
Calculate the arc hyperbolic secant of the value
float *value*
Value of the argument

::math::trig::acotanh *value*
Calculate the arc hyperbolic cotangent of the value
float *value*
Value of the argument

The following versions of the common trigonometric functions and their inverses are defined:

::math::trig::sind *angle*
Calculate the sine of the angle (in degrees)
float *angle*
Angle (in degrees)

::math::trig::cosd *angle*
Calculate the cosine of the angle (in degrees)
float *angle*
Angle (in radians)

::math::trig::tand *angle*
Calculate the cotangent of the angle (in degrees)
float *angle*
Angle (in degrees)

::math::trig::cosecd *angle*
Calculate the cosecant of the angle (in degrees)
float *angle*
Angle (in degrees)

::math::trig::secd *angle*
Calculate the secant of the angle (in degrees)
float *angle*
Angle (in degrees)

::math::trig::cotand *angle*
Calculate the cotangent of the angle (in degrees)
float *angle*
Angle (in degrees)

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib

Trackers and specify *math :: trig* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Mathematics

Copyright

```
Copyright (c) 2018 Arjen Markus
```


md4

md4
Keywords: md4, hashing, message-digest, rfc 1320, rfc 1321, rfc 2104, security

Name
md4 - MD4 Message-Digest Algorithm

Synopsis
package require Tcl 8.5 9
package require md4 ?**1.0.8**?
::md4::md4 ?*-hex*? [*-channel channel* | *-file filename* | *string*]
::md4::hmac ?*-hex*? *-key key* [*-channel channel* | *-file filename* | *string*]
::md4::MD4Init
::md4::MD4Update *token data*
::md4::MD4Final *token*
::md4::HMACInit *key*
::md4::HMACUpdate *token data*
::md4::HMACFinal *token*

Description
This package is an implementation in Tcl of the MD4 message-digest algorithm as described in RFC 1320 (1) and (2). This algorithm takes an arbitrary quantity of data and generates a 128-bit message digest from the input. The MD4 algorithm is faster but potentially weaker than the related MD5 algorithm (3).

If you have **critcl** and have built the **tcllibc** package then the implementation of the hashing function will be performed by compiled code. Alternatively if **cryptkit** is available this will be used. If no accelerator package can be found then the pure-tcl implementation is used. The programming interface remains the same in all cases.

BEWARE The commands in this package expect binary data as their input. When a **-file** is provided then this is ensured by the commands themselves, as they open the referenced file in binary mode. When literal data, or a **-channel** are provided instead, then the command's *caller is responsible* for ensuring this fact. The necessary conversion command is **encoding**

convertto utf-8 $string.

ATTENTION, there is a *Tcl 9 COMPATIBILITY ISSUE* here.

Tcl 8.x silently mishandles non-binary input by cutting it internally to size. I.e. by only using the low byte of a higher unicode point. No error is thrown, the *result is simply wrong*.

Tcl 9 throws an error instead, i.e.

```
expected byte sequence but character <location> was ...
```

Commands

::md4::md4 ?*-hex*? [*-channel channel* | *-file filename* | *string*]

Calculate the MD4 digest of the data given in string. This is returned as a binary string by default. Giving the *-hex* option will return a hexadecimal encoded version of the digest.

The data to be hashed can be specified either as a string argument to the md4 command, or as a filename or a pre-opened channel. If the *-filename* argument is given then the file is opened, the data read and hashed and the file is closed. If the *-channel* argument is given then data is read from the channel until the end of file. The channel is not closed.

Only one of *-file*, *-channel* or *string* should be given.

::md4::hmac ?*-hex*? *-key key* [*-channel channel* | *-file filename* | *string*]

Calculate an Hashed Message Authentication digest (HMAC) using the MD4 digest algorithm. HMACs are described in RFC 2104 (4) and provide an MD4 digest that includes a key. All options other than *-key* are as for the **::md4::md4** command.

Programming interface

For the programmer, the MD4 hash can be viewed as a bucket into which one pours data. When you have finished, you extract a value that is derived from the data that was poured into the bucket. The programming interface to the MD4 hash operates on a token (equivalent to the bucket). You call **MD4Init** to obtain a token and then call **MD4Update** as many times as required to add data to the hash. To release any resources and obtain the hash value, you then call **MD4Final**. An equivalent set of functions gives you a keyed digest (HMAC).

::md4::MD4Init

Begins a new MD4 hash. Returns a token ID that must be used for the remaining functions.

::md4::MD4Update *token data*

Add data to the hash identified by token. Calling *MD4Update $token* "*abcd*" is equivalent to calling *MD4Update $token* "*ab*" followed by *MD4Update $token* "*cb*". See Examples.

::md4::MD4Final *token*

Returns the hash value and releases any resources held by this token. Once this command completes the token will be invalid. The result is a binary string of 16 bytes representing the 128 bit MD4 digest value.

::md4::HMACInit *key*

This is equivalent to the **::md4::MD4Init** command except that it requires the key that will be included in the HMAC.

::md4::HMACUpdate *token data*

see below ...

::md4::HMACFinal *token*
These commands are identical to the MD4 equivalent commands.

Examples

```
% md4::md4 -hex "Tcl does MD4"
858da9b31f57648a032230447bd15f25
```

```
% md4::hmac -hex -key Sekret "Tcl does MD4"
c324088e5752872689caedf2a0464758
```

```
% set tok [md4::MD4Init]
::md4::1
% md4::MD4Update $tok "Tcl "
% md4::MD4Update $tok "does "
% md4::MD4Update $tok "MD4"
% md4::Hex [md4::MD4Final $tok]
858da9b31f57648a032230447bd15f25
```

References

1. Rivest, R., "The MD4 Message Digest Algorithm", RFC 1320, MIT, April 1992. (http://www.rfc-editor.org/rfc/rfc1320.txt)
2. Rivest, R., "The MD4 message digest algorithm", in A.J. Menezes and S.A. Vanstone, editors, Advances in Cryptology - CRYPTO '90 Proceedings, pages 303-311, Springer-Verlag, 1991.
3. Rivest, R., "The MD5 Message-Digest Algorithm", RFC 1321, MIT and RSA Data Security, Inc, April 1992. (http://www.rfc-editor.org/rfc/rfc1321.txt)
4. Krawczyk, H., Bellare, M. and Canetti, R. "HMAC: Keyed-Hashing for Message Authentication", RFC 2104, February 1997. (http://www.rfc-editor.org/rfc/rfc2104.txt)

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *md4* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Hashes, checksums, and encryption

Copyright

```
Copyright (c) 2003, Pat Thoyts <patthoyts@users.sourceforge.net>
```

md5

md5

Keywords: md5, hashing, message-digest, rfc 1320, rfc 1321, rfc 2104, security

Name

md5 - MD5 Message-Digest Algorithm

Synopsis

package require Tcl 8.5 9
package require md5 ?**2.0.9**?
::md5::md5 ?*-hex*? [*-channel channel* | *-file filename* | *string*]
::md5::hmac ?*-hex*? *-key key* [*-channel channel* | *-file filename* | *string*]
::md5::MD5Init
::md5::MD5Update *token data*
::md5::MD5Final *token*
::md5::HMACInit *key*
::md5::HMACUpdate *token data*
::md5::HMACFinal *token*

Description

This package is an implementation in Tcl of the MD5 message-digest algorithm as described in RFC 1321 (1). This algorithm takes an arbitrary quantity of data and generates a 128-bit message digest from the input. The MD5 algorithm is related to the MD4 algorithm (2) but has been strengthened against certain types of potential attack. MD5 should be used in preference to MD4 for new applications.

If you have **critcl** and have built the **tcllibc** package then the implementation of the hashing function will be performed by compiled code. Alternatively if you have either **cryptkit** or **Trf** then either of these is used to accelerate the digest computation. If no suitable compiled package is available then the pure-Tcl implementation is used. The programming interface remains the same in all cases.

Note the previous version of this package always returned a hex encoded string. This has been changed to simplify the programming interface and to make this version more compatible with other implementations. To obtain the previous usage, either explicitly specify package version 1 or use the *-hex* option to the **md5** command.

BEWARE The commands in this package expect binary data as their input. When a **-file** is provided then this is ensured by the commands themselves, as they open the referenced file in binary mode. When literal data, or a **-channel** are provided instead, then the command's *caller is responsible* for ensuring this fact. The necessary conversion command is **encoding convertto utf-8 $string**.

ATTENTION, there is a *Tcl 9 COMPATIBILITY ISSUE* here.

Tcl 8.x silently mishandles non-binary input by cutting it internally to size. I.e. by only using the low byte of a higher unicode point. No error is thrown, the *result is simply wrong*.

Tcl 9 throws an error instead, i.e.

```
expected byte sequence but character <location> was ...
```

Commands

::md5::md5 ?*-hex*? [*-channel channel* | *-file filename* | *string*]

Calculate the MD5 digest of the binary data given in string. This is returned as a binary string by default. Giving the *-hex* option will return a hexadecimal encoded version of the digest.

The data to be hashed can be specified either as a string argument to the **md5** command, or as a filename or a pre-opened channel. If the *-filename* argument is given then the file is opened, the data read and hashed and the file is closed. If the *-channel* argument is given then data is read from the channel until the end of file. The channel is not closed.

Only one of *-file*, *-channel* or *string* should be given.

::md5::hmac ?*-hex*? *-key key* [*-channel channel* | *-file filename* | *string*]

Calculate an Hashed Message Authentication digest (HMAC) using the MD5 digest algorithm. HMACs are described in RFC 2104 (3) and provide an MD5 digest that includes a key. All options other than *-key* are as for the **::md5::md5** command.

Programming interface

For the programmer, the MD5 hash can be viewed as a bucket into which one pours data. When you have finished, you extract a value that is derived from the data that was poured into the bucket. The programming interface to the MD5 hash operates on a token (equivalent to the bucket). You call **MD5Init** to obtain a token and then call **MD5Update** as many times as required to add data to the hash. To release any resources and obtain the hash value, you then call **MD5Final**. An equivalent set of functions gives you a keyed digest (HMAC).

::md5::MD5Init

Begins a new MD5 hash. Returns a token ID that must be used for the remaining functions.

::md5::MD5Update *token data*

Add data to the hash identified by token. Calling *MD5Update $token* "*abcd*" is equivalent to calling *MD5Update $token* "*ab*" followed by *MD5Update $token* "*cb*". See Examples.

::md5::MD5Final *token*

Returns the hash value and releases any resources held by this token. Once this command completes the token will be invalid. The result is a binary string of 16 bytes representing the 128 bit MD5 digest value.

::md5::HMACInit *key*

This is equivalent to the **::md5::MD5Init** command except that it requires the key that will be included in the HMAC.

::md5::HMACUpdate *token data*

see below ...

::md5::HMACFinal *token*

These commands are identical to the MD5 equivalent commands.

Examples

```
% md5::md5 -hex "Tcl does MD5"
8AAC1EE01E20BB347104FABB90310433
```

```
% md5::hmac -hex -key Sekret "Tcl does MD5"
35BBA244FD56D3EDF5F3C47474DACB5D
```

```
% set tok [md5::MD5Init]
::md5::1
% md5::MD5Update $tok "Tcl "
% md5::MD5Update $tok "does "
% md5::MD5Update $tok "MD5"
% md5::Hex [md5::MD5Final $tok]
8AAC1EE01E20BB347104FABB90310433
```

References

1. Rivest, R., "The MD5 Message-Digest Algorithm", RFC 1321, MIT and RSA Data Security, Inc, April 1992. (http://www.rfc-editor.org/rfc/rfc1321.txt)
2. Rivest, R., "The MD4 Message Digest Algorithm", RFC 1320, MIT, April 1992. (http://www.rfc-editor.org/rfc/rfc1320.txt)
3. Krawczyk, H., Bellare, M. and Canetti, R. "HMAC: Keyed-Hashing for Message Authentication", RFC 2104, February 1997. (http://www.rfc-editor.org/rfc/rfc2104.txt)

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *md5* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Hashes, checksums, and encryption

Copyright

```
Copyright (c) 2003, Pat Thoyts <patthoyts@users.sourceforge.net>
```

md5crypt

md5crypt

Keywords: md5crypt, hashing, md5, message-digest, security

Name

md5crypt - MD5-based password encryption

Synopsis

package require Tcl 8.5 9
package require md5 2.0

package require md5crypt ?**1.2.0**?
::md5crypt::md5crypt *password salt*
::md5crypt::aprcrypt *password salt*
::md5crypt::salt ?*length*?

Description

This package provides an implementation of the MD5-crypt password encryption algorithm as pioneered by FreeBSD and currently in use as a replacement for the unix crypt(3) function in many modern systems. An implementation of the closely related Apache MD5-crypt is also available. The output of these commands are compatible with the BSD and OpenSSL implementation of md5crypt and the Apache 2 htpasswd program.

Commands

::md5crypt::md5crypt *password salt*
: Generate a BSD compatible md5-encoded password hash from the plaintext password and a random salt (see SALT).

::md5crypt::aprcrypt *password salt*
: Generate an Apache compatible md5-encoded password hash from the plaintext password and a random salt (see SALT).

::md5crypt::salt ?*length*?
: Generate a random salt string suitable for use with the **md5crypt** and **aprcrypt** commands.

Salt

The salt passed to either of the encryption schemes implemented here is checked to see if it begins with the encryption scheme magic string (either "1" for MD5-crypt or "$apr1$" for Apache crypt). If so, this is removed. The remaining characters up to the next $ and up to a maximum of 8 characters are then used as the salt. The salt text should probably be restricted to the set of ASCII alphanumeric characters plus "./" (dot and forward-slash) - this is to preserve maximum compatibility with the unix password file format.

If a password is being generated rather than checked from a password file then the **salt** command may be used to generate a random salt.

Examples

```
% md5crypt::md5crypt password 01234567
$1$01234567$b5lh2mHyD2PdJjFfALlEz1
```

```
% md5crypt::aprcrypt password 01234567
$apr1$01234567$IXBaQywhAhc0d75ZbaSDp/
```

```
% md5crypt::md5crypt password [md5crypt::salt]
$1$dFmvyRmO$T.V3OmzqeEf3hqJp2WFcb.
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *md5crypt* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Hashes, checksums, and encryption

Copyright
```
Copyright (c) 2003, Pat Thoyts <patthoyts@users.sourceforge.net>
```

mime

mime
Keywords: mime, email, internet, mail, net, rfc 2045, rfc 2046, rfc 2049, rfc 821, rfc 822, smtp

Name
mime - Manipulation of MIME body parts

Synopsis
package require Tcl 8.5 9
package require mime ?**1.7.2**?
::mime::initialize ?*-canonical type/subtype ?-param {key value}...*? ?*-encoding value*? ?*-header {key value}...*? (**-file** *name* | **-string** *value* | **-parts** *{token1 ... tokenN}*)
::mime::finalize *token* ?*-subordinates all* | *dynamic* | *none*?
::mime::getproperty *token* ?*property* | *-names*?
::mime::getheader *token* ?*key* | *-names*?
::mime::setheader *token key value* ?*-mode write* | *append* | *delete*?
::mime::getbody *token* ?**-decode**? ?*-command callback -blocksize octets*?
::mime::copymessage *token channel*
::mime::buildmessage *token*
::mime::parseaddress *string*
::mime::parsedatetime (*string* | **-now**) *property*
::mime::mapencoding *encoding_name*
::mime::reversemapencoding *charset_type*

Description
The **mime** library package provides the commands to create and manipulate MIME body parts.

::mime::initialize ?-canonical *type/subtype* ?-param {*key value*}...? ?-encoding *value*? ?-header {*key value*}...?? (-file *name* | -string *value* | -parts {*token1 ... tokenN*})
Creates a MIME part and returns a token representing it.
- If the **-canonical** option is present, then the body is in canonical (raw) form and is found by consulting either the **-file**, **-string**, or **-parts** option.

  In addition, both the **-param** and **-header** options may occur zero or more times to specify **Content-Type** parameters (e.g., **charset**) and header keyword/values (e.g.,

Content-Disposition), respectively.

Also, **-encoding**, if present, specifies the **Content-Transfer-Encoding** when copying the body.

- If the **-canonical** option is not present, then the MIME part contained in either the **-file** or the **-string** option is parsed, dynamically generating subordinates as appropriate.

::mime::finalize *token* ?-subordinates all | dynamic | none?

Destroys the MIME part represented by *token*. It returns an empty string.

If the **-subordinates** option is present, it specifies which subordinates should also be destroyed. The default value is **dynamic**, destroying all subordinates which were created by **::mime::initialize** together with the containing body part.

::mime::getproperty *token* ?*property* | -names?

Returns a string or a list of strings containing the properties of a MIME part. If the command is invoked with the name of a specific property, then the corresponding value is returned; instead, if **-names** is specified, a list of all properties is returned; otherwise, a serialized array of properties and values is returned.

The possible properties are:

content
: The type/subtype describing the content

encoding
: The "Content-Transfer-Encoding"

params
: A list of "Content-Type" parameters

parts
: A list of tokens for the part's subordinates. This property is present only if the MIME part has subordinates.

size
: The approximate size of the content (unencoded)

::mime::getheader *token* ?*key* | -names?

Returns the header of a MIME part as a dictionary with possibly-redundant keys.

If *key* is provided, then a list of values of matching names, without regard to case, is returned.

If **-names** is provided, a list of all keys is returned.

::mime::setheader *token key value* ?-mode write | append | delete?

If **append** is provided, creates a new header named *key* with the value of *value* is added. If **write** is provided, deletes any existing headers whose names match *key* and then creates a new header named *key* with the value of *value*. If **delete** is provided any existing header having a name that matches *key* is deleted. Returns a list of strings containing the previous value associated with the key.

The value for **-mode** is one of:

write
: The *key*/*value* is either created or overwritten (the default).

append
A new *value* is appended for the *key* (creating it as necessary).

delete
All values associated with the key are removed (the *value* parameter is ignored).

::mime::getbody *token* ?-decode? ?-command *callback* ?-blocksize *octets*??
Returns a string containing the body of the leaf MIME part represented by *token* in canonical form.

If the **-command** option is present, then it is repeatedly invoked with a fragment of the body as this:

```
uplevel #0 $callback [list "data" $fragment]
```

(The **-blocksize** option, if present, specifies the maximum size of each fragment passed to the callback.)

When the end of the body is reached, the callback is invoked as:

```
    uplevel #0 $callback "end"
```

Alternatively, if an error occurs, the callback is invoked as:

```
    uplevel #0 $callback [list "error" reason]
```

Regardless, the return value of the final invocation of the callback is propagated upwards by **::mime::getbody**.

If the **-command** option is absent, then the return value of **::mime::getbody** is a string containing the MIME part's entire body.

If the option **-decode** is absent the return value computed above is returned as is. This means that it will be in the charset specified for the token and not the usual utf-8. If the option **-decode** is present however the command will use the charset information associated with the token to convert the string from its encoding into utf-8 before returning it.

::mime::copymessage *token channel*
Copies the MIME represented by *token* part to the specified *channel*. The command operates synchronously, and uses fileevent to allow asynchronous operations to proceed independently. It returns an empty string.

::mime::buildmessage *token*
Returns the MIME part represented by *token* as a string. It is similar to **::mime::copymessage**, only it returns the data as a return string instead of writing to a channel.

::mime::parseaddress *string*
Takes a string containing one or more 822-style address specifications and returns a list of serialized arrays, one element for each address specified in the argument. If the string contains more than one address they will be separated by commas.

Each serialized array contains the properties below. Note that one or more of these properties may be empty.

address
local@domain
comment
822-style comment
domain
the domain part (rhs)
error
non-empty on a parse error
group
this address begins a group
friendly
user-friendly rendering
local
the local part (lhs)
memberP
this address belongs to a group
phrase
the phrase part
proper
822-style address specification
route
822-style route specification (obsolete)

::mime::parsedatetime (*string* | -now) *property*
Takes a string containing an 822-style date-time specification and returns the specified property as a serialized array.

The list of properties and their ranges are:
hour
0.. 23
lmonth
January, February, ..., December
lweekday
Sunday, Monday, ... Saturday
mday
1.. 31
min
0.. 59
mon
1.. 12
month
Jan, Feb, ..., Dec
proper
822-style date-time specification
rclock

elapsed seconds between then and now

sec
0.. 59

wday
0.. 6 (Sun.. Mon)

weekday
Sun, Mon, ..., Sat

yday
1.. 366

year
1900 ...

zone
-720.. 720 (minutes east of GMT)

::mime::mapencoding *encoding_name*
Maps tcl encodings onto the proper names for their MIME charset type. This is only done for encodings whose charset types were known. The remaining encodings return "" for now.

::mime::reversemapencoding *charset_type*
Maps MIME charset types onto tcl encoding names. Those that are unknown return "".

Known bugs

Tcllib Bug #447037

This problem affects only people which are using Tcl and Mime on a 64-bit system. The currently recommended fix for this problem is to upgrade to Tcl version 8.4. This version has extended 64 bit support and the bug does not appear anymore.

The problem could have been generally solved by requiring the use of Tcl 8.4 for this package. We decided against this solution as it would force a large number of unaffected users to upgrade their Tcl interpreter for no reason.

See *Ticket 447037* [/tktview?name=447037] for additional information.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *mime* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Text processing

Copyright

```
Copyright (c) 1999-2000 Marshall T. Rose
```

smtp

Keywords: smtp, email, internet, mail, mime, net, rfc 2554, rfc 2821, rfc 3207, rfc 821, rfc 822, tls

Name

smtp - Client-side tcl implementation of the smtp protocol

Synopsis

package require Tcl
package require mime ?**1.5.4**?
package require smtp ?**1.5.2**?
::smtp::sendmessage *token option...*

Description

The **smtp** library package provides the client side of the Simple Mail Transfer Protocol (SMTP) (1) (2).

::smtp::sendmessage *token option*...

This command sends the MIME part (see package **mime**) represented by *token* to an SMTP server. *options* is a list of options and their associated values. The recognized options are:

-servers

A list of SMTP servers. The default is **localhost**.

If multiple servers are specified they are tried in sequence. Note that the **-ports** are iterated over in tandem with the servers. If there are not enough ports for the number of servers the default port (see below) is used. If there are more ports than servers the superfluous ports are ignored.

-ports

A list of SMTP ports. The default is **25**.

See option **-servers** above regardig the behaviour for then multiple servers and ports are specified.

-client

The name to use as our hostname when connecting to the server. By default this is either localhost if one of the servers is localhost, or is set to the string returned by **info hostname**.

-queue

Indicates that the SMTP server should be asked to queue the message for later processing. A boolean value.

-atleastone

Indicates that the SMTP server must find at least one recipient acceptable for the message to be sent. A boolean value.

-originator

A string containing an 822-style address specification. If present the header isn't examined

for an originator address.

-recipients
A string containing one or more 822-style address specifications. If present the header isn't examined for recipient addresses). If the string contains more than one address they will be separated by commas.

-header
A list containing two elements, an smtp header and its associated value (the -header option may occur zero or more times).

-usetls
This package supports the RFC 3207 TLS extension (3) by default provided the tls package is available. You can turn this off with this boolean option.

-tlsimport
This boolean flag is **false** by default. When this flag is set the package will import TLS on a sucessfully opened channel. This is needed for connections using native TLS negotiation instead of **STARTTLS**. The **tls** package is automatically required when needed.

-tlspolicy
This option lets you specify a command to be called if an error occurs during TLS setup. The command is called with the SMTP code and diagnostic message appended. The command should return 'secure' or 'insecure' where insecure will cause the package to continue on the unencrypted channel. Returning 'secure' will cause the socket to be closed and the next server in the **-servers** list to be tried.

-username
see below ...

-password
If your SMTP server requires authentication (RFC 2554 (4)) before accepting mail you can use **-username** and **-password** to provide your authentication details to the server. Currently this package supports DIGEST-MD5, CRAM-MD5, LOGIN and PLAIN authentication methods. The most secure method will be tried first and each method tried in turn until we are either authorized or we run out of methods. Note that if the server permits a TLS connection, then the authorization will occur after we begin using the secure channel.

Please also read the section on Authentication, it details the necessary prequisites, i.e. packages needed to support these options and authentication.

If the **-originator** option is not present, the originator address is taken from **From** (or **Resent-From**); similarly, if the **-recipients** option is not present, recipient addresses are taken from **To**, **cc**, and **Bcc** (or **Resent-To**, and so on). Note that the header key/values supplied by the **-header** option (not those present in the MIME part) are consulted. Regardless, header key/values are added to the outgoing message as necessary to ensure that a valid 822-style message is sent.

The command returns a list indicating which recipients were unacceptable to the SMTP server. Each element of the list is another list, containing the address, an SMTP error code, and a textual diagnostic. Depending on the **-atleastone** option and the intended recipients, a non-empty list may still indicate that the message was accepted by the server.

Authentication

Beware. SMTP authentication uses **SASL**. I.e. if the user has to authenticate a connection, i.e. use the options **-user** and **-password** (see above) it is necessary to have the **sasl** package available so that **smtp** can load it.

This is a soft dependency because not everybody requires authentication, and **sasl** depends on a lot of the cryptographic (secure) hashes, i.e. all of **md5**, **otp**, **md4**, **sha1**, and **ripemd160**.

Example

```
proc send_simple_message {recipient email_server subject body} {
    package require smtp
    package require mime

    set token [mime::initialize -canonical text/plain \
    -string $body]
    mime::setheader $token Subject $subject
    smtp::sendmessage $token \
    -recipients $recipient -servers $email_server
    mime::finalize $token
}

send_simple_message someone@somewhere.com localhost \
    "This is the subject." "This is the message."
```

TLS security considerations

This package uses the **TLS** package to handle the security for **https** urls and other socket connections.

Policy decisions like the set of protocols to support and what ciphers to use are not the responsibility of **TLS**, nor of this package itself however. Such decisions are the responsibility of whichever application is using the package, and are likely influenced by the set of servers the application will talk to as well.

For example, in light of the recent POODLE attack discovered by Google many servers will disable support for the SSLv3 protocol. To handle this change the applications using **TLS** must be patched, and not this package, nor **TLS** itself. Such a patch may be as simple as generally activating **tls1** support, as shown in the example below.

```
    package require tls
    tls::init -tls1 1 ;# forcibly activate support for the TLS1
    protocol

    ... your own application code ...
```

References

1. Jonathan B. Postel, "SIMPLE MAIL TRANSFER PROTOCOL", RFC 821, August 1982. (http://www.rfc-editor.org/rfc/rfc821.txt)
2. J. Klensin, "Simple Mail Transfer Protocol", RFC 2821, April 2001. (

http://www.rfc-editor.org/rfc/rfc2821.txt)
3. P. Hoffman, "SMTP Service Extension for Secure SMTP over Transport Layer Security", RFC 3207, February 2002. (http://www.rfc-editor.org/rfc/rfc3207.txt)
4. J. Myers, "SMTP Service Extension for Authentication", RFC 2554, March 1999. (http://www.rfc-editor.org/rfc/rfc2554.txt)

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *smtp* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Networking

Copyright

```
Copyright (c) 1999-2000 Marshall T. Rose and others
```

mkdoc

mkdoc

Keywords: mkdoc

Name

mkdoc - Extracts and optionally converts Markdown comments in source code to HTML

Synopsis

package require Tcl 8.6 9
package require Markdown ?**1.2.2**?
package require yaml ?**0.4.2**?
package require mkdoc ?**0.7.2**?
package require hook
::mkdoc::mkdoc *infile outfile* ?*-css cssfile*?
::mkdoc::run *infile*

Description

The package **mkdoc** provides a command to extract documentation embedded in code and optionally convert these comments into HTML. The latter uses Tcllib's **Markdown** package. Each line of the embedded documentation begins with the special comment marker **#'**.

::mkdoc::mkdoc *infile outfile* ?-css *cssfile*?

The command reads the specified *infile* and extracts the code comments introduced by the **#'** marker. If the *outfile* is either a "*.html*" or "*.htm*" file the Markdown is converted into HTML

using either a default style or the specified style sheet *cssfile*.

All arguments are paths to the files to read from or write to.

The result of the command is the empty string.

See section Formatting for the supported Markdown syntax and extensions to it.

::mkdoc::run *infile*

The command reads the specified *infile*, extracts the embedded documentation, and then executes the contents of the first example, i.e. ```-quoted block, found in the **Example** section.

Here is such an example which will be executed by the Tcl interpreter

```
#' ## <a name="example">Example</a>
#'
#' ```
#' puts "Hello mkdoc package"
#' puts "I am in the example section"
#' ```
```

DANGER, BEWARE. Failing to open the *infile* causes the command to *exit* the entire process.

Use of this command in a general context is not recommended.

Examples

The example below demonstrates the conversion of the documentation embedded into the file "*mkdoc.tcl*" itself:

```
package require mkdoc
# extracting the Markdown
mkdoc::mkdoc mkdoc.tcl mkdoc.md
# converting Markdown to HTML
mkdoc::mkdoc mkdoc.md mkdoc.html
# direct conversion without intermediate file
mkdoc::mkdoc mkdoc.tcl mkdoc.html
```

Formatting

The package supports the syntax supported by Tcllib's **Markdown** package.

It further supports a set of simple YAML headers whose information is inserted into appropriate HTML **meta**-tags. The supported keys are

author

Set the document author. Defaults to **NN**.

title

Set the document title. Defaults to **Documentation filename**].

date

Sets the document date. Defaults to the current day.

css
Sets a custom CSS stylesheet. Defaults to the internal mkdoc sheet.

Note that in Markdown output mode these headers are simply passed through into the result. This is proper, as processors like **pandoc** are able to use them as well.

See the example below for the syntax:

```
#' ---
#' title: mkdoc::mkdoc 0.7.1
#' author: Detlef Groth, Schwielowsee, Germany
#' date: 2022-04-17
#' css: mini.css
#' ---
#'
```

Another extension over standard Markdown is the support of a single level of includes.

See the example below for the syntax:

```
#' #include "path/to/include/file"
```

Note, the double-quotes around the path are part of the syntax.

Beware further that relative paths are resolved relative to the current working directory, and *not* relative to the location of the including file.

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such to the author of this package. Please also report any ideas for enhancements you may have for either package and/or documentation.

Code copyright

BSD License type:

The following terms apply to all files a ssociated with the software unless explicitly disclaimed in individual files.

The authors hereby grant permission to use, copy, modify, distribute, and license this software and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions. No written agreement, license, or royalty fee is required for any of the authorized uses. Modifications to this software may be copyrighted by their authors and need not follow the licensing terms described here, provided that the new terms are clearly indicated on the first page of each file where they apply.

In no event shall the authors or distributors be liable to any party for direct, indirect, special, incidental, or consequential damages arising out of the use of this software, its documentation, or any derivatives thereof, even if the authors have been advised of the possibility of such damage.

The authors and distributors specifically disclaim any warranties, including, but not limited to, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. This software is provided on an "as is" basis, and the authors and distributors have no obligation to provide maintenance, support, updates, enhancements, or modifications.

RESTRICTED RIGHTS: Use, duplication or disclosure by the government is subject to the restrictions as set forth in subparagraph (c) (1) (ii) of the Rights in Technical Data and Computer Software Clause as DFARS 252.227-7013 and FAR 52.227-19.

Category
Text processing

Copyright

```
Copyright (c) 2019-2024, Detlef Groth <detlef(at)dgroth(dot)de>
```


multiplexer

multiplexer
Keywords: multiplexer, chat

Name
multiplexer - One-to-many communication with sockets.

Synopsis
package require Tcl 8.5 9
package require logger
package require multiplexer ?**0.3**?
::multiplexer::create
${multiplexer_instance}::Init *port*
${multiplexer_instance}::Config *key value*
${multiplexer_instance}::AddFilter *cmdprefix*
cmdprefix *data chan clientaddress clientport*
${multiplexer_instance}::AddAccessFilter *cmdprefix*
cmdprefix *chan clientaddress clientport*
${multiplexer_instance}::AddExitFilter *cmdprefix*
cmdprefix *chan clientaddress clientport*

Description
The **multiplexer** package provides a generic system for one-to-many communication utilizing sockets. For example, think of a chat system where one user sends a message which is then broadcast to all the other connected users.

It is possible to have different multiplexers running concurrently.

::multiplexer::create

The **create** command creates a new multiplexer 'instance'. For example:

```
set mp [::multiplexer::create]
```

This instance can then be manipulated like so:

```
${mp}::Init 35100
```

${multiplexer_instance}::Init *port*

This starts the multiplexer listening on the specified port.

${multiplexer_instance}::Config *key value*

Use **Config** to configure the multiplexer instance. Configuration options currently include:

sendtoorigin

A boolean flag. If **true**, the sender will receive a copy of the sent message. Defaults to **false**.

debuglevel

Sets the debug level to use for the multiplexer instance, according to those specified by the **logger** package (debug, info, notice, warn, error, critical).

${multiplexer_instance}::AddFilter *cmdprefix*

Command to add a filter for data that passes through the multiplexer instance. The registered *cmdprefix* is called when data arrives at a multiplexer instance. If there is more than one filter command registered at the instance they will be called in the order of registristation, and each filter will get the result of the preceding filter as its argument. The first filter gets the incoming data as its argument. The result returned by the last filter is the data which will be broadcast to all clients of the multiplexer instance. The command prefix is called as

cmdprefix *data chan clientaddress clientport*

Takes the incoming *data*, modifies it, and returns that as its result. The last three arguments contain information about the client which sent the data to filter: The channel connecting us to the client, its ip-address, and its ip-port.

${multiplexer_instance}::AddAccessFilter *cmdprefix*

Command to add an access filter. The registered *cmdprefix* is called when a new client socket tries to connect to the multixer instance. If there is more than one access filter command registered at the instance they will be called in the order of registristation. If any of the called commands returns **-1** the access to the multiplexer instance is denied and the client channel is closed immediately. Any other result grants the client access to the multiplexer instance. The command prefix is called as

cmdprefix *chan clientaddress clientport*

The arguments contain information about the client which tries to connected to the instance: The channel connecting us to the client, its ip-address, and its ip-port.

${multiplexer_instance}::AddExitFilter *cmdprefix*

Adds filter to be run when client socket generates an EOF condition. The registered *cmdprefix* is called when a client socket of the multixer signals EOF. If there is more than one exit filter command registered at the instance they will be called in the order of registristation. Errors thrown by an exit filter are ignored, but logged. Any result returned by an exit filter is ignored. The command prefix is called as

cmdprefix *chan clientaddress clientport*

The arguments contain information about the client which signaled the EOF: The channel connecting us to the client, its ip-address, and its ip-port.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib

Trackers and specify *multiplexer* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Programming tools

namespacex

namespacex
Keywords: namespacex, extended namespace, info, namespace unknown, namespace utilities, state (de)serialization, unknown hooking, utilities

Name
namespacex - Namespace utility commands

Synopsis
package require Tcl 8.5 9
package require namespacex ?**0.4**?
::namespacex hook add ?*namespace*? *cmdprefix*
::namespacex hook proc ?*namespace*? *arguments body*
::namespacex hook on ?*namespace*? *guardcmdprefix actioncmdprefix*
::namespacex hook next *arg...*
::namespacex import fromns *cmdname* ?newname ...?
::namespacex info allchildren *namespace*
::namespacex info allvars *namespace*
::namespacex normalize *namespace*
::namespacex info vars *namespace* ?*pattern*?
::namespacex state get *namespace*
::namespacex state set *namespace dict*
::namespacex state drop *namespace*
::namespacex strip *prefix namespaces*

Description
This package provides a number of utility commands for working with namespaces. The commands fall into four categories:

1. Hook commands provide and manipulate a chain of commands which replaces the single regular **namespace unknown** handler.
2. An import command provides the ability to import any command from another namespace.
3. Information commands allow querying of variables and child namespaces.
4. State commands provide a means to serialize variable values in a namespace.

Commands
::namespacex hook add ?*namespace*? *cmdprefix*

Adds the *cmdprefix* to the chain of unknown command handlers that are invoked when the *namespace* would otherwise invoke its unknown handler. If *namespace* is not specified, then *cmdprefix* is added to the chain of handlers for the namespace of the caller.

The chain of *cmdprefix* are executed in reverse order of addition, *i.e.* the most recently added *cmdprefix* is executed first. When executed, *cmdprefix* has additional arguments appended to it as would any namespace unknown handler.

::namespacex hook proc ?*namespace*? *arguments body*

Adds an anonymous procedure to the chain of namespace unknown handlers for the *namespace*.

If *namespace* is not specified, then the handler is added to the chain of handlers for the namespace of the caller.

The *arguments* and *body* are specified as for the core **proc** command.

::namespacex hook on ?*namespace*? *guardcmdprefix actioncmdprefix*

Adds a guarded action to the chain of namespace unknown handlers for the *namespace*.

If *namespace* is not specified, then the handler is added to the chain of handlers for the namespace of the caller.

The *guardcmdprefix* is executed first. If it returns a value that can be interpreted as false, then the next unknown hander in the chain is executed. Otherwise, *actioncmdprefix* is executed and the return value of the handler is the value returned by *actioncmdprefix*.

When executed, both *guardcmdprefix* and *actioncmdprefix* have the same additional arguments appended as for any namespace unknown handler.

::namespacex hook next *arg*...

This command is available to namespace hooks to execute the next hook in the chain of handlers for the namespace.

::namespacex import fromns *cmdname ?newname ...?*

Imports the command *cmdname* from the *fromns* namespace into the namespace of the caller. The *cmdname* command is imported even if the *fromns* did not originally export the command.

If *newname* is specified, then the imported command will be known by that name. Otherwise, the command retains is original name as given by *cmdname*.

Additional pairs of *cmdname* / *newname* arguments may also be specified.

::namespacex info allchildren *namespace*

Returns a list containing the names of all child namespaces in the specified *namespace* and its children. The names are all fully qualified.

::namespacex info allvars *namespace*

Returns a list containing the names of all variables in the specified *namespace* and its children. The names are all given relative to *namespace*, and *not* fully qualified.

::namespacex normalize *namespace*
Returns the absolute name of *namespace*, which is resolved relative to the namespace of the caller, with all unneeded colon characters removed.

::namespacex info vars *namespace* ?*pattern*?
Returns a list containing the names of all variables in the specified *namespace*. If the *pattern* argument is specified, then only variables matching *pattern* are returned. Matching is determined using the same rules as for **string match**.

::namespacex state get *namespace*
Returns a dictionary holding the names and values of all variables in the specified *namespace* and its child namespaces.

Note that the names are all relative to *namespace*, and *not* fully qualified.

::namespacex state set *namespace dict*
Takes a dictionary holding the names and values for a set of variables and replaces the current state of the specified *namespace* and its child namespaces with this state. The result of the command is the empty string.

::namespacex state drop *namespace*
Unsets all variables in the specified *namespace* and its child namespaces. The result of the command is the empty string.

::namespacex strip *prefix namespaces*
Each item in *namespaces* must be the absolute normalized name of a child namespace of namespace *prefix*. Returns the corresponding list of relative names of child namespaces.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *namespacex* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Copyright

```
Copyright (c) 200? Neil Madden (http://wiki.tcl.tk/12790)
Copyright (c) 200? Various (http://wiki.tcl.tk/1489)
Copyright (c) 2010 Documentation, Andreas Kupries
```

ncgi

ncgi

Keywords: ncgi, CGI, cookie, form, html

Name

ncgi - Procedures to manipulate CGI values.

Synopsis

package require Tcl 8.5 9
package require ncgi ?**1.4.6**?
::ncgi::cookie *cookie*
::ncgi::decode *str*
::ncgi::empty *name*
::ncgi::exists *name*
::ncgi::encode *string*
::ncgi::header ?*type*? *args*
::ncgi::import *cginame* ?*tclname*?
::ncgi::importAll *args*
::ncgi::importFile *cmd cginame* ?*filename*?
::ncgi::input ?*fakeinput*? ?*fakecookie*?
::ncgi::multipart *type query*
::ncgi::nvlist
::ncgi::names
::ncgi::parse
::ncgi::parseMimeValue *value*
::ncgi::query
::ncgi::redirect *url*
::ncgi::reset *query type*
::ncgi::setCookie *args*
::ncgi::setDefaultValue *key defvalue*
::ncgi::setDefaultValueList *key defvaluelist*
::ncgi::setValue *key value*
::ncgi::setValueList *key valuelist*
::ncgi::type
::ncgi::urlStub ?*url*?
::ncgi::value *key* ?*default*?
::ncgi::valueList *key* ?*default*?

Description

The **ncgi** package provides commands that manipulate CGI values. These are values that come from Web forms and are processed either by CGI scripts or web pages with embedded Tcl code. Use the **ncgi** package to query these values, set and get cookies, and encode and decode www-url-encoded values.

In the simplest case, a CGI script first calls **::ncgi::parse** and then calls **::ncgi::value** to get different form values. If a CGI value is repeated, you should use **::ncgi::valueList** to get back the complete list of values.

An alternative to **::ncgi::parse** is **::ncgi::input**, which has semantics similar to Don Libes' **cgi_input** procedure. **::ncgi::input** restricts repeated CGI values to have names that end with "List". In this case, **::ncgi::value** will return the complete list of values, and **::ncgi::input** will raise errors if it find repeated form elements without the right name.

The **::ncgi::reset** procedure can be used in test suites and Web servers to initialize the source of the CGI values. Otherwise the values are read in from the CGI environment.

The complete set of procedures is described below.

::ncgi::cookie *cookie*
Return a list of values for *cookie*, if any. It is possible that more than one cookie with the same name can be present, so this procedure returns a list.

::ncgi::decode *str*
Decode strings in www-url-encoding, which represents special characters with a %xx sequence, where xx is the character code in hex.

::ncgi::empty *name*
Returns 1 if the CGI variable *name* is not present or has the empty string as its value.

::ncgi::exists *name*
The return value is a boolean. It returns **0** if the CGI variable *name* is not present, and **1** otherwise.

::ncgi::encode *string*
Encode *string* into www-url-encoded format.

::ncgi::header ?*type*? *args*
Output the CGI header to standard output. This emits a Content-Type: header and additional headers based on *args*, which is a list of header names and header values. The *type* defaults to "text/html".

::ncgi::import *cginame* ?*tclname*?
This creates a variable in the current scope with the value of the CGI variable *cginame*. The name of the variable is *tclname*, or *cginame* if *tclname* is empty (default).

::ncgi::importAll *args*
This imports several CGI variables as Tcl variables. If *args* is empty, then every CGI value is imported. Otherwise each CGI variable listed in *args* is imported.

::ncgi::importFile *cmd cginame* ?*filename*?
This provides information about an uploaded file from a form input field of type **file** with name *cginame*. *cmd* can be one of **-server -client**, **-type** or **-data**.

-client *cginame*
returns the filename as sent by the client.

-type *cginame*
returns the mime type of the uploaded file.

-data *cginame*
returns the contents of the file.

-server *cginame filename*
writes the file contents to a local temporary file (or *filename* if supplied) and returns the name of the file. The caller is responsible for deleting this file after use.

::ncgi::input ?*fakeinput*? ?*fakecookie*?
This reads and decodes the CGI values from the environment. It restricts repeated form values to have a trailing "List" in their name. The CGI values are obtained later with the **::ncgi::value** procedure.

::ncgi::multipart *type query*
This procedure parses a multipart/form-data *query*. This is used by **::ncgi::nvlist** and not normally called directly. It returns an alternating list of names and structured values. Each structure value is in turn a list of two elements. The first element is meta-data from the

multipart/form-data structure. The second element is the form value. If you use **::ncgi::value** you just get the form value. If you use **::ncgi::valueList** you get the structured value with meta data and the value.

The *type* is the whole Content-Type, including the parameters like *boundary*. This returns a list of names and values that describe the multipart data. The values are a nested list structure that has some descriptive information first, and the actual form value second. The descriptive information is list of header names and values that describe the content.

::ncgi::nvlist
This returns all the query data as a name, value list. In the case of multipart/form-data, the values are structured as described in **::ncgi::multipart**.

::ncgi::names
This returns all names found in the query data, as a list. **::ncgi::multipart**.

::ncgi::parse
This reads and decodes the CGI values from the environment. The CGI values are obtained later with the **::ncgi::value** procedure. IF a CGI value is repeated, then you should use **::ncgi::valueList** to get the complete list of values.

::ncgi::parseMimeValue *value*
This decodes the Content-Type and other MIME headers that have the form of "primary value; param=val; p2=v2" It returns a list, where the first element is the primary value, and the second element is a list of parameter names and values.

::ncgi::query
This returns the raw query data.

::ncgi::redirect *url*
Generate a response that causes a 302 redirect by the Web server. The *url* is the new URL that is the target of the redirect. The URL will be qualified with the current server and current directory, if necessary, to convert it into a full URL.

::ncgi::reset *query type*
Set the query data and Content-Type for the current CGI session. This is used by the test suite and by Web servers to initialize the ncgi module so it does not try to read standard input or use environment variables to get its data. If neither *query* or *type* are specified, then the **ncgi** module will look in the standard CGI environment for its data.

::ncgi::setCookie *args*
Set a cookie value that will be returned as part of the reply. This must be done before **::ncgi::header** or **::ncgi::redirect** is called in order for the cookie to be returned properly. The *args* are a set of flags and values:
- **-name** *name*
- **-value** *value*
- **-expires** *date*
- **-path** *path restriction*
- **-domain** *domain restriction*

::ncgi::setDefaultValue *key defvalue*
Set a CGI value if it does not already exists. This affects future calls to **::ncgi::value** (but not future calls to **::ncgi::nvlist**). If the CGI value already is present, then this procedure has no

side effects.

::ncgi::setDefaultValueList *key defvaluelist*
Like **::ncgi::setDefaultValue** except that the value already has list structure to represent multiple checkboxes or a multi-selection.

::ncgi::setValue *key value*
Set a CGI value, overriding whatever was present in the CGI environment already. This affects future calls to **::ncgi::value** (but not future calls to **::ncgi::nvlist**).

::ncgi::setValueList *key valuelist*
Like **::ncgi::setValue** except that the value already has list structure to represent multiple checkboxes or a multi-selection.

::ncgi::type
Returns the Content-Type of the current CGI values.

::ncgi::urlStub ?*url*?
Returns the current URL, but without the protocol, server, and port. If *url* is specified, then it defines the URL for the current session. That value will be returned by future calls to **::ncgi::urlStub**

::ncgi::value *key* ?*default*?
Return the CGI value identified by *key*. If the CGI value is not present, then the *default* value is returned instead. This value defaults to the empty string.

If the form value *key* is repeated, then there are two cases: if **::ncgi::parse** was called, then **::ncgi::value** only returns the first value associated with *key*. If **::ncgi::input** was called, then **::ncgi::value** returns a Tcl list value and *key* must end in "List" (e.g., "skuList"). In the case of multipart/form-data, this procedure just returns the value of the form element. If you want the meta-data associated with each form value, then use **::ncgi::valueList**.

::ncgi::valueList *key* ?*default*?
Like **::ncgi::value**, but this always returns a list of values (even if there is only one value). In the case of multipart/form-data, this procedure returns a list of two elements. The first element is meta-data in the form of a parameter, value list. The second element is the form value.

Examples

Uploading a file

```
HTML:
<html>
<form action="/cgi-bin/upload.cgi" method="POST"
    enctype="multipart/form-data">
Path: <input type="file" name="filedata"><br>
Name: <input type="text" name="filedesc"><br>
<input type="submit">
</form>
</html>

TCL: upload.cgi
```

```
#!/usr/local/bin/tclsh

::ncgi::parse
set filedata [::ncgi::value filedata]
set filedesc [::ncgi::value filedesc]

puts "<html> File uploaded at <a
    href=\"/images/$filedesc\">$filedesc</a> </html>"

set filename /www/images/$filedesc

set fh [open $filename w]
puts -nonewline $fh $filedata
close $fh
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *ncgi* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

CGI programming

nettool

nettool

Keywords: nettool, odie

Name

nettool - Tools for networked applications

Synopsis

package require Tcl 8.5 9
package require nettool ?**0.5.4**?
package require twapi 3.1
package require ip 0.2
package require platform 0.2
::cat *filename*
::nettool::allocate_port *startingport*
::nettool::arp_table
::nettool::broadcast_list

::nettool::claim_port *port* ?*protocol*?
::nettool::cpuinfo *args*
::nettool::find_port *startingport*
::nettool::hwid_list
::nettool::ip_list
::nettool::mac_list
::nettool::network_list
::nettool::port_busy *port*
::nettool::release_port *port* ?*protocol*?
::nettool::status
::nettool::user_data_root *appname*

Description

The **nettool** package consists of a Pure-tcl set of tools to perform common network functions that would normally require different packages or calls to exec, in a standard Tcl interface. At present nettool has reference implementations for the following operating systems: Windows, MacOSX, and Linux (debian).

Commands

::cat *filename*
Dump the contents of a file as a result.

::nettool::allocate_port *startingport*
Attempt to allocate *startingport*, or, if busy, advance the port number sequentially until a free port is found, and claim that port. This command uses a built-in database of known ports to avoid returning a port which is in common use. (For example: http (80))

::nettool::arp_table
Dump the contents of this computer's Address Resolution Protocol (ARP) table. The result will be a Tcl formatted list: *macid ipaddrlist* ...

::nettool::broadcast_list
Returns a list of broadcast addresses (suitable for UDP multicast) that this computer is associated with.

::nettool::claim_port *port* ?*protocol*?
Mark *port* as busy, optionally as either **tcp** (default) or **udp**.

::nettool::cpuinfo *args*
If no arguments are given, return a key/value list describing the CPU of the present machine. Included in the matrix is info on the number of cores/processors that are available for parallel tasking, installed physical RAM, and processor family.

The exact contents are platform specific.

For Linux, information is drawn from /proc/cpuinfo and /proc/meminfo.

For MacOSX, information is drawn from sysctl

For Windows, information is draw from TWAPI.

If arguments are given, the result with be a key/value list limited to the fields requested.

Canonical fields for all platforms:

cpus
Count of CPUs/cores/execution units

speed
Clock speed of processor(s) in Mhz

memory
Installed RAM (in MB)

vendor
Manufacturer

::nettool::find_port *startingport*
Return *startingport* if it is available, or the next free port after *startingport*. Note: Unlike **::nettool::allocate_port**, this command does not claim the port.

This command uses a built-in database of known ports to avoid returning a port which is in common use. (For example: http (80))

::nettool::hwid_list
Return a list of hardware specific identifiers from this computer. The source and content will vary by platform.

For MacOSX, the motherboard serial number and macids for all network devices is returned.

For Windows, the volume serial number of C and macids for all network devices is returned.

For Linux, macids for all network devices is returned.

::nettool::ip_list
Return a list of IP addresses associated with this computer.

::nettool::mac_list
Return a list of MACIDs for the network cards attached to this machine. The MACID of the primary network card is returned first.

::nettool::network_list
Return a list of networks associated with this computer. Networks are formated with **ip::nativeToPrefix**.

::nettool::port_busy *port*
Return true if *port* is claimed, false otherwise.

::nettool::release_port *port* ?*protocol*?
Mark *port* as not busy, optionally as either **tcp** (default) or **udp**.

::nettool::status
Return a key/value list describing the status of the computer. The output is designed to be comparable to the output of **top** for all platforms.

Common fields include:

load
Processes per processing unit

memory_total
Total physical RAM (MB)

memory_free
Total physical RAM unused (MB)

::nettool::user_data_root *appname*
Return a fully qualified path to a folder where *appname* should store it's data. The path is not created, only computed, by this command.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *odie* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

System

Copyright


```
Copyright (c) 2015-2018 Sean Woods <yoda@etoyoc.com>
```


nmea

nmea

Keywords: nmea, gps

Name

nmea - Process NMEA data

Synopsis

package require Tcl 8.5 9
package require nmea ?**1.1.0**?
::nmea::input *sentence*
::nmea::open_port *port* ?*speed*?
::nmea::close_port
::nmea::configure_port *settings*
::nmea::open_file *file* ?*rate*?
::nmea::close_file
::nmea::do_line
::nmea::rate
::nmea::log ?*file*?
::nmea::checksum *data*
::nmea::write *sentence data*
::nmea::event *setence* ?*command*?

Description

This package provides a standard interface for writing software which recieves NMEA standard input data. It allows for reading data from COM ports, files, or programmatic input. It also supports the checksumming and logging of incoming data. After parsing, input is dispatched to user defined handler commands for processing. To define a handler, see the **event** command. There are no GPS specific functions in this package. NMEA data consists of a sentence type, followed by a list of data.

Commands

::nmea::input *sentence*

Processes and dispatches the supplied sentence. If *sentence* contains no commas it is treated as a Tcl list, otherwise it must be standard comma delimited NMEA data, with an optional checksum and leading **$**.

```
nmea::input {$GPGSA,A,3,04,05,,09,12,,,24,,,,,2.5,1.3,2.1*39}
nmea::input [list GPGSA A 3 04 05  09 12 "" "" 24 "" "" ""  2.5 1.3
    2.1]
```

::nmea::open_port *port* ?speed?

Open the specified COM port and read NMEA sentences when available. Port speed is set to 4800bps by default or to *speed*.

::nmea::close_port

Close the com port connection if one is open.

::nmea::configure_port *settings*

Changes the current port settings. *settings* has the same format as fconfigure -mode.

::nmea::open_file *file* ?rate?

Open file *file* and read NMEA sentences, one per line, at the rate specified by ?rate? in milliseconds. The file format may omit the leading **$** and/or the checksum. If rate is <= 0 (the default) then lines will only be processed when a call to **do_line** is made.

::nmea::close_file

Close the open file if one exists.

::nmea::do_line

If there is a currently open file, this command will read and process a single line from it. Returns the number of lines read.

::nmea::rate

Sets the rate at which lines are processed from the open file, in milliseconds. The rate remains consistant across files, there does not need to be a file currently open to use this command. Set to 0 to disable automatic line processing.

::nmea::log ?file?

Starts or stops input logging. If a file name is specified then all NMEA data recieved on the open port will be logged to the ?file? in append mode. If file is an empty string then any logging will be stopped. If no file is specified then returns a boolean value indicating if logging is currently enabled. Data written to the port by **write**, data read from files, or input made using **input**, is not logged.

::nmea::checksum *data*

Returns the checksum of the supplied data.

::nmea::write *sentence data*

If there is a currently open port, this command will write the specified sentence and data to the port in proper NMEA checksummed format.

::nmea::event *setence* ?command?

Registers a handler proc for a given NMEA *sentence*. There may be at most one handler per sentence, any existing handler is replaced. If no command is specified, returns the name of the current handler for the given *setence* or an empty string if none exists. In addition to the incoming sentences there are 2 builtin types, EOF and DEFAULT. The handler for the DEFAULT setence is invoked if there is not a specific handler for that sentence. The EOF handler is invoked when End Of File is reached on the open file or port.

The handler procedures, with the exception of the builtin types,must take exactly one argument, which is a list of the data values. The DEFAULT handler should have two arguments, the sentence type and the data values. The EOF handler has no arguments.

```
nmea::event gpgsa parse_sat_detail
nmea::event default handle_unknown

proc parse_sat_detail {data} {
    puts [lindex $data 1]
}

proc handle_unknown {name data} {
    puts "unknown data type $name"
}
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *nmea* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Networking

Copyright

```
Copyright (c) 2006-2009, Aaron Faupell <afaupell@users.sourceforge.net>
```

nns

nns_auto

Keywords: nns_auto, automatic, client, name service, reconnect, restore

Name

nameserv::auto - Name service facility, Client Extension

Synopsis

package require Tcl 8.5 9
package require nameserv::auto ?**0.4**?
package require nameserv

Description

Please read the document *Name service facility, introduction* first.

This package provides the *exact* same API as is provided by package **nameserv**, i.e. the regular name service client. It differs from the former by taking measures to ensure that longer-lived data, i.e. bound names, continuous and unfullfilled async searches, survive the loss of the connection to the name server as much as is possible.

This means that the bound names and continuous and unfullfilled async searches are remembered client-side and automatically re-entered into the server when the connection comes back after its loss. For bound names there is one important limitation to such restoration: It is possible that a name of this client was bound by a different client while the connection was gone. Such names are fully lost, and the best the package can and will do is to inform the user of this.

API

The user-visible API is mainly identical to the API of **nameserv** and is therefore not described here. Please read the documentation of **nameserv**.

The differences are explained below, in the sections Options and Events.

Options

This package supports all the options of package **nameserv**, plus one more. The additional option allows the user to specify the time interval between attempts to restore a lost connection.

-delay *milliseconds*
: The value of this option is an integer value > 0 which specifies the interval to wait between attempts to restore a lost connection, in milliseconds. The default value is **1000**, i.e. one second.

Events

This package generates all of the events of package **nameserv**, plus two more. Both events are generated for the tag *nameserv*.

lost-name
: This event is generated when a bound name is truly lost, i.e. could not be restored after the temporary loss of the connection to the name server. It indicates that a different client took ownership of the name while this client was out of contact.

  The detail information of the event will be a Tcl dictionary containing two keys, **name**, and **data**. Their values hold all the information about the lost name.

re-connection

This event is generated when the connection to the server is restored. The remembered data has been restored when the event is posted.

The event has no detail information.

Design

The package is implemented on top of the regular nameservice client, i.e. package **nameserv**. It detects the loss of the connection by listening for *lost-connection* events, on the tag *nameserv* .

It reacts to such events by starting a periodic timer and trying to reconnect to the server whenver this timer triggers. On success the timer is canceled, a *re-connection* event generated, and the package proceeds to re-enter the remembered bound names and continuous searches.

Another loss of the connection, be it during or after re-entering the remembered information simply restarts the timer and subsequent reconnection attempts.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *nameserv* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Networking

Copyright


```
Copyright (c) 2007-2008 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


nns_client

Keywords: nns_client, client, name service

Name

nameserv - Name service facility, Client

Synopsis

package require Tcl 8.5 9
package require nameserv ?**0.4.3**?
package require comm
package require logger
::nameserv::bind *name data*
::nameserv::release
::nameserv::search ?**-async|-continuous**? ?*pattern*?

::nameserv::protocol
::nameserv::server_protocol
::nameserv::server_features
::nameserv::cget -option
::nameserv::configure
::nameserv::configure -option
::nameserv::configure -option *value...*
$result destroy
$result filled
$result get *name*
$result names
$result size
$result getall ?*pattern*?

Description

Please read *Name service facility, introduction* first.

This package provides a client for the name service facility implemented by the package **nameserv::server**.

This service is built in top of and for the package **comm**. It has nothing to do with the Internet's Domain Name System. If the reader is looking for a package dealing with that please see Tcllib's packages **dns** and **resolv**.

API

The package exports eight commands, as specified below:

::nameserv::bind *name data*

The caller of this command registers the given *name* as its name in the configured name service, and additionally associates a piece of *data* with it. The service does nothing with this information beyond storing it and delivering it as part of search results. The meaning is entirely up to the applications using the name service.

A generally useful choice would for example be an identifier for a communication endpoint managed by the package **comm**. Anybody retrieving the name becomes immediately able to talk to this endpoint, i.e. the registering application.

Of further importance is that a caller can register itself under more than one name, and each name can have its own piece of *data*.

Note that the name service, and thwerefore this command, will throw an error if the chosen name is already registered.

::nameserv::release

Invoking this command releases all names (and their data) registered by all previous calls to **::nameserv::bind** of this client. Note that the name service will run this command implicitly when it loses the connection to this client.

::nameserv::search ?-async|-continuous? ?*pattern*?

This command searches the name service for all registered names matching the specified

glob-*pattern*. If not specified the pattern defaults to *, matching everything. The result of the command is a dictionary mapping the matching names to the data associated with them at *bind*-time.

If either option **-async** or **-continuous** were specified the result of this command changes and becomes the Tcl command of an object holding the actual result. These two options are supported if and only if the service the client is connected to supports the protocol feature *Search/Continuous*.

For **-async** the result object is asynchronously filled with the entries matching the pattern at the time of the search and then not modified any more. The option **-continuous** extends this behaviour by additionally continuously monitoring the service for the addition and removal of entries which match the pattern, and updating the object's contents appropriately.

Note that the caller is responsible for configuring the object with a callback for proper notification when the current result (or further changes) arrive.

For more information about this object see section Asynchronous and continuous searches.

::nameserv::protocol
This command returns the highest version of the name service protocol supported by the package.

::nameserv::server_protocol
This command returns the highest version of the name service protocol supported by the name service the client is currently connected to.

::nameserv::server_features
This command returns a list containing the names of the features of the name service protocol which are supported by the name service the client is currently connected to.

::nameserv::cget -option
This command returns the currently configured value for the specified **-option**. The list of supported options and their meaning can be found in section Options.

::nameserv::configure
In this form the command returns a dictionary of all supported options, and their current values. The list of supported options and their meaning can be found in section Options.

::nameserv::configure -option
In this form the command is an alias for "**::nameserv::cget -option**". The list of supported options and their meaning can be found in section Options.

::nameserv::configure -option *value*...
In this form the command is used to configure one or more of the supported options. At least one option has to be specified, and each option is followed by its new value. The list of supported options and their meaning can be found in section Options.

This form can be used only as long as the client has not contacted the name service yet. After contact has been made reconfiguration is not possible anymore. This means that this form of the command is for the initalization of the client before it use. The command forcing a contact with the name service are
- **bind**

- **release**
- **search**
- **server_protocol**
- **server_features**

Connection handling

The client automatically connects to the service when one of the commands below is run for the first time, or whenever one of the commands is run after the connection was lost, when it was lost.

- **bind**
- **release**
- **search**
- **server_protocol**
- **server_features**

Since version 0.2 of the client it will generate an event when the connection is lost, allowing higher layers to perform additional actions. This is done via the support package **uevent**. This and all other name service related packages hereby reserve the uevent-tag *nameserv*. All their events will be posted to that tag.

Events

This package generates only one event, *lost-connection*. The detail information provided to that event is a Tcl dictionary. The only key contained in the dictionnary is **reason**, and its value will be a string describing why the connection was lost. This string is supplied by the underlying communication package, i.e. **comm**.

Options

The options supported by the client are for the specification of which name service to contact, i.e. of the location of the name service. They are:

-host *name*|*ipaddress*
: This option specifies the host name service to contact is running on, either by *name*, or by *ipaddress*. The initial default is **localhost**, i.e. it is expected to contact a name service running on the same host as the application using this package.

-port *number*
: This option specifies the port the name service to contact is listening on. It has to be a positive integer number (> 0) not greater than 65536 (unsigned short). The initial default is the number returned by the command ::nameserv::common::port, as provided by the package **::nameserv::common**.

Asynchronous and continuous searches

Asynchronous and continuous searches are invoked by using either option **-async** or **-continuous** as argument to the command **::nameserv::search**.

Note that these two options are supported if and only if the service the client is connected to supports the protocol feature *Search/Continuous*. The service provided by the package **::nameserv::server** does this since version 0.3.

For such searches the result of the search command is the Tcl command of an object holding the

actual result. The API provided by these objects is:

Options:

see below ...

-command *command_prefix*

This option has to be set if a user of the result object wishes to get asynchronous notifications when the search result or changes to it arrive.

Note that while it is possible to poll for the arrival of the initial search result via the method **filled**, and for subsequent changes by comparing the output of method **getall** against a saved copy, this is not the recommended behaviour. Setting the **-command** callback and processing the notifications as they arrive is much more efficient.

The *command_prefix* is called with two arguments, the type of change, and the data of the change. The type is either **add** or **remove**, indicating new data, or deleted data, respectively. The data of the change is always a dictionary listing the added/removed names and their associated data.

The first change reported for a search is always the set of matching entries at the time of the search.

Methods:

see below ...

$result destroy

Destroys the object and cancels any continuous monitoring of the service the object may have had active.

$result filled

The result is a boolean value indicating whether the search result has already arrived (**True**), or not (**False**).

$result get *name*

Returns the data associated with the given *name* at *bind*-time.

$result names

Returns a list containing all names known to the object at the time of the invokation.

$result size

Returns an integer value specifying the size of the result at the time of the invokation.

$result getall ?*pattern*?

Returns a dictionary containing the search result at the time of the invokation, mapping the matching names to the data associated with them at *bind*-time.

History

0.3.1

Fixed SF Bug 1954771.

0.3

Extended the client with the ability to perform asynchronous and continuous searches.

0.2

Extended the client with the ability to generate events when it loses its connection to the

name service. Based on package **uevent**.

0.1
Initial implementation of the client.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *nameserv* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Networking

Copyright


```
Copyright (c) 2007-2008 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


nns_common

Keywords: nns_common, client, name service, server

Name

nameserv::common - Name service facility, shared definitions

Synopsis

package require Tcl 8.5 9
package require nameserv::common ?**0.2**?
::nameserv::common::port

Description

Please read *Name service facility, introduction* first.

This package is internal and of no interest to users. It provides the commands of the name service facility which are shared by the client and server implemented by the packages **nameserv::server** and **nameserv** (the client).

This service is built in top of and for the package **comm**. It has nothing to do with the Internet's Domain Name System. If the reader is looking for a package dealing with that please see Tcllib's packages **dns** and **resolv**.

API

The package exports a single command, as specified below:

::nameserv::common::port
The result returned by the command is the id of the default TCP/IP port a nameservice server will listen on, and a name service client will try to connect to.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *nameserv* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Networking

Copyright


```
Copyright (c) 2007-2008 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


nns_intro

Keywords: nns_intro, client, name service, server

Name

nns_intro - Name service facility, introduction

Description

nns (short for *nano nameservice*) is a facility built for the package **comm**, adding a simple name service to it. It is also built on top of **comm**, using it for the exchange of messages between the client and server parts.

This name service facility has nothing to do with the Internet's *Domain Name System*, otherwise known as *DNS*. If the reader is looking for a package dealing with that please see either of the packages **dns** and **resolv**, both found in Tcllib too.

Tcllib provides 2 applications and 4 packages which are working together and provide access to the facility at different levels.

Applications

The application **nnsd** provides a simple name server which can be run by anybody anywhere on their system, as they see fit. It is also an example on the use of the server-side package **nameserv::server**.

Complementing this server is the **nns** client application. A possible, but no very sensible use would be to enter name/port bindings into a server from a shell script. Not sensible, as shell scripts normally do not provide a **comm**-based service.

The only case for this to make some sense would be in a shell script wrapped around a Tcl script FOO which is using comm, to register the listening port used by FOO. However even there it would much more sensible to extend FOO to use the nameservice directly. And in regard on how to that **nns** can be used as both example and template. Beyond that it may also be useful to perform nameservice queries from shell scripts.

The third application, **nnslog** is a stripped down form of the **nns** client application. It is reduced to perform a continuous search for all changes and logs all received events to stdout.

Both clients use the nameserv::auto package to automatically hande the loss and restoration of the connection to the server.

Packages

The two main packages implementing the service are **nameserv** and **nameserv::server**, i.e. client and server. The latter has not much of an API, just enough to start, stop, and configure it. See the application **nnsd** on how to use it.

The basic client, in package **nameserv**, provides the main API to manipulate and query the service. An example of its use is the application **nns**.

The second client package, nameserv::auto is API compatible to the basic client, but provides the additional functionality that it will automatically restore data like bound names when the connection to the name service was lost and then reestablished. I.e. it automatically detects the loss of the server and re-enters the data when the server comes back.

The package **nameserv::common** is of no interest to users. It is an internal package containing code and definitions common to the packages **nameserv** and **nameserv::server**.

All packages use the **uevent** package for the reporting of special circumstances via events, and reserve the uevent-tag *nameserv* for their exclusive use. All their events will be posted to that tag.

Internals

The document *Name service facility, client/server protocol* specifies the protocol used by the packages **nameserv** and **nameserv::server** to talk to each other. It is of no interest to users of either the packages or applications.

Developers wishing to modify and/or extend or to just understand the internals of the nameservice facility however are strongly advised to read it.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *nameserv* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Networking

Copyright


```
Copyright (c) 2008 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

nns_protocol

Keywords: nns_protocol, comm, name service, protocol

Name

nameserv::protocol - Name service facility, client/server protocol

Synopsis

Bind *name data*
Release
Search *pattern*
ProtocolVersion
ProtocolFeatures
Search/Continuous/Start *tag pattern*
Search/Continuous/Stop *tag*
Search/Continuous/Change *tag* **add|remove** *response*

Description

The packages **nameserv::server**, **nameserv**, and **nameserv::common** provide a simple unprotected name service facility for use in small trusted environments.

Please read *Name service facility, introduction* first.

This document contains the specification of the network protocol which is used by client and server to talk to each other, enabling implementations of the same protocol in other languages.

Nano name service protocol version 1

This protocol defines the basic set of messages to be supported by a name service, also called the *Core* feature.

Basic layer

The basic communication between client and server is done using the remote-execution protocol specified by the Tcl package **comm**. The relevant document specifying its on-the-wire protocol can be found in *comm_wire*.

All the scripts exchanged via this protocol are single commands in list form and thus can be interpreted as plain messages instead of as Tcl commands. The commands/messages specified in the next section are the only commands understood by the server-side. Command and variable substitutions are not allowed within the messages, i.e. arguments have to be literal values.

The protocol is synchronous. I.e. for each message sent a response is expected, and has to be generated. All messages are sent by the client. The server does not sent messages, only responses to messages.

Message layer

Bind *name data*

The client sends this message when it registers itself at the service with a *name* and some associated *data*. The server has to send an error response if the *name* is already in use. Otherwise the response has to be an empty string.

The server has to accept multiple names for the same client.

Release
The client sends this message to unregister all names it is known under at the service. The response has to be an empty string, always.

Search *pattern*
The client sends this message to search the service for names matching the glob-*pattern*. The response has to be a dictionary containing the matching names as keys, and mapping them to the data associated with it at **Bind**-time.

ProtocolVersion
The client sends this message to query the service for the highest version of the name service protocol it supports. The response has to be a positive integer number.

Servers supporting only *Nano Name Service Protocol Version 1* have to return **1**.

ProtocolFeatures
The client sends this message to query the service for the features of the name service protocol it supports. The response has to be a list containing feature names.

Servers supporting only *Nano Name Service Protocol Version 1* have to return **{Core}**.

Nano name service protocol extension: Continuous search

This protocol defines an extended set of messages to be supported by a name service, also called the *Search/Continuous* feature. This feature defines additional messages between client and server, and is otherwise identical to version 1 of the protocol. See the last section for the details of our foundation.

A service supporting this feature has to put the feature name **Search/Continuous** into the list of features returned by the message *ProtocolFeatures*.

For this extension the protocol is asynchronous. No direct response is expected for any of the messages in the extension. Furthermore the server will start sending messages on its own, instead of only responses to messages, and the client has to be able to handle these notifications.

Search/Continuous/Start *tag pattern*
The client sends this message to start searching the service for names matching the glob-*pattern*. In contrast to the regular *Search* request this one asks the server to continuously monitor the database for the addition and removal of matching entries and to notify the client of all such changes. The particular search is identified by the *tag*.

No direct response is expected, rather the clients expect to be notified of changes via explicit *Search/Continuous/Result* messages generated by the service.

It is further expected that the *tag* information is passed unchanged to the *Search/Continuous/Result* messages. This tagging of the results enables clients to start multiple searches and distinguish between the different results.

Search/Continuous/Stop *tag*
The client sends this message to stop the continuous search identified by the *tag*.

Search/Continuous/Change *tag* add|remove *response*

This message is sent by the service to clients with active continuous searches to transfer found changes. The first such message for a new continuous search has to contains the current set of matching entries.

To ensure this a service has to generate an **add**-message with an empty *response* if there were no matching entries at the time.

The *response* has to be a dictionary containing the matching names as keys, and mapping them to the data associated with it at **Bind**-time. The argument coming before the response tells the client whether the names in the response were added or removed from the service.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *nameserv* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Networking

Copyright


```
Copyright (c) 2007-2008 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


nns_server

Keywords: nns_server, name service, server

Name

nameserv::server - Name service facility, Server

Synopsis

package require Tcl 8.5 9
package require nameserv::server ?**0.3.3**?
package require comm
package require interp
package require logger
::nameserv::server::start
::nameserv::server::stop
::nameserv::server::active?
::nameserv::server::cget -option
::nameserv::server::configure
::nameserv::server::configure -option
::nameserv::server::configure -option *value...*

Description

Please read *Name service facility, introduction* first.

This package provides an implementation of the serviver side of the name service facility queried by the client provided by the package **nameserv**. All information required by the server will be held in memory. There is no persistent state.

This service is built in top of and for the package **comm**. It has nothing to do with the Internet's Domain Name System. If the reader is looking for a package dealing with that please see Tcllib's packages **dns** and **resolv**.

This server supports the *Core* protocol feature, and since version 0.3 the *Search/Continuous* feature as well.

API

The package exports five commands, as specified below:

::nameserv::server::start
: This command starts the server and causes it to listen on the configured port. From now on clients are able to connect and make requests. The result of the command is the empty string.

  Note that any incoming requests will only be handled if the application the server is part of does enter an event loop after this command has been run.

::nameserv::server::stop
: Invoking this command stops the server and releases all information it had. Existing connections are shut down, and no new connections will be accepted any longer. The result of the command is the empty string.

::nameserv::server::active?
: This command returns a boolean value indicating the state of the server. The result will be **true** if the server is active, i.e. has been started, and **false** otherwise.

::nameserv::server::cget -option
: This command returns the currently configured value for the specified **-option**. The list of supported options and their meaning can be found in section Options.

::nameserv::server::configure
: In this form the command returns a dictionary of all supported options, and their current values. The list of supported options and their meaning can be found in section Options.

::nameserv::server::configure -option
: In this form the command is an alias for "**::nameserv::server::cget -option**". The list of supported options and their meaning can be found in section Options.

::nameserv::server::configure -option *value*...
: In this form the command is used to configure one or more of the supported options. At least one option has to be specified, and each option is followed by its new value. The list of supported options and their meaning can be found in section Options.

  This form can be used only if the server is not active, i.e. has not been started yet, or has been stopped. While the server is active it cannot be reconfigured.

Options

The options supported by the server are for the specification of the TCP port to listen on, and whether to accept non-local connections or not. They are:

-localonly *bool*
: This option specifies whether to accept only local connections (-localonly 1) or remote connections as well (-localonly 0). The default is to accept only local connections.

-port *number*
: This option specifies the port the name service will listen on after it has been started. It has to be a positive integer number (> 0) not greater than 65536 (unsigned short). The initial default is the number returned by the command **::nameserv::server::common::port**, as provided by the package **::nameserv::server::common**.

History

0.3
: Extended the server with the ability to perform asynchronous and continuous searches.

0.2
: Changed name of -local switch to -localonly.

0.1
: Initial implementation of the server.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *nameserv* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Networking

Copyright


```
Copyright (c) 2007-2008 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


nntp

nntp

Keywords: nntp, news, nntpclient, rfc 1036, rfc 977

Name

nntp - Tcl client for the NNTP protocol

Synopsis

package require Tcl 8.5 9
package require nntp ?**0.2.3**?
::nntp::nntp ?*host*? ?*port*? ?*nntpName*?
nntpName **method** ?*arg arg* ...?
nntpName **article** ?*msgid*?
nntpName **authinfo** ?*user*? ?*pass*?
nntpName **body** ?*msgid*?
nntpName **configure**
nntpName **configure** *option*
nntpName **configure** *option value* ...
nntpName **cget** *option*
nntpName **date**
nntpName **group** ?*group*?
nntpName **head** ?*msgid*?
nntpName **help**
nntpName **last**
nntpName **list**
nntpName **listgroup** ?*group*?
nntpName **mode_reader**
nntpName **newgroups** *since*
nntpName **newnews**
nntpName **newnews** *since*
nntpName **newnews** *group* ?*since*?
nntpName **next**
nntpName **post** *article*
nntpName **slave**
nntpName **stat** ?*msgid*?
nntpName **quit**
nntpName **xgtitle** ?*group_pattern*?
nntpName **xhdr** *field* ?*range*?
nntpName **xover** ?*range*?
nntpName **xpat** *field range* ?*pattern_list*?

Description

The package **nntp** provides a simple Tcl-only client library for the NNTP protocol. It works by opening the standard NNTP socket on the server, and then providing a Tcl API to access the NNTP protocol commands. All server errors are returned as Tcl errors (thrown) which must be caught with the Tcl **catch** command.

Commands

::nntp::nntp ?*host*? ?*port*? ?*nntpName*?

The command opens a socket connection to the specified NNTP server and creates a new nntp object with an associated global Tcl command whose name is *nntpName*. This command may be used to access the various NNTP protocol commands for the new connection. The default *port* number is "119" and the default *host* is "news". These defaults can be overridden

with the environment variables **NNTPPORT** and **NNTPHOST** respectively.

Some of the commands supported by this package are not part of the nntp rfc 977 (http://www.rfc-editor.org/rfc/rfc977.txt) and will not be available (or implemented) on all nntp servers.

The access command *nntpName* has the following general form:
***nntpName* method ?*arg arg ...*?**
Option and the *arg*s determine the exact behavior of the command.

***nntpName* article ?*msgid*?**
Query the server for article *msgid* from the current group. The article is returned as a valid tcl list which contains the headers, followed by a blank line, and then followed by the body of the article. Each element in the list is one line of the article.

***nntpName* authinfo ?*user*? ?*pass*?**
Send authentication information (username and password) to the server.

***nntpName* body ?*msgid*?**
Query the server for the body of the article *msgid* from the current group. The body of the article is returned as a valid tcl list. Each element in the list is one line of the body of the article.

***nntpName* configure**
see below ...

nntpName* configure *option
see below ...

nntpName* configure *option value ...
see below ...

nntpName* cget *option
Query and configure options of the nntp connection object. Currently only one option is supported, **-binary**. When set articles are retrieved as binary data instead of text. The only methods affected by this are **article** and **body**.

One application of this option would be the download of articles containing yEnc encoded images. Although encoded the data is still mostly binary and retrieving it as text will corrupt the information.

See package **yencode** for both encoder and decoder of such data.

***nntpName* date**
Query the server for the servers current date. The date is returned in the format *YYYYMMDDHHMMSS*.

***nntpName* group ?*group*?**
Optionally set the current group, and retrieve information about the currently selected group. Returns the estimated number of articles in the group followed by the number of the first article in the group, followed by the last article in the group, followed by the name of the group.

***nntpName* head ?*msgid*?**
Query the server for the headers of the article *msgid* from the current group. The headers of

the article are returned as a valid tcl list. Each element in the list is one line of the headers of the article.

***nntpName* help**
Retrieves a list of the commands that are supported by the news server that is currently attached to.

***nntpName* last**
Sets the current article pointer to point to the previous message (if there is one) and returns the msgid of that message.

***nntpName* list**
Returns a tcl list of valid newsgroups and associated information. Each newsgroup is returned as an element in the tcl list with the following format:

```
group last first p
```

where is the name of the newsgroup, is the number of the last known article currently in that newsgroup, is the number of the first article currently in the newsgroup, and is either 'y' or 'n' indicating whether posting to this newsgroup is allowed ('y') or prohibited ('n').

The and fields will always be numeric. They may have leading zeros. If the field evaluates to less than the field, there are no articles currently on file in the newsgroup.

***nntpName* listgroup ?*group*?**
Query the server for a list of all the messages (message numbers) in the group specified by the argument *group* or by the current group if the *group* argument was not passed.

***nntpName* mode_reader**
Query the server for its nntp 'MODE READER' response string.

nntpName* newgroups *since
Query the server for a list of all the new newsgroups created since the time specified by the argument *since*. The argument *since* can be any time string that is understood by **clock scan** . The tcl list of newsgroups is returned in a similar form to the list of groups returned by the **nntpName list** command. Each element of the list has the form:

```
group last first p
```

where is the name of the newsgroup, is the number of the last known article currently in that newsgroup, is the number of the first article currently in the newsgroup, and is either 'y' or 'n' indicating whether posting to this newsgroup is allowed ('y') or prohibited ('n').

***nntpName* newnews**
Query the server for a list of new articles posted to the current group in the last day.

nntpName* newnews *since
Query the server for a list of new articles posted to the current group since the time specified by the argument *since*. The argument *since* can be any time string that is understood by **clock scan**.

***nntpName* newnews *group* ?*since*?**
Query the server for a list of new articles posted to the group specified by the argument *group* since the time specified by the argument *since* (or in the past day if no *since* argument is passed. The argument *since* can be any time string that is understood by **clock scan**.

***nntpName* next**
Sets the current article pointer to point to the next message (if there is one) and returns the msgid of that message.

nntpName* post *article
Posts an article of the form specified in RFC 1036 (http://www.rfc-editor.org/rfc/rfc1036.txt, successor to RFC 850) to the current news group.

***nntpName* slave**
Identifies a connection as being made from a slave nntp server. This might be used to indicate that the connection is serving multiple people and should be given priority. Actual use is entirely implementation dependent and may vary from server to server.

***nntpName* stat ?*msgid*?**
The stat command is similar to the article command except that no text is returned. When selecting by message number within a group, the stat command serves to set the current article pointer without sending text. The returned acknowledgment response will contain the message-id, which may be of some value. Using the stat command to select by message-id is valid but of questionable value, since a selection by message-id does NOT alter the "current article pointer"

***nntpName* quit**
Gracefully close the connection after sending a NNTP QUIT command down the socket.

***nntpName* xgtitle ?*group_pattern*?**
Returns a tcl list where each element is of the form:

```
newsgroup description
```

If a *group_pattern* is specified then only newsgroups that match the pattern will have their name and description returned.

***nntpName* xhdr *field* ?*range*?**
Returns the specified header field value for the current message or for a list of messages from the current group. *field* is the title of a field in the header such as from, subject, date, etc. If *range* is not specified or is "" then the current message is queried. The command returns a list of elements where each element has the form of:

```
    msgid value
```

Where msgid is the number of the message and value is the value set for the queried field. The *range* argument can be in any of the following forms:

""
The current message is queried.

msgid1-msgid2
All messages between *msgid1* and *msgid2* (including *msgid1* and *msgid2*) are queried.

msgid1 msgid2
All messages between *msgid1* and *msgid2* (including *msgid1* and *msgid2*) are queried.

***nntpName* xover ?*range*?**
Returns header information for the current message or for a range of messages from the current group. The information is returned in a tcl list where each element is of the form:

```
    msgid subject from date idstring bodysize headersize xref
```

If *range* is not specified or is "" then the current message is queried. The *range* argument can be in any of the following forms:

""

The current message is queried.

msgid1-msgid2

All messages between *msgid1* and *msgid2* (including *msgid1* and *msgid2*) are queried.

msgid1 msgid2

All messages between *msgid1* and *msgid2* (including *msgid1* and *msgid2*) are queried.

***nntpName* xpat *field range* ?*pattern_list*?**

Returns the specified header field value for a specified message or for a list of messages from the current group where the messages match the pattern(s) given in the pattern_list. *field* is the title of a field in the header such as from, subject, date, etc. The information is returned in a tcl list where each element is of the form:

```
    msgid value
```

Where msgid is the number of the message and value is the value set for the queried field. The *range* argument can be in any of the following forms:

msgid

The message specified by *msgid* is queried.

msgid1-msgid2

All messages between *msgid1* and *msgid2* (including *msgid1* and *msgid2*) are queried.

msgid1 msgid2

All messages between *msgid1* and *msgid2* (including *msgid1* and *msgid2*) are queried.

Example

A bigger example for posting a single article.

```
    package require nntp
    set n [nntp::nntp NNTP_SERVER]
    $n post "From: USER@DOMAIN.EXT (USER_FULL)
    Path: COMPUTERNAME!USERNAME
    Newsgroups: alt.test
    Subject: Tcl test post -ignore
    Message-ID: <[pid][clock seconds]
    @COMPUTERNAME>
    Date: [clock format [clock seconds] -format "%a, %d %
    b %y %H:%M:%S GMT" -gmt true]

    Test message body"
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you

want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *nntp* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Networking

ntp

ntp_time
Keywords: ntp_time, NTP, SNTP, rfc 2030, rfc 868, time

Name
ntp_time - Tcl Time Service Client

Synopsis
package require Tcl 8.5 9
package require time ?**1.2.2**?
::time::gettime ?*options*? *timeserver* ?*port*?
::time::getsntp ?*options*? *timeserver* ?*port*?
::time::configure ?*options*?
::time::cget *name*
::time::unixtime *token*
::time::status *token*
::time::error *token*
::time::reset *token* ?*reason*?
::time::wait *token*
::time::cleanup *token*

Description
This package implements a client for the RFC 868 TIME protocol (http://www.rfc-editor.org/rfc/rfc868.txt) and also a minimal client for the RFC 2030 Simple Network Time Protocol (http://www.rfc-editor.org/rfc/rfc2030.txt). RFC 868 returns the time in seconds since 1 January 1900 to either tcp or udp clients. RFC 2030 also gives this time but also provides a fractional part which is not used in this client.

Commands
::time::gettime ?*options*? *timeserver* ?*port*?
Get the time from *timeserver*. You may specify any of the options listed for the **configure** command here. This command returns a token which must then be used with the remaining commands in this package. Once you have finished, you should use **cleanup** to release all resources. The default port is **37**.

::time::getsntp ?*options*? *timeserver* ?*port*?

Get the time from an SNTP server. This accepts exactly the same arguments as **::time::gettime** except that the default port is **123**. The result is a token as per **::time::gettime** and should be handled in the same way.

Note that it is unlikely that any SNTP server will reply using tcp so you will require the **tcludp** or the **ceptcl** package. If a suitable package can be loaded then the udp protocol will be used by default.

::time::configure ?*options*?

Called with no arguments this command returns all the current configuration options and values. Otherwise it should be called with pairs of option name and value.

-protocol *number*

Set the default network protocol. This defaults to udp if the tcludp package is available. Otherwise it will use tcp.

-port *number*

Set the default port to use. RFC 868 uses port **37**, RFC 2030 uses port **123**.

-timeout *number*

Set the default timeout value in milliseconds. The default is 10 seconds.

-command *number*

Set a command procedure to be run when a reply is received. The procedure is called with the time token appended to the argument list.

-loglevel *number*

Set the logging level. The default is 'warning'.

::time::cget *name*

Get the current value for the named configuration option.

::time::unixtime *token*

Format the returned time for the unix epoch. RFC 868 time defines time 0 as 1 Jan 1900, while unix time defines time 0 as 1 Jan 1970. This command converts the reply to unix time.

::time::status *token*

Returns the status flag. For a successfully completed query this will be *ok*. May be *error* or *timeout* or *eof*. See also **::time::error**

::time::error *token*

Returns the error message provided for requests whose status is *error*. If there is no error message then an empty string is returned.

::time::reset *token?reason?*

Reset or cancel the query optionally specfying the reason to record for the **error** command.

::time::wait *token*

Wait for a query to complete and return the status upon completion.

::time::cleanup *token*

Remove all state variables associated with the request.

```
% set tok [::time::gettime ntp2a.mcc.ac.uk]
% set t [::time::unixtime $tok]
% ::time::cleanup $tok
```

```
% set tok [::time::getsntp pool.ntp.org]
% set t [::time::unixtime $tok]
% ::time::cleanup $tok
```

```
proc on_time {token} {
   if {[time::status $token] eq "ok"} {
      puts [clock format [time::unixtime $token]]
   } else {
      puts [time::error $token]
   }
   time::cleanup $token
}
time::getsntp -command on_time pool.ntp.org
```

Authors
Pat Thoyts

Bugs, ideas, feedback
If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *ntp* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Networking

Copyright

```
Copyright (c) 2002, Pat Thoyts <patthoyts@users.sourceforge.net>
```


oauth

oauth
Keywords: oauth, RFC 2718, RFC 5849, twitter

Name
oauth - oauth API base signature

Synopsis
package require Tcl 8.5 9
package require oauth ?**1.0.4**?
::oauth::config
::oauth::config ?*options...*?

::oauth::header *baseURL ?postQuery?*
::oauth::query *baseURL ?postQuery?*

Description

The **oauth** package provides a simple Tcl-only library for communication with oauth APIs. This current version of the package supports the Oauth 1.0 Protocol, as specified in RFC 5849.

TLS security considerations

This package uses the **TLS** package to handle the security for **https** urls and other socket connections.

Policy decisions like the set of protocols to support and what ciphers to use are not the responsibility of **TLS**, nor of this package itself however. Such decisions are the responsibility of whichever application is using the package, and are likely influenced by the set of servers the application will talk to as well.

For example, in light of the recent POODLE attack discovered by Google many servers will disable support for the SSLv3 protocol. To handle this change the applications using **TLS** must be patched, and not this package, nor **TLS** itself. Such a patch may be as simple as generally activating **tls1** support, as shown in the example below.

```
    package require tls
    tls::init -tls1 1 ;# forcibly activate support for the TLS1
    protocol

    ... your own application code ...
```

Commands

::oauth::config
When this command is invoked without arguments it returns a dictionary containing the current values of all options.

::oauth::config ?*options*...?
When invoked with arguments, options followed by their values, it is used to set and query various parameters of application and client, like proxy host and user agent for the HTTP requests. The detailed list of options is below:

-accesstoken *string*
This is the user's token.

-accesstokensecret *string*
This is the user's secret token.

-consumerkey *string*
This is the public token of your app.

-consumersecret *string*
This is the private token of your app.

-debug *bool*
The default value is **off**. If you change this option to **on**, the basic signature just created will be printed to stdout, among other debug output.

-oauthversion *version*

This is the version of the OAuth protocol to use. At the moment only **1.0** is supported, the default.

-proxyhost *hostname*
You can set up a proxy host for send contact the oauth's api server.

-proxyport *port-number*
Port number of your proxy.

-signmethod *method*
The signature method to use. OAuth 1.0 only supports **HMAC-SHA1**, the default.

-timeout *milliseconds*
Timeout in milliseconds for your query. The default value is **6000**, i.e. 6 seconds.

-urlencoding *encoding*
The encoding used for creating the x-url-encoded URLs with **::http::formatQuery**. The default is **utf-8**, as specified by RFC 2718.

::oauth::header *baseURL* ?*postQuery*?
This command is the base signature creator. With proper settings for various tokens and secrets (See **::oauth::config**) the result is the base authentication string to send to the server.

You do not need to call this procedure to create the query because **::oauth::query** (see below) will do for it for you. Doing so is useful for debugging purposes, though.

url *baseURL*
This argument is the URI path to the OAuth API server. If you plan send a GET query, you should provide a full path.

```
HTTP GET
::oauth::header {https://api.twitter.com/1.1/users/lookup.json?scre
    en_name=AbiertaMente}
```

url-encoded-string *postQuery*
When you have to send a header in POST format, you have to put the query string into this argument.

```
::oauth::header {https://api.twitter.com/1.1/friendships/create.jso
    n} {user_id=158812437&follow=true}
```

::oauth::query *baseURL* ?*postQuery*?
This procedure will use the settings made with **::oauth::config** to create the basic authentication and then send the command to the server API. It takes the same arguments as **::oauth::header**.

The returned result will be a list containing 2 elements. The first element will be a dictionary containing the HTTP header data response. This allows you, for example, to check the X-Rate-Limit from OAuth. The second element will be the raw data returned from API server. This string is usually a json object which can be further decoded with the functions of package **json**, or any other json-parser for Tcl.

Here is an example of how it would work in Twitter. Do not forget to replace the placeholder

tokens and keys of the example with your own tokens and keys when trying it out.

```
% package require oauth
% package require json
% oauth::config -consumerkey {your_consumer_key} -consumersecret
    {your_consumer_key_secret} -accesstoken {your_access_token}
    -accesstokensecret {your_access_token_secret}

% set response [oauth::query
    https://api.twitter.com/1.1/users/lookup.json?screen_name=Abierta
    Mente]
% set jsondata [lindex $response 1]
% set data [json::json2dict $jsondata]
$ set data [lindex $data 0]
% dict for {key val} $data {puts "$key => $val"}
id => 158812437
id_str => 158812437
name => Un Librepensador
screen_name => AbiertaMente
location => Explico mis tuits ahí →
description => 160Caracteres para un SMS y contaba mi vida entera
    sin recortar vocales. Ahora en Twitter, podemos usar hasta 140 y
    a mí me sobrarían 20 para contaros todo lo q
url => http://t.co/SGs3k9odBn
entities => url {urls {{url http://t.co/SGs3k9odBn expanded_url
    http://librepensamiento.es display_url librepensamiento.es
    indices {0 22}}}} description {urls {}}
protected => false
followers_count => 72705
friends_count => 53099
listed_count => 258
created_at => Wed Jun 23 18:29:58 +0000 2010
favourites_count => 297
utc_offset => 7200
time_zone => Madrid
geo_enabled => false
verified => false
statuses_count => 8996
lang => es
status => created_at {Sun Oct 12 08:02:38 +0000 2014} id
    521209314087018496 id_str 521209314087018496 text
    {@thesamethanhim http://t.co/WFoXOAofCt} source {<a
    href="http://twitter.com" rel="nofollow">Twitter Web Client</a>}
    truncated false in_reply_to_status_id 521076457490350081
    in_reply_to_status_id_str 521076457490350081 in_reply_to_user_id
    2282730867 in_reply_to_user_id_str 2282730867
```

```
    in_reply_to_screen_name thesamethanhim geo null coordinates null
    place null contributors null retweet_count 0 favorite_count 0
    entities {hashtags {} symbols {} urls {{url
    http://t.co/WFoXOAofCt expanded_url
    http://www.elmundo.es/internacional/2014/03/05/53173dc1268e3e3f23
    8b458a.html display_url elmundo.es/internacional/… indices {16
    38}}} user_mentions {{screen_name thesamethanhim name Ἑλένη id
    2282730867 id_str 2282730867 indices {0 15}}}} favorited false
    retweeted false possibly_sensitive false lang und
contributors_enabled => false
is_translator => true
is_translation_enabled => false
profile_background_color => 709397
profile_background_image_url =>
    http://pbs.twimg.com/profile_background_images/704065051/9309c02a
    a2728bdf543505ddbd408e2e.jpeg
profile_background_image_url_https =>
    https://pbs.twimg.com/profile_background_images/704065051/9309c02
    aa2728bdf543505ddbd408e2e.jpeg
profile_background_tile => true
profile_image_url =>
    http://pbs.twimg.com/profile_images/2629816665/8035fb81919b840c5c
    c149755d3d7b0b_normal.jpeg
profile_image_url_https =>
    https://pbs.twimg.com/profile_images/2629816665/8035fb81919b840c5
    cc149755d3d7b0b_normal.jpeg
profile_banner_url =>
    https://pbs.twimg.com/profile_banners/158812437/1400828874
profile_link_color => FF3300
profile_sidebar_border_color => FFFFFF
profile_sidebar_fill_color => A0C5C7
profile_text_color => 333333
profile_use_background_image => true
default_profile => false
default_profile_image => false
following => true
follow_request_sent => false
notifications => false
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *oauth* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be

made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Networking

Copyright
```
Copyright (c) 2014 Javi P. <hxm@eggdrop.es>
```

oometa

oometa
Keywords: oometa, TOOL, TclOO

Name
oometa - oo::meta A data registry for classess

Synopsis
oo::meta::info
oo::meta::info branchget ?*key*? ?*...*?
oo::meta::info branchset ?*key...*? *key value*
oo::meta::info dump *class*
oo::meta::info *class* **is** *type* ?*args*?
oo::meta::info *class* **merge** ?*dict*? ?*dict*? ?*...*?
oo::meta::info *class* **rebuild**
oo::meta::metadata *class*
oo::define meta
oo::class method meta
oo::object method meta
oo::object method meta cget ?*field*? ?*...*? *field*

Description
The **oo::meta** package provides a data registry service for TclOO classes.

Usage
```
oo::class create animal {
  meta set biodata animal: 1
}
oo::class create mammal {
  superclass animal
  meta set biodata mammal: 1
}
oo::class create cat {
  superclass mammal
  meta set biodata diet: carnivore
}
```

```
cat create felix
puts [felix meta dump biodata]
> animal: 1 mammal: 1 diet: carnivore

felix meta set biodata likes: {birds mice}
puts [felix meta get biodata]
> animal: 1 mammal: 1 diet: carnivore likes: {bird mice}

# Modify a class
mammal meta set biodata metabolism: warm-blooded
puts [felix meta get biodata]
> animal: 1 mammal: 1 metabolism: warm-blooded diet: carnivore likes:
    {birds mice}

# Overwrite class info
felix meta set biodata mammal: yes
puts [felix meta get biodata]
> animal: 1 mammal: yes metabolism: warm-blooded diet: carnivore
    likes: {birds mice}
```

Concept

The concept behind **oo::meta** is that each class contributes a snippet of *local* data. When **oo::meta::metadata** is called, the system walks through the linear ancestry produced by **oo::meta::ancestors**, and recursively combines all of that local data for all of a class' ancestors into a single dict. Instances of oo::object can also combine class data with a local dict stored in the *meta* variable.

Commands

oo::meta::info

oo::meta::info is intended to work on the metadata of a class in a manner similar to if the aggregate pieces where assembled into a single dict. The system mimics all of the standard dict commands, and addes the following:

oo::meta::info branchget ?*key*? ?...?

Returns a dict representation of the element at *args*, but with any trailing: removed from field names.

```
::oo::meta::info $myclass set option color {default: green widget:
    colorselect}
puts [::oo::meta::info $myclass get option color]
> {default: green widget: color}
puts [::oo::meta::info $myclass branchget option color]
> {default green widget color}
```

oo::meta::info branchset ?*key...*? *key value*

Merges *dict* with any other information contaned at node ?*key...*?, and adding a trailing: to all

field names.

```
::oo::meta::info $myclass branchset option color {default green
    widget colorselect}
puts [::oo::meta::info $myclass get option color]
> {default: green widget: color}
```

oo::meta::info dump *class*
Returns the complete snapshot of a class metadata, as producted by **oo::meta::metadata**

oo::meta::info *class* is *type* ?*args*?
Returns a boolean true or false if the element ?*args*? would match **string is** *type value*

```
::oo::meta::info $myclass set constant mammal 1
puts [::oo::meta::info $myclass is true constant mammal]
> 1
```

oo::meta::info *class* merge ?*dict*? ?*dict*? ?*...*?
Combines all of the arguments into a single dict, which is then stored as the new local representation for this class.

oo::meta::info *class* rebuild
Forces the meta system to destroy any cached representation of a class' metadata before the next access to **oo::meta::metadata**

oo::meta::metadata *class*
Returns an aggregate picture of the metadata for *class*, combining its *local* data with the *local* data from its ancestors.

oo::define meta
The package injects a command **oo::define::meta** which works to provide a class in the process of definition access to **oo::meta::info**, but without having to look the name up.

```
oo::define myclass {
  meta set foo bar: baz
}
```

oo::class method meta
The package injects a new method **meta** into **oo::class** which works to provide a class instance access to **oo::meta::info**.

oo::object method meta
The package injects a new method **meta** into **oo::object**. **oo::object** combines the data for its class (as provided by **oo::meta::metadata**), with a local variable *meta* to produce a local picture of metadata. This method provides the following additional commands:

oo::object method meta cget ?*field*? ?*...*? *field*
Attempts to locate a singlar leaf, and return its value. For single option lookups, this is faster than **my meta getnull** ?*field*? ?*...*? *field*], because it performs a search instead directly instead of producing the recursive merge product between the class metadata, the local *meta* variable, and THEN performing the search.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *tcloo* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
TclOO

Copyright

```
Copyright (c) 2015 Sean Woods <yoda@etoyoc.com>
```


ooutil

ooutil
Keywords: ooutil, TclOO, callback, class methods, class variables, command prefix, currying, method reference, my method, singleton

Name
oo::util - Utility commands for TclOO

Synopsis
package require Tcl 8.5 9
package require TclOO
package require oo::util ?**1.2.3**?
mymethod *method* ?*arg...*?
classmethod *name arguments body*
classvariable ?*arg...*?
link *method...*
link {alias method}...
ooutil::singleton ?*arg...*?

Description
This package provides a convenience command for the easy specification of instance methods as callback commands, like timers, file events, Tk bindings, etc.

Commands
mymethod *method* ?*arg...*?
This command is available within instance methods. It takes a method name and, possibly, arguments for the method and returns a command prefix which, when executed, will invoke the named method of the object we are in, with the provided arguments, and any others supplied at the time of actual invokation.

Note: The command is equivalent to and named after the command provided by the OO

package **snit** for the same purpose.

classmethod *name arguments body*

This command is available within class definitions. It takes a method name and, possibly, arguments for the method and creates a method on the class, available to a user of the class and of derived classes.

Note: The command is equivalent to the command **typemethod** provided by the OO package **snit** for the same purpose.

Example

```
oo::class create ActiveRecord {
    classmethod find args { puts "[self] called with arguments:
    $args" }
}
oo::class create Table {
    superclass ActiveRecord
}
puts [Table find foo bar]
# ======
# which will write
# ======
# ::Table called with arguments: foo bar
```

classvariable ?*arg*...?

This command is available within instance methods. It takes a series of variable names and makes them available in the method's scope. The originating scope for the variables is the class (instance) the object instance belongs to. In other words, the referenced variables are shared between all instances of their class.

Note: The command is roughly equivalent to the command **typevariable** provided by the OO package **snit** for the same purpose. The difference is that it cannot be used in the class definition itself.

Example:

```
% oo::class create Foo {
    method bar {z} {
        classvariable x y
        return [incr x $z],[incr y]
    }
}
::Foo
% Foo create a
::a
% Foo create b
::b
% a bar 2
2,1
```

```
% a bar 3
5,2
% b bar 7
12,3
% b bar -1
11,4
% a bar 0
11,5
```

link *method*...
see below ...

link {*alias method*}...
This command is available within instance methods. It takes a list of method names and/or pairs of alias- and method-name and makes the named methods available to all instance methods without requiring the **my** command.

The alias name under which the method becomes available defaults to the method name, except where explicitly specified through an alias/method pair.

Examples:

```
    link foo
    # The method foo is now directly accessible as foo instead of my
    foo.

    link {bar foo}
    # The method foo is now directly accessible as bar.

    link a b c
    # The methods a, b, and c all become directly acessible under
    their
    # own names.
```

The main use of this command is expected to be in instance constructors, for convenience, or to set up some methods for use in a mini DSL.

ooutil::singleton ?*arg*...?
This command is a meta-class, i.e. a variant of the builtin **oo::class** which ensures that it creates only a single instance of the classes defined with it.

Syntax and results are like for **oo::class**.

Example:

```
% oo::class create example {
   self mixin singleton
   method foo {} {self}
}
::example
% [example new] foo
```

```
::oo::Obj22
% [example new] foo
::oo::Obj22
```

Authors
Donal Fellows, Andreas Kupries

Bugs, ideas, feedback
If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *oo::util* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Utility

Copyright

```
Copyright (c) 2011-2015 Andreas Kupries, BSD licensed
```


otp

otp
Keywords: otp, hashing, message-digest, password, rfc 2289, security

Name
otp - One-Time Passwords

Synopsis
package require Tcl 8.5 9
package require otp ?**1.1.0**?
::otp::otp-md4 ?*-hex*? ?*-words*? *-seed seed -count count data*
::otp::otp-md5 ?*-hex*? ?*-words*? *-seed seed -count count data*
::otp::otp-sha1 ?*-hex*? ?*-words*? *-seed seed -count count data*
::otp::otp-rmd160 ?*-hex*? ?*-words*? *-seed seed -count count data*

Description
This package is an implementation in Tcl of the One-Time Password system as described in RFC 2289 (1). This system uses message-digest algorithms to sequentially hash a passphrase to create single-use passwords. The resulting data is then provided to the user as either hexadecimal digits or encoded using a dictionary of 2048 words. This system is used by OpenBSD for secure login and can be used as a SASL mechanism for authenticating users.

In this implementation we provide support for four algorithms that are included in the tcllib

distribution: MD5 (2), MD4 (3), RIPE-MD160 (4) and SHA-1 (5).

Commands

- **::otp::otp-md4** ?*-hex*? ?*-words*? *-seed seed -count count data*
- **::otp::otp-md5** ?*-hex*? ?*-words*? *-seed seed -count count data*
- **::otp::otp-sha1** ?*-hex*? ?*-words*? *-seed seed -count count data*
- **::otp::otp-rmd160** ?*-hex*? ?*-words*? *-seed seed -count count data*

Examples

```
% otp::otp-md5 -count 99 -seed host67821 "My Secret Pass Phrase"
(binary gibberish)
% otp::otp-md5 -words -count 99 -seed host67821 "My Secret Pass Phrase"
SOON ARAB BURG LIMB FILE WAD
% otp::otp-md5 -hex -count 99 -seed host67821 "My Secret Pass Phrase"
e249b58257c80087
```

References

1. Haller, N. et al., "A One-Time Password System", RFC 2289, February 1998. http://www.rfc-editor.org/rfc/rfc2289.txt
2. Rivest, R., "The MD5 Message-Digest Algorithm", RFC 1321, MIT and RSA Data Security, Inc, April 1992. (http://www.rfc-editor.org/rfc/rfc1321.txt)
3. Rivest, R., "The MD4 Message Digest Algorithm", RFC 1320, MIT, April 1992. (http://www.rfc-editor.org/rfc/rfc1320.txt)
4. H. Dobbertin, A. Bosselaers, B. Preneel, "RIPEMD-160, a strengthened version of RIPEMD" http://www.esat.kuleuven.ac.be/~cosicart/pdf/AB-9601/AB-9601.pdf
5. "Secure Hash Standard", National Institute of Standards and Technology, U.S. Department Of Commerce, April 1995. (http://www.itl.nist.gov/fipspubs/fip180-1.htm)

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *otp* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Hashes, checksums, and encryption

Copyright

```
Copyright (c) 2006, Pat Thoyts <patthoyts@users.sourceforge.net>
```

page

page_intro

Keywords: page_intro, page, parser generator, text processing

Name

page_intro - page introduction

Description

page (short for *parser generator*) stands for a set of related packages which help in the construction of parser generators, and other utilities doing text processing.

They are mainly geared towards supporting the Tcllib application **page**, with the package **page::pluginmgr** in a central role as the plugin management for the application. The other packages are performing low-level text processing and utility tasks geared towards parser generation and mainly accessed by **page** through plugins.

The packages implementing the plugins are not documented as regular packages, as they cannot be loaded into a general interpreter, like tclsh, without extensive preparation of the interpreter. Preparation which is done for them by the plugin manager.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *page* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Page Parser Generator

Copyright

```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

page_pluginmgr

Keywords: page_pluginmgr, page, parser generator, text processing

Name

page_pluginmgr - page plugin manager

Synopsis

package require page::pluginmgr ?**0.3**?
package require fileutil
::page::pluginmgr::reportvia *cmd*
::page::pluginmgr::report *level text* ?*from* ?*to*?
::page::pluginmgr::log *cmd*

::page::pluginmgr::configuration *name*
::page::pluginmgr::reader *name*
::page::pluginmgr::rconfigure *dict*
::page::pluginmgr::rtimeable
::page::pluginmgr::rtime
::page::pluginmgr::rgettime
::page::pluginmgr::rhelp
::page::pluginmgr::rlabel
::page::pluginmgr::read *read eof* ?*complete*?
read **num**
eof
done
::page::pluginmgr::writer *name*
::page::pluginmgr::wconfigure *dict*
::page::pluginmgr::wtimeable
::page::pluginmgr::wtime
::page::pluginmgr::wgettime
::page::pluginmgr::whelp
::page::pluginmgr::wlabel
::page::pluginmgr::write *chan data*
::page::pluginmgr::transform *name*
::page::pluginmgr::tconfigure *id dict*
::page::pluginmgr::ttimeable *id*
::page::pluginmgr::ttime *id*
::page::pluginmgr::tgettime *id*
::page::pluginmgr::thelp *id*
::page::pluginmgr::tlabel *id*
::page::pluginmgr::transform_do *id data*
page_cdefinition
page_rfeature *name*
page_rtime
page_rgettime
page_rlabel
page_rhelp
page_roptions
page_rconfigure *option value*
page_rrun
page_read *num*
page_read_done
page_eof
page_info *text* ?*from ?to*?
page_warning *text* ?*from ?to*?
page_error *text* ?*from ?to*?
page_log_info *text*
page_log_warning *text*
page_log_error *text*

page_wfeature
page_wtime
page_wgettime
page_wlabel
page_whelp
page_woptions
page_wconfigure *option value*
page_wrun *chan data*
page_info *text ?from ?to?*
page_warning *text ?from ?to?*
page_error *text ?from ?to?*
page_log_info *text*
page_log_warning *text*
page_log_error *text*
page_tfeature
page_ttime
page_tgettime
page_tlabel
page_thelp
page_toptions
page_tconfigure *option value*
page_trun *chan data*
page_info *text ?from ?to?*
page_warning *text ?from ?to?*
page_error *text ?from ?to?*
page_log_info *text*
page_log_warning *text*
page_log_error *text*

Description

This package provides the plugin manager central to the **page** application. It manages the various reader, writer, configuration, and transformation plugins which actually process the text (read, transform, and write).

All plugins are loaded into slave interpreters specially prepared for them. While implemented using packages they need this special environment and are not usable in a plain interpreter, like tclsh. Because of that they are only described in general terms in section Predefined plugins, and not documented as regular packages. It is expected that they follow the APIs specified in the sections

1. Config plugin API
2. Reader plugin API
3. Writer plugin API
4. **TRANSFORM PLUGIN API**

as per their type.

API

::page::pluginmgr::reportvia *cmd*

This command defines the callback command used by **::page::pluginmgr::report** (see below) to report input errors and warnings. The default is to write such reports to the standard error channel.

::page::pluginmgr::report *level text* ?*from* ?*to*??

This command is used to report input errors and warnings. By default such reports are written to the standard error. This can be changed by setting a user-specific callback command with **::page::pluginmgr::reportvia** (see above).

The arguments *level* and *text* specify both the importance of the message, and the message itself. For the former see the package **logger** for the allowed values.

The optional argument *from* and *to* can be used by the caller to indicate the location (or range) in the input where the reported problem occured. Each is a list containing two elements, the line and the column in the input, in this order.

::page::pluginmgr::log *cmd*

This command defines a log callback command to be used by loaded plugins for the reporting of internal errors, warnings, and general information. Specifying the empty string as callback disables logging.

Note: The *cmd* has to be created by the **logger** package, or follow the same API as such.

The command returns the empty string as its result.

::page::pluginmgr::configuration *name*

This command loads the named configuration plugin, retrieves the options encoded in it, and then immediately unloads it again.

If the *name* is the path to a file, then this files will be tried to be loaded as a plugin first, and, if that fails, opened and its contents read as a list of options and their arguments, separated by spaces, tabs and newlines, possibly quotes with single and double quotes.

See section Config plugin API for the API expected of configuration plugins.

The result of the command is the list of options retrieved.

::page::pluginmgr::reader *name*

This command loads the named reader plugin and initializes it. The result of the command is a list of options the plugin understands.

Only a single reader plugin can be loaded. Loading another reader plugin causes the previously loaded reader plugin to be de-initialized and unloaded.

See section Reader plugin API for the API expected of reader plugins.

::page::pluginmgr::rconfigure *dict*

This commands configures the loaded reader plugin. The options and their values are provided as a Tcl dictionary. The result of the command is the empty string.

::page::pluginmgr::rtimeable

This commands checks if the loaded reader plugin is able to collect timing statistics. The result of the command is a boolean flag. The result is **true** if the plugin can be timed, and **false** otherwise.

::page::pluginmgr::rtime
This command activates the collection of timing statistics in the loaded reader plugin.

::page::pluginmgr::rgettime
This command retrieves the collected timing statistics of the loaded reader plugin after it was executed.

::page::pluginmgr::rhelp
This command retrieves the help string of the loaded reader plugin. This is expected to be in *doctools* format.

::page::pluginmgr::rlabel
This command retrieves the human-readable name of the loaded reader plugin.

::page::pluginmgr::read *read eof ?complete?*
This command invokes the loaded reader plugin to process the input, and returns the results of the plugin as its own result. The input is accessible through the callback commands *read*, and *eof*. The optional *done* can be used to intrecept when the plugin has completed its processing. All arguments are command prefixes.

The plugin will invoke the various callbacks in the following situations:

read num
is invoked whenever input to process is needed, with the number of characters/bytes it asks for. The result is expected to be the input the plugin is in need of.

eof
is invoked by the plugin to check if the input has reached the of the stream. The result is expected to be a boolean flag, **true** when the input has hit EOF, and **false** otherwise.

done
is invoked when the plugin has completed the processing of the input.

::page::pluginmgr::writer *name*
This command loads the named writer plugin and initializes it. The result of the command is a list of options the plugin understands.

Only a single reader plugin can be loaded. Loading another reader plugin causes the previously loaded reader plugin to be de-initialized and unloaded.

See section Writer plugin API for the API expected of writer plugins.

::page::pluginmgr::wconfigure *dict*
This commands configures the loaded writer plugin. The options and their values are provided as a Tcl dictionary. The result of the command is the empty string.

::page::pluginmgr::wtimeable
This commands checks if the loaded writer plugin is able to measure execution times. The result of the command is a boolean flag. The result is **true** if the plugin can be timed, and **false** otherwise.

::page::pluginmgr::wtime
This command activates the collection of timing statistics in the loaded writer plugin.

::page::pluginmgr::wgettime
This command retrieves the collected timing statistics of the loaded writer plugin after it was executed.

::page::pluginmgr::whelp
This command retrieves the help string of the loaded writer plugin. This is expected to be in *doctools* format.

::page::pluginmgr::wlabel
This command retrieves the human-readable name of the loaded writer plugin.

::page::pluginmgr::write *chan data*
The loaded writer plugin is invoked to generate the output. It is given the *data* to generate the outpout from, and the Tcl handle *chan* of the channel to write the generated output to. The command returns th empty string as its result.

::page::pluginmgr::transform *name*
This command loads the named transformation plugin and initializes it. The result of the command is a 2-element list containing the plugin id and a list of options the plugin understands, in this order.

Multiple transformations plugins can be loaded and are identified by handles.

See section **TRANSFORM PLUGIN API** for the API expected of transformation plugins.

::page::pluginmgr::tconfigure *id dict*
This commands configures the identified transformation plugin. The options and their values are provided as a Tcl dictionary. The result of the command is the empty string.

::page::pluginmgr::ttimeable *id*
This commands checks if the identified transformation plugin is able to collect timing statistics. The result of the command is a boolean flag. The result is **true** if the plugin can be timed, and **false** otherwise.

::page::pluginmgr::ttime *id*
This command activates the collection of timing statistics in the identified transformation plugin.

::page::pluginmgr::tgettime *id*
This command retrieves the collected timing statistics of the identified transformation plugin after it was executed.

::page::pluginmgr::thelp *id*
This command retrieves the help string of the identified transformation plugin. This is expected to be in *doctools* format.

::page::pluginmgr::tlabel *id*
This command retrieves the human-readable name of the identified transformation plugin.

::page::pluginmgr::transform_do *id data*
The identified transformation plugin is invoked to process the specified *data*. The result of the plugin is returned as the result of the command.

Config plugin API

Configuration plugins are expected to provide a single command, described below.

page_cdefinition
This command of a configuration plugin is called by the plugin manager to execute it. Its result has to be a list of options and values to process.

Configuration plugins do not expect the environment to provide any special commands.

It is expected that a configuration plugin **FOO** is implemented by the package **page::config::FOO**.

Configuration plugins are loaded, executed, and unloaded in one step, they are not kept in memory. The command for doing this is **::page::pluginmgr::configuration**.

Reader plugin API

Reader plugins are expected to provide the following commands, described below.

page_rfeature *name*
This command takes a feature *name* and returns a boolean flag indicating whether the feature is supported by the plugin, or not. The result has to be **true** if the feature is supported, and **false** otherwise.

See section Features for the possible features the plugin manager will ask for.

page_rtime
This command is invoked to activate the collection of timing statistics.

page_rgettime
This command is invoked to retrieve the collected timing statistics.

page_rlabel
This command is invoked to retrieve a human-readable label for the plugin.

page_rhelp
This command is invoked to retrieve a help text for plugin. The text is expected to be in *doctools* format.

page_roptions
This command is invoked to retrieve the options understood by the plugin.

page_rconfigure *option value*
This command is invoked to reconfigure the plugin, specifically the given *option* is set to the new *value*.

page_rrun
This command is invoked to process the input stream per the current plugin configuration. The result of the command is the result of the processing.

Reader plugins expect the environment to provide the following special commands.

page_read *num*
This command is invoked to read *num* characters/bytes from the input. Its result has to be read characters/bytes.

page_read_done
This command is invoked to signal that the plugin has completed the processing of the input.

page_eof
This command is invoked to check if the input stream has reached its end. Its result has to be a boolean flag, **true** when the input has reached the end, **false** otherwise.

page_info *text* ?*from* ?*to*??
Invoked to report some information to the user. May indicate a location or range in the input. Each piece of location data, if provided, is a 2-element list containing line and column numbers.

page_warning *text* ?*from* ?*to*??
Invoked to report a warning to the user. May indicate a location or range in the input. Each piece of location data, if provided, is a 2-element list containing line and column numbers.

page_error *text* ?*from* ?*to*??
Invoked to report an error to the user. May indicate a location or range in the input. Each piece of location data, if provided, is a 2-element list containing line and column numbers.

page_log_info *text*
Invoked to report some internal information.

page_log_warning *text*
Invoked to report an internal warning.

page_log_error *text*
Invoked to report an internal error.

It is expected that a reader plugin **FOO** is implemented by the package **page::reader::FOO**.

Reader plugins are loaded by the command **::page::pluginmgr::reader**. At most one reader plugin can be kept in memory.

Writer plugin API

Writer plugins are expected to provide the following commands, described below.

page_wfeature
This command takes a feature *name* and returns a boolean flag indicating whether the feature is supported by the plugin, or not. The result has to be **true** if the feature is supported, and **false** otherwise.

See section Features for the possible features the plugin manager will ask for.

page_wtime
This command is invoked to activate the collection of timing statistics.

page_wgettime
This command is invoked to retrieve the collected timing statistics.

page_wlabel
This command is invoked to retrieve a human-readable label for the plugin.

page_whelp
This command is invoked to retrieve a help text for plugin. The text is expected to be in *doctools* format.

page_woptions
This command is invoked to retrieve the options understood by the plugin.

page_wconfigure *option value*
This command is invoked to reconfigure the plugin, specifically the given *option* is set to the new *value*.

page_wrun *chan data*
This command is invoked to process the specified *data* and write it to the output stream *chan*.

The latter is a Tcl channel handle opened for writing. The result of the command is the empty string.

Writer plugins expect the environment to provide the following special commands.

page_info *text* ?*from* ?*to*??
Invoked to report some information to the user. May indicate a location or range in the input. Each piece of location data, if provided, is a 2-element list containing line and column numbers.

page_warning *text* ?*from* ?*to*??
Invoked to report a warning to the user. May indicate a location or range in the input. Each piece of location data, if provided, is a 2-element list containing line and column numbers.

page_error *text* ?*from* ?*to*??
Invoked to report an error to the user. May indicate a location or range in the input. Each piece of location data, if provided, is a 2-element list containing line and column numbers.

page_log_info *text*
Invoked to report some internal information.

page_log_warning *text*
Invoked to report an internal warning.

page_log_error *text*
Invoked to report an internal error.

It is expected that a writer plugin **FOO** is implemented by the package **page::writer::FOO**.

Writer plugins are loaded by the command **::page::pluginmgr::writer**. At most one writer plugin can be kept in memory.

Transform plugin API" page::transform::*

Transformation plugins are expected to provide the following commands, described below.

page_tfeature
This command takes a feature *name* and returns a boolean flag indicating whether the feature is supported by the plugin, or not. The result has to be **true** if the feature is supported, and **false** otherwise.

See section Features for the possible features the plugin manager will ask for.

page_ttime
This command is invoked to activate the collection of timing statistics.

page_tgettime
This command is invoked to retrieve the collected timing statistics.

page_tlabel
This command is invoked to retrieve a human-readable label for the plugin.

page_thelp
This command is invoked to retrieve a help text for plugin. The text is expected to be in *doctools* format.

page_toptions
This command is invoked to retrieve the options understood by the plugin.

page_tconfigure *option value*

This command is invoked to reconfigure the plugin, specifically the given *option* is set to the new *value*.

page_trun *chan data*
This command is invoked to process the specified *data* and write it to the output stream *chan*. The latter is a Tcl channel handle opened for writing. The result of the command is the empty string.

Transformation plugins expect the environment to provide the following special commands.

page_info *text* ?*from* ?*to*??
Invoked to report some information to the user. May indicate a location or range in the input. Each piece of location data, if provided, is a 2-element list containing line and column numbers.

page_warning *text* ?*from* ?*to*??
Invoked to report a warning to the user. May indicate a location or range in the input. Each piece of location data, if provided, is a 2-element list containing line and column numbers.

page_error *text* ?*from* ?*to*??
Invoked to report an error to the user. May indicate a location or range in the input. Each piece of location data, if provided, is a 2-element list containing line and column numbers.

page_log_info *text*
Invoked to report some internal information.

page_log_warning *text*
Invoked to report an internal warning.

page_log_error *text*
Invoked to report an internal error.

It is expected that a transformation plugin **FOO** is implemented by the package **page::transform::FOO**.

Transformation plugins are loaded by the command **::page::pluginmgr::transform**. More than one transformation plugin can be kept in memory.

Predefined plugins

The following predefined plugins are known, i.e. provided by the page module.

Configuration
see below ...

peg
Returns a set of options to configure the **page** application for the processing of a PEG grammar and the generation of ME code. See the packages **grammar_peg**, **grammar_me** and relations for more details.

Reader
see below ...

hb
Expects a so-called *half-baked PEG container* as input and returns the equivalent abstract syntax tree. See the writer plugin *hb* for the plugin generating this type of input.

lemon
Expects a grammar specification as understood by Richar Hipp's LEMON parser generator

and returns an abstract syntax tree for it.

peg
Expects a grammar specification in the form of a parsing expression grammar (PEG) and returns an abstract syntax tree for it.

ser
Expect the serialized form of a parsing expression grammar as generated by the package **grammar::peg** as input, converts it into an equivalent abstract syntax tree and returns that.

treeser
Expects the serialized form of a tree as generated by the package **struct::tree** as input and returns it, after validation.

Writer
see below ...

hb
Expects an abstract syntax tree for a parsing expression grammar as input and writes it out in the form of a so-called *half-baked PEG container*.

identity
Takes any input and writes it as is.

mecpu
Expects symbolic assembler code for the MatchEngine CPU (See the package **grammar::me::cpu** and relatives) and writes it out as Tcl code for a parser.

me
Expects an abstract syntax tree for a parsing expression grammar as input and writes it out as Tcl code for the MatchEngine (See the package **grammar::me** and relatives) which parses input in that grammar.

null
Takes any input and writes nothing. The logical equivalent of /dev/null.

peg
Expects an abstract syntax tree for a parsing expression grammar as input and writes it out in the form of a canonical PEG which can be read by the reader plugin *peg*.

ser
Expects an abstract syntax tree for a parsing expression grammar as input and writes it out as a serialized PEG container which can be read by the reader plugin *ser*.

tpc
Expects an abstract syntax tree for a parsing expression grammar as input and writes it out as Tcl code initializing a PEG container as provided by the package **grammar::peg**.

tree
Takes any serialized tree (per package **struct::tree**) as input and writes it out in a generic indented format.

Transformation
see below ...

mecpu
Takes an abstract syntax tree for a parsing expression grammer as input, generates

symbolic assembler code for the MatchEngine CPU, and returns that as its result (See the package **grammar::me::cpu** and relatives).

reachable
Takes an abstract syntax tree for a parsing expression grammer as input, performs a reachability analysis, and returns the modified and annotated tree.

realizable
Takes an abstract syntax tree for a parsing expression grammer as input, performs an analysis of realizability, and returns the modified and annotated tree.

Features
The plugin manager currently checks the plugins for only one feature, **timeable**. A plugin supporting this feature is assumed to be able to collect timing statistics on request.

Bugs, ideas, feedback
If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *page* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Page Parser Generator

Copyright

```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


page_util_flow
Keywords: page_util_flow, dataflow, graph walking, page, parser generator, text processing, tree walking

Name
page_util_flow - page dataflow/treewalker utility

Synopsis
package require page::util::flow ?**0.2**?
package require snit
::page::util::flow *start flowvar nodevar script*
flow **visit** *node*
flow **visitl** *nodelist*
flow **visita** *node...*

Description

This package provides a single utility command for easy dataflow based manipulation of arbitrary data structures, especially abstract syntax trees.

API

::page::util::flow *start flowvar nodevar script*

This command contains the core logic to drive the walking of an arbitrary data structure which can partitioned into separate parts. Examples of such structures are trees and graphs.

The command makes no assumptions at all about the API of the structure to be walked, except that that its parts, here called *nodes*, are identified by strings. These strings are taken as is, from the arguments, and the body, and handed back to the body, without modification.

Access to the actual data structure, and all decisions regarding which nodes to visit in what order are delegated to the body of the loop, i.e. the *script*.

The body is invoked first for the nodes in the start-set specified via *start*), and from then on for the nodes the body has requested to be visited. The command stops when the set of nodes to visit becomes empty. Note that a node can be visited more than once. The body has complete control about this.

The body is invoked in the context of the caller. The variable named by *nodevar* will be set to the current node, and the variable named by *flowvar* will be set to the command of the flow object through which the body can request the nodes to visit next. The API provided by this object is described in the next section, Flow API.

Note that the command makes no promises regarding the order in which nodes are visited, excpt that the nodes requested to be visited by the current iteration will be visited afterward, in some order.

Flow API

This section describes the API provided by the flow object made accessible to the body script of **::page::util::flow**.

flow* visit *node

Invoking this method requests that the node *n* is visited after the current iteration.

flow* visitl *nodelist

Invoking this method requests that all the nodes found in the list *nodelist* are visited after the current iteration.

flow* visita *node...

This is the variadic arguments form of the method **visitl**, see above.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *page* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Page Parser Generator

Copyright

```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


page_util_norm_lemon
Keywords: page_util_norm_lemon, graph walking, lemon, normalization, page, parser generator, text processing, tree walking

Name
page_util_norm_lemon - page AST normalization, LEMON

Synopsis
package require page::util::norm_lemon ?**0.2**?
package require snit
::page::util::norm::lemon *tree*

Description
This package provides a single utility command which takes an AST for a lemon grammar and normalizes it in various ways. The result is called a *Normalized Lemon Grammar Tree*.

Note that this package can only be used from within a plugin managed by the package **page::pluginmgr**.

API
::page::util::norm::lemon *tree*

This command assumes the *tree* object contains for a lemon grammar. It normalizes this tree in place. The result is called a *Normalized Lemon Grammar Tree*.

The exact operations performed are left undocumented for the moment.

Bugs, ideas, feedback
If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *page* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Page Parser Generator

Copyright

```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

page_util_norm_peg

Keywords: page_util_norm_peg, PEG, graph walking, normalization, page, parser generator, text processing, tree walking

Name

page_util_norm_peg - page AST normalization, PEG

Synopsis

package require page::util::norm_peg ?**0.2**?
package require snit
::page::util::norm::peg *tree*

Description

This package provides a single utility command which takes an AST for a parsing expression grammar and normalizes it in various ways. The result is called a *Normalized PE Grammar Tree*.

Note that this package can only be used from within a plugin managed by the package **page::pluginmgr**.

API

::page::util::norm::peg *tree*

This command assumes the *tree* object contains for a parsing expression grammar. It normalizes this tree in place. The result is called a *Normalized PE Grammar Tree*.

The following operations are performd

1. The data for all terminals is stored in their grandparental nodes. The terminal nodes and their parents are removed. Type information is dropped.
2. All nodes which have exactly one child are irrelevant and are removed, with the exception of the root node. The immediate child of the root is irrelevant as well, and removed as well.
3. The name of the grammar is moved from the tree node it is stored in to an attribute of the root node, and the tree node removed.

   The node keeping the start expression separate is removed as irrelevant and the root node of the start expression tagged with a marker attribute, and its handle saved in an attribute of the root node for quick access.
4. Nonterminal hint information is moved from nodes into attributes, and the now irrelevant nodes are deleted.

   Note: This transformation is dependent on the removal of all nodes with exactly one child, as it removes the all 'Attribute' nodes already. Otherwise this transformation would have to put the information into the grandparental node.

The default mode given to the nonterminals is **value**.

Like with the global metadata definition specific information is moved out out of nodes into attributes, the now irrelevant nodes are deleted, and the root nodes of all definitions are tagged with marker attributes. This provides us with a mapping from nonterminal names to their defining nodes as well, which is saved in an attribute of the root node for quick reference.

At last the range in the input covered by a definition is computed. The left extent comes from the terminal for the nonterminal symbol it defines. The right extent comes from the rightmost child under the definition. While this not an expression tree yet the location data is sound already.

5. The remaining nodes under all definitions are transformed into proper expression trees. First character ranges, followed by unary operations, characters, and nonterminals. At last the tree is flattened by the removal of superfluous inner nodes.

   The order matters, to shed as much nodes as possible early, and to avoid unnecessary work later.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *page* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Page Parser Generator

Copyright


```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


page_util_peg

Keywords: page_util_peg, PEG, page, parser generator, parsing expression grammar, text processing, transformation

Name

page_util_peg - page PEG transformation utilities

Synopsis

package require page::util::peg ?**0.2**?
package require snit
::page::util::peg::symbolNodeOf *tree node*
::page::util::peg::symbolOf *tree node*

::page::util::peg::updateUndefinedDueRemoval *tree*
::page::util::peg::flatten *treequery tree*
::page::util::peg::getWarnings *tree*
::page::util::peg::printWarnings *msg*
::page::util::peg::peOf *tree eroot*
::page::util::peg::printTclExpr *pe*

Description

This package provides a few common operations to PEG transformations. They assume a *Normalized PE Grammar Tree* as input, see the package **page::util::norm::peg**, possibly augmented with attributes coming from transformations not in conflict with the base definition.

API

::page::util::peg::symbolNodeOf *tree node*
Given an arbitrary expression *node* in the AST *tree* it determines the node (itself or an ancestor) containing the name of the nonterminal symbol the node belongs to, and returns its id. The result is either the root of the tree (for the start expression), or a definition node.

::page::util::peg::symbolOf *tree node*
As **::page::util::peg::symbolNodeOf**, but returns the symbol name instead of the node.

::page::util::peg::updateUndefinedDueRemoval *tree*
The removal of nodes in the AST *tree* can cause symbols to lose one or more users.

```
A used by B and C,
B is reachable,
C is not,

so A now loses the node in the expression for C calling it,
or rather, not calling it anymore.
```

This command updates the cross-references and which nonterminals are now undefined.

::page::util::peg::flatten *treequery tree*
This commands flattens nested sequence and choice operators in the AST *tree*, re-using the **treeql** object *treequery* to run the query determining which nodes to cut.

::page::util::peg::getWarnings *tree*
This command looks at the attributes of the AST *tree* for problems with the grammar and issues warnings. They do not prevent us from writing the grammar, but still represent problems with it the user should be made aware of.

The result of the command is a dictionary mapping nonterminal names to their associated warnings.

::page::util::peg::printWarnings *msg*
The argument of the command is a dictionary mapping nonterminal names to their associated warnings, as generated by, for example, the command **::page::util::peg::getWarnings**.

The warnings contained therein are formatted and then printed via the log command page_info. This means that this command can be used only from within a plugin managed by

the package **page::pluginmgr**.

::page::util::peg::peOf ***tree eroot***
: This command converts the parsing expression starting at the node *eroot* in the AST *tree* into a nested list. The exact syntax of this list specified by the package **grammar::peg**.

::page::util::peg::printTclExpr ***pe***
: This command converts the parsing expression contained in the nested list *pe* into a Tcl string which can be placed into a Tcl script. See the package **grammar::peg** for the exact syntax of *pe*.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *page* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Page Parser Generator

Copyright


```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


page_util_quote

Keywords: page_util_quote, page, parser generator, quoting, text processing

Name

page_util_quote - page character quoting utilities

Synopsis

package require page::util::quote ?**0.2**?
package require snit
::page::util::quote::unquote *char*
::page::util::quote::quote'tcl *char*
::page::util::quote::quote'tclstr *char*
::page::util::quote::quote'tclcom *char*

Description

This package provides a few utility commands to convert characters into various forms.

API

::page::util::quote::unquote ***char***
: A character, as stored in an abstract syntax tree by a PEG processor (See the packages **grammar::peg::interpreter**, **grammar::me**, and their relations), i.e. in some quoted form,

is converted into the equivalent Tcl character. The character is returned as the result of the command.

::page::util::quote::quote'tcl *char*
This command takes a Tcl character (internal representation) and converts it into a string which is accepted by the Tcl parser, will regenerate the character in question and is 7bit ASCII. The string is returned as the result of this command.

::page::util::quote::quote'tclstr *char*
This command takes a Tcl character (internal representation) and converts it into a string which is accepted by the Tcl parser and will generate a human readable representation of the character in question. The string is returned as the result of this command.

The string does not use any unprintable characters. It may use backslash-quoting. High UTF characters are quoted to avoid problems with the still prevalent ascii terminals. It is assumed that the string will be used in a double-quoted environment.

::page::util::quote::quote'tclcom *char*
This command takes a Tcl character (internal representation) and converts it into a string which is accepted by the Tcl parser when used within a Tcl comment. The string is returned as the result of this command.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *page* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Page Parser Generator

Copyright


```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pki

pki

Keywords: pki, cipher, data integrity, encryption, public key cipher, rsa, security

Name

pki - Implementation of the public key cipher

Synopsis

package require Tcl 8.5 9

package require pki ?**0.22**?
::pki::encrypt ?*-binary*? ?*-hex*? ?*-pad*? ?*-nopad*? ?*-priv*? ?*-pub*? ?*--*? *input key*
::pki::decrypt ?*-binary*? ?*-hex*? ?*-unpad*? ?*-nounpad*? ?*-priv*? ?*-pub*? ?*--*? *input key*
::pki::sign *input key* ?*algo*?
::pki::verify *signedmessage plaintext key* ?*algo*?
::pki::key *key* ?*password*? ?*encodePem*?
::pki::pkcs::parse_key *key* ?*password*?
::pki::x509::parse_cert *cert*
::pki::rsa::generate *bitlength* ?*exponent*?
::pki::x509::verify_cert *cert trustedcerts* ?*intermediatecerts*?
::pki::x509::validate_cert *cert* ?*-sign_message dn_of_signer*? ?*-encrypt_message dn_of_signer* ? ?*-sign_cert dn_to_be_signed ca_depth*? ?*-ssl dn*?
::pki::pkcs::create_csr *keylist namelist* ?*encodePem*? ?*algo*?
::pki::pkcs::parse_csr *csr*
::pki::x509::create_cert *signreqlist cakeylist serial_number notBefore notAfter isCA extensions* ?*encodePem*? ?*algo*?

Description

Commands

::pki::encrypt ?*-binary*? ?*-hex*? ?*-pad*? ?*-nopad*? ?*-priv*? ?*-pub*? ?*--*? *input key*

Encrypt a message using PKI (probably RSA).

Requires the caller to specify either **-priv** to encrypt with the private key or **-pub** to encrypt with the public key. The default option is to pad and return in hex. One of **-pub** or **-priv** must be specified.

The **-hex** option causes the data to be returned in encoded as a hexidecimal string, while the **-binary** option causes the data to be returned as a binary string. If they are specified multiple times, the last one specified is used.

The **-pad** option causes the data to be padded per PKCS#1 prior to being encrypted. The **-nopad** inhibits this behaviour. If they are specified multiple times, the last one specified is used.

The input to encrypt is specified as *input*.

The *key* parameter, holding the key to use, is a return value from either **::pki::pkcs::parse_key**, **::pki::x509::parse_cert**, or **::pki::rsa::generate**.

Mapping to OpenSSL's **openssl** application:
1. "openssl rsautl -encrypt" == "::pki::encrypt -binary -pub"
2. "openssl rsautl -sign" == "::pki::encrypt -binary -priv"

::pki::decrypt ?*-binary*? ?*-hex*? ?*-unpad*? ?*-nounpad*? ?*-priv*? ?*-pub*? ?*--*? *input key*

Decrypt a message using PKI (probably RSA). See **::pki::encrypt** for option handling.

Mapping to OpenSSL's **openssl** application:
1. "openssl rsautl -decrypt" == "::pki::decrypt -binary -priv"
2. "openssl rsautl -verify" == "::pki::decrypt -binary -pub"

::pki::sign *input key* ?*algo*?

Digitally sign message *input* using the private *key*.

If *algo* is ommited "sha1" is assumed. Possible values for *algo* include "**md5**", "**sha1**", "**sha256**", and "**raw**". Specifying "**raw**" for *algo* will inhibit the building of an ASN.1 structure to encode which hashing algorithm was chosen. *Attention*: In this case the corresponding **pkgi::verify** must be called **with** algorithm information. Conversely, specifying a non-"**raw**" algorithm here means that the corresponding **pkgi::verify** invokation has to be made *without* algorithm information.

The *input* should be the plain text, hashing will be performed on it.

The *key* should include the private key.

::pki::verify *signedmessage plaintext key* ?*algo*?

Verify a digital signature using a public *key*. Returns true or false.

Attention: The algorithm information *algo* has to be specified if and only if the **pki::sign** which generated the *signedmessage* was called with algorithm "**raw**". This inhibited the building of the ASN.1 structure encoding the chosen hashing algorithm. Conversely, if a proper algorithm was specified during signing then you *must not* specify an algorithm here.

::pki::key *key* ?*password*? ?*encodePem*?

Convert a key structure into a serialized PEM (default) or DER encoded private key suitable for other applications. For RSA keys this means PKCS#1.

::pki::pkcs::parse_key *key* ?*password*?

Convert a PKCS#1 private *key* into a usable key, i.e. one which can be used as argument for **::pki::encrypt**, **::pki::decrypt**, **::pki::sign**, and **::pki::verify**.

::pki::x509::parse_cert *cert*

Convert an X.509 certificate to a usable (public) key. The returned dictionary can be used as argument for **::pki:encrypt**, **::pki::decrypt**, and **::pki::verify**. The *cert* argument can be either PEM or DER encoded. In addition to the public keying information, the dictionary contains the following keys containing certificate content as defined in RFC5280:

- **subject** holds the name of the subject from the certificate.
- **issuer** holds the name of the issuing CA.
- **serial_number** holds the serial number of the certificate.
- **notBefore** holds the starting date for certificate validity.
- **notAfter** holds the ending date for certificate validity.
- **version** holds the X.509 version format.
- **extensions** holds a dictionary containing the extensions included in the certificate (see below).

The dictionary holds additional entries related to keying. These are intended for use of the above-mentioned commands for cryptographic operations.

The **extensions** key in the returned dictionary holds a nested dictionary whose keys correspond to the names (with same exact case) in Certificate Extensions in RFC5280. The format of each value is also based on the ASN.1 structures defined there. See the *Examples* [#section3] for an illustration.

::pki::rsa::generate *bitlength* ?*exponent*?
Generate a new RSA key pair, the parts of which can be used as argument for **::pki::encrypt**, **::pki::decrypt**, **::pki::sign**, and **::pki::verify**.

The *bitlength* argument is the length of the public key modulus.

The *exponent* argument should generally not be specified unless you really know what you are doing.

::pki::x509::verify_cert *cert trustedcerts* ?*intermediatecerts*?
Verify that a trust can be found between the certificate specified in the *cert* argument and one of the certificates specified in the list of certificates in the *trustedcerts* argument. (Eventually the chain can be through untrusted certificates listed in the *intermediatecerts* argument, but this is currently unimplemented). The certificates specified in the *cert* and *trustedcerts* option should be parsed (from **::pki::x509::parse_cert**).

::pki::x509::validate_cert *cert* ?-sign_message *dn_of_signer*? ?-encrypt_message *dn_of_signer*? ?-sign_cert *dn_to_be_signed ca_depth*? ?-ssl *dn*?
Validate that a certificate is valid to be used in some capacity. If multiple options are specified they must all be met for this procedure to return "true".

Currently, only the **-sign_cert** option is functional. Its arguments are *dn_to_be_signed* and *ca_depth*. The *dn_to_be_signed* is the distinguished from the subject of a certificate to verify that the certificate specified in the *cert* argument can sign. The *ca_depth* argument is used to indicate at which depth the verification should be done at. Some certificates are limited to how far down the chain they can be used to verify a given certificate.

::pki::pkcs::create_csr *keylist namelist* ?*encodePem*? ?*algo*?
Generate a certificate signing request from a key pair specified in the *keylist* argument.

The *namelist* argument is a list of "name" followed by "value" pairs to encoding as the requested distinguished name in the CSR.

The *encodePem* option specifies whether or not the result should be PEM encoded or DER encoded. A "true" value results in the result being PEM encoded, while any other value 9results in the the result being DER encoded. DER encoding is the default.

The *algo* argument specifies the hashing algorithm we should use to sign this certificate signing request with. The default is "sha1". Other possible values include "md5" and "sha256".

::pki::pkcs::parse_csr *csr*
Parse a Certificate Signing Request. The *csr* argument can be either PEM or DER encoded. The command returns a dictionary that includes the following keys:
- **subject** - contains the subject name from the CSR.
- **type** - contains the public key algorithm name. Currently only **rsa** is supported.
- **extensionRequest** - contains a dictionary with the contents of the **extensionRequest** https://datatracker.ietf.org/doc/html/rfc2986#page-5 information in the CSR. This has the same form as described for the **extensions** dictionary in the documentation for **parse_cert** .

There may be other keys in the dictionary related to the public key algorithm in use.

::pki::x509::create_cert *signreqlist cakeylist serial_number notBefore notAfter isCA extensions ?encodePem? ?algo?*

Sign a signing request (usually from **::pki::pkcs::create_csr** or **::pki::pkcs::parse_csr**) with a Certificate Authority (CA) certificate.

The *signreqlist* argument should be the parsed signing request.

The *cakeylist* argument should be the parsed CA certificate.

The *serial_number* argument should be a serial number unique to this certificate from this certificate authority.

The *notBefore* and *notAfter* arguments should contain the time before and after which (respectively) the certificate should be considered invalid. The time should be encoded as something **clock format** will accept (i.e., the results of **clock seconds** and **clock add**).

The *isCA* argument is a boolean argument describing whether or not the signed certificate should be a a CA certificate. If specified as true the "id-ce-basicConstraints" extension is added with the arguments of "critical" being true, "allowCA" being true, and caDepth being -1 (infinite).

The *extensions* argument is a list of extensions and their parameters that should be encoded into the created certificate. Currently only one extension is understood ("id-ce-basicConstraints"). It accepts three arguments *critical allowCA caDepth*. The *critical* argument to this extension (and any extension) whether or not the validator should reject the certificate as invalid if it does not understand the extension (if set to "true") or should ignore the extension (if set to "false"). The *allowCA* argument is used to specify as a boolean value whether or not we can be used a certificate authority (CA). The *caDepth* argument indicates how many children CAs can be children of this CA in a depth-wise fashion. A value of "0" for the *caDepth* argument means that this CA cannot sign a CA certificate and have the result be valid. A value of "-1" indicates infinite depth.

Examples

The example below retrieves a certificate from *www.example.com* using the TLS extension and dumps its content.

```
% set so [tls::socket www.example.com 443]
sock00000229EB84E710
% tls::handshake $so
1
% set status [tls::status $so]
...output not shown...
% set cert_pem [dict get $status certificate]
...output not shown...
% set cert [::pki::x509::parse_cert $cert_pem]
```

```
    ...output not shown...
    % dict get $cert subject
    C=US, ST=California, L=Los Angeles, O=Internet Corporation for
    Assigned Names and Numbers, CN=www.example.org
    % dict get $cert issuer
    C=US, O=DigiCert Inc, CN=DigiCert TLS RSA SHA256 2020 CA1
    % clock format [dict get $cert notAfter]
    Sun Dec 26 05:29:59 +0530 2021
    % set extensions [dict get $cert extensions]
    ...output not shown...
    % dict keys $extensions
    authorityKeyIdentifier subjectKeyIdentifier subjectAltName
    keyUsage extKeyUsage cRLDistributionPoints certificatePolicies
    authorityInfoAccess id-ce-basicConstraints basicConstraints
    1.3.6.1.4.1.11129.2.4.2
    dict get $extensions basicConstraints
    1 {0 -1}
    % dict get $extensions keyUsage
    1 {5 digitalSignature keyEncipherment}
    % dict get $extensions extKeyUsage
    0 {serverAuth clientAuth}
    % dict get $extensions subjectAltName
    0 {dNSName www.example.org dNSName example.com dNSName example.edu
    dNSName example.net dNSName example.org dNSName www.example.com
    dNSName www.example.edu dNSName www.example.net}
    % dict get $extensions basicConstraints
    1 {0 -1}
    % dict get $extensions keyUsage
    1 {5 digitalSignature keyEncipherment}
    % dict get $extensions extKeyUsage
    0 {serverAuth clientAuth}
```

References

1. Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile
2. New ASN.1 Modules for the Public Key Infrastructure Using X.509 (PKIX)
3. PKCS #10: Certification Request Syntax Specification

Authors

Roy Keene, Ashok P. Nadkarni

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *rsa* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be

made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Hashes, checksums, and encryption

Copyright

```
Copyright (c) 2010, 2011, 2012, 2013, 2021 - 2024 Roy Keene, Andreas
    Kupries, Ashok P. Nadkarni
```


pluginmgr

pluginmgr
Keywords: pluginmgr, plugin management, plugin search

Name
pluginmgr - Manage a plugin

Synopsis
package require Tcl 8.5 9
package require pluginmgr ?**0.5**?
::pluginmgr *objectName* ?*option value...*?
::pluginmgr::paths *objectName name...*
objectName method ?*arg arg* ...?
objectName **clone**
objectName **configure**
objectName **configure** *option*
objectName **configure -option** *value...*
objectName **cget -option**
objectName **destroy**
objectName **do** *arg...*
objectName **interpreter**
objectName **plugin**
objectName **load** *string*
objectName **unload**
objectName **list**
objectName **path** *path*
objectName **paths**

Description
This package provides commands and objects for the generic management of plugins which can be loaded into an application.

To avoid the implementation of yet another system to locate Tcl code the system provides by this package is built on top of the regular package management system. Each plugin is considered as a package and a simple invokation of **package require** is enough to locate and load it, if it exists. The only time we will need additional paths is when a plugin manager is part of a

wrapped application and has to be able to search for plugins existing outside of that application. For this situation the package provides a command to create a general set of such paths based on names for the plugin manager and/or application in question.

The main contribution of this package is a generic framework which allows the easy declaration of

1. How to translate a plugin name to the name of the package implementing it, and vice versa.
2. The list of commands a plugin has to provide as API, and also of more complex checks as code.
3. The list of commands expected by the plugin from the environment.

This then allows the easy generation of plugin managers customized to particular types of plugins for an application.

It should be noted that all plugin code is considered untrusted and will always be executed within a safe interpreter. The interpreter is enabled enough to allow plugins the loading of all additional packages they may need.

Public api

Package commands

::pluginmgr *objectName* ?*option value*...?

This command creates a new plugin manager object with an associated Tcl command whose name is *objectName*. This *object* command is explained in full detail in the sections Object command and Object methods. The object command will be created under the current namespace if the *objectName* is not fully qualified, and in the specified namespace otherwise.

The options and their values coming after the name of the object are used to set the initial configuration of the mamager object, specifying the applicable plugins and their API.

::pluginmgr::paths *objectName name*...

This utility command adds a set of paths to the specified object, based on the given *name*s. It will search for:

1. The environment variable *name*_PLUGINS. Its contents will be interpreted as a list of package paths. The entries have to be separated by either**:** (unix) or**;** (windows).

   The name will be converted to upper-case letters.
2. The registry entry "HKEY_LOCAL_MACHINE\SOFTWARE*name*\PLUGINS". Its contents will be interpreted as a list of package paths. The entries have to be separated by**;**. This item is considered only when on Windows (tm).

   The casing of letters is not changed.
3. The registry entry "HKEY_CURRENT_USER\SOFTWARE*name*\PLUGINS". Its contents will be interpreted as a list of package paths. The entries have to be separated by**;**. This item is considered only when on Windows (tm).

   The casing of letters is not changed.
4. The directory "*~/.name*/plugin".
5. The directory "*~/.name*/plugins".

   The casing of letters is not changed.

and add all the paths found that way to the list of package paths maintained by the object.

If *name* is namespaced each item in the list will be repeated per prefix of *name*, with conversion of:-sequences into the proper separator (underscore for environment variables, backslash for registry entries, and / for directories).

Examples:

```
    ::pluginmgr::paths ::obj docidx

    => env  DOCIDX_PLUGINS
       reg  HKEY_LOCAL_MACHINE\SOFTWARE\docidx\PLUGINS
       reg  HKEY_CURRENT_USER\SOFTWARE\docidx\PLUGINS
       path ~/.docidx/plugins

    ::pluginmgr::paths ::obj doctools::idx

    => env  DOCTOOLS_PLUGINS
       env  DOCTOOLS_IDX_PLUGINS
       reg  HKEY_LOCAL_MACHINE\SOFTWARE\doctools\PLUGINS
       reg  HKEY_LOCAL_MACHINE\SOFTWARE\doctools\idx\PLUGINS
       reg  HKEY_CURRENT_USER\SOFTWARE\doctools\PLUGINS
       reg  HKEY_CURRENT_USER\SOFTWARE\doctools\idx\PLUGINS
       path ~/.doctools/plugin
       path ~/.doctools/idx/plugin
```

Object command

All commands created by the command **::pluginmgr** (See section Package commands) have the following general form and may be used to invoke various operations on their plugin manager object.

objectName method ?*arg arg ...*?

The method **method** and its *arg*'uments determine the exact behavior of the command. See section Object methods for the detailed specifications.

Object methods

***objectName* clone**

This method creates a new plugin management object and returns the associated object command. The generated object is a clone of the object the method was invoked on. I.e. the new object will have the same configuration as the current object. With regard to state, if the current object has a plugin loaded then this plugin and all associated state is moved to the generated clone and the current object is reset into the base state (no plugin loaded). In this manner a configured plugin manager is also a factory for loaded plugins.

***objectName* configure**

The method returns a list of all known options and their current values when called without any arguments.

objectName* configure *option

The method behaves like the method **cget** when called with a single argument and returns

the value of the option specified by said argument.

***objectName* configure -option *value*...**
The method reconfigures the specified **option**s of the object, setting them to the associated *value*s, when called with an even number of arguments, at least two.

The legal options are described in the section Object configuration.

***objectName* cget -option**
This method expects a legal configuration option as argument and will return the current value of that option for the object the method was invoked for.

The legal configuration options are described in section Object configuration.

***objectName* destroy**
This method destroys the object it is invoked for.

***objectName* do *arg*...**
This method interprets its list of arguments as the words of a command and invokes this command in the execution context of the plugin. The result of the invoked command is made the result of the method. The call will fail with an error if no valid plugin has been loaded into the manager object.

***objectName* interpreter**
This method returns the handle of the safe interpreter the current plugin is loaded into. An empty string as return value signals that the manager currently has no valid plugin loaded.

***objectName* plugin**
This method returns the name of the plugin currently loaded. An empty string as return value signals that the manager currently has no valid plugin loaded.

objectName* load *string
This method loads, validates, and initializes a named plugin into the manager object.

The algorithm to locate and load the plugin employed is:
1. If the *string* contains the path to an existing file then this file is taken as the implementation of the plugin.
2. Otherwise the plugin name is translated into a package name via the value of the option **-pattern** and then loaded through the regular package management.
3. The load fails.

The algorithm to validate and initialize the loaded code is:
1. If the option **-api** is non-empty introspection commands are used to ascertain that the plugin provides the listed commands.
2. If the option **-check** is non-empty the specified command prefix is called.
3. If either of the above fails the candidate plugin is unloaded again
4. Otherwise all the commands specified via the option **-cmds** are installed in the plugin.

A previously loaded plugin is discarded, but only if the new plugin was found and sucessfully validated and initialized. Note that there will be no intereference between old and new plugin as both will be put into separate safe interpreters.

***objectName* unload**
This method unloads the currently loaded plugin. It returns the empty string. The call will be

silently ignored if no plugin is loaded at all.

***objectName* list**
This method uses the contents of the option **-pattern** to find all packages which can be plugins under the purview of this manager object. It translates their names into plugin names and returns a list containing them.

objectName* path *path
This methods adds the specified *path* to the list of additional package paths to look at when searching for a plugin. It returns the empty string. Duplicate paths are ignored, i.e. each path is added only once. Paths are made absolute, but are not normalized.

***objectName* paths**
This method returns a list containing all additional paths which have been added to the plugin manager object since its creation.

Object configuration

All plugin manager objects understand the following configuration options:

-pattern *string*
The value of this option is a glob pattern which has to contain exactly one '*'-operator. All packages whose names match this pattern are the plugins recognized by the manager object. And vice versa, the replacement of the '*'-operator with a plugin name will yield the name of the package implementing that plugin.

This option has no default, except if option **-name** was set. It has to be set before attempting to load a plugin, either directly, or through option **-name**.

-api *list*
The value of this option is a list of command names, and any plugin loaded has to provide these commands. Names which are not fully qualified are considered to be rooted in the global namespace. If empty no expectations are made on the plugin. The default value is the empty list.

-check *cmdprefix*
The value of this option is interpreted as a command prefix. Its purpose is to perform complex checks on a loaded plugin package to validate it, which go beyond a simple list of provided commands.

It is called with the manager object command as the only argument and has to return a boolean value. A value of **true** will be interpreted to mean that the candidate plugin passed the test. The call will happen if and only if the candidate plugin already passed the basic API check specified through the option **-api**.

The default value is the empty list, which causes the manager object to suppress the call and to assume the candidate plugin passes.

-cmds *dict*
The value of this option is a dictionary. It specifies the commands which will be made available to the plugin (as keys), and the trusted commands in the environment which implement them (as values). The trusted commands will be executed in the interpreter specified by the option **-cmdip**. The default value is the empty dictionary.

-cmdip *ipspec*
The value of this option is the path of the interpreter where the trusted commands given to the plugin will be executed in. The default is the empty string, referring to the current interpreter.

-setup *cmdprefix*
The value of this option is interpreted as a command prefix.

It is called whenever a new safe interpreter for a plugin has been created, but before a plugin is loaded. It is provided with the manager object command and the interpreter handle as its only arguments. Any return value will be ignored.

Its purpose is give a user of the plugin management the ability to define commands, packages, etc. a chosen plugin may need while being loaded.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pluginmgr* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Programming tools

Copyright

```
Copyright (c) 2005 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

png

png

Keywords: png, comment, image, timestamp

Name

png - PNG querying and manipulation of meta data

Synopsis

package require Tcl 8.5 9
package require crc32
package require png ?**0.4.1**?
::png::validate *file*
::png::isPNG *file*
::png::imageInfo *file*

::png::getTimestamp *file*
::png::setTimestamp *file time*
::png::getComments *file*
::png::removeComments *file*
::png::addComment *file keyword text*
::png::addComment *file keyword lang keyword2 text*
::png::getPixelDimension *file*
::png::image *file*
::png::write *file data*

Description

This package provides commands to query and modify PNG images. PNG stands for *Portable Network Graphics* and is specified at http://www.libpng.org/pub/png/spec/1.2.

Commands

::png::validate *file*

Returns a value indicating if *file* is a valid PNG file. The file is checked for PNG signature, each chunks checksum is verified, existence of a data chunk is verified, first chunk is checked for header, last chunk is checked for ending. Things *not* checked for are: validity of values within a chunk, multiple header chunks, noncontiguous data chunks, end chunk before actual eof. This procedure can take lots of time.

Possible return values:

OK
File is a valid PNG file.

SIG
no/broken PNG signature.

BADLEN
corrupt chunk length.

EOF
premature end of file.

NOHDR
missing header chunk.

CKSUM
crc mismatch.

NODATA
missing data chunk(s).

NOEND
missing end marker.

::png::isPNG *file*

Returns a boolean value indicating if the file *file* starts with a PNG signature. This is a much faster and less intensive check than **::png::validate** as it does not check if the PNG data is valid.

::png::imageInfo *file*

Returns a dictionary with keys **width**, **height**, **depth**, **color**, **compression**, **filter**, and

interlace. The values are the associated properties of the PNG image in *file*. Throws an error if file is not a PNG image, or if the checksum of the header is invalid. For information on interpreting the values for the returned properties see http://www.libpng.org/pub/png/spec/1.2/PNG-Chunks.html.

::png::getTimestamp *file*

Returns the epoch time if a timestamp chunk is found in the PNG image contained in the *file*, otherwise returns the empty string. Does not attempt to verify the checksum of the timestamp chunk. Throws an error if the *file* is not a valid PNG image.

::png::setTimestamp *file time*

Writes a new timestamp to the *file*, either replacing the old timestamp, or adding one just before the data chunks if there was no previous timestamp. *time* is the new time in the gmt epoch format. Throws an error if *file* is not a valid PNG image.

::png::getComments *file*

Currently supports only uncompressed comments. Does not attempt to verify the checksums of the comment chunks. Returns a list where each element is a comment. Each comment is itself a list. The list for a plain text comment consists of 2 elements: the human readable keyword, and the text data. A unicode (international) comment consists of 4 elements: the human readable keyword, the language identifier, the translated keyword, and the unicode text data. Throws an error if *file* is not a valid PNG image.

::png::removeComments *file*

Removes all comments from the PNG image in *file*. Beware - This uses memory equal to the file size minus comments, to hold the intermediate result. Throws an error if *file* is not a valid PNG image.

::png::addComment *file keyword text*

Adds a plain *text* comment to the PNG image in *file*, just before the first data chunk. Will throw an error if no data chunk is found. *keyword* has to be less than 80 characters long to conform to the PNG specification.

::png::addComment *file keyword lang keyword2 text*

Adds a unicode (international) comment to the PNG image in *file*, just before the first data chunk. Will throw an error if no data chunk is found. *keyword* has to be less than 80 characters long to conform to the PNG specification. *keyword2* is the translated *keyword*, in the language specified by the language identifier *lang*.

::png::getPixelDimension *file*

Returns a dictionary with keys **ppux**, **ppuy** and **unit** if the information is present. Otherwise, it returns the empty string.

The values of **ppux** and **ppuy** return the pixel per unit value in X or Y direction.

The allowed values for key **unit** are **meter** and **unknown**. In the case of meter, the dpi value can be calculated by multiplying with the conversion factor **0.0254**.

::png::image *file*

Given a PNG file returns the image in the list of scanlines format used by Tk_GetColor.

::png::write *file data*

Takes a list of scanlines in the Tk_GetColor format and writes the represented image to *file*.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *png* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

File formats

Copyright

```
Copyright (c) 2004, Code: Aaron Faupell
    <afaupell@users.sourceforge.net>
Copyright (c) 2004, Doc:  Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

pop3

pop3

Keywords: pop3, email, mail, pop, rfc 1939, secure, ssl, tls

Name

pop3 - Tcl client for POP3 email protocol

Synopsis

package require Tcl 8.5 9
package require pop3 ?**1.11**?
::pop3::open ?*-msex 0|1*? ?*-retr-mode retr|list|slow*? ?*-socketcmd cmdprefix*? ?*-stls 0|1*? ?*-tls-callback stls-callback-command*? *host username password* ?*port*?
::pop3::config *chan*
::pop3::status *chan*
::pop3::last *chan*
::pop3::retrieve *chan startIndex* ?*endIndex*?
::pop3::delete *chan startIndex* ?*endIndex*?
::pop3::list *chan* ?*msg*?
::pop3::top *chan msg n*
::pop3::uidl *chan* ?*msg*?
::pop3::capa *chan*
::pop3::close *chan*

Description

The **pop3** package provides a simple Tcl-only client library for the POP3 email protocol as specified in RFC 1939. It works by opening the standard POP3 socket on the server, transmitting

the username and password, then providing a Tcl API to access the POP3 protocol commands. All server errors are returned as Tcl errors (thrown) which must be caught with the Tcl **catch** command.

Tls security considerations

This package uses the **TLS** package to handle the security for **https** urls and other socket connections.

Policy decisions like the set of protocols to support and what ciphers to use are not the responsibility of **TLS**, nor of this package itself however. Such decisions are the responsibility of whichever application is using the package, and are likely influenced by the set of servers the application will talk to as well.

For example, in light of the recent POODLE attack discovered by Google many servers will disable support for the SSLv3 protocol. To handle this change the applications using **TLS** must be patched, and not this package, nor **TLS** itself. Such a patch may be as simple as generally activating **tls1** support, as shown in the example below.

```
    package require tls
    tls::init -tls1 1 ;# forcibly activate support for the TLS1
    protocol

    ... your own application code ...
```

API

::pop3::open ?-msex 0|1? ?-retr-mode retr|list|slow? ?-socketcmd cmdprefix? ?-stls 0|1? ?-tls-callback stls-callback-command? *host username password* ?*port*?

Open a socket connection to the server specified by *host*, transmit the *username* and *password* as login information to the server. The default port number is **110**, which can be overridden using the optional *port* argument. The return value is a channel used by all of the other ::pop3 functions.

The command recognizes three options

-msex boolean

Setting this option tells the package that the server we are talking to is an MS Exchange server (which has some oddities we have to work around). The default is **False**.

-retr-mode retr|list|slow

The retrieval mode determines how exactly messages are read from the server. The allowed values are **retr**, **list** and **slow**. The default is **retr**. See **::pop3::retrieve** for more information.

-socketcmd cmdprefix

This option allows the user to overide the use of the builtin **socket** command with any API-compatible command. The envisioned main use is the securing of the new connection via SSL, through the specification of the command **tls::socket**. This command is specially recognized as well, changing the default port of the connection to **995**.

-stls boolean

Setting this option tells the package to secure the connection using SSL or TLS. It performs STARTTLS as described in IETF RFC 2595, it first opens a normal, unencrypted connection

and then negotiates a SSLv3 or TLSv1 connection. If the connection cannot be secured, the connection will be closed and an error will be returned

-tls-callback stls-callback-command
This option allows the user to overide the **tls::callback** used during the **-stls** SSL/TLS handshake. See the TLS manual for details on how to implement this callback.

::pop3::config *chan*
Returns the configuration of the pop3 connection identified by the channel handle *chan* as a serialized array.

::pop3::status *chan*
Query the server for the status of the mail spool. The status is returned as a list containing two elements, the first is the number of email messages on the server and the second is the size (in octets, 8 bit blocks) of the entire mail spool.

::pop3::last *chan*
Query the server for the last email message read from the spool. This value includes all messages read from all clients connecting to the login account. This command may not be supported by the email server, in which case the server may return 0 or an error.

::pop3::retrieve *chan startIndex ?endIndex?*
Retrieve a range of messages from the server. If the *endIndex* is not specified, only one message will be retrieved. The return value is a list containing each message as a separate element. See the *startIndex* and *endIndex* descriptions below.

The retrieval mode determines how exactly messages are read from the server. The mode **retr** assumes that the RETR command delivers the size of the message as part of the command status and uses this to read the message efficiently. In mode **list** RETR does not deliver the size, but the LIST command does and we use this to retrieve the message size before the actual retrieval, which can then be done efficiently. In the last mode, **slow**, the system is unable to obtain the size of the message to retrieve in any manner and falls back to reading the message from the server line by line.

It should also be noted that the system checks upon the configured mode and falls back to the slower modes if the above assumptions are not true.

::pop3::delete *chan startIndex ?endIndex?*
Delete a range of messages from the server. If the *endIndex* is not specified, only one message will be deleted. Note, the indices are not reordered on the server, so if you delete message 1, then the first message in the queue is message 2 (message index 1 is no longer valid). See the *startIndex* and *endIndex* descriptions below.

startIndex
The *startIndex* may be an index of a specific message starting with the index 1, or it have any of the following values:

start
This is a logical value for the first message in the spool, equivalent to the value 1.

next
The message immediately following the last message read, see **::pop3::last**.

end

The most recent message in the spool (the end of the spool). This is useful to retrieve only the most recent message.

endIndex
The *endIndex* is an optional parameter and defaults to the value "-1", which indicates to only retrieve the one message specified by *startIndex*. If specified, it may be an index of a specific message starting with the index "1", or it may have any of the following values:
last
The message is the last message read by a POP3 client, see **::pop3::last**.
end
The most recent message in the spool (the end of the spool).

::pop3::list *chan* ?*msg*?
Returns the scan listing of the mailbox. If parameter *msg* is given, then the listing only for that message is returned.

::pop3::top *chan msg n*
Optional POP3 command, not all servers may support this. **::pop3::top** retrieves headers of a message, specified by parameter *msg*, and number of *n* lines from the message body.

::pop3::uidl *chan* ?*msg*?
Optional POP3 command, not all servers may support this. **::pop3::uidl** returns the uid listing of the mailbox. If the parameter *msg* is specified, then the listing only for that message is returned.

::pop3::capa *chan*
Optional POP3 command, not all servers may support this. **::pop3::capa** returns a list of the capabilities of the server. TOP, SASL, UIDL, LOGIN-DELAY and STLS are typical capabilities. See IETF RFC 2449.

::pop3::close *chan*
Gracefully close the connect after sending a POP3 QUIT command down the socket.

Secure mail transfer

A pop3 connection can be secured with SSL/TLS by requiring the package **TLS** and then using either the option **-socketcmd** or the option **-stls** of the command **pop3::open**. The first method, option **-socketcmd**, will force the use of the **tls::socket** command when opening the connection. This is suitable for POP3 servers which expect SSL connections only. These will generally be listening on port 995.

```
    package require tls
    tls::init -cafile /path/to/ca/cert -keyfile ...

    # Create secured pop3 channel
    pop3::open -socketcmd tls::socket \
        $thehost $theuser $thepassword

    ...
```

The second method, option **-stls**, will connect to the standard POP3 port and then perform an

STARTTLS handshake. This will only work for POP3 servers which have this capability. The package will confirm that the server supports STARTTLS and the handshake was performed correctly before proceeding with authentication.

```
    package require tls
    tls::init -cafile /path/to/ca/cert -keyfile ...

    # Create secured pop3 channel
    pop3::open -stls 1 \
        $thehost $theuser $thepassword

    ...
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pop3* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Networking

pop3d

pop3d

Keywords: pop3d, internet, network, pop3, protocol, rfc 1939, secure, ssl, tls

Name

pop3d - Tcl POP3 server implementation

Synopsis

package require Tcl 8.5 9
package require pop3d ?**1.2.0**?
::pop3d::new ?*serverName*?
serverName *option* ?*arg arg* ...?
serverName **up**
serverName **down**
serverName **destroy** ?*mode*?
serverName **configure**
serverName **configure** *-option*
serverName **configure** *-option value*...
serverName **cget** *-option*
serverName **conn** list

serverName **conn** state *id*
authCmd **exists** *name*
authCmd **lookup** *name*
storageCmd **dele** *mbox msgList*
storageCmd **lock** *mbox*
storageCmd **unlock** *mbox*
storageCmd **size** *mbox* ?*msgId*?
storageCmd **stat** *mbox*
storageCmd **get** *mbox msgId*

Description

::pop3d::new ?*serverName*?
This command creates a new server object with an associated global Tcl command whose name is *serverName*.

The command **serverName** may be used to invoke various operations on the server. It has the following general form:

serverName *option* ?*arg arg ...*?
Option and the *arg*s determine the exact behavior of the command.

A pop3 server can be started on any port the caller has permission for from the operating system. The default port will be 110, which is the port defined by the standard specified in RFC 1939 (http://www.rfc-editor.org/rfc/rfc1939.txt). After creating, configuring and starting a the server object will listen for and accept connections on that port and handle them according to the POP3 protocol.

Note: The server provided by this module will handle only the basic protocol by itself. For the higher levels of user authentication and handling of the actual mailbox contents callbacks will be invoked.

The following commands are possible for server objects:

***serverName* up**
After this call the server will listen for connections on its configured port.

***serverName* down**
After this call the server will stop listening for connections. This does not affect existing connections.

***serverName* destroy ?*mode*?**
Destroys the server object. Currently open connections are handled depending on the chosen mode. The provided *mode*s are:

kill
Destroys the server immediately, and forcefully closes all currently open connections. This is the default mode.

defer
Stops the server from accepting new connections and will actually destroy it only after the last of the currently open connections for the server is closed.

***serverName* configure**
Returns a list containing all options and their current values in a format suitable for use by the

command **array set**. The options themselves are described in section Options.

serverName* configure *-option
Returns the current value of the specified option. This is an alias for the method **cget**. The options themselves are described in section Options.

serverName* configure *-option value...
Sets the specified option to the provided value. The options themselves are described in section Options.

serverName* cget *-option
Returns the current value of the specified option. The options themselves are described in section Options.

***serverName* conn list**
Returns a list containing the ids of all connections currently open.

serverName* conn state *id
Returns a list suitable for [**array set**] containing the state of the connection referenced by *id*.

Options

The following options are available to pop3 server objects.

-port *port*
Defines the *port* to listen on for new connections. Default is 110. This option is a bit special. If *port* is set to "0" the server, or rather the operating system, will select a free port on its own. When querying **-port** the id of this chosen port will be returned. Changing the port while the server is up will neither change the returned value, nor will it change on which port the server is listening on. Only after resetting the server via a call to **down** followed by a call to **up** will the new port take effect. It is at that time that the value returned when querying **-port** will change too.

-auth *command*
Defines a *command* prefix to call whenever the authentication of a user is required. If no such command is specified the server will reject all users. The interface which has to be provided by the command prefix is described in section Authentication.

-storage *command*
Defines a *command* prefix to call whenever the handling of mailbox contents is required. If no such command is specified the server will claim that all mailboxes are empty. The interface which has to be provided by the command prefix is described in section Mailboxes.

-socket *command*
Defines a *command* prefix to call for opening the listening socket. This can be used to make the pop3 server listen on a SSL socket as provided by the **tls** package, see the command **tls::socket**.

Authentication

Here we describe the interface which has to be provided by the authentication callback so that pop3 servers following the interface of this module are able to use it.

authCmd* exists *name
This method is given a user*name* and has to return a boolean value telling whether or not the specified user exists.

authCmd* lookup *name
This method is given a user*name* and has to return a two-element list containing the password for this user and a storage reference, in this order.

The storage reference is passed unchanged to the storage callback, see sections Options and Mailboxes for either the option defining it and or the interface to provide, respectively.

Mailboxes

Here we describe the interface which has to be provided by the storage callback so that pop3 servers following the interface of this module are able to use it. The *mbox* argument is the storage reference as returned by the **lookup** method of the authentication command, see section Authentication.

storageCmd* dele *mbox msgList
Deletes the messages whose numeric ids are contained in the *msgList* from the mailbox specified via *mbox*.

storageCmd* lock *mbox
This method locks the specified mailbox for use by a single connection to the server. This is necessary to prevent havoc if several connections to the same mailbox are open. The complementary method is **unlock**. The command will return true if the lock could be set successfully or false if not.

storageCmd* unlock *mbox
This is the complementary method to **lock**, it revokes the lock on the specified mailbox.

***storageCmd* size *mbox* ?*msgId*?**
Determines the size of the message specified through its id in *msgId*, in bytes, and returns this number. The command will return the size of the whole maildrop if no message id was specified.

storageCmd* stat *mbox
Determines the number of messages in the specified mailbox and returns this number.

storageCmd* get *mbox msgId
Returns a handle for the specified message. This handle is a mime token following the interface described in the documentation of package **mime**. The pop3 server will use the functionality of the mime token to send the mail to the requestor at the other end of a pop3 connection.

Secure mail transfer

The option **-socket** (see Options) enables users of the package to override how the server opens its listening socket. The envisioned main use is the specification of the **tls::socket** command, see package **tls**, to secure the communication.

```
package require tls
tls::init \
    ...

pop3d::new S -socket tls::socket
...
```

References

1. RFC 1939
2. RFC 2449

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pop3d* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Networking

Copyright

```
Copyright (c) 2002-2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
Copyright (c) 2005 Reinhard Max  <max@suse.de>
```

pop3d_dbox

Keywords: pop3d_dbox, internet, network, pop3, protocol, rfc 822

Name

pop3d::dbox - Simple mailbox database for pop3d

Synopsis

package require Tcl 8.5 9
package require pop3d::dbox ?**1.0.3**?
::pop3d::dbox::new ?*dbName*?
dbName *option* ?*arg arg ...*?
dbName **destroy**
dbName **base** *base*
dbName **add** *mbox*
dbName **remove** *mbox*
dbName **move** *old new*
dbName **list**
dbName **exists** *mbox*
dbName **locked** *mbox*
dbName **lock** *mbox*
dbName **unlock** *mbox*
dbName **stat** *mbox*
dbName **size** *mbox* ?*msgId*?
dbName **dele** *mbox msgList*
storageCmd **get** *mbox msgId*

Description

The package **pop3d::dbox** provides simple/basic mailbox management facilities. Each mailbox object manages a single base directory whose subdirectories represent the managed mailboxes. Mails in a mailbox are represented by files in a mailbox directory, where each of these files contains a single mail, both headers and body, in RFC 822 (http://www.rfc-editor.org/rfc/rfc822.txt) conformant format.

Any mailbox object following the interface described below can be used in conjunction with the pop3 server core provided by the package **pop3d**. It is especially possible to directly use the objects created by this package in the storage callback of pop3 servers following the same interface as servers created by the package **pop3d**.

::pop3d::dbox::new ?*dbName*?
: This command creates a new database object with an associated global Tcl command whose name is *dbName*.

The command **dbName** may be used to invoke various operations on the database. It has the following general form:

dbName *option* ?*arg arg ...*?
: *Option* and the *arg*s determine the exact behavior of the command.

The following commands are possible for database objects:

***dbName* destroy**
: Destroys the mailbox database and all transient data. The directory associated with the object is not destroyed.

dbName* base *base
: Defines the base directory containing the mailboxes to manage. If this method is not called none of the following methods will work.

dbName* add *mbox
: Adds a mailbox of name *mbox* to the database. The name must be a valid path component.

dbName* remove *mbox
: Removes the mailbox specified through *mbox*, and the mails contained therein, from the database. This method will fail if the specified mailbox is locked.

dbName* move *old new
: Changes the name of the mailbox *old* to *new*.

***dbName* list**
: Returns a list containing the names of all mailboxes in the directory associated with the database.

dbName* exists *mbox
: Returns true if the mailbox with name *mbox* exists in the database, or false if not.

dbName* locked *mbox
: Checks if the mailbox specified through *mbox* is currently locked.

dbName* lock *mbox
: This method locks the specified mailbox for use by a single connection to the server. This is necessary to prevent havoc if several connections to the same mailbox are open. The complementary method is **unlock**. The command will return true if the lock could be set successfully or false if not.

dbName* unlock *mbox
: This is the complementary method to **lock**, it revokes the lock on the specified mailbox.

dbName* stat *mbox
: Determines the number of messages in the specified mailbox and returns this number. This method fails if the mailbox *mbox* is not locked.

***dbName* size *mbox* ?*msgId*?**
: Determines the size of the message specified through its id in *msgId*, in bytes, and returns this number. The command will return the size of the whole maildrop if no message id was specified. If specified the *msgId* has to be in the range "1 ... [*dbName* **stat**]" or this call will fail. If **stat** was not called before this call, **size** will assume that there are zero messages in the mailbox.

dbName* dele *mbox msgList
: Deletes the messages whose numeric ids are contained in the *msgList* from the mailbox specified via *mbox*. The *msgList* must not be empty or this call will fail. The numeric ids in *msgList* have to be in the range "1 ... [*dbName* **stat**]" or this call will fail. If **stat** was not called before this call, **dele** will assume that there are zero messages in the mailbox.

storageCmd* get *mbox msgId
: Returns a handle for the specified message. This handle is a mime token following the interface described in the documentation of package **mime**. The token is *read-only*. In other words, the caller is allowed to do anything with the token except to modify it. The *msgId* has to be in the range "1 ... [*dbName* **stat**]" or this call will fail. If **stat** was not called before this call, **get** will assume that there are zero messages in the mailbox.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pop3d* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Networking

Copyright


```
Copyright (c) 2002 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pop3d_udb

Keywords: pop3d_udb, internet, network, pop3, protocol

Name

pop3d::udb - Simple user database for pop3d

Synopsis

package require Tcl 8.5 9
package require pop3d::udb ?**1.0.1**?
::pop3d::udb::new ?*dbName*?
dbName *option* ?*arg arg* ...?
dbName **destroy**
dbName **add** *user pwd storage*
dbName **remove** *user*
dbName **rename** *user newName*
dbName **lookup** *user*
dbName **exists** *user*
dbName **who**
dbName **save** ?*file*?
dbName **read** *file*

Description

The package **pop3d::udb** provides simple in memory databases which can be used in conjunction with the pop3 server core provided by the package **pop3d**. The databases will use the names of users as keys and associates passwords and storage references with them.

Objects created by this package can be directly used in the authentication callback of pop3 servers following the same interface as servers created by the package **pop3d**.

::pop3d::udb::new ?*dbName*?
: This command creates a new database object with an associated global Tcl command whose name is *dbName*.

The command **dbName** may be used to invoke various operations on the database. It has the following general form:

dbName *option* ?*arg arg ...*?
: *Option* and the *arg*s determine the exact behavior of the command.

The following commands are possible for database objects:

***dbName* destroy**
: Destroys the database object.

dbName* add *user pwd storage
: Add a new user or changes the data of an existing user. Stores *password* and *storage* reference for the given *user*.

dbName* remove *user
: Removes the specified *user* from the database.

dbName* rename *user newName
: Changes the name of the specified *user* to *newName*.

dbName* lookup *user
: Searches the database for the specified *user* and returns a two-element list containing the associated password and storage reference, in this order. Throws an error if the user could not be found. This is the interface as expected by the authentication callback of package **pop3d**.

dbName* exists *user
: Returns true if the specified *user* is known to the database, else false.

***dbName* who**
Returns a list of users known to the database.

***dbName* save ?*file*?**
Saves the contents of the database into the given *file*. If the file is not specified the system will use the path last used in a call to *dbName* **read**. The generated file can be read by the **read** method.

dbName* read *file
Reads the specified *file* and adds the contained user definitions to the database. As the file is actually **source**'d a safe interpreter is employed to safeguard against malicious code. This interpreter knows the **add** command for adding users and their associated data to this database. This command has the same argument signature as the method **add**. The path of the *file* is remembered internally so that it can be used in the next call of *dbName* **save** without an argument.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pop3d* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Networking

Copyright


```
Copyright (c) 2002 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


practcl

practcl

Keywords: practcl

Name

practcl - The Practcl Module

Synopsis

package require TclOO 1.0

proc **practcl::cat** *fname*

proc **practcl::docstrip** *text*

proc **putb** ?*map*? *text*

proc **Proc** *name arglist body*

proc **noop** ?*args*?
proc **practcl::debug** ?*args*?
proc **practcl::doexec** ?*args*?
proc **practcl::doexec_in** *path* ?*args*?
proc **practcl::dotclexec** ?*args*?
proc **practcl::domake** *path* ?*args*?
proc **practcl::domake.tcl** *path* ?*args*?
proc **practcl::fossil** *path* ?*args*?
proc **practcl::fossil_status** *dir*
proc **practcl::os**
proc **practcl::mkzip** *exename barekit vfspath*
proc **practcl::sort_dict** *list*
proc **practcl::local_os**
proc **practcl::config.tcl** *path*
proc **practcl::read_configuration** *path*
proc **practcl::tcllib_require** *pkg* ?*args*?
proc **practcl::platform::tcl_core_options** *os*
proc **practcl::platform::tk_core_options** *os*
proc **practcl::read_rc_file** *filename* ?*localdat* ?
proc **practcl::read_sh_subst** *line info*
proc **practcl::read_sh_file** *filename* ?*localdat* ?
proc **practcl::read_Config.sh** *filename*
proc **practcl::read_Makefile** *filename*
proc **practcl::cputs** *varname* ?*args*?
proc **practcl::tcl_to_c** *body*
proc **practcl::_tagblock** *text* ?*style* **tcl**? ?*note* ?
proc **practcl::de_shell** *data*
proc **practcl::grep** *pattern* ?*files* ?
proc **practcl::file_lexnormalize** *sp*
proc **practcl::file_relative** *base dst*
proc **practcl::findByPattern** *basedir patterns*
proc **practcl::log** *fname comment*
proc **practcl::_pkgindex_simpleIndex** *path*
proc **practcl::_pkgindex_directory** *path*
proc **practcl::_pkgindex_path_subdir** *path*
proc **practcl::pkgindex_path** ?*args*?
proc **practcl::installDir** *d1 d2*

proc **practcl::copyDir** *d1 d2* ?*toplevel* **1**?
proc **practcl::buildModule** *modpath*
proc **practcl::installModule** *modpath DEST*
proc **practcl::trigger** ?*args*?
proc **practcl::depends** ?*args*?
proc **practcl::target** *name info* ?*action* ?
method **constructor**
method **argspec** *argspec*
method **comment** *block*
method **keyword.Annotation** *resultvar commentblock type name body*
method **keyword.Class** *resultvar commentblock name body*
method **keyword.class** *resultvar commentblock name body*
method **keyword.Class_Method** *resultvar commentblock name* ?*args*?
method **keyword.method** *resultvar commentblock name* ?*args*?
method **keyword.proc** *commentblock name argspec*
method **reset**
method **Main**
method **section.method** *keyword method minfo*
method **section.annotation** *type name iinfo*
method **section.class** *class_name class_info*
method **section.command** *procinfo*
method **manpage** ?**header** *value*? ?**footer** *value*? ?**authors** *list*?
method **scan_text** *text*
method **scan_file** *filename*
method **_MorphPatterns**
method **define** *submethod* ?*args*?
method **graft** ?*args*?
method **initialize**
method **link** *command* ?*args*?
method **morph** *classname*
method **script** *script*
method **select**
method **source** *filename*
classmethod **select** *object*
method **config.sh**
method **BuildDir** *PWD*
method **MakeDir** *srcdir*

method **read_configuration**
method **build-cflags** *PROJECT DEFS namevar versionvar defsvar*
method **critcl** ?*args*?
method **Autoconf**
method **BuildDir** *PWD*
method **ConfigureOpts**
method **MakeDir** *srcdir*
method **make {} autodetect**
method **make {} clean**
method **make {} compile**
method **make {} install** *DEST*
method **build-compile-sources** *PROJECT COMPILE CPPCOMPILE INCLUDES*
method **build-Makefile** *path PROJECT*
method **build-library** *outfile PROJECT*
method **build-tclsh** *outfile PROJECT* ?*path* **auto**?
method **BuildDir** *PWD*
method **make {} autodetect**
method **make {} clean**
method **make {} compile**
method **make {} install** *DEST*
method **MakeDir** *srcdir*
method **NmakeOpts**
method **constructor** *module_object name info* ?*action_body* ?
method **do**
method **check**
method **output**
method **reset**
method **triggers**
method **constructor** *parent* ?*args*?
method **child** *method*
method **go**
method **cstructure** *name definition* ?*argdat* ?
method **include** *header*
method **include_dir** ?*args*?
method **include_directory** ?*args*?
method **c_header** *body*
method **c_code** *body*

method **c_function** *header body* ?*info* ?
method **c_tcloomethod** *name body* ?*arginfo* ?
method **cmethod** *name body* ?*arginfo* ?
method **c_tclproc_nspace** *nspace*
method **c_tclcmd** *name body* ?*arginfo* ?
method **c_tclproc_raw** *name body* ?*arginfo* ?
method **tcltype** *name argdat*
method **project-compile-products**
method **implement** *path*
method **initialize**
method **linktype**
method **generate-cfile-constant**
method **generate-cfile-header**
method **generate-cfile-tclapi**
method **generate-loader-module**
method **Collate_Source** *CWD*
method **select**
classmethod **select** *object*
method **code** *section body*
method **Collate_Source** *CWD*
method **project-compile-products**
method **generate-debug** ?*spaces* ?
method **generate-cfile-constant**
method **generate-cfile-public-structure**
method **generate-cfile-header**
method **generate-cfile-global**
method **generate-cfile-private-typedef**
method **generate-cfile-private-structure**
method **generate-cfile-functions**
method **generate-cfile-tclapi**
method **generate-hfile-public-define**
method **generate-hfile-public-macro**
method **generate-hfile-public-typedef**
method **generate-hfile-public-structure**
method **generate-hfile-public-headers**
method **generate-hfile-public-function**
method **generate-hfile-public-includes**

method **generate-hfile-public-verbatim**
method **generate-loader-external**
method **generate-loader-module**
method **generate-stub-function**
method **IncludeAdd** *headervar* ?*args*?
method **generate-tcl-loader**
method **generate-tcl-pre**
method **generate-tcl-post**
method **linktype**
method **Ofile** *filename*
method **project-static-packages**
method **toolset-include-directory**
method **target** *method* ?*args*?
method **project-compile-products**
method **generate-loader-module**
method **project-compile-products**
method **linker-products** *configdict*
method **initialize**
variable **make_object**
method **_MorphPatterns**
method **add** ?*args*?
method **install-headers** ?*args*?
method **make {} _preamble**
method **make {} pkginfo**
method **make {} objects**
method **make {} object** *name*
method **make {} reset**
method **make {} trigger** ?*args*?
method **make {} depends** ?*args*?
method **make {} filename** *name*
method **make {} target** *name Info body*
method **make {} todo**
method **make {} do**
method **child** *which*
method **generate-c**
method **generate-h**
method **generate-loader**

method **initialize**
method **implement** *path*
method **linktype**
method **_MorphPatterns**
method **constructor** ?*args*?
method **add_object** *object*
method **add_project** *pkg info* ?*oodefine* ?
method **add_tool** *pkg info* ?*oodefine* ?
method **build-tclcore**
method **child** *which*
method **linktype**
method **project** *pkg* ?*args*?
method **tclcore**
method **tkcore**
method **tool** *pkg* ?*args*?
method **clean** *PATH*
method **project-compile-products**
method **go**
method **generate-decls** *pkgname path*
method **implement** *path*
method **generate-make** *path*
method **linktype**
method **package-ifneeded** ?*args*?
method **shared_library** ?*filename* ?
method **static_library** ?*filename* ?
method **build-tclkit_main** *PROJECT PKG_OBJS*
method **Collate_Source** *CWD*
method **wrap** *PWD exename vfspath* ?*args*?
classmethod **Sandbox** *object*
classmethod **select** *object*
classmethod **claim_option**
classmethod **claim_object** *object*
classmethod **claim_path** *path*
method **scm_info**
method **DistroMixIn**
method **Sandbox**
method **SrcDir**

method **ScmTag**
method **ScmClone**
method **ScmUnpack**
method **ScmUpdate**
method **Unpack**
classmethod **claim_object** *object*
classmethod **claim_option**
classmethod **claim_path** *path*
method **ScmUnpack**
classmethod **claim_object** *obj*
classmethod **claim_option**
classmethod **claim_path** *path*
method **scm_info**
method **ScmClone**
method **ScmTag**
method **ScmUnpack**
method **ScmUpdate**
classmethod **claim_object** *obj*
classmethod **claim_option**
classmethod **claim_path** *path*
method **ScmTag**
method **ScmUnpack**
method **ScmUpdate**
method **_MorphPatterns**
method **BuildDir** *PWD*
method **child** *which*
method **compile**
method **go**
method **install** ?*args*?
method **linktype**
method **linker-products** *configdict*
method **linker-external** *configdict*
method **linker-extra** *configdict*
method **env-bootstrap**
method **env-exec**
method **env-install**
method **env-load**

method **env-present**
method **sources**
method **update**
method **unpack**
method **env-bootstrap**
method **env-present**
method **linktype**
method **env-bootstrap**
method **env-install**
method **env-present**
method **install** *DEST*
method **kettle** *path* ?*args*?
method **install** *DEST*
method **install** *DEST*
method **env-bootstrap**
method **env-install**
method **env-present**
method **install** *DEST*
method **install-module** *DEST* ?*args*?
method **env-bootstrap**
method **env-install**
method **install** *DEST*
method **install-module** *DEST* ?*args*?
method **clean**
method **env-install**
method **project-compile-products**
method **ComputeInstall**
method **go**
method **linker-products** *configdict*
method **project-static-packages**
method **BuildDir** *PWD*
method **compile**
method **Configure**
method **install** *DEST*
method **install** *DEST*
method **install** *DEST*
method **env-bootstrap**

method **env-present**

method **env-install**

method **go**

method **linktype**

Description

The Practcl module is a tool for integrating large modules for C API Tcl code that requires custom Tcl types and TclOO objects.

The concept with Practcl is that is a single file package that can assist any tcl based project with distribution, compilation, linking, VFS preparation, executable assembly, and installation. Practcl also allows one project to invoke the build system from another project, allowing complex projects such as a statically linked basekit to be assembled with relative ease.

Practcl ships as a single file, and aside from a Tcl 8.6 interpreter, has no external dependencies.

Making a practcl project

Commands

proc practcl::cat *fname*
: Concatenate a file

proc practcl::docstrip *text*
: Strip the global comments from tcl code. Used to prevent the documentation markup comments from clogging up files intended for distribution in machine readable format.

proc putb ?*map*? *text*
: Append a line of text to a variable. Optionally apply a string mapping.

proc Proc *name arglist body*
: Generate a proc if no command already exists by that name

proc noop ?*args*?
: A command to do nothing. A handy way of negating an instruction without having to comment it completely out. It's also a handy attachment point for an object to be named later

proc practcl::debug ?*args*?
: see below ...

proc practcl::doexec ?*args*?
: Drop in a static copy of Tcl

proc practcl::doexec_in *path* ?*args*?
: see below ...

proc practcl::dotclexec ?*args*?
: see below ...

proc practcl::domake *path* ?*args*?
: see below ...

proc practcl::domake.tcl *path* ?*args*?
: see below ...

proc practcl::fossil *path* ?*args*?
: see below ...

proc practcl::fossil_status *dir*

see below ...

proc practcl::os

see below ...

proc practcl::mkzip *exename barekit vfspath*

Build a zipfile. On tcl8.6 this invokes the native Zip implementation on older interpreters this invokes zip via exec

proc practcl::sort_dict *list*

Dictionary sort a key/value list. Needed because pre tcl8.6 does not have *lsort -stride 2*

proc practcl::local_os

Returns a dictionary describing the local operating system. Fields return include:
- download - Filesystem path where fossil repositories and source tarballs are downloaded for the current user
- EXEEXT - The extension to give to executables. (i.e. .exe on windows)
- fossil_mirror - A URI for a local network web server who acts as a fossil repository mirror
- local_install - Filesystem path where packages for local consumption by the current user are installed
- prefix - The prefix as given to the Tcl core/TEA for installation to local_install in ./configure
- sandbox - The file location where this project unpacks external projects
- TEACUP_PROFILE - The ActiveState/Teacup canonical name for this platform (i.e. win32-ix86 macosx10.5-i386-x86_84)
- TEACUP_OS - The local operating system (windows, macosx, openbsd, etc). Gives the same answer as tcl.m4, except that macosx is given as macosx instead of Darwin.
- TEA_PLATFORM - The platform returned by uname -s-uname -r (on Unix), or "windows" on Windows
- TEACUP_ARCH - The processor architecture for the local os (i.e. ix86, x86_64)
- TEACUP_ARCH - The processor architecture for the local os (i.e. ix86, x86_64)
- teapot - Filesystem path where teapot package files are downloaded for the current user
- userhome - File path to store localized preferences, cache download files, etc for the current user

This command uses a combination of local checks with Exec, any tclConfig.sh file that is resident, autoconf data where already computed, and data gleaned from a file named practcl.rc in userhome. The location for userhome varies by platform and operating system:
- Windows: ::env(LOCALAPPDATA)/Tcl
- Macos: ~/Library/Application Support/Tcl
- Other: ~/tcl

proc practcl::config.tcl *path*

A transparent call to ::practcl::read_configuration to preserve backward compadibility with older copies of Practcl

proc practcl::read_configuration *path*

Detect local platform. This command looks for data gleaned by autoconf or autosetup in the path specified, or perform its own logic tests if neither has been run. A file named config.site present in the location indicates that this project is cross compiling, and the data stored in that file is used for the compiler and linker.

This command looks for information from the following files, in the following order:

- config.tcl - A file generated by autoconf/configure in newer editions of TEA, encoded as a Tcl script.
- config.site - A file containing cross compiler information, encoded as a SH script
- ::env(VisualStudioVersion) - On Windows, and environmental value that indicates MS Visual Studio is installed

This command returns a dictionary containing all of the data cleaned from the sources above. In the absence of any guidance this command returns the same output as ::practcl::local_os. In this mode, if the environmental variable VisualStudioVersion exists, this command will provide a template of fields that are appropriate for compiling on Windows under Microsoft Visual Studio. The USEMSVC flag in the dictionary is a boolean flag to indicate if this is indeed the case.

proc practcl::tcllib_require *pkg* ?*args*?

Try to load a package, and failing that retrieve tcllib

proc practcl::platform::tcl_core_options *os*

Return the string to pass to ./configure to compile the Tcl core for the given OS.

- windows: --with-tzdata --with-encoding utf-8
- macosx: --enable-corefoundation=yes --enable-framework=no --with-tzdata --with-encoding utf-8
- other: --with-tzdata --with-encoding utf-8

proc practcl::platform::tk_core_options *os*

see below ...

proc practcl::read_rc_file *filename* ?*localdat* ?

Read a stylized key/value list stored in a file

proc practcl::read_sh_subst *line info*

Converts a XXX.sh file into a series of Tcl variables

proc practcl::read_sh_file *filename* ?*localdat* ?

see below ...

proc practcl::read_Config.sh *filename*

A simpler form of read_sh_file tailored to pulling data from (tcl|tk)Config.sh

proc practcl::read_Makefile *filename*

A simpler form of read_sh_file tailored to pulling data from a Makefile

proc practcl::cputs *varname* ?*args*?

Append arguments to a buffer The command works like puts in that each call will also insert a line feed. Unlike puts, blank links in the interstitial are suppressed

proc practcl::tcl_to_c *body*

see below ...

proc practcl::_tagblock *text* ?*style* tcl? ?*note* ?

see below ...

proc practcl::de_shell *data*

see below ...

proc practcl::grep *pattern* ?*files* ?

Search for the pattern *pattern* amongst $files

proc practcl::file_lexnormalize *sp*

see below ...

proc practcl::file_relative *base dst*

Calculate a relative path between base and dst

Example:

```
::practcl::file_relative ~/build/tcl/unix ~/build/tcl/library
> ../library
```

proc practcl::findByPattern *basedir patterns*

see below ...

proc practcl::log *fname comment*

Record an event in the practcl log

proc practcl::_pkgindex_simpleIndex *path*

see below ...

proc practcl::_pkgindex_directory *path*

Return true if the pkgindex file contains any statement other than "package ifneeded" and/or if any package ifneeded loads a DLL

proc practcl::_pkgindex_path_subdir *path*

Helper function for ::practcl::pkgindex_path

proc practcl::pkgindex_path ?*args*?

Index all paths given as though they will end up in the same virtual file system

proc practcl::installDir *d1 d2*

Delete the contents of *d2*, and then recusively Ccopy the contents of *d1* to *d2*.

proc practcl::copyDir *d1 d2* ?*toplevel* 1?

Recursively copy the contents of *d1* to *d2*

proc practcl::buildModule *modpath*

see below ...

proc practcl::installModule *modpath DEST*

Install a module from MODPATH to the directory specified. *dpath* is assumed to be the fully qualified path where module is to be placed. Any existing files will be deleted at that path. If the path is symlink the process will return with no error and no action. If the module has contents in the build/ directory that are newer than the .tcl files in the module source directory, and a build/build.tcl file exists, the build/build.tcl file is run. If the source directory includes a file named index.tcl, the directory is assumed to be in the tao style of modules, and the entire directory (and all subdirectories) are copied verbatim. If no index.tcl file is present, all .tcl files are copied from the module source directory, and a pkgIndex.tcl file is generated if non yet exists. I a folder named htdocs exists in the source directory, that directory is copied verbatim to the destination.

proc practcl::trigger ?*args*?

Trigger build targets, and recompute dependencies

Internals:

```
  ::practcl::LOCAL make trigger {*}$args
  foreach {name obj} [::practcl::LOCAL make objects] {
    set ::make($name) [$obj do]
  }
```

proc practcl::depends ?*args*?

Calculate if a dependency for any of the arguments needs to be fulfilled or rebuilt.

Internals:

```
  ::practcl::LOCAL make depends {*}$args
```

proc practcl::target *name info* ?*action* ?

Declare a build product. This proc is just a shorthand for *::practcl::LOCAL make task $name $info $action*

Registering a build product with this command will create an entry in the global array, and populate a value in the global array.

Internals:

```
  set obj [::practcl::LOCAL make task $name $info $action]
  set ::make($name) 0
  set filename [$obj define get filename]
  if {$filename ne {}} {
    set ::target($name) $filename
  }
```

Classes

Class practcl::doctool

```
{ set authors {
   {John Doe} {jdoe@illustrious.edu}
   {Tom RichardHarry} {tomdickharry@illustrius.edu}
 }
 # Create the object
 ::practcl::doctool create AutoDoc
 set fout [open [file join $moddir module.tcl] w]
 foreach file [glob [file join $srcdir *.tcl]] {
   set content [::practcl::cat [file join $srcdir $file]]
    # Scan the file
    AutoDoc scan_text $content
    # Strip the comments from the distribution
    puts $fout [::practcl::docstrip $content]
 }
 # Write out the manual page
 set manout [open [file join $moddir module.man] w]
 dict set args header [string map $modmap [::practcl::cat [file join
    $srcdir manual.txt]]]
```

```
 dict set args footer [string map $modmap [::practcl::cat [file join
    $srcdir footer.txt]]]
 dict set args authors $authors
 puts $manout [AutoDoc manpage {*}$args]
 close $manout


}
```

Tool for build scripts to dynamically generate manual files from comments in source code files

Methods

method constructor

see below ...

method argspec *argspec*

Process an argument list into an informational dict. This method also understands non-positional arguments expressed in the notation of Tip 471 https://core.tcl-lang.org/tips/doc/trunk/tip/479.md.

The output will be a dictionary of all of the fields and whether the fields are **positional**, **mandatory**, and whether they have a **default** value.

Example:

```
   my argspec {a b {c 10}}

   > a {positional 1 mandatory 1} b {positional 1 mandatory 1} c
    {positional 1 mandatory 0 default 10}
```

method comment *block*

Convert a block of comments into an informational dictionary. If lines in the comment start with a single word ending in a colon, all subsequent lines are appended to a dictionary field of that name. If no fields are given, all of the text is appended to the **description** field.

Example:

```
my comment {Does something cool}
> description {Does something cool}

my comment {
title : Something really cool
author : Sean Woods
author : John Doe
description :
This does something really cool!
}
> description {This does something really cool!}
  title {Something really cool}
  author {Sean Woods
  John Doe}
```

method keyword.Annotation *resultvar commentblock type name body*
see below ...

method keyword.Class *resultvar commentblock name body*
Process an oo::objdefine call that modifies the class object itself

method keyword.class *resultvar commentblock name body*
Process an oo::define, clay::define, etc statement.

method keyword.Class_Method *resultvar commentblock name* ?*args*?
Process a statement for a clay style class method

method keyword.method *resultvar commentblock name* ?*args*?
Process a statement for a tcloo style object method

method keyword.proc *commentblock name argspec*
Process a proc statement

method reset
Reset the state of the object and its embedded coroutine

method Main
Main body of the embedded coroutine for the object

method section.method *keyword method minfo*
Generate the manual page text for a method or proc

method section.annotation *type name iinfo*
see below ...

method section.class *class_name class_info*
Generate the manual page text for a class

method section.command *procinfo*
Generate the manual page text for the commands section

method manpage ?header *value*? ?footer *value*? ?authors *list*?
Generate the manual page. Returns the completed text suitable for saving in .man file. The header argument is a block of doctools text to go in before the machine generated section. footer is a block of doctools text to go in after the machine generated section. authors is a list of individual authors and emails in the form of AUTHOR EMAIL ?AUTHOR EMAIL?...

method scan_text *text*

Scan a block of text

method scan_file *filename*

Scan a file of text

Class practcl::metaclass

The metaclass for all practcl objects

Methods

- method **_MorphPatterns**
- method **define** *submethod* ?*args*?
- method **graft** ?*args*?
- method **initialize**
- method **link** *command* ?*args*?
- method **morph** *classname*
- method **script** *script*
- method **select**
- method **source** *filename*

Class practcl::toolset

Ancestor-less class intended to be a mixin which defines a family of build related behaviors that are modified when targetting either gcc or msvc

Class Methods

classmethod select *object*

Perform the selection for the toolset mixin

Methods

method config.sh

find or fake a key/value list describing this project

method BuildDir *PWD*

Compute the location where the product will be built

method MakeDir *srcdir*

Return where the Makefile is located relative to *srcdir*. For this implementation the MakeDir is always srcdir.

method read_configuration

Read information about the build process for this package. For this implementation, data is sought in the following locations in the following order: config.tcl (generated by practcl.) PKGConfig.sh. The Makefile

If the Makefile needs to be consulted, but does not exist, the Configure method is invoked

method build-cflags *PROJECT DEFS namevar versionvar defsvar*

method DEFS This method populates 4 variables: name - The name of the package version - The version of the package defs - C flags passed to the compiler includedir - A list of paths to feed to the compiler for finding headers

method critcl ?*args*?

Invoke critcl in an external process

Class practcl::toolset.gcc

ancestors: **practcl::toolset**

Methods

method Autoconf
see below ...

method BuildDir *PWD*
see below ...

method ConfigureOpts
see below ...

method MakeDir *srcdir*
Detect what directory contains the Makefile template

method make {} autodetect
see below ...

method make {} clean
see below ...

method make {} compile
see below ...

method make {} install *DEST*
see below ...

method build-compile-sources *PROJECT COMPILE CPPCOMPILE INCLUDES*
see below ...

method build-Makefile *path PROJECT*
see below ...

method build-library *outfile PROJECT*
Produce a static or dynamic library

method build-tclsh *outfile PROJECT* ?*path* auto?
Produce a static executable

Class practcl::toolset.msvc

ancestors: **practcl::toolset**

Methods

method BuildDir *PWD*
MSVC always builds in the source directory

method make {} autodetect
Do nothing

method make {} clean
see below ...

method make {} compile
see below ...

method make {} install *DEST*
see below ...

method MakeDir *srcdir*
Detect what directory contains the Makefile template

- method **NmakeOpts**

Class practcl::make_obj

ancestors: **practcl::metaclass**

A build deliverable object. Normally an object file, header, or tcl script which must be compiled or generated in some way

Methods

- method **constructor** *module_object name info* ?*action_body* ?
- method **do**
- method **check**
- method **output**
- method **reset**
- method **triggers**

Class practcl::object

ancestors: **practcl::metaclass**

A generic Practcl object

Methods

- method **constructor** *parent* ?*args*?
- method **child** *method*
- method **go**

Class practcl::dynamic

Dynamic blocks do not generate their own .c files, instead the contribute to the amalgamation of the main library file

Methods

method cstructure *name definition* ?*argdat* ?
Parser functions

method include *header*
see below ...

method include_dir ?*args*?
see below ...

method include_directory ?*args*?
see below ...

method c_header *body*
see below ...

method c_code *body*
see below ...

method c_function *header body* ?*info* ?
see below ...

method c_tcloomethod *name body* ?*arginfo* ?
see below ...

method cmethod *name body* ?*arginfo* ?
Alias to classic name

method c_tclproc_nspace *nspace*
see below ...
method c_tclcmd *name body* ?*arginfo* ?
see below ...
method c_tclproc_raw *name body* ?*arginfo* ?
Alias to classic name
method tcltype *name argdat*
see below ...
method project-compile-products
Module interactions
method implement *path*
see below ...
method initialize
Practcl internals
method linktype
see below ...
method generate-cfile-constant
see below ...
method generate-cfile-header
see below ...
method generate-cfile-tclapi
Generate code that provides implements Tcl API calls
method generate-loader-module
Generate code that runs when the package/module is initialized into the interpreter
method Collate_Source *CWD*
see below ...
method select
Once an object marks itself as some flavor of dynamic, stop trying to morph it into something else

Class practcl::product

A deliverable for the build system

Class Methods

- classmethod **select** *object*

Methods

method code *section body*
see below ...
method Collate_Source *CWD*
see below ...
method project-compile-products
see below ...
method generate-debug ?*spaces* ?
see below ...

method generate-cfile-constant
see below ...
method generate-cfile-public-structure
Populate const static data structures
method generate-cfile-header
see below ...
method generate-cfile-global
see below ...
method generate-cfile-private-typedef
see below ...
method generate-cfile-private-structure
see below ...
method generate-cfile-functions
Generate code that provides subroutines called by Tcl API methods
method generate-cfile-tclapi
Generate code that provides implements Tcl API calls
method generate-hfile-public-define
see below ...
method generate-hfile-public-macro
see below ...
method generate-hfile-public-typedef
see below ...
method generate-hfile-public-structure
see below ...
method generate-hfile-public-headers
see below ...
method generate-hfile-public-function
see below ...
method generate-hfile-public-includes
see below ...
method generate-hfile-public-verbatim
see below ...
method generate-loader-external
see below ...
method generate-loader-module
see below ...
method generate-stub-function
see below ...
method IncludeAdd *headervar* ?*args*?
see below ...
method generate-tcl-loader
see below ...
method generate-tcl-pre

This methods generates any Tcl script file which is required to pre-initialize the C library

method generate-tcl-post

see below ...

method linktype

see below ...

method Ofile *filename*

see below ...

method project-static-packages

Methods called by the master project

method toolset-include-directory

Methods called by the toolset

- method **target** *method* ?*args*?

Class practcl::product.cheader

ancestors: **practcl::product**

A product which generated from a C header file. Which is to say, nothing.

Methods

- method **project-compile-products**
- method **generate-loader-module**

Class practcl::product.csource

ancestors: **practcl::product**

A product which generated from a C source file. Normally an object (.o) file.

Methods

- method **project-compile-products**

Class practcl::product.clibrary

ancestors: **practcl::product**

A product which is generated from a compiled C library. Usually a .a or a .dylib file, but in complex cases may actually just be a conduit for one project to integrate the source code of another

Methods

- method **linker-products** *configdict*

Class practcl::product.dynamic

ancestors: **practcl::dynamic practcl::product**

A product which is generated from C code that itself is generated by practcl or some other means. This C file may or may not produce its own .o file, depending on whether it is eligible to become part of an amalgamation

Methods

- method **initialize**

Class practcl::product.critcl

ancestors: **practcl::dynamic practcl::product**

A binary product produced by critcl. Note: The implementation is not written yet, this class does nothing.

Class practcl::module

ancestors: **practcl::object practcl::product.dynamic**

In the end, all C code must be loaded into a module This will either be a dynamically loaded library implementing a tcl extension, or a compiled in segment of a custom shell/app

Variable

- variable **make_object**

Methods

method _MorphPatterns
see below ...

method add ?*args*?
see below ...

method install-headers ?*args*?
see below ...

method make {} _preamble
see below ...

method make {} pkginfo
see below ...

method make {} objects
Return a dictionary of all handles and associated objects

method make {} object *name*
Return the object associated with handle *name*

method make {} reset
Reset all deputy objects

method make {} trigger ?*args*?
Exercise the triggers method for all handles listed

method make {} depends ?*args*?
Exercise the check method for all handles listed

method make {} filename *name*
Return the file name of the build product for the listed handle

method make {} target *name Info body*
see below ...

method make {} todo
Return a list of handles for object which return true for the do method

method make {} do
For each target exercise the action specified in the *action* definition if the *do* method returns true

method child *which*
see below ...

method generate-c

This methods generates the contents of an amalgamated .c file which implements the loader for a batch of tools

method generate-h

This methods generates the contents of an amalgamated .h file which describes the public API of this module

- method **generate-loader**
- method **initialize**
- method **implement** *path*
- method **linktype**

Class practcl::project

ancestors: **practcl::module**

A toplevel project that is a collection of other projects

Methods

method _MorphPatterns

see below ...

method constructor ?*args*?

see below ...

method add_object *object*

see below ...

method add_project *pkg info* ?*oodefine* ?

see below ...

method add_tool *pkg info* ?*oodefine* ?

see below ...

method build-tclcore

Compile the Tcl core. If the define *tk* is true, compile the Tk core as well

method child *which*

see below ...

method linktype

see below ...

method project *pkg* ?*args*?

Exercise the methods of a sub-object

- method **tclcore**
- method **tkcore**
- method **tool** *pkg* ?*args*?

Class practcl::library

ancestors: **practcl::project**

A toplevel project that produces a library

Methods

method clean *PATH*

see below ...

method project-compile-products

see below ...

method go
see below ...

method generate-decls *pkgname path*
see below ...

method implement *path*
see below ...

method generate-make *path*
Backward compadible call

method linktype
see below ...

method package-ifneeded ?*args*?
Create a "package ifneeded" Args are a list of aliases for which this package will answer to

- method **shared_library** ?*filename* ?
- method **static_library** ?*filename* ?

Class practcl::tclkit

ancestors: **practcl::library**

A toplevel project that produces a self-contained executable

Methods

method build-tclkit_main *PROJECT PKG_OBJS*
see below ...

method Collate_Source *CWD*
see below ...

method wrap *PWD exename vfspath ?args?*
Wrap an executable

Class practcl::distribution

Standalone class to manage code distribution This class is intended to be mixed into another class (Thus the lack of ancestors)

Class Methods

- classmethod **Sandbox** *object*
- classmethod **select** *object*
- classmethod **claim_option**
- classmethod **claim_object** *object*
- classmethod **claim_path** *path*

Methods

- method **scm_info**
- method **DistroMixIn**
- method **Sandbox**
- method **SrcDir**
- method **ScmTag**
- method **ScmClone**

- method **ScmUnpack**
- method **ScmUpdate**
- method **Unpack**

Class practcl::distribution.snapshot

ancestors: **practcl::distribution**

A file distribution from zip, tarball, or other non-scm archive format

Class Methods

- classmethod **claim_object** *object*
- classmethod **claim_option**
- classmethod **claim_path** *path*

Methods

- method **ScmUnpack**

Class practcl::distribution.fossil

ancestors: **practcl::distribution**

A file distribution based on fossil

Class Methods

classmethod claim_object *obj*
Check for markers in the metadata

classmethod claim_option
see below ...

classmethod claim_path *path*
Check for markers in the source root

Methods

method scm_info
see below ...

method ScmClone
Clone the source

- method **ScmTag**
- method **ScmUnpack**
- method **ScmUpdate**

Class practcl::distribution.git

ancestors: **practcl::distribution**

A file distribution based on git

Class Methods

- classmethod **claim_object** *obj*
- classmethod **claim_option**
- classmethod **claim_path** *path*

Methods

- method **ScmTag**

- method **ScmUnpack**
- method **ScmUpdate**

Class practcl::subproject

ancestors: **practcl::module**

A subordinate project

Methods

method _MorphPatterns
see below ...

method BuildDir *PWD*
see below ...

method child *which*
see below ...

method compile
see below ...

method go
see below ...

method install ?*args*?
Install project into the local build system

method linktype
see below ...

method linker-products *configdict*
see below ...

method linker-external *configdict*
see below ...

method linker-extra *configdict*
see below ...

method env-bootstrap
Methods for packages/tools that can be downloaded possibly built and used internally by this Practcl process Load the facility into the interpreter

method env-exec
Return a file path that exec can call

method env-install
Install the tool into the local environment

method env-load
Do whatever is necessary to get the tool into the local environment

method env-present
Check if tool is available for load/already loaded

- method **sources**
- method **update**
- method **unpack**

Class practcl::subproject.source

ancestors: **practcl::subproject practcl::library**

A project which the kit compiles and integrates the source for itself

Methods

- method **env-bootstrap**
- method **env-present**
- method **linktype**

Class practcl::subproject.teapot

ancestors: **practcl::subproject**

a copy from the teapot

Methods

- method **env-bootstrap**
- method **env-install**
- method **env-present**
- method **install** *DEST*

Class practcl::subproject.kettle

ancestors: **practcl::subproject**

Methods

- method **kettle** *path* ?*args*?
- method **install** *DEST*

Class practcl::subproject.critcl

ancestors: **practcl::subproject**

Methods

- method **install** *DEST*

Class practcl::subproject.sak

ancestors: **practcl::subproject**

Methods

- method **env-bootstrap**
- method **env-install**
- method **env-present**
- method **install** *DEST*
- method **install-module** *DEST* ?*args*?

Class practcl::subproject.practcl

ancestors: **practcl::subproject**

Methods

- method **env-bootstrap**
- method **env-install**
- method **install** *DEST*
- method **install-module** *DEST* ?*args*?

Class practcl::subproject.binary

ancestors: **practcl::subproject**

A subordinate binary package

Methods

- method **clean**
- method **env-install**
- method **project-compile-products**
- method **ComputeInstall**
- method **go**
- method **linker-products** *configdict*
- method **project-static-packages**
- method **BuildDir** *PWD*
- method **compile**
- method **Configure**
- method **install** *DEST*

Class practcl::subproject.TEA

ancestors: **practcl::subproject.binary**

A subordinate TEA based binary package

Class practcl::subproject.library

ancestors: **practcl::subproject.binary practcl::library**

A subordinate C library built by this project

Methods

- method **install** *DEST*

Class practcl::subproject.external

ancestors: **practcl::subproject.binary**

A subordinate external C library

Methods

- method **install** *DEST*

Class practcl::subproject.core

ancestors: **practcl::subproject.binary**

Methods

- method **env-bootstrap**
- method **env-present**
- method **env-install**
- method **go**
- method **linktype**

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *practcl* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
TclOO

Copyright

```
Copyright (c) 2016-2018 Sean Woods <yoda@etoyoc.com>
```


processman

processman
Keywords: processman, odie

Name
processman - Tool for automating the period callback of commands

Synopsis
package require Tcl 8.5 9
package require twapi 3.1
package require cron 1.1
package require processman ?**0.8**?
::processman::find_exe *name*
::processman::kill *id*
::processman::kill_all
::processman::killexe *name*
::processman::onexit *id cmd*
::processman::priority *id level*
::processman::process_list
::processman::process_list *id*
::processman::spawn *id cmd args*

Description
The **processman** package provides a Pure-tcl set of utilities to manage child processes in a platform-generic nature.

Commands
::processman::find_exe *name*
Locate an executable by the name of *name* in the system path. On windows, also add the .exe extention if not given.

::processman::kill *id*
Kill a child process *id*.

::processman::kill_all
Kill all processes spawned by this program

::processman::killexe *name*
Kill a process identified by the executable. On Unix, this triggers a killall. On windows, **twapi::get_process_ids** is used to map a name one or more IDs, which are then killed.

::processman::onexit *id cmd*
Arrange to execute the script *cmd* when this programe detects that process *id* as terminated.

::processman::priority *id level*
Mark process *id* with the priorty *level*. Valid levels: low, high, default.

On Unix, the process is tagged using the **nice** command.

On Windows, the process is modifed via the **twapi::set_priority_class**

::processman::process_list
Return a list of processes that have been triggered by this program, as well as a boolean flag to indicate if the process is still running.

::processman::process_list *id*
Return true if process *id* is still running, false otherwise.

::processman::spawn *id cmd args*
Start a child process, identified by *id*. *cmd* is the name of the command to execute. *args* are arguments to pass to that command.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *odie* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

System

Copyright

```
Copyright (c) 2015 Sean Woods <yoda@etoyoc.com>
```

profiler

profiler

Keywords: profiler, performance, profile, speed

Name

profiler - Tcl source code profiler

Synopsis

package require Tcl 8.5 9

package require profiler ?**0.7**?
::profiler::init
::profiler::dump *pattern*
::profiler::print ?*pattern*?
::profiler::reset ?*pattern*?
::profiler::suspend ?*pattern*?
::profiler::resume ?*pattern*?
::profiler::new-disabled
::profiler::new-enabled
::profiler::sortFunctions *key*

Description

The **profiler** package provides a simple Tcl source code profiler. It is a function-level profiler; that is, it collects only function-level information, not the more detailed line-level information. It operates by redefining the Tcl **proc** command. Profiling is initiated via the **::profiler::init** command.

Commands

::profiler::init
Initiate profiling. All procedures created after this command is called will be profiled. To profile an entire application, this command must be called before any other commands.

::profiler::dump *pattern*
Dump profiling information for the all functions matching *pattern*. If no pattern is specified, information for all functions will be returned. The result is a list of key/value pairs that maps function names to information about that function. The information about each function is in turn a list of key/value pairs. The keys used and their values are:

totalCalls
The total number of times *functionName* was called.

callerDist
A list of key/value pairs mapping each calling function that called *functionName* to the number of times it called *functionName*.

compileTime
The runtime, in clock clicks, of *functionName* the first time that it was called.

totalRuntime
The sum of the runtimes of all calls of *functionName*.

averageRuntime
Average runtime of *functionName*.

descendantTime
Sum of the time spent in descendants of *functionName*.

averageDescendantTime
Average time spent in descendants of *functionName*.

::profiler::print ?*pattern*?
Print profiling information for all functions matching *pattern*. If no pattern is specified, information about all functions will be displayed. The return result is a human readable display of the profiling information.

::profiler::reset ?*pattern*?
Reset profiling information for all functions matching *pattern*. If no pattern is specified, information will be reset for all functions.

::profiler::suspend ?*pattern*?
Suspend profiling for all functions matching *pattern*. If no pattern is specified, profiling will be suspended for all functions. It stops gathering profiling information after this command is issued. However, it does not erase any profiling information that has been gathered previously. Use resume command to re-enable profiling.

::profiler::resume ?*pattern*?
Resume profiling for all functions matching *pattern*. If no pattern is specified, profiling will be resumed for all functions. This command should be invoked after suspending the profiler in the code.

::profiler::new-disabled
Change the initial profiling state for new procedures. Invoking this command disables profiling for all procedures created after this command until **new-enabled** is invoked. Activate profiling of specific procedures via **resume**.

::profiler::new-enabled
Change the initial profiling state for new procedures. Invoking this command enables profiling for all procedures created after this command until **new-disabled** is invoked. Prevent profiling of specific procedures via **suspend**.

::profiler::sortFunctions *key*
Return a list of functions sorted by a particular profiling statistic. Supported values for *key* are: **calls**, **exclusiveTime**, **compileTime**, **nonCompileTime**, **totalRuntime**, **avgExclusiveTime**, and **avgRuntime**. The return result is a list of lists, where each sublist consists of a function name and the value of *key* for that function.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *profiler* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Programming tools

pt

pt_astree

Keywords: pt_astree, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt::ast - Abstract Syntax Tree Serialization

Synopsis

package require Tcl 8.5 9
package require pt::ast ?**1.2**?
::pt::ast verify *serial* ?*canonvar*?
::pt::ast verify-as-canonical *serial*
::pt::ast canonicalize *serial*
::pt::ast print *serial*
::pt::ast bottomup *cmdprefix ast*
cmdprefix *ast*
::pt::ast topdown *cmdprefix pe*
::pt::ast equal *seriala serialb*
::pt::ast new0 *s loc* ?*child...*?
::pt::ast new *s start end* ?*child...*?

Description

Are you lost ? Do you have trouble understanding this document ? In that case please read the overview provided by the *Introduction to Parser Tools*. This document is the entrypoint to the whole system the current package is a part of.

This package provides commands to work with the serializations of abstract syntax trees as managed by the Parser Tools, and specified in section Ast serialization format.

This is a supporting package in the Core Layer of Parser Tools.

IMAGE: arch_core_support

API

::pt::ast verify *serial* ?*canonvar*?

This command verifies that the content of *serial* is a valid serialization of an abstract syntax tree and will throw an error if that is not the case. The result of the command is the empty string.

If the argument *canonvar* is specified it is interpreted as the name of a variable in the calling context. This variable will be written to if and only if *serial* is a valid regular serialization. Its value will be a boolean, with **True** indicating that the serialization is not only valid, but also *canonical*. **False** will be written for a valid, but non-canonical serialization.

For the specification of serializations see the section Ast serialization format.

::pt::ast verify-as-canonical *serial*

This command verifies that the content of *serial* is a valid *canonical* serialization of an abstract syntax tree and will throw an error if that is not the case. The result of the command is the empty string.

For the specification of canonical serializations see the section Ast serialization format.

::pt::ast canonicalize *serial*

This command assumes that the content of *serial* is a valid *regular* serialization of an abstract syntax and will throw an error if that is not the case.

It will then convert the input into the *canonical* serialization of the contained tree and return it as its result. If the input is already canonical it will be returned unchanged.

For the specification of regular and canonical serializations see the section Ast serialization format.

::pt::ast print *serial*

This command assumes that the argument *serial* contains a valid serialization of an abstract syntax tree and returns a string containing that tree in a human readable form.

The exact format of this form is not specified and cannot be relied on for parsing or other machine-based activities.

For the specification of serializations see the section Ast serialization format.

::pt::ast bottomup *cmdprefix ast*

This command walks the abstract syntax tree *ast* from the bottom up to the root, invoking the command prefix *cmdprefix* for each node. This implies that the children of a node N are handled before N.

The command prefix has the signature

cmdprefix *ast*

I.e. it is invoked with the ast node the walk is currently at.

The result returned by the command prefix replaces *ast* in the node it was a child of, allowing transformations of the tree.

This also means that for all inner node the contents of the children elements are the results of the command prefix invoked for the children of this node.

::pt::ast topdown *cmdprefix pe*

This command walks the abstract syntax tree *ast* from the root down to the leaves, invoking the command prefix *cmdprefix* for each node. This implies that the children of a node N are handled after N.

The command prefix has the same signature as for **bottomup**, see above.

The result returned by the command prefix is *ignored*.

::pt::ast equal *seriala serialb*

This command tests the two sbstract syntax trees *seriala* and *serialb* for structural equality. The result of the command is a boolean value. It will be set to **true** if the trees are identical, and **false** otherwise.

String equality is usable only if we can assume that the two trees are pure Tcl lists.

::pt::ast new0 *s loc ?child...?*

This command command constructs the ast for a nonterminal node refering refering to the symbol *s* at position *loc* in the input, and the set of child nodes *child* ..., from left right. The latter may be empty. The constructed node is returned as the result of the command. The end position is *loc*-1, i.e. one character before the start. This type of node is possible for rules containing optional parts.

::pt::ast new *s start end* ?*child*...?

This command command constructs the ast for a nonterminal node refering to the symbol *s* covering the range of positions *start* to *end* in the input, and the set of child nodes *child* ..., from left right. The latter may be empty. The constructed node is returned as the result of the command.

Ast serialization format

Here we specify the format used by the Parser Tools to serialize Abstract Syntax Trees (ASTs) as immutable values for transport, comparison, etc.

Each node in an AST represents a nonterminal symbol of a grammar, and the range of tokens/characters in the input covered by it. ASTs do not contain terminal symbols, i.e. tokens/characters. These can be recovered from the input given a symbol's location.

We distinguish between *regular* and *canonical* serializations. While a tree may have more than one regular serialization only exactly one of them will be *canonical*.

Regular serialization

1. The serialization of any AST is the serialization of its root node.
1. The serialization of any node is a Tcl list containing at least three elements.
    1. The first element is the name of the nonterminal symbol stored in the node.
    2. The second and third element are the locations of the first and last token in the token stream the node represents (covers).
        1. Locations are provided as non-negative integer offsets from the beginning of the token stream, with the first token found in the stream located at offset 0 (zero).
        2. The end location has to be equal to or larger than the start location.

1. All elements after the first three represent the children of the node, which are themselves nodes. This means that the serializations of nodes without children, i.e. leaf nodes, have exactly three elements. The children are stored in the list with the leftmost child first, and the rightmost child last.

Canonical serialization

The canonical serialization of an abstract syntax tree has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this tree.

1. The string representation of the value is the canonical representation of a pure Tcl list. I.e. it does not contain superfluous whitespace.

Example

Assuming the parsing expression grammar below

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

and the input string

```
 120+5
```

then a parser should deliver the abstract syntax tree below (except for whitespace)

```
set ast {Expression 0 4
    {Factor 0 4
        {Term 0 2
            {Number 0 2
                {Digit 0 0}
                {Digit 1 1}
                {Digit 2 2}
            }
        }
        {AddOp 3 3}
        {Term 4 4
            {Number 4 4
                {Digit 4 4}
            }
        }
    }
}
```

Or, more graphical

```
+- Digit 0 0 | 1
|            |
+- Term 0 2 --- Number 0 2 -+- Digit 1 1 | 2
|                           |            |
|                           +- Digit 2 2 | 0
|                                        |
Expression 0 4 --- Factor 0 4 -+----------------------------- AddOp 3
    3 | +
|                                        |
+- Term 4 4 --- Number 4 4 --- Digit 4 4 | 5
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pt* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Parsing and Grammars

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_cparam_config_critcl

Keywords: pt_cparam_config_critcl, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt::cparam::configuration::critcl - C/PARAM, Canned configuration, Critcl

Synopsis

package require Tcl 8.5 9
package require pt::cparam::configuration::critcl ?**1.0.3**?
::pt::cparam::configuration::critcl def *name pkg version cmdprefix*

Description

Are you lost ? Do you have trouble understanding this document ? In that case please read the overview provided by the *Introduction to Parser Tools*. This document is the entrypoint to the whole system the current package is a part of.

This package is an adjunct to pt::peg::to::cparam, to make the use of this highly configurable package easier by providing a canned configuration. When applied this configuration causes the package pt::peg::to::cparam to generate **critcl**-based parser packages.

It is a supporting package in the Core Layer of Parser Tools.

IMAGE: arch_core_support

API

::pt::cparam::configuration::critcl def *name pkg version cmdprefix*

The command applies the configuration provided by this package to the *cmdprefix*, causing the creation of **critcl**-based parsers whose class is *name*, in package *pkg* with *version*.

The use of a command prefix as API allows application of the configuration to not only

pt::peg::to::cparam (pt::peg::to::cparam configure), but also export manager instances and PEG containers (**$export configuration set** and **[$container exporter] configuration set** respectively).

Or anything other command prefix accepting two arguments, option and value.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pt* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Parsing and Grammars

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_cparam_config_tea

Keywords: pt_cparam_config_tea, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt::cparam::configuration::tea - C/PARAM, Canned configuration, TEA

Synopsis

package require Tcl 8.5 9
package require pt::cparam::configuration::tea ?**0.2**?
::pt::cparam::configuration::tea def *name pkg version cmdprefix*

Description

Are you lost ? Do you have trouble understanding this document ? In that case please read the overview provided by the *Introduction to Parser Tools*. This document is the entrypoint to the whole system the current package is a part of.

This package is an adjunct to pt::peg::to::cparam, to make the use of this highly configurable package easier by providing a canned configuration. When applied this configuration causes the package pt::peg::to::cparam to generate plain parser code ready for inclusion into a *TEA*-based C extension.

It is a supporting package in the Core Layer of Parser Tools.

IMAGE: arch_core_support

API

::pt::cparam::configuration::tea def *name pkg version cmdprefix*

The command applies the configuration provided by this package to the *cmdprefix*, causing the creation of **tea**-based parsers whose class is *name*, in package *pkg* with *version*.

The use of a command prefix as API allows application of the configuration to not only pt::peg::to::cparam (pt::peg::to::cparam configure), but also export manager instances and PEG containers (**$export configuration set** and **[$container exporter] configuration set** respectively).

Or anything other command prefix accepting two arguments, option and value.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pt* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Parsing and Grammars

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_from_api

Keywords: pt_from_api, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt_import_api - Parser Tools Import API

Synopsis

package require Tcl 8.5 9
CONVERTER convert *text*
IncludeFile *currentfile path*
::import *text*

Description

Are you lost ? Do you have trouble understanding this document ? In that case please read the overview provided by the *Introduction to Parser Tools*. This document is the entrypoint to the whole system the current package is a part of.

This document describes two APIs. First the API shared by all packages for the conversion of some other format into Parsing Expression Grammars, and then the API shared by the packages which implement the import plugins sitting on top of the conversion packages.

Its intended audience are people who wish to create their own converter for some type of input, and/or an import plugin for their or some other converter.

It resides in the Import section of the Core Layer of Parser Tools.

IMAGE: arch_core_import

Converter API

Any (grammar) import converter has to follow the rules set out below:

1. A converter is a package. Its name is arbitrary, however it is recommended to put it under the **::pt::peg::from** namespace.
2. The package provides either a single Tcl command following the API outlined below, or a class command whose instances follow the same API. The commands which follow the API are called *converter commands*.
3. A converter command has to provide the following single method with the given signature and semantic. Converter commands are allowed to provide more methods of their own, but not less, and they may not provide different semantics for the standardized method.

   CONVERTER convert *text*

   This method has to accept some *text*, a parsing expression grammar in some format. The result of the method has to be the canonical serialization of a parsing expression grammar, as specified in section Peg serialization format, the result of reading and converting the input text.

Plugin API

Any (grammar) import plugin has to follow the rules set out below:

1. A plugin is a package.
2. The name of a plugin package has the form pt::peg::import::**FOO**, where **FOO** is the name of the format the plugin will accept input for.
3. The plugin can expect that the package **pt::peg::import::plugin** is present, as indicator that it was invoked from a genuine plugin manager.

   It is recommended that a plugin does check for the presence of this package.
4. The plugin can expect that a command named **IncludeFile** is present, with the signature

   IncludeFile *currentfile path*

   This command has to be invoked by the plugin when it has to process an included file, if the format has the concept of such.

   The plugin has to supply the following arguments

   string *currentfile*

   The path of the file it is currently processing. This may be the empty string if no such is known.

   string *path*

   The path of the include file as specified in the include directive being processed.

```
    The result of the command will be a 5-element list containing

    1. A boolean flag indicating the success (**True**) or failure
    (**False**) of the operation.

    2. In case of success the contents of the included file, and the
    empty string otherwise.

    3. The resolved, i.e. absolute path of the included file, if
    possible, or the unchanged *path* argument. This is for display in
    an error message, or as the *currentfile* argument of another call
    to **IncludeFile** should this file contain more files.

    4. In case of success an empty string, and for failure a code
    indicating the reason for it, one of

        notfound
        : The specified file could not be found.

        notread
        : The specified file was found, but not be read into memory.


    5. An empty string in case of success of a **notfound** failure,
    and an additional error message describing the reason for a
    **notread** error in more detail.
```

1. A plugin has to provide a single command, in the global namespace, with the signature shown below. Plugins are allowed to provide more commands of their own, but not less, and they may not provide different semantics for the standardized command.

   ::import *text*

   This command has to accept the a text containing a parsing expression grammar in some format. The result of the command has to be the result of the converter invoked by the plugin for the input grammar, the canonical serialization of the parsing expression grammar contained in the input.

   string *text*

   This argument will contain the parsing expression grammar for which to generate the serialization. The specification of what a *canonical* serialization is can be found in the section Peg serialization format.

1. A single usage cycle of a plugin consists of an invokation of the command import. This call has to leave the plugin in a state where another usage cycle can be run without problems.

Usage

To use a converter do

```
    # Get the converter (single command here, not class)
    package require the-converter-package

    # Perform the conversion
    set serial [theconverter convert $thegrammartext]

    ... process the result ...
```

To use a plugin **FOO** do

```
    # Get an import plugin manager
    package require pt::peg::import
    pt::peg::import I

    # Run the plugin, and the converter inside.
    set serial [I import serial $thegrammartext FOO]

    ... process the result ...
```

Peg serialization format

Here we specify the format used by the Parser Tools to serialize Parsing Expression Grammars as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a PEG may have more than one regular serialization only exactly one of them will be *canonical*.

regular serialization

1. The serialization of any PEG is a nested Tcl dictionary.
1. This dictionary holds a single key, **pt::grammar::peg**, and its value. This value holds the contents of the grammar.
2. The contents of the grammar are a Tcl dictionary holding the set of nonterminal symbols and the starting expression. The relevant keys and their values are

   rules

   The value is a Tcl dictionary whose keys are the names of the nonterminal symbols known to the grammar.

   1. Each nonterminal symbol may occur only once.
   2. The empty string is not a legal nonterminal symbol.
   3. The value for each symbol is a Tcl dictionary itself. The relevant keys and their values in this dictionary are

      is

      The value is the serialization of the parsing expression describing the symbols sentennial structure, as specified in the section PE serialization format.

      mode

      The value can be one of three values specifying how a parser should handle the semantic value produced by the symbol.

      value

      The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal itself, which has the ASTs of

the symbol's right hand side as its children.

leaf
The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal, without any children. Any ASTs generated by the symbol's right hand side are discarded.

void
The nonterminal has no semantic value. Any ASTs generated by the symbol's right hand side are discarded (as well).

```
**start**
: The value is the serialization of the start parsing expression of
    the grammar, as specified in the section [PE serialization
    format].
```

1. The terminal symbols of the grammar are specified implicitly as the set of all terminal symbols used in the start expression and on the RHS of the grammar rules.

canonical serialization
The canonical serialization of a grammar has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this grammar.

1. The keys found in all the nested Tcl dictionaries are sorted in ascending dictionary order, as generated by Tcl's builtin command **lsort -increasing -dict**.
2. The string representation of the value is the canonical representation of a Tcl dictionary. I.e. it does not contain superfluous whitespace.

Example

Assuming the following PEG for simple mathematical expressions

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

then its canonical serialization (except for whitespace) is

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                mode value}
```

```
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}                mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                   mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                   mode value}
        MulOp      {is {/ {t *} {t /}}
                                   mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
                                   mode value}
        Sign       {is {/ {t -} {t +}}
                                   mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                   mode value}
    }
    start {n Expression}
}
```

PE serialization format

Here we specify the format used by the Parser Tools to serialize Parsing Expressions as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a parsing expression may have more than one regular serialization only exactly one of them will be *canonical*.

Regular serialization

see below ...

Atomic Parsing Expressions

1. The string **epsilon** is an atomic parsing expression. It matches the empty string.
1. The string **dot** is an atomic parsing expression. It matches any character.
2. The string **alnum** is an atomic parsing expression. It matches any Unicode alphabet or digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
3. The string **alpha** is an atomic parsing expression. It matches any Unicode alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
4. The string **ascii** is an atomic parsing expression. It matches any Unicode character below U0080. This is a custom extension of PEs based on Tcl's builtin command **string is**.
5. The string **control** is an atomic parsing expression. It matches any Unicode control character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
6. The string **digit** is an atomic parsing expression. It matches any Unicode digit character. Note that this includes characters outside of the [0..9] range. This is a custom extension of PEs based on Tcl's builtin command **string is**.
7. The string **graph** is an atomic parsing expression. It matches any Unicode printing character, except for space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
8. The string **lower** is an atomic parsing expression. It matches any Unicode lower-case

alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.

9. The string **print** is an atomic parsing expression. It matches any Unicode printing character, including space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
10. The string **punct** is an atomic parsing expression. It matches any Unicode punctuation character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
11. The string **space** is an atomic parsing expression. It matches any Unicode space character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
12. The string **upper** is an atomic parsing expression. It matches any Unicode upper-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
13. The string **wordchar** is an atomic parsing expression. It matches any Unicode word character. This is any alphanumeric character (see alnum), and any connector punctuation characters (e.g. underscore). This is a custom extension of PEs based on Tcl's builtin command **string is**.
14. The string **xdigit** is an atomic parsing expression. It matches any hexadecimal digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
15. The string **ddigit** is an atomic parsing expression. It matches any decimal digit character. This is a custom extension of PEs based on Tcl's builtin command **regexp**.
16. The expression [list t **x**] is an atomic parsing expression. It matches the terminal string **x**.
17. The expression [list n **A**] is an atomic parsing expression. It matches the nonterminal **A**.

Combined Parsing Expressions

1. For parsing expressions **e1**, **e2**, ... the result of [list / **e1 e2** ...] is a parsing expression as well. This is the *ordered choice*, aka *prioritized choice*.
1. For parsing expressions **e1**, **e2**, ... the result of [list x **e1 e2** ...] is a parsing expression as well. This is the *sequence*.
2. For a parsing expression **e** the result of [list * **e**] is a parsing expression as well. This is the *kleene closure*, describing zero or more repetitions.
3. For a parsing expression **e** the result of [list + **e**] is a parsing expression as well. This is the *positive kleene closure*, describing one or more repetitions.
4. For a parsing expression **e** the result of [list & **e**] is a parsing expression as well. This is the *and lookahead predicate*.
5. For a parsing expression **e** the result of [list! **e**] is a parsing expression as well. This is the *not lookahead predicate*.
6. For a parsing expression **e** the result of [list ? **e**] is a parsing expression as well. This is the *optional input*.

Canonical serialization

The canonical serialization of a parsing expression has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this parsing expression.

1. The string representation of the value is the canonical representation of a pure Tcl list. I.e. it does not contain superfluous whitespace.

2. Terminals are *not* encoded as ranges (where start and end of the range are identical).

Example

Assuming the parsing expression shown on the right-hand side of the rule

```
    Expression <- Term (AddOp Term)*
```

then its canonical serialization (except for whitespace) is

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pt* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Parsing and Grammars

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_introduction

Keywords: pt_introduction, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar , matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt_introduction - Introduction to Parser Tools

Synopsis

package require Tcl 8.5 9

Description

Welcome to the Parser Tools, a system for the creation and manipulation of parsers and the grammars driving them.

What are your goals which drove you here ?

1. Do you simply wish to create a parser for some language ?

   In that case have a look at our parser generator application, **pt**, or, for a slightly deeper access, the package underneath it, **pt::pgen**.
2. Do you wish to know more about the architecture of the system ?

This is described in the section Parser Tools architecture, below

3. Is your interest in the theoretical background upon which the packages and tools are build ?

   See the *Introduction to Parsing Expression Grammars*.

Parser Tools architecture

The system can be split into roughly three layers, as seen in the figure below

IMAGE: architecture

These layers are, from high to low:

1. At the top we have the application and the packages using the packages of the layer below to implement common usecases. One example is the aforementioned **pt::pgen** which provides a parser generator.

   The list of packages belonging to this layer can be found in section User packages
2. In this layer we have the packages which provide the core of the functionality for the whole system. They are, in essence, a set of blocks which can be combined in myriad ways, like Lego (tm). The packages in the previous level are 'just' pre-fabricated combinations to cover the most important use cases.

   The list of packages belonging to this layer can be found in section Core packages
3. Last, but not least is the layer containing support packages providing generic functionality which not necessarily belong into the module.

   The list of packages belonging to this layer can be found in section Support packages

User packages

- **pt::pgen**

Core packages

This layer is further split into six sections handling the storage, import, export, transformation, and execution of grammars, plus grammar specific support packages.

Storage

see below ...

- **pt::peg::container**

Export

see below ...

- **pt::peg::export**
- **pt::peg::export::container**
- **pt::peg::export::json**
- **pt::peg::export::peg**
- pt::peg::to::container
- pt::peg::to::json
- pt::peg::to::peg
- pt::peg::to::param
- pt::peg::to::tclparam
- pt::peg::to::cparam

Import

see below ...
- **pt::peg::import**
- pt::peg::import::container
- **pt::peg::import::json**
- **pt::peg::import::peg**
- pt::peg::from::container
- pt::peg::from::json
- pt::peg::from::peg

Transformation
see below ...

Execution
see below ...
- **pt::peg::interp**
- **pt::rde**

Support
see below ...
- **pt::tclparam::configuration::snit**
- **pt::tclparam::configuration::tcloo**
- **pt::cparam::configuration::critcl**
- **pt::ast**
- **pt::pe**
- **pt::peg**

Support packages
- pt::peg::container::peg
- **text::write**
- **configuration**
- **paths**
- **char**

Bugs, ideas, feedback
If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pt* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Parsing and Grammars

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

pt_json_language

Keywords: pt_json_language, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt::json_language - The JSON Grammar Exchange Format

Synopsis

package require Tcl 8.5 9

Description

Are you lost ? Do you have trouble understanding this document ? In that case please read the overview provided by the *Introduction to Parser Tools*. This document is the entrypoint to the whole system the current package is a part of.

The **json** format for parsing expression grammars was written as a data exchange format not bound to Tcl. It was defined to allow the exchange of grammars with PackRat/PEG based parser generators for other languages.

It is formally specified by the rules below:

1. The JSON of any PEG is a JSON object.
2. This object holds a single key, **pt::grammar::peg**, and its value. This value holds the contents of the grammar.
3. The contents of the grammar are a JSON object holding the set of nonterminal symbols and the starting expression. The relevant keys and their values are

   rules

   The value is a JSON object whose keys are the names of the nonterminal symbols known to the grammar.

   1. Each nonterminal symbol may occur only once.
   2. The empty string is not a legal nonterminal symbol.
   3. The value for each symbol is a JSON object itself. The relevant keys and their values in this dictionary are

      is

      The value is a JSON string holding the Tcl serialization of the parsing expression describing the symbols sentennial structure, as specified in the section PE serialization format.

      mode

      The value is a JSON holding holding one of three values specifying how a parser should handle the semantic value produced by the symbol.

      value

      The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal itself, which has the ASTs of the symbol's right hand side as its children.

leaf
The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal, without any children. Any ASTs generated by the symbol's right hand side are discarded.

void
The nonterminal has no semantic value. Any ASTs generated by the symbol's right hand side are discarded (as well).

```
**start**
: The value is a JSON string holding the Tcl serialization of the
    start parsing expression of the grammar, as specified in the
    section [PE serialization format].
```

1. The terminal symbols of the grammar are specified implicitly as the set of all terminal symbols used in the start expression and on the RHS of the grammar rules.

As an aside to the advanced reader, this is pretty much the same as the Tcl serialization of PE grammars, as specified in section PEG serialization format, except that the Tcl dictionaries and lists of that format are mapped to JSON objects and arrays. Only the parsing expressions themselves are not translated further, but kept as JSON strings containing a nested Tcl list, and there is no concept of canonicity for the JSON either.

Example

Assuming the following PEG for simple mathematical expressions

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

a JSON serialization for it is

```
{
    "pt::grammar::peg" : {
        "rules" : {
            "AddOp"     : {
                "is"   : "\/ {t -} {t +}",
                "mode" : "value"
            },
            "Digit"     : {
                "is"   : "\/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6}
    {t 7} {t 8} {t 9}",
```

```
                "mode" : "value"
            },
            "Expression" : {
                "is"   : "\/ {x {t (} {n Expression} {t )}} {x {n
    Factor} {* {x {n MulOp} {n Factor}}}}",
                "mode" : "value"
            },
            "Factor"    : {
                "is"   : "x {n Term} {* {x {n AddOp} {n Term}}}",
                "mode" : "value"
            },
            "MulOp"     : {
                "is"   : "\/ {t *} {t \/}",
                "mode" : "value"
            },
            "Number"    : {
                "is"   : "x {? {n Sign}} {+ {n Digit}}",
                "mode" : "value"
            },
            "Sign"      : {
                "is"   : "\/ {t -} {t +}",
                "mode" : "value"
            },
            "Term"      : {
                "is"   : "n Number",
                "mode" : "value"
            }
        },
        "start" : "n Expression"
    }
}
```

and a Tcl serialization of the same is

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                  mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}               mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                  mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                  mode value}
        MulOp      {is {/ {t *} {t /}}
                                  mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
```

```
                                    mode value}
        Sign       {is {/ {t -} {t +}}
                                    mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                    mode value}
    }
    start {n Expression}
}
```

The similarity of the latter to the JSON should be quite obvious.

PEG serialization format

Here we specify the format used by the Parser Tools to serialize Parsing Expression Grammars as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a PEG may have more than one regular serialization only exactly one of them will be *canonical*.

regular serialization

1. The serialization of any PEG is a nested Tcl dictionary.
1. This dictionary holds a single key, **pt::grammar::peg**, and its value. This value holds the contents of the grammar.
2. The contents of the grammar are a Tcl dictionary holding the set of nonterminal symbols and the starting expression. The relevant keys and their values are

   rules

   The value is a Tcl dictionary whose keys are the names of the nonterminal symbols known to the grammar.

   1. Each nonterminal symbol may occur only once.
   2. The empty string is not a legal nonterminal symbol.
   3. The value for each symbol is a Tcl dictionary itself. The relevant keys and their values in this dictionary are

      is

      The value is the serialization of the parsing expression describing the symbols sentennial structure, as specified in the section PE serialization format.

      mode

      The value can be one of three values specifying how a parser should handle the semantic value produced by the symbol.

      value

      The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal itself, which has the ASTs of the symbol's right hand side as its children.

      leaf

      The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal, without any children. Any ASTs generated by the symbol's right hand side are discarded.

      void

      The nonterminal has no semantic value. Any ASTs generated by the symbol's right hand side are discarded (as well).

```
**start**
: The value is the serialization of the start parsing expression of
    the grammar, as specified in the section [PE serialization
    format].
```

1. The terminal symbols of the grammar are specified implicitly as the set of all terminal symbols used in the start expression and on the RHS of the grammar rules.

canonical serialization

The canonical serialization of a grammar has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this grammar.

1. The keys found in all the nested Tcl dictionaries are sorted in ascending dictionary order, as generated by Tcl's builtin command **lsort -increasing -dict**.
2. The string representation of the value is the canonical representation of a Tcl dictionary. I.e. it does not contain superfluous whitespace.

Example

Assuming the following PEG for simple mathematical expressions

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

then its canonical serialization (except for whitespace) is

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                  mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}                mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                  mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                  mode value}
        MulOp      {is {/ {t *} {t /}}
                                  mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
                                  mode value}
        Sign       {is {/ {t -} {t +}}
```

```
                                    mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                    mode value}
    }
    start {n Expression}
}
```

PE serialization format

Here we specify the format used by the Parser Tools to serialize Parsing Expressions as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a parsing expression may have more than one regular serialization only exactly one of them will be *canonical*.

Regular serialization

see below ...

Atomic Parsing Expressions

1. The string **epsilon** is an atomic parsing expression. It matches the empty string.
1. The string **dot** is an atomic parsing expression. It matches any character.
2. The string **alnum** is an atomic parsing expression. It matches any Unicode alphabet or digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
3. The string **alpha** is an atomic parsing expression. It matches any Unicode alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
4. The string **ascii** is an atomic parsing expression. It matches any Unicode character below U0080. This is a custom extension of PEs based on Tcl's builtin command **string is**.
5. The string **control** is an atomic parsing expression. It matches any Unicode control character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
6. The string **digit** is an atomic parsing expression. It matches any Unicode digit character. Note that this includes characters outside of the [0..9] range. This is a custom extension of PEs based on Tcl's builtin command **string is**.
7. The string **graph** is an atomic parsing expression. It matches any Unicode printing character, except for space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
8. The string **lower** is an atomic parsing expression. It matches any Unicode lower-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
9. The string **print** is an atomic parsing expression. It matches any Unicode printing character, including space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
10. The string **punct** is an atomic parsing expression. It matches any Unicode punctuation character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
11. The string **space** is an atomic parsing expression. It matches any Unicode space character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
12. The string **upper** is an atomic parsing expression. It matches any Unicode upper-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command

string is.

13. The string **wordchar** is an atomic parsing expression. It matches any Unicode word character. This is any alphanumeric character (see alnum), and any connector punctuation characters (e.g. underscore). This is a custom extension of PEs based on Tcl's builtin command **string is**.
14. The string **xdigit** is an atomic parsing expression. It matches any hexadecimal digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
15. The string **ddigit** is an atomic parsing expression. It matches any decimal digit character. This is a custom extension of PEs based on Tcl's builtin command **regexp**.
16. The expression [list t **x**] is an atomic parsing expression. It matches the terminal string **x**.
17. The expression [list n **A**] is an atomic parsing expression. It matches the nonterminal **A**.

Combined Parsing Expressions

1. For parsing expressions **e1**, **e2**, ... the result of [list / **e1 e2** ...] is a parsing expression as well. This is the *ordered choice*, aka *prioritized choice*.
1. For parsing expressions **e1**, **e2**, ... the result of [list x **e1 e2** ...] is a parsing expression as well. This is the *sequence*.
2. For a parsing expression **e** the result of [list * **e**] is a parsing expression as well. This is the *kleene closure*, describing zero or more repetitions.
3. For a parsing expression **e** the result of [list + **e**] is a parsing expression as well. This is the *positive kleene closure*, describing one or more repetitions.
4. For a parsing expression **e** the result of [list & **e**] is a parsing expression as well. This is the *and lookahead predicate*.
5. For a parsing expression **e** the result of [list! **e**] is a parsing expression as well. This is the *not lookahead predicate*.
6. For a parsing expression **e** the result of [list ? **e**] is a parsing expression as well. This is the *optional input*.

Canonical serialization

The canonical serialization of a parsing expression has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this parsing expression.

1. The string representation of the value is the canonical representation of a pure Tcl list. I.e. it does not contain superfluous whitespace.
2. Terminals are *not* encoded as ranges (where start and end of the range are identical).

Example

Assuming the parsing expression shown on the right-hand side of the rule

```
    Expression <- Term (AddOp Term)*
```

then its canonical serialization (except for whitespace) is

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you

want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pt* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Parsing and Grammars

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_param

Keywords: pt_param, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer, virtual machine

Name

pt::param - PackRat Machine Specification

Synopsis

package require Tcl 8.5 9

Description

Are you lost ? Do you have trouble understanding this document ? In that case please read the overview provided by the *Introduction to Parser Tools*. This document is the entrypoint to the whole system the current package is a part of.

Welcome to the PackRat Machine (short: *PARAM*), a virtual machine geared towards the support of recursive descent parsers, especially packrat parsers. Towards this end it has features like the caching and reuse of partial results, the caching of the encountered input, and the ability to backtrack in both input and AST creation.

This document specifies the machine in terms of its architectural state and instruction set.

Architectural state

Any PARAM implementation has to manage at least the following state:

***Input* (IN)**

This is the channel the characters to process are read from.

This part of the machine's state is used and modified by the instructions defined in the section Input Handling.

***Current Character* (CC)**

The character from the *input* currently tested against its possible alternatives.

This part of the machine's state is used and modified by the instructions defined in the section Character Processing.

***Current Location* (CL)**

The location of the *current character* in the *input*, as offset relative to the beginning of the input. Character offsets are counted from **0**.

This part of the machine's state is used and modified by the instructions defined in the sections Character Processing, Location Handling, and Nonterminal Execution.

***Location Stack* (LS)**

A stack of locations in the *input*, saved for possible backtracking.

This part of the machine's state is used and modified by the instructions defined in the sections Character Processing, Location Handling, and Nonterminal Execution.

***Status* (ST)**

The status of the last attempt of testing the *input*, indicating either success or failure.

This part of the machine's state is used and modified by the instructions defined in the sections Status control, Character Processing, and Nonterminal Execution.

***Semantic Value* (SV)**

The current semantic value, either empty, or a node for AST constructed from the input.

This part of the machine's state is used and modified by the instructions defined in the sections Value Construction, and AST Construction.

***AST Reduction Stack* (ARS)**

The stack of partial ASTs constructed during the processing of nonterminal symbols.

This part of the machine's state is used and modified by the instructions defined in the sections Value Construction, and AST Construction.

***AST Stack* (AS)**

The stack of reduction stacks, saved for possible backtracking.

This part of the machine's state is used and modified by the instructions defined in the sections Value Construction, and AST Construction.

***Error Status* (ER)**

The machine's current knowledge of errors. This is either empty, or set to a pair of location in the input and the set of messages for that location.

Note that this part of the machine's state can be set even if the last test of the *current character* was successful. For example, the *-operator (matching a sub-expression zero or more times) in a PEG is always successful, even if it encounters a problem further in the input and has to backtrack. Such problems must not be forgotten when continuing the parsing.

This part of the machine's state is used and modified by the instructions defined in the sections Error Handling, Character Processing, and Nonterminal Execution.

***Error Stack* (ES)**

The stack of error stati, saved for backtracking. This enables the machine to merge current and older error stati when performing backtracking in choices after an failed match.

This part of the machine's state is used and modified by the instructions defined in the sections Error Handling, Character Processing, and Nonterminal Execution.

***Nonterminal Cache* (NC)**

A cache of machine states keyed by pairs name of nonterminal symbol and location in the input. Each pair (N, L) is associated with a 4-tuple holding the values to use for CL, ST, SV, and ER after the nonterminal N was parsed starting from the location L. It is a performance aid for backtracking parsers, allowing them to avoid an expensive reparsing of complex nonterminal symbols if they have been encountered before at a given location.

The key location is where machine started the attempt to match the named nonterminal symbol, and the location in the saved 4-tuple is where machine ended up after the attempt completed, independent of the success of the attempt.

This part of the machine's state is used and modified by the instructions defined in the section Nonterminal Execution.

***Terminal Cache* (TC)**

A cache of characters read from IN, with their location in IN as pair of line and column, keyed by the location in IN, this time as character offset from the beginning of IN. It is a performance aid for backtracking parsers, allowing them to avoid a possibly expensive rereading of characters from IN, or even enabling backtracking at, i.e. in the case of IN not randomly seekable.

This part of the machine's state is used and modified by the instructions defined in the section Input Handling.

Instruction set

With the machine's architectural state specified it is now possible to specify the instruction set operating on that state and to be implemented by any realization of the PARAM. The 37 instructions are grouped roughly by the state they influence and/or query during their execution.

Input Handling

The instructions in this section mainly access IN, pulling the characters to process into the machine.

input_next *msg*

This method reads the next character, i.e. the character after CL, from IN. If successful this character becomes CC, CL is advanced by one, ES is cleared, and the operation is recorded as a success in ST.

The operation may read the character from IN if the next character is not yet known to TC. If successful the new character is stored in TC, with its location (line, column), and the operation otherwise behaves as specified above. Future reads from the same location, possible due to backtracking, will then be satisfied from TC instead of IN.

If, on the other hand, the end of IN was reached, the operation is recorded as failed in ST, CL is left unchanged, and the pair of CL and *msg* becomes the new ES.

Character Processing

The instructions in this section mainly access CC, testing it against character classes, ranges, and individual characters.

test_alnum

This instruction implements the special PE operator "alnum", which checks if CC falls into the character class of the same name, or not.

Success and failure of the test are both recorded directly in ST. Success further clears ES, wheras failure sets the pair of CL and expected input (encoded as a leaf parsing expression) as the new ES and then rewinds CL by one character, preparing the machine for another parse attempt by a possible alternative.

test_alpha

This instruction implements the special PE operator "alpha", which checks if CC falls into the character class of the same name, or not.

Success and failure of the test are both recorded directly in ST. Success further clears ES, wheras failure sets the pair of CL and expected input (encoded as a leaf parsing expression) as the new ES and then rewinds CL by one character, preparing the machine for another parse attempt by a possible alternative.

test_ascii

This instruction implements the special PE operator "ascii", which checks if CC falls into the character class of the same name, or not.

Success and failure of the test are both recorded directly in ST. Success further clears ES, wheras failure sets the pair of CL and expected input (encoded as a leaf parsing expression) as the new ES and then rewinds CL by one character, preparing the machine for another parse attempt by a possible alternative.

test_char *char*

This instruction implements the character matching operator, i.e. it checks if CC is *char*.

Success and failure of the test are both recorded directly in ST. Success further clears ES, wheras failure sets the pair of CL and expected input (encoded as a leaf parsing expression) as the new ES and then rewinds CL by one character, preparing the machine for another parse attempt by a possible alternative.

test_ddigit

This instruction implements the special PE operator "ddigit", which checks if CC falls into the character class of the same name, or not.

Success and failure of the test are both recorded directly in ST. Success further clears ES, wheras failure sets the pair of CL and expected input (encoded as a leaf parsing expression) as the new ES and then rewinds CL by one character, preparing the machine for another parse attempt by a possible alternative.

test_digit
This instruction implements the special PE operator "digit", which checks if CC falls into the character class of the same name, or not.

Success and failure of the test are both recorded directly in ST. Success further clears ES, wheras failure sets the pair of CL and expected input (encoded as a leaf parsing expression) as the new ES and then rewinds CL by one character, preparing the machine for another parse attempt by a possible alternative.

test_graph
This instruction implements the special PE operator "graph", which checks if CC falls into the character class of the same name, or not.

Success and failure of the test are both recorded directly in ST. Success further clears ES, wheras failure sets the pair of CL and expected input (encoded as a leaf parsing expression) as the new ES and then rewinds CL by one character, preparing the machine for another parse attempt by a possible alternative.

test_lower
This instruction implements the special PE operator "lower", which checks if CC falls into the character class of the same name, or not.

Success and failure of the test are both recorded directly in ST. Success further clears ES, wheras failure sets the pair of CL and expected input (encoded as a leaf parsing expression) as the new ES and then rewinds CL by one character, preparing the machine for another parse attempt by a possible alternative.

test_print
This instruction implements the special PE operator "print", which checks if CC falls into the character class of the same name, or not.

Success and failure of the test are both recorded directly in ST. Success further clears ES, wheras failure sets the pair of CL and expected input (encoded as a leaf parsing expression) as the new ES and then rewinds CL by one character, preparing the machine for another parse attempt by a possible alternative.

test_punct
This instruction implements the special PE operator "punct", which checks if CC falls into the character class of the same name, or not.

Success and failure of the test are both recorded directly in ST. Success further clears ES, wheras failure sets the pair of CL and expected input (encoded as a leaf parsing expression) as the new ES and then rewinds CL by one character, preparing the machine for another parse attempt by a possible alternative.

test_range *chars chare*
This instruction implements the range matching operator, i.e. it checks if CC falls into the interval of characters spanned up by the two characters from *chars* to *chare*, both inclusive.

Success and failure of the test are both recorded directly in ST. Success further clears ES, wheras failure sets the pair of CL and expected input (encoded as a leaf parsing expression)

as the new ES and then rewinds CL by one character, preparing the machine for another parse attempt by a possible alternative.

test_space
This instruction implements the special PE operator "space", which checks if CC falls into the character class of the same name, or not.

Success and failure of the test are both recorded directly in ST. Success further clears ES, wheras failure sets the pair of CL and expected input (encoded as a leaf parsing expression) as the new ES and then rewinds CL by one character, preparing the machine for another parse attempt by a possible alternative.

test_upper
This instruction implements the special PE operator "upper", which checks if CC falls into the character class of the same name, or not.

Success and failure of the test are both recorded directly in ST. Success further clears ES, wheras failure sets the pair of CL and expected input (encoded as a leaf parsing expression) as the new ES and then rewinds CL by one character, preparing the machine for another parse attempt by a possible alternative.

test_wordchar
This instruction implements the special PE operator "wordchar", which checks if CC falls into the character class of the same name, or not.

Success and failure of the test are both recorded directly in ST. Success further clears ES, wheras failure sets the pair of CL and expected input (encoded as a leaf parsing expression) as the new ES and then rewinds CL by one character, preparing the machine for another parse attempt by a possible alternative.

test_xdigit
This instruction implements the special PE operator "xdigit", which checks if CC falls into the character class of the same name, or not.

Success and failure of the test are both recorded directly in ST. Success further clears ES, wheras failure sets the pair of CL and expected input (encoded as a leaf parsing expression) as the new ES and then rewinds CL by one character, preparing the machine for another parse attempt by a possible alternative.

Error Handling

The instructions in this section mainly access ER and ES.

error_clear
This instruction clears ER.

error_push
This instruction makes a copy of ER and pushes it on ES.

error_pop_merge
This instruction takes the topmost entry of ES and merges the error status it contains with ES, making the result the new ES.

The merge is governed by four rules, with the merge result

1. Empty if both states are empty.
2. The non-empty state if only one of the two states is non-empty.
3. The state with the larger location, if the two states specify different locations.
4. The pair of the location shared by the two states, and the set-union of their messages for states at the same location.

error_nonterminal *symbol*
This is a guarded instruction. It does nothing if either ES is empty, or if the location in ES is not just past the last location saved in LS. Otherwise it sets the pair of that location and the nonterminal *symbol* as the new ES.

Note: In the above "just past" means "that location plus one", or also "the location of the next character after that location".

Status control

The instructions in this section directly manipulate ST.

status_ok
This instruction sets ST to **true**, recording a success.

status_fail
This instruction sets ST to **false**, recording a failure.

status_negate
This instruction negates ST, turning a failure into a success and vice versa.

Location Handling

The instructions in this section access CL and LS.

loc_push
This instruction makes a copy of CL and pushes it on LS.

loc_pop_discard
This instructions pops the last saved location from LS.

loc_pop_rewind
This instruction pops the last saved location from LS and restores it as CL.

Nonterminal Execution

The instructions in this section access and manipulate NC.

symbol_restore *symbol*
This instruction checks if NC contains data for the nonterminal *symbol* at CL, or not. The result of the instruction is a boolean flag, with **True** indicating that data was found in the cache. In that case the instruction has further updated the architectural state of the machine with the cached information, namely CL, ST, ER, and SV.

The method with which the instruction's result is transformed into control flow is left undefined and the responsibility of the implementation.

symbol_save *symbol*
This instructions saves the current settings of CL, ST, ER, and SV in NC, using the pair of nonterminal *symbol* and the last location saved in LS as key.

Value Construction

The instructions in this section manipulate SV.

value_clear
: This instruction clears SV.

value_leaf *symbol*
: This instruction constructs an AST node for *symbol* covering the range of IN from one character after the last location saved on LS to CL and stores it in SV. ...

value_reduce *symbol*
: This instruction generally behaves like **value_nonterminal_leaf**, except that it takes all AST nodes on ARS, if any, and makes them the children of the new node, with the last node saved on ARS becoming the right-most / last child. Note that ARS is not modfied by this operation.

AST Construction

The instructions in this section manipulate ARS and AS.

ast_value_push
: This instruction makes a copy of SV and pushes it on ARS.

ast_push
: This instruction pushes the current state of ARS on AS and then clears ARS.

ast_pop_rewind
: This instruction pops the last entry saved on AS and restores it as the new state of ARS.

ast_pop_discard
: This instruction pops the last entry saved on AS.

Control flow

Normally this section would contain the specifications of the control flow instructions of the PARAM, i.e. (un)conditional jumps and the like. However, this part of the PARAM is intentionally left unspecified. This allows the implementations to freely choose how to implement control flow.

The implementation of this machine in Parser Tools, i.e the package **pt::rde**, is not only coded in Tcl, but also relies on Tcl commands to provide it with control flow (instructions).

Interaction of the instructions with the architectural state

```
Instruction     Inputs              Outputs
======================= ======================     ====================
ast_pop_discard     AS          ->  AS
ast_pop_rewind      AS          ->  AS, ARS
ast_push        ARS, AS         ->  AS
ast_value_push      SV, ARS         ->  ARS
======================= ======================     ====================
error_clear     -           ->  ER
error_nonterminal sym   ER, LS          ->  ER
error_pop_merge     ES, ER          ->  ER
error_push      ES, ER          ->  ES
======================= ======================     ====================
input_next msg      IN          ->  TC, CL, CC, ST, ER
======================= ======================     ====================
```

```
loc_pop_discard     LS          ->  LS
loc_pop_rewind      LS          ->  LS, CL
loc_push        CL, LS          ->  LS
======================= ======================     ====================
status_fail     -           ->  ST
status_negate       ST          ->  ST
status_ok       -           ->  ST
======================= ======================     ====================
symbol_restore sym  NC          ->  CL, ST, ER, SV
symbol_save    sym  CL, ST, ER, SV LS   ->  NC
======================= ======================     ====================
test_alnum          CC          ->  ST, ER
test_alpha      CC          ->  ST, ER
test_ascii      CC          ->  ST, ER
test_char char      CC          ->  ST, ER
test_ddigit     CC          ->  ST, ER
test_digit      CC          ->  ST, ER
test_graph      CC          ->  ST, ER
test_lower      CC          ->  ST, ER
test_print      CC          ->  ST, ER
test_punct      CC          ->  ST, ER
test_range chars chare  CC          ->  ST, ER
test_space      CC          ->  ST, ER
test_upper      CC          ->  ST, ER
test_wordchar       CC          ->  ST, ER
test_xdigit     CC          ->  ST, ER
======================= ======================     ====================
value_clear     -           ->  SV
value_leaf symbol   LS, CL          ->  SV
value_reduce symbol ARS, LS, CL     ->  SV
======================= ======================     ====================
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pt* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Parsing and Grammars

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

pt_parse_peg

Keywords: pt_parse_peg, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt_parse_peg - Parser Tools PEG Parser

Synopsis

package require Tcl 8.5 9
package require pt::parse::peg 1
pt::parse::peg ?*objectName*?
objectName **destroy**
objectName **parse** *chan*
objectName **parset** *text*

Description

Are you lost ? Do you have trouble understanding this document ? In that case please read the overview provided by the *Introduction to Parser Tools*. This document is the entrypoint to the whole system the current package is a part of.

This package provides a class whose instances are parsers for parsing expression grammars in textual form.

Class API

pt::parse::peg ?*objectName*?

The class command constructs parser instances, i.e. objects. The result of the command is the fully-qualified name of the instance command.

If no *objectName* is specified the class will generate and use an automatic name. If the *objectName* was specified, but is not fully qualified the command will be created in the current namespace.

Instances API

All parser instances provide at least the methods shown below:

***objectName* destroy**

This method destroys the parser instance, releasing all claimed memory and other resources, and deleting the instance command.

The result of the command is the empty string.

objectName* parse *chan

This method runs the parser using the contents of *chan* as input (starting at the current location in the channel), until parsing is not possible anymore, either because parsing has

completed, or run into a syntax error.

Note here that the Parser Tools are based on Tcl 8.5+. In other words, the channel argument is not restricted to files, sockets, etc. We have the full power of *reflected channels* available.

It should also be noted that the parser pulls the characters from the input stream as it needs them. If a parser created by this package has to be operated in a push aka event-driven manner it will be necessary to go to Tcl 8.6+ and use the **coroutine::auto** to wrap it into a coroutine where **read** is properly changed for push-operation.

Upon successful completion the command returns an abstract syntax tree as its result. This AST is in the form specified in section **AST serialization format**. As a plain nested Tcl-list it can then be processed with any Tcl commands the user likes, doing transformations, semantic checks, etc. To help in this the package **pt::ast** provides a set of convenience commands for validation of the tree's basic structure, printing it for debugging, and walking it either from the bottom up, or top down.

When encountering a syntax error the command will throw an error instead. This error will be a 4-element Tcl-list, containing, in the order listed below:

1. The string **pt::rde** identifying it as parser runtime error.
2. The location of the parse error, as character offset from the beginning of the parsed input.
3. The location of parse error, now as a 2-element list containing line-number and column in the line.
4. A set of atomic parsing expressions indicating encoding the characters and/or nonterminal symbols the parser expected to see at the location of the parse error, but did not get. For the specification of atomic parsing expressions please see the section **PE serialization format**.

objectName* parset *text

This method runs the parser using the string in *text* as input. In all other ways it behaves like the method **parse**, shown above.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pt* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Parsing and Grammars

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

pt_parser_api

Keywords: pt_parser_api, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt_parser_api - Parser API

Synopsis

package require Tcl 8.5 9
className ?*objectName*?
objectName **destroy**
objectName **parse** *chan*
objectName **parset** *text*

Description

Are you lost ? Do you have trouble understanding this document ? In that case please read the overview provided by the *Introduction to Parser Tools*. This document is the entrypoint to the whole system the current package is a part of.

This document describes the API shared by the grammar interpreter provided by the package **pt::peg::interp** and the parsers generated by the **pt** application for the result formats **critcl**, **snit**, and **oo** regarding access to the actual parsing functionality.

Its intended audience are people who wish to create a parser for some language of theirs and then use that parser within a Tcl-based package or application.

It resides in the User Layer of Parser Tools.

IMAGE: arch_user_pkg

Class API

className ?*objectName*?

The class command constructs parser instances, i.e. objects. The result of the command is the fully-qualified name of the instance command.

If no *objectName* is specified the class will generate and use an automatic name. If the *objectName* was specified, but is not fully qualified the command will be created in the current namespace.

Instance API

All parser instances provide at least the methods shown below:

***objectName* destroy**

This method destroys the parser instance, releasing all claimed memory and other resources, and deleting the instance command.

The result of the command is the empty string.

objectName* parse *chan

This method runs the parser using the contents of *chan* as input (starting at the current

location in the channel), until parsing is not possible anymore, either because parsing has completed, or run into a syntax error.

Note here that the Parser Tools are based on Tcl 8.5+. In other words, the channel argument is not restricted to files, sockets, etc. We have the full power of *reflected channels* available.

It should also be noted that the parser pulls the characters from the input stream as it needs them. If a parser created by this package has to be operated in a push aka event-driven manner it will be necessary to go to Tcl 8.6+ and use the **coroutine::auto** to wrap it into a coroutine where **read** is properly changed for push-operation.

Upon successful completion the command returns an abstract syntax tree as its result. This AST is in the form specified in section Ast serialization format. As a plain nested Tcl-list it can then be processed with any Tcl commands the user likes, doing transformations, semantic checks, etc. To help in this the package **pt::ast** provides a set of convenience commands for validation of the tree's basic structure, printing it for debugging, and walking it either from the bottom up, or top down.

When encountering a syntax error the command will throw an error instead. This error will be a 4-element Tcl-list, containing, in the order listed below:

1. The string **pt::rde** identifying it as parser runtime error.
2. The location of the parse error, as character offset from the beginning of the parsed input.
3. The location of parse error, now as a 2-element list containing line-number and column in the line.
4. A set of atomic parsing expressions indicating encoding the characters and/or nonterminal symbols the parser expected to see at the location of the parse error, but did not get. For the specification of atomic parsing expressions please see the section PE serialization format.

objectName* parset *text

This method runs the parser using the string in *text* as input. In all other ways it behaves like the method **parse**, shown above.

Usage

A generated parser is used like this

```
package require the-parser-package ;# Generated by result-formats
'critcl', 'snit' or 'oo' of 'pt'.
set parser [the-parser-class]

set ast [$parser parse $channel]
... process the abstract syntax tree ...
```

When using a grammar interpreter for parsing some differences creep in

```
package require the-grammar-package ;# Generated by result-format
'container' of 'pt'.
set grammar [the-grammar-class]

package require pt::peg::interp
```

```
    set parser [pt::peg::interp]

    $parser use $grammar

    set ast [$parser parse $channel]
    $parser destroy

    ... process the abstract syntax tree ...
```

Ast serialization format

Here we specify the format used by the Parser Tools to serialize Abstract Syntax Trees (ASTs) as immutable values for transport, comparison, etc.

Each node in an AST represents a nonterminal symbol of a grammar, and the range of tokens/characters in the input covered by it. ASTs do not contain terminal symbols, i.e. tokens/characters. These can be recovered from the input given a symbol's location.

We distinguish between *regular* and *canonical* serializations. While a tree may have more than one regular serialization only exactly one of them will be *canonical*.

Regular serialization

1. The serialization of any AST is the serialization of its root node.
1. The serialization of any node is a Tcl list containing at least three elements.
    1. The first element is the name of the nonterminal symbol stored in the node.
    2. The second and third element are the locations of the first and last token in the token stream the node represents (covers).
        1. Locations are provided as non-negative integer offsets from the beginning of the token stream, with the first token found in the stream located at offset 0 (zero).
        2. The end location has to be equal to or larger than the start location.

1. All elements after the first three represent the children of the node, which are themselves nodes. This means that the serializations of nodes without children, i.e. leaf nodes, have exactly three elements. The children are stored in the list with the leftmost child first, and the rightmost child last.

Canonical serialization

The canonical serialization of an abstract syntax tree has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this tree.

1. The string representation of the value is the canonical representation of a pure Tcl list. I.e. it does not contain superfluous whitespace.

Example

Assuming the parsing expression grammar below

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
```

```
    Expression <- Term (AddOp Term)*                                ;
    MulOp      <- '*' / '/'                                         ;
    Term       <- Factor (MulOp Factor)*                            ;
    AddOp      <- '+'/'-'                                           ;
    Factor     <- '(' Expression ')' / Number                       ;
END;
```

and the input string

```
 120+5
```

then a parser should deliver the abstract syntax tree below (except for whitespace)

```
set ast {Expression 0 4
    {Factor 0 4
        {Term 0 2
            {Number 0 2
                {Digit 0 0}
                {Digit 1 1}
                {Digit 2 2}
            }
        }
        {AddOp 3 3}
        {Term 4 4
            {Number 4 4
                {Digit 4 4}
            }
        }
    }
}
```

Or, more graphical

```
+- Digit 0 0 | 1
|            |
+- Term 0 2 --- Number 0 2 -+- Digit 1 1 | 2
|                          |            |
|                          +- Digit 2 2 | 0
|                                       |
Expression 0 4 --- Factor 0 4 -+----------------------------- AddOp 3
    3 | +
|                                       |
+- Term 4 4 --- Number 4 4 --- Digit 4 4 | 5
```

PE serialization format

Here we specify the format used by the Parser Tools to serialize Parsing Expressions as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a parsing expression may

have more than one regular serialization only exactly one of them will be *canonical*.

Regular serialization

see below ...

Atomic Parsing Expressions

1. The string **epsilon** is an atomic parsing expression. It matches the empty string.
1. The string **dot** is an atomic parsing expression. It matches any character.
2. The string **alnum** is an atomic parsing expression. It matches any Unicode alphabet or digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
3. The string **alpha** is an atomic parsing expression. It matches any Unicode alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
4. The string **ascii** is an atomic parsing expression. It matches any Unicode character below U0080. This is a custom extension of PEs based on Tcl's builtin command **string is**.
5. The string **control** is an atomic parsing expression. It matches any Unicode control character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
6. The string **digit** is an atomic parsing expression. It matches any Unicode digit character. Note that this includes characters outside of the [0..9] range. This is a custom extension of PEs based on Tcl's builtin command **string is**.
7. The string **graph** is an atomic parsing expression. It matches any Unicode printing character, except for space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
8. The string **lower** is an atomic parsing expression. It matches any Unicode lower-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
9. The string **print** is an atomic parsing expression. It matches any Unicode printing character, including space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
10. The string **punct** is an atomic parsing expression. It matches any Unicode punctuation character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
11. The string **space** is an atomic parsing expression. It matches any Unicode space character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
12. The string **upper** is an atomic parsing expression. It matches any Unicode upper-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
13. The string **wordchar** is an atomic parsing expression. It matches any Unicode word character. This is any alphanumeric character (see alnum), and any connector punctuation characters (e.g. underscore). This is a custom extension of PEs based on Tcl's builtin command **string is**.
14. The string **xdigit** is an atomic parsing expression. It matches any hexadecimal digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
15. The string **ddigit** is an atomic parsing expression. It matches any decimal digit character. This is a custom extension of PEs based on Tcl's builtin command **regexp**.
16. The expression [list t **x**] is an atomic parsing expression. It matches the terminal string **x**.
17. The expression [list n **A**] is an atomic parsing expression. It matches the nonterminal **A**.

Combined Parsing Expressions

1. For parsing expressions **e1**, **e2**, ... the result of [list / **e1 e2** ...] is a parsing expression as well. This is the *ordered choice*, aka *prioritized choice*.
1. For parsing expressions **e1**, **e2**, ... the result of [list x **e1 e2** ...] is a parsing expression as well. This is the *sequence*.
2. For a parsing expression **e** the result of [list * **e**] is a parsing expression as well. This is the *kleene closure*, describing zero or more repetitions.
3. For a parsing expression **e** the result of [list + **e**] is a parsing expression as well. This is the *positive kleene closure*, describing one or more repetitions.
4. For a parsing expression **e** the result of [list & **e**] is a parsing expression as well. This is the *and lookahead predicate*.
5. For a parsing expression **e** the result of [list! **e**] is a parsing expression as well. This is the *not lookahead predicate*.
6. For a parsing expression **e** the result of [list ? **e**] is a parsing expression as well. This is the *optional input*.

Canonical serialization

The canonical serialization of a parsing expression has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this parsing expression.

1. The string representation of the value is the canonical representation of a pure Tcl list. I.e. it does not contain superfluous whitespace.
2. Terminals are *not* encoded as ranges (where start and end of the range are identical).

Example

Assuming the parsing expression shown on the right-hand side of the rule

```
    Expression <- Term (AddOp Term)*
```

then its canonical serialization (except for whitespace) is

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pt* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Parsing and Grammars

Copyright

Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>

pt_peg_container

Keywords: pt_peg_container, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt::peg::container - PEG Storage

Synopsis

package require Tcl 8.5 9
package require snit
package require pt::peg::container ?**1**?
::pt::peg *objectName* ?=|*:=*|<--|*as*|*deserialize src*?
objectName **destroy**
objectName **clear**
objectName **importer**
objectName **importer** *object*
objectName **exporter**
objectName **exporter** *object*
objectName **=** *source*
objectName **-->** *destination*
objectName **serialize** ?*format*?
objectName **deserialize =** *data* ?*format*?
objectName **deserialize +=** *data* ?*format*?
objectName **start**
objectName **start** *pe*
objectName **nonterminals**
objectName **modes**
objectName **modes** *dict*
objectName **rules**
objectName **rules** *dict*
objectName **add** ?*nt...*?
objectName **remove** ?*nt...*?
objectName **exists** *nt*
objectName **rename** *ntold ntnew*
objectName **mode** *nt*
objectName **mode** *nt mode*
objectName **rule** *nt*
objectName **rule** *nt pe*

Description

Are you lost ? Do you have trouble understanding this document ? In that case please read the overview provided by the *Introduction to Parser Tools*. This document is the entrypoint to the

whole system the current package is a part of.

This package provides a container class for parsing expression grammars, with each instance storing a single grammar and allowing the user to manipulate and query its definition.

It resides in the Storage section of the Core Layer of Parser Tools, and is one of the three pillars the management of parsing expression grammars resides on.

IMAGE: arch_core_container

The other two pillars are, as shown above

1. *PEG Import*, and
2. *PEG Export*

Packages related to this are:

pt::rde
: This package provides an implementation of PARAM, a virtual machine for the parsing of a channel, geared towards the needs of handling PEGs.

pt::peg::interp
: This package implements an interpreter for PEGs on top of the virtual machine provided by **pt::peg::rde**

Class API

The package exports the API described here.

::pt::peg *objectName* ?=|:=|<--|as|deserialize *src*?
: The command creates a new container object for a parsing expression grammar and returns the fully qualified name of the object command as its result. The API of this object command is described in the section Object API. It may be used to invoke various operations on the object.

  The new container will be empty if no *src* is specified. Otherwise it will contain a copy of the grammar contained in the *src*. All operators except **deserialize** interpret *src* as a container object command. The **deserialize** operator interprets *src* as the serialization of a parsing expression grammar instead, as specified in section PEG serialization format.

  An empty grammar has no nonterminal symbols, and the start expression is the empty expression, i.e. epsilon. It is *valid*, but not *useful*.

Object API

All objects created by this package provide the following methods for the manipulation and querying of their contents:

***objectName* destroy**
: This method destroys the object, releasing all claimed memory, and deleting the associated object command.

***objectName* clear**
: This method resets the object to contain the empty grammar. It does *not* destroy the object itself.

***objectName* importer**
: This method returns the import manager object currently attached to the container, if any.

objectName* importer *object
This method attaches the *object* as import manager to the container, and returns it as the result of the command. Note that the *object* is *not* put into ownership of the container. I.e., destruction of the container will *not* destroy the *object*.

It is expected that *object* provides a method named **import text** which takes a text and a format name, and returns the canonical serialization of the table of contents contained in the text, assuming the given format.

***objectName* exporter**
This method returns the export manager object currently attached to the container, if any.

objectName* exporter *object
This method attaches the *object* as export manager to the container, and returns it as the result of the command. Note that the *object* is *not* put into ownership of the container. I.e., destruction of the container will *not* destroy the *object*.

It is expected that *object* provides a method named **export object** which takes the container and a format name, and returns a text encoding table of contents stored in the container, in the given format. It is further expected that the *object* will use the container's method **serialize** to obtain the serialization of the table of contents from which to generate the text.

objectName* = *source
This method assigns the contents of the PEG object *source* to ourselves, overwriting the existing definition. This is the assignment operator for grammars.

This operation is in effect equivalent to

```
    objectName deserialize = [source serialize]
```

objectName* --> *destination
This method assigns our contents to the PEG object *destination*, overwriting the existing definition. This is the reverse assignment operator for grammars.

This operation is in effect equivalent to

```
    destination deserialize = [objectName serialize]
```

***objectName* serialize ?*format*?**
This method returns our grammar in some textual form usable for transfer, persistent storage, etc. If no *format* is not specified the returned result is the canonical serialization of the grammar, as specified in the section PEG serialization format.

Otherwise the object will use the attached export manager to convert the data to the specified format. In that case the method will fail with an error if the container has no export manager attached to it.

***objectName* deserialize = *data* ?*format*?**
This is the complementary method to **serialize**. It replaces the current definition with the grammar contained in the *data*. If no *format* was specified it is assumed to be the regular serialization of a grammar, as specified in the section PEG serialization format

Otherwise the object will use the attached import manager to convert the data from the specified format to a serialization it can handle. In that case the method will fail with an error if the container has no import manager attached to it.

The result of the method is the empty string.

***objectName* deserialize += *data* ?*format*?**

This method behaves like **deserialize =** in its essentials, except that it merges the grammar in the *data* to its contents instead of replacing it. The method will fail with an error and leave the grammar unchanged if merging is not possible, i.e. would produce an invalid grammar.

The result of the method is the empty string.

***objectName* start**

This method returns the current start expression of the grammar.

objectName* start *pe

This method defines the *start expression* of the grammar. It replaces the current start expression with the parsing expression *pe*, and returns the new start expression.

The method will fail with an error and leave the grammar unchanged if *pe* does not contain a valid parsing expression as specified in the section Pe serialization format.

***objectName* nonterminals**

This method returns the set of all nonterminal symbols known to the grammar.

***objectName* modes**

This method returns a dictionary mapping the set of all nonterminal symbols known to the grammar to their semantic modes.

objectName* modes *dict

This method takes a dictionary mapping a set of nonterminal symbols known to the grammar to their semantic modes, and returns the new full mapping of nonterminal symbols to semantic modes.

The method will fail with an error if any of the nonterminal symbols in the dictionary is not known to the grammar, or the empty string, i.e. an invalid nonterminal symbol, or if any the chosen *mode*s is not one of the legal values.

***objectName* rules**

This method returns a dictionary mapping the set of all nonterminal symbols known to the grammar to their parsing expressions (right-hand sides).

objectName* rules *dict

This method takes a dictionary mapping a set of nonterminal symbols known to the grammar to their parsing expressions (right-hand sides), and returns the new full mapping of nonterminal symbols to parsing expressions.

The method will fail with an error any of the nonterminal symbols in the dictionary is not known to the grammar, or the empty string, i.e. an invalid nonterminal symbol, or any of the chosen parsing expressions is not a valid parsing expression as specified in the section Pe serialization format.

***objectName* add ?*nt*...?**

This method adds the nonterminal symbols *nt*, etc. to the grammar, and defines default semantic mode and expression for it (**value** and **epsilon** respectively). The method returns the empty string as its result.

The method will fail with an error and leaves the grammar unchanged if any of the nonterminal symbols are either already defined in our grammar, or are the empty string (an invalid nonterminal symbol).

The method does nothing if no symbol was specified as argument.

***objectName* remove ?*nt...*?**

This method removes the named nonterminal symbols *nt*, etc. from the set of nonterminal symbols known to our grammar. The method returns the empty string as its result.

The method will fail with an error and leave the grammar unchanged if any of the nonterminal symbols is not known to the grammar, or is the empty string, i.e. an invalid nonterminal symbol.

objectName* exists *nt

This method tests whether the nonterminal symbol *nt* is known to our grammar or not. The result is a boolean value. It will be set to **true** if *nt* is known, and **false** otherwise.

The method will fail with an error if *nt* is the empty string, i.e. an invalid nonterminal symbol.

objectName* rename *ntold ntnew

This method renames the nonterminal symbol *ntold* to *ntnew*. The method returns the empty string as its result.

The method will fail with an error and leave the grammar unchanged if either *ntold* is not known to the grammar, or *ntnew* is already known, or any of them is the empty string, i.e. an invalid nonterminal symbol.

objectName* mode *nt

This method returns the current semantic mode for the nonterminal symbol *nt*.

The method will fail with an error if *nt* is not known to the grammar, or the empty string, i.e. an invalid nonterminal symbol.

objectName* mode *nt mode

This mode sets the semantic mode for the nonterminal symbol *nt*, and returns the new mode. The method will fail with an error if *nt* is not known to the grammar, or the empty string, i.e. an invalid nonterminal symbol, or the chosen *mode* is not one of the legal values.

The following modes are legal:

value

The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal itself, which has the ASTs of the symbol's right hand side as its children.

leaf

The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal, without any children. Any ASTs generated by the

symbol's right hand side are discarded.

void
The nonterminal has no semantic value. Any ASTs generated by the symbol's right hand side are discarded (as well).

objectName* rule *nt
This method returns the current parsing expression (right-hand side) for the nonterminal symbol *nt*.

The method will fail with an error if *nt* is not known to the grammar, or the empty string, i.e. an invalid nonterminal symbol.

objectName* rule *nt pe
This method set the parsing expression (right-hand side) of the nonterminal *nt* to *pe*, and returns the new parsing expression.

The method will fail with an error if *nt* is not known to the grammar, or the empty string, i.e. an invalid nonterminal symbol, or *pe* does not contain a valid parsing expression as specified in the section Pe serialization format.

PEG serialization format

Here we specify the format used by the Parser Tools to serialize Parsing Expression Grammars as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a PEG may have more than one regular serialization only exactly one of them will be *canonical*.

regular serialization

1. The serialization of any PEG is a nested Tcl dictionary.
1. This dictionary holds a single key, **pt::grammar::peg**, and its value. This value holds the contents of the grammar.
2. The contents of the grammar are a Tcl dictionary holding the set of nonterminal symbols and the starting expression. The relevant keys and their values are

rules
The value is a Tcl dictionary whose keys are the names of the nonterminal symbols known to the grammar.

1. Each nonterminal symbol may occur only once.
2. The empty string is not a legal nonterminal symbol.
3. The value for each symbol is a Tcl dictionary itself. The relevant keys and their values in this dictionary are

is
The value is the serialization of the parsing expression describing the symbols sentennial structure, as specified in the section Pe serialization format.

mode
The value can be one of three values specifying how a parser should handle the semantic value produced by the symbol.

value
The semantic value of the nonterminal symbol is an abstract syntax tree

consisting of a single node node for the nonterminal itself, which has the ASTs of the symbol's right hand side as its children.

leaf
The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal, without any children. Any ASTs generated by the symbol's right hand side are discarded.

void
The nonterminal has no semantic value. Any ASTs generated by the symbol's right hand side are discarded (as well).

```
**start**
: The value is the serialization of the start parsing expression of
    the grammar, as specified in the section [Pe serialization
    format].
```

1. The terminal symbols of the grammar are specified implicitly as the set of all terminal symbols used in the start expression and on the RHS of the grammar rules.

canonical serialization
The canonical serialization of a grammar has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this grammar.

1. The keys found in all the nested Tcl dictionaries are sorted in ascending dictionary order, as generated by Tcl's builtin command **lsort -increasing -dict**.
2. The string representation of the value is the canonical representation of a Tcl dictionary. I.e. it does not contain superfluous whitespace.

Example

Assuming the following PEG for simple mathematical expressions

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

then its canonical serialization (except for whitespace) is

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}              mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                mode value}
        MulOp      {is {/ {t *} {t /}}
                                mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
                                mode value}
        Sign       {is {/ {t -} {t +}}
                                mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                mode value}
    }
    start {n Expression}
}
```

Pe serialization format

Here we specify the format used by the Parser Tools to serialize Parsing Expressions as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a parsing expression may have more than one regular serialization only exactly one of them will be *canonical*.

Regular serialization

see below ...

Atomic Parsing Expressions

1. The string **epsilon** is an atomic parsing expression. It matches the empty string.
1. The string **dot** is an atomic parsing expression. It matches any character.
2. The string **alnum** is an atomic parsing expression. It matches any Unicode alphabet or digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
3. The string **alpha** is an atomic parsing expression. It matches any Unicode alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
4. The string **ascii** is an atomic parsing expression. It matches any Unicode character below U0080. This is a custom extension of PEs based on Tcl's builtin command **string is**.
5. The string **control** is an atomic parsing expression. It matches any Unicode control character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
6. The string **digit** is an atomic parsing expression. It matches any Unicode digit character. Note that this includes characters outside of the [0..9] range. This is a custom extension of PEs based on Tcl's builtin command **string is**.

7. The string **graph** is an atomic parsing expression. It matches any Unicode printing character, except for space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
8. The string **lower** is an atomic parsing expression. It matches any Unicode lower-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
9. The string **print** is an atomic parsing expression. It matches any Unicode printing character, including space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
10. The string **punct** is an atomic parsing expression. It matches any Unicode punctuation character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
11. The string **space** is an atomic parsing expression. It matches any Unicode space character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
12. The string **upper** is an atomic parsing expression. It matches any Unicode upper-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
13. The string **wordchar** is an atomic parsing expression. It matches any Unicode word character. This is any alphanumeric character (see alnum), and any connector punctuation characters (e.g. underscore). This is a custom extension of PEs based on Tcl's builtin command **string is**.
14. The string **xdigit** is an atomic parsing expression. It matches any hexadecimal digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
15. The string **ddigit** is an atomic parsing expression. It matches any decimal digit character. This is a custom extension of PEs based on Tcl's builtin command **regexp**.
16. The expression [list t **x**] is an atomic parsing expression. It matches the terminal string **x**.
17. The expression [list n **A**] is an atomic parsing expression. It matches the nonterminal **A**.

Combined Parsing Expressions

1. For parsing expressions **e1**, **e2**, ... the result of [list / **e1 e2** ...] is a parsing expression as well. This is the *ordered choice*, aka *prioritized choice*.
1. For parsing expressions **e1**, **e2**, ... the result of [list x **e1 e2** ...] is a parsing expression as well. This is the *sequence*.
2. For a parsing expression **e** the result of [list * **e**] is a parsing expression as well. This is the *kleene closure*, describing zero or more repetitions.
3. For a parsing expression **e** the result of [list + **e**] is a parsing expression as well. This is the *positive kleene closure*, describing one or more repetitions.
4. For a parsing expression **e** the result of [list & **e**] is a parsing expression as well. This is the *and lookahead predicate*.
5. For a parsing expression **e** the result of [list! **e**] is a parsing expression as well. This is the *not lookahead predicate*.
6. For a parsing expression **e** the result of [list ? **e**] is a parsing expression as well. This is the *optional input*.

Canonical serialization

The canonical serialization of a parsing expression has the format as specified in the previous

item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this parsing expression.

1. The string representation of the value is the canonical representation of a pure Tcl list. I.e. it does not contain superfluous whitespace.
2. Terminals are *not* encoded as ranges (where start and end of the range are identical).

Example

Assuming the parsing expression shown on the right-hand side of the rule

```
    Expression <- Term (AddOp Term)*
```

then its canonical serialization (except for whitespace) is

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pt* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Parsing and Grammars

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

pt_peg_container_peg

Keywords: pt_peg_container_peg, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt::peg::container::peg - PEG Storage. Canned PEG grammar specification

Synopsis

package require Tcl 8.5 9
package require snit
package require pt::peg::container::peg ?**1**?
package require pt::peg::container

Description

Are you lost ? Do you have trouble understanding this document ? In that case please read the

overview provided by the *Introduction to Parser Tools*. This document is the entrypoint to the whole system the current package is a part of.

This package provides a sub-type of **pt::peg::container** which is preloaded with a parsing expression grammar describing a textual format for parsing expression grammars.

The sub-type provides the exact same API as **pt::peg::container**. Instead of duplicating its contents the reader is asked to read the referenced document.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pt* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Parsing and Grammars

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_peg_export

Keywords: pt_peg_export, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt::peg::export - PEG Export

Synopsis

package require Tcl 8.5 9
package require snit
package require struct::map
package require pt::peg
package require pluginmgr
package require pt::peg::export ?**1.0.2**?
::pt::peg::export *objectName*
objectName method ?*arg arg ...*?
objectName **destroy**
objectName **export serial** *serial* ?*format*?
objectName **export object** *object* ?*format*?
objectName **configuration names**
objectName **configuration get**

objectName **configuration set** *name* ?*value*?
objectName **configuration unset** *pattern...*

Description

Are you lost ? Do you have trouble understanding this document ? In that case please read the overview provided by the *Introduction to Parser Tools*. This document is the entrypoint to the whole system the current package is a part of.

This package provides a manager for parsing expression grammars, with each instance handling a set of plugins for the export of them to other formats, i.e. their conversion to, for example *nroff* , *HTML*, etc.

It resides in the Export section of the Core Layer of Parser Tools, and is one of the three pillars the management of parsing expression grammars resides on.

IMAGE: arch_core_export

The other two pillars are, as shown above

1. *PEG Import*, and
2. *PEG Storage*

For information about the data structure which is the major input to the manager objects provided by this package see the section PEG serialization format.

The plugin system of this class is based on the package **pluginmgr**, and configured to look for plugins using

1. the environment variable **GRAMMAR_PEG_EXPORT_PLUGINS**,
2. the environment variable **GRAMMAR_PEG_PLUGINS**,
3. the environment variable **GRAMMAR_PLUGINS**,
4. the path "*~/.grammar/peg/export/plugin*"
5. the path "*~/.grammar/peg/plugin*"
6. the path "*~/.grammar/plugin*"
7. the path "*~/.grammar/peg/export/plugins*"
8. the path "*~/.grammar/peg/plugins*"
9. the path "*~/.grammar/plugins*"
10. the registry entry "HKEY_CURRENT_USER\SOFTWARE\GRAMMAR\PEG\EXPORT\PLUGINS"
11. the registry entry "HKEY_CURRENT_USER\SOFTWARE\GRAMMAR\PEG\PLUGINS"
12. the registry entry "HKEY_CURRENT_USER\SOFTWARE\GRAMMAR\PLUGINS"

The last three are used only when the package is run on a machine using the Windows(tm) operating system.

The whole system is delivered with three predefined export plugins, namely

container
: See *PEG Export Plugin. To CONTAINER format* for details.

json
: See *PEG Export Plugin. To JSON format* for details.

peg
: See *PEG Export Plugin. To PEG format* for details.

For readers wishing to write their own export plugin for some format, i.e. *plugin writer*s, reading and understanding the *Parser Tools Export API* specification is an absolute necessity, as it

documents the interaction between this package and its plugins in detail.

API

Package commands

::pt::peg::export *objectName*

This command creates a new export manager object with an associated Tcl command whose name is *objectName*. This *object* command is explained in full detail in the sections Object command and Object methods. The object command will be created under the current namespace if the *objectName* is not fully qualified, and in the specified namespace otherwise.

Object command

All objects created by the **::pt::peg::export** command have the following general form:

objectName method ?*arg arg ...*?

The method **method** and its *arg*'uments determine the exact behavior of the command. See section Object methods for the detailed specifications.

Object methods

***objectName* destroy**

This method destroys the object it is invoked for.

***objectName* export serial *serial* ?*format*?**

This method takes the canonical serialization of a parsing expression grammar stored in *serial* and converts it to the specified *format*, using the export plugin for the format. This will fail with an error if no plugin could be found for the format. The string generated by the conversion process is returned as the result of this method.

If no format is specified the method defaults to **text**.

The specification of what a *canonical* serialization is can be found in the section PEG serialization format.

The plugin has to conform to the interface documented in the *Parser Tools Export API* specification.

***objectName* export object *object* ?*format*?**

This method is a convenient wrapper around the export serial method described by the previous item. It expects that *object* is an object command supporting a **serialize** method returning the canonical serialization of a parsing expression grammar. It invokes that method, feeds the result into export serial and returns the resulting string as its own result.

***objectName* configuration names**

This method returns a list containing the names of all configuration options currently known to the object.

***objectName* configuration get**

This method returns a dictionary containing the names and values of all configuration options currently known to the object.

***objectName* configuration set *name* ?*value*?**

This method sets the configuration option *name* to the specified *value* and returns the new

value of the option.

If no value is specified it simply returns the current value, without changing it.

Note that these configuration options and their values are simply passed to a plugin when the actual export is performed. It is the plugin which checks the validity, not the manager.

***objectName* configuration unset *pattern*...**

This method unsets all configuration options matching the specified glob *pattern*s. If no pattern is specified it will unset all currently defined configuration options.

PEG serialization format

Here we specify the format used by the Parser Tools to serialize Parsing Expression Grammars as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a PEG may have more than one regular serialization only exactly one of them will be *canonical*.

regular serialization

1. The serialization of any PEG is a nested Tcl dictionary.
1. This dictionary holds a single key, **pt::grammar::peg**, and its value. This value holds the contents of the grammar.
2. The contents of the grammar are a Tcl dictionary holding the set of nonterminal symbols and the starting expression. The relevant keys and their values are

rules

The value is a Tcl dictionary whose keys are the names of the nonterminal symbols known to the grammar.

1. Each nonterminal symbol may occur only once.
2. The empty string is not a legal nonterminal symbol.
3. The value for each symbol is a Tcl dictionary itself. The relevant keys and their values in this dictionary are

is

The value is the serialization of the parsing expression describing the symbols sentennial structure, as specified in the section PE serialization format.

mode

The value can be one of three values specifying how a parser should handle the semantic value produced by the symbol.

value

The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal itself, which has the ASTs of the symbol's right hand side as its children.

leaf

The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal, without any children. Any ASTs generated by the symbol's right hand side are discarded.

void

The nonterminal has no semantic value. Any ASTs generated by the symbol's right hand side are discarded (as well).

```
**start**
: The value is the serialization of the start parsing expression of
    the grammar, as specified in the section [PE serialization
    format].
```

1. The terminal symbols of the grammar are specified implicitly as the set of all terminal symbols used in the start expression and on the RHS of the grammar rules.

canonical serialization

The canonical serialization of a grammar has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this grammar.

1. The keys found in all the nested Tcl dictionaries are sorted in ascending dictionary order, as generated by Tcl's builtin command **lsort -increasing -dict**.
2. The string representation of the value is the canonical representation of a Tcl dictionary. I.e. it does not contain superfluous whitespace.

Example

Assuming the following PEG for simple mathematical expressions

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

then its canonical serialization (except for whitespace) is

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                  mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}                mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                  mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                  mode value}
        MulOp      {is {/ {t *} {t /}}
                                  mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
                                  mode value}
        Sign       {is {/ {t -} {t +}}
```

```
                                    mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                    mode value}
    }
    start {n Expression}
}
```

PE serialization format

Here we specify the format used by the Parser Tools to serialize Parsing Expressions as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a parsing expression may have more than one regular serialization only exactly one of them will be *canonical*.

Regular serialization

see below ...

Atomic Parsing Expressions

1. The string **epsilon** is an atomic parsing expression. It matches the empty string.
1. The string **dot** is an atomic parsing expression. It matches any character.
2. The string **alnum** is an atomic parsing expression. It matches any Unicode alphabet or digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
3. The string **alpha** is an atomic parsing expression. It matches any Unicode alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
4. The string **ascii** is an atomic parsing expression. It matches any Unicode character below U0080. This is a custom extension of PEs based on Tcl's builtin command **string is**.
5. The string **control** is an atomic parsing expression. It matches any Unicode control character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
6. The string **digit** is an atomic parsing expression. It matches any Unicode digit character. Note that this includes characters outside of the [0..9] range. This is a custom extension of PEs based on Tcl's builtin command **string is**.
7. The string **graph** is an atomic parsing expression. It matches any Unicode printing character, except for space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
8. The string **lower** is an atomic parsing expression. It matches any Unicode lower-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
9. The string **print** is an atomic parsing expression. It matches any Unicode printing character, including space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
10. The string **punct** is an atomic parsing expression. It matches any Unicode punctuation character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
11. The string **space** is an atomic parsing expression. It matches any Unicode space character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
12. The string **upper** is an atomic parsing expression. It matches any Unicode upper-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command

string is.

13. The string **wordchar** is an atomic parsing expression. It matches any Unicode word character. This is any alphanumeric character (see alnum), and any connector punctuation characters (e.g. underscore). This is a custom extension of PEs based on Tcl's builtin command **string is**.
14. The string **xdigit** is an atomic parsing expression. It matches any hexadecimal digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
15. The string **ddigit** is an atomic parsing expression. It matches any decimal digit character. This is a custom extension of PEs based on Tcl's builtin command **regexp**.
16. The expression [list t **x**] is an atomic parsing expression. It matches the terminal string **x**.
17. The expression [list n **A**] is an atomic parsing expression. It matches the nonterminal **A**.

Combined Parsing Expressions

1. For parsing expressions **e1**, **e2**, ... the result of [list / **e1 e2** ...] is a parsing expression as well. This is the *ordered choice*, aka *prioritized choice*.
1. For parsing expressions **e1**, **e2**, ... the result of [list x **e1 e2** ...] is a parsing expression as well. This is the *sequence*.
2. For a parsing expression **e** the result of [list * **e**] is a parsing expression as well. This is the *kleene closure*, describing zero or more repetitions.
3. For a parsing expression **e** the result of [list + **e**] is a parsing expression as well. This is the *positive kleene closure*, describing one or more repetitions.
4. For a parsing expression **e** the result of [list & **e**] is a parsing expression as well. This is the *and lookahead predicate*.
5. For a parsing expression **e** the result of [list! **e**] is a parsing expression as well. This is the *not lookahead predicate*.
6. For a parsing expression **e** the result of [list ? **e**] is a parsing expression as well. This is the *optional input*.

Canonical serialization

The canonical serialization of a parsing expression has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this parsing expression.

1. The string representation of the value is the canonical representation of a pure Tcl list. I.e. it does not contain superfluous whitespace.
2. Terminals are *not* encoded as ranges (where start and end of the range are identical).

Example

Assuming the parsing expression shown on the right-hand side of the rule

```
    Expression <- Term (AddOp Term)*
```

then its canonical serialization (except for whitespace) is

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you

want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pt* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Parsing and Grammars

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_peg_export_container

Keywords: pt_peg_export_container, CONTAINER, EBNF, LL(k), PEG, TDPL, context-free languages , export, expression, grammar, matching, parser, parsing expression, parsing expression grammar, plugin, push down automaton, recursive descent, serialization, state, top-down parsing languages, transducer

Name

pt::peg::export::container - PEG Export Plugin. Write CONTAINER format

Synopsis

package require Tcl 8.5 9
package require pt::peg::export::container ?**1**?
package require pt::peg::to::container
export *serial configuration*

Description

Are you lost ? Do you have trouble understanding this document ? In that case please read the overview provided by the *Introduction to Parser Tools*. This document is the entrypoint to the whole system the current package is a part of.

This package implements the parsing expression grammar export plugin for the generation of CONTAINER markup.

It resides in the Export section of the Core Layer of Parser Tools and is intended to be used by **pt::peg::export**, the export manager, sitting between it and the corresponding core conversion functionality provided by pt::peg::to::container.

IMAGE: arch_core_eplugins

While the direct use of this package with a regular interpreter is possible, this is strongly disrecommended and requires a number of contortions to provide the expected environment. The proper way to use this functionality depends on the situation:

1. In an untrusted environment the proper access is through the package **pt::peg::export** and the export manager objects it provides.

2. In a trusted environment however simply use the package pt::peg::to::container and access the core conversion functionality directly.

API

The API provided by this package satisfies the specification of the Plugin API found in the *Parser Tools Export API* specification.

export *serial configuration*

This command takes the canonical serialization of a parsing expression grammar, as specified in section Peg serialization format, and contained in *serial*, the *configuration*, a dictionary, and generates CONTAINER markup encoding the grammar. The created string is then returned as the result of the command.

Configuration

The CONTAINER export plugin recognizes the following configuration variables and changes its behaviour as they specify.

enum *mode*

The value of this configuration variable controls which methods of **pt::peg** instances the plugin will use to specify the grammar. There are two legal values

bulk

In this mode the methods **start**, **add**, **modes**, and **rules** are used to specify the grammar in a bulk manner, i.e. as a set of nonterminal symbols, and two dictionaries mapping from the symbols to their semantic modes and parsing expressions.

This mode is the default.

incremental

In this mode the methods **start**, **add**, **mode**, and **rule** are used to specify the grammar piecemal, with each nonterminal having its own block of defining commands.

string *template*

If this configuration variable is set it is assumed to contain a string into which to put the generated code and other configuration data. The various locations are expected to be specified with the following placeholders:

@user@

To be replaced with the value of the configuration variable **user**.

@format@

To be replaced with the the constant **CONTAINER**.

@file@

To be replaced with the value of the configuration variable **file**.

@name@

To be replaced with the value of the configuration variable **name**.

@mode@

To be replaced with the value of the configuration variable **mode**.

@code@

To be replaced with the generated code.

If this configuration variable is not set, or empty, then the plugin falls back to a standard template, which is defined as "**@code@**".

Note that this plugin may ignore the standard configuration variables **user**, **format**, **file**, and their values, depending on the chosen template.

The content of the standard configuration variable **name**, if set, is used as name of the grammar in the output. Otherwise the plugin falls back to the default name **a_pe_grammar**.

Grammar container

The **container** format is another form of describing parsing expression grammars. While data in this format is executable it does not constitute a parser for the grammar. It always has to be used in conjunction with the package **pt::peg::interp**, a grammar interpreter.

The format represents grammars by a **snit::type**, i.e. class, whose instances are API-compatible to the instances of the **pt::peg::container** package, and which are preloaded with the grammar in question.

It has no direct formal specification beyond what was said above.

Example

Assuming the following PEG for simple mathematical expressions

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

one possible CONTAINER serialization for it is

```
snit::type a_pe_grammar {
    constructor {} {
        install myg using pt::peg::container ${selfns}::G
        $myg start {n Expression}
        $myg add   AddOp Digit Expression Factor MulOp Number Sign Term
        $myg modes {
            AddOp      value
            Digit      value
            Expression value
            Factor     value
            MulOp      value
            Number     value
            Sign       value
            Term       value
        }
```

```
        $myg rules {
            AddOp      {/ {t -} {t +}}
            Digit      {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}
            Expression {/ {x {t \50} {n Expression} {t \51}} {x {n
    Factor} {* {x {n MulOp} {n Factor}}}}}
            Factor     {x {n Term} {* {x {n AddOp} {n Term}}}}
            MulOp      {/ {t *} {t /}}
            Number     {x {? {n Sign}} {+ {n Digit}}}
            Sign       {/ {t -} {t +}}
            Term       {n Number}
        }
        return
    }

    component myg
    delegate method * to myg
}
```

Peg serialization format

Here we specify the format used by the Parser Tools to serialize Parsing Expression Grammars as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a PEG may have more than one regular serialization only exactly one of them will be *canonical*.

regular serialization

1. The serialization of any PEG is a nested Tcl dictionary.
1. This dictionary holds a single key, **pt::grammar::peg**, and its value. This value holds the contents of the grammar.
2. The contents of the grammar are a Tcl dictionary holding the set of nonterminal symbols and the starting expression. The relevant keys and their values are

   rules

   The value is a Tcl dictionary whose keys are the names of the nonterminal symbols known to the grammar.

   1. Each nonterminal symbol may occur only once.
   2. The empty string is not a legal nonterminal symbol.
   3. The value for each symbol is a Tcl dictionary itself. The relevant keys and their values in this dictionary are

      is

      The value is the serialization of the parsing expression describing the symbols sentennial structure, as specified in the section PE serialization format.

      mode

      The value can be one of three values specifying how a parser should handle the semantic value produced by the symbol.

      value

      The semantic value of the nonterminal symbol is an abstract syntax tree

consisting of a single node node for the nonterminal itself, which has the ASTs of the symbol's right hand side as its children.

leaf
The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal, without any children. Any ASTs generated by the symbol's right hand side are discarded.

void
The nonterminal has no semantic value. Any ASTs generated by the symbol's right hand side are discarded (as well).

```
**start**
: The value is the serialization of the start parsing expression of
    the grammar, as specified in the section [PE serialization
    format].
```

1. The terminal symbols of the grammar are specified implicitly as the set of all terminal symbols used in the start expression and on the RHS of the grammar rules.

canonical serialization
The canonical serialization of a grammar has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this grammar.

1. The keys found in all the nested Tcl dictionaries are sorted in ascending dictionary order, as generated by Tcl's builtin command **lsort -increasing -dict**.
2. The string representation of the value is the canonical representation of a Tcl dictionary. I.e. it does not contain superfluous whitespace.

Example

Assuming the following PEG for simple mathematical expressions

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

then its canonical serialization (except for whitespace) is

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}              mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                mode value}
        MulOp      {is {/ {t *} {t /}}
                                mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
                                mode value}
        Sign       {is {/ {t -} {t +}}
                                mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                mode value}
    }
    start {n Expression}
}
```

PE serialization format

Here we specify the format used by the Parser Tools to serialize Parsing Expressions as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a parsing expression may have more than one regular serialization only exactly one of them will be *canonical*.

Regular serialization

see below ...

Atomic Parsing Expressions

1. The string **epsilon** is an atomic parsing expression. It matches the empty string.
1. The string **dot** is an atomic parsing expression. It matches any character.
2. The string **alnum** is an atomic parsing expression. It matches any Unicode alphabet or digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
3. The string **alpha** is an atomic parsing expression. It matches any Unicode alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
4. The string **ascii** is an atomic parsing expression. It matches any Unicode character below U0080. This is a custom extension of PEs based on Tcl's builtin command **string is**.
5. The string **control** is an atomic parsing expression. It matches any Unicode control character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
6. The string **digit** is an atomic parsing expression. It matches any Unicode digit character. Note that this includes characters outside of the [0..9] range. This is a custom extension of PEs based on Tcl's builtin command **string is**.

7. The string **graph** is an atomic parsing expression. It matches any Unicode printing character, except for space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
8. The string **lower** is an atomic parsing expression. It matches any Unicode lower-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
9. The string **print** is an atomic parsing expression. It matches any Unicode printing character, including space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
10. The string **punct** is an atomic parsing expression. It matches any Unicode punctuation character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
11. The string **space** is an atomic parsing expression. It matches any Unicode space character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
12. The string **upper** is an atomic parsing expression. It matches any Unicode upper-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
13. The string **wordchar** is an atomic parsing expression. It matches any Unicode word character. This is any alphanumeric character (see alnum), and any connector punctuation characters (e.g. underscore). This is a custom extension of PEs based on Tcl's builtin command **string is**.
14. The string **xdigit** is an atomic parsing expression. It matches any hexadecimal digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
15. The string **ddigit** is an atomic parsing expression. It matches any decimal digit character. This is a custom extension of PEs based on Tcl's builtin command **regexp**.
16. The expression [list t **x**] is an atomic parsing expression. It matches the terminal string **x**.
17. The expression [list n **A**] is an atomic parsing expression. It matches the nonterminal **A**.

Combined Parsing Expressions

1. For parsing expressions **e1**, **e2**, ... the result of [list / **e1 e2** ...] is a parsing expression as well. This is the *ordered choice*, aka *prioritized choice*.
1. For parsing expressions **e1**, **e2**, ... the result of [list x **e1 e2** ...] is a parsing expression as well. This is the *sequence*.
2. For a parsing expression **e** the result of [list * **e**] is a parsing expression as well. This is the *kleene closure*, describing zero or more repetitions.
3. For a parsing expression **e** the result of [list + **e**] is a parsing expression as well. This is the *positive kleene closure*, describing one or more repetitions.
4. For a parsing expression **e** the result of [list & **e**] is a parsing expression as well. This is the *and lookahead predicate*.
5. For a parsing expression **e** the result of [list! **e**] is a parsing expression as well. This is the *not lookahead predicate*.
6. For a parsing expression **e** the result of [list ? **e**] is a parsing expression as well. This is the *optional input*.

Canonical serialization

The canonical serialization of a parsing expression has the format as specified in the previous

item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this parsing expression.

1. The string representation of the value is the canonical representation of a pure Tcl list. I.e. it does not contain superfluous whitespace.
2. Terminals are *not* encoded as ranges (where start and end of the range are identical).

Example

Assuming the parsing expression shown on the right-hand side of the rule

```
    Expression <- Term (AddOp Term)*
```

then its canonical serialization (except for whitespace) is

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pt* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Parsing and Grammars

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_peg_export_json

Keywords: pt_peg_export_json, EBNF, JSON, LL(k), PEG, TDPL, context-free languages, export, expression, grammar, matching, parser, parsing expression, parsing expression grammar, plugin , push down automaton, recursive descent, serialization, state, top-down parsing languages, transducer

Name

pt::peg::export::json - PEG Export Plugin. Write JSON format

Synopsis

package require Tcl 8.5 9
package require pt::peg::export::json ?**1**?
package require pt::peg::to::json
export *serial configuration*

Description

Are you lost ? Do you have trouble understanding this document ? In that case please read the overview provided by the *Introduction to Parser Tools*. This document is the entrypoint to the whole system the current package is a part of.

This package implements the parsing expression grammar export plugin for the generation of JSON markup.

It resides in the Export section of the Core Layer of Parser Tools and is intended to be used by **pt::peg::export**, the export manager, sitting between it and the corresponding core conversion functionality provided by pt::peg::to::json.

IMAGE: arch_core_eplugins

While the direct use of this package with a regular interpreter is possible, this is strongly disrecommended and requires a number of contortions to provide the expected environment. The proper way to use this functionality depends on the situation:

1. In an untrusted environment the proper access is through the package **pt::peg::export** and the export manager objects it provides.
2. In a trusted environment however simply use the package pt::peg::to::json and access the core conversion functionality directly.

API

The API provided by this package satisfies the specification of the Plugin API found in the *Parser Tools Export API* specification.

export *serial configuration*
: This command takes the canonical serialization of a parsing expression grammar, as specified in section Peg serialization format, and contained in *serial*, the *configuration*, a dictionary, and generates JSON markup encoding the grammar. The created string is then returned as the result of the command.

Configuration

The JSON export plugin recognizes the following configuration variables and changes its behaviour as they specify.

boolean *indented*
: If this flag is set the plugin will break the generated JSON code across lines and indent it according to its inner structure, with each key of a dictionary on a separate line.

  If this flag is not set (the default), the whole JSON object will be written on a single line, with minimum spacing between all elements.

boolean *aligned*
: If this flag is set the generator ensures that the values for the keys in a dictionary are vertically aligned with each other, for a nice table effect. To make this work this also implies that **indented** is set.

  If this flag is not set (the default), the output is formatted as per the value of **indented**, without trying to align the values for dictionary keys.

Note that this plugin ignores the standard configuration variables **user**, **format**, **file**, and **name** , and their values.

JSON grammar exchange format

The **json** format for parsing expression grammars was written as a data exchange format not bound to Tcl. It was defined to allow the exchange of grammars with PackRat/PEG based parser generators for other languages.

It is formally specified by the rules below:

1. The JSON of any PEG is a JSON object.
2. This object holds a single key, **pt::grammar::peg**, and its value. This value holds the contents of the grammar.
3. The contents of the grammar are a JSON object holding the set of nonterminal symbols and the starting expression. The relevant keys and their values are

   rules
   The value is a JSON object whose keys are the names of the nonterminal symbols known to the grammar.
   1. Each nonterminal symbol may occur only once.
   2. The empty string is not a legal nonterminal symbol.
   3. The value for each symbol is a JSON object itself. The relevant keys and their values in this dictionary are

      is
      The value is a JSON string holding the Tcl serialization of the parsing expression describing the symbols sentennial structure, as specified in the section PE serialization format.

      mode
      The value is a JSON holding holding one of three values specifying how a parser should handle the semantic value produced by the symbol.

      value
      The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal itself, which has the ASTs of the symbol's right hand side as its children.

      leaf
      The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal, without any children. Any ASTs generated by the symbol's right hand side are discarded.

      void
      The nonterminal has no semantic value. Any ASTs generated by the symbol's right hand side are discarded (as well).

```
**start**
: The value is a JSON string holding the Tcl serialization of the
    start parsing expression of the grammar, as specified in the
    section [PE serialization format].
```

1. The terminal symbols of the grammar are specified implicitly as the set of all terminal symbols used in the start expression and on the RHS of the grammar rules.

As an aside to the advanced reader, this is pretty much the same as the Tcl serialization of PE

grammars, as specified in section Peg serialization format, except that the Tcl dictionaries and lists of that format are mapped to JSON objects and arrays. Only the parsing expressions themselves are not translated further, but kept as JSON strings containing a nested Tcl list, and there is no concept of canonicity for the JSON either.

Example

Assuming the following PEG for simple mathematical expressions

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

a JSON serialization for it is

```
{
    "pt::grammar::peg" : {
        "rules" : {
            "AddOp"     : {
                "is"   : "\/ {t -} {t +}",
                "mode" : "value"
            },
            "Digit"     : {
                "is"   : "\/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6}
    {t 7} {t 8} {t 9}",
                "mode" : "value"
            },
            "Expression" : {
                "is"   : "\/ {x {t (} {n Expression} {t )}} {x {n
    Factor} {* {x {n MulOp} {n Factor}}}}",
                "mode" : "value"
            },
            "Factor"    : {
                "is"   : "x {n Term} {* {x {n AddOp} {n Term}}}",
                "mode" : "value"
            },
            "MulOp"     : {
                "is"   : "\/ {t *} {t \/}",
                "mode" : "value"
            },
            "Number"    : {
                "is"   : "x {? {n Sign}} {+ {n Digit}}",
```

```
                "mode" : "value"
            },
            "Sign"      : {
                "is"   : "\/ {t -} {t +}",
                "mode" : "value"
            },
            "Term"      : {
                "is"   : "n Number",
                "mode" : "value"
            }
        },
        "start" : "n Expression"
    }
}
```

and a Tcl serialization of the same is

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}               mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                mode value}
        MulOp      {is {/ {t *} {t /}}
                                mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
                                mode value}
        Sign       {is {/ {t -} {t +}}
                                mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                mode value}
    }
    start {n Expression}
}
```

The similarity of the latter to the JSON should be quite obvious.

Peg serialization format

Here we specify the format used by the Parser Tools to serialize Parsing Expression Grammars as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a PEG may have more than one regular serialization only exactly one of them will be *canonical*.

regular serialization

1. The serialization of any PEG is a nested Tcl dictionary.
1. This dictionary holds a single key, **pt::grammar::peg**, and its value. This value holds the contents of the grammar.
2. The contents of the grammar are a Tcl dictionary holding the set of nonterminal symbols and the starting expression. The relevant keys and their values are

   rules
   The value is a Tcl dictionary whose keys are the names of the nonterminal symbols known to the grammar.
   1. Each nonterminal symbol may occur only once.
   2. The empty string is not a legal nonterminal symbol.
   3. The value for each symbol is a Tcl dictionary itself. The relevant keys and their values in this dictionary are

      is
      The value is the serialization of the parsing expression describing the symbols sentennial structure, as specified in the section PE serialization format.

      mode
      The value can be one of three values specifying how a parser should handle the semantic value produced by the symbol.

      value
      The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal itself, which has the ASTs of the symbol's right hand side as its children.

      leaf
      The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal, without any children. Any ASTs generated by the symbol's right hand side are discarded.

      void
      The nonterminal has no semantic value. Any ASTs generated by the symbol's right hand side are discarded (as well).

```
**start**
: The value is the serialization of the start parsing expression of
    the grammar, as specified in the section [PE serialization
    format].
```

1. The terminal symbols of the grammar are specified implicitly as the set of all terminal symbols used in the start expression and on the RHS of the grammar rules.

canonical serialization
The canonical serialization of a grammar has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this grammar.

1. The keys found in all the nested Tcl dictionaries are sorted in ascending dictionary order, as generated by Tcl's builtin command **lsort -increasing -dict**.
2. The string representation of the value is the canonical representation of a Tcl dictionary. I.e.

it does not contain superfluous whitespace.

Example

Assuming the following PEG for simple mathematical expressions

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

then its canonical serialization (except for whitespace) is

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}              mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                mode value}
        MulOp      {is {/ {t *} {t /}}
                                mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
                                mode value}
        Sign       {is {/ {t -} {t +}}
                                mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                mode value}
    }
    start {n Expression}
}
```

PE serialization format

Here we specify the format used by the Parser Tools to serialize Parsing Expressions as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a parsing expression may have more than one regular serialization only exactly one of them will be *canonical*.

Regular serialization

see below ...

Atomic Parsing Expressions

1. The string **epsilon** is an atomic parsing expression. It matches the empty string.
1. The string **dot** is an atomic parsing expression. It matches any character.
2. The string **alnum** is an atomic parsing expression. It matches any Unicode alphabet or digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
3. The string **alpha** is an atomic parsing expression. It matches any Unicode alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
4. The string **ascii** is an atomic parsing expression. It matches any Unicode character below U0080. This is a custom extension of PEs based on Tcl's builtin command **string is**.
5. The string **control** is an atomic parsing expression. It matches any Unicode control character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
6. The string **digit** is an atomic parsing expression. It matches any Unicode digit character. Note that this includes characters outside of the [0..9] range. This is a custom extension of PEs based on Tcl's builtin command **string is**.
7. The string **graph** is an atomic parsing expression. It matches any Unicode printing character, except for space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
8. The string **lower** is an atomic parsing expression. It matches any Unicode lower-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
9. The string **print** is an atomic parsing expression. It matches any Unicode printing character, including space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
10. The string **punct** is an atomic parsing expression. It matches any Unicode punctuation character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
11. The string **space** is an atomic parsing expression. It matches any Unicode space character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
12. The string **upper** is an atomic parsing expression. It matches any Unicode upper-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
13. The string **wordchar** is an atomic parsing expression. It matches any Unicode word character. This is any alphanumeric character (see alnum), and any connector punctuation characters (e.g. underscore). This is a custom extension of PEs based on Tcl's builtin command **string is**.
14. The string **xdigit** is an atomic parsing expression. It matches any hexadecimal digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
15. The string **ddigit** is an atomic parsing expression. It matches any decimal digit character. This is a custom extension of PEs based on Tcl's builtin command **regexp**.
16. The expression [list t **x**] is an atomic parsing expression. It matches the terminal string **x**.
17. The expression [list n **A**] is an atomic parsing expression. It matches the nonterminal **A**.

Combined Parsing Expressions

1. For parsing expressions **e1**, **e2**, ... the result of [list / **e1 e2** ...] is a parsing expression as well. This is the *ordered choice*, aka *prioritized choice*.

1. For parsing expressions **e1**, **e2**, ... the result of [list x **e1 e2** ...] is a parsing expression as well. This is the *sequence*.
2. For a parsing expression **e** the result of [list * **e**] is a parsing expression as well. This is the *kleene closure*, describing zero or more repetitions.
3. For a parsing expression **e** the result of [list + **e**] is a parsing expression as well. This is the *positive kleene closure*, describing one or more repetitions.
4. For a parsing expression **e** the result of [list & **e**] is a parsing expression as well. This is the *and lookahead predicate*.
5. For a parsing expression **e** the result of [list! **e**] is a parsing expression as well. This is the *not lookahead predicate*.
6. For a parsing expression **e** the result of [list ? **e**] is a parsing expression as well. This is the *optional input*.

Canonical serialization

The canonical serialization of a parsing expression has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this parsing expression.

1. The string representation of the value is the canonical representation of a pure Tcl list. I.e. it does not contain superfluous whitespace.
2. Terminals are *not* encoded as ranges (where start and end of the range are identical).

Example

Assuming the parsing expression shown on the right-hand side of the rule

```
    Expression <- Term (AddOp Term)*
```

then its canonical serialization (except for whitespace) is

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pt* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Parsing and Grammars

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

pt_peg_export_peg

Keywords: pt_peg_export_peg, EBNF, LL(k), PEG, TDPL, context-free languages, export, expression, grammar, matching, parser, parsing expression, parsing expression grammar, plugin , push down automaton, recursive descent, serialization, state, top-down parsing languages, transducer

Name

pt::peg::export::peg - PEG Export Plugin. Write PEG format

Synopsis

package require Tcl 8.5 9
package require pt::peg::export::peg ?**1**?
package require pt::peg::to::peg
export *serial configuration*

Description

Are you lost ? Do you have trouble understanding this document ? In that case please read the overview provided by the *Introduction to Parser Tools*. This document is the entrypoint to the whole system the current package is a part of.

This package implements the parsing expression grammar export plugin for the generation of PEG markup.

It resides in the Export section of the Core Layer of Parser Tools and is intended to be used by **pt::peg::export**, the export manager, sitting between it and the corresponding core conversion functionality provided by pt::peg::to::peg.

IMAGE: arch_core_eplugins

While the direct use of this package with a regular interpreter is possible, this is strongly disrecommended and requires a number of contortions to provide the expected environment. The proper way to use this functionality depends on the situation:

1. In an untrusted environment the proper access is through the package **pt::peg::export** and the export manager objects it provides.
2. In a trusted environment however simply use the package pt::peg::to::peg and access the core conversion functionality directly.

API

The API provided by this package satisfies the specification of the Plugin API found in the *Parser Tools Export API* specification.

export *serial configuration*
: This command takes the canonical serialization of a parsing expression grammar, as specified in section PEG serialization format, and contained in *serial*, the *configuration*, a dictionary, and generates PEG markup encoding the grammar. The created string is then returned as the result of the command.

Configuration

The PEG export plugin recognizes the following configuration variables and changes its behaviour as they specify.

string *template*

If this configuration variable is set it is assumed to contain a string into which to put the generated text and other configuration data. The various locations are expected to be specified with the following placeholders:

@user@
To be replaced with the value of the configuration variable **user**.

@format@
To be replaced with the the constant **PEG**.

@file@
To be replaced with the value of the configuration variable **file**.

@name@
To be replaced with the value of the configuration variable **name**.

@code@
To be replaced with the generated text.

If this configuration variable is not set, or empty, then the plugin falls back to a standard template, which is defined as "**@code@**".

Note that this plugin may ignore the standard configuration variables **user**, **format**, **file**, and their values, depending on the chosen template.

The content of the standard configuration variable **name**, if set, is used as name of the grammar in the output. Otherwise the plugin falls back to the default name **a_pe_grammar**.

PEG specification language

peg, a language for the specification of parsing expression grammars is meant to be human readable, and writable as well, yet strict enough to allow its processing by machine. Like any computer language. It was defined to make writing the specification of a grammar easy, something the other formats found in the Parser Tools do not lend themselves too.

It is formally specified by the grammar shown below, written in itself. For a tutorial / introduction to the language please go and read the *PEG Language Tutorial*.

```
PEG pe_grammar_for_peg (Grammar)

    # ------------------------------------------------------------------
        # Syntactical constructs

        Grammar         <- WHITESPACE Header Definition* Final EOF ;

        Header          <- PEG Identifier StartExpr ;
        Definition      <- Attribute? Identifier IS Expression
    SEMICOLON ;
        Attribute       <- (VOID / LEAF) COLON ;
        Expression      <- Sequence (SLASH Sequence)* ;
        Sequence        <- Prefix+ ;
        Prefix          <- (AND / NOT)? Suffix ;
        Suffix          <- Primary (QUESTION / STAR / PLUS)? ;
```

```
        Primary         <- ALNUM / ALPHA / ASCII / CONTROL / DDIGIT /
    DIGIT
                        /  GRAPH / LOWER / PRINTABLE / PUNCT / SPACE /
    UPPER
                        /  WORDCHAR / XDIGIT
                        / Identifier
                        /  OPEN Expression CLOSE
                        /  Literal
                        /  Class
                        /  DOT
                        ;
        Literal         <- APOSTROPH  (!APOSTROPH  Char)* APOSTROPH
    WHITESPACE
                        /  DAPOSTROPH (!DAPOSTROPH Char)* DAPOSTROPH
    WHITESPACE ;
        Class           <- OPENB (!CLOSEB Range)* CLOSEB WHITESPACE ;
        Range           <- Char TO Char / Char ;

        StartExpr       <- OPEN Expression CLOSE ;
void:   Final           <- "END" WHITESPACE SEMICOLON WHITESPACE ;

        # ------------------------------------------------------------
    -------
        # Lexing constructs

        Identifier      <- Ident WHITESPACE ;
leaf:   Ident           <- ([_:] / <alpha>) ([_:] / <alnum>)* ;
        Char            <- CharSpecial / CharOctalFull / CharOctalPart
                        /  CharUnicode / CharUnescaped
                        ;

leaf:   CharSpecial     <- "\\" [nrt'"\[\]\\] ;
leaf:   CharOctalFull   <- "\\" [0-3][0-7][0-7] ;
leaf:   CharOctalPart   <- "\\" [0-7][0-7]? ;
leaf:   CharUnicode     <- "\\" 'u' HexDigit (HexDigit (HexDigit
    HexDigit?)?)? ;
leaf:   CharUnescaped   <- !"\\" . ;

void:   HexDigit        <- [0-9a-fA-F] ;

void:   TO              <- '-'           ;
void:   OPENB           <- "["           ;
void:   CLOSEB          <- "]"           ;
void:   APOSTROPH       <- "'"           ;
void:   DAPOSTROPH      <- '"'           ;
void:   PEG             <- "PEG" !([_:] / <alnum>) WHITESPACE ;
```

```
void:   IS              <- "<-"    WHITESPACE ;
leaf:   VOID            <- "void"  WHITESPACE ; # Implies that
    definition has no semantic value.
leaf:   LEAF            <- "leaf"  WHITESPACE ; # Implies that
    definition has no terminals.
void:   SEMICOLON       <- ";"     WHITESPACE ;
void:   COLON           <- ":"     WHITESPACE ;
void:   SLASH           <- "/"     WHITESPACE ;
leaf:   AND             <- "&"     WHITESPACE ;
leaf:   NOT             <- "!"     WHITESPACE ;
leaf:   QUESTION        <- "?"     WHITESPACE ;
leaf:   STAR            <- "*"     WHITESPACE ;
leaf:   PLUS            <- "+"     WHITESPACE ;
void:   OPEN            <- "("     WHITESPACE ;
void:   CLOSE           <- ")"     WHITESPACE ;
leaf:   DOT             <- "."     WHITESPACE ;

leaf:   ALNUM           <- "<alnum>"    WHITESPACE ;
leaf:   ALPHA           <- "<alpha>"    WHITESPACE ;
leaf:   ASCII           <- "<ascii>"    WHITESPACE ;
leaf:   CONTROL         <- "<control>"  WHITESPACE ;
leaf:   DDIGIT          <- "<ddigit>"   WHITESPACE ;
leaf:   DIGIT           <- "<digit>"    WHITESPACE ;
leaf:   GRAPH           <- "<graph>"    WHITESPACE ;
leaf:   LOWER           <- "<lower>"    WHITESPACE ;
leaf:   PRINTABLE       <- "<print>"    WHITESPACE ;
leaf:   PUNCT           <- "<punct>"    WHITESPACE ;
leaf:   SPACE           <- "<space>"    WHITESPACE ;
leaf:   UPPER           <- "<upper>"    WHITESPACE ;
leaf:   WORDCHAR        <- "<wordchar>" WHITESPACE ;
leaf:   XDIGIT          <- "<xdigit>"   WHITESPACE ;

void:   WHITESPACE      <- (" " / "\t" / EOL / COMMENT)* ;
void:   COMMENT         <- '#' (!EOL .)* EOL ;
void:   EOL             <- "\n\r" / "\n" / "\r" ;
void:   EOF             <- !. ;

        # ------------------------------------------------------------
    -------
END;
```

Example

Our example specifies the grammar for a basic 4-operation calculator.

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

Using higher-level features of the notation, i.e. the character classes (predefined and custom), this example can be rewritten as

```
PEG calculator (Expression)
    Sign       <- [-+]                                         ;
    Number     <- Sign? <ddigit>+                              ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- [*/]                       ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- [-+]                       ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

PEG serialization format

Here we specify the format used by the Parser Tools to serialize Parsing Expression Grammars as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a PEG may have more than one regular serialization only exactly one of them will be *canonical*.

regular serialization

1. The serialization of any PEG is a nested Tcl dictionary.
1. This dictionary holds a single key, **pt::grammar::peg**, and its value. This value holds the contents of the grammar.
2. The contents of the grammar are a Tcl dictionary holding the set of nonterminal symbols and the starting expression. The relevant keys and their values are

   rules

   The value is a Tcl dictionary whose keys are the names of the nonterminal symbols known to the grammar.

   1. Each nonterminal symbol may occur only once.
   2. The empty string is not a legal nonterminal symbol.
   3. The value for each symbol is a Tcl dictionary itself. The relevant keys and their values in this dictionary are

      is

      The value is the serialization of the parsing expression describing the symbols sentennial structure, as specified in the section PE serialization format.

      mode

The value can be one of three values specifying how a parser should handle the semantic value produced by the symbol.

value
: The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal itself, which has the ASTs of the symbol's right hand side as its children.

leaf
: The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal, without any children. Any ASTs generated by the symbol's right hand side are discarded.

void
: The nonterminal has no semantic value. Any ASTs generated by the symbol's right hand side are discarded (as well).

```
**start**
: The value is the serialization of the start parsing expression of
    the grammar, as specified in the section [PE serialization
    format].
```

1. The terminal symbols of the grammar are specified implicitly as the set of all terminal symbols used in the start expression and on the RHS of the grammar rules.

canonical serialization
: The canonical serialization of a grammar has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this grammar.

1. The keys found in all the nested Tcl dictionaries are sorted in ascending dictionary order, as generated by Tcl's builtin command **lsort -increasing -dict**.
2. The string representation of the value is the canonical representation of a Tcl dictionary. I.e. it does not contain superfluous whitespace.

Example

Assuming the following PEG for simple mathematical expressions

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

then its canonical serialization (except for whitespace) is

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}              mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                mode value}
        MulOp      {is {/ {t *} {t /}}
                                mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
                                mode value}
        Sign       {is {/ {t -} {t +}}
                                mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                mode value}
    }
    start {n Expression}
}
```

PE serialization format

Here we specify the format used by the Parser Tools to serialize Parsing Expressions as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a parsing expression may have more than one regular serialization only exactly one of them will be *canonical*.

Regular serialization

see below ...

Atomic Parsing Expressions

1. The string **epsilon** is an atomic parsing expression. It matches the empty string.
1. The string **dot** is an atomic parsing expression. It matches any character.
2. The string **alnum** is an atomic parsing expression. It matches any Unicode alphabet or digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
3. The string **alpha** is an atomic parsing expression. It matches any Unicode alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
4. The string **ascii** is an atomic parsing expression. It matches any Unicode character below U0080. This is a custom extension of PEs based on Tcl's builtin command **string is**.
5. The string **control** is an atomic parsing expression. It matches any Unicode control character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
6. The string **digit** is an atomic parsing expression. It matches any Unicode digit character. Note that this includes characters outside of the [0..9] range. This is a custom extension of PEs based on Tcl's builtin command **string is**.

7. The string **graph** is an atomic parsing expression. It matches any Unicode printing character, except for space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
8. The string **lower** is an atomic parsing expression. It matches any Unicode lower-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
9. The string **print** is an atomic parsing expression. It matches any Unicode printing character, including space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
10. The string **punct** is an atomic parsing expression. It matches any Unicode punctuation character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
11. The string **space** is an atomic parsing expression. It matches any Unicode space character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
12. The string **upper** is an atomic parsing expression. It matches any Unicode upper-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
13. The string **wordchar** is an atomic parsing expression. It matches any Unicode word character. This is any alphanumeric character (see alnum), and any connector punctuation characters (e.g. underscore). This is a custom extension of PEs based on Tcl's builtin command **string is**.
14. The string **xdigit** is an atomic parsing expression. It matches any hexadecimal digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
15. The string **ddigit** is an atomic parsing expression. It matches any decimal digit character. This is a custom extension of PEs based on Tcl's builtin command **regexp**.
16. The expression [list t **x**] is an atomic parsing expression. It matches the terminal string **x**.
17. The expression [list n **A**] is an atomic parsing expression. It matches the nonterminal **A**.

Combined Parsing Expressions

1. For parsing expressions **e1**, **e2**, ... the result of [list / **e1 e2** ...] is a parsing expression as well. This is the *ordered choice*, aka *prioritized choice*.
1. For parsing expressions **e1**, **e2**, ... the result of [list x **e1 e2** ...] is a parsing expression as well. This is the *sequence*.
2. For a parsing expression **e** the result of [list * **e**] is a parsing expression as well. This is the *kleene closure*, describing zero or more repetitions.
3. For a parsing expression **e** the result of [list + **e**] is a parsing expression as well. This is the *positive kleene closure*, describing one or more repetitions.
4. For a parsing expression **e** the result of [list & **e**] is a parsing expression as well. This is the *and lookahead predicate*.
5. For a parsing expression **e** the result of [list! **e**] is a parsing expression as well. This is the *not lookahead predicate*.
6. For a parsing expression **e** the result of [list ? **e**] is a parsing expression as well. This is the *optional input*.

Canonical serialization

The canonical serialization of a parsing expression has the format as specified in the previous

item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this parsing expression.

1. The string representation of the value is the canonical representation of a pure Tcl list. I.e. it does not contain superfluous whitespace.
2. Terminals are *not* encoded as ranges (where start and end of the range are identical).

Example

Assuming the parsing expression shown on the right-hand side of the rule

```
    Expression <- Term (AddOp Term)*
```

then its canonical serialization (except for whitespace) is

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pt* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Parsing and Grammars

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

pt_peg_from_container

Keywords: pt_peg_from_container, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt::peg::from::container - PEG Conversion. From CONTAINER format

Synopsis

package require Tcl 8.5 9

Description

Are you lost ? Do you have trouble understanding this document ? In that case please read the overview provided by the *Introduction to Parser Tools*. This document is the entrypoint to the whole system the current package is a part of.

This package does not exist. There is no need for it. The CONTAINER format for parsing

expression grammars is a piece of Tcl code which, then sourced, provides a class whose instances have the grammar we wish to import loaded. Another way of looking at this is, the CONTAINER output is its own import package.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pt* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Parsing and Grammars

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

pt_peg_from_json

Keywords: pt_peg_from_json, EBNF, JSON, LL(k), PEG, TDPL, context-free languages, conversion, expression, format conversion, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, serialization, state, top-down parsing languages, transducer

Name

pt::peg::from::json - PEG Conversion. Read JSON format

Synopsis

package require Tcl 8.5 9
package require pt::peg::from::json ?**1.1**?
package require pt::peg
package require json
pt::peg::from::json convert *text*

Description

Are you lost ? Do you have trouble understanding this document ? In that case please read the overview provided by the *Introduction to Parser Tools*. This document is the entrypoint to the whole system the current package is a part of.

This package implements the converter from JSON markup to parsing expression grammars.

It resides in the Import section of the Core Layer of Parser Tools, and can be used either directly with the other packages of this layer, or indirectly through the import manager provided by **pt::peg::import**. The latter is intented for use in untrusted environments and done through the corresponding import plugin **pt::peg::import::json** sitting between converter and import

manager.

IMAGE: arch_core_iplugins

API

The API provided by this package satisfies the specification of the Converter API found in the *Parser Tools Import API* specification.

pt::peg::from::json convert *text*
: This command takes the JSON markup encoding a parsing expression grammar and contained in *text*, and generates the canonical serialization of said grammar, as specified in section Peg serialization format. The created value is then returned as the result of the command.

JSON grammar exchange format

The **json** format for parsing expression grammars was written as a data exchange format not bound to Tcl. It was defined to allow the exchange of grammars with PackRat/PEG based parser generators for other languages.

It is formally specified by the rules below:

1. The JSON of any PEG is a JSON object.
2. This object holds a single key, **pt::grammar::peg**, and its value. This value holds the contents of the grammar.
3. The contents of the grammar are a JSON object holding the set of nonterminal symbols and the starting expression. The relevant keys and their values are

   rules
   : The value is a JSON object whose keys are the names of the nonterminal symbols known to the grammar.
     1. Each nonterminal symbol may occur only once.
     2. The empty string is not a legal nonterminal symbol.
     3. The value for each symbol is a JSON object itself. The relevant keys and their values in this dictionary are

        is
        : The value is a JSON string holding the Tcl serialization of the parsing expression describing the symbols sentennial structure, as specified in the section PE serialization format.

        mode
        : The value is a JSON holding holding one of three values specifying how a parser should handle the semantic value produced by the symbol.

          value
          : The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal itself, which has the ASTs of the symbol's right hand side as its children.

          leaf
          : The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal, without any children. Any ASTs generated by the symbol's right hand side are discarded.

          void
          : The nonterminal has no semantic value. Any ASTs generated by the symbol's right

hand side are discarded (as well).

```
**start**
: The value is a JSON string holding the Tcl serialization of the
    start parsing expression of the grammar, as specified in the
    section [PE serialization format].
```

1. The terminal symbols of the grammar are specified implicitly as the set of all terminal symbols used in the start expression and on the RHS of the grammar rules.

As an aside to the advanced reader, this is pretty much the same as the Tcl serialization of PE grammars, as specified in section Peg serialization format, except that the Tcl dictionaries and lists of that format are mapped to JSON objects and arrays. Only the parsing expressions themselves are not translated further, but kept as JSON strings containing a nested Tcl list, and there is no concept of canonicity for the JSON either.

Example

Assuming the following PEG for simple mathematical expressions

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

a JSON serialization for it is

```
{
    "pt::grammar::peg" : {
        "rules" : {
            "AddOp"     : {
                "is"   : "\/ {t -} {t +}",
                "mode" : "value"
            },
            "Digit"     : {
                "is"   : "\/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6}
    {t 7} {t 8} {t 9}",
                "mode" : "value"
            },
            "Expression" : {
                "is"   : "\/ {x {t (} {n Expression} {t )}} {x {n
    Factor} {* {x {n MulOp} {n Factor}}}}",
                "mode" : "value"
```

```
            },
            "Factor"    : {
                "is"   : "x {n Term} {* {x {n AddOp} {n Term}}}",
                "mode" : "value"
            },
            "MulOp"     : {
                "is"   : "\/ {t *} {t \/}",
                "mode" : "value"
            },
            "Number"    : {
                "is"   : "x {? {n Sign}} {+ {n Digit}}",
                "mode" : "value"
            },
            "Sign"      : {
                "is"   : "\/ {t -} {t +}",
                "mode" : "value"
            },
            "Term"      : {
                "is"   : "n Number",
                "mode" : "value"
            }
        },
        "start" : "n Expression"
    }
}
```

and a Tcl serialization of the same is

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}               mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                mode value}
        MulOp      {is {/ {t *} {t /}}
                                mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
                                mode value}
        Sign       {is {/ {t -} {t +}}
                                mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                mode value}
    }
    start {n Expression}
}
```

The similarity of the latter to the JSON should be quite obvious.

Peg serialization format

Here we specify the format used by the Parser Tools to serialize Parsing Expression Grammars as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a PEG may have more than one regular serialization only exactly one of them will be *canonical*.

regular serialization

1. The serialization of any PEG is a nested Tcl dictionary.
1. This dictionary holds a single key, **pt::grammar::peg**, and its value. This value holds the contents of the grammar.
2. The contents of the grammar are a Tcl dictionary holding the set of nonterminal symbols and the starting expression. The relevant keys and their values are

   rules

   The value is a Tcl dictionary whose keys are the names of the nonterminal symbols known to the grammar.

   1. Each nonterminal symbol may occur only once.
   2. The empty string is not a legal nonterminal symbol.
   3. The value for each symbol is a Tcl dictionary itself. The relevant keys and their values in this dictionary are

      is

      The value is the serialization of the parsing expression describing the symbols sentennial structure, as specified in the section PE serialization format.

      mode

The value can be one of three values specifying how a parser should handle the semantic value produced by the symbol.

value
The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal itself, which has the ASTs of the symbol's right hand side as its children.

leaf
The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal, without any children. Any ASTs generated by the symbol's right hand side are discarded.

void
The nonterminal has no semantic value. Any ASTs generated by the symbol's right hand side are discarded (as well).

```
**start**
: The value is the serialization of the start parsing expression of
    the grammar, as specified in the section [PE serialization
    format].
```

1. The terminal symbols of the grammar are specified implicitly as the set of all terminal symbols used in the start expression and on the RHS of the grammar rules.

canonical serialization
The canonical serialization of a grammar has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this grammar.

1. The keys found in all the nested Tcl dictionaries are sorted in ascending dictionary order, as generated by Tcl's builtin command **lsort -increasing -dict**.
2. The string representation of the value is the canonical representation of a Tcl dictionary. I.e. it does not contain superfluous whitespace.

Example

Assuming the following PEG for simple mathematical expressions

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

then its canonical serialization (except for whitespace) is

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}               mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                mode value}
        MulOp      {is {/ {t *} {t /}}
                                mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
                                mode value}
        Sign       {is {/ {t -} {t +}}
                                mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                mode value}
    }
    start {n Expression}
}
```

PE serialization format

Here we specify the format used by the Parser Tools to serialize Parsing Expressions as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a parsing expression may have more than one regular serialization only exactly one of them will be *canonical*.

Regular serialization

see below ...

Atomic Parsing Expressions

1. The string **epsilon** is an atomic parsing expression. It matches the empty string.
1. The string **dot** is an atomic parsing expression. It matches any character.
2. The string **alnum** is an atomic parsing expression. It matches any Unicode alphabet or digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
3. The string **alpha** is an atomic parsing expression. It matches any Unicode alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
4. The string **ascii** is an atomic parsing expression. It matches any Unicode character below U0080. This is a custom extension of PEs based on Tcl's builtin command **string is**.
5. The string **control** is an atomic parsing expression. It matches any Unicode control character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
6. The string **digit** is an atomic parsing expression. It matches any Unicode digit character. Note that this includes characters outside of the [0..9] range. This is a custom extension of PEs based on Tcl's builtin command **string is**.

7. The string **graph** is an atomic parsing expression. It matches any Unicode printing character, except for space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
8. The string **lower** is an atomic parsing expression. It matches any Unicode lower-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
9. The string **print** is an atomic parsing expression. It matches any Unicode printing character, including space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
10. The string **punct** is an atomic parsing expression. It matches any Unicode punctuation character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
11. The string **space** is an atomic parsing expression. It matches any Unicode space character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
12. The string **upper** is an atomic parsing expression. It matches any Unicode upper-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
13. The string **wordchar** is an atomic parsing expression. It matches any Unicode word character. This is any alphanumeric character (see alnum), and any connector punctuation characters (e.g. underscore). This is a custom extension of PEs based on Tcl's builtin command **string is**.
14. The string **xdigit** is an atomic parsing expression. It matches any hexadecimal digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
15. The string **ddigit** is an atomic parsing expression. It matches any decimal digit character. This is a custom extension of PEs based on Tcl's builtin command **regexp**.
16. The expression [list t **x**] is an atomic parsing expression. It matches the terminal string **x**.
17. The expression [list n **A**] is an atomic parsing expression. It matches the nonterminal **A**.

Combined Parsing Expressions

1. For parsing expressions **e1**, **e2**, ... the result of [list / **e1 e2** ...] is a parsing expression as well. This is the *ordered choice*, aka *prioritized choice*.
1. For parsing expressions **e1**, **e2**, ... the result of [list x **e1 e2** ...] is a parsing expression as well. This is the *sequence*.
2. For a parsing expression **e** the result of [list * **e**] is a parsing expression as well. This is the *kleene closure*, describing zero or more repetitions.
3. For a parsing expression **e** the result of [list + **e**] is a parsing expression as well. This is the *positive kleene closure*, describing one or more repetitions.
4. For a parsing expression **e** the result of [list & **e**] is a parsing expression as well. This is the *and lookahead predicate*.
5. For a parsing expression **e** the result of [list! **e**] is a parsing expression as well. This is the *not lookahead predicate*.
6. For a parsing expression **e** the result of [list ? **e**] is a parsing expression as well. This is the *optional input*.

Canonical serialization

The canonical serialization of a parsing expression has the format as specified in the previous

item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this parsing expression.

1. The string representation of the value is the canonical representation of a pure Tcl list. I.e. it does not contain superfluous whitespace.
2. Terminals are *not* encoded as ranges (where start and end of the range are identical).

Example

Assuming the parsing expression shown on the right-hand side of the rule

```
    Expression <- Term (AddOp Term)*
```

then its canonical serialization (except for whitespace) is

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pt* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Parsing and Grammars

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

pt_peg_from_peg

Keywords: pt_peg_from_peg, EBNF, LL(k), PEG, TDPL, context-free languages, conversion, expression, format conversion, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, serialization, state, top-down parsing languages, transducer

Name

pt::peg::from::peg - PEG Conversion. Read PEG format

Synopsis

package require Tcl 8.5 9
package require pt::peg::from::peg ?**1.0.4**?
pt::peg::from::peg convert *text*

Description

Are you lost ? Do you have trouble understanding this document ? In that case please read the

overview provided by the *Introduction to Parser Tools*. This document is the entrypoint to the whole system the current package is a part of.

This package implements the converter from PEG markup to parsing expression grammars.

It resides in the Import section of the Core Layer of Parser Tools, and can be used either directly with the other packages of this layer, or indirectly through the import manager provided by **pt::peg::import**. The latter is intented for use in untrusted environments and done through the corresponding import plugin **pt::peg::import::peg** sitting between converter and import manager.

IMAGE: arch_core_iplugins

API

The API provided by this package satisfies the specification of the Converter API found in the *Parser Tools Import API* specification.

pt::peg::from::peg convert *text*

This command takes the PEG markup encoding a parsing expression grammar and contained in *text*, and generates the canonical serialization of said grammar, as specified in section PEG serialization format. The created value is then returned as the result of the command.

PEG specification language

peg, a language for the specification of parsing expression grammars is meant to be human readable, and writable as well, yet strict enough to allow its processing by machine. Like any computer language. It was defined to make writing the specification of a grammar easy, something the other formats found in the Parser Tools do not lend themselves too.

It is formally specified by the grammar shown below, written in itself. For a tutorial / introduction to the language please go and read the *PEG Language Tutorial*.

```
PEG pe_grammar_for_peg (Grammar)

    # ------------------------------------------------------------------
        # Syntactical constructs

        Grammar         <- WHITESPACE Header Definition* Final EOF ;

        Header          <- PEG Identifier StartExpr ;
        Definition      <- Attribute? Identifier IS Expression
    SEMICOLON ;
        Attribute       <- (VOID / LEAF) COLON ;
        Expression      <- Sequence (SLASH Sequence)* ;
        Sequence        <- Prefix+ ;
        Prefix          <- (AND / NOT)? Suffix ;
        Suffix          <- Primary (QUESTION / STAR / PLUS)? ;
        Primary         <- ALNUM / ALPHA / ASCII / CONTROL / DDIGIT /
    DIGIT
                        /  GRAPH / LOWER / PRINTABLE / PUNCT / SPACE /
    UPPER
```

```
                        /  WORDCHAR / XDIGIT
                        / Identifier
                        /  OPEN Expression CLOSE
                        /  Literal
                        /  Class
                        /  DOT
                        ;
        Literal         <- APOSTROPH  (!APOSTROPH  Char)* APOSTROPH
    WHITESPACE
                        /  DAPOSTROPH (!DAPOSTROPH Char)* DAPOSTROPH
    WHITESPACE ;
        Class           <- OPENB (!CLOSEB Range)* CLOSEB WHITESPACE ;
        Range           <- Char TO Char / Char ;

        StartExpr       <- OPEN Expression CLOSE ;
void:   Final           <- "END" WHITESPACE SEMICOLON WHITESPACE ;

        # ------------------------------------------------------------
    -------
        # Lexing constructs

        Identifier      <- Ident WHITESPACE ;
leaf:   Ident           <- ([_:] / <alpha>) ([_:] / <alnum>)* ;
        Char            <- CharSpecial / CharOctalFull / CharOctalPart
                        /  CharUnicode / CharUnescaped
                        ;

leaf:   CharSpecial     <- "\\" [nrt'"\[\]\\] ;
leaf:   CharOctalFull   <- "\\" [0-3][0-7][0-7] ;
leaf:   CharOctalPart   <- "\\" [0-7][0-7]? ;
leaf:   CharUnicode     <- "\\" 'u' HexDigit (HexDigit (HexDigit
    HexDigit?)?)? ;
leaf:   CharUnescaped   <- !"\\" . ;

void:   HexDigit        <- [0-9a-fA-F] ;

void:   TO              <- '-'           ;
void:   OPENB           <- "["           ;
void:   CLOSEB          <- "]"           ;
void:   APOSTROPH       <- "'"           ;
void:   DAPOSTROPH      <- '"'           ;
void:   PEG             <- "PEG" !([_:] / <alnum>) WHITESPACE ;
void:   IS              <- "<-"    WHITESPACE ;
leaf:   VOID            <- "void"  WHITESPACE ; # Implies that
    definition has no semantic value.
leaf:   LEAF            <- "leaf"  WHITESPACE ; # Implies that
```

```
    definition has no terminals.
void:   SEMICOLON       <- ";"     WHITESPACE ;
void:   COLON           <- ":"     WHITESPACE ;
void:   SLASH           <- "/"     WHITESPACE ;
leaf:   AND             <- "&"     WHITESPACE ;
leaf:   NOT             <- "!"     WHITESPACE ;
leaf:   QUESTION        <- "?"     WHITESPACE ;
leaf:   STAR            <- "*"     WHITESPACE ;
leaf:   PLUS            <- "+"     WHITESPACE ;
void:   OPEN            <- "("     WHITESPACE ;
void:   CLOSE           <- ")"     WHITESPACE ;
leaf:   DOT             <- "."     WHITESPACE ;

leaf:   ALNUM           <- "<alnum>"    WHITESPACE ;
leaf:   ALPHA           <- "<alpha>"    WHITESPACE ;
leaf:   ASCII           <- "<ascii>"    WHITESPACE ;
leaf:   CONTROL         <- "<control>"  WHITESPACE ;
leaf:   DDIGIT          <- "<ddigit>"   WHITESPACE ;
leaf:   DIGIT           <- "<digit>"    WHITESPACE ;
leaf:   GRAPH           <- "<graph>"    WHITESPACE ;
leaf:   LOWER           <- "<lower>"    WHITESPACE ;
leaf:   PRINTABLE       <- "<print>"    WHITESPACE ;
leaf:   PUNCT           <- "<punct>"    WHITESPACE ;
leaf:   SPACE           <- "<space>"    WHITESPACE ;
leaf:   UPPER           <- "<upper>"    WHITESPACE ;
leaf:   WORDCHAR        <- "<wordchar>" WHITESPACE ;
leaf:   XDIGIT          <- "<xdigit>"   WHITESPACE ;

void:   WHITESPACE      <- (" " / "\t" / EOL / COMMENT)* ;
void:   COMMENT         <- '#' (!EOL .)* EOL ;
void:   EOL             <- "\n\r" / "\n" / "\r" ;
void:   EOF             <- !. ;

        # ------------------------------------------------------------
    -------
END;
```

Example

Our example specifies the grammar for a basic 4-operation calculator.

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
```

```
    Term       <- Factor (MulOp Factor)*                        ;
    AddOp      <- '+'/'-'                                       ;
    Factor     <- '(' Expression ')' / Number                   ;
END;
```

Using higher-level features of the notation, i.e. the character classes (predefined and custom), this example can be rewritten as

```
PEG calculator (Expression)
    Sign       <- [-+]                                          ;
    Number     <- Sign? <ddigit>+                               ;
    Expression <- Term (AddOp Term)*                            ;
    MulOp      <- [*/]                        ;
    Term       <- Factor (MulOp Factor)*                        ;
    AddOp      <- [-+]                        ;
    Factor     <- '(' Expression ')' / Number                   ;
END;
```

PEG serialization format

Here we specify the format used by the Parser Tools to serialize Parsing Expression Grammars as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a PEG may have more than one regular serialization only exactly one of them will be *canonical*.

regular serialization

1. The serialization of any PEG is a nested Tcl dictionary.
1. This dictionary holds a single key, **pt::grammar::peg**, and its value. This value holds the contents of the grammar.
2. The contents of the grammar are a Tcl dictionary holding the set of nonterminal symbols and the starting expression. The relevant keys and their values are

   rules

   The value is a Tcl dictionary whose keys are the names of the nonterminal symbols known to the grammar.

   1. Each nonterminal symbol may occur only once.
   2. The empty string is not a legal nonterminal symbol.
   3. The value for each symbol is a Tcl dictionary itself. The relevant keys and their values in this dictionary are

      is

      The value is the serialization of the parsing expression describing the symbols sentennial structure, as specified in the section PE serialization format.

      mode

      The value can be one of three values specifying how a parser should handle the semantic value produced by the symbol.

      value

      The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal itself, which has the ASTs of the symbol's right hand side as its children.

leaf
The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal, without any children. Any ASTs generated by the symbol's right hand side are discarded.

void
The nonterminal has no semantic value. Any ASTs generated by the symbol's right hand side are discarded (as well).

```
**start**
: The value is the serialization of the start parsing expression of
    the grammar, as specified in the section [PE serialization
    format].
```

1. The terminal symbols of the grammar are specified implicitly as the set of all terminal symbols used in the start expression and on the RHS of the grammar rules.

canonical serialization
The canonical serialization of a grammar has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this grammar.

1. The keys found in all the nested Tcl dictionaries are sorted in ascending dictionary order, as generated by Tcl's builtin command **lsort -increasing -dict**.
2. The string representation of the value is the canonical representation of a Tcl dictionary. I.e. it does not contain superfluous whitespace.

Example

Assuming the following PEG for simple mathematical expressions

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

then its canonical serialization (except for whitespace) is

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
```

```
    7} {t 8} {t 9}}                mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                   mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                   mode value}
        MulOp      {is {/ {t *} {t /}}
                                   mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
                                   mode value}
        Sign       {is {/ {t -} {t +}}
                                   mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                   mode value}
    }
    start {n Expression}
}
```

PE serialization format

Here we specify the format used by the Parser Tools to serialize Parsing Expressions as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a parsing expression may have more than one regular serialization only exactly one of them will be *canonical*.

Regular serialization

see below ...

Atomic Parsing Expressions

1. The string **epsilon** is an atomic parsing expression. It matches the empty string.
1. The string **dot** is an atomic parsing expression. It matches any character.
2. The string **alnum** is an atomic parsing expression. It matches any Unicode alphabet or digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
3. The string **alpha** is an atomic parsing expression. It matches any Unicode alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
4. The string **ascii** is an atomic parsing expression. It matches any Unicode character below U0080. This is a custom extension of PEs based on Tcl's builtin command **string is**.
5. The string **control** is an atomic parsing expression. It matches any Unicode control character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
6. The string **digit** is an atomic parsing expression. It matches any Unicode digit character. Note that this includes characters outside of the [0..9] range. This is a custom extension of PEs based on Tcl's builtin command **string is**.
7. The string **graph** is an atomic parsing expression. It matches any Unicode printing character, except for space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
8. The string **lower** is an atomic parsing expression. It matches any Unicode lower-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command

string is.

9. The string **print** is an atomic parsing expression. It matches any Unicode printing character, including space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
10. The string **punct** is an atomic parsing expression. It matches any Unicode punctuation character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
11. The string **space** is an atomic parsing expression. It matches any Unicode space character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
12. The string **upper** is an atomic parsing expression. It matches any Unicode upper-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
13. The string **wordchar** is an atomic parsing expression. It matches any Unicode word character. This is any alphanumeric character (see alnum), and any connector punctuation characters (e.g. underscore). This is a custom extension of PEs based on Tcl's builtin command **string is**.
14. The string **xdigit** is an atomic parsing expression. It matches any hexadecimal digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
15. The string **ddigit** is an atomic parsing expression. It matches any decimal digit character. This is a custom extension of PEs based on Tcl's builtin command **regexp**.
16. The expression [list t **x**] is an atomic parsing expression. It matches the terminal string **x**.
17. The expression [list n **A**] is an atomic parsing expression. It matches the nonterminal **A**.

Combined Parsing Expressions

1. For parsing expressions **e1**, **e2**, ... the result of [list / **e1 e2** ...] is a parsing expression as well. This is the *ordered choice*, aka *prioritized choice*.
1. For parsing expressions **e1**, **e2**, ... the result of [list x **e1 e2** ...] is a parsing expression as well. This is the *sequence*.
2. For a parsing expression **e** the result of [list * **e**] is a parsing expression as well. This is the *kleene closure*, describing zero or more repetitions.
3. For a parsing expression **e** the result of [list + **e**] is a parsing expression as well. This is the *positive kleene closure*, describing one or more repetitions.
4. For a parsing expression **e** the result of [list & **e**] is a parsing expression as well. This is the *and lookahead predicate*.
5. For a parsing expression **e** the result of [list! **e**] is a parsing expression as well. This is the *not lookahead predicate*.
6. For a parsing expression **e** the result of [list ? **e**] is a parsing expression as well. This is the *optional input*.

Canonical serialization

The canonical serialization of a parsing expression has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this parsing expression.

1. The string representation of the value is the canonical representation of a pure Tcl list. I.e. it does not contain superfluous whitespace.
2. Terminals are *not* encoded as ranges (where start and end of the range are identical).

Example

Assuming the parsing expression shown on the right-hand side of the rule

```
    Expression <- Term (AddOp Term)*
```

then its canonical serialization (except for whitespace) is

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pt* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Parsing and Grammars

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

pt_peg_import

Keywords: pt_peg_import, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt::peg::import - PEG Import

Synopsis

package require Tcl 8.5 9
package require Tcl 8.5 9
package require snit
package require fileutil::paths
package require pt::peg
package require pluginmgr
package require pt::peg::import ?**1.0.2**?
::pt::peg::import *objectName*
objectName method ?*arg arg ...*?
objectName **destroy**
objectName **import text** *text* ?*format*?
objectName **import file** *path* ?*format*?
objectName **import object text** *object text* ?*format*?

objectName **import object file** *object path* ?*format*?
objectName **includes**
objectName **include add** *path*
objectName **include remove** *path*
objectName **include clear**

Description

Are you lost ? Do you have trouble understanding this document ? In that case please read the overview provided by the *Introduction to Parser Tools*. This document is the entrypoint to the whole system the current package is a part of.

This package provides a manager for parsing expression grammars, with each instance handling a set of plugins for the import of them from other formats, i.e. their conversion from, for example *peg*, *container*, *json*, etc.

It resides in the Import section of the Core Layer of Parser Tools, and is one of the three pillars the management of parsing expression grammars resides on.

IMAGE: arch_core_import

The other two pillars are, as shown above

1. *PEG Export*, and
2. *PEG Storage*

For information about the data structure which is the major output of the manager objects provided by this package see the section PEG serialization format.

The plugin system of our class is based on the package **pluginmgr**, and configured to look for plugins using

1. the environment variable **GRAMMAR_PEG_IMPORT_PLUGINS**,
2. the environment variable **GRAMMAR_PEG_PLUGINS**,
3. the environment variable **GRAMMAR_PLUGINS**,
4. the path "*~/.grammar/peg/import/plugin*"
5. the path "*~/.grammar/peg/plugin*"
6. the path "*~/.grammar/plugin*"
7. the path "*~/.grammar/peg/import/plugins*"
8. the path "*~/.grammar/peg/plugins*"
9. the path "*~/.grammar/plugins*"
10. the registry entry "HKEY_CURRENT_USER\SOFTWARE\GRAMMAR\PEG\IMPORT\PLUGINS"
11. the registry entry "HKEY_CURRENT_USER\SOFTWARE\GRAMMAR\PEG\PLUGINS"
12. the registry entry "HKEY_CURRENT_USER\SOFTWARE\GRAMMAR\PLUGINS"

The last three are used only when the package is run on a machine using the Windows(tm) operating system.

The whole system is delivered with three predefined import plugins, namely

container
: See *PEG Import Plugin. From CONTAINER format* for details.

json
: See *PEG Import Plugin. From JSON format* for details.

peg

See *PEG Import Plugin. From PEG format* for details.

For readers wishing to write their own import plugin for some format, i.e. *plugin writer*s, reading and understanding the *Parser Tools Impport API* specification is an absolute necessity, as it documents the interaction between this package and its plugins in detail.

API

Package commands

::pt::peg::import *objectName*

This command creates a new import manager object with an associated Tcl command whose name is *objectName*. This *object* command is explained in full detail in the sections Object command and Object methods. The object command will be created under the current namespace if the *objectName* is not fully qualified, and in the specified namespace otherwise.

Object command

All objects created by the **::pt::peg::import** command have the following general form:

objectName method ?*arg arg ...*?

The method **method** and its *arg*'uments determine the exact behavior of the command. See section Object methods for the detailed specifications.

Object methods

***objectName* destroy**

This method destroys the object it is invoked for.

***objectName* import text *text* ?*format*?**

This method takes the *text* and converts it from the specified *format* to the canonical serialization of a parsing expression grammar using the import plugin for the format. An error is thrown if no plugin could be found for the format. The serialization generated by the conversion process is returned as the result of this method.

If no format is specified the method defaults to **text**.

The specification of what a *canonical* serialization is can be found in the section PEG serialization format.

The plugin has to conform to the interface documented in the *Parser Tools Import API* specification.

***objectName* import file *path* ?*format*?**

This method is a convenient wrapper around the import text method described by the previous item. It reads the contents of the specified file into memory, feeds the result into import text and returns the resulting serialization as its own result.

***objectName* import object text *object text* ?*format*?**

This method is a convenient wrapper around the import text method described by the previous item. It expects that *object* is an object command supporting a **deserialize** method expecting the canonical serialization of a parsing expression grammar. It imports the text using import text and then feeds the resulting serialization into the *object* via **deserialize**. This method returns the empty string as it result.

***objectName* import object file *object path* ?*format*?**
This method behaves like **import object text**, except that it reads the text to convert from the specified file instead of being given it as argument.

***objectName* includes**
This method returns a list containing the currently specified paths to use to search for include files when processing input. The order of paths in the list corresponds to the order in which they are used, from first to last, and also corresponds to the order in which they were added to the object.

objectName* include add *path
This methods adds the specified *path* to the list of paths to use to search for include files when processing input. The path is added to the end of the list, causing it to be searched after all previously added paths. The result of the command is the empty string.

The method does nothing if the path is already known.

objectName* include remove *path
This methods removes the specified *path* from the list of paths to use to search for include files when processing input. The result of the command is the empty string.

The method does nothing if the path is not known.

***objectName* include clear**
This method clears the list of paths to use to search for include files when processing input. The result of the command is the empty string.

PEG serialization format

Here we specify the format used by the Parser Tools to serialize Parsing Expression Grammars as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a PEG may have more than one regular serialization only exactly one of them will be *canonical*.

regular serialization

1. The serialization of any PEG is a nested Tcl dictionary.
1. This dictionary holds a single key, **pt::grammar::peg**, and its value. This value holds the contents of the grammar.
2. The contents of the grammar are a Tcl dictionary holding the set of nonterminal symbols and the starting expression. The relevant keys and their values are

   rules
   The value is a Tcl dictionary whose keys are the names of the nonterminal symbols known to the grammar.
   1. Each nonterminal symbol may occur only once.
   2. The empty string is not a legal nonterminal symbol.
   3. The value for each symbol is a Tcl dictionary itself. The relevant keys and their values in this dictionary are

      is
      The value is the serialization of the parsing expression describing the symbols sentennial structure, as specified in the section PE serialization format.

      mode

The value can be one of three values specifying how a parser should handle the semantic value produced by the symbol.

value
The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal itself, which has the ASTs of the symbol's right hand side as its children.

leaf
The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal, without any children. Any ASTs generated by the symbol's right hand side are discarded.

void
The nonterminal has no semantic value. Any ASTs generated by the symbol's right hand side are discarded (as well).

```
**start**
: The value is the serialization of the start parsing expression of
    the grammar, as specified in the section [PE serialization
    format].
```

1. The terminal symbols of the grammar are specified implicitly as the set of all terminal symbols used in the start expression and on the RHS of the grammar rules.

canonical serialization
The canonical serialization of a grammar has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this grammar.

1. The keys found in all the nested Tcl dictionaries are sorted in ascending dictionary order, as generated by Tcl's builtin command **lsort -increasing -dict**.
2. The string representation of the value is the canonical representation of a Tcl dictionary. I.e. it does not contain superfluous whitespace.

Example

Assuming the following PEG for simple mathematical expressions

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

then its canonical serialization (except for whitespace) is

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}              mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                mode value}
        MulOp      {is {/ {t *} {t /}}
                                mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
                                mode value}
        Sign       {is {/ {t -} {t +}}
                                mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                mode value}
    }
    start {n Expression}
}
```

PE serialization format

Here we specify the format used by the Parser Tools to serialize Parsing Expressions as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a parsing expression may have more than one regular serialization only exactly one of them will be *canonical*.

Regular serialization

see below ...

Atomic Parsing Expressions

1. The string **epsilon** is an atomic parsing expression. It matches the empty string.
1. The string **dot** is an atomic parsing expression. It matches any character.
2. The string **alnum** is an atomic parsing expression. It matches any Unicode alphabet or digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
3. The string **alpha** is an atomic parsing expression. It matches any Unicode alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
4. The string **ascii** is an atomic parsing expression. It matches any Unicode character below U0080. This is a custom extension of PEs based on Tcl's builtin command **string is**.
5. The string **control** is an atomic parsing expression. It matches any Unicode control character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
6. The string **digit** is an atomic parsing expression. It matches any Unicode digit character. Note that this includes characters outside of the [0..9] range. This is a custom extension of PEs based on Tcl's builtin command **string is**.

7. The string **graph** is an atomic parsing expression. It matches any Unicode printing character, except for space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
8. The string **lower** is an atomic parsing expression. It matches any Unicode lower-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
9. The string **print** is an atomic parsing expression. It matches any Unicode printing character, including space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
10. The string **punct** is an atomic parsing expression. It matches any Unicode punctuation character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
11. The string **space** is an atomic parsing expression. It matches any Unicode space character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
12. The string **upper** is an atomic parsing expression. It matches any Unicode upper-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
13. The string **wordchar** is an atomic parsing expression. It matches any Unicode word character. This is any alphanumeric character (see alnum), and any connector punctuation characters (e.g. underscore). This is a custom extension of PEs based on Tcl's builtin command **string is**.
14. The string **xdigit** is an atomic parsing expression. It matches any hexadecimal digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
15. The string **ddigit** is an atomic parsing expression. It matches any decimal digit character. This is a custom extension of PEs based on Tcl's builtin command **regexp**.
16. The expression [list t **x**] is an atomic parsing expression. It matches the terminal string **x**.
17. The expression [list n **A**] is an atomic parsing expression. It matches the nonterminal **A**.

Combined Parsing Expressions

1. For parsing expressions **e1**, **e2**, ... the result of [list / **e1 e2** ...] is a parsing expression as well. This is the *ordered choice*, aka *prioritized choice*.
1. For parsing expressions **e1**, **e2**, ... the result of [list x **e1 e2** ...] is a parsing expression as well. This is the *sequence*.
2. For a parsing expression **e** the result of [list * **e**] is a parsing expression as well. This is the *kleene closure*, describing zero or more repetitions.
3. For a parsing expression **e** the result of [list + **e**] is a parsing expression as well. This is the *positive kleene closure*, describing one or more repetitions.
4. For a parsing expression **e** the result of [list & **e**] is a parsing expression as well. This is the *and lookahead predicate*.
5. For a parsing expression **e** the result of [list! **e**] is a parsing expression as well. This is the *not lookahead predicate*.
6. For a parsing expression **e** the result of [list ? **e**] is a parsing expression as well. This is the *optional input*.

Canonical serialization

The canonical serialization of a parsing expression has the format as specified in the previous

item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this parsing expression.

1. The string representation of the value is the canonical representation of a pure Tcl list. I.e. it does not contain superfluous whitespace.
2. Terminals are *not* encoded as ranges (where start and end of the range are identical).

Example

Assuming the parsing expression shown on the right-hand side of the rule

```
    Expression <- Term (AddOp Term)*
```

then its canonical serialization (except for whitespace) is

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pt* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Parsing and Grammars

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_peg_import_container

Keywords: pt_peg_import_container, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt::peg::import::container - PEG Import Plugin. From CONTAINER format

Synopsis

package require Tcl 8.5 9

Description

Are you lost ? Do you have trouble understanding this document ? In that case please read the overview provided by the *Introduction to Parser Tools*. This document is the entrypoint to the whole system the current package is a part of.

This package does not exist. There is no need for it. The CONTAINER format for parsing

expression grammars is a piece of Tcl code which, then sourced, provides a class whose instances have the grammar we wish to import loaded. Another way of looking at this is, the CONTAINER output is its own import package.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pt* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Parsing and Grammars

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_peg_import_json

Keywords: pt_peg_import_json, EBNF, JSON, LL(k), PEG, TDPL, context-free languages, expression , grammar, import, matching, parser, parsing expression, parsing expression grammar, plugin, push down automaton, recursive descent, serialization, state, top-down parsing languages, transducer

Name

pt::peg::import::json - PEG Import Plugin. Read JSON format

Synopsis

package require Tcl 8.5 9
package require pt::peg::import::json ?**1.1**?
package require pt::peg::to::json
import *text*

Description

Are you lost ? Do you have trouble understanding this document ? In that case please read the overview provided by the *Introduction to Parser Tools*. This document is the entrypoint to the whole system the current package is a part of.

This package implements the parsing expression grammar import plugin processing JSON markup.

It resides in the Import section of the Core Layer of Parser Tools and is intended to be used by **pt::peg::import**, the import manager, sitting between it and the corresponding core conversion functionality provided by pt::peg::from::json.

IMAGE: arch_core_iplugins

While the direct use of this package with a regular interpreter is possible, this is strongly disrecommended and requires a number of contortions to provide the expected environment. The proper way to use this functionality depends on the situation:

1. In an untrusted environment the proper access is through the package **pt::peg::import** and the import manager objects it provides.
2. In a trusted environment however simply use the package pt::peg::from::json and access the core conversion functionality directly.

API

The API provided by this package satisfies the specification of the Plugin API found in the *Parser Tools Import API* specification.

import *text*
: This command takes the JSON markup encoding a parsing expression grammar and contained in *text*, and generates the canonical serialization of said grammar, as specified in section Peg serialization format. The created value is then returned as the result of the command.

JSON grammar exchange format

The **json** format for parsing expression grammars was written as a data exchange format not bound to Tcl. It was defined to allow the exchange of grammars with PackRat/PEG based parser generators for other languages.

It is formally specified by the rules below:

1. The JSON of any PEG is a JSON object.
2. This object holds a single key, **pt::grammar::peg**, and its value. This value holds the contents of the grammar.
3. The contents of the grammar are a JSON object holding the set of nonterminal symbols and the starting expression. The relevant keys and their values are

   rules
   : The value is a JSON object whose keys are the names of the nonterminal symbols known to the grammar.
     1. Each nonterminal symbol may occur only once.
     2. The empty string is not a legal nonterminal symbol.
     3. The value for each symbol is a JSON object itself. The relevant keys and their values in this dictionary are

        is
        : The value is a JSON string holding the Tcl serialization of the parsing expression describing the symbols sentennial structure, as specified in the section PE serialization format.

        mode
        : The value is a JSON holding holding one of three values specifying how a parser should handle the semantic value produced by the symbol.

          value
          : The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal itself, which has the ASTs of the symbol's right hand side as its children.

          leaf

The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal, without any children. Any ASTs generated by the symbol's right hand side are discarded.

void
The nonterminal has no semantic value. Any ASTs generated by the symbol's right hand side are discarded (as well).

```
**start**
: The value is a JSON string holding the Tcl serialization of the
    start parsing expression of the grammar, as specified in the
    section [PE serialization format].
```

1. The terminal symbols of the grammar are specified implicitly as the set of all terminal symbols used in the start expression and on the RHS of the grammar rules.

As an aside to the advanced reader, this is pretty much the same as the Tcl serialization of PE grammars, as specified in section Peg serialization format, except that the Tcl dictionaries and lists of that format are mapped to JSON objects and arrays. Only the parsing expressions themselves are not translated further, but kept as JSON strings containing a nested Tcl list, and there is no concept of canonicity for the JSON either.

Example

Assuming the following PEG for simple mathematical expressions

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

a JSON serialization for it is

```
{
    "pt::grammar::peg" : {
        "rules" : {
            "AddOp"     : {
                "is"   : "\/ {t -} {t +}",
                "mode" : "value"
            },
            "Digit"     : {
                "is"   : "\/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6}
    {t 7} {t 8} {t 9}",
                "mode" : "value"
```

```
            },
            "Expression" : {
                "is"   : "\/ {x {t (} {n Expression} {t )}} {x {n
    Factor} {* {x {n MulOp} {n Factor}}}}",
                "mode" : "value"
            },
            "Factor"    : {
                "is"   : "x {n Term} {* {x {n AddOp} {n Term}}}",
                "mode" : "value"
            },
            "MulOp"     : {
                "is"   : "\/ {t *} {t \/}",
                "mode" : "value"
            },
            "Number"    : {
                "is"   : "x {? {n Sign}} {+ {n Digit}}",
                "mode" : "value"
            },
            "Sign"      : {
                "is"   : "\/ {t -} {t +}",
                "mode" : "value"
            },
            "Term"      : {
                "is"   : "n Number",
                "mode" : "value"
            }
        },
        "start" : "n Expression"
    }
}
```

and a Tcl serialization of the same is

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                  mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}               mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                  mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                  mode value}
        MulOp      {is {/ {t *} {t /}}
                                  mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
                                  mode value}
```

```
        Sign       {is {/ {t -} {t +}}
                                mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                mode value}
    }
    start {n Expression}
}
```

The similarity of the latter to the JSON should be quite obvious.

Peg serialization format

Here we specify the format used by the Parser Tools to serialize Parsing Expression Grammars as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a PEG may have more than one regular serialization only exactly one of them will be *canonical*.

regular serialization

1. The serialization of any PEG is a nested Tcl dictionary.
1. This dictionary holds a single key, **pt::grammar::peg**, and its value. This value holds the contents of the grammar.
2. The contents of the grammar are a Tcl dictionary holding the set of nonterminal symbols and the starting expression. The relevant keys and their values are

   rules

   The value is a Tcl dictionary whose keys are the names of the nonterminal symbols known to the grammar.

   1. Each nonterminal symbol may occur only once.
   2. The empty string is not a legal nonterminal symbol.
   3. The value for each symbol is a Tcl dictionary itself. The relevant keys and their values in this dictionary are

      is

      The value is the serialization of the parsing expression describing the symbols sentennial structure, as specified in the section PE serialization format.

      mode

      The value can be one of three values specifying how a parser should handle the semantic value produced by the symbol.

      value

      The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal itself, which has the ASTs of the symbol's right hand side as its children.

      leaf

      The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal, without any children. Any ASTs generated by the symbol's right hand side are discarded.

      void

      The nonterminal has no semantic value. Any ASTs generated by the symbol's right hand side are discarded (as well).

```
**start**
: The value is the serialization of the start parsing expression of
    the grammar, as specified in the section [PE serialization
    format].
```

1. The terminal symbols of the grammar are specified implicitly as the set of all terminal symbols used in the start expression and on the RHS of the grammar rules.

canonical serialization

The canonical serialization of a grammar has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this grammar.

1. The keys found in all the nested Tcl dictionaries are sorted in ascending dictionary order, as generated by Tcl's builtin command **lsort -increasing -dict**.
2. The string representation of the value is the canonical representation of a Tcl dictionary. I.e. it does not contain superfluous whitespace.

Example

Assuming the following PEG for simple mathematical expressions

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

then its canonical serialization (except for whitespace) is

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                  mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}                mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                  mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                  mode value}
        MulOp      {is {/ {t *} {t /}}
                                  mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
                                  mode value}
        Sign       {is {/ {t -} {t +}}
```

```
                                    mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                    mode value}
    }
    start {n Expression}
}
```

PE serialization format

Here we specify the format used by the Parser Tools to serialize Parsing Expressions as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a parsing expression may have more than one regular serialization only exactly one of them will be *canonical*.

Regular serialization

see below ...

Atomic Parsing Expressions

1. The string **epsilon** is an atomic parsing expression. It matches the empty string.
1. The string **dot** is an atomic parsing expression. It matches any character.
2. The string **alnum** is an atomic parsing expression. It matches any Unicode alphabet or digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
3. The string **alpha** is an atomic parsing expression. It matches any Unicode alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
4. The string **ascii** is an atomic parsing expression. It matches any Unicode character below U0080. This is a custom extension of PEs based on Tcl's builtin command **string is**.
5. The string **control** is an atomic parsing expression. It matches any Unicode control character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
6. The string **digit** is an atomic parsing expression. It matches any Unicode digit character. Note that this includes characters outside of the [0..9] range. This is a custom extension of PEs based on Tcl's builtin command **string is**.
7. The string **graph** is an atomic parsing expression. It matches any Unicode printing character, except for space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
8. The string **lower** is an atomic parsing expression. It matches any Unicode lower-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
9. The string **print** is an atomic parsing expression. It matches any Unicode printing character, including space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
10. The string **punct** is an atomic parsing expression. It matches any Unicode punctuation character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
11. The string **space** is an atomic parsing expression. It matches any Unicode space character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
12. The string **upper** is an atomic parsing expression. It matches any Unicode upper-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command

string is.

13. The string **wordchar** is an atomic parsing expression. It matches any Unicode word character. This is any alphanumeric character (see alnum), and any connector punctuation characters (e.g. underscore). This is a custom extension of PEs based on Tcl's builtin command **string is**.
14. The string **xdigit** is an atomic parsing expression. It matches any hexadecimal digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
15. The string **ddigit** is an atomic parsing expression. It matches any decimal digit character. This is a custom extension of PEs based on Tcl's builtin command **regexp**.
16. The expression [list t **x**] is an atomic parsing expression. It matches the terminal string **x**.
17. The expression [list n **A**] is an atomic parsing expression. It matches the nonterminal **A**.

Combined Parsing Expressions

1. For parsing expressions **e1**, **e2**, ... the result of [list / **e1 e2** ...] is a parsing expression as well. This is the *ordered choice*, aka *prioritized choice*.
1. For parsing expressions **e1**, **e2**, ... the result of [list x **e1 e2** ...] is a parsing expression as well. This is the *sequence*.
2. For a parsing expression **e** the result of [list * **e**] is a parsing expression as well. This is the *kleene closure*, describing zero or more repetitions.
3. For a parsing expression **e** the result of [list + **e**] is a parsing expression as well. This is the *positive kleene closure*, describing one or more repetitions.
4. For a parsing expression **e** the result of [list & **e**] is a parsing expression as well. This is the *and lookahead predicate*.
5. For a parsing expression **e** the result of [list! **e**] is a parsing expression as well. This is the *not lookahead predicate*.
6. For a parsing expression **e** the result of [list ? **e**] is a parsing expression as well. This is the *optional input*.

Canonical serialization

The canonical serialization of a parsing expression has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this parsing expression.

1. The string representation of the value is the canonical representation of a pure Tcl list. I.e. it does not contain superfluous whitespace.
2. Terminals are *not* encoded as ranges (where start and end of the range are identical).

Example

Assuming the parsing expression shown on the right-hand side of the rule

```
    Expression <- Term (AddOp Term)*
```

then its canonical serialization (except for whitespace) is

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you

want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pt* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Parsing and Grammars

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_peg_import_peg

Keywords: pt_peg_import_peg, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, import, matching, parser, parsing expression, parsing expression grammar, plugin, push down automaton, recursive descent, serialization, state, top-down parsing languages, transducer

Name

pt::peg::import::peg - PEG Import Plugin. Read PEG format

Synopsis

package require Tcl 8.5 9
package require pt::peg::import::peg ?**1.1**?
package require pt::peg::to::peg
import *text*

Description

Are you lost ? Do you have trouble understanding this document ? In that case please read the overview provided by the *Introduction to Parser Tools*. This document is the entrypoint to the whole system the current package is a part of.

This package implements the parsing expression grammar import plugin processing PEG markup.

It resides in the Import section of the Core Layer of Parser Tools and is intended to be used by **pt::peg::import**, the import manager, sitting between it and the corresponding core conversion functionality provided by pt::peg::from::peg.

IMAGE: arch_core_iplugins

While the direct use of this package with a regular interpreter is possible, this is strongly disrecommended and requires a number of contortions to provide the expected environment. The proper way to use this functionality depends on the situation:

1. In an untrusted environment the proper access is through the package **pt::peg::import** and the import manager objects it provides.

2. In a trusted environment however simply use the package pt::peg::from::peg and access the core conversion functionality directly.

API

The API provided by this package satisfies the specification of the Plugin API found in the *Parser Tools Import API* specification.

import *text*

This command takes the PEG markup encoding a parsing expression grammar and contained in *text*, and generates the canonical serialization of said grammar, as specified in section PEG serialization format. The created value is then returned as the result of the command.

PEG specification language

peg, a language for the specification of parsing expression grammars is meant to be human readable, and writable as well, yet strict enough to allow its processing by machine. Like any computer language. It was defined to make writing the specification of a grammar easy, something the other formats found in the Parser Tools do not lend themselves too.

It is formally specified by the grammar shown below, written in itself. For a tutorial / introduction to the language please go and read the *PEG Language Tutorial*.

```
PEG pe_grammar_for_peg (Grammar)

    # ------------------------------------------------------------------------
        # Syntactical constructs

        Grammar         <- WHITESPACE Header Definition* Final EOF ;

        Header          <- PEG Identifier StartExpr ;
        Definition      <- Attribute? Identifier IS Expression
    SEMICOLON ;
        Attribute       <- (VOID / LEAF) COLON ;
        Expression      <- Sequence (SLASH Sequence)* ;
        Sequence        <- Prefix+ ;
        Prefix          <- (AND / NOT)? Suffix ;
        Suffix          <- Primary (QUESTION / STAR / PLUS)? ;
        Primary         <- ALNUM / ALPHA / ASCII / CONTROL / DDIGIT /
    DIGIT
                        /  GRAPH / LOWER / PRINTABLE / PUNCT / SPACE /
    UPPER
                        /  WORDCHAR / XDIGIT
                        / Identifier
                        /  OPEN Expression CLOSE
                        /  Literal
                        /  Class
                        /  DOT
                        ;
        Literal         <- APOSTROPH  (!APOSTROPH  Char)* APOSTROPH
```

```
    WHITESPACE
                        /  DAPOSTROPH (!DAPOSTROPH Char)* DAPOSTROPH
    WHITESPACE ;
        Class           <- OPENB (!CLOSEB Range)* CLOSEB WHITESPACE ;
        Range           <- Char TO Char / Char ;

        StartExpr       <- OPEN Expression CLOSE ;
void:   Final           <- "END" WHITESPACE SEMICOLON WHITESPACE ;

        # ------------------------------------------------------------
    -------
        # Lexing constructs

        Identifier      <- Ident WHITESPACE ;
leaf:   Ident           <- ([_:] / <alpha>) ([_:] / <alnum>)* ;
        Char            <- CharSpecial / CharOctalFull / CharOctalPart
                        /  CharUnicode / CharUnescaped
                        ;

leaf:   CharSpecial     <- "\\" [nrt'"\[\]\\] ;
leaf:   CharOctalFull   <- "\\" [0-3][0-7][0-7] ;
leaf:   CharOctalPart   <- "\\" [0-7][0-7]? ;
leaf:   CharUnicode     <- "\\" 'u' HexDigit (HexDigit (HexDigit
    HexDigit?)?)? ;
leaf:   CharUnescaped   <- !"\\" . ;

void:   HexDigit        <- [0-9a-fA-F] ;

void:   TO              <- '-'           ;
void:   OPENB           <- "["           ;
void:   CLOSEB          <- "]"           ;
void:   APOSTROPH       <- "'"           ;
void:   DAPOSTROPH      <- '"'           ;
void:   PEG             <- "PEG" !([_:] / <alnum>) WHITESPACE ;
void:   IS              <- "<-"    WHITESPACE ;
leaf:   VOID            <- "void"  WHITESPACE ; # Implies that
    definition has no semantic value.
leaf:   LEAF            <- "leaf"  WHITESPACE ; # Implies that
    definition has no terminals.
void:   SEMICOLON       <- ";"     WHITESPACE ;
void:   COLON           <- ":"     WHITESPACE ;
void:   SLASH           <- "/"     WHITESPACE ;
leaf:   AND             <- "&"     WHITESPACE ;
leaf:   NOT             <- "!"     WHITESPACE ;
leaf:   QUESTION        <- "?"     WHITESPACE ;
leaf:   STAR            <- "*"     WHITESPACE ;
```

```
leaf:   PLUS            <- "+"     WHITESPACE ;
void:   OPEN            <- "("     WHITESPACE ;
void:   CLOSE           <- ")"     WHITESPACE ;
leaf:   DOT             <- "."     WHITESPACE ;

leaf:   ALNUM           <- "<alnum>"    WHITESPACE ;
leaf:   ALPHA           <- "<alpha>"    WHITESPACE ;
leaf:   ASCII           <- "<ascii>"    WHITESPACE ;
leaf:   CONTROL         <- "<control>"  WHITESPACE ;
leaf:   DDIGIT          <- "<ddigit>"   WHITESPACE ;
leaf:   DIGIT           <- "<digit>"    WHITESPACE ;
leaf:   GRAPH           <- "<graph>"    WHITESPACE ;
leaf:   LOWER           <- "<lower>"    WHITESPACE ;
leaf:   PRINTABLE       <- "<print>"    WHITESPACE ;
leaf:   PUNCT           <- "<punct>"    WHITESPACE ;
leaf:   SPACE           <- "<space>"    WHITESPACE ;
leaf:   UPPER           <- "<upper>"    WHITESPACE ;
leaf:   WORDCHAR        <- "<wordchar>" WHITESPACE ;
leaf:   XDIGIT          <- "<xdigit>"   WHITESPACE ;

void:   WHITESPACE      <- (" " / "\t" / EOL / COMMENT)* ;
void:   COMMENT         <- '#' (!EOL .)* EOL ;
void:   EOL             <- "\n\r" / "\n" / "\r" ;
void:   EOF             <- !. ;

        # ------------------------------------------------------------
    -------
END;
```

Example

Our example specifies the grammar for a basic 4-operation calculator.

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

Using higher-level features of the notation, i.e. the character classes (predefined and custom), this example can be rewritten as

```
PEG calculator (Expression)
    Sign       <- [-+]                                  ;
    Number     <- Sign? <ddigit>+                       ;
    Expression <- Term (AddOp Term)*                    ;
    MulOp      <- [*/]                      ;
    Term       <- Factor (MulOp Factor)*                ;
    AddOp      <- [-+]                      ;
    Factor     <- '(' Expression ')' / Number           ;
END;
```

PEG serialization format

Here we specify the format used by the Parser Tools to serialize Parsing Expression Grammars as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a PEG may have more than one regular serialization only exactly one of them will be *canonical*.

regular serialization

1. The serialization of any PEG is a nested Tcl dictionary.
1. This dictionary holds a single key, **pt::grammar::peg**, and its value. This value holds the contents of the grammar.
2. The contents of the grammar are a Tcl dictionary holding the set of nonterminal symbols and the starting expression. The relevant keys and their values are

   rules

   The value is a Tcl dictionary whose keys are the names of the nonterminal symbols known to the grammar.

   1. Each nonterminal symbol may occur only once.
   2. The empty string is not a legal nonterminal symbol.
   3. The value for each symbol is a Tcl dictionary itself. The relevant keys and their values in this dictionary are

      is

      The value is the serialization of the parsing expression describing the symbols sentennial structure, as specified in the section PE serialization format.

      mode

      The value can be one of three values specifying how a parser should handle the semantic value produced by the symbol.

      value

      The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal itself, which has the ASTs of the symbol's right hand side as its children.

      leaf

      The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal, without any children. Any ASTs generated by the symbol's right hand side are discarded.

      void

      The nonterminal has no semantic value. Any ASTs generated by the symbol's right hand side are discarded (as well).

```
**start**
: The value is the serialization of the start parsing expression of
    the grammar, as specified in the section [PE serialization
    format].
```

1. The terminal symbols of the grammar are specified implicitly as the set of all terminal symbols used in the start expression and on the RHS of the grammar rules.

canonical serialization

The canonical serialization of a grammar has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this grammar.

1. The keys found in all the nested Tcl dictionaries are sorted in ascending dictionary order, as generated by Tcl's builtin command **lsort -increasing -dict**.
2. The string representation of the value is the canonical representation of a Tcl dictionary. I.e. it does not contain superfluous whitespace.

Example

Assuming the following PEG for simple mathematical expressions

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

then its canonical serialization (except for whitespace) is

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                  mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}                mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                  mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                  mode value}
        MulOp      {is {/ {t *} {t /}}
                                  mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
                                  mode value}
        Sign       {is {/ {t -} {t +}}
```

```
                                    mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                    mode value}
    }
    start {n Expression}
}
```

PE serialization format

Here we specify the format used by the Parser Tools to serialize Parsing Expressions as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a parsing expression may have more than one regular serialization only exactly one of them will be *canonical*.

Regular serialization

see below ...

Atomic Parsing Expressions

1. The string **epsilon** is an atomic parsing expression. It matches the empty string.
1. The string **dot** is an atomic parsing expression. It matches any character.
2. The string **alnum** is an atomic parsing expression. It matches any Unicode alphabet or digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
3. The string **alpha** is an atomic parsing expression. It matches any Unicode alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
4. The string **ascii** is an atomic parsing expression. It matches any Unicode character below U0080. This is a custom extension of PEs based on Tcl's builtin command **string is**.
5. The string **control** is an atomic parsing expression. It matches any Unicode control character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
6. The string **digit** is an atomic parsing expression. It matches any Unicode digit character. Note that this includes characters outside of the [0..9] range. This is a custom extension of PEs based on Tcl's builtin command **string is**.
7. The string **graph** is an atomic parsing expression. It matches any Unicode printing character, except for space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
8. The string **lower** is an atomic parsing expression. It matches any Unicode lower-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
9. The string **print** is an atomic parsing expression. It matches any Unicode printing character, including space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
10. The string **punct** is an atomic parsing expression. It matches any Unicode punctuation character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
11. The string **space** is an atomic parsing expression. It matches any Unicode space character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
12. The string **upper** is an atomic parsing expression. It matches any Unicode upper-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command

string is.

13. The string **wordchar** is an atomic parsing expression. It matches any Unicode word character. This is any alphanumeric character (see alnum), and any connector punctuation characters (e.g. underscore). This is a custom extension of PEs based on Tcl's builtin command **string is**.
14. The string **xdigit** is an atomic parsing expression. It matches any hexadecimal digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
15. The string **ddigit** is an atomic parsing expression. It matches any decimal digit character. This is a custom extension of PEs based on Tcl's builtin command **regexp**.
16. The expression [list t **x**] is an atomic parsing expression. It matches the terminal string **x**.
17. The expression [list n **A**] is an atomic parsing expression. It matches the nonterminal **A**.

Combined Parsing Expressions

1. For parsing expressions **e1**, **e2**, ... the result of [list / **e1 e2** ...] is a parsing expression as well. This is the *ordered choice*, aka *prioritized choice*.
1. For parsing expressions **e1**, **e2**, ... the result of [list x **e1 e2** ...] is a parsing expression as well. This is the *sequence*.
2. For a parsing expression **e** the result of [list * **e**] is a parsing expression as well. This is the *kleene closure*, describing zero or more repetitions.
3. For a parsing expression **e** the result of [list + **e**] is a parsing expression as well. This is the *positive kleene closure*, describing one or more repetitions.
4. For a parsing expression **e** the result of [list & **e**] is a parsing expression as well. This is the *and lookahead predicate*.
5. For a parsing expression **e** the result of [list! **e**] is a parsing expression as well. This is the *not lookahead predicate*.
6. For a parsing expression **e** the result of [list ? **e**] is a parsing expression as well. This is the *optional input*.

Canonical serialization

The canonical serialization of a parsing expression has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this parsing expression.

1. The string representation of the value is the canonical representation of a pure Tcl list. I.e. it does not contain superfluous whitespace.
2. Terminals are *not* encoded as ranges (where start and end of the range are identical).

Example

Assuming the parsing expression shown on the right-hand side of the rule

```
    Expression <- Term (AddOp Term)*
```

then its canonical serialization (except for whitespace) is

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you

want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pt* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Parsing and Grammars

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_peg_interp

Keywords: pt_peg_interp, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt::peg::interp - Interpreter for parsing expression grammars

Synopsis

package require Tcl 8.5 9
package require pt::peg::interp ?**1.0.2**?
package require pt::rde ?**1**?
package require snit
::pt::peg::interp *objectName grammar*
objectName **use** *grammar*
objectName **destroy**
objectName **parse** *chan*
objectName **parset** *text*

Description

Are you lost ? Do you have trouble understanding this document ? In that case please read the overview provided by the *Introduction to Parser Tools*. This document is the entrypoint to the whole system the current package is a part of.

This package provides a class whose instances are Packrat parsers configurable with a parsing expression grammar. The grammar is executed directly, i.e. interpreted, with the underlying runtime provided by the package **pt::rde**, basing everything on the PARAM.

Like the supporting runtime this package resides in the Execution section of the Core Layer of Parser Tools.

IMAGE: arch_core_transform

The interpreted grammar is copied from an instance of **pt::peg::container**, or anything providing the same API, like the container classes created by pt::peg::to::container or the

associated export plugin **pt::peg::export::container**.

Class API

The package exports the API described here.

::pt::peg::interp *objectName grammar*

The command creates a new parser object and returns the fully qualified name of the object command as its result. The API of this object command is described in the section Object API. It may be used to invoke various operations on the object.

This new parser is configured for the execution of an empty PEG. To configure the object for any other PEG use the method **use** of the Object API.

Object API

All objects created by this package provide the following methods.

objectName* use *grammar

This method configures the grammar interpreter / parser for the execution of the PEG stored in *grammar*, an object which is API-compatible to instances of **pt::peg::container**. The parser copies the relevant information of the grammar, and does *not* take ownership of the object.

The information of any previously used grammar is overwritten.

The result of the method the empty string.

***objectName* destroy**

This method destroys the parser instance, releasing all claimed memory and other resources, and deleting the instance command.

The result of the command is the empty string.

objectName* parse *chan

This method runs the parser using the contents of *chan* as input (starting at the current location in the channel), until parsing is not possible anymore, either because parsing has completed, or run into a syntax error.

Note here that the Parser Tools are based on Tcl 8.5+. In other words, the channel argument is not restricted to files, sockets, etc. We have the full power of *reflected channels* available.

It should also be noted that the parser pulls the characters from the input stream as it needs them. If a parser created by this package has to be operated in a push aka event-driven manner it will be necessary to go to Tcl 8.6+ and use the **coroutine::auto** to wrap it into a coroutine where **read** is properly changed for push-operation.

Upon successful completion the command returns an abstract syntax tree as its result. This AST is in the form specified in section AST serialization format. As a plain nested Tcl-list it can then be processed with any Tcl commands the user likes, doing transformations, semantic checks, etc. To help in this the package **pt::ast** provides a set of convenience commands for validation of the tree's basic structure, printing it for debugging, and walking it either from the bottom up, or top down.

When encountering a syntax error the command will throw an error instead. This error will be a 4-element Tcl-list, containing, in the order listed below:

1. The string **pt::rde** identifying it as parser runtime error.
2. The location of the parse error, as character offset from the beginning of the parsed input.
3. The location of parse error, now as a 2-element list containing line-number and column in the line.
4. A set of atomic parsing expressions indicating encoding the characters and/or nonterminal symbols the parser expected to see at the location of the parse error, but did not get. For the specification of atomic parsing expressions please see the section PE serialization format.

objectName* parset *text

This method runs the parser using the string in *text* as input. In all other ways it behaves like the method **parse**, shown above.

AST serialization format

Here we specify the format used by the Parser Tools to serialize Abstract Syntax Trees (ASTs) as immutable values for transport, comparison, etc.

Each node in an AST represents a nonterminal symbol of a grammar, and the range of tokens/characters in the input covered by it. ASTs do not contain terminal symbols, i.e. tokens/characters. These can be recovered from the input given a symbol's location.

We distinguish between *regular* and *canonical* serializations. While a tree may have more than one regular serialization only exactly one of them will be *canonical*.

Regular serialization

1. The serialization of any AST is the serialization of its root node.
1. The serialization of any node is a Tcl list containing at least three elements.
    1. The first element is the name of the nonterminal symbol stored in the node.
    2. The second and third element are the locations of the first and last token in the token stream the node represents (covers).
        1. Locations are provided as non-negative integer offsets from the beginning of the token stream, with the first token found in the stream located at offset 0 (zero).
        2. The end location has to be equal to or larger than the start location.

1. All elements after the first three represent the children of the node, which are themselves nodes. This means that the serializations of nodes without children, i.e. leaf nodes, have exactly three elements. The children are stored in the list with the leftmost child first, and the rightmost child last.

Canonical serialization

The canonical serialization of an abstract syntax tree has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this tree.

1. The string representation of the value is the canonical representation of a pure Tcl list. I.e. it does not contain superfluous whitespace.

Example

Assuming the parsing expression grammar below

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

and the input string

```
 120+5
```

then a parser should deliver the abstract syntax tree below (except for whitespace)

```
set ast {Expression 0 4
    {Factor 0 4
        {Term 0 2
            {Number 0 2
                {Digit 0 0}
                {Digit 1 1}
                {Digit 2 2}
            }
        }
        {AddOp 3 3}
        {Term 4 4
            {Number 4 4
                {Digit 4 4}
            }
        }
    }
}
```

Or, more graphical

```
+- Digit 0 0 | 1
|            |
+- Term 0 2 --- Number 0 2 -+- Digit 1 1 | 2
|                           |            |
|                           +- Digit 2 2 | 0
|                                        |
Expression 0 4 --- Factor 0 4 -+---------------------------- AddOp 3
    3 | +
|                                        |
+- Term 4 4 --- Number 4 4 --- Digit 4 4 | 5
```

PE serialization format

Here we specify the format used by the Parser Tools to serialize Parsing Expressions as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a parsing expression may have more than one regular serialization only exactly one of them will be *canonical*.

Regular serialization

see below ...

Atomic Parsing Expressions

1. The string **epsilon** is an atomic parsing expression. It matches the empty string.
1. The string **dot** is an atomic parsing expression. It matches any character.
2. The string **alnum** is an atomic parsing expression. It matches any Unicode alphabet or digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
3. The string **alpha** is an atomic parsing expression. It matches any Unicode alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
4. The string **ascii** is an atomic parsing expression. It matches any Unicode character below U0080. This is a custom extension of PEs based on Tcl's builtin command **string is**.
5. The string **control** is an atomic parsing expression. It matches any Unicode control character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
6. The string **digit** is an atomic parsing expression. It matches any Unicode digit character. Note that this includes characters outside of the [0..9] range. This is a custom extension of PEs based on Tcl's builtin command **string is**.
7. The string **graph** is an atomic parsing expression. It matches any Unicode printing character, except for space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
8. The string **lower** is an atomic parsing expression. It matches any Unicode lower-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
9. The string **print** is an atomic parsing expression. It matches any Unicode printing character, including space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
10. The string **punct** is an atomic parsing expression. It matches any Unicode punctuation character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
11. The string **space** is an atomic parsing expression. It matches any Unicode space character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
12. The string **upper** is an atomic parsing expression. It matches any Unicode upper-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
13. The string **wordchar** is an atomic parsing expression. It matches any Unicode word character. This is any alphanumeric character (see alnum), and any connector punctuation characters (e.g. underscore). This is a custom extension of PEs based on Tcl's builtin command **string is**.
14. The string **xdigit** is an atomic parsing expression. It matches any hexadecimal digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.

15. The string **ddigit** is an atomic parsing expression. It matches any decimal digit character. This is a custom extension of PEs based on Tcl's builtin command **regexp**.
16. The expression [list t **x**] is an atomic parsing expression. It matches the terminal string **x**.
17. The expression [list n **A**] is an atomic parsing expression. It matches the nonterminal **A**.

Combined Parsing Expressions

1. For parsing expressions **e1**, **e2**, ... the result of [list / **e1 e2** ...] is a parsing expression as well. This is the *ordered choice*, aka *prioritized choice*.
1. For parsing expressions **e1**, **e2**, ... the result of [list x **e1 e2** ...] is a parsing expression as well. This is the *sequence*.
2. For a parsing expression **e** the result of [list * **e**] is a parsing expression as well. This is the *kleene closure*, describing zero or more repetitions.
3. For a parsing expression **e** the result of [list + **e**] is a parsing expression as well. This is the *positive kleene closure*, describing one or more repetitions.
4. For a parsing expression **e** the result of [list & **e**] is a parsing expression as well. This is the *and lookahead predicate*.
5. For a parsing expression **e** the result of [list! **e**] is a parsing expression as well. This is the *not lookahead predicate*.
6. For a parsing expression **e** the result of [list ? **e**] is a parsing expression as well. This is the *optional input*.

Canonical serialization

The canonical serialization of a parsing expression has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this parsing expression.

1. The string representation of the value is the canonical representation of a pure Tcl list. I.e. it does not contain superfluous whitespace.
2. Terminals are *not* encoded as ranges (where start and end of the range are identical).

Example

Assuming the parsing expression shown on the right-hand side of the rule

```
    Expression <- Term (AddOp Term)*
```

then its canonical serialization (except for whitespace) is

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pt* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Parsing and Grammars

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_peg_introduction
Keywords: pt_peg_introduction, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name
pt::pegrammar - Introduction to Parsing Expression Grammars

Synopsis
package require Tcl 8.5 9

Description
Are you lost ? Do you have trouble understanding this document ? In that case please read the overview provided by the *Introduction to Parser Tools*. This document is the entrypoint to the whole system the current package is a part of.

Welcome to the introduction to *Parsing Expression Grammar*s (short: *PEG*), the formalism used by the Parser Tools. It is assumed that the reader has a basic knowledge of parsing theory, i.e. *Context-Free Grammars* (short: *CFG*), *languages*, and associated terms like *LL(k)*, *LR(k)*, *terminal* and *nonterminal symbols*, etc. We do not intend to recapitulate such basic definitions or terms like *useful*, *reachable*, (left/right) *recursive*, *nullable*, first/last/follow sets, etc. Please see the References at the end instead if you are in need of places and books which provide such background information.

PEGs are formally very similar to CFGs, with terminal and nonterminal symbols, start symbol, and rules defining the structure of each nonterminal symbol. The main difference lies in the choice(sic!) of *choice* operators. Where CFGs use an *unordered choice* to represent alternatives PEGs use *prioritized choice*. Which is fancy way of saying that a parser has to try the first alternative first and can try the other alternatives if only if it fails for the first, and so on.

On the CFG side this gives rise to LL(k) and LR(k) for making the choice *deterministic* with a bounded *lookahead* of k terminal symbols, where LL is in essence *topdown* aka *recursive descent* parsing, and LR *bottomup* aka *shift reduce* parsing.

On the PEG side we can parse input with recursive descent and *backtracking* of failed choices, the latter of which amounts to unlimited lookahead. By additionally recording the success or failure of nonterminals at the specific locations they were tried at and reusing this information after backtracking we can avoid the exponential blowup of running time usually associated with backtracking and keep the parsing linear. The memory requirements are of course higher due to this cache, as we are trading space for time.

This is the basic concept behind *packrat parsers*.

A limitation pure PEGs share with LL(k) CFGs is that *left-recursive* grammars cannot be parsed, with the associated recursive descent parser entering an infinite recursion. This limitation is usually overcome by extending pure PEGs with explicit operators to specify repetition, zero or more, and one or more, or, formally spoken, for the *kleene closure* and *positive kleene closure*. This is what the Parser Tools are doing.

Another extension, specific to Parser Tools, is a set of operators which map more or less directly to various character classes built into Tcl, i.e. the classes reachable via **string is**.

The remainder of this document consists of the formal definition of PEGs for the mathematically inclined, and an appendix listing references to places with more information on PEGs specifically, and parsing in general.

Formal definition

For the mathematically inclined, a Parsing Expression Grammar is a 4-tuple (VN,VT,R,eS) where

- VN is a set of *nonterminal symbols*,
- VT is a set of *terminal symbols*,
- R is a finite set of rules, where each rule is a pair (A,e), A in VN, and *e* a *parsing expression*.
- eS is a parsing expression, the *start expression*.

Further constraints are

- The intersection of VN and VT is empty.
- For all A in VT exists exactly one pair (A,e) in R. In other words, R is a function from nonterminal symbols to parsing expressions.

Parsing expressions are inductively defined via

- The empty string (epsilon) is a parsing expression.
- A terminal symbol *a* is a parsing expression.
- A nonterminal symbol *A* is a parsing expression.
- *e1e2* is a parsing expression for parsing expressions *e1* and *2*. This is called *sequence*.
- *e1*/*e2* is a parsing expression for parsing expressions *e1* and *2*. This is called *ordered choice*.
- *e** is a parsing expression for parsing expression *e*. This is called *zero-or-more repetitions*, also known as *kleene closure*.
- *e*+ is a parsing expression for parsing expression *e*. This is called *one-or-more repetitions*, also known as *positive kleene closure*.
- !*e* is a parsing expression for parsing expression *e1*. This is called a *not lookahead predicate*.
- &*e* is a parsing expression for parsing expression *e1*. This is called an *and lookahead predicate*.

PEGs are used to define a grammatical structure for streams of symbols over VT. They are a modern phrasing of older formalisms invented by Alexander Birham. These formalisms were called TS (TMG recognition scheme), and gTS (generalized TS). Later they were renamed to TPDL (Top-Down Parsing Languages) and gTPDL (generalized TPDL).

They can be easily implemented by recursive descent parsers with backtracking. This makes them relatives of LL(k) Context-Free Grammars.

References

1. The Packrat Parsing and Parsing Expression Grammars Page, by Bryan Ford, Massachusetts Institute of Technology. This is the main entry page to PEGs, and their realization through Packrat Parsers.

2. http://en.wikipedia.org/wiki/Parsing_expression_grammar Wikipedia's entry about Parsing Expression Grammars.
3. *Parsing Techniques - A Practical Guide* http://www.cs.vu.nl/~dick/PTAPG.html, an online book offering a clear, accessible, and thorough discussion of many different parsing techniques with their interrelations and applicabilities, including error recovery techniques.
4. Compilers and Compiler Generators, an online book using CoCo/R, a generator for recursive descent parsers.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pt* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Parsing and Grammars

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_peg_language

Keywords: pt_peg_language, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt::peg_language - PEG Language Tutorial

Synopsis

package require Tcl 8.5 9

Description

Are you lost ? Do you have trouble understanding this document ? In that case please read the overview provided by the *Introduction to Parser Tools*. This document is the entrypoint to the whole system the current package is a part of.

Welcome to the tutorial / introduction for the PEG specification language. If you are already familiar with the language we are about to discuss, and only wish to refresh your memory you can, of course, skip ahead to the aforementioned section and just read the full formal specification.

What is it?

peg, a language for the specification of parsing expression grammars is meant to be human

readable, and writable as well, yet strict enough to allow its processing by machine. Like any computer language. It was defined to make writing the specification of a grammar easy, something the other formats found in the Parser Tools do not lend themselves too.

The elements of the language

Basic structure

The general outline of a textual PEG is

```
PEG <<name>> (<<start-expression>>)
   <<rules>>
END;
```

Note: We are using text in double angle-brackets as place-holders for things not yet explained.

Names

Names are mostly used to identify the nonterminal symbols of the grammar, i.e. that which occurs on the left-hand side of a. The exception to that is the name given after the keyword **PEG** (see previous section), which is the name of the whole grammar itself.

The structure of a name is simple:

1. It begins with a letter, underscore, or colon, followed by
2. zero or more letters, digits, underscores, or colons.

Or, in formal textual notation:

```
    ([_:] / <alpha>) ([_:] / <alnum>)*
```

Examples of names:

```
    Hello
    ::world
    _:submarine55_
```

Examples of text which are *not* names:

```
    12
    .bogus
    0wrong
    @location
```

Rules

The main body of the text of a grammar specification is taken up by the rules. Each rule defines the sentence structure of one nonterminal symbol. Their basic structure is

```
     <<name>>  <-  <<expression>> ;
```

The specifies the nonterminal symbol to be defined, the after the arrow (<-) then declares its structure.

Note that each rule ends in a single semicolon, even the last. I.e. the semicolon is a rule *terminator*, not a separator.

We can have as many rules as we like, as long as we define each nonterminal symbol at most

once, and have at least one rule for each nonterminal symbol which occured in an expression, i.e. in either the start expression of the grammar, or the right-hande side of a rule.

Expressions

The *parsing* expressions are the meat of any specification. They declare the structure of the whole document (<>), and of all nonterminal symbols.

All expressions are made up out of *atomic expressions* and *operators* combining them. We have operators for choosing between alternatives, repetition of parts, and for look-ahead constraints. There is no explicit operator for the sequencing (also known as *concatenation*) of parts however. This is specified by simply placing the parts adjacent to each other.

Here are the operators, from highest to lowest priority (i.e. strength of binding):

```
    # Binary operators.

    <<expression-1>>     <<expression-2>>  # sequence. parse 1, then 2.
    <<expression-1>>  /  <<expression-2>>  # alternative. try to parse
    1, and parse 2 if 1 failed to parse.

    # Prefix operators. Lookahead constraints. Same priority.

    & <<expression>>  # Parse expression, ok on successful parse.
    ! <<expression>>  # Ditto, except ok on failure to parse.

    # Suffix operators. Repetition. Same priority.

    <<expression>> ?  # Parse expression none, or once (repeat 0 or 1).
    <<expression>> *  # Parse expression zero or more times.
    <<expression>> +  # Parse expression one or more times.

    # Expression nesting

    ( <<expression>> ) # Put an expression in parens to change its
    priority.
```

With this we can now deconstruct the formal expression for names given in section Names:

```
    ([_:] / <alpha>) ([_:] / <alnum>)*
```

It is a sequence of two parts,

```
    [_:] / <alpha>
```

and

```
    ([_:] / <alnum>)*
```

The parentheses around the parts kept their inner alternatives bound together against the normally higher priority of the sequence. Each of the two parts is an alternative, with the second part additionally repeated zero or more times, leaving us with the three atomic expressions

```
    [_:]
    <alpha>
    <alnum>
```

And *atomic expressions* are our next topic. They fall into three classes:

1. names, i.e. nonterminal symbols,
2. string literals, and
3. character classes.

Names we know about already, or see section Names for a refresher.

String literals are simple. They are delimited by (i.e. start and end with) either a single or double-apostroph, and in between the delimiters we can have any character but the delimiter itself. They can be empty as well. Examples of strings are

```
    ''
    ""
    'hello'
    "umbra"
    "'"
    '"'
```

The last two examples show how to place any of the delimiters into a string.

For the last, but not least of our atomic expressions, character classes, we have a number of predefined classes, shown below, and the ability to construct or own. The predefined classes are:

```
    <alnum>    # Any unicode alphabet or digit character (string is
    alnum).
    <alpha>    # Any unicode alphabet character (string is alpha).
    <ascii>    # Any unicode character below codepoint 0x80 (string is
    ascii).
    <control>  # Any unicode control character (string is control).
    <ddigit>   # The digit characters [0-9].
    <digit>    # Any unicode digit character (string is digit).
    <graph>    # Any unicode printing character, except space (string
    is graph).
    <lower>    # Any unicode lower-case alphabet character (string is
    lower).
    <print>    # Any unicode printing character, incl. space (string
    is print).
    <punct>    # Any unicode punctuation character (string is punct).
    <space>    # Any unicode space character (string is space).
    <upper>    # Any unicode upper-case alphabet character (string is
    upper).
    <wordchar> # Any unicode word character (string is wordchar).
    <xdigit>   # The hexadecimal digit characters [0-9a-fA-F].
    .          # Any character, except end of input.
```

And the syntax of custom-defined character classes is

```
    [ <<range>>* ]
```

where each range is either a single character, or of the form

```
   <<character>> - <character>>
```

Examples for character classes we have seen already in the course of this introduction are

```
    [_:]
    [0-9]
    [0-9a-fA-F]
```

We are nearly done with expressions. The only piece left is to tell how the characters in character classes and string literals are specified.

Basically characters in the input stand for themselves, and in addition to that we several types of escape syntax to to repesent control characters, or characters outside of the encoding the text is in.

All the escaped forms are started with a backslash character ('\', unicode codepoint 0x5C). This is then followed by a series of octal digits, or 'u' and hexedecimal digits, or a regular character from a fixed set for various control characters. Some examples:

```
    \n \r \t \' \" \[ \] \\ #
    \000 up to \277         # octal escape, all ascii character,
    leading 0's can be removed.
    \u2CA7                  # hexadecimal escape, all unicode
    characters.
    #                       # Here 2ca7 <=> Koptic Small Letter Tau
```

WHITESPACE and comments

One issue not touched upon so far is whitespace and comments.

Whitespace is any unicode space character, i.e. anything in the character class, and comments. The latter are sequences of characters starting with a '#' (hash, unicode codepoint 0x23) and ending at the next end-of-line.

Whitespace can be freely used between all syntactical elements of a grammar specification. It cannot be used inside of syntactical elements, like names, string literals, predefined character classes, etc.

Nonterminal attributes

Lastly, a more advanced topic. In the section Rules we gave the structure of a rule as

```
     <<name>>  <-  <<expression>> ;
```

This is not quite true. It is possible to associate a semantic mode with the nonterminal in the rule, by writing it before the name, separated from it by a colon, i.e. writing

```
    <<mode>> : <<name>>  <-  <<expression>> ;
```

is also allowed. This mode is optional. The known modes and their meanings are:

value

The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single

node node for the nonterminal itself, which has the ASTs of the symbol's right hand side as its children.

leaf
The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal, without any children. Any ASTs generated by the symbol's right hand side are discarded.

void
The nonterminal has no semantic value. Any ASTs generated by the symbol's right hand side are discarded (as well).

Of these three modes only **leaf** and **void** can be specified directly. **value** is implicitly specified by the absence of a mode before the nonterminal.

Now, with all the above under our belt it should be possible to not only read, but understand the formal specification of the text representation shown in the next section, written in itself.

PEG specification language

peg, a language for the specification of parsing expression grammars is meant to be human readable, and writable as well, yet strict enough to allow its processing by machine. Like any computer language. It was defined to make writing the specification of a grammar easy, something the other formats found in the Parser Tools do not lend themselves too.

It is formally specified by the grammar shown below, written in itself. For a tutorial / introduction to the language please go and read the *PEG Language Tutorial*.

```
PEG pe_grammar_for_peg (Grammar)

    # ------------------------------------------------------------------
        # Syntactical constructs

        Grammar         <- WHITESPACE Header Definition* Final EOF ;

        Header          <- PEG Identifier StartExpr ;
        Definition      <- Attribute? Identifier IS Expression
    SEMICOLON ;
        Attribute       <- (VOID / LEAF) COLON ;
        Expression      <- Sequence (SLASH Sequence)* ;
        Sequence        <- Prefix+ ;
        Prefix          <- (AND / NOT)? Suffix ;
        Suffix          <- Primary (QUESTION / STAR / PLUS)? ;
        Primary         <- ALNUM / ALPHA / ASCII / CONTROL / DDIGIT /
    DIGIT
                        /  GRAPH / LOWER / PRINTABLE / PUNCT / SPACE /
    UPPER
                        /  WORDCHAR / XDIGIT
                        / Identifier
                        /  OPEN Expression CLOSE
                        /  Literal
```

```
                        /  Class
                        /  DOT
                        ;
        Literal         <- APOSTROPH  (!APOSTROPH  Char)* APOSTROPH
    WHITESPACE
                        /  DAPOSTROPH (!DAPOSTROPH Char)* DAPOSTROPH
    WHITESPACE ;
        Class           <- OPENB (!CLOSEB Range)* CLOSEB WHITESPACE ;
        Range           <- Char TO Char / Char ;

        StartExpr       <- OPEN Expression CLOSE ;
void:   Final           <- "END" WHITESPACE SEMICOLON WHITESPACE ;

        # ------------------------------------------------------------
    -------
        # Lexing constructs

        Identifier      <- Ident WHITESPACE ;
leaf:   Ident           <- ([_:] / <alpha>) ([_:] / <alnum>)* ;
        Char            <- CharSpecial / CharOctalFull / CharOctalPart
                        /  CharUnicode / CharUnescaped
                        ;

leaf:   CharSpecial     <- "\\" [nrt'"\[\]\\] ;
leaf:   CharOctalFull   <- "\\" [0-3][0-7][0-7] ;
leaf:   CharOctalPart   <- "\\" [0-7][0-7]? ;
leaf:   CharUnicode     <- "\\" 'u' HexDigit (HexDigit (HexDigit
    HexDigit?)?)? ;
leaf:   CharUnescaped   <- !"\\" . ;

void:   HexDigit        <- [0-9a-fA-F] ;

void:   TO              <- '-'           ;
void:   OPENB           <- "["           ;
void:   CLOSEB          <- "]"           ;
void:   APOSTROPH       <- "'"           ;
void:   DAPOSTROPH      <- '"'           ;
void:   PEG             <- "PEG" !([_:] / <alnum>) WHITESPACE ;
void:   IS              <- "<-"    WHITESPACE ;
leaf:   VOID            <- "void"  WHITESPACE ; # Implies that
    definition has no semantic value.
leaf:   LEAF            <- "leaf"  WHITESPACE ; # Implies that
    definition has no terminals.
void:   SEMICOLON       <- ";"     WHITESPACE ;
void:   COLON           <- ":"     WHITESPACE ;
void:   SLASH           <- "/"     WHITESPACE ;
```

```
leaf:   AND             <- "&"     WHITESPACE ;
leaf:   NOT             <- "!"     WHITESPACE ;
leaf:   QUESTION        <- "?"     WHITESPACE ;
leaf:   STAR            <- "*"     WHITESPACE ;
leaf:   PLUS            <- "+"     WHITESPACE ;
void:   OPEN            <- "("     WHITESPACE ;
void:   CLOSE           <- ")"     WHITESPACE ;
leaf:   DOT             <- "."     WHITESPACE ;

leaf:   ALNUM           <- "<alnum>"    WHITESPACE ;
leaf:   ALPHA           <- "<alpha>"    WHITESPACE ;
leaf:   ASCII           <- "<ascii>"    WHITESPACE ;
leaf:   CONTROL         <- "<control>"  WHITESPACE ;
leaf:   DDIGIT          <- "<ddigit>"   WHITESPACE ;
leaf:   DIGIT           <- "<digit>"    WHITESPACE ;
leaf:   GRAPH           <- "<graph>"    WHITESPACE ;
leaf:   LOWER           <- "<lower>"    WHITESPACE ;
leaf:   PRINTABLE       <- "<print>"    WHITESPACE ;
leaf:   PUNCT           <- "<punct>"    WHITESPACE ;
leaf:   SPACE           <- "<space>"    WHITESPACE ;
leaf:   UPPER           <- "<upper>"    WHITESPACE ;
leaf:   WORDCHAR        <- "<wordchar>" WHITESPACE ;
leaf:   XDIGIT          <- "<xdigit>"   WHITESPACE ;

void:   WHITESPACE      <- (" " / "\t" / EOL / COMMENT)* ;
void:   COMMENT         <- '#' (!EOL .)* EOL ;
void:   EOL             <- "\n\r" / "\n" / "\r" ;
void:   EOF             <- !. ;

        # ------------------------------------------------------------
    -------
END;
```

Example

Our example specifies the grammar for a basic 4-operation calculator.

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

Using higher-level features of the notation, i.e. the character classes (predefined and custom), this example can be rewritten as

```
PEG calculator (Expression)
    Sign       <- [-+]                                  ;
    Number     <- Sign? <ddigit>+                       ;
    Expression <- Term (AddOp Term)*                    ;
    MulOp      <- [*/]                      ;
    Term       <- Factor (MulOp Factor)*                ;
    AddOp      <- [-+]                      ;
    Factor     <- '(' Expression ')' / Number           ;
END;
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pt* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Parsing and Grammars

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

pt_peg_op

Keywords: pt_peg_op, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt_peg_op - Parser Tools PE Grammar Utility Operations

Synopsis

package require Tcl 8.5 9
package require pt::peg::op ?**1.2.0**?
::peg::peg::op called *container*
::peg::peg::op dechain *container*
::peg::peg::op drop unreachable *container*
::peg::peg::op drop unrealizable *container*
::peg::peg::op flatten *container*
::peg::peg::op minimize *container*

::peg::peg::op modeopt *container*
::peg::peg::op reachable *container*
::peg::peg::op realizable *container*

Description

Are you lost ? Do you have trouble understanding this document ? In that case please read the overview provided by the *Introduction to Parser Tools*. This document is the entrypoint to the whole system the current package is a part of.

This package provides a number of utility commands manipulating a PE grammar (container) in various ways.

API

::peg::peg::op called *container*

This command determines the static call structure for the nonterminal symbols of the grammar stored in the *container*.

The result of the command is a dictionary mapping from each symbol to the symbols it calls. The empty string is the key used to represent the start expression of the grammar.

The grammar in the container is not modified.

The *container* instance has to expose a method API as is provided by the package **pt::peg::container**.

::peg::peg::op dechain *container*

This command simplifies all symbols which just chain to a different symbol by inlining the right hand side of the called symbol in its callers. This works if and only the modes match properly, per the decision table below.

```
caller called | dechain | notes
--------------+---------+-----------------------
value  value  |  yes    |  value is passed
value  leaf   |  yes    |  value is passed
value  void   |  yes    |  caller is implied void
leaf   value  |  no     |  generated value was discarded, inlined
    would not. called may be implied void.
leaf   leaf   |  no     |  s.a.
leaf   void   |  no     |  s.a.
void   value  |  no     |  caller drops value, inlined would not.
void   leaf   |  no     |  s.a.
void   void   |  yes    |
```

The result of the command is the empty string.

The grammar in the container is directly modified. If that is not wanted, a copy of the original container has to be used.

The *container* instance has to expose a method API as is provided by the package **pt::peg::container**.

::peg::peg::op drop unreachable *container*
This command removes all symbols from the grammar which are not **reachable**.

The result of the command is the empty string.

The grammar in the container is directly modified. If that is not wanted, a copy of the original container has to be used.

The *container* instance has to expose a method API as is provided by the package **pt::peg::container**.

::peg::peg::op drop unrealizable *container*
This command removes all symbols from the grammar which are not **realizable**.

The result of the command is the empty string.

The grammar in the container is directly modified. If that is not wanted, a copy of the original container has to be used.

The *container* instance has to expose a method API as is provided by the package **pt::peg::container**.

::peg::peg::op flatten *container*
This command flattens (see **pt::pe::op**) all expressions in the grammar, i.e. the start expression and the right hand sides of all nonterminal symbols.

The result of the command is the empty string.

The grammar in the container is directly modified. If that is not wanted, a copy of the original container has to be used.

The *container* instance has to expose a method API as is provided by the package **pt::peg::container**.

::peg::peg::op minimize *container*
This command reduces the provided grammar by applying most of the other methods of this package.

After flattening the expressions it removes unreachable and unrealizable symbols, flattens the expressions again, then optimizes the symbol modes before collapsing symbol chains as much as possible.

The result of the command is the empty string.

The grammar in the container is directly modified. If that is not wanted, a copy of the original container has to be used.

The *container* instance has to expose a method API as is provided by the package **pt::peg::container**.

::peg::peg::op modeopt *container*

This command optimizes the semantic modes of non-terminal symbols according to the two rules below.

1. If a symbol X with mode **value** calls no other symbols, i.e. uses only terminal symbols in whatever combination, then this can be represented simpler by using mode **leaf**.
2. If a symbol X is only called from symbols with modes **leaf** or **void** then this symbol should have mode **void** also, as any AST it could generate will be discarded anyway.

The result of the command is the empty string.

The grammar in the container is directly modified. If that is not wanted, a copy of the original container has to be used.

The *container* instance has to expose a method API as is provided by the package **pt::peg::container**.

::peg::peg::op reachable *container*

This command computes the set of all nonterminal symbols which are reachable from the start expression of the grammar. This is essentially the transitive closure over **called** and the symbol's right hand sides, beginning with the start expression.

The result of the command is the list of reachable symbols.

The grammar in the container is not modified.

The *container* instance has to expose a method API as is provided by the package **pt::peg::container**.

::peg::peg::op realizable *container*

This command computes the set of all nonterminal symbols which are realizable, i.e. can derive pure terminal phrases. This is done iteratively, starting with state unrealizable for all and any, and then updating all symbols which are realizable, propagating changes, until nothing changes any more.

The result of the command is the list of realizable symbols.

The grammar in the container is not modified.

The *container* instance has to expose a method API as is provided by the package **pt::peg::container**.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pt* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Parsing and Grammars

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_peg_to_container
Keywords: pt_peg_to_container, CONTAINER, EBNF, LL(k), PEG, TDPL, context-free languages, conversion, expression, format conversion, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, serialization, state, top-down parsing languages, transducer

Name
pt::peg::to::container - PEG Conversion. Write CONTAINER format

Synopsis
package require Tcl 8.5 9
package require pt::peg::to::container ?**1.1**?
package require pt::peg
package require text::write
package require char
pt::peg::to::container reset
pt::peg::to::container configure
pt::peg::to::container configure *option*
pt::peg::to::container configure *option value...*
pt::peg::to::container convert *serial*

Description
Are you lost ? Do you have trouble understanding this document ? In that case please read the overview provided by the *Introduction to Parser Tools*. This document is the entrypoint to the whole system the current package is a part of.

This package implements the converter from parsing expression grammars to CONTAINER markup.

It resides in the Export section of the Core Layer of Parser Tools, and can be used either directly with the other packages of this layer, or indirectly through the export manager provided by **pt::peg::export**. The latter is intented for use in untrusted environments and done through the corresponding export plugin **pt::peg::export::container** sitting between converter and export manager.

IMAGE: arch_core_eplugins

API
The API provided by this package satisfies the specification of the Converter API found in the *Parser Tools Export API* specification.

pt::peg::to::container reset

This command resets the configuration of the package to its default settings.

pt::peg::to::container configure
This command returns a dictionary containing the current configuration of the package.

pt::peg::to::container configure *option*
This command returns the current value of the specified configuration *option* of the package. For the set of legal options, please read the section Options.

pt::peg::to::container configure *option value...*
This command sets the given configuration *option*s of the package, to the specified *value*s. For the set of legal options, please read the section Options.

pt::peg::to::container convert *serial*
This command takes the canonical serialization of a parsing expression grammar, as specified in section Peg serialization format, and contained in *serial*, and generates CONTAINER markup encoding the grammar, per the current package configuration. The created string is then returned as the result of the command.

Options

The converter to the CONTAINER format recognizes the following options and changes its behaviour as they specify.

-file string
The value of this option is the name of the file or other entity from which the grammar came, for which the command is run. The default value is **unknown**.

-name string
The value of this option is the name of the grammar we are processing. The default value is **a_pe_grammar**.

-user string
The value of this option is the name of the user for which the command is run. The default value is **unknown**.

-mode bulk|incremental
The value of this option controls which methods of **pt::peg::container** instances are used to specify the grammar, i.e. preload it into the container. There are two legal values, as listed below. The default is **bulk**.

bulk
In this mode the methods **start**, **add**, **modes**, and **rules** are used to specify the grammar in a bulk manner, i.e. as a set of nonterminal symbols, and two dictionaries mapping from the symbols to their semantic modes and parsing expressions.

This mode is the default.

incremental
In this mode the methods **start**, **add**, **mode**, and **rule** are used to specify the grammar piecemal, with each nonterminal having its own block of defining commands.

-template string
The value of this option is a string into which to put the generated code and the other configuration settings. The various locations for user-data are expected to be specified with the placeholders listed below. The default value is "**@code@**".

@user@
To be replaced with the value of the option **-user**.

@format@
To be replaced with the the constant **CONTAINER**.

@file@
To be replaced with the value of the option **-file**.

@name@
To be replaced with the value of the option **-name**.

@mode@
To be replaced with the value of the option **-mode**.

@code@
To be replaced with the generated code.

Grammar container

The **container** format is another form of describing parsing expression grammars. While data in this format is executable it does not constitute a parser for the grammar. It always has to be used in conjunction with the package **pt::peg::interp**, a grammar interpreter.

The format represents grammars by a **snit::type**, i.e. class, whose instances are API-compatible to the instances of the **pt::peg::container** package, and which are preloaded with the grammar in question.

It has no direct formal specification beyond what was said above.

Example

Assuming the following PEG for simple mathematical expressions

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

one possible CONTAINER serialization for it is

```
snit::type a_pe_grammar {
    constructor {} {
        install myg using pt::peg::container ${selfns}::G
        $myg start {n Expression}
        $myg add   AddOp Digit Expression Factor MulOp Number Sign Term
        $myg modes {
            AddOp      value
            Digit      value
```

```
            Expression value
            Factor     value
            MulOp      value
            Number     value
            Sign       value
            Term       value
        }
        $myg rules {
            AddOp      {/ {t -} {t +}}
            Digit      {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}
            Expression {/ {x {t \50} {n Expression} {t \51}} {x {n
    Factor} {* {x {n MulOp} {n Factor}}}}}
            Factor     {x {n Term} {* {x {n AddOp} {n Term}}}}
            MulOp      {/ {t *} {t /}}
            Number     {x {? {n Sign}} {+ {n Digit}}}
            Sign       {/ {t -} {t +}}
            Term       {n Number}
        }
        return
    }

    component myg
    delegate method * to myg
}
```

Peg serialization format

Here we specify the format used by the Parser Tools to serialize Parsing Expression Grammars as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a PEG may have more than one regular serialization only exactly one of them will be *canonical*.

regular serialization

1. The serialization of any PEG is a nested Tcl dictionary.
1. This dictionary holds a single key, **pt::grammar::peg**, and its value. This value holds the contents of the grammar.
2. The contents of the grammar are a Tcl dictionary holding the set of nonterminal symbols and the starting expression. The relevant keys and their values are

   rules

   The value is a Tcl dictionary whose keys are the names of the nonterminal symbols known to the grammar.

   1. Each nonterminal symbol may occur only once.
   2. The empty string is not a legal nonterminal symbol.
   3. The value for each symbol is a Tcl dictionary itself. The relevant keys and their values in this dictionary are

      is

The value is the serialization of the parsing expression describing the symbols sentennial structure, as specified in the section PE serialization format.

mode
The value can be one of three values specifying how a parser should handle the semantic value produced by the symbol.

value
The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal itself, which has the ASTs of the symbol's right hand side as its children.

leaf
The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal, without any children. Any ASTs generated by the symbol's right hand side are discarded.

void
The nonterminal has no semantic value. Any ASTs generated by the symbol's right hand side are discarded (as well).

```
**start**
: The value is the serialization of the start parsing expression of
    the grammar, as specified in the section [PE serialization
    format].
```

1. The terminal symbols of the grammar are specified implicitly as the set of all terminal symbols used in the start expression and on the RHS of the grammar rules.

canonical serialization
The canonical serialization of a grammar has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this grammar.

1. The keys found in all the nested Tcl dictionaries are sorted in ascending dictionary order, as generated by Tcl's builtin command **lsort -increasing -dict**.
2. The string representation of the value is the canonical representation of a Tcl dictionary. I.e. it does not contain superfluous whitespace.

Example

Assuming the following PEG for simple mathematical expressions

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

then its canonical serialization (except for whitespace) is

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                  mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}                mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                  mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                  mode value}
        MulOp      {is {/ {t *} {t /}}
                                  mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
                                  mode value}
        Sign       {is {/ {t -} {t +}}
                                  mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                  mode value}
    }
    start {n Expression}
}
```

PE serialization format

Here we specify the format used by the Parser Tools to serialize Parsing Expressions as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a parsing expression may have more than one regular serialization only exactly one of them will be *canonical*.

Regular serialization

see below ...

Atomic Parsing Expressions

1. The string **epsilon** is an atomic parsing expression. It matches the empty string.
1. The string **dot** is an atomic parsing expression. It matches any character.
2. The string **alnum** is an atomic parsing expression. It matches any Unicode alphabet or

digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.

3. The string **alpha** is an atomic parsing expression. It matches any Unicode alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
4. The string **ascii** is an atomic parsing expression. It matches any Unicode character below U0080. This is a custom extension of PEs based on Tcl's builtin command **string is**.
5. The string **control** is an atomic parsing expression. It matches any Unicode control character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
6. The string **digit** is an atomic parsing expression. It matches any Unicode digit character. Note that this includes characters outside of the [0..9] range. This is a custom extension of PEs based on Tcl's builtin command **string is**.
7. The string **graph** is an atomic parsing expression. It matches any Unicode printing character, except for space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
8. The string **lower** is an atomic parsing expression. It matches any Unicode lower-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
9. The string **print** is an atomic parsing expression. It matches any Unicode printing character, including space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
10. The string **punct** is an atomic parsing expression. It matches any Unicode punctuation character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
11. The string **space** is an atomic parsing expression. It matches any Unicode space character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
12. The string **upper** is an atomic parsing expression. It matches any Unicode upper-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
13. The string **wordchar** is an atomic parsing expression. It matches any Unicode word character. This is any alphanumeric character (see alnum), and any connector punctuation characters (e.g. underscore). This is a custom extension of PEs based on Tcl's builtin command **string is**.
14. The string **xdigit** is an atomic parsing expression. It matches any hexadecimal digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
15. The string **ddigit** is an atomic parsing expression. It matches any decimal digit character. This is a custom extension of PEs based on Tcl's builtin command **regexp**.
16. The expression [list t **x**] is an atomic parsing expression. It matches the terminal string **x**.
17. The expression [list n **A**] is an atomic parsing expression. It matches the nonterminal **A**.

Combined Parsing Expressions

1. For parsing expressions **e1**, **e2**, ... the result of [list / **e1 e2** ...] is a parsing expression as well. This is the *ordered choice*, aka *prioritized choice*.

1. For parsing expressions **e1**, **e2**, ... the result of [list x **e1 e2** ...] is a parsing expression as well. This is the *sequence*.
2. For a parsing expression **e** the result of [list * **e**] is a parsing expression as well. This is the *kleene closure*, describing zero or more repetitions.

3. For a parsing expression **e** the result of [list + **e**] is a parsing expression as well. This is the *positive kleene closure*, describing one or more repetitions.
4. For a parsing expression **e** the result of [list & **e**] is a parsing expression as well. This is the *and lookahead predicate*.
5. For a parsing expression **e** the result of [list! **e**] is a parsing expression as well. This is the *not lookahead predicate*.
6. For a parsing expression **e** the result of [list ? **e**] is a parsing expression as well. This is the *optional input*.

Canonical serialization

The canonical serialization of a parsing expression has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this parsing expression.

1. The string representation of the value is the canonical representation of a pure Tcl list. I.e. it does not contain superfluous whitespace.
2. Terminals are *not* encoded as ranges (where start and end of the range are identical).

Example

Assuming the parsing expression shown on the right-hand side of the rule

```
    Expression <- Term (AddOp Term)*
```

then its canonical serialization (except for whitespace) is

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pt* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Parsing and Grammars

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_peg_to_cparam

Keywords: pt_peg_to_cparam, CPARAM, EBNF, LL(k), PEG, TDPL, context-free languages, conversion, expression, format conversion, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, serialization, state,

top-down parsing languages, transducer

Name

pt::peg::to::cparam - PEG Conversion. Write CPARAM format

Synopsis

package require Tcl 8.5 9
package require pt::peg::to::cparam ?**1.1.3**?
pt::peg::to::cparam reset
pt::peg::to::cparam configure
pt::peg::to::cparam configure *option*
pt::peg::to::cparam configure *option value...*
pt::peg::to::cparam convert *serial*

Description

Are you lost ? Do you have trouble understanding this document ? In that case please read the overview provided by the *Introduction to Parser Tools*. This document is the entrypoint to the whole system the current package is a part of.

This package implements the converter from parsing expression grammars to CPARAM markup.

It resides in the Export section of the Core Layer of Parser Tools, and can be used either directly with the other packages of this layer, or indirectly through the export manager provided by **pt::peg::export**. The latter is intented for use in untrusted environments and done through the corresponding export plugin **pt::peg::export::cparam** sitting between converter and export manager.

IMAGE: arch_core_eplugins

API

The API provided by this package satisfies the specification of the Converter API found in the *Parser Tools Export API* specification.

pt::peg::to::cparam reset
This command resets the configuration of the package to its default settings.

pt::peg::to::cparam configure
This command returns a dictionary containing the current configuration of the package.

pt::peg::to::cparam configure *option*
This command returns the current value of the specified configuration *option* of the package. For the set of legal options, please read the section Options.

pt::peg::to::cparam configure *option value...*
This command sets the given configuration *option*s of the package, to the specified *value*s. For the set of legal options, please read the section Options.

pt::peg::to::cparam convert *serial*
This command takes the canonical serialization of a parsing expression grammar, as specified in section Peg serialization format, and contained in *serial*, and generates CPARAM markup encoding the grammar, per the current package configuration. The created string is then returned as the result of the command.

Options

The converter to C code recognizes the following configuration variables and changes its behaviour as they specify.

-file string
: The value of this option is the name of the file or other entity from which the grammar came, for which the command is run. The default value is **unknown**.

-name string
: The value of this option is the name of the grammar we are processing. The default value is **a_pe_grammar**.

-user string
: The value of this option is the name of the user for which the command is run. The default value is **unknown**.

-template string
: The value of this option is a string into which to put the generated text and the other configuration settings. The various locations for user-data are expected to be specified with the placeholders listed below. The default value is "**@code@**".

  @user@
  : To be replaced with the value of the option **-user**.

  @format@
  : To be replaced with the the constant **C/PARAM**.

  @file@
  : To be replaced with the value of the option **-file**.

  @name@
  : To be replaced with the value of the option **-name**.

  @code@
  : To be replaced with the generated Tcl code.

  The following options are special, in that they will occur within the generated code, and are replaced there as well.

  @statedecl@
  : To be replaced with the value of the option **state-decl**.

  @stateref@
  : To be replaced with the value of the option **state-ref**.

  @strings@
  : To be replaced with the value of the option **string-varname**.

  @self@
  : To be replaced with the value of the option **self-command**.

  @def@
  : To be replaced with the value of the option **fun-qualifier**.

  @ns@
  : To be replaced with the value of the option **namespace**.

  @main@
  : To be replaced with the value of the option **main**.

  @prelude@

To be replaced with the value of the option **prelude**.

-state-decl string
A C string representing the argument declaration to use in the generated parsing functions to refer to the parsing state. In essence type and argument name. The default value is the string **RDE_PARAM p**.

-state-ref string
A C string representing the argument named used in the generated parsing functions to refer to the parsing state. The default value is the string **p**.

-self-command string
A C string representing the reference needed to call the generated parser function (methods ...) from another parser fonction, per the chosen framework (template). The default value is the empty string.

-fun-qualifier string
A C string containing the attributes to give to the generated functions (methods ...), per the chosen framework (template). The default value is **static**.

-namespace string
The name of the C namespace the parser functions (methods, ...) shall reside in, or a general prefix to add to the function names. The default value is the empty string.

-main string
The name of the main function (method, ...) to be called by the chosen framework (template) to start parsing input. The default value is **__main**.

-string-varname string
The name of the variable used for the table of strings used by the generated parser, i.e. error messages, symbol names, etc. The default value is **p_string**.

-prelude string
A snippet of code to be inserted at the head of each generated parsing function. The default value is the empty string.

-indent integer
The number of characters to indent each line of the generated code by. The default value is **0**.

-comments boolean
A flag controlling the generation of code comments containing the original parsing expression a parsing function is for. The default value is **on**.

While the high parameterizability of this converter, as shown by the multitude of options it supports, is an advantage to the advanced user, allowing her to customize the output of the converter as needed, a novice user will likely not see the forest for the trees.

To help these latter users an adjunct package is provided, containing a canned configuration which will generate immediately useful full parsers. It is

pt::cparam::configuration::critcl
Generated parsers are embedded into a **Critcl**-based framework.

C/PARAM code representation of parsing expression Grammars

The **c** format is executable code, a parser for the grammar. The parser implementation is written in C and can be tweaked to the users' needs through a multitude of options.

The **critcl** format, for example, is implemented as a canned configuration of these options on top of the generator for **c**.

The bulk of such a framework has to be specified through the option **-template**. The additional options

- **-fun-qualifier** string
- **-main** string
- **-namespace** string
- **-prelude** string
- **-self-command** string
- **-state-decl** string
- **-state-ref** string
- **-string-varname** string

provide code snippets which help to glue framework and generated code together. Their placeholders are in the *generated* code. Further the options

- **-indent** integer
- **-comments** boolean

allow for the customization of the code indent (default none), and whether to generate comments showing the parsing expressions a function is for (default on).

Example

We are forgoing an example of this representation, with apologies. It would be way to large for this document.

Peg serialization format

Here we specify the format used by the Parser Tools to serialize Parsing Expression Grammars as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a PEG may have more than one regular serialization only exactly one of them will be *canonical*.

regular serialization

1. The serialization of any PEG is a nested Tcl dictionary.
1. This dictionary holds a single key, **pt::grammar::peg**, and its value. This value holds the contents of the grammar.
2. The contents of the grammar are a Tcl dictionary holding the set of nonterminal symbols and the starting expression. The relevant keys and their values are

   rules

   The value is a Tcl dictionary whose keys are the names of the nonterminal symbols known to the grammar.

   1. Each nonterminal symbol may occur only once.
   2. The empty string is not a legal nonterminal symbol.
   3. The value for each symbol is a Tcl dictionary itself. The relevant keys and their values in this dictionary are

      is

      The value is the serialization of the parsing expression describing the symbols sentennial structure, as specified in the section PE serialization format.

      mode

The value can be one of three values specifying how a parser should handle the semantic value produced by the symbol.

value
: The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal itself, which has the ASTs of the symbol's right hand side as its children.

leaf
: The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal, without any children. Any ASTs generated by the symbol's right hand side are discarded.

void
: The nonterminal has no semantic value. Any ASTs generated by the symbol's right hand side are discarded (as well).

```
**start**
: The value is the serialization of the start parsing expression of
    the grammar, as specified in the section [PE serialization
    format].
```

1. The terminal symbols of the grammar are specified implicitly as the set of all terminal symbols used in the start expression and on the RHS of the grammar rules.

canonical serialization
: The canonical serialization of a grammar has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this grammar.

1. The keys found in all the nested Tcl dictionaries are sorted in ascending dictionary order, as generated by Tcl's builtin command **lsort -increasing -dict**.
2. The string representation of the value is the canonical representation of a Tcl dictionary. I.e. it does not contain superfluous whitespace.

Example

Assuming the following PEG for simple mathematical expressions

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

then its canonical serialization (except for whitespace) is

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}              mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                mode value}
        MulOp      {is {/ {t *} {t /}}
                                mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
                                mode value}
        Sign       {is {/ {t -} {t +}}
                                mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                mode value}
    }
    start {n Expression}
}
```

PE serialization format

Here we specify the format used by the Parser Tools to serialize Parsing Expressions as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a parsing expression may have more than one regular serialization only exactly one of them will be *canonical*.

Regular serialization

see below ...

Atomic Parsing Expressions

1. The string **epsilon** is an atomic parsing expression. It matches the empty string.
1. The string **dot** is an atomic parsing expression. It matches any character.
2. The string **alnum** is an atomic parsing expression. It matches any Unicode alphabet or digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
3. The string **alpha** is an atomic parsing expression. It matches any Unicode alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
4. The string **ascii** is an atomic parsing expression. It matches any Unicode character below U0080. This is a custom extension of PEs based on Tcl's builtin command **string is**.
5. The string **control** is an atomic parsing expression. It matches any Unicode control character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
6. The string **digit** is an atomic parsing expression. It matches any Unicode digit character. Note that this includes characters outside of the [0..9] range. This is a custom extension of PEs based on Tcl's builtin command **string is**.

7. The string **graph** is an atomic parsing expression. It matches any Unicode printing character, except for space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
8. The string **lower** is an atomic parsing expression. It matches any Unicode lower-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
9. The string **print** is an atomic parsing expression. It matches any Unicode printing character, including space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
10. The string **punct** is an atomic parsing expression. It matches any Unicode punctuation character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
11. The string **space** is an atomic parsing expression. It matches any Unicode space character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
12. The string **upper** is an atomic parsing expression. It matches any Unicode upper-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
13. The string **wordchar** is an atomic parsing expression. It matches any Unicode word character. This is any alphanumeric character (see alnum), and any connector punctuation characters (e.g. underscore). This is a custom extension of PEs based on Tcl's builtin command **string is**.
14. The string **xdigit** is an atomic parsing expression. It matches any hexadecimal digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
15. The string **ddigit** is an atomic parsing expression. It matches any decimal digit character. This is a custom extension of PEs based on Tcl's builtin command **regexp**.
16. The expression [list t **x**] is an atomic parsing expression. It matches the terminal string **x**.
17. The expression [list n **A**] is an atomic parsing expression. It matches the nonterminal **A**.

Combined Parsing Expressions

1. For parsing expressions **e1**, **e2**, ... the result of [list / **e1 e2** ...] is a parsing expression as well. This is the *ordered choice*, aka *prioritized choice*.
1. For parsing expressions **e1**, **e2**, ... the result of [list x **e1 e2** ...] is a parsing expression as well. This is the *sequence*.
2. For a parsing expression **e** the result of [list * **e**] is a parsing expression as well. This is the *kleene closure*, describing zero or more repetitions.
3. For a parsing expression **e** the result of [list + **e**] is a parsing expression as well. This is the *positive kleene closure*, describing one or more repetitions.
4. For a parsing expression **e** the result of [list & **e**] is a parsing expression as well. This is the *and lookahead predicate*.
5. For a parsing expression **e** the result of [list! **e**] is a parsing expression as well. This is the *not lookahead predicate*.
6. For a parsing expression **e** the result of [list ? **e**] is a parsing expression as well. This is the *optional input*.

Canonical serialization

The canonical serialization of a parsing expression has the format as specified in the previous

item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this parsing expression.

1. The string representation of the value is the canonical representation of a pure Tcl list. I.e. it does not contain superfluous whitespace.
2. Terminals are *not* encoded as ranges (where start and end of the range are identical).

Example

Assuming the parsing expression shown on the right-hand side of the rule

```
    Expression <- Term (AddOp Term)*
```

then its canonical serialization (except for whitespace) is

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pt* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Parsing and Grammars

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_peg_to_json

Keywords: pt_peg_to_json, EBNF, JSON, LL(k), PEG, TDPL, context-free languages, conversion, expression, format conversion, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, serialization, state, top-down parsing languages, transducer

Name

pt::peg::to::json - PEG Conversion. Write JSON format

Synopsis

package require Tcl 8.5 9
package require pt::peg::to::json ?**1.1**?
package require pt::peg
package require json::write
pt::peg::to::json reset
pt::peg::to::json configure

pt::peg::to::json configure *option*
pt::peg::to::json configure *option value...*
pt::peg::to::json convert *serial*

Description

Are you lost ? Do you have trouble understanding this document ? In that case please read the overview provided by the *Introduction to Parser Tools*. This document is the entrypoint to the whole system the current package is a part of.

This package implements the converter from parsing expression grammars to JSON markup.

It resides in the Export section of the Core Layer of Parser Tools, and can be used either directly with the other packages of this layer, or indirectly through the export manager provided by **pt::peg::export**. The latter is intented for use in untrusted environments and done through the corresponding export plugin **pt::peg::export::json** sitting between converter and export manager.

IMAGE: arch_core_eplugins

API

The API provided by this package satisfies the specification of the Converter API found in the *Parser Tools Export API* specification.

pt::peg::to::json reset
: This command resets the configuration of the package to its default settings.

pt::peg::to::json configure
: This command returns a dictionary containing the current configuration of the package.

pt::peg::to::json configure *option*
: This command returns the current value of the specified configuration *option* of the package. For the set of legal options, please read the section Options.

pt::peg::to::json configure *option value...*
: This command sets the given configuration *option*s of the package, to the specified *value*s. For the set of legal options, please read the section Options.

pt::peg::to::json convert *serial*
: This command takes the canonical serialization of a parsing expression grammar, as specified in section Peg serialization format, and contained in *serial*, and generates JSON markup encoding the grammar, per the current package configuration. The created string is then returned as the result of the command.

Options

The converter to the JSON grammar exchange format recognizes the following configuration variables and changes its behaviour as they specify.

-file string
: The value of this option is the name of the file or other entity from which the grammar came, for which the command is run. The default value is **unknown**.

-name string
: The value of this option is the name of the grammar we are processing. The default value is **a_pe_grammar**.

-user string
The value of this option is the name of the user for which the command is run. The default value is **unknown**.

-indented boolean
If this option is set the system will break the generated JSON across lines and indent it according to its inner structure, with each key of a dictionary on a separate line.

If the option is not set (the default), the whole JSON object will be written on a single line, with minimum spacing between all elements.

-aligned boolean
If this option is set the system will ensure that the values for the keys in a dictionary are vertically aligned with each other, for a nice table effect. To make this work this also implies that **-indented** is set.

If the option is not set (the default), the output is formatted as per the value of **indented**, without trying to align the values for dictionary keys.

JSON grammar exchange format

The **json** format for parsing expression grammars was written as a data exchange format not bound to Tcl. It was defined to allow the exchange of grammars with PackRat/PEG based parser generators for other languages.

It is formally specified by the rules below:

1. The JSON of any PEG is a JSON object.
2. This object holds a single key, **pt::grammar::peg**, and its value. This value holds the contents of the grammar.
3. The contents of the grammar are a JSON object holding the set of nonterminal symbols and the starting expression. The relevant keys and their values are

   rules
   The value is a JSON object whose keys are the names of the nonterminal symbols known to the grammar.
   1. Each nonterminal symbol may occur only once.
   2. The empty string is not a legal nonterminal symbol.
   3. The value for each symbol is a JSON object itself. The relevant keys and their values in this dictionary are

      is
      The value is a JSON string holding the Tcl serialization of the parsing expression describing the symbols sentennial structure, as specified in the section PE serialization format.

      mode
      The value is a JSON holding holding one of three values specifying how a parser should handle the semantic value produced by the symbol.

      value
      The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal itself, which has the ASTs of the symbol's right hand side as its children.

leaf
: The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal, without any children. Any ASTs generated by the symbol's right hand side are discarded.

void
: The nonterminal has no semantic value. Any ASTs generated by the symbol's right hand side are discarded (as well).

```
**start**
: The value is a JSON string holding the Tcl serialization of the
    start parsing expression of the grammar, as specified in the
    section [PE serialization format].
```

1. The terminal symbols of the grammar are specified implicitly as the set of all terminal symbols used in the start expression and on the RHS of the grammar rules.

As an aside to the advanced reader, this is pretty much the same as the Tcl serialization of PE grammars, as specified in section Peg serialization format, except that the Tcl dictionaries and lists of that format are mapped to JSON objects and arrays. Only the parsing expressions themselves are not translated further, but kept as JSON strings containing a nested Tcl list, and there is no concept of canonicity for the JSON either.

Example

Assuming the following PEG for simple mathematical expressions

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

a JSON serialization for it is

```
{
    "pt::grammar::peg" : {
        "rules" : {
            "AddOp"     : {
                "is"   : "\/ {t -} {t +}",
                "mode" : "value"
            },
            "Digit"     : {
                "is"   : "\/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6}
    {t 7} {t 8} {t 9}",
```

```
                "mode" : "value"
            },
            "Expression" : {
                "is"   : "\/ {x {t (} {n Expression} {t )}} {x {n
    Factor} {* {x {n MulOp} {n Factor}}}}",
                "mode" : "value"
            },
            "Factor"    : {
                "is"   : "x {n Term} {* {x {n AddOp} {n Term}}}",
                "mode" : "value"
            },
            "MulOp"     : {
                "is"   : "\/ {t *} {t \/}",
                "mode" : "value"
            },
            "Number"    : {
                "is"   : "x {? {n Sign}} {+ {n Digit}}",
                "mode" : "value"
            },
            "Sign"      : {
                "is"   : "\/ {t -} {t +}",
                "mode" : "value"
            },
            "Term"      : {
                "is"   : "n Number",
                "mode" : "value"
            }
        },
        "start" : "n Expression"
    }
}
```

and a Tcl serialization of the same is

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                  mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}                mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                  mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                  mode value}
        MulOp      {is {/ {t *} {t /}}
                                  mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
```

```
                                    mode value}
        Sign       {is {/ {t -} {t +}}
                                    mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                    mode value}
    }
    start {n Expression}
}
```

The similarity of the latter to the JSON should be quite obvious.

Peg serialization format

Here we specify the format used by the Parser Tools to serialize Parsing Expression Grammars as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a PEG may have more than one regular serialization only exactly one of them will be *canonical*.

regular serialization

1. The serialization of any PEG is a nested Tcl dictionary.
1. This dictionary holds a single key, **pt::grammar::peg**, and its value. This value holds the contents of the grammar.
2. The contents of the grammar are a Tcl dictionary holding the set of nonterminal symbols and the starting expression. The relevant keys and their values are

   rules

   The value is a Tcl dictionary whose keys are the names of the nonterminal symbols known to the grammar.

   1. Each nonterminal symbol may occur only once.
   2. The empty string is not a legal nonterminal symbol.
   3. The value for each symbol is a Tcl dictionary itself. The relevant keys and their values in this dictionary are

      is

      The value is the serialization of the parsing expression describing the symbols sentennial structure, as specified in the section PE serialization format.

      mode

      The value can be one of three values specifying how a parser should handle the semantic value produced by the symbol.

      value

      The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal itself, which has the ASTs of the symbol's right hand side as its children.

      leaf

      The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal, without any children. Any ASTs generated by the symbol's right hand side are discarded.

      void

      The nonterminal has no semantic value. Any ASTs generated by the symbol's right hand side are discarded (as well).

```
**start**
: The value is the serialization of the start parsing expression of
    the grammar, as specified in the section [PE serialization
    format].
```

1. The terminal symbols of the grammar are specified implicitly as the set of all terminal symbols used in the start expression and on the RHS of the grammar rules.

canonical serialization

The canonical serialization of a grammar has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this grammar.

1. The keys found in all the nested Tcl dictionaries are sorted in ascending dictionary order, as generated by Tcl's builtin command **lsort -increasing -dict**.
2. The string representation of the value is the canonical representation of a Tcl dictionary. I.e. it does not contain superfluous whitespace.

Example

Assuming the following PEG for simple mathematical expressions

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

then its canonical serialization (except for whitespace) is

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                  mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}                mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                  mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                  mode value}
        MulOp      {is {/ {t *} {t /}}
                                  mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
                                  mode value}
        Sign       {is {/ {t -} {t +}}
```

```
                                    mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                    mode value}
    }
    start {n Expression}
}
```

PE serialization format

Here we specify the format used by the Parser Tools to serialize Parsing Expressions as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a parsing expression may have more than one regular serialization only exactly one of them will be *canonical*.

Regular serialization

see below ...

Atomic Parsing Expressions

1. The string **epsilon** is an atomic parsing expression. It matches the empty string.
1. The string **dot** is an atomic parsing expression. It matches any character.
2. The string **alnum** is an atomic parsing expression. It matches any Unicode alphabet or digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
3. The string **alpha** is an atomic parsing expression. It matches any Unicode alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
4. The string **ascii** is an atomic parsing expression. It matches any Unicode character below U0080. This is a custom extension of PEs based on Tcl's builtin command **string is**.
5. The string **control** is an atomic parsing expression. It matches any Unicode control character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
6. The string **digit** is an atomic parsing expression. It matches any Unicode digit character. Note that this includes characters outside of the [0..9] range. This is a custom extension of PEs based on Tcl's builtin command **string is**.
7. The string **graph** is an atomic parsing expression. It matches any Unicode printing character, except for space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
8. The string **lower** is an atomic parsing expression. It matches any Unicode lower-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
9. The string **print** is an atomic parsing expression. It matches any Unicode printing character, including space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
10. The string **punct** is an atomic parsing expression. It matches any Unicode punctuation character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
11. The string **space** is an atomic parsing expression. It matches any Unicode space character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
12. The string **upper** is an atomic parsing expression. It matches any Unicode upper-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command

string is.

13. The string **wordchar** is an atomic parsing expression. It matches any Unicode word character. This is any alphanumeric character (see alnum), and any connector punctuation characters (e.g. underscore). This is a custom extension of PEs based on Tcl's builtin command **string is**.
14. The string **xdigit** is an atomic parsing expression. It matches any hexadecimal digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
15. The string **ddigit** is an atomic parsing expression. It matches any decimal digit character. This is a custom extension of PEs based on Tcl's builtin command **regexp**.
16. The expression [list t **x**] is an atomic parsing expression. It matches the terminal string **x**.
17. The expression [list n **A**] is an atomic parsing expression. It matches the nonterminal **A**.

Combined Parsing Expressions

1. For parsing expressions **e1**, **e2**, ... the result of [list / **e1 e2** ...] is a parsing expression as well. This is the *ordered choice*, aka *prioritized choice*.
1. For parsing expressions **e1**, **e2**, ... the result of [list x **e1 e2** ...] is a parsing expression as well. This is the *sequence*.
2. For a parsing expression **e** the result of [list * **e**] is a parsing expression as well. This is the *kleene closure*, describing zero or more repetitions.
3. For a parsing expression **e** the result of [list + **e**] is a parsing expression as well. This is the *positive kleene closure*, describing one or more repetitions.
4. For a parsing expression **e** the result of [list & **e**] is a parsing expression as well. This is the *and lookahead predicate*.
5. For a parsing expression **e** the result of [list! **e**] is a parsing expression as well. This is the *not lookahead predicate*.
6. For a parsing expression **e** the result of [list ? **e**] is a parsing expression as well. This is the *optional input*.

Canonical serialization

The canonical serialization of a parsing expression has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this parsing expression.

1. The string representation of the value is the canonical representation of a pure Tcl list. I.e. it does not contain superfluous whitespace.
2. Terminals are *not* encoded as ranges (where start and end of the range are identical).

Example

Assuming the parsing expression shown on the right-hand side of the rule

```
    Expression <- Term (AddOp Term)*
```

then its canonical serialization (except for whitespace) is

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you

want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pt* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Parsing and Grammars

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_peg_to_param

Keywords: pt_peg_to_param, EBNF, LL(k), PARAM, PEG, TDPL, context-free languages, conversion , expression, format conversion, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, serialization, state, top-down parsing languages, transducer

Name

pt::peg::to::param - PEG Conversion. Write PARAM format

Synopsis

package require Tcl 8.5 9
package require pt::peg::to::param ?**1.1**?
package require pt::peg
package require pt::pe
pt::peg::to::param reset
pt::peg::to::param configure
pt::peg::to::param configure *option*
pt::peg::to::param configure *option value...*
pt::peg::to::param convert *serial*

Description

Are you lost ? Do you have trouble understanding this document ? In that case please read the overview provided by the *Introduction to Parser Tools*. This document is the entrypoint to the whole system the current package is a part of.

This package implements the converter from parsing expression grammars to PARAM markup.

It resides in the Export section of the Core Layer of Parser Tools, and can be used either directly with the other packages of this layer, or indirectly through the export manager provided by **pt::peg::export**. The latter is intented for use in untrusted environments and done through the corresponding export plugin **pt::peg::export::param** sitting between converter and export manager.

IMAGE: arch_core_eplugins

API

The API provided by this package satisfies the specification of the Converter API found in the *Parser Tools Export API* specification.

pt::peg::to::param reset
: This command resets the configuration of the package to its default settings.

pt::peg::to::param configure
: This command returns a dictionary containing the current configuration of the package.

pt::peg::to::param configure *option*
: This command returns the current value of the specified configuration *option* of the package. For the set of legal options, please read the section Options.

pt::peg::to::param configure *option value...*
: This command sets the given configuration *option*s of the package, to the specified *value*s. For the set of legal options, please read the section Options.

pt::peg::to::param convert *serial*
: This command takes the canonical serialization of a parsing expression grammar, as specified in section Peg serialization format, and contained in *serial*, and generates PARAM markup encoding the grammar, per the current package configuration. The created string is then returned as the result of the command.

Options

The converter to PARAM markup recognizes the following configuration variables and changes its behaviour as they specify.

-template string
: The value of this configuration variable is a string into which to put the generated text and the other configuration settings. The various locations for user-data are expected to be specified with the placeholders listed below. The default value is "**@code@**".

  @user@
  : To be replaced with the value of the configuration variable **-user**.

  @format@
  : To be replaced with the the constant **PARAM**.

  @file@
  : To be replaced with the value of the configuration variable **-file**.

  @name@
  : To be replaced with the value of the configuration variable **-name**.

  @code@
  : To be replaced with the generated text.

-name string
: The value of this configuration variable is the name of the grammar for which the conversion is run. The default value is **a_pe_grammar**.

-user string
: The value of this configuration variable is the name of the user for which the conversion is run. The default value is **unknown**.

-file string

The value of this configuration variable is the name of the file or other entity from which the grammar came, for which the conversion is run. The default value is **unknown**.

Param code representation of parsing expression grammars

The PARAM code representation of parsing expression grammars is assembler-like text using the instructions of the virtual machine documented in the *PackRat Machine Specification*, plus a few more for control flow (jump ok, jump fail, call symbol, return).

It is not really useful, except possibly as a tool demonstrating how a grammar is compiled in general, without getting distracted by the incidentials of a framework, i.e. like the supporting C and Tcl code generated by the other PARAM-derived formats.

It has no direct formal specification beyond what was said above.

Example

Assuming the following PEG for simple mathematical expressions

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

one possible PARAM serialization for it is

```
# -*- text -*-
# Parsing Expression Grammar 'TEMPLATE'.
# Generated for unknown, from file 'TEST'

#
# Grammar Start Expression
#

<<MAIN>>:
         call              sym_Expression
         halt

#
# value Symbol 'AddOp'
#

sym_AddOp:
# /
#     '-'
```

```
#     '+'

         symbol_restore    AddOp
  found! jump              found_7
         loc_push

         call              choice_5

   fail! value_clear
     ok! value_leaf        AddOp
         symbol_save       AddOp
         error_nonterminal AddOp
         loc_pop_discard

found_7:
     ok! ast_value_push
         return

choice_5:
# /
#     '-'
#     '+'

         error_clear

         loc_push
         error_push

         input_next        "t -"
     ok! test_char         "-"

         error_pop_merge
     ok! jump              oknoast_4

         loc_pop_rewind
         loc_push
         error_push

         input_next        "t +"
     ok! test_char         "+"

         error_pop_merge
     ok! jump              oknoast_4

         loc_pop_rewind
         status_fail
```

```
         return

oknoast_4:
         loc_pop_discard
         return
#
# value Symbol 'Digit'
#

sym_Digit:
# /
#     '0'
#     '1'
#     '2'
#     '3'
#     '4'
#     '5'
#     '6'
#     '7'
#     '8'
#     '9'

         symbol_restore    Digit
  found! jump              found_22
         loc_push

         call              choice_20

   fail! value_clear
     ok! value_leaf        Digit
         symbol_save       Digit
         error_nonterminal Digit
         loc_pop_discard

found_22:
     ok! ast_value_push
         return

choice_20:
# /
#     '0'
#     '1'
#     '2'
#     '3'
#     '4'
#     '5'
```

```
#     '6'
#     '7'
#     '8'
#     '9'

        error_clear

        loc_push
        error_push

        input_next        "t 0"
    ok! test_char         "0"

        error_pop_merge
    ok! jump              oknoast_19

        loc_pop_rewind
        loc_push
        error_push

        input_next        "t 1"
    ok! test_char         "1"

        error_pop_merge
    ok! jump              oknoast_19

        loc_pop_rewind
        loc_push
        error_push

        input_next        "t 2"
    ok! test_char         "2"

        error_pop_merge
    ok! jump              oknoast_19

        loc_pop_rewind
        loc_push
        error_push

        input_next        "t 3"
    ok! test_char         "3"

        error_pop_merge
    ok! jump              oknoast_19
```

```
        loc_pop_rewind
        loc_push
        error_push

        input_next        "t 4"
    ok! test_char         "4"

        error_pop_merge
    ok! jump              oknoast_19

        loc_pop_rewind
        loc_push
        error_push

        input_next        "t 5"
    ok! test_char         "5"

        error_pop_merge
    ok! jump              oknoast_19

        loc_pop_rewind
        loc_push
        error_push

        input_next        "t 6"
    ok! test_char         "6"

        error_pop_merge
    ok! jump              oknoast_19

        loc_pop_rewind
        loc_push
        error_push

        input_next        "t 7"
    ok! test_char         "7"

        error_pop_merge
    ok! jump              oknoast_19

        loc_pop_rewind
        loc_push
        error_push

        input_next        "t 8"
    ok! test_char         "8"
```

```
        error_pop_merge
    ok! jump              oknoast_19

        loc_pop_rewind
        loc_push
        error_push

        input_next        "t 9"
    ok! test_char         "9"

        error_pop_merge
    ok! jump              oknoast_19

        loc_pop_rewind
        status_fail
        return

oknoast_19:
        loc_pop_discard
        return
#
# value Symbol 'Expression'
#

sym_Expression:
# /
#     x
#         '\('
#         (Expression)
#         '\)'
#     x
#         (Factor)
#         *
#             x
#                 (MulOp)
#                 (Factor)

        symbol_restore    Expression
  found! jump             found_46
        loc_push
        ast_push

        call              choice_44

   fail! value_clear
```

```
     ok! value_reduce      Expression
         symbol_save       Expression
         error_nonterminal Expression
         ast_pop_rewind
         loc_pop_discard

found_46:
     ok! ast_value_push
         return

choice_44:
# /
#     x
#         '\('
#         (Expression)
#         '\)'
#     x
#         (Factor)
#         *
#             x
#                 (MulOp)
#                 (Factor)

         error_clear

         ast_push
         loc_push
         error_push

         call              sequence_27

         error_pop_merge
     ok! jump              ok_43

         ast_pop_rewind
         loc_pop_rewind
         ast_push
         loc_push
         error_push

         call              sequence_40

         error_pop_merge
     ok! jump              ok_43

         ast_pop_rewind
```

```
         loc_pop_rewind
         status_fail
         return

ok_43:
         ast_pop_discard
         loc_pop_discard
         return

sequence_27:
# x
#     '\('
#     (Expression)
#     '\)'

         loc_push
         error_clear

         error_push

         input_next        "t ("
     ok! test_char         "("

         error_pop_merge
   fail! jump              failednoast_29
         ast_push
         error_push

         call              sym_Expression

         error_pop_merge
   fail! jump              failed_28
         error_push

         input_next        "t )"
     ok! test_char         ")"

         error_pop_merge
   fail! jump              failed_28

         ast_pop_discard
         loc_pop_discard
         return

failed_28:
         ast_pop_rewind
```

```
failednoast_29:
         loc_pop_rewind
         return

sequence_40:
# x
#     (Factor)
#     *
#         x
#             (MulOp)
#             (Factor)

         ast_push
         loc_push
         error_clear

         error_push

         call              sym_Factor

         error_pop_merge
   fail! jump              failed_41
         error_push

         call              kleene_37

         error_pop_merge
   fail! jump              failed_41

         ast_pop_discard
         loc_pop_discard
         return

failed_41:
         ast_pop_rewind
         loc_pop_rewind
         return

kleene_37:
# *
#     x
#         (MulOp)
#         (Factor)

         loc_push
```

```
         error_push

         call              sequence_34

         error_pop_merge
   fail! jump              failed_38
         loc_pop_discard
         jump              kleene_37

failed_38:
         loc_pop_rewind
         status_ok
         return

sequence_34:
# x
#     (MulOp)
#     (Factor)

         ast_push
         loc_push
         error_clear

         error_push

         call              sym_MulOp

         error_pop_merge
   fail! jump              failed_35
         error_push

         call              sym_Factor

         error_pop_merge
   fail! jump              failed_35

         ast_pop_discard
         loc_pop_discard
         return

failed_35:
         ast_pop_rewind
         loc_pop_rewind
         return
#
# value Symbol 'Factor'
```

```
#

sym_Factor:
# x
#     (Term)
#     *
#         x
#             (AddOp)
#             (Term)

         symbol_restore    Factor
  found! jump              found_60
         loc_push
         ast_push

         call              sequence_57

   fail! value_clear
     ok! value_reduce      Factor
         symbol_save       Factor
         error_nonterminal Factor
         ast_pop_rewind
         loc_pop_discard

found_60:
     ok! ast_value_push
         return

sequence_57:
# x
#     (Term)
#     *
#         x
#             (AddOp)
#             (Term)

         ast_push
         loc_push
         error_clear

         error_push

         call              sym_Term

         error_pop_merge
   fail! jump              failed_58
```

```
         error_push

         call              kleene_54

         error_pop_merge
   fail! jump              failed_58

         ast_pop_discard
         loc_pop_discard
         return

failed_58:
         ast_pop_rewind
         loc_pop_rewind
         return

kleene_54:
# *
#     x
#          (AddOp)
#          (Term)

         loc_push
         error_push

         call              sequence_51

         error_pop_merge
   fail! jump              failed_55
         loc_pop_discard
         jump              kleene_54

failed_55:
         loc_pop_rewind
         status_ok
         return

sequence_51:
# x
#     (AddOp)
#     (Term)

         ast_push
         loc_push
         error_clear
```

```
         error_push

         call              sym_AddOp

         error_pop_merge
   fail! jump              failed_52
         error_push

         call              sym_Term

         error_pop_merge
   fail! jump              failed_52

         ast_pop_discard
         loc_pop_discard
         return

failed_52:
         ast_pop_rewind
         loc_pop_rewind
         return
#
# value Symbol 'MulOp'
#

sym_MulOp:
# /
#     '*'
#     '/'

         symbol_restore    MulOp
  found! jump              found_67
         loc_push

         call              choice_65

   fail! value_clear
     ok! value_leaf        MulOp
         symbol_save       MulOp
         error_nonterminal MulOp
         loc_pop_discard

found_67:
     ok! ast_value_push
         return
```

```
choice_65:
# /
#     '*'
#     '/'

         error_clear

         loc_push
         error_push

         input_next        "t *"
     ok! test_char         "*"

         error_pop_merge
     ok! jump              oknoast_64

         loc_pop_rewind
         loc_push
         error_push

         input_next        "t /"
     ok! test_char         "/"

         error_pop_merge
     ok! jump              oknoast_64

         loc_pop_rewind
         status_fail
         return

oknoast_64:
         loc_pop_discard
         return
#
# value Symbol 'Number'
#

sym_Number:
# x
#     ?
#         (Sign)
#     +
#         (Digit)

         symbol_restore    Number
  found! jump              found_80
```

```
         loc_push
         ast_push

         call              sequence_77

   fail! value_clear
     ok! value_reduce      Number
         symbol_save       Number
         error_nonterminal Number
         ast_pop_rewind
         loc_pop_discard

found_80:
     ok! ast_value_push
         return

sequence_77:
# x
#     ?
#         (Sign)
#     +
#         (Digit)

         ast_push
         loc_push
         error_clear

         error_push

         call              optional_70

         error_pop_merge
   fail! jump              failed_78
         error_push

         call              poskleene_73

         error_pop_merge
   fail! jump              failed_78

         ast_pop_discard
         loc_pop_discard
         return

failed_78:
         ast_pop_rewind
```

```
         loc_pop_rewind
         return

optional_70:
# ?
#     (Sign)

         loc_push
         error_push

         call              sym_Sign

         error_pop_merge
   fail! loc_pop_rewind
     ok! loc_pop_discard
         status_ok
         return

poskleene_73:
# +
#     (Digit)

         loc_push

         call              sym_Digit

   fail! jump              failed_74

loop_75:
         loc_pop_discard
         loc_push
         error_push

         call              sym_Digit

         error_pop_merge
     ok! jump              loop_75
         status_ok

failed_74:
         loc_pop_rewind
         return
#
# value Symbol 'Sign'
#
```

```
sym_Sign:
# /
#     '-'
#     '+'

         symbol_restore    Sign
  found! jump              found_86
         loc_push

         call              choice_5

   fail! value_clear
     ok! value_leaf        Sign
         symbol_save       Sign
         error_nonterminal Sign
         loc_pop_discard

found_86:
     ok! ast_value_push
         return
#
# value Symbol 'Term'
#

sym_Term:
# (Number)

         symbol_restore    Term
  found! jump              found_89
         loc_push
         ast_push

         call              sym_Number

   fail! value_clear
     ok! value_reduce      Term
         symbol_save       Term
         error_nonterminal Term
         ast_pop_rewind
         loc_pop_discard

found_89:
     ok! ast_value_push
         return

#
```

```
#
```

Peg serialization format

Here we specify the format used by the Parser Tools to serialize Parsing Expression Grammars as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a PEG may have more than one regular serialization only exactly one of them will be *canonical*.

regular serialization

1. The serialization of any PEG is a nested Tcl dictionary.
1. This dictionary holds a single key, **pt::grammar::peg**, and its value. This value holds the contents of the grammar.
2. The contents of the grammar are a Tcl dictionary holding the set of nonterminal symbols and the starting expression. The relevant keys and their values are

   rules

   The value is a Tcl dictionary whose keys are the names of the nonterminal symbols known to the grammar.

   1. Each nonterminal symbol may occur only once.
   2. The empty string is not a legal nonterminal symbol.
   3. The value for each symbol is a Tcl dictionary itself. The relevant keys and their values in this dictionary are

      is

      The value is the serialization of the parsing expression describing the symbols sentennial structure, as specified in the section PE serialization format.

      mode

      The value can be one of three values specifying how a parser should handle the semantic value produced by the symbol.

      value

      The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal itself, which has the ASTs of the symbol's right hand side as its children.

      leaf

      The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal, without any children. Any ASTs generated by the symbol's right hand side are discarded.

      void

      The nonterminal has no semantic value. Any ASTs generated by the symbol's right hand side are discarded (as well).

```
**start**
: The value is the serialization of the start parsing expression of
    the grammar, as specified in the section [PE serialization
    format].
```

1. The terminal symbols of the grammar are specified implicitly as the set of all terminal

symbols used in the start expression and on the RHS of the grammar rules.

canonical serialization

The canonical serialization of a grammar has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this grammar.

1. The keys found in all the nested Tcl dictionaries are sorted in ascending dictionary order, as generated by Tcl's builtin command **lsort -increasing -dict**.
2. The string representation of the value is the canonical representation of a Tcl dictionary. I.e. it does not contain superfluous whitespace.

Example

Assuming the following PEG for simple mathematical expressions

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

then its canonical serialization (except for whitespace) is

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                  mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}                mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                  mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                  mode value}
        MulOp      {is {/ {t *} {t /}}
                                  mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
                                  mode value}
        Sign       {is {/ {t -} {t +}}
                                  mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                  mode value}
    }
    start {n Expression}
}
```

PE serialization format

Here we specify the format used by the Parser Tools to serialize Parsing Expressions as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a parsing expression may have more than one regular serialization only exactly one of them will be *canonical*.

Regular serialization

see below ...

Atomic Parsing Expressions

1. The string **epsilon** is an atomic parsing expression. It matches the empty string.
1. The string **dot** is an atomic parsing expression. It matches any character.
2. The string **alnum** is an atomic parsing expression. It matches any Unicode alphabet or digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
3. The string **alpha** is an atomic parsing expression. It matches any Unicode alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
4. The string **ascii** is an atomic parsing expression. It matches any Unicode character below U0080. This is a custom extension of PEs based on Tcl's builtin command **string is**.
5. The string **control** is an atomic parsing expression. It matches any Unicode control character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
6. The string **digit** is an atomic parsing expression. It matches any Unicode digit character. Note that this includes characters outside of the [0..9] range. This is a custom extension of PEs based on Tcl's builtin command **string is**.
7. The string **graph** is an atomic parsing expression. It matches any Unicode printing character, except for space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
8. The string **lower** is an atomic parsing expression. It matches any Unicode lower-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
9. The string **print** is an atomic parsing expression. It matches any Unicode printing character, including space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
10. The string **punct** is an atomic parsing expression. It matches any Unicode punctuation character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
11. The string **space** is an atomic parsing expression. It matches any Unicode space character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
12. The string **upper** is an atomic parsing expression. It matches any Unicode upper-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
13. The string **wordchar** is an atomic parsing expression. It matches any Unicode word character. This is any alphanumeric character (see alnum), and any connector punctuation characters (e.g. underscore). This is a custom extension of PEs based on Tcl's builtin command **string is**.
14. The string **xdigit** is an atomic parsing expression. It matches any hexadecimal digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.

15. The string **ddigit** is an atomic parsing expression. It matches any decimal digit character. This is a custom extension of PEs based on Tcl's builtin command **regexp**.
16. The expression [list t **x**] is an atomic parsing expression. It matches the terminal string **x**.
17. The expression [list n **A**] is an atomic parsing expression. It matches the nonterminal **A**.

Combined Parsing Expressions

1. For parsing expressions **e1**, **e2**, ... the result of [list / **e1 e2** ...] is a parsing expression as well. This is the *ordered choice*, aka *prioritized choice*.
1. For parsing expressions **e1**, **e2**, ... the result of [list x **e1 e2** ...] is a parsing expression as well. This is the *sequence*.
2. For a parsing expression **e** the result of [list * **e**] is a parsing expression as well. This is the *kleene closure*, describing zero or more repetitions.
3. For a parsing expression **e** the result of [list + **e**] is a parsing expression as well. This is the *positive kleene closure*, describing one or more repetitions.
4. For a parsing expression **e** the result of [list & **e**] is a parsing expression as well. This is the *and lookahead predicate*.
5. For a parsing expression **e** the result of [list! **e**] is a parsing expression as well. This is the *not lookahead predicate*.
6. For a parsing expression **e** the result of [list ? **e**] is a parsing expression as well. This is the *optional input*.

Canonical serialization

The canonical serialization of a parsing expression has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this parsing expression.

1. The string representation of the value is the canonical representation of a pure Tcl list. I.e. it does not contain superfluous whitespace.
2. Terminals are *not* encoded as ranges (where start and end of the range are identical).

Example

Assuming the parsing expression shown on the right-hand side of the rule

```
    Expression <- Term (AddOp Term)*
```

then its canonical serialization (except for whitespace) is

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pt* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Parsing and Grammars

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_peg_to_peg
Keywords: pt_peg_to_peg, EBNF, LL(k), PEG, TDPL, context-free languages, conversion, expression, format conversion, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, serialization, state, top-down parsing languages, transducer

Name
pt::peg::to::peg - PEG Conversion. Write PEG format

Synopsis
package require Tcl 8.5 9
package require pt::peg::to::peg ?**1.0.3**?
package require pt::peg
package require pt::pe
package require text::write
pt::peg::to::peg reset
pt::peg::to::peg configure
pt::peg::to::peg configure *option*
pt::peg::to::peg configure *option value...*
pt::peg::to::peg convert *serial*

Description
Are you lost ? Do you have trouble understanding this document ? In that case please read the overview provided by the *Introduction to Parser Tools*. This document is the entrypoint to the whole system the current package is a part of.

This package implements the converter from parsing expression grammars to PEG markup.

It resides in the Export section of the Core Layer of Parser Tools, and can be used either directly with the other packages of this layer, or indirectly through the export manager provided by **pt::peg::export**. The latter is intented for use in untrusted environments and done through the corresponding export plugin **pt::peg::export::peg** sitting between converter and export manager.

IMAGE: arch_core_eplugins

API
The API provided by this package satisfies the specification of the Converter API found in the *Parser Tools Export API* specification.

pt::peg::to::peg reset
This command resets the configuration of the package to its default settings.

pt::peg::to::peg configure
This command returns a dictionary containing the current configuration of the package.

pt::peg::to::peg configure *option*
This command returns the current value of the specified configuration *option* of the package. For the set of legal options, please read the section Options.

pt::peg::to::peg configure *option value...*
This command sets the given configuration *option*s of the package, to the specified *value*s. For the set of legal options, please read the section Options.

pt::peg::to::peg convert *serial*
This command takes the canonical serialization of a parsing expression grammar, as specified in section Peg serialization format, and contained in *serial*, and generates PEG markup encoding the grammar, per the current package configuration. The created string is then returned as the result of the command.

Options

The converter to the PEG language recognizes the following options and changes its behaviour as they specify.

-file string
The value of this option is the name of the file or other entity from which the grammar came, for which the command is run. The default value is **unknown**.

-name string
The value of this option is the name of the grammar we are processing. The default value is **a_pe_grammar**.

-user string
The value of this option is the name of the user for which the command is run. The default value is **unknown**.

-template string
The value of this option is a string into which to put the generated text and the values of the other options. The various locations for user-data are expected to be specified with the placeholders listed below. The default value is "**@code@**".

@user@
To be replaced with the value of the option **-user**.

@format@
To be replaced with the the constant **PEG**.

@file@
To be replaced with the value of the option **-file**.

@name@
To be replaced with the value of the option **-name**.

@code@
To be replaced with the generated text.

Peg specification language

peg, a language for the specification of parsing expression grammars is meant to be human readable, and writable as well, yet strict enough to allow its processing by machine. Like any

computer language. It was defined to make writing the specification of a grammar easy, something the other formats found in the Parser Tools do not lend themselves too.

It is formally specified by the grammar shown below, written in itself. For a tutorial / introduction to the language please go and read the *PEG Language Tutorial*.

```
PEG pe_grammar_for_peg (Grammar)

    # ------------------------------------------------------------------
        # Syntactical constructs

        Grammar         <- WHITESPACE Header Definition* Final EOF ;

        Header          <- PEG Identifier StartExpr ;
        Definition      <- Attribute? Identifier IS Expression
    SEMICOLON ;
        Attribute       <- (VOID / LEAF) COLON ;
        Expression      <- Sequence (SLASH Sequence)* ;
        Sequence        <- Prefix+ ;
        Prefix          <- (AND / NOT)? Suffix ;
        Suffix          <- Primary (QUESTION / STAR / PLUS)? ;
        Primary         <- ALNUM / ALPHA / ASCII / CONTROL / DDIGIT /
    DIGIT
                        /  GRAPH / LOWER / PRINTABLE / PUNCT / SPACE /
    UPPER
                        /  WORDCHAR / XDIGIT
                        / Identifier
                        /  OPEN Expression CLOSE
                        /  Literal
                        /  Class
                        /  DOT
                        ;
        Literal         <- APOSTROPH  (!APOSTROPH  Char)* APOSTROPH
    WHITESPACE
                        /  DAPOSTROPH (!DAPOSTROPH Char)* DAPOSTROPH
    WHITESPACE ;
        Class           <- OPENB (!CLOSEB Range)* CLOSEB WHITESPACE ;
        Range           <- Char TO Char / Char ;

        StartExpr       <- OPEN Expression CLOSE ;
void:   Final           <- "END" WHITESPACE SEMICOLON WHITESPACE ;

        # ------------------------------------------------------------
    -------
        # Lexing constructs

        Identifier      <- Ident WHITESPACE ;
```

```
leaf:   Ident           <- ([_:] / <alpha>) ([_:] / <alnum>)* ;
        Char            <- CharSpecial / CharOctalFull / CharOctalPart
                        /  CharUnicode / CharUnescaped
                        ;

leaf:   CharSpecial     <- "\\" [nrt'"\[\]\\] ;
leaf:   CharOctalFull   <- "\\" [0-3][0-7][0-7] ;
leaf:   CharOctalPart   <- "\\" [0-7][0-7]? ;
leaf:   CharUnicode     <- "\\" 'u' HexDigit (HexDigit (HexDigit
    HexDigit?)?)? ;
leaf:   CharUnescaped   <- !"\\" . ;

void:   HexDigit        <- [0-9a-fA-F] ;

void:   TO              <- '-'           ;
void:   OPENB           <- "["           ;
void:   CLOSEB          <- "]"           ;
void:   APOSTROPH       <- "'"           ;
void:   DAPOSTROPH      <- '"'           ;
void:   PEG             <- "PEG" !([_:] / <alnum>) WHITESPACE ;
void:   IS              <- "<-"    WHITESPACE ;
leaf:   VOID            <- "void"  WHITESPACE ; # Implies that
    definition has no semantic value.
leaf:   LEAF            <- "leaf"  WHITESPACE ; # Implies that
    definition has no terminals.
void:   SEMICOLON       <- ";"     WHITESPACE ;
void:   COLON           <- ":"     WHITESPACE ;
void:   SLASH           <- "/"     WHITESPACE ;
leaf:   AND             <- "&"     WHITESPACE ;
leaf:   NOT             <- "!"     WHITESPACE ;
leaf:   QUESTION        <- "?"     WHITESPACE ;
leaf:   STAR            <- "*"     WHITESPACE ;
leaf:   PLUS            <- "+"     WHITESPACE ;
void:   OPEN            <- "("     WHITESPACE ;
void:   CLOSE           <- ")"     WHITESPACE ;
leaf:   DOT             <- "."     WHITESPACE ;

leaf:   ALNUM           <- "<alnum>"    WHITESPACE ;
leaf:   ALPHA           <- "<alpha>"    WHITESPACE ;
leaf:   ASCII           <- "<ascii>"    WHITESPACE ;
leaf:   CONTROL         <- "<control>"  WHITESPACE ;
leaf:   DDIGIT          <- "<ddigit>"   WHITESPACE ;
leaf:   DIGIT           <- "<digit>"    WHITESPACE ;
leaf:   GRAPH           <- "<graph>"    WHITESPACE ;
leaf:   LOWER           <- "<lower>"    WHITESPACE ;
leaf:   PRINTABLE       <- "<print>"    WHITESPACE ;
```

```
leaf:   PUNCT           <- "<punct>"    WHITESPACE ;
leaf:   SPACE           <- "<space>"    WHITESPACE ;
leaf:   UPPER           <- "<upper>"    WHITESPACE ;
leaf:   WORDCHAR        <- "<wordchar>" WHITESPACE ;
leaf:   XDIGIT          <- "<xdigit>"   WHITESPACE ;

void:   WHITESPACE      <- (" " / "\t" / EOL / COMMENT)* ;
void:   COMMENT         <- '#' (!EOL .)* EOL ;
void:   EOL             <- "\n\r" / "\n" / "\r" ;
void:   EOF             <- !. ;

        # ------------------------------------------------------------
    -------
END;
```

Example

Our example specifies the grammar for a basic 4-operation calculator.

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

Using higher-level features of the notation, i.e. the character classes (predefined and custom), this example can be rewritten as

```
PEG calculator (Expression)
    Sign       <- [-+]                                         ;
    Number     <- Sign? <ddigit>+                              ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- [*/]                      ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- [-+]                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

Peg serialization format

Here we specify the format used by the Parser Tools to serialize Parsing Expression Grammars as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a PEG may have more than one regular serialization only exactly one of them will be *canonical*.

regular serialization

1. The serialization of any PEG is a nested Tcl dictionary.
1. This dictionary holds a single key, **pt::grammar::peg**, and its value. This value holds the contents of the grammar.
2. The contents of the grammar are a Tcl dictionary holding the set of nonterminal symbols and the starting expression. The relevant keys and their values are

   rules
   The value is a Tcl dictionary whose keys are the names of the nonterminal symbols known to the grammar.
   1. Each nonterminal symbol may occur only once.
   2. The empty string is not a legal nonterminal symbol.
   3. The value for each symbol is a Tcl dictionary itself. The relevant keys and their values in this dictionary are

      is
      The value is the serialization of the parsing expression describing the symbols sentennial structure, as specified in the section PE serialization format.

      mode
      The value can be one of three values specifying how a parser should handle the semantic value produced by the symbol.

      value
      The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal itself, which has the ASTs of the symbol's right hand side as its children.

      leaf
      The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal, without any children. Any ASTs generated by the symbol's right hand side are discarded.

      void
      The nonterminal has no semantic value. Any ASTs generated by the symbol's right hand side are discarded (as well).

```
**start**
: The value is the serialization of the start parsing expression of
    the grammar, as specified in the section [PE serialization
    format].
```

1. The terminal symbols of the grammar are specified implicitly as the set of all terminal symbols used in the start expression and on the RHS of the grammar rules.

canonical serialization

The canonical serialization of a grammar has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this grammar.

1. The keys found in all the nested Tcl dictionaries are sorted in ascending dictionary order, as generated by Tcl's builtin command **lsort -increasing -dict**.

2. The string representation of the value is the canonical representation of a Tcl dictionary. I.e. it does not contain superfluous whitespace.

Example

Assuming the following PEG for simple mathematical expressions

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

then its canonical serialization (except for whitespace) is

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                  mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}               mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                  mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                  mode value}
        MulOp      {is {/ {t *} {t /}}
                                  mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
                                  mode value}
        Sign       {is {/ {t -} {t +}}
                                  mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                  mode value}
    }
    start {n Expression}
}
```

PE serialization format

Here we specify the format used by the Parser Tools to serialize Parsing Expressions as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a parsing expression may have more than one regular serialization only exactly one of them will be *canonical*.

Regular serialization

see below ...

Atomic Parsing Expressions

1. The string **epsilon** is an atomic parsing expression. It matches the empty string.
1. The string **dot** is an atomic parsing expression. It matches any character.
2. The string **alnum** is an atomic parsing expression. It matches any Unicode alphabet or digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
3. The string **alpha** is an atomic parsing expression. It matches any Unicode alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
4. The string **ascii** is an atomic parsing expression. It matches any Unicode character below U0080. This is a custom extension of PEs based on Tcl's builtin command **string is**.
5. The string **control** is an atomic parsing expression. It matches any Unicode control character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
6. The string **digit** is an atomic parsing expression. It matches any Unicode digit character. Note that this includes characters outside of the [0..9] range. This is a custom extension of PEs based on Tcl's builtin command **string is**.
7. The string **graph** is an atomic parsing expression. It matches any Unicode printing character, except for space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
8. The string **lower** is an atomic parsing expression. It matches any Unicode lower-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
9. The string **print** is an atomic parsing expression. It matches any Unicode printing character, including space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
10. The string **punct** is an atomic parsing expression. It matches any Unicode punctuation character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
11. The string **space** is an atomic parsing expression. It matches any Unicode space character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
12. The string **upper** is an atomic parsing expression. It matches any Unicode upper-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
13. The string **wordchar** is an atomic parsing expression. It matches any Unicode word character. This is any alphanumeric character (see alnum), and any connector punctuation characters (e.g. underscore). This is a custom extension of PEs based on Tcl's builtin command **string is**.
14. The string **xdigit** is an atomic parsing expression. It matches any hexadecimal digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
15. The string **ddigit** is an atomic parsing expression. It matches any decimal digit character. This is a custom extension of PEs based on Tcl's builtin command **regexp**.
16. The expression [list t **x**] is an atomic parsing expression. It matches the terminal string **x**.
17. The expression [list n **A**] is an atomic parsing expression. It matches the nonterminal **A**.

Combined Parsing Expressions

1. For parsing expressions **e1**, **e2**, ... the result of [list / **e1 e2** ...] is a parsing expression

as well. This is the *ordered choice*, aka *prioritized choice*.

1. For parsing expressions **e1**, **e2**, ... the result of [list x **e1 e2** ...] is a parsing expression as well. This is the *sequence*.
2. For a parsing expression **e** the result of [list * **e**] is a parsing expression as well. This is the *kleene closure*, describing zero or more repetitions.
3. For a parsing expression **e** the result of [list + **e**] is a parsing expression as well. This is the *positive kleene closure*, describing one or more repetitions.
4. For a parsing expression **e** the result of [list & **e**] is a parsing expression as well. This is the *and lookahead predicate*.
5. For a parsing expression **e** the result of [list! **e**] is a parsing expression as well. This is the *not lookahead predicate*.
6. For a parsing expression **e** the result of [list ? **e**] is a parsing expression as well. This is the *optional input*.

Canonical serialization

The canonical serialization of a parsing expression has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this parsing expression.

1. The string representation of the value is the canonical representation of a pure Tcl list. I.e. it does not contain superfluous whitespace.
2. Terminals are *not* encoded as ranges (where start and end of the range are identical).

Example

Assuming the parsing expression shown on the right-hand side of the rule

```
    Expression <- Term (AddOp Term)*
```

then its canonical serialization (except for whitespace) is

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pt* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Parsing and Grammars

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

pt_peg_to_tclparam

Keywords: pt_peg_to_tclparam, EBNF, LL(k), PEG, TCLPARAM, TDPL, context-free languages, conversion, expression, format conversion, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, serialization, state, top-down parsing languages, transducer

Name

pt::peg::to::tclparam - PEG Conversion. Write TCLPARAM format

Synopsis

package require Tcl 8.5 9
package require pt::peg::to::tclparam ?**1.0.4**?
pt::peg::to::tclparam reset
pt::peg::to::tclparam configure
pt::peg::to::tclparam configure *option*
pt::peg::to::tclparam configure *option value...*
pt::peg::to::tclparam convert *serial*

Description

Are you lost ? Do you have trouble understanding this document ? In that case please read the overview provided by the *Introduction to Parser Tools*. This document is the entrypoint to the whole system the current package is a part of.

This package implements the converter from parsing expression grammars to TCLPARAM markup.

It resides in the Export section of the Core Layer of Parser Tools, and can be used either directly with the other packages of this layer, or indirectly through the export manager provided by **pt::peg::export**. The latter is intented for use in untrusted environments and done through the corresponding export plugin **pt::peg::export::tclparam** sitting between converter and export manager.

IMAGE: arch_core_eplugins

API

The API provided by this package satisfies the specification of the Converter API found in the *Parser Tools Export API* specification.

pt::peg::to::tclparam reset
: This command resets the configuration of the package to its default settings.

pt::peg::to::tclparam configure
: This command returns a dictionary containing the current configuration of the package.

pt::peg::to::tclparam configure *option*
: This command returns the current value of the specified configuration *option* of the package. For the set of legal options, please read the section Options.

pt::peg::to::tclparam configure *option value...*
: This command sets the given configuration *option*s of the package, to the specified *value*s. For the set of legal options, please read the section Options.

pt::peg::to::tclparam convert *serial*

This command takes the canonical serialization of a parsing expression grammar, as specified in section Peg serialization format, and contained in *serial*, and generates TCLPARAM markup encoding the grammar, per the current package configuration. The created string is then returned as the result of the command.

Options

The converter to Tcl/PARAM markup recognizes the following configuration variables and changes its behaviour as they specify.

-template string

The value of this configuration variable is a string into which to put the generated text and the other configuration settings. The various locations for user-data are expected to be specified with the placeholders listed below. The default value is "**@code@**".

@user@
To be replaced with the value of the configuration variable **-user**.

@format@
To be replaced with the the constant **Tcl/PARAM**.

@file@
To be replaced with the value of the configuration variable **-file**.

@name@
To be replaced with the value of the configuration variable **-name**.

@code@
To be replaced with the generated Tcl code.

The following configuration variables are special, in that they will occur within the generated code, and are replaced there as well.

@runtime@
To be replaced with the value of the configuration variable **runtime-command**.

@self@
To be replaced with the value of the configuration variable **self-command**.

@def@
To be replaced with the value of the configuration variable **proc-command**.

@ns@
To be replaced with the value of the configuration variable **namespace**.

@main@
To be replaced with the value of the configuration variable **main**.

@prelude@
To be replaced with the value of the configuration variable **prelude**.

-name string

The value of this configuration variable is the name of the grammar for which the conversion is run. The default value is **a_pe_grammar**.

-user string

The value of this configuration variable is the name of the user for which the conversion is run. The default value is **unknown**.

-file string

The value of this configuration variable is the name of the file or other entity from which the grammar came, for which the conversion is run. The default value is **unknown**.

-runtime-command string
A Tcl string representing the Tcl command or reference to it used to call PARAM instruction from parser procedures, per the chosen framework (template). The default value is the empty string.

-self-command string
A Tcl string representing the Tcl command or reference to it used to call the parser procedures (methods ...) from another parser procedure, per the chosen framework (template). The default value is the empty string.

-proc-command string
The name of the Tcl command used to define procedures (methods ...), per the chosen framework (template). The default value is **proc**.

-namespace string
The name of the namespace the parser procedures (methods, ...) shall reside in, including the trailing '::' needed to separate it from the actual procedure name. The default value is **::**.

-main string
The name of the main procedure (method, ...) to be called by the chosen framework (template) to start parsing input. The default value is **__main**.

-prelude string
A snippet of code to be insert at the head of each generated parsing command. The default value is the empty string.

-indent integer
The number of characters to indent each line of the generated code by. The default value is **0**.

While the high parameterizability of this converter, as shown by the multitude of options it supports, is an advantage to the advanced user, allowing her to customize the output of the converter as needed, a novice user will likely not see the forest for the trees.

To help these latter users two adjunct packages are provided, each containing a canned configuration which will generate immediately useful full parsers. These are

pt::tclparam::configuration::snit
Generated parsers are classes based on the **snit** package, i.e. snit::type's.

pt::tclparam::configuration::tcloo
Generated parsers are classes based on the **OO** package.

Tcl/PARAM code representation of parsing expression grammars

The Tcl/PARAM representation of parsing expression grammars is Tcl code whose execution will parse input per the grammar. The code is based on the virtual machine documented in the *PackRat Machine Specification*, using its instructions and a few more to handle control flow.

Note that the generated code by itself is not functional. It expects to be embedded into a framework which provides services like the PARAM state, implementations for the PARAM instructions, etc. The bulk of such a framework has to be specified through the option **-template**. The additional options

- **-indent** integer
- **-main** string

- **-namespace** string
- **-prelude** string
- **-proc-command** string
- **-runtime-command** string
- **-self-command** string

provide code snippets which help to glue framework and generated code together. Their placeholders are in the *generated* code.

Peg serialization format

Here we specify the format used by the Parser Tools to serialize Parsing Expression Grammars as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a PEG may have more than one regular serialization only exactly one of them will be *canonical*.

regular serialization

1. The serialization of any PEG is a nested Tcl dictionary.
1. This dictionary holds a single key, **pt::grammar::peg**, and its value. This value holds the contents of the grammar.
2. The contents of the grammar are a Tcl dictionary holding the set of nonterminal symbols and the starting expression. The relevant keys and their values are

   rules

   The value is a Tcl dictionary whose keys are the names of the nonterminal symbols known to the grammar.

   1. Each nonterminal symbol may occur only once.
   2. The empty string is not a legal nonterminal symbol.
   3. The value for each symbol is a Tcl dictionary itself. The relevant keys and their values in this dictionary are

      is

      The value is the serialization of the parsing expression describing the symbols sentennial structure, as specified in the section PE serialization format.

      mode

      The value can be one of three values specifying how a parser should handle the semantic value produced by the symbol.

      value

      The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal itself, which has the ASTs of the symbol's right hand side as its children.

      leaf

      The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal, without any children. Any ASTs generated by the symbol's right hand side are discarded.

      void

      The nonterminal has no semantic value. Any ASTs generated by the symbol's right hand side are discarded (as well).

```
**start**
: The value is the serialization of the start parsing expression of
    the grammar, as specified in the section [PE serialization
    format].
```

1. The terminal symbols of the grammar are specified implicitly as the set of all terminal symbols used in the start expression and on the RHS of the grammar rules.

canonical serialization

The canonical serialization of a grammar has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this grammar.

1. The keys found in all the nested Tcl dictionaries are sorted in ascending dictionary order, as generated by Tcl's builtin command **lsort -increasing -dict**.
2. The string representation of the value is the canonical representation of a Tcl dictionary. I.e. it does not contain superfluous whitespace.

Example

Assuming the following PEG for simple mathematical expressions

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

then its canonical serialization (except for whitespace) is

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                  mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}                mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                  mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                  mode value}
        MulOp      {is {/ {t *} {t /}}
                                  mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
                                  mode value}
        Sign       {is {/ {t -} {t +}}
```

```
                                    mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                    mode value}
    }
    start {n Expression}
}
```

PE serialization format

Here we specify the format used by the Parser Tools to serialize Parsing Expressions as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a parsing expression may have more than one regular serialization only exactly one of them will be *canonical*.

Regular serialization

see below ...

Atomic Parsing Expressions

1. The string **epsilon** is an atomic parsing expression. It matches the empty string.
1. The string **dot** is an atomic parsing expression. It matches any character.
2. The string **alnum** is an atomic parsing expression. It matches any Unicode alphabet or digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
3. The string **alpha** is an atomic parsing expression. It matches any Unicode alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
4. The string **ascii** is an atomic parsing expression. It matches any Unicode character below U0080. This is a custom extension of PEs based on Tcl's builtin command **string is**.
5. The string **control** is an atomic parsing expression. It matches any Unicode control character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
6. The string **digit** is an atomic parsing expression. It matches any Unicode digit character. Note that this includes characters outside of the [0..9] range. This is a custom extension of PEs based on Tcl's builtin command **string is**.
7. The string **graph** is an atomic parsing expression. It matches any Unicode printing character, except for space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
8. The string **lower** is an atomic parsing expression. It matches any Unicode lower-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
9. The string **print** is an atomic parsing expression. It matches any Unicode printing character, including space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
10. The string **punct** is an atomic parsing expression. It matches any Unicode punctuation character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
11. The string **space** is an atomic parsing expression. It matches any Unicode space character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
12. The string **upper** is an atomic parsing expression. It matches any Unicode upper-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command

string is.

13. The string **wordchar** is an atomic parsing expression. It matches any Unicode word character. This is any alphanumeric character (see alnum), and any connector punctuation characters (e.g. underscore). This is a custom extension of PEs based on Tcl's builtin command **string is**.
14. The string **xdigit** is an atomic parsing expression. It matches any hexadecimal digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
15. The string **ddigit** is an atomic parsing expression. It matches any decimal digit character. This is a custom extension of PEs based on Tcl's builtin command **regexp**.
16. The expression [list t **x**] is an atomic parsing expression. It matches the terminal string **x**.
17. The expression [list n **A**] is an atomic parsing expression. It matches the nonterminal **A**.

Combined Parsing Expressions

1. For parsing expressions **e1**, **e2**, ... the result of [list / **e1 e2** ...] is a parsing expression as well. This is the *ordered choice*, aka *prioritized choice*.
1. For parsing expressions **e1**, **e2**, ... the result of [list x **e1 e2** ...] is a parsing expression as well. This is the *sequence*.
2. For a parsing expression **e** the result of [list * **e**] is a parsing expression as well. This is the *kleene closure*, describing zero or more repetitions.
3. For a parsing expression **e** the result of [list + **e**] is a parsing expression as well. This is the *positive kleene closure*, describing one or more repetitions.
4. For a parsing expression **e** the result of [list & **e**] is a parsing expression as well. This is the *and lookahead predicate*.
5. For a parsing expression **e** the result of [list! **e**] is a parsing expression as well. This is the *not lookahead predicate*.
6. For a parsing expression **e** the result of [list ? **e**] is a parsing expression as well. This is the *optional input*.

Canonical serialization

The canonical serialization of a parsing expression has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this parsing expression.

1. The string representation of the value is the canonical representation of a pure Tcl list. I.e. it does not contain superfluous whitespace.
2. Terminals are *not* encoded as ranges (where start and end of the range are identical).

Example

Assuming the parsing expression shown on the right-hand side of the rule

```
    Expression <- Term (AddOp Term)*
```

then its canonical serialization (except for whitespace) is

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you

want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pt* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Parsing and Grammars

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_pegrammar

Keywords: pt_pegrammar, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar , matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt::peg - Parsing Expression Grammar Serialization

Synopsis

package require Tcl 8.5 9
package require pt::peg ?**1.1**?
package require pt::pe
::pt::peg verify *serial* ?*canonvar*?
::pt::peg verify-as-canonical *serial*
::pt::peg canonicalize *serial*
::pt::peg print *serial*
::pt::peg merge *seriala serialb*
::pt::peg equal *seriala serialb*

Description

Are you lost ? Do you have trouble understanding this document ? In that case please read the overview provided by the *Introduction to Parser Tools*. This document is the entrypoint to the whole system the current package is a part of.

This package provides commands to work with the serializations of parsing expression grammars as managed by the Parser Tools, and specified in section Peg serialization format.

This is a supporting package in the Core Layer of Parser Tools.

IMAGE: arch_core_support

API

::pt::peg verify *serial* ?*canonvar*?

This command verifies that the content of *serial* is a valid serialization of a parsing expression

and will throw an error if that is not the case. The result of the command is the empty string.

If the argument *canonvar* is specified it is interpreted as the name of a variable in the calling context. This variable will be written to if and only if *serial* is a valid regular serialization. Its value will be a boolean, with **True** indicating that the serialization is not only valid, but also *canonical*. **False** will be written for a valid, but non-canonical serialization.

For the specification of serializations see the section PE serialization format.

::pt::peg verify-as-canonical *serial*

This command verifies that the content of *serial* is a valid *canonical* serialization of a PEG and will throw an error if that is not the case. The result of the command is the empty string.

For the specification of canonical serializations see the section Peg serialization format.

::pt::peg canonicalize *serial*

This command assumes that the content of *serial* is a valid *regular* serialization of a PEG and will throw an error if that is not the case.

It will then convert the input into the *canonical* serialization of the contained PEG and return it as its result. If the input is already canonical it will be returned unchanged.

For the specification of regular and canonical serializations see the section Peg serialization format.

::pt::peg print *serial*

This command assumes that the argument *serial* contains a valid serialization of a parsing expression and returns a string containing that PE in a human readable form.

The exact format of this form is not specified and cannot be relied on for parsing or other machine-based activities.

For the specification of serializations see the section Peg serialization format.

::pt::peg merge *seriala serialb*

This command accepts the regular serializations of two grammars and uses them to create their union. The result of the command is the canonical serialization of this unified grammar.

A merge errors occurs if for any nonterminal symbol S occuring in both input grammars the two input grammars specify different semantic modes.

The semantic mode of each nonterminal symbol S is the semantic mode of S in any of its input grammars. The previous rule made sure that for symbols occuring in both grammars these values are identical.

The right-hand side of each nonterminal symbol S occuring in both input grammars is the choice between the right-hand sides of S in the input grammars, with the parsing expression of S in *seriala* coming first, except if both expressions are identical. In that case the first expression is taken.

The right-hand side of each nonterminal symbol S occuring in only one of the input grammars is the right-hand side of S in its input grammar.

The start expression of the unified grammar is the choice between the start expressions of the input grammars, with the start expression of *seriala* coming first, except if both expressions are identical. In that case the first expression is taken

::pt::peg equal *seriala serialb*

This command tests the two grammars *seriala* and *serialb* for structural equality. The result of the command is a boolean value. It will be set to **true** if the expressions are identical, and **false** otherwise.

String equality is usable only if we can assume that the two grammars are pure Tcl lists and dictionaries.

Peg serialization format

Here we specify the format used by the Parser Tools to serialize Parsing Expression Grammars as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a PEG may have more than one regular serialization only exactly one of them will be *canonical*.

regular serialization

1. The serialization of any PEG is a nested Tcl dictionary.
1. This dictionary holds a single key, **pt::grammar::peg**, and its value. This value holds the contents of the grammar.
2. The contents of the grammar are a Tcl dictionary holding the set of nonterminal symbols and the starting expression. The relevant keys and their values are

   rules

   The value is a Tcl dictionary whose keys are the names of the nonterminal symbols known to the grammar.

   1. Each nonterminal symbol may occur only once.
   2. The empty string is not a legal nonterminal symbol.
   3. The value for each symbol is a Tcl dictionary itself. The relevant keys and their values in this dictionary are

      is

      The value is the serialization of the parsing expression describing the symbols sentennial structure, as specified in the section PE serialization format.

      mode

      The value can be one of three values specifying how a parser should handle the semantic value produced by the symbol.

      value

      The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal itself, which has the ASTs of the symbol's right hand side as its children.

      leaf

      The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal, without any children. Any

ASTs generated by the symbol's right hand side are discarded.

void
The nonterminal has no semantic value. Any ASTs generated by the symbol's right hand side are discarded (as well).

```
**start**
: The value is the serialization of the start parsing expression of
    the grammar, as specified in the section [PE serialization
    format].
```

1. The terminal symbols of the grammar are specified implicitly as the set of all terminal symbols used in the start expression and on the RHS of the grammar rules.

canonical serialization
The canonical serialization of a grammar has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this grammar.

1. The keys found in all the nested Tcl dictionaries are sorted in ascending dictionary order, as generated by Tcl's builtin command **lsort -increasing -dict**.
2. The string representation of the value is the canonical representation of a Tcl dictionary. I.e. it does not contain superfluous whitespace.

Example

Assuming the following PEG for simple mathematical expressions

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

then its canonical serialization (except for whitespace) is

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                  mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}                mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                  mode value}
```

```
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                  mode value}
        MulOp      {is {/ {t *} {t /}}
                                  mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
                                  mode value}
        Sign       {is {/ {t -} {t +}}
                                  mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                  mode value}
    }
    start {n Expression}
}
```

PE serialization format

Here we specify the format used by the Parser Tools to serialize Parsing Expressions as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a parsing expression may have more than one regular serialization only exactly one of them will be *canonical*.

Regular serialization

see below ...

Atomic Parsing Expressions

1. The string **epsilon** is an atomic parsing expression. It matches the empty string.
1. The string **dot** is an atomic parsing expression. It matches any character.
2. The string **alnum** is an atomic parsing expression. It matches any Unicode alphabet or digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
3. The string **alpha** is an atomic parsing expression. It matches any Unicode alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
4. The string **ascii** is an atomic parsing expression. It matches any Unicode character below U0080. This is a custom extension of PEs based on Tcl's builtin command **string is**.
5. The string **control** is an atomic parsing expression. It matches any Unicode control character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
6. The string **digit** is an atomic parsing expression. It matches any Unicode digit character. Note that this includes characters outside of the [0..9] range. This is a custom extension of PEs based on Tcl's builtin command **string is**.
7. The string **graph** is an atomic parsing expression. It matches any Unicode printing character, except for space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
8. The string **lower** is an atomic parsing expression. It matches any Unicode lower-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
9. The string **print** is an atomic parsing expression. It matches any Unicode printing character, including space. This is a custom extension of PEs based on Tcl's builtin

command **string is**.

10. The string **punct** is an atomic parsing expression. It matches any Unicode punctuation character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
11. The string **space** is an atomic parsing expression. It matches any Unicode space character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
12. The string **upper** is an atomic parsing expression. It matches any Unicode upper-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
13. The string **wordchar** is an atomic parsing expression. It matches any Unicode word character. This is any alphanumeric character (see alnum), and any connector punctuation characters (e.g. underscore). This is a custom extension of PEs based on Tcl's builtin command **string is**.
14. The string **xdigit** is an atomic parsing expression. It matches any hexadecimal digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
15. The string **ddigit** is an atomic parsing expression. It matches any decimal digit character. This is a custom extension of PEs based on Tcl's builtin command **regexp**.
16. The expression [list t **x**] is an atomic parsing expression. It matches the terminal string **x**.
17. The expression [list n **A**] is an atomic parsing expression. It matches the nonterminal **A**.

Combined Parsing Expressions

1. For parsing expressions **e1**, **e2**, ... the result of [list / **e1 e2** ...] is a parsing expression as well. This is the *ordered choice*, aka *prioritized choice*.
1. For parsing expressions **e1**, **e2**, ... the result of [list x **e1 e2** ...] is a parsing expression as well. This is the *sequence*.
2. For a parsing expression **e** the result of [list * **e**] is a parsing expression as well. This is the *kleene closure*, describing zero or more repetitions.
3. For a parsing expression **e** the result of [list + **e**] is a parsing expression as well. This is the *positive kleene closure*, describing one or more repetitions.
4. For a parsing expression **e** the result of [list & **e**] is a parsing expression as well. This is the *and lookahead predicate*.
5. For a parsing expression **e** the result of [list! **e**] is a parsing expression as well. This is the *not lookahead predicate*.
6. For a parsing expression **e** the result of [list ? **e**] is a parsing expression as well. This is the *optional input*.

Canonical serialization

The canonical serialization of a parsing expression has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this parsing expression.

1. The string representation of the value is the canonical representation of a pure Tcl list. I.e. it does not contain superfluous whitespace.
2. Terminals are *not* encoded as ranges (where start and end of the range are identical).

Example

Assuming the parsing expression shown on the right-hand side of the rule

```
    Expression <- Term (AddOp Term)*
```

then its canonical serialization (except for whitespace) is

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pt* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Parsing and Grammars

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_pexpr_op

Keywords: pt_pexpr_op, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt::pe::op - Parsing Expression Utilities

Synopsis

package require Tcl 8.5 9
package require pt::pe::op ?**1.0.2**?
package require pt::pe ?**1**?
package require struct::set
::pt::pe::op drop *dropset pe*
::pt::pe::op rename *nt ntnew pe*
::pt::pe::op called *pe*
::pt::pe::op flatten *pe*
::pt::pe::op fusechars *pe*

Description

Are you lost ? Do you have trouble understanding this document ? In that case please read the overview provided by the *Introduction to Parser Tools*. This document is the entrypoint to the whole system the current package is a part of.

This package provides additional commands to work with the serializations of parsing

expressions as managed by the PEG and related packages, and specified in section Pe serialization format.

This is an internal package, for use by the higher level packages handling PEGs, their conversion into and out of various other formats, or other uses.

API

::pt::pe::op drop *dropset pe*
: This command removes all occurences of any of the nonterminals symbols in the set *dropset* from the parsing expression *pe*, and simplifies it. This may result in the expression becoming "epsilon", i.e. matching nothing.

::pt::pe::op rename *nt ntnew pe*
: This command renames all occurences of the nonterminal *nt* in the parsing expression *pe* into *ntnew*.

::pt::pe::op called *pe*
: This command extracts the set of all nonterminal symbols used, i.e. 'called', in the parsing expression *pe*.

::pt::pe::op flatten *pe*
: This command transforms the parsing expression by eliminating sequences nested in sequences, and choices in choices, lifting the children of the nested expression into the parent. It further eliminates all sequences and choices with only one child, as these are redundant.

  The resulting parsing expression is returned as the result of the command.

::pt::pe::op fusechars *pe*
: This command transforms the parsing expression by fusing adjacent terminals in sequences and adjacent terminals and ranges in choices, it (re)constructs highlevel *strings* and *character classes*.

  The resulting pseudo-parsing expression is returned as the result of the command and may contain the pseudo-operators **str** for character sequences, aka strings, and **cl** for character choices, aka character classes.

  The result is called a *pseudo-parsing expression* because it is not a true parsing expression anymore, and will fail a check with ::pt::peg verify if the new pseudo-operators are present in the result, but is otherwise of sound structure for a parsing expression. Notably, the commands ::pt::peg bottomup and ::pt::peg topdown will process them without trouble.

Pe serialization format

Here we specify the format used by the Parser Tools to serialize Parsing Expressions as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a parsing expression may have more than one regular serialization only exactly one of them will be *canonical*.

Regular serialization
: see below ...

  Atomic Parsing Expressions

1. The string **epsilon** is an atomic parsing expression. It matches the empty string.
1. The string **dot** is an atomic parsing expression. It matches any character.
2. The string **alnum** is an atomic parsing expression. It matches any Unicode alphabet or digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
3. The string **alpha** is an atomic parsing expression. It matches any Unicode alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
4. The string **ascii** is an atomic parsing expression. It matches any Unicode character below U0080. This is a custom extension of PEs based on Tcl's builtin command **string is**.
5. The string **control** is an atomic parsing expression. It matches any Unicode control character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
6. The string **digit** is an atomic parsing expression. It matches any Unicode digit character. Note that this includes characters outside of the [0..9] range. This is a custom extension of PEs based on Tcl's builtin command **string is**.
7. The string **graph** is an atomic parsing expression. It matches any Unicode printing character, except for space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
8. The string **lower** is an atomic parsing expression. It matches any Unicode lower-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
9. The string **print** is an atomic parsing expression. It matches any Unicode printing character, including space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
10. The string **punct** is an atomic parsing expression. It matches any Unicode punctuation character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
11. The string **space** is an atomic parsing expression. It matches any Unicode space character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
12. The string **upper** is an atomic parsing expression. It matches any Unicode upper-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
13. The string **wordchar** is an atomic parsing expression. It matches any Unicode word character. This is any alphanumeric character (see alnum), and any connector punctuation characters (e.g. underscore). This is a custom extension of PEs based on Tcl's builtin command **string is**.
14. The string **xdigit** is an atomic parsing expression. It matches any hexadecimal digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
15. The string **ddigit** is an atomic parsing expression. It matches any decimal digit character. This is a custom extension of PEs based on Tcl's builtin command **regexp**.
16. The expression [list t **x**] is an atomic parsing expression. It matches the terminal string **x**.
17. The expression [list n **A**] is an atomic parsing expression. It matches the nonterminal **A**.

Combined Parsing Expressions

1. For parsing expressions **e1**, **e2**, ... the result of [list / **e1 e2** ...] is a parsing expression as well. This is the *ordered choice*, aka *prioritized choice*.
1. For parsing expressions **e1**, **e2**, ... the result of [list x **e1 e2** ...] is a parsing expression

as well. This is the *sequence*.
2. For a parsing expression **e** the result of [list * **e**] is a parsing expression as well. This is the *kleene closure*, describing zero or more repetitions.
3. For a parsing expression **e** the result of [list + **e**] is a parsing expression as well. This is the *positive kleene closure*, describing one or more repetitions.
4. For a parsing expression **e** the result of [list & **e**] is a parsing expression as well. This is the *and lookahead predicate*.
5. For a parsing expression **e** the result of [list! **e**] is a parsing expression as well. This is the *not lookahead predicate*.
6. For a parsing expression **e** the result of [list ? **e**] is a parsing expression as well. This is the *optional input*.

Canonical serialization

The canonical serialization of a parsing expression has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this parsing expression.

1. The string representation of the value is the canonical representation of a pure Tcl list. I.e. it does not contain superfluous whitespace.
2. Terminals are *not* encoded as ranges (where start and end of the range are identical).

Example

Assuming the parsing expression shown on the right-hand side of the rule

```
    Expression <- Term (AddOp Term)*
```

then its canonical serialization (except for whitespace) is

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pt* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Parsing and Grammars

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_pexpression

Keywords: pt_pexpression, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar , matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt::pe - Parsing Expression Serialization

Synopsis

package require Tcl 8.5 9
package require pt::pe ?**1.0.2**?
package require char
::pt::pe verify *serial* ?*canonvar*?
::pt::pe verify-as-canonical *serial*
::pt::pe canonicalize *serial*
::pt::pe print *serial*
::pt::pe bottomup *cmdprefix pe*
cmdprefix *pe op arguments*
::pt::pe topdown *cmdprefix pe*
::pt::pe equal *seriala serialb*
::pt::pe epsilon
::pt::pe dot
::pt::pe alnum
::pt::pe alpha
::pt::pe ascii
::pt::pe control
::pt::pe digit
::pt::pe graph
::pt::pe lower
::pt::pe print
::pt::pe punct
::pt::pe space
::pt::pe upper
::pt::pe wordchar
::pt::pe xdigit
::pt::pe ddigit
::pt::pe terminal *t*
::pt::pe range *ta tb*
::pt::pe nonterminal *nt*
::pt::pe choice *pe...*
::pt::pe sequence *pe...*
::pt::pe repeat0 *pe*
::pt::pe repeat1 *pe*
::pt::pe optional *pe*
::pt::pe ahead *pe*
::pt::pe notahead *pe*

Description

Are you lost ? Do you have trouble understanding this document ? In that case please read the overview provided by the *Introduction to Parser Tools*. This document is the entrypoint to the whole system the current package is a part of.

This package provides commands to work with the serializations of parsing expressions as managed by the Parser Tools, and specified in section Pe serialization format.

This is a supporting package in the Core Layer of Parser Tools.

IMAGE: arch_core_support

API

::pt::pe verify *serial* ?*canonvar*?

This command verifies that the content of *serial* is a valid serialization of a parsing expression and will throw an error if that is not the case. The result of the command is the empty string.

If the argument *canonvar* is specified it is interpreted as the name of a variable in the calling context. This variable will be written to if and only if *serial* is a valid regular serialization. Its value will be a boolean, with **True** indicating that the serialization is not only valid, but also *canonical*. **False** will be written for a valid, but non-canonical serialization.

For the specification of serializations see the section Pe serialization format.

::pt::pe verify-as-canonical *serial*

This command verifies that the content of *serial* is a valid *canonical* serialization of a parsing expression and will throw an error if that is not the case. The result of the command is the empty string.

For the specification of canonical serializations see the section Pe serialization format.

::pt::pe canonicalize *serial*

This command assumes that the content of *serial* is a valid *regular* serialization of a parsing expression and will throw an error if that is not the case.

It will then convert the input into the *canonical* serialization of this parsing expression and return it as its result. If the input is already canonical it will be returned unchanged.

For the specification of regular and canonical serializations see the section Pe serialization format.

::pt::pe print *serial*

This command assumes that the argument *serial* contains a valid serialization of a parsing expression and returns a string containing that PE in a human readable form.

The exact format of this form is not specified and cannot be relied on for parsing or other machine-based activities.

For the specification of serializations see the section Pe serialization format.

::pt::pe bottomup *cmdprefix pe*

This command walks the parsing expression *pe* from the bottom up to the root, invoking the

command prefix *cmdprefix* for each partial expression. This implies that the children of a parsing expression PE are handled before PE.

The command prefix has the signature

cmdprefix *pe op arguments*

I.e. it is invoked with the parsing expression *pe* the walk is currently at, the *op*'erator in the *pe*, and the operator's *arguments*.

The result returned by the command prefix replaces *pe* in the parsing expression it was a child of, allowing transformations of the expression tree.

This also means that for all inner parsing expressions the contents of *arguments* are the results of the command prefix invoked for the children of this inner parsing expression.

::pt::pe topdown *cmdprefix pe*

This command walks the parsing expression *pe* from the root down to the leaves, invoking the command prefix *cmdprefix* for each partial expression. This implies that the children of a parsing expression PE are handled after PE.

The command prefix has the same signature as for **bottomup**, see above.

The result returned by the command prefix is *ignored*.

::pt::pe equal *seriala serialb*

This command tests the two parsing expressions *seriala* and *serialb* for structural equality. The result of the command is a boolean value. It will be set to **true** if the expressions are identical, and **false** otherwise.

String equality is usable only if we can assume that the two parsing expressions are pure Tcl lists.

::pt::pe epsilon

This command constructs the atomic parsing expression for epsilon.

::pt::pe dot

This command constructs the atomic parsing expression for dot.

::pt::pe alnum

This command constructs the atomic parsing expression for alnum.

::pt::pe alpha

This command constructs the atomic parsing expression for alpha.

::pt::pe ascii

This command constructs the atomic parsing expression for ascii.

::pt::pe control

This command constructs the atomic parsing expression for control.

::pt::pe digit

This command constructs the atomic parsing expression for digit.

::pt::pe graph

This command constructs the atomic parsing expression for graph.

::pt::pe lower

This command constructs the atomic parsing expression for lower.

::pt::pe print
This command constructs the atomic parsing expression for print.

::pt::pe punct
This command constructs the atomic parsing expression for punct.

::pt::pe space
This command constructs the atomic parsing expression for space.

::pt::pe upper
This command constructs the atomic parsing expression for upper.

::pt::pe wordchar
This command constructs the atomic parsing expression for wordchar.

::pt::pe xdigit
This command constructs the atomic parsing expression for xdigit.

::pt::pe ddigit
This command constructs the atomic parsing expression for ddigit.

::pt::pe terminal *t*
This command constructs the atomic parsing expression for the terminal symbol *t*.

::pt::pe range *ta tb*
This command constructs the atomic parsing expression for the range of terminal symbols *ta* ... *tb*.

::pt::pe nonterminal *nt*
This command constructs the atomic parsing expression for the nonterminal symbol *nt*.

::pt::pe choice *pe...*
This command constructs the parsing expression representing the ordered or prioritized choice between the argument parsing expressions. The first argument has the highest priority.

::pt::pe sequence *pe...*
This command constructs the parsing expression representing the sequence of the argument parsing expression. The first argument is the first element of the sequence.

::pt::pe repeat0 *pe*
This command constructs the parsing expression representing the zero or more repetition of the argument parsing expression *pe*, also known as the kleene closure.

::pt::pe repeat1 *pe*
This command constructs the parsing expression representing the one or more repetition of the argument parsing expression *pe*, also known as the positive kleene closure.

::pt::pe optional *pe*
This command constructs the parsing expression representing the optionality of the argument parsing expression *pe*.

::pt::pe ahead *pe*
This command constructs the parsing expression representing the positive lookahead of the argument parsing expression *pe*.

::pt::pe notahead *pe*
This command constructs the parsing expression representing the negative lookahead of the argument parsing expression *pe*.

Pe serialization format

Here we specify the format used by the Parser Tools to serialize Parsing Expressions as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a parsing expression may have more than one regular serialization only exactly one of them will be *canonical*.

Regular serialization

see below ...

Atomic Parsing Expressions

1. The string **epsilon** is an atomic parsing expression. It matches the empty string.
1. The string **dot** is an atomic parsing expression. It matches any character.
2. The string **alnum** is an atomic parsing expression. It matches any Unicode alphabet or digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
3. The string **alpha** is an atomic parsing expression. It matches any Unicode alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
4. The string **ascii** is an atomic parsing expression. It matches any Unicode character below U0080. This is a custom extension of PEs based on Tcl's builtin command **string is**.
5. The string **control** is an atomic parsing expression. It matches any Unicode control character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
6. The string **digit** is an atomic parsing expression. It matches any Unicode digit character. Note that this includes characters outside of the [0..9] range. This is a custom extension of PEs based on Tcl's builtin command **string is**.
7. The string **graph** is an atomic parsing expression. It matches any Unicode printing character, except for space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
8. The string **lower** is an atomic parsing expression. It matches any Unicode lower-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
9. The string **print** is an atomic parsing expression. It matches any Unicode printing character, including space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
10. The string **punct** is an atomic parsing expression. It matches any Unicode punctuation character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
11. The string **space** is an atomic parsing expression. It matches any Unicode space character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
12. The string **upper** is an atomic parsing expression. It matches any Unicode upper-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
13. The string **wordchar** is an atomic parsing expression. It matches any Unicode word character. This is any alphanumeric character (see alnum), and any connector punctuation characters (e.g. underscore). This is a custom extension of PEs based on Tcl's builtin command **string is**.
14. The string **xdigit** is an atomic parsing expression. It matches any hexadecimal digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.

15. The string **ddigit** is an atomic parsing expression. It matches any decimal digit character. This is a custom extension of PEs based on Tcl's builtin command **regexp**.
16. The expression [list t **x**] is an atomic parsing expression. It matches the terminal string **x**.
17. The expression [list n **A**] is an atomic parsing expression. It matches the nonterminal **A**.

Combined Parsing Expressions

1. For parsing expressions **e1**, **e2**, ... the result of [list / **e1 e2** ...] is a parsing expression as well. This is the *ordered choice*, aka *prioritized choice*.
1. For parsing expressions **e1**, **e2**, ... the result of [list x **e1 e2** ...] is a parsing expression as well. This is the *sequence*.
2. For a parsing expression **e** the result of [list * **e**] is a parsing expression as well. This is the *kleene closure*, describing zero or more repetitions.
3. For a parsing expression **e** the result of [list + **e**] is a parsing expression as well. This is the *positive kleene closure*, describing one or more repetitions.
4. For a parsing expression **e** the result of [list & **e**] is a parsing expression as well. This is the *and lookahead predicate*.
5. For a parsing expression **e** the result of [list! **e**] is a parsing expression as well. This is the *not lookahead predicate*.
6. For a parsing expression **e** the result of [list ? **e**] is a parsing expression as well. This is the *optional input*.

Canonical serialization

The canonical serialization of a parsing expression has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this parsing expression.

1. The string representation of the value is the canonical representation of a pure Tcl list. I.e. it does not contain superfluous whitespace.
2. Terminals are *not* encoded as ranges (where start and end of the range are identical).

Example

Assuming the parsing expression shown on the right-hand side of the rule

```
    Expression <- Term (AddOp Term)*
```

then its canonical serialization (except for whitespace) is

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pt* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Parsing and Grammars

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_pgen
Keywords: pt_pgen, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name
pt::pgen - Parser Generator

Synopsis
package require Tcl 8.5 9
package require pt::pgen ?**1.2**?
::pt::pgen *inputformat text resultformat* ?*options...*?

Description
Are you lost ? Do you have trouble understanding this document ? In that case please read the overview provided by the *Introduction to Parser Tools*. This document is the entrypoint to the whole system the current package is a part of.

This package provides a command implementing a *parser generator* taking parsing expression grammars as input.

It is the implementation of method **generate** of **pt**, the *Parser Tools Application*.

As such the intended audience of this document are people wishing to modify and/or extend this part of **pt**'s functionality. Users of **pt** on the other hand are hereby refered to the applications' manpage, i.e. *Parser Tools Application*.

It resides in the User Package Layer of Parser Tools.

IMAGE: arch_user_pkg

API
::pt::pgen *inputformat text resultformat ?options...?*

This command takes the parsing expression grammar in *text* (in the format specified by *inputformat*), and returns the same grammar in the format *resultformat* as the result of the command.

The two known input formats are **peg** and **json**. Introductions to them, including their formal specifications, can be found in the *PEG Language Tutorial* and *The JSON Grammar Exchange Format*. The packages used to parse these formats are

peg
: pt::peg::from::peg

json

pt::peg::from::json

On the output side the known formats, and the packages used to generate them are

c
pt::peg::to::cparam

container
pt::peg::to::container

critcl
pt::peg::to::cparam + **pt::cparam::configuration::critcl**

json
pt::peg::to::json

oo
pt::peg::to::tclparam + **pt::tclparam::configuration::tcloo**

peg
pt::peg::to::peg

snit
pt::peg::to::tclparam + **pt::tclparam::configuration::snit**

The options supported by each of these formats are documented with their respective packages.

Example

In this section we are working a complete example, starting with a PEG grammar and ending with running the parser generated from it over some input, following the outline shown in the figure below:

IMAGE: flow

Our grammar, assumed to the stored in the file "*calculator.peg*" is

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

From this we create a snit-based parser using the script "*gen*"

```
package require Tcl 8.5
package require fileutil
package require pt::pgen

lassign $argv name
```

```
set grammar [fileutil::cat $name.peg]
set pclass  [pt::pgen peg $gr snit -class $name -file  $name.peg -name
     $name]
fileutil::writeFile $name.tcl $pclass
exit 0
```

calling it like

```
 tclsh8.5 gen calculator
```

which leaves us with the parser package and class written to the file "*calculator.tcl*". Assuming that this package is then properly installed in a place where Tcl can find it we can now use this class via a script like

```
    package require calculator

    lassign $argv input
    set channel [open $input r]

    set parser [calculator]
    set ast [$parser parse $channel]
    $parser destroy
    close $channel

    ... now process the returned abstract syntax tree ...
```

where the abstract syntax tree stored in the variable will look like

```
set ast {Expression 0 4
    {Factor 0 4
        {Term 0 2
            {Number 0 2
                {Digit 0 0}
                {Digit 1 1}
                {Digit 2 2}
            }
        }
        {AddOp 3 3}
        {Term 4 4
            {Number 4 4
                {Digit 4 4}
            }
        }
    }
}
```

assuming that the input file and channel contained the text

```
 120+5
```

A more graphical representation of the tree would be

```
+- Digit 0 0 | 1
|            |
+- Term 0 2 --- Number 0 2 -+- Digit 1 1 | 2
|                          |            |
|                          +- Digit 2 2 | 0
|                                       |
Expression 0 4 --- Factor 0 4 -+----------------------------- AddOp 3
    3 | +
|                                       |
+- Term 4 4 --- Number 4 4 --- Digit 4 4 | 5
```

Regardless, at this point it is the user's responsibility to work with the tree to reach whatever goal she desires. I.e. analyze it, transform it, etc. The package **pt::ast** should be of help here, providing commands to walk such ASTs structures in various ways.

One important thing to note is that the parsers used here return a data structure representing the structure of the input per the grammar underlying the parser. There are *no* callbacks during the parsing process, i.e. no *parsing actions*, as most other parsers will have.

Going back to the last snippet of code, the execution of the parser for some input, note how the parser instance follows the specified *Parser API*.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pt* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Parsing and Grammars

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_rdengine

Keywords: pt_rdengine, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt::rde - Parsing Runtime Support, PARAM based

Synopsis

package require Tcl 8.5 9
package require pt::rde ?**1.2**?
package require snit
package require struct::stack 1.6
package require pt::ast 1.2
::pt::rde *objectName*
objectName **destroy**
objectName **reset** *chan*
objectName **complete**
objectName **chan**
objectName **line**
objectName **column**
objectName **current**
objectName **location**
objectName **locations**
objectName **ok**
objectName **value**
objectName **error**
objectName **errors**
objectName **tokens** ?*from ?to*?
objectName **symbols**
objectName **known**
objectName **reducible**
objectName **asts**
objectName **ast**
objectName **position** *loc*
objectName **i_input_next** *msg*
objectName **i_test_alnum**
objectName **i_test_alpha**
objectName **i_test_ascii**
objectName **i_test_char** *char*
objectName **i_test_ddigit**
objectName **i_test_digit**
objectName **i_test_graph**
objectName **i_test_lower**
objectName **i_test_print**
objectName **i_test_punct**
objectName **i_test_range** *chars chare*
objectName **i_test_space**
objectName **i_test_upper**
objectName **i_test_wordchar**
objectName **i_test_xdigit**
objectName **i_error_clear**
objectName **i_error_push**
objectName **i_error_pop_merge**

objectName **i_error_nonterminal** *symbol*
objectName **i_status_ok**
objectName **i_status_fail**
objectName **i_status_negate**
objectName **i_loc_push**
objectName **i_loc_pop_discard**
objectName **i_loc_pop_rewind**
objectName **i:ok_loc_pop_rewind**
objectName **i_loc_pop_rewind/discard**
objectName **i_symbol_restore** *symbol*
objectName **i_symbol_save** *symbol*
objectName **i_value_clear**
objectName **i_value_clear/leaf**
objectName **i_value_clear/reduce**
objectName **i:ok_ast_value_push**
objectName **i_ast_push**
objectName **i_ast_pop_rewind**
objectName **i:fail_ast_pop_rewind**
objectName **i_ast_pop_rewind/discard**
objectName **i_ast_pop_discard**
objectName **i_ast_pop_discard/rewind**
objectName **i:ok_continue**
objectName **i:fail_continue**
objectName **i:fail_return**
objectName **i:ok_return**
objectName **si:void_state_push**
objectName **si:void2_state_push**
objectName **si:value_state_push**
objectName **si:void_state_merge**
objectName **si:void_state_merge_ok**
objectName **si:value_state_merge**
objectName **si:value_notahead_start**
objectName **si:void_notahead_exit**
objectName **si:value_notahead_exit**
objectName **si:kleene_abort**
objectName **si:kleene_close**
objectName **si:voidvoid_branch**
objectName **si:voidvalue_branch**
objectName **si:valuevoid_branch**
objectName **si:valuevalue_branch**
objectName **si:voidvoid_part**
objectName **si:voidvalue_part**
objectName **si:valuevalue_part**
objectName **si:value_symbol_start** *symbol*
objectName **si:value_void_symbol_start** *symbol*
objectName **si:void_symbol_start** *symbol*

objectName **si:void_void_symbol_start** *symbol*
objectName **si:reduce_symbol_end** *symbol*
objectName **si:void_leaf_symbol_end** *symbol*
objectName **si:value_leaf_symbol_end** *symbol*
objectName **si:value_clear_symbol_end** *symbol*
objectName **si:void_clear_symbol_end** *symbol*
objectName **si:next_char** *tok*
objectName **si:next_range** *toks toke*
objectName **si:next_alnum**
objectName **si:next_alpha**
objectName **si:next_ascii**
objectName **si:next_ddigit**
objectName **si:next_digit**
objectName **si:next_graph**
objectName **si:next_lower**
objectName **si:next_print**
objectName **si:next_punct**
objectName **si:next_space**
objectName **si:next_upper**
objectName **si:next_wordchar**
objectName **si:next_xdigit**

Description

Are you lost ? Do you have trouble understanding this document ? In that case please read the overview provided by the *Introduction to Parser Tools*. This document is the entrypoint to the whole system the current package is a part of.

This package provides a class whose instances provide the runtime support for recursive descent parsers with backtracking, as is needed for the execution of, for example, parsing expression grammars. It implements the *PackRat Machine Specification*, as such that document is *required* reading to understand both this manpage, and the package itself. The description below does make numerous shorthand references to the PARAM's instructions and the various parts of its architectural state.

The package resides in the Execution section of the Core Layer of Parser Tools.

IMAGE: arch_core_transform

Note: This package not only has the standard Tcl implementation, but also an accelerator, i.e. a C implementation, based on Critcl.

Class API

The package exports the API described here.

::pt::rde *objectName*

The command creates a new runtime object for a recursive descent parser with backtracking and returns the fully qualified name of the object command as its result. The API of this object command is described in the section Object API. It may be used to invoke various operations on the object.

Object API

All objects created by this package provide the following 63 methods for the manipulation and querying of their state, which is, in essence the architectural state of a PARAM.

First some general methods and the state accessors.

***objectName* destroy**
: This method destroys the object, releasing all claimed memory, and deleting the associated object command.

objectName* reset *chan
: This method resets the state of the runtme to its defaults, preparing it for the parsing of the character in the channel *chan*, which becomes IN.

  Note here that the Parser Tools are based on Tcl 8.5+. In other words, the channel argument is not restricted to files, sockets, etc. We have the full power of *reflected channels* available.

  It should also be noted that the parser pulls the characters from the input stream as it needs them. If a parser created by this package has to be operated in a push aka event-driven manner it will be necessary to go to Tcl 8.6+ and use the **coroutine::auto** to wrap it into a coroutine where **read** is properly changed for push-operation.

***objectName* complete**
: This method completes parsing, either returning the AST made from the elements of ARS, or throwing an error containing the current ER.

***objectName* chan**
: This method returns the handle of the channel which is IN.

***objectName* line**
: This method returns the line number for the position IN is currently at. Note that this may not match with the line number for CL, due to backtracking.

***objectName* column**
: This method returns the column for the position IN is currently at. Note that this may not match with the column for CL, due to backtracking.

***objectName* current**
: This method returns CC.

***objectName* location**
: This method returns CL.

***objectName* locations**
: This method returns the LS. The topmost entry of the stack will be the first element of the returned list.

***objectName* ok**
: This method returns ST.

***objectName* value**
: This method returns SV.

***objectName* error**
: This method returns ER. This is either the empty string for an empty ER, or a list of 2 elements, the location the error is for, and a set of messages which specify which symbols were expected at the location. The messages are encoded as one of the possible atomic

parsing expressions (special operators, terminal, range, and nonterminal operator).

***objectName* errors**
This method returns ES. The topmost entry of the stack will be the first element of the returned list. Each entry is encoded as described for **error**.

***objectName* tokens ?*from* ?*to*??**
This method returns the part of TC for the range of locations of IN starting at *from* and ending at *to*. If *to* is not specified it is taken as identical to *from*. If neither argument is specified the whole of TC is returned.

Each token in the returned list is a list of three elements itself, containing the character at the location, and the associated line and column numbers, in this order.

***objectName* symbols**
This method returns a dictionary containing NC. Keys are two-element lists containing nonterminal symbol and location, in this order. The values are 4-tuples containing CL, ST, ER, and SV, in this order. ER is encoded as specified for the method **error**.

***objectName* known**
This method returns a list containing the keys of SC. They are encoded in the same manner as is done by method **symbols**.

***objectName* reducible**
This method returns ARS. The topmost entry of the stack will be the first element of the returned list

***objectName* asts**
This method returns AS. The topmost entry of the stack will be the first element of the returned list

***objectName* ast**
This is a convenience method returning the topmost element of ARS.

objectName* position *loc
This method returns the line and column numbers for the specified location of IN, assuming that this location has already been reached during the parsing process.

The following methods implement all PARAM instructions. They all have the prefix "i_".

The control flow is mainly provided by Tcl's builtin commands, like **if**, **while**, etc., plus a few guarded variants of PARAM instructions and Tcl commands.. That means that these instruction variants will do nothing if their guard condition is not fulfilled. They can be recognized by the prefix "i:ok_" and "i:fail_", which denote the value ST has to have for the instruction to execute.

The instructions are listed in the same order they occur in the *PackRat Machine Specification*, with the guard variants listed after their regular implementation, if any, or in their place.

objectName* i_input_next *msg
This method implements the PARAM instruction **input_next**.

***objectName* i_test_alnum**
This method implements the PARAM instruction **test_alnum**.

***objectName* i_test_alpha**
This method implements the PARAM instruction **test_alpha**.

***objectName* i_test_ascii**
This method implements the PARAM instruction **test_ascii**.

objectName* i_test_char *char
This method implements the PARAM instruction **test_char**.

***objectName* i_test_ddigit**
This method implements the PARAM instruction **test_ddigit**.

***objectName* i_test_digit**
This method implements the PARAM instruction **test_digit**.

***objectName* i_test_graph**
This method implements the PARAM instruction **test_graph**.

***objectName* i_test_lower**
This method implements the PARAM instruction **test_lower**.

***objectName* i_test_print**
This method implements the PARAM instruction **test_print**.

***objectName* i_test_punct**
This method implements the PARAM instruction **test_punct**.

objectName* i_test_range *chars chare
This method implements the PARAM instruction **test_range**.

***objectName* i_test_space**
This method implements the PARAM instruction **test_space**.

***objectName* i_test_upper**
This method implements the PARAM instruction **test_upper**.

***objectName* i_test_wordchar**
This method implements the PARAM instruction **test_wordchar**.

***objectName* i_test_xdigit**
This method implements the PARAM instruction **test_xdigit**.

***objectName* i_error_clear**
This method implements the PARAM instruction **error_clear**.

***objectName* i_error_push**
This method implements the PARAM instruction **error_push**.

***objectName* i_error_pop_merge**
This method implements the PARAM instruction **error_pop_merge**.

objectName* i_error_nonterminal *symbol
This method implements the PARAM instruction **error_nonterminal**.

***objectName* i_status_ok**
This method implements the PARAM instruction **status_ok**.

***objectName* i_status_fail**
This method implements the PARAM instruction **status_fail**.

***objectName* i_status_negate**
This method implements the PARAM instruction **status_negate**.

***objectName* i_loc_push**
This method implements the PARAM instruction **loc_push**.

***objectName* i_loc_pop_discard**
This method implements the PARAM instruction **loc_pop_discard**.

***objectName* i_loc_pop_rewind**

This method implements the PARAM instruction **loc_pop_rewind**.

***objectName* i:ok_loc_pop_rewind**
This guarded method, a variant of **i_loc_pop_rewind**, executes only for "ST == ok".

***objectName* i_loc_pop_rewind/discard**
This method is a convenient combination of control flow and the two PARAM instructions **loc_pop_rewind** and **loc_pop_discard**. The former is executed for "ST == fail", the latter for "ST == ok".

objectName* i_symbol_restore *symbol
This method implements the PARAM instruction **symbol_restore**.

The boolean result of the check is returned as the result of the method and can be used with standard Tcl control flow commands.

objectName* i_symbol_save *symbol
This method implements the PARAM instruction **symbol_save**.

***objectName* i_value_clear**
This method implements the PARAM instruction **value_clear**.

***objectName* i_value_clear/leaf**
This method is a convenient combination of control flow and the two PARAM instructions **value_clear** and **value_leaf**. The former is executed for "ST == fail", the latter for "ST == ok".

***objectName* i_value_clear/reduce**
This method is a convenient combination of control flow and the two PARAM instructions **value_clear** and **value_reduce**. The former is executed for "ST == fail", the latter for "ST == ok".

***objectName* i:ok_ast_value_push**
This method implements a guarded variant of the the PARAM instruction **ast_value_push**, which executes only for "ST == ok".

***objectName* i_ast_push**
This method implements the PARAM instruction **ast_push**.

***objectName* i_ast_pop_rewind**
This method implements the PARAM instruction **ast_pop_rewind**.

***objectName* i:fail_ast_pop_rewind**
This guarded method, a variant of **i_ast_pop_rewind**, executes only for "ST == fail".

***objectName* i_ast_pop_rewind/discard**
This method is a convenient combination of control flow and the two PARAM instructions **ast_pop_rewind** and **ast_pop_discard**. The former is executed for "ST == fail", the latter for "ST == ok".

***objectName* i_ast_pop_discard**
This method implements the PARAM instruction **ast_pop_discard**.

***objectName* i_ast_pop_discard/rewind**
This method is a convenient combination of control flow and the two PARAM instructions **ast_pop_discard** and **ast_pop_rewind**. The former is executed for "ST == fail", the latter for "ST == ok".

***objectName* i:ok_continue**

This guarded method executes only for "ST == ok". Then it aborts the current iteration of the innermost loop in the calling Tcl procedure.

***objectName* i:fail_continue**

This guarded method executes only for "ST == fail". Then it aborts the current iteration of the innermost loop in the calling Tcl procedure.

***objectName* i:fail_return**

This guarded method executes only for "ST == fail". Then it aborts the calling Tcl procedure.

***objectName* i:ok_return**

This guarded method executes only for "ST == ok". Then it aborts the calling Tcl procedure.

The next set of methods are *super instructions*, meaning that each implements a longer sequence of instructions commonly used in parsers. The combinated instructions of the previous set, i.e. those with names matching the pattern "i_*/*", are actually super instructions as well, albeit with limited scope, handling 2 instructions with their control flow. The upcoming set is much broader in scope, folding as much as six or more PARAM instructions into a single method call.

In this we can see the reasoning behind their use well:

1. By using less instructions the generated parsers become smaller, as the common parts are now truly part of the common runtime, and not explicitly written in the parser's code over and over again.
2. Using less instructions additionally reduces the overhead associated with calls into the runtime, i.e. the cost of method dispatch and of setting up the variable context.
3. Another effect of the super instructions is that their internals can be optimized as well, especially regarding control flow, and stack use, as the runtime internals are accessible to all instructions folded into the sequence.

***objectName* si:void_state_push**

This method combines

```
i_loc_push
i_error_clear
i_error_push
```

Parsers use it at the beginning of *void* sequences and choices with a *void* initial branch.

***objectName* si:void2_state_push**

This method combines

```
i_loc_push
i_error_clear
i_error_push
```

Parsers use it at the beginning of optional and repeated expressions.

***objectName* si:value_state_push**

This method combines

```
i_ast_push
i_loc_push
i_error_clear
i_error_push
```

Parsers use it at the beginning of sequences generating an AST and choices with an initial branch generating an AST.

***objectName* si:void_state_merge**

This method combines

```
i_error_pop_merge
i_loc_pop_rewind/discard
```

Parsers use it at the end of void sequences and choices whose last branch is void.

***objectName* si:void_state_merge_ok**

This method combines

```
i_error_pop_merge
i_loc_pop_rewind/discard
i_status_ok
```

Parsers use it at the end of optional expressions

***objectName* si:value_state_merge**

This method combines

```
i_error_pop_merge
i_ast_pop_rewind/discard
i_loc_pop_rewind/discard
```

Parsers use it at the end of sequences generating ASTs and choices whose last branch generates an AST

***objectName* si:value_notahead_start**

This method combines

```
i_loc_push
i_ast_push
```

Parsers use it at the beginning of negative lookahead predicates which generate ASTs.

***objectName* si:void_notahead_exit**

This method combines

```
i_loc_pop_rewind
i_status_negate
```

Parsers use it at the end of void negative lookahead predicates.

***objectName* si:value_notahead_exit**

This method combines

```
i_ast_pop_discard/rewind
i_loc_pop_rewind
i_status_negate
```

Parsers use it at the end of negative lookahead predicates which generate ASTs.

***objectName* si:kleene_abort**

This method combines

```
i_loc_pop_rewind/discard
i:fail_return
```

Parsers use it to stop a positive repetition when its first, required, expression fails.

***objectName* si:kleene_close**

This method combines

```
i_error_pop_merge
i_loc_pop_rewind/discard
i:fail_status_ok
i:fail_return
```

Parsers use it at the end of repetitions.

***objectName* si:voidvoid_branch**

This method combines

```
i_error_pop_merge
i:ok_loc_pop_discard
i:ok_return
i_loc_rewind
i_error_push
```

Parsers use it when transiting between branches of a choice when both are void.

***objectName* si:voidvalue_branch**

This method combines

```
i_error_pop_merge
i:ok_loc_pop_discard
i:ok_return
i_ast_push
i_loc_rewind
i_error_push
```

Parsers use it when transiting between branches of a choice when the failing branch is void, and the next to test generates an AST.

***objectName* si:valuevoid_branch**

This method combines

```
i_error_pop_merge
i_ast_pop_rewind/discard
i:ok_loc_pop_discard
i:ok_return
i_loc_rewind
i_error_push
```

Parsers use it when transiting between branches of a choice when the failing branch generates an AST, and the next to test is void.

***objectName* si:valuevalue_branch**

This method combines

```
i_error_pop_merge
i_ast_pop_discard
i:ok_loc_pop_discard
i:ok_return
i_ast_rewind
i_loc_rewind
i_error_push
```

Parsers use it when transiting between branches of a choice when both generate ASTs.

***objectName* si:voidvoid_part**

This method combines

```
i_error_pop_merge
i:fail_loc_pop_rewind
i:fail_return
i_error_push
```

Parsers use it when transiting between parts of a sequence and both are void.

***objectName* si:voidvalue_part**

This method combines

```
i_error_pop_merge
i:fail_loc_pop_rewind
i:fail_return
i_ast_push
i_error_push
```

Parsers use it when transiting between parts of a sequence and the sucessfully matched part is void, and after it an AST is generated.

***objectName* si:valuevalue_part**

This method combines

```
i_error_pop_merge
i:fail_ast_pop_rewind
i:fail_loc_pop_rewind
i:fail_return
i_error_push
```

Parsers use it when transiting between parts of a sequence and both parts generate ASTs.

objectName* si:value_symbol_start *symbol

This method combines

```
if/found? i_symbol_restore $symbol
i:found:ok_ast_value_push
i:found_return
i_loc_push
i_ast_push
```

Parsers use it at the beginning of a nonterminal symbol generating an AST, whose right-hand side may have generated an AST as well.

objectName* si:value_void_symbol_start *symbol

This method combines

```
if/found? i_symbol_restore $symbol
i:found:ok_ast_value_push
i:found_return
i_loc_push
i_ast_push
```

Parsers use it at the beginning of a void nonterminal symbol whose right-hand side may generate an AST.

objectName* si:void_symbol_start *symbol

This method combines

```
if/found? i_symbol_restore $symbol
i:found_return
i_loc_push
i_ast_push
```

Parsers use it at the beginning of a nonterminal symbol generating an AST whose right-hand side is void.

objectName* si:void_void_symbol_start *symbol

This method combines

```
if/found? i_symbol_restore $symbol
i:found_return
i_loc_push
```

Parsers use it at the beginning of a void nonterminal symbol whose right-hand side is void as

well.

objectName* si:reduce_symbol_end *symbol

This method combines

```
i_value_clear/reduce $symbol
i_symbol_save        $symbol
i_error_nonterminal  $symbol
i_ast_pop_rewind
i_loc_pop_discard
i:ok_ast_value_push
```

Parsers use it at the end of a non-terminal symbol generating an AST using the AST generated by the right-hand side as child.

objectName* si:void_leaf_symbol_end *symbol

This method combines

```
i_value_clear/leaf  $symbol
i_symbol_save       $symbol
i_error_nonterminal $symbol
i_loc_pop_discard
i:ok_ast_value_push
```

Parsers use it at the end of a non-terminal symbol generating an AST whose right-hand side is void.

objectName* si:value_leaf_symbol_end *symbol

This method combines

```
i_value_clear/leaf  $symbol
i_symbol_save       $symbol
i_error_nonterminal $symbol
i_loc_pop_discard
i_ast_pop_rewind
i:ok_ast_value_push
```

Parsers use it at the end of a non-terminal symbol generating an AST discarding the AST generated by the right-hand side.

objectName* si:value_clear_symbol_end *symbol

This method combines

```
i_value_clear
i_symbol_save       $symbol
i_error_nonterminal $symbol
i_loc_pop_discard
i_ast_pop_rewind
```

Parsers use it at the end of a void non-terminal symbol, discarding the AST generated by the right-hand side.

objectName* si:void_clear_symbol_end *symbol

This method combines

```
i_value_clear
i_symbol_save       $symbol
i_error_nonterminal $symbol
i_loc_pop_discard
```

Parsers use it at the end of a void non-terminal symbol with a void right-hand side.

objectName* si:next_char *tok

see below ...

objectName* si:next_range *toks toke

see below ...

***objectName* si:next_alnum**

see below ...

***objectName* si:next_alpha**

see below ...

***objectName* si:next_ascii**

see below ...

***objectName* si:next_ddigit**

see below ...

***objectName* si:next_digit**

see below ...

***objectName* si:next_graph**

see below ...

***objectName* si:next_lower**

see below ...

***objectName* si:next_print**

see below ...

***objectName* si:next_punct**

see below ...

***objectName* si:next_space**

see below ...

***objectName* si:next_upper**

see below ...

***objectName* si:next_wordchar**

see below ...

***objectName* si:next_xdigit**

These methods all combine

```
i_input_next $msg
i:fail_return
```

with the appropriate **i_test_xxx** instruction. Parsers use them for handling atomic expressions.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pt* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Parsing and Grammars

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_tclparam_config_nx

Keywords: pt_tclparam_config_nx, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt::tclparam::configuration::nx - Tcl/PARAM, Canned configuration, NX

Synopsis

package require Tcl 8.5 9
package require pt::tclparam::configuration::nx ?**1.0.2**?
::pt::tclparam::configuration::nx def *name pkg version cmdprefix*

Description

Are you lost ? Do you have trouble understanding this document ? In that case please read the overview provided by the *Introduction to Parser Tools*. This document is the entrypoint to the whole system the current package is a part of.

This package is an adjunct to pt::peg::to::tclparam, to make the use of this highly configurable package easier by providing a canned configuration. When applied this configuration causes the package pt::peg::to::tclparam to generate **NX**-based parser packages.

It is a supporting package in the Core Layer of Parser Tools.

IMAGE: arch_core_support

API

::pt::tclparam::configuration::nx def *name pkg version cmdprefix*

The command applies the configuration provided by this package to the *cmdprefix*, causing the creation of **NX**-based parsers whose class is *name*, in package *pkg* with *version*.

The use of a command prefix as API allows application of the configuration to not only

pt::peg::to::tclparam (pt::peg::to::tclparam configure), but also export manager instances and PEG containers (**$export configuration set** and **[$container exporter] configuration set** respectively).

Or anything other command prefix accepting two arguments, option and value.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pt* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Parsing and Grammars

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_tclparam_config_snit

Keywords: pt_tclparam_config_snit, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt::tclparam::configuration::snit - Tcl/PARAM, Canned configuration, Snit

Synopsis

package require Tcl 8.5 9
package require pt::tclparam::configuration::snit ?**1.0.3**?
::pt::tclparam::configuration::snit def *name pkg version cmdprefix*

Description

Are you lost ? Do you have trouble understanding this document ? In that case please read the overview provided by the *Introduction to Parser Tools*. This document is the entrypoint to the whole system the current package is a part of.

This package is an adjunct to pt::peg::to::tclparam, to make the use of this highly configurable package easier by providing a canned configuration. When applied this configuration causes the package pt::peg::to::tclparam to generate **snit**-based parser packages.

It is a supporting package in the Core Layer of Parser Tools.

IMAGE: arch_core_support

API

::pt::tclparam::configuration::snit def *name pkg version cmdprefix*

The command applies the configuration provided by this package to the *cmdprefix*, causing the creation of **snit**-based parsers whose class is *name*, in package *pkg* with *version*.

The use of a command prefix as API allows application of the configuration to not only pt::peg::to::tclparam (pt::peg::to::tclparam configure), but also export manager instances and PEG containers (**$export configuration set** and **[$container exporter] configuration set** respectively).

Or anything other command prefix accepting two arguments, option and value.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pt* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Parsing and Grammars

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_tclparam_config_tcloo

Keywords: pt_tclparam_config_tcloo, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt::tclparam::configuration::tcloo - Tcl/PARAM, Canned configuration, Tcloo

Synopsis

package require Tcl 8.5 9
package require pt::tclparam::configuration::tcloo ?**1.0.5**?
::pt::tclparam::configuration::tcloo def *name pkg version cmdprefix*

Description

Are you lost ? Do you have trouble understanding this document ? In that case please read the overview provided by the *Introduction to Parser Tools*. This document is the entrypoint to the whole system the current package is a part of.

This package is an adjunct to pt::peg::to::tclparam, to make the use of this highly configurable package easier by providing a canned configuration. When applied this configuration causes the package pt::peg::to::tclparam to generate **OO**-based parser packages.

It is a supporting package in the Core Layer of Parser Tools.

IMAGE: arch_core_support

API

::pt::tclparam::configuration::tcloo def *name pkg version cmdprefix*

The command applies the configuration provided by this package to the *cmdprefix*, causing the creation of **OO**-based parsers whose class is *name*, in package *pkg* with *version*.

The use of a command prefix as API allows application of the configuration to not only pt::peg::to::tclparam (pt::peg::to::tclparam configure), but also export manager instances and PEG containers (**$export configuration set** and **[$container exporter] configuration set** respectively).

Or anything other command prefix accepting two arguments, option and value.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pt* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Parsing and Grammars

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_to_api

Keywords: pt_to_api, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt_export_api - Parser Tools Export API

Synopsis

package require Tcl 8.5 9
CONVERTER reset

CONVERTER configure
CONVERTER configure *option*
CONVERTER configure *option value...*
CONVERTER convert *serial*
::export *serial configuration*

Description

Are you lost ? Do you have trouble understanding this document ? In that case please read the overview provided by the *Introduction to Parser Tools*. This document is the entrypoint to the whole system the current package is a part of.

This document describes two APIs. First the API shared by all packages for the conversion of Parsing Expression Grammars into some other format, and then the API shared by the packages which implement the export plugins sitting on top of the conversion packages.

Its intended audience are people who wish to create their own converter for some type of output, and/or an export plugin for their or some other converter.

It resides in the Export section of the Core Layer of Parser Tools.

IMAGE: arch_core_export

Converter API

Any (grammar) export converter has to follow the rules set out below:

1. A converter is a package. Its name is arbitrary, however it is recommended to put it under the **::pt::peg::to** namespace.
2. The package provides either a single Tcl command following the API outlined below, or a class command whose instances follow the same API. The commands which follow the API are called *converter commands*.
3. A converter command has to provide the following three methods with the given signatures and semantics. Converter commands are allowed to provide more methods of their own, but not less, and they may not provide different semantics for the standardized methods.

   CONVERTER reset
   : This method has to reset the configuration of the converter to its default settings. The result of the method has to be the empty string.

   CONVERTER configure
   : This method, in this form, has to return a dictionary containing the current configuration of the converter.

   CONVERTER configure *option*
   : This method, in this form, has to return the current value of the specified configuration *option* of the converter.

     Please read the section Options for the set of standard options any converter has to accept. Any other options accepted by a specific converter will be described in its manpage.

   CONVERTER configure *option value...*
   : This command, in this form, sets the specified *option*s of the converter to the given *value*s.

     Please read the section Options for the set of standard options a converter has to accept. Any other options accepted by a specific converter will be described in its manpage.

CONVERTER convert *serial*

This method has to accept the canonical serialization of a parsing expression grammar, as specified in section Peg serialization format, and contained in *serial*. The result of the method has to be the result of converting the input grammar into whatever the converter is for, per its configuration.

Plugin API

Any (grammar) export plugin has to follow the rules set out below:

1. A plugin is a package.
2. The name of a plugin package has the form pt::peg::export::**FOO**, where **FOO** is the name of the format the plugin will generate output for.
3. The plugin can expect that the package **pt::peg::export::plugin** is present, as indicator that it was invoked from a genuine plugin manager.

   It is recommended that a plugin does check for the presence of this package.
4. A plugin has to provide a single command, in the global namespace, with the signature shown below. Plugins are allowed to provide more command of their own, but not less, and they may not provide different semantics for the standardized command.

   ::export *serial configuration*

   This command has to accept the canonical serialization of a parsing expression grammar and the configuration for the converter invoked by the plugin. The result of the command has to be the result of the converter invoked by the plugin for th input grammar and configuration.

   string *serial*

   This argument will contain the *canonical* serialization of the parsing expression grammar for which to generate the output. The specification of what a *canonical* serialization is can be found in the section Peg serialization format.

   dictionary *configuration*

   This argument will contain the configuration to configure the converter with before invoking it, as a dictionary mapping from options to values.

   Please read the section Options for the set of standard options any converter has to accept, and thus any plugin as well. Any other options accepted by a specific plugin will be described in its manpage.

1. A single usage cycle of a plugin consists of an invokation of the command export. This call has to leave the plugin in a state where another usage cycle can be run without problems.

Options

Each export converter and plugin for an export converter has to accept the options below in their **configure** method. Converters are allowed to ignore the contents of these options when performing a conversion, but they must not reject them. Plugins are expected to pass the options given to them to the converter they are invoking.

-file string

The value of this option is the name of the file or other entity from which the grammar came, for which the command is run. The default value is **unknown**.

-name string
The value of this option is the name of the grammar we are processing. The default value is **a_pe_grammar**.

-user string
The value of this option is the name of the user for which the command is run. The default value is **unknown**.

Usage

To use a converter do

```
# Get the converter (single command here, not class)
package require the-converter-package

# Provide a configuration
theconverter configure ...

# Perform the conversion
set result [theconverter convert $thegrammarserial]

... process the result ...
```

To use a plugin **FOO** do

```
# Get an export plugin manager
package require pt::peg::export
pt::peg::export E

# Provide a configuration
E configuration set ...

# Run the plugin, and the converter inside.
set result [E export serial $grammarserial FOO]

... process the result ...
```

Peg serialization format

Here we specify the format used by the Parser Tools to serialize Parsing Expression Grammars as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a PEG may have more than one regular serialization only exactly one of them will be *canonical*.

regular serialization

1. The serialization of any PEG is a nested Tcl dictionary.
1. This dictionary holds a single key, **pt::grammar::peg**, and its value. This value holds the contents of the grammar.
2. The contents of the grammar are a Tcl dictionary holding the set of nonterminal symbols and the starting expression. The relevant keys and their values are
   rules

The value is a Tcl dictionary whose keys are the names of the nonterminal symbols known to the grammar.

1. Each nonterminal symbol may occur only once.
2. The empty string is not a legal nonterminal symbol.
3. The value for each symbol is a Tcl dictionary itself. The relevant keys and their values in this dictionary are

is
: The value is the serialization of the parsing expression describing the symbols sentennial structure, as specified in the section PE serialization format.

mode
: The value can be one of three values specifying how a parser should handle the semantic value produced by the symbol.

value
: The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal itself, which has the ASTs of the symbol's right hand side as its children.

leaf
: The semantic value of the nonterminal symbol is an abstract syntax tree consisting of a single node node for the nonterminal, without any children. Any ASTs generated by the symbol's right hand side are discarded.

void
: The nonterminal has no semantic value. Any ASTs generated by the symbol's right hand side are discarded (as well).

```
**start**
: The value is the serialization of the start parsing expression of
    the grammar, as specified in the section [PE serialization
    format].
```

1. The terminal symbols of the grammar are specified implicitly as the set of all terminal symbols used in the start expression and on the RHS of the grammar rules.

canonical serialization
: The canonical serialization of a grammar has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this grammar.

1. The keys found in all the nested Tcl dictionaries are sorted in ascending dictionary order, as generated by Tcl's builtin command **lsort -increasing -dict**.
2. The string representation of the value is the canonical representation of a Tcl dictionary. I.e. it does not contain superfluous whitespace.

Example

Assuming the following PEG for simple mathematical expressions

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

then its canonical serialization (except for whitespace) is

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                  mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}                mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                  mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                  mode value}
        MulOp      {is {/ {t *} {t /}}
                                  mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
                                  mode value}
        Sign       {is {/ {t -} {t +}}
                                  mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                  mode value}
    }
    start {n Expression}
}
```

PE serialization format

Here we specify the format used by the Parser Tools to serialize Parsing Expressions as immutable values for transport, comparison, etc.

We distinguish between *regular* and *canonical* serializations. While a parsing expression may have more than one regular serialization only exactly one of them will be *canonical*.

Regular serialization

see below ...

Atomic Parsing Expressions

1. The string **epsilon** is an atomic parsing expression. It matches the empty string.
1. The string **dot** is an atomic parsing expression. It matches any character.
2. The string **alnum** is an atomic parsing expression. It matches any Unicode alphabet or

digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.

3. The string **alpha** is an atomic parsing expression. It matches any Unicode alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
4. The string **ascii** is an atomic parsing expression. It matches any Unicode character below U0080. This is a custom extension of PEs based on Tcl's builtin command **string is**.
5. The string **control** is an atomic parsing expression. It matches any Unicode control character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
6. The string **digit** is an atomic parsing expression. It matches any Unicode digit character. Note that this includes characters outside of the [0..9] range. This is a custom extension of PEs based on Tcl's builtin command **string is**.
7. The string **graph** is an atomic parsing expression. It matches any Unicode printing character, except for space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
8. The string **lower** is an atomic parsing expression. It matches any Unicode lower-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
9. The string **print** is an atomic parsing expression. It matches any Unicode printing character, including space. This is a custom extension of PEs based on Tcl's builtin command **string is**.
10. The string **punct** is an atomic parsing expression. It matches any Unicode punctuation character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
11. The string **space** is an atomic parsing expression. It matches any Unicode space character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
12. The string **upper** is an atomic parsing expression. It matches any Unicode upper-case alphabet character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
13. The string **wordchar** is an atomic parsing expression. It matches any Unicode word character. This is any alphanumeric character (see alnum), and any connector punctuation characters (e.g. underscore). This is a custom extension of PEs based on Tcl's builtin command **string is**.
14. The string **xdigit** is an atomic parsing expression. It matches any hexadecimal digit character. This is a custom extension of PEs based on Tcl's builtin command **string is**.
15. The string **ddigit** is an atomic parsing expression. It matches any decimal digit character. This is a custom extension of PEs based on Tcl's builtin command **regexp**.
16. The expression [list t **x**] is an atomic parsing expression. It matches the terminal string **x**.
17. The expression [list n **A**] is an atomic parsing expression. It matches the nonterminal **A**.

Combined Parsing Expressions

1. For parsing expressions **e1**, **e2**, ... the result of [list / **e1 e2** ...] is a parsing expression as well. This is the *ordered choice*, aka *prioritized choice*.

1. For parsing expressions **e1**, **e2**, ... the result of [list x **e1 e2** ...] is a parsing expression as well. This is the *sequence*.
2. For a parsing expression **e** the result of [list * **e**] is a parsing expression as well. This is the *kleene closure*, describing zero or more repetitions.

3. For a parsing expression **e** the result of [list + **e**] is a parsing expression as well. This is the *positive kleene closure*, describing one or more repetitions.
4. For a parsing expression **e** the result of [list & **e**] is a parsing expression as well. This is the *and lookahead predicate*.
5. For a parsing expression **e** the result of [list! **e**] is a parsing expression as well. This is the *not lookahead predicate*.
6. For a parsing expression **e** the result of [list ? **e**] is a parsing expression as well. This is the *optional input*.

Canonical serialization

The canonical serialization of a parsing expression has the format as specified in the previous item, and then additionally satisfies the constraints below, which make it unique among all the possible serializations of this parsing expression.

1. The string representation of the value is the canonical representation of a pure Tcl list. I.e. it does not contain superfluous whitespace.
2. Terminals are *not* encoded as ranges (where start and end of the range are identical).

Example

Assuming the parsing expression shown on the right-hand side of the rule

```
    Expression <- Term (AddOp Term)*
```

then its canonical serialization (except for whitespace) is

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pt* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Parsing and Grammars

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_util

Keywords: pt_util, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt::util - General utilities

Synopsis

package require Tcl 8.5 9
package require pt::ast ?**1.2**?
::pt::util error2readable *error text*
::pt::util error2position *error text*
::pt::util error2text *error*

Description

Are you lost ? Do you have trouble understanding this document ? In that case please read the overview provided by the *Introduction to Parser Tools*. This document is the entrypoint to the whole system the current package is a part of.

This package provides general utility commands.

This is a supporting package in the Core Layer of Parser Tools.

IMAGE: arch_core_support

API

::pt::util error2readable *error text*

This command takes the structured form of a syntax *error* as thrown by parser runtimes and the input *text* to the parser which caused that error and returns a string describing the error in a human-readable form.

The input *text* is required to convert the character position of the error into a more readable line/column format, and to provide excerpts of the input around the error position.

::pt::util error2position *error text*

This command takes the structured form of a syntax *error* as thrown by parser runtimes and the input *text* to the parser which caused that error and returns a 2-element list containing the line number and column index for the error's character position in the input, in this order.

::pt::util error2text *error*

This command takes the structured form of a syntax *error* as thrown by parser runtimes and returns a list of strings, each describing a possible expected input in a human-readable form.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *pt* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Parsing and Grammars

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


rc4

rc4

Keywords: rc4, arcfour, data integrity, encryption, security, stream cipher

Name

rc4 - Implementation of the RC4 stream cipher

Synopsis

package require Tcl 8.5 9
package require rc4 ?**1.2.0**?
::rc4::rc4 ?*-hex*? *-key keyvalue* ?*-command lst*? ?*-out channel*? [*-in channel* | *-infile filename* | *string*]
::rc4::RC4Init *keydata*
::rc4::RC4 *Key data*
::rc4::RC4Final *Key*

Description

This package is an implementation in Tcl of the RC4 stream cipher developed by Ron Rivest of RSA Data Security Inc. The cipher was a trade secret of RSA but was reverse-engineered and published to the internet in 1994. It is used in a number of network protocols for securing communications. To evade trademark restrictions this cipher is sometimes known as ARCFOUR.

Commands

::rc4::rc4 ?*-hex*? *-key keyvalue* ?*-command lst*? ?*-out channel*? [*-in channel* | *-infile filename* | *string*]

Perform the RC4 algorithm on either the data provided by the argument or on the data read from the *-in* channel. If an *-out* channel is given then the result will be written to this channel. Giving the *-hex* option will return a hexadecimal encoded version of the result if not using an *-out* channel.

The data to be processes can be specified either as a string argument to the rc4 command, or as a filename or a pre-opened channel. If the *-infile* argument is given then the file is opened, the data read and processed and the file is closed. If the *-in* argument is given then data is read from the channel until the end of file. The channel is not closed. If the *-out* argument is given then the processing result is written to this channel.

If *-command* is provided then the rc4 command does not return anything. Instead the command provided is called with the rc4 result data appended as the final parameter. This is most useful when reading from Tcl channels as a fileevent is setup on the channel and the data processed in chunks

Only one of *-infile*, *-in* or *string* should be given.

Programming interface

::rc4::RC4Init *keydata*

Initialize a new RC4 key. The *keydata* is any amount of binary data and is used to initialize the cipher internal state.

::rc4::RC4 *Key data*

Encrypt or decrypt the input data using the key obtained by calling **RC4Init**.

::rc4::RC4Final *Key*

This should be called to clean up resources associated with *Key*. Once this function has been called the key is destroyed.

Examples

```
% set keydata [binary format H* 0123456789abcdef]
% rc4::rc4 -hex -key $keydata HelloWorld
3cf1ae8b7f1c670b612f
% rc4::rc4 -key $keydata [binary format H* 3cf1ae8b7f1c670b612f]
HelloWorld
```

```
set Key [rc4::RC4Init "key data"]
append ciphertext [rc4::RC4 $Key $plaintext]
append ciphertext [rc4::RC4 $Key $additional_plaintext]
rc4::RC4Final $Key
```

```
proc ::Finish {myState data} {
    DoStuffWith $myState $data
}
rc4::rc4 -in $socket -command [list ::Finish $ApplicationState]
```

Authors

Pat Thoyts

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *rc4* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Hashes, checksums, and encryption

Copyright

Copyright (c) 2003, Pat Thoyts <patthoyts@users.sourceforge.net>

rcs

rcs

Keywords: rcs, CVS, RCS patch, SCCS, diff -n format, patching, text conversion, text differences

Name

rcs - RCS low level utilities

Synopsis

package require Tcl 8.5 9
package require rcs ?**0.2**?
::rcs::text2dict *text*
::rcs::dict2text *dict*
::rcs::file2dict *filename*
::rcs::dict2file *filename dict*
::rcs::decodeRcsPatch *text*
::rcs::encodeRcsPatch *pcmds*
::rcs::applyRcsPatch *text pcmds*

Description

The *Revision Control System*, short *RCS*, is a set of applications and related data formats which allow a system to persist the history of changes to a text. It, and its relative SCCS are the basis for many other such systems, like *CVS*, etc.

This package *does not* implement RCS.

It only provides a number of low level commands which should be useful in the implementation of any revision management system, namely:

1. The conversion of texts into and out of a data structures which allow the easy modification of such text by *patches*, i.e. sequences of instructions for the transformation of one text into an other.
2. And the conversion of one particular format for patches, the so-called *RCS patches*, into and out of data structures which allow their easy application to texts.

Commands

::rcs::text2dict *text*
: Converts the argument *text* into a dictionary containing and representing the same text in an indexed form and returns that dictionary as its result. More information about the format of the result can be found in section Text dict data structure. This command returns the *canonical* representation of the input.

::rcs::dict2text *dict*
: This command provides the complementary operation to **::rcs::text2dict**. It converts a dictionary in the form described in section Text dict data structure back into a text and returns that text as its result. The command does accept non-canonical representations of the text as its input.

::rcs::file2dict *filename*
: This command is identical to **::rcs::text2dict**, except that it reads the text to convert from the file with path *filename*. The file has to exist and must be readable as well.

::rcs::dict2file *filename dict*
: This command is identical to **::rcs::2dict2text**, except that it stores the resulting text in the file with path *filename*. The file is created if it did not exist, and must be writable. The result of the command is the empty string.

::rcs::decodeRcsPatch *text*
: Converts the *text* argument into a patch command list (PCL) as specified in the section Rcs patch command list and returns this list as its result. It is assumed that the input text is in *diff -n format*, also known as *RCS patch* format, as specified in the section Rcs patch format. Please note that the command ignores no-ops in the input, in other words the resulting PCL contains only instructions doing something.

::rcs::encodeRcsPatch *pcmds*
: This command provides the complementary operation to **::rcs::decodeRcsPatch**. It convert a patch comand list (PCL) list as specified in the section Rcs patch command list back into a text in Rcs patch format and returns that text as its result.

  Note that this command and **::rcs::decodeRcsPatch** are not exactly complementary, as the latter strips no-ops from its input, which the encoder cannot put back anymore into the generated RCS patch. In other words, the result of a decode/encode step may not match the original input at the character level, but it will match it at the functional level.

::rcs::applyRcsPatch *text pcmds*
: This operation applies a patch in the form of a PCL to a text given in the form of a dictionary and returns the modified text, again as dictionary, as its result.

  To handle actual text use the commands **::rcs::text2dict** (or equivalent) and **::rcs::decodeRcsPatch** to transform the inputs into data structures acceptable to this command. Analogously use the command **::rcs::dict2text** (or equivalent) to transform the result of this command into actuall text as required.

Text dict data structure

A text dictionary is a dictionary whose keys are integer numbers and text strings as the associated values. The keys represent the line numbers of a text and the values the text of that line. Note that one text can have many representations as a dictionary, as the index values only have to be properly ordered for reconstruction, their exact values do not matter. Similarly the strings may actually span multiple physical lines.

The text

```
Hello World,
how are you ?
Fine, and you ?
```

for example can be represented by

```
{{1 {Hello World,}} {2 {how are you ?}} {3 {Fine, and you ?}}}
```

or

```
{{5 {Hello World,}} {8 {how are you ?}} {9 {Fine, and you ?}}}
```

or

```
{{-1 {Hello World,
how are you ?}} {4 {Fine, and you ?}}}
```

The first dictionary is the *canonical* representation of the text, with line numbers starting at **1**, increasing in steps of **1** and without gaps, and each value representing exactly one physical line.

All the commands creating dictionaries from text will return the canonical representation of their input text. The commands taking a dictionary and returning text will generally accept all representations, canonical or not.

The result of applying a patch to a text dictionary will in general cause the dictionary to become non-canonical.

Rcs patch format

A *patch* is in general a series of instructions how to transform an input text T into a different text T', and also encoded in text form as well.

The text format for patches understood by this package is a very simple one, known under the names *RCS patch* or *diff -n format*.

Patches in this format contain only two different commands, for the deletion of old text, and addition of new text. The replacement of some text by a different text is handled as combination of a deletion following by an addition.

The format is line oriented, with each line containing either a command or text data associated with the preceding command. The first line of a *RCS patch* is always a command line.

The commands are:

""

The empty line is a command which does nothing.

"astart n"

A line starting with the character **a** is a command for the addition of text to the output. It is followed by **n** lines of text data. When applying the patch the data is added just between the lines **start** and **start**+1. The same effect is had by appending the data to the existing text on line **start**. A non-existing line **start** is created.

"dstart n"

A line starting with the character **d** is a command for the deletion of text from the output. When applied it deletes **n** lines of text, and the first line deleted is at index **start**.

Note that the line indices **start** always refer to the text which is transformed as it is in its original state, without taking the precending changes into account.

Note also that the instruction have to be applied in the order they occur in the patch, or in a manner which produces the same result as in-order application.

This is the format of results returned by the command **::rcs::decodeRcsPatch** and accepted by the commands **::rcs::encodeRcsPatch** and **::rcs::appplyRcsPatch** resp. Note however that the decoder will strip no-op commands, and the encoder will not generate no-ops, making them

not fully complementary at the textual level, only at the functional level.

And example of a RCS patch is

```
d1 2
d4 1
a4 2
The named is the mother of all things.

a11 3
They both may be called deep and profound.
Deeper and more profound,
The door of all subtleties!
```

Rcs patch command list

Patch command lists (sort: PCL's) are the data structures generated by patch decoder command and accepted by the patch encoder and applicator commands. They represent RCS patches in the form of Tcl data structures.

A PCL is a list where each element represents a single patch instruction, either an addition, or a deletion. The elements are lists themselves, where the first item specifies the command and the remainder represent the arguments of the command.

a
: This is the instruction for the addition of text. It has two arguments, the index of the line where to add the text, and the text to add, in this order.

d
: This is the instruction for the deletion of text. It has two arguments, the index of the line where to start deleting text, and the number of lines to delete, in this order.

This is the format returned by the patch decoder command and accepted as input by the patch encoder and applicator commands.

An example for a patch command is shown below, it represents the example RCS patch found in section Rcs patch format.

```
{{d 1 2} {d 4 1} {a 4 {The named is the mother of all things.

}} {a 11 {They both may be called deep and profound.
Deeper and more profound,
The door of all subtleties!}}}
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *rcs* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Text processing

Copyright

```
Copyright (c) 2005, Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
Copyright (c) 2005, Colin McCormack <coldstore@users.sourceforge.net>
```


report

report
Keywords: report, matrix, table

Name
report - Create and manipulate report objects

Synopsis
package require Tcl 8.5 9
package require report ?**0.5**?
::report::report *reportName columns* ?*style style arg...*?
reportName *option* ?*arg arg ...*?
::report::defstyle *styleName arguments script*
::report::rmstyle *styleName*
::report::stylearguments *styleName*
::report::stylebody *styleName*
::report::styles
reportName **destroy**
reportName **templatecode disable|enable**
reportName **templatecode enabled**
reportName **templatecode get**
reportName **templatecode set** *templatedata*
reportName **tcaption** ?*size*?
reportName **bcaption** *size*
reportName **size** *column* ?*number|dyn*?
reportName **sizes** ?*size-list*?
reportName **pad** *column* ?*left|right|both padstring*?
reportName **justify** *column* ?**left|right|center**?
reportName **printmatrix** *matrix*
reportName **printmatrix2channel** *matrix chan*
reportName **columns**

Description
This package provides report objects which can be used by the formatting methods of matrix objects to generate tabular reports of the matrix in various forms. The report objects defined here break each report down into three Regions and ten classes of *lines* (various separator- and data-lines). See the following section for more detailed explanations.

::report::report *reportName columns* ?style *style arg...*?
Creates a new report object for a report having *columns* columns with an associated global Tcl command whose name is *reportName*. This command may be used to invoke various configuration operations on the report. It has the following general form:

reportName *option* ?*arg arg ...*?
Option and the *arg*s determine the exact behavior of the command. See section Report methods for more explanations. If no **style** is specified the report will use the builtin style **plain** as its default configuration.

::report::defstyle *styleName arguments script*
Defines the new style *styleName*. See section Styles for more information.

::report::rmstyle *styleName*
Deletes the style *styleName*. Trying to delete an unknown or builtin style will result in an error. Beware, this command will not check that there are no other styles depending on the deleted one. Deleting a style which is still used by another style FOO will result in a runtime error when FOO is applied to a newly instantiated report.

::report::stylearguments *styleName*
This introspection command returns the list of arguments associated with the style *styleName* .

::report::stylebody *styleName*
This introspection command returns the script associated with the style *styleName*.

::report::styles
This introspection command returns a list containing the names of all styles known to the package at the time of the call. The order of the names in the list reflects the order in which the styles were created. In other words, the first item is the predefined style **plain**, followed by the first style defined by the user, and so on.

Regions

The three regions are the *top caption*, *data area* and *bottom caption*. These are, roughly speaking, the title, the values to report and a title at the bottom. The size of the caption regions can be specified by the user as the number of rows they occupy in the matrix to format. The size of the data area is specified implicitly.

Lines

Templates are associated with each of the ten line classes, defining the formatting for this kind of line. The user is able to enable and disable the separator lines at will, but not the data lines. Their usage is solely determined by the number of rows contained in the three regions. Data lines and all enabled separators must have a template associated with them.

Note that the data-lines in a report and the rows in the matrix the report was generated from are *not* in a 1:1 relationship if any row in the matrix has a height greater than one.

The different kinds of lines and the codes used by the report methods to address them are:

top
The topmost line of a report. Separates the report from anything which came before it. The user can enable the usage of this line at will.

topdatasep

This line is used to separate the data rows in the top caption region, if it contains more than one row and the user enabled its usage.

topcapsep
This line is used to separate the top caption and data regions, if the top caption is not empty and the user enabled its usage.

datasep
This line is used to separate the data rows in the data region, if it contains more than one row and the user enabled its usage.

botcapsep
This line is used to separate the data and bottom caption regions, if the bottom caption is not empty and the user enabled its usage.

botdatasep
This line is used to separate the data rows in the bottom caption region, if it contains more than one row and the user enabled its usage.

bottom
The bottommost line of a report. Separates the report from anything which comes after it. The user can enable the usage of this line at will.

topdata
This line defines the format of data lines in the top caption region of the report.

data
This line defines the format of data lines in the data region of the report.

botdata
This line defines the format of data lines in the bottom caption region of the report.

Templates

Each template is a list of strings used to format the line it is associated with. For a report containing **n** columns a template for a data line has to contain "**n**+1" items and a template for a separator line "2*__n__+1" items.

The items in a data template specify the strings used to separate the column information. Together with the corresponding items in the separator templates they form the vertical lines in the report.

Note that the corresponding items in all defined templates have to be of equal length. This will be checked by the report object. The first item defines the leftmost vertical line and the last item defines the rightmost vertical line. The item at index **k** ("1",...,"**n**-2") separates the information in the columns "**k**-1" and "**k**".

The items in a separator template having an even-numbered index ("0","2",...) specify the column separators. The item at index "2*__k__" ("0","2",...,"2*__n__") corresponds to the items at index "__k__" in the data templates.

The items in a separator template having an odd-numbered index ("1","3",...) specify the strings used to form the horizontal lines in the separator lines. The item at index "2*__k__+1" ("1","3",...,"2*__n__+1") corresponds to column "**k**". When generating the horizontal lines the items are replicated to be at least as long as the size of their column and then cut to the exact size.

Styles

Styles are a way for the user of this package to define common configurations for report objects and then use them later during the actual instantiation of report objects. They are defined as tcl scripts which when executed configure the report object into the requested configuration.

The command to define styles is **::report::defstyle**. Its last argument is the tcl **script** performing the actual reconfiguration of the report object to obtain the requested style.

In this script the names of all previously defined styles are available as commands, as are all commands found in a safe interpreter and the configuration methods of report objects. The latter implicitly operate on the object currently executing the style script. The **arguments** declared here are available in the **script** as variables. When calling the command of a previously declared style all the arguments expected by it have to be defined in the call.

Report methods

The following commands are possible for report objects:

***reportName* destroy**
: Destroys the report, including its storage space and associated command.

***reportName templatecode* disable|enable**
: Enables or disables the usage of the template addressed by the *templatecode*. Only the codes for separator lines are allowed here. It is not possible to enable or disable data lines.

  Enabling a template causes the report to check all used templates for inconsistencies in the definition of the vertical lines (See section Templates).

***reportName templatecode* enabled**
: Returns the whether the template addressed by the *templatecode* is currently enabled or not.

***reportName templatecode* get**
: Returns the template currently associated with the kind of line addressed by the *templatecode*. All known templatecodes are allowed here.

reportName templatecode* set *templatedata
: Sets the template associated with the kind of line addressed by the *templatecode* to the new value in *templatedata*. See section Templates for constraints on the length of templates.

***reportName* tcaption ?*size*?**
: Specifies the *size* of the top caption region as the number rows it occupies in the matrix to be formatted. Only numbers greater than or equal to zero are allowed. If no *size* is specified the command will return the current size instead.

  Setting the size of the top caption to a value greater than zero enables the corresponding data template and causes the report to check all used templates for inconsistencies in the definition of the vertical lines (See section Templates).

reportName* bcaption *size
: Specifies the *size* of the bottom caption region as the number rows it occupies in the matrix to be formatted. Only numbers greater than or equal to zero are allowed. If no *size* is specified the command will return the current size instead.

  Setting the size of the bottom caption to a value greater than zero enables the corresponding data template and causes the report to check all used templates for inconsistencies in the definition of the vertical lines (See section Templates).

***reportName* size *column* ?*number*|dyn?**
Specifies the size of the *column* in the output. The value **dyn** means that the columnwidth returned by the matrix to be formatted for the specified column shall be used. The formatting of the column is dynamic. If a fixed *number* is used instead of **dyn** it means that the column has a width of that many characters (padding excluded). Only numbers greater than zero are allowed here.

If no size specification is given the command will return the current size of the *column* instead.

***reportName* sizes ?*size-list*?**
This method allows the user to specify the sizes of all columns in one call. Its argument is a list containing the sizes to associate with the columns. The first item is associated with column 0, the next with column 1, and so on.

If no *size-list* is specified the command will return a list containing the currently set sizes instead.

***reportName* pad *column* ?left|right|both ?*padstring*??**
This method allows the user to specify padding on the left, right or both sides of a *column*. If the *padstring* is not specified it defaults to a single space character. *Note*: An alternative way of specifying the padding is to use vertical separator strings longer than one character in the templates (See section Templates).

If no pad specification is given at all the command will return the current state of padding for the column instead. This will be a list containing two elements, the first element the left padding, the second describing the right padding.

***reportName* justify *column* ?left|right|center?**
Declares how the cell values for a *column* are filled into the report given the specified size of a column in the report.

For **left** and **right** justification a cell value shorter than the width of the column is bound with its named edge to the same edge of the column. The other side is filled with spaces. In the case of **center** the spaces are placed to both sides of the value and the left number of spaces is at most one higher than the right number of spaces.

For a value longer than the width of the column the value is cut at the named edge. This means for **left** justification that the *tail* (i.e. the **right** part) of the value is made visible in the output. For **center** the value is cut at both sides to fit into the column and the number of characters cut at the left side of the value is at most one less than the number of characters cut from the right side.

If no justification was specified the command will return the current justification for the column instead.

reportName* printmatrix *matrix
Formats the *matrix* according to the configuration of the report and returns the resulting string. The matrix has to have the same number of columns as the report. The matrix also has to have enough rows so that the top and bottom caption regions do not overlap. The data

region is allowed to be empty.

reportName* printmatrix2channel *matrix chan
Formats the *matrix* according to the configuration of the report and writes the result into the channel *chan*. The matrix has to have the same number of columns as the report. The matrix also has to have enough rows so that the top and bottom caption regions do not overlap. The data region is allowed to be empty.

***reportName* columns**
Returns the number of columns in the report.

The methods **size**, **pad** and **justify** all take a column index as their first argument. This index is allowed to use all the forms of an index as accepted by the **lindex** command. The allowed range for indices is "0,...,[**reportName** columns]-1".

Examples

Our examples define some generally useful report styles.

A simple table with lines surrounding all information and vertical separators, but without internal horizontal separators.

```
::report::defstyle simpletable {} {
data    set [split "[string repeat "| "   [columns]]|"]
top set [split "[string repeat "+ - " [columns]]+"]
bottom  set [top get]
top enable
bottom  enable
}
```

An extension of a **simpletable**, see above, with a title area.

```
::report::defstyle captionedtable {{n 1}} {
simpletable
topdata   set [data get]
topcapsep set [top get]
topcapsep enable
tcaption $n
}
```

Given the definitions above now an example which actually formats a matrix into a tabular report. It assumes that the matrix actually contains useful data.

```
% ::struct::matrix m
% # ... fill m with data, assume 5 columns
% ::report::report r 5 style captionedtable 1
% r printmatrix m
+---+------------------+-------+-------+--------+
|000|VERSIONS:         |2:8.4a3|1:8.4a3|1:8.4a3%|
+---+------------------+-------+-------+--------+
|001|CATCH return ok   |7      |13     |53.85   |
|002|CATCH return error |68     |91     |74.73   |
```

```
    |003|CATCH no catch used|7      |14     |50.00   |
    |004|IF if true numeric |12     |33     |36.36   |
    |005|IF elseif          |15     |47     |31.91   |
    |   |true numeric       |       |       |        |
    +---+-------------------+-------+-------+--------+
    %
    % # alternate way of doing the above
    % m format 2string r
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *report* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Data structures

Copyright


```
Copyright (c) 2002-2014,2022 Andreas Kupries
    <andreas.kupries@gmail.com>
```


rest

rest

Keywords: rest

Name

rest - define REST web APIs and call them inline or asychronously

Synopsis

package require Tcl 8.5 9
package require rest ?**1.7**?
::rest::simple *url query* ?*config*? ?*body*?
::rest::get *url query* ?*config*? ?*body*?
::rest::post *url query* ?*config*? ?*body*?
::rest::patch *url query* ?*config*? ?*body*?
::rest::head *url query* ?*config*? ?*body*?
::rest::put *url query* ?*config*? ?*body*?
::rest::delete *url query* ?*config*? ?*body*?
::rest::save *name file*
::rest::describe *name*

::rest::parameters *url ?key?*
::rest::parse_opts *static required optional words*
::rest::substitute *string var*
::rest::create_interface *name*

Description

There are two types of usage this package supports: *simple calls*, and complete *interfaces*. In an *interface* you specify a set of rules and then the package builds the commands which correspond to the REST methods. These commands can have many options such as input and output transformations and data type specific formatting. This results in a cleaner and simpler script. On the other hand, while a *simple call* is easier and quicker to implement it is also less featureful. It takes the url and a few options about the command and returns the result directly. Any formatting or checking is up to rest of the script.

Simple usage

In simple usage you make calls using the http method procedures and then check or process the returned data yourself

::rest::simple *url query ?config? ?body?*
see below ...

::rest::get *url query ?config? ?body?*
see below ...

::rest::post *url query ?config? ?body?*
see below ...

::rest::patch *url query ?config? ?body?*
see below ...

::rest::head *url query ?config? ?body?*
see below ...

::rest::put *url query ?config? ?body?*
see below ...

::rest::delete *url query ?config? ?body?*
These commands are all equivalent except for the http method used. If you use **simple** then the method should be specified as an option in the *config* dictionary. If that is not done it defaults to **get**. If a *body* is needed then the *config* dictionary must be present, however it is allowed to be empty.

The *config* dictionary supports the following keys
- **auth**
- **content-type**
- **cookie**
- **error-body**
- **format**
- **headers**
- **method**
- **timeout**

Two quick examples:

Example 1, Yahoo Boss:

```
    set appid APPID
    set search tcl
    set res [rest::get
    http://boss.yahooapis.com/ysearch/web/v1/$search [list appid
    $appid]]
    set res [rest::format_json $res]
```

Example 2, Twitter:

```
    set url   http://twitter.com/statuses/update.json
    set query [list status $text]
    set res [rest::simple $url $query {
        method post
        auth   {basic user password}
        format json
    }]
```

Interface usage

An interface to a REST API consists of a series of definitions of REST calls contained in an array. The name of that array becomes a namespace containing the defined commands. Each key of the array specifies the name of the call, with the associated configuration a dictionary, i.e. key/value pairs. The acceptable keys, i.e. legal configuration options are described below. After creating the definitions in the array simply calling **rest::create_interface** with the array as argument will then create the desired commands.

Example, Yahoo Weather:

```
   package require rest

   set yweather(forecast) {
      url      http://weather.yahooapis.com/forecastrss
      req_args { p: }
      opt_args { u: }
   }
   rest::create_interface yweather
   puts [yweather::forecast -p 94089]
```

::rest::save *name file*

This command saves a copy of the dynamically created procedures for all the API calls specified in the array variable *name* to the *file*, for later loading.

The result of the command is the empty string

::rest::describe *name*

This command prints a description of all API calls specified in the array variable *name* to the channel **stdout**.

The result of the command is the empty string.

::rest::parameters *url* ?*key*?

This command parses an *url* query string into a dictionary and returns said dictionary as its result.

If *key* is specified the command will not return the entire dictionary, but only the value of that *key*.

::rest::parse_opts *static required optional words*

This command implements a custom parserfor command options.

dict *static*

A dictionary of options and their values that are always present in the output.

list *required*

A list of options that must be supplied by *words*

list *optional*

A list of options that may appear in the *words*, but are not required. The elements must be in one of three forms:

name

The option may be present or not, no default.

name:

When present the option requires an argument.

name:value

When not present use **value** as default.

list *words*

The words to parse into options and values.

The result of the command is a list containing two elements. The first element is a dictionary containing the parsed options and their values. The second element is a list of the remaining words.

::rest::substitute *string var*

This command takes a *string*, substitutes values for any option identifiers found inside and returns the modified string as its results.

The values to substitute are found in the variable *var*, which is expected to contain a dictionary mapping from the option identifiers to replace to their values. *Note* that option identifiers which have no key in *var* are replaced with the empty string.

The option identifiers in *string* have to follow the syntax **%...%** where**...** may contain any combination of lower-case alphanumeric characters, plus underscore, colon and dash.

::rest::create_interface *name*

This command creates procedures for all the API calls specified in the array variable *name*.

The name of that array becomes a namespace containing the defined commands. Each key of the array specifies the name of the call, with the associated configuration a dictionary, i.e. key/value pairs. The legal keys and their meanings are:

url

The value of this *required* option must be the target of the http request.

description
The value of this option must be a short string describing the call. Default to the empty string, if not specified. Used only by **::rest::describe**.

body
The value of this option indicates if arguments are required for the call's request body or not. The acceptable values are listed below. Defaults to **optional** if not specified.

none
The call has no request body, none must be supplied.

optional
A request body can be supplied, but is not required.

required
A request body must be supplied.

argument
This value must be followed by the name of an option, treating the entire string as a list. The request body will be used as the value of that option.

mime_multipart
A request body must be supplied and will be interpreted as each argument representing one part of a mime/multipart document. Arguments must be lists containing 2 elements, a list of header keys and values, and the mime part body, in this order.

mime_multipart/
Same as mime_multipart, but the **Content-Type** header is set to **multipart/**.

method
The value of this option must be the name of the HTTP method to call on the url. Defaults to GET, if not specified. The acceptable values are **GET**, **POST**, and **PUT**, regardless of letter-case.

copy
When present the value of this option specifies the name of a previously defined call. The definition of that call is copied to the current call, except for the options specified by the current call itself.

unset
When present the value of this option contains a list of options in the current call. These options are removed from the definition. Use this after **copy**ing an existing definition to remove options, instead of overriding their value.

headers
Specification of additional header fields. The value of this option must be a dictionary, interpreted to contain the new header fields and their values. The default is to not add any additional headers.

content-type
The value of this option specifies the content type for the request data.

req_args
The value of this option is a list naming the required arguments of the call. Names ending in a colon will require a value.

opt_args

The value of this option a list naming the arguments that may be present for a call but are not required.

static_args
The value of this option a list naming the arguments that are always the same. No sense in troubling the user with these. A leading dash (**-**) is allowed but not required to maintain consistency with the command line.

auth
The value of this option specifies how to authenticate the calls. No authentication is done if the option is not specified.

basic
The user may configure the *basic authentication* by overriding the procedure **basic_auth** in the namespace of interface. This procedure takes two arguments, the username and password, in this order.

bearer
The user may configure a bearer token as authentication. The value is the token passed to the HTTP authorization header.

sign
The value must actually be a list with the second element the name of a procedure which will be called to perform request signing.

callback
If this option is present then the method will be created as an *async* call. Such calls will return immediately with the value of the associated http token instead of the call's result. The event loop must be active to use this option.

The value of this option is treated as a command prefix which is invoked when the HTTP call is complete. The prefix will receive at least two additional arguments, the name of the calling procedure and the status of the result (one of **OK** or **ERROR**), in this order.

In case of **OK** a third argument is added, the data associated with the result.

If and only if the **ERROR** is a redirection, the location redirected to will be added as argument. Further, if the configuration key **error-body** is set to **true** the data associated with the result will be added as argument as well.

The http request header will be available in that procedure via **upvar token token**.

cookie
The value of this option is a list of cookies to be passed in the http header. This is a shortcut to the **headers** option.

input_transform
The value of this option is a command prefix or script to perform a transformation on the query before invoking the call. A script transform is wrapped into an automatically generated internal procedure.

If not specified no transformation is done.

The command (prefix) must accept a single argument, the query (a dictionary) to transform, and must return the modified query (again as dictionary) as its result. The request body is accessible in the transform command via **upvar body body**.

format
see below ...

result
The value of this option specifies the format of the returned data. Defaults to **auto** if not specified. The acceptable values are:

auto
Auto detect between **xml** and **json**.

discard
see below ...

json
see below ...

raw
see below ...

rss
This is formatted as a special case of **xml**.

- **tdom**
- **xml**

pre_transform
The value of this option is a command prefix or script to perform a transformation on the result of a call (*before* the application of the output transform as per **format**). A script transform is wrapped into an automatically generated internal procedure.

If not specified no transformation is done.

The command (prefix) must accept a single argument, the result to transform, and must return the modified result as its result.

The http request header is accessible in the transform command via **upvar token token**

post_transform
The value of this option is a command prefix or script to perform a transformation on the result of a call (*after* the application of the output transform as per **format**). A script transform is wrapped into an automatically generated internal procedure.

If not specified no transformation is done.

The command (prefix) must accept a single argument, the result to transform, and must return the modified result as its result.

The http request header is accessible in the transform command via **upvar token token**

check_result
The value of this option must be list of two expressions, either of which may be empty.

The first expression is checks the OK condition, it must return **true** when the result is satisfactory, and **false** otherwise.

The second expression is the ERROR condition, it must return **false** unless there is an error, then it has to return **true**.

error_body
The value of this option determines whether to return the response when encountering an HTTP error, or not. The default is to not return the response body on error.

See **callback** above for more information.

timeout
The value of this option determines whether to set a timeout on the HTTP call. By default, no timeout is set.

Timeout value is accepted in milliseconds.

Examples

Yahoo Geo:

```
set ygeo(parse) {
    url http://wherein.yahooapis.com/v1/document
    method post
    body { arg documentContent }
}
ygeo::parse "san jose ca"
# "san jose ca" will be interpreted as if it were specified as the
    -documentContent option
```

Google Docs:

```
set gdocs(upload) {
    url http://docs.google.com/feeds/default/private/full
    body mime_multipart
}
gdocs::upload [list {Content-Type application/atom+xml} $xml] [list
    {Content-Type image/jpeg} $filedata]
```

Delicious:

```
set delicious(updated) {
    url https://api.del.icio.us/v1/posts/update
    auth basic
}

rest::create_interface flickr

flickr::basic_auth username password
```

Flickr:

```
set flickr(auth.getToken) {
   url http://api.flickr.com/services/rest/
   req_args { api_key: secret: }
   auth { sign do_signature }
}

rest::create_interface flickr

proc ::flickr::do_signature {query} {
    # perform some operations on the query here
    return $query
}
```

Included

The package provides functional but incomplete implementations for the following services:

- **del.icio.us**
- **facebook**
- **flickr**
- **twitter**
- **google calendar**
- **yahoo boss**
- **yahoo weather**

Please either read the package's implementation, or use **rest::describe** after loading it for their details.

Do not forget developers' documentation on the respective sites either.

Tls

The **rest** package can be used with *https*-secured services, by requiring the Tls package and then registering it with the **http** package it is sitting on top of. Example

```
    package require tls
    http::register https 443 ::tls::socket
```

Tls security considerations

This package uses the Tls package to handle the security for **https** urls and other socket connections.

Policy decisions like the set of protocols to support and what ciphers to use are not the responsibility of Tls, nor of this package itself however. Such decisions are the responsibility of whichever application is using the package, and are likely influenced by the set of servers the application will talk to as well.

For example, in light of the recent POODLE attack discovered by Google many servers will disable support for the SSLv3 protocol. To handle this change the applications using Tls must be patched, and not this package, nor Tls itself. Such a patch may be as simple as generally activating **tls1** support, as shown in the example below.

```
    package require tls
    tls::init -tls1 1 ;# forcibly activate support for the TLS1
    protocol

    ... your own application code ...
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *rest* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

ripemd

ripemd128

Keywords: ripemd128, RIPEMD, hashing, md4, message-digest, rfc 1320, rfc 1321, rfc 2104, security

Name

ripemd128 - RIPEMD-128 Message-Digest Algorithm

Synopsis

package require Tcl 8.5 9
package require ripemd128 ?**1.0.6**?
::ripemd::ripemd128 ?*-hex*? [*-channel channel* | *-file filename* | *string*]
::ripemd::hmac128 ?*-hex*? *-key key* [*-channel channel* | *-file filename* | *string*]
::ripemd::RIPEMD128Init
::ripemd::RIPEMD128Update *token data*
::ripemd::RIPEMD128Final *token*
::ripemd::RIPEHMAC128Init *key*
::ripemd::RIPEHMAC128Update *token data*
::ripemd::RIPEHMAC128Final *token*

Description

This package is an implementation in Tcl of the RIPEMD-128 message-digest algorithm (1). This algorithm takes an arbitrary quantity of data and generates a 128-bit message digest from the input. The RIPEMD-128 algorithm is based upon the MD4 algorithm (2, 4) but has been cryptographically strengthened against weaknesses that have been found in MD4 (4). RIPEMD-128 has been designed to be a drop-in replacement for MD4 and MD5 (5). If security is the major consideration, then RIPEMD-160 or SHA1 should be considered.

This package will use **Trf** to accelerate the digest computation if available. In the absence of an accelerator package the pure-Tcl implementation will be used.

BEWARE The commands in this package expect binary data as their input. When a **-file** is provided then this is ensured by the commands themselves, as they open the referenced file in binary mode. When literal data, or a **-channel** are provided instead, then the command's *caller is responsible* for ensuring this fact. The necessary conversion command is **encoding convertto utf-8 $string**.

ATTENTION, there is a *Tcl 9 COMPATIBILITY ISSUE* here.

Tcl 8.x silently mishandles non-binary input by cutting it internally to size. I.e. by only using the low byte of a higher unicode point. No error is thrown, the *result is simply wrong*.

Tcl 9 throws an error instead, i.e.

```
expected byte sequence but character <location> was ...
```

Commands

::ripemd::ripemd128 ?*-hex*? [*-channel channel* | *-file filename* | *string*]

Calculate the RIPEMD-128 digest of the data given in string. This is returned as a binary string by default. Giving the *-hex* option will return a hexadecimal encoded version of the digest.

The data to be hashed can be specified either as a string argument to the ripemd128 command, or as a filename or a pre-opened channel. If the *-filename* argument is given then the file is opened, the data read and hashed and the file is closed. If the *-channel* argument is given then data is read from the channel until the end of file. The channel is not closed.

Only one of *-file*, *-channel* or *string* should be given.

::ripemd::hmac128 ?*-hex*? *-key key* [*-channel channel* | *-file filename* | *string*]

Calculate an Hashed Message Authentication digest (HMAC) using the RIPEMD-128 digest algorithm. HMACs are described in RFC 2104 (6) and provide a RIPEMD-128 digest that includes a key. All options other than *-key* are as for the **::ripemd::ripemd128** command.

Programming interface

For the programmer, hash functions can be viewed as a bucket into which one pours data. When you have finished, you extract a value that is uniquely derived from the data that was poured into the bucket. The programming interface to the hash operates on a token (equivalent to the bucket). You call **RIPEMD128Init** to obtain a token and then call **RIPEMD128Update** as many times as required to add data to the hash. To release any resources and obtain the hash value, you then call **RIPEMD128Final**. An equivalent set of functions gives you a keyed digest (HMAC).

If you have **critcl** and have built the **tcllibc** package then the implementation of the hashing function will be performed by compiled code. Alternatively if both the Trf and Memchan extensions are available then these will be used. Finally the package will revert to a pure-Tcl implementation. The programming interface remains the same, however.

::ripemd::RIPEMD128Init

Begins a new RIPEMD-128 hash. Returns a token ID that must be used for the remaining functions.

::ripemd::RIPEMD128Update *token data*

Add data to the hash identified by token. Calling *RIPEMD128Update $token* "*abcd*" is equivalent to calling *RIPEMD128Update $token* "*ab*" followed by *RIPEMD128Update $token*

"*cb*". See Examples.

::ripemd::RIPEMD128Final *token*
Returns the hash value and releases any resources held by this token. Once this command completes the token will be invalid. The result is a binary string of 16 bytes representing the 128 bit RIPEMD-128 digest value.

::ripemd::RIPEHMAC128Init *key*
This is equivalent to the **::ripemd::RIPEMD128Init** command except that it requires the key that will be included in the HMAC.

::ripemd::RIPEHMAC128Update *token data*
see below ...

::ripemd::RIPEHMAC128Final *token*
These commands are identical to the RIPEMD128 equivalent commands.

Examples

```
% ripemd::ripemd128 -hex "Tcl does RIPEMD-128"
3cab177bae65205d81e7978f63556c63
```

```
% ripemd::hmac128 -hex -key Sekret "Tcl does RIPEMD-128"
b359dc5971a05beea0be7b106b30e389
```

```
% set tok [ripemd::RIPEMD128Init]
::ripemd::1
% ripemd::RIPEMD128Update $tok "Tcl "
% ripemd::RIPEMD128Update $tok "does "
% ripemd::RIPEMD128Update $tok "RIPEMD-128"
% ripemd::Hex [ripemd::RIPEMD128Final $tok]
3cab177bae65205d81e7978f63556c63
```

References

1. H. Dobbertin, A. Bosselaers, B. Preneel, "RIPEMD-160, a strengthened version of RIPEMD" http://www.esat.kuleuven.ac.be/~cosicart/pdf/AB-9601/AB-9601.pdf
2. Rivest, R., "The MD4 Message Digest Algorithm", RFC 1320, MIT, April 1992. (http://www.rfc-editor.org/rfc/rfc1320.txt)
3. Rivest, R., "The MD4 message digest algorithm", in A.J. Menezes and S.A. Vanstone, editors, Advances in Cryptology - CRYPTO '90 Proceedings, pages 303-311, Springer-Verlag, 1991.
4. Dobbertin, H., "Cryptanalysis of MD4", Journal of Cryptology vol 11 (4), pp. 253-271 (1998)
5. Rivest, R., "The MD5 Message-Digest Algorithm", RFC 1321, MIT and RSA Data Security, Inc, April 1992. (http://www.rfc-editor.org/rfc/rfc1321.txt)
6. Krawczyk, H., Bellare, M. and Canetti, R. "HMAC: Keyed-Hashing for Message Authentication", RFC 2104, February 1997. (http://www.rfc-editor.org/rfc/rfc2104.txt)

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *ripemd* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Hashes, checksums, and encryption

Copyright

```
Copyright (c) 2004, Pat Thoyts <patthoyts@users.sourceforge.net>
```


ripemd160
Keywords: ripemd160, RIPEMD, hashing, md4, message-digest, rfc 1320, rfc 1321, rfc 2104, security

Name
ripemd160 - RIPEMD-160 Message-Digest Algorithm

Synopsis
package require Tcl 8.5 9
package require ripemd160 ?**1.0.7**?
::ripemd::ripemd160 ?*-hex*? [*-channel channel* | *-file filename* | *string*]
::ripemd::hmac160 ?*-hex*? *-key key* [*-channel channel* | *-file filename* | *string*]
::ripemd::RIPEMD160Init
::ripemd::RIPEMD160Update *token data*
::ripemd::RIPEMD160Final *token*
::ripemd::RIPEHMAC160Init *key*
::ripemd::RIPEHMAC160Update *token data*
::ripemd::RIPEHMAC160Final *token*

Description
This package is an implementation in Tcl of the RIPEMD-160 message-digest algorithm (1). This algorithm takes an arbitrary quantity of data and generates a 160-bit message digest from the input. The RIPEMD-160 algorithm is based upon the MD4 algorithm (2, 4) but has been cryptographically strengthened against weaknesses that have been found in MD4 (4).

This package will use **cryptkit** or **Trf** to accelerate the digest computation if either package is available. In the absence of an accelerator package the pure-Tcl implementation will be used.

BEWARE The commands in this package expect binary data as their input. When a **-file** is provided then this is ensured by the commands themselves, as they open the referenced file in binary mode. When literal data, or a **-channel** are provided instead, then the command's *caller is responsible* for ensuring this fact. The necessary conversion command is **encoding convertto utf-8 $string**.

ATTENTION, there is a *Tcl 9 COMPATIBILITY ISSUE* here.

Tcl 8.x silently mishandles non-binary input by cutting it internally to size. I.e. by only using the low byte of a higher unicode point. No error is thrown, the *result is simply wrong*.

Tcl 9 throws an error instead, i.e.

```
expected byte sequence but character <location> was ...
```

Commands

::ripemd::ripemd160 ?*-hex*? [*-channel channel* | *-file filename* | *string*]

Calculate the RIPEMD-160 digest of the data given in string. This is returned as a binary string by default. Giving the *-hex* option will return a hexadecimal encoded version of the digest.

The data to be hashed can be specified either as a string argument to the ripemd160 command, or as a filename or a pre-opened channel. If the *-filename* argument is given then the file is opened, the data read and hashed and the file is closed. If the *-channel* argument is given then data is read from the channel until the end of file. The channel is not closed.

Only one of *-file*, *-channel* or *string* should be given.

::ripemd::hmac160 ?*-hex*? *-key key* [*-channel channel* | *-file filename* | *string*]

Calculate an Hashed Message Authentication digest (HMAC) using the RIPEMD-160 digest algorithm. HMACs are described in RFC 2104 (5) and provide a RIPEMD-160 digest that includes a key. All options other than *-key* are as for the **::ripemd::ripemd160** command.

Programming interface

For the programmer, hash functions can be viewed as a bucket into which one pours data. When you have finished, you extract a value that is uniquely derived from the data that was poured into the bucket. The programming interface to the hash operates on a token (equivalent to the bucket). You call **RIPEMD160Init** to obtain a token and then call **RIPEMD160Update** as many times as required to add data to the hash. To release any resources and obtain the hash value, you then call **RIPEMD160Final**. An equivalent set of functions gives you a keyed digest (HMAC).

::ripemd::RIPEMD160Init

Begins a new RIPEMD-160 hash. Returns a token ID that must be used for the remaining functions.

::ripemd::RIPEMD160Update *token data*

Add data to the hash identified by token. Calling *RIPEMD160Update $token* "*abcd*" is equivalent to calling *RIPEMD160Update $token* "*ab*" followed by *RIPEMD160Update $token* "*cb*". See Examples.

::ripemd::RIPEMD160Final *token*

Returns the hash value and releases any resources held by this token. Once this command completes the token will be invalid. The result is a binary string of 16 bytes representing the 160 bit RIPEMD-160 digest value.

::ripemd::RIPEHMAC160Init *key*

This is equivalent to the **::ripemd::RIPEMD160Init** command except that it requires the key that will be included in the HMAC.

::ripemd::RIPEHMAC160Update *token data*

see below ...

::ripemd::RIPEHMAC160Final *token*

These commands are identical to the RIPEMD160 equivalent commands.

Examples

```
% ripemd::ripemd160 -hex "Tcl does RIPEMD-160"
0829dea75a1a7074c702896723fe37763481a0a7
```

```
% ripemd::hmac160 -hex -key Sekret "Tcl does RIPEMD-160"
bf0c927231733686731dddb470b64a9c23f7f53b
```

```
% set tok [ripemd::RIPEMD160Init]
::ripemd::1
% ripemd::RIPEMD160Update $tok "Tcl "
% ripemd::RIPEMD160Update $tok "does "
% ripemd::RIPEMD160Update $tok "RIPEMD-160"
% ripemd::Hex [ripemd::RIPEMD160Final $tok]
0829dea75a1a7074c702896723fe37763481a0a7
```

References

1. H. Dobbertin, A. Bosselaers, B. Preneel, "RIPEMD-160, a strengthened version of RIPEMD" http://www.esat.kuleuven.ac.be/~cosicart/pdf/AB-9601/AB-9601.pdf
2. Rivest, R., "The MD4 Message Digest Algorithm", RFC 1320, MIT, April 1992. (http://www.rfc-editor.org/rfc/rfc1320.txt)
3. Rivest, R., "The MD4 message digest algorithm", in A.J. Menezes and S.A. Vanstone, editors, Advances in Cryptology - CRYPTO '90 Proceedings, pages 303-311, Springer-Verlag, 1991.
4. Dobbertin, H., "Cryptanalysis of MD4", Journal of Cryptology vol 11 (4), pp. 253-271 (1998)
5. Krawczyk, H., Bellare, M. and Canetti, R. "HMAC: Keyed-Hashing for Message Authentication", RFC 2104, February 1997. (http://www.rfc-editor.org/rfc/rfc2104.txt)

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *ripemd* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Hashes, checksums, and encryption

Copyright

```
Copyright (c) 2004, Pat Thoyts <patthoyts@users.sourceforge.net>
```

sasl

gtoken

Keywords: gtoken, SASL, XGoogleToken, authentication

Name

SASL::XGoogleToken - Implementation of SASL NTLM mechanism for Tcl

Synopsis

package require Tcl 8.5 9
package require SASL::XGoogleToken ?**1.0.2**?

Description

This package provides the XGoogleToken authentication mechanism for the Simple Authentication and Security Layer (SASL).

Please read the documentation for package **sasl** for details.

TLS security considerations

This package uses the **TLS** package to handle the security for **https** urls and other socket connections.

Policy decisions like the set of protocols to support and what ciphers to use are not the responsibility of **TLS**, nor of this package itself however. Such decisions are the responsibility of whichever application is using the package, and are likely influenced by the set of servers the application will talk to as well.

For example, in light of the recent POODLE attack discovered by Google many servers will disable support for the SSLv3 protocol. To handle this change the applications using **TLS** must be patched, and not this package, nor **TLS** itself. Such a patch may be as simple as generally activating **tls1** support, as shown in the example below.

```
    package require tls
    tls::init -tls1 1 ;# forcibly activate support for the TLS1
    protocol

    ... your own application code ...
```

Authors

Pat Thoyts

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *sasl* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Networking

Copyright

```
Copyright (c) 2006, Pat Thoyts <patthoyts@users.sourceforge.net>
```

ntlm

Keywords: ntlm, SASL, authentication

Name

SASL::NTLM - Implementation of SASL NTLM mechanism for Tcl

Synopsis

package require Tcl 8.5 9
package require SASL::NTLM ?**1.1.4**?

Description

This package provides the NTLM authentication mechanism for the Simple Authentication and Security Layer (SASL).

Please read the documentation for package **sasl** for details.

References

1. No official specification is available. However, http://davenport.sourceforge.net/ntlm.html provides a good description.

Authors

Pat Thoyts

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *sasl* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Networking

Copyright

```
Copyright (c) 2005-2006, Pat Thoyts <patthoyts@users.sourceforge.net>
```

sasl

Keywords: sasl, authentication

Name

SASL - Implementation of SASL mechanisms for Tcl

Synopsis

package require Tcl 8.5 9
package require SASL ?**1.3.4**?
::SASL::new *option value* ?*...*?
::SASL::configure *option value* ?*...*?
::SASL::step *context challenge* ?*...*?
::SASL::response *context*
::SASL::reset *context*
::SASL::cleanup *context*
::SASL::mechanisms ?*type*? ?*minimum*?
::SASL::register *mechanism preference clientproc* ?*serverproc*?

Description

The Simple Authentication and Security Layer (SASL) is a framework for providing authentication and authorization to comunications protocols. The SASL framework is structured to permit negotiation among a number of authentication mechanisms. SASL may be used in SMTP, IMAP and HTTP authentication. It is also in use in XMPP, LDAP and BEEP. See Mechanisms for the set of available SASL mechanisms provided with tcllib.

The SASL framework operates using a simple multi-step challenge response mechanism. All the mechanisms work the same way although the number of steps may vary. In this implementation a callback procedure must be provided from which the SASL framework will obtain users details. See Callback procedure for details of this procedure.

Commands

::SASL::new *option value ?...?*
: Contruct a new SASL context. See Options for details of the possible options to this command. A context token is required for most of the SASL procedures.

::SASL::configure *option value ?...?*
: Modify and inspect the SASL context option. See Options for further details.

::SASL::step *context challenge ?...?*
: This is the core procedure for using the SASL framework. The **step** procedure should be called until it returns 0. Each step takes a server challenge string and the response is calculated and stored in the context. Each mechanism may require one or more steps. For some steps there may be no server challenge required in which case an empty string should be provided for this parameter. All mechanisms should accept an initial empty challenge.

::SASL::response *context*
: Returns the next response string that should be sent to the server.

::SASL::reset *context*
: Re-initialize the SASL context. Discards any internal state and permits the token to be reused.

::SASL::cleanup *context*
: Release all resources associated with the SASL context. The context token may not be used again after this procedure has been called.

::SASL::mechanisms ?*type*? ?*minimum*?
: Returns a list of all the available SASL mechanisms. The list is sorted by the mechanism preference value (see **register**) with the preferred mechanisms and the head of the list. Any

mechanism with a preference value less than the*minimum* (which defaults to 0) is removed from the returned list. This permits a security threshold to be set. Mechanisms with a preference less that 25 transmit authentication are particularly susceptible to eavesdropping and should not be provided unless a secure channel is in use (eg: tls).

The *type* parameter may be one of *client* or *server* and defaults to *client*. Only mechanisms that have an implementation matching the *type* are returned (this permits servers to correctly declare support only for mechanisms that actually provide a server implementation).

::SASL::register *mechanism preference clientproc ?serverproc?*

New mechanisms can be added to the package by registering the mechanism name and the implementing procedures. The server procedure is optional. The preference value is an integer that is used to order the list returned by the **mechanisms** command. Higher values indicate a preferred mechanism. If the mechanism is already registered then the recorded values are updated.

Options

-callback

Specify a command to be evaluated when the SASL mechanism requires information about the user. The command is called with the current SASL context and a name specifying the information desired. See Examples.

-mechanism

Set the SASL mechanism to be used. See **mechanisms** for a list of supported authentication mechanisms.

-service

Set the service type for this context. Some mechanisms may make use of this parameter (eg DIGEST-MD5, GSSAPI and Kerberos). If not set it defaults to an empty string. If the **-type** is set to 'server' then this option should be set to a valid service identity. Some examples of valid service names are smtp, ldap, beep and xmpp.

-server

This option is used to set the server name used in SASL challenges when operating as a SASL server.

-type

The context type may be one of 'client' or 'server'. The default is to operate as a client application and respond to server challenges. Mechanisms may be written to support server-side SASL and setting this option will cause each **step** to issue the next challenge. A new context must be created for each incoming client connection when in server mode.

Callback procedure

When the SASL framework requires any user details it will call the procedure provided when the context was created with an argument that specfies the item of information required.

In all cases a single response string should be returned.

login

The callback procedure should return the users authorization identity. Return an empty string unless this is to be different to the authentication identity. Read [1] for a discussion about the specific meaning of authorization and authentication identities within SASL.

username
The callback procedure should return the users authentication identity. Read [1] for a discussion about the specific meaning of authorization and authentication identities within SASL.

password
The callback procedure should return the password that matches the authentication identity as used within the current realm.

For server mechanisms the password callback should always be called with the authentication identity and the realm as the first two parameters.

realm
Some SASL mechanisms use realms to partition authentication identities. The realm string is protocol dependent and is often the current DNS domain or in the case of the NTLM mechanism it is the Windows NT domain name.

hostname
Returns the client host name - typically [info host].

Mechanisms

ANONYMOUS
As used in FTP this mechanism only passes an email address for authentication. The ANONYMOUS mechanism is specified in [2].

PLAIN
This is the simplest mechanism. The users authentication details are transmitted in plain text. This mechanism should not be provided unless an encrypted link is in use - typically after SSL or TLS has been negotiated.

LOGIN
The LOGIN [1] mechanism transmits the users details with base64 encoding. This is no more secure than PLAIN and likewise should not be used without a secure link.

CRAM-MD5
This mechanism avoids sending the users password over the network in plain text by hashing the password with a server provided random value (known as a nonce). A disadvantage of this mechanism is that the server must maintain a database of plaintext passwords for comparison. CRAM-MD5 was defined in [4].

DIGEST-MD5
This mechanism improves upon the CRAM-MD5 mechanism by avoiding the need for the server to store plaintext passwords. With digest authentication the server needs to store the MD5 digest of the users password which helps to make the system more secure. As in CRAM-MD5 the password is hashed with a server nonce and other data before being transmitted across the network. Specified in [3].

OTP
OTP is the One-Time Password system described in RFC 2289 [6]. This mechanism is secure against replay attacks and also avoids storing password or password equivalents on the server. Only a digest of a seed and a passphrase is ever transmitted across the network. Requires the **otp** package from tcllib and one or more of the cryptographic digest packages (md5 or sha-1 are the most commonly used).

NTLM
: This is a proprietary protocol developed by Microsoft [5] and is in common use for authenticating users in a Windows network environment. NTLM uses DES encryption and MD4 digests of the users password to authenticate a connection. Certain weaknesses have been found in NTLM and thus there are a number of versions of the protocol. As this mechanism has additional dependencies it is made available as a separate sub-package. To enable this mechanism your application must load the SASL::NTLM package.

X-GOOGLE-TOKEN
: This is a proprietary protocol developed by Google and used for authenticating users for the Google Talk service. This mechanism makes a pair of HTTP requests over an SSL channel and so this mechanism depends upon the availability of the tls and http packages. To enable this mechanism your application must load the SASL::XGoogleToken package. In addition you are recommended to make use of the autoproxy package to handle HTTP proxies reasonably transparently.

SCRAM
: This is a protocol specified in RFC 5802 [7]. To enable this mechanism your application must load the SASL::SCRAM package.

Examples

See the examples subdirectory for more complete samples using SASL with network protocols. The following should give an idea how the SASL commands are to be used. In reality this should be event driven. Each time the **step** command is called, the last server response should be provided as the command argument so that the SASL mechanism can take appropriate action.

```
proc ClientCallback {context command args} {
    switch -exact -- $command {
        login    { return "" }
        username { return $::tcl_platform(user) }
        password { return "SecRet" }
        realm    { return "" }
        hostname { return [info host] }
        default  { return -code error unxpected }
    }
}

proc Demo {{mech PLAIN}} {
    set ctx [SASL::new -mechanism $mech -callback ClientCallback]
    set challenge ""
    while {1} {
        set more_steps [SASL::step $ctx challenge]
        puts "Send '[SASL::response $ctx]'"
        puts "Read server response into challenge var"
        if {!$more_steps} {break}
    }
    SASL::cleanup $ctx
}
```

References

1. Myers, J. "Simple Authentication and Security Layer (SASL)", RFC 2222, October 1997. (http://www.ietf.org/rfc/rfc2222.txt)
2. Newman, C. "Anonymous SASL Mechanism", RFC 2245, November 1997. (http://www.ietf.org/rfc/rfc2245.txt)
3. Leach, P., Newman, C. "Using Digest Authentication as a SASL Mechanism", RFC 2831, May 2000, (http://www.ietf.org/rfc/rfc2831.txt)
4. Klensin, J., Catoe, R. and Krumviede, P., "IMAP/POP AUTHorize Extension for Simple Challenge/Response" RFC 2195, September 1997. (http://www.ietf.org/rfc/rfc2195.txt)
5. No official specification is available. However, http://davenport.sourceforge.net/ntlm.html provides a good description.
6. Haller, N. et al., "A One-Time Password System", RFC 2289, February 1998, (http://www.ieft.org/rfc/rfc2289.txt)
7. Newman, C. et al., "Salted Challenge Response Authentication Mechanism (SCRAM) SASL and GSS-API Mechanisms", RFC 5802, July 2010, (http://www.ieft.org/rfc/rfc5802.txt)

Authors

Pat Thoyts

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *sasl* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Networking

Copyright

```
Copyright (c) 2005-2006, Pat Thoyts <patthoyts@users.sourceforge.net>
```

scram

Keywords: scram, SASL, authentication

Name

SASL::SCRAM - Implementation of SASL SCRAM mechanism for Tcl

Synopsis

package require Tcl 8.5 9
package require SASL::SCRAM ?**0.2**?

Description

This package provides the SCRAM authentication mechanism for the Simple Authentication and

Security Layer (SASL).

Please read the documentation for package **sasl** for details.

References

1. Newman, C. et al., "Salted Challenge Response Authentication Mechanism (SCRAM) SASL and GSS-API Mechanisms", RFC 5802, July 2010, (http://www.ieft.org/rfc/rfc5802.txt)

Authors

Sergei Golovan

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *sasl* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Networking

Copyright

```
Copyright (c) 2013 Sergei Golovan <sgolovan@nes.ru>
```

sha1

sha1

Keywords: sha1, FIPS 180-1, hashing, message-digest, rfc 2104, security

Name

sha1 - SHA1 Message-Digest Algorithm

Synopsis

package require Tcl 8.5 9
package require sha1 ?**2.0.5**?
::sha1::sha1 ?**-hex|-bin**? [**-channel channel** | **-file filename** | ?**--**? *string*]
::sha1::hmac *key string*
::sha1::hmac ?**-hex|-bin**? **-key key** [**-channel channel** | **-file filename** | ?**--**? *string*]
::sha1::SHA1Init
::sha1::SHA1Update *token data*
::sha1::SHA1Final *token*
::sha1::HMACInit *key*
::sha1::HMACUpdate *token data*
::sha1::HMACFinal *token*

Description

This package provides an implementation in Tcl of the SHA1 message-digest algorithm as specified by FIPS PUB 180-1 (1). This algorithm takes a message and generates a 160-bit digest from the input. The SHA1 algorithm is related to the MD4 algorithm (2) but has been strengthend against certain types of cryptographic attack. SHA1 should be used in preference to MD4 or MD5 in new applications.

This package also includes support for creating keyed message-digests using the HMAC algorithm from RFC 2104 (3) with SHA1 as the message-digest.

BEWARE The commands in this package expect binary data as their input. When a **-file** is provided then this is ensured by the commands themselves, as they open the referenced file in binary mode. When literal data, or a **-channel** are provided instead, then the command's *caller is responsible* for ensuring this fact. The necessary conversion command is **encoding convertto utf-8 $string**.

ATTENTION, there is a *Tcl 9 COMPATIBILITY ISSUE* here.

Tcl 8.x silently mishandles non-binary input by cutting it internally to size. I.e. by only using the low byte of a higher unicode point. No error is thrown, the *result is simply wrong*.

Tcl 9 throws an error instead, i.e.

```
expected byte sequence but character <location> was ...
```

Commands

::sha1::sha1 ?-hex|-bin? [-channel channel | -file filename | ?--? *string*]

The command takes a message and returns the SHA1 digest of this message as a hexadecimal string. You may request the result as binary data by giving *-bin*.

The data to be hashed can be specified either as a string argument to the **sha1** command, or as a filename or a pre-opened channel. If the *-filename* argument is given then the file is opened, the data read and hashed and the file is closed. If the *-channel* argument is given then data is read from the channel until the end of file. The channel is not closed. *NOTE* use of the channel or filename options results in the internal use of **vwait**. To avoid nested event loops in Tk or tclhttpd applications you should use the incremental programming API (see below).

Only one of *-file*, *-channel* or *string* should be given.

If the *string* to hash can be mistaken for an option (leading dash "-"), use the option **--** before it to terminate option processing and force interpretation as a string.

::sha1::hmac *key string*

see below ...

::sha1::hmac ?-hex|-bin? -key key [-channel channel | -file filename | ?--? *string*]

Calculate an Hashed Message Authentication digest (HMAC) using the SHA1 digest algorithm. HMACs are described in RFC 2104 (3) and provide an SHA1 digest that includes a key. All options other than *-key* are as for the **::sha1::sha1** command.

If the *string* to hash can be mistaken for an option (leading dash "-"), use the option **--** before

it to terminate option processing and force interpretation as a string.

Programming interface

For the programmer, the SHA1 hash can be viewed as a bucket into which one pours data. When you have finished, you extract a value that is derived from the data that was poured into the bucket. The programming interface to the SHA1 hash operates on a token (equivalent to the bucket). You call **SHA1Init** to obtain a token and then call **SHA1Update** as many times as required to add data to the hash. To release any resources and obtain the hash value, you then call **SHA1Final**. An equivalent set of functions gives you a keyed digest (HMAC).

If you have **critcl** and have built the **tcllibc** package then the implementation of the hashing function will be performed by compiled code. Failing that if you have the **Trf** package then this can be used otherwise there is a pure-tcl equivalent. The programming interface remains the same in all cases.

::sha1::SHA1Init
Begins a new SHA1 hash. Returns a token ID that must be used for the remaining functions.

::sha1::SHA1Update *token data*
Add data to the hash identified by token. Calling *SHA1Update $token* "*abcd*" is equivalent to calling *SHA1Update $token* "*ab*" followed by *SHA1Update $token* "*cb*". See Examples.

::sha1::SHA1Final *token*
Returns the hash value and releases any resources held by this token. Once this command completes the token will be invalid. The result is a binary string of 20 bytes representing the 160 bit SHA1 digest value.

::sha1::HMACInit *key*
This is equivalent to the **::sha1::SHA1Init** command except that it requires the key that will be included in the HMAC.

::sha1::HMACUpdate *token data*
see below ...

::sha1::HMACFinal *token*
These commands are identical to the SHA1 equivalent commands.

Examples

```
% sha1::sha1 "Tcl does SHA1"
285a6a91c45a9066bf39fcf24425796ef0b2a8bf
```

```
% sha1::hmac Sekret "Tcl does SHA1"
ae6251fa51b95b18cba2be95eb031d07475ff03c
```

```
% set tok [sha1::SHA1Init]
::sha1::1
% sha1::SHA1Update $tok "Tcl "
% sha1::SHA1Update $tok "does "
% sha1::SHA1Update $tok "SHA1"
% sha1::Hex [sha1::SHA1Final $tok]
285a6a91c45a9066bf39fcf24425796ef0b2a8bf
```

References

1. "Secure Hash Standard", National Institute of Standards and Technology, U.S. Department Of Commerce, April 1995. (http://www.itl.nist.gov/fipspubs/fip180-1.htm)
2. Rivest, R., "The MD4 Message Digest Algorithm", RFC 1320, MIT, April 1992. (http://www.rfc-editor.org/rfc/rfc1320.txt)
3. Krawczyk, H., Bellare, M. and Canetti, R. "HMAC: Keyed-Hashing for Message Authentication", RFC 2104, February 1997. (http://www.rfc-editor.org/rfc/rfc2104.txt)

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *sha1* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Hashes, checksums, and encryption

Copyright

```
Copyright (c) 2005, Pat Thoyts <patthoyts@users.sourceforge.net>
```

sha256

Keywords: sha256, FIPS 180-1, hashing, message-digest, rfc 2104, security

Name

sha256 - SHA256 Message-Digest Algorithm

Synopsis

package require Tcl 8.5 9
package require sha256 ?**1.0.6**?
::sha2::sha256 ?**-hex**|**-bin**? [**-channel channel** | **-file filename** | ?**--**? *string*]
::sha2::sha224 ?**-hex**|**-bin**? [**-channel channel** | **-file filename** | ?**--**? *string*]
::sha2::hmac *key string*
::sha2::hmac ?**-hex**|**-bin**? **-key key** [**-channel channel** | **-file filename** | ?**--**? *string*]
::sha2::SHA256Init
::sha2::SHA224Init
::sha2::SHA256Update *token data*
::sha2::SHA256Final *token*
::sha2::SHA224Final *token*
::sha2::HMACInit *key*
::sha2::HMACUpdate *token data*
::sha2::HMACFinal *token*

Description

This package provides an implementation in Tcl of the SHA256 and SHA224 message-digest algorithms as specified by FIPS PUB 180-1 (1). These algorithms take a message and generates a 256-bit (224-bit) digest from the input. The SHA2 algorithms are related to the SHA1 algorithm.

This package also includes support for creating keyed message-digests using the HMAC algorithm from RFC 2104 (3) with SHA256 as the message-digest.

BEWARE The commands in this package expect binary data as their input. When a **-file** is provided then this is ensured by the commands themselves, as they open the referenced file in binary mode. When literal data, or a **-channel** are provided instead, then the command's *caller is responsible* for ensuring this fact. The necessary conversion command is **encoding convertto utf-8 $string**.

ATTENTION, there is a *Tcl 9 COMPATIBILITY ISSUE* here.

Tcl 8.x silently mishandles non-binary input by cutting it internally to size. I.e. by only using the low byte of a higher unicode point. No error is thrown, the *result is simply wrong*.

Tcl 9 throws an error instead, i.e.

```
expected byte sequence but character <location> was ...
```

Commands

::sha2::sha256 ?-hex|-bin? [-channel channel | -file filename | ?--? *string*]

The command takes a message and returns the SHA256 digest of this message as a hexadecimal string. You may request the result as binary data by giving *-bin*.

The data to be hashed can be specified either as a string argument to the **sha256** command, or as a filename or a pre-opened channel. If the *-filename* argument is given then the file is opened, the data read and hashed and the file is closed. If the *-channel* argument is given then data is read from the channel until the end of file. The channel is not closed. *NOTE* use of the channel or filename options results in the internal use of **vwait**. To avoid nested event loops in Tk or tclhttpd applications you should use the incremental programming API (see below).

Only one of *-file*, *-channel* or *string* should be given.

If the *string* to hash can be mistaken for an option (leading dash "-"), use the option **--** before it to terminate option processing and force interpretation as a string.

::sha2::sha224 ?-hex|-bin? [-channel channel | -file filename | ?--? *string*]

Like **::sha2::sha256**, except that the SHA224 digest is returned.

::sha2::hmac *key string*

see below ...

::sha2::hmac ?-hex|-bin? -key key [-channel channel | -file filename | ?--? *string*]

Calculate an Hashed Message Authentication digest (HMAC) using the SHA256 digest algorithm. HMACs are described in RFC 2104 (3) and provide an SHA256 digest that includes a key. All options other than *-key* are as for the **::sha2::sha256** command.

If the *string* to hash can be mistaken for an option (leading dash "-"), use the option **--** before

it to terminate option processing and force interpretation as a string.

Programming interface

For the programmer, the SHA256 hash can be viewed as a bucket into which one pours data. When you have finished, you extract a value that is derived from the data that was poured into the bucket. The programming interface to the SHA256 hash operates on a token (equivalent to the bucket). You call **SHA256Init** to obtain a token and then call **SHA256Update** as many times as required to add data to the hash. To release any resources and obtain the hash value, you then call **SHA256Final**. An equivalent set of functions gives you a keyed digest (HMAC).

If you have **critcl** and have built the **tcllibc** package then the implementation of the hashing function will be performed by compiled code. Failing that there is a pure-tcl equivalent. The programming interface remains the same in all cases.

::sha2::SHA256Init
see below ...

::sha2::SHA224Init
Begins a new SHA256/SHA224 hash. Returns a token ID that must be used for the remaining functions.

::sha2::SHA256Update *token data*
Add data to the hash identified by token. Calling *SHA256Update $token* "*abcd*" is equivalent to calling *SHA256Update $token* "*ab*" followed by *SHA256Update $token* "*cb*". See Examples. Note that this command is used for both SHA256 and SHA224. Only the initialization and finalization commands of both hashes differ.

::sha2::SHA256Final *token*
see below ...

::sha2::SHA224Final *token*
Returns the hash value and releases any resources held by this token. Once this command completes the token will be invalid. The result is a binary string of 32/28 bytes representing the 256/224 bit SHA256 / SHA224 digest value.

::sha2::HMACInit *key*
This is equivalent to the **::sha2::SHA256Init** command except that it requires the key that will be included in the HMAC.

::sha2::HMACUpdate *token data*
see below ...

::sha2::HMACFinal *token*
These commands are identical to the SHA256 equivalent commands.

Examples

```
% sha2::sha256 "Tcl does SHA256"
0b91043ee484abd83c3e4b08d6034d71b937026379f0f59bda6e625e6e214789
```

```
% sha2::hmac Sekret "Tcl does SHA256"
4f9352c64d655e8a36abe73e6163a9d7a54039877c1c92ec90b07d48d4e854e0
```

```
% set tok [sha2::SHA256Init]
::sha2::1
% sha2::SHA256Update $tok "Tcl "
% sha2::SHA256Update $tok "does "
% sha2::SHA256Update $tok "SHA256"
% sha2::Hex [sha2::SHA256Final $tok]
0b91043ee484abd83c3e4b08d6034d71b937026379f0f59bda6e625e6e214789
```

References

1. "Secure Hash Standard", National Institute of Standards and Technology, U.S. Department Of Commerce, April 1995. (http://www.itl.nist.gov/fipspubs/fip180-1.htm)
2. Rivest, R., "The MD4 Message Digest Algorithm", RFC 1320, MIT, April 1992. (http://www.rfc-editor.org/rfc/rfc1320.txt)
3. Krawczyk, H., Bellare, M. and Canetti, R. "HMAC: Keyed-Hashing for Message Authentication", RFC 2104, February 1997. (http://www.rfc-editor.org/rfc/rfc2104.txt)

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *sha1* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Hashes, checksums, and encryption

Copyright

```
Copyright (c) 2008, Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

simulation

annealing

Keywords: annealing, math, optimization, simulated annealing

Name

simulation::annealing - Simulated annealing

Synopsis

package require Tcl ?**8.5 9**?
package require simulation::annealing 0.3
::simulation::annealing::getOption *keyword*
::simulation::annealing::hasOption *keyword*

::simulation::annealing::setOption *keyword value*
::simulation::annealing::findMinimum *args*
::simulation::annealing::findCombinatorialMinimum *args*

Description

The technique of *simulated annealing* provides methods to estimate the global optimum of a function. It is described in some detail on the Wiki http://wiki.tcl.tk/.... The idea is simple:

- randomly select points within a given search space
- evaluate the function to be optimised for each of these points and select the point that has the lowest (or highest) function value or - sometimes - accept a point that has a less optimal value. The chance by which such a non-optimal point is accepted diminishes over time.
- Accepting less optimal points means the method does not necessarily get stuck in a local optimum and theoretically it is capable of finding the global optimum within the search space.

The method resembles the cooling of material, hence the name.

The package *simulation::annealing* offers the command *findMinimum*:

```
    puts [::simulation::annealing::findMinimum  -trials 300
    -parameters {x -5.0 5.0 y -5.0 5.0}  -function
    {$x*$x+$y*$y+sin(10.0*$x)+4.0*cos(20.0*$y)}]
```

prints the estimated minimum value of the function f(x,y) = *x**2+y**2+sin(10*x)+4*cos(20*y)* and the values of x and y where the minimum was attained:

```
result -4.9112922923 x -0.181647676593 y 0.155743646974
```

Procedures

The package defines the following auxiliary procedures:

::simulation::annealing::getOption *keyword*
Get the value of an option given as part of the *findMinimum* command.
string *keyword*
Given keyword (without leading minus)

::simulation::annealing::hasOption *keyword*
Returns 1 if the option is available, 0 if not.
string *keyword*
Given keyword (without leading minus)

::simulation::annealing::setOption *keyword value*
Set the value of the given option.
string *keyword*
Given keyword (without leading minus)
string *value*
(New) value for the option

The main procedures are *findMinimum* and *findCombinatorialMinimum*:

::simulation::annealing::findMinimum *args*
Find the minimum of a function using simulated annealing. The function and the method's

parameters is given via a list of keyword-value pairs.

int *n*

List of keyword-value pairs, all of which are available during the execution via the *getOption* command.

::simulation::annealing::findCombinatorialMinimum *args*

Find the minimum of a function of discrete variables using simulated annealing. The function and the method's parameters is given via a list of keyword-value pairs.

int *n*

List of keyword-value pairs, all of which are available during the execution via the *getOption* command.

The *findMinimum* command predefines the following options:

- *-parameters list*: triples defining parameters and ranges
- *-function expr*: expression defining the function
- *-code body*: body of code to define the function (takes precedence over *-function*). The code should set the variable "result"
- *-init code*: code to be run at start up *-final code*: code to be run at the end *-trials n*: number of trials before reducing the temperature *-reduce factor*: reduce the temperature by this factor (between 0 and 1) *-initial-temp t*: initial temperature *-scale s*: scale of the function (order of magnitude of the values) *-estimate-scale y/n*: estimate the scale (only if *-scale* is not present) *-verbose y/n*: print detailed information on progress to the report file (1) or not (0) *-reportfile file*: opened file to print to (defaults to stdout)

Any other options can be used via the getOption procedure in the body. The *findCombinatorialMinimum* command predefines the following options:

- *-number-params n*: number of binary parameters (the solution space consists of lists of 1s and 0s). This is a required option.
- *-initial-values*: list of 1s and 0s constituting the start of the search.

The other predefined options are identical to those of *findMinimum*.

Tips

The procedure *findMinimum* works by constructing a temporary procedure that does the actual work. It loops until the point representing the estimated optimum does not change anymore within the given number of trials. As the temperature gets lower and lower the chance of accepting a point with a higher value becomes lower too, so the procedure will in practice terminate.

It is possible to optimise over a non-rectangular region, but some care must be taken:

- If the point is outside the region of interest, you can specify a very high value.
- This does mean that the automatic determination of a scale factor is out of the question - the high function values that force the point inside the region would distort the estimation.

Here is an example of finding an optimum inside a circle:

```
    puts [::simulation::annealing::findMinimum  -trials 3000  -reduce
    0.98  -parameters {x -5.0 5.0 y -5.0 5.0}  -code {
            if { hypot($x-5.0,$y-5.0) < 4.0 } {
                set result [expr
    {$x*$x+$y*$y+sin(10.0*$x)+4.0*cos(20.0*$y)}]
            } else {
                set result 1.0e100
            }
        }]
```

The method is theoretically capable of determining the global optimum, but often you need to use a large number of trials and a slow reduction of temperature to get reliable and repeatable estimates.

You can use the *-final* option to use a deterministic optimization method, once you are sure you are near the required optimum.

The *findCombinatorialMinimum* procedure is suited for situations where the parameters have the values 0 or 1 (and there can be many of them). Here is an example:

- We have a function that attains an absolute minimum if the first ten numbers are 1 and the rest is 0:

```
proc cost {params} {
    set cost 0
    foreach p [lrange $params 0 9] {
        if { $p == 0 } {
            incr cost
        }
    }
    foreach p [lrange $params 10 end] {
        if { $p == 1 } {
            incr cost
        }
    }
    return $cost
}
```

- We want to find the solution that gives this minimum for various lengths of the solution vector *params*:

```
foreach n {100 1000 10000} {
    break
    puts "Problem size: $n"
    puts [::simulation::annealing::findCombinatorialMinimum  -trials
    300  -verbose 0  -number-params $n  -code {set result [cost
    $params]}]
}
```

- As the vector grows, the computation time increases, but the procedure will stop if some kind of equilibrium is reached. To achieve a useful solution you may want to try different values of

the trials parameter for instance. Also ensure that the function to be minimized depends on all or most parameters - see the source code for a counter example and run that.

Category
Mathematics

Copyright

```
Copyright (c) 2008 Arjen Markus <arjenmarkus@users.sourceforge.net>
```


montecarlo
Keywords: montecarlo, math, montecarlo simulation, stochastic modelling

Name
simulation::montecarlo - Monte Carlo simulations

Synopsis
package require Tcl ?**8.5 9**?
package require simulation::montecarlo 0.2
package require simulation::random
package require math::statistics
::simulation::montecarlo::getOption *keyword*
::simulation::montecarlo::hasOption *keyword*
::simulation::montecarlo::setOption *keyword value*
::simulation::montecarlo::setTrialResult *values*
::simulation::montecarlo::setExpResult *values*
::simulation::montecarlo::getTrialResults
::simulation::montecarlo::getExpResult
::simulation::montecarlo::transposeData *values*
::simulation::montecarlo::integral2D *...*
::simulation::montecarlo::singleExperiment *args*

Description
The technique of *Monte Carlo simulations* is basically simple:

- generate random values for one or more parameters.
- evaluate the model of some system you are interested in and record the interesting results for each realisation of these parameters.
- after a suitable number of such trials, deduce an overall characteristic of the model.

You can think of a model of a network of computers, an ecosystem of some kind or in fact anything that can be quantitatively described and has some stochastic element in it.

The package *simulation::montecarlo* offers a basic framework for such a modelling technique:

```
#
# MC experiments:
# Determine the mean and median of a set of points and compare them
#
::simulation::montecarlo::singleExperiment -init {
```

```
    package require math::statistics

    set prng [::simulation::random::prng_Normal 0.0 1.0]
} -loop {
    set numbers {}
    for { set i 0 } { $i < [getOption samples] } { incr i } {
        lappend numbers [$prng]
    }
    set mean   [::math::statistics::mean $numbers]
    set median [::math::statistics::median $numbers] ;# ? Exists?
    setTrialResult [list $mean $median]
} -final {
    set result [getTrialResults]
    set means   {}
    set medians {}
    foreach r $result {
        foreach {m M} $r break
        lappend means   $m
        lappend medians $M
    }
    puts [getOption reportfile] "Correlation:
    [::math::statistics::corr $means $medians]"

} -trials 100 -samples 10 -verbose 1 -columns {Mean Median}
```

This example attemps to find out how well the median value and the mean value of a random set of numbers correlate. Sometimes a median value is a more robust characteristic than a mean value - especially if you have a statistical distribution with "fat" tails.

Procedures

The package defines the following auxiliary procedures:

::simulation::montecarlo::getOption *keyword*
Get the value of an option given as part of the *singleExperiment* command.
string *keyword*
Given keyword (without leading minus)

::simulation::montecarlo::hasOption *keyword*
Returns 1 if the option is available, 0 if not.
string *keyword*
Given keyword (without leading minus)

::simulation::montecarlo::setOption *keyword value*
Set the value of the given option.
string *keyword*
Given keyword (without leading minus)
string *value*
(New) value for the option

::simulation::montecarlo::setTrialResult *values*
Store the results of the trial for later analysis
list *values*
List of values to be stored

::simulation::montecarlo::setExpResult *values*
Set the results of the entire experiment (typically used in the final phase).
list *values*
List of values to be stored

::simulation::montecarlo::getTrialResults
Get the results of all individual trials for analysis (typically used in the final phase or after completion of the command).

::simulation::montecarlo::getExpResult
Get the results of the entire experiment (typically used in the final phase or even after completion of the *singleExperiment* command).

::simulation::montecarlo::transposeData *values*
Interchange columns and rows of a list of lists and return the result.
list *values*
List of lists of values

There are two main procedures: *integral2D* and *singleExperiment*.

::simulation::montecarlo::integral2D*...*
Integrate a function over a two-dimensional region using a Monte Carlo approach.

Arguments PM

::simulation::montecarlo::singleExperiment *args*
Iterate code over a number of trials and store the results. The iteration is gouverned by parameters given via a list of keyword-value pairs.
int *n*
List of keyword-value pairs, all of which are available during the execution via the *getOption* command.

The *singleExperiment* command predefines the following options:

- *-init code*: code to be run at start up
- *-loop body*: body of code that defines the computation to be run time and again. The code should use *setTrialResult* to store the results of each trial (typically a list of numbers, but the interpretation is up to the implementation). Note: Required keyword.
- *-final code*: code to be run at the end
- *-trials n*: number of trials in the experiment (required)
- *-reportfile file*: opened file to send the output to (default: stdout)
- *-verbose*: write the intermediate results (1) or not (0) (default: 0)
- *-analysis proc*: either "none" (no automatic analysis), standard (basic statistics of the trial results and a correlation matrix) or the name of a procedure that will take care of the analysis.
- *-columns list*: list of column names, useful for verbose output and the analysis

Any other options can be used via the getOption procedure in the body.

Tips
The procedure *singleExperiment* works by constructing a temporary procedure that does the actual work. It loops for the given number of trials.

As it constructs a temporary procedure, local variables defined at the start continue to exist in the loop.

Category
Mathematics

Copyright
```
Copyright (c) 2008 Arjen Markus <arjenmarkus@users.sourceforge.net>
```

simulation_random
Keywords: simulation_random, math, random numbers, simulation, statistical distribution

Name
simulation::random - Pseudo-random number generators

Synopsis
package require Tcl ?**8.5 9**?
package require simulation::random 0.4
::simulation::random::prng_Bernoulli *p*
::simulation::random::prng_Discrete *n*
::simulation::random::prng_Poisson *lambda*
::simulation::random::prng_Uniform *min max*
::simulation::random::prng_Triangular *min max*
::simulation::random::prng_SymmTriangular *min max*
::simulation::random::prng_Exponential *min mean*
::simulation::random::prng_Normal *mean stdev*
::simulation::random::prng_Pareto *min steep*
::simulation::random::prng_Gumbel *min f*
::simulation::random::prng_chiSquared *df*
::simulation::random::prng_Disk *rad*
::simulation::random::prng_Sphere *rad*
::simulation::random::prng_Ball *rad*
::simulation::random::prng_Rectangle *length width*
::simulation::random::prng_Block *length width depth*

Description
This package consists of commands to generate pseudo-random number generators. These new commands deliver

- numbers that are distributed normally, uniformly, according to a Pareto or Gumbel distribution and so on
- coordinates of points uniformly spread inside a sphere or a rectangle

For example:

```
    set p [::simulation::random::prng_Normal -1.0 10.0]
```

produces a new command (whose name is stored in the variable "p") that generates normally distributed numbers with a mean of -1.0 and a standard deviation of 10.0.

Procedures

The package defines the following public procedures for *discrete* distributions:

::simulation::random::prng_Bernoulli *p*

Create a command (PRNG) that generates numbers with a Bernoulli distribution: the value is either 1 or 0, with a chance p to be 1

float *p*

Chance the outcome is 1

::simulation::random::prng_Discrete *n*

Create a command (PRNG) that generates numbers 0 to n-1 with equal probability.

int *n*

Number of different values (ranging from 0 to n-1)

::simulation::random::prng_Poisson *lambda*

Create a command (PRNG) that generates numbers according to the Poisson distribution.

float *lambda*

Mean number per time interval

The package defines the following public procedures for *continuous* distributions:

::simulation::random::prng_Uniform *min max*

Create a command (PRNG) that generates uniformly distributed numbers between "min" and "max".

float *min*

Minimum number that will be generated

float *max*

Maximum number that will be generated

::simulation::random::prng_Triangular *min max*

Create a command (PRNG) that generates triangularly distributed numbers between "min" and "max". If the argument min is lower than the argument max, then smaller values have higher probability and vice versa. In the first case the probability density function is of the form *f(x) = 2(1-x)* and the other case it is of the form *f(x) = 2x*.

float *min*

Minimum number that will be generated

float *max*

Maximum number that will be generated

::simulation::random::prng_SymmTriangular *min max*

Create a command (PRNG) that generates numbers distributed according to a symmetric triangle around the mean of "min" and "max".

float *min*

Minimum number that will be generated

float *max*
Maximum number that will be generated

::simulation::random::prng_Exponential *min mean*
Create a command (PRNG) that generates exponentially distributed numbers with a given minimum value and a given mean value.
float *min*
Minimum number that will be generated
float *mean*
Mean value for the numbers

::simulation::random::prng_Normal *mean stdev*
Create a command (PRNG) that generates normally distributed numbers with a given mean value and a given standard deviation.
float *mean*
Mean value for the numbers
float *stdev*
Standard deviation

::simulation::random::prng_Pareto *min steep*
Create a command (PRNG) that generates numbers distributed according to Pareto with a given minimum value and a given distribution steepness.
float *min*
Minimum number that will be generated
float *steep*
Steepness of the distribution

::simulation::random::prng_Gumbel *min f*
Create a command (PRNG) that generates numbers distributed according to Gumbel with a given minimum value and a given scale factor. The probability density function is:

```
    P(v) = exp( -exp(f*(v-min)))
```

float *min*
Minimum number that will be generated
float *f*
Scale factor for the values

::simulation::random::prng_chiSquared *df*
Create a command (PRNG) that generates numbers distributed according to the chi-squared distribution with df degrees of freedom. The mean is 0 and the standard deviation is 1.
float *df*
Degrees of freedom

The package defines the following public procedures for random point sets:

::simulation::random::prng_Disk *rad*
Create a command (PRNG) that generates (x,y)-coordinates for points uniformly spread over a

disk of given radius.

float *rad*
Radius of the disk

::simulation::random::prng_Sphere *rad*
Create a command (PRNG) that generates (x,y,z)-coordinates for points uniformly spread over the surface of a sphere of given radius.

float *rad*
Radius of the disk

::simulation::random::prng_Ball *rad*
Create a command (PRNG) that generates (x,y,z)-coordinates for points uniformly spread within a ball of given radius.

float *rad*
Radius of the ball

::simulation::random::prng_Rectangle *length width*
Create a command (PRNG) that generates (x,y)-coordinates for points uniformly spread over a rectangle.

float *length*
Length of the rectangle (x-direction)

float *width*
Width of the rectangle (y-direction)

::simulation::random::prng_Block *length width depth*
Create a command (PRNG) that generates (x,y,z)-coordinates for points uniformly spread over a block

float *length*
Length of the block (x-direction)

float *width*
Width of the block (y-direction)

float *depth*
Depth of the block (z-direction)

Category
Mathematics

Copyright
```
Copyright (c) 2004 Arjen Markus <arjenmarkus@users.sourceforge.net>
```

smtpd

smtpd
Keywords: smtpd, rfc 2821, rfc 821, services, smtp, socket, vwait

Name

smtpd - Tcl SMTP server implementation

Synopsis

package require Tcl 8.5 9
package require smtpd ?**1.6**?
::smtpd::start ?*myaddr*? ?*port*?
::smtpd::stop
::smptd::configure ?*option value*? ?*option value* ...?
::smtpd::cget ?*option*?

Description

The **smtpd** package provides a simple Tcl-only server library for the Simple Mail Transfer Protocol as described in RFC 821 (http://www.rfc-editor.org/rfc/rfc821.txt) and RFC 2821 (http://www.rfc-editor.org/rfc/rfc2821.txt). By default the server will bind to the default network address and the standard SMTP port (25).

This package was designed to permit testing of Mail User Agent code from a developers workstation. *It does not attempt to deliver mail to your mailbox.* Instead users of this package are expected to write a procedure that will be called when mail arrives. Once this procedure returns, the server has nothing further to do with the mail.

Security

On Unix platforms binding to the SMTP port requires root privileges. I would not recommend running any script-based server as root unless there is some method for dropping root privileges immediately after the socket is bound. Under Windows platforms, it is not necessary to have root or administrator privileges to bind low numbered sockets. However, security on these platforms is weak anyway.

In short, this code should probably not be used as a permanently running Mail Transfer Agent on an Internet connected server, even though we are careful not to evaluate remote user input. There are many other well tested and security audited programs that can be used as mail servers for internet connected hosts.

TLS security considerations

This package uses the **TLS** package to handle the security for **https** urls and other socket connections.

Policy decisions like the set of protocols to support and what ciphers to use are not the responsibility of **TLS**, nor of this package itself however. Such decisions are the responsibility of whichever application is using the package, and are likely influenced by the set of servers the application will talk to as well.

For example, in light of the recent POODLE attack discovered by Google many servers will disable support for the SSLv3 protocol. To handle this change the applications using **TLS** must be patched, and not this package, nor **TLS** itself. Such a patch may be as simple as generally activating **tls1** support, as shown in the example below.

```
    package require tls
    tls::init -tls1 1 ;# forcibly activate support for the TLS1
    protocol

    ... your own application code ...
```

Commands

::smtpd::start ?*myaddr*? ?*port*?

Start the service listening on *port* or the default port 25. If *myaddr* is given as a domain-style name or numerical dotted-quad IP address then the server socket will be bound to that network interface. By default the server is bound to all network interfaces. For example:

```
  set sock [::smtpd::start [info hostname] 0]
```

will bind to the hosts internet interface on the first available port.

At present the package only supports a single instance of a SMTP server. This could be changed if required at the cost of making the package a little more complicated to read. If there is a good reason for running multiple SMTP services then it will only be necessary to fix the **options** array and the **::smtpd::stopped** variable usage.

As the server code uses **fileevent**(n) handlers to process the input on sockets you will need to run the event loop. This means either you should be running from within **wish**(1) or you should **vwait**(n) on the **::smtpd::stopped** variable which is set when the server is stopped.

::smtpd::stop

Halt the server and release the listening socket. If the server has not been started then this command does nothing. The **::smtpd::stopped** variable is set for use with **vwait**(n).

It should be noted that stopping the server does not disconnect any currently active sessions as these are operating over an independent channel. Only explicitly tracking and closing these sessions, or exiting the server process will close down all the running sessions. This is similar to the usual unix daemon practice where the server performs a **fork**(2) and the client session continues on the child process.

::smptd::configure ?*option value*? ?*option value...*?

Set configuration options for the SMTP server. Most values are the name of a callback procedure to be called at various points in the SMTP protocol. See the Callbacks section for details of the procedures.

-banner *text*

Text of a custom banner message. The default banner is "tcllib smtpd 1.6". Note that changing the banner does not affect the bracketing text in the full greeting, printing status 220, server-address, and timestamp.

-validate_host *proc*

Callback to authenticate new connections based on the ip-address of the client.

-validate_sender *proc*

Callback to authenticate new connections based on the senders email address.

-validate_recipient *proc*

Callback to validate and authorize a recipient email address

-deliverMIME *proc*
Callback used to deliver mail as a mime token created by the tcllib **mime** package.

-deliver *proc*
Callback used to deliver email. This option has no effect if the **-deliverMIME** option has been set.

::smtpd::cget ?*option*?
If no *option* is specified the command will return a list of all options and their current values. If an option is specified it will return the value of that option.

Callbacks

validate_host callback
This procedure is called with the clients ip address as soon as a connection request has been accepted and before any protocol commands are processed. If you wish to deny access to a specific host then an error should be returned by this callback. For example:

```
proc validate_host {ipnum} {
   if {[string match "192.168.1.*" $ipnum]} {
      error "go away!"
   }
}
```

If access is denied the client will receive a standard message that includes the text of your error, such as:

```
550 Access denied: I hate you.
```

As per the SMTP protocol, the connection is not closed but we wait for the client to send a QUIT command. Any other commands cause a **503 Bad Sequence** error.

validate_sender callback
The validate_sender callback is called with the senders mail address during processing of a MAIL command to allow you to accept or reject mail based upon the declared sender. To reject mail you should throw an error. For example, to reject mail from user "denied":

```
proc validate_sender {address} {
   eval array set addr [mime::parseaddress $address]
   if {[string match "denied" $addr(local)]} {
        error "mailbox $addr(local) denied"
   }
   return
}
```

The content of any error message will not be passed back to the client.

validate_recipient callback
The validate_recipient callback is similar to the validate_sender callback and permits you to verify a local mailbox and accept mail for a local user address during RCPT command handling. To reject mail, throw an error as above. The error message is ignored.

deliverMIME callback

The deliverMIME callback is called once a mail message has been successfully passed to the server. A mime token is constructed from the sender, recipients and data and the users procedure it called with this single argument. When the call returns, the mime token is cleaned up so if the user wishes to preserve the data she must make a copy.

```
proc deliverMIME {token} {
    set sender [lindex [mime::getheader $token From] 0]
    set recipients [lindex [mime::getheader $token To] 0]
    set mail "From $sender [clock format [clock seconds]]"
    append mail "\n" [mime::buildmessage $token]
    puts $mail
}
```

deliver callback

The deliver callback is called once a mail message has been successfully passed to the server and there is no -deliverMIME option set. The procedure is called with the sender, a list of recipients and the text of the mail as a list of lines. For example:

```
proc deliver {sender recipients data} {
   set mail "From $sender  [clock format [clock seconds]]"
   append mail "\n" [join $data "\n"]
   puts "$mail"
}
```

Note that the DATA command will return an error if no sender or recipient has yet been defined.

Variables

::smtpd::stopped

This variable is set to **true** during the **::smtpd::stop** command to permit the use of the **vwait**(n) command.

Author

Written by Pat Thoyts *mailto:patthoyts@users.sourceforge.net*.

License

This software is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the file "*license.terms*" for more details.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *smtpd* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be

made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Networking

Copyright
```
Copyright (c) Pat Thoyts <patthoyts@users.sourceforge.net>
```

snit

snit
Keywords: snit, BWidget, C++, Incr Tcl, adaptors, class, mega widget, object, object oriented, type, widget, widget adaptors

Name
snit - Snit's Not Incr Tcl

Synopsis
package require Tcl 8.5 9
package require snit ?**2.3.4**?
snit::type *name definition*
typevariable *name* ?**-array**? ?*value*?
typemethod *name arglist body*
typeconstructor *body*
variable *name* ?**-array**? ?*value*?
method *name arglist body*
option *namespec* ?*defaultValue*?
option *namespec* ?*options...*?
constructor *arglist body*
destructor *body*
proc *name args body*
delegate method *name* **to** *comp* ?*as target*?
delegate method *name* ?*to comp*? **using** *pattern*
delegate method * ?*to comp*? ?*using pattern*? ?*except exceptions*?
delegate option *namespec* **to** *comp*
delegate option *namespec* **to** *comp* **as** *target*
delegate option * **to** *comp*
delegate option * **to** *comp* **except** *exceptions*
component *comp* ?*-public method*? ?*-inherit flag*?
delegate typemethod *name* **to** *comp* ?*as target*?
delegate typemethod *name* ?*to comp*? **using** *pattern*
delegate typemethod * ?*to comp*? ?*using pattern*? ?*except exceptions*?
typecomponent *comp* ?*-public typemethod*? ?*-inherit flag*?
pragma ?*options...*?
expose *comp*

expose *comp* **as** *method*
onconfigure *name arglist body*
oncget *name body*
snit::widget *name definition*
widgetclass *name*
hulltype *type*
snit::widgetadaptor *name definition*
snit::typemethod *type name arglist body*
snit::method *type name arglist body*
snit::macro *name arglist body*
snit::compile *which type body*
$type *typemethod args...*
$type create *name* ?*option value ...*?
$type info typevars ?*pattern*?
$type info typemethods ?*pattern*?
$type info args *method*
$type info body *method*
$type info default *method aname varname*
$type info instances ?*pattern*?
$type destroy
$object *method args...*
$object configure ?*option*? ?*value*? ...
$object configurelist *optionlist*
$object cget *option*
$object destroy
$object info type
$object info vars ?*pattern*?
$object info typevars ?*pattern*?
$object info typemethods ?*pattern*?
$object info options ?*pattern*?
$object info methods ?*pattern*?
$object info args *method*
$object info body *method*
$object info default *method aname varname*
mymethod *name* ?*args...*?
mytypemethod *name* ?*args...*?
myproc *name* ?*args...*?
myvar *name*
mytypevar *name*
from *argvName option* ?*defvalue*?
install *compName* **using** *objType objName args...*
installhull using *widgetType args...*
installhull *name*
variable *name*
typevariable *name*
varname *name*

typevarname *name*
codename *name*
snit::boolean validate ?*value*?
snit::boolean *name*
snit::double validate ?*value*?
snit::double *name* ?*option value...*?
snit::enum validate ?*value*?
snit::enum *name* ?*option value...*?
snit::fpixels validate ?*value*?
snit::fpixels *name* ?*option value...*?
snit::integer validate ?*value*?
snit::integer *name* ?*option value...*?
snit::listtype validate ?*value*?
snit::listtype *name* ?*option value...*?
snit::pixels validate ?*value*?
snit::pixels *name* ?*option value...*?
snit::stringtype validate ?*value*?
snit::stringtype *name* ?*option value...*?
snit::window validate ?*value*?
snit::window *name*

Description

Snit is a pure Tcl object and megawidget system. It's unique among Tcl object systems in that it's based not on inheritance but on delegation. Object systems based on inheritance only allow you to inherit from classes defined using the same system, which is limiting. In Tcl, an object is anything that acts like an object; it shouldn't matter how the object was implemented. Snit is intended to help you build applications out of the materials at hand; thus, Snit is designed to be able to incorporate and build on any object, whether it's a hand-coded object, a **Tk** widget, an **Incr Tcl** object, a **BWidget** or almost anything else.

This man page is intended to be a reference only; see the accompanying snitfaq for a gentler, more tutorial introduction to Snit concepts.

Snit versions

This man page covers both Snit 2.2 and Snit 1.3. The primary difference between the two versions is simply that Snit 2.2 contains speed optimizations based on new features of Tcl 8.5; Snit 1.3 supports all of Tcl 8.3, 8.4 and Tcl 8.5. There are a few minor inconsistencies; they are flagged in the body of the man page with the label "Snit 1.x Incompatibility"; they are also discussed in the snitfaq.

Reference

Type and widget definitions

Snit provides the following commands for defining new types:

snit::type *name definition*

Defines a new abstract data type called *name*. If *name* is not a fully qualified command name, it is assumed to be a name in the namespace in which the **snit::type** command was called (usually the global namespace). It returns the fully qualified name of the new type.

The type name is then a command that is used to create objects of the new type, along with other activities.

The **snit::type** *definition* block is a script that may contain the following definitions:

typevariable *name* ?-array? ?*value*?
: Defines a type variable with the specified *name*, and optionally the specified *value*. Type variables are shared by all instances of the type. If the **-array** option is included, then *value* should be a dictionary; it will be assigned to the variable using **array set**.

typemethod *name arglist body*
: Defines a type method, a subcommand of the new type command, with the specified name, argument list, and body. The *arglist* is a normal Tcl argument list and may contain default arguments and the **args** argument; however, it may not contain the argument names **type**, **self**, **selfns**, or **win**.

  The variable **type** is automatically defined in the *body* to the type's fully-qualified name. In addition, type variables are automatically visible in the *body* of every type method.

  If the *name* consists of two or more tokens, Snit handles it specially:

```
typemethod {a b} {arg} { puts "Got $arg" }
```

  This statement implicitly defines a type method called **a** which has a subcommand **b**. **b** is called like this:

```
$type a b "Hello, world!"
```

  a may have any number of subcommands. This makes it possible to define a hierarchical command structure; see **method**, below, for more examples.

  Type methods can call commands from the namespace in which the type is defined without importing them, e.g., if the type name is **::parentns::typename**, then the type's type methods can call **::parentns::someproc** just as **someproc**. *Snit 1.x Incompatibility:* This does not work in Snit 1.x, as it depends on **namespace path**, a new command in Tcl 8.5.

  Snit 1.x Incompatibility: In Snit 1.x, the following following two calls to this type method are equivalent:

```
$type a b "Hello, world!"
$type {a b} "Hello, world!"
```

  In Snit 2.2, the second form is invalid.

typeconstructor *body*
: The type constructor's *body* is executed once when the type is first defined; it is typically used to initialize array-valued type variables and to add entries to The Tk option database.

  The variable **type** is automatically defined in the *body*, and contains the type's fully-qualified name. In addition, type variables are automatically visible in the *body* of the type constructor.

  A type may define at most one type constructor.

The type constructor can call commands from the namespace in which the type is defined without importing them, e.g., if the type name is **::parentns::typename**, then the type constructor can call **::parentns::someproc** just as **someproc**. *Snit 1.x Incompatibility:* This does not work in Snit 1.x, as it depends on **namespace path**, a new command in Tcl 8.5.

variable *name* ?-array? ?*value*?

Defines an instance variable, a private variable associated with each instance of this type, and optionally its initial value. If the **-array** option is included, then *value* should be a dictionary; it will be assigned to the variable using **array set**.

method *name arglist body*

Defines an instance method, a subcommand of each instance of this type, with the specified name, argument list and body. The *arglist* is a normal Tcl argument list and may contain default arguments and the **args** argument.

The method is implicitly passed the following arguments as well: **type**, which contains the fully-qualified type name; **self**, which contains the current instance command name; **selfns**, which contains the name of the instance's private namespace; and **win**, which contains the original instance name. Consequently, the *arglist* may not contain the argument names **type**, **self**, **selfns**, or **win**.

An instance method defined in this way is said to be *locally defined*.

Type and instance variables are automatically visible in all instance methods. If the type has locally defined options, the **options** array is also visible.

If the *name* consists of two or more tokens, Snit handles it specially:

```
    method {a b} {} { ... }
```

This statement implicitly defines a method called **a** which has a subcommand **b**. **b** is called like this:

```
    $self a b "Hello, world!"
```

a may have any number of subcommands. This makes it possible to define a hierarchical command structure:

```
% snit::type dog {
    method {tail wag}   {} {return "Wag, wag"}
    method {tail droop} {} {return "Droop, droop"}
}
::dog
% dog spot
::spot
% spot tail wag
Wag, wag
% spot tail droop
Droop, droop
%
```

What we've done is implicitly defined a "tail" method with subcommands "wag" and "droop". Consequently, it's an error to define "tail" explicitly.

Methods can call commands from the namespace in which the type is defined without importing them, e.g., if the type name is **::parentns::typename**, then the type's methods can call **::parentns::someproc** just as **someproc**. *Snit 1.x Incompatibility:* This does not work in Snit 1.x, as it depends on **namespace path**, a new command in Tcl 8.5.

Snit 1.x Incompatibility: In Snit 1.x, the following following two calls to this method are equivalent:

```
    $self a b "Hello, world!"
    $self {a b} "Hello, world!"
```

In Snit 2.2, the second form is invalid.

option *namespec* ?*defaultValue*?

see below ...

option *namespec* ?*options...*?

Defines an option for instances of this type, and optionally gives it an initial value. The initial value defaults to the empty string if no *defaultValue* is specified.

An option defined in this way is said to be *locally defined*.

The *namespec* is a list defining the option's name, resource name, and class name, e.g.:

```
    option {-font font Font} {Courier 12}
```

The option name must begin with a hyphen, and must not contain any upper case letters. The resource name and class name are optional; if not specified, the resource name defaults to the option name, minus the hyphen, and the class name defaults to the resource name with the first letter capitalized. Thus, the following statement is equivalent to the previous example:

```
    option -font {Courier 12}
```

See The Tk option database for more information about resource and class names.

Options are normally set and retrieved using the standard instance methods **configure** and **cget**; within instance code (method bodies, etc.), option values are available through the **options** array:

```
    set myfont $options(-font)
```

If the type defines any option handlers (e.g., **-configuremethod**), then it should probably use **configure** and **cget** to access its options to avoid subtle errors.

The **option** statement may include the following options:

-default *defvalue*
: Defines the option's default value; the option's default value will be "" otherwise.

-readonly *flag*
: The *flag* can be any Boolean value recognized by Tcl. If *flag* is true, then the option is read-only--it can only be set using **configure** or **configurelist** at creation time, i.e., in the type's constructor.

-type *type*
: Every locally-defined option may define its validation type, which may be either the name of a validation type or a specification for a validation subtype

  For example, an option may declare that its value must be an integer by specifying **snit::integer** as its validation type:

```
    option -number -type snit::integer
```

  It may also declare that its value is an integer between 1 and 10 by specifying a validation subtype:

```
    option -number -type {snit::integer -min 1 -max 10}
```

  If a validation type or subtype is defined for an option, then it will be used to validate the option's value whenever it is changed by the object's **configure** or **configurelist** methods. In addition, all such options will have their values validated automatically immediately after the constructor executes.

  Snit defines a family of validation types and subtypes, and it's quite simple to define new ones. See Validation types for the complete list, and Defining validation types for an explanation of how to define your own.

-cgetmethod *methodName*
: Every locally-defined option may define a **-cgetmethod**; it is called when the option's value is retrieved using the **cget** method. Whatever the method's *body* returns will be the return value of the call to **cget**.

  The named method must take one argument, the option name. For example, this code is equivalent to (though slower than) Snit's default handling of **cget**:

```
option -font -cgetmethod GetOption
method GetOption {option} {
    return $options($option)
}
```

Note that it's possible for any number of options to share a **-cgetmethod**.

-configuremethod *methodName*
Every locally-defined option may define a **-configuremethod**; it is called when the option's value is set using the **configure** or **configurelist** methods. It is the named method's responsibility to save the option's value; in other words, the value will not be saved to the **options()** array unless the method saves it there.

The named method must take two arguments, the option name and its new value. For example, this code is equivalent to (though slower than) Snit's default handling of **configure**:

```
option -font -configuremethod SetOption
method SetOption {option value} {
    set options($option) $value
}
```

Note that it's possible for any number of options to share a single **-configuremethod**.

-validatemethod *methodName*
Every locally-defined option may define a **-validatemethod**; it is called when the option's value is set using the **configure** or **configurelist** methods, just before the **-configuremethod** (if any). It is the named method's responsibility to validate the option's new value, and to throw an error if the value is invalid.

The named method must take two arguments, the option name and its new value. For example, this code verifies that **-flag**'s value is a valid Boolean value:

```
option -font -validatemethod CheckBoolean
method CheckBoolean {option value} {
    if {![string is boolean -strict $value]} {
        error "option $option must have a boolean value."
    }
}
```

Note that it's possible for any number of options to share a single **-validatemethod**.

constructor *arglist body*
The constructor definition specifies a *body* of code to be executed when a new instance is created. The *arglist* is a normal Tcl argument list and may contain default arguments and the **args** argument.

As with methods, the arguments **type**, **self**, **selfns**, and **win** are defined implicitly, and all type and instance variables are automatically visible in its *body*.

If the *definition* doesn't explicitly define the constructor, Snit defines one implicitly. If the

type declares at least one option (whether locally or by delegation), the default constructor will be defined as follows:

```
constructor {args} {
    $self configurelist $args
}
```

For standard Tk widget behavior, the argument list should be the single name **args**, as shown.

If the *definition* defines neither a constructor nor any options, the default constructor is defined as follows:

```
constructor {} {}
```

As with methods, the constructor can call commands from the namespace in which the type is defined without importing them, e.g., if the type name is **::parentns::typename**, then the constructor can call **::parentns::someproc** just as **someproc**. *Snit 1.x Incompatibility:* This does not work in Snit 1.x, as it depends on **namespace path**, a new command in Tcl 8.5.

destructor *body*

The destructor is used to code any actions that must take place when an instance of the type is destroyed: typically, the destruction of anything created in the constructor.

The destructor takes no explicit arguments; as with methods, the arguments **type**, **self**, **selfns**, and **win**, are defined implicitly, and all type and instance variables are automatically visible in its *body*. As with methods, the destructor can call commands from the namespace in which the type is defined without importing them, e.g., if the type name is **::parentns::typename**, then the destructor can call **::parentns::someproc** just as **someproc**. *Snit 1.x Incompatibility:* This does not work in Snit 1.x, as it depends on **namespace path**, a new command in Tcl 8.5.

proc *name args body*

Defines a new Tcl procedure in the type's namespace.

The defined proc differs from a normal Tcl proc in that all type variables are automatically visible. The proc can access instance variables as well, provided that it is passed **selfns** (with precisely that name) as one of its arguments.

Although they are not implicitly defined for procs, the argument names **type**, **self**, and **win** should be avoided.

As with methods and typemethods, procs can call commands from the namespace in which the type is defined without importing them, e.g., if the type name is **::parentns::typename**, then the proc can call **::parentns::someproc** just as **someproc**. *Snit 1.x Incompatibility:* This does not work in Snit 1.x, as it depends on **namespace path**, a new command in Tcl 8.5.

delegate method *name* to *comp* ?as *target*?

Delegates method *name* to component *comp*. That is, when method *name* is called on an instance of this type, the method and its arguments will be passed to the named component's command instead. That is, the following statement

```
    delegate method wag to tail
```

is roughly equivalent to this explicitly defined method:

```
    method wag {args} {
        uplevel $tail wag $args
    }
```

As with methods, the *name* may have multiple tokens; in this case, the last token of the name is assumed to be the name of the component's method.

The optional **as** clause allows you to specify the delegated method name and possibly add some arguments:

```
    delegate method wagtail to tail as "wag briskly"
```

A method cannot be both locally defined and delegated.

Note: All forms of **delegate method** can delegate to both instance components and type components.

delegate method *name* ?to *comp*? using *pattern*

In this form of the **delegate** statement, the **using** clause is used to specify the precise form of the command to which method *name* name is delegated. In this form, the **to** clause is optional, since the chosen command might not involve any particular component.

The value of the **using** clause is a list that may contain any or all of the following substitution codes; these codes are substituted with the described value to build the delegated command prefix. Note that the following two statements are equivalent:

```
    delegate method wag to tail
    delegate method wag to tail using "%c %m"
```

Each element of the list becomes a single element of the delegated command--it is never reparsed as a string.

Substitutions:

%%
: This is replaced with a single "%". Thus, to pass the string "%c" to the command as an argument, you'd write "%%c".

%c
: This is replaced with the named component's command.

%m
: This is replaced with the final token of the method *name*; if the method *name* has one token, this is identical to **%M**.

%M
: This is replaced by the method *name*; if the *name* consists of multiple tokens, they are

joined by space characters.

%j
This is replaced by the method *name*; if the *name* consists of multiple tokens, they are joined by underscores ("_").

%t
This is replaced with the fully qualified type name.

%n
This is replaced with the name of the instance's private namespace.

%s
This is replaced with the name of the instance command.

%w
This is replaced with the original name of the instance command; for Snit widgets and widget adaptors, it will be the Tk window name. It remains constant, even if the instance command is renamed.

delegate method * ?to *comp*? ?using *pattern*? ?except *exceptions*?
The form **delegate method *** delegates all unknown method names to the specified *comp* onent. The **except** clause can be used to specify a list of *exceptions*, i.e., method names that will not be so delegated. The **using** clause is defined as given above. In this form, the statement must contain the **to** clause, the **using** clause, or both.

In fact, the "*" can be a list of two or more tokens whose last element is "*", as in the following example:

```
    delegate method {tail *} to tail
```

This implicitly defines the method **tail** whose subcommands will be delegated to the **tail** component.

delegate option *namespec* to *comp*
see below ...

delegate option *namespec* to *comp* as *target*
see below ...

delegate option * to *comp*
see below ...

delegate option * to *comp* except *exceptions*
Defines a delegated option; the *namespec* is defined as for the **option** statement. When the **configure**, **configurelist**, or **cget** instance method is used to set or retrieve the option's value, the equivalent **configure** or **cget** command will be applied to the component as though the option was defined with the following **-configuremethod** and **-cgetmethod**:

```
    method ConfigureMethod {option value} {
        $comp configure $option $value
    }

    method CgetMethod {option} {
        return [$comp cget $option]
    }
```

Note that delegated options never appear in the **options** array.

If the **as** clause is specified, then the *target* option name is used in place of *name*.

The form **delegate option *** delegates all unknown options to the specified *comp*onent. The **except** clause can be used to specify a list of *exceptions*, i.e., option names that will not be so delegated.

Warning: options can only be delegated to a component if it supports the **configure** and **cget** instance methods.

An option cannot be both locally defined and delegated. TBD: Continue from here.

component *comp* ?-public *method*? ?-inherit *flag*?

Explicitly declares a component called *comp*, and automatically defines the component's instance variable.

If the **-public** option is specified, then the option is made public by defining a *method* whose subcommands are delegated to the component e.g., specifying **-public mycomp** is equivalent to the following:

```
    component mycomp
    delegate method {mymethod *} to mycomp
```

If the **-inherit** option is specified, then *flag* must be a Boolean value; if *flag* is true then all unknown methods and options will be delegated to this component. The name **-inherit** implies that instances of this new type inherit, in a sense, the methods and options of the component. That is, **-inherit yes** is equivalent to:

```
    component mycomp
    delegate option * to mycomp
    delegate method * to mycomp
```

delegate typemethod *name* to *comp* ?as *target*?

Delegates type method *name* to type component *comp*. That is, when type method *name* is called on this type, the type method and its arguments will be passed to the named type component's command instead. That is, the following statement

```
    delegate typemethod lostdogs to pound
```

is roughly equivalent to this explicitly defined method:

```
    typemethod lostdogs {args} {
        uplevel $pound lostdogs $args
    }
```

As with type methods, the *name* may have multiple tokens; in this case, the last token of the name is assumed to be the name of the component's method.

The optional **as** clause allows you to specify the delegated method name and possibly add some arguments:

```
    delegate typemethod lostdogs to pound as "get lostdogs"
```

A type method cannot be both locally defined and delegated.

delegate typemethod *name* ?to *comp*? using *pattern*

In this form of the **delegate** statement, the **using** clause is used to specify the precise form of the command to which type method *name* name is delegated. In this form, the **to** clause is optional, since the chosen command might not involve any particular type component.

The value of the **using** clause is a list that may contain any or all of the following substitution codes; these codes are substituted with the described value to build the delegated command prefix. Note that the following two statements are equivalent:

```
    delegate typemethod lostdogs to pound
    delegate typemethod lostdogs to pound using "%c %m"
```

Each element of the list becomes a single element of the delegated command--it is never reparsed as a string.

Substitutions:

%%
: This is replaced with a single "%". Thus, to pass the string "%c" to the command as an argument, you'd write "%%c".

%c
: This is replaced with the named type component's command.

%m
: This is replaced with the final token of the type method *name*; if the type method *name* has one token, this is identical to **%M**.

%M
: This is replaced by the type method *name*; if the *name* consists of multiple tokens, they are joined by space characters.

%j
: This is replaced by the type method *name*; if the *name* consists of multiple tokens, they are joined by underscores ("_").

%t
: This is replaced with the fully qualified type name.

delegate typemethod * ?to *comp*? ?using *pattern*? ?except *exceptions*?

The form **delegate typemethod *** delegates all unknown type method names to the specified type component. The **except** clause can be used to specify a list of *exceptions*, i.e., type method names that will not be so delegated. The **using** clause is defined as given above. In this form, the statement must contain the **to** clause, the **using** clause, or both.

Note: By default, Snit interprets **$type foo**, where **foo** is not a defined type method, as equivalent to **$type create foo**, where **foo** is the name of a new instance of the type. If you use **delegate typemethod ***, then the **create** type method must always be used explicitly.

The "*" can be a list of two or more tokens whose last element is "*", as in the following example:

```
    delegate typemethod {tail *} to tail
```

This implicitly defines the type method **tail** whose subcommands will be delegated to the **tail** type component.

typecomponent *comp* ?-public *typemethod*? ?-inherit *flag*?

Explicitly declares a type component called *comp*, and automatically defines the component's type variable. A type component is an arbitrary command to which type methods and instance methods can be delegated; the command's name is stored in a type variable.

If the **-public** option is specified, then the type component is made public by defining a *typemethod* whose subcommands are delegated to the type component, e.g., specifying **-public mytypemethod** is equivalent to the following:

```
    typecomponent mycomp
    delegate typemethod {mytypemethod *} to mycomp
```

If the **-inherit** option is specified, then *flag* must be a Boolean value; if *flag* is true then all unknown type methods will be delegated to this type component. (See the note on "delegate typemethod *", above.) The name **-inherit** implies that this type inherits, in a sense, the behavior of the type component. That is, **-inherit yes** is equivalent to:

```
    typecomponent mycomp
    delegate typemethod * to mycomp
```

pragma ?*options...*?

The **pragma** statement provides control over how Snit generates a type. It takes the following options; in each case, *flag* must be a Boolean value recognized by Tcl, e.g., **0**, **1**, **yes**, **no**, and so on.

By setting the **-hastypeinfo**, **-hastypedestroy**, and **-hasinstances** pragmas to false and defining appropriate type methods, you can create an ensemble command without any extraneous behavior.

-canreplace *flag*

If false (the default) Snit will not create an instance of a **snit::type** that has the same name as an existing command; this prevents subtle errors. Setting this pragma to true restores the behavior of Snit V0.93 and earlier versions.

-hastypeinfo *flag*
If true (the default), the generated type will have a type method called **info** that is used for type introspection; the **info** type method is documented below. If false, it will not.

-hastypedestroy *flag*
If true (the default), the generated type will have a type method called **destroy** that is used to destroy the type and all of its instances. The **destroy** type method is documented below. If false, it will not.

-hastypemethods *flag*
If true (the default), the generated type's type command will have subcommands (type methods) as usual. If false, the type command will serve only to create instances of the type; the first argument is the instance name.

This pragma and **-hasinstances** cannot both be set false.

-hasinstances *flag*
If true (the default), the generated type will have a type method called **create** that is used to create instances of the type, along with a variety of instance-related features. If false, it will not.

This pragma and **-hastypemethods** cannot both be set false.

-hasinfo *flag*
If true (the default), instances of the generated type will have an instance method called **info** that is used for instance introspection; the **info** method is documented below. If false, it will not.

-simpledispatch *flag*
This pragma is intended to make simple, heavily-used abstract data types (e.g., stacks and queues) more efficient.

If false (the default), instance methods are dispatched normally. If true, a faster dispatching scheme is used instead. The speed comes at a price; with **-simpledispatch yes** you get the following limitations:
- Methods cannot be delegated.
- **uplevel** and **upvar** do not work as expected: the caller's scope is two levels up rather than one.
- The option-handling methods (**cget**, **configure**, and **configurelist**) are very slightly slower.

expose *comp*
see below ...

expose *comp* as *method*
Deprecated. To expose component *comp* publicly, use **component**'s **-public** option.

onconfigure *name arglist body*
Deprecated. Define **option**'s **-configuremethod** option instead.

As of version 0.95, the following definitions,

```
option -myoption
onconfigure -myoption {value} {
    # Code to save the option's value
}
```

are implemented as follows:

```
option -myoption -configuremethod _configure-myoption
method _configure-myoption {_option value} {
    # Code to save the option's value
}
```

oncget *name body*

Deprecated. Define **option**'s **-cgetmethod** option instead.

As of version 0.95, the following definitions,

```
option -myoption
oncget -myoption {
    # Code to return the option's value
}
```

are implemented as follows:

```
option -myoption -cgetmethod _cget-myoption
method _cget-myoption {_option} {
    # Code to return the option's value
}
```

snit::widget *name definition*

This command defines a Snit megawidget type with the specified *name*. The *definition* is defined as for **snit::type**. A **snit::widget** differs from a **snit::type** in these ways:

- Every instance of a **snit::widget** has an automatically-created component called **hull**, which is normally a Tk frame widget. Other widgets created as part of the megawidget will be created within this widget.

  The hull component is initially created with the requested widget name; then Snit does some magic, renaming the hull component and installing its own instance command in its place. The hull component's new name is saved in an instance variable called **hull**.
- The name of an instance must be valid Tk window name, and the parent window must exist.

A **snit::widget** definition can include any of statements allowed in a **snit::type** definition, and may also include the following:

widgetclass *name*

Sets the **snit::widget**'s widget class to *name*, overriding the default. See The Tk option database for more information.

hulltype *type*

Determines the kind of widget used as the **snit::widget**'s hull. The *type* may be **frame**

(the default), **toplevel**, **labelframe**; the qualified equivalents of these, **tk::frame**, **tk::toplevel**, and **tk::labelframe**; or, if available, the equivalent Tile widgets: **ttk::frame**, **ttk::toplevel**, and **ttk::labelframe**. In practice, any widget that supports the **-class** option can be used as a hull widget by **lappend**'ing its name to the variable **snit::hulltypes**.

snit::widgetadaptor *name definition*

This command defines a Snit megawidget type with the specified name. It differs from **snit::widget** in that the instance's **hull** component is not created automatically, but is created in the constructor and installed using the **installhull** command. Once the hull is installed, its instance command is renamed and replaced as with normal **snit::widget**s. The original command is again accessible in the instance variable **hull**.

Note that in general it is not possible to change the *widget class* of a **snit::widgetadaptor**'s hull widget.

See The Tk option database for information on how **snit::widgetadaptor**s interact with the option database.

snit::typemethod *type name arglist body*

Defines a new type method (or redefines an existing type method) for a previously existing *type*.

snit::method *type name arglist body*

Defines a new instance method (or redefines an existing instance method) for a previously existing *type*. Note that delegated instance methods can't be redefined.

snit::macro *name arglist body*

Defines a Snit macro with the specified *name*, *arglist*, and *body*. Macros are used to define new type and widget definition statements in terms of the statements defined in this man page.

A macro is simply a Tcl proc that is defined in the slave interpreter used to compile type and widget definitions. Thus, macros have access to all of the type and widget definition statements. See Macros and meta-programming for more details.

The macro *name* cannot be the same as any standard Tcl command, or any Snit type or widget definition statement, e.g., you can't redefine the **method** or **delegate** statements, or the standard **set**, **list**, or **string** commands.

snit::compile *which type body*

Snit defines a type, widget, or widgetadaptor by "compiling" the definition into a Tcl script; this script is then evaluated in the Tcl interpreter, which actually defines the new type.

This command exposes the "compiler". Given a definition *body* for the named *type*, where *which* is **type**, **widget**, or **widgetadaptor**, **snit::compile** returns a list of two elements. The first element is the fully qualified type name; the second element is the definition script.

snit::compile is useful when additional processing must be done on the Snit-generated code--if it must be instrumented, for example, or run through the TclDevKit compiler. In

addition, the returned script could be saved in a ".tcl" file and used to define the type as part of an application or library, thus saving the compilation overhead at application start-up. Note that the same version of Snit must be used at run-time as at compile-time.

The type command

A type or widget definition creates a type command, which is used to create instances of the type. The type command has this form:

$type *typemethod args...*

The *typemethod* can be any of the Standard type methods (e.g., **create**), or any type method defined in the type definition. The subsequent *args* depend on the specific *typemethod* chosen.

The type command is most often used to create new instances of the type; hence, the **create** method is assumed if the first argument to the type command doesn't name a valid type method, unless the type definition includes **delegate typemethod *** or the **-hasinstances** pragma is set to false.

Furthermore, if the **-hastypemethods** pragma is false, then Snit type commands can be called with no arguments at all; in this case, the type command creates an instance with an automatically generated name. In other words, provided that the **-hastypemethods** pragma is false and the type has instances, the following commands are equivalent:

```
snit::type dog { ... }

set mydog [dog create %AUTO%]
set mydog [dog %AUTO%]
set mydog [dog]
```

This doesn't work for Snit widgets, for obvious reasons.

Snit 1.x Incompatibility: In Snit 1.x, the above behavior is available whether **-hastypemethods** is true (the default) or false.

Standard type methods

In addition to any type methods in the type's definition, all type and widget commands will usually have at least the following subcommands:

$type create *name* ?*option value* ...?

Creates a new instance of the type, giving it the specified *name* and calling the type's constructor.

For **snit::type**s, if *name* is not a fully-qualified command name, it is assumed to be a name in the namespace in which the call to **snit::type** appears. The method returns the fully-qualified instance name.

For **snit::widget**s and **snit::widgetadaptor**s, *name* must be a valid widget name; the method returns the widget name.

So long as *name* does not conflict with any defined type method name the **create** keyword

may be omitted, unless the type definition includes **delegate typemethod** * or the **-hasinstances** pragma is set to false.

If the *name* includes the string **%AUTO%**, it will be replaced with the string **$type$counter** where **$type** is the type name and **$counter** is a counter that increments each time **%AUTO%** is used for this type.

By default, any arguments following the *name* will be a list of *option* names and their *value*s; however, a type's constructor can specify a different argument list.

As of Snit V0.95, **create** will throw an error if the *name* is the same as any existing command--note that this was always true for **snit::widget**s and **snit::widgetadaptor**s. You can restore the previous behavior using the **-canreplace** pragma.

$type info typevars ?*pattern*?
Returns a list of the type's type variables (excluding Snit internal variables); all variable names are fully-qualified.

If *pattern* is given, it's used as a **string match** pattern; only names that match the pattern are returned.

$type info typemethods ?*pattern*?
Returns a list of the names of the type's type methods. If the type has hierarchical type methods, whether locally-defined or delegated, only the first word of each will be included in the list.

If the type definition includes **delegate typemethod** *, the list will include only the names of those implicitly delegated type methods that have been called at least once and are still in the type method cache.

If *pattern* is given, it's used as a **string match** pattern; only names that match the pattern are returned.

$type info args *method*
Returns a list containing the names of the arguments to the type's *method*, in order. This method cannot be applied to delegated type methods.

$type info body *method*
Returns the body of typemethod *method*. This method cannot be applied to delegated type methods.

$type info default *method aname varname*
Returns a boolean value indicating whether the argument *aname* of the type's *method* has a default value (**true**) or not (**false**). If the argument has a default its value is placed into the variable *varname*.

$type info instances ?*pattern*?
Returns a list of the type's instances. For **snit::type**s, it will be a list of fully-qualified instance names; for **snit::widget**s, it will be a list of Tk widget names.

If *pattern* is given, it's used as a **string match** pattern; only names that match the pattern

are returned.

Snit 1.x Incompatibility: In Snit 1.x, the full multi-word names of hierarchical type methods are included in the return value.

$type destroy
Destroys the type's instances, the type's namespace, and the type command itself.

The instance command

A Snit type or widget's **create** type method creates objects of the type; each object has a unique name that is also a Tcl command. This command is used to access the object's methods and data, and has this form:

$object *method args...*
The *method* can be any of the Standard instance methods, or any instance method defined in the type definition. The subsequent *args* depend on the specific *method* chosen.

Standard instance methods

In addition to any delegated or locally-defined instance methods in the type's definition, all Snit objects will have at least the following subcommands:

$object configure ?*option*? ?*value*? ...
Assigns new values to one or more options. If called with one argument, an *option* name, returns a list describing the option, as Tk widgets do; if called with no arguments, returns a list of lists describing all options, as Tk widgets do.

Warning: This information will be available for delegated options only if the component to which they are delegated has a **configure** method that returns this same kind of information.

Note: Snit defines this method only if the type has at least one option.

$object configurelist *optionlist*
Like **configure**, but takes one argument, a list of options and their values. It's mostly useful in the type constructor, but can be used anywhere.

Note: Snit defines this method only if the type has at least one option.

$object cget *option*
Returns the option's value.

Note: Snit defines this method only if the type has at least one option.

$object destroy
Destroys the object, calling the **destructor** and freeing all related memory.

Note: The **destroy** method isn't defined for **snit::widget** or **snit::widgetadaptor** objects; instances of these are destroyed by calling **Tk**'s **destroy** command, just as normal widgets are.

$object info type
Returns the instance's type.

$object info vars ?*pattern*?

Returns a list of the object's instance variables (excluding Snit internal variables). The names are fully qualified.

If *pattern* is given, it's used as a **string match** pattern; only names that match the pattern are returned.

$object info typevars ?*pattern*?
Returns a list of the object's type's type variables (excluding Snit internal variables). The names are fully qualified.

If *pattern* is given, it's used as a **string match** pattern; only names that match the pattern are returned.

$object info typemethods ?*pattern*?
Returns a list of the names of the type's type methods. If the type has hierarchical type methods, whether locally-defined or delegated, only the first word of each will be included in the list.

If the type definition includes **delegate typemethod** *, the list will include only the names of those implicitly delegated type methods that have been called at least once and are still in the type method cache.

If *pattern* is given, it's used as a **string match** pattern; only names that match the pattern are returned.

Snit 1.x Incompatibility: In Snit 1.x, the full multi-word names of hierarchical type methods are included in the return value.

$object info options ?*pattern*?
Returns a list of the object's option names. This always includes local options and explicitly delegated options. If unknown options are delegated as well, and if the component to which they are delegated responds to **$object configure** like Tk widgets do, then the result will include all possible unknown options that can be delegated to the component.

If *pattern* is given, it's used as a **string match** pattern; only names that match the pattern are returned.

Note that the return value might be different for different instances of the same type, if component object types can vary from one instance to another.

$object info methods ?*pattern*?
Returns a list of the names of the instance's methods. If the type has hierarchical methods, whether locally-defined or delegated, only the first word of each will be included in the list.

If the type definition includes **delegate method** *, the list will include only the names of those implicitly delegated methods that have been called at least once and are still in the method cache.

If *pattern* is given, it's used as a **string match** pattern; only names that match the pattern are returned.

Snit 1.x Incompatibility: In Snit 1.x, the full multi-word names of hierarchical type methods are included in the return value.

$object info args *method*
Returns a list containing the names of the arguments to the instance's *method*, in order. This method cannot be applied to delegated methods.

$object info body *method*
Returns the body of the instance's method *method*. This method cannot be applied to delegated methods.

$object info default *method aname varname*
Returns a boolean value indicating whether the argument *aname* of the instance's *method* has a default value (**true**) or not (**false**). If the argument has a default its value is placed into the variable *varname*.

Commands for use in object code

Snit defines the following commands for use in your object code: that is, for use in type methods, instance methods, constructors, destructors, onconfigure handlers, oncget handlers, and procs. They do not reside in the ::snit:: namespace; instead, they are created with the type, and can be used without qualification.

mymethod *name* ?*args...*?
The **mymethod** command is used for formatting callback commands to be passed to other objects. It returns a command that when called will invoke method *name* with the specified arguments, plus of course any arguments added by the caller. In other words, both of the following commands will cause the object's **dosomething** method to be called when the **$button** is pressed:

```
    $button configure -command [list $self dosomething myargument]

    $button configure -command [mymethod dosomething myargument]
```

The chief distinction between the two is that the latter form will not break if the object's command is renamed.

mytypemethod *name* ?*args...*?
The **mytypemethod** command is used for formatting callback commands to be passed to other objects. It returns a command that when called will invoke type method *name* with the specified arguments, plus of course any arguments added by the caller. In other words, both of the following commands will cause the object's **dosomething** type method to be called when **$button** is pressed:

```
    $button configure -command [list $type dosomething myargument]

    $button configure -command [mytypemethod dosomething myargument]
```

Type commands cannot be renamed, so in practice there's little difference between the two forms. **mytypemethod** is provided for parallelism with **mymethod**.

myproc *name* ?*args...*?

The **myproc** command is used for formatting callback commands to be passed to other objects. It returns a command that when called will invoke the type proc *name* with the specified arguments, plus of course any arguments added by the caller. In other words, both of the following commands will cause the object's **dosomething** proc to be called when **$button** is pressed:

```
    $button configure -command [list ${type}::dosomething myargument]

    $button configure -command [myproc dosomething myargument]
```

myvar *name*
Given an instance variable name, returns the fully qualified name. Use this if you're passing the variable to some other object, e.g., as a **-textvariable** to a Tk label widget.

mytypevar *name*
Given an type variable name, returns the fully qualified name. Use this if you're passing the variable to some other object, e.g., as a **-textvariable** to a Tk label widget.

from *argvName option ?defvalue?*
The **from** command plucks an option value from a list of options and their values, such as is passed into a type's **constructor**. *argvName* must be the name of a variable containing such a list; *option* is the name of the specific option.

from looks for *option* in the option list. If it is found, it and its value are removed from the list, and the value is returned. If *option* doesn't appear in the list, then the *defvalue* is returned. If the option is locally-defined option, and *defvalue* is not specified, then the option's default value as specified in the type definition will be returned instead.

install *compName* using *objType objName args...*
Creates a new object of type *objType* called *objName* and installs it as component *compName* , as described in Components and delegation. Any additional *args...* are passed along with the name to the *objType* command. If this is a **snit::type**, then the following two commands are equivalent:

```
    install myComp using myObjType $self.myComp args...

    set myComp [myObjType $self.myComp args...]
```

Note that whichever method is used, *compName* must still be declared in the type definition using **component**, or must be referenced in at least one **delegate** statement.

If this is a **snit::widget** or **snit::widgetadaptor**, and if options have been delegated to component *compName*, then those options will receive default values from the Tk option database. Note that it doesn't matter whether the component to be installed is a widget or not. See The Tk option database for more information.

install cannot be used to install type components; just assign the type component's command name to the type component's variable instead.

installhull using *widgetType args...*
see below ...

installhull *name*

The constructor of a **snit::widgetadaptor** must create a widget to be the object's hull component; the widget is installed as the hull component using this command. Note that the installed widget's name must be **$win**. This command has two forms.

The first form specifies the *widgetType* and the *args...* (that is, the hardcoded option list) to use in creating the hull. Given this form, **installhull** creates the hull widget, and initializes any options delegated to the hull from the Tk option database.

In the second form, the hull widget has already been created; note that its name must be "$win". In this case, the Tk option database is *not* queried for any options delegated to the hull. The longer form is preferred; however, the shorter form allows the programmer to adapt a widget created elsewhere, which is sometimes useful. For example, it can be used to adapt a "page" widget created by a **BWidgets** tabbed notebook or pages manager widget.

See The Tk option database for more information about **snit::widgetadaptor**s and the option database.

variable *name*

Normally, instance variables are defined in the type definition along with the options, methods, and so forth; such instance variables are automatically visible in all instance code (e.g., method bodies). However, instance code can use the **variable** command to declare instance variables that don't appear in the type definition, and also to bring variables from other namespaces into scope in the usual way.

It's generally clearest to define all instance variables in the type definition, and omit declaring them in methods and so forth.

Note that this is an instance-specific version of the standard Tcl **::variable** command.

typevariable *name*

Normally, type variables are defined in the type definition, along with the instance variables; such type variables are automatically visible in all of the type's code. However, type methods, instance methods and so forth can use **typevariable** to declare type variables that don't appear in the type definition.

It's generally clearest to declare all type variables in the type definition, and omit declaring them in methods, type methods, etc.

varname *name*

Deprecated. Use **myvar** instead.

Given an instance variable name, returns the fully qualified name. Use this if you're passing the variable to some other object, e.g., as a **-textvariable** to a Tk label widget.

typevarname *name*

Deprecated. Use **mytypevar** instead.

Given a type variable name, returns the fully qualified name. Use this if you're passing the type variable to some other object, e.g., as a **-textvariable** to a Tk label widget.

codename *name*

Deprecated. Use **myproc** instead. Given the name of a proc (but not a type or instance method), returns the fully-qualified command name, suitable for passing as a callback.

Components and delegation

When an object includes other objects, as when a toolbar contains buttons or a GUI object contains an object that references a database, the included object is called a component. The standard way to handle component objects owned by a Snit object is to declare them using **component**, which creates a component instance variable. In the following example, a **dog** object has a **tail** object:

```
snit::type dog {
    component mytail

    constructor {args} {
        set mytail [tail %AUTO% -partof $self]
        $self configurelist $args
    }

    method wag {} {
        $mytail wag
    }
}

snit::type tail {
    option -length 5
    option -partof
    method wag {} { return "Wag, wag, wag."}
}
```

Because the **tail** object's name is stored in an instance variable, it's easily accessible in any method.

The **install** command provides an alternate way to create and install the component:

```
snit::type dog {
    component mytail

    constructor {args} {
        install mytail using tail %AUTO% -partof $self
        $self configurelist $args
    }

    method wag {} {
        $mytail wag
    }
}
```

For **snit::type**s, the two methods are equivalent; for **snit::widget**s and **snit::widgetadaptor**s, the **install** command properly initializes the widget's options by querying The Tk option

database.

In the above examples, the **dog** object's **wag** method simply calls the **tail** component's **wag** method. In OO jargon, this is called delegation. Snit provides an easier way to do this:

```
    snit::type dog {
        delegate method wag to mytail

        constructor {args} {
            install mytail using tail %AUTO% -partof $self
            $self configurelist $args
        }
    }
```

The **delegate** statement in the type definition implicitly defines the instance variable **mytail** to hold the component's name (though it's good form to use **component** to declare it explicitly); it also defines the **dog** object's **wag** method, delegating it to the **mytail** component.

If desired, all otherwise unknown methods can be delegated to a specific component:

```
    snit::type dog {
    delegate method * to mytail

    constructor {args} {
        set mytail [tail %AUTO% -partof $self]
        $self configurelist $args
    }

    method bark { return "Bark, bark, bark!" }
    }
```

In this case, a **dog** object will handle its own **bark** method; but **wag** will be passed along to **mytail**. Any other method, being recognized by neither **dog** nor **tail**, will simply raise an error.

Option delegation is similar to method delegation, except for the interactions with the Tk option database; this is described in The Tk option database.

Type components and delegation

The relationship between type components and instance components is identical to that between type variables and instance variables, and that between type methods and instance methods. Just as an instance component is an instance variable that holds the name of a command, so a type component is a type variable that holds the name of a command. In essence, a type component is a component that's shared by every instance of the type.

Just as **delegate method** can be used to delegate methods to instance components, as described in Components and delegation, so **delegate typemethod** can be used to delegate type methods to type components.

Note also that as of Snit 0.95 **delegate method** can delegate methods to both instance components and type components.

The Tk option database

This section describes how Snit interacts with the Tk option database, and assumes the reader has a working knowledge of the option database and its uses. The book *Practical Programming in Tcl and Tk* by Welch et al has a good introduction to the option database, as does *Effective Tcl/Tk Programming*.

Snit is implemented so that most of the time it will simply do the right thing with respect to the option database, provided that the widget developer does the right thing by Snit. The body of this section goes into great deal about what Snit requires. The following is a brief statement of the requirements, for reference.

- If the **snit::widget**'s default widget class is not what is desired, set it explicitly using **widgetclass** in the widget definition.
- When defining or delegating options, specify the resource and class names explicitly when if the defaults aren't what you want.
- Use **installhull using** to install the hull for **snit::widgetadaptor**s.
- Use **install** to install all other components.

The interaction of Tk widgets with the option database is a complex thing; the interaction of Snit with the option database is even more so, and repays attention to detail.

Setting the widget class: Every Tk widget has a widget class. For Tk widgets, the widget class name is the just the widget type name with an initial capital letter, e.g., the widget class for **button** widgets is "Button".

Similarly, the widget class of a **snit::widget** defaults to the unqualified type name with the first letter capitalized. For example, the widget class of

```
    snit::widget ::mylibrary::scrolledText { ... }
```

is "ScrolledText". The widget class can also be set explicitly using the **widgetclass** statement within the **snit::widget** definition.

Any widget can be used as the **hulltype** provided that it supports the **-class** option for changing its widget class name. See the discussion of the **hulltype** command, above. The user may pass **-class** to the widget at instantion.

The widget class of a **snit::widgetadaptor** is just the widget class of its hull widget; this cannot be changed unless the hull widget supports **-class**, in which case it will usually make more sense to use **snit::widget** rather than **snit::widgetadaptor**.

Setting option resource names and classes: In Tk, every option has three names: the option name, the resource name, and the class name. The option name begins with a hyphen and is all lowercase; it's used when creating widgets, and with the **configure** and **cget** commands.

The resource and class names are used to initialize option default values by querying the Tk option database. The resource name is usually just the option name minus the hyphen, but may contain uppercase letters at word boundaries; the class name is usually just the resource name with an initial capital, but not always. For example, here are the option, resource, and class names for several **text** widget options:

```
    -background         background         Background
    -borderwidth        borderWidth        BorderWidth
    -insertborderwidth  insertBorderWidth  BorderWidth
    -padx               padX               Pad
```

As is easily seen, sometimes the resource and class names can be inferred from the option name, but not always.

Snit options also have a resource name and a class name. By default, these names follow the rule given above: the resource name is the option name without the hyphen, and the class name is the resource name with an initial capital. This is true for both locally-defined options and explicitly delegated options:

```
    snit::widget mywidget {
        option -background
        delegate option -borderwidth to hull
        delegate option * to text
    # ...
    }
```

In this case, the widget class name is "Mywidget". The widget has the following options: **-background**, which is locally defined, and **-borderwidth**, which is explicitly delegated; all other widgets are delegated to a component called "text", which is probably a Tk **text** widget. If so, **mywidget** has all the same options as a **text** widget. The option, resource, and class names are as follows:

```
    -background  background  Background
    -borderwidth borderwidth Borderwidth
    -padx        padX        Pad
```

Note that the locally defined option, **-background**, happens to have the same three names as the standard Tk **-background** option; and **-pad**, which is delegated implicitly to the **text** component, has the same three names for **mywidget** as it does for the **text** widget. **-borderwidth**, on the other hand, has different resource and class names than usual, because the internal word "width" isn't capitalized. For consistency, it should be; this is done as follows:

```
    snit::widget mywidget {
    option -background
    delegate option {-borderwidth borderWidth} to hull
    delegate option * to text
    # ...
    }
```

The class name will default to "BorderWidth", as expected.

Suppose, however, that **mywidget** also delegated **-padx** and **-pady** to the hull. In this case, both the resource name and the class name must be specified explicitly:

```
    snit::widget mywidget {
    option -background
    delegate option {-borderwidth borderWidth} to hull
    delegate option {-padx padX Pad} to hull
    delegate option {-pady padY Pad} to hull
    delegate option * to text
    # ...
    }
```

Querying the option database: If you set your widgetclass and option names as described above, Snit will query the option database when each instance is created, and will generally do the right thing when it comes to querying the option database. The remainder of this section goes into the gory details.

Initializing locally defined options: When an instance of a snit::widget is created, its locally defined options are initialized as follows: each option's resource and class names are used to query the Tk option database. If the result is non-empty, it is used as the option's default; otherwise, the default hardcoded in the type definition is used. In either case, the default can be overridden by the caller. For example,

```
    option add *Mywidget.texture pebbled

    snit::widget mywidget {
    option -texture smooth
    # ...
    }

    mywidget .mywidget -texture greasy
```

Here, **-texture** would normally default to "smooth", but because of the entry added to the option database it defaults to "pebbled". However, the caller has explicitly overridden the default, and so the new widget will be "greasy".

Initializing options delegated to the hull: A **snit::widget**'s hull is a widget, and given that its class has been set it is expected to query the option database for itself. The only exception concerns options that are delegated to it with a different name. Consider the following code:

```
    option add *Mywidget.borderWidth 5
    option add *Mywidget.relief sunken
    option add *Mywidget.hullbackground red
    option add *Mywidget.background green

    snit::widget mywidget {
    delegate option -borderwidth to hull
    delegate option -hullbackground to hull as -background
    delegate option * to hull
    # ...
    }
```

```
    mywidget .mywidget

    set A [.mywidget cget -relief]
    set B [.mywidget cget -hullbackground]
    set C [.mywidget cget -background]
    set D [.mywidget cget -borderwidth]
```

The question is, what are the values of variables A, B, C and D?

The value of A is "sunken". The hull is a Tk frame that has been given the widget class "Mywidget"; it will automatically query the option database and pick up this value. Since the **-relief** option is implicitly delegated to the hull, Snit takes no action.

The value of B is "red". The hull will automatically pick up the value "green" for its **-background** option, just as it picked up the **-relief** value. However, Snit knows that **-hullbackground** is mapped to the hull's **-background** option; hence, it queries the option database for **-hullbackground** and gets "red" and updates the hull accordingly.

The value of C is also "red", because **-background** is implicitly delegated to the hull; thus, retrieving it is the same as retrieving **-hullbackground**. Note that this case is unusual; in practice, **-background** would probably be explicitly delegated to some other component.

The value of D is "5", but not for the reason you think. Note that as it is defined above, the resource name for **-borderwidth** defaults to "borderwidth", whereas the option database entry is "borderWidth". As with **-relief**, the hull picks up its own **-borderwidth** option before Snit does anything. Because the option is delegated under its own name, Snit assumes that the correct thing has happened, and doesn't worry about it any further.

For **snit::widgetadaptor**s, the case is somewhat altered. Widget adaptors retain the widget class of their hull, and the hull is not created automatically by Snit. Instead, the **snit::widgetadaptor** must call **installhull** in its constructor. The normal way to do this is as follows:

```
    snit::widgetadaptor mywidget {
    # ...
    constructor {args} {
        # ...
        installhull using text -foreground white
        #
    }
    #...
    }
```

In this case, the **installhull** command will create the hull using a command like this:

```
    set hull [text $win -foreground white]
```

The hull is a **text** widget, so its widget class is "Text". Just as with **snit::widget** hulls, Snit assumes that it will pick up all of its normal option values automatically; options delegated from a different name are initialized from the option database in the same way.

Initializing options delegated to other components: Non-hull components are matched against the option database in two ways. First, a component widget remains a widget still, and

therefore is initialized from the option database in the usual way. Second, the option database is queried for all options delegated to the component, and the component is initialized accordingly--provided that the **install** command is used to create it.

Before option database support was added to Snit, the usual way to create a component was to simply create it in the constructor and assign its command name to the component variable:

```
    snit::widget mywidget {
    delegate option -background to myComp

    constructor {args} {
        set myComp [text $win.text -foreground black]
    }
    }
```

The drawback of this method is that Snit has no opportunity to initialize the component properly. Hence, the following approach is now used:

```
    snit::widget mywidget {
    delegate option -background to myComp

    constructor {args} {
        install myComp using text $win.text -foreground black
    }
    }
```

The **install** command does the following:

- Builds a list of the options explicitly included in the **install** command -- in this case, **-foreground**.
- Queries the option database for all options delegated explicitly to the named component.
- Creates the component using the specified command, after inserting into it a list of options and values read from the option database. Thus, the explicitly included options (**-foreground**) will override anything read from the option database.
- If the widget definition implicitly delegated options to the component using **delegate option** * , then Snit calls the newly created component's **configure** method to receive a list of all of the component's options. From this Snit builds a list of options implicitly delegated to the component that were not explicitly included in the **install** command. For all such options, Snit queries the option database and configures the component accordingly.

Non-widget components: The option database is never queried for **snit::type**s, since it can only be queried given a Tk widget name. However, **snit::widget**s can have non-widget components. And if options are delegated to those components, and if the **install** command is used to install those components, then they will be initialized from the option database just as widget components are.

Macros and meta-programming

The **snit::macro** command enables a certain amount of meta-programming with Snit classes. For example, suppose you like to define properties: instance variables that have set/get methods. Your code might look like this:

```
    snit::type dog {
        variable mood happy

        method getmood {} {
            return $mood
        }

        method setmood {newmood} {
            set mood $newmood
        }
    }
```

That's nine lines of text per property. Or, you could define the following **snit::macro**:

```
    snit::macro property {name initValue} {
        variable $name $initValue

        method get$name {} "return $name"

        method set$name {value} "set $name \$value"
    }
```

Note that a **snit::macro** is just a normal Tcl proc defined in the slave interpreter used to compile type and widget definitions; as a result, it has access to all the commands used to define types and widgets.

Given this new macro, you can define a property in one line of code:

```
    snit::type dog {
        property mood happy
    }
```

Within a macro, the commands **variable** and **proc** refer to the Snit type-definition commands, not the standard Tcl commands. To get the standard Tcl commands, use **_variable** and **_proc**.

Because a single slave interpreter is used for compiling all Snit types and widgets in the application, there's the possibility of macro name collisions. If you're writing a reuseable package using Snit, and you use some **snit::macro**s, define them in your package namespace:

```
    snit::macro mypkg::property {name initValue} { ... }

    snit::type dog {
        mypkg::property mood happy
    }
```

This leaves the global namespace open for application authors.

Validation types

A validation type is an object that can be used to validate Tcl values of a particular kind. For example, **snit::integer** is used to validate that a Tcl value is an integer.

Every validation type has a **validate** method which is used to do the validation. This method

must take a single argument, the value to be validated; further, it must do nothing if the value is valid, but throw an error if the value is invalid:

```
    snit::integer validate 5     ;# Does nothing
    snit::integer validate 5.0   ;# Throws an error (not an integer!)
```

The **validate** method will always return the validated value on success, and throw the **-errorcode** INVALID on error.

Snit defines a family of validation types, all of which are implemented as **snit::type**'s. They can be used as is; in addition, their instances serve as parameterized subtypes. For example, a probability is a number between 0.0 and 1.0 inclusive:

```
    snit::double probability -min 0.0 -max 1.0
```

The example above creates an instance of **snit::double**--a validation subtype--called **probability**, which can be used to validate probability values:

```
    probability validate 0.5   ;# Does nothing
    probability validate 7.9   ;# Throws an error
```

Validation subtypes can be defined explicitly, as in the above example; when a locally-defined option's **-type** is specified, they may also be created on the fly:

```
    snit::enum ::dog::breed -values {mutt retriever sheepdog}

    snit::type dog {
        # Define subtypes on the fly...
        option -breed -type {
            snit::enum -values {mutt retriever sheepdog}
        }

        # Or use predefined subtypes...
        option -breed -type ::dog::breed
    }
```

Any object that has a **validate** method with the semantics described above can be used as a validation type; see Defining validation types for information on how to define new ones.

Snit defines the following validation types:

snit::boolean validate ?*value*?
see below ...

snit::boolean *name*
Validates Tcl boolean values: 1, 0, **on**, **off**, **yes**, **no**, **true**, **false**. It's possible to define subtypes--that is, instances--of **snit::boolean**, but as it has no options there's no reason to do so.

snit::double validate ?*value*?
see below ...

snit::double *name* ?*option value*...?
Validates floating-point values. Subtypes may be created with the following options:
-min *min*

Specifies a floating-point minimum bound; a value is invalid if it is strictly less than *min*.

-max *max*
Specifies a floating-point maximum bound; a value is invalid if it is strictly greater than *max*.

snit::enum validate ?*value*?
see below ...

snit::enum *name* ?*option value*...?
Validates that a value comes from an enumerated list. The base type is of little use by itself, as only subtypes actually have an enumerated list to validate against. Subtypes may be created with the following options:

-values *list*
Specifies a list of valid values. A value is valid if and only if it's included in the list.

snit::fpixels validate ?*value*?
see below ...

snit::fpixels *name* ?*option value*...?
Tk programs only. Validates screen distances, in any of the forms accepted by **winfo fpixels**. Subtypes may be created with the following options:

-min *min*
Specifies a minimum bound; a value is invalid if it is strictly less than *min*. The bound may be expressed in any of the forms accepted by **winfo fpixels**.

-max *max*
Specifies a maximum bound; a value is invalid if it is strictly greater than *max*. The bound may be expressed in any of the forms accepted by **winfo fpixels**.

snit::integer validate ?*value*?
see below ...

snit::integer *name* ?*option value*...?
Validates integer values. Subtypes may be created with the following options:

-min *min*
Specifies an integer minimum bound; a value is invalid if it is strictly less than *min*.

-max *max*
Specifies an integer maximum bound; a value is invalid if it is strictly greater than *max*.

snit::listtype validate ?*value*?
see below ...

snit::listtype *name* ?*option value*...?
Validates Tcl lists. Subtypes may be created with the following options:

-minlen *min*
Specifies a minimum list length; the value is invalid if it has fewer than *min* elements. Defaults to 0.

-maxlen *max*
Specifies a maximum list length; the value is invalid if it more than *max* elements.

-type *type*
Specifies the type of the list elements; *type* must be the name of a validation type or

subtype. In the following example, the value of **-numbers** must be a list of integers.

```
option -numbers -type {snit::listtype -type snit::integer}
```

Note that this option doesn't support defining new validation subtypes on the fly; that is, the following code will not work (yet, anyway):

```
option -numbers -type {
    snit::listtype -type {snit::integer -min 5}
}
```

Instead, define the subtype explicitly:

```
snit::integer gt4 -min 5

snit::type mytype {
    option -numbers -type {snit::listtype -type gt4}
}
```

snit::pixels validate ?*value*?
see below ...

snit::pixels *name* ?*option value*...?
Tk programs only. Validates screen distances, in any of the forms accepted by **winfo pixels**. Subtypes may be created with the following options:

-min *min*
Specifies a minimum bound; a value is invalid if it is strictly less than *min*. The bound may be expressed in any of the forms accepted by **winfo pixels**.

-max *max*
Specifies a maximum bound; a value is invalid if it is strictly greater than *max*. The bound may be expressed in any of the forms accepted by **winfo pixels**.

snit::stringtype validate ?*value*?
see below ...

snit::stringtype *name* ?*option value*...?
Validates Tcl strings. The base type is of little use by itself, since very Tcl value is also a valid string. Subtypes may be created with the following options:

-minlen *min*
Specifies a minimum string length; the value is invalid if it has fewer than *min* characters. Defaults to 0.

-maxlen *max*
Specifies a maximum string length; the value is invalid if it has more than *max* characters.

-glob *pattern*
Specifies a **string match** pattern; the value is invalid if it doesn't match the pattern.

-regexp *regexp*
Specifies a regular expression; the value is invalid if it doesn't match the regular expression.

-nocase ***flag***
By default, both **-glob** and **-regexp** matches are case-sensitive. If **-nocase** is set to true, then both **-glob** and **-regexp** matches are case-insensitive.

snit::window validate ?*value*?
see below ...

snit::window ***name***
Tk programs only. Validates Tk window names. The value must cause **winfo exists** to return true; otherwise, the value is invalid. It's possible to define subtypes--that is, instances--of **snit::window**, but as it has no options at present there's no reason to do so.

Defining validation types

There are three ways to define a new validation type: as a subtype of one of Snit's validation types, as a validation type command, and as a full-fledged validation type similar to those provided by Snit. Defining subtypes of Snit's validation types is described above, under Validation types.

The next simplest way to create a new validation type is as a validation type command. A validation type is simply an object that has a **validate** method; the **validate** method must take one argument, a value, return the value if it is valid, and throw an error with **-errorcode** INVALID if the value is invalid. This can be done with a simple **proc**. For example, the **snit::boolean** validate type could have been implemented like this:

```
    proc ::snit::boolean {"validate" value} {
        if {![string is boolean -strict $value]} {
            return -code error -errorcode INVALID  "invalid boolean
    \"$value\", should be one of: 1, 0, ..."
        }

        return $value
    }
```

A validation type defined in this way cannot be subtyped, of course; but for many applications this will be sufficient.

Finally, one can define a full-fledged, subtype-able validation type as a **snit::type**. Here's a skeleton to get you started:

```
    snit::type myinteger {
        # First, define any options you'd like to use to define
        # subtypes.  Give them defaults such that they won't take
        # effect if they aren't used, and marked them "read-only".
        # After all, you shouldn't be changing their values after
        # a subtype is defined.
        #
        # For example:

        option -min -default "" -readonly 1
        option -max -default "" -readonly 1
```

```
    # Next, define a "validate" type method which should do the
    # validation in the basic case.  This will allow the
    # type command to be used as a validation type.

    typemethod validate {value} {
        if {![string is integer -strict $value]} {
            return -code error -errorcode INVALID  "invalid value
\"$value\", expected integer"
        }

        return $value
    }

    # Next, the constructor should validate the subtype options,
    # if any.  Since they are all readonly, we don't need to worry
    # about validating the options on change.

    constructor {args} {
        # FIRST, get the options
        $self configurelist $args

        # NEXT, validate them.

        # I'll leave this to your imagination.
    }

    # Next, define a "validate" instance method; its job is to
    # validate values for subtypes.

    method validate {value} {
        # First, call the type method to do the basic validation.
        $type validate $value

        # Now we know it's a valid integer.

        if {("" != $options(-min) && $value < $options(-min))  ||
            ("" != $options(-max) && $value > $options(-max))} {
            # It's out of range; format a detailed message about
            # the error, and throw it.

            set msg "...."

            return -code error -errorcode INVALID $msg
        }
```

```
            # Otherwise, if it's valid just return it.
            return $valid
        }
    }
```

And now you have a type that can be subtyped.

The file "validate.tcl" in the Snit distribution defines all of Snit's validation types; you can find the complete implementation for **snit::integer** and the other types there, to use as examples for your own types.

Caveats

If you have problems, find bugs, or new ideas you are hereby cordially invited to submit a report of your problem, bug, or idea as explained in the section Bugs, Ideas, Feedback below.

Additionally, you might wish to join the Snit mailing list; see http://www.wjduquette.com/snit for details.

One particular area to watch is using **snit::widgetadaptor** to adapt megawidgets created by other megawidget packages; correct widget destruction depends on the order of the bindings. The wisest course is simply not to do this.

Known bugs

- Error stack traces returned by Snit 1.x are extremely ugly and typically contain far too much information about Snit internals. The error messages are much improved in Snit 2.2.
- Also see the Project Trackers as explained in the section Bugs, Ideas, Feedback below.

History

During the course of developing Notebook (See http://www.wjduquette.com/notebook), my Tcl-based personal notebook application, I found I was writing it as a collection of objects. I wasn't using any particular object-oriented framework; I was just writing objects in pure Tcl following the guidelines in my Guide to Object Commands (see http://www.wjduquette.com/tcl/objects.html), along with a few other tricks I'd picked up since. And though it was working well, it quickly became tiresome because of the amount of boilerplate code associated with each new object type.

So that was one thing--tedium is a powerful motivator. But the other thing I noticed is that I wasn't using inheritance at all, and I wasn't missing it. Instead, I was using delegation: objects that created other objects and delegated methods to them.

And I said to myself, "This is getting tedious...there has got to be a better way." And one afternoon, on a whim, I started working on Snit, an object system that works the way Tcl works. Snit doesn't support inheritance, but it's great at delegation, and it makes creating megawidgets easy.

If you have any comments or suggestions (or bug reports!) don't hesitate to send me e-mail at *will@wjduquette.com*. In addition, there's a Snit mailing list; you can find out more about it at the Snit home page (see http://www.wjduquette.com/snit).

Credits

Snit has been designed and implemented from the very beginning by William H. Duquette.

However, much credit belongs to the following people for using Snit and providing me with valuable feedback: Rolf Ade, Colin McCormack, Jose Nazario, Jeff Godfrey, Maurice Diamanti, Egon Pasztor, David S. Cargo, Tom Krehbiel, Michael Cleverly, Andreas Kupries, Marty Backe, Andy Goth, Jeff Hobbs, Brian Griffin, Donal Fellows, Miguel Sofer, Kenneth Green, and Anton Kovalenko. If I've forgotten anyone, my apologies; let me know and I'll add your name to the list.

Bugs, Ideas, Feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *snit* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Programming tools

Copyright


```
Copyright (c) 2003-2009, by William H. Duquette
```


snitfaq

Keywords: snitfaq, BWidget, C++, Incr Tcl, adaptors, class, mega widget, object, object oriented, widget, widget adaptors

Name

snitfaq - Snit Frequently Asked Questions

Description

Overview

What is this document?

This is an atypical FAQ list, in that few of the questions are frequently asked. Rather, these are the questions I think a newcomer to Snit should be asking. This file is not a complete reference to Snit, however; that information is in the **snit** man page.

What is snit?

Snit is a framework for defining abstract data types and megawidgets in pure Tcl. The name "Snit" stands for "Snit's Not Incr Tcl", signifying that Snit takes a different approach to defining objects than does Incr Tcl, the best known object framework for Tcl. Had I realized that Snit would become at all popular, I'd probably have chosen something else.

The primary purpose of Snit is to be *object glue*--to help you compose diverse objects from diverse sources into types and megawidgets with clean, convenient interfaces so that you can more easily build your application.

Snit isn't about theoretical purity or minimalist design; it's about being able to do powerful

things easily and consistently without having to think about them--so that you can concentrate on building your application.

Snit isn't about implementing thousands of nearly identical carefully-specified lightweight thingamajigs--not as individual Snit objects. Traditional Tcl methods will be much faster, and not much more complicated. But Snit *is* about implementing a clean interface to manage a collection of thousands of nearly identical carefully-specified lightweight thingamajigs (e.g., think of the text widget and text tags, or the canvas widget and canvas objects). Snit lets you hide the details of just how those thingamajigs are stored--so that you can ignore it, and concentrate on building your application.

Snit isn't a way of life, a silver bullet, or the Fountain of Youth. It's just a way of managing complexity--and of managing some of the complexity of managing complexity--so that you can concentrate on building your application.

What version of Tcl does snit require?

Snit 1.3 requires Tcl 8.3 or later; Snit 2.2 requires Tcl 8.5 or later. See Snit versions for the differences between Snit 1.3 and Snit 2.2.

Where can I download snit?

Snit is part of Tcllib, the standard Tcl library, so you might already have it. It's also available at the Snit Home Page, http://www.wjduquette.com/snit.

What are snit's goals?

- A Snit object should be at least as efficient as a hand-coded Tcl object (see http://www.wjduquette.com/tcl/objects.html).
- The fact that Snit was used in an object's implementation should be transparent (and irrelevant) to clients of that object.
- Snit should be able to encapsulate objects from other sources, particularly Tk widgets.
- Snit megawidgets should be (to the extent possible) indistinguishable in interface from Tk widgets.
- Snit should be Tclish--that is, rather than trying to emulate C++, Smalltalk, or anything else, it should try to emulate Tcl itself.
- It should have a simple, easy-to-use, easy-to-remember syntax.

How is snit different from other oo frameworks?

Snit is unique among Tcl object systems in that it is based not on inheritance but on delegation. Object systems based on inheritance only allow you to inherit from classes defined using the same system, and that's a shame. In Tcl, an object is anything that acts like an object; it shouldn't matter how the object was implemented. I designed Snit to help me build applications out of the materials at hand; thus, Snit is designed to be able to incorporate and build on any object, whether it's a hand-coded object, a Tk widget, an Incr Tcl object, a BWidget or almost anything else.

Note that you can achieve the effect of inheritance using Components and Delegation--and you can inherit from anything that looks like a Tcl object.

What can I do with snit?

Using Snit, a programmer can:

- Create abstract data types and Tk megawidgets.
- Define instance variables, type variables, and Tk-style options.
- Define constructors, destructors, instance methods, type methods, procs.
- Assemble a type out of component types. Instance methods and options can be delegated to the component types automatically.

Snit versions

Which version of snit should I use?

The current Snit distribution includes two versions, Snit 1.3 and Snit 2.2. The reason that both are included is that Snit 2.2 takes advantage of a number of new features of Tcl 8.5 to improve run-time efficiency; as a side-effect, the ugliness of Snit's error messages and stack traces has been reduced considerably. The cost of using Snit 2.2, of course, is that you must target Tcl 8.5.

Snit 1.3, on the other hand, lacks Snit 2.2's optimizations, but requires only Tcl 8.3 and later.

In short, if you're targetting Tcl 8.3 or 8.4 you should use Snit 1.3. If you can afford to target Tcl 8.5, you should definitely use Snit 2.2. If you will be targetting both, you can use Snit 1.3 exclusively, or (if your code is unaffected by the minor incompatibilities between the two versions) you can use Snit 1.3 for Tcl 8.4 and Snit 2.2 for Tcl 8.5.

How do I select the version of snit I want to use?

To always use Snit 1.3 (or a later version of Snit 1.x), invoke Snit as follows:

```
package require snit 1.3
```

To always use Snit 2.2 (or a later version of Snit 2.x), say this instead:

```
package require snit 2.2
```

Note that if you request Snit 2.2 explicitly, your application will halt with Tcl 8.4, since Snit 2.2 is unavailable for Tcl 8.4.

If you wish your application to always use the latest available version of Snit, don't specify a version number:

```
package require snit
```

Tcl will find and load the latest version that's available relative to the version of Tcl being used. In this case, be careful to avoid using any incompatible features.

How are snit 1.3 and snit 2.2 incompatible?

To the extent possible, Snit 2.2 is intended to be a drop-in replacement for Snit 1.3. Unfortunately, some incompatibilities were inevitable because Snit 2.2 uses Tcl 8.5's new **namespace ensemble** mechanism to implement subcommand dispatch. This approach is much faster than the mechanism used in Snit 1.3, and also results in much better error messages; however, it also places new constraints on the implementation.

There are four specific incompatibilities between Snit 1.3 and Snit 2.2.

- Snit 1.3 supports implicit naming of objects. Suppose you define a new snit::type called **dog**. You can create instances of **dog** in three ways:

```
dog spot               ;# Explicit naming
set obj1 [dog %AUTO%]  ;# Automatic naming
set obj2 [dog]         ;# Implicit naming
```

- In Snit 2.2, type commands are defined using the **namespace ensemble** mechanism; and **namespace ensemble** doesn't allow an ensemble command to be called without a subcommand. In short, using **namespace ensemble** there's no way to support implicit naming.

  All is not lost, however. If the type has no type methods, then the type command is a simple command rather than an ensemble, and **namespace ensemble** is not used. In this case, implicit naming is still possible.

  In short, you can have implicit naming if you're willing to do without type methods (including the standard type methods, like **$type info**). To do so, use the **-hastypemethods** pragma:

```
pragma -hastypemethods 0
```

- Hierarchical methods and type methods are implemented differently in Snit 2.2.

  A hierarchical method is an instance method which has subcommands; these subcommands are themselves methods. The Tk text widget's **tag** command and its subcommands are examples of hierarchical methods. You can implement such subcommands in Snit simply by including multiple words in the method names:

```
method {tag configure} {tag args} { ... }

method {tag cget} {tag option} {...}
```

- Here we've implicitly defined a **tag** method which has two subcommands, **configure** and **cget** .

  In Snit 1.3, hierarchical methods could be called in two ways:

```
$obj tag cget -myoption      ;# The good way
$obj {tag cget} -myoption    ;# The weird way
```

- In the second call, we see that a hierarchical method or type method is simply one whose name contains multiple words.

  In Snit 2.2 this is no longer the case, and the "weird" way of calling hierarchical methods and type methods no longer works.
- The third incompatibility derives from the second. In Snit 1.3, hierarchical methods were also simply methods whose name contains multiple words. As a result, **$obj info methods** returned the full names of all hierarchical methods. In the example above, the list returned by **$obj info methods** would include **tag configure** and **tag cget** but not **tag**, since **tag** is defined only implicitly.

  In Snit 2.2, hierarchical methods and type methods are no longer simply ones whose name contains multiple words; in the above example, the list returned by **$obj info methods** would include **tag** but not **tag configure** or **tag cget**.
- The fourth incompatibility is due to a new feature. Snit 2.2 uses the new **namespace path** command so that a type's code can call any command defined in the type's parent namespace without qualification or importation. For example, suppose you have a package called

mypackage which defines a number of commands including a type, **::mypackage::mytype**. Thanks to **namespace path**, the type's code can call any of the other commands defined in **::mypackage::**.

This is extremely convenient. However, it also means that commands defined in the parent namespace, **::mypackage::** can block the type's access to identically named commands in the global namespace. This can lead to bugs. For example, Tcllib includes a type called **::tie::std::file**. This type's code calls the standard **file** command. When run with Snit 2.2, the code broke-- the type's command, **::tie::std::file**, is itself a command in the type's parent namespace, and so instead of calling the standard **file** command, the type found itself calling itself.

Are there other differences between snit 1.x and snit 2.2?

Yes.

- Method dispatch is considerably faster.
- Many error messages and stack traces are cleaner.
- The **-simpledispatch** pragma is obsolete, and ignored if present. In Snit 1.x, **-simpledispatch** substitutes a faster mechanism for method dispatch, at the cost of losing certain features. Snit 2.2 method dispatch is faster still in all cases, so **-simpledispatch** is no longer needed.
- In Snit 2.2, a type's code (methods, type methods, etc.) can call commands from the type's parent namespace without qualifying or importing them, i.e., type **::parentns::mytype**'s code can call **::parentns::someproc** as just **someproc**.

  This is extremely useful when a type is defined as part of a larger package, and shares a parent namespace with the rest of the package; it means that the type can call other commands defined by the package without any extra work.

  This feature depends on the new Tcl 8.5 **namespace path** command, which is why it hasn't been implemented for V1.x. V1.x code can achieve something similar by placing

  ```
  namespace import [namespace parent]::*
  ```

- in a type constructor. This is less useful, however, as it picks up only those commands which have already been exported by the parent namespace at the time the type is defined.

Objects

What is an object?

A full description of object-oriented programming is beyond the scope of this FAQ, obviously. In simple terms, an object is an instance of an abstract data type--a coherent bundle of code and data. There are many ways to represent objects in Tcl/Tk; the best known examples are the Tk widgets.

A Tk widget is an object; it is represented by a Tcl command. The object's methods are subcommands of the Tcl command. The object's properties are options accessed using the **configure** and **cget** methods. Snit uses the same conventions as Tk widgets do.

What is an abstract data type?

In computer science terms, an abstract data type is a complex data structure along with a set of operations--a stack, a queue, a binary tree, etc--that is to say, in modern terms, an object. In systems that include some form of inheritance the word *class* is usually used instead of *abstract*

data type, but as Snit doesn't implement inheritance as it's ordinarily understood the older term seems more appropriate. Sometimes this is called *object-based* programming as opposed to object-oriented programming. Note that you can easily create the effect of inheritance using Components and Delegation.

In Snit, as in Tk, a *type* is a command that creates instances -- objects -- which belong to the type. Most types define some number of *options* which can be set at creation time, and usually can be changed later.

Further, an *instance* is also a Tcl command--a command that gives access to the operations which are defined for that abstract data type. Conventionally, the operations are defined as subcommands of the instance command. For example, to insert text into a Tk text widget, you use the text widget's **insert** subcommand:

```
    # Create a text widget and insert some text in it.
    text .mytext -width 80 -height 24
    .mytext insert end "Howdy!"
```

In this example, **text** is the *type* command and **.mytext** is the *instance* command.

In Snit, object subcommands are generally called Instance methods.

What kinds of abstract data types does snit provide?

Snit allows you to define three kinds of abstract data type:

- snit::type
- snit::widget
- snit::widgetadaptor

What is a snit::type?

A snit::type is a non-GUI abstract data type, e.g., a stack or a queue. snit::types are defined using the snit::type command. For example, if you were designing a kennel management system for a dog breeder, you'd need a dog type.

```
% snit::type dog {
    # ...
}
::dog
%
```

This definition defines a new command (**::dog**, in this case) that can be used to define dog objects.

An instance of a snit::type can have Instance methods, Instance variables, Options, and Components. The type itself can have Type methods, Type variables, Type components, and Procs.

What is a snit::widget?, the short story

A snit::widget is a Tk megawidget built using Snit; it is very similar to a snit::type. See Widgets.

What is a snit::widgetadaptor?, the short story

A snit::widgetadaptor uses Snit to wrap an existing widget type (e.g., a Tk label), modifying its interface to a lesser or greater extent. It is very similar to a snit::widget. See Widget adaptors.

How do I create an instance of a snit::type?

You create an instance of a snit::type by passing the new instance's name to the type's create method. In the following example, we create a **dog** object called **spot**.

```
% snit::type dog {
    # ....
}
::dog
% dog create spot
::spot
%
```

In general, the **create** method name can be omitted so long as the instance name doesn't conflict with any defined Type methods. (See Type components for the special case in which this doesn't work.) So the following example is identical to the previous example:

```
% snit::type dog {
    # ....
}
::dog
% dog spot
::spot
%
```

This document generally uses the shorter form.

If the **dog** type defines Options, these can usually be given defaults at creation time:

```
% snit::type dog {
    option -breed mongrel
    option -color brown

    method bark {} { return "$self barks." }
}
::dog
% dog create spot -breed dalmation -color spotted
::spot
% spot cget -breed
dalmation
% spot cget -color
spotted
%
```

Once created, the instance name now names a new Tcl command that is used to manipulate the object. For example, the following code makes the dog bark:

```
% spot bark
::spot barks.
%
```

How do I refer to an object indirectly?

Some programmers prefer to save the object name in a variable, and reference it that way. For example,

```
% snit::type dog { ... }
::dog
% set d [dog spot -breed dalmation -color spotted]
::spot
% $d cget -breed
dalmation
% $d bark
::spot barks.
%
```

If you prefer this style, you might prefer to have Snit generate the instance's name automatically.

How can I generate the object name automatically?

If you'd like Snit to generate an object name for you, use the **%AUTO%** keyword as the requested name:

```
% snit::type dog { ... }
::dog
% set d [dog %AUTO%]
::dog2
% $d bark
::dog2 barks.
%
```

The **%AUTO%** keyword can be embedded in a longer string:

```
% set d [dog obj_%AUTO%]
::obj_dog4
% $d bark
::obj_dog4 barks.
%
```

Can types be renamed?

Tcl's **rename** command renames other commands. It's a common technique in Tcl to modify an existing command by renaming it and defining a new command with the original name; the new command usually calls the renamed command.

snit::type commands, however, should never be renamed; to do so breaks the connection between the type and its objects.

Can objects be renamed?

Tcl's **rename** command renames other commands. It's a common technique in Tcl to modify an existing command by renaming it and defining a new command with the original name; the new command usually calls the renamed command.

All Snit objects (including *widgets* and *widgetadaptors*) can be renamed, though this flexibility has some consequences:

- In an instance method, the implicit argument **self** will always contain the object's current name, so instance methods can always call other instance methods using **$self**.
- If the object is renamed, however, then **$self**'s value will change. Therefore, don't use **$self** for anything that will break if **$self** changes. For example, don't pass a callback command to another object like this:

```
    .btn configure -command [list $self ButtonPress]
```

- You'll get an error if **.btn** calls your command after your object is renamed.
- Instead, your object should define its callback command like this:

```
    .btn configure -command [mymethod ButtonPress]
```

- The mymethod command returns code that will call the desired method safely; the caller of the callback can add additional arguments to the end of the command as usual.
- Every object has a private namespace; the name of this namespace is available in method bodies, etc., as the value of the implicit argument **selfns**. This value is constant for the life of the object. Use **$selfns** instead of **$self** if you need a unique token to identify the object.
- When a snit::widget's instance command is renamed, its Tk window name remains the same -- and is still extremely important. Consequently, the Tk window name is available in method bodies as the value of the implicit argument **win**. This value is constant for the life of the object. When creating child windows, it's best to use **$win.child** rather than **$self.child** as the name of the child window.

How do I destroy a snit object?

Any Snit object of any type can be destroyed by renaming it to the empty string using the Tcl **rename** command.

Snit megawidgets (i.e., instances of snit::widget and snit::widgetadaptor) can be destroyed like any other widget: by using the Tk **destroy** command on the widget or on one of its ancestors in the window hierarchy.

Every instance of a snit::type has a **destroy** method:

```
% snit::type dog { ... }
::dog
% dog spot
::spot
% spot bark
::spot barks.
% spot destroy
% spot barks
invalid command name "spot"
%
```

Finally, every Snit type has a type method called **destroy**; calling it destroys the type and all of its instances:

```
% snit::type dog { ... }
::dog
% dog spot
::spot
% spot bark
::spot barks.
% dog destroy
% spot bark
invalid command name "spot"
% dog fido
invalid command name "dog"
%
```

Instance methods

What is an instance method?

An instance method is a procedure associated with a specific object and called as a subcommand of the object's command. It is given free access to all of the object's type variables, instance variables, and so forth.

How do I define an instance method?

Instance methods are defined in the type definition using the **method** statement. Consider the following code that might be used to add dogs to a computer simulation:

```
% snit::type dog {
    method bark {} {
        return "$self barks."
    }

    method chase {thing} {
        return "$self chases $thing."
    }
}
::dog
%
```

A dog can bark, and it can chase things.

The **method** statement looks just like a normal Tcl proc, except that it appears in a snit::type definition. Notice that every instance method gets an implicit argument called **self**; this argument contains the object's name. (There's more on implicit method arguments below.)

How does a client call an instance method?

The method name becomes a subcommand of the object. For example, let's put a simulated dog through its paces:

```
% dog spot
::spot
% spot bark
::spot barks.
% spot chase cat
::spot chases cat.
%
```

How does an instance method call another instance method?

If method A needs to call method B on the same object, it does so just as a client does: it calls method B as a subcommand of the object itself, using the object name stored in the implicit argument **self**.

Suppose, for example, that our dogs never chase anything without barking at them:

```
% snit::type dog {
    method bark {} {
        return "$self barks."
    }

    method chase {thing} {
        return "$self chases $thing.  [$self bark]"
    }
}
::dog
% dog spot
::spot
% spot bark
::spot barks.
% spot chase cat
::spot chases cat.  ::spot barks.
%
```

Are there any limitations on instance method names?

Not really, so long as you avoid the standard instance method names: **configure**, **configurelist** , **cget**, **destroy**, and **info**. Also, method names consisting of multiple words define hierarchical methods.

What is a hierarchical method?

An object's methods are subcommands of the object's instance command. Hierarchical methods allow an object's methods to have subcommands of their own; and these can in turn have subcommands, and so on. This allows the programmer to define a tree-shaped command structure, such as is used by many of the Tk widgets--the subcommands of the Tk **text** widget's **tag** method are hierarchical methods.

How do I define a hierarchical method?

Define methods whose names consist of multiple words. These words define the hierarchy implicitly. For example, the following code defines a **tag** method with subcommands **cget** and

configure:

```
snit::widget mytext {
    method {tag configure} {tag args} { ... }

    method {tag cget} {tag option} {...}
}
```

Note that there is no explicit definition for the **tag** method; it is implicit in the definition of **tag configure** and **tag cget**. If you tried to define **tag** explicitly in this example, you'd get an error.

How do I call hierarchical methods?

As subcommands of subcommands.

```
% mytext .text
.text
% .text tag configure redtext -foreground red -background black
% .text tag cget redtext -foreground
red
%
```

How do I make an instance method private?

It's often useful to define private methods, that is, instance methods intended to be called only by other methods of the same object.

Snit doesn't implement any access control on instance methods, so all methods are *de facto* public. Conventionally, though, the names of public methods begin with a lower-case letter, and the names of private methods begin with an upper-case letter.

For example, suppose our simulated dogs only bark in response to other stimuli; they never bark just for fun. So the **bark** method becomes **Bark** to indicate that it is private:

```
% snit::type dog {
    # Private by convention: begins with uppercase letter.
    method Bark {} {
        return "$self barks."
    }

    method chase {thing} {
        return "$self chases $thing. [$self Bark]"
    }
}
::dog
% dog fido
::fido
% fido chase cat
::fido chases cat. ::fido barks.
%
```

Are there any limitations on instance method arguments?

Method argument lists are defined just like normal Tcl proc argument lists; in particular, they can include arguments with default values and the **args** argument.

However, every method also has a number of implicit arguments provided by Snit in addition to those explicitly defined. The names of these implicit arguments may not used to name explicit arguments.

What implicit arguments are passed to each instance method?

The arguments implicitly passed to every method are **type**, **selfns**, **win**, and **self**.

What is $type?

The implicit argument **type** contains the fully qualified name of the object's type:

```
% snit::type thing {
    method mytype {} {
        return $type
    }
}
::thing
% thing something
::something
% something mytype
::thing
%
```

What is $self?

The implicit argument **self** contains the object's fully qualified name.

If the object's command is renamed, then **$self** will change to match in subsequent calls. Thus, your code should not assume that **$self** is constant unless you know for sure that the object will never be renamed.

```
% snit::type thing {
    method myself {} {
        return $self
    }
}
::thing
% thing mutt
::mutt
% mutt myself
::mutt
% rename mutt jeff
% jeff myself
::jeff
%
```

What is $selfns?

Each Snit object has a private namespace in which to store its Instance variables and Options.

The implicit argument **selfns** contains the name of this namespace; its value never changes, and is constant for the life of the object, even if the object's name changes:

```
% snit::type thing {
    method myNameSpace {} {
        return $selfns
    }
}
::thing
% thing jeff
::jeff
% jeff myNameSpace
::thing::Snit_inst3
% rename jeff mutt
% mutt myNameSpace
::thing::Snit_inst3
%
```

The above example reveals how Snit names an instance's private namespace; however, you should not write code that depends on the specific naming convention, as it might change in future releases.

What is $win?

The implicit argument **win** is defined for all Snit methods, though it really makes sense only for those of Widgets and Widget adaptors. **$win** is simply the original name of the object, whether it's been renamed or not. For widgets and widgetadaptors, it is also therefore the name of a Tk window.

When a snit::widgetadaptor is used to modify the interface of a widget or megawidget, it must rename the widget's original command and replace it with its own.

Thus, using **win** whenever the Tk window name is called for means that a snit::widget or snit::widgetadaptor can be adapted by a snit::widgetadaptor. See Widgets for more information.

How do I pass an instance method as a callback?

It depends on the context.

Suppose in my application I have a **dog** object named **fido**, and I want **fido** to bark when a Tk button called **.bark** is pressed. In this case, I create the callback command in the usual way, using **list**:

```
    button .bark -text "Bark!" -command [list fido bark]
```

In typical Tcl style, we use a callback to hook two independent components together. But suppose that the **dog** object has a graphical interface and owns the button itself? In this case, the **dog** must pass one of its own instance methods to the button it owns. The obvious thing to do is this:

```
% snit::widget dog {
    constructor {args} {
        #...
        button $win.barkbtn -text "Bark!" -command [list $self bark]
        #...
    }
}
::dog
%
```

(Note that in this example, our **dog** becomes a snit::widget, because it has GUI behavior. See Widgets for more.) Thus, if we create a **dog** called **.spot**, it will create a Tk button called **.spot.barkbtn**; when pressed, the button will call **$self bark**.

Now, this will work--provided that **.spot** is never renamed to something else. But surely renaming widgets is abnormal? And so it is--unless **.spot** is the hull component of a snit::widgetadaptor. If it is, then it will be renamed, and **.spot** will become the name of the snit::widgetadaptor object. When the button is pressed, the command **$self bark** will be handled by the snit::widgetadaptor, which might or might not do the right thing.

There's a safer way to do it, and it looks like this:

```
% snit::widget dog {
    constructor {args} {
        #...
        button $win.barkbtn -text "Bark!" -command [mymethod bark]
        #...
    }
}
::dog
%
```

The command mymethod takes any number of arguments, and can be used like **list** to build up a callback command; the only difference is that mymethod returns a form of the command that won't change even if the instance's name changes.

On the other hand, you might prefer to allow a widgetadaptor to override a method such that your renamed widget will call the widgetadaptor's method instead of its own. In this case, using **[list $self bark]** will do what you want...but this is a technique which should be used only in carefully controlled circumstances.

How do I delegate instance methods to a component?

See Delegation.

Instance variables

What is an instance variable?

An instance variable is a private variable associated with some particular Snit object. Instance variables can be scalars or arrays.

How is a scalar instance variable defined?

Scalar instance variables are defined in the type definition using the variable statement. You can simply name it, or you can initialize it with a value:

```
snit::type mytype {
    # Define variable "greeting" and initialize it with "Howdy!"
    variable greeting "Howdy!"
}
```

How is an array instance variable defined?

Array instance variables are also defined in the type definition using the variable command. You can initialize them at the same time by specifying the **-array** option:

```
snit::type mytype {
    # Define array variable "greetings"
    variable greetings -array {
        formal "Good Evening"
        casual "Howdy!"
    }
}
```

What happens if I don't initialize an instance variable?

Variables do not really exist until they are given values. If you do not initialize a variable when you define it, then you must be sure to assign a value to it (in the constructor, say, or in some method) before you reference it.

Are there any limitations on instance variable names?

Just a few.

First, every Snit object has a built-in instance variable called **options**, which should never be redefined.

Second, all names beginning with "Snit_" are reserved for use by Snit internal code.

Third, instance variable names containing the namespace delimiter (**::**) are likely to cause great confusion.

Do I need to declare my instance variables in my methods?

No. Once you've defined an instance variable in the type definition, it can be used in any instance code (instance methods, the constructor, and the destructor) without declaration. This differs from normal Tcl practice, in which all non-local variables in a proc need to be declared.

There is a speed penalty to having all instance variables implicitly available in all instance code. Even though your code need not declare the variables explicitly, Snit must still declare them, and that takes time. If you have ten instance variables, a method that uses none of them must still pay the declaration penalty for all ten. In most cases, the additional runtime cost is negligible. If extreme cases, you might wish to avoid it; there are two methods for doing so.

The first is to define a single instance variable, an array, and store all of your instance data in the array. This way, you're only paying the declaration penalty for one variable--and you probably need the variable most of the time anyway. This method breaks down if your instance

variables include multiple arrays; in Tcl 8.5, however, the **dict** command might come to your rescue.

The second method is to declare your instance variables explicitly in your instance code, while *not* including them in the type definition:

```
snit::type dog {
    constructor {} {
        variable mood

        set mood happy
    }

    method setmood {newMood} {
        variable mood

        set mood $newMood
    }

    method getmood {} {
        variable mood

        return $mood
    }
}
```

This allows you to ensure that only the required variables are included in each method, at the cost of longer code and run-time errors when you forget to declare a variable you need.

How do I pass an instance variable's name to another object?

In Tk, it's common to pass a widget a variable name; for example, Tk label widgets have a **-textvariable** option which names the variable which will contain the widget's text. This allows the program to update the label's value just by assigning a new value to the variable.

If you naively pass the instance variable name to the label widget, you'll be confused by the result; Tk will assume that the name names a global variable. Instead, you need to provide a fully-qualified variable name. From within an instance method or a constructor, you can fully qualify the variable's name using the myvar command:

```
snit::widget mywidget {
    variable labeltext ""

    constructor {args} {
        # ...

        label $win.label -textvariable [myvar labeltext]

        # ...
    }
}
```

How do I make an instance variable public?

Practically speaking, you don't. Instead, you'll implement public variables as Options. Alternatively, you can write Instance methods to set and get the variable's value.

Options

What is an option?

A type's options are the equivalent of what other object-oriented languages would call public member variables or properties: they are data values which can be retrieved and (usually) set by the clients of an object.

Snit's implementation of options follows the Tk model fairly exactly, except that snit::type objects usually don't interact with The Tk option database; snit::widget and snit::widgetadaptor objects, on the other hand, always do.

How do I define an option?

Options are defined in the type definition using the **option** statement. Consider the following type, to be used in an application that manages a list of dogs for a pet store:

```
snit::type dog {
    option -breed -default mongrel
    option -color -default brown
    option -akc   -default 0
    option -shots -default 0
}
```

According to this, a dog has four notable properties: a breed, a color, a flag that says whether it's pedigreed with the American Kennel Club, and another flag that says whether it has had its shots. The default dog, evidently, is a brown mutt.

There are a number of options you can specify when defining an option; if **-default** is the only one, you can omit the word **-default** as follows:

```
snit::type dog {
    option -breed mongrel
    option -color brown
    option -akc   0
    option -shots 0
}
```

If no **-default** value is specified, the option's default value will be the empty string (but see The Tk option database).

The Snit man page refers to options like these as "locally defined" options.

How can a client set options at object creation?

The normal convention is that the client may pass any number of options and their values after the object's name at object creation. For example, the **::dog** command defined in the previous answer can now be used to create individual dogs. Any or all of the options may be set at creation time.

```
% dog spot -breed beagle -color "mottled" -akc 1 -shots 1
::spot
% dog fido -shots 1
::fido
%
```

So **::spot** is a pedigreed beagle; **::fido** is a typical mutt, but his owners evidently take care of him, because he's had his shots.

Note: If the type defines a constructor, it can specify a different object-creation syntax. See Constructors for more information.

How can a client retrieve an option's value?

Retrieve option values using the **cget** method:

```
% spot cget -color
mottled
% fido cget -breed
mongrel
%
```

How can a client set options after object creation?

Any number of options may be set at one time using the **configure** instance method. Suppose that closer inspection shows that ::fido is not a brown mongrel, but rather a rare Arctic Boar Hound of a lovely dun color:

```
% fido configure -color dun -breed "Arctic Boar Hound"
% fido cget -color
dun
% fido cget -breed
Arctic Boar Hound
```

Alternatively, the **configurelist** method takes a list of options and values; occasionally this is

more convenient:

```
% set features [list -color dun -breed "Arctic Boar Hound"]
-color dun -breed {Arctic Boar Hound}
% fido configurelist $features
% fido cget -color
dun
% fido cget -breed
Arctic Boar Hound
%
```

In Tcl 8.5, the * keyword can be used with **configure** in this case:

```
% set features [list -color dun -breed "Arctic Boar Hound"]
-color dun -breed {Arctic Boar Hound}
% fido configure {*}$features
% fido cget -color
dun
% fido cget -breed
Arctic Boar Hound
%
```

The results are the same.

How should an instance method access an option value?

There are two ways an instance method can set and retrieve an option's value. One is to use the **configure** and **cget** methods, as shown below.

```
% snit::type dog {
    option -weight 10

    method gainWeight {} {
        set wt [$self cget -weight]
        incr wt
        $self configure -weight $wt
    }
}
::dog
% dog fido
::fido
% fido cget -weight
10
% fido gainWeight
% fido cget -weight
11
%
```

Alternatively, Snit provides a built-in array instance variable called **options**. The indices are the option names; the values are the option values. The method **gainWeight** can thus be rewritten

as follows:

```
    method gainWeight {} {
        incr options(-weight)
    }
```

As you can see, using the **options** variable involves considerably less typing and is the usual way to do it. But if you use **-configuremethod** or **-cgetmethod** (described in the following answers), you might wish to use the **configure** and **cget** methods anyway, just so that any special processing you've implemented is sure to get done. Also, if the option is delegated to a component then **configure** and **cget** are the only way to access it without accessing the component directly. See Delegation for more information.

How can I make an option read-only?

Define the option with **-readonly yes**.

Suppose you've got an option that determines how instances of your type are constructed; it must be set at creation time, after which it's constant. For example, a dog never changes its breed; it might or might not have had its shots, and if not can have them at a later time. **-breed** should be read-only, but **-shots** should not be.

```
% snit::type dog {
    option -breed -default mongrel -readonly yes
    option -shots -default no
}
::dog
% dog fido -breed retriever
::fido
% fido configure -shots yes
% fido configure -breed terrier
option -breed can only be set at instance creation
%
```

How can I catch accesses to an option's value?

Define a **-cgetmethod** for the option.

What is a -cgetmethod?

A **-cgetmethod** is a method that's called whenever the related option's value is queried via the **cget** instance method. The handler can compute the option's value, retrieve it from a database, or do anything else you'd like it to do.

Here's what the default behavior would look like if written using a **-cgetmethod**:

```
snit::type dog {
    option -color -default brown -cgetmethod GetOption

    method GetOption {option} {
        return $options($option)
    }
}
```

Any instance method can be used, provided that it takes one argument, the name of the option whose value is to be retrieved.

How can I catch changes to an option's value?

Define a **-configuremethod** for the option.

What is a -configuremethod?

A **-configuremethod** is a method that's called whenever the related option is given a new value via the **configure** or **configurelist** instance methods. The method can pass the value on to some other object, store it in a database, or do anything else you'd like it to do.

Here's what the default configuration behavior would look like if written using a **-configuremethod**:

```
snit::type dog {
    option -color -default brown -configuremethod SetOption

    method SetOption {option value} {
        set options($option) $value
    }
}
```

Any instance method can be used, provided that it takes two arguments, the name of the option and the new value.

Note that if your method doesn't store the value in the **options** array, the **options** array won't get updated.

How can I validate an option's value?

Define a **-validatemethod**.

What is a -validatemethod?

A **-validatemethod** is a method that's called whenever the related option is given a new value via the **configure** or **configurelist** instance methods. It's the method's responsibility to determine whether the new value is valid, and throw an error if it isn't. The **-validatemethod**, if any, is called before the value is stored in the **options** array; in particular, it's called before the **-configuremethod**, if any.

For example, suppose an option always takes a Boolean value. You can ensure that the value is in fact a valid Boolean like this:

```
% snit::type dog {
    option -shots -default no -validatemethod BooleanOption

    method BooleanOption {option value} {
        if {![string is boolean -strict $value]} {
            error "expected a boolean value, got \"$value\""
        }
    }
}
::dog
```

```
% dog fido
% fido configure -shots yes
% fido configure -shots NotABooleanValue
expected a boolean value, got "NotABooleanValue"
%
```

Note that the same **-validatemethod** can be used to validate any number of boolean options.

Any method can be a **-validatemethod** provided that it takes two arguments, the option name and the new option value.

Type variables

What is a type variable?

A type variable is a private variable associated with a Snit type rather than with a particular instance of the type. In C++ and Java, the term *static member variable* is used for the same notion. Type variables can be scalars or arrays.

How is a scalar type variable defined?

Scalar type variables are defined in the type definition using the typevariable statement. You can simply name it, or you can initialize it with a value:

```
snit::type mytype {
    # Define variable "greeting" and initialize it with "Howdy!"
    typevariable greeting "Howdy!"
}
```

Every object of type **mytype** now has access to a single variable called **greeting**.

How is an array-valued type variable defined?

Array-valued type variables are also defined using the typevariable command; to initialize them, include the **-array** option:

```
snit::type mytype {
    # Define typearray variable "greetings"
    typevariable greetings -array {
        formal "Good Evening"
        casual "Howdy!"
    }
}
```

What happens if I don't initialize a type variable?

Variables do not really exist until they are given values. If you do not initialize a variable when you define it, then you must be sure to assign a value to it (in the type constructor, say) before you reference it.

Are there any limitations on type variable names?

Type variable names have the same restrictions as the names of Instance variables do.

Do I need to declare my type variables in my methods?

No. Once you've defined a type variable in the type definition, it can be used in Instance methods or Type methods without declaration. This differs from normal Tcl practice, in which all non-local variables in a proc need to be declared.

Type variables are subject to the same speed/readability tradeoffs as instance variables; see Do I need to declare my instance variables in my methods?

How do I pass a type variable's name to another object?

In Tk, it's common to pass a widget a variable name; for example, Tk label widgets have a **-textvariable** option which names the variable which will contain the widget's text. This allows the program to update the label's value just by assigning a new value to the variable.

If you naively pass a type variable name to the label widget, you'll be confused by the result; Tk will assume that the name names a global variable. Instead, you need to provide a fully-qualified variable name. From within an instance method or a constructor, you can fully qualify the type variable's name using the mytypevar command:

```
snit::widget mywidget {
    typevariable labeltext ""

    constructor {args} {
        # ...

        label $win.label -textvariable [mytypevar labeltext]

        # ...
    }
}
```

How do I make a type variable public?

There are two ways to do this. The preferred way is to write a pair of Type methods to set and query the type variable's value.

Type variables are stored in the type's namespace, which has the same name as the type itself. Thus, you can also publicize the type variable's name in your documentation so that clients can access it directly. For example,

```
snit::type mytype {
    typevariable myvariable
}

set ::mytype::myvariable "New Value"
```

Type methods

What is a type method?

A type method is a procedure associated with the type itself rather than with any specific instance of the type, and called as a subcommand of the type command.

How do I define a type method?

Type methods are defined in the type definition using the **typemethod** statement:

```
snit::type dog {
    # List of pedigreed dogs
    typevariable pedigreed

    typemethod pedigreedDogs {} {
        return $pedigreed
    }
}
```

Suppose the **dog** type maintains a list of the names of the dogs that have pedigrees. The **pedigreedDogs** type method returns this list.

The **typemethod** statement looks just like a normal Tcl proc, except that it appears in a snit::type definition. Notice that every type method gets an implicit argument called **type**, which contains the fully-qualified type name.

How does a client call a type method?

The type method name becomes a subcommand of the type's command. For example, assuming that the constructor adds each pedigreed dog to the list of **pedigreedDogs**,

```
snit::type dog {
    option -pedigreed 0

    # List of pedigreed dogs
    typevariable pedigreed

    typemethod pedigreedDogs {} {
        return $pedigreed
    }

    # ...
}

dog spot -pedigreed 1
dog fido

foreach dog [dog pedigreedDogs] { ... }
```

Are there any limitations on type method names?

Not really, so long as you avoid the standard type method names: **create**, **destroy**, and **info**.

How do I make a type method private?

It's sometimes useful to define private type methods, that is, type methods intended to be called only by other type or instance methods of the same object.

Snit doesn't implement any access control on type methods; by convention, the names of public methods begin with a lower-case letter, and the names of private methods begin with an upper-case letter.

Alternatively, a Snit proc can be used as a private type method; see Procs.

Are there any limitations on type method arguments?

Method argument lists are defined just like normal Tcl proc argument lists; in particular, they can include arguments with default values and the **args** argument.

However, every type method is called with an implicit argument called **type** that contains the name of the type command. In addition, type methods should by convention avoid using the names of the arguments implicitly defined for Instance methods.

How does an instance or type method call a type method?

If an instance or type method needs to call a type method, it should use **$type** to do so:

```
snit::type dog {

    typemethod pedigreedDogs {} { ... }

    typemethod printPedigrees {} {
        foreach obj [$type pedigreedDogs] { ... }
    }
}
```

How do I pass a type method as a callback?

It's common in Tcl to pass a snippet of code to another object, for it to call later. Because types cannot be renamed, you can just use the type name, or, if the callback is registered from within a type method, **type**. For example, suppose we want to print a list of pedigreed dogs when a Tk button is pushed:

```
button .btn -text "Pedigrees" -command [list dog printPedigrees]
pack .btn
```

Alternatively, from a method or type method you can use the mytypemethod command, just as you would use mymethod to define a callback command for Instance methods.

Can type methods be hierarchical?

Yes, you can define hierarchical type methods in just the same way as you can define hierarchical instance methods. See Instance methods for more.

Procs

What is a proc?

A Snit proc is really just a Tcl proc defined within the type's namespace. You can use procs for private code that isn't related to any particular instance.

How do I define a proc?

Procs are defined by including a proc statement in the type definition:

```
snit::type mytype {
    # Pops and returns the first item from the list stored in the
    # listvar, updating the listvar
   proc pop {listvar} { ... }

   # ...
}
```

Are there any limitations on proc names?

Any name can be used, so long as it does not begin with **Snit_**; names beginning with **Snit_** are reserved for Snit's own use. However, the wise programmer will avoid proc names (**set**, **list**, **if**, etc.) that would shadow standard Tcl command names.

proc names, being private, should begin with a capital letter according to convention; however, as there are typically no public procs in the type's namespace it doesn't matter much either way.

How does a method call a proc?

Just like it calls any Tcl command. For example,

```
snit::type mytype {
    # Pops and returns the first item from the list stored in the
    # listvar, updating the listvar
    proc pop {listvar} { ... }

    variable requestQueue {}

    # Get one request from the queue and process it.
    method processRequest {} {
        set req [pop requestQueue]
    }
}
```

How can I pass a proc to another object as a callback?

The myproc command returns a callback command for the proc, just as mymethod does for a method.

Type constructors

What is a type constructor?

A type constructor is a body of code that initializes the type as a whole, rather like a C++ static initializer. The body of a type constructor is executed once when the type is defined, and never again.

A type can have at most one type constructor.

How do I define a type constructor?

A type constructor is defined by using the typeconstructor statement in the type definition. For example, suppose the type uses an array-valued type variable as a look-up table, and the values in the array have to be computed at start-up.

```
% snit::type mytype {
    typevariable lookupTable

    typeconstructor {
        array set lookupTable {key value...}
    }
}
```

Constructors

What is a constructor?

In object-oriented programming, an object's constructor is responsible for initializing the object completely at creation time. The constructor receives the list of options passed to the snit::type command's **create** method and can then do whatever it likes. That might include computing instance variable values, reading data from files, creating other objects, updating type and instance variables, and so forth.

The constructor's return value is ignored (unless it's an error, of course).

How do I define a constructor?

A constructor is defined by using the constructor statement in the type definition. Suppose that it's desired to keep a list of all pedigreed dogs. The list can be maintained in a type variable and retrieved by a type method. Whenever a dog is created, it can add itself to the list--provided that it's registered with the American Kennel Club.

```
% snit::type dog {
    option -akc 0

    typevariable akcList {}

    constructor {args} {
        $self configurelist $args

        if {$options(-akc)} {
            lappend akcList $self
        }
    }

    typemethod akclist {} {
        return $akcList
    }
}
::dog
% dog spot -akc 1
::spot
% dog fido
::fido
% dog akclist
::spot
%
```

What does the default constructor do?

If you don't provide a constructor explicitly, you get the default constructor, which is identical to the explicitly-defined constructor shown here:

```
snit::type dog {
    constructor {args} {
        $self configurelist $args
    }
}
```

When the constructor is called, **args** will be set to the list of arguments that follow the object's name. The constructor is allowed to interpret this list any way it chooses; the normal convention is to assume that it's a list of option names and values, as shown in the example above. If you simply want to save the option values, you should use the **configurelist** method, as shown.

Can I choose a different set of arguments for the constructor?

Yes, you can. For example, suppose we wanted to be sure that the breed was explicitly stated for every dog at creation time, and couldn't be changed thereafter. One way to do that is as follows:

```
% snit::type dog {
    variable breed

    option -color brown
    option -akc 0

    constructor {theBreed args} {
        set breed $theBreed
        $self configurelist $args
    }

    method breed {} { return $breed }
}
::dog
% dog spot dalmatian -color spotted -akc 1
::spot
% spot breed
dalmatian
```

The drawback is that this syntax is non-standard, and may limit the compatibility of your new type with other people's code. For example, Snit assumes that it can create Components using the standard creation syntax.

Are there any limitations on constructor arguments?

Constructor argument lists are subject to the same limitations as those on instance method argument lists. It has the same implicit arguments, and can contain default values and the **args** argument.

Is there anything special about writing the constructor?

Yes. Writing the constructor can be tricky if you're delegating options to components, and there are specific issues relating to snit::widgets and snit::widgetadaptors. See Delegation, Widgets, Widget adaptors, and The Tk option database.

Destructors

What is a destructor?

A destructor is a special kind of method that's called when an object is destroyed. It's responsible for doing any necessary clean-up when the object goes away: destroying Components, closing files, and so forth.

How do I define a destructor?

Destructors are defined by using the destructor statement in the type definition.

Suppose we're maintaining a list of pedigreed dogs; then we'll want to remove dogs from it when they are destroyed.

```
snit::type dog {
    option -akc 0

    typevariable akcList {}

    constructor {args} {
        $self configurelist $args

        if {$options(-akc)} {
            lappend akcList $self
        }
    }

    destructor {
        set ndx [lsearch $akcList $self]

        if {$ndx != -1} {
            set akcList [lreplace $akcList $ndx $ndx]
        }
    }

    typemethod akclist {} {
        return $akcList
    }
}
```

Are there any limitations on destructor arguments?

Yes; a destructor has no explicit arguments.

What implicit arguments are passed to the destructor?

The destructor gets the same implicit arguments that are passed to Instance methods: **type**, **selfns**, **win**, and **self**.

Must components be destroyed explicitly?

Yes and no.

Any Tk widgets created by a snit::widget or snit::widgetadaptor will be destroyed automatically by Tk when the megawidget is destroyed, in keeping with normal Tk behavior (destroying a parent widget destroys the whole tree).

Components of normal **snit::types**, on the other hand, are never destroyed automatically, nor are non-widget components of Snit megawidgets. If your object creates them in its constructor, then it should generally destroy them in its destructor.

Is there any special about writing a destructor?

Yes. If an object's constructor throws an error, the object's destructor will be called to clean up; this means that the object might not be completely constructed when the destructor is called. This can cause the destructor to throw its own error; the result is usually misleading, confusing,

and unhelpful. Consequently, it's important to write your destructor so that it's fail-safe.

For example, a **dog** might create a **tail** component; the component will need to be destroyed. But suppose there's an error while processing the creation options--the destructor will be called, and there will be no **tail** to destroy. The simplest solution is generally to catch and ignore any errors while destroying components.

```
snit::type dog {
    component tail

    constructor {args} {
        $self configurelist $args

        set tail [tail %AUTO%]
    }

    destructor {
        catch {$tail destroy}
    }
}
```

Components

What is a component?

Often an object will create and manage a number of other objects. A Snit megawidget, for example, will often create a number of Tk widgets. These objects are part of the main object; it is composed of them, so they are called components of the object.

But Snit also has a more precise meaning for **COMPONENT**. The components of a Snit object are those objects to which methods or options can be delegated. (See Delegation for more information about delegation.)

How do I declare a component?

First, you must decide what role a component plays within your object, and give the role a name. Then, you declare the component using its role name and the component statement. The component statement declares an *instance variable* which is used to store the component's command name when the component is created.

For example, suppose your **dog** object creates a **tail** object (the better to wag with, no doubt):

```
snit::type dog {
    component mytail

    constructor {args} {
        # Create and save the component's command
        set mytail [tail %AUTO% -partof $self]
        $self configurelist $args
    }

    method wag {} {
```

```
        $mytail wag
    }
}
```

As shown here, it doesn't matter what the **tail** object's real name is; the **dog** object refers to it by its component name.

The above example shows one way to delegate the **wag** method to the **mytail** component; see Delegation for an easier way.

How is a component named?

A component has two names. The first name is that of the component variable; this represents the role the component object plays within the Snit object. This is the component name proper, and is the name used to refer to the component within Snit code. The second name is the name of the actual component object created by the Snit object's constructor. This second name is always a Tcl command name, and is referred to as the component's object name.

In the example in the previous question, the component name is **mytail**; the **mytail** component's object name is chosen automatically by Snit since **%AUTO%** was used when the component object was created.

Are there any limitations on component names?

Yes. snit::widget and snit::widgetadaptor objects have a special component called the **hull** component; thus, the name **hull** should be used for no other purpose.

Otherwise, since component names are in fact instance variable names they must follow the rules for Instance variables.

What is an owned component?

An *owned* component is a component whose object command's lifetime is controlled by the snit::type or snit::widget.

As stated above, a component is an object to which our object can delegate methods or options. Under this definition, our object will usually create its component objects, but not necessarily. Consider the following: a dog object has a tail component; but tail knows that it's part of the dog:

```
snit::type dog {
    component mytail

    constructor {args} {
        set mytail [tail %AUTO% -partof $self]
        $self configurelist $args
    }

    destructor {
        catch {$mytail destroy}
    }

    delegate method wagtail to mytail as wag
```

```
    method bark {} {
        return "$self barked."
    }
}

 snit::type tail {
     component mydog
     option -partof -readonly yes

     constructor {args} {
         $self configurelist $args
         set mydog $options(-partof)
     }

     method wag {} {
         return "Wag, wag."
     }

     method pull {} {
         $mydog bark
     }
 }
```

Thus, if you ask a dog to wag its tail, it tells its tail to wag; and if you pull the dog's tail, the tail tells the dog to bark. In this scenario, the tail is a component of the dog, and the dog is a component of the tail, but the dog owns the tail and not the other way around.

What does the install command do?

The install command creates an owned component using a specified command, and assigns the result to the component's instance variable. For example:

```
snit::type dog {
    component mytail

    constructor {args} {
        # set mytail [tail %AUTO% -partof $self]
        install mytail using tail %AUTO% -partof $self
        $self configurelist $args
    }
}
```

In a snit::type's code, the install command shown above is equivalent to the **set mytail** command that's commented out. In a snit::widget's or snit::widgetadaptor's, code, however, the install command also queries The Tk option database and initializes the new component's options accordingly. For consistency, it's a good idea to get in the habit of using install for all owned components.

Must owned components be created in the constructor?

No, not necessarily. In fact, there's no reason why an object can't destroy and recreate a component multiple times over its own lifetime.

Are there any limitations on component object names?

Yes.

Component objects which are Tk widgets or megawidgets must have valid Tk window names.

Component objects which are not widgets or megawidgets must have fully-qualified command names, i.e., names which include the full namespace of the command. Note that Snit always creates objects with fully qualified names.

Next, the object names of components and owned by your object must be unique. This is no problem for widget components, since widget names are always unique; but consider the following code:

```
snit::type tail { ... }

snit::type dog {
    delegate method wag to mytail

    constructor {} {
        install mytail using tail mytail
    }
}
```

This code uses the component name, **mytail**, as the component object name. This is not good, and here's why: Snit instance code executes in the Snit type's namespace. In this case, the **mytail** component is created in the **::dog::** namespace, and will thus have the name **::dog::mytail**.

Now, suppose you create two dogs. Both dogs will attempt to create a tail called **::dog::mytail**. The first will succeed, and the second will fail, since Snit won't let you create an object if its name is already a command. Here are two ways to avoid this situation:

First, if the component type is a snit::type you can specify **%AUTO%** as its name, and be guaranteed to get a unique name. This is the safest thing to do:

```
    install mytail using tail %AUTO%
```

If the component type isn't a snit::type you can create the component in the object's instance namespace:

```
    install mytail using tail ${selfns}::mytail
```

Make sure you pick a unique name within the instance namespace.

Must I destroy the components I own?

That depends. When a parent widget is destroyed, all child widgets are destroyed automatically. Thus, if your object is a snit::widget or snit::widgetadaptor you don't need to destroy any components that are widgets, because they will generally be children or descendants of your megawidget.

If your object is an instance of snit::type, though, none of its owned components will be destroyed automatically, nor will be non-widget components of a snit::widget be destroyed automatically. All such owned components must be destroyed explicitly, or they won't be destroyed at all.

Can I expose a component's object command as part of my interface?

Yes, and there are two ways to do it. The most appropriate way is usually to use Delegation. Delegation allows you to pass the options and methods you specify along to particular components. This effectively hides the components from the users of your type, and ensures good encapsulation.

However, there are times when it's appropriate, not to mention simpler, just to make the entire component part of your type's public interface.

How do I expose a component's object command?

When you declare the component, specify the component statement's **-public** option. The value of this option is the name of a method which will be delegated to your component's object command.

For example, supposed you've written a combobox megawidget which owns a listbox widget, and you want to make the listbox's entire interface public. You can do it like this:

```
snit::widget combobox {
     component listbox -public listbox

     constructor {args} {
         install listbox using listbox $win.listbox ....
     }
}

combobox .mycombo
.mycombo listbox configure -width 30
```

Your comobox widget, **.mycombo**, now has a **listbox** method which has all of the same subcommands as the listbox widget itself. Thus, the above code sets the listbox component's width to 30.

Usually you'll let the method name be the same as the component name; however, you can name it anything you like.

Type components

What is a type component?

A type component is a component that belongs to the type itself instead of to a particular instance of the type. The relationship between components and type components is the same as the relationship between Instance variables and Type variables. Both Instance methods and Type methods can be delegated to type components.

Once you understand Components and Delegation, type components are just more of the same.

How do I declare a type component?

Declare a type component using the typecomponent statement. It takes the same options (

-inherit and **-public**) as the component statement does, and defines a type variable to hold the type component's object command.

Suppose in your model you've got many dogs, but only one veterinarian. You might make the veterinarian a type component.

```
snit::type veterinarian { ... }

snit::type dog {
    typecomponent vet

    # ...
}
```

How do I install a type component?

Just use the **set** command to assign the component's object command to the type component. Because types (even snit::widget types) are not widgets, and do not have options anyway, the extra features of the install command are not needed.

You'll usually install type components in the type constructor, as shown here:

```
snit::type veterinarian { ... }

snit::type dog {
    typecomponent vet

    typeconstructor {
        set vet [veterinarian %AUTO%]
    }
}
```

Are there any limitations on type component names?

Yes, the same as on Instance variables, Type variables, and normal Components.

Delegation

What is delegation?

Delegation, simply put, is when you pass a task you've been given to one of your assistants. (You do have assistants, don't you?) Snit objects can do the same thing. The following example shows one way in which the **dog** object can delegate its **wag** method and its **-taillength** option to its **tail** component.

```
snit::type dog {
    variable mytail

    option -taillength -configuremethod SetTailOption -cgetmethod
    GetTailOption

    method SetTailOption {option value} {
```

```
         $mytail configure $option $value
    }

    method GetTailOption {option} {
         $mytail cget $option
    }

    method wag {} {
        $mytail wag
    }

    constructor {args} {
        install mytail using tail %AUTO% -partof $self
        $self configurelist $args
    }

}
```

This is the hard way to do it, by it demonstrates what delegation is all about. See the following answers for the easy way to do it.

Note that the constructor calls the **configurelist** method **after** it creates its **tail**; otherwise, if **-taillength** appeared in the list of **args** we'd get an error.

How can I delegate a method to a component object?

Delegation occurs frequently enough that Snit makes it easy. Any method can be delegated to any component or type component by placing a single delegate statement in the type definition. (See Components and Type components for more information about component names.)

For example, here's a much better way to delegate the **dog** object's **wag** method:

```
% snit::type dog {
    delegate method wag to mytail

    constructor {} {
        install mytail using tail %AUTO%
    }
}
::dog
% snit::type tail {
    method wag {} { return "Wag, wag, wag."}
}
::tail
% dog spot
::spot
% spot wag
Wag, wag, wag.
```

This code has the same effect as the code shown under the previous question: when a **dog**'s

wag method is called, the call and its arguments are passed along automatically to the **tail** object.

Note that when a component is mentioned in a delegate statement, the component's instance variable is defined implicitly. However, it's still good practice to declare it explicitly using the component statement.

Note also that you can define a method name using the **method** statement, or you can define it using delegate; you can't do both.

Can I delegate to a method with a different name?

Suppose you wanted to delegate the **dog**'s **wagtail** method to the **tail**'s **wag** method. After all you wag the tail, not the dog. It's easily done:

```
snit::type dog {
    delegate method wagtail to mytail as wag

    constructor {args} {
        install mytail using tail %AUTO% -partof $self
        $self configurelist $args
    }
}
```

Can I delegate to a method with additional arguments?

Suppose the **tail**'s **wag** method takes as an argument the number of times the tail should be wagged. You want to delegate the **dog**'s **wagtail** method to the **tail**'s **wag** method, specifying that the tail should be wagged exactly three times. This is easily done, too:

```
snit::type dog {
    delegate method wagtail to mytail as {wag 3}
    # ...
}

snit::type tail {
    method wag {count} {
        return [string repeat "Wag " $count]
    }
    # ...
}
```

Can I delegate a method to something other than an object?

Normal method delegation assumes that you're delegating a method (a subcommand of an object command) to a method of another object (a subcommand of a different object command). But not all Tcl objects follow Tk conventions, and not everything you'd to which you'd like to delegate a method is necessary an object. Consequently, Snit makes it easy to delegate a method to pretty much anything you like using the delegate statement's **using** clause.

Suppose your dog simulation stores dogs in a database, each dog as a single record. The database API you're using provides a number of commands to manage records; each takes the record ID (a string you choose) as its first argument. For example, **saverec** saves a record. If

you let the record ID be the name of the dog object, you can delegate the dog's **save** method to the **saverec** command as follows:

```
snit::type dog {
    delegate method save using {saverec %s}
}
```

The **%s** is replaced with the instance name when the **save** method is called; any additional arguments are the appended to the resulting command.

The **using** clause understands a number of other %-conversions; in addition to the instance name, you can substitute in the method name (**%m**), the type name (**%t**), the instance namespace (**%n**), the Tk window name (**%w**), and, if a component or typecomponent name was given in the delegate statement, the component's object command (**%c**).

How can I delegate a method to a type component object?

Just exactly as you would to a component object. The delegate method statement accepts both component and type component names in its **to** clause.

How can I delegate a type method to a type component object?

Use the delegate typemethod statement. It works like delegate method, with these differences: first, it defines a type method instead of an instance method; second, the **using** clause ignores the **%s**, **%n**, and **%w** %-conversions.

Naturally, you can't delegate a type method to an instance component...Snit wouldn't know which instance should receive it.

How can I delegate an option to a component object?

The first question in this section (see Delegation) shows one way to delegate an option to a component; but this pattern occurs often enough that Snit makes it easy. For example, every **tail** object has a **-length** option; we want to allow the creator of a **dog** object to set the tail's length. We can do this:

```
% snit::type dog {
    delegate option -length to mytail

    constructor {args} {
        install mytail using tail %AUTO% -partof $self
        $self configurelist $args
    }
}
::dog
% snit::type tail {
    option -partof
    option -length 5
}
::tail
% dog spot -length 7
::spot
% spot cget -length
7
```

This produces nearly the same result as the **-configuremethod** and **-cgetmethod** shown under the first question in this section: whenever a **dog** object's **-length** option is set or retrieved, the underlying **tail** object's option is set or retrieved in turn.

Note that you can define an option name using the **option** statement, or you can define it using delegate; you can't do both.

Can I delegate to an option with a different name?

In the previous answer we delegated the **dog**'s **-length** option down to its **tail**. This is, of course, wrong. The dog has a length, and the tail has a length, and they are different. What we'd really like to do is give the **dog** a **-taillength** option, but delegate it to the **tail**'s **-length** option:

```
snit::type dog {
    delegate option -taillength to mytail as -length

    constructor {args} {
        set mytail [tail %AUTO% -partof $self]
        $self configurelist $args
    }
}
```

How can I delegate any unrecognized method or option to a component object?

It may happen that a Snit object gets most of its behavior from one of its components. This often happens with **snit::widgetadaptors**, for example, where we wish to slightly the modify the behavior of an existing widget. To carry on with our **dog** example, however, suppose that we have a snit::type called **animal** that implements a variety of animal behaviors--moving, eating, sleeping, and so forth. We want our **dog** objects to inherit these same behaviors, while adding dog-like behaviors of its own. Here's how we can give a **dog** methods and options of its own

while delegating all other methods and options to its **animal** component:

```
snit::type dog {
    delegate option * to animal
    delegate method * to animal

    option -akc 0

    constructor {args} {
        install animal using animal %AUTO% -name $self
        $self configurelist $args
    }

    method wag {} {
        return "$self wags its tail"
    }
}
```

That's it. A **dog** is now an **animal** that has a **-akc** option and can **wag** its tail.

Note that we don't need to specify the full list of method names or option names that **animal** will receive. It gets anything **dog** doesn't recognize--and if it doesn't recognize it either, it will simply throw an error, just as it should.

You can also delegate all unknown type methods to a type component using **delegate typemethod ***.

How can I delegate all but certain methods or options to a component?

In the previous answer, we said that every **dog** is an **animal** by delegating all unknown methods and options to the **animal** component. But what if the **animal** type has some methods or options that we'd like to suppress?

One solution is to explicitly delegate all the options and methods, and forgo the convenience of **delegate method *** and **delegate option ***. But if we wish to suppress only a few options or methods, there's an easier way:

```
snit::type dog {
    delegate option * to animal except -numlegs
    delegate method * to animal except {fly climb}

    # ...

    constructor {args} {
        install animal using animal %AUTO% -name $self -numlegs 4
        $self configurelist $args
    }

    # ...
}
```

Dogs have four legs, so we specify that explicitly when we create the **animal** component, and

explicitly exclude **-numlegs** from the set of delegated options. Similarly, dogs can neither **fly** nor **climb**, so we exclude those **animal** methods as shown.

Can a hierarchical method be delegated?

Yes; just specify multiple words in the delegated method's name:

```
snit::type tail {
    method wag {} {return "Wag, wag"}
    method droop {} {return "Droop, droop"}
}

snit::type dog {
    delegate method {tail wag} to mytail
    delegate method {tail droop} to mytail

    # ...

    constructor {args} {
        install mytail using tail %AUTO%
        $self configurelist $args
    }

    # ...
}
```

Unrecognized hierarchical methods can also be delegated; the following code delegates all subcommands of the "tail" method to the "mytail" component:

```
snit::type dog {
    delegate method {tail *} to mytail

    # ...
}
```

Widgets

What is a snit::widget?

A snit::widget is the Snit version of what Tcl programmers usually call a *megawidget*: a widget-like object usually consisting of one or more Tk widgets all contained within a Tk frame.

A snit::widget is also a special kind of snit::type. Just about everything in this FAQ list that relates to **snit::types** also applies to **snit::widgets**.

How do I define a snit::widget?

snit::widgets are defined using the snit::widget command, just as **snit::types** are defined by the snit::type command.

The body of the definition can contain all of the same kinds of statements, plus a couple of others which will be mentioned below.

How do snit::widgets differ from snit::types?

- The name of an instance of a snit::type can be any valid Tcl command name, in any namespace. The name of an instance of a snit::widget must be a valid Tk widget name, and its parent widget must already exist.
- An instance of a snit::type can be destroyed by calling its **destroy** method. Instances of a snit::widget have no destroy method; use the Tk **destroy** command instead.
- Every instance of a snit::widget has one predefined component called its **hull** component. The hull is usually a Tk **frame** or **toplevel** widget; any other widgets created as part of the snit::widget will usually be contained within the hull.
- snit::widgets can have their options receive default values from The Tk option database.

What is a hull component?

Snit can't create a Tk widget object; only Tk can do that. Thus, every instance of a snit::widget must be wrapped around a genuine Tk widget; this Tk widget is called the *hull component*. Snit effectively piggybacks the behavior you define (methods, options, and so forth) on top of the hull component so that the whole thing behaves like a standard Tk widget.

For snit::widgets the hull component must be a Tk widget that defines the **-class** option.

snit::widgetadaptors differ from snit::widgets chiefly in that any kind of widget can be used as the hull component; see Widget adaptors.

How can I set the hull type for a snit::widget?

A snit::widget's hull component will usually be a Tk **frame** widget; however, it may be any Tk widget that defines the **-class** option. You can explicitly choose the hull type you prefer by including the hulltype command in the widget definition:

```
snit::widget mytoplevel {
    hulltype toplevel

    # ...
}
```

If no hulltype command appears, the hull will be a **frame**.

By default, Snit recognizes the following hull types: the Tk widgets **frame**, **labelframe**, **toplevel**, and the Tile widgets **ttk::frame**, **ttk::labelframe**, and **ttk::toplevel**. To enable the use of some other kind of widget as the hull type, you can **lappend** the widget command to the variable **snit::hulltypes** (always provided the widget defines the **-class** option. For example, suppose Tk gets a new widget type called a **prettyframe**:

```
lappend snit::hulltypes prettyframe

snit::widget mywidget {
    hulltype prettyframe

    # ...
}
```

How should I name widgets which are components of a snit::widget?

Every widget, whether a genuine Tk widget or a Snit megawidget, has to have a valid Tk window name. When a snit::widget is first created, its instance name, **self**, is a Tk window name; however, if the snit::widget is used as the hull component by a snit::widgetadaptor its instance name will be changed to something else. For this reason, every **snit::widget** method, constructor, destructor, and so forth is passed another implicit argument, **win**, which is the window name of the megawidget. Any children should be named using **win** as the root.

Thus, suppose you're writing a toolbar widget, a frame consisting of a number of buttons placed side-by-side. It might look something like this:

```
snit::widget toolbar {
    delegate option * to hull

    constructor {args} {
        button $win.open -text Open -command [mymethod open]
        button $win.save -text Save -command [mymethod save]

        # ....

        $self configurelist $args

    }
}
```

See also the question on renaming objects, toward the top of this file.

Widget adaptors

What is a snit::widgetadaptor?

A snit::widgetadaptor is a kind of snit::widget. Whereas a snit::widget's hull is automatically created and is always a Tk frame, a snit::widgetadaptor can be based on any Tk widget--or on any Snit megawidget, or even (with luck) on megawidgets defined using some other package.

It's called a *widget adaptor* because it allows you to take an existing widget and customize its behavior.

How do I define a snit::widgetadaptor?

Use the snit::widgetadaptor command. The definition for a snit::widgetadaptor looks just like that for a snit::type or snit::widget, except that the constructor must create and install the hull component.

For example, the following code creates a read-only text widget by the simple device of turning its **insert** and **delete** methods into no-ops. Then, we define new methods, **ins** and **del**, which get delegated to the hull component as **insert** and **delete**. Thus, we've adapted the text widget and given it new behavior while still leaving it fundamentally a text widget.

```
::snit::widgetadaptor rotext {

    constructor {args} {
        # Create the text widget; turn off its insert cursor
```

```
        installhull using text -insertwidth 0

        # Apply any options passed at creation time.
        $self configurelist $args
    }

    # Disable the text widget's insert and delete methods, to
    # make this readonly.
    method insert {args} {}
    method delete {args} {}

    # Enable ins and del as synonyms, so the program can insert and
    # delete.
    delegate method ins to hull as insert
    delegate method del to hull as delete

    # Pass all other methods and options to the real text widget, so
    # that the remaining behavior is as expected.
    delegate method * to hull
    delegate option * to hull
}
```

The most important part is in the constructor. Whereas snit::widget creates the hull for you, snit::widgetadaptor cannot -- it doesn't know what kind of widget you want. So the first thing the constructor does is create the hull component (a Tk text widget in this case), and then installs it using the installhull command.

Note: There is no instance command until you create one by installing a hull component. Any attempt to pass methods to **$self** prior to calling installhull will fail.

Can I adapt a widget created elsewhere in the program?

Yes.

At times, it can be convenient to adapt a pre-existing widget instead of creating your own. For example, the Bwidget **PagesManager** widget manages a set of **frame** widgets, only one of which is visible at a time. The application chooses which **frame** is visible. All of the These **frame** s are created by the **PagesManager** itself, using its **add** method. It's convenient to adapt these frames to do what we'd like them to do.

In a case like this, the Tk widget will already exist when the snit::widgetadaptor is created. Snit provides an alternate form of the installhull command for this purpose:

```
snit::widgetadaptor pageadaptor {
    constructor {args} {
        # The widget already exists; just install it.
        installhull $win

        # ...
    }
}
```

Can I adapt another megawidget?

Maybe. If the other megawidget is a snit::widget or snit::widgetadaptor, then yes. If it isn't then, again, maybe. You'll have to try it and see. You're most likely to have trouble with widget destruction--you have to make sure that your megawidget code receives the event before the megawidget you're adapting does.

The Tk option database

What is the Tk option database?

The Tk option database is a database of default option values maintained by Tk itself; every Tk application has one. The concept of the option database derives from something called the X Windows resource database; however, the option database is available in every Tk implementation, including those which do not use the X Windows system (e.g., Microsoft Windows).

Full details about the Tk option database are beyond the scope of this document; both *Practical Programming in Tcl and Tk* by Welch, Jones, and Hobbs, and *Effective Tcl/Tk Programming* by Harrison and McClennan., have good introductions to it.

Snit is implemented so that most of the time it will simply do the right thing with respect to the option database, provided that the widget developer does the right thing by Snit. The body of this section goes into great deal about what Snit requires. The following is a brief statement of the requirements, for reference.

- If the widget's default widget class is not what is desired, set it explicitly using the widgetclass statement in the widget definition.
- When defining or delegating options, specify the resource and class names explicitly when necessary.
- Use the installhull using command to create and install the hull for snit::widgetadaptors.
- Use the install command to create and install all components which are widgets.
- Use the install command to create and install components which aren't widgets if you'd like them to receive option values from the option database.

The interaction of Tk widgets with the option database is a complex thing; the interaction of Snit with the option database is even more so, and repays attention to detail.

Do snit::types use the Tk option database?

No, they don't; querying the option database requires a Tk window name, and snit::types don't have one.

If you create an instance of a snit::type as a component of a snit::widget or snit::widgetadaptor, on the other hand, and if any options are delegated to the component, and if you use install to

create and install it, then the megawidget will query the option database on the snit::type's behalf. This might or might not be what you want, so take care.

What is my snit::widget's widget class?

Every Tk widget has a "widget class": a name that is used when adding option settings to the database. For Tk widgets, the widget class is the same as the widget command name with an initial capital. For example, the widget class of the Tk **button** widget is **Button**.

Similarly, the widget class of a snit::widget defaults to the unqualified type name with the first letter capitalized. For example, the widget class of

```
snit::widget ::mylibrary::scrolledText { ... }
```

is **ScrolledText**.

The widget class can also be set explicitly using the widgetclass statement within the snit::widget definition:

```
snit::widget ::mylibrary::scrolledText {
    widgetclass Text

    # ...
}
```

The above definition says that a **scrolledText** megawidget has the same widget class as an ordinary **text** widget. This might or might not be a good idea, depending on how the rest of the megawidget is defined, and how its options are delegated.

What is my snit::widgetadaptor's widget class?

The widget class of a snit::widgetadaptor is just the widget class of its hull widget; Snit has no control over this.

Note that the widget class can be changed only for **frame** and **toplevel** widgets, which is why these are the valid hull types for snit::widgets.

Try to use snit::widgetadaptors only to make small modifications to another widget's behavior. Then, it will usually not make sense to change the widget's widget class anyway.

What are option resource and class names?

Every Tk widget option has three names: the option name, the resource name, and the class name. The option name begins with a hyphen and is all lowercase; it's used when creating widgets, and with the **configure** and **cget** commands.

The resource and class names are used to initialize option default values by querying the option database. The resource name is usually just the option name minus the hyphen, but may contain uppercase letters at word boundaries; the class name is usually just the resource name with an initial capital, but not always. For example, here are the option, resource, and class names for several Tk **text** widget options:

```
    -background          background          Background
    -borderwidth         borderWidth         BorderWidth
    -insertborderwidth   insertBorderWidth   BorderWidth
    -padx                padX                Pad
```

As is easily seen, sometimes the resource and class names can be inferred from the option name, but not always.

What are the resource and class names for my megawidget's options?

For options implicitly delegated to a component using **delegate option ***, the resource and class names will be exactly those defined by the component. The **configure** method returns these names, along with the option's default and current values:

```
% snit::widget mytext {
    delegate option * to text

    constructor {args} {
        install text using text .text
        # ...
    }

    # ...
}
::mytext
% mytext .text
.text
% .text configure -padx
-padx padX Pad 1 1
%
```

For all other options (whether locally defined or explicitly delegated), the resource and class names can be defined explicitly, or they can be allowed to have default values.

By default, the resource name is just the option name minus the hyphen; the the class name is just the option name with an initial capital letter. For example, suppose we explicitly delegate "-padx":

```
% snit::widget mytext {
    option -myvalue 5

    delegate option -padx to text
    delegate option * to text

    constructor {args} {
        install text using text .text
        # ...
    }

    # ...
}
::mytext
% mytext .text
.text
```

```
% .text configure -myvalue
-myvalue myvalue Myvalue 5 5
% .text configure -padx
-padx padx Padx 1 1
%
```

Here the resource and class names are chosen using the default rules. Often these rules are sufficient, but in the case of "-padx" we'd most likely prefer that the option's resource and class names are the same as for the built-in Tk widgets. This is easily done:

```
% snit::widget mytext {
    delegate option {-padx padX Pad} to text

    # ...
}
::mytext
% mytext .text
.text
% .text configure -padx
-padx padX Pad 1 1
%
```

How does snit initialize my megawidget's locally-defined options?

The option database is queried for each of the megawidget's locally-defined options, using the option's resource and class name. If the result isn't "", then it replaces the default value given in widget definition. In either case, the default can be overridden by the caller. For example,

```
option add *Mywidget.texture pebbled

snit::widget mywidget {
    option -texture smooth
    # ...
}

mywidget .mywidget -texture greasy
```

Here, **-texture** would normally default to "smooth", but because of the entry added to the option database it defaults to "pebbled". However, the caller has explicitly overridden the default, and so the new widget will be "greasy".

How does snit initialize delegated options?

That depends on whether the options are delegated to the hull, or to some other component.

How does snit initialize options delegated to the hull?

A snit::widget's hull is a widget, and given that its class has been set it is expected to query the option database for itself. The only exception concerns options that are delegated to it with a different name. Consider the following code:

```
option add *Mywidget.borderWidth 5
option add *Mywidget.relief sunken
option add *Mywidget.hullbackground red
option add *Mywidget.background green

snit::widget mywidget {
    delegate option -borderwidth to hull
    delegate option -hullbackground to hull as -background
    delegate option * to hull
    # ...
}

mywidget .mywidget

set A [.mywidget cget -relief]
set B [.mywidget cget -hullbackground]
set C [.mywidget cget -background]
set D [.mywidget cget -borderwidth]
```

The question is, what are the values of variables A, B, C and D?

The value of A is "sunken". The hull is a Tk frame which has been given the widget class **Mywidget**; it will automatically query the option database and pick up this value. Since the **-relief** option is implicitly delegated to the hull, Snit takes no action.

The value of B is "red". The hull will automatically pick up the value "green" for its **-background** option, just as it picked up the **-relief** value. However, Snit knows that **-hullbackground** is mapped to the hull's **-background** option; hence, it queries the option database for **-hullbackground** and gets "red" and updates the hull accordingly.

The value of C is also "red", because **-background** is implicitly delegated to the hull; thus, retrieving it is the same as retrieving **-hullbackground**. Note that this case is unusual; the **-background** option should probably have been excluded using the delegate statement's **except** clause, or (more likely) delegated to some other component.

The value of D is "5", but not for the reason you think. Note that as it is defined above, the resource name for **-borderwidth** defaults to **borderwidth**, whereas the option database entry is **borderWidth**, in accordance with the standard Tk naming for this option. As with **-relief**, the hull picks up its own **-borderwidth** option before Snit does anything. Because the option is delegated under its own name, Snit assumes that the correct thing has happened, and doesn't worry about it any further. To avoid confusion, the **-borderwidth** option should have been delegated like this:

```
    delegate option {-borderwidth borderWidth BorderWidth} to hull
```

For snit::widgetadaptors, the case is somewhat altered. Widget adaptors retain the widget class of their hull, and the hull is not created automatically by Snit. Instead, the snit::widgetadaptor must call installhull in its constructor. The normal way to do this is as follows:

```
snit::widgetadaptor mywidget {
    # ...
    constructor {args} {
        # ...
        installhull using text -foreground white
        # ...
    }
    # ...
}
```

In this case, the installhull command will create the hull using a command like this:

```
    set hull [text $win -foreground white]
```

The hull is a **text** widget, so its widget class is **Text**. Just as with snit::widget hulls, Snit assumes that it will pick up all of its normal option values automatically, without help from Snit. Options delegated from a different name are initialized from the option database in the same way as described above.

In earlier versions of Snit, snit::widgetadaptors were expected to call installhull like this:

```
    installhull [text $win -foreground white]
```

This form still works--but Snit will not query the option database as described above.

How does snit initialize options delegated to other components?

For hull components, Snit assumes that Tk will do most of the work automatically. Non-hull components are somewhat more complicated, because they are matched against the option database twice.

A component widget remains a widget still, and is therefore initialized from the option database in the usual way. A **text** widget remains a **text** widget whether it is a component of a megawidget or not, and will be created as such.

But then, the option database is queried for all options delegated to the component, and the component is initialized accordingly--provided that the install command is used to create it.

Before option database support was added to Snit, the usual way to create a component was to simply create it in the constructor and assign its command name to the component variable:

```
snit::widget mywidget {
    delegate option -background to myComp

    constructor {args} {
        set myComp [text $win.text -foreground black]
    }
}
```

The drawback of this method is that Snit has no opportunity to initialize the component properly. Hence, the following approach is now used:

```
snit::widget mywidget {
    delegate option -background to myComp

    constructor {args} {
        install myComp using text $win.text -foreground black
    }
}
```

The install command does the following:

- Builds a list of the options explicitly included in the install command--in this case, **-foreground** .
- Queries the option database for all options delegated explicitly to the named component.
- Creates the component using the specified command, after inserting into it a list of options and values read from the option database. Thus, the explicitly included options (like **-foreground**) will override anything read from the option database.
- If the widget definition implicitly delegated options to the component using **delegate option *** , then Snit calls the newly created component's **configure** method to receive a list of all of the component's options. From this Snit builds a list of options implicitly delegated to the component which were not explicitly included in the install command. For all such options, Snit queries the option database and configures the component accordingly.

You don't really need to know all of this; just use install to install your components, and Snit will try to do the right thing.

What happens if I install a non-widget as a component of widget?

A snit::type never queries the option database. However, a snit::widget can have non-widget components. And if options are delegated to those components, and if the install command is used to install those components, then they will be initialized from the option database just as widget components are.

However, when used within a megawidget, install assumes that the created component uses a reasonably standard widget-like creation syntax. If it doesn't, don't use install.

Ensemble commands

What is an ensemble command?

An ensemble command is a command with subcommands. Snit objects are all ensemble commands; however, the term more usually refers to commands like the standard Tcl commands **string**, **file**, and **clock**. In a sense, these are singleton objects--there's only one instance of them.

How can I create an ensemble command using snit?

There are two ways--as a snit::type, or as an instance of a snit::type.

How can I create an ensemble command using an instance of a snit::type?

Define a type whose Instance methods are the subcommands of your ensemble command. Then, create an instance of the type with the desired name.

For example, the following code uses Delegation to create a work-alike for the standard **string** command:

```
snit::type ::mynamespace::mystringtype {
    delegate method * to stringhandler

    constructor {} {
        set stringhandler string
    }
}

::mynamespace::mystringtype mystring
```

We create the type in a namespace, so that the type command is hidden; then we create a single instance with the desired name-- **mystring**, in this case.

This method has two drawbacks. First, it leaves the type command floating about. More seriously, your shiny new ensemble command will have **info** and **destroy** subcommands that you probably have no use for. But read on.

How can I create an ensemble command using a snit::type?

Define a type whose Type methods are the subcommands of your ensemble command.

For example, the following code uses Delegation to create a work-alike for the standard **string** command:

```
snit::type mystring {
    delegate typemethod * to stringhandler

    typeconstructor {
        set stringhandler string
    }
}
```

Now the type command itself is your ensemble command.

This method has only one drawback, and though it's major, it's also surmountable. Your new ensemble command will have **create**, **info** and **destroy** subcommands you don't want. And worse yet, since the **create** method can be implicit, users of your command will accidentally be creating instances of your **mystring** type if they should mispell one of the subcommands. The command will succeed--the first time--but won't do what's wanted. This is very bad.

The work around is to set some Pragmas, as shown here:

```
snit::type mystring {
    pragma -hastypeinfo    no
    pragma -hastypedestroy no
    pragma -hasinstances   no

    delegate typemethod * to stringhandler

    typeconstructor {
        set stringhandler string
    }
}
```

Here we've used the pragma statement to tell Snit that we don't want the **info** typemethod or the **destroy** typemethod, and that our type has no instances; this eliminates the **create** typemethod and all related code. As a result, our ensemble command will be well-behaved, with no unexpected subcommands.

Pragmas

What is a pragma?

A pragma is an option you can set in your type definitions that affects how the type is defined and how it works once it is defined.

How do I set a pragma?

Use the pragma statement. Each pragma is an option with a value; each time you use the pragma statement you can set one or more of them.

How can I get rid of the "info" type method?

Set the **-hastypeinfo** pragma to **no**:

```
snit::type dog {
    pragma -hastypeinfo no
    # ...
}
```

Snit will refrain from defining the **info** type method.

How can I get rid of the "destroy" type method?

Set the **-hastypedestroy** pragma to **no**:

```
snit::type dog {
    pragma -hastypedestroy no
    # ...
}
```

Snit will refrain from defining the **destroy** type method.

How can I get rid of the "create" type method?

Set the **-hasinstances** pragma to **no**:

```
snit::type dog {
    pragma -hasinstances no
    # ...
}
```

Snit will refrain from defining the **create** type method; if you call the type command with an unknown method name, you'll get an error instead of a new instance of the type.

This is useful if you wish to use a snit::type to define an ensemble command rather than a type with instances.

Pragmas **-hastypemethods** and **-hasinstances** cannot both be false (or there'd be nothing left).

How can I get rid of type methods altogether?

Normal Tk widget type commands don't have subcommands; all they do is create widgets--in Snit terms, the type command calls the **create** type method directly. To get the same behavior from Snit, set the **-hastypemethods** pragma to **no**:

```
snit::type dog {
    pragma -hastypemethods no
    #...
}

# Creates ::spot
dog spot

# Tries to create an instance called ::create
dog create spot
```

Pragmas **-hastypemethods** and **-hasinstances** cannot both be false (or there'd be nothing left).

Why can't I create an object that replaces an old object with the same name?

Up until Snit 0.95, you could use any name for an instance of a snit::type, even if the name was already in use by some other object or command. You could do the following, for example:

```
snit::type dog { ... }

dog proc
```

You now have a new dog named "proc", which is probably not something that you really wanted to do. As a result, Snit now throws an error if your chosen instance name names an existing command. To restore the old behavior, set the **-canreplace** pragma to **yes**:

```
snit::type dog {
    pragma -canreplace yes
    # ...
}
```

How can I make my simple type run faster?

In Snit 1.x, you can set the **-simpledispatch** pragma to **yes**.

Snit 1.x method dispatch is both flexible and fast, but the flexibility comes with a price. If your type doesn't require the flexibility, the **-simpledispatch** pragma allows you to substitute a simpler dispatch mechanism that runs quite a bit faster. The limitations are these:

- Methods cannot be delegated.
- **uplevel** and **upvar** do not work as expected: the caller's scope is two levels up rather than one.
- The option-handling methods (**cget**, **configure**, and **configurelist**) are very slightly slower.

In Snit 2.2, the **-simpledispatch** macro is obsolete, and ignored; all Snit 2.2 method dispatch is faster than Snit 1.x's **-simpledispatch**.

Macros

What is a macro?

A Snit macro is nothing more than a Tcl proc that's defined in the Tcl interpreter used to compile Snit type definitions.

What are macros good for?

You can use Snit macros to define new type definition syntax, and to support conditional compilation.

How do I do conditional compilation?

Suppose you want your type to use a fast C extension if it's available; otherwise, you'll fallback to a slower Tcl implementation. You want to define one set of methods in the first case, and another set in the second case. But how can your type definition know whether the fast C extension is available or not?

It's easily done. Outside of any type definition, define a macro that returns 1 if the extension is available, and 0 otherwise:

```
if {$gotFastExtension} {
    snit::macro fastcode {} {return 1}
} else {
    snit::macro fastcode {} {return 0}
}
```

Then, use your macro in your type definition:

```
snit::type dog {

    if {[fastcode]} {
        # Fast methods
        method bark {} {...}
        method wagtail {} {...}
    } else {
        # Slow methods
        method bark {} {...}
```

```
        method wagtail {} {...}
    }
}
```

How do I define new type definition syntax?

Use a macro. For example, your snit::widget's **-background** option should be propagated to a number of component widgets. You could implement that like this:

```
snit::widget mywidget {
    option -background -default white -configuremethod
    PropagateBackground

    method PropagateBackground {option value} {
        $comp1 configure $option $value
        $comp2 configure $option $value
        $comp3 configure $option $value
    }
}
```

For one option, this is fine; if you've got a number of options, it becomes tedious and error prone. So package it as a macro:

```
snit::macro propagate {option "to" components} {
    option $option -configuremethod Propagate$option

    set body "\n"

    foreach comp $components {
        append body "\$$comp configure $option \$value\n"
    }

    method Propagate$option {option value} $body
}
```

Then you can use it like this:

```
snit::widget mywidget {
    option -background default -white
    option -foreground default -black

    propagate -background to {comp1 comp2 comp3}
    propagate -foreground to {comp1 comp2 comp3}
}
```

Are there are restrictions on macro names?

Yes, there are. You can't redefine any standard Tcl commands or Snit type definition statements. You can use any other command name, including the name of a previously defined macro.

If you're using Snit macros in your application, go ahead and name them in the global

namespace, as shown above. But if you're using them to define types or widgets for use by others, you should define your macros in the same namespace as your types or widgets. That way, they won't conflict with other people's macros.

If my fancy snit::widget is called **::mylib::mywidget**, for example, then I should define my **propagate** macro as **::mylib::propagate**:

```
snit::macro mylib::propagate {option "to" components} { ... }

snit::widget ::mylib::mywidget {
    option -background default -white
    option -foreground default -black

    mylib::propagate -background to {comp1 comp2 comp3}
    mylib::propagate -foreground to {comp1 comp2 comp3}
}
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *snit* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Programming tools

Copyright

```
Copyright (c) 2003-2006, by William H. Duquette
```

soundex

soundex

Keywords: soundex, knuth, text comparison, text likeness

Name

soundex - Soundex

Synopsis

package require Tcl 8.5 9
package require soundex ?**1.1**?
::soundex::knuth *string*

Description

This package provides soundex algorithms which allow the comparison of words based on their

phonetic likeness.

Currently only an algorithm by Knuth is provided, which is tuned to english names and words.

::soundex::knuth *string*
Computes the soundex code of the input *string* using Knuth's algorithm and returns it as the result of the command.

Examples

```
% ::soundex::knuth Knuth
K530
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *soundex* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Hashes, checksums, and encryption

Copyright


```
Copyright (c) ????, Algorithm: Donald E. Knuth
Copyright (c) 2003, Documentation: Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
Copyright (c) 1998, Tcl port: Evan Rempel <erempel@uvic.ca>
```


stooop

stooop

Keywords: stooop, C++, class, object, object oriented

Name

stooop - Object oriented extension.

Synopsis

package require Tcl 8.5 9
package require stooop ?**4.4.2**?
::stooop::class *name body*
::stooop::new *class* ?*arg arg ...*?
::stooop::delete *object* ?*object ...*?
::stooop::virtual proc *name {this ?arg arg ...?}* ?*body*?
::stooop::classof *object*

::stooop::new *object*
::stooop::printObjects ?*pattern*?
::stooop::record
::stooop::report ?*pattern*?

Description

This package provides commands to extend Tcl in an object oriented manner, using a familiar C++ like syntax and behaviour. Stooop only introduces a few new commands: **class**, **new**, **delete**, **virtual** and **classof**. Along with a few coding conventions, that is basically all you need to know to use stooop. Stooop is meant to be as simple to use as possible.

This manual is very succinct and is to be used as a quick reminder for the programmer, who should have read the thorough *stooop_man.html* HTML documentation at this point.

::stooop::class *name body*

This command creates a class. The body, similar in contents to a Tcl namespace (which a class actually also is), contains member procedure definitions. Member procedures can also be defined outside the class body, by prefixing their name with **class::**, as you would proceed with namespace procedures.

proc *class* {this ?*arg arg ...*?} ?*base* {?*arg arg ...*?} ...? *body*

This is the constructor procedure for the class. It is invoked following a **new** invocation on the class. It must have the same name as the class and a first argument named **this**. Any number of base classes specifications, including arguments to be passed to their constructor, are allowed before the actual body of the procedure.

proc *~class* {this} *body*

This is the destructor procedure for the class. It is invoked following a **delete** invocation. Its name must be the concatenation of a single **~** character followed by the class name (as in C++). It must have a single argument named **this**.

proc *name* {this ?*arg arg ...*?} *body*

This is a member procedure of the class, as its first argument is named **this**. It allows a simple access of member data for the object referenced by **this** inside the procedure. For example:

```
set ($this,data) 0
```

proc *name* {?*arg arg ...*?} *body*

This is a static (as in C++) member procedure of the class, as its first argument is not named **this**. Static (global) class data can be accessed as in:

```
set (data) 0
```

proc *class* {this copy} *body*

This is the optional copy procedure for the class. It must have the same name as the class and exactly 2 arguments named **this** and **copy**. It is invoked following a **new** invocation on an existing object of the class.

::stooop::new *class* ?*arg arg ...*?

This command is used to create an object. The first argument is the class name and is followed by the arguments needed by the corresponding class constructor. A unique identifier

for the object just created is returned.

::stooop::delete *object* ?*object ...*?
This command is used to delete one or several objects. It takes one or more object identifiers as argument(s).

::stooop::virtual proc *name* {this ?*arg arg ...*?} ?*body*?
The **virtual** specifier may be used on member procedures to achieve dynamic binding. A procedure in a base class can then be redefined (overloaded) in the derived class(es). If the base class procedure is invoked on an object, it is actually the derived class procedure which is invoked, if it exists. If the base class procedure has no body, then it is considered to be a pure virtual and the derived class procedure is always invoked.

::stooop::classof *object*
This command returns the class of the existing object passed as single parameter.

::stooop::new *object*
This command is used to create an object by copying an existing object. The copy constructor of the corresponding class is invoked if it exists, otherwise a simple copy of the copied object data members is performed.

Debugging

Environment variables

see below ...

STOOOPCHECKDATA
Setting this variable to any true value will cause stooop to check for invalid member or class data access.

STOOOPCHECKPROCEDURES
Setting this variable to any true value will cause stooop to check for invalid member procedure arguments and pure interface classes instanciation.

STOOOPCHECKALL
Setting this variable to any true value will cause stooop to activate both procedure and data member checking.

STOOOPCHECKOBJECTS
Setting this variable to any true value will cause stooop to activate object checking. The following stooop namespace procedures then become available for debugging: **printObjects**, **record** and **report**.

STOOOPTRACEPROCEDURES
Setting this environment variable to either **stdout**, **stderr** or a file name, activates procedure tracing. The stooop library will then output to the specified channel 1 line of informational text for each member procedure invocation.

STOOOPTRACEPROCEDURESFORMAT
Defines the trace procedures output format. Defaults to **"class: %C, procedure: %p, object: %O, arguments: %a"**.

STOOOPTRACEDATA
Setting this environment variable to either **stdout**, **stderr** or a file name, activates data tracing. The stooop library will then output to the specified channel 1 line of informational text for each member data access.

STOOOPTRACEDATAFORMAT

Defines the trace data output format. Defaults to "**class: %C, procedure: %p, array: %A, object: %O, member: %m, operation: %o, value: %v**".

STOOOPTRACEDATAOPERATIONS
When tracing data output, by default, all read, write and unsetting accesses are reported, but the user can set this variable to any combination of the letters **r**, **w**, and **u** for more specific tracing (please refer to the **trace** Tcl manual page for more information).

STOOOPTRACEALL
Setting this environment variable to either **stdout**, **stderr** or a file name, enables both procedure and data tracing.

::stooop::printObjects ?*pattern*?
Prints an ordered list of existing objects, in creation order, oldest first. Each output line contains the class name, object identifier and the procedure within which the creation occurred. The optional pattern argument (as in the Tcl **string match** command) can be used to limit the output to matching class names.

::stooop::record
When invoked, a snapshot of all existing stooop objects is taken. Reporting can then be used at a later time to see which objects were created or deleted in the interval.

::stooop::report ?*pattern*?
Prints the created and deleted objects since the **::stooop::record** procedure was invoked last. If present, the pattern argument limits the output to matching class names.

Examples
Please see the full HTML documentation in *stooop_man.html*.

Bugs, ideas, feedback
If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *stooop* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Programming tools

switched
Keywords: switched, C++, class, object, object oriented

Name
switched - switch/option management.

Synopsis
package require Tcl 8.5 9
package require switched ?**2.2.2**?

complete *this*
options *this*
set-option *this value*

Description

The **switched** class serves as base class for user classes with switch / option configuration procedures. It provides facilities for managing options through a simple interface.

For example:

```
set vehicle [new car -length 4.5 -width 2 -power 100 -fuel diesel]
puts "my car was running on [switched::cget $vehicle -fuel]"
switched::configure $vehicle -power 40 -fuel electricity
puts "but is now running on clean [switched::cget $vehicle -fuel]"
```

Of course, as you might have guessed, the **car** class is derived from the **switched** class. Let us see how it works:

```
class car {
    proc car {this args} switched {$args} {
        # car specific initialization code here
        switched::complete $this
    }
    ...
}
```

The switched class constructor takes the optional configuration option / value pairs as parameters. The switched class layer then completely manages the switched options: it checks their validity, stores their values and provides a clean interface to the user layer configuration setting procedures.

The switched class members available to the programmer are:

complete *this*
: This procedure is used to tell the switched layer that the derived class object (a car in the examples) is completely built. At that time, the initial configuration of the switched object occurs, using default option values (see procedure **options**) eventually overridden by construction time values, passed at the time of the **new** operator invocation. This procedure must be called only once, usually around or at the end of the derived class constructor. (*Note*: Also check the **complete** data member later in this chapter).

options *this*
: This procedure must return the configuration description for *all* options that the switched object will accept. It is a pure virtual member procedure and therefore its implementation is *mandatory* in the derived class layer. The procedure must return a list of lists. Each list pertains to a single option and is composed of the switch name, the default value for the option and an optional initial value. For example:

```
class car {
    ...
    proc options {this} {
        return [list [list -fuel petrol petrol] [list -length {} {}]
    [list -power {} {}] [list -width {} {}] ]
    }
    proc set-fuel {this value} {
        ...
    }
    ...
}
```

In this case, 4 options are specified: **fuel**, **length**, **power** and **width**. The default and initial values for the **fuel** option are identical and set to **petrol**. For the other options, values are all empty.

For each option, there must be a corresponding **set-option** procedure defined in the derived class layer. For example, since we defined a **fuel** option, there is a **set-fuel** procedure in the car class. The parameters always are the object identifier (since this is not a static procedure, but rather a dynamically defined virtual one), followed by the new value for the option. A **set-option** procedure is only invoked if the new value differs from the current one (a caching scheme for improving performance), or if there is no initial value set in the **options** procedure for that option.

In this procedure, if the initial value differs from the default value or is omitted, then initial configuration is forced and the corresponding **set-option** procedure is invoked by the switched **complete** procedure located at the end of the derived class constructor. For example:

```
class car {
    ...
    proc options {this} {
        return [list [list -fuel petrol] [list -length {} {}] [list
    -power 100 50] [list -width {} {}] ]
    }
    ...
}
```

In this case, configuration is forced on the **fuel** and **power** options, that is the corresponding **set-option** procedures will be invoked when the switched object is constructed (see **set-option** procedures documentation below).

For the **fuel** option, since there is no initial value, the **set-fuel** procedure is called with the default value (**petrol**) as argument. For the **power** option, since the initial value differs from the default value, the **set-power** procedure is called with the initial value as argument (**50**).

For the other options, since the initial values (last elements of the option lists) are identical to their default values, the corresponding **set-option** procedures will not be invoked. It is the programmer's responsibility to insure that the initial option values are correct.

set-option *this value*

These procedures may be viewed as dynamic virtual functions. There must be one implementation per supported option, as returned by the **options** procedure. For example:

```
class car {
    ...
    proc options {this} {
        return [list ...
            [list -width {} {}] ]
    }
    ...
    proc set-width {this value} {
        ...
    }
    ...
}
```

Since the **-width** option was listed in the **options** procedure, a **set-width** procedure implementation is provided, which of course would proceed to set the width of the car (and would modify the looks of a graphical representation, for example).

As you add a supported **option** in the list returned by the **options** procedure, the corresponding **set-option** procedure may be called as soon as the switched object is complete, which occurs when the switched level **complete** procedure is invoked. For example:

```
class car {
    proc car {this args} switched {args} {
        ...
        switched::complete $this
   }
    ...
    proc options {this} {
        return [list [list -fuel petrol] [list -length 4.5] [list
    -power 350] [list -width 1.8] ]
    }
    proc set-fuel {this value} {
        ...
    }
    proc set-length {this value} {
        ...
    }
    proc set-power {this value} {
        ...
    }
    proc set-width {this value} {
        ...
```

```
    }
}

new car
```

In this case, a new car is created with no options, which causes the car constructor to be called, which in turns calls the switched level **complete** procedure after the car object layer is completely initialized. At this point, since there are no initial values in any option list in the options procedure, the **set-fuel** procedure is called with its default value of **petrol** as parameter, followed by the **set-length** call with **4.5** value, **set-power** with **350** value and finally with **set-width** with **1.8** as parameter. This is a good way to test the **set-option** procedures when debugging, and when done, just fill-in the initial option values.

The switched layer checks that an option is valid (that is, listed in the **options** procedure) but obviously does not check the validity of the value passed to the **set-option** procedure, which should throw an error (for example by using the Tcl error command) if the value is invalid.

The switched layer also keeps track of the options current values, so that a **set-option** procedure is called only when the corresponding option value passed as parameter is different from the current value (see **-option** data members description).

-option

The **-option** data member is an options current value. There is one for each option listed in the options procedure. It is a read-only value which the switched layer checks against when an option is changed. It is rarely used at the layer derived from switched, except in the few cases, such as in the following example:

```
...
proc car::options {this} {
    return {
        ...
        {-manufacturer {} {}}
        ...
    }
}

proc car::set-manufacturer {this value} {}

proc car::printData {this} {
    puts "manufacturer: $switched::($this,-manufacturer)"
    ...
}
```

In this case, the manufacturer's name is stored at the switched layer level (this is why the set-manufacturer procedure has nothing to do) and later retrieved in the printData procedure.

complete

The **complete** data member (not to be confused with the **complete** procedure) is a boolean. Its initial value is **false** and it is set to **true** at the very end of the switched **complete**

procedure. It becomes useful when some options should be set at construction time only and not dynamically, as the following example shows:

```
proc car::set-width {this value} {
    if {$switched::($this,complete)} {
        error {option -width cannot be set dynamically}
    }
    ...
}
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *stooop* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Programming tools

string

token

Keywords: token, lexing, regex, string, tokenization

Name

string::token - Regex based iterative lexing

Synopsis

package require Tcl 8.5 9
package require string::token ?**1.1**?
package require fileutil
::string token text *lex string*
::string token file *lex path*
::string token chomp *lex startVar string resultVar*

Description

This package provides commands for regular expression based lexing (tokenizing) of strings.

API

::string token text *lex string*

This command takes a dictionary, *lex*, which must map regular expressions ("regexps") to labels, and tokenizes the *string* according to the dictionary.

The command tests each regexp in the *lex* dictionary in order. Since Tcl **dict**s are insertion-ordered, it is recommended that the regexps are specified from most specific to most general.

The command returns a list of tokens. Each token is a 3-element list containing the label (from the *lex* **dict**), and the start and end indexes of the match in the *string*.

The command will throw an error if it is not able to tokenize the entire string.

For line-oriented lexers, finer-grained error handling and handling huge files can be achieved by lexing line-by-line.

::string token file *lex path*

This command is a convenience wrapper around **::string token text** that makes it easy to tokenize entire files.

Note that this command loads the whole file into memory before starting to process it.

::string token chomp *lex startVar string resultVar*

This command is exposed to enable users to write their own lexers. For example, to apply different lexing dictionaries depending on internal state as the lex progresses.

(The **::string token text** and **::string token file** commands make use of this command behind the scenes.)

This command takes a dictionary, *lex*, which must map regexps to labels, a variable *startVar* which contains the index position where the lexing should begin in the input *string*, and a result variable *resultVar* to which each successfuly lexed token will be appended.

The command returns a status code indicating the state of the lex These are

0
: No token found.

1
: Token found.

2
: End of string reached.

If a token was recognized (status **1**) the command will update the index in *startVar* to the character immediately *following* the lexed token. It will also append to the *resultVar* the lexed token which itself is a 3-element list containing the label (from the *lex* **dict**), and the start and end indexes of the match in the *string*.

If a token was not recognized (status **0** or **2**), neither *startVar* nor *resultVar* will be updated.

As noted earlier, the command tests each regexp in the *lex* dictionary in order. Since Tcl **dict**s are insertion-ordered, it is recommended that the regexps are specified from most specific to most general.

Further note that all regex patterns are implicitly prefixed with the constraint escape **\A** to ensure that a match starts exactly at the character index found in *startVar*.

Examples

This example shows how to use the **string token text** command to lex a simple line-oriented *.ini* file format. (For full-featured *.ini* file handling, see inifile.)

```
const INI_EG "; example.ini
Filename = test.db

invalid
\[Database\]
# remote data
Port: 143
IP: 192.0.2.62

junk
"

proc main {} {
    set groups [lex_ini $::INI_EG]
    puts General:
    dict for {key value} [dict get $groups General] { puts "\t$key:
    $value" }
    puts Database:
    dict for {key value} [dict get $groups Database] { puts "\t$key:
    $value" }
}

proc lex_ini ini_text {
    const INI_LEX [dict create {[#;].*$} COMMENT {\[[^]]+\]} GROUP
    {\w+\s*[:=].+$} ENTRY]
    set group General
    set linenum 0
    foreach line [split $ini_text \n] {
        incr linenum
        if {[string trim $line] eq ""} { continue }
        try {
            set token [string token text $INI_LEX $line]
            if {[llength $token]} { set token [lindex $token 0] }
            lassign $token label i j
            switch $label {
                COMMENT { continue }
                GROUP {
                    set group [string range $line $i+1 $j-1]
                    dict set groups $group {}
                }
                ENTRY {
                    set entry [string range $line $i $j]
```

```
                    set k [lindex [regexp -inline -indices {[=:]}
    $entry] 0 0]
                    set key [string trim [string range $entry 0 $k-1]]
                    set value [string trim [string range $entry $k+1
    end]]
                    dict set groups $group $key $value
                }
            }
        } on error err {
            puts "Error: line $linenum: $err"
        }
    }
    return $groups
}

main
=>
Error: line 4: Unexpected character 'i' at offset 0
Error: line 10: Unexpected character 'j' at offset 0
General:
    Filename: test.db
Database:
    Port: 143
    IP: 192.0.2.62
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *textutil* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Text processing

token_shell

Keywords: token_shell, bash, lexing, parsing, shell, string, tokenization

Name

string::token::shell - Parsing of shell command line

Synopsis

package require Tcl 8.5 9
package require string::token::shell ?**1.3**?

package require string::token ?**1**?
package require fileutil
::string token shell ?**-indices**? ?**-partial**? ?--? *string*

Description

This package provides a command which parses a line of text using basic **sh**-syntax into a list of words.

API

::string token shell ?-indices? ?-partial? ?--? *string*

By default, returns a list of words from the input *string*. The words are split from *string* in accordance with basic sh(1p)-syntax.

An error is thrown if the input does not follow the expected syntax.

The command's behaviour can be modified by specifying any of the two options **-indices** and **-partial**.

--

When specified option parsing stops at this point. This option is needed if the input *string* may start with dash. In other words, this is pretty much required if *string* is user input.

-indices

When specified the command returns a list of 4-element lists. Each 4-element list contains the type of the word, its start and end indexes in the *string*, and the word itself (copied from the *string*).

For any given 4-element list, the length of the word as calculated by the difference between the indices may not match the length of the word in the last element. This is because the indices include delimeters, intra-word quoting, etc., whereas the word in the last element has all these stripped away.

The possible token types are

PLAIN

Plain word, not quoted.

D:QUOTED

Word was delimited by double-quotes.

S:QUOTED

Word was delimited by single-quotes.

D:QUOTED:PART

see below ...

S:QUOTED:PART

Like the previous types, but the word has no closing quote, i.e., is incomplete. These token types can only occur if the **-partial** option was specified, and then only for the last word of the result. If the option **-partial** is *not* specified, an incomplete last word will cause the command to throw an error instead.

-partial

When specified, the parser will accept an incomplete quoted word (i.e., with an opening quote but no matching closing quote), at the end of the *string*, without throwing an error.

The basic shell syntax accepted by this command are unquoted, single- and double-quoted words, separated by whitespace. Leading and trailing whitespace are permitted too, and if present are stripped.

Shell variables in their various forms are *not* recognized, nor are sub-shells.

Here are the recognized forms of words:

single-quoted word
: A single-quoted word begins with a single-quote character, i.e. ' (ASCII 39; U+0027) followed by zero or more unicode characters excluding single-quote, and ending with a single-quote.

  The word must be followed by either the end of the *string*, or by whitespace. A word cannot directly follow another word with nothing inbetween.

double-quoted word
: A double-quoted word begins with a double-quote character, i.e. " (ASCII 34; U+0022) followed by zero or more unicode characters excluding double-quote, and ending with a double-quote.

  Unlike for single-quoted words, double-quotes can be embedded in double-quoted words by escaping them using a backslash character **** (ASCII 92; U+005C), e.g., **"...\"..."**. Similarly, any literal backslash can be embedded by being backslash-escaped, e.g., **"...\\..."** .

unquoted word
: Unquoted words are not delimited by quotes and thus cannot contain whitespace or single-quote characters. However, double-quote and backslash characters can be put into unquoted words by escaping them as for double-quoted words.

whitespace
: Whitespace is any unicode whiespace character, i.e., any character for which string is space returns **1** (true), or which matches the regular expression **\\s**.

  Whitespace *may* occur before the first word, and after the last word. Whitespace *must* occur between adjacent words.

Examples

An example of a command line parsed into its constituent words.

```
const CMDLINE "grep --include=*.{tcl,tm,tk} -rl * | grep -v \"The
    Cloud\""
puts "CMDLINE=«$CMDLINE»"
foreach word [string token shell $CMDLINE] {
    puts "word=«$word»"
}
=>
CMDLINE=«grep --include=*.{tcl,tm,tk} -rl * | grep -v "The Cloud"»
word=«grep»
word=«--include=*.{tcl,tm,tk}»
```

```
word=«-rl»
word=«*»
word=«|»
word=«grep»
word=«-v»
word=«The Cloud»
```

An example of the same command line used above, parsed into its constituent words, with tokens and string indices.

```
foreach element [string token shell -indices $CMDLINE] {
    lassign $element token i j word
    puts "token=$token i=$i j=$j word=«$word»"
}
=>
token=PLAIN i=0 j=3 word=«grep»
token=PLAIN i=5 j=27 word=«--include=*.{tcl,tm,tk}»
token=PLAIN i=29 j=31 word=«-rl»
token=PLAIN i=33 j=33 word=«*»
token=PLAIN i=35 j=35 word=«|»
token=PLAIN i=37 j=40 word=«grep»
token=PLAIN i=42 j=43 word=«-v»
token=D:QUOTED i=45 j=55 word=«The Cloud»
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *textutil* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Text processing

stringprep

stringprep

Keywords: stringprep, unicode

Name

stringprep - Implementation of stringprep

Synopsis

package require Tcl 8.5 9

package require stringprep 1.0.3
::stringprep::register *profile* ?*-mapping list*? ?*-normalization form*? ?*-prohibited list*? ?*-prohibitedList list*? ?*-prohibitedCommand command*? ?*-prohibitedBidi boolean*?
::stringprep::stringprep *profile string*
::stringprep::compare *profile string1 string2*

Description

This is a Tcl implementation of the Preparation of Internationalized Strings ("stringprep"). It supports the definition of stringprep profiles and their use to prepare Unicode strings for comparisons as defined in RFC 3454.

Note that RFC 3454 (2002) was superceded by RFC 7564 (2015), which itself was superceded by RFC 8264 (2017).

This package is currently unmaintained; see ticket.

API

If the only string preparation needed is Unicode normalization, the **unicode** package's commands are sufficient.

::stringprep::register *profile* ?*-mapping list*? ?*-normalization form*? ?*-prohibited list*? ?*-prohibitedList list*? ?*-prohibitedCommand command*? ?*-prohibitedBidi boolean*?

Register a **stringprep** profile called *profile* that will apply the specified options.

-mapping *list*

This option specifies **stringprep** mapping tables. The tables are those specified in RFC 3454's Appendix B. The usual list values are {B.1 B.2} or {B.1 B.3} where B.1 contains characters which commonly map to nothing, B.2 is used in profiles with unicode normalization form KC, and B.3 specifies case folding. The default is an empty list which means no mapping.

-normalization *form*

This option specifies which normalization form to apply. The *form* is a string which may be any of "D", "C", "KD", or "KC". Note that formally, RFC 3454 supports either no normalization or "KC". The default is no normalization.

-prohibited *list*

This option specifies a list of RFC 3454 tables with prohibited characters. For example, the list from RFC 3491 is {A.1 C.1.2 C.2.2 C.3 C.4 C.5 C.6 C.7 C.8 C.9}. Formally, RFC 3454 allows prohibiting either all the tables from C.3 to C.9, or none of them. The default is to to prohibit none of them.

-prohibitedList *list*

This option specifies a list of prohibited characters as Unicode code points. For example to prohibit the characters **" & ' /: < > @**, provide a list of their code points: {0x22 0x26 0x27 0x2F 0x3A 0x3C 0x3E 0x40}. The default is an empty list. (If these are specified they are in addition to any prohibited by the *-prohibited* option.)

-prohibitedCommand *command*

This option specifies a *command* to be called with a unicode code point whenever a character is mapped and normalized. If the *command* returns **1** (true), then the character is treated as prohibited. A return value of **0** (false) ensures normal processing. This option is a useful alternative to providing a very large list to the *-prohibitedList* option. The default

is no command.

-prohibitedBidi ***boolean***
If this option is specified and passed nonzero (e.g., **1**; true), then the bidirectional character processing rules defined in RFC 3454's section 6 are used. The default is not to use the bidirectional rules.

::stringprep::stringprep ***profile string***
Returns a possibly modified copy of *string* based on the given *profile*; or throws an error.

The possible errors are: *invalid_profile*—the given *profile* is not defined; *prohibited_character*—the given *string* contains a prohibited character; *prohibited_bidi*—the given *string* contains a prohibited bidirectional sequence.

::stringprep::compare ***profile string1 string2***
Performs a comparison of the two given strings by comparing the result of string-preparing each of them in accordance with the given *profile*. Returns **0** if the strings are equal (in terms of the given *profile*), or **-1** if *string1* is lexicographically less than *string2*, or **1** if *string1* is lexicographically greater than *string2*.

Examples

This example shows how to use **stringprep** to compare strings.

```
::stringprep::register basicprep -mapping {B.1 B.2} -normalization KC
# Three poor UTF-8 byte sequences (to illustrate stringprep, not to
    use in practice):
const EACUTE [encoding convertfrom utf-8 \x65\xCC\x81]
const ANGSTROM [encoding convertfrom utf-8 \xE2\x84\xAB]
const ODOTS [encoding convertfrom utf-8 \x6F\xCC\x88]
const A "Caf${EACUTE} ${ANGSTROM}ngstr${ODOTS}m"
# Best to either use a UTF-8-savvy editor:
const B "Café Ångström"
# Or to use Tcl's Unicode escapes:
const C "Caf\u00E9 \u212Bngstr\u00F6m"

puts "A '$A' hex: [binary encode hex [encoding convertto utf-8 $A]]"
puts "B '$B' hex: [binary encode hex [encoding convertto utf-8 $B]]"
puts "C '$C' hex: [binary encode hex [encoding convertto utf-8 $C]]"
puts "string compare \$A \$B → [expr {[string compare $A $B] ?
    "unequal" : "equal"}]"
puts "string compare \$A \$C → [expr {[string compare $A $C] ?
    "unequal" : "equal"}]"
puts "stringprep::compare basicprep \$A \$B → [expr
    {[stringprep::compare basicprep $A $B] ? "unequal" : "equal"}]"
puts "stringprep::compare basicprep \$A \$C → [expr
    {[stringprep::compare basicprep $A $C] ? "unequal" : "equal"}]"
=>
A 'Cafe´ Ångstro¨m' hex: 43616665cc8120e284ab6e677374726fcc886d
```

```
B 'Café Ångström' hex: 436166c3a920c3856e67737472c3b66d
C 'Café Ångström' hex: 436166c3a920e284ab6e67737472c3b66d
string compare $A $B → unequal
string compare $A $C → unequal
stringprep::compare basicprep $A $B → equal
stringprep::compare basicprep $A $C → equal
```

Legacy examples

Nameprep profile definition (see RFC 3491).

```
::stringprep::register nameprep  -mapping {B.1 B.2}  -normalization KC
     -prohibited {A.1 C.1.2 C.2.2 C.3 C.4 C.5 C.6 C.7 C.8 C.9}
    -prohibitedBidi 1
```

Nodeprep and resourceprep profile definitions (see RFC 3920).

```
::stringprep::register nodeprep  -mapping {B.1 B.2}  -normalization KC
     -prohibited {A.1 C.1.1 C.1.2 C.2.1 C.2.2 C.3 C.4 C.5 C.6 C.7 C.8
    C.9}  -prohibitedList {0x22 0x26 0x27 0x2f 0x3a 0x3c 0x3e 0x40}
    -prohibitedBidi 1

::stringprep::register resourceprep  -mapping {B.1}  -normalization KC
     -prohibited {A.1 C.1.2 C.2.1 C.2.2 C.3 C.4 C.5 C.6 C.7 C.8 C.9}
    -prohibitedBidi 1
```

Authors

Sergei Golovan

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *stringprep* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Copyright

Copyright (c) 2007-2009, Sergei Golovan <sgolovan@nes.ru>

stringprep_data

Keywords: stringprep_data, stringprep, unicode

Name

stringprep::data - stringprep data tables, generated, internal

Synopsis

package require Tcl 8.5 9
package require stringprep::data 1.0.3

Description

The **stringprep::data** package is a helper for **stringprep**, providing it with the data tables needed to perform its functions. It is an *internal* package which should not be accessed on its own. Because of that it has no publicly documented API either. Its implementation is generated by a script.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *stringprep* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Copyright


```
Copyright (c) 2007-2009, Sergei Golovan <sgolovan@nes.ru>
```


unicode

Keywords: unicode, normalization

Name

unicode - Implementation of Unicode normalization

Synopsis

package require Tcl 8.5 9
package require unicode 1.1.1
::unicode::normalizeS *form string*
::unicode::fromstring *string*
::unicode::tostring *codepointsList*
::unicode::normalize *form codepointsList*

Description

This is a Tcl implementation of Unicode normalization commands.

Note that if normalization alone is insufficient (e.g., for string comparisons), the **stringprep** package may prove useful.

API

The *form* argument in the commands listed below must be a string that has one of the following values: **D** (canonical decomposition), **C** (canonical decomposition, followed by canonical composition), **KD** (compatibility decomposition), or **KC** (compatibility decomposition, followed by canonical composition).

::unicode::normalizeS *form string*
Returns a possibly modified copy of *string*, normalized in accordance with the given *form*.

This is a convenience for: *::unicode::tostring* [*unicode::normalize $form* [*::unicode::fromstring $string*]].

::unicode::fromstring *string*
Returns a list of integer Unicode code points representing the characters in *string*.

(These integer code points are used in **unicode** for internal string representation.)

::unicode::tostring *codepointsList*
Returns a string whose characters are those corresponding to the *codepointsList* of integers.

Every integer that is a valid Unicode code point is represented in the returned string by the corresponding Unicode character. Any integer that is not a valid Unicode code point is represented using the Unicode replacement character (?), U+FFFD, unless it is sufficiently big, in which case an error is thrown.

::unicode::normalize *form codepointsList*
Returns a possibly modified copy of the *codepointsList* of integers (which must all be valid Unicode code points), normalized according to the given normalization *form*.

Examples

Some Unicode characters have more than one representation in a Unicode encoding such as UTF-8. Normally, this is not a problem *within* a programming language, since the language will typically always use the same representation for any given character. But for data read in from files or other external sources, any of the possible representations could be used. So in these cases normalization may be needed.

Here is an example of the problem:

```
set a "Å"
set b "\u212B"
set c [encoding convertfrom utf-8 \xC3\x85]
set d [encoding convertfrom utf-8 \xE2\x84\xAB]
set e [encoding convertfrom utf-8 \x41\xCC\x8A]
set s [join [list $a $b $c $d $e] ""]
puts "s=$s a=b→[expr {$a eq $b}] a=c→[expr {$a eq $c}] a=d→[expr {$a
    eq $d}] a=e→[expr {$a eq $e}] b=c→[expr {$a eq $d}] b=d→[expr {$a
    eq $d}] b=e→[expr {$a eq $e}] c=d→[expr {$a eq $d}] c=e→[expr {$a
    eq $e}]"
=>
s=ÅÅÅÅA˚ a=b→0 a=c→1 a=d→0 a=e→0 b=c→0 b=d→0 b=e→0 c=d→0 c=e→0
```

The above output shows that although all the characters are the same, they have different UTF-8 representations and do *not* compare equal.

The solution is to normalize. For example:

```
set t [unicode::normalizeS KC $s]
set a [string index $t 0]
set b [string index $t 1]
set c [string index $t 2]
set d [string index $t 3]
set e [string index $t 4]
puts "t=$t a=b→[expr {$a eq $b}] a=c→[expr {$a eq $c}] a=d→[expr {$a
    eq $d}] a=e→[expr {$a eq $e}] b=c→[expr {$a eq $d}] b=d→[expr {$a
    eq $d}] b=e→[expr {$a eq $e}] c=d→[expr {$a eq $d}] c=e→[expr {$a
    eq $e}]"
=>
t=ÅÅÅÅÅ a=b→1 a=c→1 a=d→1 a=e→1 b=c→1 b=d→1 b=e→1 c=d→1 c=e→1
```

After normalization all the characters have the same representation and correctly compare as equal.

Legacy examples

```
% ::unicode::fromstring "\u0410\u0411\u0412\u0413"
1040 1041 1042 1043
% ::unicode::tostring {49 50 51 52 53}
12345
%
```

```
% ::unicode::normalize D {7692 775}
68 803 775
% ::unicode::normalizeS KD "\u1d2c"
A
%
```

References

1. "Unicode Standard Annex #15: Unicode Normalization Forms", (http://unicode.org/reports/tr15/)

Authors

Sergei Golovan

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *stringprep* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Copyright

Copyright (c) 2007, Sergei Golovan <sgolovan@nes.ru>

unicode_data

Keywords: unicode_data, stringprep, unicode

Name

unicode::data - unicode data tables, generated, internal

Synopsis

package require Tcl 8.5 9
package require unicode::data 1.1.1

Description

The **unicode::data** package is a helper for **unicode**, providing it with the data tables needed to perform its functions. It is an *internal* package which should not be accessed on its own. Because of that it has no publicly documented API either. Its implementation is generated by a script.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *stringprep* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Copyright

Copyright (c) 2007, Sergei Golovan <sgolovan@nes.ru>

struct

disjointset

Keywords: disjointset, disjoint set, equivalence class, find, merge find, partition, partitioned set, union

Name

struct::disjointset - Disjoint set data structure

Synopsis

package require Tcl 8.6 9
package require struct::disjointset ?**1.2**?
::struct::disjointset *aDisjointSet*
aDisjointSet **add-element** *element*
aDisjointSet **add-partition** *elements*
aDisjointSet **partitions**

aDisjointSet **num-partitions**
aDisjointSet **equal** *a b*
aDisjointSet **merge** *a b*
aDisjointSet **find** *element*
aDisjointSet **exemplars**
aDisjointSet **find-exemplar** *element*
aDisjointSet **destroy**

Description

This package provides a command for handling disjoint sets. Alternative names for this kind of data structure are *union-find* and *merge-find*.

A common use case for ordinary sets is to be able to answer the question, "does set S contain element E?". The answer is a simple Boolean.

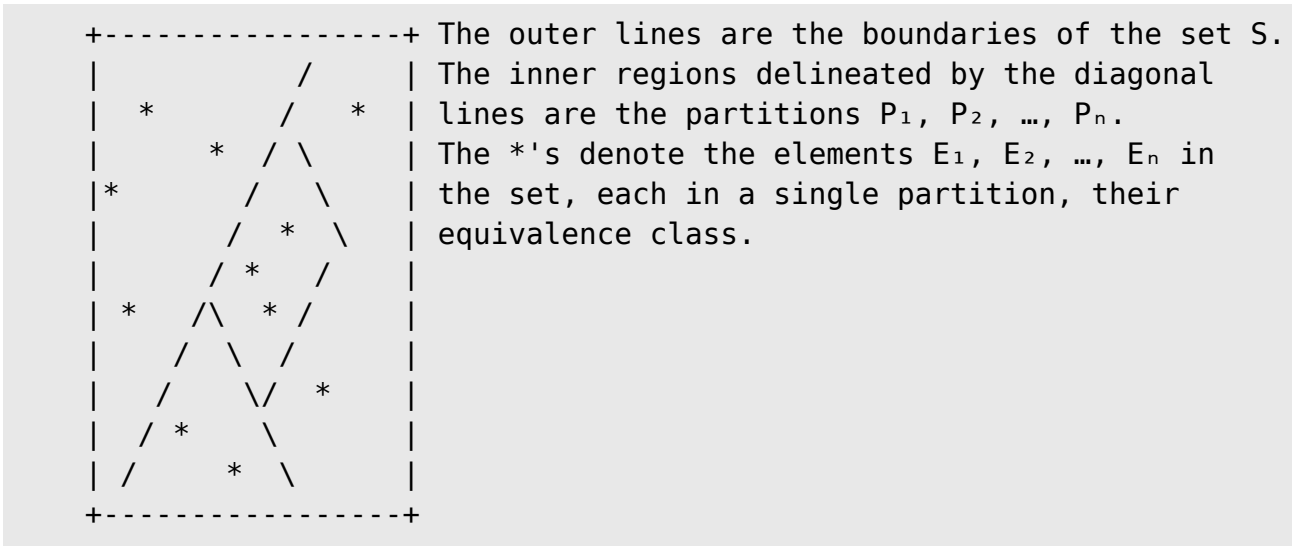
One common use case for disjoint sets is to be able to answer the question, "which of sets S_1, S_2, …, S_n contains element E?". The answer is a set—or nothing, if none of the sets contains E.

Another common use case for disjoint sets is to be able to quickly merge two sets into one, with the resultant set still fast for finding elements. Hence the term *merge-find*.

The reason this data structure is called a *disjoint set* is because it can be perceived as being a *single* set with *partitions*. In other words a disjoint set is:

- a finite *set* S, containing
- a number of *elements* E_1, E_2, …, E_n, grouped into
- a set of *partitions* P_1, P_2, …, P_n. The latter term applies because the intersection of each pair P_i, P_j of partitions is empty, i.e., $\emptyset = P_i \cap P_j$, with the set itself equal to the union of all the partitions, i.e., $S = P_1 \cup P_2 \cup \ldots \cup P_n$.

An alternative name for the *partitions* would be *equivalence classes*, where all elements in the same class are considered equal. Here is a pictorial representation of the concepts listed above:



API

The package provides a single command, **::struct::disjointset**, which provides all its functionality using methods.

::struct::disjointset *aDisjointSet*

Creates a new disjoint set object with an associated global Tcl command whose name is *aDisjointSet*. This command may be used to invoke disjoint set methods which all have the the following general form:

aDisjointSet method ?*arg ...*?

The *method* and the *arg*s specify what operation to perform.

Methods

aDisjointSet* add-element *element

Creates a new partition in the *aDisjointSet*, and adds the single *element* to the new partition. The command maintains the integrity of the disjoint set, i.e., if *element* is already in the disjoint set—no matter in which partition—this method will throw an error.

This method returns the empty string.

This method runs in constant time, *O(1)*.

aDisjointSet* add-partition *elements

Creates a new partition in the *aDisjointSet*, and adds all the given *elements* to the new partition. The command maintains the integrity of the disjoint set, i.e., if any of the elements in *elements* is already in the disjoint set—no matter in which partition—this method will throw an error.

This method returns the empty string.

This method runs in time proportional to the size of *elements*, *O(N)*, where *N* is the number of *elements*.

***aDisjointSet* partitions**

Returns the *aDisjointSet*'s set of partitions as a list of lists. The outer lists are the partitions, the inner lists contain each partition's elements.

This method runs in $O(N \times \alpha(N))$ time, where *N* is the number of elements in the disjoint set, and the α function is the inverse Ackermann function.

***aDisjointSet* num-partitions**

Returns the number of partitions in the *aDisjointSet*.

This method runs in constant time.

aDisjointSet* equal *a b

Returns **1** (true) if elements *a* and *b* in the *aDisjointSet* are in the same partition; otherwise returns **0** (false).

If one or both arguments are not in the disjoint set at all, this method will throw an error.

This method runs in amortized time $O(\alpha(N))$, where *N* is the number of elements in the larger partition, and the α function is the inverse Ackermann function.

aDisjointSet* merge *a b

Determines the partitions in the *aDisjointSet* that elements *a* and *b* are contained in, and merges these partitions into a single partition. If the two elements were already contained in the same partition the disjoint set is left unchanged.

If one or both arguments are not in the disjoint set at all, this method will throw an error.

This method returns the empty string.

This method runs in amortized time $O(\alpha(N))$, where N is the number of elements in the larger of the partitions being merged, and the α function is the inverse Ackermann function. The worst case time is $O(N)$.

aDisjointSet* find *element

Returns a list of the members of the partition of the *aDisjointSet* which contains the given *element*; or an empty string if *element* is not in the disjoint set.

This method runs in $O(N \times \alpha(N))$ time, where N is the total number of elements in the disjoint set, and the α function is the inverse Ackermann function. See **find-exemplar** for a faster method, if all that is needed is a unique partition identifier, rather than a list of its elements.

***aDisjointSet* exemplars**

Returns a list containing an exemplar—an arbitrarily chosen member—of each partition in the *aDisjointSet*.

Note that the only operation that will change the exemplar chosen for any particular partition is **merge**.

This method runs in $O(N \times \alpha(N))$ time, where N is the total number of elements in the disjoint set, and the α function is the inverse Ackermann function.

aDisjointSet* find-exemplar *element

Returns an exemplar—an arbitrarily chosen member—of the partition of the *aDisjointSet* that contains the given *element*.

If *element* is not in the disjoint set at all, this method will throw an error.

Note that the only operation that will change the exemplar chosen for any particular partition is **merge**.

This method runs in $O(\alpha(N))$ time, where N is the number of elements in the partition containing *element*, and the α function is the inverse Ackermann function.

***aDisjointSet* destroy**

Destroys the *aDisjointSet* object, including the global command *aDisjointSet*, and all its associated memory.

Examples

The examples make use of the following **proc** to show their results:

```
proc dump_disjointset name {
    puts "$name:"
    foreach partition [$name partitions] {
        puts -nonewline "  partition "
        set sep "{"
        foreach element $partition {
            puts -nonewline $sep$element
            set sep " "
        }
        puts "}"
    }
}
```

This example creates a new disjoint set called *colors* and adds one partition with the single element *black* and another partition with the elements *cyan*, *magenta*, *yellow*.

```
struct::disjointset colors
colors add-element black
puts "colors num-partitions=[colors num-partitions]"
colors add-partition {cyan magenta yellow}
puts "colors num-partitions=[colors num-partitions]"
dump_disjointset colors
=>
colors num-partitions=1
colors num-partitions=2
colors:
  partition {black}
  partition {cyan magenta yellow}
```

This example explores some of the disjoint set API on the *colors* disjoint set created above.

```
puts "colors equal black yellow=[colors equal black yellow]"
puts "colors equal cyan yellow=[colors equal cyan yellow]"
puts "colors equal cyan magenta=[colors equal cyan magenta]"
colors merge black cyan
puts "colors equal black yellow=[colors equal black yellow]"
dump_disjointset colors
=>
colors equal black yellow=0
colors equal cyan yellow=1
colors equal cyan magenta=1
colors merge black cyan
colors equal black yellow=1
colors:
  partition {black cyan magenta yellow}
```

This example continues to explore some of the disjoint set API on the *colors* disjoint set created above.

```
colors add-partition {red green blue}
dump_disjointset colors
puts "[colors find-exemplar blue] [colors find-exemplar yellow]"
colors destroy
=>
colors:
  partition {black cyan magenta yellow}
  partition {red green blue}
red cyan
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *struct :: disjointset* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Data structures

graph

Keywords: graph, adjacent, arc, cgraph, degree, edge, loop, neighbour, node, serialization, subgraph, vertex

Name

struct::graph - Create and manipulate directed graph objects

Synopsis

package require Tcl 8.5 9
package require struct::graph ?**2.4.4**?
package require struct::list ?**1.8.6**?
package require struct::set ?**2.2.4**?
::struct::graph ?*aGraph*? ?=|:=|*as*|*deserialize source*?
aGraph **=** *sourceGraph*
aGraph **-->** *destGraph*
aGraph **append** *key value*
aGraph **deserialize** *serialization*
aGraph **destroy**
aGraph **arc append** *arc key value*
aGraph **arc attr** *key*
aGraph **arc attr** *key* **-arcs** *list*
aGraph **arc attr** *key* **-glob** *globpattern*
aGraph **arc attr** *key* **-regexp** *repattern*
aGraph **arc delete** *arc* ?*arc* ...?

aGraph **arc exists** *arc*
aGraph **arc flip** *arc*
aGraph **arc get** *arc key*
aGraph **arc getall** *arc* ?*glob*?
aGraph **arc getunweighted**
aGraph **arc getweight** *arc*
aGraph **arc keys** *arc* ?*glob*?
aGraph **arc keyexists** *arc key*
aGraph **arc insert** *start end* ?*name*?
aGraph **arc lappend** *arc key value*
aGraph **arc rename** *arc newname*
aGraph **arc set** *arc key* ?*value*?
aGraph **arc setunweighted** ?*weight*?
aGraph **arc setweight** *arc weight*
aGraph **arc unsetweight** *arc*
aGraph **arc hasweight** *arc*
aGraph **arc source** *arc*
aGraph **arc target** *arc*
aGraph **arc nodes** *arc*
aGraph **arc move-source** *arc newsource*
aGraph **arc move-target** *arc newtarget*
aGraph **arc move** *arc newsource newtarget*
aGraph **arc unset** *arc key*
aGraph **arc weights**
aGraph **arcs** ?*-key key -value value*? ?*-filter cmdprefix*? ?*-in*|*-out*|*-adj*|*-inner*|*-embedding node ?node*? ...?
aGraph **lappend** *key value*
aGraph **node append** *node key value*
aGraph **node attr** *key*
aGraph **node attr** *key* **-nodes** *list*
aGraph **node attr** *key* **-glob** *globpattern*
aGraph **node attr** *key* **-regexp** *repattern*
aGraph **node degree** ?*-in*|*-out*? *node*
aGraph **node delete** *node* ?*node...*?
aGraph **node exists** *node*
aGraph **node get** *node key*
aGraph **node getall** *node* ?*glob*?
aGraph **node keys** *node* ?*glob*?
aGraph **node keyexists** *node key*
aGraph **node insert** ?*node...*?
aGraph **node lappend** *node key value*
aGraph **node opposite** *node arc*
aGraph **node rename** *node newname*
aGraph **node set** *node key* ?*value*?
aGraph **node unset** *node key*
aGraph **nodes** ?*-key key*? ?*-value value*? ?*-filter cmdprefix*? ?*-in*|*-out*|*-adj*|*-inner*|*-embedding*

node node...?
aGraph **get** *key*
aGraph **getall** ?*pattern*?
aGraph **keys** ?*pattern*?
aGraph **keyexists** *key*
aGraph **serialize** ?*node...*?
aGraph **set** *key* ?*value*?
aGraph **swap** *node1 node2*
aGraph **unset** *key*
aGraph **walk** *node* ?*-type type*? ?*-order order*? ?*-dir direction*? -command *cmd*

Description

A directed graph is a data structure containing two collections of elements, *nodes* and *arcs* (also called *edges*), along with a relation that connects the nodes and arcs.

Each arc is connected to two nodes, one of which is called the *source* and the other the *target*. This imposes a direction upon the arc, which is said to go from the source to the target. When an arc's source and target are the same node, the arc is called a *loop*.

When a node is the source or target of an arc, the node and arc are said to be *adjacent*. Furthermore, if two nodes are connected via one or more arcs, the nodes are also said to be *adjacent*.

A node can be the source of any number of *outgoing arcs*, and also the target of any number of *incoming arcs*. The number of *outgoing arcs* is a node's *out-degree*, and the number of *incoming arcs* is a node's *in-degree*.

This **struct::graph** package can manage graphs of nodes, arcs, and their relationships. In addition, the package supports the addition of any number of named *attributes* which can be associated with a graph itself, as well as with any of the graph's nodes and arcs.

See the **struct::graph::ops** package for commands that implement a wide range of common graph algorithms.

Note: The major version of the **struct::graph** package has been changed to version 2, due to backward incompatible changes in the module's API. Users of version 1 are recommended to read the section Changes for v2 for a full list of the changes, including incompatibilities with version 1.

As of version 2.2, a critcl based C implementation is now provided, which requires Tcl 8.4 or later.

API

::struct::graph ?*aGraph*? ?=|:=|as|deserialize *source*?

This command creates a new graph object with an associated global Tcl command. If *aGraph* is given, the global command will be called the given name; otherwise, a unique name will be generated by the package itself.

If a *source* is specified, the new graph will be initialized from it. For the operators **=**,**:=**, and **as**, the *source* argument is interpreted as the name of another graph object, and the assignment operator **=** will be executed. For the operator **deserialize**, the *source* is assumed to be a serialized graph object, and **deserialize** will be executed.

For example:

```
    ::struct::graph graphA = graphB
```

is equivalent to

```
    ::struct::graph graphA
    graphA = graphB
```

and

```
    ::struct::graph graphA deserialize $graphB
```

is equivalent to

```
    ::struct::graph graphA
    graphA deserialize $graphB
```

Once the graph object—and its corresponding command—has been created, the command can be used to apply various graph operations. These all have the same general form:

aGraph method ?*arg ...*?

The *method* and the *arg*s specify what operation to perform.

Methods

aGraph = sourceGraph

The **=** operator is the graph object *assignment* operator. It copies the graph contained in the graph object *sourceGraph* over the graph data in *aGraph*. The old contents of *aGraph* are deleted by this operation.

This operation is in effect equivalent to

```
    aGraph deserialize [sourceGraph serialize]
```

The operation assumes that the *sourceGraph* provides a **serialize** method which returns a valid graph serialization.

aGraph --> destGraph

The **-->** operator is the graph object *reverse assignment* operator. It copies the graph contained in the graph object *aGraph* over the graph data in the object *destGraph*. The old contents of *destGraph* are deleted by this operation.

This operation is in effect equivalent to

```
    destGraph deserialize [aGraph serialize]
```

The operation assumes that the *destGraph* provides a **deserialize** method which takes a graph serialization, and that *aGraph* provides a **serialize** method which returns a valid graph serialization.

aGraph* append *key value

Appends the *value* to the graph attribute associated with the given *key*. Returns the new value of the graph attribute associated with the *key*.

aGraph* deserialize *serialization
This is the complement of the **serialize** method. It replaces the graph data in *aGraph* with the graph described by the *serialization* value. The old contents of *aGraph* are deleted by this operation.

***aGraph* destroy**
Destroys the graph, including its storage space and associated command.

aGraph* arc append *arc key value
Appends the *value* to the given *arc*'s attribute associated with the given *key*. Returns the new value of the *arc* attribute associated with the *key*.

aGraph* arc attr *key
see below ...

aGraph* arc attr *key* -arcs *list
see below ...

aGraph* arc attr *key* -glob *globpattern
see below ...

aGraph* arc attr *key* -regexp *repattern
When no options are specified, the **arc attr** method returns a **dict** whose keys are the names of *every* arc in the *aGraph* graph. For each of the resultant dict's keys, the corresponding value is the value of the arc attribute associated with the given *key*. Arcs which do not have the given *key* are not included in the result **dict**.

If one of the options is specified, in effect the same results **dict** is created as for no options, but filtered so that only the arcs which match the given option's constraint are included in the result **dict**.

The possible constraints are:

-arcs
The value must be a list of arc names. Only the arcs mentioned in this list are searched for the *key* attribute.

-glob
The value is a glob pattern. Only the arcs in the graph whose names match this pattern are searched for the *key* attribute.

-regexp
The value is a regular expression. Only the arcs in the graph whose names match this pattern are searched for the *key* attribute.

***aGraph* arc delete *arc* ?*arc* ...?**
Removes the specified arcs from the graph.

If one or more of the arcs doesn't exit an error is thrown.

aGraph* arc exists *arc
Returns **1** (true) if the specified *arc* exists in the graph; otherwise returns **0** (false).

aGraph* arc flip *arc
Reverses the direction of the *arc*, i.e., the arc's source and target nodes are exchanged with each other.

aGraph* arc get *arc key

Returns the value associated with the *arc*'s *key*.

If the *arc* doesn't exist or if the *key* doesn't exist, an error is thrown.

***aGraph* arc getall *arc* ?*glob*?**
Returns a dictionary (suitable for use with [**array set**]) of the *arc*'s attribute keys and values.

If a *glob* pattern is specified, only the attributes whose keys match will be included.

***aGraph* arc getunweighted**
Returns a list of the names of all the graph's arcs which have *no* weight associated with them.

aGraph* arc getweight *arc
Returns the weight associated with the *arc*.

If the *arc* doesn't exist or if it has no weight, an error is thrown.

***aGraph* arc keys *arc* ?*glob*?**
Returns a list of keys for the *arc*.

If *glob* is specified only the attributes whose names match the glob will be part of the returned list.

aGraph* arc keyexists *arc key
Returns **1** (true) if the *arc* has the specified *key*; otherwise returns **0** (false).

***aGraph* arc insert *start end* ?*name*?**
Inserts an arc into the graph beginning at the *start* node and ending at the *end* node. The new arc is called *name* if specified; otherwise a unique system-generated name of form *arcx* is used.

aGraph* arc lappend *arc key value
Appends the given *value* (as a list) to the value associated with the *arc*'s *key* and returns the *arc*'s *key*'s new value.

aGraph* arc rename *arc newname
Renames the *arc* to *newname* and returns *newname*.

Thows an error if the given *arc* does not exist, or if an arc called *newname* already exists.

***aGraph* arc set *arc key* ?*value*?**
Get or set the value of the given *arc*'s associated *key*.

If *value is not* specified, the value of the given *arc*'s associated *key* is returned. If a *value is* specified, then the *value* is assigned to the given *key*, and the value of the given *arc*'s associated *key* is returned.

An arc may have any number of key-value pairs associated with it.

***aGraph* arc setunweighted ?*weight*?**
Sets the weight of all arcs without a weight to *weight*. If no *weight* is specified it defaults to **0**. Returns the empty string.

aGraph* arc setweight *arc weight
Sets the weight of the *arc* to *weight* and returns *weight*.

aGraph* arc unsetweight *arc

Removes the weight of the *arc*, if present; otherwise does nothing. Returns the empty string.

aGraph* arc hasweight *arc

Returns **1** (true) if the *arc* has a weight; otherwise returns **0** (false).

aGraph* arc source *arc

Returns the node the given *arc* begins at.

aGraph* arc target *arc

Returns the node the given *arc* ends at.

aGraph* arc nodes *arc

Returns a two-element list of the nodes the given *arc* begins and ends at.

aGraph* arc move-source *arc newsource

Changes the *arc*'s source node to *newsource*. This can be described as rotating the arc around its target node.

aGraph* arc move-target *arc newtarget

Changes the *arc*'s target node to *newtarget*. This can be described as rotating the arc around its source node.

aGraph* arc move *arc newsource newtarget

Changes the *arc*'s source and target nodes to *newsource* and *newtarget* respectively.

aGraph* arc unset *arc key

Removes the *arc*'s *key* and its corresponding value. Does nothing if the *key* doesn't exist.

***aGraph* arc weights**

Returns a **dict** where each key is the name of an arc which has a weight and each corresponding value is that arc's weight.

***aGraph* arcs ?-key *key* ?-value *value*?? ?-filter *cmdprefix*?**
?-in|-out|-adj|-inner|-embedding *node* ?node? ...?

Returns a list of the graph's arcs.

If no constraint is specified, every arc is in the returned list.

Constraints

Constraints can limit which arcs are included in the returned list based on the nodes that are connected by the arc, on the keyed values associated with the arc, or both. A general filter command can also be used. The constraints that involve connected nodes take a variable number of nodes as argument, specified after the name of the constraint itself.

At most one **-key** constraint may be specified, and if present then at most one **-value** constraint my also be used. The **key** (and if present **-value**), if specified, will be applied *second*. (If two or more **-key** options are used or if a **-value** is used without a corresponding **key**, an error will be thrown.)

-key *key*

Only return arcs that have the associated *key*.

-value *value*

Only return arcs that have the associated *key* (which *must* be specified using the **-key** option), *and* for which the *key*'s value is *value*.

At most one **-filter** command may be specified, and if present, this is applied *last*. (If two or more **-filter** options are used an error will be thrown.)

-filter *cmdprefix*
Only return arcs for which the given *cmdprefix* returns **1** (true).

For each candidate arc the *cmdprefix* is executed in the context of the caller. It is called with two arguments, the name of the graph object, and the name of the candidate arc. The *cmdprefix* command should return either **1** (true; to keep the arc in the list to be returned), or **0** (false; to remove the arc from the list to be returned).

At most one constraint may be specified using one of **-in**, **-out**, **-adj**, **-inner**, or **-embedding**. If one of these is present, it is applied *first*. (If two or more of these options is used an error will be thrown.)

-in
Only return arcs whose target is one of the specified nodes. In other words, compute the union of all the incoming arcs for the specified nodes.

-out
Only return arcs whose source is one of the specified nodes. In other words, compute the union of all the outgoing arcs for the specified nodes.

-adj
Only return arcs that are adjacent to at least one of the specified nodes. In other words, compute the union of all the incoming *and* outgoing arcs for the specified nodes (i.e., the union of using **-in** and **-out**).

-inner
Only return arcs that are adjacent to two of the specified nodes. This is the set of arcs in the subgraph formed by the specified nodes.

-embedding
Only return arcs that are adjacent to exactly one of the specified nodes. This is the set of arcs connecting the subgraph formed by the specified nodes to the rest of the graph.

Note: After specifiying one of the options above, any word with a leading dash which is not a valid option is treated as a node name instead of an invalid option. This condition holds until either a valid option terminates the list of nodes, or the end of the command is reached, whichever comes first.

aGraph* lappend *key value
Appends the *value* (as a list) to the graph attribute with the given *key*, and returns the attribute with the *key*'s new value.

aGraph* node append *node key value
Appends the *value* to the *node*'s attribute with the given *key*, and returns the attribute with the *key*'s new value.

aGraph* node attr *key
see below ...

aGraph* node attr *key* -nodes *list
see below ...

aGraph* node attr *key* -glob *globpattern
see below ...

aGraph* node attr *key* -regexp *repattern

Returns a **dict** whose keys are node names and whose corresponding values are those of the nodes' attribute values associated with the given *key*.

The returned **dict** will include all the graph's nodes that have the given *key or* if an optional constraint is specified, the subset of those nodes which match the given constraint. This means that any nodes which do *not* have the given *key* or which do *not* satisfy the constraint (if given) will not be included in the returned **dict**.

The possible constraints are:

-nodes
Only the graph's nodes that are in the given *list* and have the specified *key*, will be included in the results **dict**.

-glob
Only the graph's nodes whose names match the given *globpattern* glob pattern and have the specified *key*, will be included in the results **dict**.

-regexp
Only the graph's nodes whose names match the given *repattern* regular expression and have the specified *key*, will be included in the results **dict**.

aGraph* node degree ?-in|-out? *node
Returns the number of arcs adjacent to the specified *node*.

If the **-in** constraint is specified, only the incoming arcs are counted; if the **-out** constraint is specified, only the outgoing arcs are counted. If neither constraint is specified, then all incoming and outgoing arcs are counted.

***aGraph* node delete *node* ?*node*...?**
Deletes the specified nodes from the graph, *and also* deletes all the nodes' arcs (to prevent unconnected arcs).

aGraph* node exists *node
Returns **1** (true) if the specified *node* exists in the graph; otherwise returns **0** (false).

aGraph* node get *node key
Returns the value associated with the *node*'s *key*.

Throws an error if the *node* does not have the given *key*.

***aGraph* node getall *node* ?*glob*?**
Returns a **dict** (suitable for use with [**array set**]) for the *node*. The **dict**'s keys are attribute names (i.e., their keys) and the values are the corresponding attribute values.

If *glob* is specified, only the attributes whose names (i.e., keys) match this glob pattern, will included in the returned dictionary.

***aGraph* node keys *node* ?*glob*?**
Returns a list of the *node*'s attribute keys.

If *glob* is specified, only the attributes whose names (i.e., keys) match this glob pattern, will included in the returned list.

aGraph* node keyexists *node key
Returns **1** (true) if the *node* has the specified *key*; otherwise returns **0** (false).

***aGraph* node insert ?*node*...?**
Inserts one or more nodes into the graph, and returns a list of the inserted nodes. These new nodes are not connected to any arcs.

If no node is specified, a single new node will be inserted and returned. The new node will have a unique name generated by the system and having the form *nodex*.

aGraph* node lappend *node key value
Appends the *value* (as a list) the *node*'s attribute with the given *key*, and returns the attribute with the *key*'s new value.

aGraph* node opposite *node arc
Returns the node at the other end of the specified *arc*, which must be adjacent to the given *node*.

aGraph* node rename *node newname
Renames the *node* to *newname* and returns the node's new name.

An error is thrown if the node does not exist, or if a node called *newname* already exists.

***aGraph* node set *node key* ?*value*?**
Get or set the given *node*'s attribute that's identified by the given *key*.

If *value* is not specified, this command returns the *key*'s current value. If *value* is specified, this command assigns *value* to the *node*'s *key*.

Each node may have any number of keyed values associated with it.

aGraph* node unset *node key
Removes the *node*'s *key* and its associated value from the *node*'s attributes. Does nothing if the *node* does not have the given *key*.

***aGraph* nodes ?-key *key*? ?-value *value*? ?-filter *cmdprefix*?**
?-in|-out|-adj|-inner|-embedding *node node*...?
Returns a list of the graph's nodes. Every node is included in the list unless constraints are specified.

Constraints can limit the nodes included in the list based on the keyed values associated with the node or on neighboring nodes. The constraints that involve neighboring nodes take a list of nodes as argument, specified after the name of the constraint itself.

The constraints are essentially the same as for the **arcs** method. Although the exact meanings change slightly, since here the contraints operate on nodes rather than arcs, the general behaviour is the same, especially when it comes to the handling of words with a leading dash in node lists.

Constraints

-in
Return the *incoming neighbours*, i.e., a list of all the nodes with at least one outgoing arc ending in a node in the specified set of nodes. (Alternatively specified as the set of source nodes for the node set's **-in** arcs.)

-out
Return the *outgoing neighbours*, i.e., a list of all the nodes with at least one incoming arc

starting in a node in the specified set of nodes. (Alternatively specified as the set of target nodes for the node set's **-out** arcs.)

-adj
Return the *neighbours*, i.e., the union of the nodes returned by **-in** and **-out**.

-inner
Return the set of neighbours (see **-adj** above) which are also in the given set of nodes, i.e., the intersection of the neighbours returned by **-adj** and the set of nodes.

-embedding
Return the set of neighbours (see **-adj** above) which are *not* in the set of nodes, i.e., the difference between the neighbours returned by **-adj** and the set of nodes.

-key *key*
Return only nodes which have the given *key*.

-value *value*
Return only nodes which have the given *key* and whose associated value is *value*.

This constraint can only be used in combination with **-key**.

-filter *cmdprefix*
Return only those nodes for which the command *cmdprefix* returns **1** (true).

The command *cmdprefix* is called with two arguments, the name of the graph object, and the name of the node in question. It is executed in the context of the caller and must return a Boolean value, **0** (false; do not include the node) or **1** (true; include the node).

aGraph* get *key
Returns the value associated with the graph's *key*.

Throws an error if the graph does not have the given *key*.

See the **set** method.

***aGraph* getall ?*pattern*?**
Returns a dictionary (suitable for use with [**array set**]) containing all the graph's keys and their associated values.

If the *pattern* is specified only those attributes whose names match the pattern will be part of the returned dictionary. The pattern is a **glob** pattern.

***aGraph* keys ?*pattern*?**
Returns a list of all the graph's keys.

If the *pattern* is specified only those attributes whose names match the pattern will be part of the returned list. The pattern is a **glob** pattern.

aGraph* keyexists *key
Returns **1** (true) if the graph has the specified *key*; otherwise returns **0** (false).

***aGraph* serialize ?*node...*?**
Returns the graph (or a subgraph of the graph) serialized into a single Tcl value. The value can be deserialized back into a graph using the **deserialize** method.

The serialization is of the entire graph, unless one or more *node*s are specified, in which case only the subgraph spanned by the given nodes is serialized.

Since a serialized graph is a Tcl value it can be copied like any Tcl string, e.g., sent to a channel, persisted into a file, etc.

This method also serves as the basis for the graph's copy constructor and assignment operator.

Every graph interface's serialize method's implementation must produce semantically equivalent serialization values so that these values may be used interchangeably between implementations.

The serialized value is a list of triples, plus a **dict** at the end. This means that *[llength $serial] % 3 == 1*. Valid values include 1, 4, 7, ...

The last element of the list is a **dict** containing the attributes associated with the whole graph.

All the preceding elements are triples, one per node, each consisting of:
1. The node's name.
2. A **dict** containing the node's attributes.
3. A list of all the arcs starting at this node (see below).

Each element of a node's list of arcs is itself a list of three or four elements as follows:
1. The arc's name.
2. A reference to the arc's destination node. This reference is the integer index of the destination node in the main serialization list. (This integer will always be >= *0* and < *[llength $serial]* and a multiple of three.)

   Note: For internal consistency serialization requires that every arc name used in a graph must be unique, whether in the same node, or at some other node.
3. A **dict** containing the arc's attributes.
4. An optional arc weight. If absent it means that this arc has no associated weight.

   Note: This information is new, compared to the serialization of **graph** 2.3 and earlier. Making arc weights optional in the new format preserves some compatibility with the old. In particular, any graph that does *not* use weights will have a serialization which is deserializeable by the older graph package. However, if weights are used, older packages will not be able to deserialize.

For all attribute dictionaries, each key is an attribute name and each key's associated value holds the attribute's value.

Note: The serialization's order of nodes and of arcs within each node has no significance. In other words two semantically equivalent graphs could have different serializations.

```
    # A possible serialization for the graph structure
    #
    #        d -----> %2
    #       /          ^ \
    #      /          /   \
```

```
#     /         b     \
#    /         /       \
#  %1 <- a - %0         e
#    ^         \\      /
#     \\        c     /
#      \\        \\  /
#       \\        v v
#        f ------ %3
# is
#
# %3 {} {{f 6 {}}} %0 {} {{a 6 {}} {b 9 {}} {c 0 {}}} %1 {} {{d
9 {}}} %2 {} {{e 0 {}}} {}
#
# This assumes that the graph has neither attribute data nor
weighted arcs.
```

***aGraph* set *key* ?*value*?**

Get or set the value associated with the given graph *key*.

If no *value* is given the value associated with the *key* is returned. If *value* is given, the *key* is set to the specified *value* and the *value* is returned.

Throws an error if no *value* is specified (i.e., when getting) and the graph does not have the given *key*.

A graph may have any number of keyed values associated with it. See also the **get** method.

aGraph* swap *node1 node2

Swap the positions of *node1* and *node2* in the graph.

aGraph* unset *key

Removes the graph's *key* and its corresponding value. Does nothing if the *key* doesn't exist.

aGraph* walk *node* ?-type *type*? ?-order *order*? ?-dir *direction*? -command *cmd

Perform a breadth-first or depth-first walk of the graph starting at *node* and going **forward** (the direction of outgoing arcs) or **backward** (the opposite direction to the incoming arcs).

If specified, the walk's type must be **dfs** (depth-first; the default), or **bfs** breadth-first.

If specified, the walk's order must be **pre** (pre-order; the default), or **post** (post-order), or **both**.

Pre-order means that a node is visited before any of its neighbours (as defined by the *direction*, see below). Post-order means that a node is visited after any of its neighbours. Both-order walking means that a node is visited before *and* after any of its neighbours. Note that the breadth-first type of walk may *only* be combined with pre-order; only depth-first may be used with pre-order, post-order, or both-order.

If specified, the walk's direction must be **backward** (the direction opposite to the incoming

arcs), or **forward** (the direction of the outgoing arcs).

As the walk progresses, the command *cmd* will be evaluated at each node and passed three arguments. The arguments are the call's mode (**enter** or **leave**), the graph itself (i.e., *aGraph*), and the current node's name. For a pre-order walk, all nodes are entered (mode **enter** is passed to *cmd*); for a post-order all nodes are left (mode **leave** is passed). In a both-order walk the first visit to a node is the entry (mode **enter**), and the second visit is the leave (mode **leave**).

Changes for v2

Changes for v2.4

The **serialize** method's serialization format changed to account for the (optional) use of weights. See the method's documentation for details.

Changes for v2.x

The following noteworthy changes have occurred:

1. The API for accessing attributes and their values has been simplified.

   All functionality regarding the default attribute "data" has been removed. This default attribute no longer exists. All accesses to attributes must specify the attribute's name. This backward *incompatible* change led to simplified signatures for all methods involving attributes.

   In particular, the flag **-key** has gone. Please read the documentation for the arc and node methods **get**, **getall**, **set**, **unset**, **append**, **lappend**, **keyexists**, and **keys**, for a description of the new APIs.
2. The methods **keys** and **getall** now take an optional glob pattern argument which if specified will limit the returned attribute data to only those keys which match the pattern.
3. Arcs and nodes can now be renamed. See the documentation for the methods **arc rename** and **node rename**.
4. The structure has been extended with APIs for graph serialization and deserialization, and a number of operations based on these methods (e.g., graph assignment, copy construction).

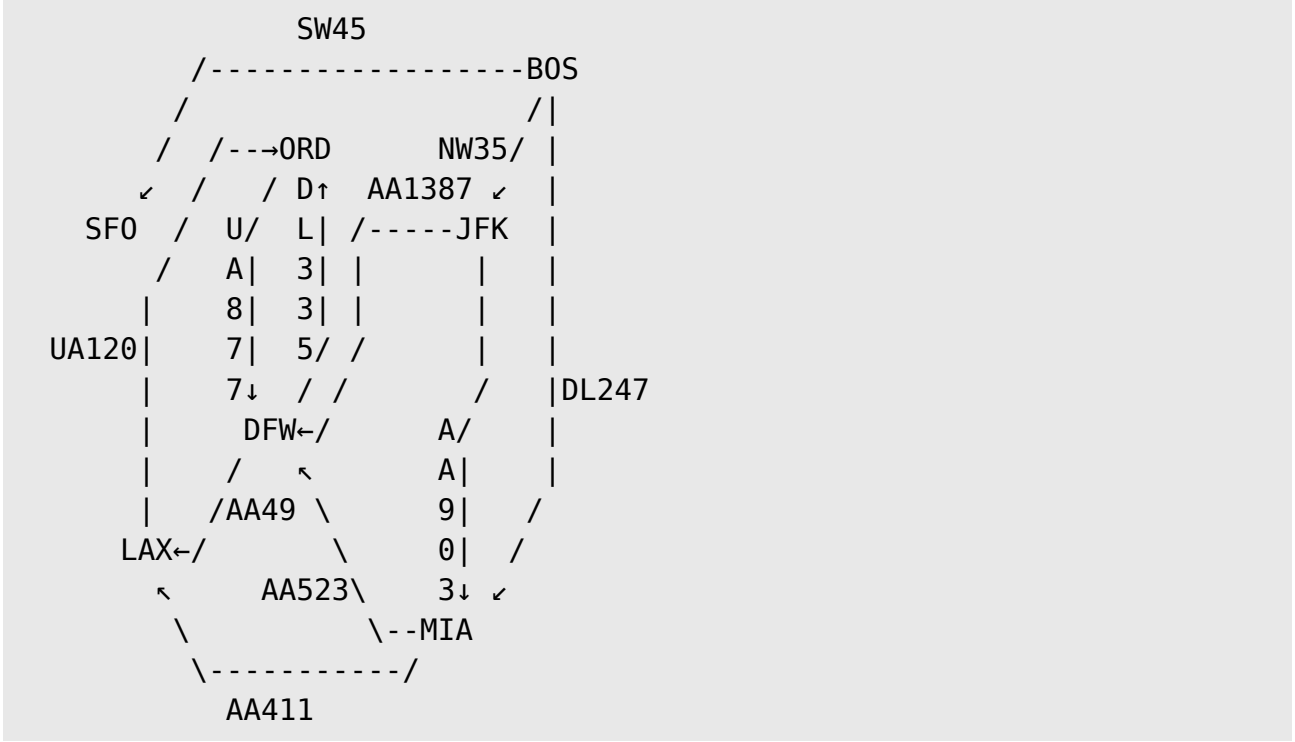
   Please read the documentation for the methods **serialize**, **deserialize**, **=**, and **-->**, and the documentation on the construction of graph objects.

   Serialization and deserialization mean that graphs can be copied like any Tcl string, e.g., sent to a channel, persisted into a file, etc.
5. Both **arc**'s and **node**'s now have an **attr** method which supports access to attribute data regardless of arc and node relationships.
6. Both **arcs** and **nodes** have been extended with the ability to specify constraints to filter the arcs or nodes they return; see their documentation for details.

Examples

The following example shows the basics of using the **struct::graph** package.

An illustration of the graph used in the example:



The code that represents the graph shown above, along with weights (flight distances in kilometers).

```
proc visit_airport {mode graph airport} {
    puts " [expr {$mode eq "enter" ? "→" : "←"}] $graph $airport"
}

try { ;# Adapted from "Data Structures and Algorithms in Java"
    ::struct::graph flights
    flights node insert SFO
    flights node insert LAX
    flights node insert ORD
    flights node insert DFW
    flights node insert JFK
    flights node insert BOS
    flights node insert MIA
    flights arc insert JFK SFO SW45
    flights arc setweight SW45 4152
    flights arc insert JFK DFW AA1387
    flights arc setweight AA1387 2235
    flights arc insert JFK MIA AA903
    flights arc setweight AA903 1756
    flights arc insert BOS JFK NW35
    flights arc setweight NW35 300
    flights arc insert BOS MIA DL247
    flights arc setweight DL247 2027
    flights arc insert MIA DFW AA523
    flights arc setweight AA523 1802
```

```
    flights arc insert MIA LAX AA411
    flights arc setweight AA411 3763
    flights arc insert LAX ORD UA120
    flights arc setweight UA120 2802
    flights arc insert DFW LAX AA49
    flights arc setweight AA49 1983
    flights arc insert DFW ORD DL335
    flights arc setweight DL335 1290
    flights arc insert ORD DFW UA877
    flights arc setweight UA877 1290
    puts "flights walk ORD:"
    puts [flights walk ORD -order both -command visit_airport]
    puts "All flights: [lsort -dictionary [flights arcs]]"
    puts "Chicago flights: [flights arcs -embedding ORD]"
    puts "Chicago direct connections: [flights nodes -embedding ORD]"
} finally {
    flights destroy
}
=>
flights walk ORD:
 → flights ORD
 → flights DFW
 → flights LAX
 ← flights LAX
 ← flights DFW
 ← flights ORD

All flights: AA49 AA411 AA523 AA903 AA1387 DL247 DL335 NW35 SW45 UA120
    UA877
Chicago flights: DL335 UA120 UA877
Chicago direct connections: DFW LAX
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *struct :: graph* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Data structures

Copyright

Copyright (c) 2002-2009,2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>

graph1

Keywords: graph1, cgraph, graph

Name

struct::graph_v1 - Create and manipulate directed graph objects

Synopsis

package require Tcl 8.5 9
package require struct::graph ?**1.2.2**?
graphName *option* ?*arg arg ...*?
graphName **destroy**
graphName **arc append** *arc* ?*-key key*? *value*
graphName **arc delete** *arc* ?*arc ...*?
graphName **arc exists** *arc*
graphName **arc get** *arc* ?*-key key*?
graphName **arc getall** *arc*
graphName **arc keys** *arc*
graphName **arc keyexists** *arc* ?*-key key*?
graphName **arc insert** *start end* ?*child*?
graphName **arc lappend** *arc* ?*-key key*? *value*
graphName **arc set** *arc* ?*-key key*? ?*value*?
graphName **arc source** *arc*
graphName **arc target** *arc*
graphName **arc unset** *arc* ?*-key key*?
graphName **arcs** ?*-key key*? ?*-value value*? ?*-in*|*-out*|*-adj*|*-inner*|*-embedding nodelist*?
graphName **node append** *node* ?*-key key*? *value*
graphName **node degree** ?*-in*|*-out*? *node*
graphName **node delete** *node* ?*node ...*?
graphName **node exists** *node*
graphName **node get** *node* ?*-key key*?
graphName **node getall** *node*
graphName **node keys** *node*
graphName **node keyexists** *node* ?*-key key*?
graphName **node insert** ?*child*?
graphName **node lappend** *node* ?*-key key*? *value*
graphName **node opposite** *node arc*
graphName **node set** *node* ?*-key key*? ?*value*?
graphName **node unset** *node* ?*-key key*?
graphName **nodes** ?*-key key*? ?*-value value*? ?*-in*|*-out*|*-adj*|*-inner*|*-embedding nodelist*?
graphName **get** ?*-key key*?
graphName **getall**
graphName **keys**
graphName **keyexists** ?*-key key*?

graphName **set** ?*-key key*? ?*value*?
graphName **swap** *node1 node2*
graphName **unset** ?*-key key*?
graphName **walk** *node* ?*-order order*? ?*-type type*? ?*-dir direction*? -command *cmd*

Description

This package has been superseded by **struct::graph** *version 2.*

The ::struct::graph command creates a new graph object with an associated global Tcl command whose name is *graphName*. This command may be used to invoke various operations on the graph. It has the following general form:

graphName *option* ?*arg arg ...*?
Option and the *arg*s determine the exact behavior of the command.

A directed graph is a structure containing two collections of elements, called *nodes* and *arcs* respectively, together with a relation ("connectivity") that places a general structure upon the nodes and arcs.

Each arc is connected to two nodes, one of which is called the *source* and the other the *target*. This imposes a direction upon the arc, which is said to go from the source to the target. It is allowed that source and target of an arc are the same node. Such an arc is called a *loop*. Whenever a node is source or target of an arc both are said to be *adjacent*. This extends into a relation between nodes, i.e. if two nodes are connected through at least one arc they are said to be *adjacent* too.

Each node can be the source and target for any number of arcs. The former are called the *outgoing arcs* of the node, the latter the *incoming arcs* of the node. The number of edges in either set is called the *in-* resp. the *out-degree* of the node.

In addition to maintaining the node and arc relationships, this graph implementation allows any number of keyed values to be associated with each node and arc.

The following commands are possible for graph objects:

***graphName* destroy**
Destroy the graph, including its storage space and associated command.

graphName* arc append *arc* ?-key *key*? *value
Appends a *value* to one of the keyed values associated with an *arc*. If no *key* is specified, the key **data** is assumed.

***graphName* arc delete *arc* ?*arc* ...?**
Remove the specified arcs from the graph.

graphName* arc exists *arc
Return true if the specified *arc* exists in the graph.

***graphName* arc get *arc* ?-key *key*?**
Return the value associated with the key *key* for the *arc*. If no key is specified, the key **data** is assumed.

graphName* arc getall *arc
Returns a serialized list of key/value pairs (suitable for use with [**array set**]) for the *arc*.

graphName* arc keys *arc
Returns a list of keys for the *arc*.

***graphName* arc keyexists *arc* ?-key *key*?**
Return true if the specified *key* exists for the *arc*. If no *key* is specified, the key **data** is assumed.

***graphName* arc insert *start end* ?*child*?**
Insert an arc named *child* into the graph beginning at the node *start* and ending at the node *end*. If the name of the new arc is not specified the system will generate a unique name of the form *arcx*.

graphName* arc lappend *arc* ?-key *key*? *value
Appends a *value* (as a list) to one of the keyed values associated with an *arc*. If no *key* is specified, the key **data** is assumed.

***graphName* arc set *arc* ?-key *key*? ?*value*?**
Set or get one of the keyed values associated with an arc. If no key is specified, the key **data** is assumed. Each arc that is added to a graph has the empty string assigned to the key **data** automatically. An arc may have any number of keyed values associated with it. If *value* is not specified, this command returns the current value assigned to the key; if *value* is specified, this command assigns that value to the key.

graphName* arc source *arc
Return the node the given *arc* begins at.

graphName* arc target *arc
Return the node the given *arc* ends at.

***graphName* arc unset *arc* ?-key *key*?**
Remove a keyed value from the arc *arc*. If no key is specified, the key **data** is assumed.

***graphName* arcs ?-key *key*? ?-value *value*? ?-in|-out|-adj|-inner|-embedding *nodelist*?**
Return a list of arcs in the graph. If no restriction is specified a list containing all arcs is returned. Restrictions can limit the list of returned arcs based on the nodes that are connected by the arc, on the keyed values associated with the arc, or both. The restrictions that involve connected nodes have a list of nodes as argument, specified after the name of the restriction itself.

-in
Return a list of all arcs whose target is one of the nodes in the *nodelist*.

-out
Return a list of all arcs whose source is one of the nodes in the *nodelist*.

-adj
Return a list of all arcs adjacent to at least one of the nodes in the *nodelist*. This is the union of the nodes returned by **-in** and **-out**.

-inner
Return a list of all arcs adjacent to two of the nodes in the *nodelist*. This is the set of arcs in the subgraph spawned by the specified nodes.

-embedding
Return a list of all arcs adjacent to exactly one of the nodes in the *nodelist*. This is the set of arcs connecting the subgraph spawned by the specified nodes to the rest of the graph.

-key *key*
Limit the list of arcs that are returned to those arcs that have an associated key *key*.

-value *value*

This restriction can only be used in combination with **-key**. It limits the list of arcs that are returned to those arcs whose associated key *key* has the value *value*.

The restrictions imposed by either **-in**, **-out**, **-adj**, **-inner**, or **-embedded** are applied first. Specifying more than one of them is illegal. At last the restrictions set via **-key** (and **-value**) are applied. Specifying more than one **-key** (and **-value**) is illegal.

graphName* node append *node* ?-key *key*? *value
Appends a *value* to one of the keyed values associated with an *node*. If no *key* is specified, the key **data** is assumed.

graphName* node degree ?-in|-out? *node
Return the number of arcs adjacent to the specified *node*. If one of the restrictions **-in** or **-out** is given only the incoming resp. outgoing arcs are counted.

***graphName* node delete *node* ?*node* ...?**
Remove the specified nodes from the graph. All of the nodes' arcs will be removed as well to prevent unconnected arcs.

graphName* node exists *node
Return true if the specified *node* exists in the graph.

***graphName* node get *node* ?-key *key*?**
Return the value associated with the key *key* for the *node*. If no key is specified, the key **data** is assumed.

graphName* node getall *node
Returns a serialized list of key/value pairs (suitable for use with [**array set**]) for the *node*.

graphName* node keys *node
Returns a list of keys for the *node*.

***graphName* node keyexists *node* ?-key *key*?**
Return true if the specified *key* exists for the *node*. If no *key* is specified, the key **data** is assumed.

***graphName* node insert ?*child*?**
Insert a node named *child* into the graph. The nodes has no arcs connected to it. If the name of the new child is not specified the system will generate a unique name of the form *nodex*.

graphName* node lappend *node* ?-key *key*? *value
Appends a *value* (as a list) to one of the keyed values associated with an *node*. If no *key* is specified, the key **data** is assumed.

graphName* node opposite *node arc
Return the node at the other end of the specified *arc*, which has to be adjacent to the given *node*.

***graphName* node set *node* ?-key *key*? ?*value*?**
Set or get one of the keyed values associated with a node. If no key is specified, the key **data** is assumed. Each node that is added to a graph has the empty string assigned to the key **data** automatically. A node may have any number of keyed values associated with it. If *value* is not specified, this command returns the current value assigned to the key; if *value* is specified, this command assigns that value to the key.

***graphName* node unset *node* ?-key *key*?**
Remove a keyed value from the node *node*. If no key is specified, the key **data** is assumed.

***graphName* nodes ?-key *key*? ?-value *value*? ?-in|-out|-adj|-inner|-embedding *nodelist*?**

Return a list of nodes in the graph. Restrictions can limit the list of returned nodes based on neighboring nodes, or based on the keyed values associated with the node. The restrictions that involve neighboring nodes have a list of nodes as argument, specified after the name of the restriction itself.

The possible restrictions are the same as for method **arcs**. The set of nodes to return is computed as the union of all source and target nodes for all the arcs satisfying the restriction as defined for **arcs**.

graphName **get ?-key *key*?**
Return the value associated with the key *key* for the graph. If no key is specified, the key **data** is assumed.

graphName **getall**
Returns a serialized list of key/value pairs (suitable for use with [**array set**]) for the whole graph.

graphName **keys**
Returns a list of keys for the whole graph.

graphName **keyexists ?-key *key*?**
Return true if the specified *key* exists for the whole graph. If no *key* is specified, the key **data** is assumed.

graphName **set ?-key *key*? ?*value*?**
Set or get one of the keyed values associated with a graph. If no key is specified, the key **data** is assumed. Each graph has the empty string assigned to the key **data** automatically. A graph may have any number of keyed values associated with it. If *value* is not specified, this command returns the current value assigned to the key; if *value* is specified, this command assigns that value to the key.

graphName **swap *node1 node2***
Swap the position of *node1* and *node2* in the graph.

graphName **unset ?-key *key*?**
Remove a keyed value from the graph. If no key is specified, the key **data** is assumed.

graphName **walk *node* ?-order *order*? ?-type *type*? ?-dir *direction*? -command *cmd***
Perform a breadth-first or depth-first walk of the graph starting at the node *node* going in either the direction of outgoing or opposite to the incoming arcs.

The type of walk, breadth-first or depth-first, is determined by the value of *type*; **bfs** indicates breadth-first, **dfs** indicates depth-first. Depth-first is the default.

The order of the walk, pre-order, post-order or both-order is determined by the value of *order*; **pre** indicates pre-order, **post** indicates post-order, **both** indicates both-order. Pre-order is the default. Pre-order walking means that a node is visited before any of its neighbors (as defined by the *direction*, see below). Post-order walking means that a parent is visited after any of its neighbors. Both-order walking means that a node is visited before *and* after any of its neighbors. The combination of a bread-first walk with post- or both-order is illegal.

The direction of the walk is determined by the value of *dir*; **backward** indicates the direction opposite to the incoming arcs, **forward** indicates the direction of the outgoing arcs.

As the walk progresses, the command *cmd* will be evaluated at each node, with the mode of the call (**enter** or **leave**) and values *graphName* and the name of the current node appended. For a pre-order walk, all nodes are **enter**ed, for a post-order all nodes are left. In a both-order walk the first visit of a node **enter**s it, the second visit **leave**s it.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *struct :: graph* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Data structures

Copyright


```
Copyright (c) 2002 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


graphops

Keywords: graphops, adjacency list, adjacency matrix, adjacent, approximation algorithm, arc, articulation point, augmenting network, augmenting path, bfs, bipartite, blocking flow, bridge, complete graph, connected component, cut edge, cut vertex, degree, degree constrained spanning tree, diameter, dijkstra, distance, eccentricity, edge, flow network, graph, heuristic, independent set, isthmus, level graph, local searching, loop, matching, max cut, maximum flow, minimal spanning tree, minimum cost flow, minimum degree spanning tree, minimum diameter spanning tree, neighbour, node, radius, residual graph, shortest path, squared graph, strongly connected component, subgraph, travelling salesman, vertex, vertex cover

Name

struct::graph::op - Operations for directed and undirected graph objects

Synopsis

package require Tcl 8.6 9
package require struct::graph::op ?**0.11.4**?
struct::graph::op::toAdjacencyMatrix *g*
struct::graph::op::toAdjacencyList *g* ?*options...*?
struct::graph::op::kruskal *g*
struct::graph::op::prim *g*
struct::graph::op::isBipartite? *g* ?*partitions*?
struct::graph::op::tarjan *g*
struct::graph::op::connectedComponents *g*
struct::graph::op::connectedComponentOf *g node*

struct::graph::op::isConnected? *g*
struct::graph::op::isCutVertex? *g node*
struct::graph::op::isBridge? *g arc*
struct::graph::op::isEulerian? *g* ?*tourvar*?
struct::graph::op::isSemiEulerian? *g* ?*pathvar*?
struct::graph::op::dijkstra *g start_node* ?*options...*?
struct::graph::op::distance *g start_node end_node* ?*-arcmode mode*?
struct::graph::op::eccentricity *g node* ?*-arcmode mode*?
struct::graph::op::radius *g* ?*-arcmode mode*?
struct::graph::op::diameter *g* ?*-arcmode mode*?
struct::graph::op::BellmanFord *g startnode*
struct::graph::op::Johnsons *g* ?*options...*?
struct::graph::op::FloydWarshall *g*
struct::graph::op::MetricTravellingSalesman *g*
struct::graph::op::Christofides *g*
struct::graph::op::GreedyMaxMatching *g*
struct::graph::op::MaxCut *g U V*
struct::graph::op::UnweightedKCenter *g k*
struct::graph::op::WeightedKCenter *g nodeWeights W*
struct::graph::op::GreedyMaxIndependentSet *g*
struct::graph::op::GreedyWeightedMaxIndependentSet *g nodeWeights*
struct::graph::op::VerticesCover *g*
struct::graph::op::EdmondsKarp *g s t*
struct::graph::op::BusackerGowen *g desiredFlow s t*
struct::graph::op::ShortestsPathsByBFS *g s outputFormat*
struct::graph::op::BFS *g s* ?*outputFormat...*?
struct::graph::op::MinimumDiameterSpanningTree *g*
struct::graph::op::MinimumDegreeSpanningTree *g*
struct::graph::op::MaximumFlowByDinic *g s t blockingFlowAlg*
struct::graph::op::BlockingFlowByDinic *g s t*
struct::graph::op::BlockingFlowByMKM *g s t*
struct::graph::op::createResidualGraph *g f*
struct::graph::op::createAugmentingNetwork *g f path*
struct::graph::op::createLevelGraph *Gf s*
struct::graph::op::TSPLocalSearching *g C*
struct::graph::op::TSPLocalSearching3Approx *g C*
struct::graph::op::createSquaredGraph *g*
struct::graph::op::createCompleteGraph *g originalEdges*

Description

This package, **struct::graph::op**, is a companion to the **struct::graph** package. It provides a series of common operations and algorithms applicable to directed or undirected graphs.

The operations provided by this package can be applied to *any* graph objects which provide the same API as the objects created by **struct::graph**. Therefore this package is not directly dependent on **struct::graph**.

Operations

struct::graph::op::toAdjacencyMatrix *g*

Returns the adjacency matrix for the graph *g*.

The elements of the outer list are the matrix's rows, the inner elements are each row's column values. The matrix has **n**+1 rows and columns, with the first row and column (index 0) containing the name of the node the row/column is for. All other elements are Boolean values, **1** (true) if there is an arc between the two nodes of the respective row and column, and **0** (false) otherwise.

Note that the matrix is symmetric. It does *not* represent the directionality of the arcs, only their presence (or absence) between nodes. It is also unable to represent parallel arcs in *g*.

struct::graph::op::toAdjacencyList *g* ?*options*...?

Returns an *adjacency list* for the graph *g*. Each list element is a **dict** whose key is a node and whose value is a list of the node's adjacent nodes. If **-weights** is used each of the value list's elements is itself a two-element list of node and weight.

Arguments:

see below ...

Graph object *g*

A graph to convert into an *adjacency list*.

Options:

see below ...

-directed

By default the graph *g* is treated as if it were an *undirected graph*. Use this option to make the command handle *g* as a directed graph.

-weights

By default the graph *g*'s weight information (if any) is ignored. Use this option to make the command include any weight information in the result.

Using this option assumes that every arc in graph *g* has an associated weight; if this is not the case an error will be thrown.

struct::graph::op::kruskal *g*

Returns a list of arc names in graph *g* as determined by Kruskal's algorithm.

For a connected graph, the arcs are those which span a minimum spanning tree (MST). For a disconnected graph the arcs are those of a minimum spanning forest (which itself is composed of an MST for each connected component; excluding single vertex components).

The algorithm's time complexity is *O(E*log E)* or *O(E*log V)*, where *V* is the number of vertices and *E* is the number of edges in graph *g*. Note that Prim's algorithm may produce different results if there is more than one possible MST.

To find the nodes each returned arc connects, use the graph's **arc nodes** method on each arc.

The command will throw an error if any of graph *g*'s arcs has no associated weight.

struct::graph::op::prim *g*

Returns a list of arc names in graph *g* as determined by Prim's algorithm.

For a connected graph, the arcs are those which span a minimum spanning tree (MST). For a disconnected graph the arcs are those of a minimum spanning forest (which itself is composed of an MST for each connected component; excluding single vertex components).

This algorithm has a time complexity between *O(E+V*log V)* and *O(V*V)*, depending on the implementation (Fibonacci heap and adjacency list versus adjacency matrix), where *V* is the number of vertices and *E* is the number of edges in graph *g*. Note that Kruskal's algorithm may produce different results if there is more than one possible MST.

To find the nodes each returned arc connects, use the graph's **arc nodes** method on each arc.

The command will throw an error if any of graph *g*'s arcs has no associated weight.

struct::graph::op::isBipartite? *g* ?*partitions*?

Returns **1** (true) if graph *g* is bipartite; otherwise returns **0** (false).

If the optional *partitions* variable name is given *and* graph *g* is bipartite, then *partitions* is created if it doesn't exist, then it is set to the list of the two partitions. If graph *g* is not bipartite, *partitions* is not created if it doesn't exist; and not changed if it does.

struct::graph::op::tarjan *g*

Returns a list of sets, each of which contains the nodes for one of graph *g*'s *strongly connected* components (SCCs). The union of all SCCs covers the whole graph, and no two SCCs intersect.

If *all* the resulting SCCs contain only one node, then graph *g* is *acyclic*. If the result contains only one SCC containing all graph *g*'s nodes, then the graph is *strongly connected*.

Note: The **struct::set** package must be required before this function is used, since it returns a set of type **struct::set**.

struct::graph::op::connectedComponents *g*

Returns a list of sets, each of which contains the nodes for one of graph *g*'s *connected* components (CCs). The union of all CCs covers the whole graph, and no two CCs intersect.

If the result contains only one CC containing all graph *g*'s nodes, then the graph is *connected*.

Note: The **struct::set** package must be required before this function is used, since it returns a set of type **struct::set**.

struct::graph::op::connectedComponentOf *g node*

Returns a list of graph *g*'s *connected* components (CCs) that contain node *node*.

The command will throw an error if *node* is not a graph *g* node.

struct::graph::op::isConnected? *g*
Returns **1** (true) if graph *g* is *connected*; otherwise returns **0** (false).

struct::graph::op::isCutVertex? *g node*
Returns **1** (true) if graph *g*'s *node* is a *cut vertex* (*also called an articulation point*); otherwise returns **0** (false).

The command will throw an error if *node* is not a graph *g* node.

struct::graph::op::isBridge? *g arc*
Returns **1** (true) if graph *g*'s *arc* is a *bridge* (*also called a cut edge or isthmus*); otherwise returns **0** (false).

The command will throw an error if *arc* is not a graph *g* arc

struct::graph::op::isEulerian? *g* ?*tourvar*?
Returns **1** (true) if graph *g* is *Eulerian*; otherwise returns **0** (false).

If the graph is Eulerian and *tourvar* is specified, then an Euler tour is computed and stored in the named variable. The tour is represented by a list of the arcs traversed, in order of traversal.

struct::graph::op::isSemiEulerian? *g* ?*pathvar*?
Returns **1** (true) if graph *g* is *semi-Eulerian*; otherwise returns **0** (false).

If the graph is semi-Eulerian and *pathvar* is specified, then an Euler path is computed and stored in the named variable. The path is represented by a list of the arcs traversed, in order of traversal.

struct::graph::op::dijkstra *g start_node* ?*options*...?
Returns a **dict** whose keys are nodes and whose values vary depending on the options. By default an undirected traversal and tree format are used, resulting in a **dict** whose values are lists of nodes. The traversal is based on the distances in the weighted graph *g* from the *start_node* to all the graph's other nodes.

Two options are supported:

-arcmode mode
The *mode* may be **undirected** (the default) or **directed**. If **directed**, all the arcs are traversed from the source to the target. For **undirected**, all the arcs are traversed both ways.

-outputformat format
The *format* may be **tree** (the default) or **distances**. In either case the result is a **dict** whose keys are the names of all graph *g*'s nodes. When the *format* is **tree**, each value is the path from the node to the *start_node* (i.e., a list of nodes; excluding the *start_node* itself). When the *format* is **distances**, each value is the distance from the node to the *start_node*. *Note:* When using *format* **distances** it is possible to have distances of **0** or of **Inf**.

struct::graph::op::distance *g start_node end_node* ?-arcmode *mode*?
Returns the distance in graph *g* from the *start_node* to the *end_node* using an undirected traversal (i.e., *-arcmode* **undirected**). Use *-arcmode* **directed** for a directed traversal. (See

struct::graph::op::dijkstra's **-arcmode** option.)

struct::graph::op::eccentricity *g node* ?-arcmode *mode*?

Returns the *eccentricity* in graph *g* of the given *node* using an undirected traversal (i.e., *-arcmode* **undirected**). Use *-arcmode* **directed** for a directed traversal. (See **struct::graph::op::dijkstra**'s **-arcmode** option.)

A node's (un)directed *eccentricity* is the maximal (un)directed distance between the node and any other node in the graph.

struct::graph::op::radius *g* ?-arcmode *mode*?

Returns the *radius* of graph *g* using an undirected traversal (i.e., *-arcmode* **undirected**). Use *-arcmode* **directed** for a directed traversal. (See **struct::graph::op::dijkstra**'s **-arcmode** option.)

A graph's (un)directed *radius* is the minimal (un)directed *eccentricity* of all nodes in the graph.

struct::graph::op::diameter *g* ?-arcmode *mode*?

Returns the *diameter* of graph *g* using an undirected traversal (i.e., *-arcmode* **undirected**). Use *-arcmode* **directed** for a directed traversal. (See **struct::graph::op::dijkstra**'s **-arcmode** option.)

A graph's (un)directed *radius* is the maximal (un)directed *eccentricity* of all nodes in the graph.

struct::graph::op::BellmanFord *g startnode*

Searching for **shortests paths** between chosen node and all other nodes in graph *g*. Based on relaxation method. In comparison to **struct::graph::op::dijkstra** it doesn't need assumption that all weights on edges in input graph *g* have to be positive.

That generality sets the complexity of algorithm to - *O(V*E)*, where *V* is the number of vertices and *E* is number of edges in graph *g*.

Arguments:

see below ...

Graph object *g* (input)

Directed, connected and edge weighted graph *g*, without any negative cycles (presence of cycles with the negative sum of weight means that there is no shortest path, since the total weight becomes lower each time the cycle is traversed). Negative weights on edges are allowed.

Node *startnode* (input)

The node for which we find all shortest paths to each other node in graph *g*.

Result:

Dictionary containing for each node (key) distances to each other node in graph *g*.

Note: If algorithm finds a negative cycle, it will return error message.

struct::graph::op::Johnsons *g* ?*options*...?

Searching for **shortest paths** between all pairs of vertices in graph. For sparse graphs asymptotically quicker than **struct::graph::op::FloydWarshall** algorithm. Johnson's

algorithm uses **struct::graph::op::BellmanFord** and **struct::graph::op::dijkstra** as subprocedures.

Time complexity: *O(n**2*log(n) +n*m)*, where *n* is the number of nodes and *m* is the number of edges in graph *g*.

Arguments:

see below ...

Graph object *g* (input)

Directed graph *g*, weighted on edges and not containing any cycles with negative sum of weights (the presence of such cycles means there is no shortest path, since the total weight becomes lower each time the cycle is traversed). Negative weights on edges are allowed.

Options:

see below ...

-filter

Returns only existing distances, cuts all *Inf* values for non-existing connections between pairs of nodes.

Result:

Dictionary containing distances between all pairs of vertices.

struct::graph::op::FloydWarshall *g*

Searching for **shortest paths** between all pairs of edges in weighted graphs.

Time complexity: *O(V^3)* - where *V* is number of vertices.

Memory complexity: *O(V^2)*.

Arguments:

see below ...

Graph object *g* (input)

Directed and weighted graph *g*.

Result:

Dictionary containing shortest distances to each node from each node.

Note: Algorithm finds solutions dynamically. It compares all possible paths through the graph between each pair of vertices. Graph shouldn't possess any cycle with negative sum of weights (the presence of such cycles means there is no shortest path, since the total weight becomes lower each time the cycle is traversed).

On the other hand algorithm can be used to find those cycles - if any shortest distance found by algorithm for any nodes *v* and *u* (when *v* is the same node as *u*) is negative, that node surely belong to at least one negative cycle.

struct::graph::op::MetricTravellingSalesman *g*

Algorithm for solving a metric variation of Travelling Salesman problem. *TSP problem* is *NP-Complete*, so there is no efficient algorithm to solve it. Greedy methods are getting extremely slow, with the increase in the set of nodes.

Arguments:

see below ...

Graph object *g* (input)

Undirected, weighted graph *g*.

Result:

Approximated solution of minimum *Hamilton Cycle* - closed path visiting all nodes, each exactly one time.

Note: **It's 2-approximation algorithm.**

struct::graph::op::Christofides *g*

Another algorithm for solving **metric** *TSP problem*. Christofides implementation uses *Max Matching* for reaching better approximation factor.

Arguments:

see below ...

Graph Object *g* (input)

Undirected, weighted graph *g*.

Result:

Approximated solution of minimum *Hamilton Cycle* - closed path visiting all nodes, each exactly one time.

Note: **It's is a 3/2 approximation algorithm.**

struct::graph::op::GreedyMaxMatching *g*

Greedy Max Matching procedure, which finds **maximal matching** (not maximum) for given graph *g*. It adds edges to solution, beginning from edges with the lowest cost.

Arguments:

see below ...

Graph Object *g* (input)

Undirected graph *g*.

Result:

Set of edges - the max matching for graph *g*.

struct::graph::op::MaxCut *g U V*

Algorithm solving a **Maximum Cut Problem**.

Arguments:

see below ...

Graph Object *g* (input)

The graph to cut.

List *U* (output)

Variable storing first set of nodes (cut) given by solution.

List *V* (output)

Variable storing second set of nodes (cut) given by solution.

Result:

Algorithm returns number of edges between found two sets of nodes.

Note: MaxCut is a **2-approximation algorithm.**

struct::graph::op::UnweightedKCenter *g k*

Approximation algorithm that solves a K-center problem.

Arguments:

see below ...

Graph Object *g* (input)

Undirected complete graph *g*, which satisfies triangle inequality.

Integer *k* (input)

Positive integer that sets the number of nodes that will be included in *k-center*.

Result:

Set of nodes - *k* center for graph *g*.

Note: UnweightedKCenter is a **2-approximation algorithm.**

struct::graph::op::WeightedKCenter *g nodeWeights W*

Approximation algorithm that solves a weighted version of K-center problem.

Arguments:

see below ...

Graph Object *g* (input)

Undirected complete graph *g*, which satisfies triangle inequality.

Integer *W* (input)

Positive integer that sets the maximum possible weight of *k-center* found by algorithm.

List *nodeWeights* (input)

List of nodes and its weights in graph *g*.

Result:

Set of nodes, which is solution found by algorithm.

Note:WeightedKCenter is a **3-approximation algorithm.**

struct::graph::op::GreedyMaxIndependentSet *g*

A *maximal independent set* is an *independent set* such that adding any other node to the set forces the set to contain an edge.

Algorithm for input graph *g* returns set of nodes (list), which are contained in Max Independent Set found by algorithm.

struct::graph::op::GreedyWeightedMaxIndependentSet *g nodeWeights*

Weighted variation of *Maximal Independent Set*. It takes as an input argument not only graph *g* but also set of weights for all vertices in graph *g*.

Note: Read also *Maximal Independent Set* description for more info.

struct::graph::op::VerticesCover *g*

Vertices cover is a set of vertices such that each edge of the graph is incident to at least one vertex of the set. This 2-approximation algorithm searches for minimum *vertices cover*, which is a classical optimization problem in computer science and is a typical example of an *NP-hard* optimization problem that has an approximation algorithm. For input graph *g* algorithm returns the set of edges (list), which is Vertex Cover found by algorithm.

struct::graph::op::EdmondsKarp *g s t*

Improved Ford-Fulkerson's algorithm, computing the **maximum flow** in given flow network *g*.

Arguments:

see below ...

Graph Object *g* (input)

Weighted and directed graph. Each edge should have set integer attribute considered as maximum throughputs that can be carried by that link (edge).

Node *s* (input)

The node that is a source for graph *g*.

Node *t* (input)

The node that is a sink for graph *g*.

Result:

Procedure returns the dictionary containing throughputs for all edges. For each key (the edge between nodes *u* and *v* in the form of *list u v*) there is a value that is a throughput for that key. Edges where throughput values are equal to 0 are not returned (it is like there was no link in the flow network between nodes connected by such edge).

The general idea of algorithm is finding the shortest augumenting paths in graph *g*, as long as they exist, and for each path updating the edge's weights along that path, with maximum possible throughput. The final (maximum) flow is found when there is no other augumenting path from source to sink.

Note: Algorithm complexity: *O(V*E)*, where *V* is the number of nodes and *E* is the number of edges in graph *g*.

struct::graph::op::BusackerGowen *g desiredFlow s t*

Algorithm finds solution for a **minimum cost flow problem**. So, the goal is to find a flow, whose max value can be *desiredFlow*, from source node *s* to sink node *t* in given flow network *g*. That network except throughputs at edges has also defined a non-negative cost on each edge - cost of using that edge when directing flow with that edge (it can illustrate e.g. fuel usage, time or any other measure dependent on usages).

Arguments:

see below ...

Graph Object *g* (input)

Flow network (directed graph), each edge in graph should have two integer attributes: *cost* and *throughput*.

Integer *desiredFlow* (input)

Max value of the flow for that network.

Node *s* (input)

The source node for graph *g*.

Node *t* (input)

The sink node for graph *g*.

Result:

Dictionary containing values of used throughputs for each edge (key). found by algorithm.

Note: Algorithm complexity: *O(V**2*desiredFlow)*, where *V* is the number of nodes in graph *g*.

struct::graph::op::ShortestsPathsByBFS *g s outputFormat*

Shortest pathfinding algorithm using BFS method. In comparison to **struct::graph::op::dijkstra** it can work with negative weights on edges. Of course negative cycles are not allowed. Algorithm is better than dijkstra for sparse graphs, but also there exist some pathological cases (those cases generally don't appear in practise) that make time complexity increase exponentially with the growth of the number of nodes.

Arguments:

see below ...

Graph Object *g* (input)

Input graph.

Node *s* (input)

Source node for which all distances to each other node in graph *g* are computed.

Options and result:

see below ...

distances

When selected *outputFormat* is **distances** - procedure returns dictionary containing distances between source node *s* and each other node in graph *g*.

paths

When selected *outputFormat* is **paths** - procedure returns dictionary containing for each node *v*, a list of nodes, which is a path between source node *s* and node *v*.

struct::graph::op::BFS *g s ?outputFormat...?*

Breadth-First Search - algorithm creates the BFS Tree. Memory and time complexity: *O(V + E)*, where *V* is the number of nodes and *E* is number of edges.

Arguments:

see below ...

Graph Object *g* (input)

Input graph.

Node *s* (input)

Source node for BFS procedure.

Options and result:

see below ...

graph

When selected **outputFormat** is **graph** - procedure returns a graph structure (**struct::graph**), which is equivalent to BFS tree found by algorithm.

tree

When selected **outputFormat** is **tree** - procedure returns a tree structure (**struct::tree**), which is equivalent to BFS tree found by algorithm.

struct::graph::op::MinimumDiameterSpanningTree *g*

The goal is to find for input graph *g*, the *spanning tree* that has the minimum *diameter* value.

General idea of algorithm is to run *BFS* over all vertices in graph *g*. If the diameter *d* of the tree is odd, then we are sure that tree given by *BFS* is minimum (considering diameter value).

When, diameter *d* is even, then optimal tree can have minimum *diameter* equal to *d* or *d-1*.

In that case, what algorithm does is rebuilding the tree given by *BFS*, by adding a vertice between root node and root's child node (nodes), such that subtree created with child node as root node is the greatest one (has the greatests height). In the next step for such rebuilded tree, we run again *BFS* with new node as root node. If the height of the tree didn't changed, we have found a better solution.

For input graph *g* algorithm returns the graph structure (**struct::graph**) that is a spanning tree with minimum diameter found by algorithm.

struct::graph::op::MinimumDegreeSpanningTree *g*

Algorithm finds for input graph *g*, a spanning tree *T* with the minimum possible degree. That problem is *NP-hard*, so algorithm is an approximation algorithm.

Let *V* be the set of nodes for graph *g* and let *W* be any subset of *V*. Lets assume also that *OPT* is optimal solution and *ALG* is solution found by algorithm for input graph *g*.

It can be proven that solution found with the algorithm must fulfil inequality:

*((|W| + k - 1) / |W|) <= ALG <= 2*OPT + log2(n) + 1*.

Arguments:

see below ...

Graph Object *g* (input)

Undirected simple graph.

Result:

Algorithm returns graph structure, which is equivalent to spanning tree *T* found by algorithm.

struct::graph::op::MaximumFlowByDinic *g s t blockingFlowAlg*

Algorithm finds **maximum flow** for the flow network represented by graph *g*. It is based on the blocking-flow finding methods, which give us different complexities what makes a better fit for different graphs.

Arguments:

see below ...

Graph Object *g* (input)

Directed graph *g* representing the flow network. Each edge should have attribute *throughput* set with integer value.

Node *s* (input)

The source node for the flow network *g*.

Node *t* (input)

The sink node for the flow network *g*.

Options:

see below ...

dinic

Procedure will find maximum flow for flow network *g* using Dinic's algorithm (**struct::graph::op::BlockingFlowByDinic**) for blocking flow computation.

mkm
Procedure will find maximum flow for flow network *g* using Malhotra, Kumar and Maheshwari's algorithm (**struct::graph::op::BlockingFlowByMKM**) for blocking flow computation.

Result:
Algorithm returns dictionary containing it's flow value for each edge (key) in network *g*.

Note: **struct::graph::op::BlockingFlowByDinic** gives *O(m*n^2)* complexity and **struct::graph::op::BlockingFlowByMKM** gives *O(n^3)* complexity, where *n* is the number of nodes and *m* is the number of edges in flow network *g*.

struct::graph::op::BlockingFlowByDinic *g s t*
Algorithm for given network *g* with source *s* and sink *t*, finds a **blocking flow**, which can be used to obtain a *maximum flow* for that network *g*.

Arguments:
see below ...

Graph Object *g* (input)
Directed graph *g* representing the flow network. Each edge should have attribute *throughput* set with integer value.

Node *s* (input)
The source node for the flow network *g*.

Node *t* (input)
The sink node for the flow network *g*.

Result:
Algorithm returns dictionary containing it's blocking flow value for each edge (key) in network *g*.

Note: Algorithm's complexity is *O(n*m)*, where *n* is the number of nodes and *m* is the number of edges in flow network *g*.

struct::graph::op::BlockingFlowByMKM *g s t*
Algorithm for given network *g* with source *s* and sink *t*, finds a **blocking flow**, which can be used to obtain a *maximum flow* for that *network g*.

Arguments:
see below ...

Graph Object *g* (input)
Directed graph *g* representing the flow network. Each edge should have attribute *throughput* set with integer value.

Node *s* (input)
The source node for the flow network *g*.

Node *t* (input)
The sink node for the flow network *g*.

Result:
Algorithm returns dictionary containing it's blocking flow value for each edge (key) in network *g*.

Note: Algorithm's complexity is *O(n^2)*, where *n* is the number of nodes in flow network *g*.

struct::graph::op::createResidualGraph *g f*

Procedure creates a *residual graph* (or **residual network**) for network *g* and given flow *f*.

Arguments:

see below ...

Graph Object *g* (input)

Flow network (directed graph where each edge has set attribute: *throughput*).

dictionary *f* (input)

Current flows in flow network *g*.

Result:

Procedure returns graph structure that is a *residual graph* created from input flow network *g*.

struct::graph::op::createAugmentingNetwork *g f path*

Procedure creates an **augmenting network** for a given residual network *g*, flow *f* and augmenting path *path*.

Arguments:

see below ...

Graph Object *g* (input)

Residual network (directed graph), where for every edge there are set two attributes: throughput and cost.

Dictionary *f* (input)

Dictionary which contains for every edge (key), current value of the flow on that edge.

List *path* (input)

Augmenting path, set of edges (list) for which we create the network modification.

Result:

Algorithm returns graph structure containing the modified augmenting network.

struct::graph::op::createLevelGraph *Gf s*

For given residual graph *Gf* procedure finds the **level graph**.

Arguments:

see below ...

Graph Object *Gf* (input)

Residual network, where each edge has it's attribute *throughput* set with certain value.

Node *s* (input)

The source node for the residual network *Gf*.

Result:

Procedure returns a *level graph* created from input *residual network*.

struct::graph::op::TSPLocalSearching *g C*

Algorithm is a *heuristic of local searching* for *Travelling Salesman Problem*. For some solution of *TSP problem*, it checks if it's possible to find a better solution. As *TSP* is well known NP-Complete problem, so algorithm is a approximation algorithm (with 2 approximation factor).

Arguments:

see below ...

Graph Object *g* (input)

Undirected and complete graph with attributes "weight" set on each single edge.

List *C* (input)

A list of edges being *Hamiltonian cycle*, which is solution of *TSP Problem* for graph *g*.

Result:

Algorithm returns the best solution for *TSP problem*, it was able to find.

Note: The solution depends on the choosing of the beginning cycle *C*. It's not true that better cycle assures that better solution will be found, but practise shows that we should give starting cycle with as small sum of weights as possible.

struct::graph::op::TSPLocalSearching3Approx *g C*

Algorithm is a *heuristic of local searching* for *Travelling Salesman Problem*. For some solution of *TSP problem*, it checks if it's possible to find a better solution. As *TSP* is well known NP-Complete problem, so algorithm is a approximation algorithm (with 3 approximation factor).

Arguments:

see below ...

Graph Object *g* (input)

Undirected and complete graph with attributes "weight" set on each single edge.

List *C* (input)

A list of edges being *Hamiltonian cycle*, which is solution of *TSP Problem* for graph *g*.

Result:

Algorithm returns the best solution for *TSP problem*, it was able to find.

Note: In practise 3-approximation algorithm turns out to be far more effective than 2-approximation, but it gives worser approximation factor. Further heuristics of local searching (e.g. 4-approximation) doesn't give enough boost to square the increase of approximation factor, so 2 and 3 approximations are mainly used.

struct::graph::op::createSquaredGraph *g*

X-Squared graph is a graph with the same set of nodes as input graph *g*, but a different set of edges. X-Squared graph has edge *(u,v)*, if and only if, the distance between *u* and *v* nodes is not greater than X and *u!= v*.

Procedure for input graph *g*, returns its two-squared graph.

Note: Distances used in choosing new set of edges are considering the number of edges, not the sum of weights at edges.

struct::graph::op::createCompleteGraph *g originalEdges*

For input graph *g* procedure adds missing arcs to make it a *complete graph*. It also holds in variable *originalEdges* the set of arcs that graph *g* possessed before that operation.

Background theory and terms

Shortest path problem

Definition (*single-pair shortest path problem*):

Formally, given a weighted graph (let *V* be the set of vertices, and *E* a set of edges), and one vertice *v* of *V*, find a path *P* from *v* to a *v*' of V so that the sum of weights on edges along the path is minimal among all paths connecting v to v'.

Generalizations:

- *The single-source shortest path problem*, in which we have to find shortest paths from a source vertex v to all other vertices in the graph.
- *The single-destination shortest path problem*, in which we have to find shortest paths from all vertices in the graph to a single destination vertex v. This can be reduced to the single-source shortest path problem by reversing the edges in the graph.
- *The all-pairs shortest path problem*, in which we have to find shortest paths between every pair of vertices v, v' in the graph.

Note: The result of *Shortest Path problem* can be *Shortest Path tree*, which is a subgraph of a given (possibly weighted) graph constructed so that the distance between a selected root node and all other nodes is minimal. It is a tree because if there are two paths between the root node and some vertex v (i.e. a cycle), we can delete the last edge of the longer path without increasing the distance from the root node to any node in the subgraph.

Travelling Salesman problem

Definition:

For given edge-weighted (weights on edges should be positive) graph the goal is to find the cycle that visits each node in graph exactly once (*Hamiltonian cycle*).

Generalizations:

- *Metric TSP* - A very natural restriction of the *TSP* is to require that the distances between cities form a *metric*, i.e., they satisfy *the triangle inequality*. That is, for any 3 cities *A*, *B* and *C* , the distance between *A* and *C* must be at most the distance from *A* to *B* plus the distance from *B* to *C*. Most natural instances of *TSP* satisfy this constraint.
- *Euclidean TSP* - Euclidean TSP, or *planar TSP*, is the *TSP* with the distance being the ordinary *Euclidean distance*. *Euclidean TSP* is a particular case of *TSP* with *triangle inequality*, since distances in plane obey triangle inequality. However, it seems to be easier than general *TSP* with *triangle inequality*. For example, *the minimum spanning tree* of the graph associated with an instance of *Euclidean TSP* is a *Euclidean minimum spanning tree*, and so can be computed in expected *O(n log n)* time for *n* points (considerably less than the number of edges). This enables the simple *2-approximation algorithm* for TSP with triangle inequality above to operate more quickly.
- *Asymmetric TSP* - In most cases, the distance between two nodes in the *TSP* network is the same in both directions. The case where the distance from *A* to *B* is not equal to the distance from *B* to *A* is called *asymmetric TSP*. A practical application of an *asymmetric TSP* is route optimisation using street-level routing (asymmetric due to one-way streets, slip-roads and motorways).

Matching problem

Definition:

Given a graph *G = (V,E)*, a matching or *edge-independent set M* in *G* is a set of pairwise

non-adjacent edges, that is, no two edges share a common vertex. A vertex is *matched* if it is incident to an edge in the *matching M*. Otherwise the vertex is *unmatched*.

Generalizations:

- *Maximal matching* - a matching *M* of a graph G with the property that if any edge not in *M* is added to *M*, it is no longer a *matching*, that is, *M* is maximal if it is not a proper subset of any other *matching* in graph G. In other words, a *matching M* of a graph G is maximal if every edge in G has a non-empty intersection with at least one edge in *M*.
- *Maximum matching* - a matching that contains the largest possible number of edges. There may be many *maximum matchings*. The *matching number* of a graph G is the size of a *maximum matching*. Note that every *maximum matching* is *maximal*, but not every *maximal matching* is a *maximum matching*.
- *Perfect matching* - a matching which matches all vertices of the graph. That is, every vertex of the graph is incident to exactly one edge of the matching. Every *perfect matching* is *maximum* and hence *maximal*. In some literature, the term *complete matching* is used. A *perfect matching* is also a *minimum-size edge cover*. Moreover, the size of a *maximum matching* is no larger than the size of a *minimum edge cover*.
- *Near-perfect matching* - a matching in which exactly one vertex is unmatched. This can only occur when the graph has an odd number of vertices, and such a *matching* must be *maximum*. If, for every vertex in a graph, there is a near-perfect matching that omits only that vertex, the graph is also called *factor-critical*.

Related terms:

- *Alternating path* - given a matching *M*, an *alternating path* is a path in which the edges belong alternatively to the matching and not to the matching.
- *Augmenting path* - given a matching *M*, an *augmenting path* is an *alternating path* that starts from and ends on free (unmatched) vertices.

Cut problems

Definition:

A *cut* is a partition of the vertices of a graph into two *disjoint subsets*. The *cut-set* of the *cut* is the set of edges whose end points are in different subsets of the partition. Edges are said to be crossing the cut if they are in its *cut-set*.

Formally:

- a *cut C = (S,T)* is a partition of *V* of a graph *G = (V, E)*.
- an *s-t cut C = (S,T)* of a *flow network N = (V, E)* is a cut of *N* such that *s* is included in *S* and *t* is included in *T*, where *s* and *t* are the *source* and the *sink* of *N* respectively.
- The *cut-set* of a *cut C = (S,T)* is such set of edges from graph *G = (V, E)* that each edge *(u, v)* satisfies condition that *u* is included in *S* and *v* is included in *T*.

In an *unweighted undirected* graph, the size or weight of a cut is the number of edges crossing the cut. In a *weighted graph*, the same term is defined by the sum of the weights of the edges crossing the cut.

In a *flow network*, an *s-t cut* is a cut that requires the *source* and the *sink* to be in different subsets, and its *cut-set* only consists of edges going from the *source*'*s* side to the *sink*'*s* side. The capacity of an *s-t cut* is defined by the sum of capacity of each edge in the *cut-set*.

The *cut* of a graph can sometimes refer to its *cut-set* instead of the partition.

Generalizations:

- *Minimum cut* - A cut is minimum if the size of the cut is not larger than the size of any other cut.
- *Maximum cut* - A cut is maximum if the size of the cut is not smaller than the size of any other cut.
- *Sparsest cut* - The *Sparsest cut problem* is to bipartition the vertices so as to minimize the ratio of the number of edges across the cut divided by the number of vertices in the smaller half of the partition.

K-center problem

Definitions:

see below ...

Unweighted K-Center

For any set *S* (which is subset of *V*) and node *v*, let the *connect(v,S)* be the cost of cheapest edge connecting *v* with any node in *S*. The goal is to find such *S*, that |*S*| = *k* and *max_v{connect(v,S)}* is possibly small.

In other words, we can use it i.e. for finding best locations in the city (nodes of input graph) for placing k buildings, such that those buildings will be as close as possible to all other locations in town.

Weighted K-Center

The variation of *unweighted k-center problem*. Besides the fact graph is edge-weighted, there are also weights on vertices of input graph *g*. We've got also restriction *W*. The goal is to choose such set of nodes *S* (which is a subset of *V*), that it's total weight is not greater than *W* and also function: *max_v { min_u { cost(u,v) }}* has the smallest possible worth (*v* is a node in *V* and *u* is a node in *S*).

Flow problems

Definitions:

- *the maximum flow problem* - the goal is to find a feasible flow through a single-source, single-sink flow network that is maximum. The *maximum flow problem* can be seen as a special case of more complex network flow problems, such as the *circulation problem*. The maximum value of an *s-t flow* is equal to the minimum capacity of an *s-t cut* in the network, as stated in the *max-flow min-cut theorem*.

```
More formally for flow network *G = (V,E)*, where for each edge *(u,
    v)* we have its throuhgput *c(u,v)* defined. As *flow* *F* we
    define set of non-negative integer attributes *f(u,v)* assigned
    to edges, satisfying such conditions:

1. for each edge *(u, v)* in *G* such condition should be satisfied:
         0 <= f(u,v) <= c(u,v)

2. Network *G* has source node *s* such that the flow *F* is equal
    to the sum of outcoming flow decreased by the sum of incoming
```

```
    flow from that source node *s*.

3. Network *G* has sink node *t* such that the the *-F* value is
    equal to the sum of the incoming flow decreased by the sum of
    outcoming flow from that sink node *t*.

4. For each node that is not a *source* or *sink* the sum of
    incoming flow and sum of outcoming flow should be equal.
```

- *the minimum cost flow problem* - the goal is finding the cheapest possible way of sending a certain amount of flow through a *flow network*.
- *blocking flow* - a *blocking flow* for a *residual network Gf* we name such flow *b* in *Gf* that:
  1. Each path from *sink* to *source* is the shortest path in *Gf*.
  2. Each shortest path in *Gf* contains an edge with fully used throughput in *Gf+b*.
- *residual network* - for a flow network *G* and flow *f residual network* is built with those edges, which can send larger flow. It contains only those edges, which can send flow larger than 0.
- *level network* - it has the same set of nodes as *residual graph*, but has only those edges *(u,v)* from *Gf* for which such equality is satisfied: *distance(s,u)+1 = distance(s,v)*.
- *augmenting network* - it is a modification of *residual network* considering the new flow values. Structure stays unchanged but values of throughputs and costs at edges are different.

Approximation algorithm

k-approximation algorithm:

Algorithm is a k-approximation, when for *ALG* (solution returned by algorithm) and *OPT* (optimal solution), such inequality is true:
- for minimalization problems: *ALG/OPT <= k*
- for maximalization problems: *OPT/ALG <= k*

References

1. Adjacency matrix
2. Adjacency list
3. Kruskal's algorithm
4. Prim's algorithm
5. Bipartite graph
6. Strongly connected components
7. Tarjan's strongly connected components algorithm
8. Cut vertex
9. Bridge
10. Bellman-Ford's algorithm
11. Johnson's algorithm
12. Floyd-Warshall's algorithm
13. Travelling Salesman Problem
14. Christofides Algorithm
15. Max Cut

16. Matching
17. Max Independent Set
18. Vertex Cover
19. Ford-Fulkerson's algorithm
20. Maximum Flow problem
21. Busacker-Gowen's algorithm
22. Dinic's algorithm
23. K-Center problem
24. BFS
25. Minimum Degree Spanning Tree
26. Approximation algorithm

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *struct :: graph* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Data structures

Copyright


```
Copyright (c) 2008 Alejandro Paz <vidriloco@gmail.com>
Copyright (c) 2008 (docs) Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
Copyright (c) 2009 Michal Antoniewski <antoniewski.m@gmail.com>
```


matrix

Keywords: matrix

Name

struct::matrix - Create and manipulate matrix objects

Synopsis

package require Tcl 8.5 9
package require struct::matrix ?**2.2**?
::struct::matrix ?*aMatrix*? ?=|:=|*as*|*deserialize source*?
aMatrix *option* ?*arg arg ...*?
aMatrix **=** *sourceMatrix*
aMatrix **-->** *destMatrix*
aMatrix **add column** ?*values*?
aMatrix **add columns** *n*
aMatrix **add row** ?*values*?

aMatrix **add rows** *n*
aMatrix **cells**
aMatrix **cellsize** *column row*
aMatrix **columns**
aMatrix **columnwidth** *column*
aMatrix **delete column** *column*
aMatrix **delete columns** *n*
aMatrix **delete row** *row*
aMatrix **delete rows** *n*
aMatrix **deserialize** *serialization*
aMatrix **destroy**
aMatrix **format 2string** ?*report*?
aMatrix **format 2chan** ??*report*? *channel?*
aMatrix **get cell** *column row*
aMatrix **get column** *column*
aMatrix **get rect** *column_tl row_tl column_br row_br*
aMatrix **get row** *row*
aMatrix **insert column** *column* ?*values*?
aMatrix **insert row** *row* ?*values*?
aMatrix **link** ?*-transpose*? *arrayvar*
aMatrix **links**
aMatrix **rowheight** *row*
aMatrix **rows**
aMatrix **search** ?*-nocase*? ?*-exact*|*-glob*|*-regexp*? **all** *pattern*
aMatrix **search** ?*-nocase*? ?*-exact*|*-glob*|*-regexp*? **column** *column pattern*
aMatrix **search** ?*-nocase*? ?*-exact*|*-glob*|*-regexp*? **row** *row pattern*
aMatrix **search** ?*-nocase*? ?*-exact*|*-glob*|*-regexp*? **rect** *column_tl row_tl column_br row_br pattern*
aMatrix **serialize** ?*column_tl row_tl column_br row_br*?
aMatrix **set cell** *column row value*
aMatrix **set column** *column values*
aMatrix **set rect** *column row values*
aMatrix **set row** *row values*
aMatrix **sort columns** ?**-increasing**|**-decreasing**? *row*
aMatrix **sort rows** ?**-increasing**|**-decreasing**? *column*
aMatrix **swap columns** *column_a column_b*
aMatrix **swap rows** *row_a row_b*
aMatrix **transpose**
aMatrix **unlink** *arrayvar*

Description

This package provides a command for creating matrices, and subcommands for manipulating matrices once created.

A matrix is a rectangular collection of cells arranged in rows and columns. Each cell contains a single value of any kind. Every cell may be addressed using a pair of **0**-based integers {column, row}.

The special non-numeric index **end** may be used to address a matrix's last row or last column, depending on the context. Indices of the form **end**-**number** are counted from the end of the row or column, just the same as for standard Tcl lists.

Using an out-of-range index will raise an error.

By default each matrix is created empty. New rows and columns may be added as required.

A matrix can export an array-view onto its contents. This can be used by, for example, *tkTable*, to link the matrix into the display.

This package's main command is:

::struct::matrix ?*aMatrix*? ?=|:=|as|deserialize *source*?

This command creates a new matrix object with an associated global Tcl command whose name is *aMatrix*. This command may be used to invoke various operations on the matrix. It has the following general form:

aMatrix *option* ?*arg arg ...*?

The *option* and the *arg*s determine what the command does.

If *aMatrix* is not specified a unique name will be generated by the package itself. If a *source* is specified the new matrix will be initialized to it. For the operators **=**,**:=**, and **as** the argument *source* is interpreted as the name of another matrix object, and the assignment operator **=** will be executed. For **deserialize** the *source* is a serialized matrix object and **deserialize** will be executed.

In other words

```
    ::struct::matrix mymatrix = b
```

is equivalent to

```
    ::struct::matrix mymatrix
    mymatrix = b
```

and

```
    ::struct::matrix mymatrix deserialize $b
```

is equivalent to

```
    ::struct::matrix mymatrix
    mymatrix deserialize $b
```

Matrix objects provide the following commands:

aMatrix = sourceMatrix

The **=** operator is the matrix object *assignment* operator. (This operator may be written as **=** or**:=** or **as**.) It copies the matrix contained in the matrix object *sourceMatrix* over the matrix data in *aMatrix*. The old contents of *aMatrix* are deleted by this operation.

This operation is in effect equivalent to

```
    aMatrix deserialize [sourceMatrix serialize]
```

aMatrix* --> *destMatrix
The **-->** operator is the matrix object *reverse assignment* operator. It copies the matrix contained in the matrix object *aMatrix* over the matrix data in the object *destMatrix*. The old contents of *destMatrix* are deleted by this operation.

This operation is in effect equivalent to

```
    destMatrix deserialize [aMatrix serialize]
```

***aMatrix* add column ?*values*?**
Extends the matrix by one column after the last column.

If no *values* is given the new cells are set to the empty string. If *values are* given, then this method in effect applies **set column** (see below) on the newly added column.

aMatrix* add columns *n
Extends the matrix by *n* columns after the last column and sets all the new cells to the empty string.

If *n <= 0* an error is raised.

***aMatrix* add row ?*values*?**
Extends the matrix by one row after the last row.

If no *values* is given the new cells are set to the empty string. If *values are* given, then this method in effect applies **set row** (see below) on the newly added row.

aMatrix* add rows *n
Extends the matrix by *n* rows after the last row and sets all the new cells to the empty string.

If *n <= 0* an error is raised.

***aMatrix* cells**
Returns the number of cells currently managed by the matrix. This is the product of **rows** and **columns**.

aMatrix* cellsize *column row
Returns the length of the string representation of the value currently contained in the addressed cell.

***aMatrix* columns**
Returns the number of columns currently managed by the matrix.

aMatrix* columnwidth *column
Returns the length of the longest string representation of all the values currently contained in the cells of the addressed column if these are all spanning only one line. For cell values spanning multiple lines the length of their longest line goes into the computation.

Note: The command recognizes ANSI color control sequences and excludes them from the width of a line, as they are logically of zero width.

Note: The command properly recognizes and accounts for Asian double-wide characters.

In other words, the returned length is the number of terminal columns taken by the string,

and not the number of characters.

aMatrix* delete column *column
Deletes the specified column from the matrix and shifts all columns with higher indices one index down.

aMatrix* delete columns *n
Deletes *n* columns from the right of the matrix. The value of *n* has to satisfy the constraint "0 < *n* < [**aMatrix columns**]"

aMatrix* delete row *row
Deletes the specified row from the matrix and shifts all row with higher indices one index down.

aMatrix* delete rows *n
Deletes *n* rows from the bottom of the matrix. The value of *n* has to satisfy the constraint "0 < *n* < [**aMatrix rows**]"

aMatrix* deserialize *serialization
This is the complement to **serialize**. It replaces matrix data in *aMatrix* with the matrix described by the *serialization* value. The old contents of *aMatrix* are deleted by this operation.

***aMatrix* destroy**
Destroys the matrix, including its storage space and associated command.

***aMatrix* format 2string ?*report*?**
Formats the matrix using the specified report object and returns the string containing the result of this operation. The report has to support the **printmatrix** method. If no *report* is specified the system will use an internal report definition to format the matrix.

***aMatrix* format 2chan ??*report*? *channel*?**
Formats the matrix using the specified report object and writes the string containing the result of this operation into the channel. The report has to support the **printmatrix2channel** method. If no *report* is specified the system will use an internal report definition to format the matrix. If no *channel* is specified the system will use **stdout**.

aMatrix* get cell *column row
Returns the value currently contained in the cell identified by row and column index.

aMatrix* get column *column
Returns a list containing the values from all cells in the column identified by the index. The contents of the cell in row 0 are stored as the first element of this list.

aMatrix* get rect *column_tl row_tl column_br row_br
Returns a list of lists of cell values. The values stored in the result come from the sub-matrix whose top-left and bottom-right cells are specified by *column_tl, row_tl* and *column_br, row_br* resp. Note that the following equations have to be true: "*column_tl* <= *column_br*" and "*row_tl* <= *row_br*". The result is organized as follows: The outer list is the list of rows, its elements are lists representing a single row. The row with the smallest index is the first element of the outer list. The elements of the row lists represent the selected cell values. The cell with the smallest index is the first element in each row list.

aMatrix* get row *row
Returns a list containing the values from all cells in the row identified by the index. The contents of the cell in column 0 are stored as the first element of this list.

***aMatrix* insert column *column* ?*values*?**

Extends the matrix by one column and then acts like **set column** (see below) on this new column if there were *values* supplied. Without *values* the new cells will be set to the empty string. The new column is inserted just before the column specified by the given index. This means, if *column* is less than or equal to zero, then the new column is inserted at the beginning of the matrix, before the first column. If *column* has the value **end**, or if it is greater than or equal to the number of columns in the matrix, then the new column is appended to the matrix, behind the last column. The old column at the chosen index and all columns with higher indices are shifted one index upward.

***aMatrix* insert row *row* ?*values*?**

Extends the matrix by one row and then acts like **set row** (see below) on this new row if there were *values* supplied. Without *values* the new cells will be set to the empty string. The new row is inserted just before the row specified by the given index. This means, if *row* is less than or equal to zero, then the new row is inserted at the beginning of the matrix, before the first row. If *row* has the value **end**, or if it is greater than or equal to the number of rows in the matrix, then the new row is appended to the matrix, behind the last row. The old row at that index and all rows with higher indices are shifted one index upward.

aMatrix* link ?-transpose? *arrayvar

Links the matrix to the specified array variable. This means that the contents of all cells in the matrix is stored in the array too, with all changes to the matrix propagated there too. The contents of the cell *(column,row)* is stored in the array using the key *column,row*. If the option **-transpose** is specified the key *row,column* will be used instead. It is possible to link the matrix to more than one array. Note that the link is bidirectional, i.e. changes to the array are mirrored in the matrix too.

***aMatrix* links**

Returns a list containing the names of all array variables the matrix was linked to through a call to method **link**.

aMatrix* rowheight *row

Returns the height of the specified row in lines. This is the highest number of lines spanned by a cell over all cells in the row.

***aMatrix* rows**

Returns the number of rows currently managed by the matrix.

aMatrix* search ?-nocase? ?-exact|-glob|-regexp? all *pattern

Searches the whole matrix for cells matching the *pattern* and returns a list with all matches. Each item in the aforementioned list is a list itself and contains the column and row index of the matching cell, in this order. The results are ordered by column first and row second, both times in ascending order. This means that matches to the left and the top of the matrix come before matches to the right and down.

The type of the pattern (string, glob, regular expression) is determined by the option after the **search** keyword. If no option is given it defaults to **-exact**.

If the option **-nocase** is specified the search will be case-insensitive.

aMatrix* search ?-nocase? ?-exact|-glob|-regexp? column *column pattern

Like **search all**, but the search is restricted to the specified column.

aMatrix* search ?-nocase? ?-exact|-glob|-regexp? row *row pattern

Like **search all**, but the search is restricted to the specified row.

aMatrix* search ?-nocase? ?-exact|-glob|-regexp? rect *column_tl row_tl column_br row_br pattern

Like **search all**, but the search is restricted to the specified rectangular area of the matrix.

***aMatrix* serialize ?*column_tl row_tl column_br row_br*?**

This method serializes the sub-matrix spanned up by the rectangle specification. In other words it returns a tcl *value* completely describing that matrix. If no rectangle is specified the whole matrix will be serialized. This allows, for example, the transfer of matrix objects (or parts thereof) over arbitrary channels, persistence, etc. This method is also the basis for both the copy constructor and the assignment operator.

The result of this method has to be semantically identical over all implementations of the matrix interface. This is what will enable us to copy matrix data between different implementations of the same interface.

The result is a list containing exactly three items.

The first two elements of the list specify the number of rows and columns of the matrix, in that order. The values integer numbers greater than or equal to zero.

The last element of the list contains the values of the matrix cells we have serialized, in the form of a value like it is returned by the **get rect**. However empty cells to the right and bottom of the matrix can be left out of that value as the size information in the serialization allows the receiver the creation of a matrix with the proper size despite the missing values.

```
# A possible serialization for the matrix structure
#
# | a b d g |
# | c e     |
# | f       |
#
# is
#
# 3 4 {{a b d g} {c e} {f}}
```

aMatrix* set cell *column row value

Sets the value in the cell identified by row and column index to the data in the third argument.

aMatrix* set column *column values

Sets the values in the cells identified by the column index to the elements of the list provided as the third argument. Each element of the list is assigned to one cell, with the first element going into the cell in row 0 and then upward. If there are less values in the list than there are rows the remaining rows are set to the empty string. If there are more values in the list than there are rows the superfluous elements are ignored. The matrix is not extended by this operation.

aMatrix* set rect *column row values

Takes a list of lists of cell values and writes them into the submatrix whose top-left cell is specified by the two indices. If the sublists of the outerlist are not of equal length the shorter sublists will be filled with empty strings to the length of the longest sublist. If the submatrix

specified by the top-left cell and the number of rows and columns in the *values* extends beyond the matrix we are modifying the over-extending parts of the values are ignored, i.e. essentially cut off. This subcommand expects its input in the format as returned by **get rect**.

aMatrix* set row *row values
Sets the values in the cells identified by the row index to the elements of the list provided as the third argument. Each element of the list is assigned to one cell, with the first element going into the cell in column 0 and then upward. If there are less values in the list than there are columns the remaining columns are set to the empty string. If there are more values in the list than there are columns the superfluous elements are ignored. The matrix is not extended by this operation.

aMatrix* sort columns ?-increasing|-decreasing? *row
Sorts the columns in the matrix using the data in the specified *row* as the key to sort by. The options **-increasing** and **-decreasing** have the same meaning as for **lsort**. If no option is specified **-increasing** is assumed.

aMatrix* sort rows ?-increasing|-decreasing? *column
Sorts the rows in the matrix using the data in the specified *column* as the key to sort by. The options **-increasing** and **-decreasing** have the same meaning as for **lsort**. If no option is specified **-increasing** is assumed.

aMatrix* swap columns *column_a column_b
Swaps the contents of the two specified columns.

aMatrix* swap rows *row_a row_b
Swaps the contents of the two specified rows.

***aMatrix* transpose**
Transposes the contents of the matrix, i.e. swaps rows for columns and vice versa.

aMatrix* unlink *arrayvar
Removes the link between the matrix and the specified arrayvariable, if there is one.

Examples

The examples below assume a 5x5 matrix M with the first row containing the values 1 to 5, with 1 in the top-left cell. Each other row contains the contents of the row above it, rotated by one cell to the right.

```
% M get rect 0 0 4 4
{{1 2 3 4 5} {5 1 2 3 4} {4 5 1 2 3} {3 4 5 1 2} {2 3 4 5 1}}
```

```
% M set rect 1 1 {{0 0 0} {0 0 0} {0 0 0}}
% M get rect 0 0 4 4
{{1 2 3 4 5} {5 0 0 0 4} {4 0 0 0 3} {3 0 0 0 2} {2 3 4 5 1}}
```

Assuming that the style definitions in the example section of the manpage for the package **report** are loaded into the interpreter now an example which formats a matrix into a tabular report. The code filling the matrix with data is not shown. contains useful data.

```
    % ::struct::matrix m
    % # ... fill m with data, assume 5 columns
    % ::report::report r 5 style captionedtable 1
    % m format 2string r
```

```
    +---+------------------+-------+-------+--------+
    |000|VERSIONS:         |2:8.4a3|1:8.4a3|1:8.4a3%|
    +---+------------------+-------+-------+--------+
    |001|CATCH return ok   |7      |13     |53.85   |
    |002|CATCH return error|68     |91     |74.73   |
    |003|CATCH no catch used|7     |14     |50.00   |
    |004|IF if true numeric|12     |33     |36.36   |
    |005|IF elseif         |15     |47     |31.91   |
    |   |true numeric      |       |       |        |
    +---+------------------+-------+-------+--------+
    %
    % # alternate way of doing the above
    % r printmatrix m
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *struct :: matrix* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Data structures

Copyright

```
Copyright (c) 2002-2013,2019,2022 Andreas Kupries
    <andreas.kupries@gmail.com>
```

pool

Keywords: pool, discrete items, finite, struct

Name

struct::pool - Create and manipulate pool objects (of discrete items)

Synopsis

package require Tcl 8.5 9
package require struct::pool ?**1.2.4**?
::struct::pool ?*poolName*? ?*maxsize*?
poolName *option* ?*arg arg* ...?
poolName **add** *itemName1* ?*itemName2 itemName3* ...?
poolName **clear** ?**-force**?
poolName **destroy** ?**-force**?
poolName **info** *type* ?*arg*?

poolName **maxsize** ?*maxsize*?
poolName **release** *itemName*
poolName **remove** *itemName* ?**-force**?
poolName **request** itemVar ?*options*?

Description

This package provides pool objects which can be used to manage finite collections of discrete items.

::struct::pool ?*poolName*? ?*maxsize*?

Creates a new pool object. If no *poolName* is supplied, then the new pool will be named pool**X**, where X is a positive integer. The optional second argument *maxsize* has to be a positive integer indicating the maximum size of the pool; this is the maximum number of items the pool may hold. The default for this value is **10**.

The pool object has an associated global Tcl command whose name is *poolName*. This command may be used to invoke various configuration operations on the report. It has the following general form:

poolName *option* ?*arg arg ...*?

Option and the *arg*s determine the exact behavior of the command. See section Pool object command for a detailed list of options and their behaviour.

Pools and allocation

The purpose of the pool command and the pool object command that it generates, is to manage pools of discrete items. Examples of a pool of discrete items are:

- the seats in a cinema, theatre, train etc.. for which visitors/travelers can make a reservation;
- the dynamic IP-addresses that an ISP can dole out to subscribers;
- a car rental's collection of cars, which can be rented by customers;
- the class rooms in a school building, which need to be scheduled;
- the database connections available to client-threads in a web-server application;
- the books in a library that customers can borrow;
- etc ...

The common denominator in the examples is that there is a more or less fixed number of items (seats, IP-addresses, cars, ...) that are supposed to be allocated on a more or less regular basis. An item can be allocated only once at a time. An item that is allocated, must be released before it can be re-allocated. While several items in a pool are being allocated and released continuously, the total number of items in the pool remains constant.

Keeping track of which items are allocated, and by whom, is the purpose of the pool command and its subordinates.

Pool parlance: If we say that an item is *allocated*, it means that the item is *busy*, *owned* or *occupied*; it is not available anymore. If an item is *free*, it is *available*. Deallocating an item is equivalent to setting free or releasing an item. The person or entity to which the item has been allotted is said to own the item.

Items

Discrete items

The **pool** command is designed for *discrete items only*. Note that there are pools where allocation occurs on a non-discrete basis, for example computer memory. There are also pools from which the shares that are doled out are not expected to be returned, for example a charity fund or a pan of soup from which you may receive a portion. Finally, there are even pools from which nothing is ever allocated or returned, like a swimming pool or a cesspool.

Unique item names

A pool cannot manage duplicate item names. Therefore, items in a pool must have unique names.

Item equivalence

From the point of view of the manager of a pool, items are equivalent. The manager of a pool is indifferent about which entity/person occupies a given item. However, clients may have preferences for a particular item, based on some item property they know.

Preferences

A future owner may have a preference for a particular item. Preference based allocation is supported (see the **-prefer** option to the request subcommand). A preference for a particular item is most likely to result from variability among features associated with the items. Note that the pool commands themselves are not designed to manage such item properties. If item properties play a role in an application, they should be managed separately.

Pool object command

The following subcommands and corresponding arguments are available to any pool object command.

***poolName* add *itemName1* ?*itemName2 itemName3 ...*?**

This command adds the items on the command line to the pool. If duplicate item names occur on the command line, an error is raised. If one or more of the items already exist in the pool, this also is considered an error.

***poolName* clear ?-force?**

Removes all items from the pool. If there are any allocated items at the time when the command is invoked, an error is raised. This behaviour may be modified through the **-force** argument. If it is supplied on the command line, the pool will be cleared regardless the allocation state of its items.

***poolName* destroy ?-force?**

Destroys the pool data structure, all associated variables and the associated pool object command. By default, the command checks whether any items are still allocated and raises an error if such is the case. This behaviour may be modified through the argument **-force**. If it is supplied on the command line, the pool data structure will be destroyed regardless allocation state of its items.

***poolName* info *type* ?*arg*?**

Returns various information about the pool for further programmatic use. The *type* argument indicates the type of information requested. Only the type **allocID** uses an additional argument.

allocID *itemName*

returns the allocID of the item whose name is *itemName*. Free items have an allocation id of **-1**.

allitems
returns a list of all items in the pool.

allocstate
Returns a list of key-value pairs, where the keys are the items and the values are the corresponding allocation id's. Free items have an allocation id of **-1**.

cursize
returns the current pool size, i.e. the number of items in the pool.

freeitems
returns a list of items that currently are not allocated.

maxsize
returns the maximum size of the pool.

***poolName* maxsize ?*maxsize*?**
Sets or queries the maximum size of the pool, depending on whether the *maxsize* argument is supplied or not. If *maxsize* is supplied, the maximum size of the pool will be set to that value. If no argument is supplied, the current maximum size of the pool is returned. In this variant, the command is an alias for:

poolName info maxsize.

The *maxsize* argument has to be a positive integer.

poolName* release *itemName
Releases the item whose name is *itemName* that was allocated previously. An error is raised if the item was not allocated at the time when the command was issued.

***poolName* remove *itemName* ?-force?**
Removes the item whose name is *itemName* from the pool. If the item was allocated at the time when the command was invoked, an error is raised. This behaviour may be modified through the optional argument **-force**. If it is supplied on the command line, the item will be removed regardless its allocation state.

***poolName* request itemVar ?options?**
Handles a request for an item, taking into account a possible preference for a particular item. There are two possible outcomes depending on the availability of items:

1. The request is honoured, an item is allocated and the variable whose name is passed with the argument *itemVar* will be set to the name of the item that was allocated. The command returns 1.
2. The request is denied. No item is allocated. The variable whose name is itemVar is not set. Attempts to read *itemVar* may raise an error if the variable was not defined before issuing the request. The command returns 0.

The return values from this command are meant to be inspected. The examples below show how to do this. Failure to check the return value may result in erroneous behaviour. If no preference for a particular item is supplied through the option **-prefer** (see below), then all requests are honoured as long as items are available.

The following options are supported:
-allocID *allocID*

If the request is honoured, an item will be allocated to the entity identified by allocID. If the allocation state of an item is queried, it is this allocation ID that will be returned. If the option **-allocID** is not supplied, the item will be given to and owned by **dummyID**. Allocation id's may be anything except the value -1, which is reserved for free items.

-prefer *preferredItem*

This option modifies the allocation strategy as follows: If the item whose name is *preferredItem* is not allocated at the time when the command is invoked, the request is honoured (return value is 1). If the item was allocated at the time when the command was invoked, the request is denied (return value is 0).

Examples

Two examples are provided. The first one mimics a step by step interactive tclsh session, where each step is explained. The second example shows the usage in a server application that talks to a back-end application.

Example 1

This example presents an interactive tclsh session which considers the case of a Car rental's collection of cars. Ten steps explain its usage in chronological order, from the creation of the pool, via the most important stages in the usage of a pool, to the final destruction.

Note aside:

In this example, brand names are used to label the various items. However, a brand name could be regarded as a property of an item. Because the pool command is not designed to manage properties of items, they need to be managed separately. In the latter case the items should be labeled with more neutral names such as: car1, car2, car3, etc ... and a separate database or array should hold the brand names associated with the car labels.

```
 1. Load the package into an interpreter
 % package require pool
 0.1

 2. Create a pool object called `CarPool' with a maximum size of
55 items (cars):
 % pool CarPool 55
 CarPool

 4. Add items to the pool:
 % CarPool add Toyota Trabant Chrysler1 Chrysler2 Volkswagen

 5. Somebody crashed the Toyota. Remove it from the pool as
follows:
 % CarPool remove Toyota

 6. Acquired a new car for the pool. Add it as follows:
 % CarPool add Nissan

 7. Check whether the pool was adjusted correctly:
 % CarPool info allitems
 Trabant Chrysler1 Chrysler2 Volkswagen Nissan
```

Suspend the interactive session temporarily, and show the programmatic use of the request subcommand:

```
 # Mrs. Swift needs a car. She doesn't have a preference for a
 # particular car. We'll issue a request on her behalf as follows:
 if { [CarPool request car -allocID "Mrs. Swift"] }  {
     # request was honoured, process the variable `car'
     puts "$car has been allocated to [CarPool info allocID $car]."
 } else {
     # request was denied
      puts "No car available."
 }
```

Note how the **if** command uses the value returned by the **request** subcommand.

```
 # Suppose Mr. Wiggly has a preference for the Trabant:
 if { [CarPool request car -allocID "Mr. Wiggly" -prefer Trabant]
}  {
     # request was honoured, process the variable `car'
     puts "$car has been allocated to [CarPool info allocID $car]."
 } else {
     # request was denied
      puts "The Trabant was not available."
 }
```

Resume the interactive session:

```
    8. When the car is returned then you can render it available by:
    % CarPool release Trabant

    9. When done, you delete the pool.
    % CarPool destroy
    Couldn't destroy `CarPool' because some items are still allocated.

    Oops, forgot that Mrs. Swift still occupies a car.

    10. We force the destruction of the pool as follows:
    % CarPool destroy -force
```

Example 2

This example describes the case from which the author's need for pool management originated. It is an example of a server application that receives requests from client applications. The client requests are dispatched onto a back-end application before being returned to the client application. In many cases there are a few equivalent instances of back-end applications to which a client request may be passed along. The file descriptors that identify the channels to these back-end instances make up a pool of connections. A particular connection may be allocated to just one client request at a time.

```
    # Create the pool of connections (pipes)
    set maxpipes 10
    pool Pipes $maxpipes
    for {set i 0} {$i < $maxpipes} {incr i} {
        set fd [open "|backendApplication" w+]
        Pipes add $fd
    }

    # A client request comes in. The request is identified as
   `clientX'.
    # Dispatch it onto an instance of a back-end application
    if { [Pipes request fd -allocID clientX] } {
        # a connection was allocated
        # communicate to the back-end application via the variable
   `fd'
        puts $fd "someInstruction"
        # ...... etc.
    } else {
        # all connections are currently occupied
        # store the client request in a queue for later processing,
        # or return a 'Server busy' message to the client.
    }
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *struct :: pool* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Data structures

Copyright

```
Copyright (c) 2002, Erik Leunissen <e.leunissen@hccnet.nl>
```


prioqueue
Keywords: prioqueue, ordered list, priority queue

Name
struct::prioqueue - Create and manipulate prioqueue objects

Synopsis
package require Tcl 8.5 9
package require struct::prioqueue ?**1.5**?
::struct::prioqueue ?**-ascii|-dictionary|-integer|-real**? ?*prioqueueName*?
prioqueueName **option** ?*arg arg ...*?
prioqueueName **clear**
prioqueueName **remove** *item*
prioqueueName **destroy**
prioqueueName **get** ?*count*?
prioqueueName **peek** ?*count*?
prioqueueName **peekpriority** ?*count*?
prioqueueName **put** *item prio* ?*item prio ...*?
prioqueueName **size**

Description
This package implements a simple priority queue using nested tcl lists.

The command **::struct::prioqueue** creates a new priority queue with default priority key type *-integer*. This means that keys given to the **put** subcommand must have this type.

This also sets the priority ordering. For key types *-ascii* and *-dictionary* the data is sorted in ascending order (as with **lsort** *-increasing*), thereas for *-integer* and *-real* the data is sorted in descending order (as with **lsort** *-decreasing*).

Prioqueue names are unrestricted, but may be recognized as options if no priority type is given.

::struct::prioqueue ?-ascii|-dictionary|-integer|-real? ?*prioqueueName*?
The **::struct::prioqueue** command creates a new prioqueue object with an associated global Tcl command whose name is *prioqueueName*. This command may be used to invoke various

operations on the prioqueue. It has the following general form:

***prioqueueName* option ?*arg arg ...*?**

option and the *arg*s determine the exact behavior of the command. The following commands are possible for prioqueue objects:

***prioqueueName* clear**

Remove all items from the prioqueue.

prioqueueName* remove *item

Remove the selected item from this priority queue.

***prioqueueName* destroy**

Destroy the prioqueue, including its storage space and associated command.

***prioqueueName* get ?*count*?**

Return the front *count* items of the prioqueue (but not their priorities) and remove them from the prioqueue. If *count* is not specified, it defaults to 1. If *count* is 1, the result is a simple string; otherwise, it is a list. If specified, *count* must be greater than or equal to 1. If there are no or too few items in the prioqueue, this command will throw an error.

***prioqueueName* peek ?*count*?**

Return the front *count* items of the prioqueue (but not their priorities), without removing them from the prioqueue. If *count* is not specified, it defaults to 1. If *count* is 1, the result is a simple string; otherwise, it is a list. If specified, *count* must be greater than or equal to 1. If there are no or too few items in the queue, this command will throw an error.

***prioqueueName* peekpriority ?*count*?**

Return the front *count* items priority keys, without removing them from the prioqueue. If *count* is not specified, it defaults to 1. If *count* is 1, the result is a simple string; otherwise, it is a list. If specified, *count* must be greater than or equal to 1. If there are no or too few items in the queue, this command will throw an error.

***prioqueueName* put *item prio* ?*item prio ...*?**

Put the *item* or items specified into the prioqueue. *prio* must be a valid priority key for this type of prioqueue, otherwise an error is thrown and no item is added. Items are inserted at their priority ranking. Items with equal priority are added in the order they were added.

***prioqueueName* size**

Return the number of items in the prioqueue.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *struct :: prioqueue* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Data structures

Copyright

Copyright (c) 2003 Michael Schlenker <mic42@users.sourceforge.net>

queue

Keywords: queue, graph, list, matrix, pool, prioqueue, record, set, skiplist, stack, tree

Name

struct::queue - Create and manipulate queue objects

Synopsis

package require Tcl 8.5 9
package require struct::queue ?**1.4.6**?
queueName **option** ?*arg arg* ...?
queueName **clear**
queueName **destroy**
queueName **get** ?*count*?
queueName **peek** ?*count*?
queueName **put** *item* ?*item* ...?
queueName **unget** *item*
queueName **size**

Description

The **::struct** namespace contains a commands for processing finite queues.

It exports a single command, **::struct::queue**. All functionality provided here can be reached through a subcommand of this command.

Note: As of version 1.4.1 of this package a critcl based C implementation is available. This implementation however requires Tcl 8.4 to run.

The **::struct::queue** command creates a new queue object with an associated global Tcl command whose name is *queueName*. This command may be used to invoke various operations on the queue. It has the following general form:

***queueName* option ?*arg arg ...*?**
Option and the *arg*s determine the exact behavior of the command. The following commands are possible for queue objects:

***queueName* clear**
Remove all items from the queue.

***queueName* destroy**
Destroy the queue, including its storage space and associated command.

***queueName* get ?*count*?**
Return the front *count* items of the queue and remove them from the queue. If *count* is not specified, it defaults to 1. If *count* is 1, the result is a simple string; otherwise, it is a list. If specified, *count* must be greater than or equal to 1. If there are not enough items in the queue to fulfull the request, this command will throw an error.

***queueName* peek ?*count*?**
Return the front *count* items of the queue, without removing them from the queue. If *count* is

not specified, it defaults to 1. If *count* is 1, the result is a simple string; otherwise, it is a list. If specified, *count* must be greater than or equal to 1. If there are not enough items in the queue to fulfull the request, this command will throw an error.

***queueName* put *item* ?*item ...*?**
Put the *item* or items specified into the queue. If more than one *item* is given, they will be added in the order they are listed.

queueName* unget *item
Put the *item* into the queue, at the front, i.e. before any other items already in the queue. This makes this operation the complement to the method **get**.

***queueName* size**
Return the number of items in the queue.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *struct :: queue* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Data structures

record

Keywords: record, data structures, struct

Name

struct::record - Define and create records (similar to 'C' structures)

Synopsis

package require Tcl 8.5 9
package require struct::record ?**1.2.4**?
record define *recordName recordMembers* ?*instanceName1 instanceName2 ...*?
record show *record*
record show *instances recordName*
record show *members recordName*
record show *values instanceName*
record exists *record recordName*
record exists *instance instanceName*
record delete *record recordName*
record delete *instance instanceName*
instanceName **cget** -member
instanceName **cget** -member1 -member2
instanceName **cget**
instanceName **configure**

instanceName
instanceName **configure** -member *value*
instanceName **configure** -member1 *value1* -member2 *value2*
recordName **instanceName|#auto** ?*-member1 value1 -member2 value2 ...*?
instanceName **cget** ?*-member1 -member2 ...*?
instanceName **configure** ?*-member1 value1 -member2 value2 ...*?

Description

The **::struct::record** package provides a mechanism to group variables together as one data structure, similar to a *C* structure. The members of a record can be variables or other records. However, a record can not contain circular records, i.e. records that contain the same record as a member.

This package was structured so that it is very similar to how Tk objects work. Each record definition creates a record object that encompasses that definition. Subsequently, that record object can create instances of that record. These instances can then be manipulated with the **cget** and **configure** methods.

The package only contains one top level command, but several sub commands (see below). It also obeys the namespace in which the record was defined, hence the objects returned are fully qualified.

Note: Tcl 9 users may find Tcl's built-in object-oriented functionality is more convenient and versatile than using this package.

record define *recordName recordMembers* ?*instanceName1 instanceName2 ...*?
Defines a record. *recordName* is the name of the record, and is also used as an object command. This object command is used to create instances of the record definition. The *recordMembers* are the members of the record that make up the record definition. These are variables and other records. If optional *instanceName* args are specified, then an instance is generated after the definition is created for each *instanceName*.

record show *record*
Returns a list of records that have been defined.

record show *instances recordName*
Returns the instances that have been instantiated by *recordName*.

record show *members recordName*
Returns the members that are defined for record *recordName*. It returns the same format as how the records were defined.

record show *values instanceName*
Returns a list of values that are set for the instance *instanceName*. The output is a list of key/value pairs. If there are nested records, then the values of the nested records will itself be a list.

record exists *record recordName*
Tests for the existence of a *record* with the name *recordName*.

record exists *instance instanceName*
Tests for the existence of a *instance* with the name *instanceName*.

record delete *record recordName*
Deletes *recordName*, and all instances of *recordName*. It will return an error if the record does

not exist.

record delete *instance instanceName*

Deletes *instance* with the name of *instanceName*. It will return an error if the instance does not exist. Note that this recursively deletes any nested instances as well.

Record members

Record members can either be variables, or other records, However, the same record can not be nested witin itself (circular). To define a nested record, you need to specify the **record** keyword, along the with name of the record, and the name of the instance of that nested record (within the container). For example, it would look like this:

```
# this is the nested record
record define mynestedrecord {
    nest1
    nest2
}

# This is the main record
record define myrecord {
    mem1
    mem2
    {record mynestedrecord mem3}
}
```

You can also assign default or initial values to the members of a record, by enclosing the member entry in braces:

```
record define myrecord {
    mem1
    {mem2 5}
}
```

All instances created from this record definition will initially have **5** as the value for member *mem2*. If no default is given, then the value will be the empty string.

Getting values

To get a value of a member, there are several ways to do this.

instanceName* cget *-member

In this form the built-in **cget** instance method returns the value of the specified *member*. Note the leading dash.

To reach a nested member use *dot notation*:

```
instanceName cget -mem3.nest1
```

instanceName* cget *-member1 -member2

In this form the built-in **cget** instance method returns a list containing the values of both specified members, in the order of specification.

***instanceName* cget**

see below ...

***instanceName* configure**

see below ...

instanceName

These forms are all equivalent. They return a dictionary of all members and the associated values.

Setting values

To set a value of a member, there are several ways to do this.

instanceName* configure *-member value

In this form the built-in **configure** instance method sets the specified *member* to the given *value*. Note the leading dash.

To reach a nested member use *dot notation*:

```
instanceName configure -mem3.nest1 value
```

instanceName* configure *-member1 value1 -member2 value2

In this form the built-in **configure** instance method sets all specified members to the associated values.

Alias access

In the original implementation, access was done by using dot notation similar to how *C* structures are accessed. However, there was a concensus to make the interface more Tcl like, which made sense. However, the original alias access still exists. It might prove to be helpful to some.

Basically, for every member of every instance, an alias is created. This alias is used to get and set values for that member. An example will illustrate the point, using the above defined records:

```
% # Create an instance first
% myrecord inst1
::inst1

% # To get a member of an instance, just use the alias. It behaves
% # like a Tcl command:
% inst1.mem1

% # To set a member via the alias, just include a value. And optionally
% # the equal sign - syntactic sugar.
% inst1.mem1 = 5
5

% inst1.mem1
5

% # For nested records, just continue with the dot notation.
% # note, no equal sign.
```

```
% inst1.mem3.nest1 10
10

% inst1.mem3.nest1
10

% # just the instance by itself gives all member/values pairs for that
% # instance
% inst1
-mem1 5 -mem2 {} -mem3 {-nest1 10 -nest2 {}}

% # and to get all members within the nested record
% inst1.mem3
-nest1 10 -nest2 {}
```

Record command

The following subcommands and corresponding arguments are available to any record command:

***recordName instanceName*|#auto ?*-member1 value1 -member2 value2 ...*?**

Using the *recordName* object command that was created from the record definition, instances of the record definition can be created. Once an instance is created, it inherits the members of the record definition, very similar to how objects work. During instance generation, an object command for the instance is created as well, using *instanceName*.

This object command is used to access the data members of the instance. During the instantiation, while values for that instance may be given, when done, *all* values must be given, and be given as key/value pairs, like for method **configure**. Nested records have to be in list format.

Optionally, **#auto** can be used in place of *instanceName*. When **#auto** is used, the instance name will be automatically generated, and of the form **recordNameN**, where **N** is a unique integer (starting at 0) that is generated.

Instance command

The following subcommands and corresponding arguments are available to any record instance command:

***instanceName* cget ?*-member1 -member2 ...*?**

Each instance has the method **cget**. This is very similar to how Tk widget's **cget** command works. It queries the values of the members for that particular instance. If no arguments are given, then a dictionary is returned.

***instanceName* configure ?*-member1 value1 -member2 value2 ...*?**

Each instance has the method **configure**. This is very similar to how Tk widget's **configure** command works. It sets the values of the particular members for that particular instance. If no arguments are given, then a dictionary list is returned.

Examples

Two examples are provided to give a good illustration on how to use this package.

Example 1 - contact information

Probably the most obvious example would be to hold contact information, such as addresses, phone numbers, comments, etc. Since a person can have multiple phone numbers, multiple email addresses, etc, we will use nested records to define these. So, the first thing we do is define the nested records:

```
##
##  This is an interactive example, to see what is returned by
##  each command as well.
##

% namespace import ::struct::record::*

% # define a nested record. Notice that country has default 'USA'.
% record define locations {
    street
    street2
    city
    state
    zipcode
    {country USA}
    phone
}
::locations
% # Define the main record. Notice that it uses the location record
    twice.
% record define contacts {
    first
    middle
    last
    {record locations home}
    {record locations work}
}
::contacts
% # Create an instance for the contacts record.
% contacts cont1
::cont1
% # Display some introspection values
% record show records
::contacts ::locations
% #
% record show values cont1
-first {} -middle {} -last {} -home {-street {} -street2 {} -city {}
```

```
    -state {} -zipcode {} -country USA -phone {}} -work {-street {}
    -street2 {} -city {} -state {} -zipcode {} -country USA -phone {}}
% #
% record show instances contacts
::cont1
% #
% cont1 config
-first {} -middle {} -last {} -home {-street {} -street2 {} -city {}
    -state {} -zipcode {} -country USA -phone {}} -work {-street {}
    -street2 {} -city {} -state {} -zipcode {} -country USA -phone {}}
% #
% cont1 cget
-first {} -middle {} -last {} -home {-street {} -street2 {} -city {}
    -state {} -zipcode {} -country USA -phone {}} -work {-street {}
    -street2 {} -city {} -state {} -zipcode {} -country USA -phone {}}
% # copy one record to another record
% record define contacts2 [record show members contacts]
::contacts2
% record show members contacts2
first middle last {record locations home} {record locations work}
% record show members contacts
first middle last {record locations home} {record locations work}
%
```

Example 2 - linked list

This next example just illustrates a simple linked list

```
% # define a very simple record for linked list
% record define linkedlist {
    value
    next
}
::linkedlist
% linkedlist lstart
::lstart
% lstart config -value 1 -next [linkedlist #auto]
% [lstart cget -next] config -value 2 -next [linkedlist #auto]
% [[lstart cget -next] cget -next] config -value 3 -next "end"
% set next lstart
lstart
% while 1 {
    lappend values [$next cget -value]
    set next [$next cget -next]
    if {[string match "end" $next]} break
}
% puts "$values"
```

```
1 2 3
% # cleanup linked list
% # We could just use delete record linkedlist also
% foreach I [record show instances linkedlist] {
    record delete instance $I
}
% record show instances linkedlist
%
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *struct :: record* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Data structures

Copyright

Copyright (c) 2002, Brett Schwarz <brett_schwarz@yahoo.com>

skiplist

Keywords: skiplist

Name

struct::skiplist - Create and manipulate skiplists

Synopsis

package require Tcl 8.5 9
package require struct::skiplist ?**1.4**?
skiplistName *option* ?*arg arg* ...?
skiplistName **delete** *node* ?*node...*?
skiplistName **destroy**
skiplistName **insert** *key value*
skiplistName **search** *node* ?*-key key*?
skiplistName **size**
skiplistName **walk** *cmd*

Description

The **::struct::skiplist** command creates a new skiplist object with an associated global Tcl command whose name is *skiplistName*. This command may be used to invoke various operations on the skiplist. It has the following general form:

skiplistName *option* ?*arg arg ...*?
Option and the *arg*s determine the exact behavior of the command.

Skip lists are an alternative data structure to binary trees. They can be used to maintain ordered lists over any sequence of insertions and deletions. Skip lists use randomness to achieve probabilistic balancing, and as a result the algorithms for insertion and deletion in skip lists are much simpler and faster than those for binary trees.

To read more about skip lists see Pugh, William. *Skip lists: a probabilistic alternative to balanced trees* In: Communications of the ACM, June 1990, 33(6) 668-676.

Currently, the key can be either a number or a string, and comparisons are performed with the built in greater than operator. The following commands are possible for skiplist objects:

***skiplistName* delete *node* ?*node*...?**
Remove the specified nodes from the skiplist.

***skiplistName* destroy**
Destroy the skiplist, including its storage space and associated command.

skiplistName* insert *key value
Insert a node with the given *key* and *value* into the skiplist. If a node with that key already exists, then the that node's value is updated and its node level is returned. Otherwise a new node is created and 0 is returned.

***skiplistName* search *node* ?-key *key*?**
Search for a given key in a skiplist. If not found then 0 is returned. If found, then a two element list of 1 followed by the node's value is retuned.

***skiplistName* size**
Return a count of the number of nodes in the skiplist.

skiplistName* walk *cmd
Walk the skiplist from the first node to the last. At each node, the command *cmd* will be evaluated with the key and value of the current node appended.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *struct :: skiplist* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Data structures

Copyright

```
Copyright (c) 2000 Keith Vetter
```

stack

Keywords: stack, graph, matrix, queue, tree

Name

struct::stack - Create and manipulate stack objects

Synopsis

package require Tcl 8.5 9
package require struct::stack ?**1.5.4**?
stackName **option** ?*arg arg* ...?
stackName **clear**
stackName **destroy**
stackName **get**
stackName **getr**
stackName **peek** ?*count*?
stackName **peekr** ?*count*?
stackName **trim** ?*newsize*?
stackName **trim*** ?*newsize*?
stackName **pop** ?*count*?
stackName **push** *item* ?*item...*?
stackName **size**

Description

The **::struct** namespace contains a commands for processing finite stacks.

It exports a single command, **::struct::stack**. All functionality provided here can be reached through a subcommand of this command.

Note: As of version 1.3.3 of this package a critcl based C implementation is available. This implementation however requires Tcl 8.4 to run.

The **::struct::stack** command creates a new stack object with an associated global Tcl command whose name is *stackName*. This command may be used to invoke various operations on the stack. It has the following general form:

***stackName* option ?*arg arg ...*?**
Option and the *arg*s determine the exact behavior of the command. The following commands are possible for stack objects:

***stackName* clear**
Remove all items from the stack.

***stackName* destroy**
Destroy the stack, including its storage space and associated command.

***stackName* get**
Returns the whole contents of the stack as a list, without removing them from the stack.

***stackName* getr**
A variant of **get**, which returns the contents in reversed order.

***stackName* peek ?*count*?**
Return the top *count* items of the stack, without removing them from the stack. If *count* is not specified, it defaults to 1. If *count* is 1, the result is a simple string; otherwise, it is a list. If specified, *count* must be greater than or equal to 1. If there are not enoughs items on the

stack to fulfull the request, this command will throw an error.

***stackName* peekr ?*count*?**
A variant of **peek**, which returns the items in reversed order.

***stackName* trim ?*newsize*?**
Shrinks the stack to contain at most *newsize* elements and returns a list containing the elements which were removed. Nothing is done if the stack is already at the specified size, or smaller. In that case the result is the empty list.

***stackName* trim* ?*newsize*?**
A variant of **trim** which performs the shrinking, but does not return the removed elements.

***stackName* pop ?*count*?**
Return the top *count* items of the stack, and remove them from the stack. If *count* is not specified, it defaults to 1. If *count* is 1, the result is a simple string; otherwise, it is a list. If specified, *count* must be greater than or equal to 1. If there are not enoughs items on the stack to fulfull the request, this command will throw an error.

***stackName* push *item* ?*item...*?**
Push the *item* or items specified onto the stack. If more than one *item* is given, they will be pushed in the order they are listed.

***stackName* size**
Return the number of items on the stack.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *struct :: stack* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Data structures

struct_list

Keywords: struct_list, Fisher-Yates, assign, common, comparison, diff, differential, equal, equality, filter, first permutation, flatten, folding, full outer join, generate permutations, inner join, join, left outer join, list, longest common subsequence, map, next permutation, outer join, permutation, reduce, repeating, repetition, reshuffle, reverse, right outer join, shuffle, subsequence, swapping

Name

struct::list - Procedures for manipulating lists

Synopsis

package require Tcl 8.5 9
package require struct::list ?**1.9**?

::struct::list longestCommonSubsequence *sequence1 sequence2* ?*maxOccurs*?
::struct::list longestCommonSubsequence2 *sequence1 sequence2* ?*maxOccurs*?
::struct::list lcsInvert *lcsData len1 len2*
::struct::list lcsInvert2 *lcs1 lcs2 len1 len2*
::struct::list lcsInvertMerge *lcsData len1 len2*
::struct::list lcsInvertMerge2 *lcs1 lcs2 len1 len2*
::struct::list reverse *sequence*
::struct::list shuffle *list*
::struct::list assign *sequence varname* ?*varname?...*?
::struct::list flatten ?**-full**? ?**--**? *sequence*
::struct::list map *sequence cmdprefix*
::struct::list mapfor *var sequence script*
::struct::list filter *sequence cmdprefix*
::struct::list filterfor *var sequence expr*
::struct::list split *sequence cmdprefix* ?*passVar failVar*?
::struct::list fold *sequence initialvalue cmdprefix*
::struct::list shift *listvar*
::struct::list iota *n*
::struct::list equal ?**-simple**? ?**--**? *a b*
::struct::list repeat *size element1* ?*element2 element3...*?
::struct::list repeatn *value size...*
::struct::list dbJoin ?**-inner|-left|-right|-full**? ?*-keys varname*? {keycol table}...
::struct::list dbJoinKeyed ?**-inner|-left|-right|-full**? ?*-keys varname*? *table...*
::struct::list swap *listvar i j*
::struct::list firstperm *list*
::struct::list nextperm *perm*
::struct::list permutations *list*
::struct::list foreachperm *var list body*

Description

The **::struct::list** namespace contains several useful commands for processing Tcl lists. Generally speaking, they implement algorithms more complex or specialized than the ones provided by Tcl itself.

It exports only a single command, **struct::list**. All functionality provided here can be reached through a subcommand of this command.

Commands

::struct::list longestCommonSubsequence *sequence1 sequence2 ?maxOccurs?*
Returns a list of indices into *sequence1* and a corresponding list where each item is an index into *sequence2* of the matching value according to the longest common subsequences algorithm. If *maxOccurs* is provided, the common subsequence is restricted to elements that occur no more than *maxOccurs* times in *sequence2*.

::struct::list longestCommonSubsequence2 *sequence1 sequence2 ?maxOccurs?*
Returns the longest common subsequence of elements in the two lists *sequence1* and *sequence2*. If *maxOccurs* is provided, the result is only an approximation, where the longest common subsequence is approximated by first determining the longest common sequence of

only those elements that occur no more than *maxOccurs* times in *sequence2*, and then using that result to align the two lists, determining the longest common subsequences of the sublists between the two elements.

The result is the same as for **longestCommonSubsequence**.

::struct::list lcsInvert *lcsData len1 len2*

Takes a description of a longest common subsequence (*lcsData*), inverts it, and returns the result. Inversion means here that as the input describes which parts of the two sequences are identical the output describes the differences instead.

To be fully defined the lengths of the two sequences have to be known and are specified through *len1* and *len2*.

The result is a list where each element describes one chunk of the differences between the two sequences. This description is a list containing three elements, a type and two pairs of indices into *sequence1* and *sequence2* respectively, in this order. The type can be one of three values:

added

Describes an addition. I.e. items which are missing in *sequence1* can be found in *sequence2*. The pair of indices into *sequence1* describes where the added range had been expected to be in *sequence1*. The first index refers to the item just before the added range, and the second index refers to the item just after the added range. The pair of indices into *sequence2* describes the range of items which has been added to it. The first index refers to the first item in the range, and the second index refers to the last item in the range.

deleted

Describes a deletion. I.e. items which are in *sequence1* are missing from *sequence2*. The pair of indices into *sequence1* describes the range of items which has been deleted. The first index refers to the first item in the range, and the second index refers to the last item in the range. The pair of indices into *sequence2* describes where the deleted range had been expected to be in *sequence2*. The first index refers to the item just before the deleted range, and the second index refers to the item just after the deleted range.

changed

Describes a general change. I.e a range of items in *sequence1* has been replaced by a different range of items in *sequence2*. The pair of indices into *sequence1* describes the range of items which has been replaced. The first index refers to the first item in the range, and the second index refers to the last item in the range. The pair of indices into *sequence2* describes the range of items replacing the original range. Again the first index refers to the first item in the range, and the second index refers to the last item in the range.

```
    sequence 1 = {a b r a c a d a b r a}
    lcs 1      =   {1 2   4 5     8 9 10}
    lcs 2      =   {0 1   3 4     5 6 7}
    sequence 2 =   {b r i c a     b r a c}

    Inversion  = {{deleted  {0  0} {-1 0}}
                  {changed  {3  3}  {2 2}}
                  {deleted  {6  7}  {4 5}}
                  {added   {10 11}  {8 8}}}
```

Notes:
- An index of **-1** in a *deleted* chunk refers to just before the first element of the second sequence.
- Also an index equal to the length of the first sequence in an *added* chunk refers to just behind the end of the sequence.

::struct::list lcsInvert2 *lcs1 lcs2 len1 len2*

Similar to **lcsInvert**. Instead of directly taking the result of a call to **longestCommonSubsequence** this subcommand expects the indices for the two sequences in two separate lists.

::struct::list lcsInvertMerge *lcsData len1 len2*

Similar to **lcsInvert**. It returns essentially the same structure as that command, except that it may contain chunks of type **unchanged** too.

These new chunks describe the parts which are unchanged between the two sequences. This means that the result of this command describes both the changed and unchanged parts of the two sequences in one structure.

```
    sequence 1 = {a b r a c a d a b r a}
    lcs 1      =   {1 2   4 5     8 9 10}
    lcs 2      =   {0 1   3 4     5 6 7}
    sequence 2 =   {b r i c a     b r a c}

    Inversion/Merge  = {{deleted   {0  0} {-1 0}}
                        {unchanged {1  2}  {0 1}}
                        {changed   {3  3}  {2 2}}
                        {unchanged {4  5}  {3 4}}
                        {deleted   {6  7}  {4 5}}
                        {unchanged {8 10}  {5 7}}
                        {added    {10 11}  {8 8}}}
```

::struct::list lcsInvertMerge2 *lcs1 lcs2 len1 len2*

Similar to **lcsInvertMerge**. Instead of directly taking the result of a call to **longestCommonSubsequence** this subcommand expects the indices for the two sequences in two separate lists.

::struct::list reverse *sequence*

The subcommand takes a single *sequence* as argument and returns a new sequence

containing the elements of the input sequence in reverse order.

::struct::list shuffle *list*

The subcommand takes a *list* and returns a copy of that list with the elements it contains in random order. Every possible ordering of elements is equally likely to be generated. The Fisher-Yates shuffling algorithm is used internally.

::struct::list assign *sequence varname* ?*varname*?...

The subcommand assigns the first **n** elements of the input *sequence* to the one or more variables whose names were listed after the sequence, where **n** is the number of specified variables.

If there are more variables specified than there are elements in the *sequence* the empty string will be assigned to the superfluous variables.

If there are more elements in the *sequence* than variable names specified the subcommand returns a list containing the unassigned elements. Else an empty list is returned.

```
tclsh> ::struct::list assign {a b c d e} foo bar
c d e
tclsh> set foo
a
tclsh> set bar
b
```

::struct::list flatten ?-full? ?--? *sequence*

The subcommand takes a single *sequence* and returns a new sequence where one level of nesting was removed from the input sequence. In other words, the sublists in the input sequence are replaced by their elements.

The subcommand will remove any nesting it finds if the option **-full** is specified.

```
tclsh> ::struct::list flatten {1 2 3 {4 5} {6 7} {{8 9}} 10}
1 2 3 4 5 6 7 {8 9} 10
tclsh> ::struct::list flatten -full {1 2 3 {4 5} {6 7} {{8 9}}
10}
1 2 3 4 5 6 7 8 9 10
```

::struct::list map *sequence cmdprefix*

The subcommand takes a *sequence* to operate on and a command prefix (*cmdprefix*) specifying an operation, applies the command prefix to each element of the sequence and returns a sequence consisting of the results of that application.

The command prefix will be evaluated with a single word appended to it. The evaluation takes place in the context of the caller of the subcommand.

```
tclsh> # squaring all elements in a list

tclsh> proc sqr {x} {expr {$x*$x}}
tclsh> ::struct::list map {1 2 3 4 5} sqr
1 4 9 16 25
```

```
tclsh> # Retrieving the second column from a matrix
tclsh> # given as list of lists.

tclsh> proc projection {n list} {::lindex $list $n}
tclsh> ::struct::list map {{a b c} {1 2 3} {d f g}} {projection
1}
b 2 f
```

::struct::list mapfor *var sequence script*

The subcommand takes a *sequence* to operate on and a tcl *script*, applies the script to each element of the sequence and returns a sequence consisting of the results of that application.

The script will be evaluated as is, and has access to the current list element through the specified iteration variable *var*. The evaluation takes place in the context of the caller of the subcommand.

```
tclsh> # squaring all elements in a list

tclsh> ::struct::list mapfor x {1 2 3 4 5} {
expr {$x * $x}
}
1 4 9 16 25

tclsh> # Retrieving the second column from a matrix
tclsh> # given as list of lists.

tclsh> ::struct::list mapfor x {{a b c} {1 2 3} {d f g}} {
lindex $x 1
}
b 2 f
```

::struct::list filter *sequence cmdprefix*

The subcommand takes a *sequence* to operate on and a command prefix (*cmdprefix*) specifying an operation, applies the command prefix to each element of the sequence and returns a sequence consisting of all elements of the *sequence* for which the command prefix returned **true**. In other words, this command filters out all elements of the input *sequence* which fail the test the *cmdprefix* represents, and returns the remaining elements.

The command prefix will be evaluated with a single word appended to it. The evaluation takes place in the context of the caller of the subcommand.

```
tclsh> # removing all odd numbers from the input

tclsh> proc even {x} {expr {($x % 2) == 0}}
tclsh> ::struct::list filter {1 2 3 4 5} even
2 4
```

Note: The **filter** is a specialized application of **fold** where the result is extended with the current item or not, depending o nthe result of the test.

::struct::list filterfor *var sequence expr*

The subcommand takes a *sequence* to operate on and a tcl expression (*expr*) specifying a condition, applies the conditionto each element of the sequence and returns a sequence consisting of all elements of the *sequence* for which the expression returned **true**. In other words, this command filters out all elements of the input *sequence* which fail the test the condition *expr* represents, and returns the remaining elements.

The expression will be evaluated as is, and has access to the current list element through the specified iteration variable *var*. The evaluation takes place in the context of the caller of the subcommand.

```
tclsh> # removing all odd numbers from the input

tclsh> ::struct::list filterfor x {1 2 3 4 5} {($x % 2) == 0}
2 4
```

::struct::list split *sequence cmdprefix* ?*passVar failVar*?

This is a variant of method **filter**, see above. Instead of returning just the elements passing the test we get lists of both passing and failing elements.

If no variable names are specified then the result of the command will be a list containing the list of passing elements, and the list of failing elements, in this order. Otherwise the lists of passing and failing elements are stored into the two specified variables, and the result will be a list containing two numbers, the number of elements passing the test, and the number of elements failing, in this order.

The interface to the test is the same as used by **filter**.

::struct::list fold *sequence initialvalue cmdprefix*

The subcommand takes a *sequence* to operate on, an arbitrary string *initial value* and a command prefix (*cmdprefix*) specifying an operation.

The command prefix will be evaluated with two words appended to it. The second of these words will always be an element of the sequence. The evaluation takes place in the context of the caller of the subcommand.

It then reduces the sequence into a single value through repeated application of the command prefix and returns that value. This reduction is done by

1
: Application of the command to the initial value and the first element of the list.

2
: Application of the command to the result of the last call and the second element of the list.

...
: see below ...

i
: Application of the command to the result of the last call and the **i**'th element of the list.

...
: see below ...

end

Application of the command to the result of the last call and the last element of the list. The result of this call is returned as the result of the subcommand.

```
tclsh> # summing the elements in a list.
tclsh> proc + {a b} {expr {$a + $b}}
tclsh> ::struct::list fold {1 2 3 4 5} 0 +
15
```

::struct::list shift *listvar*

The subcommand takes the list contained in the variable named by *listvar* and shifts it down one element. After the call *listvar* will contain a list containing the second to last elements of the input list. The first element of the ist is returned as the result of the command. Shifting the empty list does nothing.

::struct::list iota *n*

The subcommand returns a list containing the integer numbers in the range **[0,n)**. The element at index **i** of the list contain the number **i**.

For "*n* == **0**" an empty list will be returned.

::struct::list equal ?-simple? ?--? *a b*

The subcommand compares the two lists *a* and *b* for equality. In other words, they have to be of the same length and have to contain the same elements in the same order.

By default equality checks are applied recursively to list elements which are lists themselves. This behaviour is disabled when specifying the option **-simple**.

The option **--** disables option processing even if the following argument looks like such.

A boolean value will be returned as the result of the command. This value will be **true** if the two lists are equal, and **false** else.

::struct::list repeat *size element1 ?element2 element3...?*

The subcommand creates a list of length "*size* * *number of elements*" by repeating *size* times the sequence of elements *element1 element2...*. *size* must be a positive integer, *element***n** can be any Tcl value. Note that **repeat 1 arg ...** is identical to **list arg ...**, though the *arg* is required with **repeat**.

Examples:

```
tclsh> ::struct::list repeat 3 a
a a a
tclsh> ::struct::list repeat 3 [::struct::list repeat 3 0]
{0 0 0} {0 0 0} {0 0 0}
tclsh> ::struct::list repeat 3 a b c
a b c a b c a b c
tclsh> ::struct::list repeat 3 [::struct::list repeat 2 a] b c
{a a} b c {a a} b c {a a} b c
```

::struct::list repeatn *value size...*

The subcommand creates a (nested) list containing the *value* in all positions. The exact size and degree of nesting is determined by the *size* arguments, all of which have to be integer numbers greater than or equal to zero.

A single argument *size* which is a list of more than one element will be treated as if more than argument *size* was specified.

If only one argument *size* is present the returned list will not be nested, of length *size* and contain *value* in all positions. If more than one *size* argument is present the returned list will be nested, and of the length specified by the last *size* argument given to it. The elements of that list are defined as the result of **Repeat** for the same arguments, but with the last *size* value removed.

An empty list will be returned if no *size* arguments are present.

```
tclsh> ::struct::list repeatn  0 3 4
{0 0 0} {0 0 0} {0 0 0} {0 0 0}
tclsh> ::struct::list repeatn  0 {3 4}
{0 0 0} {0 0 0} {0 0 0} {0 0 0}
tclsh> ::struct::list repeatn  0 {3 4 5}
{{0 0 0} {0 0 0} {0 0 0} {0 0 0}} {{0 0 0} {0 0 0} {0 0 0} {0 0
0}} {{0 0 0} {0 0 0} {0 0 0} {0 0 0}} {{0 0 0} {0 0 0} {0 0 0}
{0 0 0}} {{0 0 0} {0 0 0} {0 0 0} {0 0 0}}
```

::struct::list dbJoin ?-inner|-left|-right|-full? ?-keys *varname*? {*keycol table*}...

The method performs a table join according to relational algebra. The execution of any of the possible outer join operation is triggered by the presence of either option **-left**, **-right**, or **-full**. If none of these options is present a regular inner join will be performed. This can also be triggered by specifying **-inner**. The various possible join operations are explained in detail in section Table join.

If the **-keys** is present its argument is the name of a variable to store the full list of found keys into. Depending on the exact nature of the input table and the join mode the output table may not contain all the keys by default. In such a case the caller can declare a variable for this information and then insert it into the output table on its own, as she will have more information about the placement than this command.

What is left to explain is the format of the arguments.

The *keycol* arguments are the indices of the columns in the tables which contain the key values to use for the joining. Each argument applies to the table following immediately after it. The columns are counted from **0**, which references the first column. The table associated with the column index has to have at least *keycol*+1 columns. An error will be thrown if there are less.

The *table* arguments represent a table or matrix of rows and columns of values. We use the same representation as generated and consumed by the methods **get rect** and **set rect** of **matrix** objects. In other words, each argument is a list, representing the whole matrix. Its elements are lists too, each representing a single rows of the matrix. The elements of the

row-lists are the column values.

The table resulting from the join operation is returned as the result of the command. We use the same representation as described above for the input *table*s.

::struct::list dbJoinKeyed ?-inner|-left|-right|-full? ?-keys *varname*? *table*...
The operations performed by this method are the same as described above for **dbJoin**. The only difference is in the specification of the keys to use. Instead of using column indices separate from the table here the keys are provided within the table itself. The row elements in each *table* are not the lists of column values, but a two-element list where the second element is the regular list of column values and the first element is the key to use.

::struct::list swap *listvar i j*
The subcommand exchanges the elements at the indices *i* and *j* in the list stored in the variable named by *listvar*. The list is modified in place, and also returned as the result of the subcommand.

::struct::list firstperm *list*
This subcommand returns the lexicographically first permutation of the input *list*.

::struct::list nextperm *perm*
This subcommand accepts a permutation of a set of elements (provided by *perm*) and returns the next permutatation in lexicographic sequence.

The algorithm used here is by Donal E. Knuth, see section References for details.

::struct::list permutations *list*
This subcommand returns a list containing all permutations of the input *list* in lexicographic order.

::struct::list foreachperm *var list body*
This subcommand executes the script *body* once for each permutation of the specified *list*. The permutations are visited in lexicographic order, and the variable *var* is set to the permutation for which *body* is currently executed. The result of the loop command is the empty string.

Longest common subsequence and file comparison

The **longestCommonSubsequence** subcommand forms the core of a flexible system for doing differential comparisons of files, similar to the capability offered by the Unix command **diff**. While this procedure is quite rapid for many tasks of file comparison, its performance degrades severely if *sequence2* contains many equal elements (as, for instance, when using this procedure to compare two files, a quarter of whose lines are blank. This drawback is intrinsic to the algorithm used (see the Reference for details).

One approach to dealing with the performance problem that is sometimes effective in practice is arbitrarily to exclude elements that appear more than a certain number of times. This number is provided as the *maxOccurs* parameter. If frequent lines are excluded in this manner, they will not appear in the common subsequence that is computed; the result will be the longest common subsequence of infrequent elements. The procedure **longestCommonSubsequence2** implements this heuristic. It functions as a wrapper around **longestCommonSubsequence**; it computes the longest common subsequence of infrequent elements, and then subdivides the subsequences that lie between the matches to approximate the true longest common

subsequence.

Table join

This is an operation from relational algebra for relational databases.

The easiest way to understand the regular inner join is that it creates the cartesian product of all the tables involved first and then keeps only all those rows in the resulting table for which the values in the specified key columns are equal to each other.

Implementing this description naively, i.e. as described above will generate a *huge* intermediate result. To avoid this the cartesian product and the filtering of row are done at the same time. What is required is a fast way to determine if a key is present in a table. In a true database this is done through indices. Here we use arrays internally.

An *outer* join is an extension of the inner join for two tables. There are three variants of outerjoins, called *left*, *right*, and *full* outer joins. Their result always contains all rows from an inner join and then some additional rows.

1. For the left outer join the additional rows are all rows from the left table for which there is no key in the right table. They are joined to an empty row of the right table to fit them into the result.
2. For the right outer join the additional rows are all rows from the right table for which there is no key in the left table. They are joined to an empty row of the left table to fit them into the result.
3. The full outer join combines both left and right outer join. In other words, the additional rows are as defined for left outer join, and right outer join, combined.

We extend all the joins from two to **n** tables (**n** > 2) by executing

```
    (...((table1 join table2) join table3) ...) join tableN
```

Examples for all the joins:

```
    Inner Join

    {0 foo}              {0 bagel}    {0 foo   0 bagel}
    {1 snarf} inner join {1 snatz}  = {1 snarf 1 snatz}
    {2 blue}             {3 driver}

    Left Outer Join

    {0 foo}                   {0 bagel}    {0 foo   0 bagel}
    {1 snarf} left outer join {1 snatz}  = {1 snarf 1 snatz}
    {2 blue}                  {3 driver}   {2 blue  {} {}}

    Right Outer Join

    {0 foo}                    {0 bagel}    {0 foo   0 bagel}
    {1 snarf} right outer join {1 snatz}  = {1 snarf 1 snatz}
    {2 blue}                   {3 driver}   {{} {}   3 driver}
```

```
    Full Outer Join

    {0 foo}                    {0 bagel}    {0 foo   0 bagel}
    {1 snarf} full outer join {1 snatz}  = {1 snarf 1 snatz}
    {2 blue}                   {3 driver}   {2 blue  {} {}}
                                            {{} {}   3 driver}
```

References

1. J. W. Hunt and M. D. McIlroy, "An algorithm for differential file comparison," Comp. Sci. Tech. Rep. #41, Bell Telephone Laboratories (1976). Available on the Web at the second author's personal site: http://www.cs.dartmouth.edu/~doug/
2. Donald E. Knuth, "Fascicle 2b of 'The Art of Computer Programming' volume 4". Available on the Web at the author's personal site: http://www-cs-faculty.stanford.edu/~knuth/fasc2b.ps.gz.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *struct :: list* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Data structures

Copyright


```
Copyright (c) 2003-2005 by Kevin B. Kenny. All rights reserved
Copyright (c) 2003-2012 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


struct_map

Keywords: struct_map

Name

struct::map - Manage key/value maps

Synopsis

package require struct::map ?**1**?
::struct::map *mapName*
mapName method ?*arg arg* ...?
mapName **get**
mapName **names**
mapName **set** *name* ?*value*?
mapName **unset** ?*pattern...*?

Description

Provides a snit class whose instances manage a key/value map. In other words, an object wrapper around Tcl arrays.

API

The main command provides construction of maps:

::struct::map *mapName*

Creates a new, empty map with an associated global Tcl command whose name is *mapName*. It may be used to invoke various operations on the map. It has the following general form:

mapName method ?*arg arg ...*?

method and *arg*uments determine the exact behavior of the command.

If *mapName* is specified as **%AUTO%** a unique name will be generated by the package itself. The result of the command is the fully-qualified name of the instance command.

The following commands are possible for map objects:

***mapName* get**

Returns the entire map as a Tcl dictionary.

***mapName* names**

Returns the list of all keys known to the map, in arbitrary order.

***mapName* set *name* ?*value*?**

Sets key *name* to the specified *value*, if the value specified. Returns the value for the key. Throws an error if the key is not known.

***mapName* unset ?*pattern*...?**

Removes all keys matching at least one of the glob *pattern*s from the map. If no pattern is specified all keys are removed. In other words, the default pattern is *. The result of the command is the empty string.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *struct :: list* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

struct_set

Keywords: struct_set, cardinality, difference, emptiness, exclusion, inclusion, intersection, membership, set, symmetric difference, union

Name

struct::set - Commands for manipulating sets

Synopsis

package require Tcl 8.5 9

package require struct::set ?**2.2.4**?
::struct::set empty *set*
::struct::set size *set*
::struct::set contains *set value*
::struct::set union ?*set1...*?
::struct::set intersect ?*set1...*?
::struct::set difference *set1 set2*
::struct::set symdiff *set1 set2*
::struct::set intersect3 *set1 set2*
::struct::set equal *set1 set2*
::struct::set include *setVar value*
::struct::set exclude *setVar value*
::struct::set add *setVar set*
::struct::set subtract *setVar set*
::struct::set subsetof *A B*

Description

The **::struct::set** namespace contains several useful commands for processing finite sets.

It exports only a single command, **struct::set**. All the functionality is available through this command's subcommands.

A new empty set can be created using **::struct::set add** *setVar {}*.

Note: As of version 2.2 of this package a critcl based C implementation will be used where available, with Tcl 8.4 or later.

Commands

In the following, *set*, *set1*, and *set2* are **set** values and *setVar* is a **set**'s variable name.

::struct::set empty *set*
Returns **1** (true) if the *set* is empty; otherwise returns **0** (false).

::struct::set size *set*
Returns the *set*'s cardinality, i.e., the number of elements in the *set*. This could be zero.

::struct::set contains *set value*
Returns **1** (true) if the *set* contains the element *value*; otherwise returns **0** (false).

::struct::set union ?*set1*...?
Returns a **set** consisting of the union of the given **set**s, i.e. *set1* ∪ *set2* ∪ .... The resulting **set** will contain every value in every given **set**, with no duplicates.

::struct::set intersect ?*set1*...?
Returns a **set** consisting of the intersection of the given **set**s, i.e. *set1* ∩ *set2* ∩ .... The resulting **set** will contain every value that *every* one of the given **set**s has in common, with no duplicates.

::struct::set difference *set1 set2*
Returns a **set** consisting of the difference between *set1* and *set2*, i.e., *set1* - *set2*. The resulting **set** will contain every value that is in *set1* but that is *not* in *set2*.

::struct::set symdiff *set1 set2*
Returns a **set** consisting of the symmetric difference between *set1* and *set2*, i.e., *set1* Δ *set2*. The resulting **set** will contain every value that is in *set1 or* is in *set2*, but *not* in both.

::struct::set intersect3 ***set1 set2***
Returns a combination of the methods **intersect** and **difference**.

The return value is a three-element list containing "*set1* ∩ *set2*", "*set1* - *set2*", and "*set2* - *set1*", in this order. In other words, the intersection of *set1* and *set2*, and their differences.

::struct::set equal ***set1 set2***
Returns **1** (true) if the two **set**s contain exactly the same values; otherwise returns **0** (false).

::struct::set include ***setVar value***
Adds element *value* to the **set** called *setVar*, creating *setVar* if it doesn't exist. Harmlessly does nothing if *value* is already in **set** *setVar*. Returns nothing. This command can be used to create new **set**s.

::struct::set exclude ***setVar value***
Removes element *value* from the **set** called *setVar*. If *value* isn't in *setVar*, the command harmlessly does nothing. Returns nothing.

::struct::set add ***setVar set***
Adds every element from the **set** value *set* to the **set** called *setVar*, excluding duplicates. The **set** *setVar* is created if it doesn't exist. Returns nothing.

Use **::struct::set add** *setVar {}* to create a new empty set. Use **::struct::set include** to add individual values.

::struct::set subtract ***setVar set***
Removes every element from the **set** value *set* from the **set** called *setVar*. Returns nothing. Use **::struct::set exclude** to remove individual values.

::struct::set subsetof ***A B***
Returns **1** (true) if $A \subseteq B$, i.e., if **set** *A* is a true subset of or equal to the **set** *B*; otherwise returns **0** (false).

Examples

Creating and populating a new **set** from scratch. The **earth_metals set** will be created on the first iteration of the loop and added to on subsequent iterations.

```
foreach element {Be Mg Ca Sr Ba Ra} {
    struct::set include earth_metals $element
}
```

Creating a **set** by unioning a couple of existing **set**s.

```
set metals [struct::set union $alkali_metals $earth_metals]
```

Querying a **set**'s properties.

```
if {[struct::set empty $metals]} {
    puts "empty"
} else {
    puts "[struct::set size $metals] elements"
}
```

References

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *struct :: set* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Data structures

Copyright


```
Copyright (c) 2004-2008 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


struct_tree

Keywords: struct_tree, breadth-first, depth-first, in-order, node, post-order, pre-order, serialization, tree

Name

struct::tree - Create and manipulate tree objects

Synopsis

package require Tcl 8.5 9
package require struct::tree ?**2.1.1**?
package require struct::list ?**1.5**?
::struct::tree ?*treeName*? ?=|:=|*as*|*deserialize source*?
treeName option ?*arg arg* ...?
::struct::tree::prune
treeName **=** *sourcetree*
treeName **-->** *desttree*
treeName **ancestors** *node*
treeName **append** *node key value*
treeName **attr** *key*
treeName **attr** *key* **-nodes** *list*
treeName **attr** *key* **-glob** *globpattern*
treeName **attr** *key* **-regexp** *repattern*
treeName **children** ?**-all**? *node* ?*filter cmdprefix*?
treeName **cut** *node*
treeName **delete** *node* ?*node* ...?
treeName **depth** *node*
treeName **descendants** *node* ?*filter cmdprefix*?
treeName **deserialize** *serialization*

treeName **destroy**
treeName **exists** *node*
treeName **get** *node key*
treeName **getall** *node* ?*pattern*?
treeName **keys** *node* ?*pattern*?
treeName **keyexists** *node key*
treeName **index** *node*
treeName **insert** *parent index* ?*child* ?*child* ...?
treeName **isleaf** *node*
treeName **lappend** *node key value*
treeName **leaves**
treeName **move** *parent index node* ?*node* ...?
treeName **next** *node*
treeName **numchildren** *node*
treeName **nodes**
treeName **parent** *node*
treeName **previous** *node*
treeName **rename** *node newname*
treeName **rootname**
treeName **serialize** ?*node*?
treeName **set** *node key* ?*value*?
treeName **size** ?*node*?
treeName **splice** *parent from* ?*to*? ?*child*?
treeName **swap** *node1 node2*
treeName **unset** *node key*
treeName **walk** *node* ?*-order order*? ?*-type type*? *loopvar script*
treeName **walkproc** *node* ?*-order order*? ?*-type type*? *cmdprefix*

Description

A tree is a collection of named elements, called nodes, one of which is distinguished as a root, along with a relation ("parenthood") that places a hierarchical structure on the nodes. (Data Structures and Algorithms; Aho, Hopcroft and Ullman; Addison-Wesley, 1987). In addition to maintaining the node relationships, this tree implementation allows any number of keyed values to be associated with each node.

The element names can be arbitrary strings.

A tree is thus similar to an array, but with three important differences:

1. Trees are accessed through an object command, whereas arrays are accessed as variables. (This means trees cannot be local to a procedure.)
2. Trees have a hierarchical structure, whereas an array is just an unordered collection.
3. Each node of a tree has a separate collection of attributes and values. This is like an array where every value is a dictionary.

Note: The major version of the package **struct** has been changed to version 2.0, due to backward incompatible changes in the API of this module. Please read the section Changes for 2.0 for a full list of all changes, incompatible and otherwise.

API

Tree class API

The main commands of the package are:

::struct::tree ?*treeName*? ?=|:=|as|deserialize *source*?

The command creates a new tree object with an associated global Tcl command whose name is *treeName*. This command may be used to invoke various operations on the tree. It has the following general form:

treeName option ?*arg arg ...*?

Option and the *arg*s determine the exact behavior of the command.

If *treeName* is not specified a unique name will be generated by the package itself. If a *source* is specified the new tree will be initialized to it. For the operators **=**,**:=**, and **as** *source* is interpreted as the name of another tree object, and the assignment operator **=** will be executed. For **deserialize** the *source* is a serialized tree object and **deserialize** will be executed.

In other words

```
    ::struct::tree mytree = b
```

is equivalent to

```
    ::struct::tree mytree
    mytree = b
```

and

```
    ::struct::tree mytree deserialize $b
```

is equivalent to

```
    ::struct::tree mytree
    mytree deserialize $b
```

::struct::tree::prune

This command is provided outside of the tree methods, as it is not a tree method per se. It however interacts tightly with the method **walk**. When used in the walk script it causes the traversal to ignore the children of the node we are currently at. This command cannot be used with the traversal modes which look at children before their parent, i.e. **post** and **in**. The only applicable orders of traversal are **pre** and **both**. An error is thrown if the command and chosen order of traversal do not fit.

Tree object API

Two general observations beforehand:

1. The root node of the tree can be used in most places where a node is asked for. The default name of the rootnode is "root", but this can be changed with the method **rename** (see below). Whatever the current name for the root node of the tree is, it can be retrieved by calling the method **rootname**.

2. The method **insert** is the only way to create new nodes, and they are automatically added to a parent. A tree object cannot have nodes without a parent, save the root node.

And now the methods supported by tree objects created by this package:

treeName = sourcetree
: This is the assignment operator for tree objects. It copies the tree contained in the tree object *sourcetree* over the tree data in *treeName*. The old contents of *treeName* are deleted by this operation.

  This operation is in effect equivalent to

```
    treeName deserialize [sourcetree serialize]
```

treeName --> desttree
: This is the reverse assignment operator for tree objects. It copies the tree contained in the tree object *treeName* over the tree data in the object *desttree*. The old contents of *desttree* are deleted by this operation.

  This operation is in effect equivalent to

```
    desttree deserialize [treeName serialize]
```

treeName* ancestors *node
: This method extends the method **parent** and returns a list containing all ancestor nodes to the specified *node*. The immediate ancestor, in other words, parent node, is the first element in that list, its parent the second element, and so on until the root node is reached, making it the last element of the returned list.

treeName* append *node key value
: Appends a *value* to one of the keyed values associated with an node. Returns the new value given to the attribute *key*.

treeName* attr *key
: see below ...

treeName* attr *key* -nodes *list
: see below ...

treeName* attr *key* -glob *globpattern
: see below ...

treeName* attr *key* -regexp *repattern
: This method retrieves the value of the attribute named *key*, for all nodes in the tree (matching the restriction specified via one of the possible options) and having the specified attribute.

  The result is a dictionary mapping from node names to the value of attribute *key* at that node. Nodes not having the attribute *key*, or not passing a specified restriction, are not listed in the result.

  The possible restrictions are:

  -nodes
  : The value is a list of nodes. Only the nodes mentioned in this list are searched for the attribute.

  -glob

The value is a glob pattern. Only the nodes in the tree whose names match this pattern are searched for the attribute.

-regexp
The value is a regular expression. Only the nodes in the tree whose names match this pattern are searched for the attribute.

***treeName* children ?-all? *node* ?filter *cmdprefix*?**
Return a list of the children of *node*. If the option **-all** is specified, then not only the direct children, but their children, and so on are returned in the result. If a filter command is specified only those nodes are listed in the final result which pass the test. The command in *cmdprefix* is called with two arguments, the name of the tree object, and the name of the node in question. It is executed in the context of the caller and has to return a boolean value. Nodes for which the command returns **false** are removed from the result list before it is returned to the caller.

Some examples:

```
mytree insert root end 0 ; mytree set 0 volume 30
mytree insert root end 1
mytree insert root end 2
mytree insert 0    end 3
mytree insert 0    end 4
mytree insert 4    end 5 ; mytree set 5 volume 50
mytree insert 4    end 6

proc vol {t n} {
$t keyexists $n volume
}
proc vgt40 {t n} {
if {![$t keyexists $n volume]} {return 0}
expr {[$t get $n volume] > 40}
}

tclsh> lsort [mytree children -all root filter vol]
0 5

tclsh> lsort [mytree children -all root filter vgt40]
5

tclsh> lsort [mytree children root filter vol]
0

tclsh> puts ([lsort [mytree children root filter vgt40]])
()
```

treeName* cut *node
Removes the node specified by *node* from the tree, but not its children. The children of *node* are made children of the parent of the *node*, at the index at which *node* was located.

***treeName* delete *node* ?*node* ...?**
Removes the specified nodes from the tree. All of the nodes' children will be removed as well to prevent orphaned nodes.

treeName* depth *node
Return the number of steps from node *node* to the root node.

***treeName* descendants *node* ?filter *cmdprefix*?**
This method extends the method **children** and returns a list containing all nodes descending from *node*, and passing the filter, if such was specified.

This is actually the same as "*treeName* **children -all**". **descendants** should be prefered, and "children -all" will be deprecated sometime in the future.

treeName* deserialize *serialization
This is the complement to **serialize**. It replaces tree data in *treeName* with the tree described by the *serialization* value. The old contents of *treeName* are deleted by this operation.

***treeName* destroy**
Destroy the tree, including its storage space and associated command.

treeName* exists *node
Returns true if the specified node exists in the tree.

treeName* get *node key
Returns the value associated with the key *key* for the node *node*.

***treeName* getall *node* ?*pattern*?**
Returns a dictionary (suitable for use with [**array set**]) containing the attribute data for the *node*. If the glob *pattern* is specified only the attributes whose names match the pattern will be part of the dictionary.

***treeName* keys *node* ?*pattern*?**
Returns a list of keys for the *node*. If the *pattern* is specified only the attributes whose names match the pattern will be part of the returned list. The pattern is a **glob** pattern.

treeName* keyexists *node key
Return true if the specified *key* exists for the *node*.

treeName* index *node
Returns the index of *node* in its parent's list of children. For example, if a node has *nodeFoo*, *nodeBar*, and *nodeBaz* as children, in that order, the index of *nodeBar* is 1.

***treeName* insert *parent index* ?*child* ?*child* ...??**
Insert one or more nodes into the tree as children of the node *parent*. The nodes will be added in the order they are given. If *parent* is **root**, it refers to the root of the tree. The new nodes will be added to the *parent* node's child list at the index given by *index*. The *index* can be **end** in which case the new nodes will be added after the current last child. Indices of the form "end-**n**" are accepted as well.

If any of the specified children already exist in *treeName*, those nodes will be moved from their original location to the new location indicated by this command.

If no *child* is specified, a single node will be added, and a name will be generated for the new node. The generated name is of the form *node***x**, where **x** is a number. If names are specified they must neither contain whitespace nor colons (":").

The return result from this command is a list of nodes added.

treeName* isleaf *node
Returns true if *node* is a leaf of the tree (if *node* has no children), false otherwise.

treeName* lappend *node key value
Appends a *value* (as a list) to one of the keyed values associated with an *node*. Returns the new value given to the attribute *key*.

***treeName* leaves**
Return a list containing all leaf nodes known to the tree.

***treeName* move *parent index node* ?*node* ...?**
Make the specified nodes children of *parent*, inserting them into the parent's child list at the index given by *index*. Note that the command will take all nodes out of the tree before inserting them under the new parent, and that it determines the position to place them into after the removal, before the re-insertion. This behaviour is important when it comes to moving one or more nodes to a different index without changing their parent node.

treeName* next *node
Return the right sibling of *node*, or the empty string if *node* was the last child of its parent.

treeName* numchildren *node
Return the number of immediate children of *node*.

***treeName* nodes**
Return a list containing all nodes known to the tree.

treeName* parent *node
Return the parent of *node*.

treeName* previous *node
Return the left sibling of *node*, or the empty string if *node* was the first child of its parent.

treeName* rename *node newname
Renames the node *node* to *newname*. An error is thrown if either the node does not exist, or a node with name *newname* does exist. The result of the command is the new name of the node.

***treeName* rootname**
Returns the name of the root node of the tree.

***treeName* serialize ?*node*?**
This method serializes the sub-tree starting at *node*. In other words it returns a tcl *value* completely describing the tree starting at *node*. This allows, for example, the transfer of tree objects (or parts thereof) over arbitrary channels, persistence, etc. This method is also the basis for both the copy constructor and the assignment operator.

The result of this method has to be semantically identical over all implementations of the tree interface. This is what will enable us to copy tree data between different implementations of the same interface.

The result is a list containing containing a multiple of three elements. It is like a serialized array except that there are two values following each key. They are the names of the nodes in the serialized tree. The two values are a reference to the parent node and the attribute data, in this order.

The reference to the parent node is the empty string for the root node of the tree. For all other nodes it is the index of the parent node in the list. This means that they are integers, greater than or equal to zero, less than the length of the list, and multiples of three. The order of the nodes in the list is important insofar as it is used to reconstruct the lists of children for each node. The children of a node have to be listed in the serialization in the same order as they are listed in their parent in the tree.

The attribute data of a node is a dictionary, i.e. a list of even length containing a serialized array. For a node without attribute data the dictionary is the empty list.

Note: While the current implementation returns the root node as the first element of the list, followed by its children and their children in a depth-first traversal this is not necessarily true for other implementations. The only information a reader of the serialized data can rely on for the structure of the tree is that the root node is signaled by the empty string for the parent reference, that all other nodes refer to their parent through the index in the list, and that children occur in the same order as in their parent.

```
A possible serialization for the tree structure

            +- d
      +- a -+
root -+- b  +- e
      +- c
is

{root {} {} a 0 {} d 3 {} e 3 {} b 0 {} c 0 {}}

The above assumes that none of the nodes have attributes.
```

***treeName* set *node key* ?*value*?**

Set or get one of the keyed values associated with a node. A node may have any number of keyed values associated with it. If *value* is not specified, this command returns the current value assigned to the key; if *value* is specified, this command assigns that value to the key, and returns it.

***treeName* size ?*node*?**

Return a count of the number of descendants of the node *node*; if no node is specified, **root** is assumed.

***treeName* splice *parent from* ?*to*? ?*child*?**

Insert a node named *child* into the tree as a child of the node *parent*. If *parent* is **root**, it refers to the root of the tree. The new node will be added to the parent node's child list at the index given by *from*. The children of *parent* which are in the range of the indices *from* and *to* are made children of *child*. If the value of *to* is not specified it defaults to **end**. If no name is given for *child*, a name will be generated for the new node. The generated name is of the form *node***x**, where **x** is a number. The return result from this command is the name of the new node.

The arguments *from* and *to* are regular list indices, i.e. the form "end-**n**" is accepted as well.

treeName* swap *node1 node2

Swap the position of *node1* and *node2* in the tree.

treeName* unset *node key

Removes a keyed value from the node *node*. The method will do nothing if the *key* does not exist.

treeName* walk *node* ?-order *order*? ?-type *type*? *loopvar script

Perform a breadth-first or depth-first walk of the tree starting at the node *node*. The type of walk, breadth-first or depth-first, is determined by the value of *type*; **bfs** indicates breadth-first, **dfs** indicates depth-first. Depth-first is the default. The order of the walk, pre-, post-, both- or in-order is determined by the value of *order*; **pre** indicates pre-order, **post** indicates post-order, **both** indicates both-order and **in** indicates in-order. Pre-order is the default.

Pre-order walking means that a parent node is visited before any of its children. For example, a breadth-first search starting from the root will visit the root, followed by all of the root's children, followed by all of the root's grandchildren. Post-order walking means that a parent node is visited after any of its children. Both-order walking means that a parent node is visited before *and* after any of its children. In-order walking means that a parent node is visited after its first child and before the second. This is a generalization of in-order walking for binary trees and will do the right thing if a binary tree is walked. The combination of a breadth-first walk with in-order is illegal.

As the walk progresses, the *script* will be evaluated at each node. The evaluation takes place in the context of the caller of the method. Regarding loop variables, these are listed in *loopvar* . If one only one variable is specified it will be set to the id of the node. When two variables are specified, i.e. *loopvar* is a true list, then the first variable will be set to the action performed at the node, and the other to the id of the node itself. All loop variables are created in the context of the caller.

There are three possible actions: **enter**, **leave**, or **visit**. **enter** actions occur during pre-order walks; **leave** actions occur during post-order walks; **visit** actions occur during in-order walks. In a both-order walk, the command will be evaluated twice for each node; the action is **enter** for the first evaluation, and **leave** for the second.

Note: The **enter** action for a node is always performed before the walker will look at the children of that node. This means that changes made by the *script* to the children of the node will immediately influence the walker and the steps it will take.

Any other manipulation, for example of nodes higher in the tree (i.e already visited), or upon leaving will have undefined results. They may succeed, error out, silently compute the wrong result, or anything in between.

At last a small table showing the relationship between the various options and the possible actions.

```
order       type    actions         notes
-----       ----    -----           -----
pre         dfs     enter           parent before children
post        dfs     leave           parent after children
in          dfs     visit           parent between first and second
   child.
```

```
both        dfs     enter, leave    parent before and after children
-----       ----    -----           -----
pre         bfs     enter           parent before children
post        bfs     leave           parent after children
in          bfs             -- illegal --
both        bfs     enter, leave    parent before and after children
-----       ----    -----           -----
```

Note the command **::struct::tree::prune**. This command can be used in the walk script to force the command to ignore the children of the node we are currently at. It will throw an error if the order of traversal is either **post** or **in** as these modes visit the children before their parent, making pruning non-sensical.

treeName* walkproc *node* ?-order *order*? ?-type *type*? *cmdprefix

This method is like method **walk** in all essentials, except the interface to the user code. This method invokes a command prefix with three additional arguments (tree, node, and action), instead of evaluating a script and passing the node via a loop variable.

Changes for 2.0

The following noteworthy changes have occurred:

1. The API for accessing attributes and their values has been simplified.

   All functionality regarding the default attribute "data" has been removed. This default attribute does not exist anymore. All accesses to attributes have to specify the name of the attribute in question. This backward *incompatible* change allowed us to simplify the signature of all methods handling attributes.

   Especially the flag **-key** is not required anymore, even more, its use is now forbidden. Please read the documentation for the methods **set**, **get**, **getall**, **unset**, **append**, **lappend**, **keyexists** and **keys** for a description of the new API's.
2. The methods **keys** and **getall** now take an optional pattern argument and will return only attribute data for keys matching this pattern.
3. Nodes can now be renamed. See the documentation for the method **rename**.
4. The structure has been extended with API's for the serialization and deserialization of tree objects, and a number of operations based on them (tree assignment, copy construction).

   Please read the documentation for the methods **serialize**, **deserialize**, **=**, and **-->**, and the documentation on the construction of tree objects.

   Beyond the copying of whole tree objects these new API's also enable the transfer of tree objects over arbitrary channels and for easy persistence.
5. The walker API has been streamlined and made more similar to the command **foreach**. In detail:
   - The superfluous option **-command** has been removed.
   - Ditto for the place holders. Instead of the placeholders two loop variables have to be specified to contain node and action information.
   - The old command argument has been documented as a script now, which it was in the past too.
   - The fact that **enter** actions are called before the walker looks at the children of a node has

been documented now. In other words it is now officially allowed to manipulate the list of children for a node under *these* circumstances. It has been made clear that changes under any other circumstances will have undefined results, from silently computing the wrong result to erroring out.

1. A new method, **attr**, was added allowing the query and retrieval of attribute data without regard to the node relationship.
2. The method **children** has been extended with the ability to select from the children of the node based on an arbitrary filtering criterium. Another extension is the ability to look not only at the immediate children of the node, but the whole tree below it.

Examples

The following example demonstrates the creation of new nodes:

```
    mytree insert root end 0   ; # Create node 0, as child of the root
    mytree insert root end 1 2 ; # Ditto nodes 1 & 2
    mytree insert 0    end 3   ; # Now create node 3 as child of node 0
    mytree insert 0    end     ; # Create another child of 0, with a
    #                            generated name. The name is returned
    #                            as the result of the command.
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *struct :: tree* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Data structures

Copyright


```
Copyright (c) 2002-2004,2012 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


struct_tree1

Keywords: struct_tree1, tree

Name

struct::tree_v1 - Create and manipulate tree objects

Synopsis

package require Tcl 8.5 9
package require struct::tree ?**1.2.3**?
treeName option ?*arg arg* ...?

treeName **append** *node* ?*-key key*? *value*
treeName **children** *node*
treeName **cut** *node*
treeName **delete** *node* ?*node* ...?
treeName **depth** *node*
treeName **destroy**
treeName **exists** *node*
treeName **get** *node* ?*-key key*?
treeName **getall** *node*
treeName **keys** *node*
treeName **keyexists** *node* ?*-key key*?
treeName **index** *node*
treeName **insert** *parent index* ?*child* ?*child* ...?
treeName **isleaf** *node*
treeName **lappend** *node* ?*-key key*? *value*
treeName **move** *parent index node* ?*node* ...?
treeName **next** *node*
treeName **numchildren** *node*
treeName **parent** *node*
treeName **previous** *node*
treeName **set** *node* ?*-key key*? ?*value*?
treeName **size** ?*node*?
treeName **splice** *parent from* ?*to*? ?*child*?
treeName **swap** *node1 node2*
treeName **unset** *node* ?*-key key*?
treeName **walk** *node* ?*-order order*? ?*-type type*? **-command** *cmd*

Description

The ::struct::tree command creates a new tree object with an associated global Tcl command whose name is *treeName*. This command may be used to invoke various operations on the tree. It has the following general form:

treeName option ?*arg arg ...*?
Option and the *arg*s determine the exact behavior of the command.

A tree is a collection of named elements, called nodes, one of which is distinguished as a root, along with a relation ("parenthood") that places a hierarchical structure on the nodes. (Data Structures and Algorithms; Aho, Hopcroft and Ullman; Addison-Wesley, 1987). In addition to maintaining the node relationships, this tree implementation allows any number of keyed values to be associated with each node.

The element names can be arbitrary strings.

A tree is thus similar to an array, but with three important differences:

1. Trees are accessed through an object command, whereas arrays are accessed as variables. (This means trees cannot be local to a procedure.)
2. Trees have a hierarchical structure, whereas an array is just an unordered collection.
3. Each node of a tree has a separate collection of attributes and values. This is like an array where every value is a dictionary.

The following commands are possible for tree objects:

treeName* append *node* ?-key *key*? *value
Appends a *value* to one of the keyed values associated with an node. If no *key* is specified, the key **data** is assumed.

treeName* children *node
Return a list of the children of *node*.

treeName* cut *node
Removes the node specified by *node* from the tree, but not its children. The children of *node* are made children of the parent of the *node*, at the index at which *node* was located.

***treeName* delete *node* ?*node* ...?**
Removes the specified nodes from the tree. All of the nodes' children will be removed as well to prevent orphaned nodes.

treeName* depth *node
Return the number of steps from node *node* to the root node.

***treeName* destroy**
Destroy the tree, including its storage space and associated command.

treeName* exists *node
Returns true if the specified node exists in the tree.

***treeName* get *node* ?-key *key*?**
Return the value associated with the key *key* for the node *node*. If no key is specified, the key **data** is assumed.

treeName* getall *node
Returns a serialized list of key/value pairs (suitable for use with [**array set**]) for the *node*.

treeName* keys *node
Returns a list of keys for the *node*.

***treeName* keyexists *node* ?-key *key*?**
Return true if the specified *key* exists for the *node*. If no *key* is specified, the key **data** is assumed.

treeName* index *node
Returns the index of *node* in its parent's list of children. For example, if a node has *nodeFoo*, *nodeBar*, and *nodeBaz* as children, in that order, the index of *nodeBar* is 1.

***treeName* insert *parent index* ?*child* ?*child* ...??**
Insert one or more nodes into the tree as children of the node *parent*. The nodes will be added in the order they are given. If *parent* is **root**, it refers to the root of the tree. The new nodes will be added to the *parent* node's child list at the index given by *index*. The *index* can be **end** in which case the new nodes will be added after the current last child.

If any of the specified children already exist in *treeName*, those nodes will be moved from their original location to the new location indicated by this command.

If no *child* is specified, a single node will be added, and a name will be generated for the new node. The generated name is of the form *node***x**, where **x** is a number. If names are specified they must neither contain whitespace nor colons (":").

The return result from this command is a list of nodes added.

treeName* isleaf *node
Returns true if *node* is a leaf of the tree (if *node* has no children), false otherwise.

treeName* lappend *node* ?-key *key*? *value
Appends a *value* (as a list) to one of the keyed values associated with an *node*. If no *key* is specified, the key **data** is assumed.

***treeName* move *parent index node* ?*node* ...?**
Make the specified nodes children of *parent*, inserting them into the parent's child list at the index given by *index*. Note that the command will take all nodes out of the tree before inserting them under the new parent, and that it determines the position to place them into after the removal, before the re-insertion. This behaviour is important when it comes to moving one or more nodes to a different index without changing their parent node.

treeName* next *node
Return the right sibling of *node*, or the empty string if *node* was the last child of its parent.

treeName* numchildren *node
Return the number of immediate children of *node*.

treeName* parent *node
Return the parent of *node*.

treeName* previous *node
Return the left sibling of *node*, or the empty string if *node* was the first child of its parent.

***treeName* set *node* ?-key *key*? ?*value*?**
Set or get one of the keyed values associated with a node. If no key is specified, the key **data** is assumed. Each node that is added to a tree has the value "" assigned to the key **data** automatically. A node may have any number of keyed values associated with it. If *value* is not specified, this command returns the current value assigned to the key; if *value* is specified, this command assigns that value to the key.

***treeName* size ?*node*?**
Return a count of the number of descendants of the node *node*; if no node is specified, **root** is assumed.

***treeName* splice *parent from* ?*to*? ?*child*?**
Insert a node named *child* into the tree as a child of the node *parent*. If *parent* is **root**, it refers to the root of the tree. The new node will be added to the parent node's child list at the index given by *from*. The children of *parent* which are in the range of the indices *from* and *to* are made children of *child*. If the value of *to* is not specified it defaults to **end**. If no name is given for *child*, a name will be generated for the new node. The generated name is of the form *node***x**, where **x** is a number. The return result from this command is the name of the new node.

treeName* swap *node1 node2
Swap the position of *node1* and *node2* in the tree.

***treeName* unset *node* ?-key *key*?**
Removes a keyed value from the node *node*. If no key is specified, the key **data** is assumed.

treeName* walk *node* ?-order *order*? ?-type *type*? -command *cmd
Perform a breadth-first or depth-first walk of the tree starting at the node *node*. The type of walk, breadth-first or depth-first, is determined by the value of *type*; **bfs** indicates

breadth-first, **dfs** indicates depth-first. Depth-first is the default. The order of the walk, pre-, post-, both- or in-order is determined by the value of *order*; **pre** indicates pre-order, **post** indicates post-order, **both** indicates both-order and **in** indicates in-order. Pre-order is the default.

Pre-order walking means that a parent node is visited before any of its children. For example, a breadth-first search starting from the root will visit the root, followed by all of the root's children, followed by all of the root's grandchildren. Post-order walking means that a parent node is visited after any of its children. Both-order walking means that a parent node is visited before *and* after any of its children. In-order walking means that a parent node is visited after its first child and before the second. This is a generalization of in-order walking for binary trees and will do the right thing if a binary is walked. The combination of a breadth-first walk with in-order is illegal.

As the walk progresses, the command *cmd* will be evaluated at each node. Percent substitution will be performed on *cmd* before evaluation, just as in a **bind** script. The following substitutions are recognized:

%%
: Insert the literal % character.

%t
: Name of the tree object.

%n
: Name of the current node.

%a
: Name of the action occurring; one of **enter**, **leave**, or **visit**. **enter** actions occur during pre-order walks; **leave** actions occur during post-order walks; **visit** actions occur during in-order walks. In a both-order walk, the command will be evaluated twice for each node; the action is **enter** for the first evaluation, and **leave** for the second.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *struct :: tree* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Data structures

Copyright

```
Copyright (c) 2002 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

tar

tar

Keywords: tar, archive, tape archive

Name

tar - Tar file creation, extraction & manipulation

Synopsis

package require Tcl 8.5 9
package require tar ?**0.15**?
::tar::contents *tarball* ?**-chan**? ?**-gzip**?
::tar::stat *tarball* ?*file*? ?**-chan**? ?**-gzip**?
::tar::untar *tarball args*
::tar::get *tarball fileName* ?**-chan**? ?**-gzip**?
::tar::create *tarball files args*
::tar::add *tarball files args*
::tar::remove *tarball files*

Description

Note: Starting with version 0.8 the tar reader commands (contents, stats, get, untar) support the GNU LongName extension (header type 'L') for large paths.

Beware

For all commands, when using **-chan** ...

1. It is assumed that the channel was opened for reading, and configured for binary input.
2. It is assumed that the channel position is at the beginning of a legal tar file.
3. The commands will *modify* the channel position as they perform their task.
4. The commands will *not* close the channel.
5. In other words, the commands leave the channel in a state very likely unsuitable for use by further **tar** commands. Still doing so will very likely results in errors, bad data, etc. pp.
6. It is the responsibility of the user to seek the channel back to a suitable position.
7. When using a channel transformation which is not generally seekable, for example **gunzip**, then it is the responsibility of the user to (a) unstack the transformation before seeking the channel back to a suitable position, and (b) for restacking it after.

Regarding support for gzip compression:

1. Errors are thrown when attempting to read from compressed tar archives while compression support (i.e. **::zlib**) is not available.
2. Errors are thrown when attempting to read an uncompressed tar archive when compression is requested by the user (**-gzip**).

   No errors are thrown when attempting to read a compressed tar archive when compression was not requested, and is supported. In that case the commands automatically activate the code handling the compression.
3. Errors are thrown when attempting to edit compressed tar archives. See the commands **tar::add**, and **tar::remove**. This is not supported.

4. Creation of compressed tar archives however is supported, as this sequentially writes the archive, allowing for streaming compression.

Commands

::tar::contents *tarball* ?-chan? ?-gzip?

Returns a list of the files contained in *tarball*. The order is not sorted and depends on the order files were stored in the archive.

If the option **-chan** is present *tarball* is interpreted as an open channel. It is assumed that the channel was opened for reading, and configured for binary input. The command will *not* close the channel.

::tar::stat *tarball* ?file? ?-chan? ?-gzip?

Returns a nested dict containing information on the named ?file? in *tarball*, or all files if none is specified. The top level are pairs of filename and info. The info is a dict with the keys "**mode uid gid size mtime type linkname uname gname devmajor devminor**"

```
% ::tar::stat tarball.tar
foo.jpg {mode 0644 uid 1000 gid 0 size 7580 mtime 811903867 type
    file linkname {} uname user gname wheel devmajor 0 devminor 0}
```

If the option **-chan** is present *tarball* is interpreted as an open channel. It is assumed that the channel was opened for reading, and configured for binary input. The command will *not* close the channel.

::tar::untar *tarball args*

Extracts *tarball*. *-file* and *-glob* limit the extraction to files which exactly match or pattern match the given argument. No error is thrown if no files match. Returns a list of filenames extracted and the file size. The size will be null for non regular files. Leading path seperators are stripped so paths will always be relative.

-dir dirName

Directory to extract to. Uses **pwd** if none is specified

-file fileName

Only extract the file with this name. The name is matched against the complete path stored in the archive including directories.

-glob pattern

Only extract files patching this glob style pattern. The pattern is matched against the complete path stored in the archive.

-nooverwrite

Dont overwrite files that already exist

-nomtime

Leave the file modification time as the current time instead of setting it to the value in the archive.

-noperms

In Unix, leave the file permissions as the current umask instead of setting them to the values in the archive.

-chan

If this option is present *tarball* is interpreted as an open channel. It is assumed that the

channel was opened for reading, and configured for binary input. The command will *not* close the channel.

```
% foreach {file size} [::tar::untar tarball.tar -glob *.jpg] {
puts "Extracted $file ($size bytes)"
}
```

::tar::get *tarball fileName* ?-chan? ?-gzip?

Returns the contents of *fileName* from the *tarball*.

```
% set readme [::tar::get tarball.tar doc/README] {
% puts $readme
}
```

If the option **-chan** is present *tarball* is interpreted as an open channel. It is assumed that the channel was opened for reading, and configured for binary input. The command will *not* close the channel.

An error is thrown when *fileName* is not found in the tar archive.

::tar::create *tarball files args*

Creates a new tar file containing the *files*. *files* must be specified as a single argument which is a proper list of filenames.

-dereference

Normally **create** will store links as an actual link pointing at a file that may or may not exist in the archive. Specifying this option will cause the actual file point to by the link to be stored instead.

-chan

If this option is present *tarball* is interpreted as an open channel. It is assumed that the channel was opened for writing, and configured for binary output. The command will *not* close the channel.

```
% ::tar::create new.tar [glob -nocomplain file*]
% ::tar::contents new.tar
file1 file2 file3
```

::tar::add *tarball files args*

Appends *files* to the end of the existing *tarball*. *files* must be specified as a single argument which is a proper list of filenames.

-dereference

Normally **add** will store links as an actual link pointing at a file that may or may not exist in the archive. Specifying this option will cause the actual file point to by the link to be stored instead.

-prefix string

Normally **add** will store files under exactly the name specified as argument. Specifying a ?-prefix? causes the *string* to be prepended to every name.

-quick

The only sure way to find the position in the *tarball* where new files can be added is to read

it from start, but if *tarball* was written with a "blocksize" of 1 (as this package does) then one can alternatively find this position by seeking from the end. The ?-quick? option tells **add** to do the latter.

::tar::remove *tarball files*
Removes *files* from the *tarball*. No error will result if the file does not exist in the tarball. Directory write permission and free disk space equivalent to at least the size of the tarball will be needed.

```
% ::tar::remove new.tar {file2 file3}
% ::tar::contents new.tar
file3
```

Bugs, ideas, feedback
If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *tar* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
File formats

tepam

tepam_argument_dialogbox
Keywords: tepam_argument_dialogbox, data entry form, parameter entry form

Name
tepam::argument_dialogbox - TEPAM argument_dialogbox, reference manual

Synopsis
package require Tcl 8.5 9
package require Tk 8.3
package require tepam ?**0.5**?
tepam::argument_dialogbox *item_name item_attributes* ?item_name item_attributes? ?...?
tepam::argument_dialogbox *{item_name item_attributes ?item_name item_attributes? ?...?}*

Description

Argument dialogbox call
The TEPAM **argument_dialogbox** is a flexible and easily usable data entry form generator that is available if Tk has been loaded. Each data entry element of a form is defined via a *data entry item* that can be provided to **argument_dialogbox** in two formats:

tepam::argument_dialogbox *item_name item_attributes ?item_name item_attributes? ?...?*

Using this first format, each *data entry item* is defined via a pair of two arguments. The first one is the *item name* that defines the entry widget that has to be used in the form. The second argument, called *item attributes*, specifies the variable which is attributed to the data entry element as well as eventual formatting and context information.

The **argument_dialogbox** returns **ok** if the entered data have been acknowledged (via the *OK* button) and validated by a data checker. If the entered data have been rejected (via the *Cancel* button) the **argument_dialogbox** returns **cancel**.

A small example illustrates how the **argument_dialogbox** can be employed:

```
set DialogResult [tepam::argument_dialogbox \
   -title "Itinerary selection" \
   -file {-label "Itinerary report" -variable report_file} \
   -frame {-label "Itinerary start"} \
      -comment {-text "Specify your itinerary start location"} \
      -entry {-label "City" -variable start_city -type string} \
      -entry {-label "Street" -variable start_street -type string
    -optional 1} \
      -entry {-label "Street number" -variable start_street_nbr
    -type integer -optional 1} \
   -frame {-label "Itinerary destination"} \
      -comment {-text "Specify your itinerary destination"} \
      -entry {-label "City" -variable dest_city -type string} \
      -entry {-label "Street" -variable dest_street -type string
    -optional 1} \
      -entry {-label "Street number" -variable dest_street_nbr -type
    integer -optional 1} \
   -frame {} \
   -checkbutton {-label "Don't use highways" -variable no_highway}]
```

This example opens a dialog box that has the title *Itinerary selection*. A first entry widget in this box allows selecting a report file. It follows two frames to define respectively an itinerary start and end location. Each of these locations that are described with a comment has three entry widgets to specify respectively the city, street and the street number. Bellow the second frame there is a check button that allows specifying if eventual highways should be ignored.

tepam::argument_dialogbox {*item_name item_attributes ?item_name item_attributes? ?...?*}

Sometimes it is simpler to provide all the data entry item definitions in form of a single list to **argument_dialogbox**, and not as individual arguments. The second format that is supported by **argument_dialogbox** corresponds exactly to the first one, except that all item definitions are packed into a single list that is provided to **argument_dialogbox**. The previous example can therefore also be written in the following way:

```
set DialogResult [tepam::argument_dialogbox {
   -title "Itinerary selection"
   -file {-label "Itinerary report" -variable report_file}
   ...
   -checkbutton {-label "Don't use highways" -variable no_highway} }]
```

The commands **argument_dialogbox** as well as **procedure** are exported from the namespace tepam. To use these commands without the **tepam::** namespace prefix, it is sufficient to import them into the main namespace:

```
namespace import tepam::*

set DialogResult [argument_dialogbox \
   -title "Itinerary selection"
   ...
```

The following subsections explain the different argument item types that are accepted by the **argument_dialogbox**, classified into three groups. The first data entry item definition format will be used in the remaining document, knowing that this format can always be transformed into the second format by putting all arguments into a single list that is then provided to **argument_dialogbox**.

Context definition items

The first item group allows specifying some context aspects of an argument dialog box. These items are taking a simple character string as item attribute:

```
tepam::argument_dialogbox \
   -<argument_name> string \
   ...
```

The following items are classified into this group:

-title *string*

The dialog box window title which is by default *Dialog* can be changed with the *-title* item:

```
tepam::argument_dialogbox \
   -title "System configuration" \
   ...
```

-window *string*

The argument dialog box uses by default *.dialog* as dialog top level window. This path can be changed with the *-window* item:

```
tepam::argument_dialogbox \
   -window .dialog \
   ...
```

-parent *string*

By defining a parent window, the argument dialog box will be displayed beside this one. Without explicit parent window definition, the top-level window will be considered as parent

window.

```
tepam::argument_dialogbox \
   -parent .my_appl \
   ...
```

-context *string*

If a context is defined the dialog box state, e.g. the entered data as well as the window size and position, is restored the next time the argument dialog box is called. The assignment of a context allows saving the dialog box state in its context to distinguish between different usages of the argument dialog box.

```
tepam::argument_dialogbox \
   -context destination_definitions \
   ...
```

Formatting and display options

Especially for big, complex forms it becomes important that the different data entry widgets are graphically well organized and commented to provide an immediate and clear overview to the user. A couple of items allow structuring and commenting the dialog boxes.

The items of this classification group require as item attributes a definition list, which contains itself attribute name and value pairs:

```
tepam::argument_dialogbox \
   ...
   -<argument_name> {
      ?-<attribute_name> <attribute_value>?
      ?-<attribute_name> <attribute_value>?
      ?...?
   }
   ...
```

The following items are classified into this group:

-frame *list*

The *-frame* item allows packing all following entry widgets into a labeled frame, until a next frame item is defined or until the last entry widget has been defined. It recognizes the following attributes inside the item attribute list:

-label *string*

An optional frame label can be specified with the *-label* statement.

Example:

```
tepam::argument_dialogbox \
   ...
   -frame {-label "Destination address"}
   ...
```

To close an open frame without opening a new one, an empty list has to be provided to the

-frame statement.

```
tepam::argument_dialogbox \
   ...
   -frame {}
   ...
```

-sep [const {{}}]
Entry widgets can be separated with the *-sep* statement which doesn't require additional definitions. The related definition list has to exist, but its content is ignored.

```
tepam::argument_dialogbox \
   ...
   -sep {}
   ...
```

-comment *string*
Comments and descriptions can be added with the *-text* attribute of the *-comment* item. Please note that each entry widget itself can also contain a *-text* attribute for comments and descriptions. But the *-comment* item allows for example adding a description between two frames.

```
tepam::argument_dialogbox \
   ...
   -comment {-text "Specify bellow the destination address"}
   ...
```

-yscroll 0|1|auto
This attribute allows controlling an eventual vertical scrollbar. Setting it to **0** will permanently disable the scrollbar, setting it to **1** will enable it. By default it is set to **auto**. The scrollbar is enabled in this mode only if the vertical data entry form size exceeds 66% of the screen height.

```
tepam::argument_dialogbox \
   ...
   -yscroll auto
   ...
```

Global custom data validation

This item group allows specifying global custom checks to validate the entered data.

-validatecommand *script*
Custom data checks can be performed via validation commands that are defined with the *-validatecommand* item. Example:

```
tepam::argument_dialogbox \
   -entry {-label "Your comment" -variable YourCom} \
   -validatecommand {IllegalWordDetector $YourCom}
```

The validation command is executed in the context of the calling procedure, once all the basic data checks have been performed and data variables are assigned. All data is accessed via the data variables. Note that there is also an entry widget specific attribute *-validatecommand* that allows declaring custom checks for specific data entries.

The attribute *-validatecommand* can be repeated to declare multiple custom checks.

-validatecommand_error_text *string*
This item allows overriding the default error message for a global custom argument validation (defined by *-validatecommand*). Also this attribute can be repeated in case multiple checks are declared.

-validatecommand2 *script*
see below ...

-validatecommand2_error_text *string*
These items are mainly for TEPAM internal usage.

These items corrspond respectively to *-validatecommand* and *-validatecommand_error_text*. However, the data validation is not performed in the next upper stack level, but two stack levels higher.

Data entry widget items

Data entry widgets are created with the widget items. These items require as item attributes a definition list, which contains itself attribute name and value pairs:

```
tepam::argument_dialogbox \
   ...
   -<argument_name> {
      ?-<attribute_name> <attribute_value>?
      ?-<attribute_name> <attribute_value>?
      ?...?
   }
   ...
```

The attribute list can contain various attributes to describe and comment an entry widget and to constrain its entered value. All entry widgets are accepting a common set of attributes that are described in the section Entry widget item attributes.

TEPAM defines a rich set of entry widgets. If necessary, this set can be extended with additional application specific entry widgets (see Application specific entry widgets):

-entry *list*
The *-entry* item generates the simplest but most universal data entry widget. It allows entering any kind of data in form of single line strings.

```
tepam::argument_dialogbox \
   -entry {-label Name -variable Entry}
```

-text *list*
The *-text* item generates a multi line text entry widget. The widget height can be selected with the *-height* attribute.

```
tepam::argument_dialogbox \
   -text {-label Name -variable Text -height 5}
```

-checkbox *list*

A group of check boxes is created with the *-checkbox* item. The number of check boxes and their option values are specified with a list assigned to the *-choices* attribute or via a variable declared with the *-choicevariable* attribute:

```
tepam::argument_dialogbox \
   -checkbox {-label "Font sytle" -variable FontStyle \
              -choices {bold italic underline} -default italic}
```

If the labels of the check boxes should differ from the option values, their labels can be defined with the *-choicelabels* attribute:

```
tepam::argument_dialogbox \
   -checkbox {-label "Font sytle" -variable FontStyle \
              -choices {bold italic underline} \
              -choicelabels {Bold Italic Underline} \
              -default italic}
```

In contrast to a radio box group, a check box group allows selecting simultaneously several choice options. The selection is stored for this reason inside the defined variable in form of a list, even if only one choice option has been selected.

-radiobox *list*

A group of radio boxes is created with the *-radiobox* item. The number of radio boxes and their option values are specified with a list assigned to the *-choices* attribute or via a variable declared with the *-choicevariable* attribute.

In contrast to a check box group, a radio box group allows selecting simultaneously only one choice option. The selected option value is stored directly, and not in form of a list, inside the defined variable.

```
tepam::argument_dialogbox \
   -radiobox {-label "Text adjustment" -variable Adjustment \
              -choices {left center right} -default left}
```

If the labels of the radio boxes should differ from the option values, their labels can be defined with the *-choicelabels* attribute:

```
tepam::argument_dialogbox \
   -radiobox {-label "Text adjustment" -variable Adjustment \
              -choices {left center right} \
              -choicelabels {Left Center Right} -default left}
```

-checkbutton *list*

The *-checkbutton* entry widget allows activating or deactivating a single choice option. The result written into the variable will either be **0** if the check button was not activated or **1** if it was activated. An eventually provided default value has also to be either **0** or **1**.

```
tepam::argument_dialogbox \
   -checkbutton {-label Capitalize -variable Capitalize -default 1}
```

Several types of list and combo boxes are available to handle selection lists.

-combobox *list*

The combobox is a combination of a normal entry widget together with a drop-down list box. The combobox allows selecting from this drop-down list box a single element. The list of the available elements can be provided either as a list to the *-choices* attribute, or via a variable that is specified with the *-choicevariable* attribute.

```
tepam::argument_dialogbox \
   -combobox {-label "Text size" -variable Size -choices {8 9 10 12
    15 18} -default 12}
```

And here is an example of using a variable to define the selection list:

```
set TextSizes {8 9 10 12 15 18}
tepam::argument_dialogbox \
   -combobox {-label "Text size" -variable Size -choicevariable
    TextSizes -default 12}
```

-listbox *list*

In contrast to the combo box, the list box is always displayed by the *listbox* entry widget. Only one element is selectable unless the *-multiple_selection* attribute is set. The list box height can be selected with the *-height* attribute. If the height is not explicitly defined, the list box height is automatically adapted to the argument dialog box size. The first example uses a variable to define the available choices:

```
set set AvailableSizes
for {set k 0} {$k<16} {incr k} {lappend AvailableSizes [expr 1<<$k]}

tepam::argument_dialogbox \
   -listbox {-label "Distance" -variable Distance \
             -choicevariable AvailableSizes -default 6 -height 5}
```

Here is a multi-element selection example. Please note that also the default selection can contain multiple elements:

```
tepam::argument_dialogbox \
   -listbox {-label "Text styles" -variable Styles \
             -choices {bold italic underline overstrike} \
             -choicelabels {Bold Italic Underline Overstrike} \
             -default {bold underline} -multiple_selection 1 \
             -height 3}
```

-disjointlistbox *list*

A disjoint list box has to be used instead of a normal list box if the selection order is important. The disjoint list box entry widget has in fact two list boxes, one to select elements

and one to display the selected elements in the chosen order.

Disjoint listboxes allow always selecting multiple elements. With the exception of the *-multiple_selection* attribute, disjointed list boxes are accepting the same attributes as the normal listbox, e.g. *-height, -choices, -choicevariable, -default*.

```
tepam::argument_dialogbox \
   -disjointlistbox {-label "Preferred scripting languages"
    -variable Languages \
             -comment "Please select your preferred languages in the
    order" \
             -choices {JavaScript Lisp Lua Octave PHP Perl Python
    Ruby Scheme Tcl} \
             -default {Tcl Perl Python}}
```

The file and directory selectors are building a next group of data entry widgets. A paragraph of section Entry widget item attributes explains the widget specific attributes that allow specifying the targeted file types, active directory etc.

-file *list*

The item *-file* creates a group composed by an entry widget together with a button that allows opening a file browser. The data type *file* is automatically selected for this entry if no data type has been explicitly defined with the *-type* attribute.

```
tepam::argument_dialogbox \
   -file {-label "Image file" -variable ImageF \
          -filetypes {{"GIF" {*.gif}} {"JPG" {*.jpg}}} \
          -initialfile "picture.gif"}
```

-existingfile *list*

The item *-existingfile* creates a group composed by an entry widget together with a button that allows opening a browser to select an existing file. The data type *existingfile* is automatically selected for this entry if no data type has been explicitly defined with the *-type* attribute.

```
tepam::argument_dialogbox \
   -existingfile {-label "Image file" -variable ImageF \
                  -filetypes {{"GIF" {*.gif}} {"JPG" {*.jpg}}} \
                  -initialfile "picture.gif"}
```

-directory *list*

The item *-directory* creates a group composed by an entry widget together with a button that allows opening a directory browser. The data type *directory* is automatically selected for this entry if no data type has been explicitly defined with the *-type* attribute.

```
tepam::argument_dialogbox \
   -directory {-label "Report directory" -variable ReportDir}
```

-existingdirectory *list*

The item *-existingdirectory* creates a group composed by an entry widget together with a

button that allows opening a browser to select an existing directory. The data type *existingdirectory* is automatically selected for this entry if no data type has been explicitly defined with the *-type* attribute.

```
tepam::argument_dialogbox \
   -existingdirectory {-label "Report directory" -variable ReportDir}
```

Finally, there is a last group of some other special data entry widgets.

-color *list*

The color selector is composed by an entry widget together with a button that allows opening a color browser. The data type *color* is automatically selected for this entry widget type if no data type has been explicitly defined with the *-type* attribute.

```
tepam::argument_dialogbox \
   -color {-label "Background color" -variable Color -default red}
```

-font *list*

The font selector is composed by an entry widget together with a button that allows opening a font browser. The data type *font* is automatically selected for this entry widget type if no data type has been explicitly defined with the *-type* attribute. The entry widget displays an example text in the format of the selected font.

The font browser allows selecting by default the font families provided by the **font families** Tk command as well as a reasonable set of different font sizes between 6 points and 40 points. Different sets of font families and font sizes can be specified respectively via the *-font_families* or *-font_sizes* attributes.

If no default font is provided via the *-default* attribute, the default font of the label widget to display the selected font will be used as default selected font. If the font family of this label widget is not part of the available families the first available family is used as default. If the font size of this label widget is not part of the available sizes the next close available size is selected as default size.

```
tepam::argument_dialogbox \
   -font {-label "Font" -variable Font \
          -font_sizes {8 10 12 16} \
          -default {Arial 20 italic}}
```

Entry widget item attributes

All the entry widget items are accepting the following attributes:

-text *string*

Eventual descriptions and comments specified with the *-text* attribute are displayed above the entry widget.

```
tepam::argument_dialogbox \
   -entry {-text "Please enter your name bellow" -variable Name}
```

-label *string*

The label attribute creates left to the entry widget a label using the provided string as label

text:

```
tepam::argument_dialogbox \
   -entry {-label Name -variable Name}
```

-variable *string*

All entry widgets require a specified variable. After accepting the entered information with the OK button, the entry widget data is stored inside the defined variables.

```
tepam::argument_dialogbox \
   -existingdirectory {-label "Report directory" -variable ReportDir}
```

-default *string*

Eventual default data for the entry widgets can be provided via the *-default* attribute. The default value is overridden if an argument dialog box with a defined context is called another time. The value acknowledged in a previous call will be used in this case as default value.

```
tepam::argument_dialogbox \
   -checkbox {-label "Font sytle" -variable FontStyle \
              -choices {bold italic underline} -default italic}
```

-optional 0|1

Data can be specified as optional or mandatory with the *-optional* attribute that requires either **0** (mandatory) or **1** (optional) as attribute data.

In case an entry is optional and no data has been entered, e.g. the entry contains an empty character string, the entry will be considered as undefined and the assigned variable will not be defined.

```
tepam::argument_dialogbox \
   -entry {-label "City" -variable start_city -type string} \
   -entry {-label "Street" -variable start_street -type string
    -optional 0} \
   -entry {-label "Street number" -variable start_street_nbr -type
    integer -optional 1} \
```

-type *string*

If the data type is defined with the *-type* attribute the argument dialog box will automatically perform a data type check after acknowledging the entered values and before the dialog box is closed. If a type incompatible value is found an error message box appears and the user can correct the value.

The argument dialog box accepts all types that have been specified by the TEPAM package and that are also used by tepam::procedure (see the *tepam::procedure reference manual*).

Some entry widgets like the file and directory widgets, as well as the color and font widgets are specifying automatically the default data type if no type has been specified explicitly with the *-type* attribute.

```
tepam::argument_dialogbox \
   -entry {-label "Street number" -variable start_street_nbr -type
    integer} \
```

-range *string*

Values can be constrained with the *-range* attribute. The valid range is defined with a list containing the minimum valid value and a maximum valid value.

The *-range* attribute has to be used only for numerical arguments, like integers and doubles.

```
tepam::argument_dialogbox \
   -entry {-label Month -variable Month -type integer -range {1 12}}
```

-validatecommand *string*

Custom argument value validations can be performed via specific validation commands that are defined with the *-validatecommand* attribute. The provided validation command can be a complete script in which the pattern *%P* is placeholder for the argument value that has to be validated.

```
tepam::argument_dialogbox \
   -entry {-label "Your comment" -variable YourCom \
           -validatecommand "IllegalWordDetector %P"}
```

While the purpose of this custom argument validation attribute is the validation of a specific argument, there is also a global data validation attribute *-validatecommand* that allows performing validation that involves multiple arguments.

-validatecommand_error_text *string*

This attribute allows overriding the default error message for a custom argument validation (defined by *-validatecommand*).

Some other attributes are supported by the list and combo boxes as well as by the radio and check buttons.

-choices *string*

Choice lists can directly be defined with the *-choices* attribute. This way to define choice lists is especially adapted for smaller, fixed selection lists.

```
tepam::argument_dialogbox \
   -listbox {-label "Text styles" -variable Styles \
             -choices {bold italic underline} -default underline
```

-choicelabels *string (only check and radio buttons)*

If the labels of the check and radio boxes should differ from the option values, they can be defined with the *-choicelabels* attribute:

```
tepam::argument_dialogbox \
   -checkbox {-label "Font sytle" -variable FontStyle \
              -choices {bold italic underline} \
              -choicelabels {Bold Italic Underline}
```

-choicevariable *string*
Another way to define the choice lists is using the *-choicevariable* attribute. This way to define choice lists is especially adapted for huge and eventually variable selection lists.

```
set TextSizes {8 9 10 12 15 18}
tepam::argument_dialogbox \
   -combobox {-label "Text size" -variable Size -choicevariable
    TextSizes}
```

-multiple_selection 0|1
The list box item (**-listbox**) allows by default selecting only one list element. By setting the *-multiple_selection* attribute to **1**, multiple elements can be selected.

```
tepam::argument_dialogbox \
   -listbox {-label "Text styles" -variable Styles \
             -choices {bold italic underline} -default underline \
             -multiple_selection 1 -height 3}
```

Some additional attributes are supported by the file and directory selection widgets.

-filetypes *string*
The file type attribute is used by the **-file** and **-existingfile** items to define the file endings that are searched by the file browser.

```
tepam::argument_dialogbox \
   -file {-label "Image file" -variable ImageF \
          -filetypes {{"GIF" {*.gif}} {"JPG" {*.jpg}}}}
```

-initialfile *string*
The initial file used by the file browsers of the **-file** and **-existingfile** widgets are by default the file defined with the *-default* attribute, unless a file is specified with the *-initialfile* attribute.

```
tepam::argument_dialogbox \
   -file {-variable ImageF -initialfile "picture.gif"}
```

-activedir *string*
The *-activedir* attribute will override the default active search directory used by the file browsers of all file and directory entry widgets. The default active search directory is defined by the directory of a specified initial file (*-initialfile*) if defined, and otherwise by the directory of the default file/directory, specified with the *-default* attribute.

```
tepam::argument_dialogbox \
   -file "-variable ImageF -activedir $pwd"
```

Finally, there is a last attribute supported by some widgets:

-height *string*
All widgets containing a selection list (**-listbox**, **-disjointlistbox**, **-font**) as well as the multi line **-text** widget are accepting the *-height* attribute that defines the number of displayed

rows of the selection lists.

```
tepam::argument_dialogbox \
   -listbox {-label "Text size" -variable Size \
             -choices {8 9 10 12 15 18} -default 12 -height 3}
```

If the no height has been explicitly specified the height of the widget will be dynamically adapted to the argument dialog box size.

Application specific entry widgets

An application specific entry widget can be made available to the argument dialog box by adding a dedicated procedure to the tepam namespace. This procedure has three arguments; the first one is the widget path, the second one a subcommand and the third argument has various purposes:

```
proc tepam::ad_form(<WidgetName>) {W Command {Par ""}} {
   upvar Option Option; # if required
   variable argument_dialogbox; # if required
   switch $Command {
      "create" <CreateCommandSequence>
      "set_choice" <SetChoiceCommandSequence>
      "set" <SetCommandv>
      "get" <GetCommandSequence>
   }
}
```

Argument_dialogbox takes care about the *-label* and *-text* attributes for all entry widgets. For any data entry widget it creates a frame into which the data entry widget components can be placed. The path to this frame is provided via the *W* argument.

The entry widget procedure has to support 3 mandatory and an optional command that are selected via the argument *Command*:

create

The entry widget is called a first time with the command *create* to build the data entry widget.

The frames that are made available by **argument_dialogbox** for the entry widgets are by default only extendable in the X direction. To make them also extendable in the Y direction, for example for extendable list boxes, the command **ad_form(make_expandable) $W** has to be executed when an entry widget is built.

set_choice

The entry widget procedure is only called with the *set_choice* command if the *-choices* or *-choicevariable* has been specified. The command is therefore only relevant for list and combo boxes.

The available selection list that is either specified with the *-choices* or *-choicevariable* attribute is provided via the *Par* argument to the entry widget procedure. This list can be used to initialize an available choice list.

set
: If a default value is either defined via the *-default* attribute or via a preceding call the entry widget procedure is called with the *set* command. The argument *Par* contains in this case the value to which the entry widget has to be initialized.

get
: The entry widget procedure command *get* has to return the current value of the entry widget.

Eventually specified entry widget item attributes are available via the **Option** array variable of the calling procedure. This variable becomes accessible inside the entry widget procedure via the **upvar** command.

There may be a need to store some information in a variable. The array variable **argument_dialogbox** has to be used for this purpose together with array indexes starting with "*$W,*", e.g. *argument_dialogbox($W,values)*.

Examples of entry widget procedures are directly provided by the TEPAM package source file that specifies the standard entry widget procedures. The simplest procedure is the one for the basic *entry* widget:

```
proc tepam::ad_form(entry) {W Command {Par ""}} {
   switch $Command {
      "create" {pack [entry \$W.entry] -fill x \
                        -expand yes -pady 4 -side left}
      "set" {\$W.entry insert 0 $Par}
      "get" {return [\$W.entry get]}
   }
}
```

It is also possible to relay on an existing entry widget procedure to derive a new, more specific one. The *radiobox* widget is used for example, to create a new entry widget that allows selecting either *left*, *center* or *right*. The original *radiobox* widget is called with the *set_choice* command immediately after the *create* command, to define the fixed list of selection options.

```
proc tepam::ad_form(rcl) {W Command {Par ""}} {
   set Res [ad_form(radiobox) $W $Command $Par]
   if {$Command=="create"} {
      ad_form(radiobox) $W set_choice {left center right}
   }
   return $Res
}
```

Please consult the TEPAM package source file to find additional and more complex examples of entry widget procedures.

Variables

The **argument_dialogbox** is using two variables inside the namespace **::tepam**:

argument_dialogbox
: Application specific entry widget procedures can use this array variable to store their own data, using as index the widget path provided to the procedure, e.g. *argument_dialogbox($W,)*.

last_parameters
This array variable is only used by an argument dialog box if its context has been specified via the *-context* attribute. The argument dialog box position and size as well as its entered data are stored inside this variable if the data are acknowledged and the form is closed. This allows the form to restore its previous state once it is called another time.

To reuse the saved parameters not just in the actual application session but also in another one, it is sufficient to store the **last_parameter** array variable contents in a configuration file which is loaded the next time an application is launched.

Category
Argument entry form, mega widget

Copyright

```
Copyright (c) 2009-2013, Andreas Drollinger
```


tepam_doc_gen
Keywords: tepam_doc_gen, automatic documentation, documentation, procedure documentation

Name
tepam::doc_gen - TEPAM DOC Generation, reference manual

Synopsis
package require Tcl 8.5 9
package require tepam 0.5
package require tepam::doc_gen ?**0.1.3**?
tepam::doc_gen::generate ?*-format format*? ?*-style style*? ?*-header_footer*? ?*-dest_file dest_file*? *name*
tepam::doc_gen::patch ?*-format format*? ?*-style style*? ?*-search_pattern search_pattern*? ?*-src_string src_string* | *-src_file src_file*? ?*-dest_file dest_file*? ?*name*?

Description
This package generates documentations of TEPAM procedures (procedures that have been declared with tepam::procedure). The documents are generated in the classic UNIX document style using the following document sections: Name, Synopsis, Description, Arguments and Example. **TEPAM Doc Gen** provides support for various document formats. Support for additional formats can be added if necessary.

The **TEPAM Doc Gen** package provides the following commands:

tepam::doc_gen::generate ?-format *format*? ?-style *style*? ?-header_footer? ?-dest_file *dest_file*? *name*
This command generates the documentation for a specified procedure (*name*) in one of the supported formats (TXT, HTML, POD (Perl Doc), DT (TclLib DocTool), or in a custom specific format. The format is specified via ?format?. The flag ?-header_footer? adds to the documentation file header and footer. If ?dest_file? is specified the documentation is stored in a file (the file header and footer are added automatically in this case) and the file name is returned. Otherwise the documentation string is returned by **generate**.

tepam::doc_gen::patch ?-format *format*? ?-style *style*? ?-search_pattern *search_pattern*? ?-src_string *src_string* | -src_file *src_file*? ?-dest_file *dest_file*? ?name?

This command inserts procedure documentations into an existing master document at the locations indicated by insertion placeholders which are matching the pattern of ?search_pattern?. The existing master document is either provided as data to the argument (?src_string?) or via a file (?src_file?). The final document is returned by **patch** if no destination file is defined (?dest_file?). Otherwise, the document is stored in the specified file, and the number of insertion placeholders that could be handled successfully is returned.

Any insertion placeholders of the master document are handled by default. By defining the argument ?name? the documentation insertion will be restricted to a particular procedure.

Arguments

?-format *format*?

Specifies the documentation format. **TEPAM Doc Gen** provides support for the following formats:
- TXT - Text format (default)
- HTML
- POD - Perl Plain Old Documentation format (PerlPOD)
- DT - TclLib DocTool format

Section Adding support for new document formats shows how support for additional formats can be added.

?-style *style*?

The documentation is by default generated in Tcl style (e.g. **command arg1 arg2 ...**). C-style documentation can be generated by setting this argument to 'C' (e.g. **command(arg1,arg2,...)**).

?-dest_file *dest_file*?

If ?dest_file? is defined the documentation is written into the specified destination file. Otherwise the documentation string is returned by the commands **generate** and **patch**.

***name* / ?name?**

This is the name of the procedure for which the documentation has to be generated. This is a mandatory argument for **generate**, but an optional argument for **patch**.

?-header_footer?

Generate adds to the generated procedure documentation the file header and footer only if a file is generated. By selecting the flag ?-header_footer? the header and footer are also generated if the documentation is returned as string by **generate**.

?-src_string *src_string* | -src_file *src_file*?

Patch inserts procedure documentations into an existing document that is either provided as string to the argument (?src_string?) or as a file (?src_file?). One of these two arguments need to be specified.

?-search_pattern *search_pattern*?

The argument ?search_pattern? defines the documentation insertion placeholder used in a document. It is a regular expression accepted by **regexp** and needs to contain a parenthesized sub-expression that contains the procedure name for which the documentation needs to be inserted.

The default insertion placeholder pattern is *\{!(.*?)!\}*, which means that the procedure name will be embedded between *{!* and*!}*. The section Examples contains a custom insertion placeholder pattern example.

Predefined document formats

TEPAM Doc Gen pre-defines the following document formats:

TXT - text format

The documentation will be generated in a simple text format if this format is selected. The format can be customized via the following variable:

tepam::doc_gen::Option(TXT,MaxLineLength)
Default: 80

This variable defines the line wrapping limit (character position).

HTML - HTML format

TEPAM Doc Gen generates CSS styled HTML files. The HTML documentation can be customized via the following variable:

tepam::doc_gen::Option(HTML,CssFile)
Default: "*tepam_doc_stylesheet.css*"

This variable specifies the CSS stylesheet file that is referred by the generated HTML files.

The CSS stylesheet can be customized to change the documentation formatting. A good starting point to create a customized CSS stylesheet is to use the CSS file provided by the **TEPAM Doc Gen** example/demo. The HTML documentation uses the following CSS class styles:

- *h1.tepam_page_title* - Document page title. Only used by **generate** if a file is created or if the header and footer are built (flag ?-header_footer? selected).
- *div.tepam_command_help* - Documentation container. The entire procedure documentation is placed inside this container.
- *p.tepam_section_title* - Section title (e.g. *Name*, *Synopsis*, *Description*, ...)
- *p.tepam_sub_section_title* - Sub-section title (used to separate the documentation of multiple sub-procedures)
- *p.tepam_name* - Name section
- *p.tepam_synopsis* - Synopsis section
- *p.tepam_description* - Single description paragraph
- *ul.tepam_description_list* - Item of a HTML bulleted/unordered list inside the description section
- *dt.tepam_argument* - Item of a HTML description list used to list the procedure arguments
- *p.tepam_argument_description* - Argument description paragraph
- *p.tepam_argument_attribute* - Argument attribute line
- *pre.tepam_example* - Example section

POD - Perl document format

The documentation is generated in the Perl Plain Old Documentation format (PerlPOD) if this format is selected.

Dt - TclLib doctools format

The documentation is generated in the Tcllib DocTools format if this format is selected.

Adding support for new document formats

Support for a new document format can be added by defining in the **tepam::doc_gen** namespace a set of procedures that generate the different document components.

The following documentation listing contains tokens that refer to the different document generation procedures:

```
     <01>
<03> <20s>   NAME<20e>
     <30s>       message_box - Displays text in a message box<30e>
     <20s>   SYNOPSYS<20e>
     <40s>       message_box [-mtype <mtype>] <text><40e>
     <20s>   DESCRIPTION<20e>
     <21s>     message_box<21e>
     <54s>       message_box [-mtype <mtype>] <text><54e>
     <50s>       This procedure allows displaying a text in an
   message box. The following
                 message types are supported:<50e>
<51> <53s>       * Info<53e>
     <53s>       * Warning<53e>
     <53s>       * Error<53e>
     <52>
     <50s>       If the text parameter is use multiple times the
   different texts are
                 concatenated to create the message text.<50e>
     <20s>   ARGUMENTS<20e>
<60> <62s>       [-mtype <mtype>]<62e>
<63> <65s>          Message type<65e>
     <66s>          Default: "Warning"<66e>
     <66s>          Multiple: yes<66e>
     <66s>          Choices: Info, Warning, Error<66e>
     <64>
     <62s>       <text><62e>
<63> <65s>          One or multiple text lines to display<65e>
     <66s>          Type: string<66e>
     <66s>          Multiple: yes<66e>
     <64><61>
     <20s>   EXAMPLE<20e>
<70> <72s>       message_box "Please save first the document"<72e>
     <73s>       -> 1<73e>
     <71><04>
     <02>
```

There are 2 types of document generation procedures:

Content generation procedures (e.g. <40s>...<40e>)

These procedures generate some document content based on the text that is provided as procedure argument. The listing above shows two tokens for these procedures to indicate the beginning and the end of the generated content.

Control generation procedures (e.g. <03>)
These procedures generate control constructs, for example to generate the prolog code and epilog code for lists, sections, etc. These procedures have no argument.

The following set of procedures needs to be defined to provide support for a new document format:

***01* - gen($Format,Header) {*Text*}**
Only called if **doc_gen** generates a file or if it is called with the flag ?-header_footer?. The procedure creates the file header. The provided parameter is the procedure name for which the documentation has to be generated.

***02* - gen($Format,Footer) {*Text*}**
Only called if **doc_gen** generates a file or if it is called with the flag ?-header_footer?. The procedure creates the file footer.

***03* - gen($Format,Begin) {}**
Generates the documentation prolog (preamble)

***04* - gen($Format,End) {}**
Generates the documentation epilog

***20* - gen($Format,SectionTitle) {*Text*}**
Generates a section title (e.g. *Name*, *Synopsis*, *Description*, ...). The raw title text is provided as parameter

***21* - gen($Format,SubSectionTitle) {*Text*}**
Generates a sub-section title. Sub-sections are used if a single documentation is generated for multiple sub-commands to make a separation between them. The raw title text is provided as parameter

***30* - gen($Format,Name) {*Text*}**
Generates the name section (without title). The raw section text is provided as parameter.

***40* - gen($Format,Synopsis) {*Text*}**
Generates the synopsis section (without title). The section text provided as parameter is pre-formatted (the argument strings are generated by **gen($Format,ArgumentString)**).

***50* - gen($Format,Description) {*Text*}**
Generates a description paragraph. The raw paragraph text is provided as parameter.

***51* - gen($Format,DescriptionListBegin) {}**
Generates the prolog of a bulleted/unordered list inside the description section. This prolog is usually the start code of a list structure.

***52* - gen($Format,DescriptionListEnd) {}**
Generates the epilog of a bulleted/unordered list inside the description section. This epilog is usually the end code of a list structure.

***53* - gen($Format,DescriptionListItem) {*Text*}**
Generates a text item in a bulleted/unordered description list. The raw item text is provided as parameter.

***54* - gen($Format,DescriptionSynopsis) {*Text*}**
Generates the synopsis line on the beginning of the description section. The command can

return an empty string if no synopsys line is required at this place.

Some formats (e.g. Tcl DocTools) require that the synopsis line is defined in the description section, to build then automatically the synopsis section. The section text provided as parameter is pre-formatted (the argument strings are generated by **gen($Format,ArgumentString)**).

***60* - gen($Format,ArgumentListBegin) {}**
Generates the prolog of argument list (definition/non-bulleted list). This prolog is usually the start code of a definition list.

***61* - gen($Format,ArgumentListEnd) {}**
Generates the epilog of the argument list. This epilog is usually the end string of a list structure.

***62* - gen($Format,ArgumentListItem) {Name IsOptional IsNamed Type}**
Generates an argument item line inside the argument list. This command can rely on **gen($Format,ArgumentDetailBegin)** since the parameters are identical.

***63* - gen($Format,ArgumentDetailBegin) {}**
Generates the argument details prolog (preamble).

***64* - gen($Format,ArgumentDetailEnd) {}**
Generates the argument details epilog

***65* - gen($Format,ArgumentDescription) {*Text*}**
Generates the argument description (single paragraph).

***66* - gen($Format,ArgumentAttribute) {*Text*}**
Generates a single argument attribute string. The command is called individually for each attribute.

***70* - gen($Format,ExampleBegin) {}**
Generates the example section prolog (preamble)

***71* - gen($Format,ExampleEnd) {}**
Generates the example section epilog

***72* - gen($Format,ExampleCommandLine) {*Text*}**
Generates a single command line in the example section. The command is called for each individual command line.

***73* - gen($Format,ExampleResultLine) {*Text*}**
Generates a command result line

***80* - gen($Format,ArgumentString) {Name IsOptional IsNamed Type}**
Generates the part of the command line or the synopsis that is specific to an argument. The generated string has to indicate if an argument is optional, named and if it is a flag.

The following parameters are provided to this procedure:

Name
Name of the argument

IsOptional
If true (=**1**) the argument is optional which should be indicated by the generated string (for example by putting the argument into brackets {[]} or into question marks '?'):

```
gen(TXT,ArgumentString) mtype 1 0 string -> "[mtype]"
```

IsNamed
If true (=**1**) an argument is a named argument (option). The generated string should in this case contain the argument/option name, followed by the argument itself:

```
gen(TXT,ArgumentString) mtype 0 1 string -> "-mtype <mtype>"
```

Named arguments can also be optional:

```
gen(TXT,ArgumentString) mtype 1 1 string -> "[-mtype <mtype>]"
```

Type
Indicates the type of the argument. If the type is set to **none** the argument is a flag, which needs to be indicated by the generated string. Example:

```
gen(TXT,ArgumentString) close 1 1 none -> "[-close]"
```

Examples

Tepam::doc_gen::generate

The **TEPAM Doc Gen** package can be explored by generating the documentation of the command **tepam::doc_gen::generate**. The following example generates the document in text format (default format):

```
tepam::doc_gen::generate tepam::doc_gen::generate
```

The next example generates the documentation in HTML format:

```
tepam::doc_gen::generate -format HTML tepam::doc_gen::generate
```

The flag ?header_footer? adds also the file header and footer:

```
tepam::doc_gen::generate -format HTML -header_footer
    tepam::doc_gen::generate
```

The documentation can directly be stored in a file. The file header and footer are automatically generated in this way:

```
tepam::doc_gen::generate -format HTML -dest_file doc_gen.html
    tepam::doc_gen::generate
```

The generated HTML file refers a CSS stylesheet file (default: tepam_doc_stylesheet.css). To display the HTML file correctly this CSS stylesheet file needs to be copied into the directory of the generated HTML file.

The Tcl DOC Tools format can be used as intermediate format to generate other formats, for example HTML:

```
# Generate the documentation in Tcl Doc Tool format
set dt [tepam::doc_gen::generate -format DT -header_footer
    tepam::doc_gen::generate]

# Create a new doc tools object (HTML format)
```

```
package require doctools
::doctools::new myDoc -format html

# Open the HTML file, and write the HTML formatted documentation
set fHtml [open doc_gen.dt.html w]
puts $fHtml [myDoc format $dt]
close $fHtml
```

Tepam::doc_gen::patch

While **generate** provides a limited number of possibilities to vary the document structure, **patch** offers more flexibility. Multiple documentations for different procedures and meta information can for example be added.

The following listing shows how the **patch** command works. It defines first a HTML master document string that contains 2 procedure documentation placeholders (*{**}*). There placeholders are replaced by **patch** with the generated documentation of the referred procedures. Since nonstandard placeholders are used, **patch** is called with an explicit placeholder pattern definition (argument *search_pattern*).

```
# Define the HTML master document
set HtmlMasterDoc {\
<html>
  <head>
    <title>tepam::doc_gen</title>
    <link rel="stylesheet" href="tepam_doc_stylesheet.css">
    <meta content="documentation" name="keywords"></meta>
  </head>
  <body>
    <h1>tepam::doc_gen</h1>
      <h2>Generate</h2>
{*tepam::doc_gen::generate*}
      <h2>Patch</h2>
{*tepam::doc_gen::patch*}
  </body>
<html>\
}

# Patch the master document: This will replace the placeholders by the
# procedure documentation divisions:
tepam::doc_gen::patch -format HTML -search_pattern {\{\*(.*?)\*\}} \
                      -src_string $HtmlMasterDoc -dest_file
    tepam_doc_gen.html
```

Category

Documentation tools

Copyright

Copyright (c) 2013, Andreas Drollinger

tepam_introduction

Keywords: tepam_introduction, argument integrity, argument validation, arguments, entry mask, parameter entry form, procedure, subcommand

Name

tepam - An introduction into TEPAM, Tcl's Enhanced Procedure and Argument Manager

Description

This document is an informal introduction into TEPAM, the Tcl's Enhanced Procedure and Argument Manager. Detailed information to the TEPAM package is provided in the *tepam::procedure* and *tepam::argument_dialogbox* reference manuals.

Overview

This package provides a new Tcl procedure declaration syntax that simplifies the implementation of procedure subcommands and the handling of the different types of procedure arguments like flags or switches, options, unnamed arguments, optional and mandatory options and arguments, default values, etc. Procedure declarations can be enriched with detailed information about the procedure and its arguments. This information is used for the following purposes:

First of all, a preamble is added in front of the body of a procedure that is declared with TEPAM. This preamble calls an argument manager that that uses the provided information to check the validity of the argument types and values before the procedure body is executed. Then, the information is used to generate help and usage texts if requested, or to generate clear error message in case an argument validation fails. The information also allows generating automatically graphical forms that allows an interactive definition of all arguments, in case a procedure is called interactively. And finally, the additional information helps self-commenting in a clean way the declaration of a procedure and of all its arguments.

The graphical form generator that creates the necessary argument specification forms for the interactive procedure calls is also available for other purposes than for procedure argument specifications. It allows creating code efficiently complex parameter entry forms that are usable independently from TEPAM's new procedure definition method.

Here is a short overview about all major TEPAM features:

- New self-documenting procedure declaration syntax: The additional information to declare properly a procedure has not to be provided with additional statements, but can be added in a natural syntax directly into the procedure header.
- Easy way to specify subcommands: A subcommand is declared like a procedure, simply with a procedure name composed by a base name followed by a subcommand name. Sub-subcommands are created identically using simply procedure names composed by 3 words.
- Flexible usage of flags (switches), options (named arguments) and unnamed arguments. Option names are optionally automatically completed.
- Support for default values, mandatory/optional options and arguments, choice lists, value ranges, multiple usable options/arguments.
- Choice of a *named arguments first, unnamed arguments later* procedure calling style (typical

for Tcl commands) or of an *unnamed arguments first, named arguments later* procedure calling style (typical for Tk commands).
- In case the *named arguments first, unnamed arguments later* style (Tcl) is selected: Clear separation between options and arguments via the "--" flag. The unnamed arguments can optionally be accessed as options (named arguments).
- Automatic type and value check before the procedure body is executed, taking into account validation ranges, choice lists and custom validation commands. Generation of clear error message if necessary.
- Many predefined types exist (integer, boolean, double, color, file, font, ...). Other application specific types can easily be added.
- Automatic help and usage text generation if a procedure is called with the *-help* flag.
- Automatic generation of an interactive argument definition form, in case a procedure is called with the *-interactive* flag.
- Procedure calls can be logged which is useful to get for interactively called procedures the command call lines.
- Powerful and code efficient generation of complex parameter definition forms.

Procedure declaration

TEPAM's procedure declaration syntax is simple and self-explaining. Instead of declaring a procedure with the Tcl key word **proc**, a procedure is declared with the TEPAM command **procedure** which takes as **proc** also 3 arguments: The procedure name, the procedure header and the procedure body.

The following example declares the subcommand **message** of the procedure **display**. This command has several named and unnamed arguments:

```
tepam::procedure {display message} {
   -return            -
   -short_description "Displays a simple message box"
   -description       "This procedure allows displaying a configurable
    message box."
   -args {
      {-mtype -default Warning -choices {Info Warning Error}
    -description "Message type"}
      {-font -type font -default {Arial 10 italic} -description
    "Message text font"}
      {-level -type integer -optional -range {1 10} -description
    "Message level"}
      {-fg -type color -default black -description "Message color"}
      {-bg -type color -optional -description "Background color"}
      {-no_border -type none -description "Use a splash window style
    (no border)"}
      {-log_file -type file -optional -description "Optional message
    log file"}
      {text -type string -multiple -description "Multiple text lines
    to display"}
   }
```

```
} {
   puts "display message:"
   foreach var {mtype font level fg bg no_border log_file text} {
      if {[info exists $var]} {
         puts  "  $var=[set $var]"
      }
   }
}
```

A call of procedure that has been declared in this way will first invoke the TEPAM argument manager, before the procedure body is executed. The argument manager parses the provided arguments, validates them, completes them eventually with some default values, and makes them finally available to the procedure body as local variables. In case an argument is missing or has a wrong type, the argument manager generates an error message that explains the reason for the error.

As the example above shows, the TEPAM command **procedure** accepts subcommand definitions as procedure name and allows defining much more information than just the argument list inside the procedure header. The procedure body on the other hand is identical between a command declared with **proc** and a command declared with **procedure**.

The procedure header allows defining in addition to the arguments some procedure attributes, like a description, information concerning the return value, etc. This information is basically used for the automatic generation of comprehensive help and usage texts.

A list of argument definition statements assigned to the *-args* argument is defining the procedure arguments. Each argument definition statement starts with the argument name, optionally followed by some argument attributes.

Three types of arguments can be defined: Unnamed arguments, named arguments and flags. The distinction between the named and unnamed arguments is made by the first argument name character which is simply "-" for named arguments. A flag is defined as named argument that has the type *none*.

Named and unnamed arguments are mandatory, unless they are declared with the *-optional* flag and unless they have a default value specified with the *-default* option. Named arguments and the last unnamed argument can have the attribute *-multiple*, which means that they can be defined multiple times. The expected argument data type is specified with the *-type* option. TEPAM defines a large set of standard data types which can easily be completed with application specific data types.

The argument declaration order has only an importance for unnamed arguments that are by default parsed after the named arguments (Tcl style). A variable allows changing this behavior in a way that unnamed arguments are parsed first, before the named arguments (Tk style).

Procedure help

The declared procedure can simply be called with the *-help* option to get the information about the usage of the procedure and its arguments:

```
display message -help
  ->
NAME
      display message - Displays a simple message box
SYNOPSYS
      display message
            [-mtype <mtype>] :
               Message type, default: "Warning", choices: {Info
    Warning Error}
            [-font <font>] :
               Message text font, type: font, default: Arial 10 italic
            [-level <level>] :
               Message level, type: integer, range: 1..10
            [-fg <fg>] :
               Message color, type: color, default: black
            [-bg <bg>] :
               Background color, type: color
            [-no_border ] :
               Use a splash window style (no border)
            [-log_file <log_file>] :
               Optional message log file, type: file
            <text> :
               Multiple text lines to display, type: string
DESCRIPTION
      This procedure allows displaying a configurable message box.
```

Procedure call

The specified procedure can be called in many ways. The following listing shows some valid procedure calls:

```
display message "The document hasn't yet been saved!"
-> display message:
     mtype=Warning
     font=Arial 10 italic
     fg=black
     no_border=0
     text={The document hasn't yet been saved!}

display message -fg red -bg black "Please save first the document"
-> display message:
     mtype=Warning
     font=Arial 10 italic
     fg=red
     bg=black
     no_border=0
     text={Please save first the document}
```

```
display message -mtype Error -no_border "Why is here no border?"
-> display message:
     mtype=Error
     font=Arial 10 italic
     fg=black
     no_border=1
     text={Why is here no border?}

display message -font {Courier 12} -level 10 \
   "Is there enough space?" "Reduce otherwise the font size!"
-> display message:
     mtype=Warning
     font=Courier 12
     level=10
     fg=black
     no_border=0
     text={Is there enough space?} {Reduce otherwise the font size!}
```

The next lines show how wrong arguments are recognized. The *text* argument that is mandatory is missing in the first procedure call:

```
display message -font {Courier 12}
  -> display message: Required argument is missing: text
```

Only known arguments are accepted:

```
display message -category warning Hello
  -> display message: Argument '-category' not known
```

Argument types are automatically checked and an error message is generated in case the argument value has not the expected type:

```
display message -fg MyColor "Hello"
  -> display message: Argument 'fg' requires type 'color'.  Provided
    value: 'MyColor'
```

Selection choices have to be respected ...

```
display message -mtype Fatal Hello
  -> display message: Argument (mtype) has to be one of the  following
    elements: Info, Warning, Error
```

... as well as valid value ranges:

```
display message -level 12 Hello
  -> display message: Argument (level) has to be between 1 and 10
```

Interactive procedure calls

The most intuitive way to call the procedure is using an form that allows specifying all

arguments interactively. This form will automatically be generated if the declared procedure is called with the *-interactive* flag. To use this feature the Tk library has to be loaded.

```
display message -interactive
```

The generated form contains for each argument a data entry widget that is adapted to the argument type. Check buttons are used to specify flags, radio boxes for tiny choice lists, disjoint list boxes for larger choice lists and files, directories, fonts and colors can be selected with dedicated browsers.

After acknowledging the specified argument data via an OK button, the entered data are first validated, before the provided arguments are transformed into local variables and the procedure body is executed. In case the entered data are invalid, a message appears and the user can correct them until they are valid.

The procedure calls can optionally be logged in a variable. This is for example useful to get the command call lines of interactively called procedures.

Flexible argument dialog box

The form generator that creates in the previous example the argument dialog box for the interactive procedure call is also available for other purposes than for the definition of procedure arguments. If Tk has been loaded TEPAM provides and argument dialog box that allows creating complex parameter definition forms in a very efficient way.

The following example tries to illustrate the simplicity to create complex data entry forms. It creates an input mask that allows specifying a file to copy, a destination folder as well as a checkbox that allows specifying if an eventual existing file can be overwritten. Comfortable browsers can be used to select files and directories. And finally, the form offers also the possibility to accept and decline the selection. Here is the code snippet that is doing all this:

```
tepam::argument_dialogbox \
   -existingfile {-label "Source file" -variable SourceFile} \
   -existingdirectory {-label "Destination folder" -variable DestDir} \
   -checkbutton {-label "Overwrite existing file" -variable Overwrite}
```

The **argument_dialogbox** returns **ok** if the entered data are validated. It will return **cancel** if the data entry has been canceled. After the validation of the entered data, the **argument_dialogbox** defines all the specified variables with the entered data inside the calling context.

An **argument_dialogbox** requires a pair of arguments for each variable that it has to handle. The first argument defines the entry widget type used to select the variable's value and the second one is a lists of attributes related to the variable and the entry widget.

Many entry widget types are available: Beside the simple generic entries, there are different kinds of list and combo boxes available, browsers for existing and new files and directories, check and radio boxes and buttons, as well as color and font pickers. If necessary, additional entry widgets can be defined.

The attribute list contains pairs of attribute names and attribute data. The primary attribute is *-variable* used to specify the variable in the calling context into which the entered data has to be stored. Another often used attribute is *-label* that allows adding a label to the data entry widget. Other attributes are available that allow specifying default values, the expected data types, valid

data ranges, etc.

The next example of a more complex argument dialog box provides a good overview about the different available entry widget types and parameter attributes. The example contains also some formatting instructions like *-frame* and *-sep* which allows organizing the different entry widgets in frames and sections:

```
set ChoiceList {"Choice 1" "Choice 2" "Choice 3" "Choice 4" "Choice 5"
    "Choice 6"}

set Result [tepam::argument_dialogbox \
   -title "System configuration" \
   -context test_1 \
   -frame {-label "Entries"} \
      -entry {-label Entry1 -variable Entry1} \
      -entry {-label Entry2 -variable Entry2 -default "my default"} \
   -frame {-label "Listbox & combobox"} \
      -listbox {-label "Listbox, single selection" -variable Listbox1 \
                -choices {1 2 3 4 5 6 7 8} -default 1 -height 3} \
      -listbox {-label "Listbox, multiple selection" -variable Listbox2
                -choicevariable ChoiceList -default {"Choice 2"
    "Choice 3"}
                -multiple_selection 1 -height 3} \
      -disjointlistbox {-label "Disjoined listbox" -variable
    DisJntListbox
                         -choicevariable ChoiceList \
                         -default {"Choice 3" "Choice 5"} -height 3} \
      -combobox {-label "Combobox" -variable Combobox \
                 -choices {1 2 3 4 5 6 7 8} -default 3} \
   -frame {-label "Checkbox, radiobox and checkbutton"} \
      -checkbox {-label Checkbox -variable Checkbox
                 -choices {bold italic underline} -choicelabels {Bold
    Italic Underline} \
                 -default italic} \
      -radiobox {-label Radiobox -variable Radiobox
                 -choices {bold italic underline} -choicelabels {Bold
    Italic Underline} \
                 -default underline} \
      -checkbutton {-label CheckButton -variable Checkbutton -default
    1} \
   -frame {-label "Files & directories"} \
      -existingfile {-label "Input file" -variable InputFile} \
      -file {-label "Output file" -variable OutputFile} \
      -sep {} \
      -existingdirectory {-label "Input directory" -variable
    InputDirectory} \
      -directory {-label "Output irectory" -variable OutputDirectory} \
```

```
   -frame {-label "Colors and fonts"} \
      -color {-label "Background color" -variable Color -default red} \
      -sep {} \
      -font {-label "Font" -variable Font -default {Courier 12
    italic}}]
```

The **argument_dialogbox** defines all the specified variables with the entered data and returns **ok** if the data have been validated via the Ok button. If the data entry is cancelled by activating the Cancel button, the **argument_dialogbox** returns **cancel**.

```
if {$Result=="cancel"} {
   puts "Canceled"
} else { # $Result=="ok"
   puts "Arguments: "
   foreach Var {
      Entry1 Entry2
      Listbox1 Listbox2 DisJntListbox
      Combobox Checkbox Radiobox Checkbutton
      InputFile OutputFile InputDirectory OutputDirectory
      Color Font
   } {
      puts "  $Var: '[set $Var]'"
   }
}
-> Arguments:
   Entry1: 'Hello, this is a trial'
   Entry2: 'my default'
   Listbox1: '1'
   Listbox2: '{Choice 2} {Choice 3}'
   DisJntListbox: '{Choice 3} {Choice 5}'
   Combobox: '3'
   Checkbox: 'italic'
   Radiobox: 'underline'
   Checkbutton: '1'
   InputFile: 'c:\tepam\in.txt'
   OutputFile: 'c:\tepam\out.txt'
   InputDirectory: 'c:\tepam\input'
   OutputDirectory: 'c:\tepam\output'
   Color: 'red'
   Font: 'Courier 12 italic'
```

Category

Procedures, arguments, parameters, options

Copyright

Copyright (c) 2009-2013, Andreas Drollinger

tepam_procedure

Keywords: tepam_procedure, argument integrity, argument validation, arguments, procedure, subcommand

Name

tepam::procedure - TEPAM procedure, reference manual

Synopsis

package require Tcl 8.5 9
package require tepam ?**0.5**?
tepam::procedure *name attributes body*

Description

This package provides an alternative way to declare Tcl procedures and to manage its arguments. There is a lot of benefit to declare a procedure with TEPAM rather than with the Tcl standard command **proc**: TEPAM allows specifying inside the procedure declaration all information that is required to generate comprehensive documentations and help support. The information is also used by an automatically invoked argument checker that validates the provided procedure arguments before the procedure body is executed. Finally, a procedure can be called interactively which will open a graphical form that allows specifying the procedure arguments.

TEPAM simplifies also the handling of the different types of argument, like the *named arguments* (often also called *options*) and the *unnamed arguments*. TEPAM supports the *named first, unnamed later* style (typical Tcl command style) as well as also the *unnamed first, named later* style (typical Tk command style). TEPAM takes care about default values for arguments, optional arguments, multiple applicable arguments, etc. and eliminates the need to check the validity of the argument inside the procedure bodies.

An informal overview of all the TEPAM procedure declaration and calling features as well as a short introduction into TEPAM is provided by *tepam(n)*.

Terminology

The exact meaning of several terms that are used in this document will be shortly explained to avoid any ambiguities and misunderstandings.

Subcommand

The usage of subcommands is heavily used in the Tcl language. Several commands are incorporated into a single main command and are selectable via the first argument.

The **string** command is an example of such a command that implements for example subcommands to check a character string length, to compare strings, to extract substrings, etc:

```
string length string
string compare string string
string range string first last
...
```

TEPAM provides a framework that allows implementing easily such subcommands in form of

Tcl procedures. It allows not only defining a first level of subcommands, but also a higher level of subcommands. The **string** command class check could be implemented as independent sub-sub-commands of the **string** command:

```
string is alnum string
string is integer string
string is double string
...
```

Procedure attribute
TEPAM allows attaching to a declared procedure different kind of attributes. Some of these attributes are *just* used for documentation purposes, but other attributes specify the way how the procedure has to be called. Also the procedure arguments are defined in form of a procedure attribute.

Argument
TEPAM uses the term *argument* for the parameters of a procedure.

The following example calls the subcommand **string compare** with several arguments:

```
string compare -nocase -length 3 "emphasized" "emphasised"
```

The following paragraphs discuss these different argument types.

Named argument
Some parameters, as *-length 3* of the subcommand **string compare** have to be provided as pairs of argument names and argument values. This parameter type is often also called *option*.

TEPAM uses the term *named argument* for such options as well as for the flags (see next item).

Flag, switch
Another parameter type is the *flag* or the *switch*. Flags are provided simply by naming the flag leading with the '-' character. The *-nocase* of the previous **string compare** example is such a flag.

Flags are considered by TEPAM like a special form of *named arguments*.

Unnamed argument
For the other parameters, e.g. the ones for which the argument name has not to be mentioned, TEPAM uses the term *unnamed argument*. The previous **string compare** example uses for the two provided character strings two *unnamed arguments*.

Argument attribute
TEPAM allows describing the purpose of each procedure argument with *argument attributes*. While some of them are just documenting the attributes, most attributes are used by an argument manager to control and validate the arguments that are provided during a procedure call. Argument attributes are used to specify default values, parameter classes (integer, xdigit, font, ...), choice validation lists, value ranges, etc.

Named arguments first, unnamed arguments later
The **string compare** command of the previous example requires that the *named arguments*

(options, flags) are provided first. The two mandatory (unnamed) arguments have to be provided as last argument.

```
string compare -nocase -length 3 Water $Text
```

This is the usual Tcl style (exceptions exist) which is referred in the TEPAM documentation as *named arguments first, unnamed arguments later style*.

Unnamed arguments first, named arguments later

In contrast to most Tcl commands, Tk uses generally (exceptions exist also here) a different calling style where the *unnamed arguments* have to be provided first, before the *named arguments* have to be provided:

```
pack .ent1 .ent2 -fill x -expand yes -side left
```

This style is referred in the TEPAM documentation as *unnamed arguments first, named arguments later style*.

Procedure declaration

TEPAM allows declaring new Tcl procedures with the command **tepam::procedure** that has similar to the standard Tcl command **proc** also 3 arguments:

tepam::procedure *name attributes body*

see below ...

The TEPAM procedure declaration syntax is demonstrated by the following example:

```
tepam::procedure {display message} {
   -short_description
      "Displays a simple message box"
   -description
      "This procedure allows displaying a configurable\
       message box. The default message type that is\
       created is a warning, but also errors and info can\
       be generated.
       The procedure accepts multiple text lines."
   -example
      {display message -mtype Warning "Save first your job"}
   -args {
      {-mtype -choices {Info Warning Error} \
              -default Warning -description "Message type"}
      {text   -type string -multiple \
              -description "Multiple text lines to display"}
   }
} {
   puts "Message type: $mtype"
   puts "Message: $text"
}
```

The 3 arguments of **procedure** are:

name

The procedure name can be used in very flexible ways. Procedure names can have namespace qualifiers. By providing a two element name list as procedure name, a subcommand of a procedure will be declared. It is even possible to declare sub-sub-commands of a procedure by providing name lists with three elements.

Here are some valid procedure declarations using different procedure names (the attribute and body arguments are empty for simplicity):

```
# Simple procedure name:
tepam::procedure display_message {} {}

# Procedure declared in the main namespace:
tepam::procedure ::display_message {} {}

# Procedure in the namespace ::ns:
tepam::procedure ::ns::display_message {} {}

# Declaration of the subcommand message of the procedure display:
tepam::procedure {display message} {} {}
```

attributes

All procedure attributes are provided in form of an option list that contains pairs of option names and option values. The example above has as procedure attribute a short and a normal description, but also the procedure arguments are defined in form of a procedure attribute.

Most procedure attributes are providing information for documentation purposes. But some of them affect also the way how the procedure can be called. The section Procedure attributes discusses in detail the available procedure attributes.

The procedure arguments are defined in form of a special procedure attribute. Most of the information provided in the argument definition is not just used for documentation purposes. This information is in fact used by the TEPAM argument manager to handle and validate the various forms of arguments that are provided during the procedure calls. The section Argument declaration discusses in detail all the argument definition attributes.

body

This is the normal procedure body. The declared arguments will be available to the procedure body in form of variables.

The procedure body will only be executed if the provided set of arguments could be validated by the TEPAM argument manager.

```
tepam::procedure {display_message} {
   -args {
      {-mtype -default Warning -choices {Warning Error}}
      {text -type string}
   }
```

```
} {
   puts "Message type: $mtype"
   puts "Message: $text"
}
```

The commands **procedure** as well as **argument_dialogbox** are exported from the namespace tepam. To use these commands without the **tepam::** namespace prefix, it is sufficient to import them into the main namespace:

```
namespace import tepam::*

procedure {display_message} {
   -args {
      ...
```

Procedure attributes

The first group of attributes affect the behavior of the declared procedure:

-named_arguments_first 0|1
: This attribute defines the calling style of a procedure. TEPAM uses by default the *named arguments first, unnamed arguments later* style (Tcl). This default behavior can globally be changed by setting the variable **tepam::named_arguments_first** to **0**. This global calling style can be changed individually for a procedure with the *-named_arguments_first* attribute.

-auto_argument_name_completion 0|1
: The declared procedures will by default automatically try to match eventually abbreviated argument names to the defined arguments names. This default behavior can globally be changed by setting the variable **tepam::auto_argument_name_completion** to **0**. This global setting of the automatic argument name completion can be changed individually for a procedure with the *-auto_argument_name_completion* procedure attribute.

-interactive_display_format extended|short
: A procedure declared with the TEPAM **procedure** command can always be called with the **-interactive** option. By doing so, a graphical form will be generated that allows specifying all procedure argument values. There are two display modes for these interactive forms. While the *extended* mode is more adapted for small procedure argument sets, the **short** form is more adequate for huge procedure argument sets.

  The choice to use short or extended forms can be globally configured via the variable **tepam::interactive_display_format**. This global setting can then be changed individually for a procedure with the *-interactive_display_format* procedure attribute.

-args *list*
: The procedure arguments are declared via the *-args* attribute. An argument is defined via a list having as first element the argument name, followed by eventual argument attributes. All these argument definition lists are packaged themselves into a global list that is assigned to the *-args* attribute.

  The argument definition syntax will be described more in detail in the following sub section.

The next attributes allow specifying custom argument checks as well as custom error messages in case these checks are failing:

-validatecommand *script*
Custom argument validations can be performed via specific validation commands that are defined with the *-validatecommand* attribute.

Validation command declaration example:

```
tepam::procedure {display_message} {
   -args {
      {text -type string -description "Message text"} }
   -validatecommand {IllegalWordDetector $text}
} {
}
```

The validation command is executed in the context of the declared procedure body. The different argument values are accessed via the argument names. Note there is also an argument attribute *-validatecommand* that allows declaring custom checks for specific arguments.

The attribute *-validatecommand* can be repeated to declare multiple custom checks.

-validatecommand_error_text *string*
This attribute allows overriding the default error message for a custom argument validation (defined by *-validatecommand*). Also this attribute can be repeated in case multiple argument checks are declared.

The following attribute allows controlling the logging settings for an individual procedure:

-command_log 0|1|"interactive"
This argument configures the logging of the procedure calls into the list variable **tepam::ProcedureCallLogList**. The default configuration defined by the variable **tepam::command_log** will be used if this argument is not defined in a procedure declaration.

Setting this argument to **0** will disable any procedure call loggings, setting it to **1** will log any procedure calls and setting it to **interactive** will log just the procedures that are called interactively (procedures called with the **-interactive** flag).

The next group of procedure attributes is just used for the purpose of documentation and help text generation:

-category *string*
A category can be assigned to a procedure for documentation purposes. Any string is accepted as category.

-short_description *string*
The short description of a procedure is used in the documentation summary of a generated procedure list as well as in the NAME section of a generated procedure manual page.

-description *string*
The (full) description assigned to a procedure is used to create user manual and help pages.

-return *string*

The *-return* attribute allows defining the expected return value of a procedure (used for documentation purposes).

-example *string*

A help text or manual page of a procedure can be enriched with eventual examples, using the *-example* attribute.

Argument declaration

The following example shows the structure that is used for the argument definitions in the context of a procedure declaration:

```
tepam::procedure {display_message} {
   -args {
      {-mtype -default Warning -choices {Info Warning Error}
    -description "Message type"}
      {-font -type font -default {Arial 10 italic} -description
    "Message text font"}
      {-level -type integer -optional -range {1 10} -description
    "Message level"}
      {-fg -type color -optional -description "Message color"}
      {-log_file -type file -optional -description "Optional message
    log file"}
      {text -type string -multiple -description "Multiple text lines
    to display"}
   }
} {
}
```

Each of the procedure arguments is declared with a list that has as first element the argument name, followed by eventual attributes. The argument definition syntax can be formalized in the following way:

```
tepam::procedure <name> {
   -args {
      {<argument_name_1> <arg_attr_name_1a> <arg_attr_value_1a>
    <arg_attr_name_1b> <arg_attr_value_1b> ...}
      {<argument_name_2> <arg_attr_name_2a> <arg_attr_value_2a>
    <arg_attr_name_2b> <arg_attr_value_2b> ...}
      ...
   }
} <body>
```

The argument names and attributes have to be used in the following way:

Argument name (>)

The provided argument name specifies whether the argument is an *unnamed argument* or a *named argument*. In addition to this, an argument name can also be blank to indicate an argument comment, or it can start with # to indicate a section comment.

""

This is the simplest form of an argument name: An argument whose name is not starting

with '-' is an *unnamed argument*. The parameter provided during a procedure call will be assigned to a variable with the name.

```
tepam::procedure {print_string} {
   -args {
      {text -type string -description "This is an unnamed
    argument"}
   }
} {
   puts $text
}

print_string "Hello"
 -> Hello
```

"-"

An argument whose name starts with '-' is a *named argument* (also called *option*). The parameter provided during a procedure call will be assigned to a variable with the name (not -).

```
tepam::procedure {print_string} {
   -args {
      {-text -type string -description "This is a named argument"}
   }
} {
   puts $text
}

print_string -text "Hello"
 -> Hello
```

"--"

This flag allows clearly specifying the end of the named arguments and the beginning of the unnamed arguments, in case the *named arguments first, unnamed arguments later style (Tcl)* has been selected.

If the *unnamed arguments first, named arguments later style (Tk)* style is selected, this flag is ignored if the unnamed arguments have already been parsed. Otherwise it will be assigned to the corresponding unnamed argument.

"-" or ""

A blank argument name (either '-' or '') starts a comment for the following arguments.

```
tepam::procedure {print_time} {
   -interactive_display_format short
   -args {
      {hours -type integer -description "Hour"}
      {minutes -type integer -description "Minute"}
```

```
      {- The following arguments are optional:}
      {seconds -type integer -default 0 -description "Seconds"}
      {milliseconds -type integer -default 0 -description
    "Milliseconds"}
   }
} {
   puts "${hour}h${minutes}:[expr $seconds+0.001*$milliseconds]"
}
```

Argument comments are basically used in the graphical argument definition forms that are created if a procedure is called interactively.

"#*"

An argument definition list that starts with '#' is considered as a section comment. The argument definition list will be trimmed from the '#' characters and the remaining string will be used as section comment.

Section comments can be used to structure visually the argument definition code. Section comments are also used to structure the generated help texts and the interactive argument definition forms.

```
tepam::procedure {complex_multiply} {
   -description "This function perform a complex multiplication"
   -args {
      {#### First complex number ####}
      {-r0 -type double -description "First number real part"}
      {-i0 -type double -description "First number imaginary part"}

      {#### Second complex number ####}
      {-r1 -type double -description "Second number real part"}
      {-i1 -type double -description "Second number imaginary
    part"}
   }
} {
   return [expr $r0*$r1 - $i0*$i1]
}
```

Argument attributes (> >)

The following argument attributes are supported:

-description *string*

The description argument attribute is used for documentation purpose. Interactive argument definition forms use this attribute to provide explanations for an argument.

-type *type*

The type argument attribute allows assigning the argument either to a predefined data type, or to an application specific data type. The argument values that are provided during a procedure call are automatically checked with respect to the defined argument type.

The section Argument types provides a list of predefined data types and explains how application specific types can be specified.

The argument type *none* has a special meaning. An argument that has the type *none* is handled as a *flag*. A flag is always optional and its related variable contains the logical value **1** if the flag has been defined during the procedure call, or otherwise **0**.

-default *value*

Eventual default values can be defined with the -default argument attribute. Arguments with default values are automatically optional arguments.

-optional|-mandatory

Arguments are by default mandatory, unless a default value is defined. The flag *-optional* transforms an argument into an optional argument.

In case an optional argument is not defined during a procedure call, the corresponding variable will not be defined. The flag *-mandatory* is the opposite to *-optional*. This flag exists only for completion reason, since an argument is anyway mandatory by default.

-multiple

Arguments that have the *-multiple* attribute can be defined multiple times during a procedure call. The values that are provided during a procedure call for such an argument are stored in a list variable. This is even the case if such an argument is only defined once during a procedure call.

The *-multiple* attribute can be attributed to unnamed arguments and to named arguments. The pair of argument name/argument value has to be repeated for each provided value in case of a named argument. In case the argument with the *-multiple* attribute is an unnamed argument, this one has to be the absolute last one of all unnamed arguments.

-choices *list*

A possible set of valid argument values can be attributed to an argument via the *-choices* attribute. The argument value provided during a procedure call will be checked against the provided choice values.

-choicelabels *list*

An eventual short description can be attributed to each choice option with the *-choicelabels* attribute. These descriptions will be used in the generated help texts and as radio and check box labels for the interactive calls.

The *-choicelabels* attribute is optional, but if it is defined, its list needs to have the identical size as the *-choices* argument list.

-range *{double double}*

Another argument constraint can be defined with the *-range* attribute. The valid range is defined with a list containing the minimum valid value and a maximum valid value. The *-range* attribute has to be used only for numerical arguments, like integers and doubles.

-validatecommand *script*

Custom argument value validations can be performed via specific validation commands that are defined with the *-validatecommand* attribute. The provided validation command

can be a complete script in which the pattern *%P* is replaced by the argument value that has to be validated.

Validation command declaration example:

```
tepam::procedure {display_message} {
   -args {
      {text -type string -description "Message text" \
            -validatecommand {IllegalWordDetector %P}}
} {
}
```

While the purpose of this custom argument validation attribute is the validation of a specific argument, there is also a global attribute *-validatecommand* that allows performing validation that involves multiple arguments.

-validatecommand_error_text *string*
This attribute allows overriding the default error message for a custom argument validation (defined by *-validatecommand*).

-widget *string*
The widgets that allow defining the different arguments in case of an interactive procedure call are normally selected automatically in function of the argument type. The *-widget* attribute allows specifying explicitly a certain widget type for an argument.

-auxargs *list*
In case a procedure is called interactively, additional argument attributes can be provided to the interactive argument definition form via the *-auxargs* attribute that is itself a list of attribute name/attribute value pairs:

```
-auxargs {-<arg_attr_name_1a> <arg_attr_value_1a> \
          -<arg_attr_name_1b> <arg_attr_value_1b>
          ...
}
```

For example, if a procedure takes as argument a file name it may be beneficial to specify the required file type for the interactive argument definition form. This information can be provided via the *-auxargs* attribute to the argument definition form:

```
tepam::procedure LoadPicture {
   -args {
      {FileName -type existingfile -description "Picture file" \
                -auxargs {-filetypes {{"GIF" {*.gif}} {"JPG"
    {*.jpg}} }}}
   }
} {
}
```

-auxargs_commands *script*
If the auxiliary argument attributes are not static but have to be dynamically adaptable, the *-auxargs_commands* allows defining them via commands that are executed during a

procedure call. A list of pairs of auxiliary attribute names and commands has to be provided to the *-auxargs_commands* attribute. The provided commands are executed in the context of the calling procedure.

```
-auxargs_commands {-<arg_attr_name_1a> <arg_attr_command_1a> \
                   -<arg_attr_name_1b> <arg_attr_command_1b>
                   ...
}
```

Variables

Several variables defined inside the **::tepam** namespace impact the mode of operation of the procedures that have been declared with the TEPAM **procedure** command.

named_arguments_first

This variable defines the general calling style of the procedures. It is by default set to **1** which selects the *named arguments first, unnamed arguments later* style (Tcl style).

By setting this variable to **0**, the *named arguments first, unnamed arguments later* style is globally selected (Tk style):

```
set tepam::named_arguments_first 0
```

While this variable defines the general calling style, the procedure attribute *-named_arguments_first* can adapt this style individually for each declared procedure.

auto_argument_name_completion

This variable controls the general automatic argument name matching mode. By default it is set to **1**, meaning that the called procedures are trying to match eventually abbreviated argument names with the declared argument names.

By setting this variable to **0** the automatic argument name matching mode is disabled:

```
set tepam::auto_argument_name_completion 0
```

While this variable defines the general matching mode, the procedure attribute *-auto_argument_name_completion* can adapt this mode individually for each declared procedure.

interactive_display_format

A procedure declared via the TEPAM **procedure** command can always be called with the **-interactive** switch. By doing so, a graphical form will be generated that allows entering interactively all procedure arguments.

There are two display modes for these interactive forms. The *extended* mode which is the default mode is more adapted for small procedure argument sets. The **short** form is more adequate for huge procedure argument sets:

```
set tepam::interactive_display_format "short"
```

The choice to use short or extended forms can be globally configured via the variable **interactive_display_format**. This global setting can be changed individually for a procedure

with the procedure attribute *-interactive_display_format*.

help_line_length

The maximum line length used by the procedure help text generator can be specified with this variable. The default length which is set to 80 (characters) can easily be adapted to the need of an application:

```
set tepam::help_line_length 120
```

Since this variable is applied directly during the help text generation, its value can continuously be adapted to the current need.

command_log

Procedure calls can be logged inside the list variable **tepam::ProcedureCallLogList**. The variable **tepam::command_log** controls the default logging settings for any procedures. The following configurations are supported:
- *0*: Disables any procedure call loggings
- *1*: Enables any procedure call loggings
- "*interactive*": Will log any procedures called interactively (e.g. procedures called with the -interactive flag). This is the default configuration.

This default logging configuration can be changed individually for each procedure with the *-command_log* attribute.

Argument types

TEPAM provides a comprehensive set of procedure argument types. They can easily be completed with application specific types if necessary.

Predefined argument types

To remember, a type can be assigned to each specified procedure argument:

```
tepam::procedure {warning} {
   -args {
      {-font -type font -default {Arial 10 italic}}
      {-severity_level -type integer -optional -range {1 10}}
      {-fg -type color -optional -description "Message color"}
      {text -type string -multiple -description "Multiple text lines
    to display"}
   }
} {
   ...
}
```

There are some *special purpose types* that are building the first category of predefined argument types:

- **none**

A *flag*, also called *switch*, is defined as a named argument that has the type **none**. Flags are always optional and the default value of the assigned variable is set to **0**. In contrast to the (normal) named arguments, no argument value has to be provided to a flag.

Since no argument value has to be provided to a flag, also no data check is performed for this argument type.

```
tepam::procedure flag_test {
   -args {
      {-flag -type none -description "This is a flag"}
   }
} {
   puts $flag
}

flag_test
-> 0

flag_test -flag
-> 1
```

- **string**

String is a generic argument data type. Any data string can be provided to a string type argument and no data type checks are therefore performed. The string type allows defining single line strings during the interactive procedure calls.

- **text**

Text is identical to **string** with the only difference that it allows entering multi line strings during interactive procedure calls.

- **{}**

A **blank** argument type signifies an undefined argument type. This is the default argument type that will be used if no type has been explicitly specified. An argument that has a **blank** type behaves identically than an argument that has a **string** type, e.g. no argument data checks are performed. The only difference is that the data type **string** is mentioned in the generated help documentation, while this is not the case for the **blank** type.

Several *numerical types* are defined by TEPAM. The type validation procedures are using the **string is -strict** commands to check the validity of the provided arguments, which assures that no empty strings are accepted as argument value. The type validation expression for the numerical types and the argument types to which this expression is applied are:

```
string is <type_to_check> -strict <argument_value>
```

- *boolean*
- *integer*
- *double*

Empty strings are accepted as argument value for all the alpha numeric argument types. The argument types that are falling into this category and validation expression used for them are:

```
string is <type_to_check> <argument_value>
```

- *alnum*
- *alpha*
- *ascii*

- *control*
- *digit*
- *graph*
- *lower*
- *print*
- *punct*
- *space*
- *upper*
- *wordchar*
- *xdigit*

In addition to the data types checked with the **string is** commands, TEPAM specifies some other useful data types:

- *char*

  Each string that has a length of 1 character meets the *character* type. The type check is made with the following expression:

  ```
  expr [string length <argument_value>]==1
  ```

- *color*

  Any character strings that are accepted by Tk as a color are considered as valid color argument. Please note that the Tk package has to be loaded to use the type *color*. TEPAM is using the following command to validate the color type:

  ```
  expr ![catch {winfo rgb . <argument_value>}]
  ```

- *font*

  Any character strings that are accepted by Tk as a font are considered as valid font argument. Please note that the Tk package has to be loaded to use the *font* type. TEPAM is using the following command to validate the color type:

  ```
  expr ![catch {font measure <argument_value> ""}]
  ```

- *file*

  Any strings that are not containing one of the following characters are considered as valid file names: * ? " < >. It is not necessary that the file and its containing directory exist. Zero-length strings are not considered as valid file names.

  The following expression is used to validate the file names:

  ```
  expr [string length <argument_value>]>0 && ![regexp {[\"*?<>:]}
      <argument_value>]
  ```

- *existingfile*

  The argument is valid if it matches with an existing file. The following check is performed to validate the arguments of this type:

  ```
  file exists <argument_value>
  ```

- *directory*

  The directory argument is validated exactly in the same way as the file arguments.

- *existingdirectory*

The argument is valid if it matches with an existing directory. The following check is performed to validate the arguments of this type:

```
file isdirectory <argument_value>
```

Defining application specific argument types

To add support for a new application specific argument type it is just necessary to add into the namespace tepam a validation function **Validation()**. This function requires one argument. It has to returns **1** if the provided argument matches with the relevant data type. The function has to return otherwise **0**.

The validation command section of the "*tepam.tcl*" package provides sufficient examples of validation functions, since it implements the ones for the standard TEPAM types.

The following additional code snippet shows the validation function for a custom argument type that requires values that have a character string length of exactly 2:

```
proc tepam::Validate(two_char) {v} {expr {[string length $v]==2}}
```

Procedure calls

Help

Each procedure can be called with the *-help* flag. The procedure will then print a generated help text to *stdout* and will then return without performing any additional actions.

Taking the first procedure declared in Procedure calls, the help request and the printed help text would be:

```
display message -help
->
NAME
      display message - Displays a simple message box
SYNOPSIS
      display message
            [-mtype <mtype>]
               Message type, default: "Warning", choices: {Info,
    Warning, Error}
            <text>
               Multiple text lines to display, type: string
DESCRIPTION
      This procedure allows displaying a configurable message box. The
    default
      message type that is created is a warning, but also errors and
    info can
      be generated.
      The procedure accepts multiple text lines.
EXAMPLE
      display message -mtype Warning "Save first your job"
```

The argument manager is checking if the last provided argument is *-help* and generates the requested help message if this is the case. So, also the following example will print the help message:

```
display message -mtype Info "It is 7:00" -help
```

On the other hand, the following call will result in an error:

```
display message -help -mtype Info "It is 7:00"
->
display message: Argument '-help' not known
```

Interactive procedure call

If Tk has been loaded a procedure can be called with the *-interactive* flag to open a graphical form that allows specifying interactively all procedure arguments. The following example assures that the Tk library is loaded and shows the command line to call interactively the procedure declared in Procedure calls:

```
package require Tk
display message -interactive
```

Also the *-interactive* flag has to be placed at the last argument position as this is also required for the *-help* flag. Arguments defined before the *-interactive* flag will be ignored. The following example is therefore also a valid interactive procedure call:

```
display message -mtype Info "It is 7:00" -interactive
```

Unnamed arguments

Unnamed arguments are typically provided to the called procedure as simple parameters. This procedure calling form requires that the provided arguments are strictly following the order of the specified arguments. Several parameters can be assigned to the last argument if this one has the *-multiple* attribute. So, the following declared procedure ...

```
tepam::procedure {display_message} {
   -args {
      {mtype -choices {Info Warning Error}}
      {text -type string -multiple}
   }
} {
   puts "$mtype: [join $text]"
}
```

... can for example be called in the following ways:

```
display_message Info "It is PM 7:00."
-> Info: It is PM 7:00.

display_message Info "It is PM 7:00." "You should go home."
-> Info: It is PM 7:00. You should go home.
```

The nice thing is that unnamed arguments can also be called as named arguments, which can

be handy, for example if the exact specified argument order is not known to a user:

```
display_message -mtype Info -text "It is PM 7:00."
-> Info: It is PM 7:00.

display_message -text "It is PM 7:00." -mtype Info
-> Info: It is PM 7:00.

display_message -mtype Info -text "It is PM 7:00." -text "You should
    go home."
-> Info: It is PM 7:00. You should go home.

display_message -text "It is PM 7:00." -text "You should go home."
    -mtype Info
-> Info: It is PM 7:00. You should go home.
```

Named arguments

Named arguments have to be provided to a procedure in form of a parameter pairs composed by the argument names and the argument values. The order how they are provided during a procedure call is irrelevant and has not to match with the argument specification order.

The following declared procedure ...

```
tepam::procedure {display_message} {
   -args {
      {-mtype -choices {Info Warning Error}}
      {-text -type string -multiple}
   }
} {
   puts "$mtype: [join $text]"
}
```

... can be called in the following ways:

```
display_message -mtype Info -text "It is PM 7:00."
-> Info: It is PM 7:00.

display_message -text "It is PM 7:00." -mtype Info
-> Info: It is PM 7:00.

display_message -mtype Info -text "It is PM 7:00." -text "You should
    go home."
-> Info: It is PM 7:00. You should go home.

display_message -text "It is PM 7:00." -text "You should go home."
    -mtype Info
-> Info: It is PM 7:00. You should go home.
```

Also named arguments that have not the *-multiple* attribute can be provided multiple times.

Only the last provided argument will be retained in such a case:

```
display_message -mtype Info -text "It is PM 7:00." -mtype Warning
-> Warning: It is PM 7:00.
```

Unnamed arguments first, named arguments later (Tk style)

A procedure that has been defined while the variable **tepam::named_arguments_first** was set to 1, or with the procedure attribute *-named_arguments_first* set to 1 has to be called in the Tcl style. The following procedure declaration will be used in this section to illustrate the meaning of this calling style:

```
set tepam::named_arguments_first 1
tepam::procedure my_proc {
   -args {
      {-n1 -default ""}
      {-n2 -default ""}
      {u1 -default ""}
      {u2 -default ""}
   }
} {
   puts "n1:'$n1', n2:'$n2', u1:'$u1', u2:'$u2'"
}
```

The unnamed arguments are placed at the end of procedure call, after the named arguments:

```
my_proc -n1 N1 -n2 N2 U1 U2
-> n1:'N1', n2:'N2', u1:'U1', u2:'U2'
```

The argument parser considers the first argument that doesn't start with the '-' character as well as all following arguments as unnamed argument:

```
my_proc U1 U2
-> n1:'', n2:'', u1:'U1', u2:'U2'
```

Named arguments can be defined multiple times. If the named argument has the *-multiply* attribute, all argument values will be collected in a list. Otherwise, only the last provided attribute value will be retained:

```
my_proc -n1 N1 -n2 N2 -n1 M1 U1 U2
-> n1:'M1', n2:'N2', u1:'U1', u2:'U2'
```

The name of the first unnamed argument has therefore not to start with the '-' character. The unnamed argument is otherwise considered as name of another named argument. This is especially important if the first unnamed argument is given by a variable that can contain any character strings:

```
my_proc -n1 N1 -n2 N2 "->" "<-"
-> my_proc: Argument '->' not known

set U1 "->"
my_proc -n1 N1 -n2 N2 $U1 U2
my_proc: Argument '->' not known
```

The '--' flag allows separating unambiguously the unnamed arguments from the named arguments. All data after the '--' flag will be considered as unnamed argument:

```
my_proc -n1 N1 -n2 N2 -- "->" "<-"
-> n1:'N1', n2:'N2', u1:'->', u2:'<-'

set U1 "->"
my_proc -n1 N1 -n2 N2 -- $U1 U2
-> n1:'N1', n2:'N2', u1:'->', u2:'<-'
```

Named arguments first, unnamed arguments later (Tcl style)

The Tk calling style will be chosen if a procedure is defined while the variable **tepam::named_arguments_first** is set to 0, or if the procedure attribute *-named_arguments_first* has been set to 0. The following procedure will be used in this section to illustrate this calling style:

```
set tepam::named_arguments_first 0
tepam::procedure my_proc {
   -args {
      {-n1 -default ""}
      {-n2 -default ""}
      {u1}
      {u2 -default "" -multiple}
   }
} {
   puts "n1:'$n1', n2:'$n2', u1:'$u1', u2:'$u2'"
}
```

The unnamed arguments have to be provided first in this case. The named arguments are provided afterwards:

```
my_proc U1 U2 -n1 N1 -n2 N2
-> n1:'N1', n1:'N1', u1:'U1', u2:'U2'
```

The argument parser will assign to each defined unnamed argument a value before it switches to read the named arguments. This default behavior changes a bit if there are unnamed arguments that are optional or that can take multiple values.

An argument value will only be assigned to an unnamed argument that is optional (that has either the *-optional* attribute or that has a default value), if the value is not beginning with the '-' character or if no named arguments are defined. The value that starts with '-' is otherwise considered as the name of a named argument.

Argument values are assigned to an argument that has the *-multiple* attribute as long as the parameter value doesn't starts with the '-' character.

Values that start with the '-' character can therefore not be assigned to optional unnamed arguments, which restricts the usage of the Tcl procedure calling style. The Tk style may be preferable in some cases, since it allows separating unambiguously the named arguments from the unnamed ones with the '--' flag.

Let's explore in a bit less theoretically the ways how the previously defined procedure can be called: The first example calls the procedure without any parameters, which leads to an error since *u1* is a mandatory argument:

```
my_proc
-> my_proc: Required argument is missing: u1
```

The procedure call is valid if one parameter is provided for *u1*:

```
my_proc U1
-> n1:'', n2:'', u1:'U1', u2:''
```

If more parameters are provided that are not starting with the '-' character, they will be attributed to the unnamed arguments. *U2* will receive 3 of these parameters, since it accepts multiple values:

```
my_proc U1 U2 U3 U4
-> n1:'', n2:'', u1:'U1', u2:'U2 U3 U4'
```

As soon as one parameter starts with '-' and all unnamed arguments have been assigned, the argument manager tries to interpret the parameter as name of a named argument. The procedure call will fail if a value beginning with '-' is assigned to an unnamed argument:

```
my_proc U1 U2 U3 U4 -U5
-> my_proc: Argument '-U5' not known
```

The attribution of a parameter to a named argument will fail if there are undefined unnamed (non optional) arguments. The name specification will in this case simply be considered as a parameter value that is attributed to the *next* unnamed argument. This was certainly not the intention in the following example:

```
my_proc -n1 N1
-> n1:'', n2:'', u1:'-n1', u2:'N1'
```

The situation is completely different if values have already been assigned to all mandatory unnamed arguments. A parameter beginning with the '-' character will in this case be considered as a name identifier for a named argument:

```
my_proc U1 -n1 N1
-> n1:'N1', n2:'', u1:'U1', u2:''
```

No unnamed arguments are allowed behind the named arguments:

```
my_proc U1 -n1 N1 U2
-> my_proc: Argument 'U2' is not an option
```

The '--' flag has no special meaning if not all mandatory arguments have got assigned a value. This flag will simply be attributed to one of the unnamed arguments:

```
my_proc -- -n1 N1
-> n1:'N1', n2:'', u1:'--', u2:''
```

But the '--' flag is simply ignored if the argument parser has started to handle the named arguments:

```
my_proc U1 -- -n1 N1
-> n1:'N1', n2:'', u1:'U1', u2:''

my_proc U1 -n1 N1 -- -n2 N2
-> n1:'N1', n2:'N2', u1:'U1', u2:''
```

Raw argument list

It may be necessary sometimes that the procedure body is able to access the entire list of arguments provided during a procedure call. This can happen via the **args** variable that contains always the unprocessed argument list:

```
tepam::procedure {display_message} {
   -args {
      {-mtype -choices {Warning Error} -default Warning}
      {text -type string -multiple}

   }
} {
   puts "args: $args"
}
display_message -mtype Warning "It is 7:00"
-> args: -mtype Warning {It is 7:00}
```

Category

Procedures, arguments, parameters, options

Copyright

```
Copyright (c) 2009-2013, Andreas Drollinger
```

term

ansi_cattr

Keywords: ansi_cattr, ansi, attribute control, color control, control, terminal

Name

term::ansi::code::attr - ANSI attribute sequences

Synopsis

package require Tcl 8.5 9

package require term::ansi::code ?**0.2**?
package require term::ansi::code::attr ?**0.2**?
::term::ansi::code::attr::names
::term::ansi::code::attr::import ?*ns*? ?*arg...*?
::term::ansi::code::attr::fgblack
::term::ansi::code::attr::fgred
::term::ansi::code::attr::fggreen
::term::ansi::code::attr::fgyellow
::term::ansi::code::attr::fgblue
::term::ansi::code::attr::fgmagenta
::term::ansi::code::attr::fgcyan
::term::ansi::code::attr::fgwhite
::term::ansi::code::attr::fgdefault
::term::ansi::code::attr::bgblack
::term::ansi::code::attr::bgred
::term::ansi::code::attr::bggreen
::term::ansi::code::attr::bgyellow
::term::ansi::code::attr::bgblue
::term::ansi::code::attr::bgmagenta
::term::ansi::code::attr::bgcyan
::term::ansi::code::attr::bgwhite
::term::ansi::code::attr::bgdefault
::term::ansi::code::attr::bold
::term::ansi::code::attr::dim
::term::ansi::code::attr::italic
::term::ansi::code::attr::underline
::term::ansi::code::attr::blink
::term::ansi::code::attr::revers
::term::ansi::code::attr::hidden
::term::ansi::code::attr::strike
::term::ansi::code::attr::nobold
::term::ansi::code::attr::noitalic
::term::ansi::code::attr::nounderline
::term::ansi::code::attr::noblink
::term::ansi::code::attr::norevers
::term::ansi::code::attr::nohidden
::term::ansi::code::attr::nostrike
::term::ansi::code::attr::reset

Description

This package provides symbolic names for the ANSI attribute control codes. For each control code a single command is provided which returns this code as its result. None of the commands of this package write to a channel; that is handled by higher level packages, like **term::ansi::send**.

API

Introspection

::term::ansi::code::attr::names
This command is for introspection. It returns as its result a list containing the names of all attribute commands.

::term::ansi::code::attr::import ?*ns*? ?*arg*...?
This command imports some or all attribute commands into the namespace *ns*. This is by default the namespace *attr*. Note that this is relative namespace name, placing the imported command into a child of the current namespace. By default all commands are imported, this can howver be restricted by listing the names of the wanted commands after the namespace argument.

Attributes

::term::ansi::code::attr::fgblack
Set text color to *Black*.

::term::ansi::code::attr::fgred
Set text color to *Red*.

::term::ansi::code::attr::fggreen
Set text color to *Green*.

::term::ansi::code::attr::fgyellow
Set text color to *Yellow*.

::term::ansi::code::attr::fgblue
Set text color to *Blue*.

::term::ansi::code::attr::fgmagenta
Set text color to *Magenta*.

::term::ansi::code::attr::fgcyan
Set text color to *Cyan*.

::term::ansi::code::attr::fgwhite
Set text color to *White*.

::term::ansi::code::attr::fgdefault
Set default text color (*Black*).

::term::ansi::code::attr::bgblack
Set background to *Black*.

::term::ansi::code::attr::bgred
Set background to *Red*.

::term::ansi::code::attr::bggreen
Set background to *Green*.

::term::ansi::code::attr::bgyellow
Set background to *Yellow*.

::term::ansi::code::attr::bgblue
Set background to *Blue*.

::term::ansi::code::attr::bgmagenta
Set background to *Magenta*.

::term::ansi::code::attr::bgcyan
Set background to *Cyan*.

::term::ansi::code::attr::bgwhite
Set background to *White*.

::term::ansi::code::attr::bgdefault
Set default background (Transparent).

::term::ansi::code::attr::bold
Bold on.

::term::ansi::code::attr::dim
Dim on.

::term::ansi::code::attr::italic
Italics on.

::term::ansi::code::attr::underline
Underscore on.

::term::ansi::code::attr::blink
Blink on.

::term::ansi::code::attr::revers
Reverse on.

::term::ansi::code::attr::hidden
Hidden on.

::term::ansi::code::attr::strike
Strike-through on.

::term::ansi::code::attr::nobold
Bold off.

::term::ansi::code::attr::noitalic
Italics off.

::term::ansi::code::attr::nounderline
Underscore off.

::term::ansi::code::attr::noblink
Blink off.

::term::ansi::code::attr::norevers
Reverse off.

::term::ansi::code::attr::nohidden
Hidden off.

::term::ansi::code::attr::nostrike
Strike-through off.

::term::ansi::code::attr::reset
Reset all attributes to their default values.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *term* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be

made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Terminal control

Copyright

```
Copyright (c) 2006 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


ansi_cctrl
Keywords: ansi_cctrl, ansi, attribute control, color control, control, terminal

Name
term::ansi::code::ctrl - ANSI control sequences

Synopsis
package require Tcl 8.5 9
package require term::ansi::code ?**0.2**?
package require term::ansi::code::ctrl ?**0.3**?
::term::ansi::code::ctrl::names
::term::ansi::code::ctrl::import ?*ns*? ?*arg...*?
::term::ansi::code::ctrl::eeol
::term::ansi::code::ctrl::esol
::term::ansi::code::ctrl::el
::term::ansi::code::ctrl::ed
::term::ansi::code::ctrl::eu
::term::ansi::code::ctrl::es
::term::ansi::code::ctrl::sd
::term::ansi::code::ctrl::su
::term::ansi::code::ctrl::ch
::term::ansi::code::ctrl::sc
::term::ansi::code::ctrl::rc
::term::ansi::code::ctrl::sca
::term::ansi::code::ctrl::rca
::term::ansi::code::ctrl::st
::term::ansi::code::ctrl::ct
::term::ansi::code::ctrl::cat
::term::ansi::code::ctrl::qdc
::term::ansi::code::ctrl::qds
::term::ansi::code::ctrl::qcp
::term::ansi::code::ctrl::rd
::term::ansi::code::ctrl::elw
::term::ansi::code::ctrl::dlw
::term::ansi::code::ctrl::eg
::term::ansi::code::ctrl::lg

::term::ansi::code::ctrl::scs0 *tag*
::term::ansi::code::ctrl::scs1 *tag*
::term::ansi::code::ctrl::sda *arg...*
::term::ansi::code::ctrl::sda_fgblack
::term::ansi::code::ctrl::sda_fgred
::term::ansi::code::ctrl::sda_fggreen
::term::ansi::code::ctrl::sda_fgyellow
::term::ansi::code::ctrl::sda_fgblue
::term::ansi::code::ctrl::sda_fgmagenta
::term::ansi::code::ctrl::sda_fgcyan
::term::ansi::code::ctrl::sda_fgwhite
::term::ansi::code::ctrl::sda_fgdefault
::term::ansi::code::ctrl::sda_bgblack
::term::ansi::code::ctrl::sda_bgred
::term::ansi::code::ctrl::sda_bggreen
::term::ansi::code::ctrl::sda_bgyellow
::term::ansi::code::ctrl::sda_bgblue
::term::ansi::code::ctrl::sda_bgmagenta
::term::ansi::code::ctrl::sda_bgcyan
::term::ansi::code::ctrl::sda_bgwhite
::term::ansi::code::ctrl::sda_bgdefault
::term::ansi::code::ctrl::sda_bold
::term::ansi::code::ctrl::sda_dim
::term::ansi::code::ctrl::sda_italic
::term::ansi::code::ctrl::sda_underline
::term::ansi::code::ctrl::sda_blink
::term::ansi::code::ctrl::sda_revers
::term::ansi::code::ctrl::sda_hidden
::term::ansi::code::ctrl::sda_strike
::term::ansi::code::ctrl::sda_nobold
::term::ansi::code::ctrl::sda_noitalic
::term::ansi::code::ctrl::sda_nounderline
::term::ansi::code::ctrl::sda_noblink
::term::ansi::code::ctrl::sda_norevers
::term::ansi::code::ctrl::sda_nohidden
::term::ansi::code::ctrl::sda_nostrike
::term::ansi::code::ctrl::sda_reset
::term::ansi::send::fcp *row col*
::term::ansi::code::ctrl::cu ?*n*?
::term::ansi::code::ctrl::cd ?*n*?
::term::ansi::code::ctrl::cf ?*n*?
::term::ansi::code::ctrl::cb ?*n*?
::term::ansi::code::ctrl::ss ?*s e*?
::term::ansi::code::ctrl::skd *code str*
::term::ansi::code::ctrl::title *str*
::term::ansi::code::ctrl::gron

::term::ansi::code::ctrl::groff
::term::ansi::code::ctrl::tlc
::term::ansi::code::ctrl::trc
::term::ansi::code::ctrl::brc
::term::ansi::code::ctrl::blc
::term::ansi::code::ctrl::ltj
::term::ansi::code::ctrl::ttj
::term::ansi::code::ctrl::rtj
::term::ansi::code::ctrl::btj
::term::ansi::code::ctrl::fwj
::term::ansi::code::ctrl::hl
::term::ansi::code::ctrl::vl
::term::ansi::code::ctrl::groptim *str*
::term::ansi::code::ctrl::clear
::term::ansi::code::ctrl::init
::term::ansi::code::ctrl::showat *row col text*

Description

This package provides symbolic names for the ANSI control sequences. For each sequence a single command is provided which returns the sequence as its result. None of the commands of this package write to a channel; that is handled by higher level packages, like **term::ansi::send** .

API

Introspection

::term::ansi::code::ctrl::names
This command is for introspection. It returns as its result a list containing the names of all attribute commands.

::term::ansi::code::ctrl::import ?*ns*? ?*arg*...?
This command imports some or all attribute commands into the namespace *ns*. This is by default the namespace *ctrl*. Note that this is relative namespace name, placing the imported command into a child of the current namespace. By default all commands are imported, this can howver be restricted by listing the names of the wanted commands after the namespace argument.

Sequences

::term::ansi::code::ctrl::eeol
Erase (to) End Of Line

::term::ansi::code::ctrl::esol
Erase (to) Start Of Line

::term::ansi::code::ctrl::el
Erase (current) Line

::term::ansi::code::ctrl::ed
Erase Down (to bottom)

::term::ansi::code::ctrl::eu

Erase Up (to top)

::term::ansi::code::ctrl::es
Erase Screen

::term::ansi::code::ctrl::sd
Scroll Down

::term::ansi::code::ctrl::su
Scroll Up

::term::ansi::code::ctrl::ch
Cursor Home

::term::ansi::code::ctrl::sc
Save Cursor

::term::ansi::code::ctrl::rc
Restore Cursor (Unsave)

::term::ansi::code::ctrl::sca
Save Cursor + Attributes

::term::ansi::code::ctrl::rca
Restore Cursor + Attributes

::term::ansi::code::ctrl::st
Set Tab (@ current position)

::term::ansi::code::ctrl::ct
Clear Tab (@ current position)

::term::ansi::code::ctrl::cat
Clear All Tabs

::term::ansi::code::ctrl::qdc
Query Device Code

::term::ansi::code::ctrl::qds
Query Device Status

::term::ansi::code::ctrl::qcp
Query Cursor Position

::term::ansi::code::ctrl::rd
Reset Device

::term::ansi::code::ctrl::elw
Enable Line Wrap

::term::ansi::code::ctrl::dlw
Disable Line Wrap

::term::ansi::code::ctrl::eg
Enter Graphics Mode

::term::ansi::code::ctrl::lg
Exit Graphics Mode

::term::ansi::code::ctrl::scs0 ***tag***
Set default character set

::term::ansi::code::ctrl::scs1 ***tag***
Set alternate character set Select Character Set.

Choose which character set is used for either default (scs0) or alternate font (scs1). This does not change whether default or alternate font are used, only their definition.

The legal tags, and their meanings, are:

A
: United Kingdom Set

B
: ASCII Set

0
: Special Graphics

1
: Alternate Character ROM Standard Character Set

2
: Alternate Character ROM Special Graphics

::term::ansi::code::ctrl::sda *arg*...
: Set Display Attributes. The arguments are the code sequences for the possible attributes, as provided by the package **term::ansi::code::attr**. For convenience this package also provides additional commands each setting a single specific attribute.

::term::ansi::code::ctrl::sda_fgblack
: Set text color to *Black*.

::term::ansi::code::ctrl::sda_fgred
: Set text color to *Red*.

::term::ansi::code::ctrl::sda_fggreen
: Set text color to *Green*.

::term::ansi::code::ctrl::sda_fgyellow
: Set text color to *Yellow*.

::term::ansi::code::ctrl::sda_fgblue
: Set text color to *Blue*.

::term::ansi::code::ctrl::sda_fgmagenta
: Set text color to *Magenta*.

::term::ansi::code::ctrl::sda_fgcyan
: Set text color to *Cyan*.

::term::ansi::code::ctrl::sda_fgwhite
: Set text color to *White*.

::term::ansi::code::ctrl::sda_fgdefault
: Set default text color (*Black*).

::term::ansi::code::ctrl::sda_bgblack
: Set background to *Black*.

::term::ansi::code::ctrl::sda_bgred
: Set background to *Red*.

::term::ansi::code::ctrl::sda_bggreen
: Set background to *Green*.

::term::ansi::code::ctrl::sda_bgyellow
: Set background to *Yellow*.

::term::ansi::code::ctrl::sda_bgblue
Set background to *Blue*.

::term::ansi::code::ctrl::sda_bgmagenta
Set background to *Magenta*.

::term::ansi::code::ctrl::sda_bgcyan
Set background to *Cyan*.

::term::ansi::code::ctrl::sda_bgwhite
Set background to *White*.

::term::ansi::code::ctrl::sda_bgdefault
Set default background (Transparent).

::term::ansi::code::ctrl::sda_bold
Bold on.

::term::ansi::code::ctrl::sda_dim
Dim on.

::term::ansi::code::ctrl::sda_italic
Italics on.

::term::ansi::code::ctrl::sda_underline
Underscore on.

::term::ansi::code::ctrl::sda_blink
Blink on.

::term::ansi::code::ctrl::sda_revers
Reverse on.

::term::ansi::code::ctrl::sda_hidden
Hidden on.

::term::ansi::code::ctrl::sda_strike
Strike-through on.

::term::ansi::code::ctrl::sda_nobold
Bold off.

::term::ansi::code::ctrl::sda_noitalic
Italics off.

::term::ansi::code::ctrl::sda_nounderline
Underscore off.

::term::ansi::code::ctrl::sda_noblink
Blink off.

::term::ansi::code::ctrl::sda_norevers
Reverse off.

::term::ansi::code::ctrl::sda_nohidden
Hidden off.

::term::ansi::code::ctrl::sda_nostrike
Strike-through off.

::term::ansi::code::ctrl::sda_reset
Reset all attributes to their default values.

::term::ansi::send::fcp *row col*

Force Cursor Position (aka Go To).

::term::ansi::code::ctrl::cu ?*n*?
Cursor Up. *n* defaults to 1.

::term::ansi::code::ctrl::cd ?*n*?
Cursor Down. *n* defaults to 1.

::term::ansi::code::ctrl::cf ?*n*?
Cursor Forward. *n* defaults to 1.

::term::ansi::code::ctrl::cb ?*n*?
Cursor Backward. *n* defaults to 1.

::term::ansi::code::ctrl::ss ?*s e*?
Scroll Screen (entire display, or between rows start end, inclusive).

::term::ansi::code::ctrl::skd *code str*
Set Key Definition.

::term::ansi::code::ctrl::title *str*
Set the terminal title.

::term::ansi::code::ctrl::gron
Switch to character/box graphics. I.e. switch to the alternate font.

::term::ansi::code::ctrl::groff
Switch to regular characters. I.e. switch to the default font.

::term::ansi::code::ctrl::tlc
Character graphics, Top Left Corner.

::term::ansi::code::ctrl::trc
Character graphics, Top Right Corner.

::term::ansi::code::ctrl::brc
Character graphics, Bottom Right Corner.

::term::ansi::code::ctrl::blc
Character graphics, Bottom Left Corner.

::term::ansi::code::ctrl::ltj
Character graphics, Left T Junction.

::term::ansi::code::ctrl::ttj
Character graphics, Top T Junction.

::term::ansi::code::ctrl::rtj
Character graphics, Right T Junction.

::term::ansi::code::ctrl::btj
Character graphics, Bottom T Junction.

::term::ansi::code::ctrl::fwj
Character graphics, Four-Way Junction.

::term::ansi::code::ctrl::hl
Character graphics, Horizontal Line.

::term::ansi::code::ctrl::vl
Character graphics, Vertical Line.

::term::ansi::code::ctrl::groptim *str*
Optimize character graphics. The generator commands above create way to many

superfluous commands shifting into and out of the graphics mode. This command removes all shifts which are not needed. To this end it also knows which characters will look the same in both modes, to handle strings created outside of this package.

::term::ansi::code::ctrl::clear
Clear screen. In essence a sequence of CursorHome + EraseDown.

::term::ansi::code::ctrl::init
Initialize default and alternate fonts to ASCII and box graphics.

::term::ansi::code::ctrl::showat *row col text*
Format the block of text for display at the specified location.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *term* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Terminal control

Copyright


```
Copyright (c) 2006-2008 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


ansi_cmacros

Keywords: ansi_cmacros, ansi, control, frame, menu, terminal

Name

term::ansi::code::macros - Macro sequences

Synopsis

package require Tcl 8.5 9
package require textutil::repeat
package require textutil::tabify
package require term::ansi::code::macros ?**0.2**?
::term::ansi::code::macros::names
::term::ansi::code::macros::import ?*ns*? ?*arg...*?
::term::ansi::code::macros::menu *menu*
::term::ansi::code::macros::frame *string*

Description

This package provides higher level control sequences for more complex shapes.

API

Introspection

::term::ansi::code::macros::names

This command is for introspection. It returns as its result a list containing the names of all attribute commands.

::term::ansi::code::macros::import ?*ns*? ?*arg*...?

This command imports some or all attribute commands into the namespace *ns*. This is by default the namespace *macros*. Note that this is relative namespace name, placing the imported command into a child of the current namespace. By default all commands are imported, this can howver be restricted by listing the names of the wanted commands after the namespace argument.

Sequences

::term::ansi::code::macros::menu *menu*

The description of a menu is converted into a formatted rectangular block of text, with the menu command characters highlighted using bold red text. The result is returned as the result of the command.

The description, *menu*, is a dictionary mapping from menu label to command character.

::term::ansi::code::macros::frame *string*

The paragraph of text contained in the string is padded with spaces at the right margin, after normalizing internal tabs, and then put into a frame made of box-graphics. The result is returned as the result of the command.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *term* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Terminal control

Copyright

```
Copyright (c) 2006 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

ansi_code

Keywords: ansi_code, control, declare, define, terminal

Name

term::ansi::code - Helper for control sequences

Synopsis

package require Tcl 8.5 9
package require term::ansi::code ?**0.2**?
::term::ansi::code::esc *str*
::term::ansi::code::escb *str*
::term::ansi::code::define *name escape code*
::term::ansi::code::const *name code*

Description

This package provides commands enabling the definition of control sequences in an easy manner.

::term::ansi::code::esc *str*
: This command returns the argument string, prefixed with the ANSI escape character, "\033."

::term::ansi::code::escb *str*
: This command returns the argument string, prefixed with a common ANSI escape sequence, "\033[".

::term::ansi::code::define *name escape code*
: This command defines a procedure *name* which returns the control sequence *code*, beginning with the specified escape sequence, either **esc**, or **escb**.

::term::ansi::code::const *name code*
: This command defines a procedure *name* which returns the control sequence *code*.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *term* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Terminal control

Copyright


```
Copyright (c) 2006 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


ansi_ctrlu

Keywords: ansi_ctrlu, ansi, columns, control, cooked, input mode, lines, raw, rows, terminal

Name

term::ansi::ctrl::unix - Control operations and queries

Synopsis

package require Tcl 8.5 9
package require term::ansi::ctrl::unix ?**0.1.2**?
::term::ansi::ctrl::unix::import ?*ns*? ?*arg...*?
::term::ansi::ctrl::unix::raw
::term::ansi::ctrl::unix::cooked
::term::ansi::ctrl::unix::columns
::term::ansi::ctrl::unix::rows

Description

WARNING: This package is unix-specific and depends on the availability of two unix system commands for terminal control, i.e. **stty** and **tput**, both of which have to be found in the **$PATH**. If any of these two commands is missing the loading of the package will fail.

The package provides commands to switch the standard input of the current process between *raw* and *cooked* input modes, and to query the size of terminals, i.e. the available number of columns and lines.

API

Introspection

::term::ansi::ctrl::unix::import ?*ns*? ?*arg*...?

This command imports some or all attribute commands into the namespace *ns*. This is by default the namespace *ctrl*. Note that this is relative namespace name, placing the imported command into a child of the current namespace. By default all commands are imported, this can howver be restricted by listing the names of the wanted commands after the namespace argument.

Operations

::term::ansi::ctrl::unix::raw

This command switches the standard input of the current process to *raw* input mode. This means that from then on all characters typed by the user are immediately reported to the application instead of waiting in the OS buffer until the Enter/Return key is received.

::term::ansi::ctrl::unix::cooked

This command switches the standard input of the current process to *cooked* input mode. This means that from then on all characters typed by the user are kept in OS buffers for editing until the Enter/Return key is received.

::term::ansi::ctrl::unix::columns

This command queries the terminal connected to the standard input for the number of columns available for display.

::term::ansi::ctrl::unix::rows

This command queries the terminal connected to the standard input for the number of rows (aka lines) available for display.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *term* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Terminal control

Copyright
```
Copyright (c) 2006-2011 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

ansi_send
Keywords: ansi_send, character output, control, terminal

Name
term::ansi::send - Output of ANSI control sequences to terminals

Synopsis
package require Tcl 8.5 9
package require term::ansi::send ?**0.2**?
::term::ansi::send::import ?*ns*? *...*
::term::ansi::send::eeol
::term::ansi::send::esol
::term::ansi::send::el
::term::ansi::send::ed
::term::ansi::send::eu
::term::ansi::send::es
::term::ansi::send::sd
::term::ansi::send::su
::term::ansi::send::ch
::term::ansi::send::sc
::term::ansi::send::rc
::term::ansi::send::sca
::term::ansi::send::rca
::term::ansi::send::st
::term::ansi::send::ct
::term::ansi::send::cat
::term::ansi::send::qdc
::term::ansi::send::qds
::term::ansi::send::qcp
::term::ansi::send::rd
::term::ansi::send::elw
::term::ansi::send::dlw
::term::ansi::send::eg
::term::ansi::send::lg

::term::ansi::send::scs0 *tag*
::term::ansi::send::scs1 *tag*
::term::ansi::send::sda *arg...*
::term::ansi::send::sda_fgblack
::term::ansi::send::sda_fgred
::term::ansi::send::sda_fggreen
::term::ansi::send::sda_fgyellow
::term::ansi::send::sda_fgblue
::term::ansi::send::sda_fgmagenta
::term::ansi::send::sda_fgcyan
::term::ansi::send::sda_fgwhite
::term::ansi::send::sda_fgdefault
::term::ansi::send::sda_bgblack
::term::ansi::send::sda_bgred
::term::ansi::send::sda_bggreen
::term::ansi::send::sda_bgyellow
::term::ansi::send::sda_bgblue
::term::ansi::send::sda_bgmagenta
::term::ansi::send::sda_bgcyan
::term::ansi::send::sda_bgwhite
::term::ansi::send::sda_bgdefault
::term::ansi::send::sda_bold
::term::ansi::send::sda_dim
::term::ansi::send::sda_italic
::term::ansi::send::sda_underline
::term::ansi::send::sda_blink
::term::ansi::send::sda_revers
::term::ansi::send::sda_hidden
::term::ansi::send::sda_strike
::term::ansi::send::sda_nobold
::term::ansi::send::sda_noitalic
::term::ansi::send::sda_nounderline
::term::ansi::send::sda_noblink
::term::ansi::send::sda_norevers
::term::ansi::send::sda_nohidden
::term::ansi::send::sda_nostrike
::term::ansi::send::sda_reset
::term::ansi::send::fcp *row col*
::term::ansi::send::cu ?*n*?
::term::ansi::send::cd ?*n*?
::term::ansi::send::cf ?*n*?
::term::ansi::send::cb ?*n*?
::term::ansi::send::ss ?*s e*?
::term::ansi::send::skd *code str*
::term::ansi::send::title *str*
::term::ansi::send::gron

::term::ansi::send::groff
::term::ansi::send::tlc
::term::ansi::send::trc
::term::ansi::send::brc
::term::ansi::send::blc
::term::ansi::send::ltj
::term::ansi::send::ttj
::term::ansi::send::rtj
::term::ansi::send::btj
::term::ansi::send::fwj
::term::ansi::send::hl
::term::ansi::send::vl
::term::ansi::send::groptim *str*
::term::ansi::send::clear
::term::ansi::send::init
::term::ansi::send::showat *row col text*

Description

This package provides commands to send ANSI terminal control sequences to a terminal. All commands come in two variants, one for sending to any channel, the other for sending to *stdout* .

The commands are defined using the control sequences provided by the package **term::ansi::code::ctrl**. They have the same arguments as the commands they are based on, with the exception of the variant for sending to any channel. Their first argument is always a channel handle, then followed by the original arguments. Below we will list only the variant sending to *stdout*.

::term::ansi::send::import ?*ns*?*...*

Imports the commands of this package into the namespace *ns*. If not specified it defaults to *send*. Note that this default is a relative namespace name, i.e. the actual namespace will be created under the current namespace.

By default all commands will be imported, this can however be restricted to specific commands, by listing them after the namespace to import them into.

::term::ansi::send::eeol

Erase (to) End Of Line.

::term::ansi::send::esol

Erase (to) Start Of Line.

::term::ansi::send::el

Erase (current) Line.

::term::ansi::send::ed

Erase Down (to bottom).

::term::ansi::send::eu

Erase Up (to top).

::term::ansi::send::es

Erase Screen.

::term::ansi::send::sd
Scroll Down.

::term::ansi::send::su
Scroll Up.

::term::ansi::send::ch
Cursor Home.

::term::ansi::send::sc
Save Cursor. Note: Only one saved position can be handled. This is no unlimited stack. Saving before restoring will overwrite the saved data.

::term::ansi::send::rc
Restore Cursor (Unsave).

::term::ansi::send::sca
Save Cursor + Attributes.

::term::ansi::send::rca
Restore Cursor + Attributes.

::term::ansi::send::st
Set Tab (@ current position).

::term::ansi::send::ct
Clear Tab (@ current position).

::term::ansi::send::cat
Clear All Tabs.

::term::ansi::send::qdc
Query Device Code.

::term::ansi::send::qds
Query Device Status.

::term::ansi::send::qcp
Query Cursor Position.

::term::ansi::send::rd
Reset Device.

::term::ansi::send::elw
Enable Line Wrap.

::term::ansi::send::dlw
Disable Line Wrap.

::term::ansi::send::eg
Enter Graphics Mode.

::term::ansi::send::lg
Exit Graphics Mode.

::term::ansi::send::scs0 ***tag***
see below ...

::term::ansi::send::scs1 ***tag***
Select Character Set.

Choose which character set is used for default (scs0) and alternate font (scs1). This does not change whether default or alternate font are used, just their definitions.

The legal tags, and their meanings, are:

A
United Kingdom Set

B
ASCII Set

0
Special Graphics

1
Alternate Character ROM Standard Character Set

2
Alternate Character ROM Special Graphics

::term::ansi::send::sda *arg*...
Set Display Attributes. The arguments are the code sequences for the possible attributes, as provided by the package **term::ansi::code::attr**. For convenience this package also provides additional commands each setting a single specific attribute.

::term::ansi::send::sda_fgblack
Set text color to *Black*.

::term::ansi::send::sda_fgred
Set text color to *Red*.

::term::ansi::send::sda_fggreen
Set text color to *Green*.

::term::ansi::send::sda_fgyellow
Set text color to *Yellow*.

::term::ansi::send::sda_fgblue
Set text color to *Blue*.

::term::ansi::send::sda_fgmagenta
Set text color to *Magenta*.

::term::ansi::send::sda_fgcyan
Set text color to *Cyan*.

::term::ansi::send::sda_fgwhite
Set text color to *White*.

::term::ansi::send::sda_fgdefault
Set default text color (*Black*).

::term::ansi::send::sda_bgblack
Set background to *Black*.

::term::ansi::send::sda_bgred
Set background to *Red*.

::term::ansi::send::sda_bggreen
Set background to *Green*.

::term::ansi::send::sda_bgyellow
Set background to *Yellow*.

::term::ansi::send::sda_bgblue
Set background to *Blue*.

::term::ansi::send::sda_bgmagenta
Set background to *Magenta*.

::term::ansi::send::sda_bgcyan
Set background to *Cyan*.

::term::ansi::send::sda_bgwhite
Set background to *White*.

::term::ansi::send::sda_bgdefault
Set default background (Transparent).

::term::ansi::send::sda_bold
Bold on.

::term::ansi::send::sda_dim
Dim on.

::term::ansi::send::sda_italic
Italics on.

::term::ansi::send::sda_underline
Underscore on.

::term::ansi::send::sda_blink
Blink on.

::term::ansi::send::sda_revers
Reverse on.

::term::ansi::send::sda_hidden
Hidden on.

::term::ansi::send::sda_strike
Strike-through on.

::term::ansi::send::sda_nobold
Bold off.

::term::ansi::send::sda_noitalic
Italics off.

::term::ansi::send::sda_nounderline
Underscore off.

::term::ansi::send::sda_noblink
Blink off.

::term::ansi::send::sda_norevers
Reverse off.

::term::ansi::send::sda_nohidden
Hidden off.

::term::ansi::send::sda_nostrike
Strike-through off.

::term::ansi::send::sda_reset
Reset all attributes to their default values.

::term::ansi::send::fcp *row col*
Force Cursor Position (aka Go To).

::term::ansi::send::cu ?*n*?

Cursor Up. *n* defaults to 1.

::term::ansi::send::cd ?*n*?
Cursor Down. *n* defaults to 1.

::term::ansi::send::cf ?*n*?
Cursor Forward. *n* defaults to 1.

::term::ansi::send::cb ?*n*?
Cursor Backward. *n* defaults to 1.

::term::ansi::send::ss ?*s e*?
Scroll Screen (entire display, or between rows start end, inclusive).

::term::ansi::send::skd *code str*
Set Key Definition.

::term::ansi::send::title *str*
Set the terminal title.

::term::ansi::send::gron
Switch to character/box graphics. I.e. switch to the alternate font.

::term::ansi::send::groff
Switch to regular characters. I.e. switch to the default font.

::term::ansi::send::tlc
Character graphics, Top Left Corner.

::term::ansi::send::trc
Character graphics, Top Right Corner.

::term::ansi::send::brc
Character graphics, Bottom Right Corner.

::term::ansi::send::blc
Character graphics, Bottom Left Corner.

::term::ansi::send::ltj
Character graphics, Left T Junction.

::term::ansi::send::ttj
Character graphics, Top T Junction.

::term::ansi::send::rtj
Character graphics, Right T Junction.

::term::ansi::send::btj
Character graphics, Bottom T Junction.

::term::ansi::send::fwj
Character graphics, Four-Way Junction.

::term::ansi::send::hl
Character graphics, Horizontal Line.

::term::ansi::send::vl
Character graphics, Vertical Line.

::term::ansi::send::groptim *str*
Optimize character graphics. The generator commands above create way to many superfluous commands shifting into and out of the graphics mode. This command removes all shifts which are not needed. To this end it also knows which characters will look the same in both modes, to handle strings created outside of this package.

::term::ansi::send::clear
Clear screen. In essence a sequence of CursorHome + EraseDown.

::term::ansi::send::init
Initialize default and alternate fonts to ASCII and box graphics.

::term::ansi::send::showat *row col text*
Show the block of text at the specified location.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *term* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Terminal control

Copyright


```
Copyright (c) 2006 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


imenu

Keywords: imenu, control, menu, terminal, text display

Name

term::interact::menu - Terminal widget, menu

Synopsis

package require Tcl 8.5 9
package require term::interact::menu ?**0.2**?
term::interact::menu *object dict* ?*options...*?
object **interact**
object **done**
object **clear**
object **configure**
object **configure** *option*
object **configure** *option value...*
object **cget** *option*

Description

This package provides a class for the creation of a simple menu control.

Class API

The package exports a single command, the class command, enabling the creation of menu

instances. Its API is:

term::interact::menu *object dict* ?*options*...?
This command creates a new menu object with the name *object*, initializes it, and returns the fully qualified name of the object command as its result.

The argument is the menu to show, possibly followed by configuration options and their values. The options are explained in the section Configuration. The menu is a dictionary maping labels to symbolic action codes.

Object API

The objects created by the class command provide the methods listed below:

***object* interact**
Shows the menu in the screen at the configured location and starts interacting with it. This opens its own event loop for the processing of incoming characters. The method returns when the interaction has completed. See section Interaction for a description of the possible interaction.

The method returns the symbolic action of the menu item selected by the user at the end of the interaction.

***object* done**
This method can be used by user supplied actions to terminate the interaction with the object.

***object* clear**
This method can be used by user supplied actions to remove the menu from the terminal.

***object* configure**
see below ...

object* configure *option
see below ...

***object* configure *option value*...**
see below ...

object* cget *option
Standard methods to retrieve and configure the options of the menu.

Configuration

A menu instance recognizes the following options:

-in chan
Specifies the channel to read character sequences from. Defaults to **stdin**.

-out chan
Specifies the channel to write the menu contents to. Defaults to **stdout**.

-column int
Specifies the column of the terminal where the left margin of the menu display should appear. Defaults to 0, i.e. the left-most column.

-line int
Specifies the line of the terminal where the top margin of the menu display should appear.

Defaults to 0, i.e. the top-most line.

-height int
Specifies the number of lines of text to show at most in the display. Defaults to 25.

-actions dict
Specifies a dictionary containing additional actions, using character sequences as keys. Note that these sequences cannot override the hardwired sequences described in section Interaction.

-hilitleft int
see below ...

-hilitright int
By default the entire selected menu entry is highlighted in revers output. However, when present these two options restrict revers dispay to the specified sub-range of the entry.

-framed bool
By default the menu is shown using only header and footer out of characters box graphics. If this flag is set the menu is fully enclosed in a box.

Interaction

A menu object recognizes the control sequences listed below and acts as described. The user can supply more control sequences to act on via the configuration, but is not able to overide these defaults.

Cursor Up
The selection is moved up one entry, except if the first entry of the menu is already selected.

Cursor Down
The selection is moved down one entry, except if the last entry of the menu is already selected.

Enter/Return
The interaction with the object is terminated.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *term* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Terminal control

Copyright

```
Copyright (c) 2006 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

ipager

Keywords: ipager, control, pager, terminal, text display

Name

term::interact::pager - Terminal widget, paging

Synopsis

package require Tcl 8.5 9
package require term::interact::pager ?**0.2**?
term::interact::pager *object text* ?*options...*?
object **interact**
object **done**
object **clear**
object **text** *text*
object **configure**
object **configure** *option*
object **configure** *option value...*
object **cget** *option*

Description

This package provides a class for the creation of a simple paging text display.

Class API

The package exports a single command, the class command, enabling the creation of pager instances. Its API is:

term::interact::pager *object text* ?*options*...?

This command creates a new pager object with the name *object*, initializes it, and returns the fully qualified name of the object command as its result.

The argument is the text to show, possibly followed by configuration options and their values. The options are explained in the section Configuration.

Object API

The objects created by the class command provide the methods listed below:

***object* interact**

Show the pager in the screen at the configured location and start interacting with it. This opens its own event loop for the processing of incoming characters. The method returns when the interaction has completed. See section Interaction for a description of the possible interaction.

***object* done**

This method can be used by user supplied actions to terminate the interaction with the object.

***object* clear**

This method can be used by user supplied actions to remove the pager from the terminal.

object* text *text

This method can be used to change the text shown by the pager. The pager will reset the dispay to show the first line of the text at the top.

***object* configure**
see below ...

object* configure *option
see below ...

object* configure *option value...
see below ...

object* cget *option
Standard methods to retrieve and configure the options of the pager.

Configuration

A pager instance recognizes the following options:

-in chan
Specifies the channel to read character sequences from. Defaults to **stdin**.

-out chan
Specifies the channel to write the pager contents to. Defaults to **stdout**.

-column int
Specifies the column of the terminal where the left margin of the pager display should appear. Defaults to 0, i.e. the left-most column.

-line int
Specifies the line of the terminal where the top margin of the pager display should appear. Defaults to 0, i.e. the top-most line.

-height int
Specifies the number of lines of text to show at most in the display. Defaults to 25.

-actions dict
Specifies a dictionary containing additional actions, using character sequences as keys. Note that these sequences cannot override the hardwired sequences described in section Interaction.

Interaction

A pager object recognizes the control sequences listed below and acts as described. The user can supply more control sequences to act on via the configuration, but is not able to overide these defaults.

Cursor Up
The text is scrolled down a single line, making one more line visible at the top. The pager will not react if the first line of the text is already shown.

Cursor Down
The text is scrolled up a single line, making one more line visible at the bottom. The pager will not react if the last line of the text is already shown.

Page Up
The text is scrolled down a page. The pager will not react if the first line of the text is already shown.

Page Down

The text is scrolled up a page. The pager will not react if the last line of the text is already shown.

Enter/Return
The interaction with the object is terminated.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *term* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Terminal control

Copyright


```
Copyright (c) 2006 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


receive

Keywords: receive, character input, control, get character, listener, receiver, terminal

Name

term::receive - General input from terminals

Synopsis

package require Tcl 8.5 9
package require term::receive ?**0.2**?
::term::receive::getch ?*chan*?
::term::receive::listen *cmd* ?*chan*?
cmd **process** *string*
cmd **eof**
::term::receive::unlisten ?*chan*?

Description

This package provides the most primitive commands for receiving characters to a terminal. They are in essence convenient wrappers around the builtin commands **read** and **fileevent**.

::term::receive::getch ?*chan*?
This command reads a single character from the channel with handle *chan* and returns it as the result of the command.

If not specified *chan* defaults to **stdin**.

It is the responsibility of the caller to make sure that the channel can provide single characters. On unix this can be done, for example, by using the command of package **term::ansi::ctrl::unix**.

::term::receive::listen *cmd* ?*chan*?

This command sets up a filevent listener for the channel with handle *chan* and invokes the command prefix *cmd* whenever characters have been received, or EOF was reached.

If not specified *chan* defaults to **stdin**.

The signature of the command prefix is

cmd* process *string

This method is invoked when characters were received, and *string* holds them for processing.

***cmd* eof**

This method is invoked when EOF was reached on the channel we listen on. It will be the last call to be received by the callback.

::term::receive::unlisten ?*chan*?

This command disables the filevent listener for the channel with handle *chan*.

If not specified *chan* defaults to **stdin**.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *term* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Terminal control

Copyright


```
Copyright (c) 2006 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


term

Keywords: term, control, terminal

Name

term - General terminal control

Synopsis

package require Tcl 8.5 9
package require term ?**0.2**?

Description

It is planned to have this package provide highlevel general terminal control commands, in the vein of ncurses or similar packages. Currently nothing has been implemented however. I.e. this package is empty. It can be loaded, yet provides nothing.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *term* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Terminal control

Copyright


```
Copyright (c) 2006 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


term_bind

Keywords: term_bind, character input, control, dispatcher, listener, receiver, terminal

Name

term::receive::bind - Keyboard dispatch from terminals

Synopsis

package require Tcl 8.5 9
package require term::receive::bind ?**0.2**?
term::receive::bind *object* ?*map*?
object **map** *str cmd*
object **default** *cmd*
object **listen** ?*chan*?
object **unlisten** ?*chan*?
object **reset**
object **next** *char*
object **process** *str*
object **eof**

Description

This package provides a class for the creation of simple dispatchers from character sequences to actions. Internally each dispatcher is in essence a deterministic finite automaton with tree structure.

Class API

The package exports a single command, the class command, enabling the creation of dispatcher instances. Its API is:

term::receive::bind *object* ?*map*?
This command creates a new dispatcher object with the name *object*, initializes it, and returns the fully qualified name of the object command as its result.

The argument is a dictionary mapping from strings, i.e. character sequences to the command prefices to invoke when the sequence is found in the input stream.

Object API

The objects created by the class command provide the methods listed below:

object* map *str cmd
This method adds an additional mapping from the string *str* to the action *cmd*. The mapping will take effect immediately should the processor be in a prefix of *str*, or at the next reset operation. The action is a command prefix and will be invoked with one argument appended to it, the character sequence causing the invokation. It is executed in the global namespace.

object* default *cmd
This method defines a default action *cmd* which will be invoked whenever an unknown character sequence is encountered. The command prefix is handled in the same as the regular action defined via method **map**.

***object* listen ?*chan*?**
This methods sets up a filevent listener for the channel with handle *chan* and invokes the dispatcher object whenever characters have been received, or EOF was reached.

If not specified *chan* defaults to **stdin**.

***object* unlisten ?*chan*?**
This methods removes the filevent listener for the channel with handle *chan*.

If not specified *chan* defaults to **stdin**.

***object* reset**
This method resets the character processor to the beginning of the tree.

object* next *char
This method causes the character processor to process the character *c*. This may simply advance the internal state, or invoke an associated action for a recognized sequence.

object* process *str
This method causes the character processor to process the character sequence *str*, advancing the internal state and invoking action as necessary. This is a callback for **listen**.

***object* eof**
This method causes the character processor to handle EOF on the input. This is currently no-op. This is a callback for **listen**.

Notes

The simplicity of the DFA means that it is not possible to recognize a character sequence with has a another recognized character sequence as its prefix.

In other words, the set of recognized strings has to form a *prefix code*.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *term* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Terminal control

Copyright

```
Copyright (c) 2006 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

term_send

Keywords: term_send, character output, control, terminal

Name

term::send - General output to terminals

Synopsis

package require Tcl 8.5 9
package require term::send ?**0.2**?
::term::send::wrch *chan str*
::term::send::wr *str*

Description

This package provides the most primitive commands for sending characters to a terminal. They are in essence convenient wrappers around the builtin command **puts**.

::term::send::wrch *chan str*
: Send the text *str* to the channel specified by the handle *chan*. In contrast to the builtin command **puts** this command does not terminate the string with a line terminator. It also forces an flush of Tcl internal and OS buffers to ensure that the characters are processed immediately.

::term::send::wr *str*
: This convenience command is like **::term::send::wrch**, except that the destination channel is fixed to *stdout*.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib

Trackers and specify *term* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Terminal control

Copyright

```
Copyright (c) 2006 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

textutil

adjust

Keywords: adjust, TeX, adjusting, formatting, hyphenation, indenting, justification, paragraph, string, undenting

Name

textutil::adjust - Procedures to adjust, indent, and undent paragraphs

Synopsis

package require Tcl 8.5 9
package require textutil::adjust ?**0.7.4**?
::textutil::adjust::adjust *string* ?*option value* ...?
::textutil::adjust::readPatterns *filename*
::textutil::adjust::listPredefined
::textutil::adjust::getPredefined *filename*
::textutil::adjust::indent *string prefix* ?*skip*?
::textutil::adjust::undent *string*

Description

The **textutil::adjust** package provides commands to indent, unindent, and justify the text held in a string.

API

::textutil::adjust::adjust *string ?option value ...?*

Returns a copy of *string* modified according to the given options.

Any newlines in the original *string* are treated as spaces, i.e., the input *string* is handled as a single paragraph of text. If the result string is shorter than the **-length**, it will not contain any newlines. Otherwise, the result will contain newlines as needed to perform the desired justification.

Together with **::textutil::adjust::indent** it is possible to create properly wrapped paragraphs with arbitrary indentations.

Under the hood, every newline, tab, or sequence of two or more spaces are treated as a single space, and single spaces are used to separate the line's words. These are the algorithm's *real* lines. Every *real* line is placed in a *logical* line, which is exactly **-length** characters wide (even if the *real* line is shorter). Every word in a *real* line is separated by a single space, except when the **-justify plain** option is used, in which case words are separated by one or more spaces as needed to fill the *real* line.

By default, trailing spaces (including tabs which are first replaced by spaces) are trimmed. See the **-full** option below.

The following options may be given after the *string* parameter. They control how the command places each *real* line in its corresponding *logical* line.

-full *boolean*
: If set to **0** (false; the default), any trailing spaces are deleted from each line before returning the string. If set to **1** (true), any trailing spaces are left at the end each line.

-hyphenate *boolean*
: If set to **0** (false; the default), no hyphenation will be done. If set to **1** (true), the command will try to hyphenate the last word of each line when appropriate.

  If **-hyphenate** is set to **1**, hyphenation patterns *must* have been loaded by calling the **::textutil::adjust::readPatterns** command, *before* the call to **::textutil::adjust::adjust** ; otherwise **::textutil::adjust::adjust** will throw an error.

-justify center|left|plain|right
: Sets the justification applied to the returned string to any one of: **left** (the default), **center**, **plain** (justified), or **right**. The justification means that any line in the returned string (possibly excluding the last line), is justified according to the given justification.

  If the justification is set to **plain** (justified), and the number of printable characters in the last line is less than 90% of **-length**, then the last line is justified **left**, avoiding the padding of this line when it is too short. The meaning of each value is:

  center
  : Each *real* line is centered in the *logical* line by padding the *real* line with spaces at the beginning and end as needed. If the last line is less than 90% of **-length**, it will be aligned **left** instead of **center**. (See also the **-full** option.)

  left
  : Each *real* line is placed at the left of the *logical* line, with no preceding spaces. If **-full** is **1** (true), the *real* line will be padded with spaces at the end to make it **-length** characters wide.

  plain
  : Each line is fully justified. This means that each *real* line's words are placed in the *logical* line, separated by one or more spaces, as needed to completely fill the line with no leading or trailing spaces.

  right
  : Each *real* line is placed at the right of the *logical* line, with no trailing spaces, and padded with leading spaces if needed to make the *real* line **-length** characters wide.

-length *integer*
: Sets the *logical* line length to be *integer*. The *integer* must be a positive integer *> 0*. The

default is **72**.

-strictlength ***boolean***
If set to **0** (false; the default), a line can exceed the specified **-length** if a single word is longer than **-length**. If set to **1** (true), words that are longer than **-length** are split so that no line exceeds the specified **-length**.

::textutil::adjust::readPatterns ***filename***
Reads hyphenation patterns from the file called *filename*.

This command is needed only if command **::textutil::adjust::adjust** is used with **-hyphenate 1** (true). In this case the call to **::textutil::adjust::readPatterns** *must* be done *before* calling **::textutil::adjust::adjust**.

The **textutil::adjust** package comes with a number of predefined pattern files. Use the **::textutil::adjust::listPredefined** command to list their names.

::textutil::adjust::listPredefined
Returns a list of the names of the hyphenation files provided by this package.

::textutil::adjust::getPredefined ***filename***
Returns the full path and filename of the *filename* hyphenation file. *Note* that *filename must* be one of the names returned by **::textutil::adjust::listPredefined**.

::textutil::adjust::indent ***string prefix*** **?*skip*?**
Returns a modified copy of *string* with every non-skipped line prefixed with *prefix* and whitespace (except for the newline), trimmed from the end. By default no lines are skipped, so all are prefixed.

The **skip** option specifies how many lines to skip at the start of the string; its default is **0**, in which case none are skipped and all are prefixed. If **skip** is set to a value > *0*, then that many initial lines in the string are skipped, i.e., not prefixed, while all the subsequent lines are prefixed. Setting **skip** to **1** will create a conventional hanging indent.

Together with **::textutil::adjust::adjust** it is possible to create properly wrapped paragraphs with arbitrary indentations.

::textutil::adjust::undent ***string***
Returns a modified copy of *string* with any common prefix of whitespace removed from each line.

Empty lines and lines containing only whitespace are ignored for the computation of the prefix and all these lines are returned as empty lines.

Note that this command's name is **undent** *not unindent.*

Examples

The following examples show some of the **::textutil::adjust** package's commands in action.

The following **proc** and **const** are used by the examples below.

```
proc replace_spc {s {sub ▲}} { regsub -all { } $s $sub }
const LINES "Four\nsiamese cats\nheard barks\nand bolted."
```

The **replace_spc** command replaces every space in the string it is passed with the ▲ symbol. The **LINES** constant provides some text for the examples to work with.

Justifying

The following example show every justification option in use. Spaces have been made visible by preserving them using **-full 1** and replacing them with the ▲ symbol.

Normally, **-full** would *not* be used and there would be *no* trailing spaces in each line.

```
set lines [textutil::adjust::adjust $LINES -length 20 -full 1 -justify
    left]
puts "======= left =======\n[replace_spc $lines]"
set lines [textutil::adjust::adjust $LINES -length 20 -full 1 -justify
    center]
puts "====== center ======\n[replace_spc $lines]"
set lines [textutil::adjust::adjust $LINES -length 20 -full 1 -justify
    plain]
puts "======= plain ======\n[replace_spc $lines]"
set lines [textutil::adjust::adjust $LINES -length 20 -full 1 -justify
    right]
puts "======= right ======\n[replace_spc $lines]"
=>
======= left =======
Four▲siamese▲cats▲▲▲
heard▲barks▲and▲▲▲▲▲
bolted.▲▲▲▲▲▲▲▲▲▲▲▲▲

====== center ======
▲▲Four▲siamese▲cats▲
▲▲▲heard▲barks▲and▲▲
▲▲▲▲▲▲▲bolted.▲▲▲▲▲▲

======= plain ======
Four▲siamese▲▲▲▲cats
heard▲▲▲barks▲▲▲▲and
bolted.▲▲▲▲▲▲▲▲▲▲▲▲▲

======= right ======
▲▲▲Four▲siamese▲cats
▲▲▲▲▲heard▲barks▲and
▲▲▲▲▲▲▲▲▲▲▲▲▲bolted.
```

Indenting and unindenting

In the following example leading spaces have been made visible by using an indent of ▪▪ rather than, say, two spaces, and other spaces have been made visible by replacing them with the ▲

symbol. There are no trailing spaces since **textutil::adjust::indent** trims them.

```
set lines [textutil::adjust::adjust $LINES -length 20 -justify left]
puts "======== left =========\n[replace_spc $lines]"
set indented [textutil::adjust::indent $lines "··"]
puts "======== indent =======\n[replace_spc $indented]"
# undent only unindents spaces
set indented [regsub -all · $indented " "]
set unindented [textutil::adjust::undent $indented]
puts "======== undent =======\n[replace_spc $unindented]"
set hanging [textutil::adjust::indent $lines "··" 1]
puts "==== hanging indent ===\n[replace_spc $hanging]"
=>
======== left =========
Four▲siamese▲cats
heard▲barks▲and
bolted.
======== indent =======
··Four▲siamese▲cats
··heard▲barks▲and
··bolted.
======== undent =======
Four▲siamese▲cats
heard▲barks▲and
bolted.
==== hanging indent ===
Four▲siamese▲cats
··heard▲barks▲and
··bolted.
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *textutil* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Text processing

expander

Keywords: expander, string, template processing, text expansion

Name

textutil::expander - Procedures to process templates and expand text.

Synopsis

package require Tcl 8.5 9
package require textutil::expander ?**1.3.2**?
::textutil::expander *expanderName*
expanderName **cappend** *text*
expanderName **cget** *varname*
expanderName **cis** *cname*
expanderName **cname**
expanderName **cpop** *cname*
expanderName **ctopandclear**
expanderName **cpush** *cname*
expanderName **cset** *varname value*
expanderName **cvar** *varname*
expanderName **errmode** *newErrmode*
expanderName **evalcmd** ?*newEvalCmd*?
expanderName **expand** *string* ?*brackets*?
expanderName **lb** ?*newbracket*?
expanderName **rb** ?*newbracket*?
expanderName **reset**
expanderName **setbrackets** *lbrack rbrack*
expanderName **textcmd** ?*newTextCmd*?
expanderName **where**

Description

The **textutil::expander** package provides a string templating facility that is more powerful and flexible than using the built-in subst command.

See the Tutorial section to learn the package, and the API section for the complete details.

Expander vs. subst

The Tcl **subst** command is often used to support a kind of template processing. Given a string with embedded variables or function calls, **subst** will interpolate the variable and function values, returning the new string:

```
% set greeting "Howdy"
Howdy
% proc place {} {return "World"}
% subst {$greeting, [place]!}
Howdy, World!
%
```

By defining a suitable set of Tcl commands, **subst** can be used to implement a markup language similar to HTML.

The **subst** command is efficient, but it has three drawbacks for this kind of template processing:

- There's no way to identify and process the plain text between two embedded Tcl commands; that makes it difficult to handle plain text in a context-sensitive way.

- Embedded commands are necessarily bracketed by **[** and **]**; it's convenient to be able to choose different brackets in special cases. Someone producing web pages that include a large quantity of Tcl code examples might easily prefer to use **<<** and **>>** as the embedded code delimiters instead.
- There's no easy way to handle incremental input, as one might wish to do when reading data from a socket.

At present, expander solves the first two problems.

The following section describes the command API to the expander; this is followed by the tutorial sections, see Tutorial.

API

The **textutil::expander** package provides only one command, described below. The rest of the section is taken by a description of the methods for the expander objects created by this command.

::textutil::expander *expanderName*

The command creates a new expander object with an associated Tcl command whose name is *expanderName*. This command may be used to invoke various methods. If the *expanderName* is not fully qualified it is interpreted as relative to the current namespace. The method calls have the following general form:

```
expanderName method ?arg ...?
```

Expander objects support the following methods:

expanderName* cappend *text

Appends a string to the output in the current context. This command should rarely be used by macros or application code.

expanderName* cget *varname

Returns the value of variable *varname*, defined in the current context.

expanderName* cis *cname

Returns **1** (true) if the current context's name is *cname*; otherwise returns **0** (false). *cname*.

***expanderName* cname**

Returns the name of the current context.

expanderName* cpop *cname

Pops a context from the context stack, returning all accumulated output in that context. The context must be called *cname*, or an error is thrown.

***expanderName* ctopandclear**

Returns the output currently captured in the topmost context and clears that buffer. This is similar to a combination of **cpop** followed by **cpush**, except that the internal state (brackets) is preserved here.

expanderName* cpush *cname

Pushes a context called *cname* onto the context stack. The context must be popped by **cpop** before expansion ends or an error is thrown.

expanderName* cset *varname value

Sets variable *varname* to *value* in the current context.

expanderName* cvar *varname

Returns the internal variable name of context variable *varname*; this allows the variable to be passed to commands like **lappend**.

expanderName* errmode *newErrmode

Sets the macro expansion error mode to one of **nothing**, **macro**, **error**, or **fail**; the default value is **fail**. The value determines what the expander does if an error is detected during the expansion of a macro.

fail
: The error propagates normally (i.e., is thrown), and can be caught or ignored by the application.

error
: The macro expands into a detailed error message, and expansion continues.

macro
: The macro expands to itself; that is, it is passed along to the output unchanged.

nothing
: The macro expands to the empty string, and is effectively ignored.

***expanderName* evalcmd ?*newEvalCmd*?**

Returns the current evaluation command, which defaults to **uplevel #0**. If specified, *newEvalCmd* will be saved for future use and then returned; it must be a Tcl command expecting one additional argument: the macro to evaluate.

***expanderName* expand *string* ?*brackets*?**

Expands the input string, replacing embedded macros with their expanded values, and returns the expanded string.

Note that this method pushes a new (empty) context on the stack of contexts while it is running, and removes it on return.

If *brackets* is given, it must be a list of two strings; the items will be used as the left and right macro expansion bracket sequences for this expansion only.

***expanderName* lb ?*newbracket*?**

Returns the current value of the left macro expansion bracket; this is for use as or within a macro, when the bracket needs to be included in the output text. If *newbracket* is specified, it becomes the new bracket, and is returned.

***expanderName* rb ?*newbracket*?**

Returns the current value of the right macro expansion bracket; this is for use as or within a macro, when the bracket needs to be included in the output text. If *newbracket* is specified, it becomes the new bracket, and is returned.

***expanderName* reset**

Resets all expander settings to their initial values. Unexpected results are likely if this command is called from within a call to **expand**.

expanderName* setbrackets *lbrack rbrack

Sets the left and right macro expansion brackets. This command is for use as or within a macro, or to permanently change the bracket definitions. By default, the brackets are **[** and **]**, but any non-empty string can be used; for example, **<** and **>** or **(*** and ***)** or even **Hello,** and **World!**.

***expanderName* textcmd ?*newTextCmd*?**
Returns the current command for processing plain text, which defaults to the empty string, meaning *identity*. If specified, *newTextCmd* will be saved for future use and then returned; it must be a Tcl command expecting one additional argument: the text to process. The expander object will use this command for all plain text it encounters, giving the user of the object the ability to process all plain text in some standard way before writing it to the output. The object expects that the command returns the processed plain text.

Note that the combination of "**textcmd** *plaintext*" is run through the *evalcmd* for the actual evaluation. In other words, the *textcmd* is treated as a special macro implicitly surrounding all plain text in the template.

***expanderName* where**
Returns a three-element list containing the current character position, line, and column the expander has reached in the processing of the current input string.

Tutorial

Basics

To begin, create an expander object:

```
% package require textutil::expander
1.2
% ::textutil::expander myexp
::myexp
%
```

The created **::myexp** object can be used to expand text strings containing embedded Tcl commands. By default, embedded commands are delimited by square brackets. Note that expander doesn't attempt to interpolate variables, since variables can be referenced by embedded commands:

```
% set greeting "Howdy"
Howdy
% proc place {} {return "World"}
% ::myexp expand {[set greeting], [place]!}
Howdy, World!
%
```

Embedding macros

An expander macro is simply a Tcl script embedded within a text string. Expander evaluates the script in the global context, and replaces it with its result string. For example,

```
% set greetings {Howdy Hi "What's up"}
Howdy Hi "What's up"
% ::myexp expand {There are many ways to say "Hello, World!":
[set result {}
foreach greeting $greetings {
append result "$greeting, World!\\n"
```

```
    }
    set result]
    And that's just a small sample!}
    There are many ways to say "Hello, World!":
    Howdy, World!
    Hi, World!
    What's up, World!

    And that's just a small sample!
    %
```

Writing macro commands

More typically, *macro commands* are used to create a markup language. A macro command is just a Tcl command that returns an output string. For example, expand can be used to implement a generic document markup language that can be retargeted to HTML or any other output format:

```
    % proc bold {} {return "<b>"}
    % proc /bold {} {return "</b>"}
    % ::myexp expand {Some of this text is in [bold]boldface[/bold]}
    Some of this text is in <b>boldface</b>
    %
```

The above definitions of **bold** and **/bold** returns HTML, but such commands can be as complicated as needed; they could, for example, decide what to return based on the desired output format.

Changing the expansion brackets

By default, embedded macros are enclosed in square brackets, **[** and **]**. If square brackets must be included in the output, the input can use the **lb** and **rb** commands. Alternatively, or if square brackets are unwanted for some other reason, the macro expansion brackets can be changed to any pair of nonempty strings.

The **setbrackets** method changes the brackets for all subsequent expansions applied to the current expansion object. For example, you can write pseudo-html by change them to **<** and **>**:

```
    % ::myexp setbrackets < >
    % ::myexp expand {<bold>This is boldface</bold>}
    <b>This is boldface</b>
```

Alternatively, you can change the expansion brackets temporarily by passing the desired brackets to the **expand** method:

```
    % ::myexp setbrackets "\\[" "\\]"
    % ::myexp expand {<bold>This is boldface</bold>} {< >}
    <b>This is boldface</b>
    %
```

Customized macro expansion

By default, macros are evaluated using the Tcl **uplevel #0** command, so that the embedded code executes in the global context. The application can provide a different evaluation command using the **evalcmd** method; this allows the application to use a safe interpreter, for example, or even to evaluate something other than Tcl code. There is one caveat: to be recognized as valid, a macro must return **1** (true) when passed to Tcl's info complete subcommand.

For example, the following code "evaluates" each macro by returning the macro text itself.

```
    proc identity {macro} {return $macro}
    ::myexp evalcmd identity
```

Using the context stack

Often it's desirable to define a pair of macros which operate in some way on the plain text between them. Consider a set of macros for adding footnotes to a web page: one could have implemented something like this:

```
    Dr. Pangloss, however, thinks that this is the best of all
    possible worlds.[footnote "See Candide, by Voltaire"]
```

The **footnote** macro would, presumably, assign a number to this footnote and save the text to be formatted later on. However, this solution is ugly if the footnote text is long or should contain additional markup. Consider the following instead:

```
    Dr. Pangloss, however, thinks that this is the best of all
    possible worlds.[footnote]See [bookTitle "Candide"], by
    [authorsName "Voltaire"], for more information.[/footnote]
```

Here the footnote text is contained between **footnote** and **/footnote** macros, continues onto a second line, and contains several macros of its own. This is both clearer and more flexible; however, with the features presented so far there's no easy way to do it. That's the purpose of the context stack.

All macro expansion takes place in a particular context. Here, the **footnote** macro pushes a new context onto the context stack. Then, all expanded text gets placed in that new context. **/footnote** retrieves it by popping the context. Here's a skeleton implementation of these two macros:

```
    proc footnote {} {
        ::myexp cpush footnote
    }

    proc /footnote {} {
        set footnoteText [::myexp cpop footnote]

        # Save the footnote text, and return an appropriate footnote
        # number and link.
    }
```

The **cpush** method pushes a new context onto the stack; the argument is the context's name. It can be any string, but would typically be the name of the macro itself. Then, **cpop** verifies that the current context has the expected name, pops it off of the stack, and returns the

accumulated text.

Expand provides several other tools related to the context stack. Suppose the first macro in a context pair takes arguments or computes values which the second macro in the pair needs. After calling **cpush**, the first macro can define one or more context variables; the second macro can retrieve their values any time before calling **cpop**. For example, suppose the document must specify the footnote number explicitly:

```
    proc footnote {footnoteNumber} {
        ::myexp cpush footnote
        ::myexp csave num $footnoteNumber
        # Return an appropriate link
    }

    proc /footnote {} {
        set footnoteNumber [::myexp cget num]
        set footnoteText [::myexp cpop footnote]

        # Save the footnote text and its footnoteNumber for future
        # output.
    }
```

At times, it might be desirable to define macros that are valid only within a particular context pair; such macros should verify that they are only called within the correct context using either **cis** or **cname**.

History

The **expander** package was written by William H. Duquette; it is a repackaging of the central algorithm of the expand macro processing tool.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *textutil* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Documentation tools

Copyright

```
Copyright (c) William H. Duquette https://github.com/wjduquette
```

patch

Keywords: patch, diff -ruN, diff, unified format, fossil, git, unified format diff

Name

textutil::patch - Application of uni-diff patches to directory trees

Synopsis

package require Tcl 8.5 9
package require textutil::patch ?**0.2**?
::textutil::patch::apply *basedirectory striplevel patch reportcmd*
{*}reportcmd apply *filename*
{*}reportcmd fail *filename hunk expected seen*
{*}reportcmd fail-already *filename hunk*

Description

This package provides a single command which applies a patch in unified diff format to the files in a directory tree.

API

::textutil::patch::apply *basedirectory striplevel patch reportcmd*

Applies the *patch* (i.e., the text of the actual patch, not a filename) to the files in the *basedirectory* using the specified *striplevel*, and reports actions with calls to the command prefix *reportcmd*. Returns an empty string.

The *striplevel* argument is equivalent to the patch command's **-p** (or **--strip**) option with a value of *striplevel*.

Throws an error if *patch* cannot be parsed, or if nothing is done to the files in *basedirectory*.

Actions that occur during the patching, including the inability to apply a hunk, are reported through the command prefix *reportcmd*. Files with problems are left unchanged, while files without problems—whether they are processed before or after problem files—will be changed in accordance with the *patch*.

Each call to the command prefix can take one of three possible forms:

{*}reportcmd apply *filename*
: This call is made at the start of each patch apply and is passed the *filename* of the file being patched. An attempt is made to apply all collected hunks for the *filename*.

{*}reportcmd fail *filename hunk expected seen*
: This call is made every time the application of a hunk fails. The command is passed the *filename* being processed, the *hunk* number, what the *expected* text was, and what the text that was actually *seen* is.

{*}reportcmd fail-already *filename hunk*
: This call is made every time the application of a hunk fails due to the given *hunk* number having *already* been applied to the *filename*.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *textutil* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Text processing

repeat
Keywords: repeat, blanks, repetition, string

Name
textutil::repeat - Procedures to repeat strings.

Synopsis
package require Tcl 8.5 9
package require textutil::repeat ?**0.8**?
::textutil::repeat::strRepeat *text count*
::textutil::repeat::blank *count*

Description
The **textutil::repeat** package provides commands to generate strings by repeating a string multiple times.

API
::textutil::repeat::strRepeat *text count*
Returns a string containing the *text* repeated *count* times.

The repetitions are joined with no separators between them. A value of *count* <= *0* causes the command to return an empty string.

Note: If the Tcl core this package is loaded in provides the **string repeat** command, then this command will be implemented in its terms, for maximum speed. Otherwise a fast Tcl implementation is used.

::textutil::repeat::blank *count*
Returns a string containing *count* spaces.

This is a convenience for calling **::textutil::repeat::strRepeat** with **" "** as its first argument.

Examples
This tiny example shows the **::textutil::repeat** package's commands in action.

```
puts [::textutil::repeat::strRepeat = 22]
puts =[::textutil::repeat::blank 20]=
=>
======================
=                     =
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *textutil* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Text processing

tabify

Keywords: tabify, formatting, string, tabstops

Name

textutil::tabify - Procedures to (un)tabify strings

Synopsis

package require Tcl 8.5 9
package require textutil::tabify ?**0.8**?
::textutil::tabify::tabify *string* ?*count*?
::textutil::tabify::tabify2 *string* ?*count*?
::textutil::tabify::untabify *string* ?*count*?
::textutil::tabify::untabify2 *string* ?*count*?

Description

The **textutil::tabify** package provides commands for converting tabs to spaces or spaces to tabs in strings.

API

::textutil::tabify::tabify *string* ?*count*?
: Returns a copy of *string* with any substring of *count* space characters replaced by a horizontal tab. The *count* defaults to 8.

::textutil::tabify::tabify2 *string* ?*count*?
: Returns a copy of *string* with substrings of *count or fewer* space characters replaced by a horizontal tab using a text editor like algorithm. The *count* defaults to 8.

  The algorithm used by this command treats each line in *string* as if it had notional tabstops every *count* columns. (So, if *count* is **8**, at columns **8**, **16**, **24**, ….) Only sequences of *two or more* space characters that occur immediately before a notional tabstop are replaced with a tab.

::textutil::tabify::untabify *string* ?*count*?
: Returns a copy of *string* with every horizontal tab replaced by *count* spaces. The *count* defaults to 8.

::textutil::tabify::untabify2 *string* ?*count*?

Returns a copy of *string* with horizontal tabs replaced by *count or fewer* space characters using a text editor like algorithm. The *count* defaults to 8.

This command is the counterpart to **::textutil::tabify::tabify2**. The algorithm used by this command treats each line in *string* as if it had notional tabstops every *count* columns. (So, if *count* is **8**, at columns **8**, **16**, **24**, ...) Instead of blindly replacing each horizontal tab with *count* spaces, each horizontal tab at a notional tabstop is replaced by enough spaces to reach the next notional tabstop.

There is one asymmetry though: A tab may be replaced by a single space, but not the other way around.

Examples

The following examples show some of the **::textutil::tabify** package's commands in action.

The examples make use of a tiny helper command to make spaces and tabs distinct and visible:

```
proc replace_tab_spc s { regsub -all { } [regsub -all \t $s →] ▴ }
```

This example shows the **::textutil::tabify::tabify** command in action.

```
const LINES1 "    if \{\$x\} \{\n        puts 1\n    \}"
puts "=== original ===\n[replace_tab_spc $LINES1]\n"
set tabbed [::textutil::tabify::tabify $LINES1]
puts "=== 8spc→tab ===\n[replace_tab_spc $tabbed]\n"
set tabbed [::textutil::tabify::tabify $LINES1 4]
puts "=== 4spc→tab ===\n[replace_tab_spc $tabbed]\n"
=>
=== original ===
▴▴▴▴if▴{$x}▴{
▴▴▴▴▴▴▴▴puts▴1
▴▴▴▴}

=== 8spc→tab ===
▴▴▴▴if▴{$x}▴{
→puts▴1
▴▴▴▴}

=== 4spc→tab ===
→if▴{$x}▴{
→→puts▴1
→}
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *textutil* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Text processing

textutil
Keywords: textutil, TeX, formatting, hyphenation, indenting, paragraph, regular expression, string, trimming

Name
textutil - Procedures to manipulate texts and strings.

Synopsis
package require Tcl 8.5 9
package require textutil ?**0.10**?
::textutil::adjust::adjust *string* ?*option value ...*?
::textutil::adjust::readPatterns *filename*
::textutil::adjust::listPredefined
::textutil::adjust::getPredefined *filename*
::textutil::adjust::indent *string prefix* ?*skip*?
::textutil::adjust::undent *string*
::textutil::repeat::strRepeat *text count*
::textutil::repeat::blank *count*
::textutil::split::splitn *string* ?*size*?
::textutil::split::splitx *string* ?*regexp*?
::textutil::string::chop *string*
::textutil::string::tail *string*
::textutil::string::cap *string*
::textutil::string::capEachWord *string*
::textutil::string::uncap *string*
::textutil::string::longestCommonPrefixList *list*
::textutil::string::longestCommonPrefix ?*string...*?
::textutil::tabify::tabify *string* ?*count*?
::textutil::tabify::tabify2 *string* ?*count*?
::textutil::tabify::untabify *string* ?*count*?
::textutil::tabify::untabify2 *string* ?*count*?
::textutil::trim::trim *string* ?*regexp*?
::textutil::trim::trimleft *string* ?*regexp*?
::textutil::trim::trimright *string* ?*regexp*?
::textutil::trim::trimPrefix *string prefix*
::textutil::trim::trimEmptyHeading *string*

Description
The **textutil** package provides commands that manipulate strings. This package is a bundle providing the commands of the six subpackages listed below, all of which are in the **textutil**

namespace.

- **textutil::adjust**
- **textutil::repeat**
- **textutil::split**
- **textutil::string**
- **textutil::tabify**
- **textutil::trim**

The bundle is *deprecated*, and it will be removed in a future Tcllib release. Instead of using this package, package require specific subpackages as needed.

For convenience, all of the **textutil** namespace's commands shown above are described in the API section below, in addition to being documented in the subpackages themselves.

Note that the following subpackages are also in the **textutil** namespace but are not part of the bundle, so they must be specifically package required as needed.

- **textutil::expander**
- **textutil::patch**
- **textutil::wcswidth**

The **textutil** namespace's commands shown above are described only in the subpackages themselves.

API

::textutil::adjust::adjust *string ?option value ...?*

Returns a copy of *string* modified according to the given options.

Any newlines in the original *string* are treated as spaces, i.e., the input *string* is handled as a single paragraph of text. If the result string is shorter than the **-length**, it will not contain any newlines. Otherwise, the result will contain newlines as needed to perform the desired justification.

Together with **::textutil::adjust::indent** it is possible to create properly wrapped paragraphs with arbitrary indentations.

Under the hood, every newline, tab, or sequence of two or more spaces are treated as a single space, and single spaces are used to separate the line's words. These are the algorithm's *real* lines. Every *real* line is placed in a *logical* line, which is exactly **-length** characters wide (even if the *real* line is shorter). Every word in a *real* line is separated by a single space, except when the **-justify plain** option is used, in which case words are separated by one or more spaces as needed to fill the *real* line.

By default, trailing spaces (including tabs which are first replaced by spaces) are trimmed. See the **-full** option below.

The following options may be given after the *string* parameter. They control how the command places each *real* line in its corresponding *logical* line.

-full *boolean*

If set to **0** (false; the default), any trailing spaces are deleted from each line before returning the string. If set to **1** (true), any trailing spaces are left at the end each line.

-hyphenate *boolean*

If set to **0** (false; the default), no hyphenation will be done. If set to **1** (true), the command

will try to hyphenate the last word of each line when appropriate.

If **-hyphenate** is set to **1**, hyphenation patterns *must* have been loaded by calling the **::textutil::adjust::readPatterns** command, *before* the call to **::textutil::adjust::adjust** ; otherwise **::textutil::adjust::adjust** will throw an error.

-justify center|left|plain|right
Sets the justification applied to the returned string to any one of: **left** (the default), **center**, **plain** (justified), or **right**. The justification means that any line in the returned string (possibly excluding the last line), is justified according to the given justification.

If the justification is set to **plain** (justified), and the number of printable characters in the last line is less than 90% of **-length**, then the last line is justified **left**, avoiding the padding of this line when it is too short. The meaning of each value is:

center
Each *real* line is centered in the *logical* line by padding the *real* line with spaces at the beginning and end as needed. If the last line is less than 90% of **-length**, it will be aligned **left** instead of **center**. (See also the **-full** option.)

left
Each *real* line is placed at the left of the *logical* line, with no preceding spaces. If **-full** is **1** (true), the *real* line will be padded with spaces at the end to make it **-length** characters wide.

plain
Each line is fully justified. This means that each *real* line's words are placed in the *logical* line, separated by one or more spaces, as needed to completely fill the line with no leading or trailing spaces.

right
Each *real* line is placed at the right of the *logical* line, with no trailing spaces, and padded with leading spaces if needed to make the *real* line **-length** characters wide.

-length *integer*
Sets the *logical* line length to be *integer*. The *integer* must be a positive integer *> 0*. The default is **72**.

-strictlength *boolean*
If set to **0** (false; the default), a line can exceed the specified **-length** if a single word is longer than **-length**. If set to **1** (true), words that are longer than **-length** are split so that no line exceeds the specified **-length**.

::textutil::adjust::readPatterns *filename*
Reads hyphenation patterns from the file called *filename*.

This command is needed only if command **::textutil::adjust::adjust** is used with **-hyphenate 1** (true). In this case the call to **::textutil::adjust::readPatterns** *must* be done *before* calling **::textutil::adjust::adjust**.

The **textutil::adjust** package comes with a number of predefined pattern files. Use the **::textutil::adjust::listPredefined** command to list their names.

::textutil::adjust::listPredefined
Returns a list of the names of the hyphenation files provided by this package.

::textutil::adjust::getPredefined *filename*
Returns the full path and filename of the *filename* hyphenation file. *Note* that *filename must* be one of the names returned by **::textutil::adjust::listPredefined**.

::textutil::adjust::indent *string prefix* ?*skip*?
Returns a modified copy of *string* with every non-skipped line prefixed with *prefix* and whitespace (except for the newline), trimmed from the end. By default no lines are skipped, so all are prefixed.

The **skip** option specifies how many lines to skip at the start of the string; its default is **0**, in which case none are skipped and all are prefixed. If **skip** is set to a value > *0*, then that many initial lines in the string are skipped, i.e., not prefixed, while all the subsequent lines are prefixed. Setting **skip** to **1** will create a conventional hanging indent.

Together with **::textutil::adjust::adjust** it is possible to create properly wrapped paragraphs with arbitrary indentations.

::textutil::adjust::undent *string*
Returns a modified copy of *string* with any common prefix of whitespace removed from each line.

Empty lines and lines containing only whitespace are ignored for the computation of the prefix and all these lines are returned as empty lines.

Note that this command's name is **undent** *not unindent.*

::textutil::repeat::strRepeat *text count*
Returns a string containing the *text* repeated *count* times.

The repetitions are joined with no separators between them. A value of *count* <= *0* causes the command to return an empty string.

Note: If the Tcl core this package is loaded in provides the **string repeat** command, then this command will be implemented in its terms, for maximum speed. Otherwise a fast Tcl implementation is used.

::textutil::repeat::blank *count*
Returns a string containing *count* spaces.

This is a convenience for calling **::textutil::repeat::strRepeat** with **" "** as its first argument.

::textutil::split::splitn *string* ?*size*?
Returns a list of strings containing consecutive substrings of *string*, each *size* long—except for the last which will be shorter if *string*'s length is not an exact multiple of *size*. The *size* defaults to **1**, i.e., return a list of *string*'s characters; this is the same as calling the built-in split command with a second argument of **""**.

If *string* is empty, an empty list is returned. If *size* <= *0*, an error will be thrown.

::textutil::split::splitx *string* ?*regexp*?
Returns a list of strings containing consecutive substrings of *string* split according to the *regexp* regular expression which defaults to *[\t \r\n]+* (tabs, spaces, newlines).

Note: If *regexp* contains capturing parentheses, then the captured groups will be included in the result list as additional elements.

If *string* is empty, an empty list is returned. If *regexp* is empty, the *string* is split at every character; this is the same as calling the built-in split command with a second argument of **""**.

::textutil::string::chop *string*
Returns a copy of *string* with the last character removed.

::textutil::string::tail *string*
Returns a copy of *string* with the first character removed.

::textutil::string::cap *string*
Returns a copy of *string* with the first character capitalized.

::textutil::string::capEachWord *string*
Returns a copy of *string* with the first character of every word capitalized.

::textutil::string::uncap *string*
Returns a copy of *string* with the first character lowercased.

::textutil::string::longestCommonPrefixList *list*
Returns the longest common prefix of the strings in the given *list*.

If no argument is given, the empty string is returned.

::textutil::string::longestCommonPrefix ?*string*...?
Returns the longest common prefix of the string arguments, *string*, ...

If a single string is given, it is returned since it is its own longest common prefix. If no argument is given, the empty string is returned.

::textutil::tabify::tabify *string* ?*count*?
Returns a copy of *string* with any substring of *count* space characters replaced by a horizontal tab. The *count* defaults to 8.

::textutil::tabify::tabify2 *string* ?*count*?
Returns a copy of *string* with substrings of *count or fewer* space characters replaced by a horizontal tab using a text editor like algorithm. The *count* defaults to 8.

The algorithm used by this command treats each line in *string* as if it had notional tabstops every *count* columns. (So, if *count* is **8**, at columns **8**, **16**, **24**, ...) Only sequences of *two or more* space characters that occur immediately before a notional tabstop are replaced with a tab.

::textutil::tabify::untabify *string* ?*count*?
Returns a copy of *string* with every horizontal tab replaced by *count* spaces. The *count* defaults to 8.

::textutil::tabify::untabify2 *string* ?*count*?
Returns a copy of *string* with horizontal tabs replaced by *count or fewer* space characters

using a text editor like algorithm. The *count* defaults to 8.

This command is the counterpart to **::textutil::tabify::tabify2**. The algorithm used by this command treats each line in *string* as if it had notional tabstops every *count* columns. (So, if *count* is **8**, at columns **8**, **16**, **24**, ...) Instead of blindly replacing each horizontal tab with *count* spaces, each horizontal tab at a notional tabstop is replaced by enough spaces to reach the next notional tabstop.

There is one asymmetry though: A tab may be replaced by a single space, but not the other way around.

::textutil::trim::trim *string* ?*regexp*?
Returns a copy of *string* with any leading or trailing substring that matches *regexp* removed from the *string* if does not contain newlines, or from *every line* within the *string* if it contains newlines. The *regexp* defaults to *[\t]+*. This default will remove leading and trailing spaces and tabs from the *string* if it does not contain newlines, or from *every line* within the *string* if it contains newlines.

::textutil::trim::trimleft *string* ?*regexp*?
Returns a copy of *string* with any leading substring that matches *regexp* removed from the *string* if does not contain newlines, or from *every line* within the *string* if it contains newlines. The *regexp* defaults to *[\t]+*. This default will remove leading spaces and tabs from the *string* if it does not contain newlines, or from *every line* within the *string* if it contains newlines.

::textutil::trim::trimright *string* ?*regexp*?
Returns a copy of *string* with any trailing substring that matches *regexp* removed from the *string* if does not contain newlines, or from *every line* within the *string* if it contains newlines. The *regexp* defaults to *[\t]+*. This default will remove trailing spaces and tabs from the *string* if it does not contain newlines, or from *every line* within the *string* if it contains newlines.

::textutil::trim::trimPrefix *string prefix*
Returns a copy of *string* with *prefix* removed from the start if it is present; otherwise returns *string* unchanged.

Contrast this command with the built-in string command's **trim**, **trimleft**, **trimright** subcommands which remove all characters in a given set of characters passed as a string.

::textutil::trim::trimEmptyHeading *string*
Returns a copy of *string* with any leading blank lines (including lines containing only whitespace) removed.

Examples

Adjust

The following examples show some of the **::textutil::adjust** package's commands in action.

The following **proc** and **const** are used by the examples below.

```
proc replace_spc {s {sub ▲}} { regsub -all { } $s $sub }
const LINES "Four\nsiamese cats\nheard barks\nand bolted."
```

The **replace_spc** command replaces every space in the string it is passed with the ▲ symbol.

The **LINES** constant provides some text for the examples to work with.

Justifying

The following example show every justification option in use. Spaces have been made visible by preserving them using **-full 1** and replacing them with the ▲ symbol.

Normally, **-full** would *not* be used and there would be *no* trailing spaces in each line.

```
set lines [textutil::adjust::adjust $LINES -length 20 -full 1 -justify
    left]
puts "======= left =======\n[replace_spc $lines]"
set lines [textutil::adjust::adjust $LINES -length 20 -full 1 -justify
    center]
puts "====== center ======\n[replace_spc $lines]"
set lines [textutil::adjust::adjust $LINES -length 20 -full 1 -justify
    plain]
puts "======= plain ======\n[replace_spc $lines]"
set lines [textutil::adjust::adjust $LINES -length 20 -full 1 -justify
    right]
puts "======= right ======\n[replace_spc $lines]"
=>
======= left =======
Four▲siamese▲cats▲▲▲
heard▲barks▲and▲▲▲▲▲
bolted.▲▲▲▲▲▲▲▲▲▲▲▲▲

====== center ======
▲▲Four▲siamese▲cats▲
▲▲▲heard▲barks▲and▲▲
▲▲▲▲▲▲▲bolted.▲▲▲▲▲▲

======= plain ======
Four▲siamese▲▲▲▲cats
heard▲▲▲barks▲▲▲▲and
bolted.▲▲▲▲▲▲▲▲▲▲▲▲▲

======= right ======
▲▲▲Four▲siamese▲cats
▲▲▲▲▲heard▲barks▲and
▲▲▲▲▲▲▲▲▲▲▲▲▲bolted.
```

Indenting and unindenting

In the following example leading spaces have been made visible by using an indent of ·· rather than, say, two spaces, and other spaces have been made visible by replacing them with the ▲ symbol. There are no trailing spaces since **textutil::adjust::indent** trims them.

```
set lines [textutil::adjust::adjust $LINES -length 20 -justify left]
puts "======== left =========\n[replace_spc $lines]"
set indented [textutil::adjust::indent $lines "··"]
puts "======== indent =======\n[replace_spc $indented]"
# undent only unindents spaces
set indented [regsub -all · $indented " "]
set unindented [textutil::adjust::undent $indented]
puts "======== undent =======\n[replace_spc $unindented]"
set hanging [textutil::adjust::indent $lines "··" 1]
puts "==== hanging indent ===\n[replace_spc $hanging]"
=>
======== left =========
Four▴siamese▴cats
heard▴barks▴and
bolted.
======== indent =======
··Four▴siamese▴cats
··heard▴barks▴and
··bolted.
======== undent =======
Four▴siamese▴cats
heard▴barks▴and
bolted.
==== hanging indent ===
Four▴siamese▴cats
··heard▴barks▴and
··bolted.
```

Repeat

This tiny example shows the **::textutil::repeat** package's commands in action.

```
puts [::textutil::repeat::strRepeat = 22]
puts =[::textutil::repeat::blank 20]=
=>
======================
=                    =
```

Split

The following examples show some of the **::textutil::split** package's commands in action.

To help make the examples' effects more visible, each element of the returned lists is bracketed using ⟨ and ⟩. For non-UTF-8 aware editors these can be replaced with **\u276C** and **\u276D**.

This first example shows how to split strings by size.

```
const LINE1 "α and 𝕫"
set chars [::textutil::split::splitn $LINE1] ;# splits into characters
puts ⟨[join $chars "⟩ ⟨"]⟩
set chars [split $LINE1 ""] ;# splits into characters
puts ⟨[join $chars "⟩ ⟨"]⟩
set chunks [::textutil::split::splitn $LINE1 3]
puts ⟨[join $chunks "⟩ ⟨"]⟩
=>
⟨α⟩ ⟨ ⟩ ⟨a⟩ ⟨n⟩ ⟨d⟩ ⟨ ⟩ ⟨𝕫⟩
⟨α⟩ ⟨ ⟩ ⟨a⟩ ⟨n⟩ ⟨d⟩ ⟨ ⟩ ⟨𝕫⟩
⟨α a⟩ ⟨nd ⟩ ⟨𝕫⟩
```

This second example shows how to split strings by regexp.

```
const LINE2 "a, be , cat, done , eagle"
set chunks [::textutil::split::splitx $LINE2] ;# splits on [\t \r\n]+
puts ⟨[join $chunks "⟩ ⟨"]⟩
set chunks [::textutil::split::splitx $LINE2 {\s*,\s*}]
puts ⟨[join $chunks "⟩ ⟨"]⟩
set chunks [::textutil::split::splitx $LINE2 {\s*(,)\s*}]
puts ⟨[join $chunks "⟩ ⟨"]⟩
=>
⟨a,⟩ ⟨be⟩ ⟨,⟩ ⟨cat,⟩ ⟨done⟩ ⟨,⟩ ⟨eagle⟩
⟨a⟩ ⟨be⟩ ⟨cat⟩ ⟨done⟩ ⟨eagle⟩
⟨a⟩ ⟨,⟩ ⟨be⟩ ⟨,⟩ ⟨cat⟩ ⟨,⟩ ⟨done⟩ ⟨,⟩ ⟨eagle⟩
```

String

The following examples show some of the **::textutil::string** package's commands in action.

This first example shows all the commands that work on a single string.

```
const LINE "has the σαβ cogs"
puts ""$LINE" const"
set line [::textutil::string::chop $LINE]
puts ""$line" chop"
set line [::textutil::string::tail $line]
puts ""$line" tail"
set line [::textutil::string::cap $line]
puts ""$line" cap"
set line [::textutil::string::capEachWord $line]
puts ""$line" capEachWord"
set line [::textutil::string::uncap $line]
puts ""$line" uncap"
=>
"has the σαβ cogs" const
"has the σαβ cog" chop
"as the σαβ cog" tail
"As the σαβ cog" cap
"As The Σαβ Cog" capEachWord
"as The Σαβ Cog" uncap
```

This example shows the longest common prefix commands in use.

```
const LIST [list handbag handcuff handful handle handy]
puts "[::textutil::string::longestCommonPrefixList $LIST]"
puts "[::textutil::string::longestCommonPrefix king queen knave]"
puts "[::textutil::string::longestCommonPrefix king kin kind kinks]"
=>
"hand"
""
"kin"
```

This example shows how to use the **::textutil::string::longestCommonPrefixList** command to find the common path from a list of paths.

```
proc longestCommonPath paths {
    const SEP [file separator]
    set path [::textutil::string::longestCommonPrefixList $paths]
    if {[string index $path end] ne $SEP} {
        if {[set j [string last $SEP $path]] > -1} {
            set path [string range $path 0 $j-1]
        }
    }
    if {$path ne $SEP} { set path [string trimright $path $SEP] }
    return $path
}

const PATHS [list /home/sally/music /home/sally/museums
    /home/sally/musings]
```

```
puts [::textutil::string::longestCommonPrefixList $PATHS]
puts [longestCommonPath $PATHS]
puts [longestCommonPath [list /bin / /sbin]]
=>
/home/sally/mus
/home/sally
/
```

Tabify

The following examples show some of the **::textutil::tabify** package's commands in action.

The examples make use of a tiny helper command to make spaces and tabs distinct and visible:

```
proc replace_tab_spc s { regsub -all { } [regsub -all \t $s →] ▴ }
```

This example shows the **::textutil::tabify::tabify** command in action.

```
const LINES1 "    if \{\$x\} \{\n        puts 1\n    \}"
puts "=== original ===\n[replace_tab_spc $LINES1]\n"
set tabbed [::textutil::tabify::tabify $LINES1]
puts "=== 8spc→tab ===\n[replace_tab_spc $tabbed]\n"
set tabbed [::textutil::tabify::tabify $LINES1 4]
puts "=== 4spc→tab ===\n[replace_tab_spc $tabbed]\n"
=>
=== original ===
▴▴▴▴if▴{$x}▴{
▴▴▴▴▴▴▴▴puts▴1
▴▴▴▴}

=== 8spc→tab ===
▴▴▴▴if▴{$x}▴{
→puts▴1
▴▴▴▴}

=== 4spc→tab ===
→if▴{$x}▴{
→→puts▴1
→}
```

Trim

The following examples show some of the **::textutil::trim** package's commands in action.

The following **proc** is used by some of the examples below.

```
proc replace_ws s { regsub -all { } [regsub -all \t [regsub -all \n $s
    ⏎] →] ▴ }
```

This example shows the use of **::textutil::trim::trimEmptyHeading** and **::textutil::trim::trim**.

```
const LINES1 "\t\n Alpha\t\n Beta \t\n Gamma \t\n"
puts "== const ============================\n[replace_ws $LINES1]"
set line [::textutil::trim::trimEmptyHeading $LINES1]
puts "== textutil::trim::trimEmptyHeading ==\n[replace_ws $line]"
set line [string trimleft $LINES1]
puts "== string trimleft ==================\n[replace_ws $line]"
set line [::textutil::trim::trim $LINES1]
puts "== textutil::trim::trim ==============\n[replace_ws $line]"
=>
== const ============================
→⏎▴Alpha→⏎▴Beta▴→⏎▴Gamma▴→⏎
== textutil::trim::trimEmptyHeading ==
▴Alpha→⏎▴Beta▴→⏎▴Gamma▴→⏎
== string trimleft ==================
Alpha→⏎▴Beta▴→⏎▴Gamma▴→⏎
== textutil::trim::trim ==============
⏎Alpha⏎Beta⏎Gamma⏎
```

Notice the subtle difference in behavior between **::textutil::trim::trimEmptyHeading** and **string trimleft**.

This example shows the use of the built-in **string trim** in combination with and **::textutil::trim::trim**.

```
const LINES2 [string trim "\t\n Delta\t\n Epsilon \t\n Zeta \t\n"]
puts "== const ============================\n[replace_ws $LINES2]"
set line [::textutil::trim::trim $LINES2]
puts "== textutil::trim::trim ==============\n[replace_ws $line]"
=>
== const ============================
Delta→⏎▴Epsilon▴→⏎▴Zeta
== textutil::trim::trim ==============
Delta⏎Epsilon⏎Zeta
```

This example contrasts the built-in **string trim** with **::textutil::trim::trimPrefix**.

```
const PATH /home/homer
puts "const='$PATH'"
puts "string trim \$PATH /home='[string trim $PATH /home]'"
puts "string trim \$PATH /home='[string trim $PATH ehmo/]'"
puts "::textutil::trim::trimPrefix \$PATH
    /home='[::textutil::trim::trimPrefix $PATH /home]'"
const LINE mimic
puts "string trim \$LINE mic='[string trim $LINE mic]'"
puts "::textutil::trim::trimPrefix \$LINE
    mic='[::textutil::trim::trimPrefix $LINE mic]'"
=>
const='/home/homer'
```

```
string trim $PATH /home='r'
string trim $PATH /home='r'
::textutil::trim::trimPrefix $PATH /home='/homer'
string trim $LINE mic=''
::textutil::trim::trimPrefix $LINE mic='mimic'
```

For the **PATH** examples, the **string trim** command trims all the individual characters that are in the string "/home", i.e., "/", "h", "o", "m", "e". The order of the characters in the string don't matter as the "ehmo/" example shows. Compare this with **::textutil::trim::trimPrefix** which trims the literal string "/home".

For the **LINE** examples, the **string trim** command trims all the individual characters that are in the string "mic", i.e., "m", "i", "c". Compare this with **::textutil::trim::trimPrefix** which trims the literal string "mic"; but since the **LINE** doesn't start with "mic", nothing is trimmed and the string is returned unchanged.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *textutil* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Text processing

textutil_split

Keywords: textutil_split, regular expression, split, string

Name

textutil::split - Procedures to split texts

Synopsis

package require Tcl 8.5 9
package require textutil::split ?**0.9**?
::textutil::split::splitn *string* ?*size*?
::textutil::split::splitx *string* ?*regexp*?

Description

The **textutil::split** package provides commands that split strings into substrings by size, or by regular expression.

To split on a particular character or set of characters, use the built-in split command. To split a string into its individual characters, use the built-in split command with a second argument of **""**.

API

::textutil::split::splitn *string* ?*size*?

Returns a list of strings containing consecutive substrings of *string*, each *size* long—except for the last which will be shorter if *string*'s length is not an exact multiple of *size*. The *size* defaults to **1**, i.e., return a list of *string*'s characters; this is the same as calling the built-in split command with a second argument of **""**.

If *string* is empty, an empty list is returned. If *size* <= *0*, an error will be thrown.

::textutil::split::splitx *string* ?*regexp*?

Returns a list of strings containing consecutive substrings of *string* split according to the *regexp* regular expression which defaults to *[\t \r\n]+* (tabs, spaces, newlines).

Note: If *regexp* contains capturing parentheses, then the captured groups will be included in the result list as additional elements.

If *string* is empty, an empty list is returned. If *regexp* is empty, the *string* is split at every character; this is the same as calling the built-in split command with a second argument of **""**.

Examples

The following examples show some of the **::textutil::split** package's commands in action.

To help make the examples' effects more visible, each element of the returned lists is bracketed using **❬** and **❭**. For non-UTF-8 aware editors these can be replaced with **\u276C** and **\u276D**.

This first example shows how to split strings by size.

```
const LINE1 "α and ℤ"
set chars [::textutil::split::splitn $LINE1] ;# splits into characters
puts ❬[join $chars "❭ ❬"]❭
set chars [split $LINE1 ""] ;# splits into characters
puts ❬[join $chars "❭ ❬"]❭
set chunks [::textutil::split::splitn $LINE1 3]
puts ❬[join $chunks "❭ ❬"]❭
=>
❬α❭ ❬ ❭ ❬a❭ ❬n❭ ❬d❭ ❬ ❭ ❬ℤ❭
❬α❭ ❬ ❭ ❬a❭ ❬n❭ ❬d❭ ❬ ❭ ❬ℤ❭
❬α a❭ ❬nd ❭ ❬ℤ❭
```

This second example shows how to split strings by regexp.

```
const LINE2 "a, be , cat, done , eagle"
set chunks [::textutil::split::splitx $LINE2] ;# splits on [\t \r\n]+
puts ⟨[join $chunks "⟩ ⟨"]⟩
set chunks [::textutil::split::splitx $LINE2 {\s*,\s*}]
puts ⟨[join $chunks "⟩ ⟨"]⟩
set chunks [::textutil::split::splitx $LINE2 {\s*(,)\s*}]
puts ⟨[join $chunks "⟩ ⟨"]⟩
=>
⟨a,⟩ ⟨be⟩ ⟨,⟩ ⟨cat,⟩ ⟨done⟩ ⟨,⟩ ⟨eagle⟩
⟨a⟩ ⟨be⟩ ⟨cat⟩ ⟨done⟩ ⟨eagle⟩
⟨a⟩ ⟨,⟩ ⟨be⟩ ⟨,⟩ ⟨cat⟩ ⟨,⟩ ⟨done⟩ ⟨,⟩ ⟨eagle⟩
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *textutil* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Text processing

textutil_string

Keywords: textutil_string, capitalize, chop, common prefix, formatting, prefix, string, uncapitalize

Name

textutil::string - Procedures to manipulate texts and strings.

Synopsis

package require Tcl 8.5 9
package require textutil::string ?**0.9**?
::textutil::string::chop *string*
::textutil::string::tail *string*
::textutil::string::cap *string*
::textutil::string::capEachWord *string*
::textutil::string::uncap *string*
::textutil::string::longestCommonPrefixList *list*
::textutil::string::longestCommonPrefix ?*string...*?

Description

The **textutil::string** package provides miscellaneous string manipulation commands.

To see how to find a common path from a list of paths, see the last of the Examples.

API

::textutil::string::chop *string*
Returns a copy of *string* with the last character removed.

::textutil::string::tail *string*
Returns a copy of *string* with the first character removed.

::textutil::string::cap *string*
Returns a copy of *string* with the first character capitalized.

::textutil::string::capEachWord *string*
Returns a copy of *string* with the first character of every word capitalized.

::textutil::string::uncap *string*
Returns a copy of *string* with the first character lowercased.

::textutil::string::longestCommonPrefixList *list*
Returns the longest common prefix of the strings in the given *list*.

If no argument is given, the empty string is returned.

::textutil::string::longestCommonPrefix ?*string*...?
Returns the longest common prefix of the string arguments, *string*, ...

If a single string is given, it is returned since it is its own longest common prefix. If no argument is given, the empty string is returned.

Examples

The following examples show some of the **::textutil::string** package's commands in action.

This first example shows all the commands that work on a single string.

```
const LINE "has the σαβ cogs"
puts ""$LINE" const"
set line [::textutil::string::chop $LINE]
puts ""$line" chop"
set line [::textutil::string::tail $line]
puts ""$line" tail"
set line [::textutil::string::cap $line]
puts ""$line" cap"
set line [::textutil::string::capEachWord $line]
puts ""$line" capEachWord"
set line [::textutil::string::uncap $line]
puts ""$line" uncap"
=>
"has the σαβ cogs" const
"has the σαβ cog" chop
"as the σαβ cog" tail
"As the σαβ cog" cap
"As The Σαβ Cog" capEachWord
"as The Σαβ Cog" uncap
```

This example shows the longest common prefix commands in use.

```
const LIST [list handbag handcuff handful handle handy]
puts "[::textutil::string::longestCommonPrefixList $LIST]"
puts "[::textutil::string::longestCommonPrefix king queen knave]"
puts "[::textutil::string::longestCommonPrefix king kin kind kinks]"
=>
"hand"
""
"kin"
```

This example shows how to use the **::textutil::string::longestCommonPrefixList** command to find the common path from a list of paths.

```
proc longestCommonPath paths {
    const SEP [file separator]
    set path [::textutil::string::longestCommonPrefixList $paths]
    if {[string index $path end] ne $SEP} {
        if {[set j [string last $SEP $path]] > -1} {
            set path [string range $path 0 $j-1]
        }
    }
    if {$path ne $SEP} { set path [string trimright $path $SEP] }
    return $path
}

const PATHS [list /home/sally/music /home/sally/museums
    /home/sally/musings]
puts [::textutil::string::longestCommonPrefixList $PATHS]
puts [longestCommonPath $PATHS]
puts [longestCommonPath [list /bin / /sbin]]
=>
/home/sally/mus
/home/sally
/
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *textutil* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Text processing

trim

Keywords: trim, prefix, regular expression, string, trimming

Name

textutil::trim - Procedures to trim strings

Synopsis

package require Tcl 8.5 9
package require textutil::trim ?**0.8**?
::textutil::trim::trim *string* ?*regexp*?
::textutil::trim::trimleft *string* ?*regexp*?
::textutil::trim::trimright *string* ?*regexp*?
::textutil::trim::trimPrefix *string prefix*
::textutil::trim::trimEmptyHeading *string*

Description

The **textutil::trim** package provides commands that trim strings in various ways, including using arbitrary regular expressions.

To trim strings of sets of characters (such as whitespace), use the built-in string command's **trim** , **trimleft**, **trimright** subcommands.

API

::textutil::trim::trim *string* ?*regexp*?
Returns a copy of *string* with any leading or trailing substring that matches *regexp* removed from the *string* if does not contain newlines, or from *every line* within the *string* if it contains newlines. The *regexp* defaults to *[\t]+*. This default will remove leading and trailing spaces and tabs from the *string* if it does not contain newlines, or from *every line* within the *string* if it contains newlines.

::textutil::trim::trimleft *string* ?*regexp*?
Returns a copy of *string* with any leading substring that matches *regexp* removed from the *string* if does not contain newlines, or from *every line* within the *string* if it contains newlines. The *regexp* defaults to *[\t]+*. This default will remove leading spaces and tabs from the *string* if it does not contain newlines, or from *every line* within the *string* if it contains newlines.

::textutil::trim::trimright *string* ?*regexp*?
Returns a copy of *string* with any trailing substring that matches *regexp* removed from the *string* if does not contain newlines, or from *every line* within the *string* if it contains newlines. The *regexp* defaults to *[\t]+*. This default will remove trailing spaces and tabs from the *string* if it does not contain newlines, or from *every line* within the *string* if it contains newlines.

::textutil::trim::trimPrefix *string prefix*
Returns a copy of *string* with *prefix* removed from the start if it is present; otherwise returns *string* unchanged.

Contrast this command with the built-in string command's **trim**, **trimleft**, **trimright** subcommands which remove all characters in a given set of characters passed as a string.

::textutil::trim::trimEmptyHeading *string*
Returns a copy of *string* with any leading blank lines (including lines containing only

whitespace) removed.

Examples

The following examples show some of the **::textutil::trim** package's commands in action.

The following **proc** is used by some of the examples below.

```
proc replace_ws s { regsub -all { } [regsub -all \t [regsub -all \n $s
    ⏎] →] ▲ }
```

This example shows the use of **::textutil::trim::trimEmptyHeading** and **::textutil::trim::trim**.

```
const LINES1 "\t\n Alpha\t\n Beta \t\n Gamma \t\n"
puts "== const ============================\n[replace_ws $LINES1]"
set line [::textutil::trim::trimEmptyHeading $LINES1]
puts "== textutil::trim::trimEmptyHeading ==\n[replace_ws $line]"
set line [string trimleft $LINES1]
puts "== string trimleft ==================\n[replace_ws $line]"
set line [::textutil::trim::trim $LINES1]
puts "== textutil::trim::trim ==============\n[replace_ws $line]"
=>
== const ============================
→⏎▲Alpha→⏎▲Beta▲→⏎▲Gamma▲→⏎
== textutil::trim::trimEmptyHeading ==
▲Alpha→⏎▲Beta▲→⏎▲Gamma▲→⏎
== string trimleft ==================
Alpha→⏎▲Beta▲→⏎▲Gamma▲→⏎
== textutil::trim::trim ==============
⏎Alpha⏎Beta⏎Gamma⏎
```

Notice the subtle difference in behavior between **::textutil::trim::trimEmptyHeading** and **string trimleft**.

This example shows the use of the built-in **string trim** in combination with and **::textutil::trim::trim**.

```
const LINES2 [string trim "\t\n Delta\t\n Epsilon \t\n Zeta \t\n"]
puts "== const ============================\n[replace_ws $LINES2]"
set line [::textutil::trim::trim $LINES2]
puts "== textutil::trim::trim ==============\n[replace_ws $line]"
=>
== const ============================
Delta→⏎▲Epsilon▲→⏎▲Zeta
== textutil::trim::trim ==============
Delta⏎Epsilon⏎Zeta
```

This example contrasts the built-in **string trim** with **::textutil::trim::trimPrefix**.

```
const PATH /home/homer
puts "const='$PATH'"
puts "string trim \$PATH /home='[string trim $PATH /home]'"
puts "string trim \$PATH /home='[string trim $PATH ehmo/]'"
puts "::textutil::trim::trimPrefix \$PATH
    /home='[::textutil::trim::trimPrefix $PATH /home]'"
const LINE mimic
puts "string trim \$LINE mic='[string trim $LINE mic]'"
puts "::textutil::trim::trimPrefix \$LINE
    mic='[::textutil::trim::trimPrefix $LINE mic]'"
=>
const='/home/homer'
string trim $PATH /home='r'
string trim $PATH /home='r'
::textutil::trim::trimPrefix $PATH /home='/homer'
string trim $LINE mic=''
::textutil::trim::trimPrefix $LINE mic='mimic'
```

For the **PATH** examples, the **string trim** command trims all the individual characters that are in the string "/home", i.e., "/", "h", "o", "m", "e". The order of the characters in the string don't matter as the "ehmo/" example shows. Compare this with **::textutil::trim::trimPrefix** which trims the literal string "/home".

For the **LINE** examples, the **string trim** command trims all the individual characters that are in the string "mic", i.e., "m", "i", "c". Compare this with **::textutil::trim::trimPrefix** which trims the literal string "mic"; but since the **LINE** doesn't start with "mic", nothing is trimmed and the string is returned unchanged.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *textutil* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Text processing

wcswidth

Keywords: wcswidth, character type, character width, double-wide character, prefix, regular expression, string

Name

textutil::wcswidth - Procedures to compute terminal width of strings

Synopsis

package require Tcl 8.5 9
package require textutil::wcswidth ?**35.3**?
::textutil::wcswidth *string*
::textutil::wcswidth_char *codepoint*
::textutil::wcswidth_type *codepoint*

Description

The **textutil::wcswidth** package provides commands that report character types, character widths, and string widths, in the context of terminal (console) output.

This package's data, and therefore its underlying the functionality, is based on the Unicode database file http://www.unicode.org/Public/UCD/latest/ucd/EastAsianWidth.txt.

API

This package's commands account for double-width characters from the various Asian and other scripts.

::textutil::wcswidth *string*
Returns the number of character cells needed by the given *string* when output to a terminal.

::textutil::wcswidth_char *codepoint*
Returns the number of character cells needed by the given *codepoint* Unicode character when output to a terminal.

Important: The *codepoint* Unicode character must be specified as an integer.

::textutil::wcswidth_type *codepoint*
Returns the character type of the specified *codepoint* Unicode character.

The returned value is a string containing one of the following constants: **A**, **F**, **H**, **N**, **Na**, or **W**, as specified in http://www.unicode.org/Public/UCD/latest/ucd/EastAsianWidth.txt.

Important: The *codepoint* Unicode character must be specified as an integer.

Examples

This example shows all three commands in action.

```
const LINE "Δ÷☕M☂𝕫"
puts "string length  = [string length $LINE]"
puts "terminal width = [textutil::wcswidth $LINE]"
foreach char [split $LINE ""] {
    set cp [scan $char %c]
    set pad [string repeat " " [expr {2 - [textutil::wcswidth_char
    $cp]}]]
    puts [format "%-2s%sU+%04X %s" $char $pad $cp
    [textutil::wcswidth_type $cp]]
}
=>
string length  = 6
```

```
terminal width = 8
Δ  U+0394 A
÷  U+00F7 A
☕ U+2615 W
M  U+004D N
☔ U+2614 W
ℤ  U+2124 N
```

On a terminal using a monospaced font, the output shown above lines up *perfectly*; any "wobble" seen on this page is an artifact.

Note that if **pad** was set to **" "**, i.e., taking no account of the character widths, the output would *not* line up, e.g.:

```
string length  = 6
terminal width = 8
Δ  U+0394 A
÷  U+00F7 A
☕  U+2615 W
M  U+004D N
☔  U+2614 W
ℤ  U+2124 N
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *textutil* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Text processing

tie

tie

Keywords: tie, array, database, file, metakit, persistence, untie

Name

tie - Array persistence

Synopsis

package require Tcl 8.5 9
package require tie ?**1.3**?
::tie::tie *arrayvarname options... dstype dsname...*

::tie::untie *arrayvarname* ?*token*?
::tie::info ties *arrayvarname*
::tie::info types
::tie::info type *dstype*
::tie::register *dsclasscmd* **as** *dstype*
dsclasscmd *objname* ?*dsname...*?
ds destroy
ds names
ds size
ds get
ds set *dict*
ds unset ?*pattern*?
ds setv *index value*
ds unsetv *index*
ds getv *index*

Description

The **tie** package provides a framework for the creation of persistent Tcl array variables. It should be noted that the provided mechanism is generic enough to also allow its usage for the distribution of the contents of Tcl arrays over multiple threads and processes, i.e. communication.

This, persistence and communication, is accomplished by *tying*) a Tcl array variable to a *data source*. Examples of data sources are other Tcl arrays and files.

It should be noted that a single Tcl array variable can be tied to more than one *data source*. It is this feature which allows the framework to be used for communication as well. Just tie several Tcl arrays in many client processes to a Tcl array in a server and all changes to any of them will be distributed to all. Less centralized variants of this are of course possible as well.

Using ties

Tie API

This section describes the basic API used to establish and remove ties between Tcl array variables and data sources. This interface is the only one a casual user has to be concerned about. The following sections about the various internal interfaces can be safely skipped.

::tie::tie *arrayvarname options... dstype dsname...*

This command establishes a tie between the Tcl array whose name is provided by the argument *arrayvarname* and the *data source* identified by the *dstype* and its series of *dsname* arguments. All changes made to the Tcl array after this command returns will be saved to the *data source* for safekeeping (or distribution).

The result of the command is always a token which identifies the new tie. This token can be used later to destroy this specific tie.

varname *arrayvarname* (in)

The name of the Tcl array variable to connect the new tie to.

name|command *dstype* (in)

This argument specifies the type of the *data source* we wish to access. The *dstype* can be one of **log**, **array**, **remotearray**, **file**, **growfile**, or **dsource**; in addition, the programmer

can register additional data source types. Each *dstype* is followed by one or more arguments that identify the *data source* to which the array is to be tied.

string *dsname* (in)
The series of *dsname* arguments coming after the *dstype* identifies the *data source* we wish to connect to, and has to be appropriate for the chosen type.

The command understands a number of additional options which guide the process of setting up the connection between Tcl array and *data source*.

-open
The Tcl array for the new tie is *loaded* from the *data source*, and the previously existing contents of the Tcl array are erased. Care is taken to *not* erase the previous contents should the creation of the tie fail.

This option and the option **-save** exclude each other. If neither this nor option **-save** are specified then this option is assumed as default.

-save
The Tcl array for the new tie is *saved* to the *data source*, and the previously existing contents of the *data source* are erased.

This option and the option **-open** exclude each other. If neither this nor option **-open** are specified then option **-open** is assumed as default.

-merge
Using this option prevents the erasure of any previously existing content and merges the data instead. It can be specified in conjunction with either **-open** or **-save**. They determine how data existing in both Tcl array and *data source*, i.e duplicates, are dealt with.

When used with **-open** data in the *data source* has precedence. In other words, for duplicates the data in the *data source* is loaded into the Tcl array.

When used with **-save** data in the Tcl array has precedence. In other words, for duplicates the data in the Tcl array is saved into the *data source*.

::tie::untie *arrayvarname* ?*token*?
This command dissolves one or more ties associated with the Tcl array named by *arrayvarname*. If no *token* is specified then all ties to that Tcl array are dissolved. Otherwise only the tie the token stands for is removed, if it is actually connected to the array. Trying to remove a specific tie not belonging to the provided array will cause an error.

It should be noted that while severing a tie will destroy management information internal to the package the *data source* which was handled by the tie will not be touched, only closed.

After the command returns none of changes made to the array will be saved to the *data source* anymore.

The result of the command is an empty string.

varname *arrayname* (in)
The name of a Tcl array variable which may have ties.

handle *token* (in)

A handle representing a specific tie. This argument is optional.

::tie::info ties *arrayvarname*
This command returns a list of ties associated with the Tcl array variable named by *arrayvarname*. The result list will be empty if the variable has no ties associated with it.

::tie::info types
This command returns a dictionary of registered types, and the class commands they are associated with.

::tie::info type *dstype*
This command returns the fully resolved class command for a type name. This means that the command will follow a chain of type definitions ot its end.

Standard data source types

This package provides the six following types as examples and standard data sources.

log
This *data source* does not maintain any actual data, nor persistence. It does not accept any identifying arguments. All changes are simply logged to **stdout**.

array
This *data source* uses a regular Tcl array as the origin of the persistent data. It accepts a single identifying argument, the name of this Tcl array. All changes are mirrored to that array.

remotearray
This *data source* is similar to **array**. The difference is that the Tcl array to which we are mirroring is not directly accessible, but through a **send**-like command.

It accepts three identifying arguments, the name of the other Tcl array, the command prefix for the **send**-like accessor command, and an identifier for the remote entity hosting the array, in this order. All changes are mirrored to that array, via the command prefix. All commands will be executed in the context of the global namespace.

send-like means that the command prefix has to have **send** syntax and semantics. I.e. it is a channel over which we can send arbitrary commands to some other entity. The remote array *data source* however uses only the commands **set**, **unset**, **array exists**, **array names**, **array set**, and **array get** to retrieve and set values in the remote array.

The command prefix and the entity id are separate to allow the data source to use options like **-async** when assembling the actual commands.

Examples of command prefixes, listed with the id of the remote entity, without options. In reality only the part before the id is the command prefix:

send *tkname*
The Tcl array is in a remote interpreter and is accessed via Tk's X communication.

comm::comm send *hostportid*
The Tcl array is in a remote interpreter and is accessed through a socket.

thread::send *threadid*
The Tcl array is in a remote interpreter in a different thread of this process.

file
This *data source* uses a single file as origin of the persistent data. It accepts a single

identifying argument, the path to this file. The file has to be both readable and writable. It may not exist, the *data source* will create it in that case. This (and only this) situation will require that the directory for the file exists and is writable as well.

All changes are saved in the file, as proper Tcl commands, one command per operation. In other words, the file will always contain a proper Tcl script.

If the file exists when the tie using it is set up, then it will be compacted, i.e. superfluous operations are removed, if the operations log stored in it contains either at least one operation clearing the whole array, or at least 1.5 times more operations than entries in the loaded array.

growfile

This *data source* is like **file** in terms of the storage medium for the array data, and how it is configured. In constrast to the former it however assumes and ensures that the tied array will never shrink. I.e. the creation of new array entries, and the modification of existing entries is allowed, but the deletion of entries is not, and causes the data source to throw errors.

This restriction allows us to simplify both file format and access to the file radically. For one, the file is read only once and the internal cache cannot be invalidated. Second, writing data is reduced to a simple append, and no compaction step is necessary. The format of the contents is the string representation of a dictionary which can be incrementally extended forever at the end.

dsource

This *data source* uses an explicitly specified *data source object* as the source for the persistent data. It accepts a single identifying argument, the command prefix, i.e. object command.

To use this type it is necessary to know how the framework manages ties and what **data source objects** are.

All changes are delegated to the specified object.

Creating new data sources

This section is of no interest to the casual user of ties. Only developers wishing to create new data sources have to know the information provided herein.

Data source objects

All ties are represented internally by an in-memory object which mediates between the tie framework and the specific *data source*, like an array, file, etc. This is the *data source object*.

Its class, the **data source class** is *not* generic, but specific to the type of the *data source*. Writing a new *data source* requires us to write such a class, and then registering it with the framework as a new type.

The following subsections describe the various APIs a **data source class** and the objects it generates will have to follow to be compatible with the tie framework.

Data source objects are normally automatically created and destroyed by the framework when a

tie is created, or removed. This management can be explicitly bypassed through the usage of the "dsource" type. The *data source* for this type is a *data source object* itself, and this object is outside of the scope of the tie framework and not managed by it. In other words, this type allows the creation of ties which talk to pre-existing *data source object*s, and these objects will survive the removal of the ties using them as well.

Registering a new data source class

After a **data source class** has been written it is necessary to register it as a new type with the framework.

::tie::register *dsclasscmd* as *dstype*

Using this command causes the tie framework to remember the class command *dsclasscmd* of a **data source class** under the type name *dstype*.

After the call the argument *dstype* of the basic user command **::tie::tie** will accept *dstype* as a type name and translate it internally to the appropriate class command for the creation of **data source objects** for the new *data source*.

Data source class

Each data source class is represented by a single command, also called the *class command*, or *object creation command*. Its syntax is

dsclasscmd *objname* ?*dsname*...?

The first argument of the class command is the name of the *data source object* to create. The framework itself will always supply the string **%AUTO%**, to signal that the class command has to generate not only the object, but the object name as well.

This is followed by a series of arguments identifying the data source the new object is for. These are the same *dsname* arguments which are given to the basic user command **::tie::tie** . Their actual meaning is dependent on the *data source class*.

The result of the class command has to be the fully-qualified name of the new *data source object*, i.e. the name of the *object command*. The interface this command has to follow is described in the section Data source object API

Data source object API

Please read the section Data source class first, to know how to generate new *object commands*.

Each *object command* for a *data source* object has to provide at least the methods listed below for proper inter-operation with the tie framework. Note that the names of most of the methods match the subcommands of the builtin **array** command.

ds destroy

This method is called when the object **ds** is destroyed. It now has to release all its internal resources associated with the external data source.

ds names

This command has to return a list containing the names of all keys found in the *data source* the object talks to. This is equivalent to **array names**.

ds size

This command has to return an integer number specifying the number of keys found in the *data source* the object talks to. This is equivalent to **array size**.

ds get
This command has to return a dictionary containing the data found in the *data source* the object talks to. This is equivalent to **array get**.

ds set *dict*
This command takes a dictionary and adds its contents to the data source the object talks to. This is equivalent to **array set**.

ds unset ?*pattern*?
This command takes a pattern and removes all elements whose keys matching it from the *data source*. If no pattern is specified it defaults to *, causing the removal of all elements. This is nearly equivalent to **array unset**.

ds setv *index value*
This command has to save the *value* in the *data source* the object talks to, under the key *index*.

The result of the command is ignored. If an error is thrown then this error will show up as error of the set operation which caused the method call.

ds unsetv *index*
This command has to remove the value under the key *index* from the *data source* the object talks to.

The result of the command is ignored. If an error is thrown then this error will show up as error of the unset operation which caused the method call.

ds getv *index*
This command has to return the value for the key *index* in the *data source* the object talks to.

And here a small table comparing the *data source* methods to the regular Tcl commands for accessing an array.

```
        Regular Tcl             Data source
        -----------             -----------
        array names a           ds names
        array size  a           ds size
        array get   a           ds get
        array set   a dict      ds set   dict
        array unset a pattern   ds unset ?pattern?
        -----------             -----------
        set a($idx) $val        ds setv   idx val
        unset a($idx)           ds unsetv idx
        $a($idx)                ds getv   idx
        -----------             -----------
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib

Trackers and specify *tie* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Programming tools

Copyright

```
Copyright (c) 2004-2021 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


tie_std
Keywords: tie_std, array, database, file, metakit, persistence, tie, untie

Name
tie - Array persistence, standard data sources

Synopsis
package require Tcl 8.5 9
package require tie::std::log ?**1.2**?
package require tie::std::array ?**1.2**?
package require tie::std::rarray ?**1.2**?
package require tie::std::file ?**1.2**?
package require tie::std::growfile ?**1.2**?
package require tie::std::dsource ?**1.2**?

Description
The packages listed as requirements for this document are internal packages providing the standard data sources of package **tie**, as described in section *STANDARD DATA SOURCE TYPES* of **tie**'s documentation.

They are automatically loaded and registered by **tie** when it itself is requested, and as such there is no need to request them on their own, although it is possible to do so.

Bugs, ideas, feedback
If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *tie* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Programming tools

Copyright


```
Copyright (c) 2008-2021 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


tiff

tiff

Keywords: tiff, image, tif

Name

tiff - TIFF reading, writing, and querying and manipulation of meta data

Synopsis

package require Tcl 8.5 9
package require tiff ?**0.2.3**?
::tiff::isTIFF *file*
::tiff::byteOrder *file*
::tiff::numImages *file*
::tiff::dimensions *file* ?*image*?
::tiff::imageInfo *file* ?*image*?
::tiff::entries *file* ?*image*?
::tiff::getEntry *file entry* ?*image*?
::tiff::addEntry *file entry* ?*image*?
::tiff::deleteEntry *file entry* ?*image*?
::tiff::getImage *file* ?*image*?
::tiff::writeImage *image file* ?*entry*?
::tiff::nametotag *names*
::tiff::tagtoname *tags*
::tiff::debug *file*

Description

This package provides commands to query, modify, read, and write TIFF images. TIFF stands for *Tagged Image File Format* and is a standard for lossless storage of photographical images and associated metadata. It is specified at http://partners.adobe.com/public/developer/tiff/index.html .

Multiple images may be stored in a single TIFF file. The ?image? options to the functions in this package are for accessing images other than the first. Data in a TIFF image is stored as a series of tags having a numerical value, which are represented in either a 4 digit hexadecimal format or a string name. For a reference on defined tags and their meanings see http://www.awaresystems.be/imaging/tiff/tifftags.html

Commands

::tiff::isTIFF *file*

Returns a boolean value indicating if *file* is a TIFF image.

::tiff::byteOrder *file*
Returns either **big** or **little**. Throws an error if *file* is not a TIFF image.

::tiff::numImages *file*
Returns the number of images in *file*. Throws an error if *file* is not a TIFF image.

::tiff::dimensions *file* ?image?
Returns the dimensions of image number ?image? in *file* as a list of the horizontal and vertical pixel count. Throws an error if *file* is not a TIFF image.

::tiff::imageInfo *file* ?image?
Returns a dictionary with keys **ImageWidth**, **ImageLength**, **BitsPerSample**, **Compression**, **PhotometricInterpretation**, **ImageDescription**, **Orientation**, **XResolution**, **YResolution**, **ResolutionUnit**, **DateTime**, **Artist**, and **HostComputer**. The values are the associated properties of the TIFF ?image? in *file*. Values may be empty if the associated tag is not present in the file.

```
    puts [::tiff::imageInfo photo.tif]

    ImageWidth 686 ImageLength 1024 BitsPerSample {8 8 8}
    Compression 1
    PhotometricInterpretation 2 ImageDescription {} Orientation 1
    XResolution 170.667 YResolution 170.667 ResolutionUnit 2
    DateTime {2005:12:28 19:44:45}
    Artist {} HostComputer {}
```

There is nothing special about these tags, this is simply a convience procedure which calls **getEntry** with common entries. Throws an error if *file* is not a TIFF image.

::tiff::entries *file* ?image?
Returns a list of all entries in the given *file* and ?image? in hexadecimal format. Throws an error if *file* is not a TIFF image.

::tiff::getEntry *file entry* ?image?
Returns the value of *entry* from image ?image? in the TIFF *file*. *entry* may be a list of multiple entries. If an entry does not exist, an empty string is returned

```
    set data [::tiff::getEntry photo.tif {0131 0132}]
    puts "file was written at [lindex $data 0] with software [lindex
    $data 1]"
```

Throws an error if *file* is not a TIFF image.

::tiff::addEntry *file entry* ?image?
Adds the specified entries to the image named by ?image? (default 0), or optionally **all**. *entry* must be a list where each element is a list of tag, type, and value. If a tag already exists, it is overwritten.

```
    ::tiff::addEntry photo.tif {{010e 2 "an example photo"} {013b 2
    "Aaron F"}}
```

The data types are defined as follows
1

BYTE (8 bit unsigned integer)

2
ASCII

3
SHORT (16 bit unsigned integer)

4
LONG (32 bit unsigned integer)

5
RATIONAL

6
SBYTE (8 bit signed byte)

7
UNDEFINED (uninterpreted binary data)

8
SSHORT (signed 16 bit integer)

9
SLONG (signed 32 bit integer)

10
SRATIONAL

11
FLOAT (32 bit floating point number)

12
DOUBLE (64 bit floating point number)

Throws an error if *file* is not a TIFF image.

::tiff::deleteEntry *file entry* ?image?
Deletes the specified entries from the image named by ?image? (default 0), or optionally **all**. Throws an error if *file* is not a TIFF image.

::tiff::getImage *file* ?image?
Returns the name of a Tk image containing the image at index ?image? from *file* Throws an error if *file* is not a TIFF image, or if image is an unsupported format. Supported formats are uncompressed 24 bit RGB and uncompressed 8 bit palette.

::tiff::writeImage *image file* ?entry?
Writes the contents of the Tk image *image* to a tiff file *file*. Files are written in the 24 bit uncompressed format, with big endian byte order. Additional entries to be added to the image may be specified, in the same format as **tiff::addEntry**

::tiff::nametotag *names*
Returns a list with *names* translated from string to 4 digit format. 4 digit names in the input are passed through unchanged. Strings without a defined tag name will throw an error.

::tiff::tagtoname *tags*
Returns a list with *tags* translated from 4 digit to string format. If a tag does not have a defined name it is passed through unchanged.

::tiff::debug *file*
Prints everything we know about the given file in a nice format.

Variables
The mapping of 4 digit tag names to string names uses the array ::tiff::tiff_tags. The reverse mapping uses the array ::tiff::tiff_sgat.

Limitations
1. Cannot write exif ifd
2. Reading limited to uncompressed 8 bit rgb and 8 bit palletized images
3. Writing limited to uncompressed 8 bit rgb

Bugs, ideas, feedback
If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *tiff* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
File formats

Copyright

```
Copyright (c) 2005-2006, Aaron Faupell <afaupell@users.sourceforge.net>
```


tool

meta
Keywords: meta, TclOO, callback, class methods, class variables, command prefix, currying, method reference, my method, singleton

Name
oo::util - Utility commands for TclOO

Synopsis
package require Tcl 8.5 9
package require TclOO
package require oo::util ?**1.2.2**?
mymethod *method* ?*arg...*?
classmethod *name arguments body*
classvariable ?*arg...*?
link *method...*
link {alias method}...
ooutil::singleton ?*arg...*?

Description
This package provides a convenience command for the easy specification of instance methods

as callback commands, like timers, file events, Tk bindings, etc.

Commands

mymethod *method* ?*arg*...?

This command is available within instance methods. It takes a method name and, possibly, arguments for the method and returns a command prefix which, when executed, will invoke the named method of the object we are in, with the provided arguments, and any others supplied at the time of actual invokation.

Note: The command is equivalent to and named after the command provided by the OO package **snit** for the same purpose.

classmethod *name arguments body*

This command is available within class definitions. It takes a method name and, possibly, arguments for the method and creates a method on the class, available to a user of the class and of derived classes.

Note: The command is equivalent to the command **typemethod** provided by the OO package **snit** for the same purpose.

Example

```
oo::class create ActiveRecord {
    classmethod find args { puts "[self] called with arguments:
    $args" }
}
oo::class create Table {
    superclass ActiveRecord
}
puts [Table find foo bar]
# ======
# which will write
# ======
# ::Table called with arguments: foo bar
```

classvariable ?*arg*...?

This command is available within instance methods. It takes a series of variable names and makes them available in the method's scope. The originating scope for the variables is the class (instance) the object instance belongs to. In other words, the referenced variables are shared between all instances of their class.

Note: The command is roughly equivalent to the command **typevariable** provided by the OO package **snit** for the same purpose. The difference is that it cannot be used in the class definition itself.

Example:

```
% oo::class create Foo {
    method bar {z} {
        classvariable x y
        return [incr x $z],[incr y]
    }
}
::Foo
% Foo create a
::a
% Foo create b
::b
% a bar 2
2,1
% a bar 3
5,2
% b bar 7
12,3
% b bar -1
11,4
% a bar 0
11,5
```

link *method*...

see below ...

link {*alias method*}...

This command is available within instance methods. It takes a list of method names and/or pairs of alias- and method-name and makes the named methods available to all instance methods without requiring the **my** command.

The alias name under which the method becomes available defaults to the method name, except where explicitly specified through an alias/method pair.

Examples:

```
    link foo
    # The method foo is now directly accessible as foo instead of my
    foo.

    link {bar foo}
    # The method foo is now directly accessible as bar.

    link a b c
    # The methods a, b, and c all become directly acessible under
    their
    # own names.
```

The main use of this command is expected to be in instance constructors, for convenience, or to set up some methods for use in a mini DSL.

ooutil::singleton ?*arg*...?

This command is a meta-class, i.e. a variant of the builtin **oo::class** which ensures that it creates only a single instance of the classes defined with it.

Syntax and results are like for **oo::class**.

Example:

```
% oo::class create example {
   self mixin singleton
   method foo {} {self}
}
::example
% [example new] foo
::oo::Obj22
% [example new] foo
::oo::Obj22
```

Authors

Donal Fellows, Andreas Kupries

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *oo::util* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Utility

Copyright

```
Copyright (c) 2011-2015 Andreas Kupries, BSD licensed
```

tool

Keywords: tool, TclOO, framework

Name

tool - TclOO Library (TOOL) Framework

Synopsis

package require Tcl 8.6 9
package require sha1
package require dicttool
package require oo::meta

package require oo::dialect
object **cget** *option*
object **configure** ?*keyvaluelist*?
object **configure** *field value* ?*field*? ?*value*? ?*...*?
object **configurelist** ?*keyvaluelist*?
object **forward** *stub forward*
object **graft** *stub forward*
object **InitializePublic**
object **Eval_Script** ?*script*?
object **Option_Default** *field*

tool::define **class_method** *arglist body*

tool::define **array** *name contents*

tool::define **array_ensemble** *methodname varname* ?cases?

tool::define **dict_ensemble** *methodname varname* ?cases?

tool::define **method** *methodname arglist body*

tool::define **option** *name dictopts*

tool::define **property** ?branch? *field value*

tool::define **variable** *name value*

Description

This module implements the Tcl Object Oriented Library framework, or *TOOL*. It is intended to be a general purpose framework that is useable in its own right, and easily extensible.

TOOL defines a metaclass with provides several additional keywords to the TclOO description langauge, default behaviors for its consituent objects, and top-down integration with the capabilities provided by the **oo::meta** package.

The TOOL metaclass was build with the **oo::dialect** package, and thus can be used as the basis for additional metaclasses. As a metaclass, TOOL has it's own "class" class, "object" class, and define namespace.

```
package require tool

# tool::class workds just like oo::class
tool::class create myclass {
}

# tool::define works just like oo::define
tool::define myclass method noop {} {}

# tool::define and tool::class understand additional keywords
tool::define myclass array_ensemble mysettings mysettings {}

# And tool interoperates with oo::define
oo::define myclass method do_something {} { return something }
```

```
# TOOL and TclOO objects are interchangeable
oo::class create myooclass {
  superclass myclass
}
```

Several manual pages go into more detail about specific keywords and methods.

- **tool::array_ensemble**
- tool::dict_ensemble
- **tool::method_ensemble**
- **tool::object**
- **tool::option_handling**

Keywords

TOOL adds new (or modifies) keywords used in the definitions of classes. However, the new keywords are only available via calls to *tool::class create* or *tool::define*

tool::define class_method *arglist body*
: Defines a method for the class object itself. This method will be passed on to descendents of the class, unlike **self method**.

tool::define array *name contents*
: Declares a variable *name* which will be initialized as an array, populated with *contents* for objects of this class, as well as any objects for classes which are descendents of this class.

tool::define array_ensemble *methodname varname* ?cases?
: Declares a method ensemble *methodname* which will control access to variable *varname*. Cases are a key/value list of method names and bodies which will be overlaid on top of the standard template. See **tool::array_ensemble**.

  One method name is reserved: **initialize**. **initialize** Declares the initial values to be populated in the array, as a key/value list, and will not be expressed as a method for the ensemble.

tool::define dict_ensemble *methodname varname* ?cases?
: Declares a method ensemble *methodname* which will control access to variable *varname*. Cases are a key/value list of method names and bodies which will be overlaid on top of the standard template. See tool::dict_ensemble.

  One method name is reserved: **initialize**. **initialize** Declares the initial values to be populated in the array, as a key/value list, and will not be expressed as a method for the ensemble.

tool::define method *methodname arglist body*
: If *methodname* contains ::, the method is considered to be part of a method ensemble. See **tool::method_ensembles**. Otherwise this command behaves exactly like the standard **oo::define method** command.

tool::define option *name dictopts*
: Declares an option. *dictopts* is a key/value list defining parameters for the option. See **tool::option_handling**.

```
tool::class create myclass {
  option color {
    post-command: {puts [list %self%'s %field% is now %value%]}
    default: green
  }
}
myclass create foo
foo configure color purple
> foo's color is now purple
```

tool::define property ?branch? *field value*
Defines a new leaf in the class metadata tree. With no branch, the leaf will appear in the *const* section, accessible by either the object's **property** method, or via **oo::meta::info** *class* **get const** *field*:

tool::define variable *name value*
Declares a variable *name* which will be initialized with the value *value* for objects of this class, as well as any objects for classes which are descendents of this class.

Public object methods

The TOOL object mother of all classes defines several methods to enforces consistent behavior throughout the framework.

object* cget *option
Return the value of this object's option *option*. If the **property options_strict** is true for this class, calling an option which was not declared by the **option** keyword will throw an error. In all other cases if the value is present in the object's *options* array that value is returned. If it does not exist, the object will attempt to retrieve a property of the same name.

***object* configure ?keyvaluelist?**
see below ...

***object* configure *field value* ?field? ?value? ?...?**
This command will inject new values into the objects *options* array, according to the rules as set forth by the option descriptions. See **tool::option_handling** for details. **configure** will strip leading -'s off of field names, allowing it to behave in a quasi-backward compatible manner to tk options.

***object* configurelist ?keyvaluelist?**
This command will inject new values into the objects *options* array, according to the rules as set forth by the option descriptions. This command will perform validation and alternate storage rules. It will not invoke trigger rules. See **tool::option_handling** for details.

object* forward *stub forward
A passthrough to **oo:objdefine [self] forward**

object* graft *stub forward
Delegates the method to the object or command designated by *forward*

```
tool::object create A
tool::object create B
A graft buddy B
A configure color red
B configure color blue
A cget color
> red
A <buddy> cget color
> blue
```

Private object methods

***object* InitializePublic**

Consults the metadata for the class to ensure every array, option, and variable which has been declared but not initialized is initialized with the default value. This method is called by the constructor and the morph method. It is safe to invoke multiple times.

***object* Eval_Script ?script?**

Executes a block of text within the namespace of the object. Lines that begin with a # are ignored as comments. Commands that begin with :: are interpreted as calling a global command. All other Tcl commands that lack a "my" prefix are given one, to allow the script to exercise internal methods. This method is intended for configuration scripts, where the object's methods are intepreting a domain specific language.

```
tool::class myclass {
  constructor script {
    my Eval_Script $script
  }
  method node {nodename info} {
    my variable node
    dict set node $nodename $info
  }
  method get {args} {
    my variable node
    return [dict get $node $args]
  }
}
myclass create movies {
  # This block of code is executed by the object
  node {The Day the Earth Stood Still} {
    date: 1952
    characters: {GORT Klatoo}
  }
}
movies get {The Day the Earth Stood Still} date:
> 1952
```

object* Option_Default *field

Computes the default value for an option. See **tool::option_handling**.

Authors
Sean Woods

Bugs, ideas, feedback
If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *tcloo* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
TclOO

Copyright

```
Copyright (c) 2015 Sean Woods <yoda@etoyoc.com>
```


tool_dict_ensemble
Keywords: tool_dict_ensemble, TOOL, TclOO

Name
tool::dict_ensemble - Dictionary Tools

Synopsis
package require tool ?**0.4.2**?
object **ensemble add** *field*

Description
The **dict_ensemble** command is a keyword added by tool. It defines a public variable (stored as a dict), and an access function to manipulated and access the values stored in that dict.

object ensemble* add *field
] *value value ...*] Adds elements to a list maintained with the *field* leaf of the dict maintained my this ensemble. Declares a variable *name* which will be initialized as an array, populated with *contents* for objects of this class, as well as any objects for classes which are descendents of this class.

Authors
Sean Woods

Bugs, ideas, feedback
If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *tool* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Utility

Copyright
```
Copyright (c) 2015 Sean Woods <yoda@etoyoc.com>
```

transfer

connect
Keywords: connect, active, channel, connection, passive, secure, ssl, tls, transfer

Name
transfer::connect - Connection setup

Synopsis
package require Tcl 8.5 9
package require snit ?**1.0**?
package require transfer::connect ?**0.3**?
transfer::connect *objectName* ?*options...*?
objectName **method** ?*arg arg* ...?
objectName **destroy**
objectName **connect** *command*

Description
This package provides objects holding enough information to enable them to either actively connect to a counterpart, or to passively wait for a connection from said counterpart. I.e. any object created by this packages is always in one of two complementary modes, called *active* (the object initiates the connection) and *passive* (the object receives the connection).

Of the two objects in a connecting pair one has to be configured for *active* mode, and the other then has to be configured for *passive* mode. This establishes which of the two partners connects to whom (the *active* to the other), or, who is waiting on whom (the *passive* on the other). Note that this is completely independent of the direction of any data transmission using the connection after it has been established. An active object can, after establishing the connection, either transmit or receive data. Equivalently the passive object can do the same after the waiting for its partner has ended.

API

Package commands
transfer::connect *objectName* ?*options*...?

This command creates a new connection object with an associated Tcl command whose name

is *objectName*. This *object* command is explained in full detail in the sections Object command and Object methods. The set of supported *options* is explained in section Options.

The object command will be created under the current namespace if the *objectName* is not fully qualified, and in the specified namespace otherwise. The fully qualified name of the object command is returned as the result of the command.

Object command

All objects created by the **::transfer::connect** command have the following general form:

***objectName* method ?*arg arg ...*?**

The method **method** and its *arg*'uments determine the exact behavior of the command. See section Object methods for the detailed specifications.

Object methods

***objectName* destroy**

This method destroys the object. This is safe to do for an *active* object when a connection has been started, as the completion callback is synchronous. For a *passive* object currently waiting for its partner to establish the connection however this is not safe and will cause errors later on, when the connection setup completes and tries to access the now missing data structures of the destroyed object.

objectName* connect *command

This method starts the connection setup per the configuration of the object. When the connection is established the callback *command* will be invoked with one additional argument, the channel handle of the socket over which data can be transfered.

The detailed behaviour of the method depends on the configured mode.

active

The connection setup is done synchronously. The object waits until the connection is established. The method returns the empty string as its result.

passive

The connection setup is done asynchronously. The method returns immediately after a listening socket has been set up. The connection will be established in the background. The method returns the port number of the listening socket, for use by the caller. One important use is the transfer of this information to the counterpart so that it knows where it has to connect to.

This is necessary as the object might have been configured for port **0**, allowing the operating system to choose the actual port it will listen on.

The listening port is closed immediately when the connection was established by the partner, to keep the time interval small within which a third party can connect to the port too. Even so it is recommended to use additional measures in the protocol outside of the connect and transfer object to ensure that a connection is not used with an unidentified/unauthorized partner One possibility for this is the use of SSL/TLS. See the option **-socketcmd** and section Secure connections for information on how to do this.

Options

Connection objects support the set of options listed below.

-mode *mode*

This option specifies the mode the object is in. It is optional and defaults to **active** mode. The two possible modes are:

active

In this mode the two options **-host** and **-port** are relevant and specify the host and TCP port the object has to connect to. The host is given by either name or IP address.

passive

In this mode the option **-host** has no relevance and is ignored should it be configured. The only option the object needs is **-port**, and it specifies the TCP port on which the listening socket is opened to await the connection from the partner.

-host *hostname-or-ipaddr*

This option specifies the host to connect to in *active* mode, either by name or ip-address. An object configured for *passive* mode ignores this option.

-port *int*

For *active* mode this option specifies the port the object is expected to connect to. For *passive* mode however it is the port where the object creates the listening socket waiting for a connection. It defaults to **0**, which allows the OS to choose the actual port to listen on.

-socketcmd *command*

This option allows the user to specify which command to use to open a socket. The default is to use the builtin **::socket**. Any compatible with that command is allowed.

The envisioned main use is the specfication of **tls::socket**. I.e. this option allows the creation of secure transfer channels, without making this package explicitly dependent on the **tls** package.

See also section Secure connections.

-encoding encodingname

see below ...

-eofchar eofspec

see below ...

-translation transspec

These options are the same as are recognized by the builtin command **fconfigure**. They provide the configuration to be set for the channel between the two partners after it has been established, but before the callback is invoked (See method **connect**).

Secure connections

One way to secure connections made by objects of this package is to require the package **tls** and then configure the option **-socketcmd** to force the use of command **tls::socket** to open the socket.

```
    # Load and initialize tls
    package require tls
    tls::init -cafile /path/to/ca/cert -keyfile ...

    # Create a connector with secure socket setup,
    transfer::connect C -socketcmd tls::socket ...
    ...
```

Tls security considerations

This package uses the **TLS** package to handle the security for **https** urls and other socket connections.

Policy decisions like the set of protocols to support and what ciphers to use are not the responsibility of **TLS**, nor of this package itself however. Such decisions are the responsibility of whichever application is using the package, and are likely influenced by the set of servers the application will talk to as well.

For example, in light of the recent POODLE attack discovered by Google many servers will disable support for the SSLv3 protocol. To handle this change the applications using **TLS** must be patched, and not this package, nor **TLS** itself. Such a patch may be as simple as generally activating **tls1** support, as shown in the example below.

```
    package require tls
    tls::init -tls1 1 ;# forcibly activate support for the TLS1
    protocol

    ... your own application code ...
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *transfer* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Transfer module

Copyright

```
Copyright (c) 2006-2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

copyops

Keywords: copyops, channel, copy, transfer

Name

transfer::copy - Data transfer foundation

Synopsis

package require Tcl 8.5 9
package require transfer::copy ?**0.3**?
transfer::copy::do chan|string *data outchannel* ?*options...*?
transfer::copy::chan *channel outchannel* ?*options...*?
transfer::copy::string *string outchannel* ?*options...*?
transfer::copy::doChan *channel outchannel optvar*
transfer::copy::doString *string outchannel optvar*
transfer::copy::options *outchannel optionlist optvar*

Description

This package provides a number of commands for the asynchronous of information contained in either a string or channel. The main point of this package is that the commands here provide a nicer callback API than the builtin command **fcopy**, making the use of these facilities simpler than the builtin.

API

transfer::copy::do chan|string *data outchannel* ?*options*...?

This command transfers the information in *data* to the *outchannel*, according to the *options*. The type of the information in *data* is determined by the first argument.

The options available to this command are the same as are available to the command **transfer::copy::options**, and explained there.

chan

The argument *data* contains the handle of a channel and the actual infomration to transfer is read from that channel.

string

The argument *data* contains a string and this is the information to be transfered.

transfer::copy::chan *channel outchannel* ?*options*...?

This command is a shorter and more direct form for the command **transfer::copy::do chan**.

transfer::copy::string *string outchannel* ?*options*...?

This command is a shorter and more direct form for the command **transfer::copy::do string**.

transfer::copy::doChan *channel outchannel optvar*

This command is an alternate form of **transfer::copy::chan** which reads its options out of the array variable named by *optvar* instead of from a variable length argument list.

transfer::copy::doString *string outchannel optvar*

This command is an alternate form of **transfer::copy::string** which reads its options out of the array variable named by *optvar* instead of from a variable length argument list.

transfer::copy::options *outchannel optionlist optvar*

This command is the option processor used by all the commands above which read their options from a variable length argument list. It first reads default settings from the channel handle *outchannel*, then processes the list of options in *optionlist*, at last stores the results in

the array variable named by *optvar*. The contents of that variable are in a format which is directly understood by all the commands above which read their options out of an array variable.

The recognized options are:

-blocksize *int*
: This option specifies the size of the chunks to transfer in one operation. It is optional and defaults to the value of **-buffersize** as configured for the output channel.

  If specified its value has to be an integer number greater than zero.

-command *commandprefix*
: This option specifies the completion callback of the operation. This option has to be specified. An error will be thrown if it is not, or if the empty list was specified as argument to it.

  Its value has to be a command prefix, i.e. a list whose first word is the command to execute, followed by words containing fixed arguments. When the callback is invoked one or two additional arguments are appended to the prefix. The first argument is the number of bytes which were transfered. The optional second argument is an error message and added if and only if an error occured during the the transfer.

-progress *commandprefix*
: This option specifies the progress callback of the operation. It is optional and defaults to the empty list. This last possibility signals that no feedback was asked for and disabled it.

  Its value has to be a command prefix, see above, **-command** for a more detailed explanation. When the callback is invoked a single additional arguments is appended to the prefix. This argument is the number of bytes which were transfered so far.

-size *int*
: This option specifies the number of bytes to read from the input data and transfer. It is optional and defaults to "Transfer everything". Its value has to be an integer number and any value less than zero has the same meaning, i.e. to transfer all available data. Any other value is the amount of bytes to transfer.

  All transfer commands will throw error an when their user tries to transfer more data than is available in the input. This happens immediately, before the transfer is actually started, should the input be a string. Otherwise the, i.e. for a channel as input, the error is thrown the moment the underflow condition is actually detected.

-encoding *encodingname*
: see below ...

-eofchar *eofspec*
: see below ...

-translation *transspec*
: These options are the same as are recognized by the builtin command **fconfigure** and provide the settings for the output channel which are to be active during the transfer, and only then. I.e. the settings of the output channel before the transfer are saved, and restored at the end of a transfer, regardless of its success or failure. None of these options

are required, and they default to the settings of the output channel if not specified.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *transfer* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Transfer module

Copyright

```
Copyright (c) 2006-2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

ddest

Keywords: ddest, channel, copy, data destination, transfer

Name

transfer::data::destination - Data destination

Synopsis

package require Tcl 8.5 9
package require snit ?**1.0**?
package require transfer::data::destination ?**0.3**?
transfer::data::destination *objectName* ?*options...*?
objectName **method** ?*arg arg* ...?
objectName **destroy**
objectName **put** *chunk*
objectName **done**
objectName **valid** *msgvar*
objectName **receive** *channel done*

Description

This package provides objects mainly describing the destination of a data transfer. They are also able to initiate the reception of information from a channel into the described destination.

API

transfer::data::destination *objectName* ?*options*...?

This command creates a new data destination object with an associated Tcl command whose name is *objectName*. This *object* command is explained in full detail in the sections Object command and Object methods. The set of supported *options* is explained in section Options.

The object command will be created under the current namespace if the *objectName* is not fully qualified, and in the specified namespace otherwise. The fully qualified name of the object command is returned as the result of the command.

Object command

All objects created by the **::transfer::data::destination** command have the following general form:

***objectName* method ?*arg arg ...*?**

The method **method** and its *arg*'uments determine the exact behavior of the command. See section Object methods for the detailed specifications.

Object methods

***objectName* destroy**

This method destroys the object. Doing so while the object is busy with the reception of information from a channel will cause errors later on, when the reception completes and tries to access the now missing data structures of the destroyed object.

objectName* put *chunk

The main receptor method. Saves the received *chunk* of data into the configured destination. It has to be called for each piece of data received.

***objectName* done**

The secondary receptor method. Finalizes the receiver. It has to be called when the receiving channel signals EOF. Afterward neither itself nor method **put** can be called anymore.

objectName* valid *msgvar

This method checks the configuration of the object for validity. It returns a boolean flag as result, whose value is **True** if the object is valid, and **False** otherwise. In the latter case the variable whose name is stored in *msgvar* is set to an error message describing the problem found with the configuration. Otherwise this variable is not touched.

objectName* receive *channel done

This method initiates the reception of data from the specified *channel*. The received data will be stored into the configured destination, via calls to the methods **put** and **done**. When the reception completes the command prefix *done* is invoked, with the number of received characters appended to it as the sole additional argument.

Options

All data destinations support the options listed below. It should be noted that all are semi-exclusive, each specifying a different type of destination and associated information. If these options are specified more than once then the last option specified is used to actually configure the object.

-channel *handle*

This option specifies that the destination of the data is a channel, and its associated argument is the handle of the channel to write the received data to.

-file *path*

This option specifies that the destination of the data is a file, and its associated argument is the path of the file to write the received data to.

-variable *varname*
This option specifies that the destination of the data is a variable, and its associated argument contains the name of the variable to write the received data to. The variable is assumed to be global or namespaced, anchored at the global namespace.

-progress *command*
This option, if specified, defines a command to be invoked for each chunk of bytes received, allowing the user to monitor the progress of the reception of the data. The callback is always invoked with one additional argument, the number of bytes received so far.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *transfer* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Transfer module

Copyright


```
Copyright (c) 2006-2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


dsource

Keywords: dsource, channel, copy, data source, transfer

Name

transfer::data::source - Data source

Synopsis

package require Tcl 8.5 9
package require snit ?**1.0**?
package require transfer::copy ?**0.3**?
package require transfer::data::source ?**0.3**?
transfer::data::source *objectName* ?*options...*?
objectName **method** ?*arg arg* ...?
objectName **destroy**
objectName **type**
objectName **data**
objectName **size**
objectName **valid** *msgvar*
objectName **transmit** *channel blocksize done*

Description

This package provides objects mainly describing the origin of some data to transfer. They are also able to initiate transfers of the described information to a channel using the foundation package **transfer::copy**.

API

Package commands

transfer::data::source *objectName* ?*options*...?

This command creates a new data source object with an associated Tcl command whose name is *objectName*. This *object* command is explained in full detail in the sections Object command and Object methods. The set of supported *options* is explained in section Options.

The object command will be created under the current namespace if the *objectName* is not fully qualified, and in the specified namespace otherwise. The fully qualified name of the object command is returned as the result of the command.

Object command

All objects created by the **::transfer::data::source** command have the following general form:

***objectName* method ?*arg arg ...*?**

The method **method** and its *arg*'uments determine the exact behavior of the command. See section Object methods for the detailed specifications.

Object methods

***objectName* destroy**

This method destroys the object. Doing so while a transfer initiated by the object is active is safe as all data required for the transfer itself was copied, and the completion of the transfer will not try to access the initiating object anymore. i.e. the transfer is completely separate from the source object itself.

***objectName* type**

This method returns a string describing the type of the data the object is refering to. The possible values and their meanings are:

undefined

No data was specified at all, or it was specified incompletely. The object does not know the type.

string

The data to transfer is contained in a string.

channel

The data to transfer is contained in a channel.

***objectName* data**

This method returns a value depending on the type of the data the object refers to, through which the data can be accessed. The method throws an error if the type is **undefined**. For type **string** the returned result is the data itself, whereas for type **channel** the returned result is the handle of the channel containing the data.

***objectName* size**

This method returns a value depending on the type of the data the object refers to, the size of the data. The method throws an error if the type is **undefined**. Return of a negative value signals that the object is unable to determine an absolute size upfront (like for data in a channel).

objectName* valid *msgvar

This method checks the configuration of the object for validity. It returns a boolean flag as result, whose value is **True** if the object is valid, and **False** otherwise. In the latter case the variable whose name is stored in *msgvar* is set to an error message describing the problem found with the configuration. Otherwise this variable is not touched.

objectName* transmit *channel blocksize done

This method initiates a transfer of the referenced data to the specified *channel*. When the transfer completes the command prefix *done* is invoked, per the rules for the option **-command** of command transfer::copy::do in the package **transfer::copy**. The *blocksize* specifies the size of the chunks to transfer in one go. See the option **-blocksize** of command transfer::copy::do in the package **transfer::copy**.

Options

All data sources support the options listed below. It should be noted that the first four options are semi-exclusive, each specifying a different type of data source and associated content. If these options are specified more than once then the last option specified is used to actually configure the object.

-string *text*

This option specifies that the source of the data is an immediate string, and its associated argument contains the string in question.

-channel *handle*

This option specifies that the source of the data is a channel, and its associated argument is the handle of the channel containing the data.

-file *path*

This option specifies that the source of the data is a file, and its associated argument is the path of the file containing the data.

-variable *varname*

This option specifies that the source of the data is a string stored in a variable, and its associated argument contains the name of the variable in question. The variable is assumed to be global or namespaced, anchored at the global namespace.

-size *int*

This option specifies the size of the data transfer. It is optional and defaults to -1. This value, and any other value less than zero signals to transfer all the data from the source.

-progress *command*

This option, if specified, defines a command to be invoked for each chunk of bytes transmitted, allowing the user to monitor the progress of the transmission of the data. The callback is always invoked with one additional argument, the number of bytes transmitted so far.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you

want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *transfer* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Transfer module

Copyright


```
Copyright (c) 2006-2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


receiver

Keywords: receiver, channel, copy, data destination, secure, ssl, tls, transfer

Name

transfer::receiver - Data source

Synopsis

package require Tcl 8.5 9
package require snit ?**1.0**?
package require transfer::data::destination ?**0.3**?
package require transfer::connect ?**0.3**?
package require transfer::receiver ?**0.3**?
transfer::receiver *object* ?*options...*?
transfer::receiver stream channel *chan host port* ?*arg...*?
transfer::receiver stream file *path host port* ?*arg...*?
objectName **method** ?*arg arg* ...?
objectName **destroy**
objectName **start**
objectName **busy**

Description

This package pulls data destinations and connection setup together into a combined object for the reception of information coming in over a socket. These objects understand all the options from objects created by the packages transfer::data::destination and transfer::connect.

API

Package commands

transfer::receiver *object* ?*options*...?

This command creates a new receiver object with an associated Tcl command whose name is *objectName*. This *object* command is explained in full detail in the sections Object command and Object methods. The set of supported *options* is explained in section Options.

The object command will be created under the current namespace if the *objectName* is not fully qualified, and in the specified namespace otherwise. The fully qualified name of the object command is returned as the result of the command.

transfer::receiver stream channel *chan host port* ?*arg*...?

This method creates a fire-and-forget transfer for the data coming from the source at host/port (details below) and writing to the channel *chan*, starting at the current seek location. The channel is configured to use binary translation and encoding for the transfer. The channel is *not* closed when the transfer has completed. This is left to the completion callback.

If both *host* and *port* are provided an **active** connection to the data source is made. If only a *port* is specified (with *host* the empty string) then a **passive** connection is made instead, i.e. the receiver then waits for a conneciton by the transmitter.

Any arguments after the port are treated as options and are used to configure the internal receiver object. See the section Options for a list of the supported options and their meaning. *Note* however that the signature of the command prefix specified for the **-command** callback differs from the signature for the same option of the receiver object. This callback is only given the number of bytes and transfered, and possibly an error message. No reference to the internally used receiver object is made.

The result returned by the command is the empty string if it was set to make an *active* connection, and the port the internal receiver object is listening on otherwise, i.e when it is configured to connect *passive*ly. See also the package transfer::connect and the description of the method **connect** for where this behaviour comes from.

transfer::receiver stream file *path host port* ?*arg*...?

This method is like **stream channel**, except that the received data is written to the file *path*, replacing any prior content.

Object command

All objects created by the **::transfer::receiver** command have the following general form:

***objectName* method ?*arg arg ...*?**

The method **method** and its *arg*'uments determine the exact behavior of the command. See section Object methods for the detailed specifications.

Object methods

***objectName* destroy**

This method destroys the object. Doing so while a reception is on progress will cause errors later on, when the reception completes and tries to access the now missing data structures of the destroyed object.

***objectName* start**

This method initiates the data reception, setting up the connection first and then copying the received information into the destination. The method will throw an error if a reception is already/still in progress. I.e. it is not possible to run two receptions in parallel, only in sequence. Errors will also be thrown if the configuration of the data destination is invalid, or if

no completion callback was specified.

The result returned by the method is the empty string for an object configured to make an *active* connection, and the port the object is listening on otherwise, i.e when it is configured to connect *passive*ly. See also the package transfer::connect and the description of the method **connect** for where this behaviour comes from.

***objectName* busy**
This method returns a boolean value telling us whether a reception is in progress (**True**), or not (**False**).

Options

All receiver objects support the union of the options supported by their connect and data destination components, plus one of their own. See also the documentation for the packages transfer::data::destination and transfer::connect.

-command *cmdprefix*
This option specifies the command to invoke when the reception of the information has been completed. The arguments given to this command are the same as given to the completion callback of the command transfer::copy::do provided by the package **transfer::copy**.

-mode *mode*
This option specifies the mode the object is in. It is optional and defaults to **active** mode. The two possible modes are:

active
In this mode the two options **-host** and **-port** are relevant and specify the host and TCP port the object has to connect to. The host is given by either name or IP address.

passive
In this mode the option **-host** has no relevance and is ignored should it be configured. The only option the object needs is **-port**, and it specifies the TCP port on which the listening socket is opened to await the connection from the partner.

-host *hostname-or-ipaddr*
This option specifies the host to connect to in *active* mode, either by name or ip-address. An object configured for *passive* mode ignores this option.

-port *int*
For *active* mode this option specifies the port the object is expected to connect to. For *passive* mode however it is the port where the object creates the listening socket waiting for a connection. It defaults to **0**, which allows the OS to choose the actual port to listen on.

-socketcmd *command*
This option allows the user to specify which command to use to open a socket. The default is to use the builtin **::socket**. Any compatible with that command is allowed.

The envisioned main use is the specfication of **tls::socket**. I.e. this option allows the creation of secure transfer channels, without making this package explicitly dependent on the **tls** package.

See also section Secure connections.

-encoding encodingname
see below ...

-eofchar eofspec
see below ...

-translation transspec
These options are the same as are recognized by the builtin command **fconfigure**. They provide the configuration to be set for the channel between the two partners after it has been established, but before the callback is invoked (See method **connect**).

-channel *handle*
This option specifies that the destination of the data is a channel, and its associated argument is the handle of the channel to write the received data to.

-file *path*
This option specifies that the destination of the data is a file, and its associated argument is the path of the file to write the received data to.

-variable *varname*
This option specifies that the destination of the data is a variable, and its associated argument contains the name of the variable to write the received data to. The variable is assumed to be global or namespaced, anchored at the global namespace.

-progress *command*
This option, if specified, defines a command to be invoked for each chunk of bytes received, allowing the user to monitor the progress of the reception of the data. The callback is always invoked with one additional argument, the number of bytes received so far.

Secure connections

One way to secure connections made by objects of this package is to require the package **tls** and then configure the option **-socketcmd** to force the use of command **tls::socket** to open the socket.

```
    # Load and initialize tls
    package require tls
    tls::init -cafile /path/to/ca/cert -keyfile ...

    # Create a connector with secure socket setup,
    transfer::receiver R -socketcmd tls::socket ...
    ...
```

Tls security considerations

This package uses the **TLS** package to handle the security for **https** urls and other socket connections.

Policy decisions like the set of protocols to support and what ciphers to use are not the responsibility of **TLS**, nor of this package itself however. Such decisions are the responsibility of whichever application is using the package, and are likely influenced by the set of servers the application will talk to as well.

For example, in light of the recent POODLE attack discovered by Google many servers will disable support for the SSLv3 protocol. To handle this change the applications using **TLS** must

be patched, and not this package, nor **TLS** itself. Such a patch may be as simple as generally activating **tls1** support, as shown in the example below.

```
    package require tls
    tls::init -tls1 1 ;# forcibly activate support for the TLS1
    protocol

    ... your own application code ...
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *transfer* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Transfer module

Copyright

```
Copyright (c) 2006 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

tqueue

Keywords: tqueue, channel, copy, queue, transfer

Name

transfer::copy::queue - Queued transfers

Synopsis

package require Tcl 8.5 9
package require snit ?**1.0**?
package require struct::queue ?**1.4**?
package require transfer::copy ?**0.3**?
package require transfer::copy::queue ?**0.2**?
transfer::copy::queue *objectName outchannel* ?*options...*?
objectName **method** ?*arg arg* ...?
objectName **destroy**
objectName **busy**
objectName **pending**
objectName **put** *request*

Description

This package provides objects which serialize transfer requests for a single channel by means of

a fifo queue. Accumulated requests are executed in order of entrance, with the first request reaching an idle object starting the execution in general. New requests can be added while the object is active and are defered until all requests entered before them have been completed successfully.

When a request causes a transfer error execution stops and all requests coming after it are not served. Currently this means that their completion callbacks are never triggered at all.

NOTE: Not triggering the completion callbacks of the unserved requests after an error stops the queue object is something I am not fully sure that it makes sense. It forces the user of the queue to remember the callbacks as well and run them. Because otherwise everything in the system which depends on getting a notification about the status of a request will hang in the air. I am slowly convincing myself that it is more sensible to trigger the relevant completion callbacks with an error message about the queue abort, and 0 bytes transfered.

All transfer requests are of the form

```
    {type data options...}
```

where *type* is in {**chan**, **string**}, and *data* specifies the information to transfer. For **chan** the data is the handle of the channel containing the actual information to transfer, whereas for **string** *data* contains directly the information to transfer. The *options* are a list of them and their values, and are the same as are accepted by the low-level copy operations of the package **transfer::copy**. Note how just prepending the request with transfer::copy::do and inserting a channel handle in between *data* and *options* easily transforms it from a pure data structure into a command whose evaluation will perform the request.

API

Package commands

transfer::copy::queue *objectName outchannel* ?*options*...?

This command creates a new queue object for the management of the channel *outchannel*, with an associated Tcl command whose name is *objectName*. This *object* command is explained in full detail in the sections Object command and Object methods. The set of supported *options* is explained in section Options.

The object command will be created under the current namespace if the *objectName* is not fully qualified, and in the specified namespace otherwise. The fully qualified name of the object command is returned as the result of the command.

Object command

All objects created by the **::transfer::copy::queue** command have the following general form:

***objectName* method ?*arg arg ...*?**

The method **method** and its *arg*'uments determine the exact behavior of the command. See section Object methods for the detailed specifications.

Object methods

***objectName* destroy**

This method destroys the object. Doing so while the object is busy will cause errors later on, when the currently executed request completes and tries to access the now missing data

structures of the destroyed object.

***objectName* busy**
This method returns a boolean value telling us if the object is currently serving a request (i.e. *busy*, value **True**), or not (i.e. *idle*, value **False**).

***objectName* pending**
This method returns the number of requests currently waiting in the queue for their execution. A request currently served is not counted as waiting.

objectName* put *request
This method enters the transfer *request* into the object's queue of waiting requests. If the object is *idle* it will become *busy*, immediately servicing the request. Otherwise servicing the new request will be defered until all preceding requests have been served.

Options

The only option known is **-on-status-change**. It is optional and defaults to the empty list, disabling the reporting of status changes. Otherwise its argument is a command prefix which is invoked whenever the internal status of the object changed. The callback is invoked with two additional arguments, the result of the methods **pending** and **busy**, in this order. This allows any user to easily know, for example, when the object has processed all outstanding requests.

Use

A possible application of this package and class is within a HTTP 1.1 server, managing the results waiting for transfer to the client.

It should be noted that in this application the system also needs an additional data structure which keeps track of outstanding results as they may come back in a different order than the requests from the client, and releases them to the actual queue in the proper order.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *transfer* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Transfer module

Copyright


```
Copyright (c) 2006 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


transmitter

Keywords: transmitter, channel, copy, data source, secure, ssl, tls, transfer

Name

transfer::transmitter - Data source

Synopsis

package require Tcl 8.5 9
package require snit ?**1.0**?
package require transfer::copy ?**0.3**?
package require transfer::data::source ?**0.3**?
package require transfer::connect ?**0.3**?
package require transfer::transmitter ?**0.3**?
transfer::transmitter *objectName* ?*options...*?
transfer::transmitter stream channel *chan host port* ?*arg...*?
transfer::transmitter stream file *path host port* ?*arg...*?
objectName **method** ?*arg arg* ...?
objectName **destroy**
objectName **start**
objectName **busy**

Description

This package pulls data sources and connection setup together into a combined object for the transmission of information over a socket. These objects understand all the options from objects created by the packages transfer::data::source and transfer::connect.

API

Package commands

transfer::transmitter *objectName ?options...?*

This command creates a new transmitter object with an associated Tcl command whose name is *objectName*. This *object* command is explained in full detail in the sections Object command and Object methods. The set of supported *options* is explained in section Options.

The object command will be created under the current namespace if the *objectName* is not fully qualified, and in the specified namespace otherwise. The fully qualified name of the object command is returned as the result of the command.

transfer::transmitter stream channel *chan host port ?arg...?*

This method creates a fire-and-forget transfer for the data contained in the channel *chan*, starting at the current seek location. The channel is configured to use binary translation and encoding for the transfer. The channel is automatically closed when the transfer has completed.

If both *host* and *port* are provided an **active** connection to the destination is made. If only a *port* is specified (with *host* the empty string) then a **passive** connection is made instead.

Any arguments after the port are treated as options and are used to configure the internal transmitter object. See the section Options for a list of the supported options and their meaning. *Note* however that the signature of the command prefix specified for the **-command** callback differs from the signature for the same option of the transmitter object.

This callback is only given the number of bytes and transfered, and possibly an error message. No reference to the internally used transmitter object is made.

The result returned by the command is the empty string if it was set to make an *active* connection, and the port the internal transmitter object is listening on otherwise, i.e when it is configured to connect *passive*ly. See also the package transfer::connect and the description of the method **connect** for where this behaviour comes from.

transfer::transmitter stream file *path host port ?arg...?*

This method is like **stream channel**, except that the data contained in the file *path* is transfered.

Object command

All objects created by the **::transfer::transmitter** command have the following general form:

***objectName* method ?*arg arg ...*?**

The method **method** and its *arg*'uments determine the exact behavior of the command. See section Object methods for the detailed specifications.

Object methods

***objectName* destroy**

This method destroys the object. Doing so while a transmission is in progress will cause errors later on, when the transmission completes and tries to access the now missing data structures of the destroyed object.

***objectName* start**

This method initiates the data transmission, setting up the connection first and then copying the information. The method will throw an error if a transmission is already/still in progress. I.e. it is not possible to run two transmissions in parallel on a single object, only in sequence. Multiple transmitter objects are needed to manage parallel transfers, one per transmission. Errors will also be thrown if the configuration of the data source is invalid, or if no completion callback was specified.

The result returned by the method is the empty string for an object configured to make an *active* connection, and the port the object is listening on otherwise, i.e when it is configured to connect *passive*ly. See also the package transfer::connect and the description of the method **connect** for where this behaviour comes from.

***objectName* busy**

This method returns a boolean value telling us whether a transmission is in progress (**True**), or not (**False**).

Options

All transmitter objects support the union of the options supported by their connect and data source components, plus two of their own. See also the documentation for the packages transfer::data::source and transfer::connect.

-blocksize *int*

This option specifies the size of the chunks to be transmitted in one block. Usage is optional, its default value is **1024**.

-command *cmdprefix*

This option specifies the command to invoke when the transmission of the information has been completed. The arguments given to this command are the same as given to the completion callback of the command transfer::copy::do provided by the package **transfer::copy**.

-mode *mode*

This option specifies the mode the object is in. It is optional and defaults to **active** mode. The two possible modes are:

active

In this mode the two options **-host** and **-port** are relevant and specify the host and TCP port the object has to connect to. The host is given by either name or IP address.

passive

In this mode the option **-host** has no relevance and is ignored should it be configured. The only option the object needs is **-port**, and it specifies the TCP port on which the listening socket is opened to await the connection from the partner.

-host *hostname-or-ipaddr*

This option specifies the host to connect to in *active* mode, either by name or ip-address. An object configured for *passive* mode ignores this option.

-port *int*

For *active* mode this option specifies the port the object is expected to connect to. For *passive* mode however it is the port where the object creates the listening socket waiting for a connection. It defaults to **0**, which allows the OS to choose the actual port to listen on.

-socketcmd *command*

This option allows the user to specify which command to use to open a socket. The default is to use the builtin **::socket**. Any compatible with that command is allowed.

The envisioned main use is the specfication of **tls::socket**. I.e. this option allows the creation of secure transfer channels, without making this package explicitly dependent on the **tls** package.

See also section Secure connections.

-encoding encodingname

see below ...

-eofchar eofspec

see below ...

-translation transspec

These options are the same as are recognized by the builtin command **fconfigure**. They provide the configuration to be set for the channel between the two partners after it has been established, but before the callback is invoked (See method **connect**).

-string *text*

This option specifies that the source of the data is an immediate string, and its associated argument contains the string in question.

-channel *handle*

This option specifies that the source of the data is a channel, and its associated argument is the handle of the channel containing the data.

-file *path*
This option specifies that the source of the data is a file, and its associated argument is the path of the file containing the data.

-variable *varname*
This option specifies that the source of the data is a string stored in a variable, and its associated argument contains the name of the variable in question. The variable is assumed to be global or namespaced, anchored at the global namespace.

-size *int*
This option specifies the size of the data transfer. It is optional and defaults to -1. This value, and any other value less than zero signals to transfer all the data from the source.

-progress *command*
This option, if specified, defines a command to be invoked for each chunk of bytes transmitted, allowing the user to monitor the progress of the transmission of the data. The callback is always invoked with one additional argument, the number of bytes transmitted so far.

Secure connections

One way to secure connections made by objects of this package is to require the package **tls** and then configure the option **-socketcmd** to force the use of command **tls::socket** to open the socket.

```
    # Load and initialize tls
    package require tls
    tls::init -cafile /path/to/ca/cert -keyfile ...

    # Create a connector with secure socket setup,
    transfer::transmitter T -socketcmd tls::socket ...
    ...
```

Tls security considerations

This package uses the **TLS** package to handle the security for **https** urls and other socket connections.

Policy decisions like the set of protocols to support and what ciphers to use are not the responsibility of **TLS**, nor of this package itself however. Such decisions are the responsibility of whichever application is using the package, and are likely influenced by the set of servers the application will talk to as well.

For example, in light of the recent POODLE attack discovered by Google many servers will disable support for the SSLv3 protocol. To handle this change the applications using **TLS** must be patched, and not this package, nor **TLS** itself. Such a patch may be as simple as generally activating **tls1** support, as shown in the example below.

```
    package require tls
    tls::init -tls1 1 ;# forcibly activate support for the TLS1
    protocol

    ... your own application code ...
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *transfer* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Transfer module

Copyright


```
Copyright (c) 2006-2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


treeql

treeql

Keywords: treeql, Cost, DOM, XPath, XSLT, structured queries, tree, tree query language

Name

treeql - Query tree objects

Synopsis

package require Tcl 8.5 9
package require snit
package require struct::list
package require struct::set
package require treeql ?**1.3.2**?
treeql *objectname* **-tree** *tree* ?*-query query*? ?*-nodes nodes*? ?*args...*?
qo **query** *args...*
qo **result**
qo **discard**

Description

This package provides objects which can be used to query and transform tree objects following the API of tree objects created by the package **struct::tree**.

The tree query and manipulation language used here, TreeQL, is inspired by Cost (See section References for more information).

treeql, the package, is a fairly thin query facility over tree-structured data types. It implements an ordered set of nodes (really a list) which are generated and filtered through the application of TreeQL operators to each node in turn.

API

TreeQL class API

The command **treeql** is a **snit**::type which implements the Treeql Query Language. This means that it follows the API for class commands as specified by the package **snit**. Its general syntax is

treeql *objectname* -tree *tree* ?-query *query*? ?-nodes *nodes*? ?*args*...?

The command creates a new tree query object and returns the fully qualified name of the object command as its result. The API the returned command is following is described in the section TreeQL object API

Each query object is associated with a single *tree* object. This is the object all queries will be run against.

If the option **-nodes** was specified then its argument is treated as a list of nodes. This list is used to initialize the node set. It defaults to the empty list.

If the option **-query** was specified then its argument will be interpreted as an object, the *parent query* of this query. It defaults to the object itself. All queries will be interpreted in the environment of this object.

Any arguments coming after the options are treated as a query and run immediately, after the *node set* has been initialized. This uses the same syntax for the query as the method **query**.

The operations of the TreeQL available for this are explained in the section about The tree query language. This section also explains the term *node set* used above.

TreeQL object API

As **treeql** has been implemented in **snit** all the standard methods of **snit**-based classes are available to the user and therefore not listed here. Please read the documentation for **snit** for what they are and what functionality they provide

The methods provided by the package **treeql** itself are listed and explained below.

***qo* query *args*...**

This method interprets its arguments as a series of TreeQL operators and interpretes them from the left to right (i.e. first to last). Note that the first operator uses the *node set* currently known to the object to perform its actions. In other words, the *node set* is *not* cleared, or modified in other ways, before the query is run. This allows the user to run several queries one after the other and have each use the results of the last. Any initialization has to be done by any query itself, using TreeQL operators. The result of the method is the *node set* after the last operator of the query has been executed.

Note that uncaught errors will leave the *node set* of the object in an intermediate state, per the TreeQL operators which were executed successfully before the error occurred.

The above means in detail that:

1. The first argument is interpreted as the name of a query operator, the number of arguments required by that operator is then determined, and taken from the immediately following arguments.

Because of this operators cannot have optional arguments, all arguments have to be present as defined. Failure to do this will, at least, confuse the query interpreter, but more likely cause errors.

2. The operator is applied to the current node set, yielding a new node set, and/or manipulating the tree object the query object is connected to.
3. The arguments used (i.e. operator name and arguments) are removed from the list of method arguments, and then the whole process is repeated from step [1], until the list of arguments is empty or an error occurred.

```
# q is the query object.

q query root children get data

# The above query
# - Resets the node set to the root node - root
# - Adds the children of root to the set - children
# - Replaces the node set with the       - get data
#   values for the attribute 'data',
#   for all nodes in the set which
#   have such an attribute.
# - And returns this information.

# Below we can see the same query, but rewritten
# to show the structure as it is seen by the query
# interpreter.

q query \
    root \
    children \
    get data
```

The operators of the TreeQL language available for this are explained in the section about The tree query language. This section also explains the term *node set* used above.

***qo* result**

This method returns a list containing the current node set.

***qo* discard**

This method returns the current node set (like method **result**), but also destroys the query object (*qo*). This is useful when constructing and using sub-queries (%AUTO% objects immediately destroyed after use).

The tree query language

This and the following sections specify the Tree Query Language used by the query objects of this package in detail.

First we explain the general concepts underneath the language which are required to comprehend it. This is followed by the specifications for all the available query operators. They fall into eight categories, and each category has its own section.

1. TreeQL concepts
2. Structural generators
3. Attribute filters
4. Attribute Mutators
5. Attribute string accessors
6. Sub-queries
7. Node set operators
8. Node set Iterators
9. Typed node support

TreeQL concepts

The main concept which has to be understood is that of the *node set*. Each query object maintains exactly one such *node set*, and essentially all operators use it and input argument and for their result. This structure simply contains the handles of all nodes which are currently of interest to the query object. To name it a *set* is a bit of a misnomer, because

1. A node (handle) can occur in the structure more than once, and
2. the order of nodes in the structure is important as well. Whenever an operator processes all nodes in the node set it will do so in the order they occur in the structure.

Regarding the possible multiple occurrence of a node, consider a node set containing two nodes A and B, both having node P as their immediate parent. Application of the TreeQL operator "parent" will then add P to the new node set twice, once per node it was parent of. I.e. the new node set will then be {P P}.

Structural generators

All tree-structural operators locate nodes in the tree based on a structural relation ship to the nodes currently in the set and then replace the current node set with the set of nodes found Nodes which fulfill such a relationship multiple times are added to the result as often as they fulfill the relationship.

It is important to note that the found nodes are collected in a separate storage area while processing the node set, and are added to (or replacing) the current node set only after the current node set has been processed completely. In other words, the new nodes are *not* processed by the operator as well and do not affect the iteration.

When describing an operator the variable **N** will be used to refer to any node in the node set.

ancestors
: Replaces the current node set with the ancestors for all nodes **N** in the node set, should **N** have a parent. In other words, nodes without a parent do not contribute to the new node set. In other words, uses all nodes on the path from node **N** to root, in this order (root last), for all nodes **N** in the node set. This includes the root, but not the node itself.

rootpath
: Replaces the current node set with the ancestors for all nodes **N** in the node set, should **N** have a parent. In other words, nodes without a parent do not contribute to the new node set. In contrast to the operator **ancestors** the nodes are added in reverse order however, i.e. the root node first.

parent
: Replaces the current node set with the parent of node **N**, for all nodes **N** in the node set,

should **N** have a parent. In other words, nodes without a parent do not contribute to the new node set.

children
Replaces the current node set with the immediate children of node **N**, for all nodes **N** in the node set, should **N** have children. In other words, nodes without children do not contribute to the new node set.

left
Replaces the current node set with the previous/left sibling for all nodes **N** in the node set, should **N** have siblings to the left. In other words, nodes without left siblings do not contribute to the new node set.

right
Replaces the current node set with the next/right sibling for all nodes **N** in the node set, should **N** have siblings to the right. In other words, nodes without right siblings do not contribute to the new node set.

prev
Replaces the current node set with all previous/left siblings of node **N**, for all nodes **N** in the node set, should **N** have siblings to the left. In other words, nodes without left siblings are ignored. The left sibling adjacent to the node is added first, and the leftmost sibling last (reverse tree order).

esib
Replaces the current node set with all previous/left siblings of node **N**, for all nodes **N** in the node set, should **N** have siblings to the left. In other words, nodes without left siblings are ignored. The leftmost sibling is added first, and the left sibling adjacent to the node last (tree order).

The method name is a shorthand for *Earlier SIBling*.

next
Replaces the current node set with all next/right siblings of node **N**, for all nodes **N** in the node set, should **N** have siblings to the right. In other words, nodes without right siblings do not contribute to the new node set. The right sibling adjacent to the node is added first, and the rightmost sibling last (tree order).

root
Replaces the current node set with a node set containing a single node, the root of the tree.

tree
Replaces the current node set with a node set containing all nodes found in the tree. The nodes are added in pre-order (parent first, then children, the latter from left to right, first to last).

descendants
Replaces the current node set with the nodes in all subtrees rooted at node **N**, for all nodes **N** in the node set, should **N** have children. In other words, nodes without children do not contribute to the new node set.

This is like the operator **children**, but covers the children of children as well, i.e. all the *proper descendants*. "Rooted at **N**" means that **N** itself is not added to the new set, which is also implied by *proper descendants*.

subtree
Like operator **descendants**, but includes the node **N**. In other words:

Replaces the current node set with the nodes of the subtree of node **N**, for all nodes **N** in the node set, should **N** have children. In other words, nodes without children do not contribute to the new node set. I.e this is like the operator **children**, but covers the children of children, etc. as well. "Of **N**" means that **N** itself is added to the new set.

forward
Replaces the current node set with the nodes in the subtrees rooted at the right siblings of node **N**, for all nodes **N** in the node set, should **N** have right siblings, and they children. In other words, nodes without right siblings, and them without children are ignored.

This is equivalent to the operator sequence

```
next descendants
```

later
This is an alias for the operator **forward**.

backward
Replaces the current node set with the nodes in the flattened previous subtrees, in reverse tree order.

This is nearly equivalent to the operator sequence

```
prev descendants
```

The only difference is that this uses the nodes in reverse order.

earlier
Replaces the current node set with the nodes in the flattened previous subtrees, in tree order.

This is equivalent to the operator sequence

```
prev subtree
```

Attribute filters

These operators filter the node set by reference to attributes of nodes and their properties. Filter means that all nodes not fulfilling the criteria are removed from the node set. In other words, the node set is replaced by the set of nodes fulfilling the filter criteria.

hasatt *attr*
Reduces the node set to nodes which have an attribute named *attr*.

withatt *attr value*
Reduces the node set to nodes which have an attribute named *attr*, and where the value of that attribute is equal to *value* (The "==" operator is **string equal -nocase**).

withatt! *attr val*
This is the same as **withatt**, but all nodes in the node set have to have the attribute, and the "==" operator is **string equal**, i.e. no **-nocase**. The operator will fail with an error if they don't have the attribute.

attof *attr vals*

Reduces the node set to nodes which which have an attribute named *attr* and where the value of that attribute is contained in the list *vals* of legal values. The contained-in operator used here does glob matching (using the attribute value as pattern) and ignores the case of the attribute value, *but not* for the elements of *vals*.

attmatch *attr match*

Same as **withatt**, but **string match** is used as the "==" operator, and *match* is the pattern checked for.

Note that *match* is a interpreted as a partial argument *list* for **string match**. This means that it is interpreted as a list containing the pattern, and the pattern element can be preceded by options understand by **string match**, like **-nocase**. This is especially important should the pattern contain spaces. It has to be wrapped into a list for correct interpretation by this operator

Attribute Mutators

These operators change node attributes within the underlying tree. In other words, all these operators have *side effects*.

set *attr val*

Sets the attribute *attr* to the value *val*, for all nodes **N** in the node set. The operator will fail if a node does not have an attribute named *attr*. The tree will be left in a partially modified state.

unset *attr*

Unsets the attribute *attr*, for all nodes **N** in the node set. The operator will fail if a node does not have an attribute named *attr*. The tree will be left in a partially modified state.

Attribute string accessors

These operators retrieve the values of node attributes from the underlying tree. The collected results are stored in the node set, but are not actually nodes.

In other words, they redefine the semantics of the node set stored by the query object to contain non-node data after their completion.

The query interpreter will terminate after it has finished processing one of these operators, silently discarding any later query elements. It also means that our talk about maintenance of a node set is not quite true. It is a node set while the interpreter is processing commands, but can be left as an attribute value set at the end of query processing.

string *op attr*

Applies the string operator *op* to the attribute named *attr*, for all nodes **N** in the node set, collects the results of that application and places them into the node set.

The operator will fail if a node does not have an attribute named *attr*.

The argument *op* is interpreted as partial argument list for the builtin command **string**. Its first word has to be any of the sub-commands understood by **string**. This has to be followed by all arguments required for the subcommand, except the last. that last argument is supplied by the attribute value.

get *pattern*

For all nodes **N** in the node set it determines all their attributes with names matching the glob *pattern*, then the values of these attributes, at last it replaces the node set with the list of these attribute values.

attlist
This is a convenience definition for the operator **getvals** *. In other words, it replaces the node set with a list of the attribute values for all attributes for all nodes **N** in the node set.

attrs *glob*
Replaces the current node set with a list of attribute lists, one attribute list per for all nodes **N** in the node set.

attval *attname*
Reduces the current node set with the operator **hasatt**, and then replaces it with a list containing the values of the attribute named *attname* for all nodes **N** in the node set.

Sub-queries

Sub-queries yield node sets which are then used to augment, reduce or replace the current node set.

andq *query*
Replaces the node set with the set-intersection of the node set generated by the sub-query *query* and itself.

The execution of the sub-query uses the current node set as its own initial node set.

orq *query*
Replaces the node set with the set-union of the node set generated by the sub-query *query* and itself. Duplicate nodes are removed.

The execution of the sub-query uses the current node set as its own initial node set.

notq *query*
Replaces the node set with the set of nodes generated by the sub-query *query* which are also not in the current node set. In other word the set difference of itself and the node set generated by the sub-query.

The execution of the sub-query uses the current node set as its own initial node set.

Node set operators

These operators change the node set directly, without referring to the tree.

unique
Removes duplicate nodes from the node set, preserving order. In other words, the earliest occurrence of a node handle is preserved, every other occurrence is removed.

select
Replaces the current node set with a node set containing only the first node from the current node set

transform *query var body*
First it interprets the sub-query *query*, using the current node set as its initial node set. Then it iterates over the result of that query, binding the handle of each node to the variable named in *var*, and executing the script *body*. The collected results of these executions is

made the new node set, replacing the current one.

The script *body* is executed in the context of the caller.

map *var body*
Iterates over the current node set, binding the handle of each node to the variable named in *var*, and executing the script *body*. The collected results of these executions is made the new node set, replacing the current one.

The script *body* is executed in the context of the caller.

quote *val*
Appends the literal value *val* to the current node set.

replace *val*
Replaces the current node set with the literal list value *val*.

Node set Iterators

foreach *query var body*
Interprets the sub-query *query*, then performs the equivalent of operator **over** on the nodes in the node set created by that query. The current node set is not changed, except through side effects from the script *body*.

The script *body* is executed in the context of the caller.

with *query body*
Interprets the *query*, then runs the script *body* on the node set generated by the query. At last it restores the current node set as it was before the execution of the query.

The script *body* is executed in the context of the caller.

over *var body*
Executes the script *body* for each node in the node set, with the variable named by *var* bound to the name of the current node. The script *body* is executed in the context of the caller.

This is like the builtin **foreach**, with the node set as the source of the list to iterate over.

The results of executing the *body* are ignored.

delete
Deletes all the nodes contained in the current node set from the tree.

Typed node support

These filters and accessors assume the existence of an attribute called **@type**, and are short-hand forms useful for cost-like tree query, html tree editing, and so on.

nodetype
Returns the node type of nodes. Attribute string accessor. This is equivalent to

```
get @type
```

oftype *t*
Reduces the node set to nodes whose type is equal to *t*, with letter case ignored.

nottype *t*
Reduces the node set to nodes whose type is not equal to *t*, with letter case ignored.

oftypes *attrs*
Reduces set to nodes whose @type is an element in the list *attrs* of types. The value of @type is used as a glob pattern, and letter case is relevant.

Examples
... TODO ...

References
1. COST on the Tcler's Wiki.
2. TreeQL on the Tcler's Wiki. Discuss this package there.

Bugs, ideas, feedback
If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *treeql* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Data structures

Copyright

```
Copyright (c) 2004 Colin McCormack <coldstore@users.sourceforge.net>
Copyright (c) 2004 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


try

tcllib_fhome
Keywords: tcllib_fhome

Name
file::home - file home - Return home directory of current or other user

Synopsis
package require Tcl 8.x
package require fhome ?**1**?
file home ?*user*?

Description
This package provides a forward-compatibility implementation of Tcl 9's file home subcommand (TIP 602), for Tcl 8.x.

file home ?*user*?
Returns the home directory of the current user, or of the specified *user* if given.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *file* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Utility

Copyright

```
Copyright (c) 2024 Andreas Kupries, BSD licensed
```

tcllib_throw

Keywords: tcllib_throw, error, return, throw

Name

throw - throw - Throw an error exception with a message

Synopsis

package require Tcl 8.5 9
package require throw ?**1.1**?
::throw *error_code error_message*

Description

This package provides a forward-compatibility implementation of Tcl 8.6's **throw** command (TIP 329), for Tcl 8.5. The code was directly pulled from Tcl 8.6 revision ?, when try/finally was implemented as Tcl procedure instead of in C.

::throw *error_code error_message*
throw is merely a reordering of the arguments of the error command. It throws an error with the indicated error code and error message.

Examples

```
throw {MYERROR CODE} "My error message"
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *try* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Utility

Copyright

```
Copyright (c) 2015 Miguel Martínez López, BSD licensed
```


tcllib_try
Keywords: tcllib_try, cleanup, error, exception, final, resource management

Name
try - try - Trap and process errors and exceptions

Synopsis
package require Tcl 8.5 9
package require try ?**1.1**?
::try *body* ?*handler...*? ?*finally script*?

Description
This package provides a forward-compatibility implementation of Tcl 8.6's **try**/**finally** command (TIP 329), for Tcl 8.5. The code was directly pulled from Tcl 8.6 revision ?, when try/finally was implemented as Tcl procedure instead of in C.

::try *body* ?*handler...*? ?finally *script*?
This command executes the script *body* and, depending on what the outcome of that script is (normal exit, error, or some other exceptional result), runs a handler script to deal with the case. Once that has all happened, if the **finally** clause is present, the *script* it includes will be run and the result of the handler (or the *body* if no handler matched) is allowed to continue to propagate. Note that the **finally** clause is processed even if an error occurs and irrespective of which, if any, *handler* is used.

The *handler* clauses are each expressed as several words, and must have one of the following forms:

on *code variableList script*
This clause matches if the evaluation of *body* completed with the exception code *code*. The *code* may be expressed as an integer or one of the following literal words: **ok**, **error**, **return**, **break**, or **continue**. Those literals correspond to the integers 0 through 4 respectively.

trap *pattern variableList script*
This clause matches if the evaluation of *body* resulted in an error and the prefix of the **-errorcode** from the interpreter's status dictionary is equal to the *pattern*. The number of prefix words taken from the **-errorcode** is equal to the list-length of *pattern*, and inter-word spaces are normalized in both the **-errorcode** and *pattern* before comparison.

The *variableList* word in each *handler* is always interpreted as a list of variable names. If the first word of the list is present and non-empty, it names a variable into which the result of the evaluation of *body* (from the main **try**) will be placed; this will contain the human-readable form of any errors. If the second word of the list is present and non-empty, it names a variable into which the options dictionary of the interpreter at the moment of completion of execution of *body* will be placed.

The *script* word of each *handler* is also always interpreted the same: as a Tcl script to evaluate if the clause is matched. If *script* is a literal **-** and the *handler* is not the last one, the *script* of the following *handler* is invoked instead (just like with the **switch** command).

Note that *handler* clauses are matched against in order, and that the first matching one is always selected. At most one *handler* clause will selected. As a consequence, an **on error** will mask any subsequent **trap** in the **try**. Also note that **on error** is equivalent to **trap {}** .

If an exception (i.e. any non-**ok** result) occurs during the evaluation of either the *handler* or the **finally** clause, the original exception's status dictionary will be added to the new exception's status dictionary under the **-during** key.

Examples

Ensure that a file is closed no matter what:

```
set f [open /some/file/name a]
try {
    puts \$f "some message"
    # ...
} finally {
    close \$f
}
```

Handle different reasons for a file to not be openable for reading:

```
try {
    set f [open /some/file/name]
} trap {POSIX EISDIR} {} {
    puts "failed to open /some/file/name: it's a directory"
} trap {POSIX ENOENT} {} {
    puts "failed to open /some/file/name: it doesn't exist"
}
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *try* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be

made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Utility

Copyright
```
Copyright (c) 2008 Donal K. Fellows, BSD licensed
```

udpcluster

udpcluster
Keywords: udpcluster, name service, server

Name
udpcluster - UDP Peer-to-Peer cluster

Synopsis
package require Tcl 8.5 9
package require udpcluster ?**0.3.4**?
package require ip
package require nettool
package require comm
package require interp
package require dicttool
package require cron

Description
This package is a lightweight alternative to Zeroconf. It utilizes UDP packets to populate a table of services provided by each node on a local network. Each participant broadcasts a key/value list in plain UTF-8 which lists what ports are open, and what protocols are expected on those ports. Developers are free to add any additional key/value pairs beyond those.

Using udpcluster.

For every service you wish to publish invoke:

```
cluster::publish echo@[cluster::macid] {port 10000 protocol echo}
```

To query what services are available on the local network:

```
set results [cluster::search PATTERN]
# And inside that result...
echo@LOCALMACID {
   port 10000
   protocol echo
}
```

To unpublish a service:

```
cluster::unpublish echo@[cluster::macid]
```

Results will Historical Notes:

This tool was originally known as nns::cluster, but as development progressed, it was clear that it wasn't interacting with any of the other facilities in NNS.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *nameserv* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Networking

Copyright

```
Copyright (c) 2016-2018 Sean Woods <yoda@etoyoc.com>
```

uev

uevent

Keywords: uevent, bind, event, generate event, hook, unbind

Name

uevent - User events

Synopsis

package require Tcl 8.5 9
package require uevent ?**0.3.2**?
package require logger
::uevent::bind *tag event command*
command *tag event details*
::uevent::unbind *token*
::uevent::generate *tag event* ?*details*?
::uevent::list
::uevent::list *tag*
::uevent::list *tag event*
::uevent::watch::tag::add *pattern command*
{*}command bound *tag*
{*}command unbound *tag*
::uevent::watch::tag::remove *token*
::uevent::watch::event::add *tag_pattern event_pattern command*

{*}command bound *tag event*
{*}command unbound *tag event*
::uevent::watch::event::remove *token*

Description

This package provides a general facility for the handling of user events. Allows the binding of arbitrary commands to arbitrary events on arbitrary tags, removal of bindings, and event generation.

The main difference to the event system built into the Tcl/Tk core is that the latter can generate only virtual events, and only for widgets. It is not possible to use the builtin facilities to bind to events on arbitrary (pseudo-)objects, nor is it able to generate events for such.

Here we can, by assuming that each object in question is represented by its own tag. Which is possible as we allow arbitrary tags.

More differences:

1. The package uses only a two-level hierarchy, tags and events, to handle everything, whereas the Tcl/Tk system uses three levels, i.e. objects, tags, and events, with a n:m relationship between objects and tags.
2. This package triggers all bound commands for a tag/event combination, and they are independent of each other. A bound command cannot force the event processing core to abort the processing of command coming after it.

API

The package exports eight commands, as specified below. Note that when the package is used from within Tcl 8.5+ all the higher commands are ensembles, i.e. the :: separators can be replaceed by spaces.

::uevent::bind *tag event command*

Using this command registers the *command* prefix to be triggered when the *event* occurs for the *tag*. The result of the command is an opaque token representing the binding. Note that if the same combination of <*tag*,*event*,*command*> is used multiple times the same token is returned by every call.

The signature of the *command* prefix is
- **command** *tag event details*

where *details* contains the argument(s) of the event. Its contents are event specific and have to be agreed upon between actual event generator and consumer. This package simply transfers the information and does not perform any processing beyond that.

::uevent::unbind *token*

This command releases the event binding represented by the *token*. The token has to be the result of a call to **::uevent::bind**. The result of the command is the empty string.

::uevent::generate *tag event* ?*details*?

This command generates an *event* for the *tag*, triggering all commands bound to that combination. The *details* argument is simply passed unchanged to all event handlers. It is the responsibility of the code generating and consuming the event to have an agreement about the format and contents of the information carried therein. The result of the command is the empty string.

Note that all bound commands are triggered, independently of each other. The event handlers cannot assume a specific order. They are also *not* called synchronously with the invokation of this command, but simply put into the event queue for processing when the system returns to the event loop.

Generating an event for an unknown tag, or for a <*tag*,*event*> combination which has no commands bound to it is allowed, such calls will be ignored.

::uevent::list

In this form the command returns a list containing the names of all tags which have events with commands bound to them.

::uevent::list *tag*

In this format the command returns a list containing the names of all events for the *tag* with commands bound to them. Specifying an unknown tag, i.e. a tag without event and commands, will cause the command to throw an error.

::uevent::list *tag event*

In this format the command returns a list containing all commands bound to the *event* for the *tag*. Specifying an unknown tag or unknown event, will cause the command to throw an error.

::uevent::watch::tag::add *pattern command*

This command sets up a sort of reverse events. Events generated, i.e. the *command* prefix invoked, when observers bind to and unbind from specific tags.

Note that the command prefix is only invoked twice per tag, first when the first command is bound to any event of the tag, and second when the last command bound to the tag is removed.

The signature of the *command* prefix is
- **{*}command bound** *tag*
- **{*}command unbound** *tag*

The result of the command is a token representing the watcher.

::uevent::watch::tag::remove *token*

This command removes a watcher for (un)bind events on tags.

The result of the command is the empty string.

::uevent::watch::event::add *tag_pattern event_pattern command*

This command sets up a sort of reverse events. Events generated, i.e. the *command* prefix invoked, when observers bind to and unbind from specific combinations of tags and events.

Note that the command prefix is only invoked twice per tag/event combination, first when the first command is bound to it, and second when the last command bound to the it is removed.

The signature of the *command* prefix is
- **{*}command bound** *tag event*
- **{*}command unbound** *tag event*

The result of the command is a token representing the watcher.

::uevent::watch::event::remove *token*

This command removes a watcher for (un)bind events on tag/event combinations.

The result of the command is the empty string.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *uevent* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Programming tools

Copyright


```
Copyright (c) 2007-2012 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


uevent_onidle

Keywords: uevent_onidle, callback, deferal, event, idle, merge, on-idle

Name

uevent::onidle - Request merging and deferal to idle time

Synopsis

package require Tcl 8.5 9
package require uevent::onidle ?**0.2**?
package require logger
::uevent::onidle *objectName commandprefix*
objectName **request**

Description

This package provides objects which can merge multiple requestes for an action and execute the action the moment the system (event loop) becomes idle. The action to be run is configured during object construction.

API

The package exports a class, **uevent::onidle**, as specified below.

::uevent::onidle *objectName commandprefix*

The command creates a new *onidle* object with an associated global Tcl command whose name is *objectName*. This command may be used to invoke various operations on the object.

The *commandprefix* is the action to perform when the event loop is idle and the user asked for it using the method **request** (See below).

The object commands created by the class commands above have the form:

***objectName* request**
: This method requests the execution of the command prefix specified during the construction of *objectName* the next time the event loop is idle. Multiple requests are merged and cause only one execution of the command prefix.

Examples
Examples of this type of deferal are buried in the (C-level) implementations all the Tk widgets, defering geometry calculations and window redraw activity in this manner.

Bugs, ideas, feedback
If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *uevent* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Copyright

```
Copyright (c) 2008 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


units

units
Keywords: units, angle, constants, conversion, distance, radians, unit

Name
units - unit conversion

Synopsis
package require Tcl 8.5 9
package require units ?**2.2.3**?
::units::convert *value targetUnits*
::units::reduce *unitString*
::units::new *name baseUnits*

Description
This library provides a conversion facility from a variety of scientific and engineering shorthand notations into floating point numbers. This allows application developers to easily convert values with different units into uniformly scaled numbers.

The units conversion facility is also able to convert between compatible units. If, for example, a application is expecting a value in *ohms* (Resistance), and the user specifies units of *milliwebers/femtocoulomb*, the conversion routine will handle it appropriately. An error will be

generated if an incorrect conversion is attempted.

Values are scaled from one set of units to another by dimensional analysis. Both the value units and the target units are reduced into primitive units and a scale factor. Units are checked for compatibility, and the scale factors are applied by multiplication and division. This technique is extremely flexible and quite robust.

New units and new unit abbreviations can be defined in terms of existing units and abbreviations. It is also possible to define a new primitive unit, although that will probably be unnecessary. New units will most commonly be defined to accommodate non-SI measurement systems, such as defining the unit *inch* as *2.54 cm*.

Commands

::units::convert *value targetUnits*

Converts the *value* string into a floating point number, scaled to the specified *targetUnits*. The *value* string may contain a number and units. If units are specified, then they must be compatible with the *targetUnits*. If units are not specified for the *value*, then it will be scaled to the target units. For example,

```
% ::units::convert "2.3 miles" km
3.7014912
% ::units::convert 300m/s miles/hour
671.080887616
% ::units::convert "1.0 m kg/s^2" newton
1.0
% ::units::convert 1.0 millimeter
1000.0
```

::units::reduce *unitString*

Returns a unit string consisting of a scale factor followed by a space separated list of sorted and reduced primitive units. The reduced unit string may include a forward-slash (separated from the surrounding primitive subunits by spaces) indicating that the remaining subunits are in the denominator. Generates an error if the *unitString* is invalid.

```
% ::units::reduce pascal
1000.0 gram / meter second second
```

::units::new *name baseUnits*

Creates a new unit conversion with the specified name. The new unit *name* must be only alphabetic (upper or lower case) letters. The *baseUnits* string can consist of any valid units conversion string, including constant factors, numerator and denominator parts, units with prefixes, and exponents. The baseUnits may contain any number of subunits, but it must reduce to primitive units. BaseUnits could also be the string *-primitive* to represent a new kind of quantity which cannot be derived from other units. But you probably would not do that unless you have discovered some kind of new universal property.

```
% ::units::new furlong "220 yards"
% ::units::new fortnight "14 days"
% ::units::convert 100m/s furlongs/fortnight
601288.475303
```

Unit string format

Value and unit string format is quite flexible. It is possible to define virtually any combination of units, prefixes, and powers. Valid unit strings must conform to these rules.

- A unit string consists of an optional scale factor followed by zero or more subunits. The scale factor must be a valid floating point number, and may or may not be separated from the subunits. The scale factor could be negative.
- Subunits are separated form each other by one or more separator characters, which are space (" "), hyphen ("-"), asterisk ("*"), and forward-slash ("/"). Sure, go ahead and complain about using a minus sign ("-") to represent multiplication. It just isn't sound mathematics, and, by rights, we should require everyone to use the asterisk ("*") to separate all units. But the bottom line is that complex unit strings like *m-kg/s^2* are pleasantly readable.
- The forward-slash seperator ("/") indicates that following subunits are in the denominator. There can be at most one forward-slash separator.
- Subunits can be floating point scale factors, but with the exception of the leading scale factor, they must be surrounded by valid separators. Subunit scale factors cannot be negative. (Remember that the hyphen is a unit separator.)
- Subunits can be valid units or abbreviations. They may include a prefix. They may include a plural suffix "s" or "es". They may also include a power string denoted by a circumflex ("^"), followed by a integer, after the unit name (or plural suffix, if there is one). Negative exponents are not allowed. (Remember that the hyphen is a unit separator.)

Example valid unit strings

```
Unit String              Reduced Unit String
-----------------------------------------------------------
meter                    1.0 meter
kilometer                1000.0 meter
km                       1000.0 meter
km/s                     1000.0 meter / second
/microsecond             1000000.0 / second
/us                      1000000.0 / second
kg-m/s^2                 1000.0 gram meter / second second
30second                 30.0 second
30 second                30.0 second
30 seconds               30.0 second
200*meter/20.5*second    9.75609756098 meter / second
```

SI units

The standard SI units are predefined according to *NIST Special Publication 330*. Standard units for both SI Base Units (Table 1) and SI Derived Units with Special Names (Tables 3a and 3b) are included here for reference. Each standard unit name and abbreviation are included in this

package.

SI Base units

```
Quantity                Unit Name    Abbr.
---------------------------------------------
Length                  meter        m
Mass                    kilogram     kg
Time                    second       s
Current                 ampere       A
Temperature             kelvin       K
Amount                  mole         mol
Luminous Intensity      candela      cd
```

SI derived units with Special names

```
Quantity                Unit Name    Abbr.   Units     Base Units
-------------------------------------------------------------------
plane angle             radian       rad     m/m       m/m
solid angle             steradian    sr      m^2/m^2   m^2/m^2
frequency               hertz        Hz                /s
force                   newton       N                 m-kg/s^2
pressure                pascal       Pa      N/m^2     kg/m-s^2
energy, work            joule        J       N-m       m^2-kg/s^2
power, radiant flux     watt         W       J/s       m^2-kg/s^3
electric charge         coulomb      C                 s-A
electric potential      volt         V       W/A       m^2-kg/s^3-A
capacitance             farad        F       C/V       s^4-A^2/m^2-kg
electric resistance     ohm                  V/A       m^2-kg/s^3-A^2
electric conductance    siemens      S       A/V       s^3-A^2/m^2-kg
magnetic flux           weber        Wb      V-s       m^2-kg/s^2-A
magnetic flux density   tesla        T       Wb/m^2    kg/s^2-A
inductance              henry        H       Wb/A      m^2-kg/s^2-A^2
luminous flux           lumen        lm                cd-sr
illuminance             lux          lx      lm/m^2    cd-sr/m^2
activity (of a
radionuclide)           becquerel    Bq                /s
absorbed dose           gray         Gy      J/kg      m^2/s^2
dose equivalent         sievert      Sv      J/kg      m^2/s^2
```

Note that the SI unit kilograms is actually implemented as grams because 1e-6 kilogram = 1 milligram, not 1 microkilogram. The abbreviation for Electric Resistance (ohms), which is the omega character, is not supported.

Also note that there is no support for Celsius or Farenheit temperature. The units conversion routines can only scale values with multiplication and division, so it is not possible to convert from thermodynamic temperature (kelvins) to absolute degrees Celsius or Farenheit. Conversion of thermodynamic quantities, such as thermal expansion (per unit temperature), however, are easy to add to the units library.

SI Units can have a multiple or sub-multiple prefix. The prefix or its abbreviation should appear before the unit, without spaces. Compound prefixes are not allowed, and a prefix should never be used alone. These prefixes are defined in Table 5 of *Special Publication 330*.

SI prefixes

```
Prefix Name     Abbr.   Factor
----------------------------------------
yotta           Y       1e24
zetta           Z       1e21
exa             E       1e18
peta            P       1e15
tera            T       1e12
giga            G       1e9
mega            M       1e6
kilo            k       1e3
hecto           h       1e2
deka            da      1e1
deca                    1e1

deci            d       1e-1
centi           c       1e-2
milli           m       1e-3
micro           u       1e-6
nano            n       1e-9
pico            p       1e-12
femto           f       1e-15
atto            a       1e-18
zepto           z       1e-21
yocto           y       1e-24
```

Note that we define the same prefix with both the USA ("deka") and non-USA ("deca") spellings. Also note that we take the liberty of allowing "micro" to be typed as a "u" instead of the Greek character mu.

Many non-SI units are commonly used in applications. Appendix B.8 of *NIST Special Publication 811* lists many non-SI conversion factors. It is not possible to include all possible unit definitions in this package. In some cases, many different conversion factors exist for a given unit, depending on the context. (The appendix lists over 40 conversions for British thermal units!) Application specific conversions can always be added using the **new** command, but some well known and often used conversions are included in this package.

Non-SI units

```
Unit Name             Abbr.    Base Units
-------------------------------------------------
angstrom                       1.0E-10 m
astronomicalUnit      AU       1.495979E11 m
atmosphere                     1.01325E5 Pa
```

```
bar                         1.0E5 Pa
calorie                     4.1868 J
curie                       3.7E10 Bq
day                         8.64E4 s
degree                      1.745329E-2 rad
erg                         1.0E-7 J
faraday                     9.648531 C
fermi                       1.0E-15 m
foot                ft      3.048E-1 m
gauss                       1.0E-4 T
gilbert                     7.957747E-1 A
grain               gr      6.479891E-5 kg
hectare             ha      1.0E4 m^2
hour                h       3.6E3 s
inch                in      2.54E-2 m
lightYear                   9.46073E15 m
liter               L       1.0E-3 m^3
maxwell             Mx      1.0E-8 Wb
mho                         1.0 S
micron                      1.0E-6 m
mil                         2.54E-5 m
mile                mi      1.609344E3 m
minute              min     6.0E1 s
parsec              pc      3.085E16 m
pica                        4.233333E-3 m
pound               lb      4.535924E-1 kg
revolution                  6.283185 rad
revolutionPerMinute rpm     1.047198E-1 rad/s
yard                yd      9.144E-1 m
year                        3.1536E7 s
```

Quantities and derived units with Special names

This units conversion package is limited specifically to unit reduction, comparison, and scaling. This package does not consider any of the quantity names for either base or derived units. A similar implementation or an extension in a typed or object-oriented language might introduce user defined types for the quantities. Quantity type checking could be used, for example, to ensure that all *length* values properly reduced to *meters*, or that all *velocity* values properly reduced to *meters/second*.

A C implementation of this package has been created to work in conjunction with the Simplified Wrapper Interface Generator (http://www.swig.org/). That package (units.i) exploits SWIG's typemap system to automatically convert script quantity strings into floating point quantities. Function arguments are specified as quantity types (e.g., *typedef float Length*), and target units (expected by the C application code) are specified in an associative array. Default units are also defined for each quantity type, and are applied to any unit-less quantity strings.

A units system enhanced with quantity type checking might benefit from inclusion of other

derived types which are expressed in terms of special units, as illustrated in Table 2 of *NIST Publication 330*. The quantity *area*, for example, could be defined as units properly reducing to *meter^2*, although the utility of defining a unit named *square meter* is arguable.

References

The unit names, abbreviations, and conversion values are derived from those published by the United States Department of Commerce Technology Administration, National Institute of Standards and Technology (NIST) in *NIST Special Publication 330: The International System of Units (SI)* and *NIST Special Publication 811: Guide for the Use of the International System of Units (SI)*. Both of these publications are available (as of December 2000) from http://physics.nist.gov/cuu/Reference/contents.html

The ideas behind implementation of this package is based in part on code written in 1993 by Adrian Mariano which performed dimensional analysis of unit strings using fixed size tables of C structs. After going missing in the late 1990's, Adrian's code has reappeared in the GNU Units program at http://www.gnu.org/software/units/

Authors

Robert W. Techentin

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *units* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Copyright

```
Copyright (c) 2000-2005 Mayo Foundation
```

uri

uri

Keywords: uri, fetching information, file, ftp, gopher, http, https, ldap, mailto, news, prospero, rfc 1630, rfc 2255, rfc 2396, rfc 3986, url, wais, www

Name

uri - URI utilities

Synopsis

package require Tcl 8.5 9
package require uri ?**1.2.8**?
uri::setQuirkOption *option* ?*value*?
uri::split *url* ?*defaultscheme*?

uri::join ?*key value* ...?
uri::resolve *base url*
uri::isrelative *url*
uri::geturl *url* ?*options...*?
uri::canonicalize *uri*
uri::register *schemeList script*

Description

This package does two things.

First, it provides a number of commands for manipulating URLs/URIs and fetching data specified by them. For fetching data this package analyses the requested URL/URI and then dispatches it to the appropriate package (**http**, **ftp**, ...) for actual retrieval. Currently these commands are defined for the schemes *http*, *https*, *ftp*, *mailto*, *news*, *ldap*, *ldaps* and *file*. The package **uri::urn** adds scheme *urn*.

Second, it provides regular expressions for a number of **registered** URL/URI schemes. Registered schemes are currently *ftp*, *ldap*, *ldaps*, *file*, *http*, *https*, *gopher*, *mailto*, *news*, *wais* and *prospero*. The package **uri::urn** adds scheme *urn*.

The commands of the package conform to RFC 3986 (https://www.rfc-editor.org/rfc/rfc3986.txt), with the exception of a loophole arising from RFC 1630 and described in RFC 3986 Sections 5.2.2 and 5.4.2. The loophole allows a relative URI to include a scheme if it is the same as the scheme of the base URI against which it is resolved. RFC 3986 recommends avoiding this usage.

Commands

uri::setQuirkOption *option* ?*value*?

uri::setQuirkOption is an accessor command for a number of "quirk options". The command has the same semantics as the command **set**: when called with one argument it reads an existing value; with two arguments it writes a new value. The value of a "quirk option" is boolean: the value **false** requests conformance with RFC 3986, while **true** requests use of the quirk. See section Quirk options for discussion of the different options and their purpose.

uri::split *url* ?*defaultscheme*?

uri::split takes a *url*, decodes it and then returns a list of key/value pairs suitable for **array set** containing the constituents of the *url*. If the scheme is missing from the *url* it defaults to the value of *defaultscheme* if it was specified, or *http* else. Currently the schemes *http*, *https*, *ftp*, *mailto*, *news*, *ldap*, *ldaps* and *file* are supported by the package itself. See section Extending on how to expand that range.

The set of constituents of a URL (= the set of keys in the returned dictionary) is dependent on the scheme of the URL. The only key which is therefore always present is **scheme**. For the following schemes the constituents and their keys are known:

ftp

user, **pwd**, **host**, **port**, **path**, **type**, **pbare**. The pbare is optional.

http(s)

user, **pwd**, **host**, **port**, **path**, **query**, **fragment**, **pbare**. The pbare is optional.

file

path, **host**. The host is optional.

mailto

user, **host**. The host is optional.

ldap(s)
host, **port**, **dn**, **attrs**, **scope**, **filter**, **extensions**

news
Either **message-id** or **newsgroup-name**.

For discussion of the boolean **pbare** see options *NoInitialSlash* and *NoExtraKeys* in Quirk options.

The constituents are returned as slices of the argument *url*, without removal of percent-encoding ("url-encoding") or other adaptations. Notably, on Windows® the **path** in scheme *file* is not a valid local filename. See Examples for more information.

uri::join ?*key value*?...
uri::join takes a list of key/value pairs (generated by **uri::split**, for example) and returns the canonical URL they represent. Currently the schemes *http*, *https*, *ftp*, *mailto*, *news*, *ldap*, *ldaps* and *file* are supported by the package itself. See section Extending on how to expand that range.

The arguments are expected to be slices of a valid URL, with percent-encoding ("url-encoding") and any other necessary adaptations. Notably, on Windows the **path** in scheme *file* is not a valid local filename. See Examples for more information.

uri::resolve *base url*
uri::resolve resolves the specified *url* relative to *base*, in conformance with RFC 3986. In other words: a non-relative *url* is returned unchanged, whereas for a relative *url* the missing parts are taken from *base* and prepended to it. The result of this operation is returned. For an empty *url* the result is *base*, without its URI fragment (if any). The command is available for schemes *http*, *https*, *ftp*, and *file*.

uri::isrelative *url*
uri::isrelative determines whether the specified *url* is absolute or relative. The command is available for a *url* of any scheme.

uri::geturl *url* ?*options*...?
uri::geturl decodes the specified *url* and then dispatches the request to the package appropriate for the scheme found in the URL. The command assumes that the package to handle the given scheme either has the same name as the scheme itself (including possible capitalization) followed by **::geturl**, or, in case of this failing, has the same name as the scheme itself (including possible capitalization). It further assumes that whatever package was loaded provides a **geturl**-command in the namespace of the same name as the package itself. This command is called with the given *url* and all given *options*. Currently **geturl** does not handle any options itself.

Note: file-URLs are an exception to the rule described above. They are handled internally.

It is not possible to specify results of the command. They depend on the **geturl**-command for the scheme the request was dispatched to.

uri::canonicalize *uri*
uri::canonicalize returns the canonical form of a URI. The canonical form of a URI is one

where relative path specifications, i.e. "." and "..", have been resolved. The command is available for all URI schemes that have **uri::split** and **uri::join** commands. The command returns a canonicalized URI if the URI scheme has a **path** component (i.e. *http*, *https*, *ftp*, and *file*). For schemes that have **uri::split** and **uri::join** commands but no **path** component (i.e. *mailto*, *news*, *ldap*, and *ldaps*), the command returns the *uri* unchanged.

uri::register *schemeList script*

uri::register registers the first element of *schemeList* as a new scheme and the remaining elements as aliases for this scheme. It creates the namespace for the scheme and executes the *script* in the new namespace. The script has to declare variables containing regular expressions relevant to the scheme. At least the variable **schemepart** has to be declared as that one is used to extend the variables keeping track of the registered schemes.

Schemes

In addition to the commands mentioned above this package provides regular expression to recognize URLs for a number of URL schemes.

For each supported scheme a namespace of the same name as the scheme itself is provided inside of the namespace *uri* containing the variable **url** whose contents are a regular expression to recognize URLs of that scheme. Additional variables may contain regular expressions for parts of URLs for that scheme.

The variable **uri::schemes** contains a list of all registered schemes. Currently these are *ftp*, *ldap* , *ldaps*, *file*, *http*, *https*, *gopher*, *mailto*, *news*, *wais* and *prospero*.

Extending

Extending the range of schemes supported by **uri::split** and **uri::join** is easy because both commands do not handle the request by themselves but dispatch it to another command in the *uri* namespace using the scheme of the URL as criterion.

uri::split and **uri::join** call **Split[string totitle]** and **Join[string totitle]** respectively.

The provision of split and join commands is sufficient to extend the commands **uri::canonicalize** and **uri::geturl** (the latter subject to the availability of a suitable package with a **geturl** command). In contrast, to extend the command **uri::resolve** to a new scheme, the command itself must be modified.

To extend the range of schemes for which pattern information is available, use the command **uri::register**.

An example of a package that provides both commands and pattern information for a new scheme is **uri::urn**, which adds scheme *urn*.

Quirk options

The value of a "quirk option" is boolean: the value **false** requests conformance with RFC 3986, while **true** requests use of the quirk. Use command **uri::setQuirkOption** to access the values of quirk options.

Quirk options are useful both for allowing backwards compatibility when a command specification changes, and for adding useful features that are not included in RFC specifications. The following quirk options are currently defined:

NoInitialSlash

This quirk option concerns the leading character of **path** (if non-empty) in the schemes *http*, *https*, and *ftp*.

RFC 3986 defines **path** in an absolute URI to have an initial "/", unless the value of **path** is the empty string. For the scheme *file*, all versions of package **uri** follow this rule. The quirk option *NoInitialSlash* does not apply to scheme *file*.

For the schemes *http*, *https*, and *ftp*, versions of **uri** before 1.2.7 define the **path** *NOT* to include an initial "/". When the quirk option *NoInitialSlash* is **true** (the default), this behavior is also used in version 1.2.7. To use instead values of **path** as defined by RFC 3986, set this quirk option to **false**.

This setting does not affect RFC 3986 conformance. If *NoInitialSlash* is **true**, then the value of **path** in the schemes *http*, *https*, or *ftp*, cannot distinguish between URIs in which the full "RFC 3986 path" is the empty string "" or a single slash "/" respectively. The missing information is recorded in an additional **uri::split** key **pbare**.

The boolean **pbare** is defined when quirk options *NoInitialSlash* and *NoExtraKeys* have values **true** and **false** respectively. In this case, if the value of **path** is the empty string "", **pbare** is **true** if the full "RFC 3986 path" is "", and **pbare** is **false** if the full "RFC 3986 path" is "/".

Using this quirk option *NoInitialSlash* is a matter of preference.

NoExtraKeys

This quirk option permits full backward compatibility with versions of **uri** before 1.2.7, by omitting the **uri::split** key **pbare** described above (see quirk option *NoInitialSlash*). The outcome is greater backward compatibility of the **uri::split** command, but an inability to distinguish between URIs in which the full "RFC 3986 path" is the empty string "" or a single slash "/" respectively - i.e. a minor non-conformance with RFC 3986.

If the quirk option *NoExtraKeys* is **false** (the default), command **uri::split** returns an additional key **pbare**, and the commands comply with RFC 3986. If the quirk option *NoExtraKeys* is **true**, the key **pbare** is not defined and there is not full conformance with RFC 3986.

Using the quirk option *NoExtraKeys* is *NOT* recommended, because if set to **true** it will reduce conformance with RFC 3986. The option is included only for compatibility with code, written for earlier versions of **uri**, that needs values of **path** without a leading "/", *AND ALSO* cannot tolerate unexpected keys in the results of **uri::split**.

HostAsDriveLetter

When handling the scheme *file* on the Windows platform, versions of **uri** before 1.2.7 use the **host** field to represent a Windows drive letter and the colon that follows it, and the **path** field to represent the filename path after the colon. Such URIs are invalid, and are not recognized by any RFC. When the quirk option *HostAsDriveLetter* is **true**, this behavior is also used in version 1.2.7. To use *file* URIs on Windows that conform to RFC 3986, set this quirk option to **false** (the default).

Using this quirk is *NOT* recommended, because if set to **true** it will cause the **uri** commands to expect and produce invalid URIs. The option is included only for compatibility with legacy code.

RemoveDoubleSlashes

When a URI is canonicalized by **uri::canonicalize**, its **path** is normalized by removal of segments "." and "..". RFC 3986 does not mandate the removal of empty segments "" (i.e. the merger of double slashes, which is a feature of filename normalization but not of URI **path** normalization): it treats URIs with excess slashes as referring to different resources. When the quirk option *RemoveDoubleSlashes* is **true** (the default), empty segments will be removed from **path**. To prevent removal, and thereby conform to RFC 3986, set this quirk option to **false**.

Using this quirk is a matter of preference. A URI with double slashes in its path was most likely generated by error, certainly so if it has a straightforward mapping to a file on a server. In some cases it may be better to sanitize the URI; in others, to keep the URI and let the server handle the possible error.

Backward compatibility

To behave as similarly as possible to versions of **uri** earlier than 1.2.7, set the following quirk options:

- **uri::setQuirkOption** *NoInitialSlash* 1
- **uri::setQuirkOption** *NoExtraKeys* 1
- **uri::setQuirkOption** *HostAsDriveLetter* 1
- **uri::setQuirkOption** *RemoveDoubleSlashes* 0

In code that can tolerate the return by **uri::split** of an additional key **pbare**, set

- **uri::setQuirkOption** *NoExtraKeys* 0

in order to achieve greater compliance with RFC 3986.

New designs

For new projects, the following settings are recommended:

- **uri::setQuirkOption** *NoInitialSlash* 0
- **uri::setQuirkOption** *NoExtraKeys* 0
- **uri::setQuirkOption** *HostAsDriveLetter* 0
- **uri::setQuirkOption** *RemoveDoubleSlashes* 0|1

Default values

The default values for package **uri** version 1.2.7 are intended to be a compromise between backwards compatibility and improved features. Different default values may be chosen in future versions of package **uri**.

- **uri::setQuirkOption** *NoInitialSlash* 1
- **uri::setQuirkOption** *NoExtraKeys* 0
- **uri::setQuirkOption** *HostAsDriveLetter* 0
- **uri::setQuirkOption** *RemoveDoubleSlashes* 1

Examples

A Windows® local filename such as "**C:\Other Files\startup.txt**" is not suitable for use as the **path** element of a URI in the scheme *file*.

The Tcl command **file normalize** will convert the backslashes to forward slashes. To generate a valid **path** for the scheme *file*, the normalized filename must be prepended with "**/**", and then any characters that do not match the **regexp** bracket expression

```
    [a-zA-Z0-9$_.+!*'(,)?:@&=-]
```

must be percent-encoded.

The result in this example is "**/C:/Other%20Files/startup.txt**" which is a valid value for **path**.

```
% uri::join path /C:/Other%20Files/startup.txt scheme file

file:///C:/Other%20Files/startup.txt

% uri::split file:///C:/Other%20Files/startup.txt

path /C:/Other%20Files/startup.txt scheme file
```

On UNIX® systems filenames begin with "**/**" which is also used as the directory separator. The only action needed to convert a filename to a valid **path** is percent-encoding.

Credits

Original code (regular expressions) by Andreas Kupries. Modularisation by Steve Ball, also the split/join/resolve functionality. RFC 3986 conformance by Keith Nash.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *uri* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Networking

urn-scheme

Keywords: urn-scheme, rfc 2141, uri, url, urn

Name

uri_urn - URI utilities, URN scheme

Synopsis

package require Tcl 8.5 9
package require uri::urn ?**1.0.4**?
uri::urn::quote *url*

uri::urn::unquote *url*

Description

This package provides two commands to quote and unquote the disallowed characters for url using the *urn* scheme, registers the scheme with the package **uri**, and provides internal helpers which will be automatically used by the commands uri::split and uri::join of package **uri** to handle urls using the *urn* scheme.

Commands

uri::urn::quote *url*

This command quotes the characters disallowed by the *urn* scheme (per RFC 2141 sec2.2) in the *url* and returns the modified url as its result.

uri::urn::unquote *url*

This commands performs the reverse of **::uri::urn::quote**. It takes an *urn* url, removes the quoting from all disallowed characters, and returns the modified urls as its result.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *uri* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Networking

uuid

uuid

Keywords: uuid, GUID

Name

uuid - UUID generation and comparison

Synopsis

package require Tcl 8.5 9
package require uuid ?**1.0.9**?
::uuid::uuid generate
::uuid::uuid equal *id1 id2*

Description

This package provides a generator of universally unique identifiers (UUID) also known as globally unique identifiers (GUID). This implementation follows the draft specification from (1) although this is actually an expired draft document.

Commands

::uuid::uuid generate
Creates a type 4 uuid by MD5 hashing a number of bits of variant data including the time and hostname. Returns the string representation of the new uuid.

::uuid::uuid equal *id1 id2*
Compares two uuids and returns true if both arguments are the same uuid.

Examples

```
% uuid::uuid generate
b12dc22c-5c36-41d2-57da-e29d0ef5839c
```

References

1. Paul J. Leach, "UUIDs and GUIDs", February 1998. (http://www.opengroup.org/dce/info/draft-leach-uuids-guids-01.txt)

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *uuid* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Hashes, checksums, and encryption

Copyright

```
Copyright (c) 2004, Pat Thoyts <patthoyts@users.sourceforge.net>
```

valtype

cc_amex

Keywords: cc_amex, AMEX, American Express, Checking, Testing, Type checking, Validation, Value checking, bank, card for credit, credit card, finance, isA

Name

valtype::creditcard::amex - Validation for AMEX creditcard number

Synopsis

package require Tcl 8.5 9
package require snit 2
package require valtype::common
package require valtype::luhn
package require valtype::creditcard::amex ?**1.1**?

valtype::creditcard::amex validate *value*
valtype::creditcard::amex checkdigit *value*

Description

This package implements a snit validation type for an AMEX creditcard number.

A validation type is an object that can be used to validate Tcl values of a particular kind. For example, **snit::integer**, a validation type defined by the **snit** package is used to validate that a Tcl value is an integer.

Every validation type has a **validate** method which is used to do the validation. This method must take a single argument, the value to be validated; further, it must do nothing if the value is valid, but throw an error if the value is invalid:

```
    valtype::creditcard::amex validate .... ;# Does nothing
    valtype::creditcard::amex validate .... ;# Throws an error (bad
    American Expresss creditcard number).
```

The **validate** method will always return the validated value on success, and throw the **-errorcode** INVALID on error, possibly with additional elements which provide more details about the problem.

API

The API provided by this package satisfies the specification of snit validation types found in the documentation of *Snit's Not Incr Tcl*.

valtype::creditcard::amex validate *value*
: This method validates the *value* and returns it, possibly in a canonical form, if it passes. If the value does not pass the validation an error is thrown.

valtype::creditcard::amex checkdigit *value*
: This method computes a check digit for the *value*. Before doing so it is validated, except for a checkdigit. If the value does not pass the validation no check digit is calculated and an error is thrown instead.

Error codes

As said in the package description, the errors thrown by the commands of this package in response to input validation failures use the **-errorcode** INVALID to distinguish themselves from package internal errors.

To provide more detailed information about why the validation failed the **-errorCode** goes actually beyond that. First, it will contain a code detailing the type itself. Here this is **CREDITCARD AMEX**. This is then followed by values detailing the reason for the failure. The full set of **-errorCode**s which can be thrown by this package are:

INVALID CREDITCARD AMEX CHARACTER
: The input value contained one or more bad characters, i.e. characters which must not occur in the input for it to be an AMEX creditcard number.

INVALID CREDITCARD AMEX CHECK-DIGIT
: The check digit of the input value is wrong. This usually signals a data-entry error, with digits transposed, forgotten, etc. Of course, th input may be an outright fake too.

INVALID CREDITCARD AMEX LENGTH

The input value is of the wrong length to be an AMEX creditcard number.

INVALID CREDITCARD AMEX PREFIX

The input value does not start with the magic value(s) required for it to be an AMEX creditcard number.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *valtype* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Validation, Type checking

Copyright


```
Copyright (c) 2011 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


cc_discover

Keywords: cc_discover, Checking, Discover, Testing, Type checking, Validation, Value checking, bank, card for credit, credit card, finance, isA

Name

valtype::creditcard::discover - Validation for Discover creditcard number

Synopsis

package require Tcl 8.5 9
package require snit 2
package require valtype::common
package require valtype::luhn
package require valtype::creditcard::discover ?**1.1**?
valtype::creditcard::discover validate *value*
valtype::creditcard::discover checkdigit *value*

Description

This package implements a snit validation type for a Discover creditcard number.

A validation type is an object that can be used to validate Tcl values of a particular kind. For example, **snit::integer**, a validation type defined by the **snit** package is used to validate that a Tcl value is an integer.

Every validation type has a **validate** method which is used to do the validation. This method must take a single argument, the value to be validated; further, it must do nothing if the value is valid, but throw an error if the value is invalid:

```
    valtype::creditcard::discover validate .... ;# Does nothing
    valtype::creditcard::discover validate .... ;# Throws an error
    (bad Discover creditcard number).
```

The **validate** method will always return the validated value on success, and throw the **-errorcode** INVALID on error, possibly with additional elements which provide more details about the problem.

API

The API provided by this package satisfies the specification of snit validation types found in the documentation of *Snit's Not Incr Tcl*.

valtype::creditcard::discover validate *value*
: This method validates the *value* and returns it, possibly in a canonical form, if it passes. If the value does not pass the validation an error is thrown.

valtype::creditcard::discover checkdigit *value*
: This method computes a check digit for the *value*. Before doing so it is validated, except for a checkdigit. If the value does not pass the validation no check digit is calculated and an error is thrown instead.

Error codes

As said in the package description, the errors thrown by the commands of this package in response to input validation failures use the **-errorcode** INVALID to distinguish themselves from package internal errors.

To provide more detailed information about why the validation failed the **-errorCode** goes actually beyond that. First, it will contain a code detailing the type itself. Here this is **CREDITCARD DISCOVER**. This is then followed by values detailing the reason for the failure. The full set of **-errorCode**s which can be thrown by this package are:

INVALID CREDITCARD DISCOVER CHARACTER
: The input value contained one or more bad characters, i.e. characters which must not occur in the input for it to be a Discover creditcard number.

INVALID CREDITCARD DISCOVER CHECK-DIGIT
: The check digit of the input value is wrong. This usually signals a data-entry error, with digits transposed, forgotten, etc. Of course, th input may be an outright fake too.

INVALID CREDITCARD DISCOVER LENGTH
: The input value is of the wrong length to be a Discover creditcard number.

INVALID CREDITCARD DISCOVER PREFIX
: The input value does not start with the magic value(s) required for it to be a Discover creditcard number.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *valtype* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be

made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Validation, Type checking

Copyright
```
Copyright (c) 2011 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

cc_mastercard
Keywords: cc_mastercard, Checking, MasterCard, Testing, Type checking, Validation, Value checking, bank, card for credit, credit card, finance, isA

Name
valtype::creditcard::mastercard - Validation for Mastercard creditcard number

Synopsis
package require Tcl 8.5 9
package require snit 2
package require valtype::common
package require valtype::luhn
package require valtype::creditcard::mastercard ?**1.1**?
valtype::creditcard::mastercard validate *value*
valtype::creditcard::mastercard checkdigit *value*

Description
This package implements a snit validation type for a Mastercard creditcard number.

A validation type is an object that can be used to validate Tcl values of a particular kind. For example, **snit::integer**, a validation type defined by the **snit** package is used to validate that a Tcl value is an integer.

Every validation type has a **validate** method which is used to do the validation. This method must take a single argument, the value to be validated; further, it must do nothing if the value is valid, but throw an error if the value is invalid:

```
    valtype::creditcard::mastercard validate .... ;# Does nothing
    valtype::creditcard::mastercard validate .... ;# Throws an error
    (bad Mastercard creditcard number).
```

The **validate** method will always return the validated value on success, and throw the **-errorcode** INVALID on error, possibly with additional elements which provide more details about the problem.

API
The API provided by this package satisfies the specification of snit validation types found in the documentation of *Snit's Not Incr Tcl*.

valtype::creditcard::mastercard validate *value*

This method validates the *value* and returns it, possibly in a canonical form, if it passes. If the value does not pass the validation an error is thrown.

valtype::creditcard::mastercard checkdigit *value*

This method computes a check digit for the *value*. Before doing so it is validated, except for a checkdigit. If the value does not pass the validation no check digit is calculated and an error is thrown instead.

Error codes

As said in the package description, the errors thrown by the commands of this package in response to input validation failures use the **-errorcode** INVALID to distinguish themselves from package internal errors.

To provide more detailed information about why the validation failed the **-errorCode** goes actually beyond that. First, it will contain a code detailing the type itself. Here this is **CREDITCARD MASTERCARD**. This is then followed by values detailing the reason for the failure. The full set of **-errorCode**s which can be thrown by this package are:

INVALID CREDITCARD MASTERCARD CHARACTER

The input value contained one or more bad characters, i.e. characters which must not occur in the input for it to be a Mastercard creditcard number.

INVALID CREDITCARD MASTERCARD CHECK-DIGIT

The check digit of the input value is wrong. This usually signals a data-entry error, with digits transposed, forgotten, etc. Of course, th input may be an outright fake too.

INVALID CREDITCARD MASTERCARD LENGTH

The input value is of the wrong length to be a Mastercard creditcard number.

INVALID CREDITCARD MASTERCARD PREFIX

The input value does not start with the magic value(s) required for it to be a Mastercard creditcard number.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *valtype* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Validation, Type checking

Copyright


```
Copyright (c) 2011 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


cc_visa

Keywords: cc_visa, Checking, Testing, Type checking, VISA, Validation, Value checking, bank, card for credit, credit card, finance, isA

Name

valtype::creditcard::visa - Validation for VISA creditcard number

Synopsis

package require Tcl 8.5 9
package require snit 2
package require valtype::common
package require valtype::luhn
package require valtype::creditcard::visa ?**1.1**?
valtype::creditcard::visa validate *value*
valtype::creditcard::visa checkdigit *value*

Description

This package implements a snit validation type for a VISA creditcard number.

A validation type is an object that can be used to validate Tcl values of a particular kind. For example, **snit::integer**, a validation type defined by the **snit** package is used to validate that a Tcl value is an integer.

Every validation type has a **validate** method which is used to do the validation. This method must take a single argument, the value to be validated; further, it must do nothing if the value is valid, but throw an error if the value is invalid:

```
    valtype::creditcard::visa validate .... ;# Does nothing
    valtype::creditcard::visa validate .... ;# Throws an error (bad
    VISA creditcard number).
```

The **validate** method will always return the validated value on success, and throw the **-errorcode** INVALID on error, possibly with additional elements which provide more details about the problem.

API

The API provided by this package satisfies the specification of snit validation types found in the documentation of *Snit's Not Incr Tcl*.

valtype::creditcard::visa validate *value*
: This method validates the *value* and returns it, possibly in a canonical form, if it passes. If the value does not pass the validation an error is thrown.

valtype::creditcard::visa checkdigit *value*
: This method computes a check digit for the *value*. Before doing so it is validated, except for a checkdigit. If the value does not pass the validation no check digit is calculated and an error is thrown instead.

Error codes

As said in the package description, the errors thrown by the commands of this package in response to input validation failures use the **-errorcode** INVALID to distinguish themselves from package internal errors.

To provide more detailed information about why the validation failed the **-errorCode** goes actually beyond that. First, it will contain a code detailing the type itself. Here this is **CREDITCARD VISA**. This is then followed by values detailing the reason for the failure. The full set of **-errorCode**s which can be thrown by this package are:

INVALID CREDITCARD VISA CHARACTER
: The input value contained one or more bad characters, i.e. characters which must not occur in the input for it to be a VISA creditcard number.

INVALID CREDITCARD VISA CHECK-DIGIT
: The check digit of the input value is wrong. This usually signals a data-entry error, with digits transposed, forgotten, etc. Of course, th input may be an outright fake too.

INVALID CREDITCARD VISA LENGTH
: The input value is of the wrong length to be a VISA creditcard number.

INVALID CREDITCARD VISA PREFIX
: The input value does not start with the magic value(s) required for it to be a VISA creditcard number.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *valtype* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Validation, Type checking

Copyright


```
Copyright (c) 2011 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


ean13

Keywords: ean13, Checking, EAN, European Article Number, International Article Number, Testing, Type checking, Validation, Value checking, isA

Name

valtype::gs1::ean13 - Validation for EAN13

Synopsis

package require Tcl 8.5 9
package require snit 2
package require valtype::common
package require valtype::gs1::ean13 ?**1.1**?
valtype::gs1::ean13 validate *value*

valtype::gs1::ean13 checkdigit *value*

Description

This package implements a snit validation type for an EAN13.

A validation type is an object that can be used to validate Tcl values of a particular kind. For example, **snit::integer**, a validation type defined by the **snit** package is used to validate that a Tcl value is an integer.

Every validation type has a **validate** method which is used to do the validation. This method must take a single argument, the value to be validated; further, it must do nothing if the value is valid, but throw an error if the value is invalid:

```
    valtype::gs1::ean13 validate .... ;# Does nothing
    valtype::gs1::ean13 validate .... ;# Throws an error (bad EAN13).
```

The **validate** method will always return the validated value on success, and throw the **-errorcode** INVALID on error, possibly with additional elements which provide more details about the problem.

API

The API provided by this package satisfies the specification of snit validation types found in the documentation of *Snit's Not Incr Tcl*.

valtype::gs1::ean13 validate *value*
: This method validates the *value* and returns it, possibly in a canonical form, if it passes. If the value does not pass the validation an error is thrown.

valtype::gs1::ean13 checkdigit *value*
: This method computes a check digit for the *value*. Before doing so it is validated, except for a checkdigit. If the value does not pass the validation no check digit is calculated and an error is thrown instead.

Error codes

As said in the package description, the errors thrown by the commands of this package in response to input validation failures use the **-errorcode** INVALID to distinguish themselves from package internal errors.

To provide more detailed information about why the validation failed the **-errorCode** goes actually beyond that. First, it will contain a code detailing the type itself. Here this is **EAN13**. This is then followed by values detailing the reason for the failure. The full set of **-errorCode**s which can be thrown by this package are:

INVALID EAN13 CHARACTER
: The input value contained one or more bad characters, i.e. characters which must not occur in the input for it to be an EAN13.

INVALID EAN13 CHECK-DIGIT
: The check digit of the input value is wrong. This usually signals a data-entry error, with digits transposed, forgotten, etc. Of course, th input may be an outright fake too.

INVALID EAN13 LENGTH
: The input value is of the wrong length to be an EAN13.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *valtype* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Validation, Type checking

Copyright

```
Copyright (c) 2011 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

iban

Keywords: iban, Checking, International Bank Account Number, Testing, Type checking, Validation, Value checking, bank, finance, isA

Name

valtype::iban - Validation for IBAN

Synopsis

package require Tcl 8.5 9
package require snit 2
package require valtype::common
package require valtype::iban ?**1.8**?
valtype::iban validate *value*
valtype::iban checkdigit *value*

Description

This package implements a snit validation type for an IBAN.

A validation type is an object that can be used to validate Tcl values of a particular kind. For example, **snit::integer**, a validation type defined by the **snit** package is used to validate that a Tcl value is an integer.

Every validation type has a **validate** method which is used to do the validation. This method must take a single argument, the value to be validated; further, it must do nothing if the value is valid, but throw an error if the value is invalid:

```
    valtype::iban validate .... ;# Does nothing
    valtype::iban validate .... ;# Throws an error (bad International
    Bank Account Number).
```

The **validate** method will always return the validated value on success, and throw the **-errorcode** INVALID on error, possibly with additional elements which provide more details

about the problem.

API

The API provided by this package satisfies the specification of snit validation types found in the documentation of *Snit's Not Incr Tcl*.

valtype::iban validate *value*
: This method validates the *value* and returns it, possibly in a canonical form, if it passes. If the value does not pass the validation an error is thrown.

valtype::iban checkdigit *value*
: This method computes a check digit for the *value*. Before doing so it is validated, except for a checkdigit. If the value does not pass the validation no check digit is calculated and an error is thrown instead.

Error codes

As said in the package description, the errors thrown by the commands of this package in response to input validation failures use the **-errorcode** INVALID to distinguish themselves from package internal errors.

To provide more detailed information about why the validation failed the **-errorCode** goes actually beyond that. First, it will contain a code detailing the type itself. Here this is **IBAN**. This is then followed by values detailing the reason for the failure. The full set of **-errorCode**s which can be thrown by this package are:

INVALID IBAN CHARACTER
: The input value contained one or more bad characters, i.e. characters which must not occur in the input for it to be an IBAN.

INVALID IBAN CHECK-DIGIT
: The check digit of the input value is wrong. This usually signals a data-entry error, with digits transposed, forgotten, etc. Of course, th input may be an outright fake too.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *valtype* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Validation, Type checking

Copyright


```
Copyright (c) 2011 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

imei

Keywords: imei, Checking, International Mobile Equipment Identity, Testing, Type checking, Validation, Value checking, cell-phone, isA, mobile phone, phone

Name

valtype::imei - Validation for IMEI

Synopsis

package require Tcl 8.5 9
package require snit 2
package require valtype::common
package require valtype::luhn
package require valtype::imei ?**1.1**?
valtype::imei validate *value*
valtype::imei checkdigit *value*

Description

This package implements a snit validation type for an IMEI.

A validation type is an object that can be used to validate Tcl values of a particular kind. For example, **snit::integer**, a validation type defined by the **snit** package is used to validate that a Tcl value is an integer.

Every validation type has a **validate** method which is used to do the validation. This method must take a single argument, the value to be validated; further, it must do nothing if the value is valid, but throw an error if the value is invalid:

```
    valtype::imei validate .... ;# Does nothing
    valtype::imei validate .... ;# Throws an error (bad International
    Mobile Equipment Identity (IMEI) number).
```

The **validate** method will always return the validated value on success, and throw the **-errorcode** INVALID on error, possibly with additional elements which provide more details about the problem.

API

The API provided by this package satisfies the specification of snit validation types found in the documentation of *Snit's Not Incr Tcl*.

valtype::imei validate *value*
: This method validates the *value* and returns it, possibly in a canonical form, if it passes. If the value does not pass the validation an error is thrown.

valtype::imei checkdigit *value*
: This method computes a check digit for the *value*. Before doing so it is validated, except for a checkdigit. If the value does not pass the validation no check digit is calculated and an error is thrown instead.

Error codes

As said in the package description, the errors thrown by the commands of this package in

response to input validation failures use the **-errorcode** INVALID to distinguish themselves from package internal errors.

To provide more detailed information about why the validation failed the **-errorCode** goes actually beyond that. First, it will contain a code detailing the type itself. Here this is **IMEI**. This is then followed by values detailing the reason for the failure. The full set of **-errorCode**s which can be thrown by this package are:

INVALID IMEI CHARACTER
: The input value contained one or more bad characters, i.e. characters which must not occur in the input for it to be an IMEI.

INVALID IMEI CHECK-DIGIT
: The check digit of the input value is wrong. This usually signals a data-entry error, with digits transposed, forgotten, etc. Of course, th input may be an outright fake too.

INVALID IMEI LENGTH
: The input value is of the wrong length to be an IMEI.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *valtype* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Validation, Type checking

Copyright


```
Copyright (c) 2011 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


isbn

Keywords: isbn, Book Number, Checking, EAN, EAN13, European Article Number, International Article Number, International Standard Book Number, Testing, Type checking, Validation, Value checking, isA

Name

valtype::isbn - Validation for ISBN

Synopsis

package require Tcl 8.5 9
package require snit 2
package require valtype::common
package require valtype::isbn ?**1.1**?
valtype::isbn validate *value*

valtype::isbn checkdigit *value*
valtype::isbn 13of *value*

Description

This package implements a snit validation type for an ISBN.

A validation type is an object that can be used to validate Tcl values of a particular kind. For example, **snit::integer**, a validation type defined by the **snit** package is used to validate that a Tcl value is an integer.

Every validation type has a **validate** method which is used to do the validation. This method must take a single argument, the value to be validated; further, it must do nothing if the value is valid, but throw an error if the value is invalid:

```
    valtype::isbn validate .... ;# Does nothing
    valtype::isbn validate .... ;# Throws an error (bad ISBN).
```

The **validate** method will always return the validated value on success, and throw the **-errorcode** INVALID on error, possibly with additional elements which provide more details about the problem.

API

The API provided by this package satisfies the specification of snit validation types found in the documentation of *Snit's Not Incr Tcl*.

valtype::isbn validate *value*
: This method validates the *value* and returns it, possibly in a canonical form, if it passes. If the value does not pass the validation an error is thrown.

valtype::isbn checkdigit *value*
: This method computes a check digit for the *value*. Before doing so it is validated, except for a checkdigit. If the value does not pass the validation no check digit is calculated and an error is thrown instead.

valtype::isbn 13of *value*
: This method expects an old-style 10-digit ISBN and returns the canonical modern 13-digit ISBN. This is used by **validate** to canonicalize the input, so that all parts of the system after the validation can expect to work with modern 13-digit ISBNs.

Error codes

As said in the package description, the errors thrown by the commands of this package in response to input validation failures use the **-errorcode** INVALID to distinguish themselves from package internal errors.

To provide more detailed information about why the validation failed the **-errorCode** goes actually beyond that. First, it will contain a code detailing the type itself. Here this is **ISBN**. This is then followed by values detailing the reason for the failure. The full set of **-errorCode**s which can be thrown by this package are:

INVALID ISBN CHARACTER
: The input value contained one or more bad characters, i.e. characters which must not occur in the input for it to be an ISBN.

INVALID ISBN CHECK-DIGIT

The check digit of the input value is wrong. This usually signals a data-entry error, with digits transposed, forgotten, etc. Of course, th input may be an outright fake too.

INVALID ISBN LENGTH
The input value is of the wrong length to be an ISBN.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *valtype* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Validation, Type checking

Copyright


```
Copyright (c) 2011 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


luhn

Keywords: luhn, Checking, Testing, Type checking, Validation, Value checking, isA

Name

valtype::luhn - Validation for plain number with a LUHN checkdigit

Synopsis

package require Tcl 8.5 9
package require snit 2
package require valtype::common
package require valtype::luhn ?**1.1**?
valtype::luhn validate *value*
valtype::luhn checkdigit *value*

Description

This package implements a snit validation type for a plain number with a LUHN checkdigit.

A validation type is an object that can be used to validate Tcl values of a particular kind. For example, **snit::integer**, a validation type defined by the **snit** package is used to validate that a Tcl value is an integer.

Every validation type has a **validate** method which is used to do the validation. This method must take a single argument, the value to be validated; further, it must do nothing if the value is valid, but throw an error if the value is invalid:

```
    valtype::luhn validate .... ;# Does nothing
    valtype::luhn validate .... ;# Throws an error (bad luhn
    checkdigit).
```

The **validate** method will always return the validated value on success, and throw the **-errorcode** INVALID on error, possibly with additional elements which provide more details about the problem.

API

The API provided by this package satisfies the specification of snit validation types found in the documentation of *Snit's Not Incr Tcl*.

valtype::luhn validate *value*
: This method validates the *value* and returns it, possibly in a canonical form, if it passes. If the value does not pass the validation an error is thrown.

valtype::luhn checkdigit *value*
: This method computes a check digit for the *value*. Before doing so it is validated, except for a checkdigit. If the value does not pass the validation no check digit is calculated and an error is thrown instead.

Error codes

As said in the package description, the errors thrown by the commands of this package in response to input validation failures use the **-errorcode** INVALID to distinguish themselves from package internal errors.

To provide more detailed information about why the validation failed the **-errorCode** goes actually beyond that. First, it will contain a code detailing the type itself. Here this is **LUHN**. This is then followed by values detailing the reason for the failure. The full set of **-errorCode**s which can be thrown by this package are:

INVALID LUHN CHARACTER
: The input value contained one or more bad characters, i.e. characters which must not occur in the input for it to be a plain number with a LUHN checkdigit.

INVALID LUHN CHECK-DIGIT
: The check digit of the input value is wrong. This usually signals a data-entry error, with digits transposed, forgotten, etc. Of course, th input may be an outright fake too.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *valtype* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Validation, Type checking

Copyright


```
Copyright (c) 2011 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


luhn5

Keywords: luhn5, Checking, Testing, Type checking, Validation, Value checking, isA, luhn, luhn-5

Name

valtype::luhn5 - Validation for plain number with a LUHN5 checkdigit

Synopsis

package require Tcl 8.5 9
package require snit 2
package require valtype::common
package require valtype::luhn5 ?**1.1**?
valtype::luhn5 validate *value*
valtype::luhn5 checkdigit *value*

Description

This package implements a snit validation type for a plain number with a LUHN5 checkdigit.

A validation type is an object that can be used to validate Tcl values of a particular kind. For example, **snit::integer**, a validation type defined by the **snit** package is used to validate that a Tcl value is an integer.

Every validation type has a **validate** method which is used to do the validation. This method must take a single argument, the value to be validated; further, it must do nothing if the value is valid, but throw an error if the value is invalid:

```
    valtype::luhn5 validate .... ;# Does nothing
    valtype::luhn5 validate .... ;# Throws an error (bad luhn5
    checkdigit).
```

The **validate** method will always return the validated value on success, and throw the **-errorcode** INVALID on error, possibly with additional elements which provide more details about the problem.

API

The API provided by this package satisfies the specification of snit validation types found in the documentation of *Snit's Not Incr Tcl*.

valtype::luhn5 validate *value*
: This method validates the *value* and returns it, possibly in a canonical form, if it passes. If the value does not pass the validation an error is thrown.

valtype::luhn5 checkdigit *value*
: This method computes a check digit for the *value*. Before doing so it is validated, except for a checkdigit. If the value does not pass the validation no check digit is calculated and an error is thrown instead.

Error codes

As said in the package description, the errors thrown by the commands of this package in response to input validation failures use the **-errorcode** INVALID to distinguish themselves from package internal errors.

To provide more detailed information about why the validation failed the **-errorCode** goes actually beyond that. First, it will contain a code detailing the type itself. Here this is **LUHN5**. This is then followed by values detailing the reason for the failure. The full set of **-errorCode**s which can be thrown by this package are:

INVALID LUHN5 CHARACTER
: The input value contained one or more bad characters, i.e. characters which must not occur in the input for it to be a plain number with a LUHN5 checkdigit.

INVALID LUHN5 CHECK-DIGIT
: The check digit of the input value is wrong. This usually signals a data-entry error, with digits transposed, forgotten, etc. Of course, th input may be an outright fake too.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *valtype* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Validation, Type checking

Copyright


```
Copyright (c) 2011 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


usnpi

Keywords: usnpi, Checking, NPI, National Provider Identifier, Testing, Type checking, US-NPI, Validation, Value checking, isA, medicare

Name

valtype::usnpi - Validation for USNPI

Synopsis

package require Tcl 8.5 9
package require snit 2
package require valtype::common
package require valtype::luhn
package require valtype::usnpi ?**1.1**?
valtype::usnpi validate *value*

valtype::usnpi checkdigit *value*

Description

This package implements a snit validation type for an USNPI.

A validation type is an object that can be used to validate Tcl values of a particular kind. For example, **snit::integer**, a validation type defined by the **snit** package is used to validate that a Tcl value is an integer.

Every validation type has a **validate** method which is used to do the validation. This method must take a single argument, the value to be validated; further, it must do nothing if the value is valid, but throw an error if the value is invalid:

```
    valtype::usnpi validate .... ;# Does nothing
    valtype::usnpi validate .... ;# Throws an error (bad US National
    Provider Identifier (US-NPI) number).
```

The **validate** method will always return the validated value on success, and throw the **-errorcode** INVALID on error, possibly with additional elements which provide more details about the problem.

API

The API provided by this package satisfies the specification of snit validation types found in the documentation of *Snit's Not Incr Tcl*.

valtype::usnpi validate *value*
: This method validates the *value* and returns it, possibly in a canonical form, if it passes. If the value does not pass the validation an error is thrown.

valtype::usnpi checkdigit *value*
: This method computes a check digit for the *value*. Before doing so it is validated, except for a checkdigit. If the value does not pass the validation no check digit is calculated and an error is thrown instead.

Error codes

As said in the package description, the errors thrown by the commands of this package in response to input validation failures use the **-errorcode** INVALID to distinguish themselves from package internal errors.

To provide more detailed information about why the validation failed the **-errorCode** goes actually beyond that. First, it will contain a code detailing the type itself. Here this is **USNPI**. This is then followed by values detailing the reason for the failure. The full set of **-errorCode**s which can be thrown by this package are:

INVALID USNPI CHARACTER
: The input value contained one or more bad characters, i.e. characters which must not occur in the input for it to be an USNPI.

INVALID USNPI CHECK-DIGIT
: The check digit of the input value is wrong. This usually signals a data-entry error, with digits transposed, forgotten, etc. Of course, th input may be an outright fake too.

INVALID USNPI LENGTH
: The input value is of the wrong length to be an USNPI.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *valtype* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Validation, Type checking

Copyright


```
Copyright (c) 2011 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


valtype_common

Keywords: valtype_common, Checking, Testing, Type checking, Validation, Value checking, isA

Name

valtype::common - Validation, common code

Synopsis

package require Tcl 8.5 9
package require valtype::common ?**1.1**?
valtype::common::reject *code text*
valtype::common::badchar *code* ?*text*?
valtype::common::badcheck *code* ?*text*?
valtype::common::badlength *code lengths* ?*text*?
valtype::common::badprefix *code prefixes* ?*text*?

Description

This package implements a number of common commands used by the validation types in this module. These commands essentially encapsulate the throwing of validation errors, ensuring that a proper **-errorcode** is used. See section Error codes.

API

valtype::common::reject *code text*

The core command of this package it throws an **INVALID** error with the specified *text*. The first argument is a list of codes extending the **INVALID** with detail information.

valtype::common::badchar *code* ?*text*?

This command throws an **INVALID CHAR** error with the specified *text*. The first argument is a list of codes providing details. These are inserted between the codes **INVALID** and **CHARACTER**.

valtype::common::badcheck *code* ?*text*?

This command throws an **INVALID CHECK-DIGIT** error with the specified *text*, if any, extended by the standard text "the check digit is incorrect". The first argument is a list of codes providing details. These are inserted between the codes **INVALID** and **CHECK_DIGIT**.

valtype::common::badlength *code lengths* ?*text*?

This command throws an **INVALID LENGTH** error with the specified *text*, if any, extended by the standard text "incorrect length, expected ... character(s)". The first argument is a list of codes providing details. These are inserted between the codes **INVALID** and **LENGTH**. The argument *lengths* is a list of the input lengths which had been expected, i.e. these are the valid lengths.

valtype::common::badprefix *code prefixes* ?*text*?

This command throws an **INVALID PREFIX** error with the specified *text*, if any, extended by the standard text "incorrect prefix, expected ...". The first argument is a list of codes providing details. These are inserted between the codes **INVALID** and **PREFIX**. The argument *prefixes* is a list of the input prefixes which had been expected, i.e. these are the valid prefixes.

Error codes

The errors thrown by the commands of this package all use the **-errorcode INVALID** to distinguish the input validation failures they represent from package internal errors.

To provide more detailed information about why the validation failed the **-errorCode** goes actually beyond that. First, it will contain a code detailing the type itself. This is supplied by the caller. This is then followed by values detailing the reason for the failure. The full set of **-errorCode**s which can be thrown by this package are shown below, with **<>** a placeholder for both the caller-supplied type-information, the type description.

INVALID <> CHARACTER

The input value contained one or more bad characters, i.e. characters which must not occur in the input for it to be a **<>**.

INVALID <> CHECK-DIGIT

The check digit of the input value is wrong. This usually signals a data-entry error, with digits transposed, forgotten, etc. Of course, th input may be an outright fake too.

INVALID <> LENGTH

The input value is of the wrong length to be a **<>**.

INVALID <> PREFIX

The input value does not start with the magic value(s) required for it to be a **<>**.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *valtype* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Validation, Type checking

Copyright


```
Copyright (c) 2011 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


verhoeff

Keywords: verhoeff, Checking, Testing, Type checking, Validation, Value checking, isA

Name

valtype::verhoeff - Validation for plain number with a VERHOEFF checkdigit

Synopsis

package require Tcl 8.5 9
package require snit 2
package require valtype::common
package require valtype::verhoeff ?**1.1**?
valtype::verhoeff validate *value*
valtype::verhoeff checkdigit *value*

Description

This package implements a snit validation type for a plain number with a VERHOEFF checkdigit.

A validation type is an object that can be used to validate Tcl values of a particular kind. For example, **snit::integer**, a validation type defined by the **snit** package is used to validate that a Tcl value is an integer.

Every validation type has a **validate** method which is used to do the validation. This method must take a single argument, the value to be validated; further, it must do nothing if the value is valid, but throw an error if the value is invalid:

```
    valtype::verhoeff validate .... ;# Does nothing
    valtype::verhoeff validate .... ;# Throws an error (bad verhoeff
    checkdigit).
```

The **validate** method will always return the validated value on success, and throw the **-errorcode** INVALID on error, possibly with additional elements which provide more details about the problem.

API

The API provided by this package satisfies the specification of snit validation types found in the documentation of *Snit's Not Incr Tcl*.

valtype::verhoeff validate *value*
: This method validates the *value* and returns it, possibly in a canonical form, if it passes. If the value does not pass the validation an error is thrown.

valtype::verhoeff checkdigit *value*
: This method computes a check digit for the *value*. Before doing so it is validated, except for a checkdigit. If the value does not pass the validation no check digit is calculated and an error is thrown instead.

Error codes

As said in the package description, the errors thrown by the commands of this package in response to input validation failures use the **-errorcode** INVALID to distinguish themselves from package internal errors.

To provide more detailed information about why the validation failed the **-errorCode** goes actually beyond that. First, it will contain a code detailing the type itself. Here this is **VERHOEFF**. This is then followed by values detailing the reason for the failure. The full set of **-errorCode**s which can be thrown by this package are:

INVALID VERHOEFF CHARACTER
: The input value contained one or more bad characters, i.e. characters which must not occur in the input for it to be a plain number with a VERHOEFF checkdigit.

INVALID VERHOEFF CHECK-DIGIT
: The check digit of the input value is wrong. This usually signals a data-entry error, with digits transposed, forgotten, etc. Of course, th input may be an outright fake too.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *valtype* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Validation, Type checking

Copyright


```
Copyright (c) 2011 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


virtchannel_base

cat

Keywords: cat, concatenation channel, reflected channel, tip 219, virtual channel

Name

tcl::chan::cat - Concatenation channel

Synopsis

package require Tcl 8.5 9
package require TclOO
package require tcl::chan::core ?**1.1**?
package require tcl::chan::cat ?**1.0.4**?
::tcl::chan::cat *chan...*

Description

The **tcl::chan::cat** package provides a command creating concatenation channels. These are non-seekable channels owning a list of subordinate channels whose contents they return in order, until all are exhausted. In this manner the channel is the concatentation of the contents of all the sub-ordinate channels.

Note that the created channels take ownership of the channels they were constructed with. Whenever they have exhausted one of their channel it will be closed. Similarly, closing the cat channel will close all the sub-ordinates it still has.

The internal **TclOO** class implementing the channel handler is a sub-class of the **tcl::chan::core** framework.

Event handling is delegated to the currently active sub-channel.

API

::tcl::chan::cat *chan*...

This command creates the concatenation channel using all the provided channels, and returns its handle.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *virtchannel* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Channels

Copyright


```
Copyright (c) 2011 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


facade

Keywords: facade, concatenation channel, reflected channel, tip 219, virtual channel

Name

tcl::chan::facade - Facade channel

Synopsis

package require Tcl 8.5 9
package require TclOO
package require logger
package require tcl::chan::core ?**1.1**?
package require tcl::chan::facade ?**1.1**?

::tcl::chan::facade *chan*

Description

The **tcl::chan::facade** package provides a command creating facades to other channels. These are channels which own a single subordinate channel and delegate all operations to.

The main use for facades is the debugging of actions on a channel. While most of the information could be tracked by a virtual channel transformation it does not have access to the event-related operation, and furthermore they are only available in Tcl 8.6.

Therefore this channel, usable with Tcl 8.5, and having access to everything going on for a channel.

The intercepted actions on channel are logged through package **logger**.

Beyond that facades provide the following additional channel configuration options:

-self
: The TclOO object handling the facade.

-fd
: The handle of the subordinate, i.e. wrapped channel.

-used
: The last time the wrapped channel was read from or written to by the facade, as per **clock milliseconds**. A value of **0** indicates that the subordinate channel was not accessed at all, yet.

-created
: The time the facade was created, as per **clock milliseconds**.

-user
: A free-form value identifying the user of the facade and its wrapped channel.

Of these only option **-user** is writable.

API

::tcl::chan::facade *chan*
: This command creates the facade channel around the provided channel *chan*, and returns its handle.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *virtchannel* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Channels

Copyright

```
Copyright (c) 2011 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

halfpipe

Keywords: halfpipe, callbacks, fifo, in-memory channel, reflected channel, tip 219, virtual channel

Name

tcl::chan::halfpipe - In-memory channel, half of a fifo2

Synopsis

package require Tcl 8.5 9
package require TclOO
package require tcl::chan::events ?**1.1**?
package require tcl::chan::halfpipe ?**1.0.2**?
::tcl::chan::halfpipe ?*-option value...*?
objectCmd **put** *bytes*

Description

The **tcl::chan::halfpipe** package provides a command creating one half of a **tcl::chan::fifo2** pair. Writing into such a channel invokes a set of callbacks which then handle the data. This is similar to a channel handler, except having a much simpler API.

The internal **TclOO** class implementing the channel handler is a sub-class of the **tcl::chan::events** framework.

API

::tcl::chan::halfpipe ?-option *value*...?

This command creates a halfpipe channel and configures it with the callbacks to run when the channel is closed, data was written to it, or ran empty. See the section Options for the list of options and associated semantics. The result of the command is a list containing two elements, the handle of the new channel, and the object command of the channel handler, in this order. The latter is supplied to the caller to provide her with access to the **put** method for adding data to the channel.

Two halfpipes with a bit of glue logic in the callbacks make for one **tcl::chan::fifo2**.

objectCmd* put *bytes

This method of the channel handler object puts the data *bytes* into the channel so that it can be read from it.

Options

-close-command cmdprefix

This callback is invoked when the channel is closed. A single argument is supplied, the handle of the channel being closed. The result of the callback is ignored.

-write-command cmdprefix

This callback is invoked when data is written to the channel. Two arguments are supplied, the handle of the channel written to, and the data written. The result of the callback is ignored.

-empty-command cmdprefix
This callback is invoked when the channel has run out of data to read. A single argument is supplied, the handle of the channel.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *virtchannel* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Channels

Copyright


```
Copyright (c) 2009, 2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


nullzero

Keywords: nullzero, /dev/null, /dev/zero, null, reflected channel, tip 219, virtual channel, zero

Name

tcl::chan::nullzero - Null/Zero channel combination

Synopsis

package require Tcl 8.5 9
package require TclOO
package require tcl::chan::events ?**1.1**?
package require tcl::chan::nullzero ?**1.1**?
::tcl::chan::nullzero

Description

The **tcl::chan::nullzero** package provides a command creating channels, which are a combination of null and zero devices. They immediately forget whatever is written to them, and on reading return an infinite stream of null characters.

Packages related to this are **tcl::chan::null** and **tcl::chan::zero**.

The internal **TclOO** class implementing the channel handler is a sub-class of the **tcl::chan::events** framework.

API

::tcl::chan::nullzero
This command creates a new nullzero channel and returns its handle.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *virtchannel* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Channels

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


randseed

Keywords: randseed, /dev/random, merge, random, reflected channel, seed, tip 219, virtual channel

Name

tcl::randomseed - Utilities for random channels

Synopsis

package require Tcl 8.5 9
package require TclOO
package require tcl::randomseed ?**1.1**?
::tcl::randomseed
::tcl::combine *seed1 seed2*

Description

The **tcl::randomseed** package provides a a few utility commands to help with the seeding of **tcl::chan::random** channels.

API

::tcl::randomseed

This command creates returns a list of seed integers suitable as seed argument for random channels. The numbers are derived from the process id, current time, and Tcl random number generator.

::tcl::combine *seed1 seed2*

This command takes to seed lists and combines them into a single list by XORing them elementwise, modulo 256. If the lists are not of equial length the shorter of the two is padded with 0s before merging.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you

want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *virtchannel* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Channels

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


std

Keywords: std, reflected channel, standard io, stdin, stdout, tip 219, virtual channel

Name

tcl::chan::std - Standard I/O, unification of stdin and stdout

Synopsis

package require Tcl 8.5 9
package require TclOO
package require tcl::chan::core ?**1.1**?
package require tcl::chan::std ?**1.2**?
::tcl::chan::std

Description

The **tcl::chan::std** package provides a command creating a standard channel which unifies stdin and stdout into a single read- and writable channel. The result is not seek-able, like the original standard channels.

The internal **TclOO** class implementing the channel handler is a sub-class of the **tcl::chan::core** framework.

API

::tcl::chan::std

This command creates the std channel and returns its handle.

The channel is created only once, on the first call, and all future calls simply return this handle.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *virtchannel* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Channels

Copyright

```
Copyright (c) 2011 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

tcllib_fifo

Keywords: tcllib_fifo, fifo, in-memory channel, reflected channel, tip 219, virtual channel

Name

tcl::chan::fifo - In-memory fifo channel

Synopsis

package require Tcl 8.5 9
package require TclOO
package require tcl::chan::events ?**1.1**?
package require tcl::chan::fifo ?**1.1**?
::tcl::chan::fifo

Description

The **tcl::chan::fifo** package provides a command creating channels which live purely in memory. Access is fifo-like, i.e. things are read out of the channel in the order they were written to it. This is equivalent to the fifo channels provided by the package **Memchan**, except that this is written in pure Tcl, not C. On the other hand, **Memchan** is usable with Tcl 8.4 and before, whereas this package requires Tcl 8.5 or higher, and **TclOO**.

The internal **TclOO** class implementing the channel handler is a sub-class of the **tcl::chan::events** framework.

API

::tcl::chan::fifo

This command creates a new fifo channel and returns its handle.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *virtchannel* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the

left-most button in the secondary navigation bar.

Category
Channels

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


tcllib_fifo2
Keywords: tcllib_fifo2, connected fifos, fifo, in-memory channel, inter-thread communication, reflected channel, tip 219, virtual channel

Name
tcl::chan::fifo2 - In-memory interconnected fifo channels

Synopsis
package require Tcl 8.5 9
package require TclOO
package require tcl::chan::events ?**1.1**?
package require tcl::chan::halfpipe ?**1.1**?
package require tcl::chan::fifo2 ?**1.1**?
::tcl::chan::fifo2

Description
The **tcl::chan::fifo2** package provides a command creating pairs of channels which live purely in memory and are connected to each other in a fifo manner. What is written to one half of the pair can be read from the other half, in the same order. One particular application for this is communication between threads, with one half of the pair moved to the thread to talk to. This is equivalent to the fifo2 channels provided by the package **Memchan**, except that this is written in pure Tcl, not C. On the other hand, **Memchan** is usable with Tcl 8.4 and before, whereas this package requires Tcl 8.5 or higher, and **TclOO**.

The internal **TclOO** class implementing the channel handler is a sub-class of the **tcl::chan::events** framework.

API
::tcl::chan::fifo2
This command creates a new connected pair of fifo channels and returns their handles, as a list containing two elements.

Bugs, ideas, feedback
If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *virtchannel* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be

made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Channels

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


tcllib_memchan
Keywords: tcllib_memchan, in-memory channel, reflected channel, tip 219, virtual channel

Name
tcl::chan::memchan - In-memory channel

Synopsis
package require Tcl 8.5 9
package require TclOO
package require tcl::chan::events ?**1.1**?
package require tcl::chan::memchan ?**1.0.5**?
::tcl::chan::memchan

Description
The **tcl::chan::memchan** package provides a command creating channels which live purely in memory. They provide random-access, i.e. are seekable. This is equivalent to the memchan channels provided by the package **Memchan**, except that this is written in pure Tcl, not C. On the other hand, **Memchan** is usable with Tcl 8.4 and before, whereas this package requires Tcl 8.5 or higher, and **TclOO**.

Packages related to this are **tcl::chan::string** and **tcl::chan::variable**.

The internal **TclOO** class implementing the channel handler is a sub-class of the **tcl::chan::events** framework.

API
::tcl::chan::memchan
This command creates a new memchan channel and returns its handle.

Bugs, ideas, feedback
If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *virtchannel* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Channels

Copyright

```
Copyright (c) 2009-2017 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


tcllib_null
Keywords: tcllib_null, /dev/null, null, reflected channel, tip 219, virtual channel

Name
tcl::chan::null - Null channel

Synopsis
package require Tcl 8.5 9
package require TclOO
package require tcl::chan::events ?**1.1**?
package require tcl::chan::null ?**1.1**?
::tcl::chan::null

Description
The **tcl::chan::null** package provides a command creating null channels, i.e. write-only channels which immediately forget whatever is written to them. This is equivalent to the null channels provided by the package **Memchan**, except that this is written in pure Tcl, not C. On the other hand, **Memchan** is usable with Tcl 8.4 and before, whereas this package requires Tcl 8.5 or higher, and **TclOO**.

Packages related to this are **tcl::chan::zero** and **tcl::chan::nullzero**.

The internal **TclOO** class implementing the channel handler is a sub-class of the **tcl::chan::events** framework.

API
::tcl::chan::null
This command creates a new null channel and returns its handle.

Bugs, ideas, feedback
If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *virtchannel* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Channels

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


tcllib_random

Keywords: tcllib_random, /dev/random, random, reflected channel, tip 219, virtual channel

Name

tcl::chan::random - Random channel

Synopsis

package require Tcl 8.5 9
package require TclOO
package require tcl::chan::events ?**1.1**?
package require tcl::chan::random ?**1.1**?
::tcl::chan::random *seed*

Description

The **tcl::chan::random** package provides a command creating random channels, i.e. read-only channels which return an infinite stream of pseudo-random characters upon reading. This is similar to the random channels provided by the package **Memchan**, except that this is written in pure Tcl, not C, and uses a much simpler generator as well. On the other hand, **Memchan** is usable with Tcl 8.4 and before, whereas this package requires Tcl 8.5 or higher, and TclOO.

The internal **TclOO** class implementing the channel handler is a sub-class of the **tcl::chan::events** framework.

API

::tcl::chan::random *seed*

This command creates a new random channel and returns its handle. The seed is a list of integer numbers used to initialize the internal feedback shift register of the generator.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *virtchannel* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Channels

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

tcllib_string

Keywords: tcllib_string, in-memory channel, reflected channel, tip 219, virtual channel

Name

tcl::chan::string - Read-only in-memory channel

Synopsis

package require Tcl 8.5 9
package require TclOO
package require tcl::chan::events ?**1.1**?
package require tcl::chan::string ?**1.0.4**?
::tcl::chan::string *content*

Description

The **tcl::chan::string** package provides a command creating channels which live purely in memory. They provide random-access, i.e. are seekable. In contrast to the channels created by **tcl::chan::memchan** they are read-only however, their content is provided at the time of construction and immutable afterward.

Packages related to this are **tcl::chan::memchan** and **tcl::chan::variable**.

The internal **TclOO** class implementing the channel handler is a sub-class of the **tcl::chan::events** framework.

API

::tcl::chan::string *content*

This command creates a new string channel and returns its handle. The channel provides random read-only access to the *content* string.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *virtchannel* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Channels

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

tcllib_variable

Keywords: tcllib_variable, in-memory channel, reflected channel, tip 219, virtual channel

Name

tcl::chan::variable - In-memory channel using variable for storage

Synopsis

package require Tcl 8.5 9
package require TclOO
package require tcl::chan::events ?**1.1**?
package require tcl::chan::variable ?**1.0.5**?
::tcl::chan::variable *varname*

Description

The **tcl::chan::variable** package provides a command creating channels which live purely in memory. They provide random-access, i.e. are seekable. In contrast to the channels created by **tcl::chan::memchan** the data is not hidden in the channel however, but stored in an associated variable, specified at the time of construction.

Packages related to this are **tcl::chan::memchan** and **tcl::chan::string**.

The internal **TclOO** class implementing the channel handler is a sub-class of the **tcl::chan::events** framework.

API

::tcl::chan::variable *varname*

This command creates a new variable channel and returns its handle. The content of the channel is stored in the associated namespace variable *varname*.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *virtchannel* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Channels

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


tcllib_zero

Keywords: tcllib_zero, /dev/zero, reflected channel, tip 219, virtual channel, zero

Name
tcl::chan::zero - Zero channel

Synopsis
package require Tcl 8.5 9
package require TclOO
package require tcl::chan::events ?**1.1**?
package require tcl::chan::zero ?**1.1**?
::tcl::chan::zero

Description
The **tcl::chan::zero** package provides a command creating zero channels, i.e. read-only channels which return an infinite stream of null characters upon reading. This is equivalent to the zero channels provided by the package **Memchan**, except that this is written in pure Tcl, not C. On the other hand, **Memchan** is usable with Tcl 8.4 and before, whereas this package requires Tcl 8.5 or higher, and TclOO.

Packages related to this are **tcl::chan::null** and **tcl::chan::nullzero**.

The internal **TclOO** class implementing the channel handler is a sub-class of the **tcl::chan::events** framework.

API
::tcl::chan::zero
This command creates a new zero channel and returns its handle.

Bugs, ideas, feedback
If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *virtchannel* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Channels

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


textwindow
Keywords: textwindow, Tk, reflected channel, text widget, tip 219, virtual channel

Name
tcl::chan::textwindow - Textwindow channel

Synopsis

package require Tcl 8.5 9
package require TclOO
package require tcl::chan::events ?**1.1**?
package require tcl::chan::textwindow ?**1.1**?
::tcl::chan::textwindow *widget*

Description

The **tcl::chan::textwindow** package provides a command creating write-only channels connected to text widgets. Anything written to the channel is printed into the associated widget.

The internal **TclOO** class implementing the channel handler is a sub-class of the **tcl::chan::events** framework.

API

::tcl::chan::textwindow *widget*

This command creates a new textwindow channel and returns its handle. Data written to this channel will appear in the associated *widget*.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *virtchannel* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Channels

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


virtchannel_core

core

Keywords: core, reflected channel, tip 219, virtual channel

Name

tcl::chan::core - Basic reflected/virtual channel support

Synopsis

package require Tcl 8.5 9
package require TclOO

package require tcl::chan::core ?**1.1**?
::tcl::chan::core *objectName*
objectName **initialize** *thechannel mode*
objectName **finalize** *thechannel*
objectName **destroy**

Description

The **tcl::chan::core** package provides a **TclOO** class implementing common behaviour needed by virtually every reflected or virtual channel (initialization, finalization).

This class expects to be used as either superclass of a concrete channel class, or to be mixed into such a class.

Class API

::tcl::chan::core *objectName*

This command creates a new channel core object with an associated global Tcl command whose name is *objectName*. This command may be used to invoke various operations on the object, as described in the section for the Instance API.

Instance API

The API of channel core instances provides only two methods, both corresponding to channel handler commands (For reference see TIP 219). They expect to be called from whichever object instance the channel core was made a part of.

objectName* initialize *thechannel mode

This method implements standard behaviour for the **initialize** method of channel handlers. Using introspection it finds the handler methods supported by the instance and returns a list containing their names, as expected by the support for reflected channels in the Tcl core.

It further remembers the channel handle in an instance variable for access by sub-classes.

objectName* finalize *thechannel

This method implements standard behaviour for the **finalize** method of channel handlers. It simply destroys itself.

***objectName* destroy**

Destroying the channel core instance closes the channel it was initialized for, see the method **initialize**. When destroyed from within a call of **finalize** this does not happen, under the assumption that the channel is being destroyed by Tcl.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *virtchannel* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Channels

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


events
Keywords: events, event management, reflected channel, tip 219, virtual channel

Name
tcl::chan::events - Event support for reflected/virtual channels

Synopsis
package require Tcl 8.5 9
package require TclOO
package require tcl::chan::core ?**1.1**?
package require tcl::chan::events ?**1.1**?
::tcl::chan::events *objectName*
objectName **finalize** *thechannel*
objectName **watch** *thechannel eventmask*
objectName **allow** *eventname...*
objectName **disallow** *eventname...*

Description
The **tcl::chan::events** package provides a **TclOO** class implementing common behaviour needed by virtually every reflected or virtual channel supporting event driven IO. It is a sub-class of **tcl::chan::core**, inheriting all of its behaviour.

This class expects to be used as either superclass of a concrete channel class, or to be mixed into such a class.

Class API
::tcl::chan::events *objectName*
: This command creates a new channel event core object with an associated global Tcl command whose name is *objectName*. This command may be used to invoke various operations on the object, as described in the section for the Instance API.

Instance API
The API of channel event core instances provides only four methods, two corresponding to channel handler commands (For reference see TIP 219), and the other two for use by sub-classes to control event generation. They former expect to be called from whichever object instance the channel event core was made a part of.

objectName* finalize *thechannel
: This method implements standard behaviour for the **finalize** method of channel handlers. It overrides the behaviour inherited from **tcl::chan::core** and additionally disables any and all event generation before destroying itself.

objectName* watch *thechannel eventmask
: This method implements standard behaviour for the **watch** method of channel handlers. Called by the IO system whenever the interest in event changes it updates the instance state to activate and/or suppress the generation of the events of (non-)interest.

objectName* allow *eventname...
: see below ...

objectName* disallow *eventname...
: These two methods are exported to sub-classes, so that their instances can notify their event core of the events the channel they implement can (allow) or cannot (disallow) generate. Together with the information about the events requested by Tcl's IO system coming in through the **watch** method the event core is able to determine which events it should (not) generate and act accordingly.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *virtchannel* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Channels

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


transformcore

Keywords: transformcore, reflected channel, tip 219, virtual channel

Name

tcl::transform::core - Basic reflected/virtual channel transform support

Synopsis

package require Tcl 8.5 9
package require TclOO
package require tcl::transform::core ?**1.1**?
::tcl::transform::core *objectName*
objectName **initialize** *thechannel mode*
objectName **finalize** *thechannel*
objectName **destroy**

Description

The **tcl::transform::core** package provides a **TclOO** class implementing common behaviour needed by virtually every reflected or virtual channel transformation (initialization, finalization).

This class expects to be used as either superclass of a concrete channel class, or to be mixed into such a class.

Class API

::tcl::transform::core *objectName*

This command creates a new transform core object with an associated global Tcl command whose name is *objectName*. This command may be used to invoke various operations on the object, as described in the section for the Instance API.

Instance API

The API of transform core instances provides only two methods, both corresponding to transform handler commands (For reference see TIP 230). They expect to be called from whichever object instance the transform core was made a part of.

objectName* initialize *thechannel mode

This method implements standard behaviour for the **initialize** method of transform handlers. Using introspection it finds the handler methods supported by the instance and returns a list containing their names, as expected by the support for reflected transformation in the Tcl core.

It further remembers the channel handle in an instance variable for access by sub-classes.

objectName* finalize *thechannel

This method implements standard behaviour for the **finalize** method of channel handlers. It simply destroys itself.

***objectName* destroy**

Destroying the transform core instance closes the channel and transform it was initialized for, see the method **initialize**. When destroyed from within a call of **finalize** this does not happen, under the assumption that the channel and transform are being destroyed by Tcl.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *virtchannel* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Channels

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

virtchannel_transform

adler32

Keywords: adler32, channel transformation, checksum, reflected channel, tip 230, transformation, virtual channel

Name

tcl::transform::adler32 - Adler32 transformation

Synopsis

package require Tcl 8.6 9
package require tcl::transform::core ?**1.1**?
package require tcl::transform::adler32 ?**1.1**?
::tcl::transform::adler32 *chan* **-option** *value...*

Description

The **tcl::transform::adler32** package provides a command creating a channel transformation which passes the read and written bytes through unchanged (like **tcl::transform::identity**), but additionally continuously computes the adler32 checksums of the data it has seen for each direction and stores them in Tcl variables specified at construction time.

Related transformations in this module are **tcl::transform::counter**, **tcl::transform::crc32**, **tcl::transform::identity**, and **tcl::transform::observe**.

The internal **TclOO** class implementing the transform handler is a sub-class of the **tcl::transform::core** framework.

API

::tcl::transform::adler32 ***chan*** **-option** ***value...***

This command creates an adler32 checksumming transformation on top of the channel *chan* and returns its handle. The accepted options are

-read-variable varname

The value of the option is the name of a global or namespaced variable, the location where the transformation has to store the adler32 checksum of the data read from the channel.

If not specified, or the empty string, the checksum of the read direction is not saved.

-write-variable varname

The value of the option is the name of a global or namespaced variable, the location where the transformation has to store the adler32 checksum of the data written to the channel.

If not specified, or the empty string, the checksum of the write direction is not saved.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *virtchannel* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Channels

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


hex
Keywords: hex, channel transformation, hexadecimal, reflected channel, tip 230, transformation, virtual channel

Name
tcl::transform::hex - Hexadecimal encoding transformation

Synopsis
package require Tcl 8.6 9
package require tcl::transform::core ?**1.1**?
package require tcl::transform::hex ?**1.1**?
::tcl::transform::hex *chan*

Description
The **tcl::transform::hex** package provides a command creating a channel transformation which hex encodes data written to it, and decodes the data read from it.

A related transformations in this module is **tcl::transform::base64**.

The internal **TclOO** class implementing the transform handler is a sub-class of the **tcl::transform::core** framework.

API
::tcl::transform::hex *chan*
This command creates a hex transformation on top of the channel *chan* and returns its handle.

Bugs, ideas, feedback
If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *virtchannel* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Channels

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

identity

Keywords: identity, channel transformation, reflected channel, tip 230, transformation, virtual channel

Name

tcl::transform::identity - Identity transformation

Synopsis

package require Tcl 8.6 9
package require tcl::transform::core ?**1.1**?
package require tcl::transform::identity ?**1.1**?
::tcl::transform::identity *chan*

Description

The **tcl::transform::identity** package provides a command creating an identity channel transformation, which does nothing but pass the read and written bytes through it unchanged. Not really useful in an application, however as the prototypical observer transformation its code is a useful starting point for any other observers people may wish to write.

The transformations in this module which derived from identity's code are **tcl::transform::adler32**, **tcl::transform::counter**, **tcl::transform::crc32**, and **tcl::transform::observe**.

The internal **TclOO** class implementing the transform handler is a sub-class of the **tcl::transform::core** framework.

API

::tcl::transform::identity *chan*

This command creates an identity transformation on top of the channel *chan* and returns its handle.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *virtchannel* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Channels

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


limitsize
Keywords: limitsize, channel transformation, reflected channel, size limit, tip 230, transformation , virtual channel

Name
tcl::transform::limitsize - limiting input

Synopsis
package require Tcl 8.6 9
package require tcl::transform::core ?**1.1**?
package require tcl::transform::limitsize ?**1.1**?
::tcl::transform::limitsize *chan max*

Description
The **tcl::transform::limitsize** package provides a command creating a channel transformation which limits the number of characters which can be read from the channel. A generator for an artificial EOF.

The internal **TclOO** class implementing the transform handler is a sub-class of the **tcl::transform::core** framework.

API
::tcl::transform::limitsize *chan max*
This command creates a size limiting transformation on top of the channel *chan* and returns its handle.

max is the number of bytes which can be read from the channel before EOF is signaled by the transformation. Note that popping the transformation clears the EOF it generated as well.

Bugs, ideas, feedback
If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *virtchannel* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Channels

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

observe

Keywords: observe, channel transformation, observer, reflected channel, stream copy, tip 230, transformation, virtual channel

Name

tcl::transform::observe - Observer transformation, stream copy

Synopsis

package require Tcl 8.6 9
package require tcl::transform::core ?**1.1**?
package require tcl::transform::observe ?**1.1**?
::tcl::transform::observe *chan logw logr*

Description

The **tcl::transform::observer** package provides a command creating a channel transformation which passes the read and written bytes through unchanged (like **tcl::transform::identity**), but additionally copies the data it has seen for each direction into channels specified at construction time.

Related transformations in this module are **tcl::transform::adler32**, **tcl::transform::counter**, **tcl::transform::crc32**, and **tcl::transform::identity**.

The internal **TclOO** class implementing the transform handler is a sub-class of the **tcl::transform::core** framework.

API

::tcl::transform::observe *chan logw logr*

This command creates an observer transformation on top of the channel *chan* and returns its handle. The channel handles *logr* and *logw* are there the data is copied to.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *virtchannel* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Channels

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


rot

Keywords: rot, caesar cipher, channel transformation, cipher, decryption, encryption, reflected channel, rot13, tip 230, transformation, virtual channel

Name

tcl::transform::rot - rot-encryption

Synopsis

package require Tcl 8.6 9
package require tcl::transform::core ?**1.1**?
package require tcl::transform::rot ?**1.1**?
::tcl::transform::rot *chan key*

Description

The **tcl::transform::rot** package provides a command creating a channel transformation which performs primitive encryption (on writing) and decryption (on reading) on the alphabetic characters. The algorithm is the Caesar-cipher, a specific variant of which is rot13.

A related transformations in this module is **tcl::transform::otp**.

The internal **TclOO** class implementing the transform handler is a sub-class of the **tcl::transform::core** framework.

API

::tcl::transform::rot *chan key*

This command creates a rot encryption transformation on top of the channel *chan* and returns its handle.

The "*key*" specifies how far characters are rotated in the alphabet, and is wrapped to the range "0...25".

Note that this transformation affects only bytes in the ranges ASCII 65...90, and 97...122, i.e. the upper- and lower-case alphabetic characters, i.e. "A...Z" and "a...z". All other bytes are passed through unchanged.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *virtchannel* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Channels

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


spacer
Keywords: spacer, channel transformation, reflected channel, spacing, tip 230, transformation, virtual channel

Name
tcl::transform::spacer - Space insertation and removal

Synopsis
package require Tcl 8.6 9
package require tcl::transform::core ?**1.1**?
package require tcl::transform::spacer ?**1.1**?
::tcl::transform::spacer *chan n* ?*space*?

Description
The **tcl::transform::spacer** package provides a command creating a channel transformation which adds spacing to the data written to it, and removes such spacing from the data read from it.

The internal **TclOO** class implementing the transform handler is a sub-class of the **tcl::transform::core** framework.

API
::tcl::transform::spacer *chan n* ?*space*?
This command creates a spacer transformation on top of the channel *chan* and returns its handle.

The *space* character sequence will be added every *n* bytes of data written, and on the read side the same is done in reverse, removing the spacing. If *space* is not specified it defaults to a single space character (ASCII 32).

Bugs, ideas, feedback
If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *virtchannel* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Channels

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


tcllib_zlib
Keywords: tcllib_zlib, channel transformation, compression, decompression, reflected channel, tip 230, tip 234, transformation, virtual channel, zlib

Name
tcl::transform::zlib - zlib (de)compression

Synopsis
package require Tcl 8.6 9
package require tcl::transform::core ?**1.1**?
package require tcl::transform::zlib ?**1.1**?
::tcl::transform::zlib *chan* ?*level*?

Description
The **tcl::transform::zlib** package provides a command creating a channel transformation which zlib compresses the written data, and decompresses on reading.

The internal **TclOO** class implementing the transform handler is a sub-class of the **tcl::transform::core** framework.

API
::tcl::transform::zlib *chan* ?*level*?

This command creates a zlib compressor transformation on top of the channel *chan* and returns its handle.

The *level* specifies how much effort is put into the compression, from **0** to **9**, and defaults to **4** .

Bugs, ideas, feedback
If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *virtchannel* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Channels

Copyright

Copyright (c) 2009 Andreas Kupries
<andreas_kupries@users.sourceforge.net>

vt_base64

Keywords: vt_base64, base64, channel transformation, reflected channel, tip 230, tip 317, transformation, virtual channel

Name

tcl::transform::base64 - Base64 encoding transformation

Synopsis

package require Tcl 8.6 9
package require tcl::transform::core ?**1.1**?
package require tcl::transform::base64 ?**1.1**?
::tcl::transform::base64 *chan*

Description

The **tcl::transform::base64** package provides a command creating a channel transformation which base64 encodes data written to it, and decodes the data read from it.

A related transformations in this module is **tcl::transform::hex**.

The internal **TclOO** class implementing the transform handler is a sub-class of the **tcl::transform::core** framework.

API

::tcl::transform::base64 ***chan***

This command creates a base64 transformation on top of the channel *chan* and returns its handle.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *virtchannel* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Channels

Copyright

Copyright (c) 2009 Andreas Kupries
<andreas_kupries@users.sourceforge.net>

vt_counter

Keywords: vt_counter, channel transformation, counter, reflected channel, tip 230, transformation, virtual channel

Name

tcl::transform::counter - Counter transformation

Synopsis

package require Tcl 8.6 9
package require tcl::transform::core ?**1.1**?
package require tcl::transform::counter ?**1.1**?
::tcl::transform::counter *chan* **-option** *value...*

Description

The **tcl::transform::counterr** package provides a command creating a channel transformation which passes the read and written bytes through unchanged (like **tcl::transform::identity**), but additionally counts the bytes it has seen for each direction and stores these counts in Tcl variables specified at construction time.

Related transformations in this module are **tcl::transform::adler32**, **tcl::transform::crc32**, **tcl::transform::identity**, and **tcl::transform::observe**.

The internal **TclOO** class implementing the transform handler is a sub-class of the **tcl::transform::core** framework.

API

::tcl::transform::counter *chan* -option *value...*

This command creates a counter transformation on top of the channel *chan* and returns its handle. The accepted options are

-read-variable varname

The value of the option is the name of a global or namespaced variable, the location where the transformation has to store the byte count of the data read from the channel.

If not specified, or the empty string, the counter of the read direction is not saved.

-write-variable varname

The value of the option is the name of a global or namespaced variable, the location where the transformation has to store the byte count of the data written to the channel.

If not specified, or the empty string, the counter of the write direction is not saved.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *virtchannel* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the

left-most button in the secondary navigation bar.

Category
Channels

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


vt_crc32
Keywords: vt_crc32, channel transformation, checksum, crc32, reflected channel, tip 230, transformation, virtual channel

Name
tcl::transform::crc32 - Crc32 transformation

Synopsis
package require Tcl 8.6 9
package require tcl::transform::core ?**1.1**?
package require tcl::transform::crc32 ?**1.1**?
::tcl::transform::crc32 *chan* **-option** *value...*

Description
The **tcl::transform::crc32** package provides a command creating a channel transformation which passes the read and written bytes through unchanged (like **tcl::transform::identity**), but additionally continuously computes the crc32 checksums of the data it has seen for each direction and stores them in Tcl variables specified at construction time. The checksum in question is zlib's crc32.

Related transformations in this module are **tcl::transform::adler32**, **tcl::transform::counter**, **tcl::transform::identity**, and **tcl::transform::observe**.

The internal **TclOO** class implementing the transform handler is a sub-class of the **tcl::transform::core** framework.

API
::tcl::transform::crc32 *chan* -option *value*...

This command creates a crc32 checksumming transformation on top of the channel *chan* and returns its handle. The accepted options are

-read-variable varname

The value of the option is the name of a global or namespaced variable, the location where the transformation has to store the crc32 checksum of the data read from the channel.

If not specified, or the empty string, the checksum of the read direction is not saved.

-write-variable varname

The value of the option is the name of a global or namespaced variable, the location where the transformation has to store the crc32 checksum of the data written to the channel.

If not specified, or the empty string, the checksum of the write direction is not saved.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *virtchannel* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Channels

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


vt_otp

Keywords: vt_otp, channel transformation, cipher, decryption, encryption, one time pad, otp, reflected channel, tip 230, transformation, virtual channel, xor

Name

tcl::transform::otp - Encryption via one-time pad

Synopsis

package require Tcl 8.6 9
package require tcl::transform::core ?**1.1**?
package require tcl::transform::otp ?**1.1**?
::tcl::transform::otp *chan keychanw keychanr*

Description

The **tcl::transform::otp** package provides a command creating a channel transformation which uses externally provided one-time pads to perform encryption (on writing) and decryption (on reading).

A related transformations in this module is **tcl::transform::rot**.

The internal **TclOO** class implementing the transform handler is a sub-class of the **tcl::transform::core** framework.

API

::tcl::transform::otp *chan keychanw keychanr*

This command creates a one-time pad based encryption transformation on top of the channel *chan* and returns its handle.

The two channels *keychanw* and *keychanr* contain the one-time pads for the write and read

directions, respectively. Their contents are reads and xored with the bytes written to and read from the channel.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *virtchannel* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Channels

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

websocket

websocket

Keywords: websocket, http, internet, net, rfc 6455

Name

websocket - Tcl implementation of the websocket protocol

Synopsis

package require Tcl 8.6 9
package require http 2.7
package require logger
package require sha1
package require base64
package require websocket ?**1.6**?
::websocket::open *url handler* ?*options*?
::websocket::send *sock type* ?*msg*? ?*final*?
::websocket::server *sock*
::websocket::live *sock path cb* ?*proto*?
::websocket::test *srvSock cliSock path* ?*hdrs*? ?*qry*?
::websocket::upgrade *sock*
::websocket::takeover *sock handler* ?*server*?
::websocket::conninfo *sock what*
::websocket::find ?*host*? ?*port*?
::websocket::configure *sock args*
::websocket::loglevel ?*loglvl*?

::websocket::close *sock* ?*code*? ?*reason*?

Description

NOTE: THIS DOCUMENTATION IS WORK IN PROGRESS...

The websocket library is a pure Tcl implementation of the WebSocket specification covering the needs of both clients and servers. Websockets provide a way to upgrade a regular HTTP connection into a long-lived and continuous binary or text communication between the client and the server. The library offers a high-level interface to receive and send data as specified in RFC 6455 (v. 13 of the protocol), relieving callers from all necessary protocol framing and reassembly. It implements the ping facility specified by the standard, together with levers to control it. Pings are server-driven and ensure the liveness of the connection across home (NAT) networks. The library has a number of introspection facilities to inquire about the current state of the connection, but also to receive notifications of incoming pings, if necessary. Finally, the library contains a number of helper procedures to facilitate the upgrading handshaking in existing web servers.

Central to the library is the procedure **websocket::takeover** that will take over a regular socket and treat it as a WebSocket, thus performing all necessary protocol framing, packetisation and reassembly in servers and clients. The procedure also takes a handler, a command that will be called back each time a (possibly reassembled) packet from the remote end is ready for delivery at the original caller. While exported by the package, the command **websocket::takeover** is seldom called in applications, since the package provides other commands that are specifically tuned for the needs of clients and servers.

Typically, clients will open a connection to a remote server by providing a WebSocket URL (*ws:* or *wss:* schemes) and the handler described above to the command **websocket::open**. The opening procedure is a wrapper around the latest http::geturl implementations: it arranges to keep the socket created within the http library opened for reuse, but confiscates it from its (internal) map of known sockets for its own use.

Servers will start by registering themselves through the command **::websocket::server** and a number of handlers for paths using the command **::websocket::live**. Then for each incoming client connection, they should test the incoming request to detect if it is an upgrade request using **::websocket::test** and perform the final handshake to place the socket connection under the control of the websocket library and its central procedure using **::websocket::upgrade**.

Apart from these main commands, the package provides a number of commands for introspection and basic operations on the websockets that it has under its control. As WebSockets connections are long-lived, most remaining communication with the library will be by way of callbacks, i.e. commands that are triggered whenever important events within the library have occur, but mostly whenever data has been received on a WebSocket.

Callbacks

A number of commands of the library take a handler handler command as an argument, a command which will be called back upon reception of data, but also upon important events within the library or events resulting from control messages sent by the remote end. For each callback being performed, the following arguments will be appended:

sock

The identifier of the WebSocket, as returned for example by **::websocket::open**

type

A textual type describing the event or message content, can be one of the following

text
: Complete text message

binary
: Complete binary message

ping
: Incoming ping message

pong
: Response to incoming ping message

connect
: Notification of successful connection to server

disconnect
: Disconnection from remote end

close
: Pending closure of connection

timeout
: Notification of connection timeout

error
: Notification of error condition

msg

Will contain the data of the message, whenever this is relevant, i.e. when the *type* is **text**, **binary**, **ping** or **pong** and whenever there is data available.

API

::websocket::open *url handler* ?*options*?

This command is used in clients to open a WebSocket to a remote websocket-enabled HTTP server. The URL provided as an argument in *url* should start with ws: or wss:, which are the WebSockets counterpart of http: and https:. The *handler* is a command that will be called back on data reception or whenever important events occur during the life of the websocket. **::websocket::open** will return a socket which serves as both the identifier of the websocket and of the physical low-level socket to the server. This socket can be used in a number of other commands for introspection or for controlling the behaviour of the library. Being essentially a wrapper around the **::http::geturl** command, this command provides mostly the same set of dash-led options than **::http::geturl**. Documented below are the options that differ from **::http::geturl** and which are specific to the WebSocket library.

-headers
: This option is supported, knowing that a number of headers will be automatically added internally in the library in order to be able to handshake the upgrading of the socket from a regular HTTP socket to a WebSocket with the server.

-validate
: This option is not supported as it has no real point for WebSockets.

-handler
: This option is used internally by the websocket library and cannot be used.

-command
This option is used internally by the websocket library and cannot be used.

-protocol
This option specifies a list of application protocols to handshake with the server. This protocols might help the server triggering application specific features. The http::geturl option -protocol is used internally by the websocket library and cannot be used.

-timeout
This option is supported, but will implemented as part of the library to enable a number of finalising cleanups.

::websocket::send *sock type* ?*msg*? ?*final*?
This command will send a fragment or a control message to the remote end of the WebSocket identified by *sock*. The type of the message specified in *type* can either be an integer according to the specification or (preferrably) one of the following case insensitive strings: "text", "binary" or "ping". The content of the message to send to the remote end is contained in *msg* and message fragmentation is made possible by the setting the argument *final* to non-true, knowing that the type of each fragment has then to be the same. The command returns the number of bytes that were effectively sent, or -1 on errors. Serious errors, such as when *sock* does not identify a known WebSocket or when the connection is not stable yet will generate errors that must be catched.

::websocket::server *sock*
This command registers the (accept) socket *sock* as the identifier fo an HTTP server that is capable of doing WebSockets. Paths onto which this server will listen for incoming connections should be declared using **::websocket::live**.

::websocket::live *sock path cb* ?*proto*?
This procedure registers callbacks that will be performed on a WebSocket compliant server registered with **::websocket::server** whenever a client connects to a matching path and protocol. *sock* is the listening socket of the websocket compliant server declared using **::websocket::server**. *path* is a glob-style path to match in client request, whenever this will occur. *cb* is the command to callback (see Callbacks). *proto* is a glob-style protocol name matcher.

::websocket::test *srvSock cliSock path* ?*hdrs*? ?*qry*?
This procedure will test if the connection from an incoming client on socket *cliSock* and on the path *path* is the opening of a WebSocket stream within a known server *srvSock*. The incoming request is not upgraded at once, instead a (temporary) context for the incoming connection is created. This allows server code to perform a number of actions, if necessary, before the WebSocket stream connection goes live. The text is made by analysing the content of the headers *hdrs* which should contain a dictionary list of the HTTP headers of the incoming client connection. The command will return **1** if this is an incoming WebSocket upgrade request and **0** otherwise.

::websocket::upgrade *sock*
Upgrade the socket *sock* that had been deemed by **::websocket::test** to be a WebSocket connection request to a true WebSocket as recognised by this library. As a result, the necessary connection handshake will be sent to the client, and the command will arrange for relevant callbacks to be made during the life of the WebSocket, notably using the specifications described by **::websocket::live**.

::websocket::takeover *sock handler* ?*server*?

Take over the existing opened socket *sock* to implement sending and receiving WebSocket framing on top of the socket. The procedure arranges for *handler* to be called back whenever messages, control messages or other important internal events are received or occured. *server* defaults to **0** and can be set to **1** (or a boolean that evaluates to true) to specify that this is a WebSocket within a server. Apart from specificities in the protocol, servers should ping their clients at regular intervals in order to keep the connection opened at all time. When *server* is set to true, the library will arrange to send these pings automatically.

::websocket::conninfo *sock what*

Provides callers with some introspection facilities in order to get some semi-internal information about an existing websocket connection. Depending on the value of the *what* argument, the procedure returns the following piece of information:

peername

Name (preferred) or IP of remote end.

sockname

or **name** Name or IP of local end.

closed

1 if the connection is closed, **0** otherwise

client

1 if the connection is a client websocket, **0** otherwise

server

1 if the connection is a server websocket, **0** otherwise

type

server if the connection is a server websocket, **client** otherwise.

handler

The handler command associated to the websocket

state

The state of the websocket, which can be one of:

CONNECTING

Connection to remote end is in progress.

CONNECTED

Connection is connected to remote end.

CLOSED

Connection is closed.

::websocket::find ?*host*? ?*port*?

Look among existing websocket connections for the ones that match the hostname and port number filters passed as parameters. This lookup takes the remote end into account and the *host* argument is matched both against the hostname (whenever available) and the IP address of the remote end. Both the *host* and *port* arguments are glob-style string matching filters and default to *, i.e. will match any host and/or port number.

::websocket::configure *sock args*

This command takes a number of dash-led options (and their values) to configure the behaviour of an existing websocket connection. The recognised options are the following (they can be shortened to the lowest common denominator):

-keepalive
is the number of seconds between each keepalive pings being sent along the connection. A zero or negative number will effectively turn off the feature. In servers, **-keepalive** defaults to 30 seconds, and in clients, no pings are initiated.

-ping
is the text that is used during the automated pings. This text defaults to the empty string, leading to an empty ping.

::websocket::loglevel ?*loglvl*?
Set or query the log level of the library, which defaults to error. Logging is built on top of the logger module, and the library makes use of the following levels: **debug**, **info**, **notice**, **warn** and **error**. When called with no argument, this procedure will simply return the current log level. Otherwise *loglvl* should contain the desired log level.

::websocket::close *sock* ?*code*? ?*reason*?
Gracefully close a websocket that was directly or indirectly passed to **::websocket::takeover**. The procedure will optionally send the *code* and describing *reason* as part of the closure handshake. Good defaults are provided, so that reasons for a number of known codes will be sent back. Only the first 125 characters of a reason string will be kept and sent as part of the handshake. The known codes are:

1000
Normal closure (the default *code* when none provided).

1001
Endpoint going away

1002
Protocol Error

1003
Received incompatible data type

1006
Abnormal closure

1007
Received data not consistent with type

1008
Policy violation

1009
Received message too big

1010
Missing extension

1011
Unexpected condition

1015
TLS handshake error

Examples

The following example script is a client that opens a websocket connection to an echo service,

waits 400ms to ensure that the connection is really established and sends a single textual message which should be echoed back by the echo service. A real example would probably use the **connect** callback to know when connection to the remote server has been establish and would only send data at that time. Finally the script closes the connection.

```
package require websocket
::websocket::loglevel debug
proc handler { sock type msg } {
    switch -glob -nocase -- $type {
    co* {
        puts "Connected on $sock"
    }
    te* {
        puts "RECEIVED: $msg"
    }
    cl* -
    dis* {
    }
    }

}
proc test { sock } {
    puts "[::websocket::conninfo $sock type] from
    [::websocket::conninfo $sock sockname] to [::websocket::conninfo
    $sock peername]"

    ::websocket::send $sock text "Testing, testing..."
    after 2000 ::websocket::close $sock
}
set sock [::websocket::open ws://ws.ifelse.io/ handler]
after 400 test $sock
vwait forever
```

Example code for a websocket server is provided in the Tcllib directory "examples/websocket".

TLS security considerations

This package uses the **TLS** package to handle the security for **https** urls and other socket connections.

Policy decisions like the set of protocols to support and what ciphers to use are not the responsibility of **TLS**, nor of this package itself however. Such decisions are the responsibility of whichever application is using the package, and are likely influenced by the set of servers the application will talk to as well.

For example, in light of the recent POODLE attack discovered by Google many servers will disable support for the SSLv3 protocol. To handle this change the applications using **TLS** must be patched, and not this package, nor **TLS** itself. Such a patch may be as simple as generally activating **tls1** support, as shown in the example below.

```
    package require tls
    tls::init -tls1 1 ;# forcibly activate support for the TLS1
    protocol

    ... your own application code ...
```

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *websocket* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

Networking

wip

wip

Keywords: wip, interpreter, list, word

Name

wip - Word Interpreter

Synopsis

package require Tcl 8.5 9
package require wip ?**2.3**?
package require snit ?**1.4**?
package require struct::set
::wip *wipName engine arg...*
def *name*
def *name method_prefix*
wipName *option* ?*arg arg* ...?
wip::dsl ?*suffix*?
wipName **def** *name* ?*method_prefix*?
wipName **defl** *names*
wipName **defd** *dict*
wipName **deflva** *name...*
wipName **defdva** (*name method_prefix*)...
wipName **undefl** *names*
wipName **undefva** *name...*
wipName **unknown** *cmdprefix*
wipName **runl** *wordlist*

wipName **run** *word...*
wipName **run_next**
wipName **run_next_while** *acceptable*
wipName **run_next_until** *rejected*
wipName **run_next_if** *acceptable*
wipName **run_next_ifnot** *rejected*
wipName **next**
wipName **peek**
wipName **peekall**
wipName **insertl** *at wordlist*
wipName **replacel** *wordlist*
wipName **pushl** *wordlist*
wipName **addl** *wordlist*
wipName **insert** *at word...*
wipName **replace** *word...*
wipName **push** *word...*
wipName **add** *word...*

Description

This package provides a micro interpreter for lists of words. Domain specific languages based on this will have a bit of a Forth feel, with the input stream segmented into words and any other structuring left to whatever the language desired. Note that we have here in essence only the core dispatch loop, and no actual commands whatsoever, making this definitely only a Forth feel and not an actual Forth.

The idea is derived from Colin McCormack's **treeql** processor, modified to require less boiler plate within the command implementations, at the expense of, likely, execution speed. In addition the interface between processor core and commands is more complex too.

General behaviour

Word interpreters have a mappping from the names of the language commands they shall recognize to the methods in the engine to invoke for them, and possibly fixed arguments for these methods. This mapping is largely static, however it is possible to change it during the execution of a word list (= program).

At the time a language command is defined the word interpreter will use **snit**'s introspection capabilities to determine the number of arguments expected by the method of the egnine, and together with the number of fixed arguments supplied in the method prefix of the mapping it then knows how many arguments the language command is expecting. This is the command's *arity*. Variable-argument methods (i.e. with the last argument named *args*) are *not* allowed and will cause the word interpreter to throw an error at definition time.

Note that while I said **snit**'s abilities the engine object can be written in any way, as long as it understands the method **info args**, which takes a method name and returns the list of arguments for that method.

When executing a list of words (aka program) the first word is always taken as the name of a language command, and the next words as its arguments, per the *arity* of the command. Command and argument words are removed from the list and then associated method of the

engine is executed with the argument words. The process then repeats using the then-first word of the list.

Note that the methods implementing the language commands may have full access to the list of words and are allowed to manipulate as they see fit.

1. This means, for example, that while we cannot specify variable-argument methods directly they can consume words after their fixed arguments before returning to the execution loop. This may be under the control of their fixed arguments.
2. Another possibility is the use of method **run_next** and its variants to execute commands coming after the current command, changing the order of execution.
3. Execution can be further changed by use of the program accessor methods which allow a command implementation to modify the remaining list of words (insert, replace, prepend, append words) without executing them immediately.
4. At last the basic **run** methods save and restore an existing list of words when used, enabling recursive use from within command implementations.

Class API

The main command of the package is:

::wip *wipName engine arg...*

The command creates a new word interpreter object with an associated global Tcl command whose name is *wipName*. If however the string **%AUTO%** was used as object name the package will generate its own unique name for the object.

The *engine* is the object the word interpreter will dispatch all recognized commands to, and the *arg*uments are a word list which defines an initial mapping from language words to engine methods.

The recognized language of this word list is

def *name*

Defines *name* as command of the language, to be mapped to a method of the *engine* having the same name.

def *name method_prefix*

Defines *name* as command of the language, to be mapped to the method of the *engine* named in the *method_prefix*.

The returned command may be used to invoke various operations on the object. It has the following general form:

wipName *option* ?*arg arg ...*?

Option and the *arg*s determine the exact behavior of the command.

The package additionally exports the command:

wip::dsl ?*suffix*?

This command is for use within snit types which wish to use one or more wip interpreters as a component. Use within the type definition installs most of the boilerplate needed to setup and use a word interpreter.

It installs a component named *wip*, and a method **wip_setup** for initializing it. This method has to be called from within the constructor of the type using the word interpreter. If further

installs a series of procedures which make the object API of the word interpreter directly available to the type's methods, without having to specify the component.

Note that this does and cannot install the language to interpret, i.e. the mapping from words to engine methods.

It is possible to instantiate multiple word interpreter components within a type by using different suffices as arguments to the command. In that case the name of the component changes to 'wip_**$suffix**', the setup command becomes 'wip_**$suffix**_setup' and all the procedures also get the suffix '_**$suffix**'.

Object API

The following commands are possible for word interpreter objects:

***wipName* def *name* ?*method_prefix*?**
Defines a language command *name* and maps it to the method named in the engine's *method_prefix*. If the *method_prefix* name is not specified it is simply the name of the language command.

wipName* defl *names
Defines a series of language commands, specified through the list of *names*, all of which are mapped to engine methods of the same name.

wipName* defd *dict
Defines a series of language commands, specified through the dictionary *dict* of names and method prefixes.

***wipName* deflva *name*...**
As method **defl**, however the list of names is specified through multiple arguments.

***wipName* defdva (*name method_prefix*)...**
As method **defd**, however the dictionary of names and method prefixes is specified through multiple arguments.

wipName* undefl *names
Removes the named series of language commands from the mapping.

***wipName* undefva *name*...**
As method **undefl**, however the list of names is specified through multiple arguments.

wipName* unknown *cmdprefix
Sets the handler for unknown words to *cmdprefix*. This command prefix takes one argument, the current word, and either throws some error, or returns the result of executing the word, as defined by the handler. The default handler simply throws an error.

wipName* runl *wordlist
Treats the list of words in *wordlist* as a program and executes the contained command one by one. The result of the command executed last is returned as the result of this command.

The *wordlist* is stored in the object for access by the other *run*-methods, and the general program accessor methods (see below). A previously stored wordlist is saved during the execution of this method and restored before it returns. This enables the recursive execution of word lists within word lists.

***wipName* run *word*...**

As method **runl**, however the list of words to execute is specified through multiple arguments.

***wipName* run_next**
Low-level method. Determines the next word in the list of words, and its arguments, and then executes it. The result of the executed word is the result of this method.

Exposed for use within command implementations. The methods **run** and **runl** use it to execute words until their word list is exhausted.

wipName* run_next_while *acceptable
Low-level method. Invokes the method **run_next** as long as the next word is in the set of *acceptable* words, and the program is not empty. The result of the command executed last is returned as the result of this command.

Exposed for use within command implementations to change the order of execution.

wipName* run_next_until *rejected
Low-level method. Invokes the method **run_next** until the next word is in the set of *rejected* words, and the program is not empty. The result of the command executed last is returned as the result of this command.

Exposed for use within command implementations to change the order of execution.

wipName* run_next_if *acceptable
Low-level method. Invokes the method **run_next** if the next word is in the set of *acceptable* words, and the program is not empty. The result of the command executed last is returned as the result of this command.

Exposed for use within command implementations to change the order of execution.

wipName* run_next_ifnot *rejected
Low-level method. Invokes the method **run_next** if the next word is not in the set of *rejected* words, and the program is not empty. The result of the command executed last is returned as the result of this command.

Exposed for use within command implementations to change the order of execution.

***wipName* next**
Returns the next word in the programm. The word is also removed.

***wipName* peek**
Returns the next word in the programm without removing it

***wipName* peekall**
Returns the remaining programm in toto.

wipName* insertl *at wordlist
Basic programm accessor method. Inserts the specified *wordlist* into the program, just before the word at position *at*. Positions are counted from **zero**.

wipName* replacel *wordlist
Basic programm accessor method. Replaces the whole stored program with the specified *wordlist*.

wipName* pushl *wordlist

Program accessor method. The specified *wordlist* is added to the front of the remaining program. Equivalent to

```
$wip insertl 0 $wordlist
```

wipName* addl *wordlist
Program accessor method. The specified *wordlist* is appended at the end of the remaining program. Equivalent to

```
$wip insertl end $wordlist
```

wipName* insert *at word...
Like method **insertl**, except the words are specified through multiple arguments.

wipName* replace *word...
Like method **setl**, except the words are specified through multiple arguments.

wipName* push *word...
Like method **pushl**, except the words are specified through multiple arguments.

wipName* add *word...
Like method **addl**, except the words are specified through multiple arguments.

Examples
No examples yet.

Bugs, ideas, feedback
If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *wip* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
Programming tools

Copyright

```
Copyright (c) 2007-2010 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


yaml

huddle
Keywords: huddle, data exchange, exchange format, json, parsing, text processing, yaml

Name
huddle - Create and manipulate huddle object

Synopsis

package require Tcl 8.5 9
package require huddle ?**0.5**?
huddle create *key value* ?*key value* ...?
huddle list ?*value value* ...?
huddle number *number*
huddle string *string*
huddle boolean *expression to evaluate as true or false*
huddle true
huddle false
huddle null
huddle get *object key* ?*key* ...?
huddle gets *object key* ?*key* ...?
huddle set *objectVar key* ?*key* ...? *value*
huddle remove *object key* ?*key* ...?
huddle combine *object1 object2* ?*object3* ...?
huddle equal *object1 object2*
huddle append *objectVar key value* ?*key value* ...?
huddle append *objectVar value* ?*value* ...?
huddle keys *object*
huddle llength *object*
huddle type *object* ?*key key...*?
huddle strip *object*
huddle jsondump *object* ?*offset*? ?*newline*? ?*begin_offset*?
huddle compile *spec data*
huddle isHuddle *object*
huddle checkHuddle *object*
huddle to_node *object* ?*tag*?
huddle wrap *tag src*
huddle call *tag command args*
huddle addType *callback*
callback *command* ?*args*?
setting
get_sub *src key*
strip *src*
set *src key value*
remove *src key value*

Description

Huddle provides a generic Tcl-based serialization/intermediary format. Currently, each node is wrapped in a tag with simple type information.

When converting huddle-notation to other serialization formats like JSON or YAML this type information is used to select the proper notation. And when going from JSON/YAML/... to huddle their notation can be used to select the proper huddle type.

In that manner huddle can serve as a common intermediary format.

```
huddle-format: >
  {HUDDLE {huddle-node}}
huddle-node: >
  {tag content}
each content of tag means:
  s: (content is a) string
  L: list, each sub node is a huddle-node
  D: dict, each sub node is a huddle-node
confirmed:
  - JSON
  - YAML(generally, but cannot discribe YAML-tags)
limitation:
  - cannot discribe aliases from a node to other node.
```

The **huddle** package returns data as a Tcl **dict**. Either the **dict** package or Tcl 8.5 is required for use.

Commands

huddle create *key value ?key value ...?*
Create a huddle object as a dict. It can contain other huddle objects.

huddle list ?*value value ...?*
Create a huddle object as a list. It can contain other huddle objects.

huddle number *number*
Create a huddle object as a number.

huddle string *string*
Create a huddle object as a string.

huddle boolean *expression to evaluate as true or false*
Create a huddle object as a boolean evaluating an expression as true or false-

huddle true
Create a huddle object as a boolean true.

huddle false
Create a huddle object as a boolean false.

huddle null
Create a huddle object as a null.

huddle get *object key ?key ...?*
Almost the same as **dict get**. Get a sub-object from the huddle object. *key* can be used to huddle-list's index.

huddle gets *object key ?key ...?*
Get a sub-object from the huddle object, stripped.

huddle set *objectVar key ?key ...? value*
Almost the same as **dict set**. Set a sub-object from the huddle object. *key* can be used to huddle-list's index.

huddle remove *object key ?key ...?*
Almost the same as **dict remove**. Remove a sub-object from the huddle object. *key* can be used to huddle-list's index.

huddle combine *object1 object2 ?object3 ...?*

Merging huddle objects given.

```
% set aa [huddle create a b c d]
HUDDLE {D {a {s b} c {s d}}}
% set bb [huddle create a k l m]
HUDDLE {D {a {s k} l {s m}}}
% huddle combine $aa $bb
HUDDLE {D {a {s k} c {s d} l {s m}}}
```

huddle equal *object1 object2*

Comparing two huddle objects recursively. When to equal, returns 1, otherwise 0.

```
% set aa [huddle create a b c d]
HUDDLE {D {a {s b} c {s d}}}
% set bb [huddle create c d a b]
HUDDLE {D {c {s d} a {s b}}}
% huddle equal $aa $bb
1
```

huddle append *objectVar key value ?key value ...?*

see below ...

huddle append *objectVar value ?value ...?*

Appending child elements. When for dicts, giving key/value. When for lists, giving values.

```
% set aa [huddle create a b c d]
HUDDLE {D {a {s b} c {s d}}}
% huddle append aa a k l m
HUDDLE {D {a {s k} c {s d} l {s m}}}
% set bb [huddle list i j k l]
HUDDLE {L {{s i} {s j} {s k} {s l}}}
% huddle append bb g h i
HUDDLE {L {{s i} {s j} {s k} {s l} {s g} {s h} {s i}}}
```

huddle keys *object*

The same as **dict keys**.

huddle llength *object*

The same as **llength**.

huddle type *object ?key key...?*

Return the element type of specified by keys. if ?key? is not given, returns the type of root node.

string

the node is a tcl's string.

dict

the node is a dict.

list

the node is a list.

number
the node is a number.
boolean
the node is a boolean.
null
the node is a null.

```
% huddle type {HUDDLE {s str}}
string
% huddle type {HUDDLE {L {{s a} {s b} {s c}}}}
list
% huddle type {HUDDLE {D {aa {s b} cc {s d}}}} cc
string
```

huddle strip *object*
Stripped all tags. Converted to normal Tcl's list/dict.
huddle jsondump *object* ?*offset*? ?*newline*? ?*begin_offset*?
dump a json-stream from the huddle-object.
offset ""
begin offset as spaces " ".

```
# normal output has some indents. some strings are escaped.
% huddle jsondump {HUDDLE {L {{L {{s i} {s baa} {s \\k} {L {{s 1.0}
    {s true} {s /g} {s h}}} {L {{s g}}}}} {s t}}}}
[
  [
    "i",
    "baa",
    "\\k",
    [
      1.0,
      true,
      "\/g",
      "h"
    ],
    ["g"]
  ],
  "t"
]
# stripped output
% huddle jsondump {HUDDLE {D {dd {D {bb {D {a {s baa} c {s {d
a}}}} cc {D {g {s h}}}}} ee {D {i {s j} k {s 1} j {s { m\a}}}}}}} ""
    ""
{"dd": {"bb": {"a": "baa","c": "d\na"},"cc": {"g": "h"}},"ee": {"i":
    "j","k": 1,"j": " m\\a"}}
```

huddle compile *spec data*

construct a huddle object from plain old tcl values. *spec* is defined as follows:

string
data is simply a string

list
data is a tcl list of strings

dict
data is a tcl dict of strings

list list
data is a tcl list of lists

list dict
data is a tcl list of dicts

dict xx list
data is a tcl dict where the value of key xx is a tcl list

dict * list
data is a tcl dict of lists *data* is plain old tcl values

```
% huddle compile {dict * list} {a {1 2 3} b {4 5}}
HUDDLE {D {a {L {{s 1} {s 2} {s 3}}} b {L {{s 4} {s 5}}}}}
% huddle compile {dict * {list {dict d list}}} {a {{c 1} {d {2 2 2}
    e 3}} b {{f 4 g 5}}}
HUDDLE {D {a {L {{D {c {s 1}}} {D {d {L {{s 2} {s 2} {s 2}}} e {s
    3}}}}} b {L {{D {f {s 4} g {s 5}}}}}}}
```

huddle isHuddle *object*

if *object* is a huddle, returns 1. the other, returns 0.

huddle checkHuddle *object*

if *object* is not a huddle, rises an error.

huddle to_node *object* ?*tag*?

for type-callbacks.

if *object* is a huddle, returns root-node. the other, returns **[list s $object]**.

```
% huddle to_node str
s str
% huddle to_node str !!str
!!str str
% huddle to_node {HUDDLE {s str}}
s str
% huddle to_node {HUDDLE {l {a b c}}}
l {a b c}
```

huddle wrap *tag src*

for type-callbacks.

Create a huddle object from *src* with specified *tag*.

```
% huddle wrap "" str
HUDDLE str
% huddle wrap s str
HUDDLE {s str}
```

huddle call *tag command args*
for type-callbacks.

devolving *command* to default *tag*-callback

huddle addType *callback*
add a user-specified-type/tag to the huddle library. To see "Additional Type".

callback
callback function name for additional type.

Type callback

The definition of callback for user-type.

callback *command* ?*args*?
see below ...

command
huddle subcomand which is needed to reply by the callback.

args
arguments of subcommand. The number of list of arguments is different for each subcommand.

The callback procedure shuould reply the following subcommands.

setting
only returns a fixed dict of the type infomation for setting the user-tag.

type typename
typename of the type

method {method1 method2 method3 ...}
method list as huddle subcommand. Then, you can call **[huddle method1 ...]**

tag {tag1 child/parent tag2 child/parent ...}
tag list for huddle-node as a dict. if the type has child-nodes, use "parent", otherwise use "child".

get_sub *src key*
returns a sub node specified by *key*.

src
a node content in huddle object.

strip *src*
returns stripped node contents. if the type has child nodes, every node must be stripped.

set *src key value*
sets a sub-node from the tagged-content, and returns self.

remove *src key value*

removes a sub-node from the tagged-content, and returns self.

strip must be defined at all types. **get_sub** must be defined at container types. **set/remove** shuould be defined, if you call them.

```
# callback sample for my-dict
proc my_dict_setting {command args} {
    switch -- $command {
        setting { ; # type definition
            return {
                type dict
                method {create keys}
                tag {d child D parent}
                constructor create
                str s
            }
            # type:   the type-name
            # method: add methods to huddle's subcommand.
            #          "get_sub/strip/set/remove/equal/append" called
    by huddle module.
            #          "strip" must be defined at all types.
            #          "get_sub" must be defined at container types.
            #          "set/remove/equal/append" shuould be defined,
    if you call them.
            # tag:    tag definition("child/parent" word is maybe
    obsoleted)
        }
        get_sub { ; # get a sub-node specified by "key" from the
    tagged-content
            foreach {src key} $args break
            return [dict get $src $key]
        }
        strip { ; # strip from the tagged-content
            foreach {src nop} $args break
            foreach {key val} $src {
                lappend result $key [huddle strip $val]
            }
            return $result
        }
        set { ; # set a sub-node from the tagged-content
            foreach {src key value} $args break
            dict set src $key $value
            return $src
        }
        remove { ; # remove a sub-node from the tagged-content
            foreach {src key value} $args break
            return [dict remove $src $key]
```

```
        }
        equal { ; # check equal for each node
            foreach {src1 src2} $args break
            if {[llength $src1] != [llength $src2]} {return 0}
            foreach {key1 val1} $src1 {
                if {![dict exists $src2 $key1]} {return 0}
                if {![huddle _equal_subs $val1 [dict get $src2
    $key1]]} {return 0}
            }
            return 1
        }
        append { ; # append nodes
            foreach {str src list} $args break
            if {[llength $list] % 2} {error {wrong # args: should be
    "huddle append objvar ?key value ...?"}}
            set resultL $src
            foreach {key value} $list {
                if {$str ne ""} {
                    lappend resultL $key [huddle to_node $value $str]
                } else {
                    lappend resultL $key $value
                }
            }
            return [eval dict create $resultL]
        }
        create { ; # $args: all arguments after "huddle create"
            if {[llength $args] % 2} {error {wrong # args: should be
    "huddle create ?key value ...?"}}
            set resultL {}
            foreach {key value} $args {
                lappend resultL $key [huddle to_node $value]
            }
            return [huddle wrap D $resultL]
        }
        keys {
            foreach {src nop} $args break
            return [dict keys [lindex [lindex $src 1] 1]]
        }
        default {
            error "$command is not callback for dict"
        }
    }
}
```

```
# inheritance sample from default dict-callback
proc ::yaml::_huddle_mapping {command args} {
    switch -- $command {
        setting { ; # type definition
            return {
                type dict
                method {mapping}
                tag {!!map parent}
                constructor mapping
                str !!str
            }
        }
        mapping { ; # $args: all arguments after "huddle mapping"
            if {[llength $args] % 2} {error {wrong # args: should be
    "huddle mapping ?key value ...?"}}
            set resultL {}
            foreach {key value} $args {
                lappend resultL $key [huddle to_node $value !!str]
            }
            return [huddle wrap !!map $resultL]
        }
        default { ; # devolving to default dict-callback
            return [huddle call D $command $args]
        }
    }
}
```

How to add type

You can add huddle-node types e.g. ::struct::tree. To do so, first, define a callback-procedure for additional tagged-type. The proc get argments as *command* and ?*args*?. It has some switch-sections.

And, addType subcommand will called.

```
huddle addType my_dict_setting
```

Working sample

```
# create as a dict
% set bb [huddle create a b c d]
HUDDLE {D {a {s b} c {s d}}}

# create as a list
% set cc [huddle list e f g h]
HUDDLE {L {{s e} {s f} {s g} {s h}}}
% set bbcc [huddle create bb $bb cc $cc]
HUDDLE {D {bb {D {a {s b} c {s d}}} cc {L {{s e} {s f} {s g} {s h}}}}}
```

```
% set folding [huddle list $bbcc p [huddle list q r] s]
HUDDLE {L {{D {bb {D {a {s b} c {s d}}} cc {L {{s e} {s f} {s g} {s
    h}}}}} {s p} {L {{s q} {s r}}} {s s}}}

# normal Tcl's notation
% huddle strip $folding
{bb {a b c d} cc {e f g h}} p {q r} s

# get a sub node
% huddle get $folding 0 bb
HUDDLE {D {a {s b} c {s d}}}
% huddle gets $folding 0 bb
a b c d

# overwrite a node
% huddle set folding 0 bb c kkk
HUDDLE {L {{D {bb {D {a {s b} c {s kkk}}} cc {L {{s e} {s f} {s g} {s
    h}}}}} {s p} {L {{s q} {s r}}} {s s}}}

# remove a node
% huddle remove $folding 2 1
HUDDLE {L {{D {bb {D {a {s b} c {s kkk}}} cc {L {{s e} {s f} {s g} {s
    h}}}}} {s p} {L {{s q}}} {s s}}}
% huddle strip $folding
{bb {a b c kkk} cc {e f g h}} p {q r} s

# dump as a JSON stream
% huddle jsondump $folding
[
  {
    "bb": {
      "a": "b",
      "c": "kkk"
    },
    "cc": [
      "e",
      "f",
      "g",
      "h"
    ]
  },
  "p",
  [
    "q",
    "r"
  ],
```

```
  "s"
]
```

Limitations
now printing.

Bugs, ideas, feedback
If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *huddle* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Copyright

```
Copyright (c) 2008-2011 KATO Kanryu <kanryu6@users.sourceforge.net>
Copyright (c) 2015 Miguel Martínez López <aplicacionamedida@gmail.com>
```


yaml
Keywords: yaml, data exchange, huddle, parsing, text processing

Name
yaml - YAML Format Encoder/Decoder

Synopsis
package require Tcl 8.5 9
package require yaml ?**0.4.2**?
::yaml::yaml2dict ?*options*? *txt*
::yaml::yaml2huddle ?*options*? *txt*
::yaml::setOption ?*options*?
::yaml::dict2yaml *dict* ?*indent*? ?*wordwrap*?
::yaml::list2yaml *list* ?*indent*? ?*wordwrap*?
::yaml::huddle2yaml *huddle* ?*indent*? ?*wordwrap*?

Description
The **yaml** package provides a simple Tcl-only library for parsing the YAML http://www.yaml.org/ data exchange format as specified in http://www.yaml.org/spec/1.1/.

The **yaml** package returns data as a Tcl **dict**. Either the **dict** package or Tcl 8.5 is required for use.

Commands
::yaml::yaml2dict ?*options*? *txt*
see below ...

::yaml::yaml2huddle ?*options*? *txt*

Parse yaml formatted text *txt* into a Tcl dict/huddle and return the value.

-file
txt is a filename of YAML-stream.

-stream
txt is just a YAML-stream.

-types *list*
The *list* is a type list for the yaml-scalar types.(e.g.!!str!!timestamp!!integer!!true ...)

```
-types {timestamp integer null true false}
```

In this case, if a string matched "timestamp", converted to the TCL internal timestamp.(e.g. "2001-12-15T02:59:43.1Z" => 1008385183)

-m:true *param*
The *param* is two elements of list for the value of true, and considered strings.

```
-m:true {1 {true on + yes y}}
```

In this case, the string "yes" found in YAML Stream, automatically converted 1.

-m:false *param*
The *param* is two elements of list for the value of false, and considered strings.

```
-m:false {0 {false off - no n}}
```

-m:null *param*
The *param* is two elements of list for the value of null, and considered strings.

```
-m:null {"" {null nil "" ~}}
```

-validate
Experiment,old: Output stream contains YAML's-tag, each node.

```
% puts [::yaml::load -validate {[aaa, bbb]}]
=>
!!seq {{!!str aaa} {!!str bbb}}
```

::yaml::setOption ?*options*?
Change implicit options for the library. Now, the params are the same as **::yaml::yaml2dict**. Arguments of**::yaml::yaml2dict** is more priority than this setting.

::yaml::dict2yaml *dict* ?*indent*? ?*wordwrap*?
see below ...

::yaml::list2yaml *list* ?*indent*? ?*wordwrap*?
see below ...

::yaml::huddle2yaml *huddle* ?*indent*? ?*wordwrap*?
Convert a dict/list/huddle object into YAML stream.

indent
spaces indent of each block node. currently default is 2.

wordwrap

word wrap for YAML stream. currently default is 40.

Examples

An example of a yaml stream converted to Tcl. A yaml stream is returned as a single item with multiple elements.

```
{
--- !<tag:clarkevans.com,2002:invoice>
invoice: 34843
date   : 2001-01-23
bill-to: &id001
    given  : Chris
    family : Dumars
    address:
        lines: |
            458 Walkman Dr.
            Suite #292
        city    : Royal Oak
        state   : MI
        postal  : 48046
ship-to: *id001
product:
    - sku         : BL394D
      quantity    : 4
      description : Basketball
      price       : 450.00
    - sku         : BL4438H
      quantity    : 1
      description : Super Hoop
      price       : 2392.00
tax  : 251.42
total: 4443.52
comments:
    Late afternoon is best.
    Backup contact is Nancy
    Billsmer @ 338-4338.
}
=>
invoice 34843 date 2001-01-23 bill-to {given Chris family Dumars
    address {lines {458 Walkman Dr.
Suite #292
} city {Royal Oak} state MI postal 48046}} ship-to {given Chris family
    Dumars address {lines {458 Walkman Dr.
Suite #292
} city {Royal Oak} state MI postal 48046}} product {{sku BL394D
    quantity 4 description Basketball price 450.00} {sku BL4438H
```

```
    quantity 1 description {Super Hoop} price 2392.00}} tax 251.42
    total 4443.52 comments {Late afternoon is best. Backup contact is
    Nancy Billsmer @ 338-4338.}
```

An example of a yaml object converted to Tcl. A yaml object is returned as a multi-element list (a dict).

```
{
---
- [name        , hr, avg  ]
- [Mark McGwire, 65, 0.278]
- [Sammy Sosa  , 63, 0.288]
-
  Mark McGwire: {hr: 65, avg: 0.278}
  Sammy Sosa: { hr: 63, avg: 0.288}
}
=>
{name hr avg} {{Mark McGwire} 65 0.278} {{Sammy Sosa} 63 0.288} {{Mark
    McGwire} {hr 65 avg 0.278} {Sammy Sosa} {hr 63 avg 0.288}}
```

Limitations

tag parser not implemented. currentry, tags are merely ignored.

Only Anchor => Aliases ordering. back alias-referring is not supported.

Too many braces, or too few braces.

Not enough character set of line feeds. Please use only "\n" as line breaks.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *yaml* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Copyright


```
Copyright (c) 2008 KATO Kanryu <kanryu6@users.sourceforge.net>
```


zip

decode

Keywords: decode, decompression, zip

Name

zipfile::decode - Access to zip archives

Synopsis

package require Tcl 8.5 9
package require fileutil::decode 0.2.1
package require Trf
package require zlibtcl
package require zipfile::decode ?**0.11.0**?
::zipfile::decode::archive
::zipfile::decode::close
::zipfile::decode::comment *adict*
::zipfile::decode::content *archive*
::zipfile::decode::copyfile *adict path dst*
::zipfile::decode::files *adict*
::zipfile::decode::filetime *adict path*
::zipfile::decode::getfile *zdict path*
::zipfile::decode::hasfile *adict path*
::zipfile::decode::filesize *zdict path*
::zipfile::decode::filecomment *zdict path*
::zipfile::decode::iszip *archive*
::zipfile::decode::open *archive*
::zipfile::decode::unzip *adict dstdir*
::zipfile::decode::unzipfile *archive dstdir*

Description

Note: packages Trf and zlibtcl are not required if TCL 8.6 is available. This package provides commands to decompress and access the contents of zip archives.

API

::zipfile::decode::archive

This command decodes the last opened (and not yet closed) zip archive file. The result of the command is a dictionary describing the contents of the archive. The structure of this dictionary is not public. Proper access should be made through the provided accessor commands of this package.

::zipfile::decode::close

This command releases all state associated with the last call of **::zipfile::decode::open**. The result of the command is the empty string.

::zipfile::decode::comment *adict*

This command takes a dictionary describing the currently open zip archive file, as returned by **::zipfile::decode::archive**, and returns the global comment of the archive.

::zipfile::decode::content *archive*

This is a convenience command which decodes the specified zip *archive* file and returns the list of paths found in it as its result.

::zipfile::decode::copyfile *adict path dst*

This command takes a dictionary describing the currently open zip archive file, as returned by **::zipfile::decode::archive**, and copies the decompressed contents of the file *path* in the archive to the the file *dst*. An error is thrown if the file is not found in the archive.

::zipfile::decode::files *adict*

This command takes a dictionary describing the currently open zip archive file, as returned by **::zipfile::decode::archive**, and returns the list of files found in the archive.

::zipfile::decode::filetime *adict path*
This command takes a dictionary describing the currently open zip archive file, as returned by **::zipfile::decode::archive**, and returns the datestamp of the file *path* in the archive.

::zipfile::decode::getfile *zdict path*
This command takes a dictionary describing the currently open zip archive file, as returned by **::zipfile::decode::archive**, and returns the decompressed contents of the file *path* in the archive. An error is thrown if the file is not found in the archive.

::zipfile::decode::hasfile *adict path*
This command takes a dictionary describing the currently open zip archive file, as returned by **::zipfile::decode::archive**, and check if the specified *path* is found in the archive. The result of the command is a boolean flag, **true** if the path is found, and **false** otherwise.

::zipfile::decode::filesize *zdict path*
This command takes a dictionary describing the currently open zip archive file, as returned by **::zipfile::decode::archive**, and returns the decompressed size of the file *path* in the archive. An error is thrown if the file is not found in the archive.

::zipfile::decode::filecomment *zdict path*
This command takes a dictionary describing the currently open zip archive file, as returned by **::zipfile::decode::archive**, and returns the per-file comment of the file *path* in the archive. An error is thrown if the file is not found in the archive.

::zipfile::decode::iszip *archive*
This command takes the path of a presumed zip *archive* file and returns a boolean flag as the result of the command telling us if it actually is a zip archive (**true**), or not (**false**).

::zipfile::decode::open *archive*
This command takes the path of a zip *archive* file and prepares it for decoding. The result of the command is the empty string. All important information is stored in global state. If multiple open calls are made one after the other only the state of the last call is available to the other commands.

::zipfile::decode::unzip *adict dstdir*
This command takes a dictionary describing the currently open zip archive file, as returned by **::zipfile::decode::archive**, and unpacks the archive in the given destination directory *dstdir* . The result of the command is the empty string.

::zipfile::decode::unzipfile *archive dstdir*
This is a convenience command which unpacks the specified zip *archive* file in the given destination directory *dstdir*.

The result of the command is the empty string.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *zipfile* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category
File

Copyright

```
Copyright (c) 2008-2022 Andreas Kupries
```


encode
Keywords: encode, compression, zip

Name
zipfile::encode - Generation of zip archives

Synopsis
package require Tcl 8.5 9
package require logger
package require Trf
package require crc32
package require snit
package require zlibtcl
package require fileutil
package require zipfile::encode ?**0.5.2**?
::zipfile::encode *objectName*
comment: *text*
file: *dst owned src* ?*noCompress*?
write *archive*

Description
Note: packages Trf and zlibtcl are not required for Tcl version 8.6 or above. This package provides a class for the generation of zip archives.

Class API
::zipfile::encode *objectName*

The class command constructs encoder instances, i.e. objects. The result of the command is the fully-qualified name of the instance command.

The argument *objectName* is required. If *objectName* is set to **%AUTO%** the class will generate and use an automatic name in the current namespace. If the value of *objectName* is not fully qualified, the instance command will be created relative to the current namespace.

Instance API
comment: *text*

This method specifies the text of the global comment for the archive. The result of the

method is the empty string. In case of multiple calls to this method for the same encoder the data from the last call prevails over all previous texts.

file: *dst owned src ?noCompress?*

This method adds a new file to the archive. The contents of the file are found in the filesystem at *src*, and will be stored in the archive under path *dst*. If the file is declared as *owned* by the archive the original file will be deleted when the archive is constructed and written. If *noCompress* is set to **true** the file will not be compressed on writing. Otherwise (the default) the file is compressed if it is advantageous. The result of the method is an empty string.

write *archive*

This method takes the global comment and all added files, encodes them as a zip archive and stores the result at path *archive* in the filesystem. All added files which were owned by the archive are deleted at this point. On the issue of ordering, the files are added to the archive in the same order as they were specified via **file:**. *Note* that this behaviour is new for version 0.4 and higher. Before 0.4 no specific order was documented. It was lexicographically sorted. The change was made to support **zip**-based file formats which require a specific order of files in the archive, for example "*.epub*".

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *zipfile* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

File

Copyright

```
Copyright (c) 2008-2009 Andreas Kupries
```

mkzip

Keywords: mkzip, decompression, zip

Name

zipfile::mkzip - Build a zip archive

Synopsis

package require Tcl 8.6 9
package require zipfile::mkzip ?**1.2.4**?
::zipfile::mkzip::mkzip *zipfile* ?**-zipkit**? ?*-runtime prefix*? ?*-comment string*? ?*-directory rootpath*? ?*-prepend-base-directory bool*? ?*-exclude exclude*? ?**--**? ?*path...*?

Description

This package utilizes the zlib functions in Tcl 8.6 to build zip archives.

API

::zipfile::mkzip::mkzip *zipfile* ?-zipkit? ?-runtime *prefix*? ?-comment *string*? ?-directory *rootpath*? ?-prepend-base-directory *bool*? ?-exclude *exclude*? ?--? ?*path*...?

From http://wiki.tcl.tk/15158

This command constructs a zip archive from a directory tree using nothing but Tcl 8.6 core features. The resulting zip file should be compatible with other **zip** programs - with the possible exception of unicode support. The files generated by this command use utf-8 encoding for all filenames and comments and it has been noticed particularly on Windows the **info-zip** and the Windows built-in zip view have rather poor support for this part of the ZIP file specification. The **7-Zip** program does correctly display utf8 filenames however and the **vfs::zip** package will use these of course.

If you use

```
::mkzip::mkzip mystuff.tm -zipkit -directory mystuff.vfs
```

it will pack your "*mystuff.vfs/*" virtual filesystem tree into a zip archive with a suitable header such that on unix you may mark it executable and it should run with tclkit. Or you can run it with **tclsh** or **wish** 8.6 if you like.

To change the executable header, specify the **-runtime** "*preface*" where preface is a file containing code you want prefixed. For instance, on Windows you can create a self-extracting zip archive using

```
mkzip mystuff.exe -directory mystuff.vfs -runtime unzipsfx.exe
```

The "*unzipsfx.exe*" is the Info-Zip self-extracting stub.

Accepted options:

-runtime path
: This option specifies a file to use as prefix to the actual zip archive. If specified **-zipkit** will be ignored.

-zipkit
: Instructs the command to generate a prefix which makes the archive a zip-based starkit. Ignored if **-runtime** is present.

-comment string
: This options specifies a global comment to place into the generated archive.

-directory path
: This option specifies the directory to place into the generated archive. If specified any argument *paths* are *ignored*.

-prepend-base-directory bool
: When this option is set the tail of the **-directory** path is prepended to all paths added to the archive.

  The option is ignored when option **-directory** is not specified.

By default this option is **false**.

-exclude list
This option specifies a list of glob patterns. All paths matching at least one of the patterns are not placed into the generated archive. This option defaults to

```
CVS/* */CVS/* *~ ".#*" "*/.#*"
```

--
This option signals the end of the options, forcing processing of all further words as arguments, even if they begin with a dash character.

Accepted arguments:
path *path*
Each path is a directory or file to place into the generated archive. Note however that these will be ignored when option **-directory** is specified.

Bugs, ideas, feedback

If you find errors in this document or bugs or problems with the package it describes, or if you want to suggest improvements for the documentation or the package, please use the Tcllib Trackers and specify *zipfile* as the category.

When proposing code changes, please provide *unified diffs*, i.e the output of **diff -u**.

Note further that *attachments* are strongly preferred over inlined patches. Attachments can be made by going to the **Edit** form of the ticket immediately after its creation, and then using the left-most button in the secondary navigation bar.

Category

File

Copyright

```
Copyright (c) 2009 Pat Thoyts
```

License

These pages were converted from the original manual pages to Markdown (TIP 700 format). The text is unchanged; only markup without a place in the format (links without a target, layout images) may be lost. For parts under the Mozilla Public License 1.1 (tDOM) this is the description of modifications required by section 3.3.

This collection is not an official publication of the projects. Each part is under the terms of its original, which follow. The copyright notices of the original pages stay with each page.

license license terms

This software is copyrighted by Ajuba Solutions and other parties. The following terms apply to all files associated with the software unless explicitly disclaimed in individual files.

The authors hereby grant permission to use, copy, modify, distribute, and license this software and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions. No written agreement,

license, or royalty fee is required for any of the authorized uses. Modifications to this software may be copyrighted by their authors and need not follow the licensing terms described here, provided that the new terms are clearly indicated on the first page of each file where they apply.

IN NO EVENT SHALL THE AUTHORS OR DISTRIBUTORS BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OF THIS SOFTWARE, ITS DOCUMENTATION, OR ANY DERIVATIVES THEREOF, EVEN IF THE AUTHORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

THE AUTHORS AND DISTRIBUTORS SPECIFICALLY DISCLAIM ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. THIS SOFTWARE IS PROVIDED ON AN "AS IS" BASIS, AND THE AUTHORS AND DISTRIBUTORS HAVE NO OBLIGATION TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

GOVERNMENT USE: If you are acquiring this software on behalf of the U.S. government, the Government shall have only "Restricted Rights" in the software and related documentation as defined in the Federal Acquisition Regulations (FARs) in Clause 52.227.19 (c) (2). If you are acquiring the software on behalf of the Department of Defense, the software shall be classified as "Commercial Computer Software" and the Government shall have only "Restricted Rights" as defined in Clause 252.227-7013 (c) (1) of DFARs. Notwithstanding the foregoing, the authors grant the U.S. Government and others acting in its behalf permission to use and distribute the software in accordance with the terms specified in this license.

Index

.

.ddt 245
.dtx 43, 237, 245

/

/dev/null 1766, 1772
/dev/random 1767, 1773
/dev/zero 1766, 1775

3

3DES 220, 223, 224

A

abstract syntax tree 536, 555
acceptance 511
acceptor 511
active 1686
adaptors 1321, 1359
adjacency list 1465
adjacency matrix 1465
adjacent 1444, 1465
adjust 1629
adjusting 1629
adler32 1782
aes 71
after 188, 189
alias 619
amazon 74
ambiguous 509
American Express 1740
AMEX 1740
angle 775, 1726
annealing 1306
anonymous procedure 637
ansi 1597, 1601, 1608, 1610
ansi_cattr 1597
ansi_cctrl 1601
ansi_cmacros 1608
ansi_code 1609
ansi_ctrlu 1610
ansi_send 1612
appender 674, 675
application 22, 24, 25
approximation algorithm 1465
arc 1444, 1465

arcfour 1266
archive 1542
argument integrity 1568, 1576
argument processing 163
argument validation 1568, 1576
arguments 1568, 1576
argv 163
argv0 163
array 1666, 1673
articulation point 1465
ascii85 111
asn 98
assembler 531
assert 186
assign 1512
AST 536
async 145
asynchronous 145
attribute control 1597, 1601
augmenting network 1465
augmenting path 1465
authentication 589, 1292, 1294, 1299
automatic 903
automatic documentation 1560
automaton 511, 513, 515, 524
autoproxy 589
aycock 509

B

bank 1740, 1742, 1744, 1746, 1749
base32 106, 108, 109
base32core 108
base32hex 109
base64 113, 1790
bash 1429
bee 122
bench 128
bench language 132, 133, 135
bench_intro 132
bench_lang_intro 133
bench_lang_spec 135
bench_read 137
bench_wcsv 138
bench_wtext 139
benchmark 128, 132, 133, 135, 137, 138, 139
ber 98
Bessel functions 822

bfs 1465
bibliography 140
bibtex 140
bigfloat 698
bignum 707
bignums 707
bind 1722
bipartite 1465
BitTorrent 122
blanks 1642
block cipher 71, 142, 220, 223, 224
blocking flow 1465
blowfish 142
Book Number 1752
breadth-first 1527
bridge 1465
BWidget 1321, 1359

C

C 384, 450
C++ 1321, 1359, 1418, 1421
cache 145, 687
caesar cipher 1787
calculus 711
callback 145, 573, 637, 933, 1677, 1725
callbacks 1765
capitalize 1659
card for credit 1740, 1742, 1744, 1746
cardinality 1524
cat 463, 1762
catalog package 340, 384, 385, 386, 387, 450, 451, 452, 453
catalogue 245
cc_amex 1740
cc_discover 1742
cc_mastercard 1744
cc_visa 1746
cell-phone 1751
cer 98
CFG 548
CFL 548
cfront 493
cgen 494
CGI 893
cgraph 1444, 1460
changelog 256, 258
changepoint 720
channel 188, 189, 1686, 1689, 1692, 1694, 1697, 1701, 1703

channel transformation 1782, 1783, 1784, 1785, 1786, 1787, 1788, 1789, 1790, 1791, 1792, 1793
character input 1624, 1626
character output 1612, 1628
character type 1664
character width 1664
chat 621, 624, 889
checkbox 578, 627
checkbutton 578
Checking 1740, 1742, 1744, 1746, 1747, 1749, 1751, 1752, 1754, 1756, 1757, 1759, 1761
checksum 194, 196, 201, 203, 1782, 1792
chop 1659
cipher 956, 1787, 1793
cksum 194, 196, 201, 203
class 1321, 1359, 1418, 1421
class methods 933, 1677
class variables 933, 1677
clay 147
cleanup 219, 1719
client 22, 25, 903, 905, 910, 911
cloud 74
cmdline 163
cmdline processing 163
color control 1597, 1601
columns 1610
combinatorics 722
comm 167, 183, 617, 618, 913
comm_wire 183
command 338
command line processing 163
command prefix 637, 933, 1677
comment 629, 967
common 1512
common prefix 1659
communication 167, 183
comparison 1512
compiler 509
complete graph 1465
complex numbers 745, 807
compression 1789, 1823
computations 698
concatenation channel 1762, 1763
connect 1686
connected component 1465
connected fifos 1770

connection 1686
constants 727, 1726
CONTAINER 1072, 1157
contents 447
context-free grammar 548
context-free languages 32, 548, 563, 571, 1014, 1019, 1020, 1021, 1028, 1031, 1038, 1047, 1049, 1055, 1064, 1065, 1072, 1079, 1088, 1096, 1097, 1105, 1113, 1120, 1121, 1129, 1137, 1143, 1145, 1153, 1157, 1164, 1172, 1181, 1204, 1213, 1220, 1226, 1230, 1236, 1239, 1254, 1255, 1256, 1257, 1264
control 186, 720, 1597, 1601, 1608, 1609, 1610, 1612, 1619, 1622, 1624, 1625, 1626, 1628
control structure 499
conversion 15, 43, 259, 279, 300, 335, 345, 350, 371, 381, 404, 411, 435, 447, 462, 816, 1097, 1105, 1157, 1164, 1172, 1181, 1204, 1213, 1726
cooked 1610
cookie 893
copy 470, 471, 1689, 1692, 1694, 1697, 1701, 1703
copyops 1689
core 1777
coro_auto 188
coroutine 188, 189, 499
Cost 1708
counter 191, 1791
counting 191
CPARAM 1164
crc 194, 196, 201, 203
crc16 196
crc32 194, 196, 201, 203, 1792
credit card 1740, 1742, 1744, 1746
cron 205
cryptography 142
CSS 336
csv 137, 138, 208
currying 637, 933, 1677
cut edge 1465
cut vertex 1465
CVS 1268
cvs 258
cvs log 258
cyclic redundancy check 194, 196, 201, 203

D

dacceptor 511
data analysis 830
data destination 1692, 1697

data entry form 1545
data exchange 632, 635, 1806, 1817
data integrity 71, 194, 196, 201, 203, 220, 223, 224, 956, 1266
data source 1694, 1703
data structures 1502
database 1666, 1673
dataflow 949
ddest 1692
DE 385, 451
debug 212, 216, 217, 218
debug_caller 216
debug_heartbeat 217
debug_timestamp 218
decimal 729
declare 1609
decode 1820
decompression 1789, 1820, 1824
decryption 1787, 1793
defer 219
deferal 1725
define 1609
degree 1444, 1465
degree constrained spanning tree 1465
degrees 727
deleg_method 617
deleg_proc 618
delegation 617, 618
depth-first 1527
der 98
DES 223, 224
des 220
deserialization 378, 392, 395, 401, 443, 458
dexec 513
diameter 1465
dict 225
dicttool 225
diff 245, 1512, 1640
diff -n format 1268
diff -ruN 1640
difference 1524
differential 1512
differential equations 711
digital 743
dijkstra 1465
directory access 642, 651
directory traversal 479

Discover 1742
discrete items 1492
disjoint set 1439
disjointset 1439
dispatcher 1626
distance 775, 1465, 1726
DNS 227
do 186
docidx 15, 259, 340, 342, 350, 371, 384, 385, 386, 387, 388, 392, 395, 462
docidx commands 265, 267, 268, 271
docidx language 265, 267, 268, 271
docidx markup 265, 267, 268, 271, 345
docidx syntax 267, 268, 271
docidx_intro 264
docidx_lang_cmdref 265
docidx_lang_faq 267
docidx_lang_intro 268
docidx_lang_syntax 271
docidx_plugin_apiref 273
docstrip 43, 237, 245
docstrip_util 245
doctoc 15, 279, 340, 398, 401, 411, 435, 450, 451, 452, 453, 454, 458, 462
doctoc commands 285, 287, 288, 293
doctoc language 285, 287, 288, 293
doctoc markup 285, 287, 288, 293, 404
doctoc syntax 287, 288, 293
doctoc_intro 284
doctoc_lang_cmdref 285
doctoc_lang_faq 287
doctoc_lang_intro 288
doctoc_lang_syntax 293
doctoc_plugin_apiref 294
doctools 15, 245, 256, 300, 336, 337, 338, 340, 342, 356, 360, 364, 366, 369, 378, 384, 385, 386, 387, 388, 392, 395, 398, 401, 417, 422, 427, 430, 433, 443, 450, 451, 452, 453, 454, 458, 462
doctools commands 308, 316, 317, 326
doctools language 308, 316, 317, 326
doctools markup 308, 316, 317, 326
doctools syntax 316, 317, 326
doctools_intro 307
doctools_lang_cmdref 308
doctools_lang_faq 316
doctools_lang_intro 317

doctools_lang_syntax 326
doctools_plugin_apiref 328
document 328
documentation 43, 237, 245, 259, 279, 300, 345, 350, 371, 404, 411, 435, 1560
DOM 1708
dom 95
domain name service 227
double-wide character 1664
dsource 1694
dtplite 15

E

e 727
EAN 1747, 1752
EAN13 1752
ean13 1747
Earley 509
EBNF 32, 1014, 1019, 1020, 1021, 1028, 1031, 1038, 1047, 1049, 1055, 1064, 1065, 1072, 1079, 1088, 1096, 1097, 1105, 1113, 1120, 1121, 1129, 1137, 1143, 1145, 1153, 1157, 1164, 1172, 1181, 1204, 1213, 1220, 1226, 1230, 1236, 1239, 1254, 1255, 1256, 1257, 1264
eccentricity 1465
edge 1444, 1465
emacs 256, 258
email 607, 878, 883, 970
emptiness 1524
empty interpreter 619
EN 386, 452
encode 1823
encoding 111, 113, 116, 119
encryption 71, 142, 220, 223, 224, 956, 1266, 1787, 1793
entry mask 1568
equal 1512
equality 1512
equivalence class 1439
error 1718, 1719
error function 822
European Article Number 1747, 1752
event 573, 1722, 1725
event management 1779
events 188, 189, 1779
exact 734
examples 133, 267, 287, 316
exception 1719
exchange format 632, 635, 1806

exclusion 1524
execution 513
exif 629
exit 188, 189
expander 1633
export 336, 337, 342, 350, 356, 360, 364, 366, 369, 398, 411, 417, 422, 427, 430, 433, 1072, 1079, 1088
export_docidx 342
export_doctoc 398
expression 32, 548, 563, 571, 1014, 1019, 1020, 1021, 1028, 1031, 1038, 1047, 1049, 1055, 1064, 1065, 1072, 1079, 1088, 1096, 1097, 1105, 1113, 1120, 1121, 1129, 1137, 1143, 1145, 1153, 1157, 1164, 1172, 1181, 1204, 1213, 1220, 1226, 1230, 1236, 1239, 1254, 1255, 1256, 1257, 1264
extended namespace 891

F

fa 515
facade 1763
faop 524
faq 267, 287, 316
fetching information 1732
FFT 745
fifo 1765, 1769, 1770
figurate 737
figurate numbers 737
file 1666, 1673, 1732
file recognition 493, 494, 495, 496
file type 493, 494, 495, 496
file utilities 463, 470, 471, 493, 494, 495, 496
filesystem 687
filetypes 495
fileutil 463
filter 499, 1512
filtergen 743
filtering 743
final 1719
finance 1740, 1742, 1744, 1746, 1749
find 1439
finite 1492
finite automaton 511, 513, 515, 524
FIPS 180-1 1300, 1303
first permutation 1512
Fisher-Yates 1512
flatten 1512
floating-point 698, 747
flow 186

flow network 1465
folding 1512
foldl 499
foldr 499
foreach 499
form 578, 893
format conversion 1097, 1105, 1157, 1164, 1172, 1181, 1204, 1213
formatter 328
formatting 137, 138, 139, 345, 350, 381, 404, 411, 447, 1629, 1643, 1645, 1659
formatting engine 273, 294, 328
fossil 1640
fourier 745
Fourier transform 745
FR 387, 453
frame 1608
framework 1680
ftp 482, 488, 489, 1732
ftp_geturl 488
ftpd 489
ftpserver 489
full outer join 1512
fuzzy 747

G

gasm 531
generate event 1722
generate permutations 1512
generation 345, 350, 404, 411
generator 499
geocoding 678
geodesy 680, 690
geography 680
get character 1624
gets 188, 189
git 1640
global 188, 189
golang 219
gopher 1732
gps 506, 901
gpx 506
grammar 32, 509, 511, 513, 515, 524, 531, 538, 542, 548, 549, 556, 563, 571, 1014, 1019, 1020, 1021, 1028, 1031, 1038, 1047, 1049, 1055, 1064, 1065, 1072, 1079, 1088, 1096, 1097, 1105, 1113, 1120, 1121, 1129, 1137, 1143, 1145, 1153, 1157, 1164, 1172, 1181, 1204, 1213, 1220, 1226, 1230, 1236, 1239, 1254, 1255, 1256, 1257, 1264

graph 531, 1444, 1460, 1465, 1501, 1511
graph walking 949, 951, 952
graph1 1460
graphops 1465
green threads 188, 189
grep 463
gtoken 1292
GUID 1739

H

halfpipe 1765
hashing 871, 874, 876, 936, 1287, 1290, 1300, 1303
heartbeat 217
heuristic 1465
hex 109, 1783
hexadecimal 1783
histogram 191
hook 573, 1722
Horspool 509
HTML 15, 259, 279, 300, 335, 336, 345, 350, 356, 404, 411, 417, 462
html 578, 585, 627, 893
html_cssdefaults 336
htmlparse 585
http 589, 593, 678, 689, 1732, 1794
httpd 593
https 1732
httpserver 593
huddle 1806, 1817
human readable 137, 139
hyphenation 1629, 1645

I

i18n 340, 384, 385, 386, 387, 450, 451, 452, 453
iban 1749
ident 606
identification 606
identity 1784
idle 1725
idx_container 345
idx_export 350
idx_export_html 356
idx_export_json 360
idx_export_nroff 364
idx_export_text 366
idx_export_wiki 369
idx_import 371

idx_import_json 378
idx_introduction 381
idx_msgcat_c 384
idx_msgcat_de 385
idx_msgcat_en 386
idx_msgcat_fr 387
idx_parse 388
idx_structure 392
image 629, 967, 1674
imap 607
imap4 607
imei 1751
imenu 1619
import 371, 378, 395, 401, 435, 443, 1121, 1129
import_docidx 395
import_doctoc 401
in-memory channel 1765, 1769, 1770, 1771, 1774, 1775
in-order 1527
inclusion 1524
Incr Tcl 1321, 1359
indenting 1629, 1645
independent set 1465
index 259, 264, 273, 342, 345, 350, 356, 360, 364, 366, 369, 371, 378, 381, 395
index formatter 273
info 891
ini 615
inner join 1512
input mode 1610
integer 816
integration 711
inter-thread communication 1770
International Article Number 1747, 1752
International Bank Account Number 1749
International Mobile Equipment Identity 1751
International Standard Book Number 1752
internationalization 340, 384, 385, 386, 387, 450, 451, 452, 453
internet 98, 482, 488, 607, 642, 651, 878, 883, 974, 978, 980, 1794
internet address 231
interpolate 750
interpolation 750
interpreter 617, 618, 619, 1801
intersection 1524
interval 698
ip 231

ipager 1622
ipc 167, 183
ipv4 231
ipv6 231
irc 621, 624
isA 1740, 1742, 1744, 1746, 1747, 1749, 1751, 1752, 1754, 1756, 1757, 1759, 1761
isbn 1752
iso8601 161
isthmus 1465
iterator 499

J

javascript 627, 632, 635
jfif 629
join 1512
jpeg 629
JSON 360, 378, 422, 443, 1079, 1097, 1121, 1172
json 345, 350, 371, 404, 411, 435, 632, 635, 1806
json_write 635
justification 1629

K

keyword index 259, 264, 345, 350, 371, 381
keywords 273
knuth 1417

L

l10n 340, 384, 385, 386, 387, 450, 451, 452, 453
lambda 637
LaTeX 43, 237, 245
latex 259, 279, 345, 404
latitute 680
lazyset 640
ldap 642, 651, 1732
ldap client 642, 651
ldapx 651
ldif 651
least squares 754
left outer join 1512
lemon 951
level graph 1465
lexer 388, 454
lexing 1426, 1429
limitsize 1785
linalg 754
line 775
linear algebra 754

linear equations 754
linear program 792
lines 1610
list 1501, 1512, 1801
listener 1624, 1626
literate programming 43, 237, 245
LL(k) 32, 548, 563, 571, 1014, 1019, 1020, 1021, 1028, 1031, 1038, 1047, 1049, 1055, 1064, 1065, 1072, 1079, 1088, 1096, 1097, 1105, 1113, 1120, 1121, 1129, 1137, 1143, 1145, 1153, 1157, 1164, 1172, 1181, 1204, 1213, 1220, 1226, 1230, 1236, 1239, 1254, 1255, 1256, 1257, 1264
local searching 1465
localization 340, 384, 385, 386, 387, 450, 451, 452, 453
location 678, 680, 687, 689
log 212, 216, 217, 218, 258, 663, 667
log level 663, 667
logger 667, 674, 675
loggerAppender 674
loggerUtils 675
longest common subsequence 1512
longitude 680
loop 1444, 1465
luhn 1754, 1756
luhn-5 1756
luhn5 1756

M

machineparameters 770
macros 337
mail 607, 878, 883, 970
mailto 1732
man_macros 337
manpage 15, 259, 279, 300, 328, 335, 345, 350, 371, 411, 435, 462
map 499, 678, 680, 687, 689, 690, 1512
map_geocode_nominatim 678
map_slippy 680
map_slippy_cache 687
map_slippy_fetcher 689
mapproj 690
markdown 259, 279, 300, 696
markup 15, 43, 259, 264, 265, 267, 268, 271, 273, 279, 284, 285, 287, 288, 293, 294, 300, 307, 308, 316, 317, 326, 328, 335, 345, 350, 371, 381, 404, 411, 435, 447, 462
MasterCard 1744
matching 32, 548, 571, 1014, 1019, 1020, 1021, 1028, 1031, 1038, 1047, 1049, 1055, 1064, 1065, 1072, 1079, 1088, 1096,

1097, 1105, 1113, 1120, 1121, 1129, 1137, 1143, 1145,
1153, 1157, 1164, 1172, 1181, 1204, 1213, 1220, 1226,
1230, 1236, 1239, 1254, 1255, 1256, 1257, 1264, 1465
math 698, 707, 711, 727, 729, 747, 750, 754, 773, 775, 792,
798, 801, 807, 813, 822, 867, 1306, 1310, 1313
math_geometry 775
mathematics 737, 745, 804, 811, 830
matrices 754
matrix 208, 754, 1272, 1484, 1501, 1511
max cut 1465
maximum 792
maximum flow 1465
md4 871, 1287, 1290
md5 874, 876
md5crypt 876
me_ast 536
me_cpu 538
me_cpucore 542
me_intro 548
me_tcl 549
me_util 555
me_vm 556
medicare 1757
mega widget 1321, 1359
membership 1524
menu 1608, 1619
merge 1725, 1767
merge find 1439
merging 128
message 167, 183, 663
message catalog 340, 384, 385, 386, 387, 450, 451, 452, 453
message level 663
message package 340, 384, 385, 386, 387, 450, 451, 452, 453
message-digest 871, 874, 876, 936, 1287, 1290, 1300, 1303
meta 1677
metakit 1666, 1673
method 617, 619
method reference 933, 1677
mime 493, 494, 496, 878, 883
minimal spanning tree 1465
minimum 792
minimum cost flow 1465
minimum degree spanning tree 1465
minimum diameter spanning tree 1465
mkdoc 20, 886
mkzip 1824

mobile phone 1751
module 245
montecarlo 1310
montecarlo simulation 1310
move 470, 471
mpexpand 335
multi 470
multi-file 470, 471
multiop 471
multiplexer 889
multiprecision 698, 707
my method 933, 1677

N

name service 22, 24, 25, 903, 905, 910, 911, 913, 915, 1721
namespace unknown 891
namespace utilities 891
namespacex 891
narrative 212, 216, 217, 218
National Provider Identifier 1757
ncgi 893
neighbour 1444, 1465
net 482, 488, 607, 878, 883, 1794
nettool 898
network 974, 978, 980
news 917, 1732
next permutation 1512
nmea 901
nns 22
nns_auto 903
nns_client 905
nns_common 910
nns_intro 911
nns_protocol 913
nns_server 915
nnsd 24
nnslog 25
nntp 917
nntpclient 917
no-op 186
node 1444, 1465, 1527
nominatim 678
normalization 128, 951, 952, 1436
NPI 1757
nroff 15, 259, 279, 300, 335, 337, 345, 350, 364, 404, 411, 427, 462
nroff_manmacros 337

ntlm 1294
NTP 923
ntp_time 923
null 1766, 1772
nullzero 1766
number theory 788
numtheory 788

O

oauth 925
object 1321, 1359, 1418, 1421
object oriented 1321, 1359, 1418, 1421
observe 1786
observer 573, 1786
odie 205, 898, 1011
on-idle 1725
one time pad 1793
oo 147
oometa 930
ooutil 933
optimisation 804
optimization 792, 1306
optimize 792
ordered list 1499
otp 936, 1793
outer join 1512

P

package 208
package indexing 245
page 26, 938, 949, 951, 952, 953, 955
page_intro 938
page_pluginmgr 938
page_util_flow 949
page_util_norm_lemon 951
page_util_norm_peg 952
page_util_peg 953
page_util_quote 955
pager 1622
paragraph 1629, 1645
PARAM 1181
parameter entry form 1545, 1568
parser 32, 95, 338, 388, 454, 509, 1014, 1019, 1020, 1021, 1028, 1031, 1038, 1047, 1049, 1055, 1064, 1065, 1072, 1079, 1088, 1096, 1097, 1105, 1113, 1120, 1121, 1129, 1137, 1143, 1145, 1153, 1157, 1164, 1172, 1181, 1204, 1213, 1220, 1226, 1230, 1236, 1239, 1254, 1255, 1256,

1257, 1264
parser generator 26, 938, 949, 951, 952, 953, 955
parsing 137, 140, 345, 371, 381, 404, 435, 447, 509, 511, 513, 515, 524, 531, 538, 542, 548, 549, 556, 563, 571, 585, 1429, 1806, 1817
parsing expression 32, 563, 571, 1014, 1019, 1020, 1021, 1028, 1031, 1038, 1047, 1049, 1055, 1064, 1065, 1072, 1079, 1088, 1096, 1097, 1105, 1113, 1120, 1121, 1129, 1137, 1143, 1145, 1153, 1157, 1164, 1172, 1181, 1204, 1213, 1220, 1226, 1230, 1236, 1239, 1254, 1255, 1256, 1257, 1264
parsing expression grammar 32, 548, 563, 571, 953, 1014, 1019, 1020, 1021, 1028, 1031, 1038, 1047, 1049, 1055, 1064, 1065, 1072, 1079, 1088, 1096, 1097, 1105, 1113, 1120, 1121, 1129, 1137, 1143, 1145, 1153, 1157, 1164, 1172, 1181, 1204, 1213, 1220, 1226, 1230, 1236, 1239, 1254, 1255, 1256, 1257, 1264
partial application 637
partition 1439
partitioned set 1439
passive 1686
password 936
patch 245, 1640
patching 1268
paths 478
pca 798
PEG 32, 548, 952, 953, 1014, 1019, 1020, 1021, 1028, 1031, 1038, 1047, 1049, 1055, 1064, 1065, 1072, 1079, 1088, 1096, 1097, 1105, 1113, 1120, 1121, 1129, 1137, 1143, 1145, 1153, 1157, 1164, 1172, 1181, 1204, 1213, 1220, 1226, 1230, 1236, 1239, 1254, 1255, 1256, 1257, 1264
peg 563
peg_interp 571
performance 128, 132, 133, 135, 137, 138, 139, 1012
permutation 1512
persistence 1666, 1673
phone 1751
pi 727
picoirc 624
pkg_dtplite 462
pki 956
plain text 366, 430
plane geometry 775
plugin 273, 294, 336, 337, 345, 350, 371, 381, 404, 411, 435, 447, 1072, 1079, 1088, 1121, 1129
plugin management 962
plugin search 962

pluginmgr 962
png 967
point 775
polynomial functions 801
polynomials 801
pool 1492, 1501
pop 970
pop3 970, 974, 978, 980
pop3d 974
pop3d_dbox 978
pop3d_udb 980
post-order 1527
practcl 982
pre-order 1527
prefix 1659, 1662, 1664
prime 788
prioqueue 1499, 1501
priority queue 1499
probabilistic calculations 804
probopt 804
proc 637
procedure 618, 1568, 1576
procedure documentation 1560
processman 1011
producer 573
profile 1012
profiler 1012
projection 690
prospero 1732
protocol 98, 642, 651, 913, 974, 978, 980
proxy 589
pt 32
pt_astree 1014
pt_cparam_config_critcl 1019
pt_cparam_config_tea 1020
pt_from_api 1021
pt_introduction 1028
pt_json_language 1031
pt_param 1038
pt_parse_peg 1047
pt_parser_api 1049
pt_peg_container 1055
pt_peg_container_peg 1064
pt_peg_export 1065
pt_peg_export_container 1072
pt_peg_export_json 1079

pt_peg_export_peg 1088
pt_peg_from_container 1096
pt_peg_from_json 1097
pt_peg_from_peg 1105
pt_peg_import 1113
pt_peg_import_container 1120
pt_peg_import_json 1121
pt_peg_import_peg 1129
pt_peg_interp 1137
pt_peg_introduction 1143
pt_peg_language 1145
pt_peg_op 1153
pt_peg_to_container 1157
pt_peg_to_cparam 1164
pt_peg_to_json 1172
pt_peg_to_param 1181
pt_peg_to_peg 1204
pt_peg_to_tclparam 1213
pt_pegrammar 1220
pt_pexpr_op 1226
pt_pexpression 1230
pt_pgen 1236
pt_rdengine 1239
pt_tclparam_config_nx 1254
pt_tclparam_config_snit 1255
pt_tclparam_config_tcloo 1256
pt_to_api 1257
pt_util 1264
public key cipher 956
publisher 573
push down automaton 32, 548, 563, 571, 1014, 1019, 1020, 1021, 1028, 1031, 1038, 1047, 1049, 1055, 1064, 1065, 1072, 1079, 1088, 1096, 1097, 1105, 1113, 1120, 1121, 1129, 1137, 1143, 1145, 1153, 1157, 1164, 1172, 1181, 1204, 1213, 1220, 1226, 1230, 1236, 1239, 1254, 1255, 1256, 1257, 1264

Q

qcomplex 807
quasi-random 811
quasirandom 811
queue 208, 585, 1501, 1511, 1701
quoting 955

R

radians 727, 1726
radiobutton 578
radius 1465

random 1767, 1773
random numbers 1313
randseed 1767
rational functions 813
rational_funcs 813
raw 1610
rc4 1266
rcs 1268
RCS patch 1268
read 188, 189
reading 137
receive 1624
receiver 1624, 1626, 1697
reconnect 903
record 1501, 1502
recursive descent 32, 548, 563, 571, 1014, 1019, 1020, 1021, 1028, 1031, 1038, 1047, 1049, 1055, 1064, 1065, 1072, 1079, 1088, 1096, 1097, 1105, 1113, 1120, 1121, 1129, 1137, 1143, 1145, 1153, 1157, 1164, 1172, 1181, 1204, 1213, 1220, 1226, 1230, 1236, 1239, 1254, 1255, 1256, 1257, 1264
reduce 499, 1512
reference 345, 350, 371, 404, 411, 435
reflected channel 1762, 1763, 1765, 1766, 1767, 1768, 1769, 1770, 1771, 1772, 1773, 1774, 1775, 1776, 1777, 1779, 1780, 1782, 1783, 1784, 1785, 1786, 1787, 1788, 1789, 1790, 1791, 1792, 1793
regex 1426
regular expression 511, 513, 515, 524, 1645, 1657, 1662, 1664
regular grammar 511, 513, 515, 524
regular languages 511, 513, 515, 524
remote communication 167, 183
remote execution 167, 183
remove 470, 471
repeat 1642
repeating 1512
repetition 1512, 1642
report 1272
reshuffle 1512
residual graph 1465
resolver 227
resource management 1719
rest 1278
restore 903
return 1718
reverse 1512
rfc 821 878, 883, 1316

rfc 822 878, 883, 978
rfc 868 923
rfc 959 482, 488, 489
rfc 977 917
rfc 1034 227
rfc 1035 227
rfc 1036 917
rfc 1320 871, 874, 1287, 1290
rfc 1321 871, 874, 1287, 1290
rfc 1413 606
rfc 1630 1732
rfc 1886 227
rfc 1939 970, 974
rfc 2030 923
rfc 2045 878
rfc 2046 878
rfc 2049 878
rfc 2104 871, 874, 1287, 1290, 1300, 1303
rfc 2141 1738
rfc 2251 642, 651
rfc 2255 1732
rfc 2289 936
rfc 2396 1732
rfc 2554 883
RFC 2718 925
rfc 2821 883, 1316
rfc 2849 651
rfc 3207 883
rfc 3513 231
rfc 3986 1732
rfc 4511 642
RFC 5849 925
rfc 6455 1794
rfc 7858 227
rfc2822 163
rfc3501 607
rfc3548 106
rfc4648 106, 109
right outer join 1512
RIPEMD 1287, 1290
ripemd128 1287
ripemd160 1290
roman 816
roman numeral 816
romberg 817
roots 711

rot 1787
rot13 1787
rounding 747
rows 1610
rpc 167, 183
rsa 956
rtcore 496
running 513

S

S3 74
SASL 1292, 1294, 1299
sasl 1294
scanl 499
SCCS 1268
scram 1299
secure 167, 970, 974, 1686, 1697, 1703
security 71, 142, 194, 196, 201, 203, 220, 223, 224, 871, 874, 876, 936, 956, 1266, 1287, 1290, 1300, 1303
seed 1767
selectionbox 627
semantic markup 264, 265, 267, 268, 271, 273, 284, 285, 287, 288, 293, 294, 307, 308, 316, 317, 326, 328, 381, 447
send 167
serialization 122, 342, 356, 360, 364, 366, 369, 392, 398, 417, 422, 427, 430, 433, 458, 1072, 1079, 1088, 1097, 1105, 1121, 1129, 1157, 1164, 1172, 1181, 1204, 1213, 1444, 1527
server 24, 678, 689, 910, 911, 915, 1721
service 667
services 489, 593, 1316
set 1501, 1524
sha1 1300
sha256 1303
shell 1429
shortest path 1465
shuffle 1512
simulated annealing 1306
simulation 1313
simulation_random 1313
singleton 933, 1677
size limit 1785
skiplist 1501, 1509
slippy 680, 687, 689
smtp 878, 883, 1316
smtpd 1316
snit 617, 619, 1321
snitfaq 1359

SNTP 923
socket 167, 183, 1316
soundex 1417
source 43, 237, 245
spacer 1788
spacing 1788
spatial interpolation 750
special 822
special functions 822
specification 135
speed 1012
split 1657
squared graph 1465
ssl 167, 607, 970, 974, 1686, 1697, 1703
stack 1501, 1511
standard io 1768
state 32, 511, 513, 515, 524, 563, 571, 1014, 1019, 1020, 1021, 1028, 1031, 1038, 1047, 1049, 1055, 1064, 1065, 1072, 1079, 1088, 1096, 1097, 1105, 1113, 1120, 1121, 1129, 1137, 1143, 1145, 1153, 1157, 1164, 1172, 1181, 1204, 1213, 1220, 1226, 1230, 1236, 1239, 1254, 1255, 1256, 1257, 1264
state (de)serialization 891
statistical distribution 1313
statistics 191, 720, 773, 798, 830
std 1768
stdin 1768
stdout 1768
stochastic modelling 1310
stooop 1418
stream cipher 1266
stream copy 1786
string 1426, 1429, 1629, 1633, 1642, 1643, 1645, 1657, 1659, 1662, 1664
stringprep 1432, 1435, 1439
stringprep_data 1435
strongly connected component 1465
struct 1492, 1502
struct_list 1512
struct_map 1523
struct_set 1524
struct_tree 1527
struct_tree1 1537
structure 186
structured queries 1708
style 336

subcommand 1568, 1576
subgraph 1444, 1465
subject 573
submitbutton 627
subscriber 573
subsequence 1512
subst 338
sum 203
swapping 1512
switched 1421
symdiff 865
symmetric difference 1524
synchronous 145
syntax tree 555

T

tabify 1643
table 404, 411, 435, 578, 1272
table of contents 279, 284, 294, 398, 401, 404, 411, 417, 422, 427, 430, 433, 435, 443, 447
tabstops 1643
tallying 191
tape archive 1542
tar 1542
tcl 698, 707, 729, 798
Tcl module 245
Tcl syntax 338
tcl_community_communication 46
tcl_parse 338
tcldes 223
tcldesjr 224
tcldocstrip 43
tcler's wiki 345, 350, 404, 411
tcllib 208
tcllib_coroutine 189
tcllib_devguide 48
tcllib_dns 227
tcllib_fhome 1717
tcllib_fifo 1769
tcllib_fifo2 1770
tcllib_installer 63
tcllib_interp 619
tcllib_ip 231
tcllib_license 68
tcllib_memchan 1771
tcllib_msgcat 340
tcllib_null 1772

tcllib_random 1773
tcllib_releasemgr 69
tcllib_sources 70
tcllib_string 1774
tcllib_throw 1718
tcllib_try 1719
tcllib_variable 1775
tcllib_zero 1775
tcllib_zlib 1789
TclOO 147, 593, 930, 933, 1677, 1680, 1685
TCLPARAM 1213
TDPL 32, 563, 571, 1014, 1019, 1020, 1021, 1028, 1031, 1038, 1047, 1049, 1055, 1064, 1065, 1072, 1079, 1088, 1096, 1097, 1105, 1113, 1120, 1121, 1129, 1137, 1143, 1145, 1153, 1157, 1164, 1172, 1181, 1204, 1213, 1220, 1226, 1230, 1236, 1239, 1254, 1255, 1256, 1257, 1264
temp file 463
template processing 1633
tepam_argument_dialogbox 1545
tepam_doc_gen 1560
tepam_introduction 1568
tepam_procedure 1576
term 1625
term_bind 1626
term_send 1628
terminal 1597, 1601, 1608, 1609, 1610, 1612, 1619, 1622, 1624, 1625, 1626, 1628
test 463
Testing 1740, 1742, 1744, 1746, 1747, 1749, 1751, 1752, 1754, 1756, 1757, 1759, 1761
testing 128, 132, 133, 135, 137, 138, 139
TeX 1629, 1645
text 137, 139, 345, 350, 404, 411
text comparison 1417
text conversion 1268
text differences 1268
text display 1619, 1622
text expansion 1633
text likeness 1417
text processing 26, 140, 938, 949, 951, 952, 953, 955, 1806, 1817
text widget 1776
textutil 1645
textutil_split 1657
textutil_string 1659
textwindow 1776
threads 188, 189

throw 1718
thumbnail 629
tie 1666, 1673
tie_std 1673
tif 1674
tiff 1674
tile 687, 689
time 923
timestamp 967
timestamps 218
tip 219 1762, 1763, 1765, 1766, 1767, 1768, 1769, 1770, 1771, 1772, 1773, 1774, 1775, 1776, 1777, 1779, 1780
tip 230 1782, 1783, 1784, 1785, 1786, 1787, 1788, 1789, 1790, 1791, 1792, 1793
tip 234 1789
tip 317 1790
Tk 1776
tls 167, 607, 883, 970, 974, 1686, 1697, 1703
TMML 15, 259, 279, 300, 335, 345, 404, 462
toc 279, 284, 295, 398, 401, 417, 422, 427, 430, 433, 443
toc formatter 295
toc_container 404
toc_export 411
toc_export_html 417
toc_export_json 422
toc_export_nroff 427
toc_export_text 430
toc_export_wiki 433
toc_import 435
toc_import_json 443
toc_introduction 447
toc_msgcat_c 450
toc_msgcat_de 451
toc_msgcat_en 452
toc_msgcat_fr 453
toc_parse 454
toc_structure 458
token 1426
token_shell 1429
tokenization 1426, 1429
TOOL 930, 1685
tool 1680
tool_dict_ensemble 1685
top-down parsing languages 32, 548, 563, 571, 1014, 1019, 1020, 1021, 1028, 1031, 1038, 1047, 1049, 1055, 1064, 1065, 1072, 1079, 1088, 1096, 1097, 1105, 1113, 1120, 1121, 1129, 1137, 1143,

1145, 1153, 1157, 1165, 1172, 1181, 1204, 1213, 1220, 1226, 1230, 1236, 1239, 1254, 1255, 1256, 1257, 1264
torrent 122
touch 463
TPDL 548
tqueue 1701
trace 212, 216, 217, 218
transducer 32, 509, 511, 513, 515, 524, 548, 563, 571, 1014, 1019, 1020, 1021, 1028, 1031, 1038, 1047, 1049, 1055, 1064, 1065, 1072, 1079, 1088, 1096, 1097, 1105, 1113, 1120, 1121, 1129, 1137, 1143, 1145, 1153, 1157, 1165, 1172, 1181, 1204, 1213, 1220, 1226, 1230, 1236, 1239, 1254, 1255, 1256, 1257, 1264
transfer 1686, 1689, 1692, 1694, 1697, 1701, 1703
transformation 953, 1782, 1783, 1784, 1785, 1786, 1787, 1788, 1789, 1790, 1791, 1792, 1793
transformcore 1780
transmitter 1703
travelling salesman 1465
traversal 479
traverse 479
tree 531, 555, 585, 1501, 1511, 1527, 1537, 1708
tree query language 1708
tree walking 949, 951, 952
treeql 1708
trig 867
trigonometry 867
trim 1662
trimming 1645, 1662
twitter 925
type 463, 493, 494, 495, 496, 1321
Type checking 1740, 1742, 1744, 1746, 1747, 1749, 1751, 1752, 1754, 1756, 1757, 1759, 1761

U

udpcluster 1721
uevent 573, 1722
uevent_onidle 1725
unbind 1722
uncapitalize 1659
undenting 1629
unicode 1432, 1435, 1436, 1439
unicode_data 1439
unified format 1640
unified format diff 1640
union 1439, 1524
unit 1726

units 1726
unknown hooking 891
untie 1666, 1673
update 188, 189
uri 1732, 1738
url 345, 350, 371, 411, 435, 678, 689, 1732, 1738
urn 1738
urn-scheme 1738
US-NPI 1757
usnpi 1757
utilities 891
uuencode 116
uuid 1739

V

Validation 1740, 1742, 1744, 1746, 1747, 1749, 1751, 1752, 1754, 1756, 1757, 1759, 1761
valtype_common 1759
Value checking 1740, 1742, 1744, 1746, 1747, 1749, 1751, 1752, 1754, 1756, 1757, 1759, 1761
vectors 754
verhoeff 1761
vertex 1444, 1465
vertex cover 1465
virtual channel 1762, 1763, 1765, 1766, 1767, 1768, 1769, 1770, 1771, 1772, 1773, 1774, 1775, 1776, 1777, 1779, 1780, 1782, 1783, 1784, 1785, 1786, 1787, 1788, 1789, 1790, 1791, 1792, 1793
virtual machine 531, 538, 542, 548, 549, 556, 571, 1038
VISA 1746
vt_base64 1790
vt_counter 1791
vt_crc32 1792
vt_otp 1793
vwait 188, 189, 1316

W

wais 1732
wcswidth 1664
websocket 1794
widget 1321, 1359
widget adaptors 1321, 1359
wiki 259, 279, 345, 350, 369, 404, 411, 433
wip 1801
word 338, 1801
WWW 593
www 1732

X

x.208 98
x.209 98
x.500 642
XGoogleToken 1292
xml 95
xor 1793
XPath 1708
XSLT 1708
xsxp 95

Y

yaml 1806, 1817
ydecode 119
yEnc 119
yencode 119

Z

zero 1766, 1775
zip 1820, 1823, 1824
zlib 1789
zoom 680, 687, 689